



MANUAL COMPLETO DEL SISTEMA POS

Waldonald's

Código Java + interfaces Swing + MySQL + operación del negocio
Guía de estudio desde cero y referencia técnica línea por línea

Versión documentada	04/10/2026
Código fuente	108 archivos Java / 28,833 líneas
Interfaz visual	23 formularios NetBeans / 6,605 líneas XML
Base de datos	6 scripts SQL / 2,283 líneas
Recursos	316 imágenes, fuentes y librerías catalogadas

Objetivo: que el equipo pueda explicar qué hace el sistema, cómo circulan los datos, cómo se relacionan las tablas y qué significa cada línea relevante del código durante la presentación.

1. Cómo usar este manual

Este documento está diseñado en capas. Primero enseña conceptos y flujos; después explica la arquitectura y la base de datos; finalmente incluye un anexo de referencia con cada archivo de texto y cada línea anotada. No hace falta memorizar el anexo: úsalo para buscar una línea concreta cuando el profesor pregunte.

Cobertura verificable: 157 archivos de texto documentados, 42,934 líneas anotadas y 316 recursos binarios catalogados.

- Ruta rápida para presentar: capítulos 2, 3, 5, 7, 8 y 13.
- Ruta para aprender Java: capítulos 4 y 6, luego buscar un archivo en el anexo.
- Ruta para aprender la base de datos: capítulos 8, 9 y 10.
- Ruta para depurar: capítulos 11 y 12, más el índice de archivos.

Alcance de 'cada archivo': se explican línea por línea los archivos Java, SQL, .form, XML, properties, Markdown, PowerShell y manifiestos. Las imágenes, fuentes y JAR no tienen líneas de código; se catalogan por carpeta, tipo y utilidad.

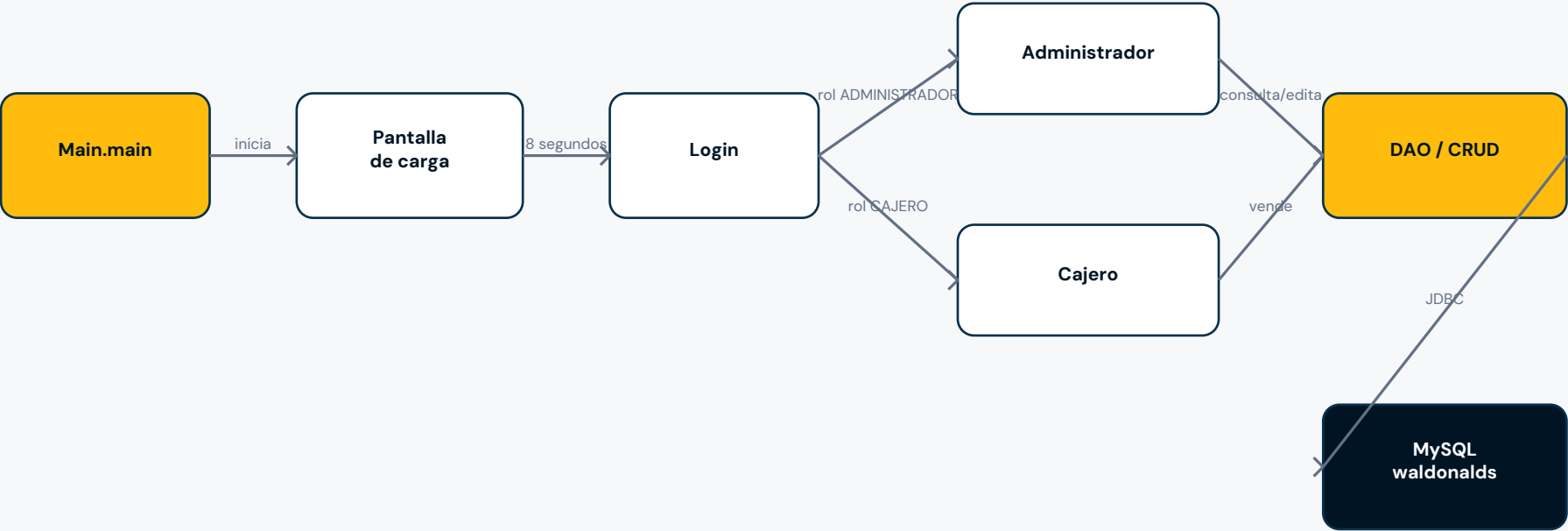
Mapa del documento

Parte	Qué aprenderán
I. Fundamentos	Conceptos de Java, Swing, JDBC, SQL y la estructura de NetBeans.
II. Arquitectura	Cómo se comunican interfaz, modelos, DAO/CRUD y MySQL.
III. Base de datos	Cada tabla, campo, relación, ejemplo y flujo de inventario/pedido.
IV. Casos prácticos	Login, WIMenú, combo predefinido, Cajita, cobro y administración.
V. Presentación	Guion, demostración, preguntas frecuentes y riesgos conocidos.
VI. Referencia	Catálogo de archivos y explicación línea por línea.

2. Qué es Waldonald's

Waldonald's es un sistema POS de escritorio para restaurante. Tiene dos áreas: el cajero arma y cobra pedidos; el administrador mantiene usuarios, productos, recetas, presentaciones, inventario y reportes. La información persiste en MySQL y la interfaz está construida con Java Swing.

Flujo de inicio y separación por roles



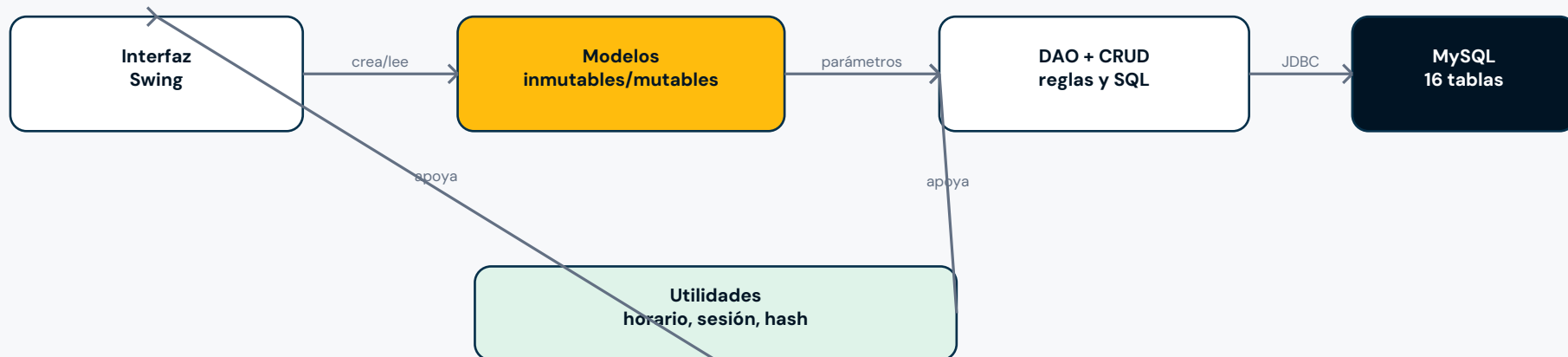
Actor	Puede hacer	No debería hacer
Cajero	Ver productos por horario, configurar presentaciones, modificar ingredientes, cobrar y ver comprobante.	Editar catálogos, usuarios o reglas comerciales.
Administrador	Gestionar usuarios, menú, recetas, inventario, movimientos y reportes.	Registrar una venta desde el flujo administrativo.
MySQL	Persistir reglas, pedidos, movimientos y estados.	Decidir la interacción visual del usuario.

3. Arquitectura del proyecto

La aplicación sigue una separación por responsabilidades. No es un framework formal, pero sí aplica una arquitectura por capas: las vistas dibujan y reciben clics, los modelos transportan datos, DAO/CRUD hablan con MySQL y las utilidades resuelven tareas compartidas.

Capa/carpeta	Responsabilidad	Ejemplos
Main / Vistas / Login	Arranque y autenticación.	main, PantallaCarga, Login
GUI_CAJERO	Venta, carrito, configuración y cobro.	Cajero, PedidoPanel, ConfiguradorProductoPanel
GUI_ADMINISTRADOR	Mantenimiento y análisis del negocio.	MenuAdmin, GestionMenuPanel, InventarioPanel, ReportesPanel
Modelos	Datos tipados que viajan entre capas.	Producto, LineaPedido, SolicitudPago, Usuario
DAO	Consultas de casos de uso complejos.	PagoDAO, ConfiguracionMenuDAO, InventarioDAO
CRUD	Altas, consultas, cambios y estado de catálogos.	ProductoCRUD, UsuarioCRUD, IngredienteCRUD
Componentes / Labels	Controles visuales reutilizables.	BotonRedondeado, TablaAdministrativa, LabelEscalable
Utilidades	Sesión, horarios, imágenes, seguridad y exportación.	SesionUsuario, HorarioMenu, SeguridadContrasena
sql	Estructura y datos iniciales.	base_datos_completa.sql, datos_menu_nueva_base.sql

Dependencias principales



4. Java desde cero: cómo leer el código

Concepto	Explicación
package	Carpeta lógica de una clase. Ejemplo: package DAO;
import	Hace disponible una clase externa. Ejemplo: import java.sql.Connection;
class	Molde que agrupa datos y métodos.
record	Modelo compacto e inmutable; Java crea constructor y accesores automáticamente.
atributo	Variable que pertenece al objeto y conserva su estado.
método	Bloque con nombre que realiza una tarea y puede recibir/devolver datos.
constructor	Método especial que se ejecuta al crear un objeto con new.
public/private	Controla desde dónde se puede acceder a un elemento.
static	Pertenece a la clase y no a una instancia concreta.
final	Impide reasignar una referencia o sobrescribir una definición.
if/else	Elige entre rutas según una condición.
for/while	Repite instrucciones.
try/catch	Ejecuta una operación riesgosa y maneja el error.
listener	Función que espera un clic, tecla o movimiento del usuario.
Scanner	Clase común para leer datos de consola. Este proyecto casi no la usa porque obtiene datos desde componentes Swing.

Ejemplo mínimo explicado

```
public BigDecimal total() {  
    return lineas.stream()  
        .map(LineaPedido::subtotal)  
        .reduce(BigDecimal.ZERO, BigDecimal::add);  
}
```

public permite llamar al método desde otra clase. BigDecimal es el tipo devuelto y se usa para dinero. total es el nombre. stream recorre las líneas; map obtiene cada subtotal; reduce los suma comenzando en cero; return entrega el resultado.

Swing y el hilo visual

Swing dibuja la interfaz en el Event Dispatch Thread. Por eso Main usa EventQueue.invokeLater. Las consultas que pueden tardar utilizan SwingWorker para no congelar la ventana. initComponents es generado por NetBeans y crea/posiciona los controles definidos en el archivo .form.

Regla de oro: no borrar los comentarios //GEN-BEGIN y //GEN-END. Si desaparecen, NetBeans abre el formulario en modo de solo lectura porque ya no reconoce sus bloques protegidos.

5. Cómo preparar y ejecutar el proyecto

Requisito	Versión/uso
JDK	Java 21, según nbproject/project.properties.
IDE	NetBeans con soporte para proyectos Ant y GUI Builder.
Base de datos	MySQL local con acceso para crear y usar waldonalds.
Driver JDBC	lib/mysql-connector-j-26.7.0.jar.
Layout	lib/absolutelayout/AbsoluteLayout.jar.
Clase principal	Main.main.

1. Abrir el proyecto Waldonalds en NetBeans.
2. Ejecutar sql/base_datos_completa.sql para crear las tablas.
3. Ejecutar sql/datos_menu_nueva_base.sql para cargar categorías, productos, recetas y presentaciones.
4. Revisar usuario y contraseña de MySQL en Conexion/Conexion.java. Para producción deben salir del código y leerse de variables de entorno.
5. Confirmar que los JAR de lib aparecen en Libraries.
6. Ejecutar Clean and Build; después Run. La pantalla de carga abre Login y el rol decide la ventana siguiente.

Orden correcto de los scripts SQL

1. sql/base_datos_completa.sql
2. sql/datos_menu_nueva_base.sql
3. sql/pagos.sql (solo si se parte de un esquema antiguo)
4. migraciones específicas (solo para bases existentes)

Para una instalación nueva, no se deben mezclar migraciones antiguas sin revisar. El esquema completo ya contiene las columnas actuales. Los scripts de migración existen para instalaciones previas.

6. Flujos funcionales principales

Flujo	Secuencia
Inicio de sesión	Login lee usuario y contraseña -> consulta usuario activo -> verifica hash -> valida turno -> guarda SesionUsuario -> abre administrador o cajero.
Catálogo del cajero	CategoriasPanel elige categoría -> ProductoDAO filtra por estado, presentación y horario -> SubCategoriasPanel crea TarjetaProducto.
Configurar producto	ConfiguracionMenuDAO carga presentaciones -> el cajero elige Individual/WIMenú/Combo -> completa grupos -> personaliza cada componente -> se crea LineaPedido.
Carrito	PedidoPanel agrega líneas, cambia cantidades, quita líneas y calcula el total con BigDecimal.
Cobro	SolicitudPago reúne clave, servicio, método y líneas -> PagoDAO revalida todo -> transacción SQL -> descuento de inventario -> comprobante.
Inventario	InventarioPanel consulta ArticuloInventario -> InventarioDAO registra ENTRADA/SALIDA/AJUSTE -> actualiza stock y bitácora.
Administración	MenuAdmin cambia paneles -> CRUD/DAO ejecutan acciones -> la tabla se recarga y muestra estado actualizado.
Reportes	ReportesPanel pide una fecha -> ReporteDAO agrupa pedidos pagados/cancelados -> filtros locales -> exportación CSV o impresión.

Por qué se vuelve a validar al cobrar

La interfaz ayuda al cajero, pero no es la autoridad final: entre configurar y pagar pudo cambiar el precio, el stock o una regla. PagoDAO vuelve a leer las reglas con la misma conexión y bloquea existencias con FOR UPDATE. Así evita vender inventario inexistente o aceptar una selección inválida.

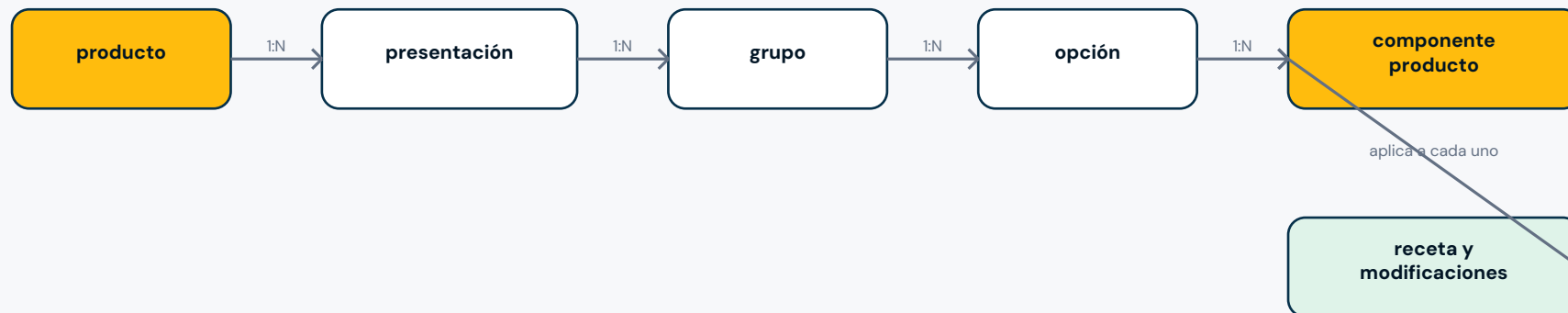
```
setAutoCommit(false) -> validar -> bloquear stock -> INSERT pedido -> INSERT detalle/opciones/productos/modificaciones -> UPDATE stock -> INSERT movimientos -> commit()  
Si algo falla: rollback()
```


7. Modelo de productos, WIMenú, Cajita y combos

La clave del diseño es que una venta no es solamente un producto. Un producto abre una o más presentaciones. Cada presentación contiene grupos; cada grupo contiene opciones; cada opción entrega uno o varios productos reales; y cada producto real puede tener modificaciones de receta.

Nivel	Ejemplo Big Mac WIMenú	Regla
Producto principal	Big Mac	Es la tarjeta que abrió el cajero.
Presentación	WIMenú - Q61	Determina precio y tipo MENU.
Grupo	Elige complemento	mínimo 1, máximo 1.
Opción	WIPatatas +Q5	Alternativa elegible y su recargo.
Componente	1 producto WIPatatas	Producto físico que se prepara/entrega.
Modificación	Big Mac sin pepinillo	Afecta solo ese componente, no toda la línea.

Jerarquía configurable



Ejemplo mental: combo de dos hamburguesas

El grupo puede requerir dos elecciones o la opción puede contener dos componentes. Cada **ProductoPedido** tiene su propia lista de **ModificacionPedido**. Por eso una hamburguesa puede ir sin pepinillos y la otra con todo, aunque ambas pertenezcan a la misma **LineaPedido**.

8. Base de datos completa

La base utiliza claves primarias para identificar filas y claves foráneas para conectarlas. Los campos de estado permiten desactivar sin borrar el historial. Los nombres copiados en las tablas de pedido son snapshots históricos: si mañana cambia el nombre del producto, el comprobante viejo conserva lo vendido.

Mapa relacional por dominios

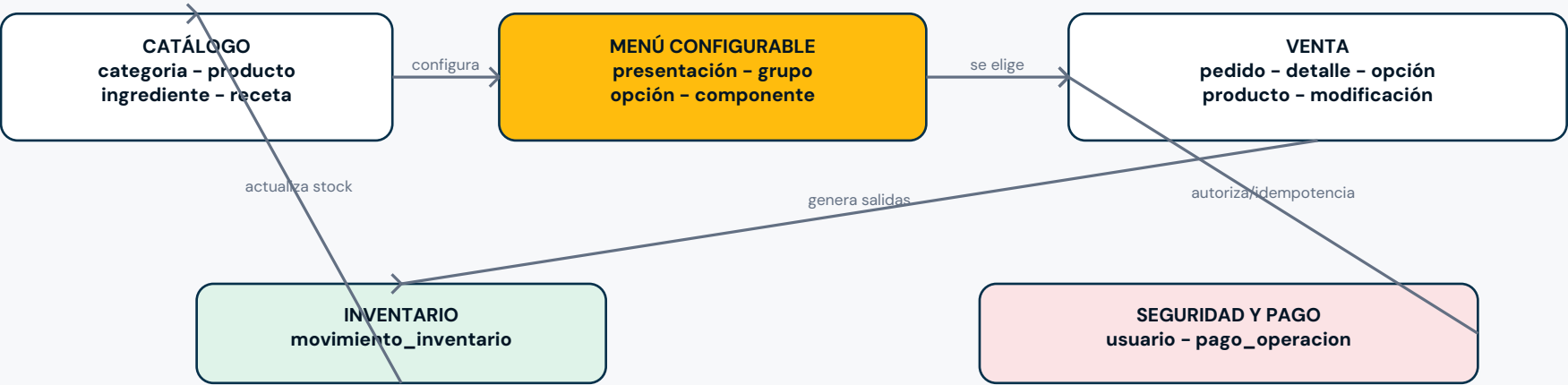


Tabla	Función	Campos
usuario	Personas que pueden iniciar sesión. Define credenciales, rol, estado y horario de trabajo.	12
categoria	Agrupar productos en las tarjetas principales del menú: Desayuno, Almuerzos, Bebidas, etc.	3
ingrediente	Materia prima controlada por inventario, con unidad, existencia y mínimo de reposición.	6
producto	Catálogo general. Incluye productos vendibles y productos internos usados como componentes de combos.	13
producto_ingrediente	Receta de cada producto y reglas de personalización: quitar, extra, cantidad y recargo.	10
presentacion_menu	Formas de vender un producto principal: Individual, WiMenú, Infantil o Combo, cada una con su precio.	7
grupo_presentacion	Pregunta o sección dentro de una presentación, con límites mínimo/máximo y permiso de repetir.	9
opcion_grupo	Respuesta disponible dentro de un grupo, con recargo y opción predeterminada.	6
opcion_componente	Productos concretos que entrega una opción. Una opción puede contener uno o varios productos.	4

Tabla	Función	Campos
pedido	Cabecera de la venta: orden, cajero, servicio, estado, pago, total y fecha.	9
pedido_detalle	Cada línea del carrito, incluyendo la presentación elegida, cantidad y subtotal.	8
pedido_opcion	Captura las elecciones realizadas en cada grupo de la línea para conservar un historial exacto.	8
pedido_producto	Expande cada opción a los productos físicos que cocina o entrega el restaurante.	5
pedido_modificacion	Cambios aplicados a cada producto interno: SIN ingrediente o EXTRA ingrediente.	7
movimiento_inventario	Bitácora de entradas, salidas y ajustes de productos directos o ingredientes.	8
pago_operacion	Control de idempotencia del cobro: evita registrar dos veces la misma solicitud de pago.	5

8.1 Tabla usuario

Personas que pueden iniciar sesión. Define credenciales, rol, estado y horario de trabajo.

Campo	Tipo	Reglas	Para qué sirve
id_usuario	INT	AUTO_INCREMENT PRIMARY KEY	Identificador único del usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre visible o copia histórica del nombre.
apellido	VARCHAR(100)	NOT NULL	Apellido del usuario.
usuario	VARCHAR(60)	NOT NULL UNIQUE	Nombre corto utilizado para iniciar sesión.
correo	VARCHAR(150)	NOT NULL UNIQUE	Correo único del usuario.
password_hash	VARCHAR(150)	NOT NULL	Resumen criptográfico de la contraseña; no guarda la contraseña en texto normal.
rol	ENUM('ADMINISTRADOR','CAJERO')	NOT NULL	Permiso principal: ADMINISTRADOR o CAJERO.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.
fecha_creacion	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Fecha y hora de creación automática.
turno	ENUM('MANANA','TARDE')	-	Turno MANANA o TARDE asignado al usuario.
hora_inicio	TIME	-	Hora opcional desde la que puede iniciar sesión.
hora_fin	TIME	-	Hora opcional hasta la que puede iniciar sesión.

Registro de ejemplo (orden de columnas del script): 1 | Ana | López | admin | ana@waldonalds.gt | [hash] | ADMINISTRADOR | 1 | ...

8.2 Tabla categoria

Agrupa productos en las tarjetas principales del menú: Desayuno, Almuerzos, Bebidas, etc.

Campo	Tipo	Reglas	Para qué sirve
id_categoria	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la categoría relacionada.
nombre	VARCHAR(100)	NOT NULL	Nombre visible o copia histórica del nombre.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Registro de ejemplo (orden de columnas del script): 2 | Almuerzos | 1

8.3 Tabla ingrediente

Materia prima controlada por inventario, con unidad, existencia y mínimo de reposición.

Campo	Tipo	Reglas	Para qué sirve
id_ingrediente	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del ingrediente relacionado.
nombre	VARCHAR(100)	NOT NULL	Nombre visible o copia histórica del nombre.
unidad_medida	VARCHAR(30)	NOT NULL	Unidad usada para contar el ingrediente, por ejemplo UNIDAD, GRAMO o ML.
stock_actual	DECIMAL(12,2)	NOT NULL DEFAULT 0	Existencia disponible en este momento.
stock_minimo	DECIMAL(12,2)	NOT NULL DEFAULT 0	Nivel que activa la advertencia de reposición.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Registro de ejemplo (orden de columnas del script): 37 | Pepinillo | UNIDAD | 250.00 | 40.00 | 1

8.4 Tabla producto

Catálogo general. Incluye productos vendibles y productos internos usados como componentes de combos.

Campo	Tipo	Reglas	Para qué sirve
id_producto	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del producto relacionado.
id_categoria	INT	NOT NULL	Identificador de la categoría relacionada.
nombre	VARCHAR(150)	NOT NULL	Nombre visible o copia histórica del nombre.
descripcion	VARCHAR(500)	-	Descripción comercial mostrada al cajero.
precio_base	DECIMAL(12,2)	NOT NULL DEFAULT 0	Precio de referencia del producto individual.
imagen	VARCHAR(500)	-	Ruta del recurso gráfico del producto.
subcategoria	VARCHAR(100)	-	Filtro secundario dentro de una categoría.
disponibilidad_menu	ENUM('TODO_DIA','DESAYUNO','ALMUERZO')	NOT NULL DEFAULT 'TODO_DIA'	Horario comercial permitido: TODO_DIA, DESAYUNO o ALMUERZO.
tipo_stock	ENUM('RECETA','DIRECTO','NINGUNO')	NOT NULL DEFAULT 'DIRECTO'	RECETA descuenta ingredientes; DIRECTO descuenta producto; NINGUNO no controla existencia.
personalizable	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el cajero puede modificar ingredientes internos.
stock_actual	INT	NOT NULL DEFAULT 0	Existencia disponible en este momento.
stock_minimo	INT	NOT NULL DEFAULT 0	Nivel que activa la advertencia de reposición.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Relaciones: id_categoria -> categoria.id_categoria

Registro de ejemplo (orden de columnas del script): 12 | 2 | Big Mac | ... | 41.00 | ... | Hamburguesas | ALMUERZO | RECETA | 1 | 0 | 0 | 1

8.5 Tabla producto_ingrediente

Receta de cada producto y reglas de personalización: quitar, extra, cantidad y recargo.

Campo	Tipo	Reglas	Para qué sirve
id_producto_ingrediente	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la regla de receta/personalización.
id_producto	INT	NOT NULL	Identificador del producto relacionado.
id_ingredient	INT	NOT NULL	Identificador del ingrediente relacionado.
cantidad_default	DECIMAL(12,2)	NOT NULL	Cantidad consumida por una preparación normal.
permite_quitar	BOOLEAN	NOT NULL DEFAULT FALSE	Permite pedir el producto sin este ingrediente.
permite_extra	BOOLEAN	NOT NULL DEFAULT FALSE	Permite agregar porciones extra de este ingrediente.
cantidad_extra	DECIMAL(12,2)	NOT NULL DEFAULT 1	Cantidad de inventario que consume cada extra.
precio_extra	DECIMAL(12,2)	NOT NULL DEFAULT 0	Recargo monetario del extra o de la opción elegida.
max_extras	INT	NOT NULL DEFAULT 1	Máximo de extras permitidos para este ingrediente.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Relaciones: id_producto -> producto id_producto: id_ingredient -> ingrediente id_ingredient

Registro de ejemplo (orden de columnas del script): 85 | 12 | 37 | 2.00 | 1 | 1 | 1.00 | 1.00 | 3 | 1

8.6 Tabla presentacion_menu

Formas de vender un producto principal: Individual, WIMenú, Infantil o Combo, cada una con su precio.

Campo	Tipo	Reglas	Para qué sirve
id_presentacion	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la presentación elegida.
id_producto_principal	INT	NOT NULL	Producto desde el que el cajero abre esta presentación.
nombre	VARCHAR(100)	NOT NULL	Nombre visible o copia histórica del nombre.
tipo	ENUM('INDIVIDUAL','MENU','INFANTIL','COMBO')	NOT NULL	Clasificación controlada del registro según la tabla.
precio	DECIMAL(12,2)	NOT NULL	Precio total de la presentación.
predeterminada	BOOLEAN	NOT NULL DEFAULT FALSE	Opción que aparece seleccionada inicialmente.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Relaciones: id_producto_principal -> producto id_producto

Registro de ejemplo (orden de columnas del script): 21 | 12 | WIMenú | MENU | 61.00 | 0 | 1

8.7 Tabla grupo_presentacion

Pregunta o sección dentro de una presentación, con límites mínimo/máximo y permiso de repetir.

Campo	Tipo	Reglas	Para qué sirve
id_grupo	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del grupo de opciones.
id_presentacion	INT	NOT NULL	Identificador de la presentación elegida.
nombre	VARCHAR(150)	NOT NULL	Nombre visible o copia histórica del nombre.
minimo	INT	NOT NULL DEFAULT 1	Cantidad mínima de elecciones requeridas.
maximo	INT	NOT NULL DEFAULT 1	Cantidad máxima de elecciones permitidas.
permite_repetir	BOOLEAN	NOT NULL DEFAULT FALSE	Autoriza elegir la misma opción más de una vez.
visible	BOOLEAN	NOT NULL DEFAULT TRUE	Decide si el grupo se muestra al cajero.
permite_personalizar	BOOLEAN	NOT NULL DEFAULT TRUE	Autoriza abrir la personalización de los productos internos.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Relaciones: id_presentacion -> presentacion_menu id_presentacion

Registro de ejemplo (orden de columnas del script): 44 | 21 | Elige la bebida | 1 | 1 | 0 | 1 | 1 | 1

8.8 Tabla opcion_grupo

Respuesta disponible dentro de un grupo, con recargo y opción predeterminada.

Campo	Tipo	Reglas	Para qué sirve
id_opcion	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la opción elegida.
id_grupo	INT	NOT NULL	Identificador del grupo de opciones.
nombre	VARCHAR(150)	NOT NULL	Nombre visible o copia histórica del nombre.
incremento_precio	DECIMAL(12,2)	NOT NULL DEFAULT 0	Monto que se suma al precio por escoger esta opción.
predeterminada	BOOLEAN	NOT NULL DEFAULT FALSE	Opción que aparece seleccionada inicialmente.
estado	BOOLEAN	NOT NULL DEFAULT TRUE	Indica si el registro está activo/disponible.

Relaciones: id_grupo -> grupo_presentacion id_grupo

Registro de ejemplo (orden de columnas del script): 190 | 44 | Coca Cola | 0.00 | 1 | 1

8.9 Tabla opcion_componente

Productos concretos que entrega una opción. Una opción puede contener uno o varios productos.

Campo	Tipo	Reglas	Para qué sirve
id_opcion_componente	INT	AUTO_INCREMENT PRIMARY KEY	Dato id opcion componente de esta tabla.
id_opcion	INT	NOT NULL	Identificador de la opción elegida.
id_producto	INT	NOT NULL	Identificador del producto relacionado.
cantidad	INT	NOT NULL DEFAULT 1	Número de unidades o cantidad decimal involucrada.

Relaciones: id_opcion -> opcion_grupo id_opcion: id_producto -> producto id_producto

Registro de ejemplo (orden de columnas del script): 201 | 190 | 65 | 1

8.10 Tabla pedido

Cabecera de la venta: orden, cajero, servicio, estado, pago, total y fecha.

Campo	Tipo	Reglas	Para qué sirve
id_pedido	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del pedido relacionado.
numero_orden	INT	NOT NULL DEFAULT 0	Número visible que identifica la orden del cliente.
id_usuario	INT	NOT NULL	Identificador único del usuario.
tipo_servicio	ENUM('COMER_AQUI','PARA_LLEVAR')	NOT NULL	COMER_AQUI o PARA_LLEVAR.
estado	ENUM('PAGADO','CANCELADO')	NOT NULL	Indica si el registro está activo/disponible.
metodo_pago	ENUM('EFECTIVO','TARJETA')	-	EFECTIVO o TARJETA.
monto_recibido	DECIMAL(12,2)	NOT NULL DEFAULT 0	Dinero entregado por el cliente para calcular el cambio.
total	DECIMAL(12,2)	NOT NULL	Total final cobrado.
fecha_hora	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Momento exacto en que ocurrió el registro.

Relaciones: id_usuario -> usuario id_usuario

Registro de ejemplo (orden de columnas del script): 500 | 104 | 7 | COMER_AQUI | PAGADO | EFECTIVO | 100.00 | 62.00 | 2026-10-04 12:30

8.11 Tabla pedido_detalle

Cada línea del carrito, incluyendo la presentación elegida, cantidad y subtotal.

Campo	Tipo	Reglas	Para qué sirve
id_detalle	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la línea del pedido.
id_pedido	INT	NOT NULL	Identificador del pedido relacionado.
id_presentacion	INT	NOT NULL	Identificador de la presentación elegida.
nombre	VARCHAR(150)	NOT NULL	Nombre visible o copia histórica del nombre.
presentacion	VARCHAR(100)	NOT NULL	Copia del nombre de la presentación al momento de vender.
cantidad	INT	NOT NULL DEFAULT 1	Número de unidades o cantidad decimal involucrada.
precio_unitario	DECIMAL(12,2)	NOT NULL	Precio de una unidad configurada.
subtotal	DECIMAL(12,2)	NOT NULL	precio_unitario multiplicado por cantidad.

Relaciones: id_pedido -> pedido id_pedido: id_presentacion -> presentacion_menu id_presentacion

Registro de ejemplo (orden de columnas del script): 700 | 500 | 21 | Big Mac | WIMenú | 1 | 62.00 | 62.00

8.12 Tabla pedido_opcion

Captura las elecciones realizadas en cada grupo de la línea para conservar un historial exacto.

Campo	Tipo	Reglas	Para qué sirve
id_pedido_opcion	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la elección guardada.
id_detalle	INT	NOT NULL	Identificador de la línea del pedido.
id_grupo	INT	NOT NULL	Identificador del grupo de opciones.
id_opcion	INT	NOT NULL	Identificador de la opción elegida.
numero	INT	NOT NULL DEFAULT 1	Número de repetición de una selección dentro del grupo.
grupo	VARCHAR(150)	NOT NULL	Copia histórica del nombre del grupo.
opcion	VARCHAR(150)	NOT NULL	Copia histórica del nombre de la opción.
precio_extra	DECIMAL(12,2)	NOT NULL DEFAULT 0	Recargo monetario del extra o de la opción elegida.

Relaciones: id_detalle -> pedido_detalle id_detalle: id_grupo -> grupo_presentacion id_grupo: id_opcion -> opcion_grupo id_opcion

Registro de ejemplo (orden de columnas del script): 900 | 700 | 44 | 190 | 1 | Elige la bebida | Coca Cola | 0.00

8.13 Tabla pedido_producto

Expande cada opción a los productos físicos que cocina o entrega el restaurante.

Campo	Tipo	Reglas	Para qué sirve
id_pedido_producto	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del producto interno del pedido.
id_pedido_opcion	INT	NOT NULL	Identificador de la elección guardada.
id_producto	INT	NOT NULL	Identificador del producto relacionado.
nombre	VARCHAR(150)	NOT NULL	Nombre visible o copia histórica del nombre.
cantidad	INT	NOT NULL DEFAULT 1	Número de unidades o cantidad decimal involucrada.

Relaciones: id_pedido_opcion -> pedido_opcion id_pedido_opcion: id_producto -> producto id_producto

Registro de ejemplo (orden de columnas del script): 1000 | 900 | 65 | Coca Cola | 1

8.14 Tabla pedido_modificacion

Cambios aplicados a cada producto interno: SIN ingrediente o EXTRA ingrediente.

Campo	Tipo	Reglas	Para qué sirve
id_modificacion	INT	AUTO_INCREMENT PRIMARY KEY	Identificador de la modificación.
id_pedido_producto	INT	NOT NULL	Identificador del producto interno del pedido.
id_producto_ingredient	INT	NOT NULL	Identificador de la regla de receta/personalización.
tipo	ENUM('SIN','EXTRA')	NOT NULL	Clasificación controlada del registro según la tabla.
ingrediente	VARCHAR(100)	NOT NULL	Copia histórica del nombre del ingrediente modificado.
cantidad	DECIMAL(12,2)	NOT NULL	Número de unidades o cantidad decimal involucrada.
precio_extra	DECIMAL(12,2)	NOT NULL DEFAULT 0	Recargo monetario del extra o de la opción elegida.

Relaciones: id_pedido_producto -> pedido_producto id_pedido_producto: id_producto_ingredient -> producto_ingredient id_producto_ingredient

Registro de ejemplo (orden de columnas del script): 1200 | 1001 | 85 | SIN | Pepinillo | 2.00 | 0.00

8.15 Tabla movimiento_inventario

Bitácora de entradas, salidas y ajustes de productos directos o ingredientes.

Campo	Tipo	Reglas	Para qué sirve
id_movimiento	INT	AUTO_INCREMENT PRIMARY KEY	Identificador del movimiento de inventario.
id_producto	INT	-	Identificador del producto relacionado.
id_ingredient	INT	-	Identificador del ingrediente relacionado.
id_pedido	INT	-	Identificador del pedido relacionado.
tipo_movimiento	ENUM('ENTRADA','SALIDA','AJUSTE')	NOT NULL	ENTRADA, SALIDA o AJUSTE.
cantidad	DECIMAL(12,2)	NOT NULL	Número de unidades o cantidad decimal involucrada.
motivo	VARCHAR(150)	-	Explicación humana de por qué cambió el inventario.
fecha_hora	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Momento exacto en que ocurrió el registro.

Relaciones: id_producto -> producto id_producto: id_ingredient -> ingrediente id_ingredient: id_pedido -> pedido id_pedido

Registro de ejemplo (orden de columnas del script): 1500 | NULL | 37 | 500 | SALIDA | 2.00 | Preparación de pedido | 2026-10-04 12:30

8.16 Tabla pago_operacion

Control de idempotencia del cobro: evita registrar dos veces la misma solicitud de pago.

Campo	Tipo	Reglas	Para qué sirve
clave	VARCHAR(36)	PRIMARY KEY	UUID o clave única enviada por el cliente para hacer el cobro idempotente.
solicitud	LONGTEXT	NOT NULL	Copia serializada de la solicitud original.
id_pedido	INT	UNIQUE	Identificador del pedido relacionado.
comprobante	LONGTEXT	-	Texto del comprobante generado tras cobrar.
referencia	VARCHAR(100)	NOT NULL DEFAULT ""	Referencia externa o administrativa del pago.

Relaciones: id_pedido -> pedido id_pedido

Registro de ejemplo (orden de columnas del script): 550e8400-e29b-41d4-a716-446655440000 | {solicitud} | 500 | {comprobante} | POS-104

9. Reglas de datos que deben poder explicar

Regla/campo	Interpretación
tipo_stock = RECETA	El producto no descuenta su propio stock; consume ingrediente según producto_ingrediente.
tipo_stock = DIRECTO	El producto se entrega como unidad terminada y descuenta producto.stock_actual.
tipo_stock = NINGUNO	Producto lógico o agrupador sin control directo de existencias.
disponibilidad_menu	Horario comercial del producto: TODO_DIA, DESAYUNO o ALMUERZO. HorarioMenu decide si se puede vender.
minimo / maximo	Cantidad de elecciones obligatorias y límite dentro de un grupo.
permite_repetir	Permite elegir varias veces la misma opción si el grupo acepta más de una selección.
permite_personalizar	Decide si los componentes internos abren cambios de ingredientes.
predeterminada	Selección inicial para acelerar el trabajo del cajero; debe seguir cumpliendo reglas.
estado	Borrado lógico. Oculta un registro sin destruir relaciones o ventas históricas.
snapshot de nombres	pedido_detalle, pedido_opcion, pedido_producto y pedido_modificacion copian textos para preservar historia.
clave de pago	Hace idempotente el cobro: repetir la misma solicitud devuelve el mismo resultado y no duplica venta.

Ejemplo de cálculo de precio

Presentación WlMenú	Q61.00
Cambio a WlPatatas	+ Q5.00
Extra queso en la hamburguesa	+ Q3.00
Precio unitario configurado	Q69.00
Cantidad 2	x 2
Subtotal	Q138.00

El precio se calcula con BigDecimal. No se debe usar double para dinero porque algunos decimales binarios generan diferencias de centavos.

Ejemplo de consumo de inventario

Si Big Mac es RECETA, PagoDAO suma la cantidad_default de cada ingrediente, resta los ingredientes quitados y agrega cantidad_extra por cada extra. Si Coca Cola es DIRECTO, descuenta unidades del producto. Todo ocurre dentro de la misma transacción del pedido.

10. Casos prácticos para demostrar

Demostración	Qué deben explicar
Caso A – Individual	Abrir Big Mac, elegir Individual, quitar pepinillo, agregar extra queso y añadir al carrito. Explicar que la modificación pertenece al ProductoPedido interno.
Caso B – WIMenú	Abrir la misma tarjeta, cambiar a WIMenú, elegir complemento y bebida. Mostrar cómo cada opción proviene de un grupo con mínimo=1 y máximo=1.
Caso C – Combo predefinido	Abrir Caja Grande, configurar sus elecciones, modificar cada hamburguesa por separado y observar un único subtotal de combo.
Caso D – Cajita	Elegir entrada, complemento, bebida y juguete. Demostrar opciones mutuamente excluyentes y recargos.
Caso E – Horario	Intentar ver un desayuno fuera de horario. ProductoDAO/HorarioMenu deben ocultarlo o ConfiguracionMenuDAO rechazarlo.
Caso F – Inventario	Registrar una entrada, revisar stock, cobrar un producto y comprobar la salida automática ligada al pedido.
Caso G – Reporte	Elegir la fecha del pedido, filtrar por cajero/método/servicio y exportar CSV.

Objeto conceptual de una línea del carrito

```
LineaPedido {  
    presentacion: 'WIMenú', cantidad: 1, precioUnitario: 69.00,  
    opciones: [  
        { grupo: 'Principal', productos: [Big Mac { SIN Pepinillo, EXTRA Queso }] },  
        { grupo: 'Complemento', productos: [WIPatatas] },  
        { grupo: 'Bebida', productos: [Coca Cola] }  
    ]  
}
```

En el código real estos datos son records Java: LineaPedido, OpcionPedido, ProductoPedido y ModificacionPedido. Cada método con... crea una copia con una lista nueva, evitando modificar accidentalmente la selección anterior.

11. Seguridad, consistencia y puntos de mejora

Tema	Qué hace hoy	Recomendación profesional
Contraseñas	SeguridadContrasena genera/verifica hashes y Login migra claves antiguas.	No aceptar contraseñas planas nuevas y usar un algoritmo adaptativo como BCrypt/Argon2.
Credenciales MySQL	Están configuradas en Conexion.java.	Mover URL, usuario y contraseña a variables de entorno o un archivo local no versionado.
SQL injection	Las consultas usan PreparedStatement en las operaciones principales.	Mantener parámetros; nunca concatenar texto ingresado por el usuario.
Cobro doble	pago_operacion usa una clave única para idempotencia.	Generar una clave por intento lógico y reutilizarla al reintentar por error de red.
Inventario	PagoDAO bloquea y descuenta dentro de la transacción.	Mantener índices y pruebas concurrentes cuando haya múltiples cajas.
Borrado	estado permite desactivar registros.	Evitar DELETE de datos con historia, salvo tablas auxiliares seguras.
GUI Builder	Los .form dependen de bloques protegidos en Java.	Editar el diseño desde NetBeans y conservar las marcas GEN-BEGIN/GEN-END.

Guía de diagnóstico

Síntoma	Qué revisar
No conecta a MySQL	Revisar servicio, puerto 3306, base waldonalds, credenciales y JAR JDBC.
Producto no aparece	Revisar estado, categoría, presentación activa, disponibilidad_menu y consulta de subcategoría.
No aparece WIMenú	Debe existir presentacion_menu tipo MENU activa con grupos/opciones/componentes válidos.
No deja agregar al carrito	Algún grupo no cumple mínimo/máximo o un componente está inactivo/fuera de horario.
Cobro falla por stock	Comparar stock_actual con consumo de recetas y productos DIRECTO.
NetBeans dice form corrupted	Restaurar comentarios protegidos y mantener sincronizados .java y .form.
La interfaz se congela	Mover consultas largas a SwingWorker y actualizar Swing en done().

12. Pruebas y verificación

Prueba	Objetivo
FormReportesTest.java	Prueba ejecutable FormReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
InventarioPanelTest.java	Prueba ejecutable InventarioPanelTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
MenuAnimacionTest.java	Prueba ejecutable MenuAnimacionTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
PagosTest.java	Prueba ejecutable PagosTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
ReporteDAOTest.java	Prueba ejecutable ReporteDAOTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
ReportesTest.java	Prueba ejecutable ReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
scripts/ValidarDatosMenu.java	Ejecuta scripts en una base temporal y valida que no haya duplicados, recetas incompletas ni relaciones inválidas.
VisualCheckMenuAdmin.java	Abre una comprobación visual del menú administrativo.

Estas pruebas son clases con main, no JUnit. Se compilan con el proyecto y se ejecutan individualmente. Algunas usan MySQL real; antes de correrlas hay que leer si crean o eliminan datos temporales.

Qué significa una prueba útil

- Preparar un estado conocido.
- Ejecutar una acción concreta.
- Comparar el resultado real con el esperado.
- Limpiar datos temporales y fallar con un mensaje claro si algo no coincide.

13. Guion sugerido para la presentación

Tiempo	Sección	Mensaje
1 minuto	Problema	Un restaurante necesita controlar opciones, personalizaciones, horarios e inventario; un combo no puede ser un producto estático.
2 minutos	Arquitectura	Mostrar las capas y explicar que Swing no contiene directamente toda la lógica SQL.
3 minutos	Cajero	Demostrar Individual/WIMenú, grupo obligatorio, personalización interna y carrito.
2 minutos	Cobro	Explicar transacción, revalidación, idempotencia y descuento de inventario.
3 minutos	Administrador	Usuarios, menú, recetas/presentaciones, inventario y reportes.
2 minutos	Base de datos	Mostrar el recorrido producto -> presentación -> grupo -> opción -> componente y pedido -> detalle -> opción -> producto -> modificación.
1 minuto	Cierre	Ventajas, pruebas y mejoras futuras.

Preguntas que podrían hacerles

Pregunta	Respuesta breve
¿Por qué no guardaron el combo como texto?	Porque cocina, inventario y personalización necesitan conocer los productos internos y cada elección.
¿Por qué existen snapshots de nombres?	Para que una venta histórica no cambie cuando se renombra un producto o una opción.
¿Por qué BigDecimal?	Porque el dinero requiere precisión decimal exacta.
¿Qué evita vender sin stock?	PagoDAO calcula consumo, bloquea filas con FOR UPDATE y confirma dentro de una transacción.
¿Para qué sirve pago_operacion?	Evita que un reintento registre dos pedidos idénticos.
¿Cuál es la diferencia entre DAO y modelo?	El modelo contiene datos; el DAO sabe leerlos y escribirlos en la base.
¿Por qué hay .java y .form?	Java contiene comportamiento; .form conserva el diseño editable de NetBeans.

14. Glosario

Término	Definición
API	Contrato mediante el que dos piezas de software se comunican.
CRUD	Create, Read, Update, Delete: operaciones básicas de mantenimiento.
DAO	Clase que centraliza acceso a datos de un caso de uso.
DTO/modelo	Objeto que transporta datos tipados.
JDBC	API estándar de Java para conectarse y ejecutar SQL.
Swing	Biblioteca de interfaz gráfica de escritorio de Java.
EDT	Hilo de Swing encargado de dibujar y procesar eventos visuales.
GUI Builder	Diseñador visual de NetBeans que mantiene archivos .form y parte de initComponents.
Clave primaria	Columna que identifica una fila de forma única.
Clave foránea	Columna que apunta a una fila de otra tabla.
Transacción	Grupo de cambios que se confirma completo o se revierte completo.
Rollback	Deshacer los cambios pendientes de una transacción fallida.
Idempotencia	Repetir la misma petición no produce un segundo efecto.
Hash	Resultado no reversible usado para verificar contraseñas sin guardarlas en claro.
Snapshot	Copia histórica de un valor tal como era en el momento de la venta.
Listener	Objeto que reacciona a un evento del usuario.
Renderizador	Código que decide cómo se dibuja una celda o control.
Paginación	Dividir muchos registros en páginas pequeñas.
Stock	Existencia disponible de un producto o ingrediente.

15. Catálogo de archivos

Esta sección permite localizar rápidamente una clase. El anexo posterior contiene la explicación línea por línea.

DOCUMENTACION_GESTION_USUARIOS.md

Archivo	Líneas/tamaño	Utilidad
DOCUMENTACION_GESTION_USUARIOS.md	445 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

DOCUMENTACION_INVENTARIO.md

Archivo	Líneas/tamaño	Utilidad
DOCUMENTACION_INVENTARIO.md	175 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

DOCUMENTACION_MENU_CONFIGURABLE.md

Archivo	Líneas/tamaño	Utilidad
DOCUMENTACION_MENU_CONFIGURABLE.md	77 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

DOCUMENTACION_PAGOS.md

Archivo	Líneas/tamaño	Utilidad
DOCUMENTACION_PAGOS.md	36 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

GUIA_REPORTES.md

Archivo	Líneas/tamaño	Utilidad
GUIA_REPORTES.md	92 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

MANUAL_TABLA_ADMINISTRATIVA.md

Archivo	Líneas/tamaño	Utilidad
MANUAL_TABLA_ADMINISTRATIVA.md	326 líneas	Documentación complementaria escrita durante el desarrollo del proyecto.

VisualCheckMenuAdmin.java

Archivo	Líneas/tamaño	Utilidad
VisualCheckMenuAdmin.java	42 líneas	Archivo de apoyo del proyecto: VisualCheckMenuAdmin.java.

build.xml

Archivo	Líneas/tamaño	Utilidad
build.xml	73 líneas	Archivo Ant que delega la compilación y ejecución a la infraestructura generada por NetBeans.

imagenes_productos

Archivo	Líneas/tamaño	Utilidad
imagenes_productos/bigmac.png	2416.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.

lib

Archivo	Líneas/tamaño	Utilidad
lib/absolutelayout/AbsoluteLayout.jar	16.8 KB	Librería de NetBeans para AbsoluteLayout.
lib/CopyLibs/org-netbeans-modules-java-j2seproject-copylibstask.jar	108.7 KB	Librería compilada requerida por el proceso de construcción.
lib/mysql-connector-j-26.7.0.jar	2552.0 KB	Driver JDBC que permite a Java comunicarse con MySQL.
lib/nblibraries.properties	7 líneas	Archivo de propiedades clave=valor utilizado por NetBeans o por la compilación.

manifest.mf

Archivo	Líneas/tamaño	Utilidad
manifest.mf	3 líneas	Archivo de apoyo del proyecto: manifest.mf.

nbproject

Archivo	Líneas/tamaño	Utilidad
nbproject/build-impl.xml	1796 líneas	Configuración XML utilizada por NetBeans/Ant para describir y construir el proyecto.
nbproject/genfiles.properties	8 líneas	Archivo de propiedades clave=valor utilizado por NetBeans o por la compilación.
nbproject/project.properties	100 líneas	Configuración del proyecto NetBeans: Java 21, librerías, clase principal y rutas de compilación.
nbproject/project.xml	18 líneas	Configuración XML utilizada por NetBeans/Ant para describir y construir el proyecto.

scripts

Archivo	Líneas/tamaño	Utilidad
scripts/generar_datos_menu_nuevo.ps1	1446 líneas	Herramienta auxiliar generar_datos_menu_nuevo usada para generar o validar datos del proyecto.
scripts/ValidarDatosMenu.java	235 líneas	Herramienta auxiliar ValidarDatosMenu usada para generar o validar datos del proyecto.

sql

Archivo	Líneas/tamaño	Utilidad
sql/agregar_wlmenu_producto_prueba.sql	478 líneas	Script SQL agregar_wlmenu_producto_prueba. Prepara, migra o prueba una parte de la base de datos.
sql/base_datos_completa.sql	197 líneas	Esquema principal. Crea la base waldonalds y sus tablas relacionales.
sql/datos_menu_nueva_base.sql	1544 líneas	Carga idempotente del catálogo, ingredientes, recetas, presentaciones, WIMenús, Cajitas y combos.
sql/migracion_productos_existentes.sql	47 líneas	Script SQL migracion_productos_existentes. Prepara, migra o prueba una parte de la base de datos.
sql/migracion_turnos_usuario.sql	8 líneas	Script SQL migracion_turnos_usuario. Prepara, migra o prueba una parte de la base de datos.
sql/pagos.sql	9 líneas	Script SQL pagos. Prepara, migra o prueba una parte de la base de datos.

src/BaseDatos

Archivo	Líneas/tamaño	Utilidad
src/BaseDatos/BaseDatos.java	13 líneas	Archivo de apoyo del proyecto: src/BaseDatos/BaseDatos.java.

src/CRUD

Archivo	Líneas/tamaño	Utilidad
src/CRUD/CRUD.java	13 líneas	Operaciones de creación, consulta, actualización y estado para el catálogo usando JDBC.
src/CRUD/IngredienteCRUD.java	256 líneas	Operaciones de creación, consulta, actualización y estado para ingrediente usando JDBC.
src/CRUD/ProductoCRUD.java	768 líneas	Operaciones de creación, consulta, actualización y estado para producto usando JDBC.
src/CRUD/UsuarioCRUD.java	340 líneas	Operaciones de creación, consulta, actualización y estado para usuario usando JDBC.

src/Componentes

Archivo	Líneas/tamaño	Utilidad
src/Componentes/AnimacionLetras.java	141 líneas	Componente visual reutilizable AnimacionLetras. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BarraTituloForm.form	102 líneas	Metadatos visuales de NetBeans para diseñar BarraTituloForm. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/Componentes/BarraTituloForm.java	467 líneas	Componente visual reutilizable BarraTituloForm. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/Boton.java	10 líneas	Componente visual reutilizable Boton. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BotonCategoria.java	240 líneas	Componente visual reutilizable BotonCategoria. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Archivo	Líneas/tamaño	Utilidad
src/Componentes/BotonDerretido.java	258 líneas	Componente visual reutilizable BotonDerretido. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BotonDesplegable.java	462 líneas	Componente visual reutilizable BotonDesplegable. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BotonLetras.java	232 líneas	Componente visual reutilizable BotonLetras. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BotonMenuLateral.java	275 líneas	Componente visual reutilizable BotonMenuLateral. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/BotonRedondeado.java	221 líneas	Componente visual reutilizable BotonRedondeado. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/CampoBusquedaAdmin.java	152 líneas	Componente visual reutilizable CampoBusquedaAdmin. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/CargaW.java	317 líneas	Componente visual reutilizable CargaW. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/DesplazamientoSuave.java	89 líneas	Componente visual reutilizable DesplazamientoSuave. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/PanelCircular.java	126 líneas	Componente visual reutilizable PanelCircular. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/PanelFlotante.java	233 líneas	Componente visual reutilizable PanelFlotante. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/PanelProductosVertical.java	78 líneas	Componente visual reutilizable PanelProductosVertical. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/Scroll_Categorias.java	465 líneas	Componente visual reutilizable Scroll_Categorias. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/Scroll_Vertical.java	289 líneas	Componente visual reutilizable Scroll_Vertical. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/TablaAdministrativa.java	523 líneas	Componente visual reutilizable TablaAdministrativa. Mantiene el estilo de la aplicación y evita duplicar código de Swing.
src/Componentes/TarjetaProducto.java	298 líneas	Componente visual reutilizable TarjetaProducto. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

src/Conexion

Archivo	Líneas/tamaño	Utilidad
src/Conexion/Conexion.java	49 líneas	Centraliza la conexión JDBC con MySQL y devuelve objetos Connection para los DAO y CRUD.

src/DAO

Archivo	Líneas/tamaño	Utilidad
src/DAO/ConfiguracionMenuDAO.java	304 líneas	Carga y administra presentaciones, grupos, opciones, componentes y recetas configurables.
src/DAO/InventarioDAO.java	326 líneas	Consulta existencias combinadas de productos directos e ingredientes y registra entradas, salidas y ajustes.
src/DAO/PagoDAO.java	324 líneas	Caso de uso transaccional de cobro. Revalida reglas, registra el pedido completo, descuenta inventario y genera el comprobante.
src/DAO/ProductoDAO.java	381 líneas	Consulta los productos visibles para el cajero por categoría, subcategoría y horario.
src/DAO/ReporteDAO.java	136 líneas	Obtiene indicadores y ventas del día para el panel de reportes.

src/Font

Archivo	Líneas/tamaño	Utilidad
src/Font/DMSans/DMSans-Black.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-BlackItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Bold.ttf	54.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-BoldItalic.ttf	59.7 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-ExtraBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-ExtraBoldItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-ExtraLight.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-ExtraLightItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Italic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Light.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-LightItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Medium.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-MediumItalic.ttf	60.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Regular.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-SemiBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-SemiBoldItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-Thin.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans-ThinItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Black.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-BlackItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Bold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-BoldItalic.ttf	59.7 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-ExtraBold.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-ExtraBoldItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-ExtraLight.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-ExtraLightItalic.ttf	60.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Italic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Light.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-LightItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Medium.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-MediumItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Regular.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.

Archivo	Líneas/tamaño	Utilidad
src/Font/DMSans/DMSans_18pt-SemiBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-SemiBoldItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-Thin.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_18pt-ThinItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Black.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-BlackItalic.ttf	59.4 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Bold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-BoldItalic.ttf	59.4 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-ExtraBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-ExtraBoldItalic.ttf	59.4 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-ExtraLight.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-ExtraLightItalic.ttf	60.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Italic.ttf	59.5 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Light.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-LightItalic.ttf	59.6 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Medium.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-MediumItalic.ttf	59.5 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Regular.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-SemiBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-SemiBoldItalic.ttf	59.5 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-Thin.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_24pt-ThinItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Black.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-BlackItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Bold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-BoldItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-ExtraBold.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-ExtraBoldItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-ExtraLight.ttf	55.2 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-ExtraLightItalic.ttf	60.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Italic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Light.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-LightItalic.ttf	59.9 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.

Archivo	Líneas/tamaño	Utilidad
src/Font/DMSans/DMSans_36pt-Medium.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-MediumItalic.ttf	60.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Regular.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-SemiBold.ttf	55.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-SemiBoldItalic.ttf	59.8 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-Thin.ttf	55.1 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.
src/Font/DMSans/DMSans_36pt-ThinItalic.ttf	60.0 KB	Archivo de fuente DM Sans para mantener la tipografía visual del sistema.

src/GUI_ADMINISTRADOR

Archivo	Líneas/tamaño	Utilidad
src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.form	248 líneas	Metadatos visuales de NetBeans para diseñar CatalogoIngredientesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.java	568 líneas	Interfaz del módulo administrador: CatalogoIngredientesPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/CombosPanel.form	33 líneas	Metadatos visuales de NetBeans para diseñar CombosPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/CombosPanel.java	46 líneas	Interfaz del módulo administrador: CombosPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/ConfiguracionMenuAdminDialog.java	266 líneas	Editor administrativo de presentaciones, grupos, opciones, componentes y receta del producto.
src/GUI_ADMINISTRADOR/DashboardPanel.form	410 líneas	Metadatos visuales de NetBeans para diseñar DashboardPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/DashboardPanel.java	211 líneas	Interfaz del módulo administrador: DashboardPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/GestionInventarioPanel.form	813 líneas	Metadatos visuales de NetBeans para diseñar GestionInventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/GestionInventarioPanel.java	1257 líneas	Pantalla compuesta de inventario con pestañas Existencias, Ingredientes y Movimientos.
src/GUI_ADMINISTRADOR/GestionMenuPanel.form	739 líneas	Metadatos visuales de NetBeans para diseñar GestionMenuPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/GestionMenuPanel.java	1759 líneas	Mantenimiento de productos del menú: filtros, resumen, alta, edición, estado y configuración comercial.
src/GUI_ADMINISTRADOR/IconoMenuAdmin.java	29 líneas	Interfaz del módulo administrador: IconoMenuAdmin. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/IngredienteFormDialog.java	46 líneas	Interfaz del módulo administrador: IngredienteFormDialog. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/IngredienteFormPanel.form	60 líneas	Metadatos visuales de NetBeans para diseñar IngredienteFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/IngredienteFormPanel.java	295 líneas	Interfaz del módulo administrador: IngredienteFormPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/IngredientesPanel.form	708 líneas	Metadatos visuales de NetBeans para diseñar IngredientesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/IngredientesPanel.java	1463 líneas	Mantenimiento del catálogo de ingredientes, stock mínimo, estado y movimientos.

Archivo	Líneas/tamaño	Utilidad
src/GUI_ADMINISTRADOR/InicioAdminForm.form	417 líneas	Metadatos visuales de NetBeans para diseñar InicioAdminForm. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/InicioAdminForm.java	314 líneas	Ventana contenedora del área administrativa y su barra de título.
src/GUI_ADMINISTRADOR/InventarioPanel.form	87 líneas	Metadatos visuales de NetBeans para diseñar InventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/InventarioPanel.java	133 líneas	Vista de existencias, búsqueda, filtros, paginación, entradas/salidas y exportación.
src/GUI_ADMINISTRADOR/MenuAdmin.form	338 líneas	Metadatos visuales de NetBeans para diseñar MenuAdmin. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/MenuAdmin.java	750 líneas	Menú lateral del administrador. Cambia entre dashboard, usuarios, menú, inventario y reportes.
src/GUI_ADMINISTRADOR/MovimientoInventarioFormDialog.java	46 líneas	Interfaz del módulo administrador: MovimientoInventarioFormDialog. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/MovimientoInventarioFormPanel.form	33 líneas	Metadatos visuales de NetBeans para diseñar MovimientoInventarioFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/MovimientoInventarioFormPanel.java	265 líneas	Interfaz del módulo administrador: MovimientoInventarioFormPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.form	20 líneas	Metadatos visuales de NetBeans para diseñar MovimientosInventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.java	257 líneas	Historial de entradas, salidas y ajustes de inventario.
src/GUI_ADMINISTRADOR/ProductoFormDialog.java	1798 líneas	Formulario emergente para crear o editar productos, categorías, precios, stock, horario e imagen.
src/GUI_ADMINISTRADOR/ReportesPanel.form	807 líneas	Metadatos visuales de NetBeans para diseñar ReportesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/ReportesPanel.java	1239 líneas	Dashboard de reportes con indicadores, filtros, tabla, paginación, exportación e impresión.
src/GUI_ADMINISTRADOR/UsuarioFormDialog.java	53 líneas	Interfaz del módulo administrador: UsuarioFormDialog. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/UsuarioFormPanel.form	435 líneas	Metadatos visuales de NetBeans para diseñar UsuarioFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/UsuarioFormPanel.java	674 líneas	Interfaz del módulo administrador: UsuarioFormPanel. Combina componentes visuales con acciones del usuario.
src/GUI_ADMINISTRADOR/UsuariosPanel.form	663 líneas	Metadatos visuales de NetBeans para diseñar UsuariosPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_ADMINISTRADOR/UsuariosPanel.java	1152 líneas	Gestión paginada de usuarios con búsqueda, filtros, resumen, alta, edición y activación.

src/GUI_CAJERO

Archivo	Líneas/tamaño	Utilidad
src/GUI_CAJERO/Cajero.form	72 líneas	Metadatos visuales de NetBeans para diseñar Cajero. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_CAJERO/Cajero.java	873 líneas	Ventana principal del cajero. Coordina menú, pedido, carrito y barra de título.
src/GUI_CAJERO/CategoriasPanel.form	153 líneas	Metadatos visuales de NetBeans para diseñar CategoriasPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_CAJERO/CategoriasPanel.java	197 líneas	Interfaz del módulo cajero: CategoriasPanel. Participa en la selección y venta de productos.

Archivo	Líneas/tamaño	Utilidad
src/GUI_CAJERO/ConfiguradorProductoDialog.java	469 líneas	Diálogo que aloja el configurador de producto y devuelve la línea terminada al carrito.
src/GUI_CAJERO/ConfiguradorProductoPanel.java	1045 líneas	Constructor visual de un producto: presentación Individual/WIMenú/Combo, grupos de opciones y cambios internos por ingrediente.
src/GUI_CAJERO/MenuCajeroPanel.form	64 líneas	Metadatos visuales de NetBeans para diseñar MenuCajeroPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_CAJERO/MenuCajeroPanel.java	109 líneas	Interfaz del módulo cajero: MenuCajeroPanel. Participa en la selección y venta de productos.
src/GUI_CAJERO/PedidoPanel.java	660 líneas	Carrito de compra. Mantiene líneas del pedido, cantidades, total, pago y acceso al último comprobante.
src/GUI_CAJERO/promoCategoriasPanel.form	53 líneas	Metadatos visuales de NetBeans para diseñar promoCategoriasPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_CAJERO/promoCategoriasPanel.java	51 líneas	Interfaz del módulo cajero: promoCategoriasPanel. Participa en la selección y venta de productos.
src/GUI_CAJERO/promocionesPanel.form	71 líneas	Metadatos visuales de NetBeans para diseñar promocionesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/GUI_CAJERO/promocionesPanel.java	123 líneas	Interfaz del módulo cajero: promocionesPanel. Participa en la selección y venta de productos.
src/GUI_CAJERO/SubCatDesayunos.java	119 líneas	Interfaz del módulo cajero: SubCatDesayunos. Participa en la selección y venta de productos.
src/GUI_CAJERO/SubCategoriasPanel.java	199 líneas	Interfaz del módulo cajero: SubCategoriasPanel. Participa en la selección y venta de productos.

src/Imágenes

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/advertencia_icono.png	9.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/antojos/derretido clasico.png	1518.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/hash brown.png	1053.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/papas.png	1389.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/salsa cheddar.png	1189.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/tostado de loroco.png	1642.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/tostado queso y frijol.png	1763.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/tostado queso y jamon.png	1589.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/tostado queso y tomate.png	1707.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/wcnuggets 4 pz.png	1265.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/antojos/wcpatatas.png	1079.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/BebidaLogo.png	881.0 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/bebidas/agua.png	799.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/cafe_con_leche.png	884.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/café.png	891.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/chocolate.png	945.3 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/bebidas/coca_cola.png	1468.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/coca_cola_zero.png	1490.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/fanta.png	1298.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/jamaica.png	1304.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/jugo_manzana.png	885.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/jugo_naranja.png	840.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/sprite.png	1344.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/te_lipton.png	1229.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/té_caliente.png	958.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/wlfrizz_A.M.png	1237.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/wlfrizz_blue.png	1319.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/wlfrizz_manzanaverde.png	1545.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bebidas/wlfrizz_pink.png	1287.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/bolsa_icono.png	218.5 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/caja_icono.png	283.4 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/cajita feliz/cajita feliz de derretido almuerzo-cena.png	1882.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de derretido.png	1709.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de hamburguesa jr.png	1799.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de hamburguesa.png	1772.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de hotcakes.png	1603.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de pollo wlcrispy.png	1775.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de quesoburguesa.png	1916.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de wlmuffin de frijol.png	1698.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de wlmuffin de huevo y queso.png	1668.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de wlmuffin de salchicha.png	1767.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/cajita feliz/cajita feliz de wlnuggets.png	1732.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/CargaW.png	115.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/cheque.png	10.9 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Desayunos/burrito.png	1248.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/burritos.png	1104.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/Caja_grande_deluxe.png	1705.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/Caja_grande_desayuno.png	1727.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/desayuno_clásico.png	1608.1 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/Desayunos/desayuno_deluxe.png	1470.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/desayuno_tradicional.png	1679.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/egg_wlmuffin.png	2017.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/egg_wlmuffin_doble.png	1910.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/eggwlmuffin_doble_huevo.png	1979.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/hotcakes.png	1230.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/pan_wlgriddle.png	1953.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wcmuffin_tocino_doblehuevo.png	1890.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlgriddle de salchicha y huevo.png	2203.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlgriddle_salchicha.png	1996.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlgriddle_tocinohuevo.png	1999.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlMuffi_Salchich_Doble_y_Huevo.png	2098.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffiinsuper_chapin_salchicha.png	1877.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_chapin_jamon.png	2153.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_chapin_salchicha.png	2109.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_cheddar_wlmelt.png	2116.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_huevofrijol.png	1889.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_huevoqueso.png	1698.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_salchicha.png	2010.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlMuffin_Salchicha_y_doble_huevo.png	2039.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_salchichay_huevo.png	1992.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_tocino_doble_huevo.png	1837.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffin_tocinohuevo.png	1930.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/Desayunos/wlmuffiinsuper_chapin_con_jamon.png	2028.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/flecha_boton.png	0.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Fondo (1).png	138.7 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Fondo (2).png	618.2 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Fondo.png	674.5 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/FondoLogo.png	157.8 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/HAMBURGUESAS/10 mcnuggets de pollo.png	1469.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/bacon cheddar mcmelt.png	2112.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/bacon_cheddar_mcmelt.png	2112.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/bacon_cheddar_wlmelt.png	2591.8 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/imagenes/HAMBURGUESAS/big tasty bacon doble.png	2744.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big tasty bacon.png	2671.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big tasty de pollo.png	2183.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big tasty doble (1).png	2715.6 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big tasty doble .png	2715.6 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big tasty.png	2425.0 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big_tasty.png	2425.0 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/big_tasty_pollo.png	2183.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/bigmac.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/bigtasty_bacon.png	2671.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/bigtasty_bacon_doble.png	2744.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/c.png	1897.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/clasica gourmet doble res doble.png	1704.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/clasica_gourmet.png	1699.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/clasica_gourmet_doble_res_doble.png	1704.5 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/cuarto de libra bacon con queso.png	2472.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/cuarto de libra bacon doble con queso.png	2586.8 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/cuarto de libra con queso.png	1906.2 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/cuarto_libra_bacon_doble_queso.png	2586.8 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble big mac.png	3050.7 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble cuarto de libra con queso.png	2297.0 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble hamburguesa con queso.png	1914.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble_bigmac.png	3050.7 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble_cuarto_libracon_queso.png	2297.0 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/doble_hamburguesa_con_queso.png	1914.4 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/hamburguesa con queso.png	1765.6 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/hamburguesa deluxe.png	2468.8 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/hamburguesa.png	1712.8 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/hamburguesa_deluxe.png	2468.8 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/hamburguesacon_queso.png	1765.6 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/McCrispy Chicken Deluxe.png	2263.7 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/mcnifica de res doble.png	2236.9 KB	Imagen de producto usada por las tarjetas del menú.
src/imagenes/HAMBURGUESAS/mcnifica doble.png	1922.9 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/HAMBURGUESAS/mcnifica_doble.png	1922.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/mcnifica_res_doble.png	2236.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pico_guacamol_gourmet_res.png	1844.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo mc crispy 1 pieza.png	1024.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo mc crispy 10 piezas.png	1264.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo mc crispy bacon ranch.png	1979.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo mc crispy dos piezas.png	942.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo_wlcrispy.png	1264.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollo_wlcrispy_1_pieza.png	1024.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/pollowlcrispy _dospiezas.png	942.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/quesoburguesa.png	1940.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/sandwich mcpollo doble.png	2146.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/sandwich_mcpollo_doble.png	2146.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/smoke tocino gourmet de res.png	1772.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/smoke tocino gourmet doble.png	2021.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/smoke_tocino_gourmetres.png	1772.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/triple bacon.png	2613.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/triple hamburguesa con queso.png	2838.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/triple_bacon.png	2613.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/triplehamburguesa_queso.png	2838.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/wcdouble.png	1889.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/WICrispy_Chicken_Deluxe.png	2263.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/HAMBURGUESAS/wlnuggets.png	1469.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/icono_admin.png	248.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_agregar.png	0.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_agregarusuario.png	170.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_editarUsuario.png	201.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_usuario.png	172.7 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_usuario_inactivo.png	167.7 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/icono_usuariostotal.png	364.7 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/IconoAdmin.png	602.0 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/ICONOS_BARRA_TITULO/icono_cerrar.png	0.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/ICONOS_BARRA_TITULO/icono_maximizar.png	0.5 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/ICONOS_BARRA_TITULO/icono_minimizar.png	0.2 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/ICONOS_BARRA_TITULO/icono_restaurar.png	0.8 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/iconoUsuario.png	808.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Imagen.java	13 líneas	Archivo de apoyo del proyecto: src/Imagenes/Imagen.java.
src/Imagenes/lista_icono.png	352.9 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Logo.png	467.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/LogoW.png	115.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/MesaLogo.png	1394.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/MiniLogo 2.png	170.4 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/MiniLogo.png	99.3 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Panel2.png	749.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/PapasLogo.png	1584.6 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/para compartir/bucket para todos.png	2302.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/Bucket Pollo McCrispy ® Para Tres.png	1543.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/Bucket Pollo McCrispy ® Snack.png	2213.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/Bucket Pollo McCrispy.png	1360.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/caja de 24 wlnuggets.png	2464.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/caja grande con postre (1).png	2253.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/caja grande snack.png	2103.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/caja grande.png	2256.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/para compartir/caja_grande_postre.png	2253.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mc cono vainilla.png	524.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry m&ms fresa.png	1482.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry m&ms.png	1587.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry oreo caramelo.png	1699.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry oreo fresa.png	1640.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry oreo.png	1571.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mcflurry oreo chocolate.png	1670.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/mflurry m&ms chocolate.png	1767.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/pastel de manzana.png	19.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/pastel de queso.png	16.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/sundae caramelo.png	689.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/postres/sundae chocolate.png	719.6 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/postres/sundae fresa.png	808.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/productos/bigmac_1789280814794.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/productos/bigmac_1789346320008.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/productos/bigmac_1789346343047.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/productos/bigmac_1790647194892.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/productos/bigmac_1790647252664.png	2416.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/PuntosLogo.png	749.2 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/quetzal_icono.png	196.2 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/ticket_icono.png	280.0 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/Usuarios.png	8.0 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/visualizar.png	1.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
src/Imagenes/wlcafe/brownie_bites.png	1617.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/cafe_blend_grano.png	1151.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/cafe_blend_molido.png	1437.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/cafe_guatemalteco.png	917.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/capuccino.png	1019.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/cheescake_fresa.png	1600.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/cheescake_mora.png	1510.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/chocolate_caliente.png	766.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/Frappe original.png	1254.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/frappe_caramelo.png	1322.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/frappe_vainilla.png	1262.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/frappé chocolate.png	1426.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/Frappé Oreo.png	1515.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/frappé vainilla light.png	1271.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/Horchata.png	1096.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/iced coffe caramelo.png	1231.4 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/iced coffe chocolate.png	1249.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/lced coffe horchata.png	1199.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/lced coffe original.png	1229.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/iced coffe vainilla light.png	1106.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/lced coffe vainilla.png	1341.5 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/latte.png	891.3 KB	Imagen de producto usada por las tarjetas del menú.

Archivo	Líneas/tamaño	Utilidad
src/Imagenes/wlcafe/pastel_elote.png	1568.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/smothie_berries.png	1458.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/smothie_mango.png	1357.7 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/te_chai_vainilla.png	998.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/te_guatealteco_melocoton_mix.png	673.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/te_guatemalteco_borabora.png	820.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/te_guatemalteco_menta_fusion.png	676.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/techai_original.png	906.2 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/techai_te_verde.png	1014.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/techai_vainilla_light.png	910.1 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/teguatemalteco_vainilla_relax.png	831.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/tostado_queso_frijol.png	1768.3 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/tostado_queso_tomate.png	1647.6 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/wl fizz_A.M..png	1237.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/wlfrizz_blue.png	1319.0 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/wlfrizz_manzanaverde.png	1545.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/wlfrizz_pink.png	1287.9 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/wlcafe/wlpop_chocolate.png	1351.8 KB	Imagen de producto usada por las tarjetas del menú.
src/Imagenes/x_icono.png	3.4 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.

src/Labels

Archivo	Líneas/tamaño	Utilidad
src/Labels/LabelEscalable.java	151 líneas	Control de entrada o etiqueta personalizado LabelEscalable, diseñado para reutilizar estilos y comportamiento.
src/Labels/Prueba.form	76 líneas	Metadatos visuales de NetBeans para diseñar Prueba. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/Labels/Prueba.java	112 líneas	Control de entrada o etiqueta personalizado Prueba, diseñado para reutilizar estilos y comportamiento.
src/Labels/TextBoxPassword.java	225 líneas	Control de entrada o etiqueta personalizado TextBoxPassword, diseñado para reutilizar estilos y comportamiento.
src/Labels/Textbox_Contrasena.java	604 líneas	Control de entrada o etiqueta personalizado Textbox_Contrasena, diseñado para reutilizar estilos y comportamiento.
src/Labels/TextBox_Login.java	710 líneas	Control de entrada o etiqueta personalizado TextBox_Login, diseñado para reutilizar estilos y comportamiento.

src/Login

Archivo	Líneas/tamaño	Utilidad
src/Login/Login.form	203 líneas	Metadatos visuales de NetBeans para diseñar Login. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.
src/Login/Login.java	396 líneas	Pantalla de autenticación. Busca al usuario, verifica la contraseña, valida estado y turno, guarda la sesión y abre el panel correcto según el rol.

src/Main

Archivo	Líneas/tamaño	Utilidad
src/Main/main.java	33 líneas	Punto de entrada. Inicia la interfaz en el hilo de Swing, muestra la pantalla de carga y luego abre el inicio de sesión.

src/Modelos

Archivo	Líneas/tamaño	Utilidad
src/Modelos/ArticuloInventario.java	70 líneas	Modelo de datos ArticuloInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ComponenteMenu.java	11 líneas	Modelo de datos ComponenteMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ConfiguracionProducto.java	11 líneas	Modelo de datos ConfiguracionProducto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/GrupoMenu.java	12 líneas	Modelo de datos GrupoMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/Ingrediente.java	33 líneas	Modelo de datos Ingrediente. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/IngredienteProducto.java	14 líneas	Modelo de datos IngredienteProducto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/LineaPedido.java	51 líneas	Modelo de datos LineaPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ModificacionPedido.java	15 líneas	Modelo de datos ModificacionPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/MovimientoInventario.java	15 líneas	Modelo de datos MovimientoInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/OpcionMenu.java	16 líneas	Modelo de datos OpcionMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/OpcionPedido.java	16 líneas	Modelo de datos OpcionPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/PaginalIngredientes.java	6 líneas	Modelo de datos PaginalIngredientes. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/PaginalInventario.java	8 líneas	Modelo de datos PaginalInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/PaginaProductos.java	9 líneas	Modelo de datos PaginaProductos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Archivo	Líneas/tamaño	Utilidad
src/Modelos/PaginaUsuarios.java	11 líneas	Modelo de datos PaginaUsuarios. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/PresentacionMenu.java	14 líneas	Modelo de datos PresentacionMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/Producto.java	88 líneas	Modelo de datos Producto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ProductoPedido.java	15 líneas	Modelo de datos ProductoPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/Productos.java	70 líneas	Modelo de datos Productos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ResumenIngredientes.java	5 líneas	Modelo de datos ResumenIngredientes. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ResumenProductos.java	9 líneas	Modelo de datos ResumenProductos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/ResumenUsuarios.java	9 líneas	Modelo de datos ResumenUsuarios. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/SolicitudPago.java	37 líneas	Modelo de datos SolicitudPago. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.
src/Modelos/Usuario.java	66 líneas	Modelo de datos Usuario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

src/Utilidades

Archivo	Líneas/tamaño	Utilidad
src/Utilidades/GestorImagenProducto.java	462 líneas	Utilidad compartida GestorImagenProducto. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/HorarioMenu.java	33 líneas	Utilidad compartida HorarioMenu. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/HorarioTurno.java	41 líneas	Utilidad compartida HorarioTurno. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/IconosAdmin.java	75 líneas	Utilidad compartida IconosAdmin. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/IconosUsuarios.java	172 líneas	Utilidad compartida IconosUsuarios. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/ImageCache.java	80 líneas	Utilidad compartida ImageCache. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/PagoPendienteStore.java	107 líneas	Utilidad compartida PagoPendienteStore. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/ReporteCsv.java	53 líneas	Utilidad compartida ReporteCsv. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/SeguridadContrasena.java	105 líneas	Utilidad compartida SeguridadContrasena. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/SesionUsuario.java	43 líneas	Utilidad compartida SesionUsuario. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/TemaAdmin.java	76 líneas	Utilidad compartida TemaAdmin. Resuelve una tarea transversal usada por varias pantallas.
src/Utilidades/UltimoComprobanteStore.java	46 líneas	Utilidad compartida UltimoComprobanteStore. Resuelve una tarea transversal usada por varias pantallas.

src/Vistas

Archivo	Líneas/tamaño	Utilidad
src/Vistas/PantallaCarga.java	156 líneas	Archivo de apoyo del proyecto: src/Vistas/PantallaCarga.java.

test

Archivo	Líneas/tamaño	Utilidad
test/FormReportesTest.java	73 líneas	Prueba ejecutable FormReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
test/InventarioPanelTest.java	189 líneas	Prueba ejecutable InventarioPanelTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
test/MenuAnimacionTest.java	49 líneas	Prueba ejecutable MenuAnimacionTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
test/PagosTest.java	167 líneas	Prueba ejecutable PagosTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
test/ReporteDAOTest.java	48 líneas	Prueba ejecutable ReporteDAOTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.
test/ReportesTest.java	85 líneas	Prueba ejecutable ReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

tmp

Archivo	Líneas/tamaño	Utilidad
tmp/pdfs/manual_render/p0001.png	52.2 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0005.png	104.7 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0010.png	117.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0020.png	14.1 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0027.png	105.9 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0046.png	47.0 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p0500.png	178.5 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.
tmp/pdfs/manual_render/p1203.png	46.5 KB	Recurso visual de interfaz: logotipo, icono, fondo o indicador.

16. Anexo: explicación archivo por archivo y línea por línea

Formato de lectura: la primera columna es el número original, la segunda conserva el código y la tercera explica su función en lenguaje sencillo. Las credenciales literales se ocultan en el PDF por seguridad, pero su línea se sigue explicando.

Las explicaciones de líneas triviales como llaves, cierres XML o separaciones son deliberadamente breves. El bloque de estructura antes de cada archivo explica el significado conjunto de sus métodos.

16.1 build.xml

Archivo Ant que delega la compilación y ejecución a la infraestructura generada por NetBeans.

Tipo: .xml | Líneas: 73 | Archivo 1 de 157

1	<?xml version="1.0" encoding="UTF-8"?>		Declara un elemento XML de configuración, con sus atributos si corresponde.
2	<!-- You may freely edit this file. See commented blocks below for -->		Comentario XML para humanos o para NetBeans.
3	<!-- some examples of how to customize the build. -->		Comentario XML para humanos o para NetBeans.
4	<!-- (If you delete it and reopen the project it will be recreated.) -->		Comentario XML para humanos o para NetBeans.
5	<!-- By default, only the Clean and Build commands use this build script. -->		Comentario XML para humanos o para NetBeans.
6	<!-- Commands such as Run, Debug, and Test only use this build script if -->		Comentario XML para humanos o para NetBeans.
7	<!-- the Compile on Save feature is turned off for the project. -->		Comentario XML para humanos o para NetBeans.
8	<!-- You can turn off the Compile on Save (or Deploy on Save) setting -->		Comentario XML para humanos o para NetBeans.
9	<!-- in the project's Project Properties dialog box.-->		Comentario XML para humanos o para NetBeans.
10	<project name="Waldonalds" default="default" basedir=".">		Declara un elemento XML de configuración, con sus atributos si corresponde.
11	<description>Builds, tests, and runs the project Waldonalds.</description>		Declara un elemento XML de configuración, con sus atributos si corresponde.
12	<import file="nbproject/build-impl.xml"/>		Declara un elemento XML de configuración, con sus atributos si corresponde.
13	<!--		Comentario XML para humanos o para NetBeans.
14			Separación visual dentro del XML.
15	There exist several targets which are by default empty and which can be		Contenido de configuración o documentación que completa el archivo.
16	used for execution of your tasks. These targets are usually executed		Contenido de configuración o documentación que completa el archivo.
17	before and after some main targets. They are:		Contenido de configuración o documentación que completa el archivo.
18			Separación visual dentro del XML.
19	-pre-init:	called before initialization of project properties	Contenido de configuración o documentación que completa el archivo.
20	-post-init:	called after initialization of project properties	Contenido de configuración o documentación que completa el archivo.
21	-pre-compile:	called before javac compilation	Contenido de configuración o documentación que completa el archivo.
22	-post-compile:	called after javac compilation	Contenido de configuración o documentación que completa el archivo.
23	-pre-compile-single:	called before javac compilation of single file	Contenido de configuración o documentación que completa el archivo.
24	-post-compile-single:	called after javac compilation of single file	Contenido de configuración o documentación que completa el archivo.
25	-pre-compile-test:	called before javac compilation of JUnit tests	Contenido de configuración o documentación que completa el archivo.
26	-post-compile-test:	called after javac compilation of JUnit tests	Contenido de configuración o documentación que completa el archivo.
27	-pre-compile-test-single:	called before javac compilation of single JUnit test	Contenido de configuración o documentación que completa el archivo.
28	-post-compile-test-single:	called after javac compilation of single JUnit test	Contenido de configuración o documentación que completa el archivo.
29	-pre-jar:	called before JAR building	Contenido de configuración o documentación que completa el archivo.
30	-post-jar:	called after JAR building	Contenido de configuración o documentación que completa el archivo.

31	-post-clean:	called after cleaning build products	Contenido de configuración o documentación que completa el archivo.
32			Separación visual dentro del XML.
33	(Targets beginning with '-' are not intended to be called on their own.)		Contenido de configuración o documentación que completa el archivo.
34			Separación visual dentro del XML.
35	Example of inserting an obfuscator after compilation could look like this:		Contenido de configuración o documentación que completa el archivo.
36			Separación visual dentro del XML.
37	<target name="-post-compile">		Declara un elemento XML de configuración, con sus atributos si corresponde.
38	<obfuscate>		Declara un elemento XML de configuración, con sus atributos si corresponde.
39	<fileset dir="\${build.classes.dir}"/>		Declara un elemento XML de configuración, con sus atributos si corresponde.
40	</obfuscate>		Cierra el elemento XML abierto anteriormente.
41	</target>		Cierra el elemento XML abierto anteriormente.
42			Separación visual dentro del XML.
43	For list of available properties check the imported		Contenido de configuración o documentación que completa el archivo.
44	nbproject/build-impl.xml file.		Contenido de configuración o documentación que completa el archivo.
45			Separación visual dentro del XML.
46			Separación visual dentro del XML.
47	Another way to customize the build is by overriding existing main targets.		Contenido de configuración o documentación que completa el archivo.
48	The targets of interest are:		Contenido de configuración o documentación que completa el archivo.
49			Separación visual dentro del XML.
50	-init-macrodef-javac:	defines macro for javac compilation	Contenido de configuración o documentación que completa el archivo.
51	-init-macrodef-junit:	defines macro for junit execution	Contenido de configuración o documentación que completa el archivo.
52	-init-macrodef-debug:	defines macro for class debugging	Contenido de configuración o documentación que completa el archivo.
53	-init-macrodef-java:	defines macro for class execution	Contenido de configuración o documentación que completa el archivo.
54	-do-jar:	JAR building	Contenido de configuración o documentación que completa el archivo.
55	run:	execution of project	Contenido de configuración o documentación que completa el archivo.
56	-javadoc-build:	Javadoc generation	Contenido de configuración o documentación que completa el archivo.
57	test-report:	JUnit report generation	Contenido de configuración o documentación que completa el archivo.
58			Separación visual dentro del XML.
59	An example of overriding the target for project execution could look like this:		Contenido de configuración o documentación que completa el archivo.
60			Separación visual dentro del XML.
61	<target name="run" depends="waldonalds-impl.jar">		Declara un elemento XML de configuración, con sus atributos si corresponde.
62	<exec dir="bin" executable="launcher.exe">		Declara un elemento XML de configuración, con sus atributos si corresponde.
63	<arg file="\${dist.jar}"/>		Declara un elemento XML de configuración, con sus atributos si corresponde.
64	</exec>		Cierra el elemento XML abierto anteriormente.
65	</target>		Cierra el elemento XML abierto anteriormente.
66			Separación visual dentro del XML.
67	Notice that the overridden target depends on the jar target and not only on		Contenido de configuración o documentación que completa el archivo.
68	the compile target as the regular run target does. Again, for a list of available		Contenido de configuración o documentación que completa el archivo.
69	properties which you can use, check the target you are overriding in the		Contenido de configuración o documentación que completa el archivo.
70	nbproject/build-impl.xml file.		Contenido de configuración o documentación que completa el archivo.
71			Separación visual dentro del XML.
72	-->		Comentario XML para humanos o para NetBeans.
73	</project>		Cierra el elemento XML abierto anteriormente.

16.2 DOCUMENTACION_GESTION_USUARIOS.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 445 | Archivo 2 de 157

1	# Gestión de usuarios de Waldonald's	Título o subtítulo Markdown que organiza la documentación.
2		Separación entre párrafos del documento.
3	Esta guía explica la implementación de la pantalla de usuarios, los archivos	Texto de documentación destinado a explicar una decisión o procedimiento.
4	creados, los componentes reutilizados y el flujo de cada operación. El módulo	Texto de documentación destinado a explicar una decisión o procedimiento.
5	está escrito para Java 21, Swing, NetBeans y MySQL.	Texto de documentación destinado a explicar una decisión o procedimiento.
6		Separación entre párrafos del documento.
7	## 1. Resultado funcional	Título o subtítulo Markdown que organiza la documentación.
8		Separación entre párrafos del documento.
9	La pantalla permite:	Texto de documentación destinado a explicar una decisión o procedimiento.
10		Separación entre párrafos del documento.
11	- Mostrar total de usuarios, administradores activos, cajeros activos e	Elemento de una lista explicativa.
12	inactivos.	Texto de documentación destinado a explicar una decisión o procedimiento.
13	- Buscar por nombre, apellido, usuario o correo.	Elemento de una lista explicativa.
14	- Combinar la búsqueda con filtros de rol y estado.	Elemento de una lista explicativa.
15	- Limpiar todos los filtros.	Elemento de una lista explicativa.
16	- Mostrar exactamente 10 usuarios por página.	Elemento de una lista explicativa.
17	- Navegar únicamente con anterior, números de página y siguiente.	Elemento de una lista explicativa.
18	- Agregar usuarios validando datos y duplicados.	Elemento de una lista explicativa.
19	- Editar datos sin volver a escribir la contraseña.	Elemento de una lista explicativa.
20	- Cambiar la contraseña sólo cuando se solicita.	Elemento de una lista explicativa.
21	- Desactivar usuarios conservando su historial de pedidos.	Elemento de una lista explicativa.
22	- Exportar a CSV todos los resultados de los filtros actuales.	Elemento de una lista explicativa.
23		Separación entre párrafos del documento.
24	No existe un selector de "usuarios por página". El tamaño siempre es 10.	Texto de documentación destinado a explicar una decisión o procedimiento.
25		Separación entre párrafos del documento.
26	## 2. Requisito del esquema	Título o subtítulo Markdown que organiza la documentación.
27		Separación entre párrafos del documento.
28	El código asume que `usuario` ya tiene una columna `correo`. Si la base todavía	Texto de documentación destinado a explicar una decisión o procedimiento.
29	no la tiene, debe agregarse antes de ejecutar la pantalla:	Texto de documentación destinado a explicar una decisión o procedimiento.
30		Separación entre párrafos del documento.
31	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
32	ALTER TABLE usuario	Texto de documentación destinado a explicar una decisión o procedimiento.
33	ADD COLUMN correo VARCHAR(120) NULL AFTER usuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
34		Separación entre párrafos del documento.
35	UPDATE usuario	Texto de documentación destinado a explicar una decisión o procedimiento.
36	SET correo = CONCAT(usuario, '@waldonalds.com')	Texto de documentación destinado a explicar una decisión o procedimiento.
37	WHERE correo IS NULL;	Texto de documentación destinado a explicar una decisión o procedimiento.
38		Separación entre párrafos del documento.

39	ALTER TABLE usuario	Texto de documentación destinado a explicar una decisión o procedimiento.
40	MODIFY correo VARCHAR(120) NOT NULL;	Texto de documentación destinado a explicar una decisión o procedimiento.
41		Separación entre párrafos del documento.
42	ALTER TABLE usuario	Texto de documentación destinado a explicar una decisión o procedimiento.
43	ADD CONSTRAINT uq_usuario_correo UNIQUE (correo);	Texto de documentación destinado a explicar una decisión o procedimiento.
44	```	Abre o cierra un ejemplo de código con formato monoespaciado.
45		Separación entre párrafos del documento.
46	La implementación no ejecuta automáticamente este `ALTER TABLE`, porque los	Texto de documentación destinado a explicar una decisión o procedimiento.
47	cambios de estructura deben administrarse de forma explícita.	Texto de documentación destinado a explicar una decisión o procedimiento.
48		Separación entre párrafos del documento.
49	## 3. Archivos de la solución	Título o subtítulo Markdown que organiza la documentación.
50		Separación entre párrafos del documento.
51	### Archivos nuevos	Título o subtítulo Markdown que organiza la documentación.
52		Separación entre párrafos del documento.
53	Archivo Responsabilidad	Texto de documentación destinado a explicar una decisión o procedimiento.
54	--- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
55	`src/Modelos/Usuario.java` Representa un usuario sin exponer su contraseña.	Texto de documentación destinado a explicar una decisión o procedimiento.
56	`src/Modelos/PaginaUsuarios.java` Agrupa las filas de una página y el total filtrado.	Texto de documentación destinado a explicar una decisión o procedimiento.
57	`src/Modelos/ResumenUsuarios.java` Contiene los cuatro conteos superiores.	Texto de documentación destinado a explicar una decisión o procedimiento.
58	`src/CRUD/UsuarioCRUD.java` Ejecuta consultas, altas, ediciones y desactivaciones.	Texto de documentación destinado a explicar una decisión o procedimiento.
59	`src/Utilidades/SeguridadContrasena.java` Genera el SHA-256 usado por el login.	Texto de documentación destinado a explicar una decisión o procedimiento.
60	`src/Utilidades/SesionUsuario.java` Conserva el usuario autenticado.	Texto de documentación destinado a explicar una decisión o procedimiento.
61	`src/Utilidades/IconosUsuarios.java` Dibuja iconos vectoriales sin archivos externos.	Texto de documentación destinado a explicar una decisión o procedimiento.
62	`src/Componentes/CampoBusquedaAdmin.java` Buscador blanco, neutro y editable como JavaBean.	Texto de documentación destinado a explicar una decisión o procedimiento.
63	`src/GUI_ADMINISTRADOR/UsuarioFormPanel.java` Lógica del formulario de alta/edición.	Texto de documentación destinado a explicar una decisión o procedimiento.
64	`src/GUI_ADMINISTRADOR/UsuarioFormPanel.form` Diseño visual editable del formulario.	Texto de documentación destinado a explicar una decisión o procedimiento.
65	`src/GUI_ADMINISTRADOR/UsuarioFormDialog.java` Contenedor modal mínimo del panel visual.	Texto de documentación destinado a explicar una decisión o procedimiento.
66		Separación entre párrafos del documento.
67	### Archivos modificados	Título o subtítulo Markdown que organiza la documentación.
68		Separación entre párrafos del documento.
69	Archivo Cambio	Texto de documentación destinado a explicar una decisión o procedimiento.
70	--- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
71	`src/GUI_ADMINISTRADOR/UsuariosPanel.java` Tabla, filtros, tarjetas, acciones, páginas y exportación.	Texto de documentación destinado a explicar una decisión o procedimiento.
72	`src/GUI_ADMINISTRADOR/UsuariosPanel.form` Contiene buscador, filtros, tabla, exportación y paginación en De sign.	Texto de documentación destinado a explicar una decisión o procedimiento.
73	`src/Login/Login.java` Usa el hash compartido y registra la sesión autenticada.	Texto de documentación destinado a explicar una decisión o procedimiento.
74		Separación entre párrafos del documento.
75	Los componentes visuales permanentes están declarados en los archivos `.form`.	Texto de documentación destinado a explicar una decisión o procedimiento.
76	Después de `initComponents()` el código solamente aplica fuentes, conecta	Texto de documentación destinado a explicar una decisión o procedimiento.
77	eventos, instala renderers y carga datos. Por eso se pueden mover controles	Texto de documentación destinado a explicar una decisión o procedimiento.
78	desde la pestaña **Design** sin reconstruir la pantalla en Java.	Texto de documentación destinado a explicar una decisión o procedimiento.
79		Separación entre párrafos del documento.
80	## 4. Componentes existentes reutilizados	Título o subtítulo Markdown que organiza la documentación.
81		Separación entre párrafos del documento.
82	Componente Uso en el módulo	Texto de documentación destinado a explicar una decisión o procedimiento.

83	--- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
84	`PanelFlotante` Contenedor de filtros, tabla y fondo del modal.	Texto de documentación destinado a explicar una decisión o procedimiento.
85	`PanelCircular` Fondo circular del icono principal del formulario.	Texto de documentación destinado a explicar una decisión o procedimiento.
86	`BotonDerretido` Botón principal “Agregar usuario”.	Texto de documentación destinado a explicar una decisión o procedimiento.
87	`BotonRedondeado` Exportar, limpiar, paginar, guardar, cancelar y acciones por fila.	Texto de documentación destinado a explicar una decisión o procedimiento.
88	`BotonDesplegable` Filtros y selectores de rol/estado.	Texto de documentación destinado a explicar una decisión o procedimiento.
89	`CampoBusquedaAdmin` Buscador blanco con lupa gris; no utiliza círculo ni fondo amarillo.	Texto de documentación destinado a explicar una decisión o procedimiento.
90	`LabelEscalable` Iconos de las tarjetas creadas en el `.form`.	Texto de documentación destinado a explicar una decisión o procedimiento.
91	`TemaAdmin` DM Sans, colores, medidas y calidad gráfica.	Texto de documentación destinado a explicar una decisión o procedimiento.
92		Separación entre párrafos del documento.
93	`JTable`, `JScrollPane`, `JPasswordField`, `JFileChooser` y `SwingWorker` son	Texto de documentación destinado a explicar una decisión o procedimiento.
94	componentes estándar de Swing usados donde el proyecto no tenía un equivalente.	Texto de documentación destinado a explicar una decisión o procedimiento.
95		Separación entre párrafos del documento.
96	## 5. Modelo de usuario	Título o subtítulo Markdown que organiza la documentación.
97		Separación entre párrafos del documento.
98	`Usuario` contiene solamente información que puede mostrarse en la interfaz.	Texto de documentación destinado a explicar una decisión o procedimiento.
99	No contiene `password_hash`.	Texto de documentación destinado a explicar una decisión o procedimiento.
100		Separación entre párrafos del documento.
101	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
102	public class Usuario {	Texto de documentación destinado a explicar una decisión o procedimiento.
103	private final int idUsuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
104	private final String nombre;	Texto de documentación destinado a explicar una decisión o procedimiento.
105	private final String apellido;	Texto de documentación destinado a explicar una decisión o procedimiento.
106	private final String nombreUsuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
107	private final String correo;	Texto de documentación destinado a explicar una decisión o procedimiento.
108	private final String rol;	Texto de documentación destinado a explicar una decisión o procedimiento.
109	private final boolean activo;	Texto de documentación destinado a explicar una decisión o procedimiento.
110	private final LocalDateTime fechaCreacion;	Texto de documentación destinado a explicar una decisión o procedimiento.
111		Separación entre párrafos del documento.
112	// La tabla usa este método para formar la columna Nombre.	Texto de documentación destinado a explicar una decisión o procedimiento.
113	public String getNombreCompleto() {	Texto de documentación destinado a explicar una decisión o procedimiento.
114	return nombre + " " + apellido;	Texto de documentación destinado a explicar una decisión o procedimiento.
115	}	Texto de documentación destinado a explicar una decisión o procedimiento.
116	}	Texto de documentación destinado a explicar una decisión o procedimiento.
117	```	Abre o cierra un ejemplo de código con formato monoespaciado.
118		Separación entre párrafos del documento.
119	Esto impide que un hash termine accidentalmente en una celda, formulario o	Texto de documentación destinado a explicar una decisión o procedimiento.
120	archivo exportado.	Texto de documentación destinado a explicar una decisión o procedimiento.
121		Separación entre párrafos del documento.
122	## 6. Capa CRUD	Título o subtítulo Markdown que organiza la documentación.
123		Separación entre párrafos del documento.
124	`UsuarioCRUD` recibe filtros y devuelve objetos `Usuario`. Todas las variables	Texto de documentación destinado a explicar una decisión o procedimiento.
125	se colocan con `PreparedStatement`.	Texto de documentación destinado a explicar una decisión o procedimiento.
126		Separación entre párrafos del documento.

127	### Consulta paginada	Título o subtítulo Markdown que organiza la documentación.
128		Separación entre párrafos del documento.
129	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
130	public PaginaUsuarios listarPagina(	Texto de documentación destinado a explicar una decisión o procedimiento.
131	String busqueda,	Texto de documentación destinado a explicar una decisión o procedimiento.
132	String rol,	Texto de documentación destinado a explicar una decisión o procedimiento.
133	Boolean estado,	Texto de documentación destinado a explicar una decisión o procedimiento.
134	int pagina,	Texto de documentación destinado a explicar una decisión o procedimiento.
135	int tamanoPagina) throws SQLException {	Texto de documentación destinado a explicar una decisión o procedimiento.
136		Separación entre párrafos del documento.
137	// La página nunca puede ser menor que 1.	Texto de documentación destinado a explicar una decisión o procedimiento.
138	int paginaSegura = Math.max(1, pagina);	Texto de documentación destinado a explicar una decisión o procedimiento.
139	int desplazamiento = (paginaSegura - 1) * tamanoPagina;	Texto de documentación destinado a explicar una decisión o procedimiento.
140		Separación entre párrafos del documento.
141	// construirFiltro agrega sólo las condiciones seleccionadas.	Texto de documentación destinado a explicar una decisión o procedimiento.
142	FiltroSql filtro = construirFiltro(busqueda, rol, estado);	Texto de documentación destinado a explicar una decisión o procedimiento.
143		Separación entre párrafos del documento.
144	try (Connection conexion = abrirConexion()) {	Texto de documentación destinado a explicar una decisión o procedimiento.
145	int total = contarFiltrados(conexion, filtro);	Texto de documentación destinado a explicar una decisión o procedimiento.
146	List<Usuario> usuarios = consultarUsuarios(	Texto de documentación destinado a explicar una decisión o procedimiento.
147	conexion, filtro, tamanoPagina, desplazamiento, true);	Texto de documentación destinado a explicar una decisión o procedimiento.
148	return new PaginaUsuarios(usuarios, total);	Texto de documentación destinado a explicar una decisión o procedimiento.
149	}	Texto de documentación destinado a explicar una decisión o procedimiento.
150	}	Texto de documentación destinado a explicar una decisión o procedimiento.
151	```	Abre o cierra un ejemplo de código con formato monoespaciado.
152		Separación entre párrafos del documento.
153	La consulta resultante termina con:	Texto de documentación destinado a explicar una decisión o procedimiento.
154		Separación entre párrafos del documento.
155	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
156	ORDER BY id_usuario ASC	Texto de documentación destinado a explicar una decisión o procedimiento.
157	LIMIT ? OFFSET ?	Texto de documentación destinado a explicar una decisión o procedimiento.
158	```	Abre o cierra un ejemplo de código con formato monoespaciado.
159		Separación entre párrafos del documento.
160	### Filtros combinados	Título o subtítulo Markdown que organiza la documentación.
161		Separación entre párrafos del documento.
162	La búsqueda utiliza:	Texto de documentación destinado a explicar una decisión o procedimiento.
163		Separación entre párrafos del documento.
164	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
165	LOWER(CONCAT_WS(' ', nombre, apellido, usuario, correo)) LIKE LOWER(?)	Texto de documentación destinado a explicar una decisión o procedimiento.
166	```	Abre o cierra un ejemplo de código con formato monoespaciado.
167		Separación entre párrafos del documento.
168	Después puede agregar `rol = ?` y `estado = ?`. Las condiciones se unen con	Texto de documentación destinado a explicar una decisión o procedimiento.
169	`AND`, por lo que los tres controles funcionan al mismo tiempo.	Texto de documentación destinado a explicar una decisión o procedimiento.
170		Separación entre párrafos del documento.

171	### Resumen	Título o subtítulo Markdown que organiza la documentación.
172		Separación entre párrafos del documento.
173	Las cuatro tarjetas salen de una consulta:	Texto de documentación destinado a explicar una decisión o procedimiento.
174		Separación entre párrafos del documento.
175	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
176	SELECT COUNT(*) AS total,	Texto de documentación destinado a explicar una decisión o procedimiento.
177	COALESCE(SUM(rol = 'ADMINISTRADOR' AND estado = TRUE), 0)	Texto de documentación destinado a explicar una decisión o procedimiento.
178	AS administradores,	Texto de documentación destinado a explicar una decisión o procedimiento.
179	COALESCE(SUM(rol = 'CAJERO' AND estado = TRUE), 0)	Texto de documentación destinado a explicar una decisión o procedimiento.
180	AS cajeros,	Texto de documentación destinado a explicar una decisión o procedimiento.
181	COALESCE(SUM(estado = FALSE), 0) AS inactivos	Texto de documentación destinado a explicar una decisión o procedimiento.
182	FROM usuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
183	```	Abre o cierra un ejemplo de código con formato monoespaciado.
184		Separación entre párrafos del documento.
185	Los conteos son globales; no cambian al filtrar la tabla.	Texto de documentación destinado a explicar una decisión o procedimiento.
186		Separación entre párrafos del documento.
187	## 7. Paginación fija	Título o subtítulo Markdown que organiza la documentación.
188		Separación entre párrafos del documento.
189	El tamaño se declara una sola vez en `UsuariosPanel`:	Texto de documentación destinado a explicar una decisión o procedimiento.
190		Separación entre párrafos del documento.
191	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
192	private static final int USUARIOS_POR_PAGINA = 10;	Texto de documentación destinado a explicar una decisión o procedimiento.
193	```	Abre o cierra un ejemplo de código con formato monoespaciado.
194		Separación entre párrafos del documento.
195	La cantidad de páginas se calcula así:	Texto de documentación destinado a explicar una decisión o procedimiento.
196		Separación entre párrafos del documento.
197	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
198	totalPaginas = Math.max(1, (int) Math.ceil(	Texto de documentación destinado a explicar una decisión o procedimiento.
199	totalRegistros / (double) USUARIOS_POR_PAGINA));	Texto de documentación destinado a explicar una decisión o procedimiento.
200	```	Abre o cierra un ejemplo de código con formato monoespaciado.
201		Separación entre párrafos del documento.
202	Al buscar o cambiar un filtro, `paginaActual` vuelve a 1. Anterior se desactiva	Texto de documentación destinado a explicar una decisión o procedimiento.
203	en la primera página y siguiente en la última. Se muestran como máximo tres	Texto de documentación destinado a explicar una decisión o procedimiento.
204	botones numéricos alrededor de la página actual.	Texto de documentación destinado a explicar una decisión o procedimiento.
205		Separación entre párrafos del documento.
206	El texto inferior es informativo, por ejemplo:	Texto de documentación destinado a explicar una decisión o procedimiento.
207		Separación entre párrafos del documento.
208	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
209	Mostrando 11-20 de 28 usuarios	Texto de documentación destinado a explicar una decisión o procedimiento.
210	```	Abre o cierra un ejemplo de código con formato monoespaciado.
211		Separación entre párrafos del documento.
212	No modifica el tamaño de página.	Texto de documentación destinado a explicar una decisión o procedimiento.
213		Separación entre párrafos del documento.
214	## 8. Búsqueda y carga asíncrona	Título o subtítulo Markdown que organiza la documentación.

215		Separación entre párrafos del documento.
216	El documento del campo de búsqueda reinicia un `Timer` de 300 milisegundos:	Texto de documentación destinado a explicar una decisión o procedimiento.
217		Separación entre párrafos del documento.
218	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
219	temporizadorBusqueda = new Timer(300, evento -> {	Texto de documentación destinado a explicar una decisión o procedimiento.
220	paginaActual = 1;	Texto de documentación destinado a explicar una decisión o procedimiento.
221	cargarPagina();	Texto de documentación destinado a explicar una decisión o procedimiento.
222	});	Texto de documentación destinado a explicar una decisión o procedimiento.
223	temporizadorBusqueda.setRepeats(false);	Texto de documentación destinado a explicar una decisión o procedimiento.
224	```	Abre o cierra un ejemplo de código con formato monoespaciado.
225		Separación entre párrafos del documento.
226	La consulta corre dentro de `SwingWorker`. De esa forma MySQL no bloquea el	Texto de documentación destinado a explicar una decisión o procedimiento.
227	hilo gráfico y la ventana continúa respondiendo.	Texto de documentación destinado a explicar una decisión o procedimiento.
228		Separación entre párrafos del documento.
229	`secuenciaCarga` identifica cada petición. Si una consulta anterior termina	Texto de documentación destinado a explicar una decisión o procedimiento.
230	después de que el usuario cambió nuevamente el filtro, su respuesta se ignora.	Texto de documentación destinado a explicar una decisión o procedimiento.
231		Separación entre párrafos del documento.
232	## 9. Tabla y acciones por fila	Título o subtítulo Markdown que organiza la documentación.
233		Separación entre párrafos del documento.
234	`ModeloTablaUsuarios` extiende `AbstractTableModel`. Las columnas de estado y	Texto de documentación destinado a explicar una decisión o procedimiento.
235	acciones devuelven el objeto `Usuario` completo, permitiendo trabajar con el ID	Texto de documentación destinado a explicar una decisión o procedimiento.
236	correcto sin intentar leerlo desde el texto pintado.	Texto de documentación destinado a explicar una decisión o procedimiento.
237		Separación entre párrafos del documento.
238	El estado usa `RenderEstado` para dibujar una insignia verde o gris.	Texto de documentación destinado a explicar una decisión o procedimiento.
239		Separación entre párrafos del documento.
240	La columna de acciones requiere dos partes:	Texto de documentación destinado a explicar una decisión o procedimiento.
241		Separación entre párrafos del documento.
242	- `RenderAcciones`: dibuja los botones.	Elemento de una lista explicativa.
243	- `EditorAcciones`: recibe los clics reales.	Elemento de una lista explicativa.
244		Separación entre párrafos del documento.
245	Un `TableCellRenderer` por si solo no es interactivo. El editor llama:	Texto de documentación destinado a explicar una decisión o procedimiento.
246		Separación entre párrafos del documento.
247	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
248	private void editar(ActionEvent evento) {	Texto de documentación destinado a explicar una decisión o procedimiento.
249	Usuario seleccionado = usuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
250	fireEditingStopped(); // Finaliza la edición de la celda.	Texto de documentación destinado a explicar una decisión o procedimiento.
251	abrirFormulario(seleccionado);	Texto de documentación destinado a explicar una decisión o procedimiento.
252	}	Texto de documentación destinado a explicar una decisión o procedimiento.
253		Separación entre párrafos del documento.
254	private void eliminar(ActionEvent evento) {	Texto de documentación destinado a explicar una decisión o procedimiento.
255	Usuario seleccionado = usuario;	Texto de documentación destinado a explicar una decisión o procedimiento.
256	fireEditingStopped();	Texto de documentación destinado a explicar una decisión o procedimiento.
257	desactivarUsuario(seleccionado);	Texto de documentación destinado a explicar una decisión o procedimiento.
258	}	Texto de documentación destinado a explicar una decisión o procedimiento.

259	```	Abre o cierra un ejemplo de código con formato monoespaciado.
260		Separación entre párrafos del documento.
261	## 10. Formulario reutilizable	Título o subtítulo Markdown que organiza la documentación.
262		Separación entre párrafos del documento.
263	`UsuarioFormDialog.mostrar(...)` recibe `null` para agregar o un `Usuario` para	Texto de documentación destinado a explicar una decisión o procedimiento.
264	editar:	Texto de documentación destinado a explicar una decisión o procedimiento.
265		Separación entre párrafos del documento.
266	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
267	// Alta	Texto de documentación destinado a explicar una decisión o procedimiento.
268	UsuarioFormDialog.mostrar(ventana, crud, null);	Texto de documentación destinado a explicar una decisión o procedimiento.
269		Separación entre párrafos del documento.
270	// Edición	Texto de documentación destinado a explicar una decisión o procedimiento.
271	UsuarioFormDialog.mostrar(ventana, crud, usuarioSeleccionado);	Texto de documentación destinado a explicar una decisión o procedimiento.
272	```	Abre o cierra un ejemplo de código con formato monoespaciado.
273		Separación entre párrafos del documento.
274	`UsuarioFormDialog` no construye controles. Crea un	Texto de documentación destinado a explicar una decisión o procedimiento.
275	`UsuarioFormPanel(crud, usuario)` y lo muestra como contenido modal. El diseño	Texto de documentación destinado a explicar una decisión o procedimiento.
276	completo se edita abriendo `UsuarioFormPanel.form` en la pestaña Design .	Texto de documentación destinado a explicar una decisión o procedimiento.
277	El mismo panel cambia título, subtítulo, valores y botón principal según reciba	Texto de documentación destinado a explicar una decisión o procedimiento.
278	`null` o un usuario existente.	Texto de documentación destinado a explicar una decisión o procedimiento.
279		Separación entre párrafos del documento.
280	### Cómo modificar el diseño en NetBeans	Título o subtítulo Markdown que organiza la documentación.
281		Separación entre párrafos del documento.
282	Para la pantalla principal:	Texto de documentación destinado a explicar una decisión o procedimiento.
283		Separación entre párrafos del documento.
284	1. Abrir `UsuariosPanel.java`.	Texto de documentación destinado a explicar una decisión o procedimiento.
285	2. Seleccionar la pestaña Design .	Texto de documentación destinado a explicar una decisión o procedimiento.
286	3. Mover o redimensionar `panelFiltros`, `campoBusqueda`, `filtroRol`,	Texto de documentación destinado a explicar una decisión o procedimiento.
287	`filtroEstado`, `botonLimpiarFiltros`, `panelTabla`, `scrollUsuarios`,	Texto de documentación destinado a explicar una decisión o procedimiento.
288	`panelPaginacion` y `botonExportar`.	Texto de documentación destinado a explicar una decisión o procedimiento.
289	4. No colocar consultas SQL dentro de `initComponents()`.	Texto de documentación destinado a explicar una decisión o procedimiento.
290		Separación entre párrafos del documento.
291	Para agregar y editar:	Texto de documentación destinado a explicar una decisión o procedimiento.
292		Separación entre párrafos del documento.
293	1. Abrir `UsuarioFormPanel.java`.	Texto de documentación destinado a explicar una decisión o procedimiento.
294	2. Seleccionar Design .	Texto de documentación destinado a explicar una decisión o procedimiento.
295	3. Editar campos, etiquetas, botones y el panel de contraseñas.	Texto de documentación destinado a explicar una decisión o procedimiento.
296	4. Ambos modos comparten el mismo diseño; `cargarModo()` decide qué elementos	Texto de documentación destinado a explicar una decisión o procedimiento.
297	se muestran durante la ejecución.	Texto de documentación destinado a explicar una decisión o procedimiento.
298		Separación entre párrafos del documento.
299	El buscador es `CampoBusquedaAdmin`, un JavaBean con propiedades `placeholder`,	Texto de documentación destinado a explicar una decisión o procedimiento.
300	`colorBorde`, `colorIcono`, `colorPlaceholder` y `radio`. Su lupa se pinta en	Texto de documentación destinado a explicar una decisión o procedimiento.
301	gris oscuro directamente dentro del campo, sin la pieza amarilla del login.	Texto de documentación destinado a explicar una decisión o procedimiento.
302		Separación entre párrafos del documento.

303	### Validaciones	Título o subtítulo Markdown que organiza la documentación.
304		Separación entre párrafos del documento.
305	- Nombre, apellido, usuario, correo y rol obligatorios.	Elemento de una lista explicativa.
306	- Límites iguales a los `VARCHAR` del esquema.	Elemento de una lista explicativa.
307	- Usuario de al menos cuatro caracteres, sin espacios.	Elemento de una lista explicativa.
308	- Formato básico de correo.	Elemento de una lista explicativa.
309	- Usuario y correo no repetidos.	Elemento de una lista explicativa.
310	- Contraseña de al menos ocho caracteres.	Elemento de una lista explicativa.
311	- Confirmación igual a la contraseña.	Elemento de una lista explicativa.
312	- Siempre debe quedar un administrador activo.	Elemento de una lista explicativa.
313	- El usuario conectado no puede desactivarse a sí mismo.	Elemento de una lista explicativa.
314		Separación entre párrafos del documento.
315	### Contraseña al editar	Título o subtítulo Markdown que organiza la documentación.
316		Separación entre párrafos del documento.
317	La opción "Cambiar contraseña" oculta los campos inicialmente. Cuando no está	Texto de documentación destinado a explicar una decisión o procedimiento.
318	marcada, el CRUD usa un `UPDATE` que no incluye `password_hash`.	Texto de documentación destinado a explicar una decisión o procedimiento.
319		Separación entre párrafos del documento.
320	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
321	String hash = requiereContrasena	Texto de documentación destinado a explicar una decisión o procedimiento.
322	? SeguridadContrasena.sha256(contrasena)	Texto de documentación destinado a explicar una decisión o procedimiento.
323	: null;	Texto de documentación destinado a explicar una decisión o procedimiento.
324		Separación entre párrafos del documento.
325	crud.actualizar(datos, hash);	Texto de documentación destinado a explicar una decisión o procedimiento.
326	```	Abre o cierra un ejemplo de código con formato monoespaciado.
327		Separación entre párrafos del documento.
328	El formulario nunca recibe ni muestra el hash anterior.	Texto de documentación destinado a explicar una decisión o procedimiento.
329		Separación entre párrafos del documento.
330	## 11. Contraseñas y login	Título o subtítulo Markdown que organiza la documentación.
331		Separación entre párrafos del documento.
332	El login original ya usaba SHA-256. El cálculo se movió a	Texto de documentación destinado a explicar una decisión o procedimiento.
333	`SeguridadContrasena.sha256(...)` , utilizado tanto por `Login` como por el alta	Texto de documentación destinado a explicar una decisión o procedimiento.
334	y la edición:	Texto de documentación destinado a explicar una decisión o procedimiento.
335		Separación entre párrafos del documento.
336	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
337	public static String sha256(String contrasena) {	Texto de documentación destinado a explicar una decisión o procedimiento.
338	MessageDigest digest = MessageDigest.getInstance("SHA-256");	Texto de documentación destinado a explicar una decisión o procedimiento.
339	byte[] hash = digest.digest(	Texto de documentación destinado a explicar una decisión o procedimiento.
340	contrasena.getBytes(StandardCharsets.UTF_8));	Texto de documentación destinado a explicar una decisión o procedimiento.
341	// Después convierte cada byte a hexadecimal.	Texto de documentación destinado a explicar una decisión o procedimiento.
342	}	Texto de documentación destinado a explicar una decisión o procedimiento.
343	```	Abre o cierra un ejemplo de código con formato monoespaciado.
344		Separación entre párrafos del documento.
345	Esto conserva compatibilidad con las contraseñas existentes. Para producción	Texto de documentación destinado a explicar una decisión o procedimiento.
346	conviene migrar posteriormente a BCrypt o Argon2, porque SHA-256 sin sal no es	Texto de documentación destinado a explicar una decisión o procedimiento.

347	un algoritmo específico para almacenar contraseñas.	Texto de documentación destinado a explicar una decisión o procedimiento.
348		Separación entre párrafos del documento.
349	## 12. Desactivación en lugar de DELETE	Título o subtítulo Markdown que organiza la documentación.
350		Separación entre párrafos del documento.
351	El icono rojo ejecuta:	Texto de documentación destinado a explicar una decisión o procedimiento.
352		Separación entre párrafos del documento.
353	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
354	UPDATE usuario	Texto de documentación destinado a explicar una decisión o procedimiento.
355	SET estado = FALSE	Texto de documentación destinado a explicar una decisión o procedimiento.
356	WHERE id_usuario = ?;	Texto de documentación destinado a explicar una decisión o procedimiento.
357	```	Abre o cierra un ejemplo de código con formato monoespaciado.
358		Separación entre párrafos del documento.
359	No se usa 'DELETE FROM usuario'. La tabla 'pedido' tiene una llave foránea con	Texto de documentación destinado a explicar una decisión o procedimiento.
360	'ON DELETE RESTRICT'; borrar usuarios rompería la trazabilidad de ventas.	Texto de documentación destinado a explicar una decisión o procedimiento.
361		Separación entre párrafos del documento.
362	Antes de desactivar se confirma la acción y se comprueba que:	Texto de documentación destinado a explicar una decisión o procedimiento.
363		Separación entre párrafos del documento.
364	- No sea el usuario autenticado.	Elemento de una lista explicativa.
365	- No sea el último administrador activo.	Elemento de una lista explicativa.
366	- El usuario no esté ya inactivo.	Elemento de una lista explicativa.
367		Separación entre párrafos del documento.
368	Los usuarios se reactivan abriendo Editar y seleccionando 'Activo'.	Texto de documentación destinado a explicar una decisión o procedimiento.
369		Separación entre párrafos del documento.
370	## 13. Exportación CSV	Título o subtítulo Markdown que organiza la documentación.
371		Separación entre párrafos del documento.
372	El botón Exportar conserva los filtros actuales, pero consulta todos los	Texto de documentación destinado a explicar una decisión o procedimiento.
373	resultados sin 'LIMIT'. El usuario elige la ubicación mediante 'JFileChooser'.	Texto de documentación destinado a explicar una decisión o procedimiento.
374		Separación entre párrafos del documento.
375	El archivo utiliza UTF-8 con BOM para que Excel muestre correctamente tildes y	Texto de documentación destinado a explicar una decisión o procedimiento.
376	la 'ñ'. Cada valor se encierra entre comillas y las comillas internas se	Texto de documentación destinado a explicar una decisión o procedimiento.
377	duplican.	Texto de documentación destinado a explicar una decisión o procedimiento.
378		Separación entre párrafos del documento.
379	## 14. Sesión autenticada	Título o subtítulo Markdown que organiza la documentación.
380		Separación entre párrafos del documento.
381	Después de validar el login se ejecuta:	Texto de documentación destinado a explicar una decisión o procedimiento.
382		Separación entre párrafos del documento.
383	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
384	SesionUsuario.iniciar(idUsuario, nombre, apellido, rol);	Texto de documentación destinado a explicar una decisión o procedimiento.
385	```	Abre o cierra un ejemplo de código con formato monoespaciado.
386		Separación entre párrafos del documento.
387	'UsuariosPanel' y 'UsuarioFormDialog' consultan ese ID para impedir que el	Texto de documentación destinado a explicar una decisión o procedimiento.
388	administrador conectado desactive su propia cuenta.	Texto de documentación destinado a explicar una decisión o procedimiento.
389		Separación entre párrafos del documento.
390	## 15. Iconos	Título o subtítulo Markdown que organiza la documentación.

391		Separación entre párrafos del documento.
392	`IconosUsuarios` implementa `javax.swing.Icon` y dibuja con `Graphics2D`:	Texto de documentación destinado a explicar una decisión o procedimiento.
393		Separación entre párrafos del documento.
394	- búsqueda;	Elemento de una lista explicativa.
395	- editar;	Elemento de una lista explicativa.
396	- eliminar/desactivar;	Elemento de una lista explicativa.
397	- filtros;	Elemento de una lista explicativa.
398	- exportar;	Elemento de una lista explicativa.
399	- agregar;	Elemento de una lista explicativa.
400	- flechas;	Elemento de una lista explicativa.
401	- usuarios, administradores, cajeros e inactivos;	Elemento de una lista explicativa.
402	- mostrar contraseña.	Elemento de una lista explicativa.
403		Separación entre párrafos del documento.
404	Así los iconos mantienen colores nítidos y no requieren instalar una librería	Texto de documentación destinado a explicar una decisión o procedimiento.
405	ni agregar múltiples PNG.	Texto de documentación destinado a explicar una decisión o procedimiento.
406		Separación entre párrafos del documento.
407	## 16. Cómo probar	Título o subtítulo Markdown que organiza la documentación.
408		Separación entre párrafos del documento.
409	1. Confirmar que MySQL esté iniciado.	Texto de documentación destinado a explicar una decisión o procedimiento.
410	2. Confirmar que `usuario.correo` exista.	Texto de documentación destinado a explicar una decisión o procedimiento.
411	3. Abrir el proyecto `Waldonalds` en NetBeans.	Texto de documentación destinado a explicar una decisión o procedimiento.
412	4. Ejecutar Clean and Build.	Texto de documentación destinado a explicar una decisión o procedimiento.
413	5. Iniciar sesión como administrador.	Texto de documentación destinado a explicar una decisión o procedimiento.
414	6. Abrir Gestión de Usuarios.	Texto de documentación destinado a explicar una decisión o procedimiento.
415	7. Probar búsqueda, los dos filtros y Limpiar filtros.	Texto de documentación destinado a explicar una decisión o procedimiento.
416	8. Crear al menos 11 usuarios para comprobar el cambio de página.	Texto de documentación destinado a explicar una decisión o procedimiento.
417	9. Editar un usuario sin cambiar contraseña y comprobar que todavía inicia	Texto de documentación destinado a explicar una decisión o procedimiento.
418	sesión.	Texto de documentación destinado a explicar una decisión o procedimiento.
419	10. Cambiar su contraseña y probar nuevamente el login.	Texto de documentación destinado a explicar una decisión o procedimiento.
420	11. Desactivar un usuario y comprobar que no puede iniciar sesión.	Texto de documentación destinado a explicar una decisión o procedimiento.
421	12. Editarlo, reactivarlo y comprobar el acceso.	Texto de documentación destinado a explicar una decisión o procedimiento.
422	13. Exportar con filtros y revisar el CSV.	Texto de documentación destinado a explicar una decisión o procedimiento.
423		Separación entre párrafos del documento.
424	También puede compilarse desde una terminal que tenga Ant:	Texto de documentación destinado a explicar una decisión o procedimiento.
425		Separación entre párrafos del documento.
426	```powershell	Abre o cierra un ejemplo de código con formato monoespaciado.
427	cd Waldonalds	Texto de documentación destinado a explicar una decisión o procedimiento.
428	ant clean jar	Texto de documentación destinado a explicar una decisión o procedimiento.
429	```	Abre o cierra un ejemplo de código con formato monoespaciado.
430		Separación entre párrafos del documento.
431	En el entorno donde se desarrolló, Ant no estaba instalado; se verificó con	Texto de documentación destinado a explicar una decisión o procedimiento.
432	`javac 21.0.4` usando `AbsoluteLayout.jar` y el conector MySQL del directorio	Texto de documentación destinado a explicar una decisión o procedimiento.
433	`lib`.	Texto de documentación destinado a explicar una decisión o procedimiento.
434		Separación entre párrafos del documento.

435	## 17. Verificaciones realizadas	Título o subtítulo Markdown que organiza la documentación.
436		Separación entre párrafos del documento.
437	- Todos los archivos Java compilan correctamente con Java 21.	Elemento de una lista explicativa.
438	- `UsuariosPanel.form` y `UsuarioFormPanel.form` son XML válidos.	Elemento de una lista explicativa.
439	- La construcción visual principal quedó dentro de los dos `.form`.	Elemento de una lista explicativa.
440	- El conector logra abrir la base `waldonalds`.	Elemento de una lista explicativa.
441	- La consulta de resumen se ejecuta correctamente.	Elemento de una lista explicativa.
442	- La base encontrada durante la verificación todavía no tenía físicamente la	Elemento de una lista explicativa.
443	columna `correo`; por eso la consulta de filas sólo funcionará después de	Texto de documentación destinado a explicar una decisión o procedimiento.
444	aplicar el requisito de la sección 2, aunque el código se implementó según	Texto de documentación destinado a explicar una decisión o procedimiento.
445	la instrucción de asumir que el campo existe.	Texto de documentación destinado a explicar una decisión o procedimiento.

16.3 DOCUMENTACION_INVENTARIO.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 175 | Archivo 3 de 157

1	# Módulo de inventario	Título o subtítulo Markdown que organiza la documentación.
2		Separación entre párrafos del documento.
3	## Objetivo	Título o subtítulo Markdown que organiza la documentación.
4		Separación entre párrafos del documento.
5	El módulo reúne en una sola sección administrativa el catálogo de	Texto de documentación destinado a explicar una decisión o procedimiento.
6	ingredientes, las existencias controlables y su historial. En el menú lateral	Texto de documentación destinado a explicar una decisión o procedimiento.
7	solo aparece Inventario ; dentro de ese módulo existen tres vistas:	Texto de documentación destinado a explicar una decisión o procedimiento.
8		Separación entre párrafos del documento.
9	Vista Responsabilidad	Texto de documentación destinado a explicar una decisión o procedimiento.
10	--- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
11	Existencias Consultar stock y registrar entradas, salidas o ajustes.	Texto de documentación destinado a explicar una decisión o procedimiento.
12	Ingredientes Crear y editar nombre, unidad, mínimo y estado del ingrediente.	Texto de documentación destinado a explicar una decisión o procedimiento.
13	Movimientos Consultar la trazabilidad de entradas, salidas, ajustes y ventas.	Texto de documentación destinado a explicar una decisión o procedimiento.
14		Separación entre párrafos del documento.
15	La vista Existencias muestra únicamente los artículos cuyo stock se puede	Texto de documentación destinado a explicar una decisión o procedimiento.
16	modificar de forma directa:	Texto de documentación destinado a explicar una decisión o procedimiento.
17		Separación entre párrafos del documento.
18	Artículo mostrado Tabla de origen Motivo	Texto de documentación destinado a explicar una decisión o procedimiento.
19	--- --- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
20	Ingrediente `ingrediente` Es la existencia consumida por los productos con receta.	Texto de documentación destinado a explicar una decisión o procedimiento.
21	Producto directo `producto` con `tipo_stock = 'DIRECTO'` Se vende y descuenta como una unidad terminada.	Texto de documentación destinado a explicar una decisión o procedimiento.
22		Separación entre párrafos del documento.
23	Los productos `RECETA` no aparecen como una existencia adicional porque su	Texto de documentación destinado a explicar una decisión o procedimiento.
24	stock real está en los ingredientes de `producto_ingrediente`. Los productos	Texto de documentación destinado a explicar una decisión o procedimiento.
25	`NINGUNO` tampoco aparecen porque fueron definidos sin control de stock.	Texto de documentación destinado a explicar una decisión o procedimiento.
26		Separación entre párrafos del documento.
27	No fue necesario agregar ni alterar tablas. Se reutilizan `ingrediente`,	Texto de documentación destinado a explicar una decisión o procedimiento.
28	`producto`, `categoria` y `movimiento_inventario` de la base actual.	Texto de documentación destinado a explicar una decisión o procedimiento.
29		Separación entre párrafos del documento.
30	## Archivos y organización	Título o subtítulo Markdown que organiza la documentación.
31		Separación entre párrafos del documento.
32	- `src/GUI_ADMINISTRADOR/InventarioPanel.form`: contenedor visual de las tres	Elemento de una lista explicativa.
33	pestañas. Los botones Existencias, Ingredientes y Movimientos se pueden	Texto de documentación destinado a explicar una decisión o procedimiento.
34	mover o redimensionar desde Design .	Texto de documentación destinado a explicar una decisión o procedimiento.
35	- `src/GUI_ADMINISTRADOR/InventarioPanel.java`: cambia la tarjeta visible con	Elemento de una lista explicativa.
36	`CardLayout`; no duplica consultas ni reglas de inventario.	Texto de documentación destinado a explicar una decisión o procedimiento.
37		Separación entre párrafos del documento.
38	- `src/GUI_ADMINISTRADOR/GestionInventarioPanel.form`: diseño visual principal	Elemento de una lista explicativa.

39	editable desde la pestaña Design de NetBeans. Contiene títulos, botones,	Texto de documentación destinado a explicar una decisión o procedimiento.
40	filtros, tabla y paginación con sus posiciones y tamaños.	Texto de documentación destinado a explicar una decisión o procedimiento.
41	- `src/GUI_ADMINISTRADOR/GestionInventarioPanel.java`: lógica de la vista	Elemento de una lista explicativa.
42	Existencias, filtros, paginación y exportación.	Texto de documentación destinado a explicar una decisión o procedimiento.
43	- `src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.form`: diseño editable del	Elemento de una lista explicativa.
44	catálogo de ingredientes, sin imágenes en sus filas.	Texto de documentación destinado a explicar una decisión o procedimiento.
45	- `src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.java`: carga, filtra,	Elemento de una lista explicativa.
46	pagina, crea, edita y desactiva ingredientes.	Texto de documentación destinado a explicar una decisión o procedimiento.
47	- `src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.form`: diseño editable del	Elemento de una lista explicativa.
48	historial integrado.	Texto de documentación destinado a explicar una decisión o procedimiento.
49	- `src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.java`: carga los últimos	Elemento de una lista explicativa.
50	movimientos y filtra por texto o tipo.	Texto de documentación destinado a explicar una decisión o procedimiento.
51	- `src/GUI_ADMINISTRADOR/IngredienteFormPanel.form`: diseño de la mini ventana	Elemento de una lista explicativa.
52	para agregar o editar un ingrediente.	Texto de documentación destinado a explicar una decisión o procedimiento.
53	- `src/GUI_ADMINISTRADOR/IngredienteFormDialog.java`: envoltorio modal liviano	Elemento de una lista explicativa.
54	del formulario anterior.	Texto de documentación destinado a explicar una decisión o procedimiento.
55	- `src/GUI_ADMINISTRADOR/MovimientoInventarioFormPanel.form`: diseño de la mini	Elemento de una lista explicativa.
56	ventana para entradas, salidas y ajustes.	Texto de documentación destinado a explicar una decisión o procedimiento.
57	- `src/GUI_ADMINISTRADOR/MovimientoInventarioFormDialog.java`: envoltorio modal	Elemento de una lista explicativa.
58	liviano del formulario de movimiento.	Texto de documentación destinado a explicar una decisión o procedimiento.
59	- `src/DAO/InventarioDAO.java`: consultas y transacciones de inventario.	Elemento de una lista explicativa.
60	- `src/Modelos/ArticuloInventario.java`: representa un ingrediente o producto	Elemento de una lista explicativa.
61	directo con una estructura común.	Texto de documentación destinado a explicar una decisión o procedimiento.
62	- `src/Modelos/PaginaInventario.java`: resultado paginado.	Elemento de una lista explicativa.
63	- `src/Modelos/MovimientoInventario.java`: fila del historial.	Elemento de una lista explicativa.
64	- `src/GUI_ADMINISTRADOR/MenuAdmin.form`: contiene únicamente el botón	Elemento de una lista explicativa.
65	Inventario en el menú lateral. La opción separada Ingredientes fue retirada.	Texto de documentación destinado a explicar una decisión o procedimiento.
66	- `src/GUI_ADMINISTRADOR/MenuAdmin.java`: abre el panel mediante el	Elemento de una lista explicativa.
67	`CardLayout` existente.	Texto de documentación destinado a explicar una decisión o procedimiento.
68	- `src/Componentes/BotonMenuLateral.java`: dibuja el icono vectorial de la caja.	Elemento de una lista explicativa.
69	- `test/InventarioPanelTest.java`: verifica navegación, carga y que la columna	Elemento de una lista explicativa.
70	de nombre sea texto sin imágenes.	Texto de documentación destinado a explicar una decisión o procedimiento.
71		Separación entre párrafos del documento.
72	## Interfaz	Título o subtítulo Markdown que organiza la documentación.
73		Separación entre párrafos del documento.
74	Se reutilizaron los componentes visuales del proyecto:	Texto de documentación destinado a explicar una decisión o procedimiento.
75		Separación entre párrafos del documento.
76	- `BotonDerretido` para la acción principal Registrar entrada.	Elemento de una lista explicativa.
77	- `BotonRedondeado` para salida, historial, exportación, paginación y acciones.	Elemento de una lista explicativa.
78	- `CampoBusquedaAdmin` para la búsqueda por nombre, categoría o unidad.	Elemento de una lista explicativa.
79	- `BotonDesplegable` para los filtros de tipo y estado.	Elemento de una lista explicativa.
80	- `PanelFlotante` para filtros, tablas y tarjetas de los formularios.	Elemento de una lista explicativa.
81	- `TablaAdministrativa` para conservar cabecera, colores, filas y selección.	Elemento de una lista explicativa.
82	- `TemaAdmin` e `IconosUsuarios` para tipografía e iconos vectoriales.	Elemento de una lista explicativa.

83		Separación entre párrafos del documento.
84	La tabla no carga rutas ni imágenes. La columna Producto / ingrediente	Texto de documentación destinado a explicar una decisión o procedimiento.
85	recibe directamente un <code>`String`</code> con el nombre.	Texto de documentación destinado a explicar una decisión o procedimiento.
86		Separación entre párrafos del documento.
87	La distribución de cada pantalla y mini ventana está guardada en su archivo	Texto de documentación destinado a explicar una decisión o procedimiento.
88	<code>`form`</code> . El código Java se concentra en eventos, validación y acceso a datos.	Texto de documentación destinado a explicar una decisión o procedimiento.
89	Por eso, para cambiar posiciones, tamaños o textos se abre el <code>`form`</code> en	Texto de documentación destinado a explicar una decisión o procedimiento.
90	NetBeans y se usa la pestaña Design .	Texto de documentación destinado a explicar una decisión o procedimiento.
91		Separación entre párrafos del documento.
92	## Patrón de las mini ventanas	Título o subtítulo Markdown que organiza la documentación.
93		Separación entre párrafos del documento.
94	Se siguió el mismo patrón de <code>`UsuarioForm`</code> :	Texto de documentación destinado a explicar una decisión o procedimiento.
95		Separación entre párrafos del documento.
96	1. Un <code>`...FormPanel.form`</code> contiene todo lo visible: tarjeta, títulos, campos,	Texto de documentación destinado a explicar una decisión o procedimiento.
97	error y botones.	Texto de documentación destinado a explicar una decisión o procedimiento.
98	2. Su clase <code>`...FormPanel.java`</code> valida y guarda los datos.	Texto de documentación destinado a explicar una decisión o procedimiento.
99	3. Un <code>`...FormDialog.java`</code> crea una ventana modal sin bordes, coloca el panel,	Texto de documentación destinado a explicar una decisión o procedimiento.
100	escucha los eventos Guardado/Cancelar y se cierra.	Texto de documentación destinado a explicar una decisión o procedimiento.
101		Separación entre párrafos del documento.
102	Así el diseño no queda construido con decenas de componentes dentro del	Texto de documentación destinado a explicar una decisión o procedimiento.
103	controlador y puede editarse visualmente. Los diálogos anteriores construidos	Texto de documentación destinado a explicar una decisión o procedimiento.
104	directamente en código ya no forman parte del flujo nuevo de Inventario.	Texto de documentación destinado a explicar una decisión o procedimiento.
105		Separación entre párrafos del documento.
106	Las columnas son: ID, nombre, tipo, categoría, unidad, stock actual, stock	Texto de documentación destinado a explicar una decisión o procedimiento.
107	mínimo, último movimiento, estado y acción.	Texto de documentación destinado a explicar una decisión o procedimiento.
108		Separación entre párrafos del documento.
109	## Estados calculados	Título o subtítulo Markdown que organiza la documentación.
110		Separación entre párrafos del documento.
111	El estado visible no necesita otra columna en la base:	Texto de documentación destinado a explicar una decisión o procedimiento.
112		Separación entre párrafos del documento.
113	- Inactivo : <code>`estado = false`</code> .	Elemento de una lista explicativa.
114	- Agotado : artículo activo con <code>`stock_actual <= 0`</code> .	Elemento de una lista explicativa.
115	- Stock bajo : artículo activo, con mínimo mayor que cero y stock actual	Elemento de una lista explicativa.
116	menor o igual al mínimo.	Texto de documentación destinado a explicar una decisión o procedimiento.
117	- Disponible : cualquier otro artículo activo con existencias.	Elemento de una lista explicativa.
118		Separación entre párrafos del documento.
119	## Entradas, salidas y ajustes	Título o subtítulo Markdown que organiza la documentación.
120		Separación entre párrafos del documento.
121	1. El administrador selecciona una fila.	Texto de documentación destinado a explicar una decisión o procedimiento.
122	2. Puede pulsar Registrar entrada, Registrar salida, Ajustar stock o <code>`± Stock`</code> .	Texto de documentación destinado a explicar una decisión o procedimiento.
123	3. El diálogo solicita cantidad y un motivo opcional.	Texto de documentación destinado a explicar una decisión o procedimiento.
124	4. Para Entrada , la cantidad se suma.	Texto de documentación destinado a explicar una decisión o procedimiento.
125	5. Para Salida , la cantidad se resta y nunca puede dejar stock negativo.	Texto de documentación destinado a explicar una decisión o procedimiento.
126	6. Para Ajuste , la cantidad escrita es el stock final que debe quedar.	Texto de documentación destinado a explicar una decisión o procedimiento.

127		Separación entre párrafos del documento.
128	<code>`InventarioDAO.registrarMovimiento`</code> bloquea la fila con <code>`FOR UPDATE`</code> , calcula	Texto de documentación destinado a explicar una decisión o procedimiento.
129	el nuevo stock, actualiza el artículo e inserta su fila en	Texto de documentación destinado a explicar una decisión o procedimiento.
130	<code>`movimiento_inventario`</code> . Ambas operaciones se confirman juntas; si una falla,	Texto de documentación destinado a explicar una decisión o procedimiento.
131	se hace <code>`rollback`</code> y no queda un stock sin historial.	Texto de documentación destinado a explicar una decisión o procedimiento.
132		Separación entre párrafos del documento.
133	Los productos directos solo aceptan unidades enteras. Los ingredientes aceptan	Texto de documentación destinado a explicar una decisión o procedimiento.
134	hasta dos decimales, de acuerdo con <code>`DECIMAL(12,2)`</code> .	Texto de documentación destinado a explicar una decisión o procedimiento.
135		Separación entre párrafos del documento.
136	<code>## Historial y ventas</code>	Título o subtítulo Markdown que organiza la documentación.
137		Separación entre párrafos del documento.
138	La pestaña Movimientos muestra hasta 300 movimientos recientes. También	Texto de documentación destinado a explicar una decisión o procedimiento.
139	aparecen las salidas creadas automáticamente durante el cobro por <code>`PagoDAO`</code> ,	Texto de documentación destinado a explicar una decisión o procedimiento.
140	porque utilizan la misma tabla <code>`movimiento_inventario`</code> .	Texto de documentación destinado a explicar una decisión o procedimiento.
141		Separación entre párrafos del documento.
142	En la vista, las entradas llevan signo positivo y las salidas se presentan con	Texto de documentación destinado a explicar una decisión o procedimiento.
143	signo negativo. En un ajuste se conserva la diferencia registrada entre el	Texto de documentación destinado a explicar una decisión o procedimiento.
144	stock anterior y el nuevo.	Texto de documentación destinado a explicar una decisión o procedimiento.
145		Separación entre párrafos del documento.
146	<code>## Filtros, paginación y exportación</code>	Título o subtítulo Markdown que organiza la documentación.
147		Separación entre párrafos del documento.
148	- La búsqueda espera 350 ms antes de consultar para evitar una consulta por	Elemento de una lista explicativa.
149	cada tecla.	Texto de documentación destinado a explicar una decisión o procedimiento.
150	- Los filtros permiten separar ingredientes, productos directos y estados.	Elemento de una lista explicativa.
151	- La consulta trae 10 filas por página.	Elemento de una lista explicativa.
152	- Exportar genera un CSV con todos los registros que cumplen los filtros	Elemento de una lista explicativa.
153	actuales, no solamente la página visible.	Texto de documentación destinado a explicar una decisión o procedimiento.
154		Separación entre párrafos del documento.
155	Las consultas se ejecutan con <code>`SwingWorker`</code> , así la interfaz no se congela	Texto de documentación destinado a explicar una decisión o procedimiento.
156	mientras MySQL responde.	Texto de documentación destinado a explicar una decisión o procedimiento.
157		Separación entre párrafos del documento.
158	<code>## Uso</code>	Título o subtítulo Markdown que organiza la documentación.
159		Separación entre párrafos del documento.
160	1. Abre <code>**Administrador > Inventario**</code> .	Texto de documentación destinado a explicar una decisión o procedimiento.
161	2. En <code>**Existencias**</code> , localiza y selecciona el artículo.	Texto de documentación destinado a explicar una decisión o procedimiento.
162	3. Registra entrada, salida o ajuste con los botones superiores o <code>`± Stock`</code> .	Texto de documentación destinado a explicar una decisión o procedimiento.
163	4. En <code>**Ingredientes**</code> , crea o edita los datos descriptivos y el stock mínimo.	Texto de documentación destinado a explicar una decisión o procedimiento.
164	5. En <code>**Movimientos**</code> , revisa la trazabilidad y filtra el historial.	Texto de documentación destinado a explicar una decisión o procedimiento.
165	6. Usa Exportar desde Existencias para guardar el resultado actual en CSV.	Texto de documentación destinado a explicar una decisión o procedimiento.
166		Separación entre párrafos del documento.
167	<code>## Verificación realizada</code>	Título o subtítulo Markdown que organiza la documentación.
168		Separación entre párrafos del documento.
169	- Compilación completa con JDK 21.	Elemento de una lista explicativa.
170	- Consulta real de solo lectura: 89 artículos cargados en la base probada, 54	Elemento de una lista explicativa.

171	ingredientes y 35 productos directos.	Texto de documentación destinado a explicar una decisión o procedimiento.
172	- Filtros de tipo y estado ejecutados correctamente.	Elemento de una lista explicativa.
173	- Prueba visual del menú, panel y navegación interna.	Elemento de una lista explicativa.
174	- Validación XML de los siete formularios `.form` relacionados.	Elemento de una lista explicativa.
175	- Validación de que el nombre de la tabla es texto y no una celda con imagen.	Elemento de una lista explicativa.

16.4 DOCUMENTACION_MENU_CONFIGURABLE.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 77 | Archivo 4 de 157

1	# Menú configurable de Waldonald's	Título o subtítulo Markdown que organiza la documentación.
2		Separación entre párrafos del documento.
3	## Puesta en marcha	Título o subtítulo Markdown que organiza la documentación.
4		Separación entre párrafos del documento.
5	Para una instalación nueva, ejecuta `sql/base_datos_completa.sql`. El programa	Texto de documentación destinado a explicar una decisión o procedimiento.
6	usa directamente las tablas `presentacion_menu`, `grupo_presentacion`,	Texto de documentación destinado a explicar una decisión o procedimiento.
7	`opcion_grupo`, `opcion_componente` y `producto_ingrediente`; no vuelve a usar	Texto de documentación destinado a explicar una decisión o procedimiento.
8	los campos antiguos `es_combo` ni `tamano_bebida`.	Texto de documentación destinado a explicar una decisión o procedimiento.
9		Separación entre párrafos del documento.
10	Si ya conservas productos creados antes del cambio, ejecuta una sola vez	Texto de documentación destinado a explicar una decisión o procedimiento.
11	`sql/migracion_productos_existentes.sql`. Ese archivo solo les crea su	Texto de documentación destinado a explicar una decisión o procedimiento.
12	presentación individual inicial; no borra ventas, productos ni inventario.	Texto de documentación destinado a explicar una decisión o procedimiento.
13		Separación entre párrafos del documento.
14	Al crear un producto desde Gestión del menú , el sistema crea automáticamente	Texto de documentación destinado a explicar una decisión o procedimiento.
15	su presentación Individual y un grupo interno que contiene ese producto.	Texto de documentación destinado a explicar una decisión o procedimiento.
16	Después selecciona el producto en la tabla y pulsa Opciones y receta .	Texto de documentación destinado a explicar una decisión o procedimiento.
17		Separación entre párrafos del documento.
18	## Tipos de inventario	Título o subtítulo Markdown que organiza la documentación.
19		Separación entre párrafos del documento.
20	- DIRECTO : descuenta unidades de `producto.stock_actual` (bebida embotellada,	Elemento de una lista explicativa.
21	juguete, postre ya preparado).	Texto de documentación destinado a explicar una decisión o procedimiento.
22	- RECETA : descuenta los ingredientes de `producto_ingrediente`. Es el tipo	Elemento de una lista explicativa.
23	normal para hamburguesas y otros productos preparados.	Texto de documentación destinado a explicar una decisión o procedimiento.
24	- NINGUNO : no descuenta inventario por sí mismo. Se usa para un producto que	Elemento de una lista explicativa.
25	funciona únicamente como nombre/contenedor de una Cajita o Caja Grande.	Texto de documentación destinado a explicar una decisión o procedimiento.
26		Separación entre párrafos del documento.
27	Solo los productos RECETA muestran opciones Sin y Extra . En la pestaña	Texto de documentación destinado a explicar una decisión o procedimiento.
28	Ingredientes y personalización se define la cantidad normal, si se puede	Texto de documentación destinado a explicar una decisión o procedimiento.
29	quitar, la cantidad de cada extra, su precio y el máximo permitido.	Texto de documentación destinado a explicar una decisión o procedimiento.
30		Separación entre párrafos del documento.
31	## Big Mac individual o McMenú	Título o subtítulo Markdown que organiza la documentación.
32		Separación entre párrafos del documento.
33	1. Crea el producto Big Mac como RECETA y configura sus ingredientes.	Texto de documentación destinado a explicar una decisión o procedimiento.
34	2. Conserva la presentación Individual creada automáticamente.	Texto de documentación destinado a explicar una decisión o procedimiento.
35	3. Agrega una presentación McMenú , tipo MENU , con su precio base.	Texto de documentación destinado a explicar una decisión o procedimiento.
36	4. Dentro de McMenú crea un grupo oculto que incluya el Big Mac, o agrega el	Texto de documentación destinado a explicar una decisión o procedimiento.
37	producto principal como componente fijo.	Texto de documentación destinado a explicar una decisión o procedimiento.
38	5. Crea los grupos visibles Elige el tamaño , Elige el complemento y	Texto de documentación destinado a explicar una decisión o procedimiento.

39	**Elige la bebida**, todos con mínimo 1 y máximo 1.	Texto de documentación destinado a explicar una decisión o procedimiento.
40	6. Cada opción de complemento o bebida apunta al producto real correspondiente.	Texto de documentación destinado a explicar una decisión o procedimiento.
41	El precio mayor de una opción se guarda en `incremento_precio`.	Texto de documentación destinado a explicar una decisión o procedimiento.
42		Separación entre párrafos del documento.
43	El cajero verá Individual y McMenú como pestañas. En Individual podrá quitar o	Texto de documentación destinado a explicar una decisión o procedimiento.
44	agregar ingredientes; en McMenú además tendrá que completar los grupos.	Texto de documentación destinado a explicar una decisión o procedimiento.
45		Separación entre párrafos del documento.
46	## Cajita infantil	Título o subtítulo Markdown que organiza la documentación.
47		Separación entre párrafos del documento.
48	1. Crea un producto contenedor con tipo de inventario `NINGUNO`.	Texto de documentación destinado a explicar una decisión o procedimiento.
49	2. Crea una presentación de tipo `INFANTIL`.	Texto de documentación destinado a explicar una decisión o procedimiento.
50	3. Agrega grupos con mínimo 1 y máximo 1: comida principal, complemento, bebida	Texto de documentación destinado a explicar una decisión o procedimiento.
51	y juguete.	Texto de documentación destinado a explicar una decisión o procedimiento.
52	4. Para un adicional opcional que deba responderse explícitamente, usa mínimo 1	Texto de documentación destinado a explicar una decisión o procedimiento.
53	y máximo 1 y agrega una opción **No gracias** sin productos internos.	Texto de documentación destinado a explicar una decisión o procedimiento.
54		Separación entre párrafos del documento.
55	## Combo predefinido con cuatro hamburguesas	Título o subtítulo Markdown que organiza la documentación.
56		Separación entre párrafos del documento.
57	1. Crea el producto contenedor de tipo `NINGUNO` y una presentación `COMBO`.	Texto de documentación destinado a explicar una decisión o procedimiento.
58	2. Crea **Elige tus hamburguesas** con mínimo 4, máximo 4 y	Texto de documentación destinado a explicar una decisión o procedimiento.
59	`permite_repetir = true`.	Texto de documentación destinado a explicar una decisión o procedimiento.
60	3. Cada opción contiene una hamburguesa real. El cajero verá cuatro selectores.	Texto de documentación destinado a explicar una decisión o procedimiento.
61	4. Para papas o bebida fijas, crea grupos ocultos con una opción predeterminada	Texto de documentación destinado a explicar una decisión o procedimiento.
62	y agrega esos productos como componentes con la cantidad incluida.	Texto de documentación destinado a explicar una decisión o procedimiento.
63		Separación entre párrafos del documento.
64	Aunque se elija la misma hamburguesa varias veces, el programa crea una unidad	Texto de documentación destinado a explicar una decisión o procedimiento.
65	interna independiente para cada una. Por eso sus ingredientes pueden cambiarse	Texto de documentación destinado a explicar una decisión o procedimiento.
66	por separado y el ticket de cocina imprime cada preparación individualmente.	Texto de documentación destinado a explicar una decisión o procedimiento.
67		Separación entre párrafos del documento.
68	## Reglas importantes	Título o subtítulo Markdown que organiza la documentación.
69		Separación entre párrafos del documento.
70	- `minimo` y `maximo` se vuelven a validar al cobrar; la interfaz no es la única	Elemento de una lista explicativa.
71	protección.	Texto de documentación destinado a explicar una decisión o procedimiento.
72	- Los precios y el inventario se vuelven a consultar dentro de la transacción de	Elemento de una lista explicativa.
73	venta para evitar cobros viejos o sobreventa entre dos cajas.	Texto de documentación destinado a explicar una decisión o procedimiento.
74	- `disponibilidad_menu` conserva `TODO_DIA`, `DESAYUNO` y `ALMUERZO`. El horario	Elemento de una lista explicativa.
75	existente del proyecto no fue modificado.	Texto de documentación destinado a explicar una decisión o procedimiento.
76	- El comprobante conserva el detalle de la configuración y el cajero puede abrir	Elemento de una lista explicativa.
77	un ticket de cocina separado, con opciones y modificaciones resaltadas.	Texto de documentación destinado a explicar una decisión o procedimiento.

16.5 DOCUMENTACION_PAGOS.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 36 | Archivo 5 de 157

		Título o subtítulo Markdown que organiza la documentación.
1	# Pagos en caja	Separación entre párrafos del documento.
2		
3	El cajero permite agregar productos al pedido, ajustar cantidades, quitar líneas y cancelar el carrito. Al conti nuar se elige comer aquí o para llevar, y efectivo o tarjeta cobrada en una terminal externa. Los importes se mu estran en quetzales (Q), incluidos el menú, la administración y los comprobantes. Cancelar cierra el panel y dev uelve el espacio al menú; si existe un cobro pendiente, se conserva para reintentar al abrir «Ordenar».	Texto de documentación destinado a explicar una decisión o procedimiento.
4		Separación entre párrafos del documento.
5	En efectivo se validan hasta dos decimales, monto suficiente y cambio. En tarjeta se exige confirmar la aprobaci ón externa y escribir la referencia de la terminal. Esta aplicación registra el pago externo; no envía cargos ba ncarios ni almacena números de tarjeta, CVV o PIN.	Texto de documentación destinado a explicar una decisión o procedimiento.
6		Separación entre párrafos del documento.
7	El cobro verifica usuario activo, productos activos, horario, precio vigente y stock. Registra pedido pagado, de talles y salidas de inventario en una única transacción InnoDB. El número de orden usa el identificador autoincr emental del pedido, sin reiniciarlo diariamente. Se descuentan existencias de productos terminados; la receta ac tual no define cantidades de ingredientes para descontarlos.	Texto de documentación destinado a explicar una decisión o procedimiento.
8		Separación entre párrafos del documento.
9	## Instalación	Título o subtítulo Markdown que organiza la documentación.
10		Separación entre párrafos del documento.
11	1. Usar Java 21 o posterior y la conexión MySQL configurada en `src/Conexion/Conexion.java`.	Texto de documentación destinado a explicar una decisión o procedimiento.
12	2. Ejecutar `sql/pagos.sql` en la base `waldonalds`. Es una migración aditiva e idempotente para guardar claves de operación y comprobantes; no modifica pedidos anteriores.	Texto de documentación destinado a explicar una decisión o procedimiento.
13	3. Compilar y ejecutar el proyecto desde NetBeans. Las dependencias son AbsoluteLayout y el conector MySQL que y a incluye el proyecto.	Texto de documentación destinado a explicar una decisión o procedimiento.
14	4. Iniciar sesión con un usuario activo. Los productos deben tener stock mayor que cero y estar disponibles en e l horario actual.	Texto de documentación destinado a explicar una decisión o procedimiento.
15		Separación entre párrafos del documento.
16	## Fallos y recuperación	Título o subtítulo Markdown que organiza la documentación.
17		Separación entre párrafos del documento.
18	Antes de enviar el cobro, se guarda su solicitud en `pagos_pendientes/caja-ID.properties`, relativa al directori o de ejecución. Mantener ese directorio estable y escribible, y usar una instancia por cajero. No borrar el resp aldo mientras el resultado esté pendiente.	Texto de documentación destinado a explicar una decisión o procedimiento.
19		Separación entre párrafos del documento.
20	Ante un error SQL, el carrito queda bloqueado y ofrece reintentar la misma operación. Si MySQL ya confirmó la ve nta pero se perdió la respuesta, devuelve el comprobante original sin duplicar el pedido ni descontar más stock. Al reiniciar desde el mismo directorio, el cajero recupera la solicitud pendiente. No repetir el cargo en la te rminal externa al reintentar su registro. Si la terminal aprobó pero el pedido no se puede completar, se debe re solver o revertir ese cargo en la propia terminal.	Texto de documentación destinado a explicar una decisión o procedimiento.
21		Separación entre párrafos del documento.
22	Tras un pago exitoso se muestra el comprobante y puede imprimirse; el botón «Último comprobante» permite reabrir lo durante la sesión. La copia persistente está en `pago_operacion.comprobante`. Es un comprobante de venta inte rno; no implementa facturación electrónica.	Texto de documentación destinado a explicar una decisión o procedimiento.
23		Separación entre párrafos del documento.
24	## Pruebas	Título o subtítulo Markdown que organiza la documentación.
25		Separación entre párrafos del documento.
26	Desde la carpeta `waldonalds`, en PowerShell con JDK disponible:	Texto de documentación destinado a explicar una decisión o procedimiento.

27		Separación entre párrafos del documento.
28	```powershell	Abre o cierra un ejemplo de código con formato monoespaciado.
29	\$fuentes = @(Get-ChildItem src -Recurse -Filter *.java ForEach-Object { \$_.FullName })	Texto de documentación destinado a explicar una decisión o procedimiento.
30	javac -encoding UTF-8 --release 21 -cp 'dist/lib/AbsoluteLayout.jar;lib/mysql-connector-j-26.7.0.jar' -d build/p ayment-check \$fuentes test/PagosTest.java	Texto de documentación destinado a explicar una decisión o procedimiento.
31	java '-Djava.awt.headless=true' -cp 'build/payment-check;dist/lib/AbsoluteLayout.jar;lib/mysql-connector-j-26.7. 0.jar' PagosTest	Texto de documentación destinado a explicar una decisión o procedimiento.
32	```	Abre o cierra un ejemplo de código con formato monoespaciado.
33		Separación entre párrafos del documento.
34	Las pruebas requieren permisos para crear una base temporal. Copian únicamente el esquema necesario a una base n ueva con nombre aleatorio `waldonalds_test_pagos_*`, insertan datos ficticios y eliminan exclusivamente esa base en `finally`. No insertan ventas ni cambian existencias en la base de trabajo.	Texto de documentación destinado a explicar una decisión o procedimiento.
35		Separación entre párrafos del documento.
36	Cubren decimales, pago insuficiente, pedido vacío, referencia de tarjeta, total y cambio, recuperación del respa ldo, stock, precio modificado, usuario y producto inactivos, horario, rollback ante fallo de inventario, reinten tos y pérdida de respuesta del commit, y concurrencia por la última existencia.	Texto de documentación destinado a explicar una decisión o procedimiento.

16.6 GUIA_REPORTES.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 92 | Archivo 6 de 157

1	# Panel de reportes de Waldonald's	Título o subtítulo Markdown que organiza la documentación.
2		Separación entre párrafos del documento.
3	## Abrir y modificar el diseño	Título o subtítulo Markdown que organiza la documentación.
4		Separación entre párrafos del documento.
5	Abre `src/GUI_ADMINISTRADOR/ReportesPanel.java` en NetBeans y selecciona Design . El diseño se guarda en `ReportesPanel.form`, junto al Java. Ambos archivos se actualizaron. Si NetBeans tenía abierta una versión anterior, recarga los cambios del disco antes de guardar; la captura compartida y el archivo inicialmente guardado tenían textos diferentes.	Texto de documentación destinado a explicar una decisión o procedimiento.
6		Separación entre párrafos del documento.
7	Los componentes están en el mismo panel. Puedes moverlos, redimensionarlos y cambiar sus propiedades en el diseñador. Se mantuvieron los componentes `PanelFlotante`, `PanelCircular` y `LabelEscalable` que ya usabas. Las tablas utilizan `TablaAdministrativa`, que ya existía en el proyecto.	Texto de documentación destinado a explicar una decisión o procedimiento.
8		Separación entre párrafos del documento.
9	Elemento Nombre en el Inspector Propiedades útiles	Texto de documentación destinado a explicar una decisión o procedimiento.
10	--- --- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
11	Tarjeta de ventas `panelFlotante1` `colorFondo`, `colorBorde`, `radio`, `sombra`	Texto de documentación destinado a explicar una decisión o procedimiento.
12	Tarjeta de pedidos `panelFlotante2` Las mismas propiedades	Texto de documentación destinado a explicar una decisión o procedimiento.
13	Tarjeta de ticket promedio `panelFlotante4` Las mismas propiedades	Texto de documentación destinado a explicar una decisión o procedimiento.
14	Tarjeta de stock bajo `panelFlotante3` Las mismas propiedades	Texto de documentación destinado a explicar una decisión o procedimiento.
15	Círculos de iconos `panelCircular2`, `3`, `5`, `4` `colorFondo`	Texto de documentación destinado a explicar una decisión o procedimiento.
16	Imágenes de tarjetas `labelEscalable2`, `3`, `5`, `4` `icon`	Texto de documentación destinado a explicar una decisión o procedimiento.
17	Valores de tarjetas `labelTitulo5`, `8`, `14`, `11` `font`, `foreground`; el texto se actualiza con datos	Texto de documentación destinado a explicar una decisión o procedimiento.
18	Selector de fecha `fechaReporte` Posición, tamaño y fuente	Texto de documentación destinado a explicar una decisión o procedimiento.
19	Controles `botonHoy`, `botonActualizar`, `botonExportar` Texto, fondo, fuente y tamaño	Texto de documentación destinado a explicar una decisión o procedimiento.
20	Tablas `tablaPedidos`, `tablaProductos`, `tablaAlertas` `colorCabecera`, `fuenteCabecera`, `altoCabecera`, `colorFilas`, `colorFilaAlterna`, `filasAlternadas`, `paddingHorizontal`	Texto de documentación destinado a explicar una decisión o procedimiento.
21	Área de cada tabla `scrollPedidos`, `scrollProductos`, `scrollAlertas` Posición y tamaño del área visible	Texto de documentación destinado a explicar una decisión o procedimiento.
22	Encabezados `tituloPedidos`, `tituloProductos`, `tituloAlertas` Fuente, color y posición	Texto de documentación destinado a explicar una decisión o procedimiento.
23	Estado de la consulta `estadoCarga` Fuente, color y posición	Texto de documentación destinado a explicar una decisión o procedimiento.
24		Separación entre párrafos del documento.
25	La interfaz utiliza directamente las medidas guardadas en los formularios de	Texto de documentación destinado a explicar una decisión o procedimiento.
26	NetBeans. El panel de reportes ocupa 1580 × 980 dentro del diseño administrativo	Texto de documentación destinado a explicar una decisión o procedimiento.
27	de 1920 × 1080. No existe una transformación automática de posiciones, tamaños	Texto de documentación destinado a explicar una decisión o procedimiento.
28	o fuentes.	Texto de documentación destinado a explicar una decisión o procedimiento.
29		Separación entre párrafos del documento.
30	## Qué muestra y cómo se calcula	Título o subtítulo Markdown que organiza la documentación.
31		Separación entre párrafos del documento.
32	- Ventas del día: suma de `pedido.total` solo cuando `estado='PAGADO'`.	Elemento de una lista explicativa.
33	- Pedidos del día: todos los pedidos de la fecha, incluidos pendientes y cancelados. La tarjeta lo indica.	Elemento de una lista explicativa.

34	- **Ticket promedio:** promedio del total de los pedidos pagados. Si no existen, muestra Q0.00.	Elemento de una lista explicativa.
35	- **Productos con stock bajo:** productos activos cuyo `stock_actual <= stock_minimo`. Es inventario actual, independiente de la fecha seleccionada.	Elemento de una lista explicativa.
36	- **Pedidos:** todos los pedidos del día, ordenados inicialmente del más reciente al más antiguo. Incluye cajero, hora, servicio, estado y total. Puedes ordenar pulsando los encabezados y desplazarte para ver todos.	Elemento de una lista explicativa.
37	- **Productos más vendidos:** los 10 primeros por unidades de pedidos pagados. Cuenta líneas principales (`id_de_talle_padre IS NULL`), de modo que los componentes no vuelvan a contabilizar una venta de combo. Su estado de disponibilidad es el actual.	Elemento de una lista explicativa.
38	- **Alertas:** productos e ingredientes activos en o por debajo del mínimo, con cantidad, umbral y unidad. Los agotados aparecen primero.	Elemento de una lista explicativa.
39		Separación entre párrafos del documento.
40	Las ventas son ingresos cobrados, no ganancias: el esquema no contiene costos para calcular utilidad. Los únicos estados mostrados son los que tu base guarda: pendiente, pagado y cancelado. Las alertas se limitan a existencias porque el esquema proporcionado no registra vencimientos ni mantenimiento programado.	Texto de documentación destinado a explicar una decisión o procedimiento.
41		Separación entre párrafos del documento.
42	La fecha incluye desde las 00:00:00 hasta antes de las 00:00:00 del día siguiente. Se usan parámetros SQL y el campo DATETIME del pedido. Se conserva la conexión existente del proyecto.	Texto de documentación destinado a explicar una decisión o procedimiento.
43		Separación entre párrafos del documento.
44	## Uso y organización del código	Título o subtítulo Markdown que organiza la documentación.
45		Separación entre párrafos del documento.
46	1. Entra a **Reportes** : la sección consulta automáticamente. Al regresar desde otra sección vuelve a consultar.	Texto de documentación destinado a explicar una decisión o procedimiento.
47	2. Para otra fecha, escríbela como `dd/MM/aaaa` y pulsa **Actualizar** . **Hoy** vuelve a la fecha actual.	Texto de documentación destinado a explicar una decisión o procedimiento.
48	3. **Exportar CSV** guarda los indicadores y las tres tablas del reporte cargado. Incluye la fecha y aclara que las existencias son actuales. Los importes se exportan con decimales, texto UTF-8 y protección para nombres que podrían interpretarse como fórmulas.	Texto de documentación destinado a explicar una decisión o procedimiento.
49		Separación entre párrafos del documento.
50	`ReportesPanel.java` conserva la presentación generada y contiene eventos/carga fuera de ` initComponents()`. `DAO/ReporteDAO.java` contiene las consultas SQL. `Utilidades/ReporteCsv.java` contiene la exportación. No es necesario editar esas clases para mover componentes o cambiar los colores del formulario.	Texto de documentación destinado a explicar una decisión o procedimiento.
51		Separación entre párrafos del documento.
52	La carga utiliza `SwingWorker`: MySQL se consulta fuera del hilo que dibuja Swing. Los resultados se aplican juntos. Mientras se consulta, se bloquean las acciones duplicadas. Ante un error se muestran guiones y un mensaje para reintentar; no se presentan ceros como si la consulta hubiera tenido éxito. Las consultas usan una transacción de lectura para mantener coherentes los indicadores y las tablas.	Texto de documentación destinado a explicar una decisión o procedimiento.
53		Separación entre párrafos del documento.
54	## Animación del menú	Título o subtítulo Markdown que organiza la documentación.
55		Separación entre párrafos del documento.
56	En `MenuAdmin.java` se mantiene un temporizador Swing de aproximadamente 60 actualizaciones por segundo. La transición usa una curva suave de aceleración y frenado, con duración máxima de 300 ms. Si pulsas otra vez durante la transición, se invierte desde la posición actual y la duración se adapta al recorrido pendiente.	Texto de documentación destinado a explicar una decisión o procedimiento.
57		Separación entre párrafos del documento.
58	En `BotonMenuLateral.java`, `aperturaMenu` controla la opacidad del texto y la	Texto de documentación destinado a explicar una decisión o procedimiento.
59	posición horizontal del icono. El texto permanece disponible para accesibilidad	Texto de documentación destinado a explicar una decisión o procedimiento.
60	y ayudas emergentes. El nombre y rol de la marca también se atenúan. Esta	Texto de documentación destinado a explicar una decisión o procedimiento.
61	animación modifica únicamente el ancho del menú lateral; no escala la interfaz.	Texto de documentación destinado a explicar una decisión o procedimiento.
62	El control admite Espacio y Enter cuando tiene el foco y el temporizador se	Texto de documentación destinado a explicar una decisión o procedimiento.
63	detiene al cerrar la ventana.	Texto de documentación destinado a explicar una decisión o procedimiento.
64		Separación entre párrafos del documento.
65	## Iconos	Título o subtítulo Markdown que organiza la documentación.
66		Separación entre párrafos del documento.

67	No se generaron ni dibujaron iconos nuevos. Las cuatro tarjetas reutilizan estos archivos de `src/Imagenes`:	Texto de documentación destinado a explicar una decisión o procedimiento.
68		Separación entre párrafos del documento.
69	- `quetzal_icono.png`: ventas.	Elemento de una lista explicativa.
70	- `bolsa_icono.png`: pedidos.	Elemento de una lista explicativa.
71	- `ticket_icono.png`: ticket promedio.	Elemento de una lista explicativa.
72	- `caja_icono.png`: existencias.	Elemento de una lista explicativa.
73		Separación entre párrafos del documento.
74	El panel actual no necesita más imágenes para funcionar. Para añadir iconos de encabezado similares a tu referencia, puedes preparar:	Texto de documentación destinado a explicar una decisión o procedimiento.
75		Separación entre párrafos del documento.
76	- `reportes_pedidos.png`: portapapeles/lista, amarillo.	Elemento de una lista explicativa.
77	- `reportes_productos.png`: hamburguesa, amarillo.	Elemento de una lista explicativa.
78	- `reportes_alertas.png`: triángulo de advertencia, rojo; también se puede evaluar el `advertencia_icono.png` que ya tienes.	Elemento de una lista explicativa.
79		Separación entre párrafos del documento.
80	PNG transparente de 64 × 64 o 96 × 96 px, con el mismo grosor de trazo y margen interno. Se mostrarían aproximadamente a 24-28 px. Los iconos que ya usaba el menú lateral se conservaron; se cambió su posición durante la animación, no su dibujo.	Texto de documentación destinado a explicar una decisión o procedimiento.
81		Separación entre párrafos del documento.
82	## Verificación realizada	Título o subtítulo Markdown que organiza la documentación.
83		Separación entre párrafos del documento.
84	- Compilación completa con Java 21 y construcción del JAR mediante Ant: correctas.	Elemento de una lista explicativa.
85	- Consultas a tu MySQL en modo de lectura: correctas. En la comprobación del 27/09/2026 no había pedidos de ese día.	Elemento de una lista explicativa.
86	- Pruebas SQL con tablas TEMPORARY exclusivas de la conexión de prueba: pagos, pendientes, cancelaciones, medianos, combos, mínimos inclusivos, registros inactivos y día vacío. Se destruyen al cerrar la conexión; no alteran las tablas reales.	Elemento de una lista explicativa.
87	- Validación de fecha imposible, constructor sin conexión obligatoria, tablas de solo lectura y escape de CSV.	Elemento de una lista explicativa.
88	- Prueba de apertura/cierre e inversión durante la animación con las medidas	Elemento de una lista explicativa.
89	fijas del formulario de 1920 × 1080.	Texto de documentación destinado a explicar una decisión o procedimiento.
90	- Lectura del modelo de fecha y los tres modelos de tabla mediante los editores reales de NetBeans 24.	Elemento de una lista explicativa.
91		Separación entre párrafos del documento.
92	Los programas de comprobación están en `test/ReportesTest.java`, `test/ReporteDAOTest.java`, `test/MenuAnimacionTest.java` y `test/FormReportesTest.java`. Las capturas de verificación están en `build/reportes-check/`. La prueba de formulario necesita las bibliotecas del NetBeans instalado; las otras usan las dependencias normales del proyecto.	Texto de documentación destinado a explicar una decisión o procedimiento.

16.7 lib/nblibraries.properties

Archivo de propiedades clave=valor utilizado por NetBeans o por la compilación.

Tipo: .properties | Líneas: 7 | Archivo 7 de 157

1	libs.absolutelayout.classpath=\	Asigna un valor a la propiedad libs.absolutelayout.classpath; NetBeans/Ant la usa al compilar o ejecutar.
2	\${base}/absolutelayout/AbsoluteLayout.jar	Contenido de configuración o documentación que completa el archivo.
3	libs.absolutelayout.displayName=Absolute Layout	Asigna un valor a la propiedad libs.absolutelayout.displayName; NetBeans/Ant la usa al compilar o ejecutar.
4	libs.CopyLibs.classpath=\	Asigna un valor a la propiedad libs.CopyLibs.classpath; NetBeans/Ant la usa al compilar o ejecutar.
5	\${base}/CopyLibs/org-netbeans-modules-java-j2seproject-copylibstask.jar	Contenido de configuración o documentación que completa el archivo.
6	libs.CopyLibs.displayName=CopyLibs Task	Asigna un valor a la propiedad libs.CopyLibs.displayName; NetBeans/Ant la usa al compilar o ejecutar.
7	libs.CopyLibs.prop-version=3.0	Asigna un valor a la propiedad libs.CopyLibs.prop-version; NetBeans/Ant la usa al compilar o ejecutar.

16.8 manifest.mf

Archivo de apoyo del proyecto: manifest.mf.

Tipo: .mf | Líneas: 3 | Archivo 8 de 157

		Manifest-Version: 1.0	Contenido de configuración o documentación que completa el archivo.
		X-COMMENT: Main-Class will be added automatically by build	Contenido de configuración o documentación que completa el archivo.
			Línea en blanco usada para separar secciones.

16.9 MANUAL_TABLA_ADMINISTRATIVA.md

Documentación complementaria escrita durante el desarrollo del proyecto.

Tipo: .md | Líneas: 326 | Archivo 9 de 157

1	# Manual de TablaAdministrativa	Título o subtítulo Markdown que organiza la documentación.
2		Separación entre párrafos del documento.
3	## 1. Qué es	Título o subtítulo Markdown que organiza la documentación.
4		Separación entre párrafos del documento.
5	`TablaAdministrativa` es un componente reutilizable basado en	Texto de documentación destinado a explicar una decisión o procedimiento.
6	`JTable`. Centraliza el diseño de las tablas administrativas y permite	Texto de documentación destinado a explicar una decisión o procedimiento.
7	que cada pantalla conserve su propio `TableModel`, consultas y acciones.	Texto de documentación destinado a explicar una decisión o procedimiento.
8		Separación entre párrafos del documento.
9	Archivo del componente:	Texto de documentación destinado a explicar una decisión o procedimiento.
10		Separación entre párrafos del documento.
11	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
12	src/Componentes/TablaAdministrativa.java	Texto de documentación destinado a explicar una decisión o procedimiento.
13	```	Abre o cierra un ejemplo de código con formato monoespaciado.
14		Separación entre párrafos del documento.
15	La tabla de usuarios ya utiliza este componente en `UsuariosPanel.form`.	Texto de documentación destinado a explicar una decisión o procedimiento.
16		Separación entre párrafos del documento.
17	## 2. Agregarlo a un formulario desde NetBeans	Título o subtítulo Markdown que organiza la documentación.
18		Separación entre párrafos del documento.
19	1. Ejecutar Clean and Build para compilar el componente.	Texto de documentación destinado a explicar una decisión o procedimiento.
20	2. Abrir Tools > Palette > Swing/AWT Components .	Texto de documentación destinado a explicar una decisión o procedimiento.
21	3. Usar Add from Project y seleccionar `TablaAdministrativa`.	Texto de documentación destinado a explicar una decisión o procedimiento.
22	4. Colocar primero un `JScrollPane` en el formulario.	Texto de documentación destinado a explicar una decisión o procedimiento.
23	5. Arrastrar `TablaAdministrativa` dentro del `JScrollPane`.	Texto de documentación destinado a explicar una decisión o procedimiento.
24	6. Cambiar el nombre de variable, por ejemplo `tablaProductos`.	Texto de documentación destinado a explicar una decisión o procedimiento.
25		Separación entre párrafos del documento.
26	También se puede cambiar una tabla existente en el archivo `.form` para	Texto de documentación destinado a explicar una decisión o procedimiento.
27	que su clase sea `Componentes.TablaAdministrativa`, como se hizo con	Texto de documentación destinado a explicar una decisión o procedimiento.
28	`tablaUsuarios`.	Texto de documentación destinado a explicar una decisión o procedimiento.
29		Separación entre párrafos del documento.
30	## 3. Propiedades visuales	Título o subtítulo Markdown que organiza la documentación.
31		Separación entre párrafos del documento.
32	Seleccionar la tabla en Design y abrir Properties .	Texto de documentación destinado a explicar una decisión o procedimiento.
33		Separación entre párrafos del documento.
34	Propiedad Función	Texto de documentación destinado a explicar una decisión o procedimiento.
35	--- ---	Texto de documentación destinado a explicar una decisión o procedimiento.
36	`font` Fuente utilizada por las filas.	Texto de documentación destinado a explicar una decisión o procedimiento.
37	`rowHeight` Altura de cada fila.	Texto de documentación destinado a explicar una decisión o procedimiento.
38	`fuenteCabecera` Fuente de los títulos de columna.	Texto de documentación destinado a explicar una decisión o procedimiento.

39	`altoCabecera` Altura de la cabecera.	Texto de documentación destinado a explicar una decisión o procedimiento.
40	`colorCabecera` Fondo de la cabecera.	Texto de documentación destinado a explicar una decisión o procedimiento.
41	`colorTextoCabecera` Texto de los títulos.	Texto de documentación destinado a explicar una decisión o procedimiento.
42	`colorFilas` Fondo normal de las filas.	Texto de documentación destinado a explicar una decisión o procedimiento.
43	`colorFilaAlterna` Fondo de filas pares cuando se activa la alternancia.	Texto de documentación destinado a explicar una decisión o procedimiento.
44	`filasAlternadas` Activa o desactiva filas de dos colores.	Texto de documentación destinado a explicar una decisión o procedimiento.
45	`colorTexto` Color del texto de las filas.	Texto de documentación destinado a explicar una decisión o procedimiento.
46	`colorLineas` Color de las divisiones horizontales.	Texto de documentación destinado a explicar una decisión o procedimiento.
47	`colorSeleccion` Fondo de la fila seleccionada.	Texto de documentación destinado a explicar una decisión o procedimiento.
48	`colorTextoSeleccion` Texto de la fila seleccionada.	Texto de documentación destinado a explicar una decisión o procedimiento.
49	`paddingHorizontal` Separación entre el texto y los bordes.	Texto de documentación destinado a explicar una decisión o procedimiento.
50	`columnasCentradas` Índices separados por coma, por ejemplo `5,7`.	Texto de documentación destinado a explicar una decisión o procedimiento.
51	`columnasDerecha` Columnas alineadas a la derecha, por ejemplo `2,4`.	Texto de documentación destinado a explicar una decisión o procedimiento.
52		Separación entre párrafos del documento.
53	Los índices comienzan en cero. En una tabla con ocho columnas:	Texto de documentación destinado a explicar una decisión o procedimiento.
54		Separación entre párrafos del documento.
55	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
56	0 1 2 3 4 5 6 7	Texto de documentación destinado a explicar una decisión o procedimiento.
57	ID Nombre Usuario Rol Correo Estado Fecha creación Acciones	Texto de documentación destinado a explicar una decisión o procedimiento.
58	```	Abre o cierra un ejemplo de código con formato monoespaciado.
59		Separación entre párrafos del documento.
60	Por eso la tabla de usuarios tiene:	Texto de documentación destinado a explicar una decisión o procedimiento.
61		Separación entre párrafos del documento.
62	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
63	columnasCentradas = 5,7	Texto de documentación destinado a explicar una decisión o procedimiento.
64	```	Abre o cierra un ejemplo de código con formato monoespaciado.
65		Separación entre párrafos del documento.
66	Todas las demás columnas quedan alineadas a la izquierda.	Texto de documentación destinado a explicar una decisión o procedimiento.
67		Separación entre párrafos del documento.
68	## 4. Modelo básico	Título o subtítulo Markdown que organiza la documentación.
69		Separación entre párrafos del documento.
70	La tabla reutiliza el modelo que necesite cada pantalla:	Texto de documentación destinado a explicar una decisión o procedimiento.
71		Separación entre párrafos del documento.
72	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
73	private final class ModeloProductos extends AbstractTableModel {	Texto de documentación destinado a explicar una decisión o procedimiento.
74		Separación entre párrafos del documento.
75	private final String[] columnas = {	Texto de documentación destinado a explicar una decisión o procedimiento.
76	"Producto", "Categoría", "Precio", "Estado"	Texto de documentación destinado a explicar una decisión o procedimiento.
77	};	Texto de documentación destinado a explicar una decisión o procedimiento.
78		Separación entre párrafos del documento.
79	private List<Producto> productos = new ArrayList<>();	Texto de documentación destinado a explicar una decisión o procedimiento.
80		Separación entre párrafos del documento.
81	@Override	Texto de documentación destinado a explicar una decisión o procedimiento.
82	public int getRowCount() {	Texto de documentación destinado a explicar una decisión o procedimiento.

83	return productos.size();	Texto de documentación destinado a explicar una decisión o procedimiento.
84	}	Texto de documentación destinado a explicar una decisión o procedimiento.
85		Separación entre párrafos del documento.
86	@Override	Texto de documentación destinado a explicar una decisión o procedimiento.
87	public int getColumnCount() {	Texto de documentación destinado a explicar una decisión o procedimiento.
88	return columnas.length;	Texto de documentación destinado a explicar una decisión o procedimiento.
89	}	Texto de documentación destinado a explicar una decisión o procedimiento.
90		Separación entre párrafos del documento.
91	@Override	Texto de documentación destinado a explicar una decisión o procedimiento.
92	public String getColumnName(int columna) {	Texto de documentación destinado a explicar una decisión o procedimiento.
93	return columnas[columna];	Texto de documentación destinado a explicar una decisión o procedimiento.
94	}	Texto de documentación destinado a explicar una decisión o procedimiento.
95		Separación entre párrafos del documento.
96	@Override	Texto de documentación destinado a explicar una decisión o procedimiento.
97	public Object getValueAt(int fila, int columna) {	Texto de documentación destinado a explicar una decisión o procedimiento.
98	Producto producto = productos.get(fila);	Texto de documentación destinado a explicar una decisión o procedimiento.
99		Separación entre párrafos del documento.
100	return switch (columna) {	Texto de documentación destinado a explicar una decisión o procedimiento.
101	case 0 -> producto.getNombre();	Texto de documentación destinado a explicar una decisión o procedimiento.
102	case 1 -> producto.getCategoria();	Texto de documentación destinado a explicar una decisión o procedimiento.
103	case 2 -> producto.getPrecio();	Texto de documentación destinado a explicar una decisión o procedimiento.
104	case 3 -> producto.getEstado();	Texto de documentación destinado a explicar una decisión o procedimiento.
105	default -> "";	Texto de documentación destinado a explicar una decisión o procedimiento.
106	};	Texto de documentación destinado a explicar una decisión o procedimiento.
107	}	Texto de documentación destinado a explicar una decisión o procedimiento.
108	}	Texto de documentación destinado a explicar una decisión o procedimiento.
109	```	Abre o cierra un ejemplo de código con formato monoespaciado.
110		Separación entre párrafos del documento.
111	Se conecta normalmente:	Texto de documentación destinado a explicar una decisión o procedimiento.
112		Separación entre párrafos del documento.
113	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
114	ModeloProductos modelo = new ModeloProductos();	Texto de documentación destinado a explicar una decisión o procedimiento.
115	tablaProductos.setModel(modelo);	Texto de documentación destinado a explicar una decisión o procedimiento.
116	tablaProductos.aplicarEstilo();	Texto de documentación destinado a explicar una decisión o procedimiento.
117	```	Abre o cierra un ejemplo de código con formato monoespaciado.
118		Separación entre párrafos del documento.
119	Se llama `aplicarEstilo()` después de cambiar el modelo para reaplicar la	Texto de documentación destinado a explicar una decisión o procedimiento.
120	cabecera y las propiedades del componente.	Texto de documentación destinado a explicar una decisión o procedimiento.
121		Separación entre párrafos del documento.
122	## 5. Mostrar imagen, nombre y descripción	Título o subtítulo Markdown que organiza la documentación.
123		Separación entre párrafos del documento.
124	### Base de datos	Título o subtítulo Markdown que organiza la documentación.
125		Separación entre párrafos del documento.
126	Conviene guardar una ruta relativa:	Texto de documentación destinado a explicar una decisión o procedimiento.

127		Separación entre párrafos del documento.
128	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
129	ALTER TABLE producto	Texto de documentación destinado a explicar una decisión o procedimiento.
130	ADD COLUMN ruta_imagen VARCHAR(255);	Texto de documentación destinado a explicar una decisión o procedimiento.
131	```	Abre o cierra un ejemplo de código con formato monoespaciado.
132		Separación entre párrafos del documento.
133	Ejemplos:	Texto de documentación destinado a explicar una decisión o procedimiento.
134		Separación entre párrafos del documento.
135	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
136	productos/big-wald.png	Texto de documentación destinado a explicar una decisión o procedimiento.
137	productos/papas.png	Texto de documentación destinado a explicar una decisión o procedimiento.
138	productos/coca-cola.png	Texto de documentación destinado a explicar una decisión o procedimiento.
139	```	Abre o cierra un ejemplo de código con formato monoespaciado.
140		Separación entre párrafos del documento.
141	No se recomienda guardar una ruta absoluta como	Texto de documentación destinado a explicar una decisión o procedimiento.
142	`C:\Users\Humberto\Desktop\...` porque dejaría de funcionar en otra	Texto de documentación destinado a explicar una decisión o procedimiento.
143	computadora.	Texto de documentación destinado a explicar una decisión o procedimiento.
144		Separación entre párrafos del documento.
145	Una estructura apropiada es:	Texto de documentación destinado a explicar una decisión o procedimiento.
146		Separación entre párrafos del documento.
147	```text	Abre o cierra un ejemplo de código con formato monoespaciado.
148	Waldonalds/	Texto de documentación destinado a explicar una decisión o procedimiento.
149	datos/	Texto de documentación destinado a explicar una decisión o procedimiento.
150	■ imagenes/	Texto de documentación destinado a explicar una decisión o procedimiento.
151	■ productos/	Texto de documentación destinado a explicar una decisión o procedimiento.
152	■ big-wald.png	Texto de documentación destinado a explicar una decisión o procedimiento.
153	■ papas.png	Texto de documentación destinado a explicar una decisión o procedimiento.
154	Waldonalds.jar	Texto de documentación destinado a explicar una decisión o procedimiento.
155	```	Abre o cierra un ejemplo de código con formato monoespaciado.
156		Separación entre párrafos del documento.
157	### Valor devuelto por el modelo	Título o subtítulo Markdown que organiza la documentación.
158		Separación entre párrafos del documento.
159	La primera columna debe devolver una `CeldaImagenTexto`:	Texto de documentación destinado a explicar una decisión o procedimiento.
160		Separación entre párrafos del documento.
161	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
162	case 0 -> new TablaAdministrativa.CeldaImagenTexto(	Texto de documentación destinado a explicar una decisión o procedimiento.
163	producto.getNombre(),	Texto de documentación destinado a explicar una decisión o procedimiento.
164	producto.getDescripcion(),	Texto de documentación destinado a explicar una decisión o procedimiento.
165	producto.getRutaImagen()	Texto de documentación destinado a explicar una decisión o procedimiento.
166	);	Texto de documentación destinado a explicar una decisión o procedimiento.
167	```	Abre o cierra un ejemplo de código con formato monoespaciado.
168		Separación entre párrafos del documento.
169	También debe declarar la clase de esa columna:	Texto de documentación destinado a explicar una decisión o procedimiento.
170		Separación entre párrafos del documento.

171	``java	Abre o cierra un ejemplo de código con formato monoespaciado.
172	@Override	Texto de documentación destinado a explicar una decisión o procedimiento.
173	public Class<?> getColumnClass(int columna) {	Texto de documentación destinado a explicar una decisión o procedimiento.
174	if (columna == 0) {	Texto de documentación destinado a explicar una decisión o procedimiento.
175	return TablaAdministrativa.CeldaImagenTexto.class;	Texto de documentación destinado a explicar una decisión o procedimiento.
176	}	Texto de documentación destinado a explicar una decisión o procedimiento.
177	return Object.class;	Texto de documentación destinado a explicar una decisión o procedimiento.
178	}	Texto de documentación destinado a explicar una decisión o procedimiento.
179	``	Abre o cierra un ejemplo de código con formato monoespaciado.
180		Separación entre párrafos del documento.
181	Después de colocar el modelo, se configura la columna:	Texto de documentación destinado a explicar una decisión o procedimiento.
182		Separación entre párrafos del documento.
183	``java	Abre o cierra un ejemplo de código con formato monoespaciado.
184	Path carpetaImágenes = Paths.get("datos", "imagenes");	Texto de documentación destinado a explicar una decisión o procedimiento.
185		Separación entre párrafos del documento.
186	tablaProductos.configurarColumnaImagenTexto(	Texto de documentación destinado a explicar una decisión o procedimiento.
187	0, // Columna Producto	Texto de documentación destinado a explicar una decisión o procedimiento.
188	carpetaImágenes, // Carpeta base permitida	Texto de documentación destinado a explicar una decisión o procedimiento.
189	48, // Ancho de imagen	Texto de documentación destinado a explicar una decisión o procedimiento.
190	48 // Alto de imagen	Texto de documentación destinado a explicar una decisión o procedimiento.
191	);	Texto de documentación destinado a explicar una decisión o procedimiento.
192	``	Abre o cierra un ejemplo de código con formato monoespaciado.
193		Separación entre párrafos del documento.
194	El componente:	Texto de documentación destinado a explicar una decisión o procedimiento.
195		Separación entre párrafos del documento.
196	- Une la carpeta base con la ruta obtenida de MySQL.	Elemento de una lista explicativa.
197	- Comprueba que el archivo exista.	Elemento de una lista explicativa.
198	- Impide salir de la carpeta permitida mediante `..`.	Elemento de una lista explicativa.
199	- Escala la imagen.	Elemento de una lista explicativa.
200	- Conserva las imágenes en caché para no leerlas en cada repintado.	Elemento de una lista explicativa.
201	- Ajusta la altura de fila cuando sea necesario.	Elemento de una lista explicativa.
202		Separación entre párrafos del documento.
203	Importaciones:	Texto de documentación destinado a explicar una decisión o procedimiento.
204		Separación entre párrafos del documento.
205	``java	Abre o cierra un ejemplo de código con formato monoespaciado.
206	import Componentes.TablaAdministrativa;	Texto de documentación destinado a explicar una decisión o procedimiento.
207	import java.nio.file.Path;	Texto de documentación destinado a explicar una decisión o procedimiento.
208	import java.nio.file.Paths;	Texto de documentación destinado a explicar una decisión o procedimiento.
209	``	Abre o cierra un ejemplo de código con formato monoespaciado.
210		Separación entre párrafos del documento.
211	## 6. Mostrar estrellas	Título o subtítulo Markdown que organiza la documentación.
212		Separación entre párrafos del documento.
213	Las estrellas deben representar una valoración real, por ejemplo el promedio	Texto de documentación destinado a explicar una decisión o procedimiento.
214	de calificaciones de clientes entre 0 y 5.	Texto de documentación destinado a explicar una decisión o procedimiento.

215		Separación entre párrafos del documento.
216	En la base de datos se guarda el número, no los símbolos:	Texto de documentación destinado a explicar una decisión o procedimiento.
217		Separación entre párrafos del documento.
218	```sql	Abre o cierra un ejemplo de código con formato monoespaciado.
219	popularidad DECIMAL(2,1)	Texto de documentación destinado a explicar una decisión o procedimiento.
220	```	Abre o cierra un ejemplo de código con formato monoespaciado.
221		Separación entre párrafos del documento.
222	El modelo devuelve:	Texto de documentación destinado a explicar una decisión o procedimiento.
223		Separación entre párrafos del documento.
224	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
225	case 6 -> new TablaAdministrativa.CeldaEstrellas(	Texto de documentación destinado a explicar una decisión o procedimiento.
226	producto.getPopularidad(),	Texto de documentación destinado a explicar una decisión o procedimiento.
227	5	Texto de documentación destinado a explicar una decisión o procedimiento.
228	);	Texto de documentación destinado a explicar una decisión o procedimiento.
229	```	Abre o cierra un ejemplo de código con formato monoespaciado.
230		Separación entre párrafos del documento.
231	Y declara la clase:	Texto de documentación destinado a explicar una decisión o procedimiento.
232		Separación entre párrafos del documento.
233	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
234	if (columna == 6) {	Texto de documentación destinado a explicar una decisión o procedimiento.
235	return TablaAdministrativa.CeldaEstrellas.class;	Texto de documentación destinado a explicar una decisión o procedimiento.
236	}	Texto de documentación destinado a explicar una decisión o procedimiento.
237	```	Abre o cierra un ejemplo de código con formato monoespaciado.
238		Separación entre párrafos del documento.
239	Después de asignar el modelo:	Texto de documentación destinado a explicar una decisión o procedimiento.
240		Separación entre párrafos del documento.
241	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
242	tablaProductos.configurarColumnaEstrellas(	Texto de documentación destinado a explicar una decisión o procedimiento.
243	6,	Texto de documentación destinado a explicar una decisión o procedimiento.
244	new Color(255, 184, 0)	Texto de documentación destinado a explicar una decisión o procedimiento.
245	);	Texto de documentación destinado a explicar una decisión o procedimiento.
246	```	Abre o cierra un ejemplo de código con formato monoespaciado.
247		Separación entre párrafos del documento.
248	La celda muestra estrellas llenas y vacías. El valor numérico continúa	Texto de documentación destinado a explicar una decisión o procedimiento.
249	disponible en un tooltip.	Texto de documentación destinado a explicar una decisión o procedimiento.
250		Separación entre párrafos del documento.
251	## 7. Mostrar porcentajes	Título o subtítulo Markdown que organiza la documentación.
252		Separación entre párrafos del documento.
253	Para progreso, disponibilidad, ventas alcanzadas o popularidad estadística,	Texto de documentación destinado a explicar una decisión o procedimiento.
254	un porcentaje suele comunicar mejor que estrellas.	Texto de documentación destinado a explicar una decisión o procedimiento.
255		Separación entre párrafos del documento.
256	El modelo devuelve:	Texto de documentación destinado a explicar una decisión o procedimiento.
257		Separación entre párrafos del documento.
258	```java	Abre o cierra un ejemplo de código con formato monoespaciado.

259	case 6 -> new TablaAdministrativa.CeldaPorcentaje(	Texto de documentación destinado a explicar una decisión o procedimiento.
260	producto.getPorcentajePopularidad()	Texto de documentación destinado a explicar una decisión o procedimiento.
261	);	Texto de documentación destinado a explicar una decisión o procedimiento.
262	```	Abre o cierra un ejemplo de código con formato monoespaciado.
263		Separación entre párrafos del documento.
264	Se configura así:	Texto de documentación destinado a explicar una decisión o procedimiento.
265		Separación entre párrafos del documento.
266	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
267	tablaProductos.configurarColumnaPorcentaje(	Texto de documentación destinado a explicar una decisión o procedimiento.
268	6,	Texto de documentación destinado a explicar una decisión o procedimiento.
269	new Color(255, 184, 0)	Texto de documentación destinado a explicar una decisión o procedimiento.
270	);	Texto de documentación destinado a explicar una decisión o procedimiento.
271	```	Abre o cierra un ejemplo de código con formato monoespaciado.
272		Separación entre párrafos del documento.
273	El valor debe estar entre 0 y 100. El componente lo limita automáticamente	Texto de documentación destinado a explicar una decisión o procedimiento.
274	si accidentalmente llega un número fuera de ese rango.	Texto de documentación destinado a explicar una decisión o procedimiento.
275		Separación entre párrafos del documento.
276	## 8. ¿Estrellas o porcentaje?	Título o subtítulo Markdown que organiza la documentación.
277		Separación entre párrafos del documento.
278	Usar estrellas cuando:	Texto de documentación destinado a explicar una decisión o procedimiento.
279		Separación entre párrafos del documento.
280	- Los clientes califican el producto.	Elemento de una lista explicativa.
281	- Existe un promedio de valoraciones.	Elemento de una lista explicativa.
282	- El significado es calidad percibida.	Elemento de una lista explicativa.
283		Separación entre párrafos del documento.
284	Usar porcentaje cuando:	Texto de documentación destinado a explicar una decisión o procedimiento.
285		Separación entre párrafos del documento.
286	- Se mide popularidad por ventas.	Elemento de una lista explicativa.
287	- Se compara contra una meta.	Elemento de una lista explicativa.
288	- Se muestra disponibilidad o avance.	Elemento de una lista explicativa.
289	- El dato proviene de una fórmula estadística.	Elemento de una lista explicativa.
290		Separación entre párrafos del documento.
291	No conviene mostrar estrellas calculadas únicamente por cantidad de ventas,	Texto de documentación destinado a explicar una decisión o procedimiento.
292	porque el usuario normalmente interpreta las estrellas como calificaciones.	Texto de documentación destinado a explicar una decisión o procedimiento.
293	Para ventas puede mostrarse `82%`, una barra o el número de unidades.	Texto de documentación destinado a explicar una decisión o procedimiento.
294		Separación entre párrafos del documento.
295	## 9. Columnas con botones	Título o subtítulo Markdown que organiza la documentación.
296		Separación entre párrafos del documento.
297	Los botones Editar, Eliminar y el interruptor necesitan un	Texto de documentación destinado a explicar una decisión o procedimiento.
298	`TableCellRenderer` para verse y un `TableCellEditor` para recibir	Texto de documentación destinado a explicar una decisión o procedimiento.
299	clics. Esos componentes permanecen en cada módulo porque las acciones cambian	Texto de documentación destinado a explicar una decisión o procedimiento.
300	entre usuarios, productos, ingredientes y pedidos.	Texto de documentación destinado a explicar una decisión o procedimiento.
301		Separación entre párrafos del documento.
302	La tabla reutilizable se encarga del diseño general; cada pantalla conserva	Texto de documentación destinado a explicar una decisión o procedimiento.

303	sus reglas de negocio.	Texto de documentación destinado a explicar una decisión o procedimiento.
304		Separación entre párrafos del documento.
305	## 10. Orden recomendado al configurar	Título o subtítulo Markdown que organiza la documentación.
306		Separación entre párrafos del documento.
307	```java	Abre o cierra un ejemplo de código con formato monoespaciado.
308	// 1. Colocar los datos.	Texto de documentación destinado a explicar una decisión o procedimiento.
309	tablaProductos.setModel(modeloProductos);	Texto de documentación destinado a explicar una decisión o procedimiento.
310		Separación entre párrafos del documento.
311	// 2. Reaplicar el diseño del componente.	Texto de documentación destinado a explicar una decisión o procedimiento.
312	tablaProductos.aplicarEstilo();	Texto de documentación destinado a explicar una decisión o procedimiento.
313		Separación entre párrafos del documento.
314	// 3. Configurar columnas especiales.	Texto de documentación destinado a explicar una decisión o procedimiento.
315	tablaProductos.configurarColumnaImagenTexto(	Texto de documentación destinado a explicar una decisión o procedimiento.
316	0, Paths.get("datos", "imagenes"), 48, 48);	Texto de documentación destinado a explicar una decisión o procedimiento.
317	tablaProductos.configurarColumnaEstrellas(	Texto de documentación destinado a explicar una decisión o procedimiento.
318	6, new Color(255, 184, 0));	Texto de documentación destinado a explicar una decisión o procedimiento.
319		Separación entre párrafos del documento.
320	// 4. Ajustar anchos.	Texto de documentación destinado a explicar una decisión o procedimiento.
321	tablaProductos.getColumnModel()	Texto de documentación destinado a explicar una decisión o procedimiento.
322	.getColumn(0).setPreferredWidth(300);	Texto de documentación destinado a explicar una decisión o procedimiento.
323	```	Abre o cierra un ejemplo de código con formato monoespaciado.
324		Separación entre párrafos del documento.
325	Si se vuelve a cambiar el modelo, deben configurarse nuevamente las columnas	Texto de documentación destinado a explicar una decisión o procedimiento.
326	especiales porque Swing crea un nuevo `TableColumnModel`.	Texto de documentación destinado a explicar una decisión o procedimiento.

16.10 nbproject/build-impl.xml

Configuración XML utilizada por NetBeans/Ant para describir y construir el proyecto.

Tipo: .xml | Líneas: 1796 | Archivo 10 de 157

1	<?xml version="1.0" encoding="UTF-8"?>	Declara un elemento XML de configuración, con sus atributos si corresponde.
2	<!--	Comentario XML para humanos o para NetBeans.
3	*** GENERATED FROM project.xml - DO NOT EDIT ***	Contenido de configuración o documentación que completa el archivo.
4	*** EDIT ../build.xml INSTEAD ***	Contenido de configuración o documentación que completa el archivo.
5		Separación visual dentro del XML.
6	For the purpose of easier reading the script	Contenido de configuración o documentación que completa el archivo.
7	is divided into following sections:	Contenido de configuración o documentación que completa el archivo.
8		Separación visual dentro del XML.
9	- initialization	Contenido de configuración o documentación que completa el archivo.
10	- compilation	Contenido de configuración o documentación que completa el archivo.
11	- jar	Contenido de configuración o documentación que completa el archivo.
12	- execution	Contenido de configuración o documentación que completa el archivo.
13	- debugging	Contenido de configuración o documentación que completa el archivo.
14	- javadoc	Contenido de configuración o documentación que completa el archivo.
15	- test compilation	Contenido de configuración o documentación que completa el archivo.
16	- test execution	Contenido de configuración o documentación que completa el archivo.
17	- test debugging	Contenido de configuración o documentación que completa el archivo.
18	- applet	Contenido de configuración o documentación que completa el archivo.
19	- cleanup	Contenido de configuración o documentación que completa el archivo.
20		Separación visual dentro del XML.
21	-->	Comentario XML para humanos o para NetBeans.
22	<project xmlns:if="ant:if" xmlns:j2seproject1="http://www.netbeans.org/ns/j2se-project/1" xmlns:j2seproject3="http://www.netbeans.org/ns/j2se-project/3" xmlns:jaxrpc="http://www.netbeans.org/ns/j2se-project/jax-rpc" xmlns:unless="ant:unless" basedir=".." default="default" name="Waldonalds-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
23	<fail message="Please build using Ant 1.8.0 or higher.">	Declara un elemento XML de configuración, con sus atributos si corresponde.
24	<condition>	Declara un elemento XML de configuración, con sus atributos si corresponde.
25	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
26	<antversion atleast="1.8.0"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
27	</not>	Cierra el elemento XML abierto anteriormente.
28	</condition>	Cierra el elemento XML abierto anteriormente.
29	</fail>	Cierra el elemento XML abierto anteriormente.
30	<target depends="test,jar,javadoc" description="Build and test whole project." name="default"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
31	<!--	Comentario XML para humanos o para NetBeans.
32	=====	Contenido de configuración o documentación que completa el archivo.
33	INITIALIZATION SECTION	Contenido de configuración o documentación que completa el archivo.
34	=====	Contenido de configuración o documentación que completa el archivo.
35	-->	Comentario XML para humanos o para NetBeans.
36	<target name="-pre-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
37	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.

38	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
39	</target>	Cierra el elemento XML abierto anteriormente.
40	<target depends="-pre-init" name="-init-private">	Declara un elemento XML de configuración, con sus atributos si corresponde.
41	<property file="nbproject/private/config.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
42	<property file="nbproject/private/configs/\${config}.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
43	<property file="nbproject/private/private.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
44	</target>	Cierra el elemento XML abierto anteriormente.
45	<target name="-pre-init-libraries">	Declara un elemento XML de configuración, con sus atributos si corresponde.
46	<property location=".\\lib\\nblibraries.properties" name="libraries.path"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
47	<dirname file="\${libraries.path}" property="libraries.dir.nativedirsep"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
48	<pathconvert dirsep="/" property="libraries.dir">	Declara un elemento XML de configuración, con sus atributos si corresponde.
49	<path path="\${libraries.dir.nativedirsep}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
50	</pathconvert>	Cierra el elemento XML abierto anteriormente.
51	<basename file="\${libraries.path}" property="libraries.basename" suffix=".properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
52	<available file="\${libraries.dir}/\${libraries.basename}-private.properties" property="private.properties.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
53	</target>	Cierra el elemento XML abierto anteriormente.
54	<target depends="-pre-init-libraries" if="private.properties.available" name="-init-private-libraries">	Declara un elemento XML de configuración, con sus atributos si corresponde.
55	<loadproperties encoding="ISO-8859-1" srcfile="\${libraries.dir}/\${libraries.basename}-private.properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
56	<filterchain>	Declara un elemento XML de configuración, con sus atributos si corresponde.
57	<replacestring from="\${base}" to="\${libraries.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
58	<escapeunicode/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
59	</filterchain>	Cierra el elemento XML abierto anteriormente.
60	</loadproperties>	Cierra el elemento XML abierto anteriormente.
61	</target>	Cierra el elemento XML abierto anteriormente.
62	<target depends="-pre-init,-init-private,-init-private-libraries" name="-init-libraries">	Declara un elemento XML de configuración, con sus atributos si corresponde.
63	<loadproperties encoding="ISO-8859-1" srcfile="\${libraries.path}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
64	<filterchain>	Declara un elemento XML de configuración, con sus atributos si corresponde.
65	<replacestring from="\${base}" to="\${libraries.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
66	<escapeunicode/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
67	</filterchain>	Cierra el elemento XML abierto anteriormente.
68	</loadproperties>	Cierra el elemento XML abierto anteriormente.
69	</target>	Cierra el elemento XML abierto anteriormente.
70	<target depends="-pre-init,-init-private,-init-libraries" name="-init-user">	Declara un elemento XML de configuración, con sus atributos si corresponde.
71	<property file="\${user.properties.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
72	<!-- The two properties below are usually overridden -->	Comentario XML para humanos o para NetBeans.
73	<!-- by the active platform. Just a fallback. -->	Comentario XML para humanos o para NetBeans.
74	<property name="default.javac.source" value="1.8"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
75	<property name="default.javac.target" value="1.8"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
76	</target>	Cierra el elemento XML abierto anteriormente.
77	<target depends="-pre-init,-init-private,-init-libraries,-init-user" name="-init-project">	Declara un elemento XML de configuración, con sus atributos si corresponde.
78	<property file="nbproject/configs/\${config}.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
79	<property file="nbproject/project.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
80	</target>	Cierra el elemento XML abierto anteriormente.

81	<target name="-init-modules-supported">	Declara un elemento XML de configuración, con sus atributos si corresponde.
82	<condition property="modules.supported.internal" value="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
83	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
84	<matches pattern="1\.[0-8](\..*)?" string="\${javac.source}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
85	</not>	Cierra el elemento XML abierto anteriormente.
86	</condition>	Cierra el elemento XML abierto anteriormente.
87	</target>	Cierra el elemento XML abierto anteriormente.
88	<target depends="-init-modules-supported" if="modules.supported.internal" name="-init-macrodef-modulename">	Declara un elemento XML de configuración, con sus atributos si corresponde.
89	<macrodef name="modulename" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
90	<attribute name="property"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
91	<attribute name="sourcepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
92	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
93	<loadresource property="@{property}" quiet="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
94	<javaresource classpath="@{sourcepath}" name="module-info.java" parentFirst="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
95	<filterchain>	Declara un elemento XML de configuración, con sus atributos si corresponde.
96	<stripjavacomments/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
97	<linecontainsregexp>	Declara un elemento XML de configuración, con sus atributos si corresponde.
98	<regexp pattern="module .* \{">	Declara un elemento XML de configuración, con sus atributos si corresponde.
99	</linecontainsregexp>	Cierra el elemento XML abierto anteriormente.
100	<tokenfilter>	Declara un elemento XML de configuración, con sus atributos si corresponde.
101	<linetokenizer/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
102	<replaceregex flags="s" pattern="(\s*module\s+)(\s*)(\s*\{.*)" replace="\2"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
103	</tokenfilter>	Cierra el elemento XML abierto anteriormente.
104	<striplinebreaks/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
105	</filterchain>	Cierra el elemento XML abierto anteriormente.
106	</loadresource>	Cierra el elemento XML abierto anteriormente.
107	</sequential>	Cierra el elemento XML abierto anteriormente.
108	</macrodef>	Cierra el elemento XML abierto anteriormente.
109	</target>	Cierra el elemento XML abierto anteriormente.
110	<target depends="-init-modules-supported, -init-macrodef-modulename" if="modules.supported.internal" name="-init-source-module-properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
111	<fail message="Java 9 support requires Ant 1.10.0 or higher.">	Declara un elemento XML de configuración, con sus atributos si corresponde.
112	<condition>	Declara un elemento XML de configuración, con sus atributos si corresponde.
113	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
114	<antversion atleast="1.10.0"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
115	</not>	Cierra el elemento XML abierto anteriormente.
116	</condition>	Cierra el elemento XML abierto anteriormente.
117	</fail>	Cierra el elemento XML abierto anteriormente.
118	<j2seproject3:modulename property="module.name" sourcepath="\${src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
119	<condition property="named.module.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
120	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
121	<isset property="module.name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
122	<length length="0" string="\${module.name}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
123	</and>	Cierra el elemento XML abierto anteriormente.
124	</condition>	Cierra el elemento XML abierto anteriormente.

125	<condition property="unnamed.module.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
126	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
127	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
128	</not>	Cierra el elemento XML abierto anteriormente.
129	</condition>	Cierra el elemento XML abierto anteriormente.
130	<property name="javac.modulepath" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
131	<property name="run.modulepath" value="\${javac.modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
132	<property name="module.build.classes.dir" value="\${build.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
133	<property name="debug.modulepath" value="\${run.modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
134	<property name="javac.upgrademodulepath" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
135	<property name="run.upgrademodulepath" value="\${javac.upgrademodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
136	<condition else="" property="javac.systemmodulepath.cmd.line.arg" value="--system '\${javac.systemmodulep ath}'">	Declara un elemento XML de configuración, con sus atributos si corresponde.
137	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
138	<isset property="javac.systemmodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
139	<length length="0" string="\${javac.systemmodulepath}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
140	</and>	Cierra el elemento XML abierto anteriormente.
141	</condition>	Cierra el elemento XML abierto anteriormente.
142	<property name="dist.jlink.dir" value="\${dist.dir}/jlink"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
143	<property name="dist.jlink.output" value="\${dist.jlink.dir}/\${application.title}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
144	<property name="module.name" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
145	</target>	Cierra el elemento XML abierto anteriormente.
146	<target depends="-pre-init,-init-private,-init-libraries,-init-user,-init-project,-init-macrodef-property,-i nit-modules-supported" name="-do-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
147	<property name="platform.java" value="\${java.home}/bin/java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
148	<available file="\${manifest.file}" property="manifest.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
149	<condition property="splashscreen.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
150	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
151	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
152	<equals arg1="\${application.splash}" arg2="" trim="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
153	</not>	Cierra el elemento XML abierto anteriormente.
154	<available file="\${application.splash}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
155	</and>	Cierra el elemento XML abierto anteriormente.
156	</condition>	Cierra el elemento XML abierto anteriormente.
157	<condition property="main.class.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
158	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
159	<isset property="main.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
160	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
161	<equals arg1="\${main.class}" arg2="" trim="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
162	</not>	Cierra el elemento XML abierto anteriormente.
163	</and>	Cierra el elemento XML abierto anteriormente.
164	</condition>	Cierra el elemento XML abierto anteriormente.
165	<condition property="profile.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
166	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
167	<isset property="javac.profile"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

168	<length length="0" string="{javac.profile}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
169	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
170	<matches pattern="1\.[0-7](\..*)?" string="{javac.source}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
171	</not>	Cierra el elemento XML abierto anteriormente.
172	</and>	Cierra el elemento XML abierto anteriormente.
173	</condition>	Cierra el elemento XML abierto anteriormente.
174	<condition property="do.archive">	Declara un elemento XML de configuración, con sus atributos si corresponde.
175	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
176	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
177	<istruer value="{jar.archive.disabled}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
178	</not>	Cierra el elemento XML abierto anteriormente.
179	<istruer value="{not.archive.disabled}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
180	</or>	Cierra el elemento XML abierto anteriormente.
181	</condition>	Cierra el elemento XML abierto anteriormente.
182	<condition property="do.archive+manifest.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
183	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
184	<isset property="manifest.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
185	<istruer value="{do.archive}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
186	</and>	Cierra el elemento XML abierto anteriormente.
187	</condition>	Cierra el elemento XML abierto anteriormente.
188	<condition property="do.archive+main.class.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
189	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
190	<isset property="main.class.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
191	<istruer value="{do.archive}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
192	</and>	Cierra el elemento XML abierto anteriormente.
193	</condition>	Cierra el elemento XML abierto anteriormente.
194	<condition property="do.archive+splashscreen.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
195	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
196	<isset property="splashscreen.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
197	<istruer value="{do.archive}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
198	</and>	Cierra el elemento XML abierto anteriormente.
199	</condition>	Cierra el elemento XML abierto anteriormente.
200	<condition property="do.archive+profile.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
201	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
202	<isset property="profile.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
203	<istruer value="{do.archive}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
204	</and>	Cierra el elemento XML abierto anteriormente.
205	</condition>	Cierra el elemento XML abierto anteriormente.
206	<condition property="have.tests">	Declara un elemento XML de configuración, con sus atributos si corresponde.
207	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
208	<available file="{test.src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
209	</or>	Cierra el elemento XML abierto anteriormente.
210	</condition>	Cierra el elemento XML abierto anteriormente.
211	<condition property="have.sources">	Declara un elemento XML de configuración, con sus atributos si corresponde.

212	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
213	<available file="\${src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
214	</or>	Cierra el elemento XML abierto anteriormente.
215	</condition>	Cierra el elemento XML abierto anteriormente.
216	<condition property="netbeans.home+have.tests">	Declara un elemento XML de configuración, con sus atributos si corresponde.
217	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
218	<isset property="netbeans.home"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
219	<isset property="have.tests"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
220	</and>	Cierra el elemento XML abierto anteriormente.
221	</condition>	Cierra el elemento XML abierto anteriormente.
222	<condition property="no.javadoc.preview">	Declara un elemento XML de configuración, con sus atributos si corresponde.
223	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
224	<isset property="javadoc.preview"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
225	<isfalse value="{javadoc.preview}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
226	</and>	Cierra el elemento XML abierto anteriormente.
227	</condition>	Cierra el elemento XML abierto anteriormente.
228	<property name="run.jvmargs" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
229	<property name="run.jvmargs.ide" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
230	<property name="javac.compilerargs" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
231	<property name="work.dir" value="{basedir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
232	<condition property="no.deps">	Declara un elemento XML de configuración, con sus atributos si corresponde.
233	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
234	<istrue value="{no.dependencies}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
235	</and>	Cierra el elemento XML abierto anteriormente.
236	</condition>	Cierra el elemento XML abierto anteriormente.
237	<property name="javac.debug" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
238	<property name="javadoc.preview" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
239	<property name="application.args" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
240	<property name="source.encoding" value="{file.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
241	<property name="runtime.encoding" value="{source.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
242	<property name="manifest.encoding" value="{source.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
243	<condition property="javadoc.encoding.used" value="{javadoc.encoding}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
244	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
245	<isset property="javadoc.encoding"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
246	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
247	<equals arg1="{javadoc.encoding}" arg2=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
248	</not>	Cierra el elemento XML abierto anteriormente.
249	</and>	Cierra el elemento XML abierto anteriormente.
250	</condition>	Cierra el elemento XML abierto anteriormente.
251	<property name="javadoc.encoding.used" value="{source.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
252	<property name="includes" value="*/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
253	<property name="excludes" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
254	<property name="do.depend" value="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
255	<condition property="do.depend.true">	Declara un elemento XML de configuración, con sus atributos si corresponde.

256	<istruer value="\${do.depend}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
257	</condition>	Cierra el elemento XML abierto anteriormente.
258	<path id="endorsed.classpath.path" path="\${endorsed.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
259	<condition else="" property="endorsed.classpath.cmd.line.arg" value="-Xbootclasspath/p: '\${toString:endorsed.classpath.path}'">	Declara un elemento XML de configuración, con sus atributos si corresponde.
260	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
261	<isset property="endorsed.classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
262	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
263	<equals arg1="\${endorsed.classpath}" arg2="" trim="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
264	</not>	Cierra el elemento XML abierto anteriormente.
265	</and>	Cierra el elemento XML abierto anteriormente.
266	</condition>	Cierra el elemento XML abierto anteriormente.
267	<condition else="" property="javac.profile.cmd.line.arg" value="-profile \${javac.profile}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
268	<isset property="profile.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
269	</condition>	Cierra el elemento XML abierto anteriormente.
270	<condition else="false" property="jdkBug6558476">	Declara un elemento XML de configuración, con sus atributos si corresponde.
271	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
272	<matches pattern="1\.[56]" string="\${java.specification.version}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
273	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
274	<os family="unix"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
275	</not>	Cierra el elemento XML abierto anteriormente.
276	</and>	Cierra el elemento XML abierto anteriormente.
277	</condition>	Cierra el elemento XML abierto anteriormente.
278	<condition else="false" property="javac.fork">	Declara un elemento XML de configuración, con sus atributos si corresponde.
279	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
280	<istruer value="\${jdkBug6558476}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
281	<istruer value="\${javac.external.vm}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
282	</or>	Cierra el elemento XML abierto anteriormente.
283	</condition>	Cierra el elemento XML abierto anteriormente.
284	<property name="jar.index" value="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
285	<property name="jar.index.metainf" value="\${jar.index}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
286	<property name="copylibs.rebase" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
287	<available file="\${meta.inf.dir}/persistence.xml" property="has.persistence.xml"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
288	<condition property="junit.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
289	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
290	<available classname="org.junit.Test" classpath="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
291	<available classname="junit.framework.Test" classpath="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
292	</or>	Cierra el elemento XML abierto anteriormente.
293	</condition>	Cierra el elemento XML abierto anteriormente.
294	<condition property="testng.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
295	<available classname="org.testng.annotations.Test" classpath="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
296	</condition>	Cierra el elemento XML abierto anteriormente.
297	<condition property="junit+testng.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.
298	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
299	<istruer value="\${junit.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

300	<istruer value="\${testng.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
301	</and>	Cierra el elemento XML abierto anteriormente.
302	</condition>	Cierra el elemento XML abierto anteriormente.
303	<condition else="testng" property="testng.mode" value="mixed">	Declara un elemento XML de configuración, con sus atributos si corresponde.
304	<istruer value="\${junit+testng.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
305	</condition>	Cierra el elemento XML abierto anteriormente.
306	<condition else="" property="testng.debug.mode" value="-mixed">	Declara un elemento XML de configuración, con sus atributos si corresponde.
307	<istruer value="\${junit+testng.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
308	</condition>	Cierra el elemento XML abierto anteriormente.
309	<property name="java.failonerror" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
310	</target>	Cierra el elemento XML abierto anteriormente.
311	<target name="-post-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
312	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
313	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
314	</target>	Cierra el elemento XML abierto anteriormente.
315	<target depends="-pre-init,-init-private,-init-libraries,-init-user,-init-project,-do-init" name="-init-check">	Declara un elemento XML de configuración, con sus atributos si corresponde.
316	<fail unless="src.dir">Must set src.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
317	<fail unless="test.src.dir">Must set test.src.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
318	<fail unless="build.dir">Must set build.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
319	<fail unless="dist.dir">Must set dist.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
320	<fail unless="build.classes.dir">Must set build.classes.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
321	<fail unless="dist.javadoc.dir">Must set dist.javadoc.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
322	<fail unless="build.test.classes.dir">Must set build.test.classes.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
323	<fail unless="build.test.results.dir">Must set build.test.results.dir</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
324	<fail unless="build.classes.excludes">Must set build.classes.excludes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
325	<fail unless="dist.jar">Must set dist.jar</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
326	</target>	Cierra el elemento XML abierto anteriormente.
327	<target name="-init-macrodef-property">	Declara un elemento XML de configuración, con sus atributos si corresponde.
328	<macrodef name="property" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
329	<attribute name="name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
330	<attribute name="value"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
331	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
332	<property name="@{name}" value="\${@{value}}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
333	</sequential>	Cierra el elemento XML abierto anteriormente.
334	</macrodef>	Cierra el elemento XML abierto anteriormente.
335	</target>	Cierra el elemento XML abierto anteriormente.
336	<target depends="-init-ap-cmdline-properties,-init-source-module-properties" if="modules.supported.internal" name="-init-macrodef-javac-with-module">	Declara un elemento XML de configuración, con sus atributos si corresponde.
337	<macrodef name="javac" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
338	<attribute default="\${src.dir}" name="srcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
339	<attribute default="\${build.classes.dir}" name="destdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
340	<attribute default="\${javac.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
341	<attribute default="\${javac.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
342	<attribute default="\${javac.upgrademodulepath}" name="upgrademodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

343	<attribute default="{javac.processorpath}" name="processorpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
344	<attribute default="{javac.processormodulepath}" name="processormodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
345	<attribute default="{build.generated.sources.dir}/ap-source-output" name="apgeneratedsrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
346	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
347	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
348	<attribute default="{javac.debug}" name="debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
349	<attribute default="{empty.dir}" name="sourcepath" unless:set="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
350	<attribute default="{src.dir}" if:set="named.module.internal" name="sourcepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
351	<attribute default="{empty.dir}" name="gensrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
352	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
353	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
354	<condition property="warn.excludes.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
355	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
356	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
357	<length length="0" string="@{excludes}" trim="true" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
358	</and>	Cierra el elemento XML abierto anteriormente.
359	</condition>	Cierra el elemento XML abierto anteriormente.
360	<echo if:set="warn.excludes.internal" level="warning" message="The javac excludes are not supported in the JDK 9 Named Module."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
361	<property location="{build.dir}/empty" name="empty.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
362	<mkdir dir="{empty.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
363	<mkdir dir="@{apgeneratedsrcdir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
364	<condition property="processormodulepath.set">	Declara un elemento XML de configuración, con sus atributos si corresponde.
365	<resourcecount count="0" when="greater">	Declara un elemento XML de configuración, con sus atributos si corresponde.
366	<path>	Declara un elemento XML de configuración, con sus atributos si corresponde.
367	<pathelement path="@{processormodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
368	</path>	Cierra el elemento XML abierto anteriormente.
369	</resourcecount>	Cierra el elemento XML abierto anteriormente.
370	</condition>	Cierra el elemento XML abierto anteriormente.
371	<javac debug="@{debug}" deprecation="{javac.deprecation}" destdir="@{destdir}" encoding="{source.encoding}" excludes="@{excludes}" fork="{javac.fork}" includeantruntime="false" includes="@{includes}" source="{javac.source}" sourcepath="{sourcepath}" srcdir="@{srcdir}" target="{javac.target}" tempdir="{java.io.tmpdir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
372	<src>	Declara un elemento XML de configuración, con sus atributos si corresponde.
373	<dirset dir="@{gensrcdir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
374	<include name="*/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
375	</dirset>	Cierra el elemento XML abierto anteriormente.
376	</src>	Cierra el elemento XML abierto anteriormente.
377	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
378	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
379	</classpath>	Cierra el elemento XML abierto anteriormente.
380	<modulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
381	<path path="@{modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
382	</modulepath>	Cierra el elemento XML abierto anteriormente.
383	<upgrademodulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
384	<path path="@{upgrademodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

385	</upgrademodulepath>	Cierra el elemento XML abierto anteriormente.
386	<compilerarg line="{javac.systemmodulepath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
387	<compilerarg line="{javac.profile.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
388	<compilerarg line="{javac.compilerargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
389	<compilerarg if:set="processormodulepath.set" value="--processor-module-path"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
390	<compilerarg if:set="processormodulepath.set" path="@{processormodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
391	<compilerarg unless:set="processormodulepath.set" value="-processorpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
392	<compilerarg path="@{processorpath}:{empty.dir}" unless:set="processormodulepath.set"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
393	<compilerarg line="{ap.processors.internal}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
394	<compilerarg line="{annotation.processing.processor.options}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
395	<compilerarg value="-s"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
396	<compilerarg path="@{apgeneratedsrcdir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
397	<compilerarg line="{ap.proc.none.internal}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
398	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
399	</javac>	Cierra el elemento XML abierto anteriormente.
400	</sequential>	Cierra el elemento XML abierto anteriormente.
401	</macrodef>	Cierra el elemento XML abierto anteriormente.
402	</target>	Cierra el elemento XML abierto anteriormente.
403	<target depends="-init-ap-cmdline-properties, -init-source-module-properties" if="ap.supported.internal" name="-init-macrodef-javac-with-processors" unless="modules.supported.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
404	<macrodef name="javac" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
405	<attribute default="{src.dir}" name="srcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
406	<attribute default="{build.classes.dir}" name="destdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
407	<attribute default="{javac.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
408	<attribute default="{javac.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
409	<attribute default="{javac.upgrademodulepath}" name="upgrademodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
410	<attribute default="{javac.processorpath}" name="processorpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
411	<attribute default="{build.generated.sources.dir}/ap-source-output" name="apgeneratedsrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
412	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
413	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
414	<attribute default="{javac.debug}" name="debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
415	<attribute default="{empty.dir}" name="sourcepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
416	<attribute default="{empty.dir}" name="gensrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
417	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
418	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
419	<property location="{build.dir}/empty" name="empty.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
420	<mkdir dir="{empty.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
421	<mkdir dir="@{apgeneratedsrcdir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
422	<javac debug="@{debug}" deprecation="{javac.deprecation}" destdir="@{destdir}" encoding="{source.encoding}" excludes="@{excludes}" fork="{javac.fork}" includeantruntime="false" includes="@{includes}" source="{javac.source}" sourcepath="@{sourcepath}" srcdir="@{srcdir}" target="{javac.target}" tempdir="{java.io.tmpdir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
423	<src>	Declara un elemento XML de configuración, con sus atributos si corresponde.
424	<dirset dir="@{gensrcdir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
425	<include name=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
426	</dirset>	Cierra el elemento XML abierto anteriormente.

427	</src>	Cierra el elemento XML abierto anteriormente.
428	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
429	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
430	</classpath>	Cierra el elemento XML abierto anteriormente.
431	<compilerarg line="\${endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
432	<compilerarg line="\${javac.profile.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
433	<compilerarg line="\${javac.compilerargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
434	<compilerarg value="-processorpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
435	<compilerarg path="@{processorpath}:{empty.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
436	<compilerarg line="\${ap.processors.internal}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
437	<compilerarg line="\${annotation.processing.processor.options}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
438	<compilerarg value="-s"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
439	<compilerarg path="@{apgeneratedsrcdir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
440	<compilerarg line="\${ap.proc.none.internal}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
441	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
442	</javac>	Cierra el elemento XML abierto anteriormente.
443	</sequential>	Cierra el elemento XML abierto anteriormente.
444	</macrodef>	Cierra el elemento XML abierto anteriormente.
445	</target>	Cierra el elemento XML abierto anteriormente.
446	<target depends="-init-ap-cmdline-properties, -init-source-module-properties" name="-init-macrodef-javac-with-out-processors" unless="ap.supported.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
447	<macrodef name="javac" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
448	<attribute default="\${src.dir}" name="srcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
449	<attribute default="\${build.classes.dir}" name="destdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
450	<attribute default="\${javac.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
451	<attribute default="\${javac.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
452	<attribute default="\${javac.upgrademodulepath}" name="upgrademodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
453	<attribute default="\${javac.processorpath}" name="processorpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
454	<attribute default="\${build.generated.sources.dir}/ap-source-output" name="apgeneratedsrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
455	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
456	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
457	<attribute default="{javac.debug}" name="debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
458	<attribute default="{empty.dir}" name="sourcepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
459	<attribute default="{empty.dir}" name="gensrcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
460	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
461	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
462	<property location="{build.dir}/empty" name="empty.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
463	<mkdir dir="{empty.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
464	<javac debug="@{debug}" deprecation="{javac.deprecation}" destdir="@{destdir}" encoding="{source.encoding}" excludes="@{excludes}" fork="{javac.fork}" includeantruntime="false" includes="@{includes}" source="{javac.source}" sourcepath="@{sourcepath}" srcdir="@{srcdir}" target="{javac.target}" tempdir="{java.io.tmpdir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
465	<src>	Declara un elemento XML de configuración, con sus atributos si corresponde.
466	<dirset dir="@{gensrcdir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
467	<include name="*/"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
468	</dirset>	Cierra el elemento XML abierto anteriormente.

469	</src>	Cierra el elemento XML abierto anteriormente.
470	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
471	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
472	</classpath>	Cierra el elemento XML abierto anteriormente.
473	<compilerarg line="\${endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
474	<compilerarg line="\${javac.profile.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
475	<compilerarg line="\${javac.compilerargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
476	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
477	</javac>	Cierra el elemento XML abierto anteriormente.
478	</sequential>	Cierra el elemento XML abierto anteriormente.
479	</macrodef>	Cierra el elemento XML abierto anteriormente.
480	</target>	Cierra el elemento XML abierto anteriormente.
481	<target depends="-init-macrodef-javac-with-module,-init-macrodef-javac-with-processors,-init-macrodef-javac-without-processors" name="-init-macrodef-javac">	Declara un elemento XML de configuración, con sus atributos si corresponde.
482	<macrodef name="depend" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
483	<attribute default="\${src.dir}" name="srcdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
484	<attribute default="\${build.classes.dir}" name="destdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
485	<attribute default="\${javac.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
486	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
487	<depend cache="\${build.dir}/depcache" destdir="@{destdir}" excludes="\${excludes}" includes="{includes}" srcdir="@{srcdir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
488	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
489	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
490	</classpath>	Cierra el elemento XML abierto anteriormente.
491	</depend>	Cierra el elemento XML abierto anteriormente.
492	</sequential>	Cierra el elemento XML abierto anteriormente.
493	</macrodef>	Cierra el elemento XML abierto anteriormente.
494	<macrodef name="force-recompile" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
495	<attribute default="\${build.classes.dir}" name="destdir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
496	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
497	<fail unless="javac.includes">Must set javac.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
498	<pathconvert pathsep="\${line.separator}" property="javac.includes.binary">	Declara un elemento XML de configuración, con sus atributos si corresponde.
499	<path>	Declara un elemento XML de configuración, con sus atributos si corresponde.
500	<filelist dir="@{destdir}" files="\${javac.includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
501	</path>	Cierra el elemento XML abierto anteriormente.
502	<globmapper from="*.java" to="*.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
503	</pathconvert>	Cierra el elemento XML abierto anteriormente.
504	<tempfile deleteonexit="true" property="javac.includesfile.binary"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
505	<echo file="\${javac.includesfile.binary}" message="\${javac.includes.binary}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
506	<delete>	Declara un elemento XML de configuración, con sus atributos si corresponde.
507	<files includesfile="\${javac.includesfile.binary}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
508	</delete>	Cierra el elemento XML abierto anteriormente.
509	<delete>	Declara un elemento XML de configuración, con sus atributos si corresponde.
510	<fileset file="\${javac.includesfile.binary}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
511	</delete>	Cierra el elemento XML abierto anteriormente.

512	</sequential>	Cierra el elemento XML abierto anteriormente.
513	</macrodef>	Cierra el elemento XML abierto anteriormente.
514	</target>	Cierra el elemento XML abierto anteriormente.
515	<target if="{junit.available}" name="-init-macrodef-junit-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
516	<condition else="false" property="nb.junit.batch" value="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
517	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
518	<istrue value="{junit.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
519	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
520	<isset property="test.method"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
521	</not>	Cierra el elemento XML abierto anteriormente.
522	</and>	Cierra el elemento XML abierto anteriormente.
523	</condition>	Cierra el elemento XML abierto anteriormente.
524	<condition else="false" property="nb.junit.single" value="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
525	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
526	<istrue value="{junit.available}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
527	<isset property="test.method"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
528	</and>	Cierra el elemento XML abierto anteriormente.
529	</condition>	Cierra el elemento XML abierto anteriormente.
530	</target>	Cierra el elemento XML abierto anteriormente.
531	<target name="-init-test-properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
532	<property name="test.binaryincludes" value="<nothing>"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
533	<property name="test.binarytestincludes" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
534	<property name="test.binaryexcludes" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
535	</target>	Cierra el elemento XML abierto anteriormente.
536	<target depends="-init-modules-supported" if="modules.supported.internal" name="-init-macrodef-junit-prototype-with-module">	Declara un elemento XML de configuración, con sus atributos si corresponde.
537	<macrodef name="junit-prototype" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
538	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
539	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
540	<element name="customizePrototype" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
541	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
542	<property name="junit.forkmode" value="perTest"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
543	<junit dir="{work.dir}" errorproperty="tests.failed" failureproperty="tests.failed" fork="true" forkmode="{junit.forkmode}" showoutput="true" tempdir="{build.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
544	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
545	<propertyref prefix="test-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
546	<mapper from="test-sys-prop.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
547	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
548	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
549	<path path="{run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
550	</classpath>	Cierra el elemento XML abierto anteriormente.
551	<modulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
552	<path path="{run.test.modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
553	</modulepath>	Cierra el elemento XML abierto anteriormente.
554	<formatter type="brief" usefile="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

555	<formatter type="xml"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
556	<jvmarg line="\${endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
557	<jvmarg value="-ea"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
558	<jvmarg line="\${run.test.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
559	<customizePrototype/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
560	</junit>	Cierra el elemento XML abierto anteriormente.
561	</sequential>	Cierra el elemento XML abierto anteriormente.
562	</macrodef>	Cierra el elemento XML abierto anteriormente.
563	</target>	Cierra el elemento XML abierto anteriormente.
564	<target depends="-init-modules-supported" name="-init-macrodef-junit-prototype-without-module" unless="modules.supported.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
565	<macrodef name="junit-prototype" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
566	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
567	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
568	<element name="customizePrototype" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
569	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
570	<property name="junit.forkmode" value="perTest"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
571	<junit dir="\${work.dir}" errorproperty="tests.failed" failureproperty="tests.failed" fork="true" forkmode="\${junit.forkmode}" showoutput="true" tempdir="\${build.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
572	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
573	<propertyref prefix="test-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
574	<mapper from="test-sys-prop.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
575	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
576	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
577	<path path="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
578	</classpath>	Cierra el elemento XML abierto anteriormente.
579	<formatter type="brief" usefile="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
580	<formatter type="xml"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
581	<jvmarg line="\${endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
582	<jvmarg value="-ea"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
583	<customizePrototype/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
584	</junit>	Cierra el elemento XML abierto anteriormente.
585	</sequential>	Cierra el elemento XML abierto anteriormente.
586	</macrodef>	Cierra el elemento XML abierto anteriormente.
587	</target>	Cierra el elemento XML abierto anteriormente.
588	<target depends="-init-test-properties,-init-macrodef-junit-prototype-with-module,-init-macrodef-junit-prototype-without-module" if="\${nb.junit.single}" name="-init-macrodef-junit-single" unless="\${nb.junit.batch}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
589	<macrodef name="junit" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
590	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
591	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
592	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
593	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
594	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
595	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
596	<j2seproject3:junit-prototype>	Declara un elemento XML de configuración, con sus atributos si corresponde.

597	<customizePrototype>	Declara un elemento XML de configuración, con sus atributos si corresponde.
598	<test methods="@{testmethods}" name="@{testincludes}" todir="\${build.test.results.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
599	</customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
600	</customizePrototype>	Cierra el elemento XML abierto anteriormente.
601	</j2seproject3:junit-prototype>	Cierra el elemento XML abierto anteriormente.
602	</sequential>	Cierra el elemento XML abierto anteriormente.
603	</macrodef>	Cierra el elemento XML abierto anteriormente.
604	</target>	Cierra el elemento XML abierto anteriormente.
605	<target depends="-init-test-properties,-init-macrodef-junit-prototype-with-module,-init-macrodef-junit-prototype-without-module" if="\${nb.junit.batch}" name="-init-macrodef-junit-batch" unless="\${nb.junit.single}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
606	<macrodef name="junit" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
607	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
608	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
609	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
610	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
611	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
612	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
613	<j2seproject3:junit-prototype>	Declara un elemento XML de configuración, con sus atributos si corresponde.
614	<customizePrototype>	Declara un elemento XML de configuración, con sus atributos si corresponde.
615	<batchtest todir="\${build.test.results.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
616	<fileset dir="\${test.src.dir}" excludes="@{excludes},\${excludes}" includes="@{includes}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
617	<filename name="@{testincludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
618	</fileset>	Cierra el elemento XML abierto anteriormente.
619	<fileset dir="\${build.test.classes.dir}" excludes="@{excludes},\${excludes},\${test.binaryexcludes}" includes="\${test.binaryincludes}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
620	<filename name="\${test.binarytestincludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
621	</fileset>	Cierra el elemento XML abierto anteriormente.
622	</batchtest>	Cierra el elemento XML abierto anteriormente.
623	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
624	</customizePrototype>	Cierra el elemento XML abierto anteriormente.
625	</j2seproject3:junit-prototype>	Cierra el elemento XML abierto anteriormente.
626	</sequential>	Cierra el elemento XML abierto anteriormente.
627	</macrodef>	Cierra el elemento XML abierto anteriormente.
628	</target>	Cierra el elemento XML abierto anteriormente.
629	<target depends="-init-macrodef-junit-init,-init-macrodef-junit-single,-init-macrodef-junit-batch" if="{junit.available}" name="-init-macrodef-junit">	Declara un elemento XML de configuración, con sus atributos si corresponde.
630	<target if="{testng.available}" name="-init-macrodef-testng">	Declara un elemento XML de configuración, con sus atributos si corresponde.
631	<macrodef name="testng" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
632	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
633	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
634	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
635	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
636	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
637	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.

638	<condition else="" property="testng.methods.arg" value="@{testincludes}.@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
639	<isset property="test.method"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
640	</condition>	Cierra el elemento XML abierto anteriormente.
641	<union id="test.set">	Declara un elemento XML de configuración, con sus atributos si corresponde.
642	<fileset dir="\${test.src.dir}" excludes="@{excludes},**/*.xml,{excludes}" includes="@{inclu des}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
643	<filename name="@{testincludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
644	</fileset>	Cierra el elemento XML abierto anteriormente.
645	</union>	Cierra el elemento XML abierto anteriormente.
646	<taskdef classname="org.testng.TestNGAntTask" classpath="\${run.test.classpath}" name="testng"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
647	<testng classfilesetref="test.set" failureProperty="tests.failed" listeners="org.testng.reporter s.VerboseReporter" methods="\${testng.methods.arg}" mode="\${testng.mode}" outputdir="\${build.test.results.dir}" s uiteName="waldonalds" testName="TestNG tests" workingDir="\${work.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
648	<xmlfileset dir="\${build.test.classes.dir}" includes="@{testincludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
649	<propertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
650	<propertyref prefix="test-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
651	<mapper from="test-sys-prop.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
652	</propertyset>	Cierra el elemento XML abierto anteriormente.
653	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
654	<path path="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
655	</classpath>	Cierra el elemento XML abierto anteriormente.
656	<jvmarg line="\${endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
657	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
658	</testng>	Cierra el elemento XML abierto anteriormente.
659	</sequential>	Cierra el elemento XML abierto anteriormente.
660	</macrodef>	Cierra el elemento XML abierto anteriormente.
661	</target>	Cierra el elemento XML abierto anteriormente.
662	<target name="-init-macrodef-test-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
663	<macrodef name="test-impl" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
664	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
665	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
666	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
667	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
668	<element implicit="true" name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
669	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
670	<echo>No tests executed.</echo>	Declara un elemento XML de configuración, con sus atributos si corresponde.
671	</sequential>	Cierra el elemento XML abierto anteriormente.
672	</macrodef>	Cierra el elemento XML abierto anteriormente.
673	</target>	Cierra el elemento XML abierto anteriormente.
674	<target depends="-init-macrodef-junit" if="\${junit.available}" name="-init-macrodef-junit-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
675	<macrodef name="test-impl" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
676	<attribute default="\${includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
677	<attribute default="\${excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
678	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
679	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

680	<element implicit="true" name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
681	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
682	<j2seproject3:junit excludes="@{excludes}" includes="@{includes}" testincludes="@{testincludes}" testmethods="@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
683	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
684	</j2seproject3:junit>	Cierra el elemento XML abierto anteriormente.
685	</sequential>	Cierra el elemento XML abierto anteriormente.
686	</macrodef>	Cierra el elemento XML abierto anteriormente.
687	</target>	Cierra el elemento XML abierto anteriormente.
688	<target depends="-init-macrodef-testng" if="{testng.available}" name="-init-macrodef-testng-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
689	<macrodef name="test-impl" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
690	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
691	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
692	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
693	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
694	<element implicit="true" name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
695	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
696	<j2seproject3:testng excludes="@{excludes}" includes="@{includes}" testincludes="@{testincludes}" testmethods="@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
697	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
698	</j2seproject3:testng>	Cierra el elemento XML abierto anteriormente.
699	</sequential>	Cierra el elemento XML abierto anteriormente.
700	</macrodef>	Cierra el elemento XML abierto anteriormente.
701	</target>	Cierra el elemento XML abierto anteriormente.
702	<target depends="-init-macrodef-test-impl,-init-macrodef-junit-impl,-init-macrodef-testng-impl" name="-init-macrodef-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
703	<macrodef name="test" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
704	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
705	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
706	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
707	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
708	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
709	<j2seproject3:test-impl excludes="@{excludes}" includes="@{includes}" testincludes="@{testincludes}" testmethods="@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
710	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
711	<jvmarg line="{run.jvmargs}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
712	<jvmarg line="{run.jvmargs.ide}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
713	</customize>	Cierra el elemento XML abierto anteriormente.
714	</j2seproject3:test-impl>	Cierra el elemento XML abierto anteriormente.
715	</sequential>	Cierra el elemento XML abierto anteriormente.
716	</macrodef>	Cierra el elemento XML abierto anteriormente.
717	</target>	Cierra el elemento XML abierto anteriormente.
718	<target depends="-init-macrodef-junit" if="{junit.available}" name="-init-macrodef-junit-debug-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
719	<macrodef name="test-debug-impl" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
720	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
721	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

722	<attribute default="*" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
723	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
724	<element name="customizeDebuggee" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
725	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
726	<j2seproject3:junit excludes="@{excludes}" includes="@{includes}" testincludes="@{testincludes}" testmethods="@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
727	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
728	<jvmarg value="-agentlib:jdwp=transport=\${debug-transport},address=\${jpda.address}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
729	<customizeDebuggee/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
730	</customize>	Cierra el elemento XML abierto anteriormente.
731	</j2seproject3:junit>	Cierra el elemento XML abierto anteriormente.
732	</sequential>	Cierra el elemento XML abierto anteriormente.
733	</macrodef>	Cierra el elemento XML abierto anteriormente.
734	</target>	Cierra el elemento XML abierto anteriormente.
735	<target if="\${testng.available}" name="-init-macrodef-testng-debug">	Declara un elemento XML de configuración, con sus atributos si corresponde.
736	<macrodef name="testng-debug" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
737	<attribute default="\${main.class}" name="testClass"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
738	<attribute default="" name="testMethod"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
739	<element name="customize2" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
740	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
741	<condition else="-testclass @{testClass}" property="test.class.or.method" value="-methods @{testClass}.@{testMethod}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
742	<isset property="test.method"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
743	</condition>	Cierra el elemento XML abierto anteriormente.
744	<condition else="-suiteName Waldonalds -testname @{testClass} \${test.class.or.method}" property="testng.cmd.args" value="@{testClass}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
745	<matches pattern="*.\\.xml" string="@{testClass}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
746	</condition>	Cierra el elemento XML abierto anteriormente.
747	<delete dir="\${build.test.results.dir}" quiet="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
748	<mkdir dir="\${build.test.results.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
749	<j2seproject3:debug classname="org.testng.TestNG" classpath="\${debug.test.classpath}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
750	<customizeDebuggee>	Declara un elemento XML de configuración, con sus atributos si corresponde.
751	<customize2/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
752	<jvmarg value="-ea"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
753	<arg line="\${testng.debug.mode}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
754	<arg line="-d \${build.test.results.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
755	<arg line="-listener org.testng.reporters.VerboseReporter"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
756	<arg line="\${testng.cmd.args}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
757	</customizeDebuggee>	Cierra el elemento XML abierto anteriormente.
758	</j2seproject3:debug>	Cierra el elemento XML abierto anteriormente.
759	</sequential>	Cierra el elemento XML abierto anteriormente.
760	</macrodef>	Cierra el elemento XML abierto anteriormente.
761	</target>	Cierra el elemento XML abierto anteriormente.
762	<target depends="-init-macrodef-testng-debug" if="\${testng.available}" name="-init-macrodef-testng-debug-impl">	Declara un elemento XML de configuración, con sus atributos si corresponde.
763	<macrodef name="testng-debug-impl" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.

764	<attribute default="{main.class}" name="testClass"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
765	<attribute default="" name="testMethod"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
766	<element implicit="true" name="customize2" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
767	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
768	<j2seproject3:testng-debug testClass="@{testClass}" testMethod="@{testMethod}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
769	<customize2/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
770	</j2seproject3:testng-debug>	Cierra el elemento XML abierto anteriormente.
771	</sequential>	Cierra el elemento XML abierto anteriormente.
772	</macrodef>	Cierra el elemento XML abierto anteriormente.
773	</target>	Cierra el elemento XML abierto anteriormente.
774	<target depends="-init-macrodef-junit-debug-impl" if="{junit.available}" name="-init-macrodef-test-debug-junit">	Declara un elemento XML de configuración, con sus atributos si corresponde.
775	<macrodef name="test-debug" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
776	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
777	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
778	<attribute default="" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
779	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
780	<attribute default="{main.class}" name="testClass"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
781	<attribute default="" name="testMethod"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
782	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
783	<j2seproject3:test-debug-impl excludes="@{excludes}" includes="@{includes}" testincludes="@{testincludes}" testmethods="@{testmethods}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
784	<customizeDebuggee>	Declara un elemento XML de configuración, con sus atributos si corresponde.
785	<jvmarg line="{run.jvmargs}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
786	<jvmarg line="{run.jvmargs.ide}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
787	</customizeDebuggee>	Cierra el elemento XML abierto anteriormente.
788	</j2seproject3:test-debug-impl>	Cierra el elemento XML abierto anteriormente.
789	</sequential>	Cierra el elemento XML abierto anteriormente.
790	</macrodef>	Cierra el elemento XML abierto anteriormente.
791	</target>	Cierra el elemento XML abierto anteriormente.
792	<target depends="-init-macrodef-testng-debug-impl" if="{testng.available}" name="-init-macrodef-test-debug-testng">	Declara un elemento XML de configuración, con sus atributos si corresponde.
793	<macrodef name="test-debug" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
794	<attribute default="{includes}" name="includes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
795	<attribute default="{excludes}" name="excludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
796	<attribute default="" name="testincludes"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
797	<attribute default="" name="testmethods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
798	<attribute default="{main.class}" name="testClass"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
799	<attribute default="" name="testMethod"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
800	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
801	<j2seproject3:testng-debug-impl testClass="@{testClass}" testMethod="@{testMethod}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
802	<customize2>	Declara un elemento XML de configuración, con sus atributos si corresponde.
803	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
804	<propertyref prefix="test-sys-prop.">	Declara un elemento XML de configuración, con sus atributos si corresponde.
805	<mapper from="test-sys-prop.*" to="" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

806	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
807	</customize2>	Cierra el elemento XML abierto anteriormente.
808	</j2seproject3:testng-debug-impl>	Cierra el elemento XML abierto anteriormente.
809	</sequential>	Cierra el elemento XML abierto anteriormente.
810	</macrodef>	Cierra el elemento XML abierto anteriormente.
811	</target>	Cierra el elemento XML abierto anteriormente.
812	<target depends="-init-macrodef-test-debug-junit, -init-macrodef-test-debug-testng" name="-init-macrodef-test-debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
813	<!--	Comentario XML para humanos o para NetBeans.
814	pre NB7.2 profiling section; consider it deprecated	Contenido de configuración o documentación que completa el archivo.
815	-->	Comentario XML para humanos o para NetBeans.
816	<target depends="-profile-pre-init, init, -profile-post-init, -profile-init-macrodef-profile, -profile-init-check" if="profiler.info.jvmargs.agent" name="profile-init"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
817	<target if="profiler.info.jvmargs.agent" name="-profile-pre-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
818	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
819	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
820	</target>	Cierra el elemento XML abierto anteriormente.
821	<target if="profiler.info.jvmargs.agent" name="-profile-post-init">	Declara un elemento XML de configuración, con sus atributos si corresponde.
822	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
823	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
824	</target>	Cierra el elemento XML abierto anteriormente.
825	<target if="profiler.info.jvmargs.agent" name="-profile-init-macrodef-profile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
826	<macrodef name="resolve">	Declara un elemento XML de configuración, con sus atributos si corresponde.
827	<attribute name="name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
828	<attribute name="value"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
829	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
830	<property name="@{name}" value="\${env.@{value}}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
831	</sequential>	Cierra el elemento XML abierto anteriormente.
832	</macrodef>	Cierra el elemento XML abierto anteriormente.
833	<macrodef name="profile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
834	<attribute default="\${main.class}" name="classname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
835	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
836	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
837	<property environment="env"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
838	<resolve name="profiler.current.path" value="\${profiler.info.pathvar}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
839	<java classname="@{classname}" dir="{profiler.info.dir}" failonerror="{java.failonerror}" fork="true" jvm="{profiler.info.jvm}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
840	<jvmarg line="{endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
841	<jvmarg value="{profiler.info.jvmargs.agent}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
842	<jvmarg line="{profiler.info.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
843	<env key="{profiler.info.pathvar}" path="{profiler.info.agentpath}:{profiler.current.path}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
844	<arg line="{application.args}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
845	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
846	<path path="{run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
847	</classpath>	Cierra el elemento XML abierto anteriormente.

848	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
849	<propertyref prefix="run-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
850	<mapper from="run-sys-prop.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
851	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
852	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
853	</java>	Cierra el elemento XML abierto anteriormente.
854	</sequential>	Cierra el elemento XML abierto anteriormente.
855	</macrodef>	Cierra el elemento XML abierto anteriormente.
856	</target>	Cierra el elemento XML abierto anteriormente.
857	<target depends="-profile-pre-init, init, -profile-post-init, -profile-init-macrodef-profile" if="profiler.info.jvmargs.agent" name="-profile-init-check">	Declara un elemento XML de configuración, con sus atributos si corresponde.
858	<fail unless="profiler.info.jvm">Must set JVM to use for profiling in profiler.info.jvm</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
859	<fail unless="profiler.info.jvmargs.agent">Must set profiler agent JVM arguments in profiler.info.jvmargs.agent</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
860	</target>	Cierra el elemento XML abierto anteriormente.
861	<!--	Comentario XML para humanos o para NetBeans.
862	end of pre NB7.2 profiling section	Contenido de configuración o documentación que completa el archivo.
863	-->	Comentario XML para humanos o para NetBeans.
864	<target depends="-init-debug-args" name="-init-macrodef-nbjpda">	Declara un elemento XML de configuración, con sus atributos si corresponde.
865	<macrodef name="nbjpdastart" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
866	<attribute default="{main.class}" name="name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
867	<attribute default="{debug.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
868	<attribute default="{debug.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
869	<attribute default="" name="stopclassname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
870	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
871	<nbjpdastart addressproperty="jpda.address" name="@{name}" stopclassname="@{stopclassname}" transport="{debug-transport}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
872	<modulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
873	<path path="@{modulepath}" />	Declara un elemento XML de configuración, con sus atributos si corresponde.
874	</modulepath>	Cierra el elemento XML abierto anteriormente.
875	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
876	<path path="@{classpath}" />	Declara un elemento XML de configuración, con sus atributos si corresponde.
877	</classpath>	Cierra el elemento XML abierto anteriormente.
878	</nbjpdastart>	Cierra el elemento XML abierto anteriormente.
879	</sequential>	Cierra el elemento XML abierto anteriormente.
880	</macrodef>	Cierra el elemento XML abierto anteriormente.
881	<macrodef name="nbjpdareload" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
882	<attribute default="{build.classes.dir}" name="dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
883	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
884	<nbjpdareload>	Declara un elemento XML de configuración, con sus atributos si corresponde.
885	<fileset dir="@{dir}" includes="{fix.classes}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
886	<include name="{fix.includes}*.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
887	</fileset>	Cierra el elemento XML abierto anteriormente.
888	</nbjpdareload>	Cierra el elemento XML abierto anteriormente.
889	</sequential>	Cierra el elemento XML abierto anteriormente.

890	</macrodef>	Cierra el elemento XML abierto anteriormente.
891	</target>	Cierra el elemento XML abierto anteriormente.
892	<target name="-init-debug-args">	Declara un elemento XML de configuración, con sus atributos si corresponde.
893	<condition else="dt_socket" property="debug-transport-by-os" value="dt_shmem">	Declara un elemento XML de configuración, con sus atributos si corresponde.
894	<os family="windows"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
895	</condition>	Cierra el elemento XML abierto anteriormente.
896	<condition else="\${debug-transport-by-os}" property="debug-transport" value="\${debug.transport}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
897	<isset property="debug.transport"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
898	</condition>	Cierra el elemento XML abierto anteriormente.
899	</target>	Cierra el elemento XML abierto anteriormente.
900	<target depends="-init-debug-args" name="-init-macrodef-debug">	Declara un elemento XML de configuración, con sus atributos si corresponde.
901	<macrodef name="debug" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
902	<attribute default="\${module.name}" name="modulename"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
903	<attribute default="\${main.class}" name="classname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
904	<attribute default="\${debug.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
905	<attribute default="\${debug.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
906	<element name="customizeDebuggee" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
907	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
908	<j2seproject1:java classname="@{classname}" classpath="@{classpath}" modulename="@{modulename}" modulepath="@{modulepath}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
909	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
910	<jvmarg value="-agentlib:jdwp=transport=\${debug-transport},address=\${jpda.address}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
911	<customizeDebuggee/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
912	</customize>	Cierra el elemento XML abierto anteriormente.
913	</j2seproject1:java>	Cierra el elemento XML abierto anteriormente.
914	</sequential>	Cierra el elemento XML abierto anteriormente.
915	</macrodef>	Cierra el elemento XML abierto anteriormente.
916	</target>	Cierra el elemento XML abierto anteriormente.
917	<target depends="-init-source-module-properties" if="named.module.internal" name="-init-macrodef-java-with-module">	Declara un elemento XML de configuración, con sus atributos si corresponde.
918	<macrodef name="java" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
919	<attribute default="\${module.name}" name="modulename"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
920	<attribute default="\${main.class}" name="classname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
921	<attribute default="\${run.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
922	<attribute default="\${run.upgrademodulepath}" name="upgrademodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
923	<attribute default="\${run.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
924	<attribute default="jvm" name="jvm"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
925	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
926	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
927	<java classname="@{classname}" dir="\${work.dir}" failonerror="\${java.failonerror}" fork="true" module="@{modulename}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
928	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
929	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
930	</classpath>	Cierra el elemento XML abierto anteriormente.
931	<modulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.

932	<pathelement path="@{modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
933	<pathelement location="\${module.build.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
934	</modulepath>	Cierra el elemento XML abierto anteriormente.
935	<upgrademodulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
936	<path path="@{upgrademodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
937	</upgrademodulepath>	Cierra el elemento XML abierto anteriormente.
938	<jvmarg value="-Dfile.encoding=\${runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
939	<redirector errorencoding="\${runtime.encoding}" inputencoding="\${runtime.encoding}" outputencoding="\${runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
940	<jvmarg line="\${run.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
941	<jvmarg line="\${run.jvmargs.ide}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
942	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
943	<propertyref prefix="run-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
944	<mapper from="run-sys-prop.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
945	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
946	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
947	</java>	Cierra el elemento XML abierto anteriormente.
948	</sequential>	Cierra el elemento XML abierto anteriormente.
949	</macrodef>	Cierra el elemento XML abierto anteriormente.
950	</target>	Cierra el elemento XML abierto anteriormente.
951	<target depends="-init-source-module-properties" if="unnamed.module.internal" name="-init-macrodef-java-with-unnamed-module">	Declara un elemento XML de configuración, con sus atributos si corresponde.
952	<macrodef name="java" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
953	<attribute default="" name="modulename"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
954	<attribute default="\${main.class}" name="classname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
955	<attribute default="\${run.modulepath}" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
956	<attribute default="\${run.upgrademodulepath}" name="upgrademodulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
957	<attribute default="\${run.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
958	<attribute default="jvm" name="jvm"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
959	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
960	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
961	<java classname="@{classname}" dir="\${work.dir}" failonerror="{java.failonerror}" fork="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
962	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
963	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
964	</classpath>	Cierra el elemento XML abierto anteriormente.
965	<modulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
966	<path path="@{modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
967	</modulepath>	Cierra el elemento XML abierto anteriormente.
968	<upgrademodulepath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
969	<path path="@{upgrademodulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
970	</upgrademodulepath>	Cierra el elemento XML abierto anteriormente.
971	<jvmarg value="-Dfile.encoding=\${runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
972	<redirector errorencoding="\${runtime.encoding}" inputencoding="\${runtime.encoding}" outputencoding="\${runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
973	<jvmarg line="\${run.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

974	<jvmarg line="\${run.jvmargs.ide}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
975	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
976	<propertyref prefix="run-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
977	<mapper from="run-sys-prop.*" to="" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
978	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
979	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
980	</java>	Cierra el elemento XML abierto anteriormente.
981	</sequential>	Cierra el elemento XML abierto anteriormente.
982	</macrodef>	Cierra el elemento XML abierto anteriormente.
983	</target>	Cierra el elemento XML abierto anteriormente.
984	<target depends="-init-source-module-properties" name="-init-macrodef-java-without-module" unless="modules.supported.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
985	<macrodef name="java" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
986	<attribute default="" name="modulename"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
987	<attribute default="\${main.class}" name="classname"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
988	<attribute default="" name="modulepath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
989	<attribute default="\${run.classpath}" name="classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
990	<attribute default="jvm" name="jvm"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
991	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
992	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
993	<java classname="@{classname}" dir="{work.dir}" failonerror="{java.failonerror}" fork="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.
994	<jvmarg line="{endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
995	<jvmarg value="-Dfile.encoding={runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
996	<redirector errencoding="{runtime.encoding}" inputencoding="{runtime.encoding}" outputencoding="{runtime.encoding}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
997	<jvmarg line="{run.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
998	<jvmarg line="{run.jvmargs.ide}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
999	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1000	<path path="@{classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1001	</classpath>	Cierra el elemento XML abierto anteriormente.
1002	<syspropertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1003	<propertyref prefix="run-sys-prop."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1004	<mapper from="run-sys-prop.*" to="" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1005	</syspropertyset>	Cierra el elemento XML abierto anteriormente.
1006	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1007	</java>	Cierra el elemento XML abierto anteriormente.
1008	</sequential>	Cierra el elemento XML abierto anteriormente.
1009	</macrodef>	Cierra el elemento XML abierto anteriormente.
1010	</target>	Cierra el elemento XML abierto anteriormente.
1011	<target depends="-init-macrodef-java-with-module, -init-macrodef-java-with-unnamed-module, -init-macrodef-java-without-module" name="-init-macrodef-java">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1012	<target name="-init-macrodef-copylibs">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1013	<macrodef name="copylibs" uri="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1014	<attribute default="{manifest.file}" name="manifest"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1015	<element name="customize" optional="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1016	<sequential>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1017	<property location="\${build.classes.dir}" name="build.classes.dir.resolved"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1018	<pathconvert property="run.classpath.without.build.classes.dir">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1019	<path path="\${run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1020	<map from="\${build.classes.dir.resolved}" to=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1021	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1022	<pathconvert pathsep=" " property="jar.classpath">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1023	<path path="\${run.classpath.without.build.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1024	<chainedmapper>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1025	<flattenmapper/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1026	<filtermapper>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1027	<replacestring from=" " to="%20"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1028	</filtermapper>	Cierra el elemento XML abierto anteriormente.
1029	<globmapper from="*" to="lib/**"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1030	</chainedmapper>	Cierra el elemento XML abierto anteriormente.
1031	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1032	<taskdef classname="org.netbeans.modules.java.j2seproject.copylibtask.CopyLibs" classpath="\${libs.CopyLibs.classpath}" name="copylibs"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1033	<copylibs compress="\${jar.compress}" excludeFromCopy="\${copylibs.excludes}" index="\${jar.index}" indexMetaInf="\${jar.index.metainf}" jarfile="\${dist.jar}" manifest="@{manifest}" manifestencoding="UTF-8" rebase="\${copylibs.rebase}" runtimeclasspath="\${run.classpath.without.build.classes.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1034	<fileset dir="\${build.classes.dir}" excludes="\${dist.archive.excludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1035	<manifest>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1036	<attribute name="Class-Path" value="\${jar.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1037	<customize/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1038	</manifest>	Cierra el elemento XML abierto anteriormente.
1039	</copylibs>	Cierra el elemento XML abierto anteriormente.
1040	</sequential>	Cierra el elemento XML abierto anteriormente.
1041	</macrodef>	Cierra el elemento XML abierto anteriormente.
1042	</target>	Cierra el elemento XML abierto anteriormente.
1043	<target name="-init-presetdef-jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1044	<presetdef name="jar" uri="http://www.netbeans.org/ns/j2se-project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1045	<jar compress="\${jar.compress}" index="\${jar.index}" jarfile="\${dist.jar}" manifestencoding="UTF-8">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1046	<j2seproject1:fileset dir="\${build.classes.dir}" excludes="\${dist.archive.excludes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1047	</jar>	Cierra el elemento XML abierto anteriormente.
1048	</presetdef>	Cierra el elemento XML abierto anteriormente.
1049	</target>	Cierra el elemento XML abierto anteriormente.
1050	<target name="-init-ap-cmdline-properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1051	<property name="annotation.processing.enabled" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1052	<property name="annotation.processing.processors.list" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1053	<property name="annotation.processing.processor.options" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1054	<property name="annotation.processing.run.all.processors" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1055	<property name="javac.processorpath" value="\${javac.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1056	<property name="javac.test.processorpath" value="\${javac.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1057	<condition property="ap.supported.internal" value="true">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1058	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1059	<matches pattern="1\.[0-5](\..*)?" string="{javac.source}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1060	</not>	Cierra el elemento XML abierto anteriormente.
1061	</condition>	Cierra el elemento XML abierto anteriormente.
1062	</target>	Cierra el elemento XML abierto anteriormente.
1063	<target depends="-init-ap-cmdline-properties" if="ap.supported.internal" name="-init-ap-cmdline-supported">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1064	<condition else="" property="ap.processors.internal" value="-processor \${annotation.processing.processor s.list}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1065	<isfalse value="{annotation.processing.run.all.processors}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1066	</condition>	Cierra el elemento XML abierto anteriormente.
1067	<condition else="" property="ap.proc.none.internal" value="-proc:none">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1068	<isfalse value="{annotation.processing.enabled}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1069	</condition>	Cierra el elemento XML abierto anteriormente.
1070	</target>	Cierra el elemento XML abierto anteriormente.
1071	<target depends="-init-ap-cmdline-properties, -init-ap-cmdline-supported" name="-init-ap-cmdline">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1072	<property name="ap.cmd.line.internal" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1073	</target>	Cierra el elemento XML abierto anteriormente.
1074	<target depends="-pre-init, -init-private, -init-libraries, -init-user, -init-project, -do-init, -post-init, -init- check, -init-macrodef-property, -init-macrodef-javac, -init-macrodef-test, -init-macrodef-test-debug, -init-macrodef- nbjpd, -init-macrodef-debug, -init-macrodef-java, -init-presetdef-jar, -init-ap-cmdline" name="init"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1075	<!--	Comentario XML para humanos o para NetBeans.
1076	=====	Contenido de configuración o documentación que completa el archivo.
1077	COMPILE SECTION	Contenido de configuración o documentación que completa el archivo.
1078	=====	Contenido de configuración o documentación que completa el archivo.
1079	-->	Comentario XML para humanos o para NetBeans.
1080	<target name="-deps-jar-init" unless="built-jar.properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1081	<property location="{build.dir}/built-jar.properties" name="built-jar.properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1082	<delete file="{built-jar.properties}" quiet="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1083	</target>	Cierra el elemento XML abierto anteriormente.
1084	<target if="already.built.jar.{basedir}" name="-warn-already-built-jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1085	<echo level="warn" message="Cycle detected: Waldonalds was already built"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1086	</target>	Cierra el elemento XML abierto anteriormente.
1087	<target depends="init, -deps-jar-init" name="deps-jar" unless="no.deps">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1088	<mkdir dir="{build.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1089	<touch file="{built-jar.properties}" verbose="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1090	<property file="{built-jar.properties}" prefix="already.built.jar."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1091	<antcall target="-warn-already-built-jar"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1092	<propertyfile file="{built-jar.properties}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1093	<entry key="{basedir}" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1094	</propertyfile>	Cierra el elemento XML abierto anteriormente.
1095	</target>	Cierra el elemento XML abierto anteriormente.
1096	<target depends="init, -check-automatic-build, -clean-after-automatic-build" name="-verify-automatic-build"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1097	<target depends="init" name="-check-automatic-build">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1098	<available file="{build.classes.dir}/.netbeans_automatic_build" property="netbeans.automatic.build"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1099	</target>	Cierra el elemento XML abierto anteriormente.

1100	<target depends="init" if="netbeans.automatic.build" name="-clean-after-automatic-build">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1101	<antcall target="clean">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1102	<param name="no.dependencies" value="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1103	</antcall>	Cierra el elemento XML abierto anteriormente.
1104	</target>	Cierra el elemento XML abierto anteriormente.
1105	<target depends="init,deps-jar" name="-pre-pre-compile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1106	<mkdir dir="\${build.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1107	</target>	Cierra el elemento XML abierto anteriormente.
1108	<target name="-pre-compile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1109	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1110	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1111	</target>	Cierra el elemento XML abierto anteriormente.
1112	<target if="do.depend.true" name="-compile-depend">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1113	<pathconvert property="build.generated.subdirs">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1114	<dirset dir="\${build.generated.sources.dir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1115	<include name="**"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1116	</dirset>	Cierra el elemento XML abierto anteriormente.
1117	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1118	<j2seproject3:depend srcdir="\${src.dir}:\${build.generated.subdirs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1119	</target>	Cierra el elemento XML abierto anteriormente.
1120	<target depends="init,deps-jar,-pre-pre-compile,-pre-compile,-copy-persistence-xml,-compile-depend" if="have.sources" name="-do-compile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1121	<j2seproject3:javac gensrcdir="\${build.generated.sources.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1122	<copy todir="\${build.classes.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1123	<fileset dir="\${src.dir}" excludes="\${build.classes.excludes},\${excludes}" includes="{includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1124	</copy>	Cierra el elemento XML abierto anteriormente.
1125	</target>	Cierra el elemento XML abierto anteriormente.
1126	<target if="has.persistence.xml" name="-copy-persistence-xml">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1127	<mkdir dir="\${build.classes.dir}/META-INF"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1128	<copy todir="\${build.classes.dir}/META-INF">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1129	<fileset dir="\${meta.inf.dir}" includes="persistence.xml orm.xml"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1130	</copy>	Cierra el elemento XML abierto anteriormente.
1131	</target>	Cierra el elemento XML abierto anteriormente.
1132	<target name="-post-compile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1133	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1134	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1135	</target>	Cierra el elemento XML abierto anteriormente.
1136	<target depends="init,deps-jar,-verify-automatic-build,-pre-pre-compile,-pre-compile,-do-compile,-post-compile" description="Compile project." name="compile"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1137	<target name="-pre-compile-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1138	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1139	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1140	</target>	Cierra el elemento XML abierto anteriormente.
1141	<target depends="init,deps-jar,-pre-pre-compile" name="-do-compile-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1142	<fail unless="javac.includes">Must select some files in the IDE or set javac.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1143	<j2seproject3:force-recompile/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1144	<j2seproject3:javac excludes="" gensrcdir="\${build.generated.sources.dir}" includes="\${javac.includes}, module-info.java" sourcepath="\${src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1145	</target>	Cierra el elemento XML abierto anteriormente.
1146	<target name="-post-compile-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1147	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1148	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1149	</target>	Cierra el elemento XML abierto anteriormente.
1150	<target depends="init,deps-jar,-verify-automatic-build,-pre-pre-compile,-pre-compile-single,-do-compile-single,-post-compile-single" name="compile-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1151	<!--	Comentario XML para humanos o para NetBeans.
1152	=====	Contenido de configuración o documentación que completa el archivo.
1153	JAR BUILDING SECTION	Contenido de configuración o documentación que completa el archivo.
1154	=====	Contenido de configuración o documentación que completa el archivo.
1155	-->	Comentario XML para humanos o para NetBeans.
1156	<target depends="init" name="-pre-pre-jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1157	<dirname file="\${dist.jar}" property="dist.jar.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1158	<mkdir dir="\${dist.jar.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1159	</target>	Cierra el elemento XML abierto anteriormente.
1160	<target name="-pre-jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1161	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1162	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1163	</target>	Cierra el elemento XML abierto anteriormente.
1164	<target depends="init,compile" name="-check-module-main-class">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1165	<pathconvert property="main.class.file">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1166	<string value="\${main.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1167	<unpackagemapper from="*" to="*.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1168	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1169	<condition property="do.module.main.class">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1170	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1171	<isset property="main.class.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1172	<available file="\${build.classes.dir}/module-info.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1173	<available file="\${build.classes.dir}/\${main.class.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1174	<isset property="libs.CopyLibs.classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1175	<available classname="org.netbeans.modules.java.j2seproject.moduletask.ModuleMainClass" classpath="\${libs.CopyLibs.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1176	</and>	Cierra el elemento XML abierto anteriormente.
1177	</condition>	Cierra el elemento XML abierto anteriormente.
1178	</target>	Cierra el elemento XML abierto anteriormente.
1179	<target depends="-check-module-main-class" if="do.module.main.class" name="-set-module-main-class">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1180	<taskdef classname="org.netbeans.modules.java.j2seproject.moduletask.ModuleMainClass" classpath="\${libs.CopyLibs.classpath}" name="modulemainclass"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1181	<modulemainclass failonerror="false" mainclass="\${main.class}" moduleinfo="\${build.classes.dir}/module-info.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1182	</target>	Cierra el elemento XML abierto anteriormente.
1183	<target depends="init" if="do.archive" name="-do-jar-create-manifest" unless="manifest.available">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1184	<tempfile deleteonexit="true" destdir="\${build.dir}" property="tmp.manifest.file"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1185	<touch file="\${tmp.manifest.file}" verbose="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1186	</target>	Cierra el elemento XML abierto anteriormente.
1187	<target depends="init" if="do.archive+manifest.available" name="-do-jar-copy-manifest">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1188	<tempfile deleteonexit="true" destdir="\${build.dir}" property="tmp.manifest.file"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1189	<copy encoding="\${manifest.encoding}" file="\${manifest.file}" outputencoding="UTF-8" tofile="\${tmp.manifest.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1190	</target>	Cierra el elemento XML abierto anteriormente.
1191	<target depends="init,-do-jar-create-manifest,-do-jar-copy-manifest" if="do.archive+main.class.available" name="-do-jar-set-mainclass">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1192	<manifest encoding="UTF-8" file="\${tmp.manifest.file}" mode="update">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1193	<attribute name="Main-Class" value="\${main.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1194	</manifest>	Cierra el elemento XML abierto anteriormente.
1195	</target>	Cierra el elemento XML abierto anteriormente.
1196	<target depends="init,-do-jar-create-manifest,-do-jar-copy-manifest" if="do.archive+profile.available" name="-do-jar-set-profile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1197	<manifest encoding="UTF-8" file="\${tmp.manifest.file}" mode="update">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1198	<attribute name="Profile" value="\${javac.profile}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1199	</manifest>	Cierra el elemento XML abierto anteriormente.
1200	</target>	Cierra el elemento XML abierto anteriormente.
1201	<target depends="init,-do-jar-create-manifest,-do-jar-copy-manifest" if="do.archive+splashscreen.available" name="-do-jar-set-splashscreen">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1202	<basename file="\${application.splash}" property="splashscreen.basename"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1203	<mkdir dir="\${build.classes.dir}/META-INF"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1204	<copy failonerror="false" file="\${application.splash}" todir="\${build.classes.dir}/META-INF"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1205	<manifest encoding="UTF-8" file="\${tmp.manifest.file}" mode="update">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1206	<attribute name="SplashScreen-Image" value="META-INF/\${splashscreen.basename}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1207	</manifest>	Cierra el elemento XML abierto anteriormente.
1208	</target>	Cierra el elemento XML abierto anteriormente.
1209	<target depends="init,compile" name="-check-do-mkdist">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1210	<condition property="do.mkdist">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1211	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1212	<isset property="do.archive"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1213	<isset property="libs.CopyLibs.classpath"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1214	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1215	<istrue value="\${mkdist.disabled}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1216	</not>	Cierra el elemento XML abierto anteriormente.
1217	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1218	<available file="\${build.classes.dir}/module-info.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1219	</not>	Cierra el elemento XML abierto anteriormente.
1220	</and>	Cierra el elemento XML abierto anteriormente.
1221	</condition>	Cierra el elemento XML abierto anteriormente.
1222	</target>	Cierra el elemento XML abierto anteriormente.
1223	<target depends="init,-init-macrodef-copylibs,compile,-pre-pre-jar,-pre-jar,-do-jar-create-manifest,-do-jar-copy-manifest,-do-jar-set-mainclass,-do-jar-set-profile,-do-jar-set-splashscreen,-check-do-mkdist" if="do.mkdist" name="-do-jar-copylibs">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1224	<j2seproject3:copylibs manifest="\${tmp.manifest.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1225	<echo level="info">To run this application from the command line without Ant, try:</echo>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1226	<property location="\${dist.jar}" name="dist.jar.resolved"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1227	<echo level="info">java -jar "\${dist.jar.resolved}"</echo>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1228	</target>	Cierra el elemento XML abierto anteriormente.
1229	<target depends="init,compile,-pre-pre-jar,-pre-jar,-do-jar-create-manifest,-do-jar-copy-manifest,-do-jar-se t-mainclass,-do-jar-set-profile,-do-jar-set-splashscreen,-check-do-mkdist" if="do.archive" name="-do-jar-jar" un less="do.mkdist">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1230	<j2seproject1:jar manifest="\${tmp.manifest.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1231	<property location="\${build.classes.dir}" name="build.classes.dir.resolved"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1232	<property location="\${dist.jar}" name="dist.jar.resolved"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1233	<condition else="\${dist.jar.resolved}" property="jar.usage.message.class.path.replacement" value="">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1234	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1235	</condition>	Cierra el elemento XML abierto anteriormente.
1236	<pathconvert property="run.classpath.with.dist.jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1237	<path path="\${run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1238	<map from="\${build.classes.dir.resolved}" to="\${jar.usage.message.class.path.replacement}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1239	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1240	<pathconvert property="run.modulepath.with.dist.jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1241	<path location="\${dist.jar.resolved}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1242	<path path="\${run.modulepath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1243	<map from="\${build.classes.dir.resolved}" to="\${dist.jar.resolved}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1244	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1245	<condition else="\${run.modulepath}" property="jar.usage.message.run.modulepath.with.dist.jar" value="\${r un.modulepath.with.dist.jar}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1246	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1247	</condition>	Cierra el elemento XML abierto anteriormente.
1248	<condition else="" property="jar.usage.message.module.path" value=" -p \${jar.usage.message.run.modulepat h.with.dist.jar}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1249	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1250	<isset property="modules.supported.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1251	<length length="0" string="\${jar.usage.message.run.modulepath.with.dist.jar}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1252	</and>	Cierra el elemento XML abierto anteriormente.
1253	</condition>	Cierra el elemento XML abierto anteriormente.
1254	<condition else="" property="jar.usage.message.class.path" value=" -cp \${run.classpath.with.dist.jar}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1255	<length length="0" string="\${run.classpath.with.dist.jar}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1256	</condition>	Cierra el elemento XML abierto anteriormente.
1257	<condition else="/\${main.class}" property="jar.usage.message.main.class.class.selector" value="">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1258	<isset property="do.module.main.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1259	</condition>	Cierra el elemento XML abierto anteriormente.
1260	<condition else=" \${main.class}" property="jar.usage.message.main.class" value=" -m \${module.name}\${jar. usage.message.main.class.class.selector}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1261	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1262	</condition>	Cierra el elemento XML abierto anteriormente.
1263	<condition else="" property="jar.usage.message" value="To run this application from the command line wit hout Ant, try:\${line.separator}\${platform.java}\${jar.usage.message.module.path}\${jar.usage.message.class.path}\${ jar.usage.message.main.class}">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1264	<isset property="main.class.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1265	</condition>	Cierra el elemento XML abierto anteriormente.
1266	<condition else="debug" property="jar.usage.level" value="info">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1267	<isset property="main.class.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1268	</condition>	Cierra el elemento XML abierto anteriormente.
1269	<echo level="{jar.usage.level}" message="{jar.usage.message}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1270	</target>	Cierra el elemento XML abierto anteriormente.
1271	<target depends="-do-jar-copylibs" if="do.archive" name="-do-jar-delete-manifest">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1272	<delete>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1273	<fileset file="{tmp.manifest.file}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1274	</delete>	Cierra el elemento XML abierto anteriormente.
1275	</target>	Cierra el elemento XML abierto anteriormente.
1276	<target depends="init,compile,-pre-pre-jar,-pre-jar,-do-jar-create-manifest,-do-jar-copy-manifest,-do-jar-se t-mainclass,-do-jar-set-profile,-do-jar-set-splashscreen,-do-jar-jar,-do-jar-delete-manifest" name="-do-jar-with out-libraries"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1277	<target depends="init,compile,-pre-pre-jar,-pre-jar,-do-jar-create-manifest,-do-jar-copy-manifest,-do-jar-se t-mainclass,-do-jar-set-profile,-do-jar-set-splashscreen,-do-jar-copylibs,-do-jar-delete-manifest" name="-do-jar -with-libraries"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1278	<target name="-post-jar">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1279	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1280	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1281	</target>	Cierra el elemento XML abierto anteriormente.
1282	<target depends="init,compile,-pre-jar,-set-module-main-class,-do-jar-without-libraries,-do-jar-with-librari es,-post-jar" name="-do-jar"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1283	<target depends="init,compile,-pre-jar,-do-jar,-post-jar,deploy" description="Build JAR." name="jar"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1284	<!--	Comentario XML para humanos o para NetBeans.
1285	=====	Contenido de configuración o documentación que completa el archivo.
1286	DEPLOY SECTION	Contenido de configuración o documentación que completa el archivo.
1287	=====	Contenido de configuración o documentación que completa el archivo.
1288	-->	Comentario XML para humanos o para NetBeans.
1289	<target name="-pre-deploy">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1290	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1291	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1292	</target>	Cierra el elemento XML abierto anteriormente.
1293	<target depends="init" name="-check-jlink">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1294	<condition property="do.jlink.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1295	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1296	<istrue value="{do.jlink}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1297	<isset property="do.archive"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1298	<isset property="named.module.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1299	</and>	Cierra el elemento XML abierto anteriormente.
1300	</condition>	Cierra el elemento XML abierto anteriormente.
1301	</target>	Cierra el elemento XML abierto anteriormente.
1302	<target depends="init,-do-jar,-post-jar,-pre-deploy,-check-jlink" if="do.jlink.internal" name="-do-deploy">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1303	<delete dir="{dist.jlink.dir}" failonerror="false" quiet="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1304	<property name="jlink.launcher.name" value="{application.title}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1305	<condition else="\${module.name}" property="jlink.add.modules" value="\${module.name},\${jlink.additionalmodules}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1306	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1307	<isset property="jlink.additionalmodules"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1308	<length length="0" string="\${jlink.additionalmodules}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1309	</and>	Cierra el elemento XML abierto anteriormente.
1310	</condition>	Cierra el elemento XML abierto anteriormente.
1311	<condition property="jlink.do.strip.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1312	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1313	<isset property="jlink.strip"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1314	<istrue value="\${jlink.strip}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1315	</and>	Cierra el elemento XML abierto anteriormente.
1316	</condition>	Cierra el elemento XML abierto anteriormente.
1317	<condition property="jlink.do.additionalparam.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1318	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1319	<isset property="jlink.additionalparam"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1320	<length length="0" string="\${jlink.additionalparam}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1321	</and>	Cierra el elemento XML abierto anteriormente.
1322	</condition>	Cierra el elemento XML abierto anteriormente.
1323	<condition property="jlink.do.launcher.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1324	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1325	<istrue value="\${jlink.launcher}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1326	<isset property="main.class.available"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1327	</and>	Cierra el elemento XML abierto anteriormente.
1328	</condition>	Cierra el elemento XML abierto anteriormente.
1329	<property name="platform.jlink" value="\${jdk.home}/bin/jlink"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1330	<property name="jlink.systemmodules.internal" value="\${jdk.home}/jmods"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1331	<exec executable="\${platform.jlink}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1332	<arg value="--module-path"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1333	<arg path="\${jlink.systemmodules.internal}:\${run.modulepath}:\${dist.jar}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1334	<arg value="--add-modules"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1335	<arg value="\${jlink.add.modules}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1336	<arg if:set="jlink.do.strip.internal" value="--strip-debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1337	<arg if:set="jlink.do.launcher.internal" value="--launcher"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1338	<arg if:set="jlink.do.launcher.internal" value="\${jlink.launcher.name}=\${module.name}/\${main.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1339	<arg if:set="jlink.do.additionalparam.internal" line="\${jlink.additionalparam}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1340	<arg value="--output"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1341	<arg value="\${dist.jlink.output}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1342	</exec>	Cierra el elemento XML abierto anteriormente.
1343	</target>	Cierra el elemento XML abierto anteriormente.
1344	<target name="-post-deploy">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1345	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1346	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1347	</target>	Cierra el elemento XML abierto anteriormente.

1348	<target depends="-do-jar, -post-jar, -pre-deploy, -do-deploy, -post-deploy" name="deploy"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1349	<!--	Comentario XML para humanos o para NetBeans.
1350	=====	Contenido de configuración o documentación que completa el archivo.
1351	EXECUTION SECTION	Contenido de configuración o documentación que completa el archivo.
1352	=====	Contenido de configuración o documentación que completa el archivo.
1353	-->	Comentario XML para humanos o para NetBeans.
1354	<target depends="init,compile" description="Run a main class." name="run">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1355	<j2seproject1:java>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1356	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1357	<arg line="\${application.args}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1358	</customize>	Cierra el elemento XML abierto anteriormente.
1359	</j2seproject1:java>	Cierra el elemento XML abierto anteriormente.
1360	</target>	Cierra el elemento XML abierto anteriormente.
1361	<target name="-do-not-recompile">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1362	<property name="javac.includes.binary" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1363	</target>	Cierra el elemento XML abierto anteriormente.
1364	<target depends="init,compile-single" name="run-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1365	<fail unless="run.class">Must select one file in the IDE or set run.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1366	<j2seproject1:java classname="\${run.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1367	</target>	Cierra el elemento XML abierto anteriormente.
1368	<target depends="init,compile-test-single" name="run-test-with-main">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1369	<fail unless="run.class">Must select one file in the IDE or set run.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1370	<j2seproject1:java classname="\${run.class}" classpath="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1371	</target>	Cierra el elemento XML abierto anteriormente.
1372	<!--	Comentario XML para humanos o para NetBeans.
1373	=====	Contenido de configuración o documentación que completa el archivo.
1374	DEBUGGING SECTION	Contenido de configuración o documentación que completa el archivo.
1375	=====	Contenido de configuración o documentación que completa el archivo.
1376	-->	Comentario XML para humanos o para NetBeans.
1377	<target depends="init" if="netbeans.home" name="-debug-start-debugger">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1378	<j2seproject1:nbjpdastart name="\${debug.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1379	</target>	Cierra el elemento XML abierto anteriormente.
1380	<target depends="init" if="netbeans.home" name="-debug-start-debugger-main-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1381	<j2seproject1:nbjpdastart classpath="\${debug.test.classpath}" name="\${debug.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1382	</target>	Cierra el elemento XML abierto anteriormente.
1383	<target depends="init,compile" name="-debug-start-debuggee">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1384	<j2seproject3:debug>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1385	<customizeDebuggee>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1386	<arg line="\${application.args}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1387	</customizeDebuggee>	Cierra el elemento XML abierto anteriormente.
1388	</j2seproject3:debug>	Cierra el elemento XML abierto anteriormente.
1389	</target>	Cierra el elemento XML abierto anteriormente.
1390	<target depends="init,compile, -debug-start-debugger, -debug-start-debuggee" description="Debug project in IDE" if="netbeans.home" name="debug"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1391	<target depends="init" if="netbeans.home" name="-debug-start-debugger-stepinto">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1392	<j2seproject1:nbjpdastart stopclassname="\${main.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1393	</target>	Cierra el elemento XML abierto anteriormente.
1394	<target depends="init,compile,-debug-start-debugger-stepinto,-debug-start-debuggee" if="netbeans.home" name="debug-stepinto"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1395	<target depends="init,compile-single" if="netbeans.home" name="-debug-start-debuggee-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1396	<fail unless="debug.class">Must select one file in the IDE or set debug.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1397	<j2seproject3:debug classname="\${debug.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1398	</target>	Cierra el elemento XML abierto anteriormente.
1399	<target depends="init,compile-single,-debug-start-debugger,-debug-start-debuggee-single" if="netbeans.home" name="debug-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1400	<target depends="init,compile-test-single" if="netbeans.home" name="-debug-start-debuggee-main-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1401	<fail unless="debug.class">Must select one file in the IDE or set debug.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1402	<j2seproject3:debug classname="\${debug.class}" classpath="\${debug.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1403	</target>	Cierra el elemento XML abierto anteriormente.
1404	<target depends="init,compile-test-single,-debug-start-debugger-main-test,-debug-start-debuggee-main-test" if="netbeans.home" name="debug-test-with-main"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1405	<target depends="init" name="-pre-debug-fix">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1406	<fail unless="fix.includes">Must set fix.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1407	<property name="javac.includes" value="\${fix.includes}.java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1408	</target>	Cierra el elemento XML abierto anteriormente.
1409	<target depends="init,-pre-debug-fix,compile-single" if="netbeans.home" name="-do-debug-fix">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1410	<j2seproject1:nbjpdareload/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1411	</target>	Cierra el elemento XML abierto anteriormente.
1412	<target depends="init,-pre-debug-fix,-do-debug-fix" if="netbeans.home" name="debug-fix"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1413	<!--	Comentario XML para humanos o para NetBeans.
1414	=====	Contenido de configuración o documentación que completa el archivo.
1415	PROFILING SECTION	Contenido de configuración o documentación que completa el archivo.
1416	=====	Contenido de configuración o documentación que completa el archivo.
1417	-->	Comentario XML para humanos o para NetBeans.
1418	<!--	Comentario XML para humanos o para NetBeans.
1419	pre NB7.2 profiler integration	Contenido de configuración o documentación que completa el archivo.
1420	-->	Comentario XML para humanos o para NetBeans.
1421	<target depends="profile-init,compile" description="Profile a project in the IDE." if="profiler.info.jvmargs.agent" name="-profile-pre72">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1422	<fail unless="netbeans.home">This target only works when run from inside the NetBeans IDE.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1423	<nbprofiledirect>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1424	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1425	<path path="\${run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1426	</classpath>	Cierra el elemento XML abierto anteriormente.
1427	</nbprofiledirect>	Cierra el elemento XML abierto anteriormente.
1428	<profile/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1429	</target>	Cierra el elemento XML abierto anteriormente.
1430	<target depends="profile-init,compile-single" description="Profile a selected class in the IDE." if="profile.r.info.jvmargs.agent" name="-profile-single-pre72">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1431	<fail unless="profile.class">Must select one file in the IDE or set profile.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1432	<fail unless="netbeans.home">This target only works when run from inside the NetBeans IDE.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1433	<nbprofiledirect>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1434	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1435	<path path="\${run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1436	</classpath>	Cierra el elemento XML abierto anteriormente.
1437	</nbprofiledirect>	Cierra el elemento XML abierto anteriormente.
1438	<profile classname="\${profile.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1439	</target>	Cierra el elemento XML abierto anteriormente.
1440	<target depends="profile-init,compile-single" if="profiler.info.jvmargs.agent" name="-profile-applet-pre72">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1441	<fail unless="netbeans.home">This target only works when run from inside the NetBeans IDE.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1442	<nbprofiledirect>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1443	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1444	<path path="\${run.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1445	</classpath>	Cierra el elemento XML abierto anteriormente.
1446	</nbprofiledirect>	Cierra el elemento XML abierto anteriormente.
1447	<profile classname="sun.applet.AppletViewer">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1448	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1449	<arg value="\${applet.url}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1450	</customize>	Cierra el elemento XML abierto anteriormente.
1451	</profile>	Cierra el elemento XML abierto anteriormente.
1452	</target>	Cierra el elemento XML abierto anteriormente.
1453	<target depends="-init-macrodéf-junit,profile-init,compile-test-single" if="profiler.info.jvmargs.agent" name="-profile-test-single-pre72">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1454	<fail unless="netbeans.home">This target only works when run from inside the NetBeans IDE.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1455	<nbprofiledirect>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1456	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1457	<path path="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1458	</classpath>	Cierra el elemento XML abierto anteriormente.
1459	</nbprofiledirect>	Cierra el elemento XML abierto anteriormente.
1460	<j2seproject3:junit excludes="\${excludes}" includes="\${includes}" testincludes="\${profile.class}" testmethods="">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1461	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1462	<jvmarg value="-agentlib:jdwp=transport=\${debug-transport},address=\${jpda.address}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1463	<env key="\${profiler.info.pathvar}" path="\${profiler.info.agentpath}:\${profiler.current.path}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1464	<jvmarg value="\${profiler.info.jvmargs.agent}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1465	<jvmarg line="\${profiler.info.jvmargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1466	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1467	<path path="\${run.test.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1468	</classpath>	Cierra el elemento XML abierto anteriormente.
1469	</customize>	Cierra el elemento XML abierto anteriormente.
1470	</j2seproject3:junit>	Cierra el elemento XML abierto anteriormente.
1471	</target>	Cierra el elemento XML abierto anteriormente.
1472	<!--	Comentario XML para humanos o para NetBeans.
1473	end of pre NB72 profiling section	Contenido de configuración o documentación que completa el archivo.
1474	-->	Comentario XML para humanos o para NetBeans.
1475	<target if="netbeans.home" name="-profile-check">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1476	<condition property="profiler.configured">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1477	<or>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1478	<contains casesensitive="true" string="{run.jvmargs.ide}" substring="-agentpath:"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1479	<contains casesensitive="true" string="{run.jvmargs.ide}" substring="-javaagent:"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1480	</or>	Cierra el elemento XML abierto anteriormente.
1481	</condition>	Cierra el elemento XML abierto anteriormente.
1482	</target>	Cierra el elemento XML abierto anteriormente.
1483	<target depends="-profile-check,-profile-pre72" description="Profile a project in the IDE." if="profiler.configured" name="profile" unless="profiler.info.jvmargs.agent">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1484	<startprofiler/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1485	<antcall target="run"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1486	</target>	Cierra el elemento XML abierto anteriormente.
1487	<target depends="-profile-check,-profile-single-pre72" description="Profile a selected class in the IDE." if="profiler.configured" name="profile-single" unless="profiler.info.jvmargs.agent">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1488	<fail unless="run.class">Must select one file in the IDE or set run.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1489	<startprofiler/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1490	<antcall target="run-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1491	</target>	Cierra el elemento XML abierto anteriormente.
1492	<target depends="-profile-test-single-pre72" description="Profile a selected test in the IDE." name="profile-test-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1493	<target depends="-profile-check" description="Profile a selected test in the IDE." if="profiler.configured" name="profile-test" unless="profiler.info.jvmargs">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1494	<fail unless="test.includes">Must select some files in the IDE or set test.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1495	<startprofiler/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1496	<antcall target="test-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1497	</target>	Cierra el elemento XML abierto anteriormente.
1498	<target depends="-profile-check" description="Profile a selected class in the IDE." if="profiler.configured" name="profile-test-with-main">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1499	<fail unless="run.class">Must select one file in the IDE or set run.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1500	<startprofiler/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1501	<antcall target="run-test-with-main"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1502	</target>	Cierra el elemento XML abierto anteriormente.
1503	<target depends="-profile-check,-profile-applet-pre72" if="profiler.configured" name="profile-applet" unless="profiler.info.jvmargs.agent">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1504	<fail unless="applet.url">Must select one file in the IDE or set applet.url</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1505	<startprofiler/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1506	<antcall target="run-applet"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1507	</target>	Cierra el elemento XML abierto anteriormente.
1508	<!--	Comentario XML para humanos o para NetBeans.
1509	=====	Contenido de configuración o documentación que completa el archivo.
1510	JAVADOC SECTION	Contenido de configuración o documentación que completa el archivo.
1511	=====	Contenido de configuración o documentación que completa el archivo.
1512	-->	Comentario XML para humanos o para NetBeans.
1513	<target depends="init" if="have.sources" name="-javadoc-build">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1514	<mkdir dir="{dist.javadoc.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1515	<condition else="" property="javadoc.endorsed.classpath.cmd.line.arg" value="-J\${endorsed.classpath.cmd.line.arg}">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1516	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1517	<isset property="endorsed.classpath.cmd.line.arg"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1518	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1519	<equals arg1="{endorsed.classpath.cmd.line.arg}" arg2=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1520	</not>	Cierra el elemento XML abierto anteriormente.
1521	</and>	Cierra el elemento XML abierto anteriormente.
1522	</condition>	Cierra el elemento XML abierto anteriormente.
1523	<condition else="" property="bug5101868workaround" value="*.java">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1524	<matches pattern="1\.[56](\..*)?" string="{java.version}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1525	</condition>	Cierra el elemento XML abierto anteriormente.
1526	<condition else="" property="javadoc.html5.cmd.line.arg" value="-html5">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1527	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1528	<isset property="javadoc.html5"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1529	<available file="{jdk.home}{file.separator}lib{file.separator}jrt-fs.jar"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1530	</and>	Cierra el elemento XML abierto anteriormente.
1531	</condition>	Cierra el elemento XML abierto anteriormente.
1532	<javadoc additionalparam="-J-Dfile.encoding={file.encoding} {javadoc.additionalparam}" author="{javadoc.author}" charset="UTF-8" destdir="{dist.javadoc.dir}" docencoding="UTF-8" encoding="{javadoc.encoding.used}" failonerror="true" noindex="{javadoc.noindex}" nonavbar="{javadoc.nonavbar}" notree="{javadoc.notree}" private="{javadoc.private}" source="{javac.source}" splitindex="{javadoc.splitindex}" use="{javadoc.use}" useexternalfile="true" version="{javadoc.version}" windowtitle="{javadoc.windowtitle}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1533	<classpath>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1534	<path path="{javac.classpath}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1535	</classpath>	Cierra el elemento XML abierto anteriormente.
1536	<fileset dir="{src.dir}" excludes="{bug5101868workaround},{excludes}" includes="{includes}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1537	<filename name="**/*.java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1538	</fileset>	Cierra el elemento XML abierto anteriormente.
1539	<fileset dir="{build.generated.sources.dir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1540	<include name="**/*.java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1541	<exclude name="*.java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1542	</fileset>	Cierra el elemento XML abierto anteriormente.
1543	<arg line="{javadoc.endorsed.classpath.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1544	<arg line="{javadoc.html5.cmd.line.arg}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1545	</javadoc>	Cierra el elemento XML abierto anteriormente.
1546	<copy todir="{dist.javadoc.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1547	<fileset dir="{src.dir}" excludes="{excludes}" includes="{includes}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1548	<filename name="**/doc-files/**"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1549	</fileset>	Cierra el elemento XML abierto anteriormente.
1550	<fileset dir="{build.generated.sources.dir}" erroronmissingdir="false">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1551	<include name="**/doc-files/**"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1552	</fileset>	Cierra el elemento XML abierto anteriormente.
1553	</copy>	Cierra el elemento XML abierto anteriormente.
1554	</target>	Cierra el elemento XML abierto anteriormente.
1555	<target depends="init,-javadoc-build" if="netbeans.home" name="-javadoc-browse" unless="no.javadoc.preview">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1556	<nbbrowse file="{dist.javadoc.dir}/index.html"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1557	</target>	Cierra el elemento XML abierto anteriormente.

1558	<target depends="init,-javadoc-build,-javadoc-browse" description="Build Javadoc." name="javadoc"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1559	<!--	Comentario XML para humanos o para NetBeans.
1560	=====	Contenido de configuración o documentación que completa el archivo.
1561	TEST COMPILATION SECTION	Contenido de configuración o documentación que completa el archivo.
1562	=====	Contenido de configuración o documentación que completa el archivo.
1563	-->	Comentario XML para humanos o para NetBeans.
1564	<target depends="init,compile" if="have.tests" name="-pre-pre-compile-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1565	<mkdir dir="\${build.test.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1566	</target>	Cierra el elemento XML abierto anteriormente.
1567	<target name="-pre-compile-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1568	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1569	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1570	</target>	Cierra el elemento XML abierto anteriormente.
1571	<target depends="-init-source-module-properties" if="named.module.internal" name="-init-test-javac-module-properties-with-module">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1572	<j2seproject3:modulename property="test.module.name" sourcepath="\${test.src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1573	<condition else="\${empty.dir}" property="javac.test.sourcepath" value="\${test.src.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1574	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1575	<isset property="test.module.name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1576	<length length="0" string="\${test.module.name}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1577	</and>	Cierra el elemento XML abierto anteriormente.
1578	</condition>	Cierra el elemento XML abierto anteriormente.
1579	<condition else="--patch-module \${module.name}=\${test.src.dir} --add-reads \${module.name}=ALL-UNNAMED" property="javac.test.compilerargs" value="--add-reads \${test.module.name}=ALL-UNNAMED">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1580	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1581	<isset property="test.module.name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1582	<length length="0" string="\${test.module.name}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1583	</and>	Cierra el elemento XML abierto anteriormente.
1584	</condition>	Cierra el elemento XML abierto anteriormente.
1585	</target>	Cierra el elemento XML abierto anteriormente.
1586	<target depends="-init-source-module-properties" if="named.module.internal" name="-init-test-run-module-properties">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1587	<condition else="\${module.name}" property="run.test.addexport.source.module.internal" value="\${test.module.name}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1588	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1589	<isset property="test.module.name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1590	<length length="0" string="\${test.module.name}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1591	</and>	Cierra el elemento XML abierto anteriormente.
1592	</condition>	Cierra el elemento XML abierto anteriormente.
1593	<fileset dir="\${build.test.classes.dir}" id="run.test.packages.internal" includes="**/*.class"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1594	<property location="\${build.test.classes.dir}" name="build.test.classes.dir.abs.internal"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1595	<pathconvert pathsep=" " property="run.test.addexports.internal" refid="run.test.packages.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1596	<chainedmapper>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1597	<regexmapper from="^(.*)\Q\${file.separator}\E.*\.class\$\$" to="\1"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1598	<filtermapper>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1599	<uniqfilter/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1600	<replacestring from="\${build.test.classes.dir.abs.internal}" to=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1601	</filtermapper>	Cierra el elemento XML abierto anteriormente.
1602	<cutdirsmapper dirs="1"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1603	<packagemapper from="" to="--add-exports \${run.test.addexport.source.module.internal}/*=ALL-UNNAMED"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1604	</chainedmapper>	Cierra el elemento XML abierto anteriormente.
1605	</pathconvert>	Cierra el elemento XML abierto anteriormente.
1606	<condition else="--patch-module \${module.name}=\${build.test.classes.dir} --add-modules \${module.name} --add-reads \${module.name}=ALL-UNNAMED \${run.test.addexports.internal}" property="run.test.jvmargs" value="--add-modules \${test.module.name} --add-reads \${test.module.name}=ALL-UNNAMED \${run.test.addexports.internal}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1607	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1608	<isset property="test.module.name"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1609	<length length="0" string="\${test.module.name}" when="greater"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1610	</and>	Cierra el elemento XML abierto anteriormente.
1611	</condition>	Cierra el elemento XML abierto anteriormente.
1612	</target>	Cierra el elemento XML abierto anteriormente.
1613	<target depends="-init-source-module-properties" name="-init-test-module-properties-without-module" unless="named.module.internal">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1614	<property name="javac.test.sourcepath" value="\${empty.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1615	<property name="javac.test.compilerargs" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1616	<property name="run.test.jvmargs" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1617	</target>	Cierra el elemento XML abierto anteriormente.
1618	<target depends="-init-test-javac-module-properties-with-module,-init-test-module-properties-without-module" name="-init-test-module-properties"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1619	<target if="do.depend.true" name="-compile-test-depend">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1620	<j2seproject3:depend classpath="\${javac.test.classpath}" destdir="\${build.test.classes.dir}" srcdir="\${test.src.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1621	</target>	Cierra el elemento XML abierto anteriormente.
1622	<target depends="init,deps-jar,compile,-init-test-module-properties,-pre-pre-compile-test,-pre-compile-test,-compile-test-depend" if="have.tests" name="-do-compile-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1623	<j2seproject3:javac apgeneratedsrcdir="\${build.test.classes.dir}" classpath="\${javac.test.classpath}" debug="true" destdir="\${build.test.classes.dir}" modulepath="\${javac.test.modulepath}" processorpath="\${javac.test.processorpath}" sourcepath="\${javac.test.sourcepath}" srcdir="\${test.src.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1624	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1625	<compilerarg line="\${javac.test.compilerargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1626	</customize>	Cierra el elemento XML abierto anteriormente.
1627	</j2seproject3:javac>	Cierra el elemento XML abierto anteriormente.
1628	<copy todir="\${build.test.classes.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1629	<fileset dir="\${test.src.dir}" excludes="\${build.classes.excludes},\${excludes}" includes="\${includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1630	</copy>	Cierra el elemento XML abierto anteriormente.
1631	</target>	Cierra el elemento XML abierto anteriormente.
1632	<target name="-post-compile-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1633	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1634	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1635	</target>	Cierra el elemento XML abierto anteriormente.
1636	<target depends="init,compile,-pre-pre-compile-test,-pre-compile-test,-do-compile-test,-post-compile-test" name="compile-test"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1637	<target name="-pre-compile-test-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1638	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1639	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1640	</target>	Cierra el elemento XML abierto anteriormente.
1641	<target depends="init,deps-jar,compile,-init-test-module-properties,-pre-pre-compile-test,-pre-compile-test-single" if="have.tests" name="-do-compile-test-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1642	<fail unless="javac.includes">Must select some files in the IDE or set javac.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1643	<j2seproject3:force-recompile destdir="\${build.test.classes.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1644	<j2seproject3:javac apgeneratedsrcdir="\${build.test.classes.dir}" classpath="\${javac.test.classpath}" debug="true" destdir="\${build.test.classes.dir}" excludes="" includes="\${javac.includes}, module-info.java" modulepath="\${javac.test.modulepath}" processorpath="\${javac.test.processorpath}" sourcepath="\${test.src.dir}" srcdir="\${test.src.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1645	<customize>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1646	<compilerarg line="\${javac.test.compilerargs}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1647	</customize>	Cierra el elemento XML abierto anteriormente.
1648	</j2seproject3:javac>	Cierra el elemento XML abierto anteriormente.
1649	<copy todir="\${build.test.classes.dir}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1650	<fileset dir="\${test.src.dir}" excludes="\${build.classes.excludes},\${excludes}" includes="\${includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1651	</copy>	Cierra el elemento XML abierto anteriormente.
1652	</target>	Cierra el elemento XML abierto anteriormente.
1653	<target name="-post-compile-test-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1654	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1655	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1656	</target>	Cierra el elemento XML abierto anteriormente.
1657	<target depends="init,compile,-pre-pre-compile-test,-pre-compile-test-single,-do-compile-test-single,-post-compile-test-single" name="compile-test-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1658	<!--	Comentario XML para humanos o para NetBeans.
1659	=====	Contenido de configuración o documentación que completa el archivo.
1660	TEST EXECUTION SECTION	Contenido de configuración o documentación que completa el archivo.
1661	=====	Contenido de configuración o documentación que completa el archivo.
1662	-->	Comentario XML para humanos o para NetBeans.
1663	<target depends="init" if="have.tests" name="-pre-test-run">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1664	<mkdir dir="\${build.test.results.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1665	</target>	Cierra el elemento XML abierto anteriormente.
1666	<target depends="init,compile-test,-init-test-run-module-properties,-pre-test-run" if="have.tests" name="-do-test-run">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1667	<j2seproject3:test includes="\${includes}" testincludes="**/*Test.java"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1668	</target>	Cierra el elemento XML abierto anteriormente.
1669	<target depends="init,compile-test,-pre-test-run,-do-test-run" if="have.tests" name="-post-test-run">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1670	<fail if="tests.failed" unless="ignore.failing.tests">Some tests failed; see details above.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1671	</target>	Cierra el elemento XML abierto anteriormente.
1672	<target depends="init" if="have.tests" name="test-report">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1673	<target depends="init" if="netbeans.home+have.tests" name="-test-browse">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1674	<target depends="init,compile-test,-pre-test-run,-do-test-run,test-report,-post-test-run,-test-browse" description="Run unit tests." name="test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1675	<target depends="init" if="have.tests" name="-pre-test-run-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.

1676	<mkdir dir="\${build.test.results.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1677	</target>	Cierra el elemento XML abierto anteriormente.
1678	<target depends="init,compile-test-single,-init-test-run-module-properties,-pre-test-run-single" if="have.tests" name="-do-test-run-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1679	<fail unless="test.includes">Must select some files in the IDE or set test.includes</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1680	<j2seproject3:test excludes="" includes="\${test.includes}" testincludes="\${test.includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1681	</target>	Cierra el elemento XML abierto anteriormente.
1682	<target depends="init,compile-test-single,-pre-test-run-single,-do-test-run-single" if="have.tests" name="-post-test-run-single">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1683	<fail if="tests.failed" unless="ignore.failing.tests">Some tests failed; see details above.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1684	</target>	Cierra el elemento XML abierto anteriormente.
1685	<target depends="init,compile-test-single,-init-test-run-module-properties,-pre-test-run-single,-do-test-run-single,-post-test-run-single" description="Run single unit test." name="test-single"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1686	<target depends="init,compile-test-single,-pre-test-run-single" if="have.tests" name="-do-test-run-single-method">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1687	<fail unless="test.class">Must select some files in the IDE or set test.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1688	<fail unless="test.method">Must select some method in the IDE or set test.method</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1689	<j2seproject3:test excludes="" includes="\${javac.includes}" testincludes="\${test.class}" testmethods="\${test.method}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1690	</target>	Cierra el elemento XML abierto anteriormente.
1691	<target depends="init,compile-test-single,-pre-test-run-single,-do-test-run-single-method" if="have.tests" name="-post-test-run-single-method">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1692	<fail if="tests.failed" unless="ignore.failing.tests">Some tests failed; see details above.</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1693	</target>	Cierra el elemento XML abierto anteriormente.
1694	<target depends="init,compile-test-single,-init-test-run-module-properties,-pre-test-run-single,-do-test-run-single-method,-post-test-run-single-method" description="Run single unit test." name="test-single-method"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1695	<!--	Comentario XML para humanos o para NetBeans.
1696	=====	Contenido de configuración o documentación que completa el archivo.
1697	TEST DEBUGGING SECTION	Contenido de configuración o documentación que completa el archivo.
1698	=====	Contenido de configuración o documentación que completa el archivo.
1699	-->	Comentario XML para humanos o para NetBeans.
1700	<target depends="init,compile-test-single,-pre-test-run-single" if="have.tests" name="-debug-start-debuggee-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1701	<fail unless="test.class">Must select one file in the IDE or set test.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1702	<j2seproject3:test-debug excludes="" includes="\${javac.includes}" testClass="\${test.class}" testincludes="\${javac.includes}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1703	</target>	Cierra el elemento XML abierto anteriormente.
1704	<target depends="init,compile-test-single,-pre-test-run-single" if="have.tests" name="-debug-start-debuggee-test-method">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1705	<fail unless="test.class">Must select one file in the IDE or set test.class</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1706	<fail unless="test.method">Must select some method in the IDE or set test.method</fail>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1707	<j2seproject3:test-debug excludes="" includes="\${javac.includes}" testClass="\${test.class}" testMethod="\${test.method}" testincludes="\${test.class}" testmethods="\${test.method}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1708	</target>	Cierra el elemento XML abierto anteriormente.
1709	<target depends="init,compile-test" if="netbeans.home+have.tests" name="-debug-start-debugger-test">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1710	<j2seproject1:nbjpdastart classpath="\${debug.test.classpath}" name="\${test.class}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1711	</target>	Cierra el elemento XML abierto anteriormente.
1712	<target depends="init,compile-test-single,-init-test-run-module-properties,-debug-start-debugger-test,-debug-start-debuggee-test" name="debug-test"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1713	<code><target depends="init,compile-test-single,-init-test-run-module-properties,-debug-start-debugger-test,-debug-start-debuggee-test-method" name="debug-test-method"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1714	<code><target depends="debug-test-method" name="debug-single-method"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1715	<code><target depends="init,-pre-debug-fix,compile-test-single" if="netbeans.home" name="-do-debug-fix-test"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1716	<code><j2seproject1:nbjpdareload dir="\${build.test.classes.dir}"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1717	<code></target></code>	Cierra el elemento XML abierto anteriormente.
1718	<code><target depends="init,-pre-debug-fix,-do-debug-fix-test" if="netbeans.home" name="debug-fix-test"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1719	<code><!--</code>	Comentario XML para humanos o para NetBeans.
1720	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1721	<code>APPLET EXECUTION SECTION</code>	Contenido de configuración o documentación que completa el archivo.
1722	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1723	<code>--></code>	Comentario XML para humanos o para NetBeans.
1724	<code><target depends="init,compile-single" name="run-applet"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1725	<code><fail unless="applet.url">Must select one file in the IDE or set applet.url</fail></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1726	<code><j2seproject1:java classname="sun.applet.AppletViewer"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1727	<code><customize></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1728	<code><arg value="\${applet.url}"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1729	<code></customize></code>	Cierra el elemento XML abierto anteriormente.
1730	<code></j2seproject1:java></code>	Cierra el elemento XML abierto anteriormente.
1731	<code></target></code>	Cierra el elemento XML abierto anteriormente.
1732	<code><!--</code>	Comentario XML para humanos o para NetBeans.
1733	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1734	<code>APPLET DEBUGGING SECTION</code>	Contenido de configuración o documentación que completa el archivo.
1735	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1736	<code>--></code>	Comentario XML para humanos o para NetBeans.
1737	<code><target depends="init,compile-single" if="netbeans.home" name="-debug-start-debuggee-applet"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1738	<code><fail unless="applet.url">Must select one file in the IDE or set applet.url</fail></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1739	<code><j2seproject3:debug classname="sun.applet.AppletViewer"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1740	<code><customizeDebuggee></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1741	<code><arg value="\${applet.url}"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1742	<code></customizeDebuggee></code>	Cierra el elemento XML abierto anteriormente.
1743	<code></j2seproject3:debug></code>	Cierra el elemento XML abierto anteriormente.
1744	<code></target></code>	Cierra el elemento XML abierto anteriormente.
1745	<code><target depends="init,compile-single,-debug-start-debugger,-debug-start-debuggee-applet" if="netbeans.home" name="debug-applet"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1746	<code><!--</code>	Comentario XML para humanos o para NetBeans.
1747	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1748	<code>CLEANUP SECTION</code>	Contenido de configuración o documentación que completa el archivo.
1749	<code>=====</code>	Contenido de configuración o documentación que completa el archivo.
1750	<code>--></code>	Comentario XML para humanos o para NetBeans.
1751	<code><target name="-deps-clean-init" unless="built-clean.properties"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1752	<code><property location="\${build.dir}/built-clean.properties" name="built-clean.properties"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1753	<code><delete file="\${built-clean.properties}" quiet="true"/></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1754	<code></target></code>	Cierra el elemento XML abierto anteriormente.
1755	<code><target if="already.built.clean.\${basedir}" name="-warn-already-built-clean"></code>	Declara un elemento XML de configuración, con sus atributos si corresponde.

1756	<echo level="warn" message="Cycle detected: Waldonalds was already built"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1757	</target>	Cierra el elemento XML abierto anteriormente.
1758	<target depends="init, -deps-clean-init" name="deps-clean" unless="no.deps">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1759	<mkdir dir="\${build.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1760	<touch file="\${built-clean.properties}" verbose="false"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1761	<property file="\${built-clean.properties}" prefix="already.built.clean."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1762	<antcall target="-warn-already-built-clean"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1763	<propertyfile file="\${built-clean.properties}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1764	<entry key="\${basedir}" value=""/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1765	</propertyfile>	Cierra el elemento XML abierto anteriormente.
1766	</target>	Cierra el elemento XML abierto anteriormente.
1767	<target depends="init" name="-do-clean">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1768	<delete dir="\${build.dir}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1769	<delete dir="\${dist.jlink.output}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1770	<delete dir="\${dist.dir}" followsymlinks="false" includeemptydirs="true"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1771	</target>	Cierra el elemento XML abierto anteriormente.
1772	<target name="-post-clean">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1773	<!-- Empty placeholder for easier customization. -->	Comentario XML para humanos o para NetBeans.
1774	<!-- You can override this target in the ../build.xml file. -->	Comentario XML para humanos o para NetBeans.
1775	</target>	Cierra el elemento XML abierto anteriormente.
1776	<target depends="init,deps-clean,-do-clean,-post-clean" description="Clean build products." name="clean"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1777	<target name="-check-call-dep">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1778	<property file="\${call.built.properties}" prefix="already.built."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1779	<condition property="should.call.dep">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1780	<and>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1781	<not>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1782	<isset property="already.built.\${call.subproject}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1783	</not>	Cierra el elemento XML abierto anteriormente.
1784	<available file="\${call.script}"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1785	</and>	Cierra el elemento XML abierto anteriormente.
1786	</condition>	Cierra el elemento XML abierto anteriormente.
1787	</target>	Cierra el elemento XML abierto anteriormente.
1788	<target depends="-check-call-dep" if="should.call.dep" name="-maybe-call-dep">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1789	<ant antfile="\${call.script}" inheritall="false" target="\${call.target}">	Declara un elemento XML de configuración, con sus atributos si corresponde.
1790	<propertyset>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1791	<propertyref prefix="transfer."/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1792	<mapper from="transfer.*" to="*" type="glob"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
1793	</propertyset>	Cierra el elemento XML abierto anteriormente.
1794	</ant>	Cierra el elemento XML abierto anteriormente.
1795	</target>	Cierra el elemento XML abierto anteriormente.
1796	</project>	Cierra el elemento XML abierto anteriormente.

16.11 nbproject/genfiles.properties

Archivo de propiedades clave=valor utilizado por NetBeans o por la compilación.

Tipo: .properties | Líneas: 8 | Archivo 11 de 157

1 build.xml.data.CRC32=1065cc3c	Asigna un valor a la propiedad build.xml.data.CRC32; NetBeans/Ant la usa al compilar o ejecutar.
2 build.xml.script.CRC32=1104f90c	Asigna un valor a la propiedad build.xml.script.CRC32; NetBeans/Ant la usa al compilar o ejecutar.
3 build.xml.stylesheet.CRC32=f85dc8f2@1.113.0.48	Asigna un valor a la propiedad build.xml.stylesheet.CRC32; NetBeans/Ant la usa al compilar o ejecutar.
4 # This file is used by a NetBeans-based IDE to track changes in generated files such as build-impl.xml.	Comentario o separación dentro del archivo de propiedades.
5 # Do not edit this file. You may delete it but then the IDE will never regenerate such files for you.	Comentario o separación dentro del archivo de propiedades.
6 nbproject/build-impl.xml.data.CRC32=1065cc3c	Asigna un valor a la propiedad nbproject/build-impl.xml.data.CRC32; NetBeans/Ant la usa al compilar o ejecutar.
7 nbproject/build-impl.xml.script.CRC32=cd6dbf2a	Asigna un valor a la propiedad nbproject/build-impl.xml.script.CRC32; NetBeans/Ant la usa al compilar o ejecutar.
8 nbproject/build-impl.xml.stylesheet.CRC32=12e0a6c2@1.113.0.48	Asigna un valor a la propiedad nbproject/build-impl.xml.stylesheet.CRC32; NetBeans/Ant la usa al compilar o ejecutar.

16.12 nbproject/project.properties

Configuración del proyecto NetBeans: Java 21, librerías, clase principal y rutas de compilación.

Tipo: .properties | Líneas: 100 | Archivo 12 de 157

1	annotation.processing.enabled=true	Asigna un valor a la propiedad annotation.processing.enabled; NetBeans/Ant la usa al compilar o ejecutar.
2	annotation.processing.enabled.in.editor=false	Asigna un valor a la propiedad annotation.processing.enabled.in.editor; NetBeans/Ant la usa al compilar o ejecutar.
3	annotation.processing.processors.list=	Asigna un valor a la propiedad annotation.processing.processors.list; NetBeans/Ant la usa al compilar o ejecutar.
4	annotation.processing.run.all.processors=true	Asigna un valor a la propiedad annotation.processing.run.all.processors; NetBeans/Ant la usa al compilar o ejecutar.
5	annotation.processing.source.output=\${build.generated.sources.dir}/ap-source-output	Asigna un valor a la propiedad annotation.processing.source.output; NetBeans/Ant la usa al compilar o ejecutar.
6	application.title=Waldonalds	Asigna un valor a la propiedad application.title; NetBeans/Ant la usa al compilar o ejecutar.
7	application.vendor=Computacion	Asigna un valor a la propiedad application.vendor; NetBeans/Ant la usa al compilar o ejecutar.
8	build.classes.dir=\${build.dir}/classes	Asigna un valor a la propiedad build.classes.dir; NetBeans/Ant la usa al compilar o ejecutar.
9	build.classes.excludes=**/*.java,**/*.form	Asigna un valor a la propiedad build.classes.excludes; NetBeans/Ant la usa al compilar o ejecutar.
10	# This directory is removed when the project is cleaned:	Comentario o separación dentro del archivo de propiedades.
11	build.dir=build	Asigna un valor a la propiedad build.dir; NetBeans/Ant la usa al compilar o ejecutar.
12	build.generated.dir=\${build.dir}/generated	Asigna un valor a la propiedad build.generated.dir; NetBeans/Ant la usa al compilar o ejecutar.
13	build.generated.sources.dir=\${build.dir}/generated-sources	Asigna un valor a la propiedad build.generated.sources.dir; NetBeans/Ant la usa al compilar o ejecutar.
14	# Only compile against the classpath explicitly listed here:	Comentario o separación dentro del archivo de propiedades.
15	build.sysclasspath=ignore	Asigna un valor a la propiedad build.sysclasspath; NetBeans/Ant la usa al compilar o ejecutar.
16	build.test.classes.dir=\${build.dir}/test/classes	Asigna un valor a la propiedad build.test.classes.dir; NetBeans/Ant la usa al compilar o ejecutar.
17	build.test.results.dir=\${build.dir}/test/results	Asigna un valor a la propiedad build.test.results.dir; NetBeans/Ant la usa al compilar o ejecutar.
18	# Uncomment to specify the preferred debugger connection transport:	Comentario o separación dentro del archivo de propiedades.
19	#debug.transport=dt_socket	Comentario o separación dentro del archivo de propiedades.
20	debug.classpath=\	Asigna un valor a la propiedad debug.classpath; NetBeans/Ant la usa al compilar o ejecutar.
21	\${run.classpath}	Contenido de configuración o documentación que completa el archivo.
22	debug.modulepath=\	Asigna un valor a la propiedad debug.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
23	\${run.modulepath}	Contenido de configuración o documentación que completa el archivo.
24	debug.test.classpath=\	Asigna un valor a la propiedad debug.test.classpath; NetBeans/Ant la usa al compilar o ejecutar.
25	\${run.test.classpath}	Contenido de configuración o documentación que completa el archivo.
26	debug.test.modulepath=\	Asigna un valor a la propiedad debug.test.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
27	\${run.test.modulepath}	Contenido de configuración o documentación que completa el archivo.
28	# Files in build.classes.dir which should be excluded from distribution jar	Comentario o separación dentro del archivo de propiedades.
29	dist.archive.excludes=	Asigna un valor a la propiedad dist.archive.excludes; NetBeans/Ant la usa al compilar o ejecutar.
30	# This directory is removed when the project is cleaned:	Comentario o separación dentro del archivo de propiedades.
31	dist.dir=dist	Asigna un valor a la propiedad dist.dir; NetBeans/Ant la usa al compilar o ejecutar.
32	dist.jar=\${dist.dir}/Waldonalds.jar	Asigna un valor a la propiedad dist.jar; NetBeans/Ant la usa al compilar o ejecutar.
33	dist.javadoc.dir=\${dist.dir}/javadoc	Asigna un valor a la propiedad dist.javadoc.dir; NetBeans/Ant la usa al compilar o ejecutar.
34	dist.jlink.dir=\${dist.dir}/jlink	Asigna un valor a la propiedad dist.jlink.dir; NetBeans/Ant la usa al compilar o ejecutar.
35	dist.jlink.output=\${dist.jlink.dir}/Waldonalds	Asigna un valor a la propiedad dist.jlink.output; NetBeans/Ant la usa al compilar o ejecutar.
36	endorsed.classpath=	Asigna un valor a la propiedad endorsed.classpath; NetBeans/Ant la usa al compilar o ejecutar.
37	excludes=	Asigna un valor a la propiedad excludes; NetBeans/Ant la usa al compilar o ejecutar.
38	file.reference.mysql-connector-j-26.7.0.jar=lib/mysql-connector-j-26.7.0.jar	Asigna un valor a la propiedad file.reference.mysql-connector-j-26.7.0.jar; NetBeans/Ant la usa al compilar o ejecutar.

39	includes=**	Asigna un valor a la propiedad includes; NetBeans/Ant la usa al compilar o ejecutar.
40	jar.compress=false	Asigna un valor a la propiedad jar.compress; NetBeans/Ant la usa al compilar o ejecutar.
41	javac.classpath=\	Asigna un valor a la propiedad javac.classpath; NetBeans/Ant la usa al compilar o ejecutar.
42	\${libs.absolutelayout.classpath}:\	Contenido de configuración o documentación que completa el archivo.
43	\${file.reference.mysql-connector-j-26.7.0.jar}	Contenido de configuración o documentación que completa el archivo.
44	# Space-separated list of extra javac options	Comentario o separación dentro del archivo de propiedades.
45	javac.compilerargs=	Asigna un valor a la propiedad javac.compilerargs; NetBeans/Ant la usa al compilar o ejecutar.
46	javac.deprecation=false	Asigna un valor a la propiedad javac.deprecation; NetBeans/Ant la usa al compilar o ejecutar.
47	javac.external.vm=true	Asigna un valor a la propiedad javac.external.vm; NetBeans/Ant la usa al compilar o ejecutar.
48	javac.modulepath=	Asigna un valor a la propiedad javac.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
49	javac.processormodulepath=	Asigna un valor a la propiedad javac.processormodulepath; NetBeans/Ant la usa al compilar o ejecutar.
50	javac.processorpath=\	Asigna un valor a la propiedad javac.processorpath; NetBeans/Ant la usa al compilar o ejecutar.
51	\${javac.classpath}	Contenido de configuración o documentación que completa el archivo.
52	javac.source=21	Asigna un valor a la propiedad javac.source; NetBeans/Ant la usa al compilar o ejecutar.
53	javac.target=21	Asigna un valor a la propiedad javac.target; NetBeans/Ant la usa al compilar o ejecutar.
54	javac.test.classpath=\	Asigna un valor a la propiedad javac.test.classpath; NetBeans/Ant la usa al compilar o ejecutar.
55	\${javac.classpath}:\	Contenido de configuración o documentación que completa el archivo.
56	\${build.classes.dir}	Contenido de configuración o documentación que completa el archivo.
57	javac.test.modulepath=\	Asigna un valor a la propiedad javac.test.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
58	\${javac.modulepath}	Contenido de configuración o documentación que completa el archivo.
59	javac.test.processorpath=\	Asigna un valor a la propiedad javac.test.processorpath; NetBeans/Ant la usa al compilar o ejecutar.
60	\${javac.test.classpath}	Contenido de configuración o documentación que completa el archivo.
61	javadoc.additionalparam=	Asigna un valor a la propiedad javadoc.additionalparam; NetBeans/Ant la usa al compilar o ejecutar.
62	javadoc.author=false	Asigna un valor a la propiedad javadoc.author; NetBeans/Ant la usa al compilar o ejecutar.
63	javadoc.encoding=\${source.encoding}	Asigna un valor a la propiedad javadoc.encoding; NetBeans/Ant la usa al compilar o ejecutar.
64	javadoc.html5=false	Asigna un valor a la propiedad javadoc.html5; NetBeans/Ant la usa al compilar o ejecutar.
65	javadoc.noindex=false	Asigna un valor a la propiedad javadoc.noindex; NetBeans/Ant la usa al compilar o ejecutar.
66	javadoc.nonavbar=false	Asigna un valor a la propiedad javadoc.nonavbar; NetBeans/Ant la usa al compilar o ejecutar.
67	javadoc.notree=false	Asigna un valor a la propiedad javadoc.notree; NetBeans/Ant la usa al compilar o ejecutar.
68	javadoc.private=false	Asigna un valor a la propiedad javadoc.private; NetBeans/Ant la usa al compilar o ejecutar.
69	javadoc.splitindex=true	Asigna un valor a la propiedad javadoc.splitindex; NetBeans/Ant la usa al compilar o ejecutar.
70	javadoc.use=true	Asigna un valor a la propiedad javadoc.use; NetBeans/Ant la usa al compilar o ejecutar.
71	javadoc.version=false	Asigna un valor a la propiedad javadoc.version; NetBeans/Ant la usa al compilar o ejecutar.
72	javadoc.windowtitle=	Asigna un valor a la propiedad javadoc.windowtitle; NetBeans/Ant la usa al compilar o ejecutar.
73	# The jlink additional root modules to resolve	Comentario o separación dentro del archivo de propiedades.
74	jlink.additionalmodules=	Asigna un valor a la propiedad jlink.additionalmodules; NetBeans/Ant la usa al compilar o ejecutar.
75	# The jlink additional command line parameters	Comentario o separación dentro del archivo de propiedades.
76	jlink.additionalparam=	Asigna un valor a la propiedad jlink.additionalparam; NetBeans/Ant la usa al compilar o ejecutar.
77	jlink.launcher=true	Asigna un valor a la propiedad jlink.launcher; NetBeans/Ant la usa al compilar o ejecutar.
78	jlink.launcher.name=Waldonalds	Asigna un valor a la propiedad jlink.launcher.name; NetBeans/Ant la usa al compilar o ejecutar.
79	main.class=Main.main	Asigna un valor a la propiedad main.class; NetBeans/Ant la usa al compilar o ejecutar.
80	manifest.file=manifest.mf	Asigna un valor a la propiedad manifest.file; NetBeans/Ant la usa al compilar o ejecutar.
81	meta.inf.dir=\${src.dir}/META-INF	Asigna un valor a la propiedad meta.inf.dir; NetBeans/Ant la usa al compilar o ejecutar.
82	mkdist.disabled=false	Asigna un valor a la propiedad mkdist.disabled; NetBeans/Ant la usa al compilar o ejecutar.

83	platform.active=default_platform	Asigna un valor a la propiedad platform.active; NetBeans/Ant la usa al compilar o ejecutar.
84	run.classpath=\	Asigna un valor a la propiedad run.classpath; NetBeans/Ant la usa al compilar o ejecutar.
85	\${javac.classpath}:\	Contenido de configuración o documentación que completa el archivo.
86	\${build.classes.dir}	Contenido de configuración o documentación que completa el archivo.
87	# Space-separated list of JVM arguments used when running the project.	Comentario o separación dentro del archivo de propiedades.
88	# You may also define separate properties like run-sys-prop.name=value instead of -Dname=value.	Comentario o separación dentro del archivo de propiedades.
89	# To set system properties for unit tests define test-sys-prop.name=value:	Comentario o separación dentro del archivo de propiedades.
90	run.jvmargs=	Asigna un valor a la propiedad run.jvmargs; NetBeans/Ant la usa al compilar o ejecutar.
91	run.modulepath=\	Asigna un valor a la propiedad run.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
92	\${javac.modulepath}	Contenido de configuración o documentación que completa el archivo.
93	run.test.classpath=\	Asigna un valor a la propiedad run.test.classpath; NetBeans/Ant la usa al compilar o ejecutar.
94	\${javac.test.classpath}:\	Contenido de configuración o documentación que completa el archivo.
95	\${build.test.classes.dir}	Contenido de configuración o documentación que completa el archivo.
96	run.test.modulepath=\	Asigna un valor a la propiedad run.test.modulepath; NetBeans/Ant la usa al compilar o ejecutar.
97	\${javac.test.modulepath}	Contenido de configuración o documentación que completa el archivo.
98	source.encoding=UTF-8	Asigna un valor a la propiedad source.encoding; NetBeans/Ant la usa al compilar o ejecutar.
99	src.dir=src	Asigna un valor a la propiedad src.dir; NetBeans/Ant la usa al compilar o ejecutar.
100	test.src.dir=test	Asigna un valor a la propiedad test.src.dir; NetBeans/Ant la usa al compilar o ejecutar.

16.13 nbproject/project.xml

Configuración XML utilizada por NetBeans/Ant para describir y construir el proyecto.

Tipo: .xml | Líneas: 18 | Archivo 13 de 157

1	<?xml version="1.0" encoding="UTF-8"?>	Declara un elemento XML de configuración, con sus atributos si corresponde.
2	<project xmlns="http://www.netbeans.org/ns/project/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
3	<type>org.netbeans.modules.java.j2seproject</type>	Declara un elemento XML de configuración, con sus atributos si corresponde.
4	<configuration>	Declara un elemento XML de configuración, con sus atributos si corresponde.
5	<data xmlns="http://www.netbeans.org/ns/j2se-project/3">	Declara un elemento XML de configuración, con sus atributos si corresponde.
6	<name>Waldonalds</name>	Declara un elemento XML de configuración, con sus atributos si corresponde.
7	<source-roots>	Declara un elemento XML de configuración, con sus atributos si corresponde.
8	<root id="src.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
9	</source-roots>	Cierra el elemento XML abierto anteriormente.
10	<test-roots>	Declara un elemento XML de configuración, con sus atributos si corresponde.
11	<root id="test.src.dir"/>	Declara un elemento XML de configuración, con sus atributos si corresponde.
12	</test-roots>	Cierra el elemento XML abierto anteriormente.
13	</data>	Cierra el elemento XML abierto anteriormente.
14	<libraries xmlns="http://www.netbeans.org/ns/ant-project-libraries/1">	Declara un elemento XML de configuración, con sus atributos si corresponde.
15	<definitions>.\lib\nblibraries.properties</definitions>	Declara un elemento XML de configuración, con sus atributos si corresponde.
16	</libraries>	Cierra el elemento XML abierto anteriormente.
17	</configuration>	Cierra el elemento XML abierto anteriormente.
18	</project>	Cierra el elemento XML abierto anteriormente.

16.14 scripts/generar_datos_menu_nuevo.ps1

Herramienta auxiliar generar_datos_menu_nuevo usada para generar o validar datos del proyecto.

Tipo: .ps1 | Líneas: 1446 | Archivo 14 de 157

1 param(	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
2 [Parameter(Mandatory = \$true)]	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
3 [string]\$SourcePath,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
4	Separación visual entre instrucciones de PowerShell.
5 [string]\$OutputPath = "sql/datos_menu_nueva_base.sql"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
6)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
7	Separación visual entre instrucciones de PowerShell.
8 \$ErrorActionPreference = "Stop"	Declara o actualiza una variable de PowerShell.
9	Separación visual entre instrucciones de PowerShell.
10 function Escape-Sql([string]\$Value) {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
11 if (\$null -eq \$Value) { return "" }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
12 return \$Value.Replace("'", "'")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
13 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
14	Separación visual entre instrucciones de PowerShell.
15 function Get-StockType(\$Product) {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
16 if (\$Product.Categoria -eq "Cajita Feliz" -or	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
17 \$Product.Nombre -match "^(Caja Grande Bucket Caja de 24)") {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
18 return "NINGUNO"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
19 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
20 if (\$Product.Categoria -eq "Antojos" -and	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
21 \$Product.Nombre -match "Derretido Tostado") {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
22 return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
23 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
24 if (\$Product.Categoria -eq "Bebidas" -and	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
25 \$Product.Nombre -match "Café Chocolate Té caliente wLFizz wLFrizz") {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
26 return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
27 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
28 if (\$Product.Categoria -eq "Desayunos") {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
29 return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
30 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
31 if (\$Product.Categoria -eq "Almuerzos" -and	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
32 \$Product.Nombre -notmatch "wLNuggets Nuggets Pollo wLCrispy (1 Pieza Dos Piezas 10 Piezas) ^Pollo wLCri	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
33 spy\$") {	
34 return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
35 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
36 if (\$Product.Categoria -eq "Postres" -and	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
37 \$Product.Nombre -match "wLCono wLFlurry Sundae") {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
38 return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
39 }	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

39	if (\$Product.Categoria -eq "WlCafé" -and	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
40	\$Product.Subcategoria -notmatch "Postres[Café en bolsa"] {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
41	return "RECETA"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
42	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
43	return "DIRECTO"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
44	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
45		Separación visual entre instrucciones de PowerShell.
46	\$source = Get-Content -LiteralPath \$SourcePath -Raw	Declara o actualiza una variable de PowerShell.
47	\$pattern = @"\\(SELECT id_categoria FROM categoria WHERE nombre = '([^\r\n]+)'\, \s*((?:' [^\r\n])*)', \s*((?:' [^\r\n])*)', \s*((?:' [^\r\n])*)', \s*([0-9.]+), \s*((?:' [^\r\n])*)', \s*([A-Z_]+), \s*NULL, \s*(TRUE FALSE), \s*([0-9.]+), \s*([0-9.]+), \s*TRUE\)"	Declara o actualiza una variable de PowerShell.
48		Separación visual entre instrucciones de PowerShell.
49	\$products = [regex]::Matches(	Declara o actualiza una variable de PowerShell.
50	\$source,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
51	\$pattern,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
52	[System.Text.RegularExpressions.RegexOptions]::SingleLine	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
53	) ForEach-Object {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
54	[pscustomobject]@{	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
55	Categoria = \$_.Groups[1].Value	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
56	Subcategoria = \$_.Groups[2].Value.Replace("'", "")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
57	Nombre = \$_.Groups[3].Value.Replace("'", "")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
58	Descripcion = \$_.Groups[4].Value.Replace("'", "")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
59	Precio = \$_.Groups[5].Value	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
60	Imagen = \$_.Groups[6].Value	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
61	Horario = \$_.Groups[7].Value	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
62	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
63	} Group-Object Categoria, Nombre ForEach-Object {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
64	# Cuando el archivo viejo repite un producto, se conserva su última versión.	Comentario de PowerShell; no se ejecuta.
65	\$_Group[-1]	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
66	} Sort-Object Categoria, Nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
67		Separación visual entre instrucciones de PowerShell.
68	\$imageCorrections = @{	Declara o actualiza una variable de PowerShell.
69	"Bacon Cheddar WlMelt" = "/Imágenes/HAMBURGUESAS/bacon_cheddar_mcmelt.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
70	"Bucket Pollo WlCrispy Para Tres" = "/Imágenes/PARA COMPARTIR/Bucket Pollo McCrispy ® Para Tres.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
71	"Bucket Pollo WlCrispy Snack" = "/Imágenes/PARA COMPARTIR/Bucket Pollo McCrispy ® Snack.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
72	"Café guatemalteco" = "/Imágenes/wlcafe/cafe_guatemalteco.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
73	"Caja Grande con Postre" = "/Imágenes/para compartir/caja grande con postre (1).png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
74	"Clásica Gourmet" = "/Imágenes/HAMBURGUESAS/clasica_gourmet_res.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
75	"Clásica Gourmet Doble Res Doble" = "/Imágenes/HAMBURGUESAS/clasica_gourmet_doble_res_doble.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
76	"Coca-Cola Zero" = "/Imágenes/BEBIDAS/coca_cola_zero.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
77	"Cuarto de Libra Bacon Doble con Queso" = "/Imágenes/HAMBURGUESAS/cuarto_de_libra_bacon_doble_queso.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
78	"Frappé Caramelo" = "/Imágenes/wlcafe/frappé_caramelo.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
79	"Frappé Vainilla" = "/Imágenes/wlcafe/Frappé Vainilla.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
80	"Pollo WlCrispy Dos Piezas" = "/Imágenes/HAMBURGUESAS/pollowlcrispy _dospiezas.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
81	"Res" = "/Imágenes/HAMBURGUESAS/hamburguesa.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

82	"Té Guatemalteco Bora Bora" = "/Imágenes/wlcafe/te_guatemalteco_borabora.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
83	"Té Guatemalteco Melocotón Mix" = "/Imágenes/wlcafe/te_guatealteco_melocoton_mix.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
84	"Té Guatemalteco Menta Fusión" = "/Imágenes/wlcafe/te_guatemalteco_menta_fusion.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
85	"Té Guatemalteco Vainilla Relax" = "/Imágenes/wlcafe/teguatemalteco_vainilla_relax.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
86	"WlCrispy Chicken Deluxe" = "/Imágenes/HAMBURGUESAS/WlCrispy_Chicken_Deluxe.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
87	"WlFizz A.M." = "/Imágenes/wlcafe/wlfizz_A.M..png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
88	"WlGriddle Tocino Huevo" = "/Imágenes/DESAYUNOS/wlgriddle_tocinohuevo.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
89	"WlMuffin de Doble Huevo" = "/Imágenes/DESAYUNOS/eggwlmuffin_doble_huevo.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
90	"WlMuffin Salchicha Doble y Huevo" = "/Imágenes/DESAYUNOS/wlMuffi_Salchich_Doble_y_Huevo.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
91	"WlMuffin Salchicha y doble huevo" = "/Imágenes/DESAYUNOS/wlMuffin_Salchicha_y_doble_huevo.png"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
92	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
93		Separación visual entre instrucciones de PowerShell.
94	\$productRows = New-Object System.Collections.Generic.List[string]	Declara o actualiza una variable de PowerShell.
95	foreach (\$product in \$products) {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
96	\$stockType = Get-StockType \$product	Declara o actualiza una variable de PowerShell.
97	\$customizable = if (\$stockType -eq "DIRECTO") { "FALSE" } else { "TRUE" }	Declara o actualiza una variable de PowerShell.
98	\$stock = if (\$stockType -eq "DIRECTO") { 100 } else { 0 }	Declara o actualiza una variable de PowerShell.
99	\$minimumStock = if (\$stockType -eq "DIRECTO") { 10 } else { 0 }	Declara o actualiza una variable de PowerShell.
100	\$image = if (\$imageCorrections.ContainsKey(\$product.Nombre)) {	Declara o actualiza una variable de PowerShell.
101	\$imageCorrections[\$product.Nombre]	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
102	} else {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
103	\$product.Imagen -replace "^/imagenes/", "/Imágenes/"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
104	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
105	\$productRows.Add((	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
106	"('{0}', '{1}', '{2}', '{3}', {4}, '{5}', '{6}', '{7}', {8}, {9}, {10})" -f	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
107	(Escape-Sql \$product.Categoria),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
108	(Escape-Sql \$product.Subcategoria),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
109	(Escape-Sql \$product.Nombre),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
110	(Escape-Sql \$product.Descripcion),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
111	\$product.Precio,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
112	(Escape-Sql \$image),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
113	\$product.Horario,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
114	\$stockType,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
115	\$customizable,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
116	\$stock,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
117	\$minimumStock	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
118	))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
119	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
120		Separación visual entre instrucciones de PowerShell.
121	# Productos auxiliares que no estaban en el catálogo viejo pero son necesarios	Comentario de PowerShell; no se ejecuta.
122	# para construir una Cajita o una caja familiar real.	Comentario de PowerShell; no se ejecuta.
123	\$productRows.Add("{'Cajita Feliz', 'Complementos', 'Papas Kids', 'Porción infantil de papas.', 10.00, '/Imágenes/ANTOJOS/papas.png', 'TODO_DIA', 'DIRECTO', FALSE, 200, 20}")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
124	\$productRows.Add("{'Cajita Feliz', 'Complementos', 'Puré de manzana', 'Puré de manzana para menú infantil.', 9.00, '/Imágenes/BEBIDAS/jugo_manzana.png', 'TODO_DIA', 'DIRECTO', FALSE, 150, 15}")	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

125	<code>\$productRows.Add("('Cajita Feliz', 'Complementos', 'Yogur de fresa', 'Yogur de fresa para menú infantil.', 9.00, NULL, 'TODO_DIA', 'DIRECTO', FALSE, 150, 15) ")</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
126	<code>\$productRows.Add("('Cajita Feliz', 'Bebidas', 'Jugo de manzana Kids', 'Jugo de manzana en tamaño infantil.', 8.00, '/Imágenes/BEBIDAS/jugo_manzana.png', 'TODO_DIA', 'DIRECTO', FALSE, 150, 15) ")</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
127	<code>\$productRows.Add("('Cajita Feliz', 'Juguetes', 'Juguete sorpresa', 'Juguete disponible para la promoción infantil.', 12.00, NULL, 'TODO_DIA', 'DIRECTO', FALSE, 200, 20) ")</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
128	<code>\$productRows.Add("('Bebidas', 'Sodas', 'Coca-Cola 1.5 L', 'Bebida familiar para cajas y combos.', 25.00, '/Imágenes/EBIDAS/coca_cola.png', 'TODO_DIA', 'DIRECTO', FALSE, 100, 10) ")</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
129		Separación visual entre instrucciones de PowerShell.
130	<code>\$header = @'</code>	Declara o actualiza una variable de PowerShell.
131	<code>-- =====</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
132	<code>-- DATOS COMPLETOS PARA LA NUEVA BASE WALDONALDS</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
133	<code>-- Generado desde los INSERT antiguos y adaptado al modelo de</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
134	<code>-- recetas, presentaciones, WlMenús, Cajitas y combos.</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
135	<code>-- =====</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
136		Separación visual entre instrucciones de PowerShell.
137	<code>USE waldonalds;</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
138		Separación visual entre instrucciones de PowerShell.
139	<code>START TRANSACTION;</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
140		Separación visual entre instrucciones de PowerShell.
141	<code>-- =====</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
142	<code>-- 1. CATEGORÍAS</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
143	<code>-- =====</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
144		Separación visual entre instrucciones de PowerShell.
145	<code>INSERT INTO categoria (nombre, estado)</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
146	<code>SELECT datos.nombre, TRUE</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
147	<code>FROM (</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
148	<code> SELECT 'Antojos' AS nombre</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
149	<code> UNION ALL SELECT 'Bebidas'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
150	<code> UNION ALL SELECT 'Cajita Feliz'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
151	<code> UNION ALL SELECT 'Desayunos'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
152	<code> UNION ALL SELECT 'Almuerzos'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
153	<code> UNION ALL SELECT 'Postres'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
154	<code> UNION ALL SELECT 'WlCafé'</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
155	<code>) datos</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
156	<code>WHERE NOT EXISTS (</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
157	<code> SELECT 1 FROM categoria c WHERE c.nombre = datos.nombre</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
158	<code>);</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
159		Separación visual entre instrucciones de PowerShell.
160	<code>UPDATE categoria</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
161	<code>SET estado = TRUE</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
162	<code>WHERE nombre IN ('Antojos', 'Bebidas', 'Cajita Feliz', 'Desayunos',</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
163	<code> 'Almuerzos', 'Postres', 'WlCafé');</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
164		Separación visual entre instrucciones de PowerShell.
165	<code>-- =====</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
166	<code>-- 2. PRODUCTOS</code>	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

167	-- Se usa una tabla temporal para evitar duplicados si el archivo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
168	-- se ejecuta más de una vez.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
169	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
170		Separación visual entre instrucciones de PowerShell.
171	DROP TEMPORARY TABLE IF EXISTS carga_producto;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
172		Separación visual entre instrucciones de PowerShell.
173	CREATE TEMPORARY TABLE carga_producto (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
174	categoria VARCHAR(100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
175	subcategoria VARCHAR(100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
176	nombre VARCHAR(150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
177	descripcion VARCHAR(500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
178	precio DECIMAL(12,2),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
179	imagen VARCHAR(500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
180	disponibilidad VARCHAR(20),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
181	tipo_stock VARCHAR(20),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
182	personalizable BOOLEAN,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
183	stock_actual INT,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
184	stock_minimo INT	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
185	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
186		Separación visual entre instrucciones de PowerShell.
187	INSERT INTO carga_producto (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
188	categoria, subcategoria, nombre, descripcion, precio, imagen,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
189	disponibilidad, tipo_stock, personalizable, stock_actual, stock_minimo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
190	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
191	VALUES	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
192	'@	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
193		Separación visual entre instrucciones de PowerShell.
194	\$afterProducts = '@'	Declara o actualiza una variable de PowerShell.
195		Separación visual entre instrucciones de PowerShell.
196	INSERT INTO producto (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
197	id_categoria, nombre, descripcion, precio_base, imagen, subcategoria,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
198	disponibilidad_menu, tipo_stock, personalizable,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
199	stock_actual, stock_minimo, estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
200	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
201	SELECT	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
202	c.id_categoria,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
203	cp.nombre,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
204	cp.descripcion,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
205	cp.precio,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
206	cp.imagen,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
207	cp.subcategoria,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
208	cp.disponibilidad,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
209	cp.tipo_stock,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
210	cp.personalizable,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

211	cp.stock_actual,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
212	cp.stock_minimo,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
213	TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
214	FROM carga_producto cp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
215	INNER JOIN categoria c ON c.nombre = cp.categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
216	WHERE NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
217	SELECT 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
218	FROM producto p	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
219	WHERE p.id_categoria = c.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
220	AND p.nombre = cp.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
221	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
222		Separación visual entre instrucciones de PowerShell.
223	-- Si el producto ya existía, se actualizan sus campos comerciales.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
224	UPDATE producto p	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
225	INNER JOIN categoria c ON c.id_categoria = p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
226	INNER JOIN carga_producto cp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
227	ON cp.categoria = c.nombre AND cp.nombre = p.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
228	SET p.descripcion = cp.descripcion,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
229	p.precio_base = cp.precio,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
230	p.imagen = cp.imagen,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
231	p.subcategoria = cp.subcategoria,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
232	p.disponibilidad_menu = cp.disponibilidad,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
233	p.tipo_stock = cp.tipo_stock,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
234	p.personalizable = cp.personalizable,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
235	p.stock_actual = CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
236	WHEN cp.tipo_stock = 'DIRECTO' AND p.stock_actual = 0	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
237	THEN cp.stock_actual	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
238	ELSE p.stock_actual	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
239	END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
240	p.stock_minimo = cp.stock_minimo,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
241	p.estado = TRUE;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
242		Separación visual entre instrucciones de PowerShell.
243	DROP TEMPORARY TABLE carga_producto;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
244		Separación visual entre instrucciones de PowerShell.
245	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
246	-- 3. INGREDIENTES	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
247	-- Las cantidades están expresadas según unidad_medida.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
248	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
249		Separación visual entre instrucciones de PowerShell.
250	DROP TEMPORARY TABLE IF EXISTS carga_ingredientes;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
251		Separación visual entre instrucciones de PowerShell.
252	CREATE TEMPORARY TABLE carga_ingredientes (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
253	nombre VARCHAR(100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
254	unidad VARCHAR(30),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

255	stock DECIMAL(12,2),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
256	mínimo DECIMAL(12,2)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
257	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
258		Separación visual entre instrucciones de PowerShell.
259	INSERT INTO carga_ingrediente (nombre, unidad, stock, mínimo) VALUES	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
260	('Pan hamburguesa', 'unidad', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
261	('Pan muffin', 'unidad', 800, 80),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
262	('Pan WlGriddle', 'unidad', 800, 80),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
263	('Pan tostado', 'rebanada', 1500, 150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
264	('Tortilla de harina', 'unidad', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
265	('Tortilla de maíz', 'unidad', 1500, 150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
266	('Carne de res', 'unidad', 1500, 150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
267	('Filete de pollo', 'unidad', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
268	('Huevo', 'unidad', 2000, 200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
269	('Salchicha desayuno', 'unidad', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
270	('Jamón', 'rebanada', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
271	('Tocino', 'tira', 1500, 150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
272	('Queso americano', 'rebanada', 2500, 250),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
273	('Queso cheddar', 'rebanada', 1000, 100),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
274	('Lechuga', 'gramo', 20000, 2000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
275	('Tomate', 'rodaja', 3000, 300),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
276	('Cebolla', 'gramo', 12000, 1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
277	('Pepinillo', 'rodaja', 5000, 500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
278	('Ketchup', 'mililitro', 20000, 2000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
279	('Mostaza', 'mililitro', 15000, 1500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
280	('Mayonesa', 'mililitro', 15000, 1500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
281	('Salsa Big Mac', 'mililitro', 10000, 1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
282	('Salsa Big Tasty', 'mililitro', 10000, 1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
283	('Salsa ranch', 'mililitro', 8000, 800),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
284	('Salsa ahumada', 'mililitro', 8000, 800),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
285	('Guacamole', 'gramo', 8000, 800),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
286	('Pico de gallo', 'gramo', 8000, 800),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
287	('Frijol', 'gramo', 15000, 1500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
288	('Loroco', 'gramo', 5000, 500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
289	('Vegetales mixtos', 'gramo', 12000, 1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
290	('Plátano', 'gramo', 10000, 1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
291	('Crema', 'mililitro', 10000, 1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
292	('Masa de hot cake', 'porcion', 1500, 150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
293	('Mantequilla', 'gramo', 8000, 800),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
294	('Miel', 'mililitro', 10000, 1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
295	('Café molido', 'gramo', 20000, 2000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
296	('Agua preparada', 'mililitro', 100000, 10000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
297	('Leche', 'mililitro', 80000, 8000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
298	('Chocolate en polvo', 'gramo', 15000, 1500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

299	('Bolsa de té','unidad',2000,200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
300	('Mezcla chai','gramo',10000,1000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
301	('Hielo','gramo',100000,10000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
302	('Jarabe caramelo','mililitro',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
303	('Jarabe chocolate','mililitro',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
304	('Jarabe fresa','mililitro',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
305	('Jarabe vainilla','mililitro',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
306	('Jarabe frutal','mililitro',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
307	('Base horchata','mililitro',15000,1500),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
308	('Pulpa de mango','gramo',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
309	('Pulpa de berries','gramo',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
310	('Helado de vainilla','gramo',50000,5000),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
311	('Cono','unidad',1500,150),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
312	('Galleta Oreo','gramo',12000,1200),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
313	('Chocolate M&M','gramo',12000,1200);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
314		Separación visual entre instrucciones de PowerShell.
315	INSERT INTO ingrediente (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
316	nombre, unidad_medida, stock_actual, stock_minimo, estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
317	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
318	SELECT ci.nombre, ci.unidad, ci.stock, ci.minimo, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
319	FROM carga_ingrediente ci	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
320	WHERE NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
321	SELECT 1 FROM ingrediente i WHERE i.nombre = ci.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
322	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
323		Separación visual entre instrucciones de PowerShell.
324	UPDATE ingrediente i	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
325	INNER JOIN carga_ingrediente ci ON ci.nombre = i.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
326	SET i.unidad_medida = ci.unidad,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
327	i.stock_actual = CASE WHEN i.stock_actual = 0 THEN ci.stock ELSE i.stock_actual END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
328	i.stock_minimo = ci.minimo,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
329	i.estado = TRUE;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
330		Separación visual entre instrucciones de PowerShell.
331	DROP TEMPORARY TABLE carga_ingrediente;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
332		Separación visual entre instrucciones de PowerShell.
333	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
334	-- 4. RECETAS	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
335	-- Se cargan primero en una tabla temporal para fusionar reglas	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
336	-- repetidas antes de insertarlas en producto_ingrediente.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
337	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
338		Separación visual entre instrucciones de PowerShell.
339	DROP TEMPORARY TABLE IF EXISTS carga_receta;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
340		Separación visual entre instrucciones de PowerShell.
341	CREATE TEMPORARY TABLE carga_receta (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
342	id_producto INT,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

343	id_ingrediente INT,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
344	cantidad DECIMAL(12,2),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
345	permite_quitar BOOLEAN,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
346	permite_extra BOOLEAN,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
347	cantidad_extra DECIMAL(12,2),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
348	precio_extra DECIMAL(12,2),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
349	max_extras INT	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
350	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
351		Separación visual entre instrucciones de PowerShell.
352	-- Tostados y derretidos.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
353	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
354	SELECT p.id_producto,i.id_ingrediente,2,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
355	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
356	JOIN ingrediente i ON i.nombre='Pan tostado'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
357	WHERE p.tipo_stock='RECETA' AND c.nombre IN ('Antojos','WlCafé')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
358	AND (p.nombre LIKE '%Tostado%' OR p.nombre LIKE '%Derretido%');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
359		Separación visual entre instrucciones de PowerShell.
360	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
361	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,3.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
362	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
363	JOIN ingrediente i ON i.nombre='Queso americano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
364	WHERE p.tipo_stock='RECETA' AND c.nombre IN ('Antojos','WlCafé')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
365	AND (p.nombre LIKE '%Tostado%' OR p.nombre LIKE '%Derretido%');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
366		Separación visual entre instrucciones de PowerShell.
367	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
368	SELECT p.id_producto,i.id_ingrediente,40,TRUE,TRUE,20,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
369	FROM producto p JOIN ingrediente i ON i.nombre='Frijol'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
370	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Frijol%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
371		Separación visual entre instrucciones de PowerShell.
372	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
373	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
374	FROM producto p JOIN ingrediente i ON i.nombre='Loroco'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
375	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%loroco%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
376		Separación visual entre instrucciones de PowerShell.
377	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
378	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
379	FROM producto p JOIN ingrediente i ON i.nombre='Jamón'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
380	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%jamón%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
381		Separación visual entre instrucciones de PowerShell.
382	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
383	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,1.50,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
384	FROM producto p JOIN ingrediente i ON i.nombre='Tomate'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
385	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tomate%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
386		Separación visual entre instrucciones de PowerShell.

387	-- Hamburguesas y sándwiches de almuerzo.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
388	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
389	SELECT p.id_producto,i.id_ingredient,1,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
390	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
391	JOIN ingrediente i ON i.nombre='Pan hamburguesa'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
392	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
393		Separación visual entre instrucciones de PowerShell.
394	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
395	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
396	CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
397	WHEN p.nombre='Doble Big Mac' THEN 4	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
398	WHEN p.nombre='Big Mac' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
399	WHEN p.nombre LIKE '%Triple%' THEN 3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
400	WHEN p.nombre LIKE '%Doble%' OR p.nombre='WlDouble' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
401	ELSE 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
402	END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
403	FALSE,TRUE,1,12.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
404	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
405	JOIN ingrediente i ON i.nombre='Carne de res'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
406	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
407	AND p.nombre NOT LIKE '%Pollo%' AND p.nombre NOT LIKE '%Crispy%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
408	AND p.nombre NOT LIKE '%Sándwich%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
409		Separación visual entre instrucciones de PowerShell.
410	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
411	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
412	CASE WHEN p.nombre LIKE '%Doble%' THEN 2 ELSE 1 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
413	FALSE,TRUE,1,10.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
414	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
415	JOIN ingrediente i ON i.nombre='Filete de pollo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
416	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
417	AND (p.nombre LIKE '%Pollo%' OR p.nombre LIKE '%Crispy%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
418	OR p.nombre LIKE '%Sándwich%');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
419		Separación visual entre instrucciones de PowerShell.
420	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
421	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
422	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre LIKE '%Triple%' THEN 2 ELSE 1 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
423	TRUE,TRUE,1,3.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
424	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
425	JOIN ingrediente i ON i.nombre='Queso americano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
426	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
427	AND p.nombre NOT LIKE '%Cheddar%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
428	AND (p.descripcion LIKE '%queso%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
429	OR p.nombre REGEXP 'Queso WlMelt Big Mac Big Tasty WlNífica WlDouble Triple Bacon Pico Guacamol Smoke Tocino Gourmet');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
430		Separación visual entre instrucciones de PowerShell.

431	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
432	SELECT p.id_producto,i.id_ingrediente,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
433	CASE WHEN p.nombre LIKE '%Doble%' THEN 2 ELSE 1 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
434	TRUE,TRUE,1,3.50,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
435	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
436	JOIN ingrediente i ON i.nombre='Queso cheddar'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
437	WHERE c.nombre IN ('Almuerzos','Desayunos') AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
438	AND p.nombre LIKE '%Cheddar%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
439		Separación visual entre instrucciones de PowerShell.
440	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
441	SELECT p.id_producto,i.id_ingrediente,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
442	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre LIKE '%Triple%' THEN 3 ELSE 2 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
443	TRUE,TRUE,1,4.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
444	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
445	JOIN ingrediente i ON i.nombre='Tocino'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
446	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
447	AND (p.nombre LIKE '%Bacon%' OR p.nombre LIKE '%Tocino%');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
448		Separación visual entre instrucciones de PowerShell.
449	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
450	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
451	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
452	JOIN ingrediente i ON i.nombre='Lechuga'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
453	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
454	AND p.nombre REGEXP 'Big Tasty Deluxe Gourmet WlNifica Pollo Crispy Res Sándwich';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
455		Separación visual entre instrucciones de PowerShell.
456	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
457	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,1.50,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
458	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
459	JOIN ingrediente i ON i.nombre='Tomate'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
460	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
461	AND p.nombre REGEXP 'Tasty Deluxe Gourmet WlNifica Pollo Crispy Res Sándwich'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
462	AND p.nombre NOT LIKE '%Big Mac%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
463		Separación visual entre instrucciones de PowerShell.
464	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
465	SELECT p.id_producto,i.id_ingrediente,10,TRUE,TRUE,5,0.75,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
466	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
467	JOIN ingrediente i ON i.nombre='Cebolla'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
468	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
469	AND p.nombre NOT REGEXP 'Pollo Crispy Sándwich';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
470		Separación visual entre instrucciones de PowerShell.
471	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
472	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,1.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
473	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
474	JOIN ingrediente i ON i.nombre='Pepinillo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

475	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
476	AND p.nombre NOT REGEXP 'Pollo Crispy Sándwich Gourmet Tasty';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
477		Separación visual entre instrucciones de PowerShell.
478	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
479	SELECT p.id_producto,i.id_ingredient,12,TRUE,TRUE,6,0.75,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
480	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
481	JOIN ingrediente i ON i.nombre='Ketchup'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
482	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
483	AND p.nombre REGEXP 'Hamburguesa Cuarto Triple WDouble Quesoburguesa';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
484		Separación visual entre instrucciones de PowerShell.
485	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
486	SELECT p.id_producto,i.id_ingredient,8,TRUE,TRUE,4,0.75,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
487	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
488	JOIN ingrediente i ON i.nombre='Mostaza'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
489	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
490	AND p.nombre REGEXP 'Hamburguesa Cuarto Triple WDouble Quesoburguesa';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
491		Separación visual entre instrucciones de PowerShell.
492	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
493	SELECT p.id_producto,i.id_ingredient,15,TRUE,TRUE,8,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
494	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
495	JOIN ingrediente i ON i.nombre='Mayonesa'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
496	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
497	AND p.nombre REGEXP 'Pollo Crispy Sándwich Deluxe';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
498		Separación visual entre instrucciones de PowerShell.
499	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
500	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
501	FROM producto p JOIN ingrediente i ON i.nombre='Salsa Big Mac'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
502	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Big Mac%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
503		Separación visual entre instrucciones de PowerShell.
504	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
505	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
506	FROM producto p JOIN ingrediente i ON i.nombre='Salsa Big Tasty'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
507	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Big Tasty%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
508		Separación visual entre instrucciones de PowerShell.
509	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
510	SELECT p.id_producto,i.id_ingredient,35,TRUE,TRUE,20,4.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
511	FROM producto p JOIN ingrediente i ON i.nombre='Guacamole'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
512	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Guacamol%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
513		Separación visual entre instrucciones de PowerShell.
514	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
515	SELECT p.id_producto,i.id_ingredient,30,TRUE,TRUE,15,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
516	FROM producto p JOIN ingrediente i ON i.nombre='Pico de gallo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
517	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Pico%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
518		Separación visual entre instrucciones de PowerShell.

519	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
520	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
521	FROM producto p JOIN ingrediente i ON i.nombre='Salsa ahumada'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
522	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Smoke%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
523		Separación visual entre instrucciones de PowerShell.
524	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
525	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
526	FROM producto p JOIN ingrediente i ON i.nombre='Salsa ranch'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
527	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Ranch%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
528		Separación visual entre instrucciones de PowerShell.
529	-- Desayunos: WLMuffin y WlGriddle.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
530	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
531	SELECT p.id_producto,i.id_ingredient,1,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
532	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
533	JOIN ingrediente i ON i.nombre='Pan muffin'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
534	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
535	AND p.nombre LIKE '%WLMuffin%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
536		Separación visual entre instrucciones de PowerShell.
537	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
538	SELECT p.id_producto,i.id_ingredient,1,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
539	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
540	JOIN ingrediente i ON i.nombre='Pan WlGriddle'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
541	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
542	AND p.nombre LIKE '%WlGriddle%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
543		Separación visual entre instrucciones de PowerShell.
544	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
545	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
546	CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
547	WHEN p.nombre='Burritos' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
548	WHEN p.nombre LIKE '%Doble Huevo%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
549	OR p.nombre LIKE '%Doble de Huevo%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
550	OR p.nombre LIKE '%doble huevo%' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
551	ELSE 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
552	END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
553	TRUE,TRUE,1,4.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
554	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
555	JOIN ingrediente i ON i.nombre='Huevo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
556	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
557	AND (p.nombre LIKE '%Huevo%' OR p.descripcion LIKE '%huevo%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
558	OR p.nombre LIKE '%Chapín%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
559	OR p.nombre IN ('Burrito','Burritos','Desayuno clásico',	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
560	'Desayuno Deluxe','Desayuno tradicional')));	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
561		Separación visual entre instrucciones de PowerShell.
562	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

563	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
564	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre='Burritos' THEN 2 ELSE 1 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
565	TRUE,TRUE,1,5.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
566	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
567	JOIN ingrediente i ON i.nombre='Salchicha desayuno'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
568	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
569	AND (p.nombre LIKE '%Salchicha%' OR p.descripcion LIKE '%salchicha%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
570	OR p.nombre LIKE '%Burrito%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
571	OR p.nombre IN ('Desayuno clásico','Desayuno Deluxe'));	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
572		Separación visual entre instrucciones de PowerShell.
573	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
574	SELECT p.id_producto,i.id_ingredient,1,TRUE,TRUE,1,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
575	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
576	JOIN ingrediente i ON i.nombre='Jamón'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
577	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
578	AND (p.nombre LIKE '%Jamón%' OR p.descripcion LIKE '%jamón%'));	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
579		Separación visual entre instrucciones de PowerShell.
580	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
581	SELECT p.id_producto,i.id_ingredient,2,TRUE,TRUE,1,4.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
582	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
583	JOIN ingrediente i ON i.nombre='Tocino'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
584	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
585	AND p.nombre LIKE '%Tocino%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
586		Separación visual entre instrucciones de PowerShell.
587	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
588	SELECT p.id_producto,i.id_ingredient,1,TRUE,TRUE,1,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
589	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
590	JOIN ingrediente i ON i.nombre='Queso americano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
591	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
592	AND (p.descripcion LIKE '%queso%' OR p.nombre LIKE '%Queso%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
593	OR p.nombre LIKE '%Cheddar%')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
594	AND p.nombre NOT LIKE '%Cheddar%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
595		Separación visual entre instrucciones de PowerShell.
596	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
597	SELECT p.id_producto,i.id_ingredient,40,TRUE,TRUE,20,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
598	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
599	JOIN ingrediente i ON i.nombre='Frijol'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
600	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
601	AND p.nombre LIKE '%Chapín%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
602		Separación visual entre instrucciones de PowerShell.
603	-- Burritos.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
604	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
605	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
606	CASE WHEN p.nombre='Burritos' THEN 2 ELSE 1 END,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

607	FROM producto p JOIN ingrediente i ON i.nombre='Tortilla de harina'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
608	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
609		Separación visual entre instrucciones de PowerShell.
610	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
611	SELECT p.id_producto,i.id_ingrediente,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
612	CASE WHEN p.nombre='Burritos' THEN 60 ELSE 30 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
613	TRUE,TRUE,20,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
614	FROM producto p JOIN ingrediente i ON i.nombre='Vegetales mixtos'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
615	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
616		Separación visual entre instrucciones de PowerShell.
617	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
618	SELECT p.id_producto,i.id_ingrediente,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
619	CASE WHEN p.nombre='Burritos' THEN 2 ELSE 1 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
620	TRUE,TRUE,1,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
621	FROM producto p JOIN ingrediente i ON i.nombre='Queso americano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
622	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
623		Separación visual entre instrucciones de PowerShell.
624	-- Hot Cakes y platos de desayuno.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
625	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
626	SELECT p.id_producto,i.id_ingrediente,3,FALSE,TRUE,1,5.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
627	FROM producto p JOIN ingrediente i ON i.nombre='Masa de hot cake'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
628	WHERE p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
629	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
630		Separación visual entre instrucciones de PowerShell.
631	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
632	SELECT p.id_producto,i.id_ingrediente,10,TRUE,TRUE,5,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
633	FROM producto p JOIN ingrediente i ON i.nombre='Mantequilla'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
634	WHERE p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
635	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
636		Separación visual entre instrucciones de PowerShell.
637	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
638	SELECT p.id_producto,i.id_ingrediente,30,TRUE,TRUE,15,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
639	FROM producto p JOIN ingrediente i ON i.nombre='Miel'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
640	WHERE p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
641	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
642		Separación visual entre instrucciones de PowerShell.
643	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
644	SELECT p.id_producto,i.id_ingrediente,2,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
645	FROM producto p JOIN ingrediente i ON i.nombre='Pan tostado'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
646	WHERE p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
647	AND p.nombre IN ('Desayuno clásico','Desayuno Deluxe');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
648		Separación visual entre instrucciones de PowerShell.
649	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
650	SELECT p.id_producto,i.id_ingrediente,80,TRUE,TRUE,40,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

651	FROM producto p JOIN ingrediente i ON i.nombre='Frijol'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
652	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
653		Separación visual entre instrucciones de PowerShell.
654	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
655	SELECT p.id_producto,i.id_ingredient,80,TRUE,TRUE,40,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
656	FROM producto p JOIN ingrediente i ON i.nombre='Plátano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
657	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
658		Separación visual entre instrucciones de PowerShell.
659	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
660	SELECT p.id_producto,i.id_ingredient,30,TRUE,TRUE,15,1.50,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
661	FROM producto p JOIN ingrediente i ON i.nombre='Crema'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
662	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
663		Separación visual entre instrucciones de PowerShell.
664	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
665	SELECT p.id_producto,i.id_ingredient,3,FALSE,TRUE,1,1.00,3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
666	FROM producto p JOIN ingrediente i ON i.nombre='Tortilla de maíz'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
667	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
668		Separación visual entre instrucciones de PowerShell.
669	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
670	SELECT p.id_producto,i.id_ingredient,1,TRUE,TRUE,1,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
671	FROM producto p JOIN ingrediente i ON i.nombre='Queso americano'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
672	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
673		Separación visual entre instrucciones de PowerShell.
674	-- Bebidas calientes del catálogo general.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
675	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
676	SELECT p.id_producto,i.id_ingredient,18,FALSE,TRUE,9,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
677	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
678	JOIN ingrediente i ON i.nombre='Café molido'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
679	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
680	AND p.nombre LIKE '%Café%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
681		Separación visual entre instrucciones de PowerShell.
682	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
683	SELECT p.id_producto,i.id_ingredient,250,FALSE,FALSE,50,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
684	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
685	JOIN ingrediente i ON i.nombre='Agua preparada'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
686	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
687	AND p.nombre REGEXP 'Café Chocolate Té caliente';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
688		Separación visual entre instrucciones de PowerShell.
689	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
690	SELECT p.id_producto,i.id_ingredient,180,FALSE,FALSE,50,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
691	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
692	JOIN ingrediente i ON i.nombre='Agua preparada'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
693	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
694	AND p.nombre REGEXP 'WlFizz WlFrizz';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

695		Separación visual entre instrucciones de PowerShell.
696	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
697	SELECT p.id_producto,i.id_ingredient,160,FALSE,TRUE,60,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
698	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
699	JOIN ingrediente i ON i.nombre='Hielo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
700	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
701	AND p.nombre REGEXP 'w\Fizz w\Frizz';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
702		Separación visual entre instrucciones de PowerShell.
703	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
704	SELECT p.id_producto,i.id_ingredient,150,FALSE,TRUE,50,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
705	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
706	JOIN ingrediente i ON i.nombre='Leche'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
707	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
708	AND p.nombre REGEXP 'con leche Chocolate';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
709		Separación visual entre instrucciones de PowerShell.
710	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
711	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
712	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate en polvo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
713	WHERE p.tipo_stock='RECETA' AND p.nombre='Chocolate';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
714		Separación visual entre instrucciones de PowerShell.
715	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
716	SELECT p.id_producto,i.id_ingredient,1,FALSE,TRUE,1,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
717	FROM producto p JOIN ingrediente i ON i.nombre='Bolsa de té'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
718	WHERE p.tipo_stock='RECETA' AND p.nombre='Té caliente';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
719		Separación visual entre instrucciones de PowerShell.
720	-- w\Café: café, frappés, bebidas frías, té y smoothies.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
721	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
722	SELECT p.id_producto,i.id_ingredient,18,FALSE,TRUE,9,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
723	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
724	JOIN ingrediente i ON i.nombre='Café molido'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
725	WHERE c.nombre='w\Café' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
726	AND p.nombre REGEXP 'Café Coffee Capuccino Latte Frappé';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
727		Separación visual entre instrucciones de PowerShell.
728	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
729	SELECT p.id_producto,i.id_ingredient,180,FALSE,FALSE,50,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
730	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
731	JOIN ingrediente i ON i.nombre='Agua preparada'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
732	WHERE c.nombre='w\Café' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
733	AND p.subcategoria IN ('Bebidas Calientes','Bebidas Frías');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
734		Separación visual entre instrucciones de PowerShell.
735	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
736	SELECT p.id_producto,i.id_ingredient,180,FALSE,TRUE,50,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
737	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
738	JOIN ingrediente i ON i.nombre='Leche'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

739	WHERE c.nombre='lCafé' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
740	AND p.nombre REGEXP 'Capuccino Chocolate Frappé Coffee Latte Chai';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
741		Separación visual entre instrucciones de PowerShell.
742	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
743	SELECT p.id_producto,i.id_ingredient,160,FALSE,TRUE,60,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
744	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
745	JOIN ingrediente i ON i.nombre='Hielo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
746	WHERE c.nombre='lCafé' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
747	AND p.subcategoria='Bebidas Frías';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
748		Separación visual entre instrucciones de PowerShell.
749	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
750	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
751	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate en polvo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
752	WHERE p.tipo_stock='RECETA' AND p.nombre='Chocolate caliente';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
753		Separación visual entre instrucciones de PowerShell.
754	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
755	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
756	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe caramelo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
757	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Caramelo%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
758		Separación visual entre instrucciones de PowerShell.
759	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
760	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
761	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe chocolate'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
762	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Chocolate%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
763	AND p.nombre<>'Chocolate caliente';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
764		Separación visual entre instrucciones de PowerShell.
765	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
766	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
767	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe vainilla'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
768	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Vainilla%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
769		Separación visual entre instrucciones de PowerShell.
770	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
771	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
772	FROM producto p JOIN ingrediente i ON i.nombre='Galleta Oreo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
773	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Oreo%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
774		Separación visual entre instrucciones de PowerShell.
775	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
776	SELECT p.id_producto,i.id_ingredient,200,FALSE,TRUE,50,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
777	FROM producto p JOIN ingrediente i ON i.nombre='Base horchata'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
778	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Horchata%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
779		Separación visual entre instrucciones de PowerShell.
780	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
781	SELECT p.id_producto,i.id_ingredient,120,FALSE,TRUE,40,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
782	FROM producto p JOIN ingrediente i ON i.nombre='Pulpa de mango'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

783	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Mango%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
784		Separación visual entre instrucciones de PowerShell.
785	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
786	SELECT p.id_producto,i.id_ingredient,120,FALSE,TRUE,40,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
787	FROM producto p JOIN ingrediente i ON i.nombre='Pulpa de berries'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
788	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Berries%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
789		Separación visual entre instrucciones de PowerShell.
790	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
791	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
792	FROM producto p JOIN ingrediente i ON i.nombre='Mezcla chai'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
793	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Chai%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
794		Separación visual entre instrucciones de PowerShell.
795	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
796	SELECT p.id_producto,i.id_ingredient,1,FALSE,TRUE,1,1.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
797	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
798	JOIN ingrediente i ON i.nombre='Bolsa de té'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
799	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
800	AND p.nombre LIKE '%Té Guatemalteco%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
801		Separación visual entre instrucciones de PowerShell.
802	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
803	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
804	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe frutal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
805	WHERE p.tipo_stock='RECETA'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
806	AND p.nombre REGEXP 'WlFizz WlFrizz Bora Bora Melocotón Menta';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
807		Separación visual entre instrucciones de PowerShell.
808	-- Helados y postres preparados.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
809	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
810	SELECT p.id_producto,i.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
811	CASE WHEN p.nombre LIKE '%WlFlurry%' THEN 180 ELSE 120 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
812	FALSE,TRUE,60,5.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
813	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
814	JOIN ingrediente i ON i.nombre='Helado de vainilla'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
815	WHERE c.nombre='Postres' AND p.tipo_stock='RECETA';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
816		Separación visual entre instrucciones de PowerShell.
817	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
818	SELECT p.id_producto,i.id_ingredient,1,FALSE,FALSE,1,0,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
819	FROM producto p JOIN ingrediente i ON i.nombre='Cono'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
820	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Cono%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
821		Separación visual entre instrucciones de PowerShell.
822	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
823	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
824	FROM producto p JOIN ingrediente i ON i.nombre='Galleta Oreo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
825	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%WlFlurry Oreo%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
826		Separación visual entre instrucciones de PowerShell.

827	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
828	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,3.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
829	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate M&M'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
830	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%M&M%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
831		Separación visual entre instrucciones de PowerShell.
832	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
833	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
834	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe caramelo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
835	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Caramelo%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
836		Separación visual entre instrucciones de PowerShell.
837	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
838	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
839	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe chocolate'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
840	WHERE p.tipo_stock='RECETA' AND p.nombre REGEXP 'Sundae Chocolate WLFflurry.*Chocolate';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
841		Separación visual entre instrucciones de PowerShell.
842	INSERT INTO carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
843	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
844	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe fresa'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
845	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Fresa%';	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
846		Separación visual entre instrucciones de PowerShell.
847	-- Inserción final de recetas, evitando pares repetidos.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
848	INSERT INTO producto_ingredient (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
849	id_producto, id_ingredient, cantidad_default,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
850	permite_quitar, permite_extra, cantidad_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
851	precio_extra, max_extras, estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
852	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
853	SELECT	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
854	receta.id_producto,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
855	receta.id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
856	receta.cantidad,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
857	receta.permite_quitar,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
858	receta.permite_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
859	receta.cantidad_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
860	receta.precio_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
861	receta.max_extras,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
862	TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
863	FROM (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
864	SELECT	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
865	id_producto,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
866	id_ingredient,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
867	MAX(cantidad) AS cantidad,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
868	MAX(permite_quitar) AS permite_quitar,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
869	MAX(permite_extra) AS permite_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
870	MAX(cantidad_extra) AS cantidad_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

871	MAX(precio_extra) AS precio_extra,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
872	MAX(max_extras) AS max_extras	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
873	FROM carga_receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
874	GROUP BY id_producto, id_ingredient	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
875	) receta	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
876	WHERE NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
877	SELECT 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
878	FROM producto_ingredient pi	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
879	WHERE pi.id_producto = receta.id_producto	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
880	AND pi.id_ingredient = receta.id_ingredient	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
881	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
882		Separación visual entre instrucciones de PowerShell.
883	DROP TEMPORARY TABLE carga_receta;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
884		Separación visual entre instrucciones de PowerShell.
885	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
886	-- 5. PRESENTACIÓN INDIVIDUAL	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
887	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
888		Separación visual entre instrucciones de PowerShell.
889	INSERT INTO presentacion_menu (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
890	id_producto_principal, nombre, tipo, precio, predeterminada, estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
891	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
892	SELECT p.id_producto, 'Individual', 'INDIVIDUAL', p.precio_base, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
893	FROM producto p	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
894	WHERE p.estado=TRUE AND p.tipo_stock<>'NINGUNO'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
895	AND p.nombre NOT IN ('Papas Kids', 'Puré de manzana', 'Yogur de fresa',	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
896	'Jugo de manzana Kids', 'Juguete sorpresa', 'Coca-Cola 1.5 L')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
897	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
898	SELECT 1 FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
899	WHERE pm.id_producto_principal=p.id_producto	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
900	AND pm.tipo='INDIVIDUAL'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
901	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
902		Separación visual entre instrucciones de PowerShell.
903	INSERT INTO grupo_presentacion (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
904	id_presentacion, nombre, minimo, maximo, permite_repetir,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
905	visible, permite_personalizar, estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
906	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
907	SELECT pm.id_presentacion, 'Producto principal', 1, 1, FALSE, FALSE, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
908	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
909	WHERE pm.tipo='INDIVIDUAL' AND pm.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
910	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
911	SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
912	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
913	AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
914	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

915		Separación visual entre instrucciones de PowerShell.
916	INSERT INTO opcion_grupo (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
917	id_grupo,nombre,incremento_precio,predeterminada,estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
918	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
919	SELECT gp.id_grupo,p.nombre,0,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
920	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
921	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
922	JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
923	WHERE pm.tipo='INDIVIDUAL' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
924	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
925	SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
926	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
927		Separación visual entre instrucciones de PowerShell.
928	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
929	SELECT og.id_opcion,pm.id_producto_principal,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
930	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
931	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
932	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
933	WHERE pm.tipo='INDIVIDUAL' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
934	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
935	SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
936	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
937	AND oc.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
938	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
939		Separación visual entre instrucciones de PowerShell.
940	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
941	-- 6. WLMENÚS DE ALMUERZO Y DESAYUNO	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
942	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
943		Separación visual entre instrucciones de PowerShell.
944	-- Almuerzos: precio individual + Q20.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
945	INSERT INTO presentacion_menu (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
946	id_producto_principal,nombre,tipo,precio,predeterminada,estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
947	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
948	SELECT p.id_producto,'WlMenú', 'MENU',p.precio_base+20,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
949	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
950	WHERE c.nombre='Almuerzos' AND p.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
951	AND p.subcategoria<>'Para compartir'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
952	AND p.nombre NOT LIKE '%10 Piezas%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
953	AND p.nombre NOT LIKE '%Dos Piezas%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
954	AND p.nombre NOT LIKE '%1 Pieza%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
955	AND p.nombre<>'Pollo WlCrispy'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
956	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
957	SELECT 1 FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
958	WHERE pm.id_producto_principal=p.id_producto AND pm.tipo='MENU'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

959	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
960		Separación visual entre instrucciones de PowerShell.
961	-- Desayunos portátiles: precio individual + Q15.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
962	INSERT INTO presentacion_menu (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
963	id_producto_principal,nombre, tipo,precio,predeterminada,estado	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
964	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
965	SELECT p.id_producto,'WlMenú Desayuno','MENU',p.precio_base+15,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
966	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
967	WHERE c.nombre='Desayunos' AND p.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
968	AND (p.nombre LIKE '%WlMuffin%' OR p.nombre LIKE '%WlGriddle%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
969	OR p.nombre IN ('Burrito','Burritos'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
970	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
971	SELECT 1 FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
972	WHERE pm.id_producto_principal=p.id_producto AND pm.tipo='MENU'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
973	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
974		Separación visual entre instrucciones de PowerShell.
975	-- Grupos de toda presentación MENU.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
976	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
977	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
978	SELECT pm.id_presentacion,'Producto principal',1,1,FALSE,FALSE,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
979	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
980	WHERE pm.tipo='MENU'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
981	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
982	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
983	AND gp.nombre='Producto principal');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
984		Separación visual entre instrucciones de PowerShell.
985	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
986	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
987	SELECT pm.id_presentacion,'Elige el complemento',1,1,FALSE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
988	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
989	WHERE pm.tipo='MENU'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
990	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
991	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
992	AND gp.nombre='Elige el complemento');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
993		Separación visual entre instrucciones de PowerShell.
994	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
995	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
996	SELECT pm.id_presentacion,'Elige la bebida',1,1,FALSE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
997	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
998	WHERE pm.tipo='MENU'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
999	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1000	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1001	AND gp.nombre='Elige la bebida');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1002		Separación visual entre instrucciones de PowerShell.

1003	-- Producto principal de cada WlMenú.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1004	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1005	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1006	SELECT gp.id_grupo,p.nombre,0,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1007	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1008	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1009	JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1010	WHERE pm.tipo='MENU' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1011	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1012		Separación visual entre instrucciones de PowerShell.
1013	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1014	SELECT og.id_opcion,pm.id_producto_principal,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1015	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1016	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1017	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1018	WHERE pm.tipo='MENU' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1019	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1020	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1021	AND oc.id_producto=pm.id_producto_principal);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1022		Separación visual entre instrucciones de PowerShell.
1023	-- Complementos. Los menús de desayuno usan Hash Brown; los demás, Papas.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1024	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1025	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1026	SELECT gp.id_grupo,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1027	CASE WHEN principal.disponibilidad_menu='DESAYUNO'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1028	THEN 'Hash Brown' ELSE 'Papas' END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1029	0,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1030	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1031	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1032	JOIN producto principal ON principal.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1033	WHERE pm.tipo='MENU' AND gp.nombre='Elige el complemento'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1034	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1035		Separación visual entre instrucciones de PowerShell.
1036	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1037	SELECT og.id_opcion,componente.id_producto,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1038	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1039	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1040	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1041	JOIN producto componente ON componente.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1042	WHERE pm.tipo='MENU' AND gp.nombre='Elige el complemento'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1043	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1044	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1045	AND oc.id_producto=componente.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1046		Separación visual entre instrucciones de PowerShell.

1047	-- Bebidas permitidas según horario del producto principal.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1048	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1049	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1050	SELECT gp.id_grupo,bebida.nombre,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1051	CASE WHEN bebida.nombre LIKE 'Jugo%' THEN 2 ELSE 0 END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1052	CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1053	WHEN principal.disponibilidad_menu='DESAYUNO' AND bebida.nombre='Café' THEN TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1054	WHEN principal.disponibilidad_menu<>'DESAYUNO' AND bebida.nombre='Coca-Cola' THEN TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1055	ELSE FALSE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1056	END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1057	TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1058	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1059	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1060	JOIN producto principal ON principal.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1061	JOIN producto bebida ON bebida.nombre IN (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1062	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Café' ELSE 'Coca-Cola' END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1063	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Jugo naranja' ELSE 'Coca-Cola Zero' END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1064	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Jugo manzana' ELSE 'Sprite' END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1065	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Agua' ELSE 'Fanta' END	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1066	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1067	WHERE pm.tipo='MENU' AND gp.nombre='Elige la bebida'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1068	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1069	WHERE og.id_grupo=gp.id_grupo AND og.nombre=bebida.nombre);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1070		Separación visual entre instrucciones de PowerShell.
1071	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1072	SELECT og.id_opcion,bebida.id_producto,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1073	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1074	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1075	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1076	JOIN producto bebida ON bebida.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1077	WHERE pm.tipo='MENU' AND gp.nombre='Elige la bebida'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1078	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1079	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1080	AND oc.id_producto=bebida.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1081		Separación visual entre instrucciones de PowerShell.
1082	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1083	-- 7. CAJITAS FELICES	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1084	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1085		Separación visual entre instrucciones de PowerShell.
1086	INSERT INTO presentacion_menu	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1087	(id_producto_principal,nombre,tipo,precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1088	SELECT p.id_producto,'Cajita Feliz','INFANTIL',p.precio_base,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1089	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1090	WHERE c.nombre='Cajita Feliz' AND p.nombre LIKE 'Cajita Feliz%'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1091	AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1092	WHERE pm.id_producto_principal=p.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1093		Separación visual entre instrucciones de PowerShell.
1094	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1095	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1096	SELECT pm.id_presentacion,'Producto principal',1,1,FALSE,FALSE,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1097	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1098	WHERE pm.tipo='INFANTIL'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1099	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1100	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1101	AND gp.nombre='Producto principal');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1102		Separación visual entre instrucciones de PowerShell.
1103	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1104	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1105	SELECT pm.id_presentacion,'Elige el complemento',1,1,FALSE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1106	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1107	WHERE pm.tipo='INFANTIL'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1108	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1109	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1110	AND gp.nombre='Elige el complemento');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1111		Separación visual entre instrucciones de PowerShell.
1112	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1113	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1114	SELECT pm.id_presentacion,'Elige la bebida',1,1,FALSE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1115	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1116	WHERE pm.tipo='INFANTIL'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1117	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1118	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1119	AND gp.nombre='Elige la bebida');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1120		Separación visual entre instrucciones de PowerShell.
1121	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1122	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1123	SELECT pm.id_presentacion,'Elige el juguete',1,1,FALSE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1124	FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1125	WHERE pm.tipo='INFANTIL'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1126	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1127	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1128	AND gp.nombre='Elige el juguete');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1129		Separación visual entre instrucciones de PowerShell.
1130	-- Entrada fija de cada Cajita.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1131	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1132	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1133	SELECT gp.id_grupo,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1134	CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1135	WHEN raiz.nombre LIKE '%derretido%' THEN 'Derretido clásico'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1136	WHEN raiz.nombre LIKE '%Hot Cakes%' THEN 'Hot Cakes'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1137	WHEN raiz.nombre LIKE '%Pollo WLCrispy%' THEN 'Pollo WLCrispy 1 Pieza'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1138	WHEN raiz.nombre LIKE '%quesoburguesa%' THEN 'Quesoburguesa'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1139	WHEN raiz.nombre LIKE '%wNuggets%' THEN 'wNuggets 4 pz'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1140	WHEN raiz.nombre LIKE '%frijol%' THEN 'wMuffin Huevo y Frijol'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1141	WHEN raiz.nombre LIKE '%Huevo y queso%' THEN 'wMuffin Huevo y Queso'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1142	WHEN raiz.nombre LIKE '%salchicha%' THEN 'wMuffin Salchicha'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1143	ELSE 'Hamburguesa'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1144	END,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1145	0, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1146	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1147	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1148	JOIN producto raiz ON raiz.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1149	WHERE pm.tipo='INFANTIL' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1150	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1151		Separación visual entre instrucciones de PowerShell.
1152	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1153	SELECT og.id_opcion,componente.id_producto,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1154	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1155	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1156	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1157	JOIN producto componente ON componente.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1158	WHERE pm.tipo='INFANTIL' AND gp.nombre='Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1159	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1160	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1161	AND oc.id_producto=componente.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1162		Separación visual entre instrucciones de PowerShell.
1163	-- Complementos infantiles.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1164	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1165	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1166	SELECT gp.id_grupo,componente.nombre,0,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1167	componente.nombre='Papas Kids',TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1168	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1169	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1170	JOIN producto componente ON componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1171	('Papas Kids','Puré de manzana','Yogur de fresa')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1172	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige el complemento'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1173	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1174	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1175		Separación visual entre instrucciones de PowerShell.
1176	-- Bebidas infantiles.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1177	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1178	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1179	SELECT gp.id_grupo, componente.nombre, 0,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1180	componente.nombre='Jugo de manzana Kids', TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1181	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1182	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1183	JOIN producto componente ON componente.nombre IN ('Jugo de manzana Kids', 'Agua')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1184	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige la bebida'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1185	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1186	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1187		Separación visual entre instrucciones de PowerShell.
1188	-- Juguete.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1189	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1190	(id_grupo, nombre, incremento_precio, determinada, estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1191	SELECT gp.id_grupo, 'Juguete sorpresa', 0, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1192	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1193	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1194	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige el juguete'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1195	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1196		Separación visual entre instrucciones de PowerShell.
1197	-- Componentes de complemento, bebida y juguete.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1198	INSERT INTO opcion_componente (id_opcion, id_producto, cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1199	SELECT og.id_opcion, componente.id_producto, 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1200	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1201	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1202	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1203	JOIN producto componente ON componente.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1204	WHERE pm.tipo='INFANTIL' AND gp.nombre<>'Producto principal'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1205	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1206	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1207	AND oc.id_producto=componente.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1208		Separación visual entre instrucciones de PowerShell.
1209	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1210	-- 8. COMBOS Y CAJAS PREDEFINIDAS	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1211	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1212		Separación visual entre instrucciones de PowerShell.
1213	INSERT INTO presentacion_menu	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1214	(id_producto_principal, nombre, tipo, precio, determinada, estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1215	SELECT p.id_producto, 'Combo', 'COMBO', p.precio_base, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1216	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1217	WHERE p.tipo_stock='NINGUNO' AND c.nombre<>'Cajita Feliz'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1218	AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1219	WHERE pm.id_producto_principal=p.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1220		Separación visual entre instrucciones de PowerShell.
1221	-- Cajas Grandes configurables: cuatro principales independientes.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1222	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1223	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1224	SELECT pm.id_presentacion,'Elige tus productos',4,4,TRUE,TRUE,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1225	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1226	WHERE pm.tipo='COMBO' AND p.nombre IN ('Caja Grande','Caja Grande con Postre')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1227	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1228	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1229	AND gp.nombre='Elige tus productos');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1230		Separación visual entre instrucciones de PowerShell.
1231	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1232	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1233	SELECT gp.id_grupo,componente.nombre,0,componente.nombre='Big Mac',TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1234	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1235	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1236	JOIN producto componente ON componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1237	('Big Mac','Cuarto de Libra con Queso','WlNifica Doble','10 WlNuggets de Pollo')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1238	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus productos'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1239	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1240	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1241		Separación visual entre instrucciones de PowerShell.
1242	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1243	SELECT og.id_opcion,componente.id_producto,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1244	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1245	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1246	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1247	JOIN producto componente ON componente.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1248	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus productos'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1249	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1250	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1251	AND oc.id_producto=componente.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1252		Separación visual entre instrucciones de PowerShell.
1253	-- Contenido fijo de cajas grandes: cuatro papas y bebida familiar.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1254	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1255	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1256	SELECT pm.id_presentacion,'Incluye',1,1,FALSE,FALSE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1257	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1258	WHERE pm.tipo='COMBO' AND p.nombre IN ('Caja Grande','Caja Grande con Postre')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1259	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1260	WHERE gp.id_presentacion=pm.id_presentacion AND gp.nombre='Incluye');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1261		Separación visual entre instrucciones de PowerShell.
1262	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1263	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1264	SELECT gp.id_grupo,'Papas y bebida familiar',0,TRUE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1265	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1266	WHERE gp.nombre='Incluye'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1267	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1268		Separación visual entre instrucciones de PowerShell.
1269	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1270	SELECT og.id_opcion,p.id_producto,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1271	CASE WHEN p.nombre='Papas' THEN 4 ELSE 1 END	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1272	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1273	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1274	JOIN producto p ON p.nombre IN ('Papas','Coca-Cola 1.5 L')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1275	WHERE gp.nombre='Incluye'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1276	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1277	WHERE oc.id_opcion=og.id_opcion AND oc.id_producto=p.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1278		Separación visual entre instrucciones de PowerShell.
1279	-- Postres seleccionables para Caja Grande con Postre.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1280	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1281	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1282	SELECT pm.id_presentacion,'Elige tus postres',2,2,TRUE,TRUE,FALSE,TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1283	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1284	WHERE pm.tipo='COMBO' AND p.nombre='Caja Grande con Postre'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1285	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1286	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1287	AND gp.nombre='Elige tus postres');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1288		Separación visual entre instrucciones de PowerShell.
1289	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1290	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1291	SELECT gp.id_grupo,postre.nombre,0,postre.nombre='Pastel de manzana',TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1292	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1293	JOIN producto postre ON postre.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1294	('Pastel de manzana','Pastel de queso','Sundae Chocolate','Sundae Fresa')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1295	WHERE gp.nombre='Elige tus postres'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1296	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1297	WHERE og.id_grupo=gp.id_grupo AND og.nombre=postre.nombre);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1298		Separación visual entre instrucciones de PowerShell.
1299	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1300	SELECT og.id_opcion,postre.id_producto,1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1301	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1302	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1303	JOIN producto postre ON postre.nombre=og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1304	WHERE gp.nombre='Elige tus postres'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1305	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1306	WHERE oc.id_opcion=og.id_opcion AND oc.id_producto=postre.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1307		Separación visual entre instrucciones de PowerShell.
1308	-- Resto de combos predefinidos: una opción fija con varios componentes.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1309	INSERT INTO grupo_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1310	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1311	SELECT pm.id_presentacion, 'Contenido del combo', 1, 1, FALSE, FALSE, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1312	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1313	WHERE pm.tipo='COMBO' AND p.nombre NOT IN ('Caja Grande', 'Caja Grande con Postre')	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1314	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1315	WHERE gp.id_presentacion=pm.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1316	AND gp.nombre='Contenido del combo');	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1317		Separación visual entre instrucciones de PowerShell.
1318	INSERT INTO opcion_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1319	(id_grupo, nombre, incrementoPrecio, determinada, estado)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1320	SELECT gp.id_grupo, p.nombre, 0, TRUE, TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1321	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1322	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1323	JOIN producto p ON p.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1324	WHERE pm.tipo='COMBO' AND gp.nombre='Contenido del combo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1325	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1326		Separación visual entre instrucciones de PowerShell.
1327	-- Componentes fijos de cada caja o bucket.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1328	INSERT INTO opcion_componente (id_opcion, id_producto, cantidad)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1329	SELECT og.id_opcion, componente.id_producto,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1330	CASE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1331	WHEN raiz.nombre='Caja de 24 WlNuggets'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1332	AND componente.nombre='10 WlNuggets de Pollo' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1333	WHEN raiz.nombre='Caja Grande Snack' AND componente.nombre='Quesoburguesa' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1334	WHEN raiz.nombre='Caja Grande Snack' AND componente.nombre='Papas' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1335	WHEN raiz.nombre LIKE 'Bucket%' AND componente.nombre='Papas' THEN 3	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1336	WHEN raiz.nombre LIKE 'Caja Grande D%' AND componente.nombre='Hash Brown' THEN 4	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1337	WHEN raiz.nombre LIKE 'Caja Grande D%' AND componente.nombre='Café' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1338	WHEN raiz.nombre='Caja Grande Deluxe' AND componente.nombre='Hot Cakes' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1339	WHEN raiz.nombre='Caja Grande Deluxe' AND componente.nombre='WlMuffin de Huevo' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1340	WHEN raiz.nombre='Caja Grande Desayuno' AND componente.nombre='WlMuffin Salchicha' THEN 2	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1341	ELSE 1	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1342	END	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1343	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1344	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1345	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1346	JOIN producto raiz ON raiz.id_producto=pm.id_producto_principal	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1347	JOIN producto componente ON	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1348	(raiz.nombre='Bucket para Todos' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1349	('Pollo WlCrispy 10 Piezas', 'Papas', 'Coca-Cola 1.5 L'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1350	OR (raiz.nombre='Bucket Pollo WlCrispy' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1351	('Pollo WlCrispy 10 Piezas', 'Papas'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1352	OR (raiz.nombre='Bucket Pollo WlCrispy Para Tres' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1353	('Pollo WlCrispy 10 Piezas', 'Papas', 'Coca-Cola 1.5 L'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1354	OR (raiz.nombre='Bucket Pollo WlCrispy Snack' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1355	('Pollo wLCrispy Dos Piezas', 'wLNuggets 4 pz', 'Papas'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1356	OR (raiz.nombre='Caja de 24 wLNuggets' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1357	('10 wLNuggets de Pollo', 'wLNuggets 4 pz'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1358	OR (raiz.nombre='Caja Grande Snack' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1359	('Quesoburguesa', 'wLNuggets 4 pz', 'Papas', 'Coca-Cola 1.5 L'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1360	OR (raiz.nombre='Caja Grande Deluxe' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1361	('Hot Cakes', 'wLMuffin de Huevo', 'Hash Brown', 'Café'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1362	OR (raiz.nombre='Caja Grande Desayuno' AND componente.nombre IN	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1363	('wLMuffin Salchicha', 'Burritos', 'Hash Brown', 'Café'))	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1364	WHERE pm.tipo='COMBO' AND gp.nombre='Contenido del combo'	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1365	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1366	WHERE oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1367	AND oc.id_producto=componente.id_producto);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1368		Separación visual entre instrucciones de PowerShell.
1369	COMMIT;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1370		Separación visual entre instrucciones de PowerShell.
1371	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1372	-- 9. COMPROBACIONES	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1373	-- Todas estas consultas deben devolver cero filas, salvo el resumen.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1374	-- =====	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1375		Separación visual entre instrucciones de PowerShell.
1376	-- Productos RECETA sin ingredientes.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1377	SELECT p.id_producto,p.nombre AS receta_sin_ingredientes	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1378	FROM producto p	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1379	LEFT JOIN producto_ingrediente pi ON pi.id_producto=p.id_producto AND pi.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1380	WHERE p.tipo_stock='RECETA' AND p.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1381	GROUP BY p.id_producto,p.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1382	HAVING COUNT(pi.id_producto_ingrediente)=0;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1383		Separación visual entre instrucciones de PowerShell.
1384	-- Productos vendibles sin presentación.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1385	SELECT p.id_producto,p.nombre AS producto_sin_presentacion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1386	FROM producto p	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1387	LEFT JOIN presentacion_menu pm ON pm.id_producto_principal=p.id_producto AND pm.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1388	WHERE p.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1389	GROUP BY p.id_producto,p.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1390	HAVING COUNT(pm.id_presentacion)=0	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1391	AND NOT EXISTS (	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1392	SELECT 1 FROM opcion_componente oc WHERE oc.id_producto=p.id_producto	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1393	);	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1394		Separación visual entre instrucciones de PowerShell.
1395	-- Grupos sin opciones.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1396	SELECT gp.id_grupo,gp.nombre AS grupo_sin_opciones	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1397	FROM grupo_presentacion gp	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1398	LEFT JOIN opcion_grupo og ON og.id_grupo=gp.id_grupo AND og.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1399	WHERE gp.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1400	GROUP BY gp.id_grupo, gp.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1401	HAVING COUNT(og.id_opcion)=0;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1402		Separación visual entre instrucciones de PowerShell.
1403	-- Opciones sin productos componentes.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1404	SELECT og.id_opcion, og.nombre AS opcion_sin_componentes	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1405	FROM opcion_grupo og	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1406	LEFT JOIN opcion_componente oc ON oc.id_opcion=og.id_opcion	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1407	WHERE og.estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1408	GROUP BY og.id_opcion, og.nombre	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1409	HAVING COUNT(oc.id_opcion_componente)=0;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1410		Separación visual entre instrucciones de PowerShell.
1411	-- Resumen final por tipo de stock.	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1412	SELECT tipo_stock, COUNT(*) AS productos	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1413	FROM producto	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1414	WHERE estado=TRUE	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1415	GROUP BY tipo_stock	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1416	ORDER BY tipo_stock;	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1417	'@	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1418		Separación visual entre instrucciones de PowerShell.
1419	\$lines = New-Object System.Collections.Generic.List[string]	Declara o actualiza una variable de PowerShell.
1420	\$lines.Add(\$header.TrimEnd())	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1421	for (\$index = 0; \$index -lt \$productRows.Count; \$index++) {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1422	\$suffix = if (\$index -eq \$productRows.Count - 1) { ";" } else { "," }	Declara o actualiza una variable de PowerShell.
1423	\$lines.Add(\$productRows[\$index] + \$suffix)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1424	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1425	\$lines.Add(\$afterProducts.TrimStart())	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1426		Separación visual entre instrucciones de PowerShell.
1427	\$resolvedOutput = if ([System.IO.Path]::IsPathRooted(\$OutputPath)) {	Declara o actualiza una variable de PowerShell.
1428	\$OutputPath	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1429	} else {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1430	Join-Path (Get-Location) \$OutputPath	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1431	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1432		Separación visual entre instrucciones de PowerShell.
1433	\$outputDirectory = Split-Path -Parent \$resolvedOutput	Declara o actualiza una variable de PowerShell.
1434	if (-not (Test-Path -LiteralPath \$outputDirectory)) {	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1435	New-Item -ItemType Directory -Path \$outputDirectory -Force Out-Null	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1436	}	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1437		Separación visual entre instrucciones de PowerShell.
1438	[System.IO.File]::WriteAllText(	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1439	\$resolvedOutput,	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1440	(\$lines -join [Environment]::NewLine),	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1441	[System.Text.UTF8Encoding]::new(\$false)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1442	)	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

1443		Separación visual entre instrucciones de PowerShell.
1444	Write-Output "Archivo generado: \$resolvedOutput"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1445	Write-Output "Productos fuente únicos: \$(\$products.Count)"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.
1446	Write-Output "Productos auxiliares agregados: 6"	Instrucción de PowerShell utilizada por el proceso auxiliar del proyecto.

16.15 scripts/ValidarDatosMenu.java

Herramienta auxiliar ValidarDatosMenu usada para generar o validar datos del proyecto.

Tipo: java | Líneas: 235 | Archivo 15 de 157

Clases/tipos detectados: ValidarDatosMenu

Línea	Constructor o método detectado
19	private ValidarDatosMenu() {
22	public static void main(String[] args) throws Exception {
96	private static void ejecutarArchivo(Statement sentencia, Path ruta) throws Exception {
112	private static List<String> separarSentencias(String contenido) {
181	private static int numero(Statement sentencia, String sql) throws Exception {
188	private static void comprobar(Statement sentencia, String nombre,
197	private static void comprobarCero(Statement sentencia, String nombre,
202	private static void imprimirResumen(Statement sentencia) throws Exception {
212	private static int[] cantidadesConfiguracion(Statement sentencia) throws Exception {
223	private static void imprimirRecetasCortas(Statement sentencia) throws Exception {

1	import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
2	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
3	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
4	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
5	import java.sql.DriverManager;	Importa java.sql.DriverManager para poder usar esa clase sin escribir su nombre completo.
6	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
7	import java.sql.Statement;	Importa java.sql.Statement para poder usar esa clase sin escribir su nombre completo.
8	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
9	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
10	import java.util.UUID;	Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	/** Ejecuta el esquema y los datos del menu en una base temporal. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	public final class ValidarDatosMenu {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14	private static final String URL =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15	"jdbc:mysql://localhost:3306/?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
16	private static final String USUARIO = "root";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
17	private static final String CONTRASENA = "[VALOR OCULTO]"; "[VALOR OCULTO]";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	private ValidarDatosMenu() {	Constructor de ValidarDatosMenu: prepara un objeto nuevo y deja sus componentes listos para usarse.
20	}	Cierra el bloque abierto anteriormente.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	public static void main(String[] args) throws Exception {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
23	String base = "waldonalds_validacion_"	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

24	+ UUID.randomUUID().toString().replace("-", "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
25	try (Connection conexion = DriverManager.getConnection(URL, USUARIO, CONTRASENA);	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
26	Statement sentencia = conexion.createStatement()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	sentencia.execute("CREATE DATABASE `" + base + "`");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
29	sentencia.execute("USE `" + base + "`");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	ejecutarArchivo(sentencia, Path.of("sql/base_datos_completa.sql"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	ejecutarArchivo(sentencia, Path.of("sql/datos_menu_nueva_base.sql"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	comprobar(sentencia, "productos", 171,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	"SELECT COUNT(*) FROM producto");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	comprobarCero(sentencia, "productos RECETA sin ingredientes",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	"SELECT COUNT(*) FROM producto p "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	+ "WHERE p.tipo_stock='RECETA' AND NOT EXISTS ("	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
38	+ "SELECT 1 FROM producto_ingredientes pi "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	+ "WHERE pi.id_producto=p.id_producto AND pi.estado=TRUE)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
40	comprobarCero(sentencia, "productos activos sin uso comercial",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	"SELECT COUNT(*) FROM producto p WHERE p.estado=TRUE "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
42	+ "AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	+ "WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE) "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
44	+ "AND NOT EXISTS (SELECT 1 FROM opcion_componente oc "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	+ "WHERE oc.id_producto=p.id_producto)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
46	comprobar(sentencia, "productos auxiliares ocultos del catalogo", 6,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	"SELECT COUNT(*) FROM producto p WHERE p.nombre IN ("	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	+ "'Papas Kids','Puré de manzana','Yogur de fresa', "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	+ "'Jugo de manzana Kids','Juguete sorpresa','Coca-Cola 1.5 L') "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	+ "AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	+ "WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE) "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
52	+ "AND EXISTS (SELECT 1 FROM opcion_componente oc "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	+ "WHERE oc.id_producto=p.id_producto)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
54	comprobarCero(sentencia, "grupos activos sin opciones",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	"SELECT COUNT(*) FROM grupo_presentacion gp WHERE gp.estado=TRUE "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
56	+ "AND NOT EXISTS (SELECT 1 FROM opcion_grupo og "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	+ "WHERE og.id_grupo=gp.id_grupo AND og.estado=TRUE)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58	comprobarCero(sentencia, "opciones activas sin componentes",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	"SELECT COUNT(*) FROM opcion_grupo og WHERE og.estado=TRUE "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
60	+ "AND NOT EXISTS (SELECT 1 FROM opcion_componente oc "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	+ "WHERE oc.id_opcion=og.id_opcion)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
62	comprobar(sentencia, "presentaciones Big Mac", 2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	"SELECT COUNT(*) FROM presentacion_menu pm "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	+ "JOIN producto p ON p.id_producto=pm.id_producto_principal "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
65	+ "WHERE p.nombre='Big Mac' AND pm.tipo IN ('INDIVIDUAL','MENU')");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
66	comprobar(sentencia, "presentacion infantil de Cajita", 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	"SELECT COUNT(*) FROM presentacion_menu pm "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

68	+ "JOIN producto p ON p.id_producto=pm.id_producto_principal "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
69	+ "WHERE p.nombre='Cajita Feliz de Hamburguesa' AND pm.tipo='INFANTIL'");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
70	comprobar(sentencia, "grupo de cuatro principales de Caja Grande", 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	"SELECT COUNT(*) FROM grupo_presentacion gp "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	+ "JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
73	+ "JOIN producto p ON p.id_producto=pm.id_producto_principal "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
74	+ "WHERE p.nombre='Caja Grande' AND gp.minimo=4 "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
75	+ "AND gp.maximo=4 AND gp.permite_repetir=TRUE");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	int[] antes = cantidadesConfiguracion(sentencia);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
78	ejecutarArchivo(sentencia, Path.of("sql/datos_menu_nueva_base.sql"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	int[] despues = cantidadesConfiguracion(sentencia);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
80	if (!java.util.Arrays.equals(antes, despues)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
81	throw new AssertionError("la segunda carga duplicó datos: antes="	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
82	+ java.util.Arrays.toString(antes) + ", despues="	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
83	+ java.util.Arrays.toString(despues));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85	System.out.println("OK: ejecutar el archivo dos veces no duplica datos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	imprimirResumen(sentencia);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	imprimirRecetasCortas(sentencia);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	System.out.println("VALIDACION COMPLETA: todos los datos son compatibles con el esquema nuevo.")	Ejecuta una instrucción Java completa; el punto y coma marca su final.
	;	
90	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
91	sentencia.execute("DROP DATABASE IF EXISTS `" + base + "`");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	}	Cierra el bloque abierto anteriormente.
93	}	Cierra el bloque abierto anteriormente.
94	}	Cierra el bloque abierto anteriormente.
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96	private static void ejecutarArchivo(Statement sentencia, Path ruta) throws Exception {	Declara el método ejecutarArchivo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
97	String contenido = Files.readString(ruta, StandardCharsets.UTF_8);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
98	int ejecutadas = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
99	for (String sql : separarSentencias(contenido)) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
100	String limpia = sql.trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
101	String mayuscula = limpia.toUpperCase();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
102	if (limpia.isEmpty() mayuscula.startsWith("CREATE DATABASE"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
103	mayuscula.startsWith("USE ")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
104	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
105	}	Cierra el bloque abierto anteriormente.
106	sentencia.execute(limpia);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107	ejecutadas++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
108	}	Cierra el bloque abierto anteriormente.
109	System.out.println(ruta + ": " + ejecutadas + " sentencias ejecutadas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	}	Cierra el bloque abierto anteriormente.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.

112	<code>private static List<String> separarSentencias(String contenido) {</code>	Declara el método separarSentencias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
113	<code> List<String> sentencias = new ArrayList<>();</code>	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
114	<code> StringBuilder actual = new StringBuilder();</code>	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
115	<code> boolean comillaSimple = false;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
116	<code> boolean comillaDoble = false;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
117	<code> boolean acento = false;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
118	<code> boolean comentarioLinea = false;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119	<code> boolean comentarioBloque = false;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	<code> for (int i = 0; i < contenido.length(); i++) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
122	<code> char c = contenido.charAt(i);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
123	<code> char siguiente = i + 1 < contenido.length() ? contenido.charAt(i + 1) : '\0';</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	<code> if (comentarioLinea) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
126	<code> if (c == '\n') {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
127	<code> comentarioLinea = false;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
128	<code> actual.append(c);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129	<code> }</code>	Cierra el bloque abierto anteriormente.
130	<code> continue;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	<code> }</code>	Cierra el bloque abierto anteriormente.
132	<code> if (comentarioBloque) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
133	<code> if (c == '*' && siguiente == '/') {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
134	<code> comentarioBloque = false;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
135	<code> i++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	<code> }</code>	Cierra el bloque abierto anteriormente.
137	<code> continue;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	<code> }</code>	Cierra el bloque abierto anteriormente.
139	<code> if (!comillaSimple && !comillaDoble && !acento) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
140	<code> if (c == '-' && siguiente == '-') {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
141	<code> comentarioLinea = true;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
142	<code> i++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	<code> continue;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	<code> }</code>	Cierra el bloque abierto anteriormente.
145	<code> if (c == '/' && siguiente == '*') {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
146	<code> comentarioBloque = true;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
147	<code> i++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	<code> continue;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	<code> }</code>	Cierra el bloque abierto anteriormente.
150	<code> }</code>	Cierra el bloque abierto anteriormente.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	<code> if (c == '\\' && !comillaDoble && !acento) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
153	<code> if (comillaSimple && siguiente == '\\') {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
154	<code> actual.append(c).append(siguiente);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	<code> i++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.

156	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
157	}	Cierra el bloque abierto anteriormente.
158	if (i == 0 contenido.charAt(i - 1) != '\\') {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
159	comillaSimple = !comillaSimple;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
160	}	Cierra el bloque abierto anteriormente.
161	} else if (c == '"' && !comillaSimple && !acento	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
162	&& (i == 0 contenido.charAt(i - 1) != '\\')) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
163	comillaDoble = !comillaDoble;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
164	} else if (c == '\'' && !comillaSimple && !comillaDoble) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
165	acento = !acento;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
166	}	Cierra el bloque abierto anteriormente.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	if (c == ';' && !comillaSimple && !comillaDoble && !acento) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
169	sentencias.add(actual.toString());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	actual.setLength(0);	Configura una propiedad mediante setLength en el objeto indicado.
171	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
172	actual.append(c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173	}	Cierra el bloque abierto anteriormente.
174	}	Cierra el bloque abierto anteriormente.
175	if (!actual.toString().isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
176	sentencias.add(actual.toString());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177	}	Cierra el bloque abierto anteriormente.
178	return sentencias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
179	}	Cierra el bloque abierto anteriormente.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	private static int numero(Statement sentencia, String sql) throws Exception {	Declara el método numero. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
182	try (ResultSet resultado = sentencia.executeQuery(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
183	resultado.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
184	return resultado.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
185	}	Cierra el bloque abierto anteriormente.
186	}	Cierra el bloque abierto anteriormente.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	private static void comprobar(Statement sentencia, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	int esperado, String sql) throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	int obtenido = numero(sentencia, sql);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
191	if (obtenido != esperado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
192	throw new AssertionError(nombre + ": esperado=" + esperado + ", obtenido=" + obtenido);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
193	}	Cierra el bloque abierto anteriormente.
194	System.out.println("OK: " + nombre + " = " + obtenido);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
195	}	Cierra el bloque abierto anteriormente.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	private static void comprobarCero(Statement sentencia, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198	String sql) throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	comprobar(sentencia, nombre, 0, sql);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

200	}	Cierra el bloque abierto anteriormente.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	private static void imprimirResumen(Statement sentencia) throws Exception {	Declara el método imprimirResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
203	String sql = "SELECT tipo_stock, COUNT(*) FROM producto GROUP BY tipo_stock ORDER BY tipo_stock";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
204	try (ResultSet resultado = sentencia.executeQuery(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
205	while (resultado.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
206	System.out.println("RESUMEN: " + resultado.getString(1)	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
207	+ " = " + resultado.getInt(2));	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
208	}	Cierra el bloque abierto anteriormente.
209	}	Cierra el bloque abierto anteriormente.
210	}	Cierra el bloque abierto anteriormente.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	private static int[] cantidadesConfiguracion(Statement sentencia) throws Exception {	Declara el método cantidadesConfiguracion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
213	String[] tablas = {"producto", "ingrediente", "producto_ingrediente",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
214	"presentacion_menu", "grupo_presentacion", "opcion_grupo",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	"opcion_componente"};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
216	int[] cantidades = new int[tablas.length];	Crea una nueva instancia de int y la usa o guarda para continuar el proceso.
217	for (int i = 0; i < tablas.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
218	cantidades[i] = numero(sentencia, "SELECT COUNT(*) FROM " + tablas[i]);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
219	}	Cierra el bloque abierto anteriormente.
220	return cantidades;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
221	}	Cierra el bloque abierto anteriormente.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	private static void imprimirRecetasCortas(Statement sentencia) throws Exception {	Declara el método imprimirRecetasCortas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
224	String sql = "SELECT p.nombre, COUNT(pi.id_producto_ingrediente) cantidad "	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
225	+ "FROM producto p JOIN producto_ingrediente pi ON pi.id_producto=p.id_producto "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
226	+ "WHERE p.tipo_stock='RECETA' AND pi.estado=TRUE "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
227	+ "GROUP BY p.id_producto,p.nombre HAVING cantidad<=2 ORDER BY p.nombre";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
228	try (ResultSet resultado = sentencia.executeQuery(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
229	while (resultado.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
230	System.out.println("AUDITAR RECETA CORTA: " + resultado.getString(1)	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
231	+ " (" + resultado.getInt(2) + " ingredientes)");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
232	}	Cierra el bloque abierto anteriormente.
233	}	Cierra el bloque abierto anteriormente.
234	}	Cierra el bloque abierto anteriormente.
235	}	Cierra el bloque abierto anteriormente.

16.16 sql/agregar_wlmenu_producto_prueba.sql

Script SQL agregar_wlmenu_producto_prueba. Prepara, migra o prueba una parte de la base de datos.

Tipo: .sql | Líneas: 478 | Archivo 16 de 157

1	USE waldonalds;	Selecciona la base de datos donde se ejecutarán las siguientes sentencias.
2		Línea en blanco usada para separar secciones del script SQL.
3	START TRANSACTION;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
4		Línea en blanco usada para separar secciones del script SQL.
5	-- Producto al que se le agregará la presentación WlMenú.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
6	SET @id_hamburguesa = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
7	SELECT id_producto	Consulta columnas o valores calculados sin modificar los datos.
8	FROM producto	Indica la tabla principal desde la que se leen los datos.
9	WHERE nombre = 'Hamburguesa de Prueba POS'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
10	AND estado = TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
11	ORDER BY id_producto DESC	Ordena el resultado según las columnas indicadas.
12	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
13	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
14		Línea en blanco usada para separar secciones del script SQL.
15	-- Se utiliza la misma categoría activa del producto principal.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
16	SET @id_categoria = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
17	SELECT id_categoria	Consulta columnas o valores calculados sin modificar los datos.
18	FROM producto	Indica la tabla principal desde la que se leen los datos.
19	WHERE id_producto = @id_hamburguesa	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
20	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
21	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
22		Línea en blanco usada para separar secciones del script SQL.
23	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
24	-- COMPONENTE: PAPAS MEDIANAS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
25	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
26		Línea en blanco usada para separar secciones del script SQL.
27	SET @id_papas = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
28	SELECT id_producto	Consulta columnas o valores calculados sin modificar los datos.
29	FROM producto	Indica la tabla principal desde la que se leen los datos.
30	WHERE nombre = 'Papas Medianas WlMenú'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
31	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
32	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
33		Línea en blanco usada para separar secciones del script SQL.
34	INSERT INTO producto (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
35	id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
36	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
37	descripcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
38	precio_base,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

39	imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
40	subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
41	disponibilidad_menu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
42	tipo_stock,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
43	personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
44	stock_actual,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
45	stock_minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
46	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
47	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
48	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
49	@id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
50	'Papas Medianas wlMenú',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
51	'Papas medianas utilizadas como complemento del wlMenú.',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
52	15.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
53	NULL,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
54	'Papas',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
55	'TODO_DIA',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
56	'DIRECTO',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
57	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
58	100,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
59	10,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
60	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
61	WHERE @id_papas IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
62		Línea en blanco usada para separar secciones del script SQL.
63	SET @id_papas = COALESCE(@id_papas, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
64		Línea en blanco usada para separar secciones del script SQL.
65		Línea en blanco usada para separar secciones del script SQL.
66	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
67	-- COMPONENTES: BEBIDAS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
68	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
69		Línea en blanco usada para separar secciones del script SQL.
70	SET @id_coca = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
71	SELECT id_producto	Consulta columnas o valores calculados sin modificar los datos.
72	FROM producto	Indica la tabla principal desde la que se leen los datos.
73	WHERE nombre = 'Coca Cola Mediana wlMenú'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
74	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
75	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
76		Línea en blanco usada para separar secciones del script SQL.
77	INSERT INTO producto (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
78	id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
79	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
80	descripcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
81	precio_base,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
82	imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

83	subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
84	disponibilidad_menu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
85	tipo_stock,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
86	personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
87	stock_actual,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
88	stock_minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
89	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
90	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
91	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
92	@id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
93	'Coca Cola Mediana wlMenú',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
94	'Bebida mediana para el wlMenú.',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
95	12.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
96	NULL,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
97	'Bebidas',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
98	'TODO_DIA',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
99	'DIRECTO',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
100	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
101	100,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
102	10,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
103	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
104	WHERE @id_coca IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
105		Línea en blanco usada para separar secciones del script SQL.
106	SET @id_coca = COALESCE(@id_coca, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
107		Línea en blanco usada para separar secciones del script SQL.
108		Línea en blanco usada para separar secciones del script SQL.
109	SET @id_sprite = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
110	SELECT id_producto	Consulta columnas o valores calculados sin modificar los datos.
111	FROM producto	Indica la tabla principal desde la que se leen los datos.
112	WHERE nombre = 'Sprite Mediana wlMenú'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
113	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
114	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
115		Línea en blanco usada para separar secciones del script SQL.
116	INSERT INTO producto (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
117	id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
118	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
119	descripcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
120	precio_base,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
121	imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
122	subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
123	disponibilidad_menu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
124	tipo_stock,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
125	personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
126	stock_actual,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

127	stock_minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
128	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
129	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
130	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
131	@id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
132	'Sprite Mediana wlMenú',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
133	'Bebida mediana para el wlMenú.',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
134	12.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
135	NULL,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
136	'Bebidas',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
137	'TODO_DIA',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
138	'DIRECTO',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
139	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
140	100,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
141	10,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
142	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
143	WHERE @id_sprite IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
144		Línea en blanco usada para separar secciones del script SQL.
145	SET @id_sprite = COALESCE(@id_sprite, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
146		Línea en blanco usada para separar secciones del script SQL.
147		Línea en blanco usada para separar secciones del script SQL.
148	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
149	-- PRESENTACIÓN WLMENÚ	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
150	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
151		Línea en blanco usada para separar secciones del script SQL.
152	SET @id_wlmenu = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
153	SELECT id_presentacion	Consulta columnas o valores calculados sin modificar los datos.
154	FROM presentacion_menu	Indica la tabla principal desde la que se leen los datos.
155	WHERE id_producto_principal = @id_hamburguesa	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
156	AND tipo = 'MENU'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
157	AND estado = TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
158	ORDER BY id_presentacion	Ordena el resultado según las columnas indicadas.
159	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
160	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
161		Línea en blanco usada para separar secciones del script SQL.
162	INSERT INTO presentacion_menu (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
163	id_producto_principal,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
164	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
165	tipo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
166	precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
167	predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
168	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
169	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
170	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

171	@id_hamburguesa,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
172	'wLMenú',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
173	'MENU',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
174	55.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
175	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
176	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
177	WHERE @id_wlmenu IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
178		Línea en blanco usada para separar secciones del script SQL.
179	SET @id_wlmenu = COALESCE(@id_wlmenu, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
180		Línea en blanco usada para separar secciones del script SQL.
181		Línea en blanco usada para separar secciones del script SQL.
182	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
183	-- GRUPO INTERNO: HAMBURGUESA PRINCIPAL	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
184	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
185		Línea en blanco usada para separar secciones del script SQL.
186	SET @id_grupo_principal = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
187	SELECT id_grupo	Consulta columnas o valores calculados sin modificar los datos.
188	FROM grupo_presentacion	Indica la tabla principal desde la que se leen los datos.
189	WHERE id_presentacion = @id_wlmenu	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
190	AND nombre = 'Producto principal'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
191	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
192	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
193		Línea en blanco usada para separar secciones del script SQL.
194	INSERT INTO grupo_presentacion (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
195	id_presentacion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
196	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
197	minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
198	maximo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
199	permite_repetir,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
200	visible,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
201	permite_personalizar,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
202	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
203	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
204	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
205	@id_wlmenu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
206	'Producto principal',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
207	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
208	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
209	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
210	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
211	TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
212	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
213	WHERE @id_grupo_principal IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
214		Línea en blanco usada para separar secciones del script SQL.

215	SET @id_grupo_principal = COALESCE(	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
216	@id_grupo_principal,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
217	LAST_INSERT_ID()	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
218	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
219		Línea en blanco usada para separar secciones del script SQL.
220	SET @id_opcion_principal = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
221	SELECT id_opcion	Consulta columnas o valores calculados sin modificar los datos.
222	FROM opcion_grupo	Indica la tabla principal desde la que se leen los datos.
223	WHERE id_grupo = @id_grupo_principal	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
224	AND nombre = 'Hamburguesa de Prueba POS'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
225	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
226	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
227		Línea en blanco usada para separar secciones del script SQL.
228	INSERT INTO opcion_grupo (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
229	id_grupo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
230	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
231	incremento_precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
232	predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
233	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
234	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
235	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
236	@id_grupo_principal,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
237	'Hamburguesa de Prueba POS',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
238	0.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
239	TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
240	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
241	WHERE @id_opcion_principal IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
242		Línea en blanco usada para separar secciones del script SQL.
243	SET @id_opcion_principal = COALESCE(	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
244	@id_opcion_principal,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
245	LAST_INSERT_ID()	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
246	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
247		Línea en blanco usada para separar secciones del script SQL.
248	INSERT INTO opcion_componente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
249	id_opcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
250	id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
251	cantidad	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
252	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
253	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
254	@id_opcion_principal,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
255	@id_hamburguesa,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
256	1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
257	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
258	SELECT 1	Consulta columnas o valores calculados sin modificar los datos.

259	FROM opcion_componente	Indica la tabla principal desde la que se leen los datos.
260	WHERE id_opcion = @id_opcion_principal	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
261	AND id_producto = @id_hamburguesa	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
262	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
263		Línea en blanco usada para separar secciones del script SQL.
264		Línea en blanco usada para separar secciones del script SQL.
265	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
266	-- GRUPO VISIBLE: COMPLEMENTO	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
267	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
268		Línea en blanco usada para separar secciones del script SQL.
269	SET @id_grupo_complemento = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
270	SELECT id_grupo	Consulta columnas o valores calculados sin modificar los datos.
271	FROM grupo_presentacion	Indica la tabla principal desde la que se leen los datos.
272	WHERE id_presentacion = @id_wlmenu	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
273	AND nombre = 'Elige el complemento'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
274	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
275	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
276		Línea en blanco usada para separar secciones del script SQL.
277	INSERT INTO grupo_presentacion (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
278	id_presentacion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
279	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
280	minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
281	maximo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
282	permite_repetir,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
283	visible,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
284	permite_personalizar,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
285	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
286	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
287	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
288	@id_wlmenu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
289	'Elige el complemento',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
290	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
291	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
292	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
293	TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
294	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
295	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
296	WHERE @id_grupo_complemento IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
297		Línea en blanco usada para separar secciones del script SQL.
298	SET @id_grupo_complemento = COALESCE(	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
299	@id_grupo_complemento,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
300	LAST_INSERT_ID()	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
301	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
302		Línea en blanco usada para separar secciones del script SQL.

303 SET @id_opcion_papas = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
304 SELECT id_opcion	Consulta columnas o valores calculados sin modificar los datos.
305 FROM opcion_grupo	Indica la tabla principal desde la que se leen los datos.
306 WHERE id_grupo = @id_grupo_complemento	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
307 AND nombre = 'Papas medianas'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
308 LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
309);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
310	Línea en blanco usada para separar secciones del script SQL.
311 INSERT INTO opcion_grupo (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
312 id_grupo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
313 nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
314 incremento_precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
315 predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
316 estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
317)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
318 SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
319 @id_grupo_complemento,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
320 'Papas medianas',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
321 0.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
322 TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
323 TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
324 WHERE @id_opcion_papas IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
325	Línea en blanco usada para separar secciones del script SQL.
326 SET @id_opcion_papas = COALESCE(@id_opcion_papas, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
327	Línea en blanco usada para separar secciones del script SQL.
328 INSERT INTO opcion_componente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
329 id_opcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
330 id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
331 cantidad	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
332)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
333 SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
334 @id_opcion_papas,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
335 @id_papas,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
336 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
337 WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
338 SELECT 1	Consulta columnas o valores calculados sin modificar los datos.
339 FROM opcion_componente	Indica la tabla principal desde la que se leen los datos.
340 WHERE id_opcion = @id_opcion_papas	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
341 AND id_producto = @id_papas	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
342);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
343	Línea en blanco usada para separar secciones del script SQL.
344	Línea en blanco usada para separar secciones del script SQL.
345 -- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
346 -- GRUPO VISIBLE: BEBIDA	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.

347	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
348		Línea en blanco usada para separar secciones del script SQL.
349	SET @id_grupo_bebida = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
350	SELECT id_grupo	Consulta columnas o valores calculados sin modificar los datos.
351	FROM grupo_presentacion	Indica la tabla principal desde la que se leen los datos.
352	WHERE id_presentacion = @id_wlmenu	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
353	AND nombre = 'Elige la bebida'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
354	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
355	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
356		Línea en blanco usada para separar secciones del script SQL.
357	INSERT INTO grupo_presentacion (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
358	id_presentacion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
359	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
360	minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
361	maximo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
362	permite_repetir,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
363	visible,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
364	permite_personalizar,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
365	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
366	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
367	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
368	@id_wlmenu,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
369	'Elige la bebida',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
370	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
371	1,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
372	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
373	TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
374	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
375	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
376	WHERE @id_grupo_bebida IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
377		Línea en blanco usada para separar secciones del script SQL.
378	SET @id_grupo_bebida = COALESCE(	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
379	@id_grupo_bebida,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
380	LAST_INSERT_ID()	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
381	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
382		Línea en blanco usada para separar secciones del script SQL.
383	SET @id_opcion_coca = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
384	SELECT id_opcion	Consulta columnas o valores calculados sin modificar los datos.
385	FROM opcion_grupo	Indica la tabla principal desde la que se leen los datos.
386	WHERE id_grupo = @id_grupo_bebida	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
387	AND nombre = 'Coca Cola mediana'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
388	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
389	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
390		Línea en blanco usada para separar secciones del script SQL.

391	INSERT INTO opcion_grupo (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
392	id_grupo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
393	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
394	incremento_precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
395	predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
396	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
397	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
398	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
399	@id_grupo_bebida,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
400	'Coca Cola mediana',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
401	0.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
402	TRUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
403	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
404	WHERE @id_opcion_coca IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
405		Línea en blanco usada para separar secciones del script SQL.
406	SET @id_opcion_coca = COALESCE(@id_opcion_coca, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
407		Línea en blanco usada para separar secciones del script SQL.
408	INSERT INTO opcion_componente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
409	id_opcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
410	id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
411	cantidad	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
412	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
413	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
414	@id_opcion_coca,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
415	@id_coca,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
416	1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
417	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
418	SELECT 1	Consulta columnas o valores calculados sin modificar los datos.
419	FROM opcion_componente	Indica la tabla principal desde la que se leen los datos.
420	WHERE id_opcion = @id_opcion_coca	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
421	AND id_producto = @id_coca	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
422	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
423		Línea en blanco usada para separar secciones del script SQL.
424		Línea en blanco usada para separar secciones del script SQL.
425	SET @id_opcion_sprite = (	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
426	SELECT id_opcion	Consulta columnas o valores calculados sin modificar los datos.
427	FROM opcion_grupo	Indica la tabla principal desde la que se leen los datos.
428	WHERE id_grupo = @id_grupo_bebida	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
429	AND nombre = 'Sprite mediana'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
430	LIMIT 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
431	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
432		Línea en blanco usada para separar secciones del script SQL.
433	INSERT INTO opcion_grupo (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
434	id_grupo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

435	nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
436	incremento_precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
437	predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
438	estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
439	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
440	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
441	@id_grupo_bebida,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
442	'Sprite mediana',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
443	0.00,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
444	FALSE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
445	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
446	WHERE @id_opcion_sprite IS NULL;	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
447		Línea en blanco usada para separar secciones del script SQL.
448	SET @id_opcion_sprite = COALESCE(@id_opcion_sprite, LAST_INSERT_ID());	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
449		Línea en blanco usada para separar secciones del script SQL.
450	INSERT INTO opcion_componente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
451	id_opcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
452	id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
453	cantidad	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
454	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
455	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
456	@id_opcion_sprite,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
457	@id_sprite,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
458	1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
459	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
460	SELECT 1	Consulta columnas o valores calculados sin modificar los datos.
461	FROM opcion_componente	Indica la tabla principal desde la que se leen los datos.
462	WHERE id_opcion = @id_opcion_sprite	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
463	AND id_producto = @id_sprite	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
464	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
465		Línea en blanco usada para separar secciones del script SQL.
466	COMMIT;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
467		Línea en blanco usada para separar secciones del script SQL.
468	-- Comprobación final. Deben aparecer dos filas: Individual y wlMenú.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
469	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
470	pm.id_presentacion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
471	pm.nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
472	pm.tipo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
473	pm.precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
474	pm.predeterminada,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
475	pm.estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
476	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
477	WHERE pm.id_producto_principal = @id_hamburguesa	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
478	ORDER BY pm.tipo DESC;	Ordena el resultado según las columnas indicadas.

16.17 sql/base_datos_completa.sql

Esquema principal. Crea la base waldonalds y sus tablas relacionales.

Tipo: .sql | Líneas: 197 | Archivo 17 de 157

1	CREATE DATABASE IF NOT EXISTS waldonalds;	Crea la base de datos si todavía no existe.
2	USE waldonalds;	Selecciona la base de datos donde se ejecutarán las siguientes sentencias.
3		Línea en blanco usada para separar secciones del script SQL.
4	CREATE TABLE usuario (	Comienza la definición de la tabla usuario.
5	id_usuario INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
6	nombre VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
7	apellido VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
8	usuario VARCHAR(60) NOT NULL UNIQUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
9	correo VARCHAR(150) NOT NULL UNIQUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
10	password_hash VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
11	rol ENUM('ADMINISTRADOR', 'CAJERO') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
12	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
13	fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,	Define una columna obligatoria; no se permite guardar NULL en ella.
14	turno ENUM('MANANA', 'TARDE'),	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
15	hora_inicio TIME,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
16	hora_fin TIME	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
17	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
18		Línea en blanco usada para separar secciones del script SQL.
19	CREATE TABLE categoria (	Comienza la definición de la tabla categoria.
20	id_categoria INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
21	nombre VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
22	estado BOOLEAN NOT NULL DEFAULT TRUE	Define una columna obligatoria; no se permite guardar NULL en ella.
23	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
24		Línea en blanco usada para separar secciones del script SQL.
25	CREATE TABLE ingrediente (	Comienza la definición de la tabla ingrediente.
26	id_ingrediente INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
27	nombre VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
28	unidad_medida VARCHAR(30) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
29	stock_actual DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
30	stock_minimo DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
31	estado BOOLEAN NOT NULL DEFAULT TRUE	Define una columna obligatoria; no se permite guardar NULL en ella.
32	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
33		Línea en blanco usada para separar secciones del script SQL.
34	CREATE TABLE producto (	Comienza la definición de la tabla producto.
35	id_producto INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
36	id_categoria INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
37	nombre VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
38	descripcion VARCHAR(500),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

39	precio_base DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
40	imagen VARCHAR(500),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
41	subcategoria VARCHAR(100),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
42	disponibilidad_menu ENUM('TODO_DIA', 'DESAYUNO', 'ALMUERZO') NOT NULL DEFAULT 'TODO_DIA',	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
43	tipo_stock ENUM('RECETA', 'DIRECTO', 'NINGUNO') NOT NULL DEFAULT 'DIRECTO',	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
44	personalizable BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
45	stock_actual INT NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
46	stock_minimo INT NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
47	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
48	FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
49	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
50		Línea en blanco usada para separar secciones del script SQL.
51	CREATE TABLE producto_ingrediente (	Comienza la definición de la tabla producto_ingrediente.
52	id_producto_ingrediente INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
53	id_producto INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
54	id_ingrediente INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
55	cantidad_default DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
56	permite_quitar BOOLEAN NOT NULL DEFAULT FALSE,	Define una columna obligatoria; no se permite guardar NULL en ella.
57	permite_extra BOOLEAN NOT NULL DEFAULT FALSE,	Define una columna obligatoria; no se permite guardar NULL en ella.
58	cantidad_extra DECIMAL(12,2) NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
59	precio_extra DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
60	max_extras INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
61	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
62	FOREIGN KEY (id_producto) REFERENCES producto(id_producto),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
63	FOREIGN KEY (id_ingrediente) REFERENCES ingrediente(id_ingrediente)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
64	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
65		Línea en blanco usada para separar secciones del script SQL.
66	CREATE TABLE presentacion_menu (	Comienza la definición de la tabla presentacion_menu.
67	id_presentacion INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
68	id_producto_principal INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
69	nombre VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
70	tipo ENUM('INDIVIDUAL', 'MENU', 'INFANTIL', 'COMBO') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
71	precio DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
72	predeterminada BOOLEAN NOT NULL DEFAULT FALSE,	Define una columna obligatoria; no se permite guardar NULL en ella.
73	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
74	FOREIGN KEY (id_producto_principal) REFERENCES producto(id_producto)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
75	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
76		Línea en blanco usada para separar secciones del script SQL.
77	CREATE TABLE grupo_presentacion (	Comienza la definición de la tabla grupo_presentacion.
78	id_grupo INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
79	id_presentacion INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
80	nombre VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
81	minimo INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
82	maximo INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.

83	permite_repetir BOOLEAN NOT NULL DEFAULT FALSE,	Define una columna obligatoria; no se permite guardar NULL en ella.
84	visible BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
85	permite_personalizar BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
86	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
87	FOREIGN KEY (id_presentacion) REFERENCES presentacion_menu(id_presentacion)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
88	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
89		Línea en blanco usada para separar secciones del script SQL.
90	CREATE TABLE opcion_grupo (	Comienza la definición de la tabla opcion_grupo.
91	id_opcion INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
92	id_grupo INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
93	nombre VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
94	incremento_precio DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
95	predeterminada BOOLEAN NOT NULL DEFAULT FALSE,	Define una columna obligatoria; no se permite guardar NULL en ella.
96	estado BOOLEAN NOT NULL DEFAULT TRUE,	Define una columna obligatoria; no se permite guardar NULL en ella.
97	FOREIGN KEY (id_grupo) REFERENCES grupo_presentacion(id_grupo)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
98	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
99		Línea en blanco usada para separar secciones del script SQL.
100	CREATE TABLE opcion_componente (	Comienza la definición de la tabla opcion_componente.
101	id_opcion_componente INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
102	id_opcion INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
103	id_producto INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
104	cantidad INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
105	FOREIGN KEY (id_opcion) REFERENCES opcion_grupo(id_opcion),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
106	FOREIGN KEY (id_producto) REFERENCES producto(id_producto)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
107	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
108		Línea en blanco usada para separar secciones del script SQL.
109	CREATE TABLE pedido (	Comienza la definición de la tabla pedido.
110	id_pedido INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
111	numero_orden INT NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
112	id_usuario INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
113	tipo_servicio ENUM('COMER_AQUI', 'PARA_LLEVAR') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
114	estado ENUM('PAGADO', 'CANCELADO') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
115	metodo_pago ENUM('EFECTIVO', 'TARJETA'),	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
116	monto_recibido DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
117	total DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
118	fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,	Define una columna obligatoria; no se permite guardar NULL en ella.
119	FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
120	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
121		Línea en blanco usada para separar secciones del script SQL.
122	CREATE TABLE pedido_detalle (	Comienza la definición de la tabla pedido_detalle.
123	id_detalle INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
124	id_pedido INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
125	id_presentacion INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
126	nombre VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.

127	presentacion VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
128	cantidad INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
129	precio_unitario DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
130	subtotal DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
131	FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
132	FOREIGN KEY (id_presentacion) REFERENCES presentacion_menu(id_presentacion)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
133	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
134		Línea en blanco usada para separar secciones del script SQL.
135	CREATE TABLE pedido_opcion (	Comienza la definición de la tabla pedido_opcion.
136	id_pedido_opcion INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
137	id_detalle INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
138	id_grupo INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
139	id_opcion INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
140	numero INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
141	grupo VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
142	opcion VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
143	precio_extra DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
144	FOREIGN KEY (id_detalle) REFERENCES pedido_detalle(id_detalle),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
145	FOREIGN KEY (id_grupo) REFERENCES grupo_presentacion(id_grupo),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
146	FOREIGN KEY (id_opcion) REFERENCES opcion_grupo(id_opcion)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
147	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
148		Línea en blanco usada para separar secciones del script SQL.
149	CREATE TABLE pedido_producto (	Comienza la definición de la tabla pedido_producto.
150	id_pedido_producto INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
151	id_pedido_opcion INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
152	id_producto INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
153	nombre VARCHAR(150) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
154	cantidad INT NOT NULL DEFAULT 1,	Define una columna obligatoria; no se permite guardar NULL en ella.
155	FOREIGN KEY (id_pedido_opcion) REFERENCES pedido_opcion(id_pedido_opcion),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
156	FOREIGN KEY (id_producto) REFERENCES producto(id_producto)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
157	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
158		Línea en blanco usada para separar secciones del script SQL.
159	CREATE TABLE pedido_modificacion (	Comienza la definición de la tabla pedido_modificacion.
160	id_modificacion INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
161	id_pedido_producto INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
162	id_producto_ingrediente INT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
163	tipo ENUM('SIN', 'EXTRA') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
164	ingrediente VARCHAR(100) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
165	cantidad DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
166	precio_extra DECIMAL(12,2) NOT NULL DEFAULT 0,	Define una columna obligatoria; no se permite guardar NULL en ella.
167	FOREIGN KEY (id_pedido_producto) REFERENCES pedido_producto(id_pedido_producto),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
168	FOREIGN KEY (id_producto_ingrediente) REFERENCES producto_ingrediente(id_producto_ingrediente)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
169	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
170		Línea en blanco usada para separar secciones del script SQL.

171	CREATE TABLE movimiento_inventario (	Comienza la definición de la tabla movimiento_inventario.
172	id_movimiento INT AUTO_INCREMENT PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
173	id_producto INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
174	id_ingredient INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
175	id_pedido INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
176	tipo_movimiento ENUM('ENTRADA','SALIDA','AJUSTE') NOT NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
177	cantidad DECIMAL(12,2) NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
178	motivo VARCHAR(150),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
179	fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,	Define una columna obligatoria; no se permite guardar NULL en ella.
180	FOREIGN KEY (id_producto) REFERENCES producto(id_producto),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
181	FOREIGN KEY (id_ingredient) REFERENCES ingrediente(id_ingredient),	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
182	FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
183	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
184		Línea en blanco usada para separar secciones del script SQL.
185	CREATE TABLE pago_operacion (	Comienza la definición de la tabla pago_operacion.
186	clave VARCHAR(36) PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.
187	solicitud LONGTEXT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.
188	id_pedido INT UNIQUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
189	comprobante LONGTEXT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
190	referencia VARCHAR(100) NOT NULL DEFAULT '',	Define una columna obligatoria; no se permite guardar NULL en ella.
191	FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.
192	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
193		Línea en blanco usada para separar secciones del script SQL.
194	-- Cada producto vendible necesita al menos una presentación.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
195	-- Ejemplo de presentación individual para un producto existente:	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
196	-- INSERT INTO presentacion_menu(id_producto_principal,nombre,tipo,precio,predeterminada)	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
197	-- VALUES (1,'Individual','INDIVIDUAL',41.00,TRUE);	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.

16.18 sql/datos_menu_nueva_base.sql

Carga idempotente del catálogo, ingredientes, recetas, presentaciones, WlMenús, Cajitas y combos.

Tipo: .sql | Líneas: 1544 | Archivo 18 de 157

1	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
2	-- DATOS COMPLETOS PARA LA NUEVA BASE WALDONALDS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
3	-- Generado desde los INSERT antiguos y adaptado al modelo de	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
4	-- recetas, presentaciones, WlMenús, Cajitas y combos.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
5	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
6		Línea en blanco usada para separar secciones del script SQL.
7	USE waldonalds;	Selecciona la base de datos donde se ejecutarán las siguientes sentencias.
8		Línea en blanco usada para separar secciones del script SQL.
9	START TRANSACTION;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
10		Línea en blanco usada para separar secciones del script SQL.
11	-- Recargo por cambiar una porción de Papas por WlPatatas.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
12	-- Modifica únicamente este valor si cambia el precio del reemplazo.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
13	SET @precio_extra_wlpatatas = 5.00;	Guarda un valor en una variable de sesión de MySQL para reutilizarlo más adelante.
14		Línea en blanco usada para separar secciones del script SQL.
15	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
16	-- 1. CATEGORÍAS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
17	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
18		Línea en blanco usada para separar secciones del script SQL.
19	INSERT INTO categoria (nombre, estado)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
20	SELECT datos.nombre, TRUE	Consulta columnas o valores calculados sin modificar los datos.
21	FROM (	Indica la tabla principal desde la que se leen los datos.
22	SELECT 'Antojos' AS nombre	Consulta columnas o valores calculados sin modificar los datos.
23	UNION ALL SELECT 'Bebidas'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
24	UNION ALL SELECT 'Cajita Feliz'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
25	UNION ALL SELECT 'Desayunos'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
26	UNION ALL SELECT 'Almuerzos'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
27	UNION ALL SELECT 'Postres'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
28	UNION ALL SELECT 'WlCafé'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
29	) datos	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
30	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
31	SELECT 1 FROM categoria c WHERE c.nombre = datos.nombre	Consulta columnas o valores calculados sin modificar los datos.
32	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
33		Línea en blanco usada para separar secciones del script SQL.
34	UPDATE categoria	Actualiza filas existentes de la tabla indicada.
35	SET estado = TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
36	WHERE nombre IN ('Antojos', 'Bebidas', 'Cajita Feliz', 'Desayunos',	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
37	'Almuerzos', 'Postres', 'WlCafé');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
38		Línea en blanco usada para separar secciones del script SQL.

39	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
40	-- 2. PRODUCTOS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
41	-- Se usa una tabla temporal para evitar duplicados si el archivo	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
42	-- se ejecuta más de una vez.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
43	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
44		Línea en blanco usada para separar secciones del script SQL.
45	DROP TEMPORARY TABLE IF EXISTS carga_producto;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
46		Línea en blanco usada para separar secciones del script SQL.
47	CREATE TEMPORARY TABLE carga_producto (	Crea una tabla temporal que existe solo durante la sesión de carga y ayuda a fusionar datos sin duplicarlos.
48	categoria VARCHAR(100),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
49	subcategoria VARCHAR(100),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
50	nombre VARCHAR(150),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
51	descripcion VARCHAR(500),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
52	precio DECIMAL(12,2),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
53	imagen VARCHAR(500),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
54	disponibilidad VARCHAR(20),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
55	tipo_stock VARCHAR(20),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
56	personalizable BOOLEAN,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
57	stock_actual INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
58	stock_minimo INT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
59	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
60		Línea en blanco usada para separar secciones del script SQL.
61	INSERT INTO carga_producto (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
62	categoria, subcategoria, nombre, descripcion, precio, imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
63	disponibilidad, tipo_stock, personalizable, stock_actual, stock_minimo	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
64	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
65	VALUES	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
66	('Almuerzos','Otras opciones','10 WlNuggets de Pollo','Diez piezas de pollo empanizado, tiernas por dentro y cru jientes por fuera.','39.00','/Imágenes/HAMBURGUESAS/10 mcnuggets de pollo.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
67	('Almuerzos','Hamburguesas','Bacon Cheddar WlMelt','Hamburguesa con carne, tocino y queso cheddar derretido.','49 .00','/Imágenes/HAMBURGUESAS/bacon_cheddar_mcmelt.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
68	('Almuerzos','Hamburguesas','Big Mac','Hamburguesa con dos tortas de res, queso, lechuga, pepinillos y salsa esp ecial.','41.00','/Imágenes/HAMBURGUESAS/bigmac.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
69	('Almuerzos','Hamburguesas','Big Tasty','Hamburguesa de res con queso, vegetales y salsa estilo Sabrosa.','48.00, /Imágenes/HAMBURGUESAS/big_tasty.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
70	('Almuerzos','Hamburguesas','Big Tasty Bacon','Hamburguesa de res con tocino, queso, vegetales y salsa estilo Sa brosa.','54.00','/Imágenes/HAMBURGUESAS/bigtasty_bacon.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
71	('Almuerzos','Hamburguesas','Big Tasty Bacon Doble','Hamburguesa doble de res con tocino, queso, vegetales y sal sa estilo Sabrosa.','65.00','/Imágenes/HAMBURGUESAS/bigtasty_bacon_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
72	('Almuerzos','Pollo','Big Tasty de Pollo','Hamburguesa de pollo con queso, vegetales y salsa estilo Sabrosa.','47 .00','/Imágenes/HAMBURGUESAS/big_tasty_pollo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
73	('Almuerzos','Para compartir','Bucket para todos','Cubeta de pollo crujiente pensado para compartir entre varias personas.','179.00','/Imágenes/PARA_COMPARTIR/bucket para todos.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
74	('Almuerzos','Para compartir','Bucket Pollo WlCrispy','Cubeta con selección de piezas de pollo crujiente para co mpartir.','179.00','/Imágenes/PARA_COMPARTIR/Bucket Pollo McCrispy.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
75	('Almuerzos','Para compartir','Bucket Pollo WlCrispy Para Tres','Cubeta de pollo crujiente ideal para compartir entre tres personas.','179.00','/Imágenes/PARA_COMPARTIR/Bucket Pollo McCrispy @ Para Tres.png','TODO_DIA','NINGUN O',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

76	('Almuerzos','Para compartir','Bucket Pollo WlCrispy Snack','Cubeta tipo aperitivo con piezas de pollo crujiente .',110.00,'/Imágenes/PARA COMPARTIR/Bucket Pollo McCrispy @ Snack.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
77	('Almuerzos','Para compartir','Caja de 24 WlNuggets','Caja para compartir con 24 bocaditos de pollo.',129.00,'/Imágenes/PARA COMPARTIR/caja de 24 wlnuggets.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
78	('Almuerzos','Para compartir','Caja Grande','Caja familiar con varias opciones principales, papas y bebida.',190.00,'/Imágenes/PARA COMPARTIR/caja grande.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
79	('Almuerzos','Para compartir','Caja Grande con Postre','Caja familiar con productos principales, papas, bebida y postres.',215.00,'/Imágenes/para compartir/caja grande con postre (1).png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
80	('Almuerzos','Para compartir','Caja Grande Snack','Caja para compartir con quesoburguesas, papas y bocaditos.',145.00,'/Imágenes/PARA COMPARTIR/caja grande snack.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
81	('Almuerzos','Creaciones Gourmet','Clásica Gourmet','Hamburguesa gourmet de res con queso, tomate, lechuga y cebolla.',49.00,'/Imágenes/HAMBURGUESAS/clasica_gourmet_res.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
82	('Almuerzos','Creaciones Gourmet','Clásica Gourmet Doble Res Doble','Hamburguesa gourmet doble de res con queso y vegetales frescos.',62.00,'/Imágenes/HAMBURGUESAS/clasica_gourmet_doble_res_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
83	('Almuerzos','Hamburguesas','Cuarto de Libra Bacon con Queso','Carne de res con queso y tocino estilo Cuarto de Libra.',48.00,'/Imágenes/HAMBURGUESAS/cuarto de libra bacon con queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
84	('Almuerzos','Hamburguesas','Cuarto de Libra Bacon Doble con Queso','Doble carne de res con queso y tocino estilo Cuarto de Libra.',61.00,'/Imágenes/HAMBURGUESAS/cuarto_de_libra_bacon_doble_queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
85	('Almuerzos','Hamburguesas','Cuarto de Libra con Queso','Carne de res con queso, cebolla, pepinillos, ketchup y mostaza.',39.00,'/Imágenes/HAMBURGUESAS/cuarto de libra con queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
86	('Almuerzos','Hamburguesas','Doble Big Mac','Gran Wl con cuatro tortas de res, queso, lechuga, pepinillos y salsa especial.',54.00,'/Imágenes/HAMBURGUESAS/doble_bigmac.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
87	('Almuerzos','Hamburguesas','Doble Cuarto de Libra con Queso','Doble carne de res con queso estilo Cuarto de Libra.',55.00,'/Imágenes/HAMBURGUESAS/doble_cuarto_libracon_queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
88	('Almuerzos','Hamburguesas','Doble Hamburguesa con Queso','Doble hamburguesa de res con queso cheddar y condimentos.',44.00,'/Imágenes/HAMBURGUESAS/doble_hamburguesa_con_queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
89	('Almuerzos','Hamburguesas','Hamburguesa','Hamburguesa clásica de res con cebolla, pepinillos, ketchup y mostaza.',20.00,'/Imágenes/HAMBURGUESAS/hamburguesa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
90	('Almuerzos','Hamburguesas','Hamburguesa con Queso','Hamburguesa de res con queso cheddar, pepinillos y condimentos.',32.00,'/Imágenes/HAMBURGUESAS/hamburguesacon_queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
91	('Almuerzos','Hamburguesas','Hamburguesa Deluxe','Hamburguesa de res con vegetales frescos y aderezo cremoso.',29.00,'/Imágenes/HAMBURGUESAS/hamburguesa_deluxe.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
92	('Almuerzos','Creaciones Gourmet','Pico Guacamol Gourmet Doble','Hamburguesa gourmet doble de res con guacamole y pico de gallo.',62.00,'/Imágenes/HAMBURGUESAS/pico_guacamol_gourmetdoble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
93	('Almuerzos','Creaciones Gourmet','Pico Guacamol Gourmet Res','Hamburguesa gourmet de res con guacamole, pico de gallo y queso.',49.00,'/Imágenes/HAMBURGUESAS/pico_guacamol_gourmet_res.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
94	('Almuerzos','Pollo','Pollo WlCrispy','Pollo crujiente sazonado, dorado por fuera y jugoso por dentro.',30.00,'/Imágenes/HAMBURGUESAS/pollo_wlcrispy.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
95	('Almuerzos','Pollo','Pollo WlCrispy 1 Pieza','Una pieza de pollo crujiente, dorada por fuera y jugosa por dentro.',25.00,'/Imágenes/HAMBURGUESAS/pollo_wlcrispy_1_pieza.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
96	('Almuerzos','Pollo','Pollo WlCrispy 10 Piezas','Diez piezas de pollo crujiente para compartir.',175.00,'/Imágenes/HAMBURGUESAS/pollo_mc_crispy_10_piezas.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
97	('Almuerzos','Pollo','Pollo WlCrispy Bacon Ranch','Pollo crujiente acompañado de tocino y salsa campestre.',50.00,'/Imágenes/HAMBURGUESAS/pollo_mc_crispy_bacon_ranch.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
98	('Almuerzos','Pollo','Pollo WlCrispy Dos Piezas','Dos piezas de pollo crujiente, doradas y jugosas.',40.00,'/Imágenes/HAMBURGUESAS/pollowlcrispy_dospiezas.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
99	('Almuerzos','Hamburguesas','Quesoburguesa','Hamburguesa de res con queso cheddar, pepinillos y condimentos.',32.00,'/Imágenes/HAMBURGUESAS/quesoburguesa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
100	('Almuerzos','Hamburguesas','Res','Hamburguesa de res con queso y vegetales frescos.',42.00,'/Imágenes/HAMBURGUESAS/hamburguesa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
101	('Almuerzos','Pollo','Sándwich WlPollo Doble','Sándwich doble de pollo con lechuga y mayonesa.',39.00,'/Imágenes/HAMBURGUESAS/sandwich_mcpollo_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

102	('Almuerzos','Creaciones Gourmet','Smoke Tocino Gourmet de Res','Hamburguesa gourmet de res con tocino, queso y cebolla crujiente.',50.00,'/Imágenes/HAMBURGUESAS/smoke_tocino_gourmetres.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
103	('Almuerzos','Creaciones Gourmet','Smoke Tocino Gourmet Doble','Hamburguesa gourmet doble de res con tocino, queso y cebolla crujiente.',63.00,'/Imágenes/HAMBURGUESAS/smoke_tocino_gourmet_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
104	('Almuerzos','Hamburguesas','Triple Bacon','Hamburguesa triple de res con queso y tocino.',46.00,'/Imágenes/HAMBURGUESAS/triple_bacon.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
105	('Almuerzos','Hamburguesas','Triple Hamburguesa con Queso','Tres tortas de res acompañadas de queso y condimento s.',46.00,'/Imágenes/HAMBURGUESAS/triplehamburguesa_queso.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
106	('Almuerzos','Pollo','WlCrispy Chicken Deluxe','Sándwich de pollo crujiente con vegetales frescos y aderezo.',39.00,'/Imágenes/HAMBURGUESAS/WlCrispy_Chicken_Deluxe.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
107	('Almuerzos','Hamburguesas','WlDouble','Doble hamburguesa de res con queso y condimentos.',44.00,'/Imágenes/HAMBURGUESAS/wcdouble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
108	('Almuerzos','Hamburguesas','WlNifica de Res Doble','Hamburguesa doble de res con queso, lechuga, tomate y cebolla.',57.00,'/Imágenes/HAMBURGUESAS/mcnifica de res doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
109	('Almuerzos','Hamburguesas','WlNifica Doble','Hamburguesa doble de res con queso, lechuga, tomate y cebolla.',57.00,'/Imágenes/HAMBURGUESAS/mcnifica_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
110	('Almuerzos','Hamburguesas','WlNifica Res Doble','Hamburguesa doble de res con queso, lechuga, tomate y cebolla.',57.00,'/Imágenes/HAMBURGUESAS/mcnifica_res_doble.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
111	('Almuerzos','Otras opciones','WlNuggets','Piezas de pollo empanizado, tiernas por dentro y crujientes por fuera.',39.00,'/Imágenes/HAMBURGUESAS/wlnuggets.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
112	('Antojos','Antojos Almuerzo y Cena','Derretido clásico','Pan ligeramente tostado con queso caliente y derretido.',22.00,'/Imágenes/ANTOJOS/derretido clasico.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
113	('Antojos','Antojos Desayuno','Hash Brown','Papa rallada dorada y crujiente, ideal para acompañar el desayuno.',18.00,'/Imágenes/ANTOJOS/hash brown.png','DESAYUNO','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
114	('Antojos','Antojos Almuerzo y Cena','Papas','Papas doradas y crujientes, servidas como acompañamiento.',28.00,'/Imágenes/ANTOJOS/papas.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
115	('Antojos','Antojos Almuerzo y Cena','Salsa cheddar','Salsa cremosa de queso cheddar para acompañar tus favorito s.',15.00,'/Imágenes/ANTOJOS/salsa cheddar.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
116	('Antojos','Antojos Almuerzo y Cena','Tostado de loroco','Tostado caliente con queso y un toque de loroco.',25.00,'/Imágenes/ANTOJOS/tostado de loroco.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
117	('Antojos','Antojos Almuerzo y Cena','Tostado queso y frijol','Tostado caliente con queso y frijoles.',25.00,'/Imágenes/ANTOJOS/tostado queso y frijol.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
118	('Antojos','Antojos Almuerzo y Cena','Tostado queso y jamón','Tostado caliente con queso y jamón.',25.00,'/Imágenes/ANTOJOS/tostado queso y jamon.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
119	('Antojos','Antojos Almuerzo y Cena','Tostado queso y tomate','Tostado caliente con queso y tomate.',25.00,'/Imágenes/ANTOJOS/tostado queso y tomate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
120	('Antojos','Antojos Almuerzo y Cena','WlNuggets 4 pz','Cuatro piezas de pollo empanizado, doradas y crujientes.',22.00,'/Imágenes/ANTOJOS/wcnuggets 4 pz.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
121	('Antojos','Antojos Almuerzo y Cena','WlPatatas','Papas doradas y crujientes, servidas como acompañamiento.',28.00,'/Imágenes/ANTOJOS/wcpatatas.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
122	('Bebidas','Naturales','Agua','Agua pura refrescante para acompañar cualquier comida.',15.00,'/Imágenes/BEBIDAS/agua.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
123	('Bebidas','Calientes','Café','Café caliente de sabor suave y aromático.',18.00,'/Imágenes/BEBIDAS/café.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
124	('Bebidas','Calientes','Café con leche','Café caliente combinado con leche cremosa.',22.00,'/Imágenes/BEBIDAS/cafe_con_leche.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
125	('Bebidas','Calientes','Chocolate','Bebida caliente de chocolate, cremosa y reconfortante.',22.00,'/Imágenes/BEBIDAS/chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
126	('Bebidas','Sodas','Coca-Cola','Bebida gaseosa Coca-Cola servida fría.',23.00,'/Imágenes/BEBIDAS/coca_col.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
127	('Bebidas','Sodas','Coca-Cola Zero','Bebida gaseosa Coca-Cola sin azúcar, fría y sin azúcar.',23.00,'/Imágenes/BEBIDAS/coca_col.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

128	('Bebidas','Sodas','Fanta','Bebida gaseosa sabor naranja servida fría.',23.00,'/Imágenes/BEBIDAS/fanta.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
129	('Bebidas','Naturales','Jamaica','Bebida refrescante de jamaica servida fría.',20.00,'/Imágenes/BEBIDAS/jamaica.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
130	('Bebidas','Naturales','Jugo manzana','Jugo de manzana frío y refrescante.',18.00,'/Imágenes/BEBIDAS/jugo_manzana.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
131	('Bebidas','Naturales','Jugo naranja','Jugo de naranja frío para acompañar el desayuno o la comida.',18.00,'/Imágenes/BEBIDAS/jugo_naranja.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
132	('Bebidas','Sodas','Sprite','Bebida gaseosa lima-limón servida fría.',23.00,'/Imágenes/BEBIDAS/sprite.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
133	('Bebidas','Calientes','Té caliente','Té caliente de sabor ligero y aromático.',18.00,'/Imágenes/BEBIDAS/té_caliente.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
134	('Bebidas','Naturales','Té Lipton','Té frío Lipton, refrescante y listo para acompañar tu comida.',20.00,'/Imágenes/BEBIDAS/te_lipton.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
135	('Bebidas','Naturales','WlFizz A.M.','Bebida fría y burbujeante con sabor frutal.',26.00,'/Imágenes/wlcafe/wlfizz_A.M..png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
136	('Bebidas','Naturales','WlFrizz Blue','Bebida fría y burbujeante con sabor frutal.',26.00,'/Imágenes/BEBIDAS/wlfrizz_blue.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
137	('Bebidas','Naturales','WlFrizz Manzana Verde','Bebida fría y burbujeante con sabor frutal.',26.00,'/Imágenes/BEBIDAS/wlfrizz_manzanaverde.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
138	('Bebidas','Naturales','WlFrizz Pink','Bebida fría y burbujeante con sabor frutal.',26.00,'/Imágenes/BEBIDAS/wlfrizz_pink.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
139	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de derretido','Menú infantil con derretido, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de derretido.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
140	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de derretido almuerzo-cena','Menú infantil con derretido, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de derretido almuerzo-cena.png','ALMUERZO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
141	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de hamburguesa','Menú infantil con hamburguesa, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de hamburguesa.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
142	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de hamburguesa Jr.','Menú infantil con hamburguesa junior, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de hamburguesa jr.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
143	('Cajita Feliz','Desayuno','Cajita Feliz de Hot Cakes','Menú infantil de desayuno con panqueques y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de hotcakes.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
144	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de Pollo WlCrispy','Menú infantil con pollo crujiente, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de pollo wlcrispy.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
145	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de quesoburguesa','Menú infantil con quesoburguesa, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de quesoburguesa.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
146	('Cajita Feliz','Desayuno','Cajita Feliz de WlMuffin de frijol','Menú infantil de desayuno con panecillo de frijol y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de wluffin de frijol.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
147	('Cajita Feliz','Desayuno','Cajita Feliz de WlMuffin de Huevo y queso','Menú infantil de desayuno con panecillo de huevo y queso.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de wluffin de huevo y queso.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
148	('Cajita Feliz','Desayuno','Cajita Feliz de WlMuffin de salchicha','Menú infantil de desayuno con panecillo de salchicha.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de wluffin de salchicha.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
149	('Cajita Feliz','Almuerzo/Cena','Cajita Feliz de WlNuggets','Menú infantil con bocaditos de pollo, acompañamiento y bebida.',45.00,'/Imágenes/CAJITA FELIZ/cajita feliz de wluffin de pollo.png','TODO_DIA','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
150	('Desayunos','Desayunos','Burrito','Burrito de desayuno con huevo, salchicha, vegetales y queso.',16.00,'/Imágenes/DESAYUNOS/burrito.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
151	('Desayunos','Desayunos','Burritos','Porción de burritos con huevo, salchicha, vegetales y queso.',27.00,'/Imágenes/DESAYUNOS/burritos.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
152	('Desayunos','Caja Grande','Caja Grande Deluxe','Caja para compartir con una selección de desayunos estilo de lujo.',160.00,'/Imágenes/DESAYUNOS/Caja_grande_deluxe.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

153	('Desayunos','Caja Grande','Caja Grande Desayuno','Caja para compartir con una selección de productos de desayuno.',140.00,'/Imagenes/DESAYUNOS/Caja_grande_desayuno.png','DESAYUNO','NINGUNO',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
154	('Desayunos','Desayunos','Desayuno clásico','Huevos revueltos con salchicha y pan panecillo tostado.',30.00,'/Imagenes/DESAYUNOS/desayuno_clásico.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
155	('Desayunos','Desayunos','Desayuno Deluxe','Huevos revueltos, salchicha, panqueques y pan panecillo tostado.',44.00,'/Imagenes/DESAYUNOS/desayuno_deluxe.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
156	('Desayunos','Desayunos','Desayuno tradicional','Desayuno chapin con huevo, frijoles, plátanos, queso, crema y tortillas.',45.00,'/Imagenes/DESAYUNOS/desayuno_tradicional.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
157	('Desayunos','Desayunos','Hot Cakes','Tres panqueques calientes acompañados de mantequilla y miel.',33.00,'/Imagenes/DESAYUNOS/hotcakes.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
158	('Desayunos','WlGriddle','Pan WlGriddle','Pan suave tipo WlGriddle con un toque dulce de arce.',18.00,'/Imagenes/DESAYUNOS/pan_wlgriddle.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
159	('Desayunos','WlGriddle','WlGriddle de Salchicha y Huevo','Pan tipo WlGriddle con salchicha, huevo y queso.',36.00,'/Imagenes/DESAYUNOS/wlgriddle_de_salchicha_y_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
160	('Desayunos','WlGriddle','WlGriddle Salchicha','Pan tipo WlGriddle con salchicha y queso.',31.00,'/Imagenes/DESAYUNOS/wlgriddle_salchicha.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
161	('Desayunos','WlGriddle','WlGriddle Tocino Huevo','Pan tipo WlGriddle con tocino, huevo y queso.',33.00,'/Imagenes/DESAYUNOS/wlgriddle_tocinohuevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
162	('Desayunos','WlMuffin','WlMuffin Chapin Jamón','Pan panecillo tostado con jamón y acompañamientos estilo chapin.',36.00,'/Imagenes/DESAYUNOS/wlmuffin_chapin_jamon.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
163	('Desayunos','WlMuffin','WlMuffin Chapin Salchicha','Pan panecillo tostado con salchicha y acompañamientos estilo chapin.',28.00,'/Imagenes/DESAYUNOS/wlmuffin_chapin_salchicha.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
164	('Desayunos','WlMuffin','WlMuffin Cheddar WlMelt','Pan panecillo tostado con salchicha, huevo y queso cheddar.',42.00,'/Imagenes/DESAYUNOS/wlmuffin_cheddarwlmelt.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
165	('Desayunos','WlMuffin','WlMuffin de Doble Huevo','Pan panecillo tostado con jamón, queso y doble huevo.',41.00,'/Imagenes/DESAYUNOS/eggwlmuffin_doble_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
166	('Desayunos','WlMuffin','WlMuffin de Huevo','Pan panecillo tostado con jamón, huevo y queso.',36.00,'/Imagenes/DESAYUNOS/egg_wlmuffin.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
167	('Desayunos','WlMuffin','WlMuffin Doble de Huevo','Pan panecillo tostado con jamón, huevo y queso.',37.00,'/Imagenes/DESAYUNOS/egg_wlmuffin_doble.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
168	('Desayunos','WlMuffin','WlMuffin Huevo y Frijol','Pan panecillo tostado con huevo y frijoles.',24.00,'/Imagenes/DESAYUNOS/wlmuffin_huevofrijol.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
169	('Desayunos','WlMuffin','WlMuffin Huevo y Queso','Pan panecillo tostado con huevo y queso.',21.00,'/Imagenes/DESAYUNOS/wlmuffin_huevoqueso.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
170	('Desayunos','WlMuffin','WlMuffin Salchicha','Pan panecillo tostado con salchicha.',28.00,'/Imagenes/DESAYUNOS/wlmuffin_salchicha.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
171	('Desayunos','WlMuffin','WlMuffin Salchicha Doble y Huevo','Panecillo de desayuno con doble salchicha, huevo y queso.',42.00,'/Imagenes/DESAYUNOS/wlmuffi_salchich_doble_y_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
172	('Desayunos','WlMuffin','WlMuffin Salchicha y doble huevo','Pan panecillo tostado con salchicha y doble huevo.',42.00,'/Imagenes/DESAYUNOS/wlmuffin_salchicha_y_doble_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
173	('Desayunos','WlMuffin','WlMuffin Salchicha y huevo','Pan panecillo tostado con salchicha, huevo y queso.',37.00,'/Imagenes/DESAYUNOS/wlmuffin_salchichay_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
174	('Desayunos','WlMuffin','WlMuffin Súper Chapin con Jamón','Panecillo estilo chapin con jamón y acompañamientos de desayuno.',42.00,'/Imagenes/DESAYUNOS/wlmuffinsuper_chapin_con_jamon.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
175	('Desayunos','WlMuffin','WlMuffin Súper Chapin Salchicha','Panecillo estilo chapin con salchicha y acompañamientos de desayuno.',35.00,'/Imagenes/DESAYUNOS/wlmuffiinsuper_chapin_salchicha.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
176	('Desayunos','WlMuffin','WlMuffin Tocino Doble Huevo','Pan panecillo tostado con tocino y doble huevo.',42.00,'/Imagenes/DESAYUNOS/wlmuffin_tocino_doble_huevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
177	('Desayunos','WlMuffin','WlMuffin Tocino y Huevo','Pan panecillo tostado con tocino, huevo y queso.',30.00,'/Imagenes/DESAYUNOS/wlmuffin_tocinohuevo.png','DESAYUNO','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
178	('Postres','Pasteles','Pastel de manzana','Pastel caliente con relleno dulce de manzana.',17.00,'/Imagenes/POSTRES/pastel_de_manzana.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
179	('Postres','Pasteles','Pastel de queso','Pastel caliente con relleno cremoso de queso.',17.00,'/Imagenes/POSTRES/pastel_de_queso.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

180	('Postres','Helados','Sundae Caramelo','Helado suave de vainilla cubierto con salsa de caramelo.','15.00','/Imagenes/POSTRES/sundae_caramelo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
181	('Postres','Helados','Sundae Chocolate','Helado suave de vainilla cubierto con salsa de chocolate.','15.00','/Imagenes/POSTRES/sundae_chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
182	('Postres','Helados','Sundae Fresa','Helado suave de vainilla cubierto con salsa de fresa.','15.00','/Imagenes/POSTRES/sundae_fresa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
183	('Postres','Helados','WlCono Vainilla','Cono crujiente con helado suave de vainilla.','10.00','/Imagenes/POSTRES/mccono_vainilla.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
184	('Postres','Helados','WlFlurry M&M'ss','Helado suave de vainilla mezclado con M&M.','24.00','/Imagenes/POSTRES/mcflurry_m&ms.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
185	('Postres','Helados','WlFlurry M&M'ss Chocolate','Helado suave de vainilla mezclado con M&M y chocolate.','25.00','/Imagenes/POSTRES/mcflurry_m&ms_chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
186	('Postres','Helados','WlFlurry M&M'ss Fresa','Helado suave de vainilla mezclado con M&M y fresa.','25.00','/Imagenes/POSTRES/mcflurry_m&ms_fresa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
187	('Postres','Helados','WlFlurry Oreo','Helado suave de vainilla mezclado con Oreo.','24.00','/Imagenes/POSTRES/mcflurry_oreo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
188	('Postres','Helados','WlFlurry Oreo Caramelo','Helado suave de vainilla mezclado con Oreo y caramelo.','25.00','/Imagenes/POSTRES/mcflurry_oreo_caramelo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
189	('Postres','Helados','WlFlurry Oreo Chocolate','Helado suave de vainilla mezclado con Oreo y chocolate.','25.00','/Imagenes/POSTRES/mcflurry_oreo_chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
190	('Postres','Helados','WlFlurry Oreo Fresa','Helado suave de vainilla mezclado con Oreo y fresa.','25.00','/Imagenes/POSTRES/mcflurry_oreo_fresa.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
191	('WlCafé','Postres WlCafé','Brownie Bites','Bizcocho de chocolate suave de chocolate con trozos de chocolate.','16.00','/Imagenes/WLCAFE/brownie_bites.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
192	('WlCafé','Café en bolsa','Café Blend Grano','Café mezcla en grano para preparar en casa.','55.00','/Imagenes/WLCAFE/cafe_blend_grano.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
193	('WlCafé','Café en bolsa','Café Blend Molido','Café mezcla molido para preparar en casa.','55.00','/Imagenes/WLCAFE/cafe_blend_molido.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
194	('WlCafé','Bebidas Calientes','Café guatemalteco','Café guatemalteco de aroma intenso y sabor balanceado.','25.00','/Imagenes/wlcafe/cafe_guatemalteco.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
195	('WlCafé','Bebidas Calientes','Capuccino','Café expreso con leche vaporizada y espuma cremosa.','29.00','/Imagenes/WLCAFE/capuccino.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
196	('WlCafé','Postres WlCafé','Cheesecake Fresa','Porción de pastel de queso con cobertura de fresa.','34.00','/Imagenes/WLCAFE/cheesecake_fresa.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
197	('WlCafé','Postres WlCafé','Cheesecake Mora','Porción de pastel de queso con cobertura de mora.','34.00','/Imagenes/WLCAFE/cheesecake_mora.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
198	('WlCafé','Bebidas Calientes','Chocolate caliente','Chocolate caliente, cremoso y reconfortante.','29.00','/Imagenes/WLCAFE/chocolate_caliente.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
199	('WlCafé','Bebidas Frías','Frappé Caramelo','Bebida frappé fría y cremosa sabor caramelo.','39.00','/Imagenes/wlcafe/frappé_caramelo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
200	('WlCafé','Bebidas Frías','Frappé Chocolate','Bebida frappé fría y cremosa sabor chocolate.','39.00','/Imagenes/WLCAFE/frappé_chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
201	('WlCafé','Bebidas Frías','Frappé Oreo','Bebida frappé fría y cremosa con sabor a Oreo.','39.00','/Imagenes/WLCAFE/frappé_oreo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
202	('WlCafé','Bebidas Frías','Frappé Original','Bebida frappé fría y cremosa sabor original.','39.00','/Imagenes/WLCAFE/frappé_original.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
203	('WlCafé','Bebidas Frías','Frappé Vainilla','Bebida frappé fría y cremosa sabor vainilla.','39.00','/Imagenes/wlcafe/frappé_vainilla.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
204	('WlCafé','Bebidas Frías','Frappé Vainilla Light','Bebida frappé fría y cremosa sabor vainilla.','39.00','/Imagenes/WLCAFE/frappé_vainilla_light.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
205	('WlCafé','Bebidas Frías','Horchata','Bebida fría de horchata con sabor dulce y especiado.','30.00','/Imagenes/WLCAFE/horchata.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
206	('WlCafé','Bebidas Frías','Iced Coffee Caramelo','Café frío con hielo y sabor a caramelo.','35.00','/Imagenes/WLCAFE/iced_coffee_caramelo.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

207	('WlCafé','Bebidas Frías','Iced Coffee Chocolate','Café frío con hielo y sabor a chocolate.','35.00','/Imágenes/WLCAFE/iced coffe chocolate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
208	('WlCafé','Bebidas Frías','Iced Coffee Horchata','Café frío con hielo y sabor a horchata.','35.00','/Imágenes/WLCAFE/iced coffe horchata.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
209	('WlCafé','Bebidas Frías','Iced Coffee Original','Café frío con hielo de sabor original.','35.00','/Imágenes/WLCAFE/iced coffe original.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
210	('WlCafé','Bebidas Frías','Iced Coffee Vainilla','Café frío con hielo y sabor a vainilla.','35.00','/Imágenes/WLCAFE/iced coffe vainilla.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
211	('WlCafé','Bebidas Frías','Iced Coffee Vainilla Light','Café frío con hielo y sabor a vainilla.','35.00','/Imágenes/WLCAFE/iced coffe vainilla light.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
212	('WlCafé','Bebidas Calientes','Latte','Café expreso combinado con leche caliente y textura cremosa.','30.00','/Imágenes/WLCAFE/latte.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
213	('WlCafé','Postres WlCafé','Pastel de elote','Porción de pastel de elote suave y dulce.','32.00','/Imágenes/WLCAFE/paste_l_elote.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
214	('WlCafé','Bebidas Frías','Smoothie Berries','Batido frío de frutos rojos con hielo triturado.','39.00','/Imágenes/WLCAFE/smothie_berries.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
215	('WlCafé','Bebidas Frías','Smoothie Mango','Batido frío de mango con hielo triturado.','39.00','/Imágenes/WLCAFE/smothie_mango.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
216	('WlCafé','Bebidas Calientes','Té Chai Original','Bebida de té chai con especias y textura cremosa.','41.00','/Imágenes/WLCAFE/techai_original.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
217	('WlCafé','Bebidas Calientes','Té Chai Té Verde','Bebida de té chai combinada con té verde.','41.00','/Imágenes/WLCAFE/techai_te_verde.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
218	('WlCafé','Bebidas Calientes','Té Chai Vainilla','Bebida de té chai con especias y un toque de vainilla.','41.00','/Imágenes/WLCAFE/te_chai_vainilla.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
219	('WlCafé','Bebidas Calientes','Té Chai Vainilla Light','Té chai ligero con sabor a vainilla.','41.00','/Imágenes/WLCAFE/techai_vainilla_light.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
220	('WlCafé','Bebidas Calientes','Té Guatemalteco Bora Bora','Infusión aromática con mezcla de sabores frutales.','29.00','/Imágenes/wlcafe/te_guatemalteco_borabora.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
221	('WlCafé','Bebidas Calientes','Té Guatemalteco Melocotón Mix','Infusión de té guatemalteco con notas de melocotón.','29.00','/Imágenes/wlcafe/te_guatealteco_melocoton_mix.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
222	('WlCafé','Bebidas Calientes','Té Guatemalteco Menta Fusión','Infusión aromática con sabor refrescante a menta.','29.00','/Imágenes/wlcafe/te_guatemalteco_menta_fusion.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
223	('WlCafé','Bebidas Calientes','Té Guatemalteco Vainilla Relax','Infusión de té con notas suaves de vainilla.','29.00','/Imágenes/wlcafe/teguatemalteco_vainilla_relax.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
224	('WlCafé','Tostados','Tostado Queso Frijol','Tostado caliente con queso y frijoles.','25.00','/Imágenes/WLCAFE/tostado_queso_frijol.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
225	('WlCafé','Tostados','Tostado Queso Tomate','Tostado caliente con queso y tomate.','25.00','/Imágenes/WLCAFE/tostado_queso_tomate.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
226	('WlCafé','Bebidas Frías','WlFizz A.M.','Bebida fría y burbujeante con sabor frutal.','26.00','/Imágenes/wlcafe/wl_fizz_A.M..png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
227	('WlCafé','Bebidas Frías','WlFrizz Blue','Bebida fría y burbujeante con sabor frutal.','26.00','/Imágenes/WLCAFE/wlfrizz_blue.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
228	('WlCafé','Bebidas Frías','WlFrizz Manzana Verde','Bebida fría y burbujeante con sabor a manzana verde.','26.00','/Imágenes/WLCAFE/wlfrizz_manzanaverde.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
229	('WlCafé','Bebidas Frías','WlFrizz Pink','Bebida fría y burbujeante con sabor frutal.','26.00','/Imágenes/WLCAFE/wlfrizz_pink.png','TODO_DIA','RECETA',TRUE,0,0),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
230	('WlCafé','Postres WlCafé','WlPop Chocolate','Bizcocho individual con relleno de chocolate.','16.00','/Imágenes/WLCAFE/wlpop_chocolate.png','TODO_DIA','DIRECTO',FALSE,100,10),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
231	('Cajita Feliz','Complementos','Papas Kids','Porción infantil de papas.','10.00','/Imágenes/ANTOJOS/papas.png','TODO_DIA','DIRECTO',FALSE,200,20),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
232	('Cajita Feliz','Complementos','Puré de manzana','Puré de manzana para menú infantil.','9.00','/Imágenes/BEBIDAS/jugo_manzana.png','TODO_DIA','DIRECTO',FALSE,150,15),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
233	('Cajita Feliz','Complementos','Yogur de fresa','Yogur de fresa para menú infantil.','9.00',NULL,'TODO_DIA','DIRECTO',FALSE,150,15),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

234	('Cajita Feliz','Bebidas','Jugo de manzana Kids','Jugo de manzana en tamaño infantil.',8.00,'/Imagenes/BEBIDAS/jugo_manzana.png','TODO_DIA','DIRECTO',FALSE,150,15),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
235	('Cajita Feliz','Juguetes','Juguete sorpresa','Juguete disponible para la promoción infantil.',12.00,NULL,'TODO_DIA','DIRECTO',FALSE,200,20),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
236	('Bebidas','Sodas','Coca-Cola 1.5 L','Bebida familiar para cajas y combos.',25.00,'/Imagenes/BEBIDAS/coca_cola.png','TODO_DIA','DIRECTO',FALSE,100,10);	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
237	INSERT INTO producto (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
238	id_categoria, nombre, descripcion, precio_base, imagen, subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
239	disponibilidad_menu, tipo_stock, personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
240	stock_actual, stock_minimo, estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
241	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
242	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
243	c.id_categoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
244	cp.nombre,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
245	cp.descripcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
246	cp.precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
247	cp.imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
248	cp.subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
249	cp.disponibilidad,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
250	cp.tipo_stock,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
251	cp.personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
252	cp.stock_actual,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
253	cp.stock_minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
254	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
255	FROM carga_producto cp	Indica la tabla principal desde la que se leen los datos.
256	INNER JOIN categoria c ON c.nombre = cp.categoria	Relaciona filas de dos tablas mediante sus identificadores vinculados.
257	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
258	SELECT 1	Consulta columnas o valores calculados sin modificar los datos.
259	FROM producto p	Indica la tabla principal desde la que se leen los datos.
260	WHERE p.id_categoria = c.id_categoria	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
261	AND p.nombre = cp.nombre	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
262	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
263		Línea en blanco usada para separar secciones del script SQL.
264	-- Si el producto ya existía, se actualizan sus campos comerciales.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
265	UPDATE producto p	Actualiza filas existentes de la tabla indicada.
266	INNER JOIN categoria c ON c.id_categoria = p.id_categoria	Relaciona filas de dos tablas mediante sus identificadores vinculados.
267	INNER JOIN carga_producto cp	Relaciona filas de dos tablas mediante sus identificadores vinculados.
268	ON cp.categoria = c.nombre AND cp.nombre = p.nombre	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
269	SET p.descripcion = cp.descripcion,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
270	p.precio_base = cp.precio,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
271	p.imagen = cp.imagen,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
272	p.subcategoria = cp.subcategoria,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
273	p.disponibilidad_menu = cp.disponibilidad,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
274	p.tipo_stock = cp.tipo_stock,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
275	p.personalizable = cp.personalizable,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

276	p.stock_actual = CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
277	WHEN cp.tipo_stock = 'DIRECTO' AND p.stock_actual = 0	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
278	THEN cp.stock_actual	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
279	ELSE p.stock_actual	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
280	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
281	p.stock_minimo = cp.stock_minimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
282	p.estado = TRUE;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
283		Línea en blanco usada para separar secciones del script SQL.
284	DROP TEMPORARY TABLE carga_producto;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
285		Línea en blanco usada para separar secciones del script SQL.
286	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
287	-- 3. INGREDIENTES	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
288	-- Las cantidades están expresadas según unidad_medida.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
289	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
290		Línea en blanco usada para separar secciones del script SQL.
291	DROP TEMPORARY TABLE IF EXISTS carga_ingrediente;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
292		Línea en blanco usada para separar secciones del script SQL.
293	CREATE TEMPORARY TABLE carga_ingrediente (	Crea una tabla temporal que existe solo durante la sesión de carga y ayuda a fusionar datos sin duplicarlos.
294	nombre VARCHAR(100),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
295	unidad VARCHAR(30),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
296	stock DECIMAL(12,2),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
297	minimo DECIMAL(12,2)	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
298	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
299		Línea en blanco usada para separar secciones del script SQL.
300	INSERT INTO carga_ingrediente (nombre, unidad, stock, minimo) VALUES	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
301	('Pan hamburguesa', 'unidad', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
302	('Pan muffin', 'unidad', 800, 80),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
303	('Pan WlGriddle', 'unidad', 800, 80),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
304	('Pan tostado', 'rebanada', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
305	('Tortilla de harina', 'unidad', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
306	('Tortilla de maíz', 'unidad', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
307	('Carne de res', 'unidad', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
308	('Filete de pollo', 'unidad', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
309	('Huevo', 'unidad', 2000, 200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
310	('Salchicha desayuno', 'unidad', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
311	('Jamón', 'rebanada', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
312	('Tocino', 'tira', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
313	('Queso americano', 'rebanada', 2500, 250),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
314	('Queso cheddar', 'rebanada', 1000, 100),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
315	('Lechuga', 'gramo', 20000, 2000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
316	('Tomate', 'rodaja', 3000, 300),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
317	('Cebolla', 'gramo', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
318	('Pepinillo', 'rodaja', 5000, 500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
319	('Ketchup', 'mililitro', 20000, 2000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

320	('Mostaza', 'mililitro', 15000, 1500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
321	('Mayonesa', 'mililitro', 15000, 1500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
322	('Salsa Big Mac', 'mililitro', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
323	('Salsa Big Tasty', 'mililitro', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
324	('Salsa ranch', 'mililitro', 8000, 800),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
325	('Salsa ahumada', 'mililitro', 8000, 800),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
326	('Guacamole', 'gramo', 8000, 800),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
327	('Pico de gallo', 'gramo', 8000, 800),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
328	('Frijol', 'gramo', 15000, 1500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
329	('Loroco', 'gramo', 5000, 500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
330	('Vegetales mixtos', 'gramo', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
331	('Plátano', 'gramo', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
332	('Crema', 'mililitro', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
333	('Masa de hot cake', 'porcion', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
334	('Mantequilla', 'gramo', 8000, 800),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
335	('Miel', 'mililitro', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
336	('Café molido', 'gramo', 20000, 2000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
337	('Agua preparada', 'mililitro', 100000, 10000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
338	('Leche', 'mililitro', 80000, 8000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
339	('Chocolate en polvo', 'gramo', 15000, 1500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
340	('Bolsa de té', 'unidad', 2000, 200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
341	('Mezcla chai', 'gramo', 10000, 1000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
342	('Hielo', 'gramo', 100000, 10000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
343	('Jarabe caramelo', 'mililitro', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
344	('Jarabe chocolate', 'mililitro', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
345	('Jarabe fresa', 'mililitro', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
346	('Jarabe vainilla', 'mililitro', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
347	('Jarabe frutal', 'mililitro', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
348	('Base horchata', 'mililitro', 15000, 1500),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
349	('Pulpa de mango', 'gramo', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
350	('Pulpa de berries', 'gramo', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
351	('Helado de vainilla', 'gramo', 50000, 5000),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
352	('Cono', 'unidad', 1500, 150),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
353	('Galleta Oreo', 'gramo', 12000, 1200),	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
354	('Chocolate M&M', 'gramo', 12000, 1200);	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
355		Línea en blanco usada para separar secciones del script SQL.
356	INSERT INTO ingrediente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
357	nombre, unidad_medida, stock_actual, stock_minimo, estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
358	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
359	SELECT ci.nombre, ci.unidad, ci.stock, ci.minimo, TRUE	Consulta columnas o valores calculados sin modificar los datos.
360	FROM carga_ingrediente ci	Indica la tabla principal desde la que se leen los datos.
361	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
362	SELECT 1 FROM ingrediente i WHERE i.nombre = ci.nombre	Consulta columnas o valores calculados sin modificar los datos.
363	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.

364		Línea en blanco usada para separar secciones del script SQL.
365	UPDATE ingrediente i	Actualiza filas existentes de la tabla indicada.
366	INNER JOIN carga_ingrediente ci ON ci.nombre = i.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
367	SET i.unidad_medida = ci.unidad,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
368	i.stock_actual = CASE WHEN i.stock_actual = 0 THEN ci.stock ELSE i.stock_actual END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
369	i.stock_mínimo = ci.mínimo,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
370	i.estado = TRUE;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
371		Línea en blanco usada para separar secciones del script SQL.
372	DROP TEMPORARY TABLE carga_ingrediente;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
373		Línea en blanco usada para separar secciones del script SQL.
374	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
375	-- 4. RECETAS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
376	-- Se cargan primero en una tabla temporal para fusionar reglas	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
377	-- repetidas antes de insertarlas en producto_ingrediente.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
378	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
379		Línea en blanco usada para separar secciones del script SQL.
380	DROP TEMPORARY TABLE IF EXISTS carga_receta;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
381		Línea en blanco usada para separar secciones del script SQL.
382	CREATE TEMPORARY TABLE carga_receta (	Crea una tabla temporal que existe solo durante la sesión de carga y ayuda a fusionar datos sin duplicarlos.
383	id_producto INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
384	id_ingrediente INT,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
385	cantidad DECIMAL(12,2),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
386	permite_quitar BOOLEAN,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
387	permite_extra BOOLEAN,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
388	cantidad_extra DECIMAL(12,2),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
389	precio_extra DECIMAL(12,2),	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
390	max_extras INT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
391	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
392		Línea en blanco usada para separar secciones del script SQL.
393	-- Tostados y derretidos.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
394	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
395	SELECT p.id_producto,i.id_ingrediente,2,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
396	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
397	JOIN ingrediente i ON i.nombre='Pan tostado'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
398	WHERE p.tipo_stock='RECETA' AND c.nombre IN ('Antojos','WlCafé')	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
399	AND (p.nombre LIKE '%Tostado%' OR p.nombre LIKE '%Derretido%');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
400		Línea en blanco usada para separar secciones del script SQL.
401	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
402	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,3.00,3	Consulta columnas o valores calculados sin modificar los datos.
403	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
404	JOIN ingrediente i ON i.nombre='Queso americano'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
405	WHERE p.tipo_stock='RECETA' AND c.nombre IN ('Antojos','WlCafé')	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
406	AND (p.nombre LIKE '%Tostado%' OR p.nombre LIKE '%Derretido%');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
407		Línea en blanco usada para separar secciones del script SQL.

408	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
409	SELECT p.id_producto,i.id_ingredient,40,TRUE,TRUE,20,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
410	FROM producto p JOIN ingrediente i ON i.nombre='Frijol'	Indica la tabla principal desde la que se leen los datos.
411	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Frijol%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
412		Línea en blanco usada para separar secciones del script SQL.
413	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
414	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
415	FROM producto p JOIN ingrediente i ON i.nombre='Loroco'	Indica la tabla principal desde la que se leen los datos.
416	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%loroco%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
417		Línea en blanco usada para separar secciones del script SQL.
418	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
419	SELECT p.id_producto,i.id_ingredient,2,TRUE,TRUE,1,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
420	FROM producto p JOIN ingrediente i ON i.nombre='Jamón'	Indica la tabla principal desde la que se leen los datos.
421	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%jamón%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
422		Línea en blanco usada para separar secciones del script SQL.
423	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
424	SELECT p.id_producto,i.id_ingredient,2,TRUE,TRUE,1,1.50,3	Consulta columnas o valores calculados sin modificar los datos.
425	FROM producto p JOIN ingrediente i ON i.nombre='Tomate'	Indica la tabla principal desde la que se leen los datos.
426	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tomate%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
427		Línea en blanco usada para separar secciones del script SQL.
428	-- Hamburguesas y sándwiches de almuerzo.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
429	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
430	SELECT p.id_producto,i.id_ingredient,1,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
431	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
432	JOIN ingrediente i ON i.nombre='Pan hamburguesa'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
433	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
434		Línea en blanco usada para separar secciones del script SQL.
435	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
436	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
437	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
438	WHEN p.nombre='Doble Big Mac' THEN 4	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
439	WHEN p.nombre='Big Mac' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
440	WHEN p.nombre LIKE '%Triple%' THEN 3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
441	WHEN p.nombre LIKE '%Doble%' OR p.nombre='WlDouble' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
442	ELSE 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
443	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
444	FALSE,TRUE,1,12.00,3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
445	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
446	JOIN ingrediente i ON i.nombre='Carne de res'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
447	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
448	AND p.nombre NOT LIKE '%Pollo%' AND p.nombre NOT LIKE '%Crispy%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
449	AND p.nombre NOT LIKE '%Sándwich%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
450		Línea en blanco usada para separar secciones del script SQL.
451	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.

452	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
453	CASE WHEN p.nombre LIKE '%Doble%' THEN 2 ELSE 1 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
454	FALSE,TRUE,1,10.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
455	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
456	JOIN ingrediente i ON i.nombre='Filete de pollo'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
457	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
458	AND (p.nombre LIKE '%Pollo%' OR p.nombre LIKE '%Crispy%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
459	OR p.nombre LIKE '%Sándwich%');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
460		Línea en blanco usada para separar secciones del script SQL.
461	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
462	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
463	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre LIKE '%Triple%' THEN 2 ELSE 1 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
464	TRUE,TRUE,1,3.00,3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
465	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
466	JOIN ingrediente i ON i.nombre='Queso americano'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
467	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
468	AND p.nombre NOT LIKE '%Cheddar%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
469	AND (p.descripcion LIKE 'queso%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
470	OR p.nombre REGEXP 'Queso WlMelt Big Mac Big Tasty WlNífica WlDouble Triple Bacon Pico Guacamol Smoke Tocino Gourmet');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
471		Línea en blanco usada para separar secciones del script SQL.
472	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
473	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
474	CASE WHEN p.nombre LIKE '%Doble%' THEN 2 ELSE 1 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
475	TRUE,TRUE,1,3.50,3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
476	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
477	JOIN ingrediente i ON i.nombre='Queso cheddar'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
478	WHERE c.nombre IN ('Almuerzos','Desayunos') AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
479	AND p.nombre LIKE '%Cheddar%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
480		Línea en blanco usada para separar secciones del script SQL.
481	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
482	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
483	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre LIKE '%Triple%' THEN 3 ELSE 2 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
484	TRUE,TRUE,1,4.00,3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
485	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
486	JOIN ingrediente i ON i.nombre='Tocino'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
487	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
488	AND (p.nombre LIKE '%Bacon%' OR p.nombre LIKE '%Tocino%');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
489		Línea en blanco usada para separar secciones del script SQL.
490	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
491	SELECT p.id_producto,i.id_ingredient,20,TRUE,TRUE,10,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
492	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
493	JOIN ingrediente i ON i.nombre='Lechuga'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
494	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
495	AND p.nombre REGEXP 'Big Tasty Deluxe Gourmet WlNífica Pollo Crispy Res Sándwich';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

496		Línea en blanco usada para separar secciones del script SQL.
497	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
498	SELECT p.id_producto,i.id_ingredient,2,TRUE,TRUE,1,1.50,2	Consulta columnas o valores calculados sin modificar los datos.
499	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
500	JOIN ingrediente i ON i.nombre='Tomate'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
501	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
502	AND p.nombre REGEXP 'Tasty Deluxe Gourmet WLnifica Pollo Crispy Res Sándwich'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
503	AND p.nombre NOT LIKE '%Big Mac%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
504		Línea en blanco usada para separar secciones del script SQL.
505	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
506	SELECT p.id_producto,i.id_ingredient,10,TRUE,TRUE,5,0.75,2	Consulta columnas o valores calculados sin modificar los datos.
507	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
508	JOIN ingrediente i ON i.nombre='Cebolla'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
509	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
510	AND p.nombre NOT REGEXP 'Pollo Crispy Sándwich';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
511		Línea en blanco usada para separar secciones del script SQL.
512	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
513	SELECT p.id_producto,i.id_ingredient,2,TRUE,TRUE,1,1.00,3	Consulta columnas o valores calculados sin modificar los datos.
514	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
515	JOIN ingrediente i ON i.nombre='Pepinillo'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
516	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
517	AND p.nombre NOT REGEXP 'Pollo Crispy Sándwich Gourmet Tasty';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
518		Línea en blanco usada para separar secciones del script SQL.
519	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
520	SELECT p.id_producto,i.id_ingredient,12,TRUE,TRUE,6,0.75,2	Consulta columnas o valores calculados sin modificar los datos.
521	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
522	JOIN ingrediente i ON i.nombre='Ketchup'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
523	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
524	AND p.nombre REGEXP 'Hamburguesa Cuarto Triple WDouble Quesoburguesa';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
525		Línea en blanco usada para separar secciones del script SQL.
526	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
527	SELECT p.id_producto,i.id_ingredient,8,TRUE,TRUE,4,0.75,2	Consulta columnas o valores calculados sin modificar los datos.
528	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
529	JOIN ingrediente i ON i.nombre='Mostaza'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
530	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
531	AND p.nombre REGEXP 'Hamburguesa Cuarto Triple WDouble Quesoburguesa';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
532		Línea en blanco usada para separar secciones del script SQL.
533	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
534	SELECT p.id_producto,i.id_ingredient,15,TRUE,TRUE,8,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
535	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
536	JOIN ingrediente i ON i.nombre='Mayonesa'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
537	WHERE c.nombre='Almuerzos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
538	AND p.nombre REGEXP 'Pollo Crispy Sándwich Deluxe';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
539		Línea en blanco usada para separar secciones del script SQL.

540	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
541	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
542	FROM producto p JOIN ingrediente i ON i.nombre='Salsa Big Mac'	Indica la tabla principal desde la que se leen los datos.
543	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Big Mac%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
544		Línea en blanco usada para separar secciones del script SQL.
545	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
546	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
547	FROM producto p JOIN ingrediente i ON i.nombre='Salsa Big Tasty'	Indica la tabla principal desde la que se leen los datos.
548	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Big Tasty%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
549		Línea en blanco usada para separar secciones del script SQL.
550	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
551	SELECT p.id_producto,i.id_ingrediente,35,TRUE,TRUE,20,4.00,2	Consulta columnas o valores calculados sin modificar los datos.
552	FROM producto p JOIN ingrediente i ON i.nombre='Guacamole'	Indica la tabla principal desde la que se leen los datos.
553	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Guacamol%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
554		Línea en blanco usada para separar secciones del script SQL.
555	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
556	SELECT p.id_producto,i.id_ingrediente,30,TRUE,TRUE,15,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
557	FROM producto p JOIN ingrediente i ON i.nombre='Pico de gallo'	Indica la tabla principal desde la que se leen los datos.
558	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Pico%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
559		Línea en blanco usada para separar secciones del script SQL.
560	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
561	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
562	FROM producto p JOIN ingrediente i ON i.nombre='Salsa ahumada'	Indica la tabla principal desde la que se leen los datos.
563	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Smoke%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
564		Línea en blanco usada para separar secciones del script SQL.
565	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
566	SELECT p.id_producto,i.id_ingrediente,20,TRUE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
567	FROM producto p JOIN ingrediente i ON i.nombre='Salsa ranch'	Indica la tabla principal desde la que se leen los datos.
568	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Ranch%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
569		Línea en blanco usada para separar secciones del script SQL.
570	-- Desayunos: WlMuffin y WlGriddle.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
571	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
572	SELECT p.id_producto,i.id_ingrediente,1,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
573	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
574	JOIN ingrediente i ON i.nombre='Pan muffin'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
575	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
576	AND p.nombre LIKE '%WlMuffin%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
577		Línea en blanco usada para separar secciones del script SQL.
578	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
579	SELECT p.id_producto,i.id_ingrediente,1,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
580	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
581	JOIN ingrediente i ON i.nombre='Pan WlGriddle'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
582	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
583	AND p.nombre LIKE '%WlGriddle%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

584		Línea en blanco usada para separar secciones del script SQL.
585	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
586	SELECT p.id_producto,i.id_ingrediente,	Consulta columnas o valores calculados sin modificar los datos.
587	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
588	WHEN p.nombre='Burritos' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
589	WHEN p.nombre LIKE '%Doble Huevo%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
590	OR p.nombre LIKE '%Doble de Huevo%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
591	OR p.nombre LIKE '%doble huevo%' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
592	ELSE 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
593	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
594	TRUE,TRUE,1,4.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
595	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
596	JOIN ingrediente i ON i.nombre='Huevo'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
597	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
598	AND (p.nombre LIKE '%Huevo%' OR p.descripcion LIKE '%huevo%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
599	OR p.nombre LIKE '%Chapín%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
600	OR p.nombre IN ('Burrito','Burritos','Desayuno clásico',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
601	'Desayuno Deluxe','Desayuno tradicional'));	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
602		Línea en blanco usada para separar secciones del script SQL.
603	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
604	SELECT p.id_producto,i.id_ingrediente,	Consulta columnas o valores calculados sin modificar los datos.
605	CASE WHEN p.nombre LIKE '%Doble%' OR p.nombre='Burritos' THEN 2 ELSE 1 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
606	TRUE,TRUE,1,5.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
607	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
608	JOIN ingrediente i ON i.nombre='Salchicha desayuno'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
609	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
610	AND (p.nombre LIKE '%Salchicha%' OR p.descripcion LIKE '%salchicha%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
611	OR p.nombre LIKE '%Burrito%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
612	OR p.nombre IN ('Desayuno clásico','Desayuno Deluxe'));	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
613		Línea en blanco usada para separar secciones del script SQL.
614	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
615	SELECT p.id_producto,i.id_ingrediente,1,TRUE,TRUE,1,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
616	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
617	JOIN ingrediente i ON i.nombre='Jamón'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
618	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
619	AND (p.nombre LIKE '%Jamón%' OR p.descripcion LIKE '%jamón%'));	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
620		Línea en blanco usada para separar secciones del script SQL.
621	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
622	SELECT p.id_producto,i.id_ingrediente,2,TRUE,TRUE,1,4.00,3	Consulta columnas o valores calculados sin modificar los datos.
623	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
624	JOIN ingrediente i ON i.nombre='Tocino'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
625	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
626	AND p.nombre LIKE '%Tocino%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
627		Línea en blanco usada para separar secciones del script SQL.

628	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
629	SELECT p.id_producto,i.id_ingredient,1,TRUE,TRUE,1,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
630	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
631	JOIN ingrediente i ON i.nombre='Queso americano'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
632	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
633	AND (p.descripcion LIKE '%queso%' OR p.nombre LIKE '%Queso%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
634	OR p.nombre LIKE '%Cheddar%')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
635	AND p.nombre NOT LIKE '%Cheddar%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
636		Línea en blanco usada para separar secciones del script SQL.
637	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
638	SELECT p.id_producto,i.id_ingredient,40,TRUE,TRUE,20,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
639	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
640	JOIN ingrediente i ON i.nombre='Frijol'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
641	WHERE c.nombre='Desayunos' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
642	AND p.nombre LIKE '%Chapín%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
643		Línea en blanco usada para separar secciones del script SQL.
644	-- Burritos.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
645	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
646	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
647	CASE WHEN p.nombre='Burritos' THEN 2 ELSE 1 END,FALSE,FALSE,1,0,1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
648	FROM producto p JOIN ingrediente i ON i.nombre='Tortilla de harina'	Indica la tabla principal desde la que se leen los datos.
649	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
650		Línea en blanco usada para separar secciones del script SQL.
651	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
652	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
653	CASE WHEN p.nombre='Burritos' THEN 60 ELSE 30 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
654	TRUE,TRUE,20,2.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
655	FROM producto p JOIN ingrediente i ON i.nombre='Vegetales mixtos'	Indica la tabla principal desde la que se leen los datos.
656	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
657		Línea en blanco usada para separar secciones del script SQL.
658	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
659	SELECT p.id_producto,i.id_ingredient,	Consulta columnas o valores calculados sin modificar los datos.
660	CASE WHEN p.nombre='Burritos' THEN 2 ELSE 1 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
661	TRUE,TRUE,1,3.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
662	FROM producto p JOIN ingrediente i ON i.nombre='Queso americano'	Indica la tabla principal desde la que se leen los datos.
663	WHERE p.tipo_stock='RECETA' AND p.nombre IN ('Burrito','Burritos');	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
664		Línea en blanco usada para separar secciones del script SQL.
665	-- Hot Cakes y platos de desayuno.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
666	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
667	SELECT p.id_producto,i.id_ingredient,3,FALSE,TRUE,1,5.00,2	Consulta columnas o valores calculados sin modificar los datos.
668	FROM producto p JOIN ingrediente i ON i.nombre='Masa de hot cake'	Indica la tabla principal desde la que se leen los datos.
669	WHERE p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
670	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
671		Línea en blanco usada para separar secciones del script SQL.

672	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
673	SELECT p.id_producto,i.id_ingredient,10,TRUE,TRUE,5,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
674	FROM producto p JOIN ingrediente i ON i.nombre='Mantequilla'	Indica la tabla principal desde la que se leen los datos.
675	WHERE p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
676	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
677		Línea en blanco usada para separar secciones del script SQL.
678	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
679	SELECT p.id_producto,i.id_ingredient,30,TRUE,TRUE,15,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
680	FROM producto p JOIN ingrediente i ON i.nombre='Miel'	Indica la tabla principal desde la que se leen los datos.
681	WHERE p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
682	AND p.nombre IN ('Hot Cakes','Desayuno Deluxe');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
683		Línea en blanco usada para separar secciones del script SQL.
684	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
685	SELECT p.id_producto,i.id_ingredient,2,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
686	FROM producto p JOIN ingrediente i ON i.nombre='Pan tostado'	Indica la tabla principal desde la que se leen los datos.
687	WHERE p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
688	AND p.nombre IN ('Desayuno clásico','Desayuno Deluxe');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
689		Línea en blanco usada para separar secciones del script SQL.
690	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
691	SELECT p.id_producto,i.id_ingredient,80,TRUE,TRUE,40,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
692	FROM producto p JOIN ingrediente i ON i.nombre='Frijol'	Indica la tabla principal desde la que se leen los datos.
693	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
694		Línea en blanco usada para separar secciones del script SQL.
695	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
696	SELECT p.id_producto,i.id_ingredient,80,TRUE,TRUE,40,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
697	FROM producto p JOIN ingrediente i ON i.nombre='Plátano'	Indica la tabla principal desde la que se leen los datos.
698	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
699		Línea en blanco usada para separar secciones del script SQL.
700	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
701	SELECT p.id_producto,i.id_ingredient,30,TRUE,TRUE,15,1.50,2	Consulta columnas o valores calculados sin modificar los datos.
702	FROM producto p JOIN ingrediente i ON i.nombre='Crema'	Indica la tabla principal desde la que se leen los datos.
703	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
704		Línea en blanco usada para separar secciones del script SQL.
705	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
706	SELECT p.id_producto,i.id_ingredient,3,FALSE,TRUE,1,1.00,3	Consulta columnas o valores calculados sin modificar los datos.
707	FROM producto p JOIN ingrediente i ON i.nombre='Tortilla de maíz'	Indica la tabla principal desde la que se leen los datos.
708	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
709		Línea en blanco usada para separar secciones del script SQL.
710	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
711	SELECT p.id_producto,i.id_ingredient,1,TRUE,TRUE,1,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
712	FROM producto p JOIN ingrediente i ON i.nombre='Queso americano'	Indica la tabla principal desde la que se leen los datos.
713	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%tradicional%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
714		Línea en blanco usada para separar secciones del script SQL.
715	-- Bebidas calientes del catálogo general.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.

716	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
717	SELECT p.id_producto,i.id_ingredient,18,FALSE,TRUE,9,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
718	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
719	JOIN ingrediente i ON i.nombre='Café molido'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
720	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
721	AND p.nombre LIKE '%Café%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
722		Línea en blanco usada para separar secciones del script SQL.
723	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
724	SELECT p.id_producto,i.id_ingredient,250,FALSE,FALSE,50,0,1	Consulta columnas o valores calculados sin modificar los datos.
725	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
726	JOIN ingrediente i ON i.nombre='Agua preparada'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
727	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
728	AND p.nombre REGEXP 'Café Chocolate Té caliente';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
729		Línea en blanco usada para separar secciones del script SQL.
730	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
731	SELECT p.id_producto,i.id_ingredient,180,FALSE,FALSE,50,0,1	Consulta columnas o valores calculados sin modificar los datos.
732	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
733	JOIN ingrediente i ON i.nombre='Agua preparada'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
734	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
735	AND p.nombre REGEXP 'WlFizz WlFrizz';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
736		Línea en blanco usada para separar secciones del script SQL.
737	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
738	SELECT p.id_producto,i.id_ingredient,160,FALSE,TRUE,60,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
739	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
740	JOIN ingrediente i ON i.nombre='Hielo'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
741	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
742	AND p.nombre REGEXP 'WlFizz WlFrizz';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
743		Línea en blanco usada para separar secciones del script SQL.
744	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
745	SELECT p.id_producto,i.id_ingredient,150,FALSE,TRUE,50,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
746	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
747	JOIN ingrediente i ON i.nombre='Leche'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
748	WHERE c.nombre='Bebidas' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
749	AND p.nombre REGEXP 'con leche Chocolate';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
750		Línea en blanco usada para separar secciones del script SQL.
751	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
752	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
753	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate en polvo'	Indica la tabla principal desde la que se leen los datos.
754	WHERE p.tipo_stock='RECETA' AND p.nombre='Chocolate';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
755		Línea en blanco usada para separar secciones del script SQL.
756	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
757	SELECT p.id_producto,i.id_ingredient,1,FALSE,TRUE,1,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
758	FROM producto p JOIN ingrediente i ON i.nombre='Bolsa de té'	Indica la tabla principal desde la que se leen los datos.
759	WHERE p.tipo_stock='RECETA' AND p.nombre='Té caliente';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

760		Línea en blanco usada para separar secciones del script SQL.
761	-- WlCafé: café, frappés, bebidas frías, té y smoothies.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
762	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
763	SELECT p.id_producto,i.id_ingredient,18,FALSE,TRUE,9,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
764	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
765	JOIN ingrediente i ON i.nombre='Café molido'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
766	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
767	AND p.nombre REGEXP 'Café Coffee Capuccino Latte Frappé';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
768		Línea en blanco usada para separar secciones del script SQL.
769	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
770	SELECT p.id_producto,i.id_ingredient,180,FALSE,FALSE,50,0,1	Consulta columnas o valores calculados sin modificar los datos.
771	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
772	JOIN ingrediente i ON i.nombre='Agua preparada'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
773	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
774	AND p.subcategoria IN ('Bebidas Calientes','Bebidas Frías');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
775		Línea en blanco usada para separar secciones del script SQL.
776	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
777	SELECT p.id_producto,i.id_ingredient,180,FALSE,TRUE,50,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
778	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
779	JOIN ingrediente i ON i.nombre='Leche'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
780	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
781	AND p.nombre REGEXP 'Capuccino Chocolate Frappé Coffee Latte Chai';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
782		Línea en blanco usada para separar secciones del script SQL.
783	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
784	SELECT p.id_producto,i.id_ingredient,160,FALSE,TRUE,60,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
785	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
786	JOIN ingrediente i ON i.nombre='Hielo'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
787	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
788	AND p.subcategoria='Bebidas Frías';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
789		Línea en blanco usada para separar secciones del script SQL.
790	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
791	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
792	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate en polvo'	Indica la tabla principal desde la que se leen los datos.
793	WHERE p.tipo_stock='RECETA' AND p.nombre='Chocolate caliente';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
794		Línea en blanco usada para separar secciones del script SQL.
795	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
796	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
797	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe caramelo'	Indica la tabla principal desde la que se leen los datos.
798	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Caramelo%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
799		Línea en blanco usada para separar secciones del script SQL.
800	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
801	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
802	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe chocolate'	Indica la tabla principal desde la que se leen los datos.
803	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Chocolate%'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

804	AND p.nombre<>'Chocolate caliente';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
805		Línea en blanco usada para separar secciones del script SQL.
806	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
807	SELECT p.id_producto,i.id_ingredient,20,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
808	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe vainilla'	Indica la tabla principal desde la que se leen los datos.
809	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Vainilla%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
810		Línea en blanco usada para separar secciones del script SQL.
811	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
812	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
813	FROM producto p JOIN ingrediente i ON i.nombre='Galleta Oreo'	Indica la tabla principal desde la que se leen los datos.
814	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Oreo%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
815		Línea en blanco usada para separar secciones del script SQL.
816	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
817	SELECT p.id_producto,i.id_ingredient,200,FALSE,TRUE,50,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
818	FROM producto p JOIN ingrediente i ON i.nombre='Base horchata'	Indica la tabla principal desde la que se leen los datos.
819	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Horchata%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
820		Línea en blanco usada para separar secciones del script SQL.
821	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
822	SELECT p.id_producto,i.id_ingredient,120,FALSE,TRUE,40,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
823	FROM producto p JOIN ingrediente i ON i.nombre='Pulpa de mango'	Indica la tabla principal desde la que se leen los datos.
824	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Mango%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
825		Línea en blanco usada para separar secciones del script SQL.
826	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
827	SELECT p.id_producto,i.id_ingredient,120,FALSE,TRUE,40,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
828	FROM producto p JOIN ingrediente i ON i.nombre='Pulpa de berries'	Indica la tabla principal desde la que se leen los datos.
829	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Berries%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
830		Línea en blanco usada para separar secciones del script SQL.
831	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
832	SELECT p.id_producto,i.id_ingredient,30,FALSE,TRUE,15,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
833	FROM producto p JOIN ingrediente i ON i.nombre='Mezcla chai'	Indica la tabla principal desde la que se leen los datos.
834	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Chai%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
835		Línea en blanco usada para separar secciones del script SQL.
836	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
837	SELECT p.id_producto,i.id_ingredient,1,FALSE,TRUE,1,1.00,2	Consulta columnas o valores calculados sin modificar los datos.
838	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
839	JOIN ingrediente i ON i.nombre='Bolsa de té'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
840	WHERE c.nombre='WlCafé' AND p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
841	AND p.nombre LIKE '%Té Guatemalteco%';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
842		Línea en blanco usada para separar secciones del script SQL.
843	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
844	SELECT p.id_producto,i.id_ingredient,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
845	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe frutal'	Indica la tabla principal desde la que se leen los datos.
846	WHERE p.tipo_stock='RECETA'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
847	AND p.nombre REGEXP 'WlFizz WlFrizz Bora Bora Melocotón Menta';	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

848		Línea en blanco usada para separar secciones del script SQL.
849	-- Helados y postres preparados.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
850	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
851	SELECT p.id_producto,i.id_ingrediente,	Consulta columnas o valores calculados sin modificar los datos.
852	CASE WHEN p.nombre LIKE '%wLfurry%' THEN 180 ELSE 120 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
853	FALSE,TRUE,60,5.00,2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
854	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
855	JOIN ingrediente i ON i.nombre='Helado de vainilla'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
856	WHERE c.nombre='Postres' AND p.tipo_stock='RECETA';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
857		Línea en blanco usada para separar secciones del script SQL.
858	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
859	SELECT p.id_producto,i.id_ingrediente,1,FALSE,FALSE,1,0,1	Consulta columnas o valores calculados sin modificar los datos.
860	FROM producto p JOIN ingrediente i ON i.nombre='Cono'	Indica la tabla principal desde la que se leen los datos.
861	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Cono%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
862		Línea en blanco usada para separar secciones del script SQL.
863	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
864	SELECT p.id_producto,i.id_ingrediente,30,FALSE,TRUE,15,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
865	FROM producto p JOIN ingrediente i ON i.nombre='Galleta Oreo'	Indica la tabla principal desde la que se leen los datos.
866	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%wLfurry Oreo%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
867		Línea en blanco usada para separar secciones del script SQL.
868	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
869	SELECT p.id_producto,i.id_ingrediente,30,FALSE,TRUE,15,3.00,2	Consulta columnas o valores calculados sin modificar los datos.
870	FROM producto p JOIN ingrediente i ON i.nombre='Chocolate M&M'	Indica la tabla principal desde la que se leen los datos.
871	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%M&M%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
872		Línea en blanco usada para separar secciones del script SQL.
873	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
874	SELECT p.id_producto,i.id_ingrediente,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
875	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe caramelo'	Indica la tabla principal desde la que se leen los datos.
876	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Caramelo%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
877		Línea en blanco usada para separar secciones del script SQL.
878	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
879	SELECT p.id_producto,i.id_ingrediente,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
880	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe chocolate'	Indica la tabla principal desde la que se leen los datos.
881	WHERE p.tipo_stock='RECETA' AND p.nombre REGEXP 'Sundae Chocolate wLfurry.*Chocolate';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
882		Línea en blanco usada para separar secciones del script SQL.
883	INSERT INTO carga_receta	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
884	SELECT p.id_producto,i.id_ingrediente,25,FALSE,TRUE,10,2.00,2	Consulta columnas o valores calculados sin modificar los datos.
885	FROM producto p JOIN ingrediente i ON i.nombre='Jarabe fresa'	Indica la tabla principal desde la que se leen los datos.
886	WHERE p.tipo_stock='RECETA' AND p.nombre LIKE '%Fresa%';	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
887		Línea en blanco usada para separar secciones del script SQL.
888	-- Inserción final de recetas, evitando pares repetidos.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
889	INSERT INTO producto_ingrediente (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
890	id_producto, id_ingrediente, cantidad_default,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
891	permite_quitar, permite_extra, cantidad_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

892	precio_extra, max_extras, estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
893	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
894	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
895	receta.id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
896	receta.id_ingredientes,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
897	receta.cantidad,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
898	receta.permite_quitar,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
899	receta.permite_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
900	receta.cantidad_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
901	receta.precio_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
902	receta.max_extras,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
903	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
904	FROM (	Indica la tabla principal desde la que se leen los datos.
905	SELECT	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
906	id_producto,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
907	id_ingredientes,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
908	MAX(cantidad) AS cantidad,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
909	MAX(permite_quitar) AS permite_quitar,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
910	MAX(permite_extra) AS permite_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
911	MAX(cantidad_extra) AS cantidad_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
912	MAX(precio_extra) AS precio_extra,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
913	MAX(max_extras) AS max_extras	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
914	FROM carga_receta	Indica la tabla principal desde la que se leen los datos.
915	GROUP BY id_producto, id_ingredientes	Agrupar filas iguales para calcular totales o resúmenes por categoría.
916	) receta	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
917	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
918	SELECT 1	Consulta columnas o valores calculados sin modificar los datos.
919	FROM producto_ingredientes pi	Indica la tabla principal desde la que se leen los datos.
920	WHERE pi.id_producto = receta.id_producto	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
921	AND pi.id_ingredientes = receta.id_ingredientes	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
922	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
923		Línea en blanco usada para separar secciones del script SQL.
924	DROP TEMPORARY TABLE carga_receta;	Elimina la tabla temporal porque su trabajo de carga ya terminó.
925		Línea en blanco usada para separar secciones del script SQL.
926	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
927	-- 5. PRESENTACIÓN INDIVIDUAL	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
928	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
929		Línea en blanco usada para separar secciones del script SQL.
930	INSERT INTO presentacion_menu (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
931	id_producto_principal, nombre, tipo, precio, predeterminada, estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
932	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
933	SELECT p.id_producto, 'Individual', 'INDIVIDUAL', p.precio_base, TRUE, TRUE	Consulta columnas o valores calculados sin modificar los datos.
934	FROM producto p	Indica la tabla principal desde la que se leen los datos.
935	WHERE p.estado=TRUE AND p.tipo_stock<>'NINGUNO'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

936	AND p.nombre NOT IN ('Papas Kids','Puré de manzana','Yogur de fresa',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
937	'Jugo de manzana Kids','Juguete sorpresa','Coca-Cola 1.5 L')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
938	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
939	SELECT 1 FROM presentacion_menu pm	Consulta columnas o valores calculados sin modificar los datos.
940	WHERE pm.id_producto_principal=p.id_producto	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
941	AND pm.tipo='INDIVIDUAL'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
942	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
943		Línea en blanco usada para separar secciones del script SQL.
944	INSERT INTO grupo_presentacion (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
945	id_presentacion,nombre,minimo,maximo,permite_repetir,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
946	visible,permite_personalizar,estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
947	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
948	SELECT pm.id_presentacion,'Producto principal',1,1,FALSE,FALSE,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
949	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
950	WHERE pm.tipo='INDIVIDUAL' AND pm.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
951	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
952	SELECT 1 FROM grupo_presentacion gp	Consulta columnas o valores calculados sin modificar los datos.
953	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
954	AND gp.nombre='Producto principal'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
955	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
956		Línea en blanco usada para separar secciones del script SQL.
957	INSERT INTO opcion_grupo (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
958	id_grupo,nombre,incremento_precio,predeterminada,estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
959	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
960	SELECT gp.id_grupo,p.nombre,0,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
961	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
962	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
963	JOIN producto p ON p.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
964	WHERE pm.tipo='INDIVIDUAL' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
965	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
966	SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo	Consulta columnas o valores calculados sin modificar los datos.
967	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
968		Línea en blanco usada para separar secciones del script SQL.
969	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
970	SELECT og.id_opcion,pm.id_producto_principal,1	Consulta columnas o valores calculados sin modificar los datos.
971	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
972	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
973	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
974	WHERE pm.tipo='INDIVIDUAL' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
975	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
976	SELECT 1 FROM opcion_componente oc	Consulta columnas o valores calculados sin modificar los datos.
977	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
978	AND oc.id_producto=pm.id_producto_principal	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
979	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.

980		Línea en blanco usada para separar secciones del script SQL.
981	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
982	-- 6. WLMENÚS DE ALMUERZO Y DESAYUNO	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
983	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
984		Línea en blanco usada para separar secciones del script SQL.
985	-- Almuerzos: precio individual + Q20.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
986	INSERT INTO presentacion_menu (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
987	id_producto_principal,nombre,tipo,precio,predeterminada,estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
988	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
989	SELECT p.id_producto,'WlMenú','MENU',p.precio_base+20,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
990	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
991	WHERE c.nombre='Almuerzos' AND p.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
992	AND p.subcategoria<>'Para compartir'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
993	AND p.nombre NOT LIKE '%10 Piezas%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
994	AND p.nombre NOT LIKE '%Dos Piezas%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
995	AND p.nombre NOT LIKE '%1 Pieza%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
996	AND p.nombre<>'Pollo WlCrispy'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
997	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
998	SELECT 1 FROM presentacion_menu pm	Consulta columnas o valores calculados sin modificar los datos.
999	WHERE pm.id_producto_principal=p.id_producto AND pm.tipo='MENU'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1000	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1001		Línea en blanco usada para separar secciones del script SQL.
1002	-- Desayunos portátiles: precio individual + Q15.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1003	INSERT INTO presentacion_menu (	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1004	id_producto_principal,nombre,tipo,precio,predeterminada,estado	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1005	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1006	SELECT p.id_producto,'WlMenú Desayuno','MENU',p.precio_base+15,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1007	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
1008	WHERE c.nombre='Desayunos' AND p.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1009	AND (p.nombre LIKE '%WlMuffin%' OR p.nombre LIKE '%WlGriddle%'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1010	OR p.nombre IN ('Burrito','Burritos'))	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1011	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1012	SELECT 1 FROM presentacion_menu pm	Consulta columnas o valores calculados sin modificar los datos.
1013	WHERE pm.id_producto_principal=p.id_producto AND pm.tipo='MENU'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1014	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1015		Línea en blanco usada para separar secciones del script SQL.
1016	-- Grupos de toda presentación MENU.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1017	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1018	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1019	SELECT pm.id_presentacion,'Producto principal',1,1,FALSE,FALSE,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1020	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1021	WHERE pm.tipo='MENU'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1022	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1023	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

1024	AND gp.nombre='Producto principal');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1025		Línea en blanco usada para separar secciones del script SQL.
1026	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1027	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1028	SELECT pm.id_presentacion, 'Elige el complemento',1,1,FALSE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1029	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1030	WHERE pm.tipo='MENU'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1031	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1032	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1033	AND gp.nombre='Elige el complemento');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1034		Línea en blanco usada para separar secciones del script SQL.
1035	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1036	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1037	SELECT pm.id_presentacion, 'Elige la bebida',1,1,FALSE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1038	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1039	WHERE pm.tipo='MENU'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1040	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1041	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1042	AND gp.nombre='Elige la bebida');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1043		Línea en blanco usada para separar secciones del script SQL.
1044	-- Producto principal de cada WlMenú.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1045	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1046	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1047	SELECT gp.id_grupo,p.nombre,0,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1048	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1049	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1050	JOIN producto p ON p.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1051	WHERE pm.tipo='MENU' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1052	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1053		Línea en blanco usada para separar secciones del script SQL.
1054	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1055	SELECT og.id_opcion,pm.id_producto_principal,1	Consulta columnas o valores calculados sin modificar los datos.
1056	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1057	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1058	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1059	WHERE pm.tipo='MENU' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1060	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1061	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1062	AND oc.id_producto=pm.id_producto_principal);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1063		Línea en blanco usada para separar secciones del script SQL.
1064	-- Complementos. Los menús de desayuno usan Hash Brown; los demás, Papas.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1065	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1066	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1067	SELECT gp.id_grupo,	Consulta columnas o valores calculados sin modificar los datos.

1068	CASE WHEN principal.disponibilidad_menu='DESAYUNO'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1069	THEN 'Hash Brown' ELSE 'Papas' END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1070	0,TRUE,TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1071	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1072	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1073	JOIN producto principal ON principal.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1074	WHERE pm.tipo='MENU' AND gp.nombre='Elige el complemento'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1075	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1076		Línea en blanco usada para separar secciones del script SQL.
1077	-- En los menús que incluyen Papas también se permite cambiarlas por	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1078	-- WlPatatas. El precio adicional corresponde a una porción.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1079	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1080	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1081	SELECT gp.id_grupo,'WlPatatas',@precio_extra_wlpatatas,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1082	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1083	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1084	WHERE pm.tipo='MENU' AND gp.nombre='Elige el complemento'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1085	AND EXISTS (SELECT 1 FROM opcion_grupo papas	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1086	WHERE papas.id_grupo=gp.id_grupo	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1087	AND papas.nombre='Papas')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1088	AND NOT EXISTS (SELECT 1 FROM opcion_grupo existente	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1089	WHERE existente.id_grupo=gp.id_grupo	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1090	AND existente.nombre='WlPatatas');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1091		Línea en blanco usada para separar secciones del script SQL.
1092	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1093	SELECT og.id_opcion,componente.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1094	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1095	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1096	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1097	JOIN producto componente ON componente.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1098	WHERE pm.tipo='MENU' AND gp.nombre='Elige el complemento'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1099	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1100	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1101	AND oc.id_producto=componente.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1102		Línea en blanco usada para separar secciones del script SQL.
1103	-- Bebidas permitidas según horario del producto principal.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1104	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1105	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1106	SELECT gp.id_grupo,bebida.nombre,	Consulta columnas o valores calculados sin modificar los datos.
1107	CASE WHEN bebida.nombre LIKE 'Jugo%' THEN 2 ELSE 0 END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1108	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1109	WHEN principal.disponibilidad_menu='DESAYUNO' AND bebida.nombre='Café' THEN TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1110	WHEN principal.disponibilidad_menu<>'DESAYUNO' AND bebida.nombre='Coca-Cola' THEN TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1111	ELSE FALSE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

1112	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1113	TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1114	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1115	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1116	JOIN producto_principal ON principal.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1117	JOIN producto_bebida ON bebida.nombre IN (	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1118	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Café' ELSE 'Coca-Cola' END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1119	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Jugo naranja' ELSE 'Coca-Cola Zero' END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1120	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Jugo manzana' ELSE 'Sprite' END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1121	CASE WHEN principal.disponibilidad_menu='DESAYUNO' THEN 'Agua' ELSE 'Fanta' END	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1122	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1123	WHERE pm.tipo='MENU' AND gp.nombre='Elige la bebida'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1124	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1125	WHERE og.id_grupo=gp.id_grupo AND og.nombre=bebida.nombre);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1126		Línea en blanco usada para separar secciones del script SQL.
1127	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1128	SELECT og.id_opcion,bebida.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1129	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1130	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1131	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1132	JOIN producto_bebida ON bebida.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1133	WHERE pm.tipo='MENU' AND gp.nombre='Elige la bebida'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1134	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1135	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1136	AND oc.id_producto=bebida.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1137		Línea en blanco usada para separar secciones del script SQL.
1138	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1139	-- 7. CAJITAS FELICES	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1140	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1141		Línea en blanco usada para separar secciones del script SQL.
1142	INSERT INTO presentacion_menu	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1143	(id_producto_principal,nombre,tipo,precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1144	SELECT p.id_producto,'Cajita Feliz','INFANTIL',p.precio_base,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1145	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
1146	WHERE c.nombre='Cajita Feliz' AND p.nombre LIKE 'Cajita Feliz%'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1147	AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1148	WHERE pm.id_producto_principal=p.id_producto);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1149		Línea en blanco usada para separar secciones del script SQL.
1150	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1151	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1152	SELECT pm.id_presentacion,'Producto principal',1,1,FALSE,FALSE,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1153	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1154	WHERE pm.tipo='INFANTIL'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1155	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

1156	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1157	AND gp.nombre='Producto principal');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1158		Línea en blanco usada para separar secciones del script SQL.
1159	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1160	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1161	SELECT pm.id_presentacion,'Elige el complemento',1,1,FALSE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1162	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1163	WHERE pm.tipo='INFANTIL'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1164	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1165	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1166	AND gp.nombre='Elige el complemento');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1167		Línea en blanco usada para separar secciones del script SQL.
1168	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1169	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1170	SELECT pm.id_presentacion,'Elige la bebida',1,1,FALSE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1171	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1172	WHERE pm.tipo='INFANTIL'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1173	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1174	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1175	AND gp.nombre='Elige la bebida');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1176		Línea en blanco usada para separar secciones del script SQL.
1177	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1178	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1179	SELECT pm.id_presentacion,'Elige el juguete',1,1,FALSE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1180	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1181	WHERE pm.tipo='INFANTIL'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1182	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1183	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1184	AND gp.nombre='Elige el juguete');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1185		Línea en blanco usada para separar secciones del script SQL.
1186	-- Entrada fija de cada Cajita.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1187	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1188	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1189	SELECT gp.id_grupo,	Consulta columnas o valores calculados sin modificar los datos.
1190	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1191	WHEN raiz.nombre LIKE '%derretido%' THEN 'Derretido clásico'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1192	WHEN raiz.nombre LIKE '%Hot Cakes%' THEN 'Hot Cakes'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1193	WHEN raiz.nombre LIKE '%Pollo wCrispy%' THEN 'Pollo wCrispy 1 Pieza'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1194	WHEN raiz.nombre LIKE '%quesoburguesa%' THEN 'Quesoburguesa'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1195	WHEN raiz.nombre LIKE '%wNuggets%' THEN 'wNuggets 4 pz'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1196	WHEN raiz.nombre LIKE '%frijol%' THEN 'wMuffin Huevo y Frijol'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1197	WHEN raiz.nombre LIKE '%Huevo y queso%' THEN 'wMuffin Huevo y Queso'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1198	WHEN raiz.nombre LIKE '%salchicha%' THEN 'wMuffin Salchicha'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1199	ELSE 'Hamburguesa'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

1200	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1201	0, TRUE, TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1202	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1203	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1204	JOIN producto raiz ON raiz.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1205	WHERE pm.tipo='INFANTIL' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1206	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1207		Línea en blanco usada para separar secciones del script SQL.
1208	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1209	SELECT og.id_opcion,componente.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1210	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1211	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1212	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1213	JOIN producto componente ON componente.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1214	WHERE pm.tipo='INFANTIL' AND gp.nombre='Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1215	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1216	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1217	AND oc.id_producto=componente.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1218		Línea en blanco usada para separar secciones del script SQL.
1219	-- Complementos infantiles.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1220	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1221	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1222	SELECT gp.id_grupo,componente.nombre,0,	Consulta columnas o valores calculados sin modificar los datos.
1223	componente.nombre='Papas Kids',TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1224	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1225	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1226	JOIN producto componente ON componente.nombre IN	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1227	('Papas Kids','Puré de manzana','Yogur de fresa')	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1228	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige el complemento'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1229	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1230	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1231		Línea en blanco usada para separar secciones del script SQL.
1232	-- Cambio opcional de Papas Kids por una porción de WlPatatas.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1233	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1234	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1235	SELECT gp.id_grupo,'WlPatatas',@precio_extra_wlpatatas,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1236	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1237	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1238	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige el complemento'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1239	AND EXISTS (SELECT 1 FROM opcion_grupo papas	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1240	WHERE papas.id_grupo=gp.id_grupo	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1241	AND papas.nombre='Papas Kids')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1242	AND NOT EXISTS (SELECT 1 FROM opcion_grupo existente	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1243	WHERE existente.id_grupo=gp.id_grupo	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

1244	AND existente.nombre='WlPatatas');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1245		Línea en blanco usada para separar secciones del script SQL.
1246	-- Bebidas infantiles.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1247	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1248	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1249	SELECT gp.id_grupo,componente.nombre,0,	Consulta columnas o valores calculados sin modificar los datos.
1250	componente.nombre='Jugo de manzana Kids',TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1251	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1252	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1253	JOIN producto componente ON componente.nombre IN ('Jugo de manzana Kids','Agua')	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1254	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige la bebida'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1255	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1256	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1257		Línea en blanco usada para separar secciones del script SQL.
1258	-- Juguete.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1259	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1260	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1261	SELECT gp.id_grupo,'Juguete sorpresa',0,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1262	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1263	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1264	WHERE pm.tipo='INFANTIL' AND gp.nombre='Elige el juguete'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1265	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1266		Línea en blanco usada para separar secciones del script SQL.
1267	-- Componentes de complemento, bebida y juguete.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1268	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1269	SELECT og.id_opcion,componente.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1270	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1271	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1272	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1273	JOIN producto componente ON componente.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1274	WHERE pm.tipo='INFANTIL' AND gp.nombre<>'Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1275	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1276	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1277	AND oc.id_producto=componente.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1278		Línea en blanco usada para separar secciones del script SQL.
1279	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1280	-- 8. COMBOS Y CAJAS PREDEFINIDAS	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1281	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1282		Línea en blanco usada para separar secciones del script SQL.
1283	INSERT INTO presentacion_menu	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1284	(id_producto_principal,nombre,tipo,precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1285	SELECT p.id_producto,'Combo','COMBO',p.precio_base,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1286	FROM producto p JOIN categoria c ON c.id_categoria=p.id_categoria	Indica la tabla principal desde la que se leen los datos.
1287	WHERE p.tipo_stock='NINGUNO' AND c.nombre<>'Cajita Feliz'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

1288	AND NOT EXISTS (SELECT 1 FROM presentacion_menu pm	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1289	WHERE pm.id_producto_principal=p.id_producto);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1290		Línea en blanco usada para separar secciones del script SQL.
1291	-- Cajas Grandes configurables: cuatro principales independientes.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1292	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1293	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1294	SELECT pm.id_presentacion,'Elige tus productos',4,4,TRUE,TRUE,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1295	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Indica la tabla principal desde la que se leen los datos.
1296	WHERE pm.tipo='COMBO' AND p.nombre IN ('Caja Grande','Caja Grande con Postre'))	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1297	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1298	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1299	AND gp.nombre='Elige tus productos');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1300		Línea en blanco usada para separar secciones del script SQL.
1301	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1302	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1303	SELECT gp.id_grupo,componente.nombre,0,componente.nombre='Big Mac',TRUE	Consulta columnas o valores calculados sin modificar los datos.
1304	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1305	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1306	JOIN producto componente ON componente.nombre IN	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1307	('Big Mac','Cuarto de Libra con Queso','WLNifica Doble','10 WLNuggets de Pollo')	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1308	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus productos'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1309	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1310	WHERE og.id_grupo=gp.id_grupo AND og.nombre=componente.nombre);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1311		Línea en blanco usada para separar secciones del script SQL.
1312	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1313	SELECT og.id_opcion,componente.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1314	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1315	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1316	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1317	JOIN producto componente ON componente.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1318	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus productos'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1319	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1320	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1321	AND oc.id_producto=componente.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1322		Línea en blanco usada para separar secciones del script SQL.
1323	-- Contenido fijo de cajas grandes. Las papas se crean más adelante como	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1324	-- una elección visible, por lo que aquí solo queda la bebida familiar.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1325	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1326	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1327	SELECT pm.id_presentacion,'Incluye',1,1,FALSE,FALSE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1328	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Indica la tabla principal desde la que se leen los datos.
1329	WHERE pm.tipo='COMBO' AND p.nombre IN ('Caja Grande','Caja Grande con Postre'))	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1330	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1331	WHERE gp.id_presentacion=pm.id_presentacion AND gp.nombre='Incluye');	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.

1332		Línea en blanco usada para separar secciones del script SQL.
1333	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1334	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1335	SELECT gp.id_grupo, 'Bebida familiar',0,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1336	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1337	WHERE gp.nombre='Incluye'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1338	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1339		Línea en blanco usada para separar secciones del script SQL.
1340	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1341	SELECT og.id_opcion,p.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1342	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1343	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1344	JOIN producto p ON p.nombre='Coca-Cola 1.5 L'	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1345	WHERE gp.nombre='Incluye'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1346	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1347	WHERE oc.id_opcion=og.id_opcion AND oc.id_producto=p.id_producto);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1348		Línea en blanco usada para separar secciones del script SQL.
1349	-- Postres seleccionables para Caja Grande con Postre.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1350	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1351	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1352	SELECT pm.id_presentacion, 'Elige tus postres',2,2,TRUE,TRUE,FALSE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1353	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Indica la tabla principal desde la que se leen los datos.
1354	WHERE pm.tipo='COMBO' AND p.nombre='Caja Grande con Postre'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1355	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1356	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1357	AND gp.nombre='Elige tus postres');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1358		Línea en blanco usada para separar secciones del script SQL.
1359	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1360	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1361	SELECT gp.id_grupo,postre.nombre,0,postre.nombre='Pastel de manzana',TRUE	Consulta columnas o valores calculados sin modificar los datos.
1362	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1363	JOIN producto postre ON postre.nombre IN	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1364	('Pastel de manzana','Pastel de queso','Sundae Chocolate','Sundae Fresa')	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1365	WHERE gp.nombre='Elige tus postres'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1366	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1367	WHERE og.id_grupo=gp.id_grupo AND og.nombre=postre.nombre);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1368		Línea en blanco usada para separar secciones del script SQL.
1369	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1370	SELECT og.id_opcion,postre.id_producto,1	Consulta columnas o valores calculados sin modificar los datos.
1371	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1372	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1373	JOIN producto postre ON postre.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1374	WHERE gp.nombre='Elige tus postres'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1375	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

1376	WHERE oc.id_opcion=og.id_opcion AND oc.id_producto=postre.id_producto);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1377		Línea en blanco usada para separar secciones del script SQL.
1378	-- Resto de combos predefinidos: una opción fija con varios componentes.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1379	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1380	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1381	SELECT pm.id_presentacion,'Contenido del combo',1,1,FALSE,FALSE,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1382	FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal	Indica la tabla principal desde la que se leen los datos.
1383	WHERE pm.tipo='COMBO' AND p.nombre NOT IN ('Caja Grande','Caja Grande con Postre')	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1384	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion gp	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1385	WHERE gp.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1386	AND gp.nombre='Contenido del combo');	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1387		Línea en blanco usada para separar secciones del script SQL.
1388	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1389	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1390	SELECT gp.id_grupo,p.nombre,0,TRUE,TRUE	Consulta columnas o valores calculados sin modificar los datos.
1391	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1392	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1393	JOIN producto p ON p.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1394	WHERE pm.tipo='COMBO' AND gp.nombre='Contenido del combo'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1395	AND NOT EXISTS (SELECT 1 FROM opcion_grupo og WHERE og.id_grupo=gp.id_grupo);	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1396		Línea en blanco usada para separar secciones del script SQL.
1397	-- Componentes fijos de cada caja o bucket.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1398	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1399	SELECT og.id_opcion,componente.id_producto,	Consulta columnas o valores calculados sin modificar los datos.
1400	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1401	WHEN raiz.nombre='Caja de 24 WlNuggets'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1402	AND componente.nombre='10 WlNuggets de Pollo' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1403	WHEN raiz.nombre='Caja Grande Snack' AND componente.nombre='Quesoburguesa' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1404	WHEN raiz.nombre LIKE 'Caja Grande D%' AND componente.nombre='Hash Brown' THEN 4	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1405	WHEN raiz.nombre LIKE 'Caja Grande D%' AND componente.nombre='Café' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1406	WHEN raiz.nombre='Caja Grande Deluxe' AND componente.nombre='Hot Cakes' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1407	WHEN raiz.nombre='Caja Grande Deluxe' AND componente.nombre='WlMuffin de Huevo' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1408	WHEN raiz.nombre='Caja Grande Desayuno' AND componente.nombre='WlMuffin Salchicha' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1409	ELSE 1	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1410	END	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1411	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1412	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1413	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1414	JOIN producto raiz ON raiz.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1415	JOIN producto componente ON	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1416	(raiz.nombre='Bucket para Todos' AND componente.nombre IN	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1417	('Pollo WlCrispy 10 Piezas','Coca-Cola 1.5 L'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1418	OR (raiz.nombre='Bucket Pollo WlCrispy' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1419	('Pollo WlCrispy 10 Piezas'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.

1420	OR (raiz.nombre='Bucket Pollo WlCrispy Para Tres' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1421	('Pollo WlCrispy 10 Piezas', 'Coca-Cola 1.5 L'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1422	OR (raiz.nombre='Bucket Pollo WlCrispy Snack' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1423	('Pollo WlCrispy Dos Piezas', 'WlNuggets 4 pz'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1424	OR (raiz.nombre='Caja de 24 WlNuggets' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1425	('10 WlNuggets de Pollo', 'WlNuggets 4 pz'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1426	OR (raiz.nombre='Caja Grande Snack' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1427	('Quesoburguesa', 'WlNuggets 4 pz', 'Coca-Cola 1.5 L'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1428	OR (raiz.nombre='Caja Grande Deluxe' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1429	('Hot Cakes', 'WlMuffin de Huevo', 'Hash Brown', 'Café'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1430	OR (raiz.nombre='Caja Grande Desayuno' AND componente.nombre IN	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1431	('WlMuffin Salchicha', 'Burritos', 'Hash Brown', 'Café'))	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1432	WHERE pm.tipo='COMBO' AND gp.nombre='Contenido del combo'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1433	AND NOT EXISTS (SELECT 1 FROM opcion_componente oc	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1434	WHERE oc.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1435	AND oc.id_producto=componente.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1436		Línea en blanco usada para separar secciones del script SQL.
1437	-- Elección de papas de los combos que originalmente incluyen Papas.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1438	-- Caja Grande lleva 4 porciones, los buckets 3 y Caja Grande Snack 2.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1439	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1440	(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1441	SELECT pm.id_presentacion, 'Elige tus papas', 1, 1, FALSE, TRUE, FALSE, TRUE	Consulta columnas o valores calculados sin modificar los datos.
1442	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
1443	JOIN producto_principal ON principal.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1444	WHERE pm.tipo='COMBO'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1445	AND principal.nombre IN (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1446	'Bucket para Todos',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1447	'Bucket Pollo WlCrispy',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1448	'Bucket Pollo WlCrispy Para Tres',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1449	'Bucket Pollo WlCrispy Snack',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1450	'Caja Grande',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1451	'Caja Grande con Postre',	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1452	'Caja Grande Snack'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1453	)	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1454	AND NOT EXISTS (SELECT 1 FROM grupo_presentacion existente	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1455	WHERE existente.id_presentacion=pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1456	AND existente.nombre='Elige tus papas'));	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1457		Línea en blanco usada para separar secciones del script SQL.
1458	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1459	(id_grupo,nombre,incremento_precio,predeterminada,estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
1460	SELECT gp.id_grupo,acompanamiento.nombre,	Consulta columnas o valores calculados sin modificar los datos.
1461	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1462	WHEN acompanamiento.nombre='Papas' THEN 0	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1463	WHEN principal.nombre IN ('Caja Grande', 'Caja Grande con Postre')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

1464	THEN @precio_extra_wlpatatas * 4	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1465	WHEN principal.nombre='Caja Grande Snack'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1466	THEN @precio_extra_wlpatatas * 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1467	ELSE @precio_extra_wlpatatas * 3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1468	END,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1469	acompanamiento.nombre='Papas',TRUE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1470	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1471	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1472	JOIN producto_principal ON principal.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1473	JOIN producto_acompanamiento ON acompanamiento.nombre IN ('Papas','WlPatatas')	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1474	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus papas'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1475	AND NOT EXISTS (SELECT 1 FROM opcion_grupo existente	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1476	WHERE existente.id_grupo=gp.id_grupo	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1477	AND existente.nombre=acompanamiento.nombre);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1478		Línea en blanco usada para separar secciones del script SQL.
1479	INSERT INTO opcion_componente (id_opcion,id_producto,cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
1480	SELECT og.id_opcion,acompanamiento.id_producto,	Consulta columnas o valores calculados sin modificar los datos.
1481	CASE	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1482	WHEN principal.nombre IN ('Caja Grande','Caja Grande con Postre') THEN 4	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1483	WHEN principal.nombre='Caja Grande Snack' THEN 2	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1484	ELSE 3	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1485	END	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1486	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1487	JOIN grupo_presentacion gp ON gp.id_grupo=og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1488	JOIN presentacion_menu pm ON pm.id_presentacion=gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1489	JOIN producto_principal ON principal.id_producto=pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1490	JOIN producto_acompanamiento ON acompanamiento.nombre=og.nombre	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1491	WHERE pm.tipo='COMBO' AND gp.nombre='Elige tus papas'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1492	AND og.nombre IN ('Papas','WlPatatas')	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1493	AND NOT EXISTS (SELECT 1 FROM opcion_componente existente	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1494	WHERE existente.id_opcion=og.id_opcion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1495	AND existente.id_producto=acompanamiento.id_producto);	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1496		Línea en blanco usada para separar secciones del script SQL.
1497	COMMIT;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1498		Línea en blanco usada para separar secciones del script SQL.
1499	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1500	-- 9. COMPROBACIONES	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1501	-- Todas estas consultas deben devolver cero filas, salvo el resumen.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1502	-- =====	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1503		Línea en blanco usada para separar secciones del script SQL.
1504	-- Productos RECETA sin ingredientes.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1505	SELECT p.id_producto,p.nombre AS receta_sin_ingredientes	Consulta columnas o valores calculados sin modificar los datos.
1506	FROM producto p	Indica la tabla principal desde la que se leen los datos.
1507	LEFT JOIN producto_ingrediente pi ON pi.id_producto=p.id_producto AND pi.estado=TRUE	Relaciona filas de dos tablas mediante sus identificadores vinculados.

1508	WHERE p.tipo_stock='RECETA' AND p.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1509	GROUP BY p.id_producto,p.nombre	Agrupar filas iguales para calcular totales o resúmenes por categoría.
1510	HAVING COUNT(pi.id_producto_ingredient)=0;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1511		Línea en blanco usada para separar secciones del script SQL.
1512	-- Productos vendibles sin presentación.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1513	SELECT p.id_producto,p.nombre AS producto_sin_presentacion	Consulta columnas o valores calculados sin modificar los datos.
1514	FROM producto p	Indica la tabla principal desde la que se leen los datos.
1515	LEFT JOIN presentacion_menu pm ON pm.id_producto_principal=p.id_producto AND pm.estado=TRUE	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1516	WHERE p.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1517	GROUP BY p.id_producto,p.nombre	Agrupar filas iguales para calcular totales o resúmenes por categoría.
1518	HAVING COUNT(pm.id_presentacion)=0	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1519	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1520	SELECT 1 FROM opcion_componente oc WHERE oc.id_producto=p.id_producto	Consulta columnas o valores calculados sin modificar los datos.
1521	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
1522		Línea en blanco usada para separar secciones del script SQL.
1523	-- Grupos sin opciones.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1524	SELECT gp.id_grupo,gp.nombre AS grupo_sin_opciones	Consulta columnas o valores calculados sin modificar los datos.
1525	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
1526	LEFT JOIN opcion_grupo og ON og.id_grupo=gp.id_grupo AND og.estado=TRUE	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1527	WHERE gp.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1528	GROUP BY gp.id_grupo,gp.nombre	Agrupar filas iguales para calcular totales o resúmenes por categoría.
1529	HAVING COUNT(og.id_opcion)=0;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1530		Línea en blanco usada para separar secciones del script SQL.
1531	-- Opciones sin productos componentes.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1532	SELECT og.id_opcion,og.nombre AS opcion_sin_componentes	Consulta columnas o valores calculados sin modificar los datos.
1533	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
1534	LEFT JOIN opcion_componente oc ON oc.id_opcion=og.id_opcion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
1535	WHERE og.estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1536	GROUP BY og.id_opcion,og.nombre	Agrupar filas iguales para calcular totales o resúmenes por categoría.
1537	HAVING COUNT(oc.id_opcion_componente)=0;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
1538		Línea en blanco usada para separar secciones del script SQL.
1539	-- Resumen final por tipo de stock.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
1540	SELECT tipo_stock,COUNT(*) AS productos	Consulta columnas o valores calculados sin modificar los datos.
1541	FROM producto	Indica la tabla principal desde la que se leen los datos.
1542	WHERE estado=TRUE	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
1543	GROUP BY tipo_stock	Agrupar filas iguales para calcular totales o resúmenes por categoría.
1544	ORDER BY tipo_stock;	Ordena el resultado según las columnas indicadas.

16.19 sql/migracion_productos_existentes.sql

Script SQL migracion_productos_existentes. Prepara, migra o prueba una parte de la base de datos.

Tipo: .sql | Líneas: 47 | Archivo 19 de 157

1	USE waldonalds;	Selecciona la base de datos donde se ejecutarán las siguientes sentencias.
2		Línea en blanco usada para separar secciones del script SQL.
3	-- Crea una presentación individual para los productos anteriores a este cambio.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
4	-- No duplica productos que ya tengan alguna presentación configurada.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
5	INSERT INTO presentacion_menu	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
6	(id_producto_principal, nombre, tipo, precio, determinada, estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
7	SELECT p.id_producto, 'Individual', 'INDIVIDUAL', p.precio_base, TRUE, TRUE	Consulta columnas o valores calculados sin modificar los datos.
8	FROM producto p	Indica la tabla principal desde la que se leen los datos.
9	WHERE NOT EXISTS (	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
10	SELECT 1 FROM presentacion_menu pm	Consulta columnas o valores calculados sin modificar los datos.
11	WHERE pm.id_producto_principal = p.id_producto	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
12	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
13		Línea en blanco usada para separar secciones del script SQL.
14	-- Grupo interno que permite tratar al producto principal igual que a cualquier	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
15	-- producto incluido en un menú o combo.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
16	INSERT INTO grupo_presentacion	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
17	(id_presentacion, nombre, minimo, maximo, permite_repetir,	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
18	visible, permite_personalizar, estado)	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
19	SELECT pm.id_presentacion, '__Producto principal', 1, 1, FALSE, FALSE, TRUE, TRUE	Consulta columnas o valores calculados sin modificar los datos.
20	FROM presentacion_menu pm	Indica la tabla principal desde la que se leen los datos.
21	WHERE pm.tipo = 'INDIVIDUAL'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
22	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
23	SELECT 1 FROM grupo_presentacion gp	Consulta columnas o valores calculados sin modificar los datos.
24	WHERE gp.id_presentacion = pm.id_presentacion	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
25	AND gp.nombre = '__Producto principal'	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
26	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
27		Línea en blanco usada para separar secciones del script SQL.
28	INSERT INTO opcion_grupo	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
29	(id_grupo, nombre, incremento_precio, determinada, estado)	Aporta uno o varios registros concretos para la inserción iniciada anteriormente.
30	SELECT gp.id_grupo, p.nombre, 0, TRUE, TRUE	Consulta columnas o valores calculados sin modificar los datos.
31	FROM grupo_presentacion gp	Indica la tabla principal desde la que se leen los datos.
32	JOIN presentacion_menu pm ON pm.id_presentacion = gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
33	JOIN producto p ON p.id_producto = pm.id_producto_principal	Relaciona filas de dos tablas mediante sus identificadores vinculados.
34	WHERE gp.nombre = '__Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
35	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
36	SELECT 1 FROM opcion_grupo og WHERE og.id_grupo = gp.id_grupo	Consulta columnas o valores calculados sin modificar los datos.
37	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.
38		Línea en blanco usada para separar secciones del script SQL.

39	INSERT INTO opcion_componente (id_opcion, id_producto, cantidad)	Inserta nuevas filas en la tabla indicada; las líneas siguientes definen columnas y valores.
40	SELECT og.id_opcion, pm.id_producto_principal, 1	Consulta columnas o valores calculados sin modificar los datos.
41	FROM opcion_grupo og	Indica la tabla principal desde la que se leen los datos.
42	JOIN grupo_presentacion gp ON gp.id_grupo = og.id_grupo	Relaciona filas de dos tablas mediante sus identificadores vinculados.
43	JOIN presentacion_menu pm ON pm.id_presentacion = gp.id_presentacion	Relaciona filas de dos tablas mediante sus identificadores vinculados.
44	WHERE gp.nombre = '__Producto principal'	Filtra las filas para afectar o devolver únicamente las que cumplen la condición.
45	AND NOT EXISTS (	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
46	SELECT 1 FROM opcion_componente oc WHERE oc.id_opcion = og.id_opcion	Consulta columnas o valores calculados sin modificar los datos.
47	);	Cierra y finaliza la sentencia SQL iniciada en las líneas anteriores.

16.20 sql/migracion_turnos_usuario.sql

Script SQL migracion_turnos_usuario. Prepara, migra o prueba una parte de la base de datos.

Tipo: .sql | Líneas: 8 | Archivo 20 de 157

1	USE waldonalds;	Selecciona la base de datos donde se ejecutarán las siguientes sentencias.
2		Línea en blanco usada para separar secciones del script SQL.
3	-- Ejecutar una sola vez sobre una base existente.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
4	-- Estas columnas son administradas desde Gestión de Usuarios.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.
5	ALTER TABLE usuario	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
6	ADD COLUMN turno ENUM('MANANA', 'TARDE') NULL,	Define una columna que solo acepta una de las opciones enumeradas en esta línea.
7	ADD COLUMN hora_inicio TIME NULL,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.
8	ADD COLUMN hora_fin TIME NULL;	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.

16.21 sql/pagos.sql

Script SQL pagos. Prepara, migra o prueba una parte de la base de datos.

Tipo: .sql | Líneas: 9 | Archivo 21 de 157

1	-- Ejecutar una vez en la base waldonalds. No modifica las ventas existentes.	Comentario SQL para explicar la intención del bloque; MySQL no lo ejecuta.	
2	CREATE TABLE IF NOT EXISTS pago_operacion (	Comienza la definición de la tabla IF.	
3	clave CHAR(36) NOT NULL PRIMARY KEY,	Define la clave primaria: identifica cada fila de manera única.	
4	solicitud TEXT NOT NULL,	Define una columna obligatoria; no se permite guardar NULL en ella.	
5	id_pedido INT NULL UNIQUE,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.	
6	comprobante TEXT NULL,	Continúa la sentencia SQL actual con columnas, valores, relaciones o condiciones.	
7	referencia VARCHAR(100) NOT NULL DEFAULT '',	Define una columna obligatoria; no se permite guardar NULL en ella.	
8	FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido)	Define una clave foránea que conecta esta tabla con la clave primaria de otra tabla.	
9	) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;	Define una columna y el valor automático que recibirá si el INSERT no envía otro.	

16.22 src/BaseDatos/BaseDatos.java

Archivo de apoyo del proyecto: src/BaseDatos/BaseDatos.java.

Tipo: java | Líneas: 13 | Archivo 22 de 157

Clases/tipos detectados: BaseDatos

1	/*		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package BaseDatos;		Declara el paquete o carpeta lógica a la que pertenece esta clase.
6			Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author Computacion		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class BaseDatos {		Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12			Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	}		Cierra el bloque abierto anteriormente.

16.23 src/Componentes/AnimacionLetras.java

Componente visual reutilizable AnimacionLetras. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 141 | Archivo 23 de 157

Clases/tipos detectados: AnimacionLetras

Línea	Constructor o método detectado
16	public AnimacionLetras() {
34	public void addNotify() {
59	public void iniciarAnimacion() {
97	public void reiniciarAnimacion() {
106	public String getTextoAnimado() {
111	public void setTextoAnimado(String textoAnimado) {
127	public int getVelocidad() {
132	public void setVelocidad(int velocidad) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
4	import java.beans.Beans;	Importa java.beans.Beans para poder usar esa clase sin escribir su nombre completo.
5	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
6	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	public class AnimacionLetras extends JLabel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	private String textoAnimado = "¡Bienvenido a Waldonald's";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
11	private int velocidad = 80;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	private Timer timer;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
14	private int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	public AnimacionLetras() {	Constructor de AnimacionLetras: prepara un objeto nuevo y deja sus componentes listos para usarse.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	// Texto visible mientras diseñas en NetBeans	Comentario para humanos; Java no ejecuta este texto. Texto visible mientras diseñas en NetBeans
19	setText(textoAnimado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	setFont(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
23	"DM Sans",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25	36	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

26	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	}	Cierra el bloque abierto anteriormente.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	// CUANDO EL COMPONENTE APARECE EN PANTALLA	Comentario para humanos; Java no ejecuta este texto. CUANDO EL COMPONENTE APARECE EN PANTALLA
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
34	public void addNotify() {	Declara el método addNotify. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	super.addNotify();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	if (!Beans.isDesignTime()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	Timer inicio = new Timer(200, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	((Timer) e.getSource()).stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	inicio.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
48	inicio.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	setText(textoAnimado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	}	Cierra el bloque abierto anteriormente.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	// ANIMACIÓN	Comentario para humanos; Java no ejecuta este texto. ANIMACIÓN
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	public void iniciarAnimacion() {	Declara el método iniciarAnimacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	// Detener animación anterior si existe	Comentario para humanos; Java no ejecuta este texto. Detener animación anterior si existe
62	if (timer != null && timer.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
63	timer.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	}	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	indice = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	// Borra el texto antes de iniciar	Comentario para humanos; Java no ejecuta este texto. Borra el texto antes de iniciar
69	setText("");	Ejecuta una instrucción Java completa; el punto y coma marca su final.

70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	timer = new Timer(velocidad, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	if (indice < textoAnimado.length()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	setText(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	textoAnimado.substring(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	indice	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
81	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	timer.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	}	Cierra el bloque abierto anteriormente.
88	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	}	Cierra el bloque abierto anteriormente.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	// REINICIAR ANIMACIÓN	Comentario para humanos; Java no ejecuta este texto. REINICIAR ANIMACIÓN
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	public void reiniciarAnimacion() {	Declara el método reiniciarAnimacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	// TEXTO	Comentario para humanos; Java no ejecuta este texto. TEXTO
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	public String getTextoAnimado() {	Declara el método getTextoAnimado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	return textoAnimado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
109	}	Cierra el bloque abierto anteriormente.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	public void setTextoAnimado(String textoAnimado) {	Declara el método setTextoAnimado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	this.textoAnimado = textoAnimado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
116	* En el diseñador muestra el texto completo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
117	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
118	if (Beans.isDesignTime()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
119	setText(textoAnimado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	}	Cierra el bloque abierto anteriormente.
121	}	Cierra el bloque abierto anteriormente.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	// VELOCIDAD	Comentario para humanos; Java no ejecuta este texto. VELOCIDAD
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	public int getVelocidad() {	Declara el método getVelocidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	return velocidad;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
130	}	Cierra el bloque abierto anteriormente.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	public void setVelocidad(int velocidad) {	Declara el método setVelocidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	if (velocidad < 10) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
135	velocidad = 10;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
136	}	Cierra el bloque abierto anteriormente.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	this.velocidad = velocidad;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
139	}	Cierra el bloque abierto anteriormente.
140	}	Cierra el bloque abierto anteriormente.
141		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.24 src/Componentes/BarraTituloForm.form

Metadatos visuales de NetBeans para diseñar BarraTituloForm. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 102 | Archivo 24 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="1e" green="f" red="1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<Dimension value="[1530, 45]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</Property>	Cierra la sección XML abierta anteriormente.
11	</Properties>	Cierra la sección XML abierta anteriormente.
12	<AccessibilityProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<Property name="AccessibleContext.accessibleName" type="java.lang.String" value="">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
14	</AccessibilityProperties>	Cierra la sección XML abierta anteriormente.
15	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
24	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84, -19, 0, 5, 115, 114, 0, 18, 106, 97, 118, 97, 46, 97, 119, 116, 46, 68, 105, 109, 101, 110, 115, 105, 111, 110, 65, -114, -39, -41, -84, 95, 68, 20, 2, 0, 2, 73, 0, 6, 104, 101, 105, 103, 104, 116, 73, 0, 5, 119, 105, 100, 116, 104, 120, 112, 0, 0, 0, 45, 0, 0, 5, -6"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	</AuxValues>	Cierra la sección XML abierta anteriormente.
27		Línea en blanco para organizar el XML del formulario.
28	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
29	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
30	</Layout>	Cierra la sección XML abierta anteriormente.
31	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	<Component class="Labels.LabelEscalable" name="labelEscalable1">	Declara un componente o contenedor visual y su clase Java.
33	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
34	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	<Image iconType="3" name="/Imagenes/Logow.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
36	</Property>	Cierra la sección XML abierta anteriormente.

37	<code><Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor"></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
38	<code><Dimension value="[38, 30]"/></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
39	<code></Property></code>	Cierra la sección XML abierta anteriormente.
40	<code></Properties></code>	Cierra la sección XML abierta anteriormente.
41	<code><Constraints></code>	Define la posición y el tamaño del componente dentro del diseño.
42	<code><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"></code>	Define la posición y el tamaño del componente dentro del diseño.
43	<code><AbsoluteConstraints x="17" y="8" width="-1" height="-1"/></code>	Define la posición y el tamaño del componente dentro del diseño.
44	<code></Constraint></code>	Cierra la sección XML abierta anteriormente.
45	<code></Constraints></code>	Cierra la sección XML abierta anteriormente.
46	<code></Component></code>	Cierra la sección XML abierta anteriormente.
47	<code><Component class="javax.swing.JButton" name="botonCerrar"></code>	Declara un componente o contenedor visual y su clase Java.
48	<code><Properties></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
49	<code><Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor"></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
50	<code><Image iconType="3" name="/Imágenes/ICONOS_BARRA_TITULO/icono_cerrar.png"/></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
51	<code></Property></code>	Cierra la sección XML abierta anteriormente.
52	<code><Property name="borderPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
53	<code><Property name="contentAreaFilled" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
54	<code><Property name="focusPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
55	<code></Properties></code>	Cierra la sección XML abierta anteriormente.
56	<code><Events></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
57	<code><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonCerrarActionPerformed"/></code>	Asocia un evento visual con el método Java que lo atenderá.
58	<code></Events></code>	Cierra la sección XML abierta anteriormente.
59	<code><Constraints></code>	Define la posición y el tamaño del componente dentro del diseño.
60	<code><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"></code>	Define la posición y el tamaño del componente dentro del diseño.
61	<code><AbsoluteConstraints x="1456" y="-14" width="-1" height="-1"/></code>	Define la posición y el tamaño del componente dentro del diseño.
62	<code></Constraint></code>	Cierra la sección XML abierta anteriormente.
63	<code></Constraints></code>	Cierra la sección XML abierta anteriormente.
64	<code></Component></code>	Cierra la sección XML abierta anteriormente.
65	<code><Component class="javax.swing.JButton" name="botonMinimizar"></code>	Declara un componente o contenedor visual y su clase Java.
66	<code><Properties></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	<code><Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor"></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	<code><Image iconType="3" name="/Imágenes/ICONOS_BARRA_TITULO/icono_minimizar.png"/></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
69	<code></Property></code>	Cierra la sección XML abierta anteriormente.
70	<code><Property name="borderPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
71	<code><Property name="contentAreaFilled" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
72	<code><Property name="focusPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
73	<code></Properties></code>	Cierra la sección XML abierta anteriormente.
74	<code><Events></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
75	<code><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonMinimizarActionPerformed"/></code>	Asocia un evento visual con el método Java que lo atenderá.
76	<code></Events></code>	Cierra la sección XML abierta anteriormente.
77	<code><Constraints></code>	Define la posición y el tamaño del componente dentro del diseño.

78	<code><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"></code>	Define la posición y el tamaño del componente dentro del diseño.
79	<code><AbsoluteConstraints x="1308" y="-19" width="-1" height="-1"/></code>	Define la posición y el tamaño del componente dentro del diseño.
80	<code></Constraint></code>	Cierra la sección XML abierta anteriormente.
81	<code></Constraints></code>	Cierra la sección XML abierta anteriormente.
82	<code></Component></code>	Cierra la sección XML abierta anteriormente.
83	<code><Component class="javax.swing.JButton" name="botonMaximizar"></code>	Declara un componente o contenedor visual y su clase Java.
84	<code><Properties></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
85	<code><Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor"></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	<code><Image iconType="3" name="/Imágenes/ICONOS_BARRA_TITULO/icono_maximizar.png"/></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
87	<code></Property></code>	Cierra la sección XML abierta anteriormente.
88	<code><Property name="borderPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
89	<code><Property name="contentAreaFilled" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
90	<code><Property name="focusPainted" type="boolean" value="false"/></code>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
91	<code></Properties></code>	Cierra la sección XML abierta anteriormente.
92	<code><Events></code>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
93	<code><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonMaximizarActionPerformed"/></code>	Asocia un evento visual con el método Java que lo atenderá.
94	<code></Events></code>	Cierra la sección XML abierta anteriormente.
95	<code><Constraints></code>	Define la posición y el tamaño del componente dentro del diseño.
96	<code><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"></code>	Define la posición y el tamaño del componente dentro del diseño.
97	<code><AbsoluteConstraints x="1382" y="-14" width="-1" height="-1"/></code>	Define la posición y el tamaño del componente dentro del diseño.
98	<code></Constraint></code>	Cierra la sección XML abierta anteriormente.
99	<code></Constraints></code>	Cierra la sección XML abierta anteriormente.
100	<code></Component></code>	Cierra la sección XML abierta anteriormente.
101	<code></SubComponents></code>	Cierra la sección XML abierta anteriormente.
102	<code></Form></code>	Cierra la sección XML abierta anteriormente.

16.25 src/Componentes/BarraTituloForm.java

Componente visual reutilizable BarraTituloForm. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 467 | Archivo 25 de 157

Clases/tipos detectados: BarraTituloForm, TipolconoVentana, IconoVentana

Línea	Constructor o método detectado
21	public static BarraTituloForm instalar(javax.swing.JFrame ventana,
50	public static BarraTituloForm instalar(javax.swing.JFrame ventana) {
63	public BarraTituloForm(javax.swing.JFrame ventana) {
75	public void setSeccion(String seccion) {
87	private void configurarDisenoAdaptable() {
128	private javax.swing.JLabel etiqueta(String texto, int estilo, int tamano) {
137	private javax.swing.JLabel separador() {
143	private void prepararBotonVisual(javax.swing.JButton boton,
173	public void paintIcon(java.awt.Component componente, java.awt.Graphics graphics,
200	private void configurarBotones() {
269	private void actualizarIconoMaximizar() {
277	private void agregarHover(
288	public void mouseEntered(
299	public void mouseExited(
309	private void configurarMovimiento() {
315	public void mousePressed(
323	public void mouseClicked(
337	public void mouseDragged(
357	private void alternarMaximizado() {
385	public BarraTituloForm() {
397	private void initComponents() {
417	public void actionPerformed(java.awt.event.ActionEvent evt) {
428	public void actionPerformed(java.awt.event.ActionEvent evt) {
439	public void actionPerformed(java.awt.event.ActionEvent evt) {
448	private void botonCerrarActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonCerrarActionPerformed
452	private void botonMinimizarActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonMinimizarActionPerformed
456	private void botonMaximizarActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonMaximizarActionPerformed

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JPanel.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author EMILIANISOMASCOS	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class BarraTituloForm extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12	private static final String BARRA_INSTALADA = "barraTituloForm";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13	private static final java.awt.Color AZUL = new java.awt.Color(1, 20, 36);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
14	private static final java.awt.Color AMARILLO = new java.awt.Color(255, 188, 13);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	private enum TipoIconoVentana {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
17	MINIMIZAR, MAXIMIZAR, RESTAURAR, CERRAR	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18	}	Cierra el bloque abierto anteriormente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	/** Instala una sola barra adaptable encima del contenido de la ventana. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	public static BarraTituloForm instalar(javax.swing.JFrame ventana,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	String seccion) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	Object instalada = ventana.getRootPane().getClientProperty(BARRA_INSTALADA);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
25	if (instalada instanceof BarraTituloForm barraExistente) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
26	barraExistente.setSeccion(seccion);	Configura una propiedad mediante setSeccion en el objeto indicado.
27	return barraExistente;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
28	}	Cierra el bloque abierto anteriormente.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	java.awt.Container contenidoOriginal = ventana.getContentPane();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	javax.swing.JPanel contenedor = new javax.swing.JPanel(	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
33	new java.awt.BorderLayout());	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	contenedor.setBackground(contenidoOriginal.getBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	BarraTituloForm barra = new BarraTituloForm(ventana);	Crea una nueva instancia de BarraTituloForm y la usa o guarda para continuar el proceso.
38	barra.setSeccion(seccion);	Configura una propiedad mediante setSeccion en el objeto indicado.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	ventana.setContentPane(contenedor);	Configura una propiedad mediante setContentPane en el objeto indicado.
41	contenedor.add(barra, java.awt.BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	contenedor.add(contenidoOriginal, java.awt.BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	ventana.getRootPane().putClientProperty(BARRA_INSTALADA, barra);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	contenedor.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	contenedor.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	return barra;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
48	}	Cierra el bloque abierto anteriormente.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	public static BarraTituloForm instalar(javax.swing.JFrame ventana) {	Declara el método instalar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
51	return instalar(ventana, "");	Finaliza el método y devuelve el valor indicado a quien lo llamó.
52	}	Cierra el bloque abierto anteriormente.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	private javax.swing.JFrame ventana;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	private int mouseX;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	private int mouseY;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	private boolean maximizada = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	private javax.swing.JLabel etiquetaMarca;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
61	private javax.swing.JLabel etiquetaSeccion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	public BarraTituloForm(javax.swing.JFrame ventana) {	Constructor de BarraTituloForm: prepara un objeto nuevo y deja sus componentes listos para usarse.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	this.ventana = ventana;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69	configurarDisenoAdaptable();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	configurarBotones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	configurarMovimiento();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	}	Cierra el bloque abierto anteriormente.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	/** Cambia el texto visible que identifica dónde está el usuario. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
75	public void setSeccion(String seccion) {	Declara el método setSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
76	String texto = seccion == null ? "" : seccion.trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
77	etiquetaSeccion.setText(texto);	Cambia el texto visible de un componente de la interfaz.
78	if (ventana != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
79	ventana.setTitle(texto.isEmpty()	Configura una propiedad mediante setTitle en el objeto indicado.
80	? "Waldonald's"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
81	: "Waldonald's - " + texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	}	Cierra el bloque abierto anteriormente.
83	revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	}	Cierra el bloque abierto anteriormente.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	private void configurarDisenoAdaptable() {	Declara el método configurarDisenoAdaptable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

88	<code>removeAll();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	<code>setLayout(new java.awt.BorderLayout());</code>	Crea una nueva instancia de <code>java.awt.BorderLayout</code> y la usa o guarda para continuar el proceso.
90	<code>setOpaque(true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	<code>setBackground(AZUL);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	<code>setPreferredSize(new java.awt.Dimension(1, 48));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
93	<code>setMinimumSize(new java.awt.Dimension(1, 48));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
94	<code>setBorder(javax.swing.BorderFactory.createMatteBorder(0, 0, 3, 0,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	<code>AMARILLO));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	<code>javax.swing.JPanel izquierda = new javax.swing.JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT, 0, 0));</code>	Crea una nueva instancia de <code>javax.swing.JPanel</code> y la usa o guarda para continuar el proceso.
98		Crea una nueva instancia de <code>java.awt.FlowLayout</code> y la usa o guarda para continuar el proceso.
99	<code>izquierda.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
100	<code>izquierda.setBorder(javax.swing.BorderFactory.createEmptyBorder(0, 18, 0, 0));</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	<code>labelEscalable1.setPreferredSize(new java.awt.Dimension(48, 42));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
103	<code>labelEscalable1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);</code>	Configura una propiedad mediante <code>setHorizontalAlignment</code> en el objeto indicado.
104	<code>labelEscalable1.setMantenerProporcion(true);</code>	Configura una propiedad mediante <code>setMantenerProporcion</code> en el objeto indicado.
105	<code>izquierda.add(labelEscalable1);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	<code>etiquetaMarca = etiqueta("Waldonald's", java.awt.Font.BOLD, 19);</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
108	<code>izquierda.add(etiquetaMarca);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	<code>izquierda.add(separador());</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	<code>etiquetaSeccion = etiqueta("", java.awt.Font.PLAIN, 16);</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
112	<code>izquierda.add(etiquetaSeccion);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	<code>javax.swing.JPanel controles = new javax.swing.JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.RIGHT, 0, 0));</code>	Crea una nueva instancia de <code>javax.swing.JPanel</code> y la usa o guarda para continuar el proceso.
115		Crea una nueva instancia de <code>java.awt.FlowLayout</code> y la usa o guarda para continuar el proceso.
116	<code>controles.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
117	<code>prepararBotonVisual(botonMinimizar, TipoIconoVentana.MINIMIZAR, false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	<code>prepararBotonVisual(botonMaximizar, TipoIconoVentana.MAXIMIZAR, false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	<code>prepararBotonVisual(botonCerrar, TipoIconoVentana.CERRAR, true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	<code>controles.add(botonMinimizar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	<code>controles.add(botonMaximizar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	<code>controles.add(botonCerrar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	<code>add(izquierda, java.awt.BorderLayout.WEST);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	<code>add(controles, java.awt.BorderLayout.EAST);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126	<code>}</code>	Cierra el bloque abierto anteriormente.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	<code>private javax.swing.JLabel etiqueta(String texto, int estilo, int tamano) { javax.swing.JLabel etiqueta = new javax.swing.JLabel(texto);</code>	Declara el método <code>etiqueta</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
129		Crea una nueva instancia de <code>javax.swing.JLabel</code> y la usa o guarda para continuar el proceso.
130	<code>etiqueta.setForeground(java.awt.Color.WHITE);</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
131	<code>etiqueta.setFont(new java.awt.Font("Dialog", estilo, tamano));</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.

132	<code>etiqueta.setBorder(javax.swing.BorderFactory.createEmptyBorder(0, 10, 0, 0));</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
133	<code>etiqueta.setVerticalAlignment(javax.swing.SwingConstants.CENTER);</code>	Configura una propiedad mediante <code>setVerticalAlignment</code> en el objeto indicado.
134	<code>return etiqueta;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
135	<code>}</code>	Cierra el bloque abierto anteriormente.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	<code>private javax.swing.JLabel separador() {</code>	Declara el método <code>separador</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
138	<code>javax.swing.JLabel separador = etiqueta("", java.awt.Font.BOLD, 17);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
139	<code>separador.setBorder(javax.swing.BorderFactory.createEmptyBorder(0, 16, 0, 8));</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
140	<code>return separador;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
141	<code>}</code>	Cierra el bloque abierto anteriormente.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	<code>private void prepararBotonVisual(javax.swing.JButton boton,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
144	<code>TipoIconoVentana tipo, boolean cerrar) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	<code>boton.setIcon(new IconoVentana(tipo));</code>	Crea una nueva instancia de <code>IconoVentana</code> y la usa o guarda para continuar el proceso.
146	<code>boton.setText("");</code>	Cambia el texto visible de un componente de la interfaz.
147	<code>boton.setForeground(java.awt.Color.WHITE);</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
148	<code>boton.setPreferredSize(new java.awt.Dimension(52, 45));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
149	<code>boton.setMinimumSize(new java.awt.Dimension(52, 45));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
150	<code>boton.setMaximumSize(new java.awt.Dimension(52, 45));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
151	<code>boton.setBorderPainted(false);</code>	Configura una propiedad mediante <code>setBorderPainted</code> en el objeto indicado.
152	<code>boton.setFocusPainted(false);</code>	Configura una propiedad mediante <code>setFocusPainted</code> en el objeto indicado.
153	<code>boton.setContentAreaFilled(false);</code>	Configura una propiedad mediante <code>setContentAreaFilled</code> en el objeto indicado.
154	<code>boton.setOpaque(true);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
155	<code>boton.setBackground(AZUL);</code>	Configura una propiedad mediante <code>setBackground</code> en el objeto indicado.
156	<code>boton.setCursor(java.awt.Cursor.getPredefinedCursor(</code>	Configura una propiedad mediante <code>setCursor</code> en el objeto indicado.
157	<code>java.awt.Cursor.HAND_CURSOR));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158	<code>}</code>	Cierra el bloque abierto anteriormente.
159		Línea en blanco: separa visualmente bloques para que el código sea más legible.
160	<code>/** Iconos vectoriales para no depender de fuentes ni de imágenes escaladas. */</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
161	<code>private static final class IconoVentana implements javax.swing.Icon {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
162	<code>private final TipoIconoVentana tipo;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	<code>IconoVentana(TipoIconoVentana tipo) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165	<code>this.tipo = tipo;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
166	<code>}</code>	Cierra el bloque abierto anteriormente.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	<code>@Override public int getIconWidth() { return 20; }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	<code>@Override public int getIconHeight() { return 20; }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
173	<code>public void paintIcon(java.awt.Component componente, java.awt.Graphics graphics,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	<code>int x, int y) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	<code>java.awt.Graphics2D g2 = (java.awt.Graphics2D) graphics.create();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

176	<code>g2.setColor(java.awt.Color.WHITE);</code>	Configura una propiedad mediante setColor en el objeto indicado.
177	<code>g2.setStroke(new java.awt.BasicStroke(2.2f,</code>	Crea una nueva instancia de java.awt.BasicStroke y la usa o guarda para continuar el proceso.
178	<code>java.awt.BasicStroke.CAP_ROUND,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	<code>java.awt.BasicStroke.JOIN_ROUND));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
180	<code>int izquierda = x + 3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
181	<code>int derecha = x + 17;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
182	<code>int arriba = y + 3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
183	<code>int abajo = y + 17;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
184	<code>switch (tipo) {</code>	Selecciona una ruta entre varias según el valor evaluado.
185	<code>case MINIMIZAR -> g2.drawLine(izquierda, y + 14, derecha, y + 14);</code>	Define una de las rutas posibles dentro del switch.
186	<code>case MAXIMIZAR -> g2.drawRect(izquierda, arriba, 14, 14);</code>	Define una de las rutas posibles dentro del switch.
187	<code>case RESTAURAR -> {</code>	Define una de las rutas posibles dentro del switch.
188	<code>g2.drawRect(x + 6, y + 3, 11, 11);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189	<code>g2.drawRect(x + 3, y + 6, 11, 11);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	<code>}</code>	Cierra el bloque abierto anteriormente.
191	<code>case CERRAR -> {</code>	Define una de las rutas posibles dentro del switch.
192	<code>g2.drawLine(izquierda, arriba, derecha, abajo);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
193	<code>g2.drawLine(derecha, arriba, izquierda, abajo);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
194	<code>}</code>	Cierra el bloque abierto anteriormente.
195	<code>}</code>	Cierra el bloque abierto anteriormente.
196	<code>g2.dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	<code>}</code>	Cierra el bloque abierto anteriormente.
198	<code>}</code>	Cierra el bloque abierto anteriormente.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	<code>private void configurarBotones() {</code>	Declara el método configurarBotones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
203	<code>// MINIMIZAR</code>	Comentario para humanos; Java no ejecuta este texto. MINIMIZAR
204	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
205	<code>botonMinimizar.addActionListener(e -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	<code>ventana.setState(</code>	Configura una propiedad mediante setState en el objeto indicado.
208	<code>java.awt.Frame.ICONIFIED</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
214	<code>// MAXIMIZAR / RESTAURAR</code>	Comentario para humanos; Java no ejecuta este texto. MAXIMIZAR / RESTAURAR
215	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
216	<code>botonMaximizar.addActionListener(e -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	<code>boolean estaMaximizada</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
219	<code>= (ventana.getExtendedState()</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

220	& java.awt.Frame.MAXIMIZED_BOTH)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
221	== java.awt.Frame.MAXIMIZED_BOTH;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	if (estaMaximizada) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	ventana.setExtendedState(	Configura una propiedad mediante setExtendedState en el objeto indicado.
226	java.awt.Frame.NORMAL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
228		Línea en blanco: separa visualmente bloques para que el código sea más legible.
229	actualizarIconoMaximizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	ventana.setExtendedState(	Configura una propiedad mediante setExtendedState en el objeto indicado.
234	java.awt.Frame.MAXIMIZED_BOTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
235	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	actualizarIconoMaximizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238	}	Cierra el bloque abierto anteriormente.
239	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240		Línea en blanco: separa visualmente bloques para que el código sea más legible.
241	// =====	Comentario para humanos; Java no ejecuta este texto. =====
242	// CERRAR	Comentario para humanos; Java no ejecuta este texto. CERRAR
243	// =====	Comentario para humanos; Java no ejecuta este texto. =====
244	botonCerrar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	ventana.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247		Línea en blanco: separa visualmente bloques para que el código sea más legible.
248	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	// =====	Comentario para humanos; Java no ejecuta este texto. =====
251	// HOVER	Comentario para humanos; Java no ejecuta este texto. HOVER
252	// =====	Comentario para humanos; Java no ejecuta este texto. =====
253	agregarHover(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	botonMinimizar,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
255	new java.awt.Color(35, 45, 60)	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
256	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
257		Línea en blanco: separa visualmente bloques para que el código sea más legible.
258	agregarHover(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	botonMaximizar,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	new java.awt.Color(35, 45, 60)	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
261	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
262		Línea en blanco: separa visualmente bloques para que el código sea más legible.
263	agregarHover(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

264	botonCerrar,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	new java.awt.Color(200, 0, 10)	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
266	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
267	}	Cierra el bloque abierto anteriormente.
268		Línea en blanco: separa visualmente bloques para que el código sea más legible.
269	private void actualizarIconoMaximizar() {	Declara el método actualizarIconoMaximizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
270	boolean estaMaximizada = ventana != null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
271	&& (ventana.getExtendedState() & java.awt.Frame.MAXIMIZED_BOTH)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272	== java.awt.Frame.MAXIMIZED_BOTH;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
273	botonMaximizar.setIcon(new IconoVentana(estaMaximizada	Crea una nueva instancia de IconoVentana y la usa o guarda para continuar el proceso.
274	? TipoIconoVentana.RESTAURAR : TipoIconoVentana.MAXIMIZAR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
275	}	Cierra el bloque abierto anteriormente.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	private void agregarHover(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
278	javax.swing.JButton boton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
279	java.awt.Color colorHover) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281	java.awt.Color colorNormal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	= new java.awt.Color(1, 15, 30);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
283		Línea en blanco: separa visualmente bloques para que el código sea más legible.
284	boton.addMouseListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
285	new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
288	public void mouseEntered(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
289	java.awt.event.MouseEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290		Línea en blanco: separa visualmente bloques para que el código sea más legible.
291	boton.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	boton.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
294	colorHover	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
295	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
296	}	Cierra el bloque abierto anteriormente.
297		Línea en blanco: separa visualmente bloques para que el código sea más legible.
298	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
299	public void mouseExited(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
300	java.awt.event.MouseEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
301		Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	boton.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
303	colorNormal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
304	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
305	}	Cierra el bloque abierto anteriormente.
306	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
307	}	Cierra el bloque abierto anteriormente.

308		Línea en blanco: separa visualmente bloques para que el código sea más legible.
309	private void configurarMovimiento() {	Declara el método configurarMovimiento. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
311	this.addMouseListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
312	new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
315	public void mousePressed(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	java.awt.event.MouseEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318	mouseX = evt.getX();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
319	mouseY = evt.getY();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
320	}	Cierra el bloque abierto anteriormente.
321		Línea en blanco: separa visualmente bloques para que el código sea más legible.
322	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
323	public void mouseClicked(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
324	java.awt.event.MouseEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
325		Línea en blanco: separa visualmente bloques para que el código sea más legible.
326	if (evt.getClickCount() == 2) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
327		Línea en blanco: separa visualmente bloques para que el código sea más legible.
328	alternarMaximizado();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
329	}	Cierra el bloque abierto anteriormente.
330	}	Cierra el bloque abierto anteriormente.
331	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332		Línea en blanco: separa visualmente bloques para que el código sea más legible.
333	this.addMouseMotionListener(	Accede al atributo del objeto actual para asignarlo o utilizarlo.
334	new java.awt.event.MouseMotionAdapter() {	Crea una nueva instancia de java.awt.event.MouseMotionAdapter y la usa o guarda para continuar el proceso.
335		Línea en blanco: separa visualmente bloques para que el código sea más legible.
336	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
337	public void mouseDragged(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338	java.awt.event.MouseEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339		Línea en blanco: separa visualmente bloques para que el código sea más legible.
340	boolean estaMaximizada	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341	= (ventana.getExtendedState()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
342	& java.awt.Frame.MAXIMIZED_BOTH)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
343	== java.awt.Frame.MAXIMIZED_BOTH;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
345	if (estaMaximizada) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
346	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
347	}	Cierra el bloque abierto anteriormente.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349	ventana.setLocation(	Configura una propiedad mediante setLocation en el objeto indicado.
350	evt.getXOnScreen() - mouseX,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
351	evt.getYOnScreen() - mouseY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

352	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
353	}	Cierra el bloque abierto anteriormente.
354	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
355	}	Cierra el bloque abierto anteriormente.
356		Línea en blanco: separa visualmente bloques para que el código sea más legible.
357	private void alternarMaximizado() {	Declara el método alternarMaximizado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
358		Línea en blanco: separa visualmente bloques para que el código sea más legible.
359	boolean estaMaximizada	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
360	= (ventana.getExtendedState()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
361	& java.awt.Frame.MAXIMIZED_BOTH)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
362	== java.awt.Frame.MAXIMIZED_BOTH;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
363		Línea en blanco: separa visualmente bloques para que el código sea más legible.
364	if (estaMaximizada) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
365		Línea en blanco: separa visualmente bloques para que el código sea más legible.
366	ventana.setExtendedState(	Configura una propiedad mediante setExtendedState en el objeto indicado.
367	java.awt.Frame.NORMAL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
368	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
370	actualizarIconoMaximizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
371		Línea en blanco: separa visualmente bloques para que el código sea más legible.
372	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
373		Línea en blanco: separa visualmente bloques para que el código sea más legible.
374	ventana.setExtendedState(	Configura una propiedad mediante setExtendedState en el objeto indicado.
375	java.awt.Frame.MAXIMIZED_BOTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
376	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377		Línea en blanco: separa visualmente bloques para que el código sea más legible.
378	actualizarIconoMaximizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
379	}	Cierra el bloque abierto anteriormente.
380	}	Cierra el bloque abierto anteriormente.
381		Línea en blanco: separa visualmente bloques para que el código sea más legible.
382	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
383	* Creates new form BarraTituloForm	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
384	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
385	public BarraTituloForm() {	Constructor de BarraTituloForm: prepara un objeto nuevo y deja sus componentes listos para usarse.
386	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
387	configurarDisenoAdaptable();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
388	}	Cierra el bloque abierto anteriormente.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
391	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
392	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
393	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
394	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
395	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

396	// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
397	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
398		Línea en blanco: separa visualmente bloques para que el código sea más legible.
399	labelEscalable1 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
400	botonCerrar = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
401	botonMinimizar = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
402	botonMaximizar = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	setBackground(new java.awt.Color(1, 15, 30));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
405	setPreferredSize(new java.awt.Dimension(1530, 45));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
406	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
407		Línea en blanco: separa visualmente bloques para que el código sea más legible.
408	labelEscalable1.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/Logow.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
409	labelEscalable1.setPreferredSize(new java.awt.Dimension(38, 30));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
410	add(labelEscalable1, new org.netbeans.lib.awtextra.AbsoluteConstraints(17, 8, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
411		Línea en blanco: separa visualmente bloques para que el código sea más legible.
412	botonCerrar.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/ICONOS_BARRA_TITULO/icono_cerrar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
413	botonCerrar.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
414	botonCerrar.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
415	botonCerrar.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
416	botonCerrar.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
417	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
418	botonCerrarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
419	}	Cierra el bloque abierto anteriormente.
420	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
421	add(botonCerrar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1456, -14, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
422		Línea en blanco: separa visualmente bloques para que el código sea más legible.
423	botonMinimizar.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/ICONOS_BARRA_TITULO/icono_minimizar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
424	botonMinimizar.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
425	botonMinimizar.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
426	botonMinimizar.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
427	botonMinimizar.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
428	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
429	botonMinimizarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430	}	Cierra el bloque abierto anteriormente.
431	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
432	add(botonMinimizar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1308, -19, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
433		Línea en blanco: separa visualmente bloques para que el código sea más legible.
434	botonMaximizar.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/ICONOS_BARRA_TITULO/icono_maximizar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
435	botonMaximizar.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
436	botonMaximizar.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.

437	botonMaximizar.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
438	botonMaximizar.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
439	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
440	botonMaximizarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
441	}	Cierra el bloque abierto anteriormente.
442	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
443	add(botonMaximizar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1382, -14, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
444		Línea en blanco: separa visualmente bloques para que el código sea más legible.
445	getAccessibleContext().setAccessibleName("");	Configura una propiedad mediante setAccessibleName en el objeto indicado.
446	} // </editor-fold> // GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447		Línea en blanco: separa visualmente bloques para que el código sea más legible.
448	private void botonCerrarActionPerformed(java.awt.event.ActionEvent evt) { // GEN-FIRST:event_botonCerrarAction	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
449	Performed	
449	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
450	} // GEN-LAST:event_botonCerrarActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
451		Línea en blanco: separa visualmente bloques para que el código sea más legible.
452	private void botonMinimizarActionPerformed(java.awt.event.ActionEvent evt) { // GEN-FIRST:event_botonMinimizar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	ActionPerformed	
453	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
454	} // GEN-LAST:event_botonMinimizarActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	private void botonMaximizarActionPerformed(java.awt.event.ActionEvent evt) { // GEN-FIRST:event_botonMaximizar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
457	ActionPerformed	
457	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
458	} // GEN-LAST:event_botonMaximizarActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459		Línea en blanco: separa visualmente bloques para que el código sea más legible.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	// Variables declaration - do not modify // GEN-BEGIN: variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
462	private javax.swing.JButton botonCerrar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
463	private javax.swing.JButton botonMaximizar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
464	private javax.swing.JButton botonMinimizar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
465	private javax.swing.JLabel labelEscalable1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
466	// End of variables declaration // GEN-END: variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
467	}	Cierra el bloque abierto anteriormente.

16.26 src/Componentes/Boton.java

Componente visual reutilizable Boton. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 10 | Archivo 26 de 157

Clases/tipos detectados: Boton

Línea	Constructor o método detectado	
7	public Boton() {	
package Componentes;		
1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public class Boton extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	public Boton() {	Constructor de Boton: prepara un objeto nuevo y deja sus componentes listos para usarse.
8	super();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
9	}	Cierra el bloque abierto anteriormente.
10	}	Cierra el bloque abierto anteriormente.

16.27 src/Componentes/BotonCategoria.java

Componente visual reutilizable BotonCategoria. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 240 | Archivo 27 de 157

Clases/tipos detectados: BotonCategoria

Línea	Constructor o método detectado
35	public BotonCategoria() {
75	public void mouseEntered(MouseEvent e) {
81	public void mouseExited(MouseEvent e) {
98	private void seleccionarEsteBoton() {
133	protected void paintComponent(Graphics g) {
219	public void setSeleccionado(boolean seleccionado) {
224	public boolean isSeleccionado() {
232	public void setRadio(int radio) {
237	public int getRadio() {

1 package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
7 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
8 import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
9 import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
10 import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
11 import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
12 import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
13 import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
14	Línea en blanco: separa visualmente bloques para que el código sea más legible.
15 import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
16 import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
17	Línea en blanco: separa visualmente bloques para que el código sea más legible.
18 public class BotonCategoria extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19	Línea en blanco: separa visualmente bloques para que el código sea más legible.
20 // Indica si el mouse está encima	Comentario para humanos; Java no ejecuta este texto. Indica si el mouse está encima
21 private boolean mouseEncima = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	Línea en blanco: separa visualmente bloques para que el código sea más legible.
23 // Indica si esta categoría está seleccionada	Comentario para humanos; Java no ejecuta este texto. Indica si esta categoría está seleccionada
24 private boolean seleccionado = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	// Colores	Comentario para humanos; Java no ejecuta este texto. Colores
27	private final Color colorNormal = new Color(255, 255, 255);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
28	private final Color colorHover = new Color(255, 249, 230);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
29	private final Color amarillo = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
30	private final Color bordeNormal = new Color(235, 235, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	// Redondeado	Comentario para humanos; Java no ejecuta este texto. Redondeado
33	private int radio = 22;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	public BotonCategoria() {	Constructor de BotonCategoria: prepara un objeto nuevo y deja sus componentes listos para usarse.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	// Texto inicial	Comentario para humanos; Java no ejecuta este texto. Texto inicial
38	setText("Categoría");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	// Tamaño recomendado	Comentario para humanos; Java no ejecuta este texto. Tamaño recomendado
41	setPreferredSize(new Dimension(105, 105));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	// Fuente	Comentario para humanos; Java no ejecuta este texto. Fuente
44	setFont(new Font("Arial", Font.BOLD, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	// Color del texto	Comentario para humanos; Java no ejecuta este texto. Color del texto
47	setForeground(new Color(25, 25, 25));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	// Imagen arriba y texto abajo	Comentario para humanos; Java no ejecuta este texto. Imagen arriba y texto abajo
50	setHorizontalAlignment(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51	setVerticalAlignment(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	setHorizontalTextPosition(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	setVerticalTextPosition(SwingConstants.BOTTOM);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	// Separación entre imagen y texto	Comentario para humanos; Java no ejecuta este texto. Separación entre imagen y texto
57	setIconTextGap(4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	// Quitar diseño normal del JButton	Comentario para humanos; Java no ejecuta este texto. Quitar diseño normal del JButton
60	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	// Cursor de mano	Comentario para humanos; Java no ejecuta este texto. Cursor de mano
66	setCursor(new Cursor(Cursor.HAND_CURSOR));	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	// =====	Comentario para humanos; Java no ejecuta este texto. =====

69	// HOVER	Comentario para humanos; Java no ejecuta este texto. HOVER
70	// =====	Comentario para humanos; Java no ejecuta este texto. =====
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
75	public void mouseEntered(MouseEvent e) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
76	mouseEncima = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
77	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	}	Cierra el bloque abierto anteriormente.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
81	public void mouseExited(MouseEvent e) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
82	mouseEncima = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
83	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	// =====	Comentario para humanos; Java no ejecuta este texto. =====
88	// CLIC / SELECCIÓN EXCLUSIVA	Comentario para humanos; Java no ejecuta este texto. CLIC / SELECCIÓN EXCLUSIVA
89	// =====	Comentario para humanos; Java no ejecuta este texto. =====
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	addActionListener(e -> seleccionarEsteBoton());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	// =====	Comentario para humanos; Java no ejecuta este texto. =====
95	// SELECCIONAR ESTE BOTÓN	Comentario para humanos; Java no ejecuta este texto. SELECCIONAR ESTE BOTÓN
96	// =====	Comentario para humanos; Java no ejecuta este texto. =====
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	private void seleccionarEsteBoton() {	Declara el método seleccionarEsteBoton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
101	* Busca todos los componentes que están dentro	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
102	* del mismo JPanel o contenedor de este botón.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
103	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
104	if (getParent() != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	Component[] componentes = getParent().getComponents();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	for (Component componente : componentes) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
109		Línea en blanco: separa visualmente bloques para que el código sea más legible.
110	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
111	* Si encuentra otro BotonCategoria,	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
112	* lo deselectiona.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

113	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
114	if (componente instanceof BotonCategoria) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
116	BotonCategoria botonCategoria =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
117	(BotonCategoria) componente;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	botonCategoria.setSelected(false);	Configura una propiedad mediante setSelected en el objeto indicado.
120	}	Cierra el bloque abierto anteriormente.
121	}	Cierra el bloque abierto anteriormente.
122	}	Cierra el bloque abierto anteriormente.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	// Este botón queda seleccionado	Comentario para humanos; Java no ejecuta este texto. Este botón queda seleccionado
125	setSelected(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126	}	Cierra el bloque abierto anteriormente.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	// =====	Comentario para humanos; Java no ejecuta este texto. =====
129	// DIBUJAR BOTÓN	Comentario para humanos; Java no ejecuta este texto. DIBUJAR BOTÓN
130	// =====	Comentario para humanos; Java no ejecuta este texto. =====
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
133	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	// Mejor calidad gráfica	Comentario para humanos; Java no ejecuta este texto. Mejor calidad gráfica
138	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
139	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
141	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	int ancho = getWidth();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
144	int alto = getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	// =====	Comentario para humanos; Java no ejecuta este texto. =====
147	// FONDO	Comentario para humanos; Java no ejecuta este texto. FONDO
148	// =====	Comentario para humanos; Java no ejecuta este texto. =====
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
151	* Si el mouse está encima y el botón	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
152	* NO está seleccionado:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
153	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
154	* fondo crema.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
155	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
156	* Si ya está seleccionado:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

157	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
158	* fondo blanco.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
159	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
160	if (mouseEncima && !seleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	g2.setColor(colorHover);	Configura una propiedad mediante setColor en el objeto indicado.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	g2.setColor(colorNormal);	Configura una propiedad mediante setColor en el objeto indicado.
167	}	Cierra el bloque abierto anteriormente.
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	ancho - 2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	alto - 2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	// =====	Comentario para humanos; Java no ejecuta este texto. =====
179	// BORDE	Comentario para humanos; Java no ejecuta este texto. BORDE
180	// =====	Comentario para humanos; Java no ejecuta este texto. =====
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
183	* Hover o seleccionado:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
184	* borde amarillo de 2 px.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
185	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
186	* Normal:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
187	* borde gris de 1 px.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
188	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
189	if (mouseEncima seleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
191	g2.setColor(amarillo);	Configura una propiedad mediante setColor en el objeto indicado.
192	g2.setStroke(new BasicStroke(2f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
194	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196	g2.setColor(bordeNormal);	Configura una propiedad mediante setColor en el objeto indicado.
197	g2.setStroke(new BasicStroke(1f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
198	}	Cierra el bloque abierto anteriormente.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

201	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
202	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
203	ancho - 3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	alto - 3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
206	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
207	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
208		Línea en blanco: separa visualmente bloques para que el código sea más legible.
209	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211	// Dibujar texto e imagen del JButton	Comentario para humanos; Java no ejecuta este texto. Dibujar texto e imagen del JButton
212	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
213	}	Cierra el bloque abierto anteriormente.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	// =====	Comentario para humanos; Java no ejecuta este texto. =====
216	// SELECCIONAR / DESELECCIONAR	Comentario para humanos; Java no ejecuta este texto. SELECCIONAR / DESELECCIONAR
217	// =====	Comentario para humanos; Java no ejecuta este texto. =====
218		Línea en blanco: separa visualmente bloques para que el código sea más legible.
219	public void setSeleccionado(boolean seleccionado) {	Declara el método setSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
220	this.seleccionado = seleccionado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
221	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
222	}	Cierra el bloque abierto anteriormente.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	public boolean isSeleccionado() {	Declara el método isSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
225	return seleccionado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
226	}	Cierra el bloque abierto anteriormente.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	// =====	Comentario para humanos; Java no ejecuta este texto. =====
229	// CAMBIAR RADIO	Comentario para humanos; Java no ejecuta este texto. CAMBIAR RADIO
230	// =====	Comentario para humanos; Java no ejecuta este texto. =====
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
233	this.radio = radio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
234	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
235	}	Cierra el bloque abierto anteriormente.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	public int getRadio() {	Declara el método getRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
238	return radio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
239	}	Cierra el bloque abierto anteriormente.
240	}	Cierra el bloque abierto anteriormente.

16.28 src/Componentes/BotonDerretido.java

Componente visual reutilizable BotonDerretido. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 258 | Archivo 28 de 157

Clases/tipos detectados: BotonDerretido

Línea	Constructor o método detectado
24	public Color getColorBoton() {
28	public void setColorBoton(Color colorBoton) {
45	public BotonDerretido() {
96	public void mouseEntered(MouseEvent e) {
105	public void mouseExited(MouseEvent e) {
116	protected void paintComponent(Graphics g) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.geom.Path2D;	Importa java.awt.geom.Path2D para poder usar esa clase sin escribir su nombre completo.
13	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
16	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	public class BotonDerretido extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	// Color amarillo del botón	Comentario para humanos; Java no ejecuta este texto. Color amarillo del botón
21	private Color colorBoton = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	@BeanProperty(preferred = true, description = "Color de fondo del botón derretido.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
24	public Color getColorBoton() {	Declara el método getColorBoton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
25	return colorBoton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
26	}	Cierra el bloque abierto anteriormente.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	public void setColorBoton(Color colorBoton) {	Declara el método setColorBoton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

29	Color anterior = this.colorBoton;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	this.colorBoton = colorBoton != null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
31	? colorBoton	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	: new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
33	firePropertyChange("colorBoton", anterior, this.colorBoton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	}	Cierra el bloque abierto anteriormente.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	// Cuánto se ha derretido	Comentario para humanos; Java no ejecuta este texto. Cuánto se ha derretido
38	private float derretido = 0f;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	// Indica si el mouse está encima	Comentario para humanos; Java no ejecuta este texto. Indica si el mouse está encima
41	private boolean mouseEncima = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	private Timer timer;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	public BotonDerretido() {	Constructor de BotonDerretido: prepara un objeto nuevo y deja sus componentes listos para usarse.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	setText("BOTÓN");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	setForeground(Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	setFont(new Font("Arial", Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	setCursor(new Cursor(Cursor.HAND_CURSOR));	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
58	// Espacio extra abajo para que puedan aparecer las gotas	Comentario para humanos; Java no ejecuta este texto. Espacio extra abajo para que puedan aparecer las gotas
59	setBorder(new EmptyBorder(5, 15, 18, 15));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	setPreferredSize(new Dimension(180, 60));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	// Animación	Comentario para humanos; Java no ejecuta este texto. Animación
64	timer = new Timer(15, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	if (mouseEncima) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	// Derretir	Comentario para humanos; Java no ejecuta este texto. Derretir
69	derretido += 0.06f;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	if (derretido >= 1f) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
72	derretido = 1f;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

73	}	Cierra el bloque abierto anteriormente.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	// Regresar a la normalidad	Comentario para humanos; Java no ejecuta este texto. Regresar a la normalidad
78	derretido -= 0.08f;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	if (derretido <= 0f) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
81	derretido = 0f;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
82	}	Cierra el bloque abierto anteriormente.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	// Si terminó la animación se detiene	Comentario para humanos; Java no ejecuta este texto. Si terminó la animación se detiene
88	if (!mouseEncima && derretido <= 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
89	timer.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	}	Cierra el bloque abierto anteriormente.
91	}};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
96	public void mouseEntered(MouseEvent e) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
97	mouseEncima = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	if (!timer.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
100	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	}	Cierra el bloque abierto anteriormente.
102	}	Cierra el bloque abierto anteriormente.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
105	public void mouseExited(MouseEvent e) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
106	mouseEncima = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	if (!timer.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
109	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	}	Cierra el bloque abierto anteriormente.
111	}	Cierra el bloque abierto anteriormente.
112	}};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113	}	Cierra el bloque abierto anteriormente.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
116	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
121	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
122	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
123	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	int ancho = getWidth();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
126	int alto = getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	// Altura principal del botón	Comentario para humanos; Java no ejecuta este texto. Altura principal del botón
129	int baseY = alto - 18;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	// Cuánto bajan las gotas	Comentario para humanos; Java no ejecuta este texto. Cuánto bajan las gotas
132	float gota1 = 14 * derretido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
133	float gota2 = 8 * derretido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
134	float gota3 = 17 * derretido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
135	float gota4 = 10 * derretido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	Path2D.Float forma = new Path2D.Float();	Crea una nueva instancia de Path2D.Float y la usa o guarda para continuar el proceso.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	// Empezamos arriba a la izquierda	Comentario para humanos; Java no ejecuta este texto. Empezamos arriba a la izquierda
140	forma.moveTo(15, 0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141		Línea en blanco: separa visualmente bloques para que el código sea más legible.
142	// Parte superior	Comentario para humanos; Java no ejecuta este texto. Parte superior
143	forma.quadTo(0, 0, 0, 15);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	// Lado izquierdo	Comentario para humanos; Java no ejecuta este texto. Lado izquierdo
146	forma.lineTo(0, baseY - 10);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.
148	forma.quadTo(0, baseY, 10, baseY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	// PARTE DERRETIDA	Comentario para humanos; Java no ejecuta este texto. PARTE DERRETIDA
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	// Primera curva	Comentario para humanos; Java no ejecuta este texto. Primera curva
154	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	ancho * 0.10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
156	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	ancho * 0.13,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
158	baseY + gota1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
159	ancho * 0.18,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
160	baseY + gota1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

161	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
164	ancho * 0.23,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165	baseY + gota1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	ancho * 0.23,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
168	ancho * 0.30,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
169	baseY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	// Segunda gota	Comentario para humanos; Java no ejecuta este texto. Segunda gota
173	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	ancho * 0.36,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	ancho * 0.37,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	baseY + gota2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	ancho * 0.42,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	baseY + gota2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
183	ancho * 0.47,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	baseY + gota2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
185	ancho * 0.48,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	ancho * 0.54,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	baseY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
191	// Tercera gota, más larga	Comentario para humanos; Java no ejecuta este texto. Tercera gota, más larga
192	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	ancho * 0.60,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	ancho * 0.61,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	baseY + gota3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197	ancho * 0.66,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198	baseY + gota3	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
202	ancho * 0.71,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
203	baseY + gota3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	ancho * 0.72,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

205	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
206	ancho * 0.78,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
207	baseY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
208	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	// Cuarta gota	Comentario para humanos; Java no ejecuta este texto. Cuarta gota
211	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212	ancho * 0.84,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	ancho * 0.85,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	baseY + gota4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	ancho * 0.90,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	baseY + gota4	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	forma.curveTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
221	ancho * 0.95,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222	baseY + gota4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	ancho * 0.95,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	ancho - 10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	baseY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
228		Línea en blanco: separa visualmente bloques para que el código sea más legible.
229	// Esquina derecha	Comentario para humanos; Java no ejecuta este texto. Esquina derecha
230	forma.quadTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
231	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
232	baseY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
233	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
234	baseY - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
235	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	forma.lineTo(ancho, 15);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	forma.quadTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
241	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
242	ancho - 15,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
243	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	forma.closePath();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247		Línea en blanco: separa visualmente bloques para que el código sea más legible.
248	// Pintar botón	Comentario para humanos; Java no ejecuta este texto. Pintar botón

249	<code>g2.setColor(colorBoton);</code>	Configura una propiedad mediante setColor en el objeto indicado.
250	<code>g2.fill(forma);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
251		Línea en blanco: separa visualmente bloques para que el código sea más legible.
252	<code>g2.dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
253		Línea en blanco: separa visualmente bloques para que el código sea más legible.
254	<code>// Pintar texto</code>	Comentario para humanos; Java no ejecuta este texto. Pintar texto
255	<code>super.paintComponent(g);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
256	<code>}</code>	Cierra el bloque abierto anteriormente.
257	<code>}</code>	Cierra el bloque abierto anteriormente.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.29 src/Componentes/BotonDesplegable.java

Componente visual reutilizable BotonDesplegable. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 462 | Archivo 29 de 157

Clases/tipos detectados: BotonDesplegable

Línea	Constructor o método detectado
58	public BotonDesplegable() {
74	public void popupMenuWillBecomeVisible(PopupMenuEvent evento) {
79	public void popupMenuWillBecomeInvisible(PopupMenuEvent evento) {
84	public void popupMenuCanceled(PopupMenuEvent evento) {
92	public void mouseEntered(MouseEvent evento) {
98	public void mouseExited(MouseEvent evento) {
108	public String getTextoDesplegable() {
112	public void setTextoDesplegable(String textoDesplegable) {
120	public Color getColorFondo() {
124	public void setColorFondo(Color colorFondo) {
135	public Color getColorHover() {
139	public void setColorHover(Color colorHover) {
147	public Color getColorDesplegado() {
151	public void setColorDesplegado(Color colorDesplegado) {
159	public Color getColorFondoMenu() {
163	public void setColorFondoMenu(Color colorFondoMenu) {
172	public Color getColorTextoOpcion() {
176	public void setColorTextoOpcion(Color colorTextoOpcion) {
184	public Color getColorHoverOpcion() {
188	public void setColorHoverOpcion(Color colorHoverOpcion) {
196	public Color getColorTextoHoverOpcion() {
200	public void setColorTextoHoverOpcion(Color colorTextoHoverOpcion) {
209	public Color getColorBordeMenu() {
213	public void setColorBordeMenu(Color colorBordeMenu) {
221	public int getRadio() {
225	public void setRadio(int radio) {
234	public int getAnchoMenu() {
238	public void setAnchoMenu(int anchoMenu) {
246	public int getAltoOpcion() {

Línea	Constructor o método detectado
250	public void setAltoOpcion(int altoOpcion) {
258	public int getSeparacionMenu() {
262	public void setSeparacionMenu(int separacionMenu) {
269	public String getOpcionSeleccionada() {
274	public int getIndiceOpcionSeleccionada() {
279	public void setFont(Font fuente) {
284	public void addMenuOpcionListener(ActionListener escucha) {
288	public void removeMenuOpcionListener(ActionListener escucha) {
292	private void configurarAspectoMenu() {
304	private void reconstruirOpciones() {
326	private JMenuItem crearOpcion(String textoOpcion, int indice) {
345	private void seleccionarOpcion(String opcion, int indice) {
365	private void alternarMenu() {
377	private void setDesplegado(boolean desplegado) {
383	private void iniciarAnimacion() {
389	private void actualizarAnimacion() {
402	protected void paintComponent(Graphics graphics) {
416	private void pintarFlecha(Graphics graphics) {
433	private static void aplicarCalidad(Graphics2D g2) {
442	private static Color colorSeguro(Color color) {
446	private static Color mezclar(Color origen, Color destino, float proporcion) {
456	private static boolean coloresCercanos(Color primero, Color segundo) {

1 package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7 import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
8 import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
9 import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
10 import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
11 import java.awt.event.ActionEvent;	Importa java.awt.event.ActionEvent para poder usar esa clase sin escribir su nombre completo.
12 import java.awt.event.ActionListener;	Importa java.awt.event.ActionListener para poder usar esa clase sin escribir su nombre completo.
13 import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.

14	<code>import java.awt.event.MouseEvent;</code>	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
15	<code>import java.beans.BeanProperty;</code>	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
16	<code>import javax.swing.BorderFactory;</code>	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
17	<code>import javax.swing.JButton;</code>	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
18	<code>import javax.swing.JMenuItem;</code>	Importa javax.swing.JMenuItem para poder usar esa clase sin escribir su nombre completo.
19	<code>import javax.swing.JPopupMenu;</code>	Importa javax.swing.JPopupMenu para poder usar esa clase sin escribir su nombre completo.
20	<code>import javax.swing.Timer;</code>	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
21	<code>import javax.swing.border.EmptyBorder;</code>	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
22	<code>import javax.swing.border.LineBorder;</code>	Importa javax.swing.border.LineBorder para poder usar esa clase sin escribir su nombre completo.
23	<code>import javax.swing.event.EventListenerList;</code>	Importa javax.swing.event.EventListenerList para poder usar esa clase sin escribir su nombre completo.
24	<code>import javax.swing.event.PopupMenuEvent;</code>	Importa javax.swing.event.PopupMenuEvent para poder usar esa clase sin escribir su nombre completo.
25	<code>import javax.swing.event.PopupMenuListener;</code>	Importa javax.swing.event.PopupMenuListener para poder usar esa clase sin escribir su nombre completo.
26	<code>import javax.swing.plaf.basic.BasicMenuItemUI;</code>	Importa javax.swing.plaf.basic.BasicMenuItemUI para poder usar esa clase sin escribir su nombre completo.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	<code>/**</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
29	<code> * Botón desplegable reutilizable y editable desde Properties de NetBeans.</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
30	<code> * Las opciones se escriben separadas por punto y coma.</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
31	<code> */</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
32	<code>@SuppressWarnings({"serial", "this-escape"})</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
33	<code>public class BotonDesplegable extends JButton {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	<code> private String textoDesplegable = "Mi perfil;Cerrar sesión";</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36	<code> private Color colorFondo = new Color(0, 0, 0, 0);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
37	<code> private Color colorHover = new Color(255, 255, 255, 28);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
38	<code> private Color colorDesplegado = new Color(255, 255, 255, 42);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
39	<code> private Color colorFondoMenu = Color.WHITE;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
40	<code> private Color colorTextoOpcion = new Color(24, 31, 42);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
41	<code> private Color colorHoverOpcion = new Color(255, 244, 210);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
42	<code> private Color colorTextoHoverOpcion = new Color(13, 17, 23);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
43	<code> private Color colorBordeMenu = new Color(226, 231, 238);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
44	<code> private int radio = 18;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	<code> private int anchoMenu = 230;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
46	<code> private int altoOpcion = 48;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	<code> private int separacionMenu = 6;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
48	<code> private boolean mouseEncima;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
49	<code> private boolean desplegado;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
50	<code> private Color colorActual = colorFondo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
51	<code> private String opcionSeleccionada = "";</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52	<code> private int indiceOpcionSeleccionada = -1;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	<code> private final Timer animacion;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
55	<code> private final JPopupMenu menuDesplegable;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
56	<code> private final EventListenerList escuchasOpciones = new EventListenerList();</code>	Crea una nueva instancia de EventListenerList y la usa o guarda para continuar el proceso.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.

58	public BotonDesplegable() {	Constructor de BotonDesplegable: prepara un objeto nuevo y deja sus componentes listos para usarse.
59	setText("Administrador");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	setForeground(Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	setBorder(new EmptyBorder(0, 18, 0, 42));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
62	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	setHorizontalAlignment(LEFT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69	menuDesplegable = new JPopupMenu();	Crea una nueva instancia de JPopupMenu y la usa o guarda para continuar el proceso.
70	configurarAspectoMenu();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	menuDesplegable.addPopupMenuListener(new PopupMenuListener() {	Crea una nueva instancia de PopupMenuListener y la usa o guarda para continuar el proceso.
73	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
74	public void popupMenuWillBecomeVisible(PopupMenuEvent evento) {	Declara el método popupMenuWillBecomeVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
75	setDesplegado(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76	}	Cierra el bloque abierto anteriormente.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
79	public void popupMenuWillBecomeInvisible(PopupMenuEvent evento) {	Declara el método popupMenuWillBecomeInvisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
80	setDesplegado(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	}	Cierra el bloque abierto anteriormente.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
84	public void popupMenuCanceled(PopupMenuEvent evento) {	Declara el método popupMenuCanceled. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
85	setDesplegado(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	}	Cierra el bloque abierto anteriormente.
87	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	animacion = new Timer(15, evento -> actualizarAnimacion());	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
90	addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
91	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
92	public void mouseEntered(MouseEvent evento) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
93	mouseEncima = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
94	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	}	Cierra el bloque abierto anteriormente.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
98	public void mouseExited(MouseEvent evento) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
99	mouseEncima = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

100	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	}	Cierra el bloque abierto anteriormente.
102	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	addActionListener(evento -> alternarMenu());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	}	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	@BeanProperty(preferred = true,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
107	description = "Opciones del menú separadas por punto y coma. Usa - para insertar un separador.")	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
108	public String getTextoDesplegable() {	Declara el método getTextoDesplegable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
109	return textoDesplegable;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
110	}	Cierra el bloque abierto anteriormente.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	public void setTextoDesplegable(String textoDesplegable) {	Declara el método setTextoDesplegable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
113	String anterior = this.textoDesplegable;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	this.textoDesplegable = textoDesplegable != null ? textoDesplegable : "";	Accede al atributo del objeto actual para asignarlo o utilizarlo.
115	firePropertyChange("textoDesplegable", anterior, this.textoDesplegable);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	}	Cierra el bloque abierto anteriormente.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	@BeanProperty(preferred = true, description = "Color normal del botón.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
120	public Color getColorFondo() {	Declara el método getColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
121	return colorFondo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
122	}	Cierra el bloque abierto anteriormente.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	public void setColorFondo(Color colorFondo) {	Declara el método setColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
125	Color anterior = this.colorFondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
126	this.colorFondo = colorSeguro(colorFondo);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
127	if (!mouseEncima && !desplegado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
128	colorActual = this.colorFondo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
129	}	Cierra el bloque abierto anteriormente.
130	firePropertyChange("colorFondo", anterior, this.colorFondo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132	}	Cierra el bloque abierto anteriormente.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	@BeanProperty(preferred = true, description = "Color del botón al pasar el mouse.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
135	public Color getColorHover() {	Declara el método getColorHover. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
136	return colorHover;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
137	}	Cierra el bloque abierto anteriormente.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	public void setColorHover(Color colorHover) {	Declara el método setColorHover. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
140	Color anterior = this.colorHover;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
141	this.colorHover = colorSeguro(colorHover);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
142	firePropertyChange("colorHover", anterior, this.colorHover);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

143	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	}	Cierra el bloque abierto anteriormente.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	@BeanProperty(description = "Color del botón mientras el menú está abierto.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
147	public Color getColorDesplegado() {	Declara el método getColorDesplegado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
148	return colorDesplegado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
149	}	Cierra el bloque abierto anteriormente.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	public void setColorDesplegado(Color colorDesplegado) {	Declara el método setColorDesplegado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
152	Color anterior = this.colorDesplegado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
153	this.colorDesplegado = colorSeguro(colorDesplegado);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
154	firePropertyChange("colorDesplegado", anterior, this.colorDesplegado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156	}	Cierra el bloque abierto anteriormente.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	@BeanProperty(preferred = true, description = "Color de fondo del menú desplegable.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
159	public Color getColorFondoMenu() {	Declara el método getColorFondoMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
160	return colorFondoMenu;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	public void setColorFondoMenu(Color colorFondoMenu) {	Declara el método setColorFondoMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
164	Color anterior = this.colorFondoMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
165	this.colorFondoMenu = colorFondoMenu != null ? colorFondoMenu : Color.WHITE;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
166	firePropertyChange("colorFondoMenu", anterior, this.colorFondoMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	configurarAspectoMenu();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	}	Cierra el bloque abierto anteriormente.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	@BeanProperty(description = "Color normal del texto de las opciones.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
172	public Color getColorTextoOpcion() {	Declara el método getColorTextoOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
173	return colorTextoOpcion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
174	}	Cierra el bloque abierto anteriormente.
175		Línea en blanco: separa visualmente bloques para que el código sea más legible.
176	public void setColorTextoOpcion(Color colorTextoOpcion) {	Declara el método setColorTextoOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
177	Color anterior = this.colorTextoOpcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
178	this.colorTextoOpcion = colorTextoOpcion != null ? colorTextoOpcion : Color.BLACK;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
179	firePropertyChange("colorTextoOpcion", anterior, this.colorTextoOpcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
180	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181	}	Cierra el bloque abierto anteriormente.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.

183	<code>@BeanProperty(preferred = true, description = "Color de las opciones al pasar el mouse.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
184	<code>public Color getColorHoverOpcion() {</code>	Declara el método <code>getColorHoverOpcion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
185	<code> return colorHoverOpcion;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
186	<code>}</code>	Cierra el bloque abierto anteriormente.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	<code>public void setColorHoverOpcion(Color colorHoverOpcion) {</code>	Declara el método <code>setColorHoverOpcion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
189	<code> Color anterior = this.colorHoverOpcion;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
190	<code> this.colorHoverOpcion = colorHoverOpcion != null ? colorHoverOpcion : Color.LIGHT_GRAY;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
191	<code> firePropertyChange("colorHoverOpcion", anterior, this.colorHoverOpcion);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
192	<code> reconstruirOpciones();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
193	<code>}</code>	Cierra el bloque abierto anteriormente.
194		Línea en blanco: separa visualmente bloques para que el código sea más legible.
195	<code>@BeanProperty(description = "Color del texto de una opción durante el hover.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
196	<code>public Color getColorTextoHoverOpcion() {</code>	Declara el método <code>getColorTextoHoverOpcion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
197	<code> return colorTextoHoverOpcion;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
198	<code>}</code>	Cierra el bloque abierto anteriormente.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	<code>public void setColorTextoHoverOpcion(Color colorTextoHoverOpcion) {</code>	Declara el método <code>setColorTextoHoverOpcion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
201	<code> Color anterior = this.colorTextoHoverOpcion;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
202	<code> this.colorTextoHoverOpcion = colorTextoHoverOpcion != null</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
203	<code> ? colorTextoHoverOpcion : Color.BLACK;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
204	<code> firePropertyChange("colorTextoHoverOpcion", anterior, this.colorTextoHoverOpcion);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
205	<code> reconstruirOpciones();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	<code>}</code>	Cierra el bloque abierto anteriormente.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	<code>@BeanProperty(description = "Color del borde del menú desplegable.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
209	<code>public Color getColorBordeMenu() {</code>	Declara el método <code>getColorBordeMenu</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
210	<code> return colorBordeMenu;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
211	<code>}</code>	Cierra el bloque abierto anteriormente.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	<code>public void setColorBordeMenu(Color colorBordeMenu) {</code>	Declara el método <code>setColorBordeMenu</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
214	<code> Color anterior = this.colorBordeMenu;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
215	<code> this.colorBordeMenu = colorSeguro(colorBordeMenu);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
216	<code> firePropertyChange("colorBordeMenu", anterior, this.colorBordeMenu);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
217	<code> configurarAspectoMenu();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218	<code>}</code>	Cierra el bloque abierto anteriormente.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	<code>@BeanProperty(preferred = true, description = "Radio de las esquinas del botón.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
221	<code>public int getRadio() {</code>	Declara el método <code>getRadio</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
222	<code> return radio;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.

223	}	Cierra el bloque abierto anteriormente.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
226	int anterior = this.radio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
227	this.radio = Math.max(0, radio);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
228	firePropertyChange("radio", anterior, this.radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
229	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
230	}	Cierra el bloque abierto anteriormente.
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	@BeanProperty(description = "Ancho del menú desplegable.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
234	public int getAnchoMenu() {	Declara el método getAnchoMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
235	return anchoMenu;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
236	}	Cierra el bloque abierto anteriormente.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	public void setAnchoMenu(int anchoMenu) {	Declara el método setAnchoMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
239	int anterior = this.anchoMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
240	this.anchoMenu = Math.max(80, anchoMenu);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
241	firePropertyChange("anchoMenu", anterior, this.anchoMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
242	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
243	}	Cierra el bloque abierto anteriormente.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	@BeanProperty(description = "Altura de cada opción del menú.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
246	public int getAltoOpcion() {	Declara el método getAltoOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
247	return altoOpcion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
248	}	Cierra el bloque abierto anteriormente.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	public void setAltoOpcion(int altoOpcion) {	Declara el método setAltoOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
251	int anterior = this.altoOpcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
252	this.altoOpcion = Math.max(24, altoOpcion);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
253	firePropertyChange("altoOpcion", anterior, this.altoOpcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	}	Cierra el bloque abierto anteriormente.
256		Línea en blanco: separa visualmente bloques para que el código sea más legible.
257	@BeanProperty(description = "Separación entre el botón y el menú.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
258	public int getSeparacionMenu() {	Declara el método getSeparacionMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
259	return separacionMenu;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
260	}	Cierra el bloque abierto anteriormente.
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	public void setSeparacionMenu(int separacionMenu) {	Declara el método setSeparacionMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
263	int anterior = this.separacionMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
264	this.separacionMenu = Math.max(0, separacionMenu);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
265	firePropertyChange("separacionMenu", anterior, this.separacionMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

266	}	Cierra el bloque abierto anteriormente.
267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
268	@BeanProperty(hidden = true)	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
269	public String getOpcionSeleccionada() {	Declara el método getOpcionSeleccionada. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
270	return opcionSeleccionada;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
271	}	Cierra el bloque abierto anteriormente.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	@BeanProperty(hidden = true)	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
274	public int getIndiceOpcionSeleccionada() {	Declara el método getIndiceOpcionSeleccionada. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
275	return indiceOpcionSeleccionada;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
276	}	Cierra el bloque abierto anteriormente.
277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
278	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
279	public void setFont(Font fuente) {	Declara el método setFont. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
280	super.setFont(fuente);	Configura una propiedad mediante setFont en el objeto indicado.
281	reconstruirOpciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
282	}	Cierra el bloque abierto anteriormente.
283		Línea en blanco: separa visualmente bloques para que el código sea más legible.
284	public void addMenuOpcionListener(ActionListener escucha) {	Declara el método addMenuOpcionListener. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
285	escuchasOpciones.add(ActionListener.class, escucha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
286	}	Cierra el bloque abierto anteriormente.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	public void removeMenuOpcionListener(ActionListener escucha) {	Declara el método removeMenuOpcionListener. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
289	escuchasOpciones.remove(ActionListener.class, escucha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
290	}	Cierra el bloque abierto anteriormente.
291		Línea en blanco: separa visualmente bloques para que el código sea más legible.
292	private void configurarAspectoMenu() {	Declara el método configurarAspectoMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
293	if (menuDesplegable == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
294	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
295	}	Cierra el bloque abierto anteriormente.
296	menuDesplegable.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
297	menuDesplegable.setBackground(colorFondoMenu);	Configura una propiedad mediante setBackground en el objeto indicado.
298	menuDesplegable.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
299	new LineBorder(colorBordeMenu, 1, true),	Crea una nueva instancia de LineBorder y la usa o guarda para continuar el proceso.
300	new EmptyBorder(7, 7, 7, 7)	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
301	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	}	Cierra el bloque abierto anteriormente.
303		Línea en blanco: separa visualmente bloques para que el código sea más legible.
304	private void reconstruirOpciones() {	Declara el método reconstruirOpciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
305	if (menuDesplegable == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

306	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
307	}	Cierra el bloque abierto anteriormente.
308	menuDesplegable.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
310	for (String valor : textoDesplegable.split(";")) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
311	String textoOpcion = valor.trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
312	if (textoOpcion.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
313	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
314	}	Cierra el bloque abierto anteriormente.
315	if ("-".equals(textoOpcion)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
316	menuDesplegable.addSeparator();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
318	}	Cierra el bloque abierto anteriormente.
319	menuDesplegable.add(crearOpcion(textoOpcion, indice));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
320	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321	}	Cierra el bloque abierto anteriormente.
322	menuDesplegable.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
323	menuDesplegable.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
324	}	Cierra el bloque abierto anteriormente.
325		Línea en blanco: separa visualmente bloques para que el código sea más legible.
326	private JMenuItem crearOpcion(String textoOpcion, int indice) {	Declara el método crearOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
327	JMenuItem opcion = new JMenuItem(textoOpcion);	Crea una nueva instancia de JMenuItem y la usa o guarda para continuar el proceso.
328	opcion.setFont(getFont());	Configura una propiedad mediante setFont en el objeto indicado.
329	opcion.setForeground(colorTextoOpcion);	Configura una propiedad mediante setForeground en el objeto indicado.
330	opcion.setBackground(colorFondoMenu);	Configura una propiedad mediante setBackground en el objeto indicado.
331	opcion.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
332	opcion.setBorder(new EmptyBorder(0, 16, 0, 16));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
333	opcion.setPreferredSize(new Dimension(anchaMenu, altoOpcion));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
334	opcion.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Configura una propiedad mediante setCursor en el objeto indicado.
335	opcion.setUI(new BasicMenuItemUI() {	Crea una nueva instancia de BasicMenuItemUI y la usa o guarda para continuar el proceso.
336	{	Abre un bloque de instrucciones perteneciente a la declaración anterior.
337	selectionBackground = colorHoverOpcion;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
338	selectionForeground = colorTextoHoverOpcion;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
339	}	Cierra el bloque abierto anteriormente.
340	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
341	opcion.addActionListener(evento -> seleccionarOpcion(textoOpcion, indice));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
342	return opcion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
343	}	Cierra el bloque abierto anteriormente.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
345	private void seleccionarOpcion(String opcion, int indice) {	Declara el método seleccionarOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
346	String opcionAnterior = opcionSeleccionada;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
347	int indiceAnterior = indiceOpcionSeleccionada;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
348	opcionSeleccionada = opcion;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
349	indiceOpcionSeleccionada = indice;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

350	firePropertyChange("opcionSeleccionada", opcionAnterior, opcionSeleccionada);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
351	firePropertyChange("indiceOpcionSeleccionada", indiceAnterior,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
352	indiceOpcionSeleccionada);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
353		Línea en blanco: separa visualmente bloques para que el código sea más legible.
354	ActionEvent evento = new ActionEvent(	Crea una nueva instancia de ActionEvent y la usa o guarda para continuar el proceso.
355	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
356	ActionEvent.ACTION_PERFORMED,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	opcionSeleccionada	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
358	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
359	for (ActionListener escucha	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
360	: escuchasOpciones.getListeners(ActionListener.class)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
361	escucha.actionPerformed(evento);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
362	}	Cierra el bloque abierto anteriormente.
363	}	Cierra el bloque abierto anteriormente.
364		Línea en blanco: separa visualmente bloques para que el código sea más legible.
365	private void alternarMenu() {	Declara el método alternarMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
366	if (menuDesplegable.isVisible()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
367	menuDesplegable.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
368	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
369	}	Cierra el bloque abierto anteriormente.
370	if (!isShowing() menuDesplegable.getComponentCount() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
371	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
372	}	Cierra el bloque abierto anteriormente.
373	int posicionX = getWidth() - anchoMenu - 14;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
374	menuDesplegable.show(this, posicionX, getHeight() + separacionMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
375	}	Cierra el bloque abierto anteriormente.
376		Línea en blanco: separa visualmente bloques para que el código sea más legible.
377	private void setDesplegado(boolean desplegado) {	Declara el método setDesplegado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
378	this.desplegado = desplegado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
379	iniciarAnimacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
380	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
381	}	Cierra el bloque abierto anteriormente.
382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
383	private void iniciarAnimacion() {	Declara el método iniciarAnimacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
384	if (animacion != null && !animacion.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
385	animacion.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
386	}	Cierra el bloque abierto anteriormente.
387	}	Cierra el bloque abierto anteriormente.
388		Línea en blanco: separa visualmente bloques para que el código sea más legible.
389	private void actualizarAnimacion() {	Declara el método actualizarAnimacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
390	Color objetivo = desplegado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
391	? colorDesplegado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	: mouseEncima ? colorHover : colorFondo;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
393	colorActual = mezclar(colorActual, objetivo, 0.24f);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

394	if (coloresCercanos(colorActual, objetivo)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
395	colorActual = objetivo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
396	animacion.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
397	}	Cierra el bloque abierto anteriormente.
398	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
399	}	Cierra el bloque abierto anteriormente.
400		Línea en blanco: separa visualmente bloques para que el código sea más legible.
401	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
402	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
403	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
404	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
405	if (colorActual.getAlpha() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
406	g2.setColor(colorActual);	Configura una propiedad mediante setColor en el objeto indicado.
407	g2.fillRoundRect(0, 2, Math.max(1, getWidth() - 1),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
408	Math.max(1, getHeight() - 4), radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	}	Cierra el bloque abierto anteriormente.
410	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
411		Línea en blanco: separa visualmente bloques para que el código sea más legible.
412	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413	pintarFlecha(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
414	}	Cierra el bloque abierto anteriormente.
415		Línea en blanco: separa visualmente bloques para que el código sea más legible.
416	private void pintarFlecha(Graphics graphics) {	Declara el método pintarFlecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
417	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
418	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
419	g2.setColor(isEnabled() ? getForeground() : getForeground().darker());	Configura una propiedad mediante setColor en el objeto indicado.
420	g2.setStroke(new BasicStroke(2f, BasicStroke.CAP_ROUND,	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
421	BasicStroke.JOIN_ROUND));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
422		Línea en blanco: separa visualmente bloques para que el código sea más legible.
423	int centroX = getWidth() - 23;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
424	int centroY = getHeight() / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
425	int direccion = desplegado ? -1 : 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
426	g2.drawLine(centroX - 5, centroY - 2 * direccion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
427	centroX, centroY + 3 * direccion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
428	g2.drawLine(centroX, centroY + 3 * direccion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
429	centroX + 5, centroY - 2 * direccion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
431	}	Cierra el bloque abierto anteriormente.
432		Línea en blanco: separa visualmente bloques para que el código sea más legible.
433	private static void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
434	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
435	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
436	g2.setRenderingHint(RenderingHints.KEY_RENDERING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
437	RenderingHints.VALUE_RENDER_QUALITY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

438	<code>g2.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL,</code>	Configura una propiedad mediante <code>setRenderingHint</code> en el objeto indicado.
439	<code>RenderingHints.VALUE_STROKE_PURE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
440	<code>}</code>	Cierra el bloque abierto anteriormente.
441		Línea en blanco: separa visualmente bloques para que el código sea más legible.
442	<code>private static Color colorSeguro(Color color) {</code>	Declara el método <code>colorSeguro</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
443	<code>return color != null ? color : new Color(0, 0, 0, 0);</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
444	<code>}</code>	Cierra el bloque abierto anteriormente.
445		Línea en blanco: separa visualmente bloques para que el código sea más legible.
446	<code>private static Color mezclar(Color origen, Color destino, float proporcion) {</code>	Declara el método <code>mezclar</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
447	<code>float inversa = 1f - proporcion;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
448	<code>return new Color(</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
449	<code>Math.round(origen.getRed() * inversa + destino.getRed() * proporcion),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
450	<code>Math.round(origen.getGreen() * inversa + destino.getGreen() * proporcion),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
451	<code>Math.round(origen.getBlue() * inversa + destino.getBlue() * proporcion),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
452	<code>Math.round(origen.getAlpha() * inversa + destino.getAlpha() * proporcion)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
454	<code>}</code>	Cierra el bloque abierto anteriormente.
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	<code>private static boolean coloresCercanos(Color primero, Color segundo) {</code>	Declara el método <code>coloresCercanos</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
457	<code>return Math.abs(primero.getRed() - segundo.getRed()) <= 2</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
458	<code>&& Math.abs(primero.getGreen() - segundo.getGreen()) <= 2</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459	<code>&& Math.abs(primero.getBlue() - segundo.getBlue()) <= 2</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
460	<code>&& Math.abs(primero.getAlpha() - segundo.getAlpha()) <= 2;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
461	<code>}</code>	Cierra el bloque abierto anteriormente.
462	<code>}</code>	Cierra el bloque abierto anteriormente.

16.30 src/Componentes/BotonLetras.java

Componente visual reutilizable BotonLetras. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 232 | Archivo 30 de 157

Clases/tipos detectados: BotonLetras

Línea	Constructor o método detectado
35	public BotonLetras() {
68	public void mouseEntered(MouseEvent e) {
76	public void mouseExited(MouseEvent e) {
97	private void seleccionarEsteBoton() {
126	protected void paintComponent(Graphics g) {
206	public void setSeleccionado(boolean seleccionado) {
213	public boolean isSeleccionado() {
221	public void setRadio(int radio) {
228	public int getRadio() {

1 package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Container;	Importa java.awt.Container para poder usar esa clase sin escribir su nombre completo.
7 import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
8 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
9 import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
10 import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
11 import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
12 import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
13 import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
14 import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
15 import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
16 import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
17	Línea en blanco: separa visualmente bloques para que el código sea más legible.
18 public class BotonLetras extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19	Línea en blanco: separa visualmente bloques para que el código sea más legible.
20 // Mouse encima	Comentario para humanos; Java no ejecuta este texto. Mouse encima
21 private boolean mouseEncima = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	Línea en blanco: separa visualmente bloques para que el código sea más legible.
23 // Si el botón está seleccionado	Comentario para humanos; Java no ejecuta este texto. Si el botón está seleccionado
24 private boolean seleccionado = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	// Colores	Comentario para humanos; Java no ejecuta este texto. Colores
27	private final Color colorFondoNormal = Color.WHITE;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
28	private final Color bordeNormal = new Color(235, 235, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
29	private final Color colorAmarillo = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
30	private final Color colorTexto = new Color(30, 30, 30);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	// Redondeado	Comentario para humanos; Java no ejecuta este texto. Redondeado
33	private int radio = 22;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	public BotonLetras() {	Constructor de BotonLetras: prepara un objeto nuevo y deja sus componentes listos para usarse.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	setText("Categoría");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	setPreferredSize(new Dimension(105, 105));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	setFont(new Font("Arial", Font.BOLD, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
42	setForeground(colorTexto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	// Centrar texto	Comentario para humanos; Java no ejecuta este texto. Centrar texto
45	setHorizontalAlignment(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	setVerticalAlignment(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	setHorizontalTextPosition(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	setVerticalTextPosition(SwingConstants.BOTTOM);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	setIconTextGap(4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	// Quitar diseño normal del JButton	Comentario para humanos; Java no ejecuta este texto. Quitar diseño normal del JButton
54	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
57	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	// Cursor	Comentario para humanos; Java no ejecuta este texto. Cursor
60	setCursor(new Cursor(Cursor.HAND_CURSOR));	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	// =====	Comentario para humanos; Java no ejecuta este texto. =====
63	// MOUSE	Comentario para humanos; Java no ejecuta este texto. MOUSE
64	// =====	Comentario para humanos; Java no ejecuta este texto. =====
65	addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
68	public void mouseEntered(MouseEvent e) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	mouseEncima = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73	}	Cierra el bloque abierto anteriormente.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
76	public void mouseExited(MouseEvent e) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	mouseEncima = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	}	Cierra el bloque abierto anteriormente.
82	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	// =====	Comentario para humanos; Java no ejecuta este texto. =====
85	// CLICK	Comentario para humanos; Java no ejecuta este texto. CLICK
86	// =====	Comentario para humanos; Java no ejecuta este texto. =====
87	addActionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	seleccionarEsteBoton();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	// =====	Comentario para humanos; Java no ejecuta este texto. =====
95	// SELECCIONAR ESTE BOTÓN	Comentario para humanos; Java no ejecuta este texto. SELECCIONAR ESTE BOTÓN
96	// =====	Comentario para humanos; Java no ejecuta este texto. =====
97	private void seleccionarEsteBoton() {	Declara el método seleccionarEsteBoton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	Container contenedor = getParent();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	if (contenedor == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
102	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	}	Cierra el bloque abierto anteriormente.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	// Buscar todos los componentes	Comentario para humanos; Java no ejecuta este texto. Buscar todos los componentes
106	for (Component componente : contenedor.getComponents()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	// Si es un BotonLetras	Comentario para humanos; Java no ejecuta este texto. Si es un BotonLetras
109	if (componente instanceof BotonLetras) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	BotonLetras boton = (BotonLetras) componente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.

113	// Quitar selección	Comentario para humanos; Java no ejecuta este texto. Quitar selección
114	boton.setSeleccionado(false);	Configura una propiedad mediante setSeleccionado en el objeto indicado.
115	}	Cierra el bloque abierto anteriormente.
116	}	Cierra el bloque abierto anteriormente.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	// Seleccionar únicamente este	Comentario para humanos; Java no ejecuta este texto. Seleccionar únicamente este
119	setSeleccionado(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	}	Cierra el bloque abierto anteriormente.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	// =====	Comentario para humanos; Java no ejecuta este texto. =====
123	// DIBUJAR BOTÓN	Comentario para humanos; Java no ejecuta este texto. DIBUJAR BOTÓN
124	// =====	Comentario para humanos; Java no ejecuta este texto. =====
125	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
126	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
130	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
131	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	int ancho = getWidth();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136	int alto = getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	// =====	Comentario para humanos; Java no ejecuta este texto. =====
139	// AMARILLO	Comentario para humanos; Java no ejecuta este texto. AMARILLO
140	// =====	Comentario para humanos; Java no ejecuta este texto. =====
141	// Si:	Comentario para humanos; Java no ejecuta este texto. Si:
142	// - está seleccionado	Comentario para humanos; Java no ejecuta este texto. - está seleccionado
143	// - o el mouse está encima	Comentario para humanos; Java no ejecuta este texto. - o el mouse está encima
144	if (seleccionado mouseEncima) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	g2.setColor(colorAmarillo);	Configura una propiedad mediante setColor en el objeto indicado.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.
148	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
150	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
151	ancho - 4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
152	alto - 4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
153	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.

157	g2.setColor(colorAmarillo);	Configura una propiedad mediante setColor en el objeto indicado.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
160	new BasicStroke(2.5f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
161	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	// =====	Comentario para humanos; Java no ejecuta este texto. =====
166	// BLANCO	Comentario para humanos; Java no ejecuta este texto. BLANCO
167	// =====	Comentario para humanos; Java no ejecuta este texto. =====
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	g2.setColor(colorFondoNormal);	Configura una propiedad mediante setColor en el objeto indicado.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	ancho - 4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	alto - 4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179		Línea en blanco: separa visualmente bloques para que el código sea más legible.
180	g2.setColor(bordeNormal);	Configura una propiedad mediante setColor en el objeto indicado.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
183	new BasicStroke(2f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
184	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	}	Cierra el bloque abierto anteriormente.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	// Borde	Comentario para humanos; Java no ejecuta este texto. Borde
188	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	ancho - 5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	alto - 5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199	// Texto e imagen	Comentario para humanos; Java no ejecuta este texto. Texto e imagen
200	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

201	}	Cierra el bloque abierto anteriormente.
202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
203	// =====	Comentario para humanos; Java no ejecuta este texto. =====
204	// SELECCIONAR / DESELECCIONAR	Comentario para humanos; Java no ejecuta este texto. SELECCIONAR / DESELECCIONAR
205	// =====	Comentario para humanos; Java no ejecuta este texto. =====
206	public void setSeleccionado(boolean seleccionado) {	Declara el método setSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	this.seleccionado = seleccionado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
211	}	Cierra el bloque abierto anteriormente.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	public boolean isSeleccionado() {	Declara el método isSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	return seleccionado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
216	}	Cierra el bloque abierto anteriormente.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	// =====	Comentario para humanos; Java no ejecuta este texto. =====
219	// RADIO	Comentario para humanos; Java no ejecuta este texto. RADIO
220	// =====	Comentario para humanos; Java no ejecuta este texto. =====
221	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	this.radio = radio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
226	}	Cierra el bloque abierto anteriormente.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	public int getRadio() {	Declara el método getRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230	return radio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
231	}	Cierra el bloque abierto anteriormente.
232	}	Cierra el bloque abierto anteriormente.

16.31 src/Componentes/BotonMenuLateral.java

Componente visual reutilizable BotonMenuLateral. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 275 | Archivo 31 de 157

Clases/tipos detectados: BotonMenuLateral

Línea	Constructor o método detectado
37	public void setAperturaMenu(double apertura) {
42	public double getAperturaMenu() {
46	public BotonMenuLateral() {
57	protected void paintComponent(Graphics graphics) {
101	private void pintarIcono(Graphics2D g2, int centroX, int centroY, Color color) {
175	private Color mezclar(Color base, Color destino, float proporcion) {
184	public Color getColorNormal() {
188	public void setColorNormal(Color colorNormal) {
193	public Color getColorHover() {
197	public void setColorHover(Color colorHover) {
202	public Color getColorSeleccionado() {
206	public void setColorSeleccionado(Color colorSeleccionado) {
211	public Color getColorTextoNormal() {
215	public void setColorTextoNormal(Color colorTextoNormal) {
220	public Color getColorTextoSeleccionado() {
224	public void setColorTextoSeleccionado(Color colorTextoSeleccionado) {
230	public Color getColorBorde() {
234	public void setColorBorde(Color colorBorde) {
239	public int getRadio() {
243	public void setRadio(int radio) {
248	public boolean isSeleccionado() {
252	public void setSeleccionado(boolean seleccionado) {
257	public boolean isMostrarBorde() {
261	public void setMostrarBorde(boolean mostrarBorde) {
266	public String getTipIcono() {
270	public void setTipIcono(String tipIcono) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.FontMetrics;	Importa java.awt.FontMetrics para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Shape;	Importa java.awt.Shape para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.geom.Arc2D;	Importa java.awt.geom.Arc2D para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.geom.Ellipse2D;	Importa java.awt.geom.Ellipse2D para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.geom.Line2D;	Importa java.awt.geom.Line2D para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.geom.Path2D;	Importa java.awt.geom.Path2D para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.geom.RoundRectangle2D;	Importa java.awt.geom.RoundRectangle2D para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	* Boton reutilizable para la navegacion lateral del modulo administrativo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
20	* El fondo, el hover, la seleccion y los iconos se pintan como vectores.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	public class BotonMenuLateral extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	private Color colorNormal = new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
25	private Color colorHover = new Color(255, 255, 255, 28);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
26	private Color colorSeleccionado = new Color(255, 190, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
27	private Color colorTextoNormal = Color.WHITE;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	private Color colorTextoSeleccionado = new Color(0, 22, 47);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
29	private Color colorBorde = new Color(255, 255, 255, 170);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
30	private int radio = 14;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31	private boolean seleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
32	private boolean mostrarBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33	private String tipoIcono = "DASHBOARD";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34	private double aperturaMenu = 1.0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	/** Interpola el contenido sin borrar el texto ni su nombre accesible. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
37	public void setAperturaMenu(double apertura) {	Declara el método setAperturaMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
38	aperturaMenu = Math.max(0.0, Math.min(1.0, apertura));	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
39	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	}	Cierra el bloque abierto anteriormente.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	public double getAperturaMenu() {	Declara el método getAperturaMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

43	return aperturaMenu;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
44	}	Cierra el bloque abierto anteriormente.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	public BotonMenuLateral() {	Constructor de BotonMenuLateral: prepara un objeto nuevo y deja sus componentes listos para usarse.
47	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51	setRolloverEnabled(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52	setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	setHorizontalAlignment(LEFT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
57	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
58	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
60	g2.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL, RenderingHints.VALUE_STROKE_PURE);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	Color fondo = seleccionado ? colorSeleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
63	: getModel().isRollover() ? colorHover : colorNormal;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	if (getModel().isPressed()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
65	fondo = mezclar(fondo, Color.BLACK, 0.10f);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
66	}	Cierra el bloque abierto anteriormente.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	Shape forma = new RoundRectangle2D.Float(	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
69	1, 1, getWidth() - 2f, getHeight() - 2f, radio * 2f, radio * 2f);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	if (fondo.getAlpha() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
71	g2.setColor(fondo);	Configura una propiedad mediante setColor en el objeto indicado.
72	g2.fill(forma);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73	}	Cierra el bloque abierto anteriormente.
74	if (mostrarBorde) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
75	g2.setColor(colorBorde);	Configura una propiedad mediante setColor en el objeto indicado.
76	g2.setStroke(new BasicStroke(1.4f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
77	g2.draw(forma);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	}	Cierra el bloque abierto anteriormente.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	Color colorContenido = seleccionado ? colorTextoSeleccionado : colorTextoNormal;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	if (!isEnabled()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
82	colorContenido = new Color(colorContenido.getRed(), colorContenido.getGreen(),	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
83	colorContenido.getBlue(), 110);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	int centroY = getHeight() / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

87	int centroIcono = (int) Math.round((getWidth() / 2.0) * (1.0 - aperturaMenu)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
88	+ 31 * aperturaMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	pintarIcono(g2, centroIcono, centroY, colorContenido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	float opacidadTexto = (float) Math.max(0.0, Math.min(1.0, (aperturaMenu - 0.35) / 0.65));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
91	g2.setComposite(java.awt.AlphaComposite.SrcOver.derive(opacidadTexto));	Configura una propiedad mediante setComposite en el objeto indicado.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	g2.setColor(colorContenido);	Configura una propiedad mediante setColor en el objeto indicado.
94	g2.setFont(getFont());	Configura una propiedad mediante setFont en el objeto indicado.
95	FontMetrics metricas = g2.getFontMetrics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
96	int textoY = centroY + (metricas.getAscent() - metricas.getDescent()) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
97	g2.drawString(getText() == null ? "" : getText(), 66, textoY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99	}	Cierra el bloque abierto anteriormente.
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	private void pintarIcono(Graphics2D g2, int centroX, int centroY, Color color) {	Declara el método pintarIcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
102	Graphics2D icono = (Graphics2D) g2.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
103	icono.setColor(color);	Configura una propiedad mediante setColor en el objeto indicado.
104	icono.setStroke(new BasicStroke(2.35f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
105	icono.translate(centroX - 13, centroY - 13);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	switch (tipoIcono == null ? "" : tipoIcono.trim().toUpperCase()) {	Selecciona una ruta entre varias según el valor evaluado.
108	case "USUARIOS":	Define una de las rutas posibles dentro del switch.
109	icono.draw(new Ellipse2D.Float(8, 2, 8, 8));	Crea una nueva instancia de Ellipse2D.Float y la usa o guarda para continuar el proceso.
110	icono.draw(new Arc2D.Float(4, 10, 16, 14, 15, 150, Arc2D.OPEN));	Crea una nueva instancia de Arc2D.Float y la usa o guarda para continuar el proceso.
111	icono.draw(new Arc2D.Float(14, 5, 8, 8, 260, 220, Arc2D.OPEN));	Crea una nueva instancia de Arc2D.Float y la usa o guarda para continuar el proceso.
112	icono.draw(new Arc2D.Float(15, 13, 10, 10, 340, 125, Arc2D.OPEN));	Crea una nueva instancia de Arc2D.Float y la usa o guarda para continuar el proceso.
113	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	case "MENU":	Define una de las rutas posibles dentro del switch.
115	icono.draw(new Arc2D.Float(2, 7, 22, 13, 0, 180, Arc2D.OPEN));	Crea una nueva instancia de Arc2D.Float y la usa o guarda para continuar el proceso.
116	icono.draw(new Line2D.Float(2, 14, 24, 14));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
117	icono.draw(new RoundRectangle2D.Float(2, 17, 22, 4, 2, 2));	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
118	icono.draw(new Line2D.Float(12, 4, 14, 4));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
119	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	case "INGREDIENTES":	Define una de las rutas posibles dentro del switch.
121	Path2D hoja = new Path2D.Float();	Crea una nueva instancia de Path2D.Float y la usa o guarda para continuar el proceso.
122	hoja.moveTo(4, 21);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	hoja.curveTo(4, 8, 13, 3, 24, 2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	hoja.curveTo(23, 14, 17, 23, 7, 22);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	hoja.closePath();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126	icono.draw(hoja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	icono.draw(new Line2D.Float(4, 24, 18, 8));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
128	icono.draw(new Line2D.Float(10, 17, 10, 11));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
129	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	case "INVENTARIO":	Define una de las rutas posibles dentro del switch.

131	Path2D caja = new Path2D.Float();	Crea una nueva instancia de Path2D.Float y la usa o guarda para continuar el proceso.
132	caja.moveTo(3, 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	caja.lineTo(13, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	caja.lineTo(23, 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
135	caja.lineTo(13, 13);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	caja.closePath();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	icono.draw(caja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	icono.draw(new Line2D.Float(3, 8, 3, 19));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
139	icono.draw(new Line2D.Float(3, 19, 13, 24));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
140	icono.draw(new Line2D.Float(13, 13, 13, 24));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
141	icono.draw(new Line2D.Float(23, 8, 23, 19));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
142	icono.draw(new Line2D.Float(23, 19, 13, 24));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
143	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	case "REPORTES":	Define una de las rutas posibles dentro del switch.
145	icono.draw(new RoundRectangle2D.Float(2, 15, 4, 9, 1, 1));	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
146	icono.draw(new RoundRectangle2D.Float(10, 9, 4, 15, 1, 1));	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
147	icono.draw(new RoundRectangle2D.Float(18, 3, 4, 21, 1, 1));	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
148	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	case "SALIR":	Define una de las rutas posibles dentro del switch.
150	icono.draw(new RoundRectangle2D.Float(3, 2, 12, 22, 2, 2));	Crea una nueva instancia de RoundRectangle2D.Float y la usa o guarda para continuar el proceso.
151	icono.draw(new Line2D.Float(11, 13, 25, 13));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
152	icono.draw(new Line2D.Float(20, 8, 25, 13));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
153	icono.draw(new Line2D.Float(20, 18, 25, 13));	Crea una nueva instancia de Line2D.Float y la usa o guarda para continuar el proceso.
154	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	case "DASHBOARD":	Define una de las rutas posibles dentro del switch.
156	default:	Define una de las rutas posibles dentro del switch.
157	Path2D casa = new Path2D.Float();	Crea una nueva instancia de Path2D.Float y la usa o guarda para continuar el proceso.
158	casa.moveTo(2, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	casa.lineTo(13, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
160	casa.lineTo(24, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161	casa.moveTo(5, 10);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	casa.lineTo(5, 23);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	casa.lineTo(21, 23);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
164	casa.lineTo(21, 10);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165	casa.moveTo(10, 23);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
166	casa.lineTo(10, 16);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	casa.lineTo(16, 16);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	casa.lineTo(16, 23);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	icono.draw(casa);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
171	}	Cierra el bloque abierto anteriormente.
172	icono.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173	}	Cierra el bloque abierto anteriormente.
174		Línea en blanco: separa visualmente bloques para que el código sea más legible.

175	private Color mezclar(Color base, Color destino, float proporcion) {	Declara el método mezclar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
176	float inversa = 1f - proporcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
177	return new Color(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
178	Math.round(base.getRed() * inversa + destino.getRed() * proporcion),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	Math.round(base.getGreen() * inversa + destino.getGreen() * proporcion),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	Math.round(base.getBlue() * inversa + destino.getBlue() * proporcion),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
181	base.getAlpha());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
182	}	Cierra el bloque abierto anteriormente.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	public Color getColorNormal() {	Declara el método getColorNormal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
185	return colorNormal;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
186	}	Cierra el bloque abierto anteriormente.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	public void setColorNormal(Color colorNormal) {	Declara el método setColorNormal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
189	this.colorNormal = colorNormal == null ? new Color(0, 0, 0, 0) : colorNormal;	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
190	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
191	}	Cierra el bloque abierto anteriormente.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	public Color getColorHover() {	Declara el método getColorHover. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
194	return colorHover;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
195	}	Cierra el bloque abierto anteriormente.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	public void setColorHover(Color colorHover) {	Declara el método setColorHover. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
198	this.colorHover = colorHover == null ? new Color(255, 255, 255, 28) : colorHover;	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
199	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
200	}	Cierra el bloque abierto anteriormente.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	public Color getColorSeleccionado() {	Declara el método getColorSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
203	return colorSeleccionado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
204	}	Cierra el bloque abierto anteriormente.
205		Línea en blanco: separa visualmente bloques para que el código sea más legible.
206	public void setColorSeleccionado(Color colorSeleccionado) {	Declara el método setColorSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
207	this.colorSeleccionado = colorSeleccionado == null ? new Color(255, 190, 0) : colorSeleccionado;	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
208	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209	}	Cierra el bloque abierto anteriormente.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211	public Color getColorTextoNormal() {	Declara el método getColorTextoNormal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
212	return colorTextoNormal;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
213	}	Cierra el bloque abierto anteriormente.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	public void setColorTextoNormal(Color colorTextoNormal) {	Declara el método setColorTextoNormal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
216	this.colorTextoNormal = colorTextoNormal == null ? Color.WHITE : colorTextoNormal;	Accede al atributo del objeto actual para asignarlo o utilizarlo.

217	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218	}	Cierra el bloque abierto anteriormente.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	public Color getColorTextoSeleccionado() {	Declara el método getColorTextoSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
221	return colorTextoSeleccionado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
222	}	Cierra el bloque abierto anteriormente.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	public void setColorTextoSeleccionado(Color colorTextoSeleccionado) {	Declara el método setColorTextoSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
225	this.colorTextoSeleccionado = colorTextoSeleccionado == null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
226	? new Color(0, 22, 47) : colorTextoSeleccionado;	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
227	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
228	}	Cierra el bloque abierto anteriormente.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230	public Color getColorBorde() {	Declara el método getColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
231	return colorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
232	}	Cierra el bloque abierto anteriormente.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	public void setColorBorde(Color colorBorde) {	Declara el método setColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
235	this.colorBorde = colorBorde == null ? new Color(255, 255, 255, 170) : colorBorde;	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
236	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
237	}	Cierra el bloque abierto anteriormente.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	public int getRadio() {	Declara el método getRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
240	return radio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
241	}	Cierra el bloque abierto anteriormente.
242		Línea en blanco: separa visualmente bloques para que el código sea más legible.
243	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
244	this.radio = Math.max(0, radio);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
245	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
246	}	Cierra el bloque abierto anteriormente.
247		Línea en blanco: separa visualmente bloques para que el código sea más legible.
248	public boolean isSeleccionado() {	Declara el método isSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
249	return seleccionado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
250	}	Cierra el bloque abierto anteriormente.
251		Línea en blanco: separa visualmente bloques para que el código sea más legible.
252	public void setSeleccionado(boolean seleccionado) {	Declara el método setSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
253	this.seleccionado = seleccionado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
254	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	}	Cierra el bloque abierto anteriormente.
256		Línea en blanco: separa visualmente bloques para que el código sea más legible.
257	public boolean isMostrarBorde() {	Declara el método isMostrarBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
258	return mostrarBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
259	}	Cierra el bloque abierto anteriormente.

260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	public void setMostrarBorde(boolean mostrarBorde) {	Declara el método setMostrarBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
262	this.mostrarBorde = mostrarBorde;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
263	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
264	}	Cierra el bloque abierto anteriormente.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	public String getTipoIcono() {	Declara el método getTipolcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
267	return tipoIcono;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
268	}	Cierra el bloque abierto anteriormente.
269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
270	public void setTipoIcono(String tipoIcono) {	Declara el método setTipolcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
271	this.tipoIcono = tipoIcono == null ? "DASHBOARD" : tipoIcono;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
272	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
273	}	Cierra el bloque abierto anteriormente.
274	}	Cierra el bloque abierto anteriormente.
275		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.32 src/Componentes/BotonRedondeado.java

Componente visual reutilizable BotonRedondeado. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 221 | Archivo 32 de 157

Clases/tipos detectados: BotonRedondeado

Línea	Constructor o método detectado
29	public BotonRedondeado() {
41	public Color getColorInicio() {
45	public void setColorInicio(Color colorInicio) {
53	public Color getColorFinal() {
57	public void setColorFinal(Color colorFinal) {
65	public Color getColorBorde() {
69	public void setColorBorde(Color colorBorde) {
77	public int getRadio() {
81	public void setRadio(int radio) {
89	public float getGrosorBorde() {
93	public void setGrosorBorde(float grosorBorde) {
101	public boolean isDegradado() {
105	public void setDegradado(boolean degradado) {
113	public boolean isMostrarFlecha() {
117	public void setMostrarFlecha(boolean mostrarFlecha) {
125	public int getSeparacionFlecha() {
129	public void setSeparacionFlecha(int separacionFlecha) {
137	protected void paintComponent(Graphics graphics) {
173	private void pintarFlecha(Graphics graphics) {
191	private static void aplicarCalidad(Graphics2D g2) {
198	private static Color ajustar(Color color, int cantidad) {
207	private static Color mezclar(Color origen, Color destino, float proporcion) {
217	private static int limitar(int valor) {

1 package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.

7	import java.awt.FontMetrics;	Importa java.awt.FontMetrics para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.GradientPaint;	Importa java.awt.GradientPaint para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
12	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	/** Botón reutilizable redondeado, sólido o con degradado y flecha opcional. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
18	public class BotonRedondeado extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	private Color colorInicio = new Color(255, 184, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
21	private Color colorFinal = new Color(255, 205, 26);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
22	private Color colorBorde = new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
23	private int radio = 18;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24	private float grosorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
25	private boolean degradado = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
26	private boolean mostrarFlecha;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	private int separacionFlecha = 28;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	public BotonRedondeado() {	Constructor de BotonRedondeado: prepara un objeto nuevo y deja sus componentes listos para usarse.
30	setText("Botón");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	setForeground(Color.BLACK);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	setPreferredSize(new Dimension(220, 58));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
38	}	Cierra el bloque abierto anteriormente.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	@BeanProperty(preferred = true, description = "Color inicial o color sólido del botón.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
41	public Color getColorInicio() {	Declara el método getColorInicio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	return colorInicio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	public void setColorInicio(Color colorInicio) {	Declara el método setColorInicio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
46	Color anterior = this.colorInicio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	this.colorInicio = colorInicio != null ? colorInicio : Color.WHITE;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
48	firePropertyChange("colorInicio", anterior, this.colorInicio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	}	Cierra el bloque abierto anteriormente.

51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	@BeanProperty(description = "Color final del degradado.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
53	public Color getColorFinal() {	Declara el método getColorFinal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
54	return colorFinal;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
55	}	Cierra el bloque abierto anteriormente.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	public void setColorFinal(Color colorFinal) {	Declara el método setColorFinal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
58	Color anterior = this.colorFinal;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	this.colorFinal = colorFinal != null ? colorFinal : colorInicio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
60	firePropertyChange("colorFinal", anterior, this.colorFinal);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	}	Cierra el bloque abierto anteriormente.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	@BeanProperty(description = "Color del contorno.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
65	public Color getColorBorde() {	Declara el método getColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
66	return colorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
67	}	Cierra el bloque abierto anteriormente.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69	public void setColorBorde(Color colorBorde) {	Declara el método setColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
70	Color anterior = this.colorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
71	this.colorBorde = colorBorde != null ? colorBorde : new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
72	firePropertyChange("colorBorde", anterior, this.colorBorde);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	}	Cierra el bloque abierto anteriormente.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	@BeanProperty(preferred = true, description = "Radio de las esquinas en píxeles.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
77	public int getRadio() {	Declara el método getRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
78	return radio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
79	}	Cierra el bloque abierto anteriormente.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
82	int anterior = this.radio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	this.radio = Math.max(0, radio);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
84	firePropertyChange("radio", anterior, this.radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	}	Cierra el bloque abierto anteriormente.
87		Línea en blanco: separa visualmente bloques para que el código sea más legible.
88	@BeanProperty(description = "Grosor del contorno en píxeles.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
89	public float getGrosorBorde() {	Declara el método getGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
90	return grosorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
91	}	Cierra el bloque abierto anteriormente.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	public void setGrosorBorde(float grosorBorde) {	Declara el método setGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
94	float anterior = this.grosorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

95	<code>this.grosorBorde = Math.max(0f, grosorBorde);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
96	<code>firePropertyChange("grosorBorde", anterior, this.grosorBorde);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	<code>}</code>	Cierra el bloque abierto anteriormente.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	<code>@BeanProperty(preferred = true, description = "Activa el degradado entre ambos colores.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
101	<code>public boolean isDegradado() {</code>	Declara el método <code>isDegradado</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
102	<code>return degradado;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	<code>}</code>	Cierra el bloque abierto anteriormente.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	<code>public void setDegradado(boolean degradado) {</code>	Declara el método <code>setDegradado</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
106	<code>boolean anterior = this.degradado;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107	<code>this.degradado = degradado;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
108	<code>firePropertyChange("degradado", anterior, degradado);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	<code>}</code>	Cierra el bloque abierto anteriormente.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	<code>@BeanProperty(preferred = true, description = "Dibuja una flecha después del texto.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
113	<code>public boolean isMostrarFlecha() {</code>	Declara el método <code>isMostrarFlecha</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
114	<code>return mostrarFlecha;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
115	<code>}</code>	Cierra el bloque abierto anteriormente.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	<code>public void setMostrarFlecha(boolean mostrarFlecha) {</code>	Declara el método <code>setMostrarFlecha</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
118	<code>boolean anterior = this.mostrarFlecha;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119	<code>this.mostrarFlecha = mostrarFlecha;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
120	<code>firePropertyChange("mostrarFlecha", anterior, mostrarFlecha);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	<code>}</code>	Cierra el bloque abierto anteriormente.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	<code>@BeanProperty(description = "Separación entre el texto y la flecha.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
125	<code>public int getSeparacionFlecha() {</code>	Declara el método <code>getSeparacionFlecha</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
126	<code>return separacionFlecha;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
127	<code>}</code>	Cierra el bloque abierto anteriormente.
128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	<code>public void setSeparacionFlecha(int separacionFlecha) {</code>	Declara el método <code>setSeparacionFlecha</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
130	<code>int anterior = this.separacionFlecha;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	<code>this.separacionFlecha = Math.max(0, separacionFlecha);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
132	<code>firePropertyChange("separacionFlecha", anterior, this.separacionFlecha);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	<code>}</code>	Cierra el bloque abierto anteriormente.
135		Línea en blanco: separa visualmente bloques para que el código sea más legible.
136	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

137	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
138	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
139	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	Color inicio = colorInicio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
142	Color fin = degradado ? colorFinal : colorInicio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
143	if (!isEnabled()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
144	inicio = mezclar(inicio, Color.LIGHT_GRAY, 0.55f);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
145	fin = mezclar(fin, Color.LIGHT_GRAY, 0.55f);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
146	} else if (getModel().isPressed()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
147	inicio = ajustar(inicio, -18);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
148	fin = ajustar(fin, -18);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
149	} else if (getModel().isRollover()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
150	inicio = ajustar(inicio, 12);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
151	fin = ajustar(fin, 12);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
152	}	Cierra el bloque abierto anteriormente.
153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154	g2.setPaint(new GradientPaint(0, 0, inicio, getWidth(), 0, fin));	Crea una nueva instancia de GradientPaint y la usa o guarda para continuar el proceso.
155	g2.fillRoundRect(0, 0, getWidth(), getHeight(), radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	if (grosorBorde > 0f && colorBorde.getAlpha() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
158	g2.setStroke(new BasicStroke(grosorBorde));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
159	g2.setColor(colorBorde);	Configura una propiedad mediante setColor en el objeto indicado.
160	int ajuste = Math.max(1, Math.round(grosorBorde));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
161	g2.drawRoundRect(0, 0, getWidth() - ajuste, getHeight() - ajuste,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
162	radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	}	Cierra el bloque abierto anteriormente.
164	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	if (mostrarFlecha && getText() != null && !getText().isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
169	pintarFlecha(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	}	Cierra el bloque abierto anteriormente.
171	}	Cierra el bloque abierto anteriormente.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	private void pintarFlecha(Graphics graphics) {	Declara el método pintarFlecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
174	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
175	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
176	FontMetrics metricas = g2.getFontMetrics(getFont());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
177	int centroTexto = getWidth() / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
178	int x = centroTexto + metricas.stringWidth(getText()) / 2 + separacionFlecha;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
179	int y = getHeight() / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
180	int largo = Math.max(10, Math.min(22, getHeight() / 3));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	<code>g2.setColor(isEnabled() ? getForeground() : getForeground().darker());</code>	Configura una propiedad mediante setColor en el objeto indicado.
183	<code>g2.setStroke(new BasicStroke(2.4f, BasicStroke.CAP_ROUND,</code>	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
184	<code>BasicStroke.JOIN_ROUND));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	<code>g2.drawLine(x - largo, y, x, y);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
186	<code>g2.drawLine(x, y, x - largo / 2, y - largo / 2);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	<code>g2.drawLine(x, y, x - largo / 2, y + largo / 2);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188	<code>g2.dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189	<code>}</code>	Cierra el bloque abierto anteriormente.
190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
191	<code>private static void aplicarCalidad(Graphics2D g2) {</code>	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
192	<code>g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,</code>	Configura una propiedad mediante setRenderingHint en el objeto indicado.
193	<code>RenderingHints.VALUE_ANTIALIAS_ON);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
194	<code>g2.setRenderingHint(RenderingHints.KEY_RENDERING,</code>	Configura una propiedad mediante setRenderingHint en el objeto indicado.
195	<code>RenderingHints.VALUE_RENDER_QUALITY);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
196	<code>}</code>	Cierra el bloque abierto anteriormente.
197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
198	<code>private static Color ajustar(Color color, int cantidad) {</code>	Declara el método ajustar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
199	<code>return new Color(</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
200	<code>limitar(color.getRed() + cantidad),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
201	<code>limitar(color.getGreen() + cantidad),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
202	<code>limitar(color.getBlue() + cantidad),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
203	<code>color.getAlpha()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
205	<code>}</code>	Cierra el bloque abierto anteriormente.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	<code>private static Color mezclar(Color origen, Color destino, float proporcion) {</code>	Declara el método mezclar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
208	<code>float p = Math.max(0f, Math.min(1f, proporcion));</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
209	<code>return new Color(</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
210	<code>Math.round(origen.getRed() * (1f - p) + destino.getRed() * p),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	<code>Math.round(origen.getGreen() * (1f - p) + destino.getGreen() * p),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212	<code>Math.round(origen.getBlue() * (1f - p) + destino.getBlue() * p),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	<code>origen.getAlpha()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215	<code>}</code>	Cierra el bloque abierto anteriormente.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	<code>private static int limitar(int valor) {</code>	Declara el método limitar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
218	<code>return Math.max(0, Math.min(255, valor));</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
219	<code>}</code>	Cierra el bloque abierto anteriormente.
220	<code>}</code>	Cierra el bloque abierto anteriormente.
221		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.33 src/Componentes/CampoBusquedaAdmin.java

Componente visual reutilizable CampoBusquedaAdmin. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 152 | Archivo 33 de 157

Clases/tipos detectados: CampoBusquedaAdmin

Línea	Constructor o método detectado
32	public CampoBusquedaAdmin() {
41	public void focusGained(FocusEvent evento) {
46	public void focusLost(FocusEvent evento) {
53	public String getPlaceholder() {
57	public void setPlaceholder(String placeholder) {
63	public Color getColorBorde() {
67	public void setColorBorde(Color colorBorde) {
73	public Color getColorIcono() {
77	public void setColorIcono(Color colorIcono) {
83	public Color getColorPlaceholder() {
87	public void setColorPlaceholder(Color colorPlaceholder) {
94	public int getRadio() {
98	public void setRadio(int radio) {
104	public Insets getInsets() {
109	protected void paintComponent(Graphics graphics) {
137	private void pintarLupa(Graphics2D g2) {
146	private static void aplicarCalidad(Graphics2D g2) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.event.FocusAdapter;	Importa java.awt.event.FocusAdapter para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.event.FocusEvent;	Importa java.awt.event.FocusEvent para poder usar esa clase sin escribir su nombre completo.
12	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
13	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.

15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	* Buscador blanco y neutro para las pantallas administrativas.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
18	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	* Es un JavaBean con constructor vacío: puede agregarse a la paleta y editarse	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
20	* directamente en el diseñador de NetBeans. El icono se dibuja en gris, sin	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	* el círculo amarillo utilizado por los campos del login.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
24	public class CampoBusquedaAdmin extends JTextField {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	private String placeholder = "Buscar...";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	private Color colorBorde = new Color(222, 227, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
28	private Color colorIcono = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
29	private Color colorPlaceholder = new Color(127, 137, 154);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
30	private int radio = 14;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	public CampoBusquedaAdmin() {	Constructor de CampoBusquedaAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
33	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	setForeground(new Color(0, 20, 43));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
35	setCaretColor(new Color(0, 20, 43));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
36	setBackground(Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	setBorder(new EmptyBorder(0, 48, 0, 16));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
38	setCursor(Cursor.getPredefinedCursor(Cursor.TEXT_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	addFocusListener(new FocusAdapter() {	Crea una nueva instancia de FocusAdapter y la usa o guarda para continuar el proceso.
40	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
41	public void focusGained(FocusEvent evento) {	Declara el método focusGained. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
46	public void focusLost(FocusEvent evento) {	Declara el método focusLost. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
47	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	}	Cierra el bloque abierto anteriormente.
49	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	}	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	@BeanProperty(preferred = true, description = "Texto mostrado cuando el campo está vacío.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
53	public String getPlaceholder() {	Declara el método getPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
54	return placeholder;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
55	}	Cierra el bloque abierto anteriormente.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	public void setPlaceholder(String placeholder) {	Declara el método setPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
58	this.placeholder = placeholder != null ? placeholder : "";	Accede al atributo del objeto actual para asignarlo o utilizarlo.

59	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	}	Cierra el bloque abierto anteriormente.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	@BeanProperty(preferred = true, description = "Color del contorno.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
63	public Color getColorBorde() {	Declara el método getColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
64	return colorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
65	}	Cierra el bloque abierto anteriormente.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	public void setColorBorde(Color colorBorde) {	Declara el método setColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
68	this.colorBorde = colorBorde != null ? colorBorde : Color.LIGHT_GRAY;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
69	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	}	Cierra el bloque abierto anteriormente.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	@BeanProperty(preferred = true, description = "Color de la lupa.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
73	public Color getColorIcono() {	Declara el método getColorIcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
74	return colorIcono;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
75	}	Cierra el bloque abierto anteriormente.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	public void setColorIcono(Color colorIcono) {	Declara el método setColorIcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
78	this.colorIcono = colorIcono != null ? colorIcono : Color.GRAY;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
79	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	}	Cierra el bloque abierto anteriormente.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	@BeanProperty(description = "Color del placeholder.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
83	public Color getColorPlaceholder() {	Declara el método getColorPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
84	return colorPlaceholder;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
85	}	Cierra el bloque abierto anteriormente.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	public void setColorPlaceholder(Color colorPlaceholder) {	Declara el método setColorPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
88	this.colorPlaceholder = colorPlaceholder != null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
89	? colorPlaceholder : Color.GRAY;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	}	Cierra el bloque abierto anteriormente.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	@BeanProperty(preferred = true, description = "Radio de las esquinas.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
94	public int getRadio() {	Declara el método getRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
95	return radio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
96	}	Cierra el bloque abierto anteriormente.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	public void setRadio(int radio) {	Declara el método setRadio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
99	this.radio = Math.max(0, radio);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
100	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	}	Cierra el bloque abierto anteriormente.

102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
104	public Insets getInsets() {	Declara el método getInsets. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
105	return new Insets(0, 48, 0, 16);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
106	}	Cierra el bloque abierto anteriormente.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
109	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
110	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	g2.setColor(Color.WHITE);	Configura una propiedad mediante setColor en el objeto indicado.
114	g2.fillRoundRect(1, 1, getWidth() - 3, getHeight() - 3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
115	radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	g2.setColor(hasFocus() ? new Color(179, 188, 201) : colorBorde);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
117	g2.setStroke(new BasicStroke(1f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
118	g2.drawRoundRect(1, 1, getWidth() - 3, getHeight() - 3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
119	radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	g2 = (Graphics2D) graphics.create();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
125	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126	pintarLupa(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	if (getText().isEmpty() && !hasFocus() && !placeholder.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
128	g2.setFont(getFont());	Configura una propiedad mediante setFont en el objeto indicado.
129	g2.setColor(colorPlaceholder);	Configura una propiedad mediante setColor en el objeto indicado.
130	int base = (getHeight() + g2.getFontMetrics().getAscent()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	- g2.getFontMetrics().getDescent()) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132	g2.drawString(placeholder, 48, base);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	}	Cierra el bloque abierto anteriormente.
134	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
135	}	Cierra el bloque abierto anteriormente.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	private void pintarLupa(Graphics2D g2) {	Declara el método pintarLupa. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
138	g2.setColor(colorIcono);	Configura una propiedad mediante setColor en el objeto indicado.
139	g2.setStroke(new BasicStroke(	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
140	1.8f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	int centroY = getHeight() / 2 - 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
142	g2.drawOval(19, centroY - 7, 13, 13);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	g2.drawLine(30, centroY + 5, 36, centroY + 11);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	}	Cierra el bloque abierto anteriormente.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.

146	private static void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
147	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
148	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
150	RenderingHints.VALUE_TEXT_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	}	Cierra el bloque abierto anteriormente.
152	}	Cierra el bloque abierto anteriormente.

16.34 src/Componentes/CargaW.java

Componente visual reutilizable CargaW. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 317 | Archivo 34 de 157

Clases/tipos detectados: CargaW

Línea	Constructor o método detectado
40	public CargaW() {
49	private void cargarImagen() {
67	private BufferedImage recortarTransparencia(BufferedImage original) {
100	private BufferedImage recolorearImagen(BufferedImage original, Color color) {
130	private BufferedImage escalarAltaCalidad(
148	private void iniciarAnimacion() {
156	protected void paintComponent(Graphics graphics) {
222	private void pintarDestello(
263	private void aplicarCalidad(Graphics2D g2) {
286	public void setTamanoLogo(int tamanoLogo) {
292	public void setVelocidad(float velocidad) {
296	public void setAnchoLuz(int anchoLuz) {
300	public void setOpacidadBase(float opacidadBase) {
305	public void iniciar() {
312	public void detener() {

1 package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.AlphaComposite;	Importa java.awt.AlphaComposite para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
7 import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
8 import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
9 import java.awt.Shape;	Importa java.awt.Shape para poder usar esa clase sin escribir su nombre completo.
10 import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
11 import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
12 import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
13 import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
14 import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
15	Línea en blanco: separa visualmente bloques para que el código sea más legible.
16 /** Logo animado utilizado en la pantalla de carga. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

17	<code>public class CargaW extends JPanel {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	<code>private static final Color AMARILLO_W = new Color(255, 199, 44);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
20	<code>private static final int INTERVALO_ANIMACION_MS = 16;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
21	<code>private static final int UMBRAL_ALPHA = 88;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	<code>// A velocidad 1.0, el destello tarda dos segundos en recorrer la W.</code>	Comentario para humanos; Java no ejecuta este texto. A velocidad 1.0, el destello tarda dos segundos en recorrer la W.
24	<code>private static final double DURACION_VUELTA_SEGUNDOS = 2.0;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	<code>private BufferedImage imagenOriginal;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	<code>private BufferedImage imagenEscalada;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	<code>private int tamanoLogo = 184;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	<code>private int anchoLuz = 70;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31	<code>private float opacidadBase = 0.20f;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
32	<code>private float velocidad = 1.0f;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33	<code>private final int margen = 8;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	<code>private int ultimoAncho = -1;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36	<code>private int ultimoAlto = -1;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	<code>private long inicioAnimacionNanos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38	<code>private Timer timer;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	<code>public CargaW() {</code>	Constructor de CargaW: prepara un objeto nuevo y deja sus componentes listos para usarse.
41	<code>setOpaque(false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	<code>setDoubleBuffered(true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	<code>setPreferredSize(new Dimension(280, 180));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
44	<code>setMinimumSize(new Dimension(280, 180));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
45	<code>cargarImagen();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	<code>iniciarAnimacion();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	<code>}</code>	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	<code>private void cargarImagen() {</code>	Declara el método cargarImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
50	<code>try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
51	<code>java.net.URL ruta = CargaW.class.getResource("/Imágenes/CargaW.png");</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	<code>if (ruta == null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
54	<code>System.err.println("No se encontro /Imágenes/CargaW.png");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	<code>return;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
56	<code>}</code>	Cierra el bloque abierto anteriormente.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	<code>BufferedImage original = ImageIO.read(ruta);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	<code>BufferedImage recortada = recortarTransparencia(original);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	<code>imagenOriginal = recolorrearImagen(recortada, AMARILLO_W);</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

61	} catch (IOException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
62	System.err.println("Error leyendo Cargaw.png: " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	}	Cierra el bloque abierto anteriormente.
64	}	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	/** Retira el espacio transparente y el halo amplio del PNG original. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
67	private BufferedImage recortarTransparencia(BufferedImage original) {	Declara el método recortarTransparencia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
68	int minX = original.getWidth();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
69	int minY = original.getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
70	int maxX = -1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
71	int maxY = -1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	for (int y = 0; y < original.getHeight(); y++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
74	for (int x = 0; x < original.getWidth(); x++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
75	int alpha = (original.getRGB(x, y) >>> 24) & 0xFF;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	if (alpha > UMBRAL_ALPHA) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
77	minX = Math.min(minX, x);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
78	minY = Math.min(minY, y);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
79	maxX = Math.max(maxX, x);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
80	maxY = Math.max(maxY, y);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
81	}	Cierra el bloque abierto anteriormente.
82	}	Cierra el bloque abierto anteriormente.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	if (maxX < minX maxY < minY) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
86	return original;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
87	}	Cierra el bloque abierto anteriormente.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	BufferedImage recortada = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
90	maxX - minX + 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
91	maxY - minY + 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
92	BufferedImage.TYPE_INT_ARGB	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
93	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	Graphics2D g2 = recortada.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
95	g2.drawImage(original, -minX, -minY, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	return recortada;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
98	}	Cierra el bloque abierto anteriormente.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	private BufferedImage recolorearImagen(BufferedImage original, Color color) {	Declara el método recolorearImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
101	BufferedImage resultado = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
102	original.getWidth(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
103	original.getHeight(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

104	BufferedImage.TYPE_INT_ARGB	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
105	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	for (int y = 0; y < original.getHeight(); y++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
108	for (int x = 0; x < original.getWidth(); x++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
109	int alpha = (original.getRGB(x, y) >>> 24) & 0xFF;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
110	if (alpha <= UMBRAL_ALPHA) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
111	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	}	Cierra el bloque abierto anteriormente.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	int alphaLimpio = Math.min(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	255,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
116	Math.round((alpha - UMBRAL_ALPHA)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117	* 255f / (255 - UMBRAL_ALPHA))	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
118	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	int nuevoPixel = (alphaLimpio << 24)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
120	(color.getRed() << 16)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
121	(color.getGreen() << 8)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
122	color.getBlue();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	resultado.setRGB(x, y, nuevoPixel);	Configura una propiedad mediante setRGB en el objeto indicado.
124	}	Cierra el bloque abierto anteriormente.
125	}	Cierra el bloque abierto anteriormente.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
128	}	Cierra el bloque abierto anteriormente.
129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
130	private BufferedImage escalarAltaCalidad(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	BufferedImage original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	int anchoFinal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	int altoFinal) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	BufferedImage resultado = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
136	anchoFinal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	altoFinal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
138	BufferedImage.TYPE_INT_ARGB	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	Graphics2D g2 = resultado.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
141	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	g2.setComposite(AlphaComposite.Src);	Configura una propiedad mediante setComposite en el objeto indicado.
143	g2.drawImage(original, 0, 0, anchoFinal, altoFinal, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
145	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
146	}	Cierra el bloque abierto anteriormente.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.

148	private void iniciarAnimacion() {	Declara el método iniciarAnimacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
149	inicioAnimacionNanos = System.nanoTime();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
150	timer = new Timer(INTERVALO_ANIMACION_MS, e -> repaint());	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
151	timer.setCoalesce(true);	Configura una propiedad mediante setCoalesce en el objeto indicado.
152	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
153	}	Cierra el bloque abierto anteriormente.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
156	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
157	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	if (imagenOriginal == null getWidth() <= 0 getHeight() <= 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
160	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
164	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	double proporcion = (double) imagenOriginal.getHeight()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
167	/ imagenOriginal.getWidth();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	int anchoDeseado = Math.min(tamanoLogo, getWidth() - margen * 2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
169	int altoDeseado = (int) Math.round(anchoDeseado * proporcion);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
170	int altoMaximo = getHeight() - margen * 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	if (altoDeseado > altoMaximo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
173	altoDeseado = altoMaximo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
174	anchoDeseado = (int) Math.round(altoDeseado / proporcion);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
175	}	Cierra el bloque abierto anteriormente.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	if (anchoDeseado <= 0 altoDeseado <= 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
178	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
180	}	Cierra el bloque abierto anteriormente.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	if (imagenEscalada == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
183	anchoDeseado != ultimoAncho	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	altoDeseado != ultimoAlto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
185	imagenEscalada = escalarAltaCalidad(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
186	imagenOriginal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	anchoDeseado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	altoDeseado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	ultimoAncho = anchoDeseado;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
191	ultimoAlto = altoDeseado;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

192	}	Cierra el bloque abierto anteriormente.
193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
194	int x = (getWidth() - anchoDeseado) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
195	int y = (getHeight() - altoDeseado) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	g2.setComposite(AlphaComposite.getInstance(	Configura una propiedad mediante setComposite en el objeto indicado.
198	AlphaComposite.SRC_OVER,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	opacidadBase	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
200	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
201	g2.drawImage(imagenEscalada, x, y, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
203	double segundos = (System.nanoTime() - inicioAnimacionNanos)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
204	/ 1_000_000_000.0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
205	double duracionVuelta = DURACION_VUELTA_SEGUNDOS	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
206	/ Math.max(0.15f, velocidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
207	double progreso = (segundos % duracionVuelta) / duracionVuelta;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
208	int centroLuz = x + (int) Math.round(progreso * anchoDeseado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	pintarDestello(g2, centroLuz, x, y, anchoDeseado, altoDeseado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
211	pintarDestello(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212	g2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	centroLuz - anchoDeseado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	anchoDeseado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	altoDeseado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220	}	Cierra el bloque abierto anteriormente.
221		Línea en blanco: separa visualmente bloques para que el código sea más legible.
222	private void pintarDestello(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	Graphics2D g2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	int centroLuz,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	int xLogo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	int yLogo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	int anchoLogo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	int altoLogo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230	Shape clipOriginal = g2.getClip();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
231	int inicio = centroLuz - anchoLuz / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	for (int i = 0; i < anchoLuz; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
234	int xActual = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
235	if (xActual < xLogo xActual >= xLogo + anchoLogo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

236	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
237	}	Cierra el bloque abierto anteriormente.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	double distancia = Math.abs((i + 0.5) - anchoLuz / 2.0)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
240	/ (anchoLuz / 2.0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
241	float intensidad = (float) Math.pow(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
242	Math.max(0.0, 1.0 - distancia),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
243	1.35	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245	float alpha = intensidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
246		Línea en blanco: separa visualmente bloques para que el código sea más legible.
247	if (alpha <= 0.01f) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
248	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
249	}	Cierra el bloque abierto anteriormente.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	g2.setComposite(AlphaComposite.getInstance(	Configura una propiedad mediante setComposite en el objeto indicado.
252	AlphaComposite.SRC_OVER,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	alpha	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	g2.setClip(xActual, yLogo, 1, altoLogo);	Configura una propiedad mediante setClip en el objeto indicado.
256	g2.drawImage(imagenEscalada, xLogo, yLogo, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
257	}	Cierra el bloque abierto anteriormente.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.
259	g2.setClip(clipOriginal);	Configura una propiedad mediante setClip en el objeto indicado.
260	g2.setComposite(AlphaComposite.SrcOver);	Configura una propiedad mediante setComposite en el objeto indicado.
261	}	Cierra el bloque abierto anteriormente.
262		Línea en blanco: separa visualmente bloques para que el código sea más legible.
263	private void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
264	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
265	RenderingHints.KEY_INTERPOLATION,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
266	RenderingHints.VALUE_INTERPOLATION_BICUBIC	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
267	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
269	RenderingHints.KEY_RENDERING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
270	RenderingHints.VALUE_RENDER_QUALITY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
271	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
272	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
273	RenderingHints.KEY_ALPHA_INTERPOLATION,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	RenderingHints.VALUE_ALPHA_INTERPOLATION_QUALITY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
277	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
278	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
279	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

280	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
281	RenderingHints.KEY_STROKE_CONTROL,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	RenderingHints.VALUE_STROKE_NORMALIZE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
284	}	Cierra el bloque abierto anteriormente.
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	public void setTamanoLogo(int tamanoLogo) {	Declara el método setTamanoLogo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
287	this.tamanoLogo = Math.max(1, tamanoLogo);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
288	imagenEscalada = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
289	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
290	}	Cierra el bloque abierto anteriormente.
291		Línea en blanco: separa visualmente bloques para que el código sea más legible.
292	public void setVelocidad(float velocidad) {	Declara el método setVelocidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
293	this.velocidad = Math.max(0.15f, velocidad);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
294	}	Cierra el bloque abierto anteriormente.
295		Línea en blanco: separa visualmente bloques para que el código sea más legible.
296	public void setAnchoLuz(int anchoLuz) {	Declara el método setAnchoLuz. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
297	this.anchoLuz = Math.max(8, anchoLuz);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
298	}	Cierra el bloque abierto anteriormente.
299		Línea en blanco: separa visualmente bloques para que el código sea más legible.
300	public void setOpacidadBase(float opacidadBase) {	Declara el método setOpacidadBase. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
301	this.opacidadBase = Math.max(0f, Math.min(1f, opacidadBase));	Accede al atributo del objeto actual para asignarlo o utilizarlo.
302	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
303	}	Cierra el bloque abierto anteriormente.
304		Línea en blanco: separa visualmente bloques para que el código sea más legible.
305	public void iniciar() {	Declara el método iniciar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
306	inicioAnimacionNanos = System.nanoTime();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
307	if (timer != null && !timer.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
308	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	}	Cierra el bloque abierto anteriormente.
310	}	Cierra el bloque abierto anteriormente.
311		Línea en blanco: separa visualmente bloques para que el código sea más legible.
312	public void detener() {	Declara el método detener. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
313	if (timer != null && timer.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
314	timer.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
315	}	Cierra el bloque abierto anteriormente.
316	}	Cierra el bloque abierto anteriormente.
317	}	Cierra el bloque abierto anteriormente.

16.35 src/Componentes/DesplazamientoSuave.java

Componente visual reutilizable DesplazamientoSuave. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 89 | Archivo 35 de 157

Clases/tipos detectados: DesplazamientoSuave

Línea	Constructor o método detectado
18	private DesplazamientoSuave() {
22	public static void ocultarBarras(JScrollPane scroll) {
27	public static void instalarVertical(JScrollPane scroll) {
32	public static void instalarHorizontal(JScrollPane scroll) {
38	public static void instalarSobreVista(JComponent vista, JScrollBar barra,
47	private static void instalar(JScrollPane scroll, JScrollBar barra, boolean horizontal) {
58	public static void desplazar(JScrollBar barra, int distancia) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.event.MouseWheelEvent;	Importa java.awt.event.MouseWheelEvent para poder usar esa clase sin escribir su nombre completo.
4	import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
5	import java.util.WeakHashMap;	Importa java.util.WeakHashMap para poder usar esa clase sin escribir su nombre completo.
6	import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.JScrollBar;	Importa javax.swing.JScrollBar para poder usar esa clase sin escribir su nombre completo.
8	import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
9	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
10		Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	/** Desplaza una barra con una animación corta en lugar de dar saltos bruscos. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
12	public final class DesplazamientoSuave {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	private static final int DURACION_MILLIS = 180;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15	private static final Map<JScrollBar, Integer> objetivos = new WeakHashMap<>();	Crea una nueva instancia de WeakHashMap y la usa o guarda para continuar el proceso.
16	private static final Map<JScrollBar, Timer> animaciones = new WeakHashMap<>();	Crea una nueva instancia de WeakHashMap y la usa o guarda para continuar el proceso.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	private DesplazamientoSuave() {	Constructor de DesplazamientoSuave: prepara un objeto nuevo y deja sus componentes listos para usarse.
19	}	Cierra el bloque abierto anteriormente.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	/** Oculta las barras sin desactivar el desplazamiento del contenido. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	public static void ocultarBarras(JScrollPane scroll) {	Declara el método ocultarBarras. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
23	scroll.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
24	scroll.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
25	}	Cierra el bloque abierto anteriormente.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.

27	public static void instalarVertical(JScrollPane scroll) {	Declara el método instalarVertical. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28	ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	instalar(scroll, scroll.getVerticalScrollBar(), false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	}	Cierra el bloque abierto anteriormente.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	public static void instalarHorizontal(JScrollPane scroll) {	Declara el método instalarHorizontal. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
33	ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	instalar(scroll, scroll.getHorizontalScrollBar(), true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	}	Cierra el bloque abierto anteriormente.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	/** Conecta la rueda al contenido interno, donde normalmente se origina. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
38	public static void instalarSobreVista(JComponent vista, JScrollBar barra,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	boolean horizontal) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	vista.addMouseWheelListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	desplazar(barra, (int) Math.round(evento.getPreciseWheelRotation()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	* (horizontal ? 45 : 32)));	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
43	evento.consume();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	}	Cierra el bloque abierto anteriormente.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	private static void instalar(JScrollPane scroll, JScrollBar barra, boolean horizontal) {	Declara el método instalar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
48	scroll.addMouseWheelListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	if (horizontal) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
50	desplazar(barra, (int) Math.round(evento.getPreciseWheelRotation() * 45));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
52	desplazar(barra, (int) Math.round(evento.getPreciseWheelRotation() * 32));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	}	Cierra el bloque abierto anteriormente.
54	evento.consume();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	}	Cierra el bloque abierto anteriormente.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	public static void desplazar(JScrollBar barra, int distancia) {	Declara el método desplazar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
59	if (distancia == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
60	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
61	}	Cierra el bloque abierto anteriormente.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	int minimo = barra.getMinimum();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
64	int maximo = Math.max(minimo, barra.getMaximum() - barra.getVisibleAmount());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	int objetivo = objetivos.getDefault(barra, barra.getValue()) + distancia;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	objetivos.put(barra, Math.max(minimo, Math.min(maximo, objetivo)));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	Timer anterior = animaciones.get(barra);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
69	if (anterior != null && anterior.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
70	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

71	}	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	Timer animacion = new Timer(15, evento -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
74	int actual = barra.getValue();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
75	int destino = objetivos.getDefault(barra, actual);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	int diferencia = destino - actual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
77	if (Math.abs(diferencia) <= 1) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
78	barra.setValue(destino);	Configura una propiedad mediante setValue en el objeto indicado.
79	((Timer) evento.getSource()).stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	animaciones.remove(barra);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
82	}	Cierra el bloque abierto anteriormente.
83	barra.setValue(actual + (int) Math.round(diferencia * 0.24));	Configura una propiedad mediante setValue en el objeto indicado.
84	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	animacion.setCoalesce(true);	Configura una propiedad mediante setCoalesce en el objeto indicado.
86	animaciones.put(barra, animacion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	animacion.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	}	Cierra el bloque abierto anteriormente.
89	}	Cierra el bloque abierto anteriormente.

16.36 src/Componentes/PanelCircular.java

Componente visual reutilizable PanelCircular. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 126 | Archivo 36 de 157

Clases/tipos detectados: PanelCircular

Línea	Constructor o método detectado
22	public PanelCircular() {
29	public Color getColorFondo() {
33	public void setColorFondo(Color colorFondo) {
43	public Color getColorBorde() {
47	public void setColorBorde(Color colorBorde) {
57	public float getGrosorBorde() {
61	public void setGrosorBorde(float grosorBorde) {
69	protected void paintComponent(Graphics graphics) {
99	public boolean contains(int x, int y) {
111	private static void aplicarCalidad(Graphics2D g2) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** Contenedor circular para iconos, imágenes, avatares o indicadores. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
15	public class PanelCircular extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
16		Línea en blanco: separa visualmente bloques para que el código sea más legible.
17	private Color colorFondo = new Color(255, 246, 216);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
18	private Color colorBorde = new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
19	private float grosorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	/** Constructor vacío requerido por la Paleta de NetBeans. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	public PanelCircular() {	Constructor de PanelCircular: prepara un objeto nuevo y deja sus componentes listos para usarse.
23	setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.

24	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
25	setPreferredSize(new Dimension(140, 140));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
26	}	Cierra el bloque abierto anteriormente.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	@BeanProperty(preferred = true, description = "Color interior del círculo.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
29	public Color getColorFondo() {	Declara el método getColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
30	return colorFondo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
31	}	Cierra el bloque abierto anteriormente.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	public void setColorFondo(Color colorFondo) {	Declara el método setColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
34	Color anterior = this.colorFondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
35	this.colorFondo = colorFondo != null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
36	? colorFondo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	: new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
38	firePropertyChange("colorFondo", anterior, this.colorFondo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	}	Cierra el bloque abierto anteriormente.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	@BeanProperty(description = "Color del contorno del círculo.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
43	public Color getColorBorde() {	Declara el método getColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
44	return colorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
45	}	Cierra el bloque abierto anteriormente.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	public void setColorBorde(Color colorBorde) {	Declara el método setColorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
48	Color anterior = this.colorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
49	this.colorBorde = colorBorde != null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
50	? colorBorde	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	: new Color(0, 0, 0, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
52	firePropertyChange("colorBorde", anterior, this.colorBorde);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	@BeanProperty(description = "Grosor del contorno en píxeles.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
57	public float getGrosorBorde() {	Declara el método getGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
58	return grosorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
59	}	Cierra el bloque abierto anteriormente.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	public void setGrosorBorde(float grosorBorde) {	Declara el método setGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
62	float anterior = this.grosorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
63	this.grosorBorde = Math.max(0f, grosorBorde);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
64	firePropertyChange("grosorBorde", anterior, this.grosorBorde);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	}	Cierra el bloque abierto anteriormente.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.

68	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
69	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
70	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
71	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	int margen = Math.max(1, (int) Math.ceil(grosorBorde));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
74	int diametro = Math.max(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
75	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
76	Math.min(getWidth(), getHeight()) - margen * 2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
77	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	int x = (getWidth() - diametro) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	int y = (getHeight() - diametro) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	g2.setColor(colorFondo);	Configura una propiedad mediante setColor en el objeto indicado.
82	g2.fillOval(x, y, diametro, diametro);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	if (grosorBorde > 0f && colorBorde.getAlpha() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
85	g2.setStroke(new BasicStroke(grosorBorde));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
86	g2.setColor(colorBorde);	Configura una propiedad mediante setColor en el objeto indicado.
87	int ajuste = Math.max(1, Math.round(grosorBorde));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
88	g2.drawOval(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
89	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
90	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
91	Math.max(1, diametro - ajuste),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
92	Math.max(1, diametro - ajuste)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
93	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	}	Cierra el bloque abierto anteriormente.
95	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	}	Cierra el bloque abierto anteriormente.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
99	public boolean contains(int x, int y) {	Declara el método contains. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
100	double radioX = getWidth() / 2.0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
101	double radioY = getHeight() / 2.0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
102	if (radioX <= 0 radioY <= 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
103	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
104	}	Cierra el bloque abierto anteriormente.
105	double normalizadoX = (x - radioX) / radioX;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
106	double normalizadoY = (y - radioY) / radioY;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107	return normalizadoX * normalizadoX	Finaliza el método y devuelve el valor indicado a quien lo llamó.
108	+ normalizadoY * normalizadoY <= 1.0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	}	Cierra el bloque abierto anteriormente.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	private static void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

112	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
113	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
115	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
117	RenderingHints.KEY_RENDERING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
118	RenderingHints.VALUE_RENDER_QUALITY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
119	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
121	RenderingHints.KEY_STROKE_CONTROL,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
122	RenderingHints.VALUE_STROKE_PURE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
123	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	}	Cierra el bloque abierto anteriormente.
125	}	Cierra el bloque abierto anteriormente.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.37 src/Componentes/PanelFlotante.java

Componente visual reutilizable PanelFlotante. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 233 | Archivo 37 de 157

Clases/tipos detectados: PanelFlotante

Línea	Constructor o método detectado
32	public PanelFlotante() {
38	public Color getColorFondo() {
42	public void setColorFondo(Color colorFondo) {
50	public Color getColorFondoFinal() {
54	public void setColorFondoFinal(Color colorFondoFinal) {
62	public Color getColorBorde() {
66	public void setColorBorde(Color colorBorde) {
74	public Color getColorSombra() {
78	public void setColorSombra(Color colorSombra) {
86	public int getRadio() {
90	public void setRadio(int radio) {
98	public float getGrosorBorde() {
102	public void setGrosorBorde(float grosorBorde) {
110	public boolean isDegradado() {
114	public void setDegradado(boolean degradado) {
122	public boolean isSombra() {
126	public void setSombra(boolean sombra) {
134	public int getTamanoSombra() {
138	public void setTamanoSombra(int tamanoSombra) {
146	public int getDesplazamientoSombraY() {
150	public void setDesplazamientoSombraY(int desplazamientoSombraY) {
158	protected void paintComponent(Graphics graphics) {
202	private void pintarSombra(Graphics2D g2, int x, int y, int ancho, int alto) {
224	private static void aplicarCalidad(Graphics2D g2) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.

6	import java.awt.GradientPaint;	Importa java.awt.GradientPaint para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	* Contenedor reutilizable con esquinas redondeadas, borde, degradado y sombra.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	* Tiene constructor vacío para poder instalarse en la Paleta de NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
18	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
19	public class PanelFlotante extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private Color colorFondo = Color.WHITE;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	private Color colorFondoFinal = Color.WHITE;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	private Color colorBorde = new Color(225, 225, 225);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
24	private Color colorSombra = new Color(25, 32, 43, 35);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
25	private int radio = 24;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
26	private float grosorBorde = 1f;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	private boolean degradado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	private boolean sombra = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
29	private int tamanoSombra = 10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	private int desplazamientoSombraY = 5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	public PanelFlotante() {	Constructor de PanelFlotante: prepara un objeto nuevo y deja sus componentes listos para usarse.
33	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	setPreferredSize(new Dimension(320, 220));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
35	}	Cierra el bloque abierto anteriormente.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	@BeanProperty(preferred = true, description = "Color principal del fondo.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
38	public Color getColorFondo() {	Declara el método getColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
39	return colorFondo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
40	}	Cierra el bloque abierto anteriormente.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	public void setColorFondo(Color colorFondo) {	Declara el método setColorFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
43	Color anterior = this.colorFondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
44	this.colorFondo = colorFondo != null ? colorFondo : Color.WHITE;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
45	firePropertyChange("colorFondo", anterior, this.colorFondo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	@BeanProperty(description = "Segundo color utilizado cuando el degradado está activo.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

50	<code>public Color getColorFondoFinal() {</code>	Declara el método <code>getColorFondoFinal</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
51	<code> return colorFondoFinal;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
52	<code>}</code>	Cierra el bloque abierto anteriormente.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	<code>public void setColorFondoFinal(Color colorFondoFinal) {</code>	Declara el método <code>setColorFondoFinal</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
55	<code> Color anterior = this.colorFondoFinal;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
56	<code> this.colorFondoFinal = colorFondoFinal != null ? colorFondoFinal : Color.WHITE;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
57	<code> firePropertyChange("colorFondoFinal", anterior, this.colorFondoFinal);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	<code> repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	<code>}</code>	Cierra el bloque abierto anteriormente.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	<code>@BeanProperty(preferred = true, description = "Color del contorno.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
62	<code>public Color getColorBorde() {</code>	Declara el método <code>getColorBorde</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
63	<code> return colorBorde;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
64	<code>}</code>	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	<code>public void setColorBorde(Color colorBorde) {</code>	Declara el método <code>setColorBorde</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
67	<code> Color anterior = this.colorBorde;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	<code> this.colorBorde = colorBorde != null ? colorBorde : new Color(0, 0, 0, 0);</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
69	<code> firePropertyChange("colorBorde", anterior, this.colorBorde);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	<code> repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	<code>}</code>	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	<code>@BeanProperty(description = "Color de la sombra, incluido su nivel de transparencia.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
74	<code>public Color getColorSombra() {</code>	Declara el método <code>getColorSombra</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
75	<code> return colorSombra;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
76	<code>}</code>	Cierra el bloque abierto anteriormente.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	<code>public void setColorSombra(Color colorSombra) {</code>	Declara el método <code>setColorSombra</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
79	<code> Color anterior = this.colorSombra;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
80	<code> this.colorSombra = colorSombra != null ? colorSombra : new Color(0, 0, 0, 0);</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
81	<code> firePropertyChange("colorSombra", anterior, this.colorSombra);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	<code> repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	<code>}</code>	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	<code>@BeanProperty(preferred = true, description = "Radio de las esquinas en píxeles.")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
86	<code>public int getRadio() {</code>	Declara el método <code>getRadio</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
87	<code> return radio;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
88	<code>}</code>	Cierra el bloque abierto anteriormente.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	<code>public void setRadio(int radio) {</code>	Declara el método <code>setRadio</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
91	<code> int anterior = this.radio;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	<code> this.radio = Math.max(0, radio);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.

93	firePropertyChange("radio", anterior, this.radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	}	Cierra el bloque abierto anteriormente.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	@BeanProperty(description = "Grosor del contorno en píxeles.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
98	public float getGrosorBorde() {	Declara el método getGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
99	return grosorBorde;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
100	}	Cierra el bloque abierto anteriormente.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	public void setGrosorBorde(float grosorBorde) {	Declara el método setGrosorBorde. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
103	float anterior = this.grosorBorde;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
104	this.grosorBorde = Math.max(0f, grosorBorde);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
105	firePropertyChange("grosorBorde", anterior, this.grosorBorde);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107	}	Cierra el bloque abierto anteriormente.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	@BeanProperty(description = "Pinta el fondo usando los dos colores configurados.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
110	public boolean isDegradado() {	Declara el método isDegradado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
111	return degradado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
112	}	Cierra el bloque abierto anteriormente.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	public void setDegradado(boolean degradado) {	Declara el método setDegradado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
115	boolean anterior = this.degradado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
116	this.degradado = degradado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
117	firePropertyChange("degradado", anterior, degradado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	}	Cierra el bloque abierto anteriormente.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	@BeanProperty(preferred = true, description = "Activa la elevación visual del panel.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
122	public boolean isSombra() {	Declara el método isSombra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
123	return sombra;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
124	}	Cierra el bloque abierto anteriormente.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	public void setSombra(boolean sombra) {	Declara el método setSombra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
127	boolean anterior = this.sombra;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
128	this.sombra = sombra;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
129	firePropertyChange("sombra", anterior, sombra);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	}	Cierra el bloque abierto anteriormente.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	@BeanProperty(description = "Extensión visual de la sombra.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
134	public int getTamanoSombra() {	Declara el método getTamanoSombra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
135	return tamanoSombra;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
136	}	Cierra el bloque abierto anteriormente.

137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	public void setTamanoSombra(int tamanoSombra) {	Declara el método setTamanoSombra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
139	int anterior = this.tamanoSombra;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
140	this.tamanoSombra = Math.max(0, tamanoSombra);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
141	firePropertyChange("tamanoSombra", anterior, this.tamanoSombra);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	}	Cierra el bloque abierto anteriormente.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	@BeanProperty(description = "Desplazamiento vertical de la sombra.")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
146	public int getDesplazamientoSombraY() {	Declara el método getDesplazamientoSombraY. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
147	return desplazamientoSombraY;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
148	}	Cierra el bloque abierto anteriormente.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	public void setDesplazamientoSombraY(int desplazamientoSombraY) {	Declara el método setDesplazamientoSombraY. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
151	int anterior = this.desplazamientoSombraY;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
152	this.desplazamientoSombraY = desplazamientoSombraY;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
153	firePropertyChange("desplazamientoSombraY", anterior, desplazamientoSombraY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
158	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
159	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
160	aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	int margen = sombra ? Math.max(2, tamanoSombra) : 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
163	int x = margen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
164	int y = margen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
165	int ancho = Math.max(1, getWidth() - margen * 2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
166	int alto = Math.max(1, getHeight() - margen * 2 - Math.max(0, desplazamientoSombraY));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	if (sombra && tamanoSombra > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
169	pintarSombra(g2, x, y, ancho, alto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	}	Cierra el bloque abierto anteriormente.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	if (degradado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
173	g2.setPaint(new GradientPaint(	Crea una nueva instancia de GradientPaint y la usa o guarda para continuar el proceso.
174	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	colorFondo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	x + ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	y + alto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

179	colorFondoFinal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
182	g2.setColor(colorFondo);	Configura una propiedad mediante setColor en el objeto indicado.
183	}	Cierra el bloque abierto anteriormente.
184	g2.fillRoundRect(x, y, ancho, alto, radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185		Línea en blanco: separa visualmente bloques para que el código sea más legible.
186	if (grosorBorde > 0f && colorBorde.getAlpha() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
187	g2.setStroke(new BasicStroke(grosorBorde));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
188	g2.setColor(colorBorde);	Configura una propiedad mediante setColor en el objeto indicado.
189	int ajuste = Math.max(1, Math.round(grosorBorde));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
190	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	Math.max(1, ancho - ajuste),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	Math.max(1, alto - ajuste),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	radio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	radio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198	}	Cierra el bloque abierto anteriormente.
199	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
200	}	Cierra el bloque abierto anteriormente.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	private void pintarSombra(Graphics2D g2, int x, int y, int ancho, int alto) {	Declara el método pintarSombra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
203	int alphaBase = colorSombra.getAlpha();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
204	for (int nivel = tamanoSombra; nivel >= 1; nivel--) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
205	float proporcion = 1f - (nivel - 1f) / Math.max(1f, tamanoSombra);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
206	int alpha = Math.max(1, Math.round(alphaBase * proporcion * 0.32f));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
207	g2.setColor(new Color(	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
208	colorSombra.getRed(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	colorSombra.getGreen(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210	colorSombra.getBlue(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	alpha	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
213	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	x - nivel / 2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	y + desplazamientoSombraY + nivel / 2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	ancho + nivel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	alto + nivel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	radio + nivel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
219	radio + nivel	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
220	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
221	}	Cierra el bloque abierto anteriormente.
222	}	Cierra el bloque abierto anteriormente.

223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	private static void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
225	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
226	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227	g2.setRenderingHint(RenderingHints.KEY_RENDERING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
228	RenderingHints.VALUE_RENDER_QUALITY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
229	g2.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
230	RenderingHints.VALUE_STROKE_PURE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
231	}	Cierra el bloque abierto anteriormente.
232	}	Cierra el bloque abierto anteriormente.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.38 src/Componentes/PanelProductosVertical.java

Componente visual reutilizable PanelProductosVertical. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 78 | Archivo 38 de 157

Clases/tipos detectados: PanelProductosVertical

Línea	Constructor o método detectado
19	public PanelProductosVertical() {
31	public Dimension getPreferredSize() {
61	public Dimension getPreferredSizeScrollableViewportSize() { return getPreferredSize(); }
64	public int getScrollableUnitIncrement(Rectangle visibleRect, int orientation, int direction) {
69	public int getScrollableBlockIncrement(Rectangle visibleRect, int orientation, int direction) {
74	public boolean getScrollableTracksViewportWidth() { return true; }
77	public boolean getScrollableTracksViewportHeight() { return false; }

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Rectangle;	Importa java.awt.Rectangle para poder usar esa clase sin escribir su nombre completo.
6	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.Scrollable;	Importa javax.swing.Scrollable para poder usar esa clase sin escribir su nombre completo.
8	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	/** Panel de tarjetas que se ajusta al ancho y solo crece verticalmente. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class PanelProductosVertical extends JPanel implements Scrollable {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** Máximo de tarjetas por fila para conservar un layout predecible. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	private static final int MAX_COLUMNAS = 6;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15	// Evita que entren siete tarjetas en la vista de 1810 px.	Comentario para humanos; Java no ejecuta este texto. Evita que entren siete tarjetas en la vista de 1810 px.
16	private static final int ESPACIO_HORIZONTAL = 40;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
17	private static final int ESPACIO_VERTICAL = 15;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	public PanelProductosVertical() {	Constructor de PanelProductosVertical: prepara un objeto nuevo y deja sus componentes listos para usarse.
20	super(new FlowLayout(FlowLayout.LEFT, ESPACIO_HORIZONTAL, ESPACIO_VERTICAL));	Crea una nueva instancia de FlowLayout y la usa o guarda para continuar el proceso.
21	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
22	}	Cierra el bloque abierto anteriormente.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	* FlowLayout calcula por defecto una sola fila cuando el panel todavía	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
26	* no tiene un ancho definitivo. Eso hace que el JScrollPane crea que no	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

27	<code>* existe contenido vertical. Calculamos aquí la altura real del panel</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
28	<code>* según el ancho visible y la cantidad de tarjetas.</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
29	<code>*/</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
30	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
31	<code>public Dimension getPreferredSize() {</code>	Declara el método <code>getPreferredSize</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
32	<code> if (!(getLayout() instanceof FlowLayout) getComponentCount() == 0) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
33	<code> return super.getPreferredSize();</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
34	<code> }</code>	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	<code> int anchoDisponible = getWidth();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	<code> if (anchoDisponible <= 0 && getParent() != null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
38	<code> anchoDisponible = getParent().getWidth();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
39	<code> }</code>	Cierra el bloque abierto anteriormente.
40	<code> if (anchoDisponible <= 0) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
41	<code> anchoDisponible = 1_810;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
42	<code> }</code>	Cierra el bloque abierto anteriormente.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	<code> int anchoTarjeta = 230;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	<code> if (getComponentCount() > 0</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
46	<code> && getComponent(0).getPreferredSize().width > 0) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	<code> anchoTarjeta = getComponent(0).getPreferredSize().width;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
48	<code> }</code>	Cierra el bloque abierto anteriormente.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	<code> int columnas = Math.max(1, Math.min(MAX_COLUMNAS,</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
51	<code> (anchoDisponible + ESPACIO_HORIZONTAL)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	<code> / (anchoTarjeta + ESPACIO_HORIZONTAL)));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	<code> int filas = (getComponentCount() + columnas - 1) / columnas;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
54	<code> int altoTarjeta = getComponent(0).getPreferredSize().height;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	<code> return new Dimension(anchoDisponible,</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
57	<code> ESPACIO_VERTICAL + filas * (altoTarjeta + ESPACIO_VERTICAL));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	<code> }</code>	Cierra el bloque abierto anteriormente.
59		Línea en blanco: separa visualmente bloques para que el código sea más legible.
60	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
61	<code>public Dimension getPreferredSizeScrollableViewportSize() { return getPreferredSize(); }</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
64	<code>public int getScrollableUnitIncrement(Rectangle visibleRect, int orientation, int direction) {</code>	Declara el método <code>getScrollableUnitIncrement</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
65	<code> return orientation == SwingConstants.VERTICAL ? 32 : 0;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
66	<code>}</code>	Cierra el bloque abierto anteriormente.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
69	<code>public int getScrollableBlockIncrement(Rectangle visibleRect, int orientation, int direction) {</code>	Declara el método <code>getScrollableBlockIncrement</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

70	return orientation == SwingConstants.VERTICAL ? Math.max(visibleRect.height - 32, 32) : 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
71	}	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
74	public boolean getScrollableTracksViewportWidth() { return true; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
77	public boolean getScrollableTracksViewportHeight() { return false; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	}	Cierra el bloque abierto anteriormente.

16.39 src/Componentes/Scroll_Categorias.java

Componente visual reutilizable Scroll_Categorias. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 465 | Archivo 39 de 157

Clases/tipos detectados: Scroll_Categorias, BarraNaranja

Línea	Constructor o método detectado
28	public Scroll_Categorias() {
147	public void setPanelCategorias(
208	public void agregarCategoria(
239	public JPanel getPanelCategorias() {
249	public void irAllnicio() {
292	protected Dimension getMinimumThumbSize() {
302	protected Dimension getMaximumThumbSize() {
316	protected JButton createDecreaseButton(
328	protected JButton createIncreaseButton(
339	private JButton crearBotonInvisible() {
399	protected void paintThumb(
457	protected void paintTrack(

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.ComponentOrientation;	Importa java.awt.ComponentOrientation para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Point;	Importa java.awt.Point para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.Rectangle;	Importa java.awt.Rectangle para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
13	import javax.swing.Box;	Importa javax.swing.Box para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.JScrollBar;	Importa javax.swing.JScrollBar para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
20	import javax.swing.plaf.basic.BasicScrollBarUI;	Importa javax.swing.plaf.basic.BasicScrollBarUI para poder usar esa clase sin escribir su nombre completo.

21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	<code>public class Scroll_Categorias extends JScrollPane {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	<code>// Este panel ahora vendrá desde GUI_CAJERO</code>	Comentario para humanos; Java no ejecuta este texto. Este panel ahora vendrá desde GUI_CAJERO
25	<code>private JPanel panelCategorias;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	<code>public Scroll_Categorias() {</code>	Constructor de Scroll_Categorias: prepara un objeto nuevo y deja sus componentes listos para usarse.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
32	<code>// ORIENTACIÓN IZQUIERDA A DERECHA</code>	Comentario para humanos; Java no ejecuta este texto. ORIENTACIÓN IZQUIERDA A DERECHA
33	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	<code>setComponentOrientation(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	<code> ComponentOrientation.LEFT_TO_RIGHT</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	<code>getViewport().setComponentOrientation(</code>	Configura una propiedad mediante setComponentOrientation en el objeto indicado.
41	<code> ComponentOrientation.LEFT_TO_RIGHT</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	<code>getViewport().setBackground(</code>	Configura una propiedad mediante setBackground en el objeto indicado.
46	<code> Color.WHITE</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
51	<code>// CONFIGURACIÓN DEL SCROLLPANE</code>	Comentario para humanos; Java no ejecuta este texto. CONFIGURACIÓN DEL SCROLLPANE
52	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	<code>setHorizontalScrollBarPolicy(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	<code> JScrollPane.HORIZONTAL_SCROLLBAR_NEVER</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	<code>setVerticalScrollBarPolicy(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	<code> JScrollPane.VERTICAL_SCROLLBAR_NEVER</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	<code>// Quitar borde exterior</code>	Comentario para humanos; Java no ejecuta este texto. Quitar borde exterior

65	setBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	BorderFactory.createEmptyBorder()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	// Quitar borde interno	Comentario para humanos; Java no ejecuta este texto. Quitar borde interno
71	setViewportBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	BorderFactory.createEmptyBorder()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	setBackground(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
77	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	// =====	Comentario para humanos; Java no ejecuta este texto. =====
82	// BARRA HORIZONTAL	Comentario para humanos; Java no ejecuta este texto. BARRA HORIZONTAL
83	// =====	Comentario para humanos; Java no ejecuta este texto. =====
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	JScrollBar barra =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
86	getHorizontalScrollBar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87		Línea en blanco: separa visualmente bloques para que el código sea más legible.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	barra.setComponentOrientation(	Configura una propiedad mediante setComponentOrientation en el objeto indicado.
90	ComponentOrientation.LEFT_TO_RIGHT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
91	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	// Quitar borde	Comentario para humanos; Java no ejecuta este texto. Quitar borde
95	barra.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
96	BorderFactory.createEmptyBorder()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	// Fondo transparente	Comentario para humanos; Java no ejecuta este texto. Fondo transparente
101	barra.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	barra.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
105	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
106	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.

109	// Altura del espacio donde está la barra	Comentario para humanos; Java no ejecuta este texto. Altura del espacio donde está la barra
110	barra.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
111	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
112	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	12	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
115	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	// Velocidad del scroll	Comentario para humanos; Java no ejecuta este texto. Velocidad del scroll
119	barra.setUnitIncrement(	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
120	25	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
121	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	barra.setBlockIncrement(	Configura una propiedad mediante setBlockIncrement en el objeto indicado.
125	120	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	// Aplicar diseño	Comentario para humanos; Java no ejecuta este texto. Aplicar diseño
130	barra.setUI(	Configura una propiedad mediante setUI en el objeto indicado.
131	new BarraNaranja()	Crea una nueva instancia de BarraNaranja y la usa o guarda para continuar el proceso.
132	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	// =====	Comentario para humanos; Java no ejecuta este texto. =====
136	// RUEDA DEL MOUSE	Comentario para humanos; Java no ejecuta este texto. RUEDA DEL MOUSE
137	// =====	Comentario para humanos; Java no ejecuta este texto. =====
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	DesplazamientoSuave.instalarHorizontal(this);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	}	Cierra el bloque abierto anteriormente.
141		Línea en blanco: separa visualmente bloques para que el código sea más legible.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	// =====	Comentario para humanos; Java no ejecuta este texto. =====
144	// METER CATEGORIASPANEL DENTRO DEL SCROLL	Comentario para humanos; Java no ejecuta este texto. METER CATEGORIASPANEL DENTRO DEL SCROLL
145	// =====	Comentario para humanos; Java no ejecuta este texto. =====
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	public void setPanelCategorias(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
148	JPanel panel) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	this.panelCategorias =	Accede al atributo del objeto actual para asignarlo o utilizarlo.
152	panel;	Ejecuta una instrucción Java completa; el punto y coma marca su final.

153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	panelCategorias.setComponentOrientation(	Configura una propiedad mediante setComponentOrientation en el objeto indicado.
156	ComponentOrientation.LEFT_TO_RIGHT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159		Línea en blanco: separa visualmente bloques para que el código sea más legible.
160	// Meter el panel visual dentro del scroll	Comentario para humanos; Java no ejecuta este texto. Meter el panel visual dentro del scroll
161	setViewportView(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
162	panelCategorias	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
163	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	DesplazamientoSuave.instalarSobreVista(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	panelCategorias,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	getHorizontalScrollBar(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
168	true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
169	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	panelCategorias.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	panelCategorias.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
175		Línea en blanco: separa visualmente bloques para que el código sea más legible.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
179	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	// Iniciar desde la izquierda	Comentario para humanos; Java no ejecuta este texto. Iniciar desde la izquierda
183	SwingUtilities.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	getViewport().setViewPosition(	Configura una propiedad mediante setViewPosition en el objeto indicado.
186	new Point(	Crea una nueva instancia de Point y la usa o guarda para continuar el proceso.
187	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	JScrollBar barra =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
194	getHorizontalScrollBar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.

197	barra.setValue(	Configura una propiedad mediante setValue en el objeto indicado.
198	barra.getMinimum()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
200	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
201	}	Cierra el bloque abierto anteriormente.
202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	// =====	Comentario para humanos; Java no ejecuta este texto. =====
205	// AGREGAR CATEGORÍAS	Comentario para humanos; Java no ejecuta este texto. AGREGAR CATEGORÍAS
206	// =====	Comentario para humanos; Java no ejecuta este texto. =====
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	public void agregarCategoria(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	JComponent componente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	if (panelCategorias == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
213	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
214	}	Cierra el bloque abierto anteriormente.
215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	panelCategorias.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	componente	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
219	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221		Línea en blanco: separa visualmente bloques para que el código sea más legible.
222	panelCategorias.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	Box.createHorizontalStrut(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	15	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228		Línea en blanco: separa visualmente bloques para que el código sea más legible.
229	panelCategorias.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	panelCategorias.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
232	}	Cierra el bloque abierto anteriormente.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234		Línea en blanco: separa visualmente bloques para que el código sea más legible.
235	// =====	Comentario para humanos; Java no ejecuta este texto. =====
236	// OBTENER PANEL	Comentario para humanos; Java no ejecuta este texto. OBTENER PANEL
237	// =====	Comentario para humanos; Java no ejecuta este texto. =====
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	public JPanel getPanelCategorias() {	Declara el método getPanelCategorias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
240		Línea en blanco: separa visualmente bloques para que el código sea más legible.

241	return panelCategorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
242	}	Cierra el bloque abierto anteriormente.
243		Línea en blanco: separa visualmente bloques para que el código sea más legible.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	// =====	Comentario para humanos; Java no ejecuta este texto. =====
246	// REGRESAR AL INICIO	Comentario para humanos; Java no ejecuta este texto. REGRESAR AL INICIO
247	// =====	Comentario para humanos; Java no ejecuta este texto. =====
248		Línea en blanco: separa visualmente bloques para que el código sea más legible.
249	public void irAlInicio() {	Declara el método irAlInicio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251		Línea en blanco: separa visualmente bloques para que el código sea más legible.
252	JScrollBar barra =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
253	getHorizontalScrollBar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	barra.setValue(	Configura una propiedad mediante setValue en el objeto indicado.
257	barra.getMinimum()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
259		Línea en blanco: separa visualmente bloques para que el código sea más legible.
260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	getViewport().setViewPosition(	Configura una propiedad mediante setViewPosition en el objeto indicado.
262	new Point(	Crea una nueva instancia de Point y la usa o guarda para continuar el proceso.
263	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
264	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
266	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
267	}	Cierra el bloque abierto anteriormente.
268		Línea en blanco: separa visualmente bloques para que el código sea más legible.
269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
270	// =====	Comentario para humanos; Java no ejecuta este texto. =====
271	// DISEÑO DE LA BARRA	Comentario para humanos; Java no ejecuta este texto. DISEÑO DE LA BARRA
272	// =====	Comentario para humanos; Java no ejecuta este texto. =====
273		Línea en blanco: separa visualmente bloques para que el código sea más legible.
274	private static class BarraNaranja	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
275	extends BasicScrollBarUI {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
278	// Color naranja	Comentario para humanos; Java no ejecuta este texto. Color naranja
279	private final Color naranja =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
280	new Color(	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
281	255,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	174,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
284	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	// =====	Comentario para humanos; Java no ejecuta este texto. =====
288	// TAMAÑO REAL DE LA BARRA	Comentario para humanos; Java no ejecuta este texto. TAMAÑO REAL DE LA BARRA
289	// =====	Comentario para humanos; Java no ejecuta este texto. =====
290		Línea en blanco: separa visualmente bloques para que el código sea más legible.
291	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
292	protected Dimension getMinimumThumbSize() {	Declara el método getMinimumThumbSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
293		Línea en blanco: separa visualmente bloques para que el código sea más legible.
294	return new Dimension(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
295	70,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
296	6	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
297	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	}	Cierra el bloque abierto anteriormente.
299		Línea en blanco: separa visualmente bloques para que el código sea más legible.
300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
301	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
302	protected Dimension getMaximumThumbSize() {	Declara el método getMaximumThumbSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
303		Línea en blanco: separa visualmente bloques para que el código sea más legible.
304	return new Dimension(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
305	70,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	6	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
307	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
308	}	Cierra el bloque abierto anteriormente.
309		Línea en blanco: separa visualmente bloques para que el código sea más legible.
310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
311	// =====	Comentario para humanos; Java no ejecuta este texto. =====
312	// QUITAR FLECHA IZQUIERDA	Comentario para humanos; Java no ejecuta este texto. QUITAR FLECHA IZQUIERDA
313	// =====	Comentario para humanos; Java no ejecuta este texto. =====
314		Línea en blanco: separa visualmente bloques para que el código sea más legible.
315	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
316	protected JButton createDecreaseButton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
317	int orientation) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318		Línea en blanco: separa visualmente bloques para que el código sea más legible.
319	return crearBotonInvisible();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
320	}	Cierra el bloque abierto anteriormente.
321		Línea en blanco: separa visualmente bloques para que el código sea más legible.
322		Línea en blanco: separa visualmente bloques para que el código sea más legible.
323	// =====	Comentario para humanos; Java no ejecuta este texto. =====
324	// QUITAR FLECHA DERECHA	Comentario para humanos; Java no ejecuta este texto. QUITAR FLECHA DERECHA
325	// =====	Comentario para humanos; Java no ejecuta este texto. =====
326		Línea en blanco: separa visualmente bloques para que el código sea más legible.
327	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

328	protected JButton createIncreaseButton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
329	int orientation) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330		Línea en blanco: separa visualmente bloques para que el código sea más legible.
331	return crearBotonInvisible();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
332	}	Cierra el bloque abierto anteriormente.
333		Línea en blanco: separa visualmente bloques para que el código sea más legible.
334		Línea en blanco: separa visualmente bloques para que el código sea más legible.
335	// =====	Comentario para humanos; Java no ejecuta este texto. =====
336	// BOTÓN INVISIBLE	Comentario para humanos; Java no ejecuta este texto. BOTÓN INVISIBLE
337	// =====	Comentario para humanos; Java no ejecuta este texto. =====
338		Línea en blanco: separa visualmente bloques para que el código sea más legible.
339	private JButton crearBotonInvisible() {	Declara el método crearBotonInvisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
340		Línea en blanco: separa visualmente bloques para que el código sea más legible.
341		Línea en blanco: separa visualmente bloques para que el código sea más legible.
342	JButton boton =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
343	new JButton();	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
346	boton.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
347	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
348	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
349	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
350	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
351	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
352		Línea en blanco: separa visualmente bloques para que el código sea más legible.
353		Línea en blanco: separa visualmente bloques para que el código sea más legible.
354	boton.setMinimumSize(	Configura una propiedad mediante setMinimumSize en el objeto indicado.
355	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
356	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
358	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
360		Línea en blanco: separa visualmente bloques para que el código sea más legible.
361		Línea en blanco: separa visualmente bloques para que el código sea más legible.
362	boton.setMaximumSize(	Configura una propiedad mediante setMaximumSize en el objeto indicado.
363	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
364	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
366	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
367	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368		Línea en blanco: separa visualmente bloques para que el código sea más legible.
369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
370	boton.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
371	BorderFactory.createEmptyBorder()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

372	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
373		Línea en blanco: separa visualmente bloques para que el código sea más legible.
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.
375	boton.setContentAreaFilled(	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
376	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
377	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379		Línea en blanco: separa visualmente bloques para que el código sea más legible.
380	boton.setFocusPainted(	Configura una propiedad mediante setFocusPainted en el objeto indicado.
381	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
382	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
383		Línea en blanco: separa visualmente bloques para que el código sea más legible.
384		Línea en blanco: separa visualmente bloques para que el código sea más legible.
385	boton.setOpaque(	Configura una propiedad mediante setOpaque en el objeto indicado.
386	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
388		Línea en blanco: separa visualmente bloques para que el código sea más legible.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
391	}	Cierra el bloque abierto anteriormente.
392		Línea en blanco: separa visualmente bloques para que el código sea más legible.
393		Línea en blanco: separa visualmente bloques para que el código sea más legible.
394	// =====	Comentario para humanos; Java no ejecuta este texto. =====
395	// DIBUJAR BARRA NARANJA	Comentario para humanos; Java no ejecuta este texto. DIBUJAR BARRA NARANJA
396	// =====	Comentario para humanos; Java no ejecuta este texto. =====
397		Línea en blanco: separa visualmente bloques para que el código sea más legible.
398	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
399	protected void paintThumb(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
400	Graphics g,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
401	JComponent c,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
402	Rectangle thumbBounds) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404		Línea en blanco: separa visualmente bloques para que el código sea más legible.
405	if (thumbBounds.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
406		Línea en blanco: separa visualmente bloques para que el código sea más legible.
407	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
408	}	Cierra el bloque abierto anteriormente.
409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
410		Línea en blanco: separa visualmente bloques para que el código sea más legible.
411	Graphics2D g2 =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
412	(Graphics2D) g.create();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
414		Línea en blanco: separa visualmente bloques para que el código sea más legible.
415	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.

416	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
417	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
419		Línea en blanco: separa visualmente bloques para que el código sea más legible.
420		Línea en blanco: separa visualmente bloques para que el código sea más legible.
421	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
422	naranja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
423	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
424		Línea en blanco: separa visualmente bloques para que el código sea más legible.
425		Línea en blanco: separa visualmente bloques para que el código sea más legible.
426	int alto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
427	6;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
428		Línea en blanco: separa visualmente bloques para que el código sea más legible.
429		Línea en blanco: separa visualmente bloques para que el código sea más legible.
430	int y =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
431	thumbBounds.y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
432	+ (	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
433	thumbBounds.height	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
434	- alto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
435	) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
436		Línea en blanco: separa visualmente bloques para que el código sea más legible.
437		Línea en blanco: separa visualmente bloques para que el código sea más legible.
438	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439	thumbBounds.x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
440	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
441	thumbBounds.width,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
442	alto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
446		Línea en blanco: separa visualmente bloques para que el código sea más legible.
447		Línea en blanco: separa visualmente bloques para que el código sea más legible.
448	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
449	}	Cierra el bloque abierto anteriormente.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451		Línea en blanco: separa visualmente bloques para que el código sea más legible.
452	// =====	Comentario para humanos; Java no ejecuta este texto. =====
453	// NO DIBUJAR FONDO DEL TRACK	Comentario para humanos; Java no ejecuta este texto. NO DIBUJAR FONDO DEL TRACK
454	// =====	Comentario para humanos; Java no ejecuta este texto. =====
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
457	protected void paintTrack(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
458	Graphics g,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459	JComponent c,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

460	Rectangle trackBounds) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461		Línea en blanco: separa visualmente bloques para que el código sea más legible.
462	// Vacío intencionalmente	Comentario para humanos; Java no ejecuta este texto. Vacío intencionalmente
463	}	Cierra el bloque abierto anteriormente.
464	}	Cierra el bloque abierto anteriormente.
465	}	Cierra el bloque abierto anteriormente.

16.40 src/Componentes/Scroll_Vertical.java

Componente visual reutilizable Scroll_Vertical. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 289 | Archivo 40 de 157

Clases/tipos detectados: Scroll_Vertical, BarraNaranjaVertical

Línea	Constructor o método detectado
24	public Scroll_Vertical() {
87	public void setPanelContenido(JPanel panel) {
125	public void agregarComponente(
142	public JPanel getPanelContenido() {
150	public void irArriba() {
176	protected Dimension getMinimumThumbSize() {
186	protected Dimension getMaximumThumbSize() {
196	protected JButton createDecreaseButton(
204	protected JButton createIncreaseButton(
210	private JButton crearBotonInvisible() {
242	protected void paintThumb(
281	protected void paintTrack(

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Point;	Importa java.awt.Point para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Rectangle;	Importa java.awt.Rectangle para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
10		Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
13	import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.JScrollBar;	Importa javax.swing.JScrollBar para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.plaf.basic.BasicScrollBarUI;	Importa javax.swing.plaf.basic.BasicScrollBarUI para poder usar esa clase sin escribir su nombre completo.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	public class Scroll_Vertical extends JScrollPane {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.

21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	<code>private JPanel panelContenido;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	<code>public Scroll_Vertical() {</code>	Constructor de Scroll_Vertical: prepara un objeto nuevo y deja sus componentes listos para usarse.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	<code>// Fondo blanco</code>	Comentario para humanos; Java no ejecuta este texto. Fondo blanco
27	<code>getViewport().setBackground(Color.WHITE);</code>	Configura una propiedad mediante setBackground en el objeto indicado.
28	<code>setBackground(Color.WHITE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
31	<code>// CONFIGURACIÓN DEL SCROLL</code>	Comentario para humanos; Java no ejecuta este texto. CONFIGURACIÓN DEL SCROLL
32	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	<code>// No queremos scroll horizontal</code>	Comentario para humanos; Java no ejecuta este texto. No queremos scroll horizontal
35	<code>setHorizontalScrollBarPolicy(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	<code> JScrollPane.HORIZONTAL_SCROLLBAR_NEVER</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	<code>// El contenido conserva el desplazamiento, pero sin mostrar la barra.</code>	Comentario para humanos; Java no ejecuta este texto. El contenido conserva el desplazamiento, pero sin mostrar la barra.
40	<code>setVerticalScrollBarPolicy(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	<code> JScrollPane.VERTICAL_SCROLLBAR_NEVER</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	<code>// Quitar bordes</code>	Comentario para humanos; Java no ejecuta este texto. Quitar bordes
45	<code>setBorder(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	<code> BorderFactory.createEmptyBorder()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	<code>setViewportBorder(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	<code> BorderFactory.createEmptyBorder()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
54	<code>// BARRA VERTICAL</code>	Comentario para humanos; Java no ejecuta este texto. BARRA VERTICAL
55	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	<code>JScrollBar barra = getVerticalScrollBar();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	<code>barra.setBorder(</code>	Configura una propiedad mediante setBorder en el objeto indicado.
60	<code> BorderFactory.createEmptyBorder()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	<code>barra.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
64	<code>barra.setBackground(Color.WHITE);</code>	Configura una propiedad mediante setBackground en el objeto indicado.

65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	// Ancho de la barra	Comentario para humanos; Java no ejecuta este texto. Ancho de la barra
67	barra.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
68	new Dimension(12, 0)	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
69	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	// Velocidad del scroll	Comentario para humanos; Java no ejecuta este texto. Velocidad del scroll
72	barra.setUnitIncrement(25);	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
73	barra.setBlockIncrement(120);	Configura una propiedad mediante setBlockIncrement en el objeto indicado.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	// Diseño personalizado	Comentario para humanos; Java no ejecuta este texto. Diseño personalizado
76	barra.setUI(	Configura una propiedad mediante setUI en el objeto indicado.
77	new BarraNaranjaVertical()	Crea una nueva instancia de BarraNaranjaVertical y la usa o guarda para continuar el proceso.
78	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	DesplazamientoSuave.instalarVertical(this);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	}	Cierra el bloque abierto anteriormente.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	// =====	Comentario para humanos; Java no ejecuta este texto. =====
84	// COLOCAR PANEL DENTRO DEL SCROLL	Comentario para humanos; Java no ejecuta este texto. COLOCAR PANEL DENTRO DEL SCROLL
85	// =====	Comentario para humanos; Java no ejecuta este texto. =====
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	public void setPanelContenido(JPanel panel) {	Declara el método setPanelContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	this.panelContenido = panel;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	setViewportView(panelContenido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	DesplazamientoSuave.instalarSobreVista(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
94	panelContenido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	getVerticalScrollBar(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	panelContenido.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	panelContenido.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	// Empezar arriba	Comentario para humanos; Java no ejecuta este texto. Empezar arriba
106	SwingUtilities.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	getViewport().setViewPosition(	Configura una propiedad mediante setViewPosition en el objeto indicado.

109	new Point(0, 0)	Crea una nueva instancia de Point y la usa o guarda para continuar el proceso.
110	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	JScrollBar barra =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
113	getVerticalScrollBar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	barra.setValue(	Configura una propiedad mediante setValue en el objeto indicado.
116	barra.getMinimum()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	}	Cierra el bloque abierto anteriormente.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	// =====	Comentario para humanos; Java no ejecuta este texto. =====
122	// AGREGAR COMPONENTES	Comentario para humanos; Java no ejecuta este texto. AGREGAR COMPONENTES
123	// =====	Comentario para humanos; Java no ejecuta este texto. =====
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	public void agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	JComponent componente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	if (panelContenido == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
129	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
130	}	Cierra el bloque abierto anteriormente.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	panelContenido.add(componente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	panelContenido.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
135	panelContenido.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	}	Cierra el bloque abierto anteriormente.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	// =====	Comentario para humanos; Java no ejecuta este texto. =====
139	// OBTENER PANEL	Comentario para humanos; Java no ejecuta este texto. OBTENER PANEL
140	// =====	Comentario para humanos; Java no ejecuta este texto. =====
141		Línea en blanco: separa visualmente bloques para que el código sea más legible.
142	public JPanel getPanelContenido() {	Declara el método getPanelContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
143	return panelContenido;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
144	}	Cierra el bloque abierto anteriormente.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	// =====	Comentario para humanos; Java no ejecuta este texto. =====
147	// REGRESAR ARRIBA	Comentario para humanos; Java no ejecuta este texto. REGRESAR ARRIBA
148	// =====	Comentario para humanos; Java no ejecuta este texto. =====
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	public void irArriba() {	Declara el método irArriba. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	JScrollBar barra =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

153	getVerticalScrollBar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	barra.setValue(	Configura una propiedad mediante setValue en el objeto indicado.
156	barra.getMinimum()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	getViewport().setViewPosition(	Configura una propiedad mediante setViewPosition en el objeto indicado.
160	new Point(0, 0)	Crea una nueva instancia de Point y la usa o guarda para continuar el proceso.
161	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	// =====	Comentario para humanos; Java no ejecuta este texto. =====
165	// DISEÑO DE BARRA VERTICAL	Comentario para humanos; Java no ejecuta este texto. DISEÑO DE BARRA VERTICAL
166	// =====	Comentario para humanos; Java no ejecuta este texto. =====
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	private static class BarraNaranjaVertical	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
169	extends BasicScrollBarUI {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	private final Color naranja =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
172	new Color(255, 174, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	// Tamaño mínimo del indicador	Comentario para humanos; Java no ejecuta este texto. Tamaño mínimo del indicador
175	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
176	protected Dimension getMinimumThumbSize() {	Declara el método getMinimumThumbSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	return new Dimension(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
179	6,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	70	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
181	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
182	}	Cierra el bloque abierto anteriormente.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	// Tamaño máximo del indicador	Comentario para humanos; Java no ejecuta este texto. Tamaño máximo del indicador
185	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
186	protected Dimension getMaximumThumbSize() {	Declara el método getMaximumThumbSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	return new Dimension(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
189	6,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	70	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
192	}	Cierra el bloque abierto anteriormente.
193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
194	// Quitar flecha superior	Comentario para humanos; Java no ejecuta este texto. Quitar flecha superior
195	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

196	protected JButton createDecreaseButton(197 int orientation) { 198 199 return crearBotonInvisible(); 200 }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Línea en blanco: separa visualmente bloques para que el código sea más legible. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	// Quitar flecha inferior	Comentario para humanos; Java no ejecuta este texto. Quitar flecha inferior
203	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
204	protected JButton createIncreaseButton(205 int orientation) { 206 207 return crearBotonInvisible(); 208 }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Línea en blanco: separa visualmente bloques para que el código sea más legible. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	private JButton crearBotonInvisible() { 211 212 JButton boton = new JButton(); 213 214 boton.setPreferredSize(215 new Dimension(0, 0) 216); 217 218 boton.setMinimumSize(219 new Dimension(0, 0) 220); 221 222 boton.setMaximumSize(223 new Dimension(0, 0) 224); 225 226 boton.setBorder(227 BorderFactory.createEmptyBorder() 228); 229 230 boton.setContentAreaFilled(false); 231 boton.setFocusPainted(false); 232 boton.setOpaque(false); 233 234 return boton; 235 }	Declara el método crearBotonInvisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Línea en blanco: separa visualmente bloques para que el código sea más legible. Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso. Línea en blanco: separa visualmente bloques para que el código sea más legible. Configura una propiedad mediante setPreferredSize en el objeto indicado. Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso. Ejecuta una instrucción Java completa; el punto y coma marca su final. Línea en blanco: separa visualmente bloques para que el código sea más legible. Configura una propiedad mediante setMinimumSize en el objeto indicado. Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso. Ejecuta una instrucción Java completa; el punto y coma marca su final. Línea en blanco: separa visualmente bloques para que el código sea más legible. Configura una propiedad mediante setMaximumSize en el objeto indicado. Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso. Ejecuta una instrucción Java completa; el punto y coma marca su final. Línea en blanco: separa visualmente bloques para que el código sea más legible. Configura una propiedad mediante setBorder en el objeto indicado. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final. Línea en blanco: separa visualmente bloques para que el código sea más legible. Configura una propiedad mediante setContentAreaFilled en el objeto indicado. Configura una propiedad mediante setFocusPainted en el objeto indicado. Configura una propiedad mediante setOpaque en el objeto indicado. Línea en blanco: separa visualmente bloques para que el código sea más legible. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	// =====	Comentario para humanos; Java no ejecuta este texto. =====
238	// DIBUJAR INDICADOR NARANJA	Comentario para humanos; Java no ejecuta este texto. DIBUJAR INDICADOR NARANJA
239	// =====	Comentario para humanos; Java no ejecuta este texto. =====

240		Línea en blanco: separa visualmente bloques para que el código sea más legible.
241	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
242	protected void paintThumb(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
243	Graphics g,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244	JComponent c,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
245	Rectangle thumbBounds) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246		Línea en blanco: separa visualmente bloques para que el código sea más legible.
247	if (thumbBounds.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
248	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
249	}	Cierra el bloque abierto anteriormente.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	Graphics2D g2 =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
252	(Graphics2D) g.create();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
253		Línea en blanco: separa visualmente bloques para que el código sea más legible.
254	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
255	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
256	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.
259	g2.setColor(naranja);	Configura una propiedad mediante setColor en el objeto indicado.
260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	int ancho = 6;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
262		Línea en blanco: separa visualmente bloques para que el código sea más legible.
263	int x =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
264	thumbBounds.x	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	+ (thumbBounds.width - ancho) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
268	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
269	thumbBounds.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
270	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
271	thumbBounds.height,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
275		Línea en blanco: separa visualmente bloques para que el código sea más legible.
276	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
277	}	Cierra el bloque abierto anteriormente.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	// No dibujar el fondo de la barra	Comentario para humanos; Java no ejecuta este texto. No dibujar el fondo de la barra
280	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
281	protected void paintTrack(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	Graphics g,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	JComponent c,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

284	Rectangle trackBounds) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	// Vacío intencionalmente	Comentario para humanos; Java no ejecuta este texto. Vacío intencionalmente
287	}	Cierra el bloque abierto anteriormente.
288	}	Cierra el bloque abierto anteriormente.
289	}	Cierra el bloque abierto anteriormente.

16.41 src/Componentes/TablaAdministrativa.java

Componente visual reutilizable TablaAdministrativa. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 523 | Archivo 41 de 157

Clases/tipos detectados: TablaAdministrativa, RenderCabecera, RenderTexto, RenderImagenTexto, RenderEstrellas, RenderPorcentaje, CeldaimagenTexto, CeldaEstrellas, CeldaPorcentaje

Línea	Constructor o método detectado
59	public TablaAdministrativa() {
73	public final void aplicarEstilo() {
95	public void configurarColumnaImagenTexto(
107	public void configurarColumnaEstrellas(int columna, Color colorEstrella) {
114	public void configurarColumnaPorcentaje(int columna, Color colorBarra) {
120	private void validarColumna(int columna) {
127	private Color fondoCelda(boolean seleccionada, int fila) {
137	private int alineacion(int columna) {
147	private Set<Integer> convertirIndices(String texto) {
164	private RenderCabecera() {
169	public Component getTableCellRendererComponent(
194	public Component getTableCellRendererComponent(
225	private RenderImagenTexto(
253	public Component getTableCellRendererComponent(
271	private Icon cargarImagen(String rutaGuardada) {
279	private Icon leerImagen(String rutaGuardada) {
304	private RenderEstrellas(Color color) {
312	public Component getTableCellRendererComponent(
341	private RenderPorcentaje(Color color) {
350	public Component getTableCellRendererComponent(
368	public record CeldaimagenTexto(
374	public record CeldaEstrellas(double valor, int maximo) {
377	public record CeldaPorcentaje(double valor) {
380	public Color getColorCabecera() {
385	public void setColorCabecera(Color colorCabecera) {
390	public Color getColorTextoCabecera() {
395	public void setColorTextoCabecera(Color colorTextoCabecera) {
400	public Color getColorFilas() {
405	public void setColorFilas(Color colorFilas) {

Línea	Constructor o método detectado
410	public Color getColorFilaAlterna() {
415	public void setColorFilaAlterna(Color colorFilaAlterna) {
420	public Color getColorTexto() {
425	public void setColorTexto(Color colorTexto) {
430	public Color getColorLineas() {
435	public void setColorLineas(Color colorLineas) {
440	public Color getColorSeleccion() {
445	public void setColorSeleccion(Color colorSeleccion) {
450	public Color getColorTextoSeleccion() {
455	public void setColorTextoSeleccion(Color colorTextoSeleccion) {
460	public Font getFuenteCabecera() {
465	public void setFuenteCabecera(Font fuenteCabecera) {
470	public int getAltoCabecera() {
475	public void setAltoCabecera(int altoCabecera) {
480	public int getPaddingHorizontal() {
485	public void setPaddingHorizontal(int paddingHorizontal) {
490	public boolean isFilasAlternadas() {
495	public void setFilasAlternadas(boolean filasAlternadas) {
500	public String getColumnasCentradas() {
505	public void setColumnasCentradas(String columnasCentradas) {
512	public String getColumnasDerecha() {
517	public void setColumnasDerecha(String columnasDerecha) {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.GridLayout;	Importa java.awt.GridLayout para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Image;	Importa java.awt.Image para poder usar esa clase sin escribir su nombre completo.
11	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
12	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
13	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.

14	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
15	import java.util.HashMap;	Importa java.util.HashMap para poder usar esa clase sin escribir su nombre completo.
16	import java.util.HashSet;	Importa java.util.HashSet para poder usar esa clase sin escribir su nombre completo.
17	import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
18	import java.util.Set;	Importa java.util.Set: colección que evita elementos repetidos.
19	import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
20	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
21	import javax.swing.Icon;	Importa javax.swing.Icon para poder usar esa clase sin escribir su nombre completo.
22	import javax.swing.ImageIcon;	Importa javax.swing.ImageIcon para poder usar esa clase sin escribir su nombre completo.
23	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
24	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
25	import javax.swing.JProgressBar;	Importa javax.swing.JProgressBar para poder usar esa clase sin escribir su nombre completo.
26	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
27	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
28	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
29	import javax.swing.table.DefaultTableCellRenderer;	Importa javax.swing.table.DefaultTableCellRenderer para poder usar esa clase sin escribir su nombre completo.
30	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
33	* JTable reutilizable para los módulos administrativos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
34	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
35	* Sus propiedades de apariencia aparecen en Properties de NetBeans. Las	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
36	* columnas especiales se habilitan por código porque dependen del tipo de	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
37	* datos que entrega cada TableModel.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
38	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
39	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
40	public class TablaAdministrativa extends JTable {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	private Color colorCabecera = Color.WHITE;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43	private Color colorTextoCabecera = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
44	private Color colorFilas = Color.WHITE;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	private Color colorFilaAlternas = new Color(250, 251, 252);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
46	private Color colorTexto = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
47	private Color colorLineas = new Color(235, 238, 242);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
48	private Color colorSeleccion = new Color(255, 247, 222);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
49	private Color colorTextoSeleccion = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
50	private Font fuenteCabecera = new Font("Dialog", Font.BOLD, 12);	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
51	private int altoCabecera = 42;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52	private int paddingHorizontal = 12;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	private boolean filasAlternadas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
54	private String columnasCentradas = "";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55	private String columnasDerecha = "";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
56	private Set<Integer> indicesCentrados = Set.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	private Set<Integer> indicesDerecha = Set.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	public TablaAdministrativa() {	Constructor de TablaAdministrativa: prepara un objeto nuevo y deja sus componentes listos para usarse.
60	setFont(new Font("Dialog", Font.PLAIN, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
61	setRowHeight(40);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	setShowHorizontalLines(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	setShowVerticalLines(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	setInterCellSpacing(new Dimension(0, 1));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
65	setFillsViewportHeight(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	setAutoResizeMode(JTable.AUTO_RESIZE_ALL_COLUMNS);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	setDefaultRenderer(Object.class, new RenderTexto());	Crea una nueva instancia de RenderTexto y la usa o guarda para continuar el proceso.
68	setDefaultRenderer(String.class, new RenderTexto());	Crea una nueva instancia de RenderTexto y la usa o guarda para continuar el proceso.
69	aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	}	Cierra el bloque abierto anteriormente.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	/** Reaplica al JTable los valores elegidos desde Properties. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
73	public final void aplicarEstilo() {	Declara el método aplicarEstilo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
74	setBackground(colorFilas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75	setForeground(colorTexto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76	setGridColor(colorLineas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	setSelectionBackground(colorSeleccion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	setSelectionForeground(colorTextoSeleccion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	if (getTableHeader() != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
80	getTableHeader().setBackground(colorCabecera);	Configura una propiedad mediante setBackground en el objeto indicado.
81	getTableHeader().setForeground(colorTextoCabecera);	Configura una propiedad mediante setForeground en el objeto indicado.
82	getTableHeader().setFont(fuenteCabecera);	Configura una propiedad mediante setFont en el objeto indicado.
83	getTableHeader().setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
84	new Dimension(0, altoCabecera));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
85	getTableHeader().setReorderingAllowed(false);	Configura una propiedad mediante setReorderingAllowed en el objeto indicado.
86	getTableHeader().setDefaultRenderer(new RenderCabecera());	Crea una nueva instancia de RenderCabecera y la usa o guarda para continuar el proceso.
87	}	Cierra el bloque abierto anteriormente.
88	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	}	Cierra el bloque abierto anteriormente.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
92	* Configura una columna cuyo valor sea CeldaImagenTexto.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
93	* La ruta almacenada en la celda se resuelve dentro de carpetaBase.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
94	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
95	public void configurarColumnaImagenTexto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	int columna,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	Path carpetaBase,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
98	int anchoImagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
99	int altoImagen) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
100	validarColumna(columna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	getColumnModel().getColumn(columna).setCellRenderer(	Configura una propiedad mediante setCellRenderer en el objeto indicado.

102	<code>new RenderImagenTextto(carpetabase, anchoImagen, altoImagen));</code>	Crea una nueva instancia de RenderImagenTextto y la usa o guarda para continuar el proceso.
103	<code>setRowHeight(Math.max(getRowHeight(), altoImagen + 12));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	<code>}</code>	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	<code>/** Configura una columna cuyo valor sea CeldaEstrellas. */</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
107	<code>public void configurarColumnaEstrellas(int columna, Color colorEstrella) {</code>	Declara el método configurarColumnaEstrellas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
108	<code>validarColumna(columna);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	<code>getColumnModel().getColumn(columna).setCellRenderer(</code>	Configura una propiedad mediante setCellRenderer en el objeto indicado.
110	<code>new RenderEstrellas(colorEstrella));</code>	Crea una nueva instancia de RenderEstrellas y la usa o guarda para continuar el proceso.
111	<code>}</code>	Cierra el bloque abierto anteriormente.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	<code>/** Configura una columna cuyo valor sea CeldaPorcentaje. */</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
114	<code>public void configurarColumnaPorcentaje(int columna, Color colorBarra) {</code>	Declara el método configurarColumnaPorcentaje. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
115	<code>validarColumna(columna);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	<code>getColumnModel().getColumn(columna).setCellRenderer(</code>	Configura una propiedad mediante setCellRenderer en el objeto indicado.
117	<code>new RenderPorcentaje(colorBarra));</code>	Crea una nueva instancia de RenderPorcentaje y la usa o guarda para continuar el proceso.
118	<code>}</code>	Cierra el bloque abierto anteriormente.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	<code>private void validarColumna(int columna) {</code>	Declara el método validarColumna. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
121	<code>if (columna < 0 columna >= getColumnCount()) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
122	<code>throw new IllegalArgumentException(</code>	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
123	<code>"No existe la columna " + columna + ".");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	<code>}</code>	Cierra el bloque abierto anteriormente.
125	<code>}</code>	Cierra el bloque abierto anteriormente.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	<code>private Color fondoCelda(boolean seleccionada, int fila) {</code>	Declara el método fondoCelda. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
128	<code>if (seleccionada) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
129	<code>return colorSeleccion;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
130	<code>}</code>	Cierra el bloque abierto anteriormente.
131	<code>if (filasAlternadas && fila % 2 != 0) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
132	<code>return colorFilaAlterna;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
133	<code>}</code>	Cierra el bloque abierto anteriormente.
134	<code>return colorFilas;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
135	<code>}</code>	Cierra el bloque abierto anteriormente.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	<code>private int alineacion(int columna) {</code>	Declara el método alineacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
138	<code>if (indicesCentrados.contains(columna)) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
139	<code>return SwingConstants.CENTER;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
140	<code>}</code>	Cierra el bloque abierto anteriormente.
141	<code>if (indicesDerecha.contains(columna)) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
142	<code>return SwingConstants.RIGHT;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
143	<code>}</code>	Cierra el bloque abierto anteriormente.
144	<code>return SwingConstants.LEFT;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.

145	}	Cierra el bloque abierto anteriormente.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	private Set<Integer> convertirIndices(String texto) {	Declara el método convertirIndices. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
148	Set<Integer> resultado = new HashSet<>();	Crea una nueva instancia de HashSet y la usa o guarda para continuar el proceso.
149	if (texto == null texto.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
150	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
151	}	Cierra el bloque abierto anteriormente.
152	for (String parte : texto.split("[,;]+")) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
153	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
154	resultado.add(Integer.valueOf(parte.trim()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	} catch (NumberFormatException ignorada) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
156	// Properties puede quedar temporalmente incompleto al editar.	Comentario para humanos; Java no ejecuta este texto. Properties puede quedar temporalmente incompleto al editar.
157	}	Cierra el bloque abierto anteriormente.
158	}	Cierra el bloque abierto anteriormente.
159	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
160	}	Cierra el bloque abierto anteriormente.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	private final class RenderCabecera extends DefaultTableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	private RenderCabecera() {	Declara el método RenderCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
165	setOpaque(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
166	}	Cierra el bloque abierto anteriormente.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
169	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	boolean seleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	JLabel etiqueta = (JLabel) super.getTableCellRendererComponent(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
177	tabla, valor, seleccionada, foco, fila, columna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
178	etiqueta.setBackground(colorCabecera);	Configura una propiedad mediante setBackground en el objeto indicado.
179	etiqueta.setForeground(colorTextoCabecera);	Configura una propiedad mediante setForeground en el objeto indicado.
180	etiqueta.setFont(fuenteCabecera);	Configura una propiedad mediante setFont en el objeto indicado.
181	etiqueta.setHorizontalAlignment(alineacion(columna));	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
182	etiqueta.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
183	BorderFactory.createMatteBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	0, 0, 1, 0, colorLineas),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
185	new EmptyBorder(0, paddingHorizontal, 0,	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
186	paddingHorizontal));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
188	}	Cierra el bloque abierto anteriormente.

189	}	Cierra el bloque abierto anteriormente.
190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
191	private final class RenderTexto extends DefaultTableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
194	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197	boolean seleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
200	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
201	JLabel etiqueta = (JLabel) super.getTableCellRendererComponent(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
202	tabla, valor, seleccionada, foco, fila, columna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
203	etiqueta.setBackground(fondoCelda(seleccionada, fila));	Configura una propiedad mediante setBackground en el objeto indicado.
204	etiqueta.setForeground(seleccionada	Configura una propiedad mediante setForeground en el objeto indicado.
205	? colorTextoSeleccion : colorTexto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	etiqueta.setFont(tabla.getFont());	Configura una propiedad mediante setFont en el objeto indicado.
207	etiqueta.setHorizontalAlignment(alineacion(columna));	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
208	etiqueta.setBorder(new EmptyBorder(	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
209	0, paddingHorizontal, 0, paddingHorizontal));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
210	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
211	}	Cierra el bloque abierto anteriormente.
212	}	Cierra el bloque abierto anteriormente.
213		Línea en blanco: separa visualmente bloques para que el código sea más legible.
214	private final class RenderImagenTexto extends JPanel	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
215	implements TableCellRenderer {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	private final JLabel imagen = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
218	private final JLabel titulo = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
219	private final JLabel detalle = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
220	private final Path carpetaBase;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
221	private final int anchoImagen;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
222	private final int altoImagen;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
223	private final Map<String, Icon> cacheImagenes = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	private RenderImagenTexto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	Path carpetaBase,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	int anchoImagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	int altoImagen) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
229	this.carpetaBase = carpetaBase.toAbsolutePath().normalize();	Accede al atributo del objeto actual para asignarlo o utilizarlo.
230	this.anchoImagen = Math.max(1, anchoImagen);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
231	this.altoImagen = Math.max(1, altoImagen);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.

233	setLayout(new BorderLayout(10, 0));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
234	setBorder(new EmptyBorder(5, paddingHorizontal, 5,	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
235	paddingHorizontal));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	imagen.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
237	imagen.setPreferredSize(new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
238	this.anchoSImagen, this.altoImagen));	Accede al atributo del objeto actual para asignarlo o utilizarlo.
239		Línea en blanco: separa visualmente bloques para que el código sea más legible.
240	JPanel textos = new JPanel(new GridLayout(2, 1, 0, 1));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
241	textos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
242	titulo.setFont(getFont().deriveFont(Font.BOLD));	Configura una propiedad mediante setFont en el objeto indicado.
243	detalle.setFont(getFont().deriveFont(	Configura una propiedad mediante setFont en el objeto indicado.
244	Math.max(10f, getFont().getSize2D() - 2f));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245	detalle.setForeground(new Color(92, 103, 124));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
246	textos.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247	textos.add(detalle);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
248	add(imagen, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
249	add(textos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
250	}	Cierra el bloque abierto anteriormente.
251		Línea en blanco: separa visualmente bloques para que el código sea más legible.
252	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
253	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
255	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
256	boolean seleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	CeldaImagenTexto celda = valor instanceof CeldaImagenTexto dato	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
261	? dato : new CeldaImagenTexto("", "", null);	Crea una nueva instancia de CeldaimagenTexto y la usa o guarda para continuar el proceso.
262	titulo.setText(celda.titulo());	Cambia el texto visible de un componente de la interfaz.
263	detalle.setText(celda.detalle());	Cambia el texto visible de un componente de la interfaz.
264	imagen.setIcon(cargarImagen(celda.rutaImagen()));	Configura una propiedad mediante setIcon en el objeto indicado.
265	setBackground(fondoCelda(seleccionada, fila));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266	titulo.setForeground(seleccionada	Configura una propiedad mediante setForeground en el objeto indicado.
267	? colorTextoSeleccion : colorTexto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268	return this;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
269	}	Cierra el bloque abierto anteriormente.
270		Línea en blanco: separa visualmente bloques para que el código sea más legible.
271	private Icon cargarImagen(String rutaGuardada) {	Declara el método cargarImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
272	if (rutaGuardada == null rutaGuardada.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
273	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
274	}	Cierra el bloque abierto anteriormente.
275	return cacheImagenes.computeIfAbsent(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
276	rutaGuardada, this::leerImagen);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

277	}	Cierra el bloque abierto anteriormente.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	private Icon leerImagen(String rutaGuardada) {	Declara el método leerImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
280	Path archivo = carpetaBase.resolve(rutaGuardada).normalize();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
281	if (!archivo.startsWith(carpetaBase)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
282	!Files.isRegularFile(archivo)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
284	}	Cierra el bloque abierto anteriormente.
285	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
286	Image original = ImageIO.read(archivo.toFile());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
287	if (original == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
288	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
289	}	Cierra el bloque abierto anteriormente.
290	Image escalada = original.getScaledInstance(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
291	anchoImagen, altoImagen, Image.SCALE_SMOOTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
292	return new ImageIcon(escalada);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
293	} catch (IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
294	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
295	}	Cierra el bloque abierto anteriormente.
296	}	Cierra el bloque abierto anteriormente.
297	}	Cierra el bloque abierto anteriormente.
298		Línea en blanco: separa visualmente bloques para que el código sea más legible.
299	private final class RenderEstrellas extends JLabel	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
300	implements TableCellRenderer {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
301		Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	private final String colorHtml;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
303		Línea en blanco: separa visualmente bloques para que el código sea más legible.
304	private RenderEstrellas(Color color) {	Declara el método RenderEstrellas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
305	setOpaque(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306	setHorizontalAlignment(SwingConstants.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
307	colorHtml = String.format("#%02x%02x%02x",	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
308	color.getRed(), color.getGreen(), color.getBlue());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	}	Cierra el bloque abierto anteriormente.
310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
311	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
312	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
313	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
314	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	boolean seleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
317	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
319	CeldaEstrellas celda = valor instanceof CeldaEstrellas dato	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
320	? dato : new CeldaEstrellas(0, 5);	Crea una nueva instancia de CeldaEstrellas y la usa o guarda para continuar el proceso.

321	int maximo = Math.max(1, celda.maximo());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
322	int llenas = Math.max(0, Math.min(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
323	maximo, (int) Math.round(celda.valor())));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
324	String texto = "★".repeat(llenas);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
325	String vacias = "■".repeat(maximo - llenas);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
326	setText("<html>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
327	+ texto + "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
328	+ vacias + "</html>");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
329	setToolTipText(String.format("%.1f de %d",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	celda.valor(), maximo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
331	setBackground(fondoCelda(seleccionada, fila));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332	return this;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
333	}	Cierra el bloque abierto anteriormente.
334	}	Cierra el bloque abierto anteriormente.
335		Línea en blanco: separa visualmente bloques para que el código sea más legible.
336	private final class RenderPorcentaje extends JPanel	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
337	implements TableCellRenderer {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338		Línea en blanco: separa visualmente bloques para que el código sea más legible.
339	private final JProgressBar barra = new JProgressBar(0, 100);	Crea una nueva instancia de JProgressBar y la usa o guarda para continuar el proceso.
340		Línea en blanco: separa visualmente bloques para que el código sea más legible.
341	private RenderPorcentaje(Color color) {	Declara el método RenderPorcentaje. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
342	setLayout(new FlowLayout(FlowLayout.CENTER, 8, 9));	Crea una nueva instancia de FlowLayout y la usa o guarda para continuar el proceso.
343	barra.setPreferredSize(new Dimension(105, 18));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
344	barra.setForeground(color);	Configura una propiedad mediante setForeground en el objeto indicado.
345	barra.setStringPainted(true);	Configura una propiedad mediante setStringPainted en el objeto indicado.
346	add(barra);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
347	}	Cierra el bloque abierto anteriormente.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
350	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
351	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
352	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
353	boolean seleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
354	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
356	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	double valorNumerico = valor instanceof CeldaPorcentaje dato	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
358	? dato.valor() : 0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
359	int porcentaje = (int) Math.round(Math.max(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
360	0, Math.min(100, valorNumerico)));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
361	barra.setValue(porcentaje);	Configura una propiedad mediante setValue en el objeto indicado.
362	barra.setString(porcentaje + "%");	Configura una propiedad mediante setString en el objeto indicado.
363	setBackground(fondoCelda(seleccionada, fila));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
364	return this;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

365	}	Cierra el bloque abierto anteriormente.
366	}	Cierra el bloque abierto anteriormente.
367		Línea en blanco: separa visualmente bloques para que el código sea más legible.
368	public record CeldaImagenTexto(369 String titulo, 370 String detalle, 371 String rutaImagen) { 372 }	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
373		Cierra el bloque abierto anteriormente.
374	public record CeldaEstrellas(double valor, int maximo) { 375 }	Línea en blanco: separa visualmente bloques para que el código sea más legible. Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema. Cierra el bloque abierto anteriormente.
376		Línea en blanco: separa visualmente bloques para que el código sea más legible.
377	public record CeldaPorcentaje(double valor) { 378 }	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema. Cierra el bloque abierto anteriormente.
379		Línea en blanco: separa visualmente bloques para que el código sea más legible.
380	public Color getColorCabecera() { 381 return colorCabecera; 382 }	Declara el método getColorCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
383		Línea en blanco: separa visualmente bloques para que el código sea más legible.
384	@BeanProperty(description = "Color de fondo de la cabecera") 385 public void setColorCabecera(Color colorCabecera) { 386 this.colorCabecera = colorCabecera; 387 aplicarEstilo(); 388 }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente. Declara el método setColorCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Accede al atributo del objeto actual para asignarlo o utilizarlo. Ejecuta una instrucción Java completa; el punto y coma marca su final. Cierra el bloque abierto anteriormente.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	public Color getColorTextoCabecera() { 391 return colorTextoCabecera; 392 }	Declara el método getColorTextoCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
393		Línea en blanco: separa visualmente bloques para que el código sea más legible.
394	@BeanProperty(description = "Color del texto de la cabecera") 395 public void setColorTextoCabecera(Color colorTextoCabecera) { 396 this.colorTextoCabecera = colorTextoCabecera; 397 aplicarEstilo(); 398 }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente. Declara el método setColorTextoCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Accede al atributo del objeto actual para asignarlo o utilizarlo. Ejecuta una instrucción Java completa; el punto y coma marca su final. Cierra el bloque abierto anteriormente.
399		Línea en blanco: separa visualmente bloques para que el código sea más legible.
400	public Color getColorFilas() { 401 return colorFilas; 402 }	Declara el método getColorFilas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Finaliza el método y devuelve el valor indicado a quien lo llamó. Cierra el bloque abierto anteriormente.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	@BeanProperty(description = "Color principal de las filas") 405 public void setColorFilas(Color colorFilas) { 406 this.colorFilas = colorFilas;	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente. Declara el método setColorFilas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve. Accede al atributo del objeto actual para asignarlo o utilizarlo.

407	aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
408	}	Cierra el bloque abierto anteriormente.
409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
410	public Color getColorFilaAlterna() {	Declara el método getColorFilaAlterna. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
411	return colorFilaAlterna;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
412	}	Cierra el bloque abierto anteriormente.
413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
414	@BeanProperty(description = "Color de las filas alternas")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
415	public void setColorFilaAlterna(Color colorFilaAlterna) {	Declara el método setColorFilaAlterna. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
416	this.colorFilaAlterna = colorFilaAlterna;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
417	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
418	}	Cierra el bloque abierto anteriormente.
419		Línea en blanco: separa visualmente bloques para que el código sea más legible.
420	public Color getColorTexto() {	Declara el método getColorTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
421	return colorTexto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
422	}	Cierra el bloque abierto anteriormente.
423		Línea en blanco: separa visualmente bloques para que el código sea más legible.
424	@BeanProperty(description = "Color del texto de las filas")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
425	public void setColorTexto(Color colorTexto) {	Declara el método setColorTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
426	this.colorTexto = colorTexto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
427	aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
428	}	Cierra el bloque abierto anteriormente.
429		Línea en blanco: separa visualmente bloques para que el código sea más legible.
430	public Color getColorLineas() {	Declara el método getColorLineas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
431	return colorLineas;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
432	}	Cierra el bloque abierto anteriormente.
433		Línea en blanco: separa visualmente bloques para que el código sea más legible.
434	@BeanProperty(description = "Color de las líneas divisorias")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
435	public void setColorLineas(Color colorLineas) {	Declara el método setColorLineas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
436	this.colorLineas = colorLineas;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
437	aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
438	}	Cierra el bloque abierto anteriormente.
439		Línea en blanco: separa visualmente bloques para que el código sea más legible.
440	public Color getColorSeleccion() {	Declara el método getColorSeleccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
441	return colorSeleccion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
442	}	Cierra el bloque abierto anteriormente.
443		Línea en blanco: separa visualmente bloques para que el código sea más legible.
444	@BeanProperty(description = "Color de fondo de una fila seleccionada")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
445	public void setColorSeleccion(Color colorSeleccion) {	Declara el método setColorSeleccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
446	this.colorSeleccion = colorSeleccion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
447	aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448	}	Cierra el bloque abierto anteriormente.

449		Línea en blanco: separa visualmente bloques para que el código sea más legible.
450	<code>public Color getColorTextoSeleccion() {</code>	Declara el método <code>getColorTextoSeleccion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
451	<code> return colorTextoSeleccion;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
452	<code>}</code>	Cierra el bloque abierto anteriormente.
453		Línea en blanco: separa visualmente bloques para que el código sea más legible.
454	<code>@BeanProperty(description = "Color del texto seleccionado")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
455	<code>public void setColorTextoSeleccion(Color colorTextoSeleccion) {</code>	Declara el método <code>setColorTextoSeleccion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
456	<code> this.colorTextoSeleccion = colorTextoSeleccion;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
457	<code> aplicarEstilo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
458	<code>}</code>	Cierra el bloque abierto anteriormente.
459		Línea en blanco: separa visualmente bloques para que el código sea más legible.
460	<code>public Font getFuenteCabecera() {</code>	Declara el método <code>getFuenteCabecera</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
461	<code> return fuenteCabecera;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
462	<code>}</code>	Cierra el bloque abierto anteriormente.
463		Línea en blanco: separa visualmente bloques para que el código sea más legible.
464	<code>@BeanProperty(description = "Fuente de los títulos de columna")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
465	<code>public void setFuenteCabecera(Font fuenteCabecera) {</code>	Declara el método <code>setFuenteCabecera</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
466	<code> this.fuenteCabecera = fuenteCabecera;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
467	<code> aplicarEstilo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
468	<code>}</code>	Cierra el bloque abierto anteriormente.
469		Línea en blanco: separa visualmente bloques para que el código sea más legible.
470	<code>public int getAltoCabecera() {</code>	Declara el método <code>getAltoCabecera</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
471	<code> return altoCabecera;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
472	<code>}</code>	Cierra el bloque abierto anteriormente.
473		Línea en blanco: separa visualmente bloques para que el código sea más legible.
474	<code>@BeanProperty(description = "Altura de la cabecera en pixeles")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
475	<code>public void setAltoCabecera(int altoCabecera) {</code>	Declara el método <code>setAltoCabecera</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
476	<code> this.altoCabecera = Math.max(1, altoCabecera);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
477	<code> aplicarEstilo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
478	<code>}</code>	Cierra el bloque abierto anteriormente.
479		Línea en blanco: separa visualmente bloques para que el código sea más legible.
480	<code>public int getPaddingHorizontal() {</code>	Declara el método <code>getPaddingHorizontal</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
481	<code> return paddingHorizontal;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
482	<code>}</code>	Cierra el bloque abierto anteriormente.
483		Línea en blanco: separa visualmente bloques para que el código sea más legible.
484	<code>@BeanProperty(description = "Espacio izquierdo y derecho del texto")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
485	<code>public void setPaddingHorizontal(int paddingHorizontal) {</code>	Declara el método <code>setPaddingHorizontal</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
486	<code> this.paddingHorizontal = Math.max(0, paddingHorizontal);</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
487	<code> repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
488	<code>}</code>	Cierra el bloque abierto anteriormente.

489		Línea en blanco: separa visualmente bloques para que el código sea más legible.
490	public boolean isFilasAlternadas() {	Declara el método isFilasAlternadas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
491	return filasAlternadas;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
492	}	Cierra el bloque abierto anteriormente.
493		Línea en blanco: separa visualmente bloques para que el código sea más legible.
494	@BeanProperty(description = "Alterna el color de las filas")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
495	public void setFilasAlternadas(boolean filasAlternadas) {	Declara el método setFilasAlternadas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
496	this.filasAlternadas = filasAlternadas;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
497	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
498	}	Cierra el bloque abierto anteriormente.
499		Línea en blanco: separa visualmente bloques para que el código sea más legible.
500	public String getColumnasCentradas() {	Declara el método getColumnasCentradas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
501	return columnasCentradas;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
502	}	Cierra el bloque abierto anteriormente.
503		Línea en blanco: separa visualmente bloques para que el código sea más legible.
504	@BeanProperty(description = "Índices centrados separados por coma: 0,5,7")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
505	public void setColumnasCentradas(String columnasCentradas) {	Declara el método setColumnasCentradas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
506	this.columnasCentradas = columnasCentradas == null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
507	? "" : columnasCentradas;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
508	indicesCentrados = convertirIndices(this.columnasCentradas);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
509	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
510	}	Cierra el bloque abierto anteriormente.
511		Línea en blanco: separa visualmente bloques para que el código sea más legible.
512	public String getColumnasDerecha() {	Declara el método getColumnasDerecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
513	return columnasDerecha;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
514	}	Cierra el bloque abierto anteriormente.
515		Línea en blanco: separa visualmente bloques para que el código sea más legible.
516	@BeanProperty(description = "Índices a la derecha separados por coma")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
517	public void setColumnasDerecha(String columnasDerecha) {	Declara el método setColumnasDerecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
518	this.columnasDerecha = columnasDerecha == null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
519	? "" : columnasDerecha;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
520	indicesDerecha = convertirIndices(this.columnasDerecha);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
521	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
522	}	Cierra el bloque abierto anteriormente.
523	}	Cierra el bloque abierto anteriormente.

16.42 src/Componentes/TarjetaProducto.java

Componente visual reutilizable TarjetaProducto. Mantiene el estilo de la aplicación y evita duplicar código de Swing.

Tipo: java | Líneas: 298 | Archivo 42 de 157

Clases/tipos detectados: TarjetaProducto

Línea	Constructor o método detectado
26	public TarjetaProducto(
36	public TarjetaProducto(
49	private void init(
166	public void mouseEntered(MouseEvent e) {
172	public void mouseExited(MouseEvent e) {
181	public void mouseEntered(MouseEvent e) {
186	public void mouseExited(MouseEvent e) {
192	private void cargarImagenDiferida(String ruta) {
208	protected void paintComponent(Graphics g) {
239	protected void paintChildren(Graphics g) {
291	public int getIdProducto() {
295	public JButton getBtnAgregar() {

1	package Componentes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.*;	Importa java.awt.* para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
6	import javax.swing.*;	Importa javax.swing.* para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.ImageCache;	Importa Utilidades.ImageCache para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	public class TarjetaProducto extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	private static final String FUENTE = "Arial";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	private int idProducto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	private JLabel lblImagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
17	private JLabel lblNombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
18	private JLabel lblDescripcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
19	private JLabel lblPrecio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
20	private JButton btnAgregar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

21	private JLabel lblEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	private boolean hover = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24	private boolean disponible;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	public TarjetaProducto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	String descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	double precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	String imagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	this(idProducto, nombre, descripcion, precio, imagen, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	}	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	public TarjetaProducto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	String descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	double precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	String imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	boolean disponible	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	this.idProducto = idProducto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
45	this.disponible = disponible;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
46	init(nombre, descripcion, precio, imagen);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	private void init(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	String descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	double precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	String imagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
57	Dimension tamano = new Dimension(230, 250);	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
58	setPreferredSize(tamano);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	setMinimumSize(tamano);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	setMaximumSize(tamano);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	setLayout(new BorderLayout(0, 8));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
62	setBorder(new EmptyBorder(12, 12, 12, 12));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
63	setCursor(new Cursor(disponible ? Cursor.HAND_CURSOR : Cursor.DEFAULT_CURSOR));	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.

65	// =====	Comentario para humanos; Java no ejecuta este texto. =====
66	// IMAGEN	Comentario para humanos; Java no ejecuta este texto. IMAGEN
67	// =====	Comentario para humanos; Java no ejecuta este texto. =====
68	lblImagen = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
69	lblImagen.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
70	lblImagen.setPreferredSize(new Dimension(180, 100));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	cargarImagenDiferida(imagen);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	add(lblImagen, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	// =====	Comentario para humanos; Java no ejecuta este texto. =====
77	// CENTRO	Comentario para humanos; Java no ejecuta este texto. CENTRO
78	// =====	Comentario para humanos; Java no ejecuta este texto. =====
79	JPanel panelCentro = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
80	panelCentro.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
81	panelCentro.setLayout(new BoxLayout(panelCentro, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	lblNombre = new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
84	"<html>" + nombre + "</html>"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
85	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	lblNombre.setFont(new Font(FUENTE, Font.BOLD, 17));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
87	lblNombre.setForeground(new Color(20, 27, 35));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
88	lblNombre.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	lblEstado = new JLabel("No disponible ahora");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
91	lblEstado.setFont(new Font(FUENTE, Font.BOLD, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
92	lblEstado.setForeground(new Color(102, 108, 115));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
93	lblEstado.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
94	lblEstado.setBackground(new Color(232, 235, 238));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
95	lblEstado.setBorder(new EmptyBorder(4, 7, 4, 7));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
96	lblEstado.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
97	lblEstado.setVisible(!disponible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	panelCentro.add(lblNombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	panelCentro.add(Box.createVerticalStrut(6));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	panelCentro.add(lblEstado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	if (!disponible) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
103	panelCentro.add(Box.createVerticalStrut(5));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	lblNombre.setForeground(new Color(112, 118, 124));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
105	}	Cierra el bloque abierto anteriormente.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	if (descripcion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
108	descripcion = "";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

109	}	Cierra el bloque abierto anteriormente.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	lblDescripcion = new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
112	"<html><div style='width:155px;'"	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
113	+ descripcion	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	+ "</div></html>"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
115	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	lblDescripcion.setFont(new Font(FUENTE, Font.PLAIN, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
117	lblDescripcion.setForeground(disponible	Configura una propiedad mediante setForeground en el objeto indicado.
118	? new Color(95, 95, 95) : new Color(142, 147, 152));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
119	lblDescripcion.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	panelCentro.add(lblDescripcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	add(panelCentro, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	// =====	Comentario para humanos; Java no ejecuta este texto. =====
126	// INFERIOR	Comentario para humanos; Java no ejecuta este texto. INFERIOR
127	// =====	Comentario para humanos; Java no ejecuta este texto. =====
128	JPanel panelInferior = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
129	panelInferior.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	lblPrecio = new JLabel(String.format("%.2f", precio));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
132	lblPrecio.setFont(new Font(FUENTE, Font.BOLD, 18));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
133	lblPrecio.setForeground(disponible	Configura una propiedad mediante setForeground en el objeto indicado.
134	? new Color(20, 27, 35) : new Color(135, 140, 145));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
135		Línea en blanco: separa visualmente bloques para que el código sea más legible.
136	btnAgregar = new JButton("Agregar");	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
137	btnAgregar.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
138	btnAgregar.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
139	btnAgregar.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
140	btnAgregar.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
141	btnAgregar.setCursor(new Cursor(Cursor.HAND_CURSOR));	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
142	btnAgregar.setFont(new Font(FUENTE, Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
143	btnAgregar.setForeground(new Color(20, 27, 35));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
144	btnAgregar.setPreferredSize(new Dimension(95, 34));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
145	btnAgregar.setEnabled(disponible);	Habilita o deshabilita la interacción con el componente.
146	if (!disponible) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
147	btnAgregar.setToolTipText("Este producto no está disponible en este horario.");	Configura una propiedad mediante setToolTipText en el objeto indicado.
148	}	Cierra el bloque abierto anteriormente.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	panelInferior.add(lblPrecio, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	panelInferior.add(btnAgregar, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.

153	add(panelInferior, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	btnAgregar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
156	if (!disponible) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
157	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
158	}	Cierra el bloque abierto anteriormente.
159	java.awt.Window ventana = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
160	if (ventana instanceof GUI_CAJERO.Cajero cajero) cajero.agregarProducto(idProducto);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
161	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	// Hover de tarjeta	Comentario para humanos; Java no ejecuta este texto. Hover de tarjeta
164	addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
165	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
166	public void mouseEntered(MouseEvent e) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
167	hover = disponible;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
168	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	}	Cierra el bloque abierto anteriormente.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
172	public void mouseExited(MouseEvent e) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
173	hover = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
174	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
175	}	Cierra el bloque abierto anteriormente.
176	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	// Hover botón	Comentario para humanos; Java no ejecuta este texto. Hover botón
179	btnAgregar.addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
180	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
181	public void mouseEntered(MouseEvent e) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
182	btnAgregar.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
183	}	Cierra el bloque abierto anteriormente.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
186	public void mouseExited(MouseEvent e) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
187	btnAgregar.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188	}	Cierra el bloque abierto anteriormente.
189	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	}	Cierra el bloque abierto anteriormente.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	private void cargarImagenDiferida(String ruta) {	Declara el método cargarImagenDiferida. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
193	lblImagen.setText("Cargando...");	Cambia el texto visible de un componente de la interfaz.
194	lblImagen.setIcon(null);	Configura una propiedad mediante setIcon en el objeto indicado.
195	ImageCache.cargarAsync(ruta, 125, 95, icono -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	if (icono == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

197	lblImagen.setText("Sin imagen");	Cambia el texto visible de un componente de la interfaz.
198	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
199	lblImagen.setText("");	Cambia el texto visible de un componente de la interfaz.
200	lblImagen.setIcon(icono);	Configura una propiedad mediante setIcon en el objeto indicado.
201	}	Cierra el bloque abierto anteriormente.
202	lblImagen.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
203	lblImagen.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
204	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
205	}	Cierra el bloque abierto anteriormente.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
208	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
209	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
212	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
216	// sombra suave	Comentario para humanos; Java no ejecuta este texto. sombra suave
217	g2.setColor(new Color(0, 0, 0, 12));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
218	g2.fillRoundRect(4, 4, getWidth() - 8, getHeight() - 8, 24, 24);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	// fondo	Comentario para humanos; Java no ejecuta este texto. fondo
221	if (!disponible) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
222	g2.setColor(new Color(242, 243, 244));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
223	} else if (hover) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
224	g2.setColor(new Color(255, 252, 245));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
225	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
226	g2.setColor(disponible ? Color.WHITE : new Color(235, 237, 239));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
227	}	Cierra el bloque abierto anteriormente.
228	g2.fillRoundRect(0, 0, getWidth() - 6, getHeight() - 6, 24, 24);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230	// borde	Comentario para humanos; Java no ejecuta este texto. borde
231	g2.setColor(new Color(230, 230, 230));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
232	g2.drawRoundRect(0, 0, getWidth() - 7, getHeight() - 7, 24, 24);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
235	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	}	Cierra el bloque abierto anteriormente.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
239	protected void paintChildren(Graphics g) {	Declara el método paintChildren. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
240	super.paintChildren(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

241		Línea en blanco: separa visualmente bloques para que el código sea más legible.
242	if (btnAgregar != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
243	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	Point p = SwingUtilities.convertPoint(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
246	btnAgregar.getParent(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	btnAgregar.getLocation(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
248	this	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
249	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
252	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	g2.setColor(Color.WHITE);	Configura una propiedad mediante setColor en el objeto indicado.
257	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	p.x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	p.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	btnAgregar.getWidth(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
261	btnAgregar.getHeight(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
262	22,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
263	22	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
264	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	g2.setColor(disponible	Configura una propiedad mediante setColor en el objeto indicado.
267	? new Color(255, 188, 13) : new Color(205, 208, 211));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
268	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
269	p.x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
270	p.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
271	btnAgregar.getWidth() - 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272	btnAgregar.getHeight() - 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	22,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	22	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	FontMetrics fm = g2.getFontMetrics(btnAgregar.getFont());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
278	String texto = btnAgregar.getText();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
279	int tx = p.x + (btnAgregar.getWidth() - fm.stringWidth(texto)) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
280	int ty = p.y + ((btnAgregar.getHeight() - fm.getHeight()) / 2) + fm.getAscent();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
281		Línea en blanco: separa visualmente bloques para que el código sea más legible.
282	g2.setFont(btnAgregar.getFont());	Configura una propiedad mediante setFont en el objeto indicado.
283	g2.setColor(disponible	Configura una propiedad mediante setColor en el objeto indicado.
284	? new Color(20, 27, 35) : new Color(145, 149, 153));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

285	g2.drawString(texto, tx, ty);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
288	}	Cierra el bloque abierto anteriormente.
289	}	Cierra el bloque abierto anteriormente.
290		Línea en blanco: separa visualmente bloques para que el código sea más legible.
291	public int getIdProducto() {	Declara el método getIdProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
292	return idProducto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
293	}	Cierra el bloque abierto anteriormente.
294		Línea en blanco: separa visualmente bloques para que el código sea más legible.
295	public JButton getBtnAgregar() {	Declara el método getBtnAgregar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
296	return btnAgregar;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
297	}	Cierra el bloque abierto anteriormente.
298	}	Cierra el bloque abierto anteriormente.

16.43 src/Conexion/Conexion.java

Centraliza la conexión JDBC con MySQL y devuelve objetos Connection para los DAO y CRUD.

Tipo: java | Líneas: 49 | Archivo 43 de 157

Clases/tipos detectados: Conexion

Línea	Constructor o método detectado
16	public static Connection conectar() {
38	public static void main(String[] args) {

Conexion.java

1	package Conexion;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
4	import java.sql.DriverManager;	Importa java.sql.DriverManager para poder usar esa clase sin escribir su nombre completo.
5	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	public class Conexion {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	private static final String URL =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
10	"jdbc:mysql://localhost:3306/waldonalds?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
	";	
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	private static final String USUARIO = "root";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	private static final String CONTRASENA = "[VALOR OCULTO]"; "[VALOR OCULTO]";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	public static Connection conectar() {	Declara el método conectar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	Connection conexion = DriverManager.getConnection(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
21	URL,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	USUARIO,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
23	CONTRASENA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	System.out.println("Conexion exitosa con MySQL");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	System.out.println("Error al conectar con MySQL:");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	System.out.println(e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.

33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
35	}	Cierra el bloque abierto anteriormente.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	public static void main(String[] args) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	if (conexion != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
43	System.out.println("Conexion con Waldonalds realizada correctamente.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
45	System.out.println("No se pudo conectar.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	}	Cierra el bloque abierto anteriormente.
47	}	Cierra el bloque abierto anteriormente.
48	}	Cierra el bloque abierto anteriormente.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.44 src/CRUD/CRUD.java

Operaciones de creación, consulta, actualización y estado para el catálogo usando JDBC.

Tipo: java | Líneas: 13 | Archivo 44 de 157

Clases/tipos detectados: CRUD

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package CRUD;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author Computacion	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class CRUD {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	}	Cierra el bloque abierto anteriormente.

16.45 src/CRUD/IngredienteCRUD.java

Operaciones de creación, consulta, actualización y estado para ingrediente usando JDBC.

Tipo: java | Líneas: 256 | Archivo 45 de 157

Clases/tipos detectados: IngredienteCRUD

Línea	Constructor o método detectado
21	private Connection conectar() throws SQLException {
27	public ResumenIngredientes obtenerResumen() throws SQLException {
46	public PaginaIngredientes listarPagina(String busqueda, Boolean stockBajo,
93	public Ingrediente obtenerPorId(int id) throws SQLException {
108	public int insertar(Ingrediente ingrediente) throws SQLException {
134	public void actualizar(Ingrediente ingrediente) throws SQLException {
152	public void cambiarEstado(int id, boolean activo) throws SQLException {
169	public BigDecimal registrarMovimiento(int id, String tipo,
239	private static int colocarParametros(PreparedStatement ps,
246	private static Ingrediente mapear(ResultSet rs) throws SQLException {

1	package CRUD;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.PaginaIngredientes;	Importa Modelos.PaginaIngredientes para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.ResumenIngredientes;	Importa Modelos.ResumenIngredientes para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
9	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
10	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
11	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
12	import java.sql.Statement;	Importa java.sql.Statement para poder usar esa clase sin escribir su nombre completo.
13	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
14	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	public class IngredienteCRUD {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
17	private static final BigDecimal MAX_STOCK = new BigDecimal("99999999.99");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
18	private static final String STOCK_BAJO =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	"(stock_minimo > 0 AND stock_actual <= stock_minimo)";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private Connection conectar() throws SQLException {	Declara el método conectar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
22	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	if (conexion == null) throw new SQLException("No fue posible conectar con MySQL.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

24	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
25	}	Cierra el bloque abierto anteriormente.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	public ResumenIngredientes obtenerResumen() throws SQLException {	Declara el método obtenerResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
29	SELECT COUNT(*) AS total,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	COALESCE(SUM(CASE WHEN estado = TRUE AND stock_minimo > 0	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
31	AND stock_actual <= stock_minimo THEN 1 ELSE 0 END), 0) AS stock_bajo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	COALESCE(SUM(CASE WHEN estado = TRUE THEN 1 ELSE 0 END), 0) AS activos,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
33	COALESCE(SUM(CASE WHEN estado = FALSE THEN 1 ELSE 0 END), 0) AS inactivos	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
34	FROM ingrediente	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
37	PreparedStatement ps = conexion.prepareStatement(sql);	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
38	ResultSet rs = ps.executeQuery() {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
39	rs.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
40	return new ResumenIngredientes(rs.getInt("total"),	Finaliza el método y devuelve el valor indicado a quien lo llamó.
41	rs.getInt("stock_bajo"), rs.getInt("activos"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
42	rs.getInt("inactivos"));	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
43	}	Cierra el bloque abierto anteriormente.
44	}	Cierra el bloque abierto anteriormente.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	public PaginaIngredientes listarPagina(String busqueda, Boolean stockBajo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	Boolean estado, int pagina, int porPagina) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	StringBuilder condiciones = new StringBuilder(" WHERE 1 = 1");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
49	List<Object> parametros = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
50	if (busqueda != null && !busqueda.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
51	condiciones.append(" AND (nombre LIKE ? OR unidad_medida LIKE ?)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52	String texto = "%" + busqueda.trim() + "%";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	}	Cierra el bloque abierto anteriormente.
56	if (stockBajo != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
57	condiciones.append(stockBajo ? " AND " + STOCK_BAJO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
58	: " AND NOT " + STOCK_BAJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	}	Cierra el bloque abierto anteriormente.
60	if (estado != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
61	condiciones.append(" AND estado = ?");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
62	parametros.add(estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	}	Cierra el bloque abierto anteriormente.
64	String contar = "SELECT COUNT(*) AS total FROM ingrediente" + condiciones;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	String listar = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	SELECT id_ingrediente, nombre, unidad_medida, stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	stock_minimo, estado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

68	FROM ingrediente	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	"" + condiciones + " ORDER BY id_ingrediente DESC LIMIT ? OFFSET ?";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
72	int total;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
73	try (PreparedStatement ps = conexion.prepareStatement(contar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
74	colocarParametros(ps, parametros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
76	rs.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
77	total = rs.getInt("total");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
78	}	Cierra el bloque abierto anteriormente.
79	}	Cierra el bloque abierto anteriormente.
80	List<Ingrediente> ingredientes = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
81	try (PreparedStatement ps = conexion.prepareStatement(listar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
82	int indice = colocarParametros(ps, parametros);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	ps.setInt(indice++, porPagina);	Configura una propiedad mediante setInt en el objeto indicado.
84	ps.setInt(indice, (pagina - 1) * porPagina);	Configura una propiedad mediante setInt en el objeto indicado.
85	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
86	while (rs.next()) ingredientes.add(mapear(rs));	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
87	}	Cierra el bloque abierto anteriormente.
88	}	Cierra el bloque abierto anteriormente.
89	return new PaginaIngredientes(ingredientes, total);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
90	}	Cierra el bloque abierto anteriormente.
91	}	Cierra el bloque abierto anteriormente.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	public Ingrediente obtenerPorId(int id) throws SQLException {	Declara el método obtenerPorId. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
94	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
95	SELECT id_ingrediente, nombre, unidad_medida, stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	stock_minimo, estado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	FROM ingrediente WHERE id_ingrediente = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
98	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
100	PreparedStatement ps = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
101	ps.setInt(1, id);	Configura una propiedad mediante setInt en el objeto indicado.
102	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
103	return rs.next() ? mapear(rs) : null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
104	}	Cierra el bloque abierto anteriormente.
105	}	Cierra el bloque abierto anteriormente.
106	}	Cierra el bloque abierto anteriormente.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	public int insertar(Ingrediente ingrediente) throws SQLException {	Declara el método insertar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
109	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
110	INSERT INTO ingrediente	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
111	(nombre, unidad_medida, stock_actual, stock_minimo, estado)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

112	VALUES (?, ?, 0.00, ?, ?)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
115	PreparedStatement ps = conexion.prepareStatement(sql,	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
116	Statement.RETURN_GENERATED_KEYS)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117	ps.setString(1, ingrediente.getNombre());	Configura una propiedad mediante setString en el objeto indicado.
118	ps.setString(2, ingrediente.getUnidadMedida());	Configura una propiedad mediante setString en el objeto indicado.
119	ps.setBigDecimal(3, ingrediente.getStockMinimo());	Configura una propiedad mediante setBigDecimal en el objeto indicado.
120	ps.setBoolean(4, ingrediente.isActivo());	Configura una propiedad mediante setBoolean en el objeto indicado.
121	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
122	ingrediente.setStockActual(BigDecimal.ZERO);	Configura una propiedad mediante setStockActual en el objeto indicado.
123	try (ResultSet rs = ps.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
124	if (rs.next()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
125	ingrediente.setIdIngrediente(rs.getInt(1));	Configura una propiedad mediante setIdIngrediente en el objeto indicado.
126	return ingrediente.getIdIngrediente();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
127	}	Cierra el bloque abierto anteriormente.
128	}	Cierra el bloque abierto anteriormente.
129	}	Cierra el bloque abierto anteriormente.
130	return 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
131	}	Cierra el bloque abierto anteriormente.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	/** No modifica stock_actual; las existencias se cambian con movimientos. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
134	public void actualizar(Ingrediente ingrediente) throws SQLException {	Declara el método actualizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
135	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136	UPDATE ingrediente SET nombre = ?, unidad_medida = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
137	stock_minimo = ?, estado = ? WHERE id_ingrediente = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
138	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
139	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
140	PreparedStatement ps = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
141	ps.setString(1, ingrediente.getNombre());	Configura una propiedad mediante setString en el objeto indicado.
142	ps.setString(2, ingrediente.getUnidadMedida());	Configura una propiedad mediante setString en el objeto indicado.
143	ps.setBigDecimal(3, ingrediente.getStockMinimo());	Configura una propiedad mediante setBigDecimal en el objeto indicado.
144	ps.setBoolean(4, ingrediente.isActivo());	Configura una propiedad mediante setBoolean en el objeto indicado.
145	ps.setInt(5, ingrediente.getIdIngrediente());	Configura una propiedad mediante setInt en el objeto indicado.
146	if (ps.executeUpdate() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
147	throw new SQLException("No se encontró el ingrediente.");	Interrompe el flujo lanzando un error con una explicación para quien llamó al método.
148	}	Cierra el bloque abierto anteriormente.
149	}	Cierra el bloque abierto anteriormente.
150	}	Cierra el bloque abierto anteriormente.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	public void cambiarEstado(int id, boolean activo) throws SQLException {	Declara el método cambiarEstado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
153	String sql = "UPDATE ingrediente SET estado = ? WHERE id_ingrediente = ?";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
154	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
155	PreparedStatement ps = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.

156	ps.setBoolean(1, activo);	Configura una propiedad mediante setBoolean en el objeto indicado.
157	ps.setInt(2, id);	Configura una propiedad mediante setInt en el objeto indicado.
158	if (ps.executeUpdate() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
159	throw new SQLException("No se encontró el ingrediente.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
160	}	Cierra el bloque abierto anteriormente.
161	}	Cierra el bloque abierto anteriormente.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
165	* ENTRADA y SALIDA reciben una cantidad positiva. AJUSTE recibe el nuevo	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
166	* stock absoluto y registra como cantidad la diferencia con el anterior.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
167	* La actualización y el movimiento se confirman en una sola transacción.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
168	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
169	public BigDecimal registrarMovimiento(int id, String tipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170	BigDecimal cantidad, String motivo) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	if (!(("ENTRADA".equals(tipo) "SALIDA".equals(tipo)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
172	"AJUSTE".equals(tipo))) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	throw new IllegalArgumentException("Tipo de movimiento inválido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
174	}	Cierra el bloque abierto anteriormente.
175	if (cantidad == null cantidad.scale() > 2	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
176	cantidad.compareTo(MAX_STOCK) > 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	("AJUSTE".equals(tipo) ? cantidad.signum() < 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	: cantidad.signum() <= 0)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	throw new IllegalArgumentException("Cantidad inválida; usa hasta dos decimales.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
180	}	Cierra el bloque abierto anteriormente.
181	if (motivo != null && motivo.length() > 150) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
182	throw new IllegalArgumentException("El motivo admite hasta 150 caracteres.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
183	}	Cierra el bloque abierto anteriormente.
184	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
185	conexion.setAutoCommit(false);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
186	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
187	BigDecimal anterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
188	try (PreparedStatement ps = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
189	"SELECT stock_actual FROM ingrediente WHERE id_ingrediente = ? FOR UPDATE")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
190	ps.setInt(1, id);	Configura una propiedad mediante setInt en el objeto indicado.
191	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
192	if (!rs.next()) throw new SQLException("No se encontró el ingrediente.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
193	anterior = rs.getBigDecimal("stock_actual");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
194	}	Cierra el bloque abierto anteriormente.
195	}	Cierra el bloque abierto anteriormente.
196	BigDecimal nuevo = switch (tipo) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	case "ENTRADA" -> anterior.add(cantidad);	Define una de las rutas posibles dentro del switch.
198	case "SALIDA" -> anterior.subtract(cantidad);	Define una de las rutas posibles dentro del switch.
199	default -> cantidad;	Define una de las rutas posibles dentro del switch.

200	};	Cierra el bloque abierto anteriormente y termina la declaración.
201	if (nuevo.signum() < 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
202	throw new SQLException("No hay existencias suficientes para la salida.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
203	}	Cierra el bloque abierto anteriormente.
204	if (nuevo.compareTo(MAX_STOCK) > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
205	throw new SQLException("El stock supera el máximo permitido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
206	}	Cierra el bloque abierto anteriormente.
207	BigDecimal cantidadRegistrada = "AJUSTE".equals(tipo)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
208	? nuevo.subtract(anterior) : cantidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209	if (cantidadRegistrada.signum() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
210	throw new SQLException("El ajuste no cambia el stock.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
211	}	Cierra el bloque abierto anteriormente.
212	try (PreparedStatement ps = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
213	"UPDATE ingrediente SET stock_actual = ? WHERE id_ingrediente = ?")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
214	ps.setBigDecimal(1, nuevo);	Configura una propiedad mediante setBigDecimal en el objeto indicado.
215	ps.setInt(2, id);	Configura una propiedad mediante setInt en el objeto indicado.
216	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
217	}	Cierra el bloque abierto anteriormente.
218	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
219	INSERT INTO movimiento_inventario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
220	(id_ingrediente, tipo_movimiento, cantidad, motivo)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
221	VALUES (?, ?, ?, ?)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223	try (PreparedStatement ps = conexion.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
224	ps.setInt(1, id);	Configura una propiedad mediante setInt en el objeto indicado.
225	ps.setString(2, tipo);	Configura una propiedad mediante setString en el objeto indicado.
226	ps.setBigDecimal(3, cantidadRegistrada);	Configura una propiedad mediante setBigDecimal en el objeto indicado.
227	ps.setString(4, motivo == null motivo.isBlank() ? null : motivo.trim());	Configura una propiedad mediante setString en el objeto indicado.
228	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
229	}	Cierra el bloque abierto anteriormente.
230	conexion.commit();	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
231	return nuevo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
232	} catch (SQLException RuntimeException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
233	conexion.rollback();	Revierde la transacción porque algo falló, evitando guardar un pedido o inventario incompleto.
234	throw ex;	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
235	}	Cierra el bloque abierto anteriormente.
236	}	Cierra el bloque abierto anteriormente.
237	}	Cierra el bloque abierto anteriormente.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	private static int colocarParametros(PreparedStatement ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240	List<Object> parametros) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
241	int indice = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
242	for (Object parametro : parametros) ps.setObject(indice++, parametro);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
243	return indice;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

244	}	Cierra el bloque abierto anteriormente.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	private static Ingrediente mapear(ResultSet rs) throws SQLException {	Declara el método mapear. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
247	Ingrediente ingrediente = new Ingrediente();	Crea una nueva instancia de Ingrediente y la usa o guarda para continuar el proceso.
248	ingrediente.setIdIngrediente(rs.getInt("id_ingrediente"));	Configura una propiedad mediante setIdIngrediente en el objeto indicado.
249	ingrediente.setNombre(rs.getString("nombre"));	Configura una propiedad mediante setNombre en el objeto indicado.
250	ingrediente.setUnidadMedida(rs.getString("unidad_medida"));	Configura una propiedad mediante setUnidadMedida en el objeto indicado.
251	ingrediente.setStockActual(rs.getBigDecimal("stock_actual"));	Configura una propiedad mediante setStockActual en el objeto indicado.
252	ingrediente.setStockMinimo(rs.getBigDecimal("stock_minimo"));	Configura una propiedad mediante setStockMinimo en el objeto indicado.
253	ingrediente.setActivo(rs.getBoolean("estado"));	Configura una propiedad mediante setActivo en el objeto indicado.
254	return ingrediente;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
255	}	Cierra el bloque abierto anteriormente.
256	}	Cierra el bloque abierto anteriormente.

16.46 src/CRUD/ProductoCRUD.java

Operaciones de creación, consulta, actualización y estado para producto usando JDBC.

Tipo: java | Líneas: 768 | Archivo 46 de 157

Clases/tipos detectados: ProductoCRUD

Línea	Constructor o método detectado
22	private Connection conectar() throws SQLException {
39	public ResumenProductos obtenerResumen()
106	public PaginaProductos listarPagina(
292	public int insertar(
357	public void actualizar(
417	public void cambiarEstado(
447	public void desactivar(
457	public void activar(
471	public Producto obtenerPorId(
639	private Producto mapearProducto(
661	private int colocarParametros(
680	private int insertarId(Connection conexion, String sql, Object... valores)
693	private void completarParametrosProducto(

1	package CRUD;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.PaginaProductos;	Importa Modelos.PaginaProductos para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.Producto;	Importa Modelos.Producto para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.ResumenProductos;	Importa Modelos.ResumenProductos para poder usar esa clase sin escribir su nombre completo.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
9	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
10	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
11	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
12	import java.sql.Statement;	Importa java.sql.Statement para poder usar esa clase sin escribir su nombre completo.
13	import java.sql.Types;	Importa java.sql.Types para poder usar esa clase sin escribir su nombre completo.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
16	import java.util.LinkedHashMap;	Importa java.util.LinkedHashMap para poder usar esa clase sin escribir su nombre completo.
17	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
18	import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.

20	public class ProductoCRUD {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	private Connection conectar() throws SQLException {	Declara el método conectar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
27	throw new SQLException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
28	"No fue posible establecer conexión con MySQL."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	}	Cierra el bloque abierto anteriormente.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
33	}	Cierra el bloque abierto anteriormente.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	// =====	Comentario para humanos; Java no ejecuta este texto. =====
36	// RESUMEN	Comentario para humanos; Java no ejecuta este texto. RESUMEN
37	// =====	Comentario para humanos; Java no ejecuta este texto. =====
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	public ResumenProductos obtenerResumen()	Declara el método obtenerResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
40	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43	SELECT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	COUNT(*) AS total,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	SUM(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	CASE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	WHEN estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
49	AND tipo_stock = 'DIRECTO'	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
50	AND stock_minimo > 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	AND stock_actual <= stock_minimo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	THEN 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	ELSE 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	END	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	) AS stock_bajo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	SUM(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
58	CASE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	WHEN estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	THEN 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	ELSE 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	END	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

63	) AS activos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	SUM(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	CASE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	WHEN estado = FALSE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	THEN 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	ELSE 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	END	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	) AS inactivos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	FROM producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
74	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
77	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
78	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	conexion.prepareStatement(sql);	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
80	ResultSet rs =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	if (rs.next()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	return new ResumenProductos(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
86	rs.getInt("total"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
87	rs.getInt("stock_bajo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
88	rs.getInt("activos"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
89	rs.getInt("inactivos")	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
90	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	}	Cierra el bloque abierto anteriormente.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	return new ResumenProductos(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
95	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
98	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
99	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	// =====	Comentario para humanos; Java no ejecuta este texto. =====
103	// LISTAR CON FILTROS Y PAGINACIÓN	Comentario para humanos; Java no ejecuta este texto. LISTAR CON FILTROS Y PAGINACIÓN
104	// =====	Comentario para humanos; Java no ejecuta este texto. =====
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.

106	public PaginaProductos listarPagina(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	String busqueda,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
108	Integer idCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
109	String disponibilidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
110	Boolean estado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
111	int pagina,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
112	int porPagina)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	StringBuilder condiciones =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
116	new StringBuilder(" WHERE 1 = 1 ");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	List<Object> parametros =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119	new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	// BÚSQUEDA	Comentario para humanos; Java no ejecuta este texto. BÚSQUEDA
122	if (busqueda != null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
123	&& !busqueda.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	condiciones.append(""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	AND (	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127	p.nombre LIKE ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	OR p.descripcion LIKE ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	OR c.nombre LIKE ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	OR p.subCategoria LIKE ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	""");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	String texto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
135	"%" + busqueda.trim() + "%";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
139	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	}	Cierra el bloque abierto anteriormente.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	// CATEGORÍA	Comentario para humanos; Java no ejecuta este texto. CATEGORÍA
144	if (idCategoria != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	condiciones.append(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
147	" AND p.id_categoria = ? "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
148	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.

150	parametros.add(idCategoria);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	}	Cierra el bloque abierto anteriormente.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	// DISPONIBILIDAD	Comentario para humanos; Java no ejecuta este texto. DISPONIBILIDAD
154	if (disponibilidad != null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
155	&& !disponibilidad.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	condiciones.append(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
158	" AND p.disponibilidad_menu = ? "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
159	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
160		Línea en blanco: separa visualmente bloques para que el código sea más legible.
161	parametros.add(disponibilidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	// ESTADO	Comentario para humanos; Java no ejecuta este texto. ESTADO
165	if (estado != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
166		Línea en blanco: separa visualmente bloques para que el código sea más legible.
167	condiciones.append(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
168	" AND p.estado = ? "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
169	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	parametros.add(estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
172	}	Cierra el bloque abierto anteriormente.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	String sqlContar = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
175	SELECT COUNT(*) AS total	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	INNER JOIN categoria c	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	ON c.id_categoria = p.id_categoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
179	""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	+ condiciones;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	String sqlDatos = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
183	SELECT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	p.id_producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
185	p.id_categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186	c.nombre AS categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	p.subCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	p.nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	p.descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	p.precio_base,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	p.imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	p.disponibilidad_menu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	p.tipo_stock,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

194	p.personalizable,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	p.stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	p.stock_minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197	p.estado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	INNER JOIN categoria c	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
202	ON c.id_categoria = p.id_categoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
203	""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	+ condiciones	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	+ ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	ORDER BY p.id_producto DESC	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
208		Línea en blanco: separa visualmente bloques para que el código sea más legible.
209	LIMIT ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210	OFFSET ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	int totalRegistros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	// CONTAR	Comentario para humanos; Java no ejecuta este texto. CONTAR
218	try (PreparedStatement ps =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
219	conexion.prepareStatement(	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
220	sqlContar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
221	)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	colocarParametros(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	parametros	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	try (ResultSet rs =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
229	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	rs.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	totalRegistros =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
234	rs.getInt(	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
235	"total"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
236	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
237	}	Cierra el bloque abierto anteriormente.

238	}	Cierra el bloque abierto anteriormente.
239		Línea en blanco: separa visualmente bloques para que el código sea más legible.
240	// CONSULTAR PRODUCTOS	Comentario para humanos; Java no ejecuta este texto. CONSULTAR PRODUCTOS
241	List<Producto> productos =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
242	new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
243		Línea en blanco: separa visualmente bloques para que el código sea más legible.
244	try (PreparedStatement ps =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
245	conexion.prepareStatement(	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
246	sqlDatos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
248		Línea en blanco: separa visualmente bloques para que el código sea más legible.
249	int indice =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
250	colocarParametros(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
251	ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
252	parametros	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
256	indice++,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	porPagina	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
259		Línea en blanco: separa visualmente bloques para que el código sea más legible.
260	int offset =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
261	(pagina - 1)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
262	* porPagina;	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
263		Línea en blanco: separa visualmente bloques para que el código sea más legible.
264	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
265	indice,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
266	offset	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
267	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268		Línea en blanco: separa visualmente bloques para que el código sea más legible.
269	try (ResultSet rs =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
270	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.
272	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
273		Línea en blanco: separa visualmente bloques para que el código sea más legible.
274	productos.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	mapearProducto(rs)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
276	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
277	}	Cierra el bloque abierto anteriormente.
278	}	Cierra el bloque abierto anteriormente.
279	}	Cierra el bloque abierto anteriormente.
280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281	return new PaginaProductos(	Finaliza el método y devuelve el valor indicado a quien lo llamó.

282	productos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	totalRegistros	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
284	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
285	}	Cierra el bloque abierto anteriormente.
286	}	Cierra el bloque abierto anteriormente.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	// =====	Comentario para humanos; Java no ejecuta este texto. =====
289	// INSERTAR	Comentario para humanos; Java no ejecuta este texto. INSERTAR
290	// =====	Comentario para humanos; Java no ejecuta este texto. =====
291		Línea en blanco: separa visualmente bloques para que el código sea más legible.
292	public int insertar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
293	Producto producto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
294	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
295		Línea en blanco: separa visualmente bloques para que el código sea más legible.
296	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
297	INSERT INTO producto (	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
298	id_categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
299	nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
300	descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
301	precio_base,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
302	imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
303	disponibilidad_menu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
304	tipo_stock,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
305	personalizable,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
307	stock_minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
308	estado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
309	subCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
310	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
315	conexion.setAutoCommit(false);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
316	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
317	int id;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
318	try (PreparedStatement ps = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
319	sql, Statement.RETURN_GENERATED_KEYS)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
320	completarParametrosProducto(ps, producto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
322	try (ResultSet rs = ps.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
323	if (!rs.next()) throw new SQLException(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
324	"No se obtuvo el identificador del producto.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.

325	id = rs.getInt(1);	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
326	}	Cierra el bloque abierto anteriormente.
327	}	Cierra el bloque abierto anteriormente.
328	int presentacion = insertarId(conexion,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
329	"INSERT INTO presentacion_menu(id_producto_principal,nombre,tipo,precio,predeterminada,e stado) VALUES(?, 'Individual', 'INDIVIDUAL',?,TRUE,TRUE)",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	id, producto.getPrecioBase());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
331	int grupo = insertarId(conexion,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
332	"INSERT INTO grupo_presentacion(id_presentacion,nombre,minimo,maximo,permite_repetir,vis ible,permite_personalizar,estado) VALUES(?, '__Producto_principal',1,1,FALSE,FALSE,TRUE,TRUE)",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
333	presentacion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
334	int opcion = insertarId(conexion,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
335	"INSERT INTO opcion_grupo(id_grupo,nombre,incremento_precio,predeterminada,estado) VALUE S(?,?,0,TRUE,TRUE)",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	grupo, producto.getNombre());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337	try (PreparedStatement componente = conexion.prepareStatement(338 "INSERT INTO opcion_componente(id_opcion,id_producto,cantidad) VALUES(?,?,1)")) {	Inicio una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
339	componente.setInt(1, opcion);	Configura una propiedad mediante setInt en el objeto indicado.
340	componente.setInt(2, id);	Configura una propiedad mediante setInt en el objeto indicado.
341	componente.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
342	}	Cierra el bloque abierto anteriormente.
343	conexion.commit();	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
344	producto.setIdProducto(id);	Configura una propiedad mediante setIdProducto en el objeto indicado.
345	return id;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
346	} catch (SQLException RuntimeException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
347	conexion.rollback();	Revierte la transacción porque algo falló, evitando guardar un pedido o inventario incompleto.
348	throw ex;	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
349	}	Cierra el bloque abierto anteriormente.
350	}	Cierra el bloque abierto anteriormente.
351	}	Cierra el bloque abierto anteriormente.
352		Línea en blanco: separa visualmente bloques para que el código sea más legible.
353	// =====	Comentario para humanos; Java no ejecuta este texto. =====
354	// ACTUALIZAR	Comentario para humanos; Java no ejecuta este texto. ACTUALIZAR
355	// =====	Comentario para humanos; Java no ejecuta este texto. =====
356		Línea en blanco: separa visualmente bloques para que el código sea más legible.
357	public void actualizar(358 Producto producto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
360		Línea en blanco: separa visualmente bloques para que el código sea más legible.
361	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
362	UPDATE producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
363	SET	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
364	id_categoria = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
365	nombre = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

366	descripcion = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
367	precio_base = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
368	imagen = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
369	disponibilidad_menu = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
370	tipo_stock = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
371	personalizable = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
372	stock_actual = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
373	stock_minimo = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
374	estado = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
375	subCategoria = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
376	WHERE id_producto = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
377	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
380	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
381	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
382	conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
383		Línea en blanco: separa visualmente bloques para que el código sea más legible.
384	completarParametrosProducto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
385	ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
386	producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
388		Línea en blanco: separa visualmente bloques para que el código sea más legible.
389	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
390	13,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
391	producto.getIdProducto()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
393		Línea en blanco: separa visualmente bloques para que el código sea más legible.
394	int filas =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
395	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397	if (filas == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
398		Línea en blanco: separa visualmente bloques para que el código sea más legible.
399	throw new SQLException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
400	"No se encontró el producto que se desea actualizar."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
401	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
402	}	Cierra el bloque abierto anteriormente.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	try (PreparedStatement precioIndividual = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
405	"UPDATE presentacion_menu SET precio=? WHERE id_producto_principal=? AND tipo='INDIVIDUAL' A ND nombre='Individual'")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
406	precioIndividual.setBigDecimal(1, producto.getPrecioBase());	Configura una propiedad mediante setBigDecimal en el objeto indicado.
407	precioIndividual.setInt(2, producto.getIdProducto());	Configura una propiedad mediante setInt en el objeto indicado.
408	precioIndividual.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
409	}	Cierra el bloque abierto anteriormente.

410	}	Cierra el bloque abierto anteriormente.
411	}	Cierra el bloque abierto anteriormente.
412		Línea en blanco: separa visualmente bloques para que el código sea más legible.
413	// =====	Comentario para humanos; Java no ejecuta este texto. =====
414	// ACTIVAR / DESACTIVAR	Comentario para humanos; Java no ejecuta este texto. ACTIVAR / DESACTIVAR
415	// =====	Comentario para humanos; Java no ejecuta este texto. =====
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417	public void cambiarEstado(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
419	boolean estado)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
420	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
421		Línea en blanco: separa visualmente bloques para que el código sea más legible.
422	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
423	UPDATE producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
424	SET estado = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
425	WHERE id_producto = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
426	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
427		Línea en blanco: separa visualmente bloques para que el código sea más legible.
428	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
429	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
430	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
431	conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
432		Línea en blanco: separa visualmente bloques para que el código sea más legible.
433	ps.setBoolean(	Configura una propiedad mediante setBoolean en el objeto indicado.
434	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
435	estado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
436	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
437		Línea en blanco: separa visualmente bloques para que el código sea más legible.
438	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
439	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
440	idProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
441	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
442		Línea en blanco: separa visualmente bloques para que el código sea más legible.
443	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
444	}	Cierra el bloque abierto anteriormente.
445	}	Cierra el bloque abierto anteriormente.
446		Línea en blanco: separa visualmente bloques para que el código sea más legible.
447	public void desactivar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
448	int idProducto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
449	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	cambiarEstado(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
452	idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

453	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
454	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
455	}	Cierra el bloque abierto anteriormente.
456		Línea en blanco: separa visualmente bloques para que el código sea más legible.
457	public void activar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
458	int idProducto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	cambiarEstado(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
462	idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
463	true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
464	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
465	}	Cierra el bloque abierto anteriormente.
466		Línea en blanco: separa visualmente bloques para que el código sea más legible.
467	// =====	Comentario para humanos; Java no ejecuta este texto. =====
468	// OBTENER POR ID	Comentario para humanos; Java no ejecuta este texto. OBTENER POR ID
469	// =====	Comentario para humanos; Java no ejecuta este texto. =====
470		Línea en blanco: separa visualmente bloques para que el código sea más legible.
471	public Producto obtenerPorId(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
472	int idProducto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
473	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
474		Línea en blanco: separa visualmente bloques para que el código sea más legible.
475	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
476	SELECT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
477	p.id_producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
478	p.id_categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
479	c.nombre AS categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
480	p.subCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
481	p.nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
482	p.descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
483	p.precio_base,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
484	p.imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
485	p.disponibilidad_menu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
486	p.tipo_stock,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
487	p.personalizable,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
488	p.stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
489	p.stock_minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
490	p.estado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
491		Línea en blanco: separa visualmente bloques para que el código sea más legible.
492	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
493		Línea en blanco: separa visualmente bloques para que el código sea más legible.
494	INNER JOIN categoria c	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
495	ON c.id_categoria = p.id_categoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

496		Línea en blanco: separa visualmente bloques para que el código sea más legible.
497	WHERE p.id_producto = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
498	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
499		Línea en blanco: separa visualmente bloques para que el código sea más legible.
500	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
501	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
502	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
503	conexion.prepareStatement(sql) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
506	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
507	idProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
508	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
509		Línea en blanco: separa visualmente bloques para que el código sea más legible.
510	try (ResultSet rs =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
511	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
512		Línea en blanco: separa visualmente bloques para que el código sea más legible.
513	if (rs.next()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
514		Línea en blanco: separa visualmente bloques para que el código sea más legible.
515	return mapearProducto(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
516	rs	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
517	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
518	}	Cierra el bloque abierto anteriormente.
519	}	Cierra el bloque abierto anteriormente.
520	}	Cierra el bloque abierto anteriormente.
521		Línea en blanco: separa visualmente bloques para que el código sea más legible.
522	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
523	}	Cierra el bloque abierto anteriormente.
524		Línea en blanco: separa visualmente bloques para que el código sea más legible.
525	// =====	Comentario para humanos; Java no ejecuta este texto. =====
526	// CATEGORÍAS ACTIVAS	Comentario para humanos; Java no ejecuta este texto. CATEGORÍAS ACTIVAS
527	// =====	Comentario para humanos; Java no ejecuta este texto. =====
528		Línea en blanco: separa visualmente bloques para que el código sea más legible.
529	public Map<Integer, String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
530	listarCategoriasActivas()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
531	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
532		Línea en blanco: separa visualmente bloques para que el código sea más legible.
533	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
534	SELECT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
535	id_categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
536	nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
537	FROM categoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
538	WHERE estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

539	ORDER BY nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
540	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
541		Línea en blanco: separa visualmente bloques para que el código sea más legible.
542	Map<Integer, String> categorias =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
543	new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
544		Línea en blanco: separa visualmente bloques para que el código sea más legible.
545	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
546	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
547	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
548	conexion.prepareStatement(sql);	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
549	ResultSet rs =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
550	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
551		Línea en blanco: separa visualmente bloques para que el código sea más legible.
552	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
553		Línea en blanco: separa visualmente bloques para que el código sea más legible.
554	categorias.put(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
555	rs.getInt(	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
556	"id_categoria"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
557	),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
558	rs.getString(	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
559	"nombre"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
560	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
561	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
562	}	Cierra el bloque abierto anteriormente.
563	}	Cierra el bloque abierto anteriormente.
564		Línea en blanco: separa visualmente bloques para que el código sea más legible.
565	return categorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
566	}	Cierra el bloque abierto anteriormente.
567		Línea en blanco: separa visualmente bloques para que el código sea más legible.
568	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
569	* En edición incluimos también la categoría actual,	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
570	* aunque haya sido desactivada posteriormente.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
571	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
572	public Map<Integer, String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
573	listarCategoriasParaFormulario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
574	Integer categoriaActual)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
575	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
576		Línea en blanco: separa visualmente bloques para que el código sea más legible.
577	String sql;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
578		Línea en blanco: separa visualmente bloques para que el código sea más legible.
579	if (categoriaActual == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
580		Línea en blanco: separa visualmente bloques para que el código sea más legible.
581	sql = ""	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
582	SELECT id_categoria, nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

583	FROM categoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
584	WHERE estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
585	ORDER BY nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
586	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
587		Línea en blanco: separa visualmente bloques para que el código sea más legible.
588	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
589		Línea en blanco: separa visualmente bloques para que el código sea más legible.
590	sql = ""	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
591	SELECT id_categoria, nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
592	FROM categoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
593	WHERE estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
594	OR id_categoria = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
595	ORDER BY nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
596	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	}	Cierra el bloque abierto anteriormente.
598		Línea en blanco: separa visualmente bloques para que el código sea más legible.
599	Map<Integer, String> categorias =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
600	new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
601		Línea en blanco: separa visualmente bloques para que el código sea más legible.
602	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
603	Connection conexion = conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
604	PreparedStatement ps =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
605	conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
606		Línea en blanco: separa visualmente bloques para que el código sea más legible.
607	if (categoriaActual != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
608		Línea en blanco: separa visualmente bloques para que el código sea más legible.
609	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
610	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
611	categoriaActual	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
612	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
613	}	Cierra el bloque abierto anteriormente.
614		Línea en blanco: separa visualmente bloques para que el código sea más legible.
615	try (ResultSet rs =	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
616	ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
617		Línea en blanco: separa visualmente bloques para que el código sea más legible.
618	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
619		Línea en blanco: separa visualmente bloques para que el código sea más legible.
620	categorias.put(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
621	rs.getInt(	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
622	"id_categoria"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
623	),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
624	rs.getString(	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
625	"nombre"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
626	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

627	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
628	}	Cierra el bloque abierto anteriormente.
629	}	Cierra el bloque abierto anteriormente.
630	}	Cierra el bloque abierto anteriormente.
631		Línea en blanco: separa visualmente bloques para que el código sea más legible.
632	return categorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
633	}	Cierra el bloque abierto anteriormente.
634		Línea en blanco: separa visualmente bloques para que el código sea más legible.
635	// =====	Comentario para humanos; Java no ejecuta este texto. =====
636	// MÉTODOS AUXILIARES	Comentario para humanos; Java no ejecuta este texto. MÉTODOS AUXILIARES
637	// =====	Comentario para humanos; Java no ejecuta este texto. =====
638		Línea en blanco: separa visualmente bloques para que el código sea más legible.
639	private Producto mapearProducto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
640	ResultSet rs)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
641	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
642		Línea en blanco: separa visualmente bloques para que el código sea más legible.
643	Producto producto = new Producto();	Crea una nueva instancia de Producto y la usa o guarda para continuar el proceso.
644	producto.setIdProducto(rs.getInt("id_producto"));	Configura una propiedad mediante setIdProducto en el objeto indicado.
645	producto.setIdCategoria(rs.getInt("id_categoria"));	Configura una propiedad mediante setIdCategoria en el objeto indicado.
646	producto.setCategoria(rs.getString("categoria"));	Configura una propiedad mediante setCategoria en el objeto indicado.
647	producto.setNombre(rs.getString("nombre"));	Configura una propiedad mediante setNombre en el objeto indicado.
648	producto.setDescripcion(rs.getString("descripcion"));	Configura una propiedad mediante setDescripcion en el objeto indicado.
649	producto.setPrecioBase(rs.getBigDecimal("precio_base"));	Configura una propiedad mediante setPrecioBase en el objeto indicado.
650	producto.setImagen(rs.getString("imagen"));	Configura una propiedad mediante setImagen en el objeto indicado.
651	producto.setDisponibilidadMenu(rs.getString("disponibilidad_menu"));	Configura una propiedad mediante setDisponibilidadMenu en el objeto indicado.
652	producto.setTipoStock(rs.getString("tipo_stock"));	Configura una propiedad mediante setTipoStock en el objeto indicado.
653	producto.setPersonalizable(rs.getBoolean("personalizable"));	Configura una propiedad mediante setPersonalizable en el objeto indicado.
654	producto.setStockActual(rs.getInt("stock_actual"));	Configura una propiedad mediante setStockActual en el objeto indicado.
655	producto.setStockMinimo(rs.getInt("stock_minimo"));	Configura una propiedad mediante setStockMinimo en el objeto indicado.
656	producto.setActivo(rs.getBoolean("estado"));	Configura una propiedad mediante setActivo en el objeto indicado.
657	producto.setSubCategoria(rs.getString("subCategoria"));	Configura una propiedad mediante setSubCategoria en el objeto indicado.
658	return producto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
659	}	Cierra el bloque abierto anteriormente.
660		Línea en blanco: separa visualmente bloques para que el código sea más legible.
661	private int colocarParametros(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
662	PreparedStatement ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
663	List<Object> parametros)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
664	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
665		Línea en blanco: separa visualmente bloques para que el código sea más legible.
666	int indice = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
667		Línea en blanco: separa visualmente bloques para que el código sea más legible.
668	for (Object parametro	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
669	: parametros) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

670		Línea en blanco: separa visualmente bloques para que el código sea más legible.
671	ps.setObject(	Configura una propiedad mediante setObject en el objeto indicado.
672	indice++,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
673	parametro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
674	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
675	}	Cierra el bloque abierto anteriormente.
676		Línea en blanco: separa visualmente bloques para que el código sea más legible.
677	return indice;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
678	}	Cierra el bloque abierto anteriormente.
679		Línea en blanco: separa visualmente bloques para que el código sea más legible.
680	private int insertarId(Connection conexion, String sql, Object... valores)	Declara el método insertarId. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
681	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
682	try (PreparedStatement ps = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
683	sql, Statement.RETURN_GENERATED_KEYS)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
684	for (int i = 0; i < valores.length; i++) ps.setObject(i + 1, valores[i]);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
685	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
686	try (ResultSet rs = ps.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
687	if (!rs.next()) throw new SQLException("No se obtuvo el registro creado.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
688	return rs.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
689	}	Cierra el bloque abierto anteriormente.
690	}	Cierra el bloque abierto anteriormente.
691	}	Cierra el bloque abierto anteriormente.
692		Línea en blanco: separa visualmente bloques para que el código sea más legible.
693	private void completarParametrosProducto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
694	PreparedStatement ps,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
695	Producto producto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
696	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
697		Línea en blanco: separa visualmente bloques para que el código sea más legible.
698	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
699	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
700	producto.getIdCategoria())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
701	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
702		Línea en blanco: separa visualmente bloques para que el código sea más legible.
703	ps.setString(	Configura una propiedad mediante setString en el objeto indicado.
704	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
705	producto.getNombre())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
706	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
707		Línea en blanco: separa visualmente bloques para que el código sea más legible.
708	ps.setString(	Configura una propiedad mediante setString en el objeto indicado.
709	3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
710	producto.getDescripcion())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
711	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
712		Línea en blanco: separa visualmente bloques para que el código sea más legible.
713	ps.setBigDecimal(	Configura una propiedad mediante setBigDecimal en el objeto indicado.

714	4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
715	producto.getPrecioBase()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
716	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
717		Línea en blanco: separa visualmente bloques para que el código sea más legible.
718	if (producto.getImagen() == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
719	producto.getImagen().isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
720		Línea en blanco: separa visualmente bloques para que el código sea más legible.
721	ps.setNull(	Configura una propiedad mediante setNull en el objeto indicado.
722	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
723	Types.VARCHAR	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
724	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
725		Línea en blanco: separa visualmente bloques para que el código sea más legible.
726	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
727		Línea en blanco: separa visualmente bloques para que el código sea más legible.
728	ps.setString(	Configura una propiedad mediante setString en el objeto indicado.
729	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
730	producto.getImagen()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
731	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
732	}	Cierra el bloque abierto anteriormente.
733		Línea en blanco: separa visualmente bloques para que el código sea más legible.
734	ps.setString(	Configura una propiedad mediante setString en el objeto indicado.
735	6,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
736	producto.getDisponibilidadMenu()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
737	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
738		Línea en blanco: separa visualmente bloques para que el código sea más legible.
739	ps.setString(7, producto.getTipoStock());	Configura una propiedad mediante setString en el objeto indicado.
740		Línea en blanco: separa visualmente bloques para que el código sea más legible.
741	ps.setBoolean(	Configura una propiedad mediante setBoolean en el objeto indicado.
742	8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
743	producto.isPersonalizable()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
744	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
745		Línea en blanco: separa visualmente bloques para que el código sea más legible.
746	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
747	9,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
748	producto.getStockActual()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
749	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
750		Línea en blanco: separa visualmente bloques para que el código sea más legible.
751	ps.setInt(	Configura una propiedad mediante setInt en el objeto indicado.
752	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
753	producto.getStockMinimo()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
754	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
755		Línea en blanco: separa visualmente bloques para que el código sea más legible.
756	ps.setBoolean(	Configura una propiedad mediante setBoolean en el objeto indicado.
757	11,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

758	producto.isActivo()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
759	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
760		Línea en blanco: separa visualmente bloques para que el código sea más legible.
761	String subCategoria = producto.getSubCategoria();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
762	if (subCategoria == null subCategoria.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
763	ps.setNull(12, Types.VARCHAR);	Configura una propiedad mediante setNull en el objeto indicado.
764	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
765	ps.setString(12, subCategoria.trim());	Configura una propiedad mediante setString en el objeto indicado.
766	}	Cierra el bloque abierto anteriormente.
767	}	Cierra el bloque abierto anteriormente.
768	}	Cierra el bloque abierto anteriormente.

16.47 src/CRUD/UsuarioCRUD.java

Operaciones de creación, consulta, actualización y estado para usuario usando JDBC.

Tipo: java | Líneas: 340 | Archivo 47 de 157

Clases/tipos detectados: UsuarioCRUD, FiltroSql

Línea	Constructor o método detectado
27	public PaginaUsuarios listarPagina(
48	public List<Usuario> listarParaExportar(
59	public ResumenUsuarios obtenerResumen() throws SQLException {
82	public int insertar(Usuario usuario, String passwordHash)
111	public void actualizar(Usuario usuario, String passwordHash)
149	public void desactivar(int idUsuario) throws SQLException {
158	public boolean existeNombreUsuario(String nombreUsuario, int idExcluido)
163	public boolean existeCorreo(String correo, int idExcluido)
168	public int contarAdministradoresActivos() throws SQLException {
182	private boolean existeValor(String columna, String valor, int idExcluido)
201	private List<Usuario> consultarUsuarios(
231	private int contarFiltrados(Connection conexion, FiltroSql filtro)
243	private FiltroSql construirFiltro(
272	private int colocarParametros(
286	private void colocarDatosComunes(
295	private void colocarHorario(
312	private Usuario mapear(ResultSet resultado) throws SQLException {
330	private Connection abrirConexion() throws SQLException {
338	private record FiltroSql(String where, List<Object> parametros) {

1	package CRUD;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.PaginaUsuarios;	Importa Modelos.PaginaUsuarios para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.ResumenUsuarios;	Importa Modelos.ResumenUsuarios para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.Usuario;	Importa Modelos.Usuario para poder usar esa clase sin escribir su nombre completo.
7	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
8	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
9	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
10	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
11	import java.sql.Timestamp;	Importa java.sql.Timestamp para poder usar esa clase sin escribir su nombre completo.

12	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
13	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	* Centraliza todas las consultas del módulo de usuarios.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
18	* La vista nunca arma SQL: solamente entrega filtros, página y datos. Todos	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	* los valores variables se envían con PreparedStatement.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
20	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	public class UsuarioCRUD {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	* Obtiene una página de resultados y el total filtrado usando la misma	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	* conexión. Así la interfaz puede calcular la cantidad de páginas.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
26	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
27	public PaginaUsuarios listarPagina(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	String busqueda,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	String rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	Boolean estado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	int pagina,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	int tamanoPagina) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	int paginaSegura = Math.max(1, pagina);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
35	int tamanoSeguro = Math.max(1, tamanoPagina);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36	int desplazamiento = (paginaSegura - 1) * tamanoSeguro;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	FiltroSql filtro = construirFiltro(busqueda, rol, estado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	try (Connection conexion = abrirConexion()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
40	int total = contarFiltrados(conexion, filtro);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
41	List<Usuario> usuarios = consultarUsuarios(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42	conexion, filtro, tamanoSeguro, desplazamiento, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	return new PaginaUsuarios(usuarios, total);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
44	}	Cierra el bloque abierto anteriormente.
45	}	Cierra el bloque abierto anteriormente.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	/** Devuelve todos los resultados filtrados, sin LIMIT, para exportar. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
48	public List<Usuario> listarParaExportar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	String busqueda,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	String rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	Boolean estado) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	FiltroSql filtro = construirFiltro(busqueda, rol, estado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	try (Connection conexion = abrirConexion()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
54	return consultarUsuarios(conexion, filtro, 0, 0, false);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
55	}	Cierra el bloque abierto anteriormente.

56	}	Cierra el bloque abierto anteriormente.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	/** Las tarjetas son globales y no cambian al aplicar filtros. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
59	public ResumenUsuarios obtenerResumen() throws SQLException {	Declara el método obtenerResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
60	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
61	SELECT COUNT(*) AS total,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	COALESCE(SUM(rol = 'ADMINISTRADOR' AND estado = TRUE), 0)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
63	AS administradores,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	COALESCE(SUM(rol = 'CAJERO' AND estado = TRUE), 0)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
65	AS cajeros,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	COALESCE(SUM(estado = FALSE), 0) AS inactivos	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
67	FROM usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	try (Connection conexion = abrirConexion();	Inicio una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
71	PreparedStatement sentencia = conexion.prepareStatement(sql);	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
72	ResultSet resultado = sentencia.executeQuery() {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
73	resultado.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
74	return new ResumenUsuarios(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
75	resultado.getInt("total"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
76	resultado.getInt("administradores"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
77	resultado.getInt("cajeros"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
78	resultado.getInt("inactivos"));	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
79	}	Cierra el bloque abierto anteriormente.
80	}	Cierra el bloque abierto anteriormente.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	public int insertar(Usuario usuario, String passwordHash)	Declara el método insertar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
83	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
85	INSERT INTO usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	(nombre, apellido, usuario, correo, password_hash, rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
87	estado, turno, hora_inicio, hora_fin)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
89	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	try (Connection conexion = abrirConexion();	Inicio una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
92	PreparedStatement sentencia = conexion.prepareStatement(	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
93	sql, PreparedStatement.RETURN_GENERATED_KEYS) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
94	colocarDatosComunes(sentencia, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	sentencia.setString(5, passwordHash);	Configura una propiedad mediante setString en el objeto indicado.
96	sentencia.setString(6, usuario.getRol());	Configura una propiedad mediante setString en el objeto indicado.
97	sentencia.setBoolean(7, usuario.isActivo());	Configura una propiedad mediante setBoolean en el objeto indicado.
98	colocarHorario(sentencia, 8, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99	sentencia.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.

100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	try (ResultSet claves = sentencia.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
102	return claves.next() ? claves.getInt(1) : 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	}	Cierra el bloque abierto anteriormente.
104	}	Cierra el bloque abierto anteriormente.
105	}	Cierra el bloque abierto anteriormente.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
108	* Si passwordHash es null, conserva la contraseña actual. Nunca se carga	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
109	* el hash existente en el formulario ni se vuelve a escribir sin motivo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
110	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
111	public void actualizar(Usuario usuario, String passwordHash)	Declara el método actualizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
112	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	boolean cambiaContrasena = passwordHash != null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	String sql = cambiaContrasena	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	? ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
116	UPDATE usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117	SET nombre = ?, apellido = ?, usuario = ?, correo = ?,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
118	rol = ?, estado = ?, turno = ?, hora_inicio = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
119	hora_fin = ?, password_hash = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
120	WHERE id_usuario = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
121	""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
122	:	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
123	UPDATE usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
124	SET nombre = ?, apellido = ?, usuario = ?, correo = ?,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
125	rol = ?, estado = ?, turno = ?, hora_inicio = ?,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
126	hora_fin = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
127	WHERE id_usuario = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
128	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
130	try (Connection conexion = abrirConexion());	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
131	PreparedStatement sentencia = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
132	colocarDatosComunes(sentencia, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	sentencia.setString(5, usuario.getRol());	Configura una propiedad mediante setString en el objeto indicado.
134	sentencia.setBoolean(6, usuario.isActivo());	Configura una propiedad mediante setBoolean en el objeto indicado.
135	colocarHorario(sentencia, 7, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	int indiceId;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
137	if (cambiaContrasena) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
138	sentencia.setString(10, passwordHash);	Configura una propiedad mediante setString en el objeto indicado.
139	indiceId = 11;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
140	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
141	indiceId = 10;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
142	}	Cierra el bloque abierto anteriormente.
143	sentencia.setInt(indiceId, usuario.getIdUsuario());	Configura una propiedad mediante setInt en el objeto indicado.

144	sentencia.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
145	}	Cierra el bloque abierto anteriormente.
146	}	Cierra el bloque abierto anteriormente.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.
148	/** Eliminación lógica: conserva pedidos y permite una futura reactivación. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
149	public void desactivar(int idUsuario) throws SQLException {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
150	String sql = "UPDATE usuario SET estado = FALSE WHERE id_usuario = ?";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
151	try (Connection conexion = abrirConexion();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
152	PreparedStatement sentencia = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
153	sentencia.setInt(1, idUsuario);	Configura una propiedad mediante setInt en el objeto indicado.
154	sentencia.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
155	}	Cierra el bloque abierto anteriormente.
156	}	Cierra el bloque abierto anteriormente.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	public boolean existeNombreUsuario(String nombreUsuario, int idExcluido)	Declara el método existeNombreUsuario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
159	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
160	return existeValor("usuario", nombreUsuario, idExcluido);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	public boolean existeCorreo(String correo, int idExcluido)	Declara el método existeCorreo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
164	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165	return existeValor("correo", correo, idExcluido);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
166	}	Cierra el bloque abierto anteriormente.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	public int contarAdministradoresActivos() throws SQLException {	Declara el método contarAdministradoresActivos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
169	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
170	SELECT COUNT(*)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	FROM usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	WHERE rol = 'ADMINISTRADOR' AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
173	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
174	try (Connection conexion = abrirConexion();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
175	PreparedStatement sentencia = conexion.prepareStatement(sql);	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
176	ResultSet resultado = sentencia.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
177	resultado.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
178	return resultado.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
179	}	Cierra el bloque abierto anteriormente.
180	}	Cierra el bloque abierto anteriormente.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	private boolean existeValor(String columna, String valor, int idExcluido)	Declara el método existeValor. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
183	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	// "columna" nunca viene del usuario; solamente se llama con las dos	Comentario para humanos; Java no ejecuta este texto. "columna" nunca viene del usuario; solamente se llama con las dos
185	// constantes anteriores. El valor sí se parametriza.	Comentario para humanos; Java no ejecuta este texto. constantes anteriores. El valor sí se parametriza.
186	String sql = "SELECT COUNT(*) FROM usuario WHERE " + columna + " = ?"	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

187	+ (idExcluido > 0 ? " AND id_usuario <> ?" : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188	try (Connection conexion = abrirConexion());	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
189	PreparedStatement sentencia = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
190	sentencia.setString(1, valor);	Configura una propiedad mediante setString en el objeto indicado.
191	if (idExcluido > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
192	sentencia.setInt(2, idExcluido);	Configura una propiedad mediante setInt en el objeto indicado.
193	}	Cierra el bloque abierto anteriormente.
194	try (ResultSet resultado = sentencia.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
195	resultado.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
196	return resultado.getInt(1) > 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
197	}	Cierra el bloque abierto anteriormente.
198	}	Cierra el bloque abierto anteriormente.
199	}	Cierra el bloque abierto anteriormente.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	private List<Usuario> consultarUsuarios(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
202	Connection conexion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
203	FiltroSql filtro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	int limite,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	int desplazamiento,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
206	boolean paginar) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
207	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
208	SELECT id_usuario, nombre, apellido, usuario, correo, rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	estado, fecha_creacion, turno, hora_inicio, hora_fin	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210	FROM usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	"" + filtro.where() + " ORDER BY id_usuario ASC"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212	+ (paginar ? " LIMIT ? OFFSET ?" : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
213		Línea en blanco: separa visualmente bloques para que el código sea más legible.
214	try (PreparedStatement sentencia = conexion.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
215	int indice = colocarParametros(sentencia, filtro.parametros());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
216	if (paginar) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
217	sentencia.setInt(indice++, limite);	Configura una propiedad mediante setInt en el objeto indicado.
218	sentencia.setInt(indice, desplazamiento);	Configura una propiedad mediante setInt en el objeto indicado.
219	}	Cierra el bloque abierto anteriormente.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	List<Usuario> usuarios = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
222	try (ResultSet resultado = sentencia.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
223	while (resultado.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
224	usuarios.add(mapear(resultado));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
225	}	Cierra el bloque abierto anteriormente.
226	}	Cierra el bloque abierto anteriormente.
227	return usuarios;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
228	}	Cierra el bloque abierto anteriormente.
229	}	Cierra el bloque abierto anteriormente.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.

231	private int contarFiltrados(Connection conexion, FiltroSql filtro)	Declara el método contarFiltrados. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
232	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
233	String sql = "SELECT COUNT(*) FROM usuario " + filtro.where();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
234	try (PreparedStatement sentencia = conexion.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
235	colocarParametros(sentencia, filtro.parametros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	try (ResultSet resultado = sentencia.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
237	resultado.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
238	return resultado.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
239	}	Cierra el bloque abierto anteriormente.
240	}	Cierra el bloque abierto anteriormente.
241	}	Cierra el bloque abierto anteriormente.
242		Línea en blanco: separa visualmente bloques para que el código sea más legible.
243	private FiltroSql construirFiltro(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244	String busqueda,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
245	String rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246	Boolean estado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	List<String> condiciones = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
248	List<Object> parametros = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	if (busqueda != null && !busqueda.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
251	condiciones.add("""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
252	LOWER(CONCAT_WS(' ', nombre, apellido, usuario, correo))	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	LIKE LOWER(?)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	""").trim());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	parametros.add("%" + busqueda.trim() + "%");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
256	}	Cierra el bloque abierto anteriormente.
257	if (rol != null && !rol.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
258	condiciones.add("rol = ?");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
259	parametros.add(rol);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
260	}	Cierra el bloque abierto anteriormente.
261	if (estado != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
262	condiciones.add("estado = ?");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
263	parametros.add(estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
264	}	Cierra el bloque abierto anteriormente.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	String where = condiciones.isEmpty()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
267	? ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
268	: "WHERE " + String.join(" AND ", condiciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	return new FiltroSql(where, parametros);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
270	}	Cierra el bloque abierto anteriormente.
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.
272	private int colocarParametros(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	PreparedStatement sentencia,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	List<Object> parametros) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

275	int indice = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
276	for (Object parametro : parametros) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
277	if (parametro instanceof Boolean valorBooleano) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
278	sentencia.setBoolean(indice++, valorBooleano);	Configura una propiedad mediante setBoolean en el objeto indicado.
279	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
280	sentencia.setString(indice++, String.valueOf(parametro));	Configura una propiedad mediante setString en el objeto indicado.
281	}	Cierra el bloque abierto anteriormente.
282	}	Cierra el bloque abierto anteriormente.
283	return indice;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
284	}	Cierra el bloque abierto anteriormente.
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	private void colocarDatosComunes(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
287	PreparedStatement sentencia,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
288	Usuario usuario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
289	sentencia.setString(1, usuario.getNombre());	Configura una propiedad mediante setString en el objeto indicado.
290	sentencia.setString(2, usuario.getApellido());	Configura una propiedad mediante setString en el objeto indicado.
291	sentencia.setString(3, usuario.getNombreUsuario());	Configura una propiedad mediante setString en el objeto indicado.
292	sentencia.setString(4, usuario.getCorreo());	Configura una propiedad mediante setString en el objeto indicado.
293	}	Cierra el bloque abierto anteriormente.
294		Línea en blanco: separa visualmente bloques para que el código sea más legible.
295	private void colocarHorario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
296	PreparedStatement sentencia,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
297	int indice,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
298	Usuario usuario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
299	sentencia.setString(indice++, usuario.getTurno());	Configura una propiedad mediante setString en el objeto indicado.
300	if (usuario.getHoraInicio() == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
301	sentencia.setNull(indice++, java.sql.Types.TIME);	Configura una propiedad mediante setNull en el objeto indicado.
302	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
303	sentencia.setString(indice++, usuario.getHoraInicio().toString());	Configura una propiedad mediante setString en el objeto indicado.
304	}	Cierra el bloque abierto anteriormente.
305	if (usuario.getHoraFin() == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
306	sentencia.setNull(indice, java.sql.Types.TIME);	Configura una propiedad mediante setNull en el objeto indicado.
307	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
308	sentencia.setString(indice, usuario.getHoraFin().toString());	Configura una propiedad mediante setString en el objeto indicado.
309	}	Cierra el bloque abierto anteriormente.
310	}	Cierra el bloque abierto anteriormente.
311		Línea en blanco: separa visualmente bloques para que el código sea más legible.
312	private Usuario mapear(ResultSet resultado) throws SQLException {	Declara el método mapear. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
313	Timestamp fecha = resultado.getTimestamp("fecha_creacion");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
314	String inicio = resultado.getString("hora_inicio");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
315	String fin = resultado.getString("hora_fin");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
316	return new Usuario(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
317	resultado.getInt("id_usuario"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
318	resultado.getString("nombre"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.

319	resultado.getString("apellido"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
320	resultado.getString("usuario"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
321	resultado.getString("correo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
322	resultado.getString("rol"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
323	resultado.getBoolean("estado"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
324	fecha != null ? fecha.toLocalDateTime() : null,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
325	resultado.getString("turno"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
326	inicio != null ? java.time.LocalDateTime.parse(inicio) : null,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
327	fin != null ? java.time.LocalDateTime.parse(fin) : null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
328	}	Cierra el bloque abierto anteriormente.
329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
330	private Connection abrirConexion() throws SQLException {	Declara el método abrirConexion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
331	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
332	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
333	throw new SQLException("No fue posible conectarse a Waldonalds.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
334	}	Cierra el bloque abierto anteriormente.
335	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
336	}	Cierra el bloque abierto anteriormente.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	private record FiltroSql(String where, List<Object> parametros) {	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
339	}	Cierra el bloque abierto anteriormente.
340	}	Cierra el bloque abierto anteriormente.

16.48 src/DAO/ConfiguracionMenuDAO.java

Carga y administra presentaciones, grupos, opciones, componentes y recetas configurables.

Tipo: java | Líneas: 304 | Archivo 48 de 157

Clases/tipos detectados: ConfiguracionMenuDAO

Línea	Constructor o método detectado
14	private Connection conectar() throws SQLException {
20	public ConfiguracionProducto cargarProducto(int idProducto) throws SQLException {
29	public ConfiguracionProducto cargarProductoAdmin(int idProducto) throws SQLException {
33	public ConfiguracionProducto cargarProducto(Connection c, int idProducto,
38	private ConfiguracionProducto cargarProducto(Connection c, int idProducto,
81	private List<GrupoMenu> cargarGrupos(Connection c, int idPresentacion,
103	private List<OpcionMenu> cargarOpciones(Connection c, int idGrupo,
125	private int contarComponentes(Connection c, int idOpcion) throws SQLException {
133	private List<ComponenteMenu> cargarComponentes(Connection c, int idOpcion,
161	public List<IngredienteProducto> cargarIngredientes(Connection c, int idProducto,
183	public List<IngredienteProducto> cargarIngredientes(int idProducto) throws SQLException {
187	public void asegurarPresentacionIndividual(int idProducto) throws SQLException {
213	public int guardarPresentacion(Integer id, int idProducto, String nombre,
223	public int guardarGrupo(Integer id, int idPresentacion, String nombre, int minimo,
232	public int guardarOpcion(Integer id, int idGrupo, String nombre,
242	public void agregarComponente(int idOpcion, int idProducto, int cantidad) throws SQLException {
246	public void quitarComponente(int idOpcion, int idProducto) throws SQLException {
252	public void desactivar(String tabla, String campold, int id) throws SQLException {
258	public Map<Integer,String> listarProductos() throws SQLException {
262	public Map<Integer,String> listarIngredientes() throws SQLException {
266	private Map<Integer,String> listarNombres(String sql) throws SQLException {
274	public void guardarIngredienteProducto(Integer id, int idProducto, int idIngrediente,
283	public void desactivarIngredienteProducto(int id) throws SQLException {
287	private static int insertarId(Connection c, String sql, Object... valores) throws SQLException {
297	private static void ejecutar(Connection c, String sql, Object... valores) throws SQLException {
301	private static void asignar(PreparedStatement s, Object... valores) throws SQLException {

1	package DAO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.*;	Importa Modelos.* para poder usar esa clase sin escribir su nombre completo.
5	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
6	import java.sql.*;	Importa java.sql.* para poder usar esa clase sin escribir su nombre completo.
7	import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa una hora sin fecha.
8	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
9	import Utilidades.HorarioMenu;	Importa Utilidades.HorarioMenu para poder usar esa clase sin escribir su nombre completo.
10		Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	/** Lee y administra presentaciones, elecciones, componentes y recetas. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
12	public class ConfiguracionMenuDAO {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	private Connection conectar() throws SQLException {	Declara el método conectar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
15	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
16	if (conexion == null) throw new SQLException("No fue posible conectar con MySQL.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
17	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
18	}	Cierra el bloque abierto anteriormente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	public ConfiguracionProducto cargarProducto(int idProducto) throws SQLException {	Declara el método cargarProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
21	try (Connection c = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
22	ConfiguracionProducto resultado = cargarProducto(c, idProducto, true, true);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	if (!HorarioMenu.estaDisponible(resultado.disponibilidadMenu(), LocalDateTime.now()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
24	throw new IllegalArgumentException("Este producto no está disponible en este horario.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
25	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
26	}	Cierra el bloque abierto anteriormente.
27	}	Cierra el bloque abierto anteriormente.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	public ConfiguracionProducto cargarProductoAdmin(int idProducto) throws SQLException {	Declara el método cargarProductoAdmin. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
30	try (Connection c = conectar()) { return cargarProducto(c, idProducto, true, false); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
31	}	Cierra el bloque abierto anteriormente.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	public ConfiguracionProducto cargarProducto(Connection c, int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	boolean soloActivos) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	return cargarProducto(c, idProducto, soloActivos, true);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	private ConfiguracionProducto cargarProducto(Connection c, int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	boolean soloActivos, boolean validarHorario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	String nombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
41	String descripcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42	String imagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

43	String disponibilidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
44	try (PreparedStatement s = c.prepareStatement("SELECT nombre,descripcion,imagen,disponibilidad_menu,esta do FROM producto WHERE id_producto=?")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
45	s.setInt(1, idProducto);	Configura una propiedad mediante setInt en el objeto indicado.
46	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
47	if (!r.next()) throw new IllegalArgumentException("El producto ya no existe.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
48	if (soloActivos && !r.getBoolean("estado"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
49	throw new IllegalArgumentException("El producto está inactivo.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
50	nombre = r.getString("nombre");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
51	descripcion = r.getString("descripcion");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
52	imagen = r.getString("imagen");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
53	disponibilidad = r.getString("disponibilidad_menu");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
54	}	Cierra el bloque abierto anteriormente.
55	}	Cierra el bloque abierto anteriormente.
56	List<PresentacionMenu> presentaciones = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
57	String sql = "SELECT id_presentacion,nombre,tipo,precio,predeterminada FROM presentacion_menu "	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58	+ "WHERE id_producto_principal=?" + (soloActivos ? " AND estado=TRUE" : "")	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
59	+ " ORDER BY predeterminada DESC,id_presentacion";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
61	s.setInt(1, idProducto);	Configura una propiedad mediante setInt en el objeto indicado.
62	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
63	while (r.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
64	int idPresentacion = r.getInt("id_presentacion");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
65	List<GrupoMenu> grupos = cargarGrupos(c, idPresentacion, soloActivos, validarHorario);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	if (validarHorario && (grupos.isEmpty() grupos.stream()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
67	.anyMatch(g -> g.minimo() > 0 && g.opciones().isEmpty())) continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68	presentaciones.add(new PresentacionMenu(idPresentacion, idProducto,	Crea una nueva instancia de PresentacionMenu y la usa o guarda para continuar el proceso.
69	r.getString("nombre"), r.getString("tipo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
70	r.getBigDecimal("precio"), r.getBoolean("predeterminada"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
71	grupos));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	}	Cierra el bloque abierto anteriormente.
73	}	Cierra el bloque abierto anteriormente.
74	}	Cierra el bloque abierto anteriormente.
75	if (presentaciones.isEmpty())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
76	throw new IllegalArgumentException("Este producto todavía no tiene una presentación configurada.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
77	return new ConfiguracionProducto(idProducto, nombre, descripcion, imagen,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
78	disponibilidad, presentaciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	}	Cierra el bloque abierto anteriormente.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	private List<GrupoMenu> cargarGrupos(Connection c, int idPresentacion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	boolean soloActivos, boolean validarHorario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	List<GrupoMenu> grupos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
84	String sql = "SELECT id_grupo,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar "	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
85	+ "FROM grupo_presentacion WHERE id_presentacion=?"	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
86	+ (soloActivos ? " AND estado=TRUE" : "") + " ORDER BY id_grupo";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

87	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
88	s.setInt(1, idPresentacion);	Configura una propiedad mediante setInt en el objeto indicado.
89	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
90	while (r.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
91	int idGrupo = r.getInt("id_grupo");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
92	grupos.add(new GrupoMenu(idGrupo, r.getString("nombre"),	Crea una nueva instancia de GrupoMenu y la usa o guarda para continuar el proceso.
93	r.getInt("minimo"), r.getInt("maximo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
94	r.getBoolean("permite_repetir"), r.getBoolean("visible"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
95	r.getBoolean("permite_personalizar"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
96	cargarOpciones(c, idGrupo, soloActivos, validarHorario));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	}	Cierra el bloque abierto anteriormente.
98	}	Cierra el bloque abierto anteriormente.
99	}	Cierra el bloque abierto anteriormente.
100	return grupos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
101	}	Cierra el bloque abierto anteriormente.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	private List<OpcionMenu> cargarOpciones(Connection c, int idGrupo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
104	boolean soloActivos, boolean validarHorario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
105	List<OpcionMenu> opciones = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
106	String sql = "SELECT id_opcion,nombre,incremento_precio,predeterminada FROM opcion_grupo "	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107	+ "WHERE id_grupo=?" + (soloActivos ? " AND estado=TRUE" : "")	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
108	+ " ORDER BY predeterminada DESC,id_opcion";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
110	s.setInt(1, idGrupo);	Configura una propiedad mediante setInt en el objeto indicado.
111	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
112	while (r.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
113	int idOpcion = r.getInt("id_opcion");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
114	List<ComponenteMenu> componentes = cargarComponentes(c, idOpcion, soloActivos, validarHorario);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	if (soloActivos && componentes.size() != contarComponentes(c, idOpcion)) continue;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
116	opciones.add(new OpcionMenu(idOpcion, r.getString("nombre"),	Crea una nueva instancia de OpcionMenu y la usa o guarda para continuar el proceso.
117	r.getBigDecimal("incremento_precio"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
118	r.getBoolean("predeterminada"), componentes));	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
119	}	Cierra el bloque abierto anteriormente.
120	}	Cierra el bloque abierto anteriormente.
121	}	Cierra el bloque abierto anteriormente.
122	return opciones;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
123	}	Cierra el bloque abierto anteriormente.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	private int contarComponentes(Connection c, int idOpcion) throws SQLException {	Declara el método contarComponentes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
126	try (PreparedStatement s = c.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
127	"SELECT COUNT(*) FROM opcion_componente WHERE id_opcion=?") {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
128	s.setInt(1, idOpcion);	Configura una propiedad mediante setInt en el objeto indicado.
129	try (ResultSet r = s.executeQuery()) { r.next(); return r.getInt(1); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

130	}	Cierra el bloque abierto anteriormente.
131	}	Cierra el bloque abierto anteriormente.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	private List<ComponenteMenu> cargarComponentes(Connection c, int idOpcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
134	boolean soloActivos, boolean validarHorario) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	List<ComponenteMenu> componentes = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
136	String sql = "SELECT oc.id_producto,oc.cantidad,p.nombre,p.tipo_stock,p.personalizable,p.disponibilidad_	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
	menu,p.estado "	
137	+ "FROM opcion_componente oc JOIN producto p ON p.id_producto=oc.id_producto "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
138	+ "WHERE oc.id_opcion=? ORDER BY oc.id_opcion_componente";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
139	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
140	s.setInt(1, idOpcion);	Configura una propiedad mediante setInt en el objeto indicado.
141	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
142	while (r.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
143	if (soloActivos && (!r.getBoolean("estado"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
144	(validarHorario && !HorarioMenu.estaDisponible(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	r.getString("disponibilidad_menu"), LocalDateTime.now()))	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
146	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	int idProducto = r.getInt("id_producto");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
148	String tipoStock = r.getString("tipo_stock");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
149	boolean personalizable = r.getBoolean("personalizable");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
150	componentes.add(new ComponenteMenu(idProducto, r.getString("nombre"),	Crea una nueva instancia de ComponenteMenu y la usa o guarda para continuar el proceso.
151	r.getInt("cantidad"), tipoStock,	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
152	personalizable, r.getString("disponibilidad_menu"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
153	"RECETA".equals(tipoStock) && personalizable	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	? cargarIngredientes(c, idProducto, soloActivos) : List.of());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156	}	Cierra el bloque abierto anteriormente.
157	}	Cierra el bloque abierto anteriormente.
158	return componentes;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
159	}	Cierra el bloque abierto anteriormente.
160		Línea en blanco: separa visualmente bloques para que el código sea más legible.
161	public List<IngredienteProducto> cargarIngredientes(Connection c, int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
162	boolean soloActivos) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
163	List<IngredienteProducto> ingredientes = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
164	String sql = "SELECT pi.id_producto_ingrediente,pi.id_ingrediente,i.nombre,pi.cantidad_default,"	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
165	+ "pi.permite_quitar,pi.permite_extra,pi.cantidad_extra,pi.precio_extra,pi.max_extras "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	+ "FROM producto_ingrediente pi JOIN ingrediente i ON i.id_ingrediente=pi.id_ingrediente "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
167	+ "WHERE pi.id_producto=?" + (soloActivos ? " AND pi.estado=TRUE AND i.estado=TRUE" : "")	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
168	+ " ORDER BY i.nombre";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
170	s.setInt(1, idProducto);	Configura una propiedad mediante setInt en el objeto indicado.
171	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
172	while (r.next()) ingredientes.add(new IngredienteProducto(	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
173	r.getInt("id_producto_ingrediente"), r.getInt("id_ingrediente"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.

174	<code>r.getString("nombre"), r.getBigDecimal("cantidad_default"),</code>	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
175	<code>r.getBoolean("permite_quitar"), r.getBoolean("permite_extra"),</code>	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
176	<code>r.getBigDecimal("cantidad_extra"), r.getBigDecimal("precio_extra"),</code>	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
177	<code>r.getInt("max_extras")));</code>	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
178	<code>}</code>	Cierra el bloque abierto anteriormente.
179	<code>}</code>	Cierra el bloque abierto anteriormente.
180	<code>return ingredientes;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
181	<code>}</code>	Cierra el bloque abierto anteriormente.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.
183	<code>public List<IngredienteProducto> cargarIngredientes(int idProducto) throws SQLException {</code>	Declara el método cargarIngredientes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
184	<code>try (Connection c = conectar()) { return cargarIngredientes(c, idProducto, true); }</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
185	<code>}</code>	Cierra el bloque abierto anteriormente.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	<code>public void asegurarPresentacionIndividual(int idProducto) throws SQLException {</code>	Declara el método asegurarPresentacionIndividual. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
188	<code>try (Connection c = conectar()) {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
189	<code>c.setAutoCommit(false);</code>	Configura una propiedad mediante setAutoCommit en el objeto indicado.
190	<code>try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
191	<code>try (PreparedStatement existe = c.prepareStatement("SELECT id_presentacion FROM presentacion_menu WHERE id_producto_principal=? LIMIT 1")) {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
192	<code>existe.setInt(1, idProducto);</code>	Configura una propiedad mediante setInt en el objeto indicado.
193	<code>try (ResultSet r = existe.executeQuery()) { if (r.next()) { c.rollback(); return; } }</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
194	<code>}</code>	Cierra el bloque abierto anteriormente.
195	<code>String nombre;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
196	<code>BigDecimal precio;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	<code>try (PreparedStatement p = c.prepareStatement("SELECT nombre,precio_base FROM producto WHERE id_producto=?")) {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
198	<code>p.setInt(1, idProducto);</code>	Configura una propiedad mediante setInt en el objeto indicado.
199	<code>try (ResultSet r = p.executeQuery()) {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
200	<code>if (!r.next()) throw new SQLException("No se encontró el producto.");</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
201	<code>nombre = r.getString(1); precio = r.getBigDecimal(2);</code>	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
202	<code>}</code>	Cierra el bloque abierto anteriormente.
203	<code>}</code>	Cierra el bloque abierto anteriormente.
204	<code>int presentacion = insertarId(c, "INSERT INTO presentacion_menu(id_producto_principal,nombre,tipo,precio,predeterminada,estado) VALUES(?,?, 'INDIVIDUAL',?,TRUE,TRUE)", idProducto, "Individual", precio);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
205	<code>int grupo = insertarId(c, "INSERT INTO grupo_presentacion(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado) VALUES(?, '___Producto_principal',1,1,FALSE,FALSE,TRUE,TRUE)", presentacion);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
206	<code>int opcion = insertarId(c, "INSERT INTO opcion_grupo(id_grupo,nombre,incremento_precio,predeterminada,estado) VALUES(?,?,0,TRUE,TRUE)", grupo, nombre);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
207	<code>ejecutar(c, "INSERT INTO opcion_componente(id_opcion,id_producto,cantidad) VALUES(?,?,1)", opcion, idProducto);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
208	<code>c.commit();</code>	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
209	<code>} catch (SQLException RuntimeException ex) { c.rollback(); throw ex; }</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
210	<code>}</code>	Cierra el bloque abierto anteriormente.
211	<code>}</code>	Cierra el bloque abierto anteriormente.

212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	public int guardarPresentacion(Integer id, int idProducto, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	String tipo, BigDecimal precio, boolean predeterminada) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	try (Connection c = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
216	if (predeterminada) ejecutar(c, "UPDATE presentacion_menu SET predeterminada=FALSE WHERE id_producto_principal=?", idProducto);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
217	if (id == null) return insertarId(c, "INSERT INTO presentacion_menu(id_producto_principal,nombre,tipo,precio,predeterminada,estado) VALUES(?,?,?,?,?,TRUE)", idProducto,nombre,tipo,precio,predeterminada);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
218	ejecutar(c, "UPDATE presentacion_menu SET nombre=?,tipo=?,precio=?,predeterminada=?,estado=TRUE WHERE id_presentacion=?", nombre,tipo,precio,predeterminada,id);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
219	return id;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
220	}	Cierra el bloque abierto anteriormente.
221	}	Cierra el bloque abierto anteriormente.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	public int guardarGrupo(Integer id, int idPresentacion, String nombre, int minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	int maximo, boolean repetir, boolean visible, boolean personalizar) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	try (Connection c = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
226	if (id == null) return insertarId(c, "INSERT INTO grupo_presentacion(id_presentacion,nombre,minimo,maximo,permite_repetir,visible,permite_personalizar,estado) VALUES(?,?,?,?,?,?,?,TRUE)", idPresentacion,nombre,minimo,maximo,repetir,visible,personalizar);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
227	ejecutar(c, "UPDATE grupo_presentacion SET nombre=?,minimo=?,maximo=?,permite_repetir=?,visible=?,permite_personalizar=?,estado=TRUE WHERE id_grupo=?", nombre,minimo,maximo,repetir,visible,personalizar,id);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
228	return id;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
229	}	Cierra el bloque abierto anteriormente.
230	}	Cierra el bloque abierto anteriormente.
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	public int guardarOpcion(Integer id, int idGrupo, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
233	BigDecimal incremento, boolean predeterminada) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
234	try (Connection c = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
235	if (predeterminada) ejecutar(c, "UPDATE opcion_grupo SET predeterminada=FALSE WHERE id_grupo=?", idGrupo);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
236	if (id == null) return insertarId(c, "INSERT INTO opcion_grupo(id_grupo,nombre,incrementoPrecio,predeterminada,estado) VALUES(?,?,?,?,?,TRUE)", idGrupo,nombre,incremento,predeterminada);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
237	ejecutar(c, "UPDATE opcion_grupo SET nombre=?,incrementoPrecio=?,predeterminada=?,estado=TRUE WHERE id_opcion=?", nombre,incremento,predeterminada,id);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
238	return id;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
239	}	Cierra el bloque abierto anteriormente.
240	}	Cierra el bloque abierto anteriormente.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.
242	public void agregarComponente(int idOpcion, int idProducto, int cantidad) throws SQLException {	Declara el método agregarComponente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
243	try (Connection c = conectar()) { ejecutar(c, "INSERT INTO opcion_componente(id_opcion,id_producto,cantidad) VALUES(?,?,?)", idOpcion,idProducto,cantidad); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
244	}	Cierra el bloque abierto anteriormente.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	public void quitarComponente(int idOpcion, int idProducto) throws SQLException {	Declara el método quitarComponente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
247	try (Connection c = conectar()) { ejecutar(c,	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
248	"DELETE FROM opcion_componente WHERE id_opcion=? AND id_producto=?",	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

249	idOpcion,idProducto)); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
250	}	Cierra el bloque abierto anteriormente.
251		Línea en blanco: separa visualmente bloques para que el código sea más legible.
252	public void desactivar(String tabla, String campoId, int id) throws SQLException {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
253	if (!Set.of("presentacion_menu", "grupo_presentacion", "opcion_grupo").contains(tabla))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
254	throw new IllegalArgumentException("Tabla inválida.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
255	try (Connection c = conectar()) { ejecutar(c, "UPDATE " + tabla + " SET estado=FALSE WHERE " + campoId + "=?", id); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
256	}	Cierra el bloque abierto anteriormente.
257		Línea en blanco: separa visualmente bloques para que el código sea más legible.
258	public Map<Integer,String> listarProductos() throws SQLException {	Declara el método listarProductos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
259	return listarNombres("SELECT id_producto,nombre FROM producto WHERE estado=TRUE ORDER BY nombre");	Finaliza el método y devuelve el valor indicado a quien lo llamó.
260	}	Cierra el bloque abierto anteriormente.
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	public Map<Integer,String> listarIngredientes() throws SQLException {	Declara el método listarIngredientes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
263	return listarNombres("SELECT id_ingrediente,nombre FROM ingrediente WHERE estado=TRUE ORDER BY nombre");	Finaliza el método y devuelve el valor indicado a quien lo llamó.
264	}	Cierra el bloque abierto anteriormente.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	private Map<Integer,String> listarNombres(String sql) throws SQLException {	Declara el método listarNombres. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
267	Map<Integer,String> resultado = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
268	try (Connection c = conectar(); PreparedStatement s = c.prepareStatement(sql); ResultSet r = s.executeQu ery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
269	while (r.next()) resultado.put(r.getInt(1), r.getString(2));	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
270	}	Cierra el bloque abierto anteriormente.
271	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
272	}	Cierra el bloque abierto anteriormente.
273		Línea en blanco: separa visualmente bloques para que el código sea más legible.
274	public void guardarIngredienteProducto(Integer id, int idProducto, int idIngrediente,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	BigDecimal cantidadDefault, boolean quitar, boolean extra,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
276	BigDecimal cantidadExtra, BigDecimal precioExtra, int maxExtras) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
277	try (Connection c = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
278	if (id == null) ejecutar(c, "INSERT INTO producto_ingrediente(id_producto,id_ingredient,cantidad_de fault,permite_quitar,permite_extra,cantidad_extra,precio_extra,max_extras,estado) VALUES(?,?,?, ?, ?, ?, TRUE)", idProducto,idIngrediente,cantidadDefault,quitar,extra,cantidadExtra,precioExtra,maxExtras);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
279	else ejecutar(c, "UPDATE producto_ingrediente SET id_ingredient=?,cantidad_default=?,permite_quitar =?,permite_extra=?,cantidad_extra=?,precio_extra=?,max_extras=?,estado=TRUE WHERE id_producto_ingrediente=?", id Ingrediente,cantidadDefault,quitar,extra,cantidadExtra,precioExtra,maxExtras,id);	Ruta alternativa que se ejecuta cuando la condición anterior fue falsa.
280	}	Cierra el bloque abierto anteriormente.
281	}	Cierra el bloque abierto anteriormente.
282		Línea en blanco: separa visualmente bloques para que el código sea más legible.
283	public void desactivarIngredienteProducto(int id) throws SQLException {	Declara el método desactivarIngredienteProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
284	try (Connection c = conectar()) { ejecutar(c, "UPDATE producto_ingrediente SET estado=FALSE WHERE id_pro ducto_ingrediente=?", id); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
285	}	Cierra el bloque abierto anteriormente.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	private static int insertarId(Connection c, String sql, Object... valores) throws SQLException {	Declara el método insertarId. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

288	try (PreparedStatement s = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
289	asignar(s, valores); s.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
290	try (ResultSet r = s.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
291	if (!r.next()) throw new SQLException("No se obtuvo el identificador creado.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
292	return r.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
293	}	Cierra el bloque abierto anteriormente.
294	}	Cierra el bloque abierto anteriormente.
295	}	Cierra el bloque abierto anteriormente.
296		Línea en blanco: separa visualmente bloques para que el código sea más legible.
297	private static void ejecutar(Connection c, String sql, Object... valores) throws SQLException {	Declara el método ejecutar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
298	try (PreparedStatement s = c.prepareStatement(sql)) { asignar(s, valores); s.executeUpdate(); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
299	}	Cierra el bloque abierto anteriormente.
300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
301	private static void asignar(PreparedStatement s, Object... valores) throws SQLException {	Declara el método asignar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
302	for (int i=0; i<valores.length; i++) s.setObject(i+1, valores[i]);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
303	}	Cierra el bloque abierto anteriormente.
304	}	Cierra el bloque abierto anteriormente.

16.49 src/DAO/InventarioDAO.java

Consulta existencias combinadas de productos directos e ingredientes y registra entradas, salidas y ajustes.

Tipo: java | Líneas: 326 | Archivo 49 de 157

Clases/tipos detectados: InventarioDAO

Línea	Constructor o método detectado
56	private Connection conectar() throws SQLException {
64	public PaginalInventario listarPagina(String busqueda, String tipo,
107	public List<ArticuloInventario> listarParaExportar(String busqueda,
124	public List<MovimientoInventario> listarMovimientos(int limite)
168	public BigDecimal registrarMovimiento(ArticuloInventario articulo,
256	private void agregarFiltros(StringBuilder condiciones,
286	private void validarMovimiento(ArticuloInventario articulo,
309	private int colocarParametros(PreparedStatement ps, List<Object> parametros)
316	private ArticuloInventario mapearArticulo(ResultSet rs)

1	package DAO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.ArticuloInventario;	Importa Modelos.ArticuloInventario para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.MovimientoInventario;	Importa Modelos.MovimientoInventario para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.PaginaInventario;	Importa Modelos.PaginaInventario para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
9	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
10	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
11	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
12	import java.sql.Timestamp;	Importa java.sql.Timestamp para poder usar esa clase sin escribir su nombre completo.
13	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
14	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	/** Consultas y movimientos del inventario unificado. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	public class InventarioDAO {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	private static final BigDecimal MAX_STOCK = new BigDecimal("99999999.99");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
20	private static final String INVENTARIO_BASE = ""	Declara una constante o referencia que no puede reasignarse después de inicializarse.
21	SELECT tipo, id, nombre, categoria, unidad, stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	stock_minimo, activo, ultimo_movimiento	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
23	FROM (	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	SELECT 'INGREDIENTE' AS tipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

25	i.id_ingrediente AS id,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26	i.nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	'Ingredientes' AS categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	i.unidad_medida AS unidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	i.stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	i.stock_minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	i.estado AS activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	(SELECT MAX(mi.fecha_hora)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	FROM movimiento_inventario mi	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	WHERE mi.id_ingrediente = i.id_ingrediente)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
35	AS ultimo_movimiento	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	FROM ingrediente i	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	UNION ALL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	SELECT 'PRODUCTO' AS tipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	p.id_producto AS id,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	p.nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	c.nombre AS categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	'unidad' AS unidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	CAST(p.stock_actual AS DECIMAL(12,2)) AS stock_actual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	CAST(p.stock_minimo AS DECIMAL(12,2)) AS stock_minimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	p.estado AS activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	(SELECT MAX(mi.fecha_hora)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	FROM movimiento_inventario mi	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	WHERE mi.id_producto = p.id_producto)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
49	AS ultimo_movimiento	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	JOIN categoria c ON c.id_categoria = p.id_categoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
52	WHERE p.tipo_stock = 'DIRECTO'	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
53	) inventario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	private Connection conectar() throws SQLException {	Declara el método conectar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
57	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
59	throw new SQLException("No fue posible conectar con MySQL.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
60	}	Cierra el bloque abierto anteriormente.
61	return conexion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
62	}	Cierra el bloque abierto anteriormente.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	public PaginaInventario listarPagina(String busqueda, String tipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	String estado, int pagina, int porPagina) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	StringBuilder condiciones = new StringBuilder(" WHERE 1 = 1");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
67	List<Object> parametros = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
68	agregarFiltros(condiciones, parametros, busqueda, tipo, estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	String contar = "SELECT COUNT(*) AS total FROM ("	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
71	+ INVENTARIO_BASE + ") lista" + condiciones;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	String listar = INVENTARIO_BASE + condiciones + ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	ORDER BY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75	CASE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
76	WHEN activo = FALSE THEN 4	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
77	WHEN stock_actual <= 0 THEN 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	WHEN stock_minimo > 0 AND stock_actual <= stock_minimo THEN 2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	ELSE 3	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	END,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
81	nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	LIMIT ? OFFSET ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
86	int total;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
87	try (PreparedStatement ps = conexion.prepareStatement(contar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
88	colocarParametros(ps, parametros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
90	rs.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
91	total = rs.getInt("total");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
92	}	Cierra el bloque abierto anteriormente.
93	}	Cierra el bloque abierto anteriormente.
94	List<ArticuloInventario> articulos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
95	try (PreparedStatement ps = conexion.prepareStatement(listar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
96	int indice = colocarParametros(ps, parametros);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
97	ps.setInt(indice++, porPagina);	Configura una propiedad mediante setInt en el objeto indicado.
98	ps.setInt(indice, (pagina - 1) * porPagina);	Configura una propiedad mediante setInt en el objeto indicado.
99	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
100	while (rs.next()) articulos.add(mapearArticulo(rs));	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
101	}	Cierra el bloque abierto anteriormente.
102	}	Cierra el bloque abierto anteriormente.
103	return new PaginaInventario(articulos, total);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
104	}	Cierra el bloque abierto anteriormente.
105	}	Cierra el bloque abierto anteriormente.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	public List<ArticuloInventario> listarParaExportar(String busqueda,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
108	String tipo, String estado) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
109	StringBuilder condiciones = new StringBuilder(" WHERE 1 = 1");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
110	List<Object> parametros = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
111	agregarFiltros(condiciones, parametros, busqueda, tipo, estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	String sql = INVENTARIO_BASE + condiciones + " ORDER BY tipo, nombre";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

113	List<ArticuloInventario> articulos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
114	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
115	PreparedStatement ps = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
116	colocarParametros(ps, parametros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
118	while (rs.next()) articulos.add(mapearArticulo(rs));	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
119	}	Cierra el bloque abierto anteriormente.
120	}	Cierra el bloque abierto anteriormente.
121	return articulos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
122	}	Cierra el bloque abierto anteriormente.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	public List<MovimientoInventario> listarMovimientos(int limite)	Declara el método listarMovimientos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
125	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
127	SELECT mi.id_movimiento, mi.fecha_hora,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	CASE WHEN mi.id_ingrediente IS NOT NULL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	THEN 'Ingrediente' ELSE 'Producto directo' END tipo_articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	COALESCE(i.nombre, p.nombre, 'Artículo no disponible') articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	mi.tipo_movimiento, mi.cantidad, mi.motivo, mi.id_pedido	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	FROM movimiento_inventario mi	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	LEFT JOIN ingrediente i ON i.id_ingrediente = mi.id_ingrediente	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
134	LEFT JOIN producto p ON p.id_producto = mi.id_producto	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
135	WHERE mi.id_ingrediente IS NOT NULL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
136	OR (mi.id_producto IS NOT NULL AND p.tipo_stock = 'DIRECTO')	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
137	ORDER BY mi.fecha_hora DESC, mi.id_movimiento DESC	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
138	LIMIT ?	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	List<MovimientoInventario> movimientos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
141	try (Connection conexion = conectar();	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
142	PreparedStatement ps = conexion.prepareStatement(sql)) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
143	ps.setInt(1, Math.max(1, limite));	Configura una propiedad mediante setInt en el objeto indicado.
144	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
145	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
146	Timestamp fecha = rs.getTimestamp("fecha_hora");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
147	int pedido = rs.getInt("id_pedido");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
148	boolean pedidoNulo = rs.wasNull();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
149	movimientos.add(new MovimientoInventario(	Crea una nueva instancia de MovimientoInventario y la usa o guarda para continuar el proceso.
150	rs.getInt("id_movimiento"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
151	fecha == null ? null : fecha.toLocalDateTime(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
152	rs.getString("tipo_articulo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
153	rs.getString("articulo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
154	rs.getString("tipo_movimiento"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
155	rs.getBigDecimal("cantidad"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
156	rs.getString("motivo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.

157	pedidoNulo ? null : pedido));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158	}	Cierra el bloque abierto anteriormente.
159	}	Cierra el bloque abierto anteriormente.
160	}	Cierra el bloque abierto anteriormente.
161	return movimientos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
165	* Entrada y salida reciben cuánto se mueve. Ajuste recibe el stock final.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
166	* La actualización y el historial se guardan dentro de una transacción.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
167	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
168	public BigDecimal registrarMovimiento(ArticuloInventario articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
169	String tipoMovimiento, BigDecimal cantidad, String motivo)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
170	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	validarMovimiento(articulo, tipoMovimiento, cantidad, motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
172	boolean producto = articulo.esProducto();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
173	if (producto && cantidad.stripTrailingZeros().scale() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
174	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
175	"Los productos directos se manejan en unidades completas.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
176	}	Cierra el bloque abierto anteriormente.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	String tabla = producto ? "producto" : "ingrediente";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
179	String columnName = producto ? "id_producto" : "id_ingrediente";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
180	String seleccionar = "SELECT stock_actual, estado"	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
181	+ (producto ? ", tipo_stock" : "")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
182	+ " FROM " + tabla + " WHERE " + columnName + " = ? FOR UPDATE";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	try (Connection conexion = conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
185	conexion.setAutoCommit(false);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
186	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
187	BigDecimal anterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
188	try (PreparedStatement ps = conexion.prepareStatement(seleccionar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
189	ps.setInt(1, articulo.getId());	Configura una propiedad mediante setInt en el objeto indicado.
190	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
191	if (!rs.next()) throw new SQLException("No se encontró el artículo.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
192	if (!rs.getBoolean("estado")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
193	throw new SQLException("El artículo está inactivo.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
194	}	Cierra el bloque abierto anteriormente.
195	if (producto && !"DIRECTO".equals(rs.getString("tipo_stock"))) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
196	throw new SQLException("El producto ya no usa stock directo.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
197	}	Cierra el bloque abierto anteriormente.
198	anterior = rs.getBigDecimal("stock_actual");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
199	}	Cierra el bloque abierto anteriormente.
200	}	Cierra el bloque abierto anteriormente.

201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	BigDecimal nuevo = switch (tipoMovimiento) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
203	case "ENTRADA" -> anterior.add(cantidad);	Define una de las rutas posibles dentro del switch.
204	case "SALIDA" -> anterior.subtract(cantidad);	Define una de las rutas posibles dentro del switch.
205	default -> cantidad;	Define una de las rutas posibles dentro del switch.
206	};	Cierra el bloque abierto anteriormente y termina la declaración.
207	if (nuevo.signum() < 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
208	throw new SQLException("No hay existencias suficientes para la salida.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
209	}	Cierra el bloque abierto anteriormente.
210	if (nuevo.compareTo(MAX_STOCK) > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
211	throw new SQLException("El stock supera el máximo permitido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
212	}	Cierra el bloque abierto anteriormente.
213	if (producto && nuevo.stripTrailingZeros().scale() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
214	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
215	"El stock final del producto debe ser un número entero.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
216	}	Cierra el bloque abierto anteriormente.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	BigDecimal cantidadGuardada = "AJUSTE".equals(tipoMovimiento)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
219	? nuevo.subtract(anterior) : cantidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220	if (cantidadGuardada.signum() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
221	throw new SQLException("El ajuste no cambia el stock.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
222	}	Cierra el bloque abierto anteriormente.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	try (PreparedStatement ps = conexion.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
225	"UPDATE " + tabla + " SET stock_actual = ? WHERE "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
226	+ columnId + " = ?")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
227	ps.setBigDecimal(1, nuevo);	Configura una propiedad mediante setBigDecimal en el objeto indicado.
228	ps.setInt(2, articulo.getId());	Configura una propiedad mediante setInt en el objeto indicado.
229	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
230	}	Cierra el bloque abierto anteriormente.
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	String insertar = producto	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
233	? "INSERT INTO movimiento_inventario "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
234	+ "(id_producto, tipo_movimiento, cantidad, motivo) VALUES (?, ?, ?, ?)"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
235	: "INSERT INTO movimiento_inventario "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
236	+ "(id_ingredient, tipo_movimiento, cantidad, motivo) VALUES (?, ?, ?, ?)";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
237	try (PreparedStatement ps = conexion.prepareStatement(insertar)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
238	ps.setInt(1, articulo.getId());	Configura una propiedad mediante setInt en el objeto indicado.
239	ps.setString(2, tipoMovimiento);	Configura una propiedad mediante setString en el objeto indicado.
240	ps.setBigDecimal(3, cantidadGuardada);	Configura una propiedad mediante setBigDecimal en el objeto indicado.
241	ps.setString(4, motivo == null motivo.isBlank()	Configura una propiedad mediante setString en el objeto indicado.
242	? null : motivo.trim());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
243	ps.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
244	}	Cierra el bloque abierto anteriormente.

245	conexion.commit();	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
246	return nuevo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
247	} catch (SQLException RuntimeException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
248	conexion.rollback();	Revierte la transacción porque algo falló, evitando guardar un pedido o inventario incompleto.
249	throw ex;	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
250	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
251	conexion.setAutoCommit(true);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
252	}	Cierra el bloque abierto anteriormente.
253	}	Cierra el bloque abierto anteriormente.
254	}	Cierra el bloque abierto anteriormente.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	private void agregarFiltros(StringBuilder condiciones,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	List<Object> parametros, String busqueda, String tipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	String estado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	if (busqueda != null && !busqueda.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
260	condiciones.append(" AND (nombre LIKE ? OR categoria LIKE ? OR unidad LIKE ?)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
261	String texto = "%" + busqueda.trim() + "%";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
262	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
263	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
264	parametros.add(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
265	}	Cierra el bloque abierto anteriormente.
266	if (tipo != null && !tipo.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
267	condiciones.append(" AND tipo = ?");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
268	parametros.add(tipo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	}	Cierra el bloque abierto anteriormente.
270	if (estado != null && !estado.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
271	switch (estado) {	Selecciona una ruta entre varias según el valor evaluado.
272	case "DISPONIBLE" -> condiciones.append(	Define una de las rutas posibles dentro del switch.
273	" AND activo = TRUE AND stock_actual > 0"	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
274	+ " AND (stock_minimo = 0 OR stock_actual > stock_minimo)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
275	case "BAJO" -> condiciones.append(	Define una de las rutas posibles dentro del switch.
276	" AND activo = TRUE AND stock_actual > 0"	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
277	+ " AND stock_minimo > 0 AND stock_actual <= stock_minimo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
278	case "AGOTADO" -> condiciones.append(	Define una de las rutas posibles dentro del switch.
279	" AND activo = TRUE AND stock_actual <= 0");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
280	case "INACTIVO" -> condiciones.append(" AND activo = FALSE");	Define una de las rutas posibles dentro del switch.
281	default -> { }	Define una de las rutas posibles dentro del switch.
282	}	Cierra el bloque abierto anteriormente.
283	}	Cierra el bloque abierto anteriormente.
284	}	Cierra el bloque abierto anteriormente.
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	private void validarMovimiento(ArticuloInventario articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
287	String tipoMovimiento, BigDecimal cantidad, String motivo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
288	if (articulo == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

289	throw new IllegalArgumentException("Selecciona un artículo.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
290	}	Cierra el bloque abierto anteriormente.
291	if (!("ENTRADA".equals(tipoMovimiento)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
292	"SALIDA".equals(tipoMovimiento)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
293	"AJUSTE".equals(tipoMovimiento))) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
294	throw new IllegalArgumentException("Tipo de movimiento inválido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
295	}	Cierra el bloque abierto anteriormente.
296	boolean ajuste = "AJUSTE".equals(tipoMovimiento);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
297	if (cantidad == null cantidad.scale() > 2	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
298	cantidad.compareTo(MAX_STOCK) > 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
299	(ajuste ? cantidad.signum() < 0 : cantidad.signum() <= 0)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
300	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
301	"Cantidad inválida; usa hasta dos decimales.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	}	Cierra el bloque abierto anteriormente.
303	if (motivo != null && motivo.trim().length() > 150) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
304	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
305	"El motivo admite hasta 150 caracteres.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306	}	Cierra el bloque abierto anteriormente.
307	}	Cierra el bloque abierto anteriormente.
308		Línea en blanco: separa visualmente bloques para que el código sea más legible.
309	private int colocarParametros(PreparedStatement ps, List<Object> parametros)	Declara el método colocarParametros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
310	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	int indice = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
312	for (Object valor : parametros) ps.setObject(indice++, valor);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
313	return indice;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
314	}	Cierra el bloque abierto anteriormente.
315		Línea en blanco: separa visualmente bloques para que el código sea más legible.
316	private ArticuloInventario mapearArticulo(ResultSet rs)	Declara el método mapearArticulo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
317	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	Timestamp fecha = rs.getTimestamp("ultimo_movimiento");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
319	return new ArticuloInventario(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
320	rs.getString("tipo"), rs.getInt("id"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
321	rs.getString("nombre"), rs.getString("categoria"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
322	rs.getString("unidad"), rs.getBigDecimal("stock_actual"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
323	rs.getBigDecimal("stock_minimo"), rs.getBoolean("activo"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
324	fecha == null ? null : fecha.toLocalDateTime());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
325	}	Cierra el bloque abierto anteriormente.
326	}	Cierra el bloque abierto anteriormente.

16.50 src/DAO/PagoDAO.java

Caso de uso transaccional de cobro. Revalida reglas, registra el pedido completo, descuenta inventario y genera el comprobante.

Tipo: java | Líneas: 324 | Archivo 50 de 157

Clases/tipos detectados: PagoDAO, Conexiones, Consumo, ReglaGrupo, ReglaIngrediente

Línea	Constructor o método detectado
16	public PagoDAO() { this(Conexion::conectar); }
17	public PagoDAO(Conexiones conexiones) { this.conexiones = conexiones; }
19	public String cobrar(SolicitudPago pago) throws SQLException {
76	private void validarLinea(Connection c, LineaPedido linea, Consumo consumo)
137	private void validarProductosOpcion(Connection c, LineaPedido linea,
177	private void validarReceta(Connection c, ProductoPedido producto,
226	private void bloquearInventario(Connection c, Consumo consumo) throws SQLException {
249	private void guardarLinea(Connection c, int idPedido, LineaPedido linea) throws SQLException {
265	private void descontarInventario(Connection c, int idPedido, Consumo consumo) throws SQLException {
276	private static int insertarId(Connection c, String sql, Object... args) throws SQLException {
291	private record ReglaGrupo(int minimo,int maximo,boolean repetir) {}
292	private record ReglaIngrediente(int idIngrediente,String nombre,
296	private static void ejecutar(Connection c, String sql, Object... args) throws SQLException {
303	private static String comprobante(int id, SolicitudPago p) {

1	package DAO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.*;	Importa Modelos.* para poder usar esa clase sin escribir su nombre completo.
5	import Utilidades.HorarioMenu;	Importa Utilidades.HorarioMenu para poder usar esa clase sin escribir su nombre completo.
6	import java.sql.*;	Importa java.sql.* para poder usar esa clase sin escribir su nombre completo.
7	import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa fecha y hora juntas.
8	import java.time.LocalTime;	Importa java.time.LocalTime: representa una hora sin fecha.
9	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
10	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	/** Cada cobro confirma pedido, detalle e inventario en una sola transacción. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	public class PagoDAO {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14	@FunctionalInterface public interface Conexiones { Connection abrir() throws SQLException; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
15	private final Conexiones conexiones;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
16	public PagoDAO() { this(Conexion::conectar); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	public PagoDAO(Conexiones conexiones) { this.conexiones = conexiones; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	public String cobrar(SolicitudPago pago) throws SQLException {	Declara el método cobrar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
20	try (Connection c = conexiones.abrir()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
21	if (c == null) throw new SQLException("No se pudo conectar con MySQL. Reintente cuando esté disponib le.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
22	c.setAutoCommit(false);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
23	boolean confirmando = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
25	// La clave se conserva al reintentar. El INSERT bloquea los intentos simultáneos.	Comentario para humanos; Java no ejecuta este texto. La clave se conserva al reintentar. El INSERT bloquea los intentos simultáneos.
26	ejecutar(c, "INSERT INTO pago_operacion(clave,solicitud,referencia) VALUES(?,?,?) "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	+ "ON DUPLICATE KEY UPDATE clave=clave", pago.clave(), pago.toString(), pago.referencia());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
28	try (PreparedStatement s = c.prepareStatement("SELECT solicitud,comprobante FROM pago_operacion WHERE clave=? FOR UPDATE")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
29	s.setString(1, pago.clave());	Configura una propiedad mediante setString en el objeto indicado.
30	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
31	r.next();	Avanza el ResultSet a la siguiente fila; devuelve false cuando ya no quedan resultados.
32	if (!pago.toString().equals(r.getString(1))) throw new IllegalArgumentException("La oper ación pertenece a otro pedido.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
33	String previo = r.getString(2);	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
34	if (previo != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
35	c.rollback();	Revierte la transacción porque algo falló, evitando guardar un pedido o inventario incompleto.
36	// Los comprobantes antiguos conservan sus valores; se actualiza solo el símbolo al mostrarlos.	Comentario para humanos; Java no ejecuta este texto. Los comprobantes antiguos conservan sus valores; se actualiza solo el símbolo al mostrarlos.
37	return previo.replace(" \$", " Q")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
38	.replace("\nTotal: \$", "\nTotal: Q")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	.replace("\nRecibido: \$", "\nRecibido: Q")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	.replace("\nCambio: \$", "\nCambio: Q");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	}	Cierra el bloque abierto anteriormente.
42	}	Cierra el bloque abierto anteriormente.
43	}	Cierra el bloque abierto anteriormente.
44	try (PreparedStatement s = c.prepareStatement("SELECT estado,rol FROM usuario WHERE id_usuario=? FOR UPDATE")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
45	s.setInt(1, pago.usuario());	Configura una propiedad mediante setInt en el objeto indicado.
46	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
47	if (!r.next() !r.getBoolean(1) !("CAJERO".equals(r.getString(2)) "ADMINISTRADOR ".equals(r.getString(2))))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
48	throw new IllegalArgumentException("La sesión no está autorizada para cobrar.");	Interrompe el flujo lanzando un error con una explicación para quien llamó al método.
49	}	Cierra el bloque abierto anteriormente.
50	}	Cierra el bloque abierto anteriormente.
51	Consumo consumo = new Consumo();	Crea una nueva instancia de Consumo y la usa o guarda para continuar el proceso.
52	for (LineaPedido linea : pago.lineas()) validarLinea(c, linea, consumo);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
53	bloquearInventario(c, consumo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	int id;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55	try (PreparedStatement s = c.prepareStatement("INSERT INTO pedido(numero_orden,id_usuario,tipo_s ervicio,estado,metodo_pago,monto_recibido,total) VALUES(0,?,?, 'PAGADO',?,?,?)", Statement.RETURN_GENERATED_KEYS)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

56	s.setInt(1, pago.usuario()); s.setString(2, pago.servicio()); s.setString(3, pago.metodo());	Configura una propiedad mediante setInt en el objeto indicado.
57	s.setBigDecimal(4, pago.recibido()); s.setBigDecimal(5, pago.total()); s.executeUpdate();	Configura una propiedad mediante setBigDecimal en el objeto indicado.
58	try (ResultSet r = s.getGeneratedKeys()) { if (!r.next()) throw new SQLException("No se obtuvo el número de pedido."); id = r.getInt(1); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
59	}	Cierra el bloque abierto anteriormente.
60	ejecutar(c, "UPDATE pedido SET numero_orden=? WHERE id_pedido=?", id, id);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
61	for (LineaPedido linea : pago.lineas()) guardarLinea(c, id, linea);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
62	descontarInventario(c, id, consumo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	String ticket = comprobante(id, pago);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
64	ejecutar(c, "UPDATE pago_operacion SET id_pedido=?,comprobante=? WHERE clave=?", id, ticket, pago.clave());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
65	confirmando = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
66	c.commit();	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
67	return ticket;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
68	} catch (SQLException RuntimeException error) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
69	try { c.rollback(); } catch (SQLException rollback) { error.addSuppressed(rollback); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
70	if (confirmando) throw new SQLException("No se pudo verificar la confirmación. Reintente este mismo cobro; no vuelva a cobrar en la terminal.", error);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
71	throw error;	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
72	}	Cierra el bloque abierto anteriormente.
73	}	Cierra el bloque abierto anteriormente.
74	}	Cierra el bloque abierto anteriormente.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	private void validarLinea(Connection c, LineaPedido linea, Consumo consumo)	Declara el método validarLinea. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
77	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	BigDecimal precioCatalogo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	try (PreparedStatement s = c.prepareStatement("SELECT pm.id_producto_principal,pm.nombre,pm.precio,pm.estado AS presentacion_activa,p.nombre,p.estado AS productoactivo,p.disponibilidad_menu "	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
80	+ "FROM presentacion_menu pm JOIN producto p ON p.id_producto=pm.id_producto_principal "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
81	+ "WHERE pm.id_presentacion=? FOR UPDATE")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
82	s.setInt(1, linea.idPresentacion());	Configura una propiedad mediante setInt en el objeto indicado.
83	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
84	if (!r.next() r.getInt("id_producto_principal") != linea.idProducto())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
85	!r.getBoolean("presentacion_activa") !r.getBoolean("productoactivo")	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
86	!HorarioMenu.estaDisponible(r.getString("disponibilidad_menu"), LocalTime.now()))	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
87	throw new IllegalArgumentException(linea.nombre() + ": la presentación ya no está disponible.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
88	precioCatalogo = r.getBigDecimal("precio");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
89	if (precioCatalogo.compareTo(linea.precioBase()) != 0)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
90	throw new IllegalArgumentException(linea.nombre() + ": cambió el precio. Configúralo nuevamente.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
91	}	Cierra el bloque abierto anteriormente.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	Map<Integer, ReglaGrupo> reglas = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
95	try (PreparedStatement s = c.prepareStatement("SELECT id_grupo,minimo,maximo,permite_repetir FROM grupo_presentacion WHERE id_presentacion=? AND estado=TRUE")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

96	s.setInt(1, linea.idPresentacion());	Configura una propiedad mediante setInt en el objeto indicado.
97	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
98	while (r.next()) reglas.put(r.getInt(1), new ReglaGrupo(r.getInt(2),r.getInt(3),r.getBoolean(4))	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
99	};	
100	}	Cierra el bloque abierto anteriormente.
101	Map<Integer,Integer> cantidadesGrupo = new HashMap<>();	Cierra el bloque abierto anteriormente.
102	Map<Integer,Set<Integer>> opcionesGrupo = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
103	for (OpcionPedido opcion : linea.opciones()) {	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
104	ReglaGrupo regla = reglas.get(opcion.idGrupo());	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
105	if (regla == null) throw new IllegalArgumentException("La configuración de " + linea.nombre() + " ca	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
106	mbió.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
107	BigDecimal incremento;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
108	try (PreparedStatement s = c.prepareStatement("SELECT incremento_precio,estado FROM opcion_grupo WHE	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
109	RE id_opcion=? AND id_grupo=?")) {	
110	s.setInt(1, opcion.idOpcion()); s.setInt(2, opcion.idGrupo());	Configura una propiedad mediante setInt en el objeto indicado.
111	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
112	if (!r.next() !r.getBoolean("estado"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
113	throw new IllegalArgumentException(opcion.opcion() + ": ya no está disponible.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
114	incremento = r.getBigDecimal("incremento_precio");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
115	}	Cierra el bloque abierto anteriormente.
116	}	Cierra el bloque abierto anteriormente.
117	if (incremento.compareTo(opcion.precioExtra()) != 0)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
118	throw new IllegalArgumentException(opcion.opcion() + ": cambió el precio.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
119	precioCatalogo = precioCatalogo.add(incremento);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
120	cantidadesGrupo.merge(opcion.idGrupo(), 1, Integer::sum);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	opcionesGrupo.computeIfAbsent(opcion.idGrupo(), k -> new HashSet<>()).add(opcion.idOpcion());	Crea una nueva instancia de HashSet y la usa o guarda para continuar el proceso.
122	validarProductosOpcion(c, linea, opcion, consumo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	for (ProductoPedido producto : opcion.productos())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
124	for (ModificacionPedido cambio : producto.modificaciones())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
125	precioCatalogo = precioCatalogo.add(cambio.precioExtra());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
126	}	Cierra el bloque abierto anteriormente.
127	for (Map.Entry<Integer,ReglaGrupo> entrada : reglas.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
128	int cantidad = cantidadesGrupo.getOrDefault(entrada.getKey(), 0);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
129	ReglaGrupo regla = entrada.getValue();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
130	if (cantidad < regla.minimo cantidad > regla.maximo)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
131	throw new IllegalArgumentException(linea.nombre() + ": faltan elecciones obligatorias.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
132	if (!regla.repetir && opcionesGrupo.getOrDefault(entrada.getKey(), Set.of()).size() != cantidad)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
133	throw new IllegalArgumentException(linea.nombre() + ": una elección no permite repetirse.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
134	}	Cierra el bloque abierto anteriormente.
135	if (precioCatalogo.setScale(2).compareTo(linea.precio()) != 0)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
136	throw new IllegalArgumentException(linea.nombre() + ": el total cambió. Configúralo nuevamente.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
137	}	Cierra el bloque abierto anteriormente.
		Línea en blanco: separa visualmente bloques para que el código sea más legible.
	private void validarProductosOpcion(Connection c, LineaPedido linea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

138	OpcionPedido opcion, Consumo consumo) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	Map<Integer,Integer> esperados = new TreeMap<>();	Crea una nueva instancia de TreeMap y la usa o guarda para continuar el proceso.
140	try (PreparedStatement s = c.prepareStatement("SELECT id_producto,SUM(cantidad) cantidad FROM opcion_componente WHERE id_opcion=? GROUP BY id_producto")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
141	s.setInt(1, opcion.idOpcion());	Configura una propiedad mediante setInt en el objeto indicado.
142	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
143	while (r.next()) esperados.put(r.getInt(1), r.getInt(2));	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
144	}	Cierra el bloque abierto anteriormente.
145	}	Cierra el bloque abierto anteriormente.
146	Map<Integer,Integer> recibidos = new TreeMap<>();	Crea una nueva instancia de TreeMap y la usa o guarda para continuar el proceso.
147	for (ProductoPedido producto : opcion.productos())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
148	recibidos.merge(producto.idProducto(), producto.cantidad(), Integer::sum);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	if (!esperados.equals(recibidos))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
150	throw new IllegalArgumentException(opcion.opcion() + ": sus productos internos cambiaron.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	for (ProductoPedido producto : opcion.productos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
153	String tipoStock;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
154	String nombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
155	try (PreparedStatement s = c.prepareStatement("SELECT nombre,tipo_stock,estado,disponibilidad_menu FROM producto WHERE id_producto=?")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
156	s.setInt(1, producto.idProducto());	Configura una propiedad mediante setInt en el objeto indicado.
157	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
158	if (!r.next() !r.getBoolean("estado")	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
159	!HorarioMenu.estaDisponible(r.getString("disponibilidad_menu"), LocalDateTime.now()))	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
160	throw new IllegalArgumentException(producto.nombre() + ": ya no está disponible.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
161	nombre = r.getString("nombre"); tipoStock = r.getString("tipo_stock");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
162	}	Cierra el bloque abierto anteriormente.
163	}	Cierra el bloque abierto anteriormente.
164	BigDecimal multiplicador = BigDecimal.valueOf((long) linea.cantidad() * producto.cantidad());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
165	if ("DIRECTO".equals(tipoStock)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
166	consumo.productos.merge(producto.idProducto(), multiplicador, BigDecimal::add);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	if (!producto.modificaciones().isEmpty())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
168	throw new IllegalArgumentException(nombre + " no admite cambios por ingredientes.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
169	} else if ("RECETA".equals(tipoStock)) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
170	validarReceta(c, producto, multiplicador, consumo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
171	} else if (!producto.modificaciones().isEmpty()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
172	throw new IllegalArgumentException(nombre + " no admite cambios por ingredientes.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
173	}	Cierra el bloque abierto anteriormente.
174	}	Cierra el bloque abierto anteriormente.
175	}	Cierra el bloque abierto anteriormente.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	private void validarReceta(Connection c, ProductoPedido producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	BigDecimal multiplicador, Consumo consumo) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	Map<Integer, ReglaIngrediente> receta = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.

180	try (PreparedStatement s = c.prepareStatement("SELECT pi.id_producto_ingredient,pi.id_ingredient,i.nombre,pi.cantidad_default,pi.permite_quitar,pi.permite_extra,pi.cantidad_extra,pi.precio_extra,pi.max_extras "	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
181	+ "FROM producto_ingredient pi JOIN ingrediente i ON i.id_ingredient=pi.id_ingredient "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
182	+ "WHERE pi.id_producto=? AND pi.estado=TRUE AND i.estado=TRUE")) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
183	s.setInt(1, producto.idProducto());	Configura una propiedad mediante setInt en el objeto indicado.
184	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
185	while (r.next()) receta.put(r.getInt("id_producto_ingredient"), new ReglaIngrediente(	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
186	r.getInt("id_ingredient"),r.getString("nombre"),r.getBigDecimal("cantidad_default"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
187	r.getBoolean("permite_quitar"),r.getBoolean("permite_extra"),r.getBigDecimal("cantidad_e	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
188	xtra"),	
189	r.getBigDecimal("precio_extra"),r.getInt("max_extras")));	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
190	}	Cierra el bloque abierto anteriormente.
191	Set<Integer> quitados = new HashSet<>();	Cierra el bloque abierto anteriormente.
192	Set<Integer> conExtra = new HashSet<>();	Crea una nueva instancia de HashSet y la usa o guarda para continuar el proceso.
193	Set<String> cambiosUnicos = new HashSet<>();	Crea una nueva instancia de HashSet y la usa o guarda para continuar el proceso.
194	for (ModificacionPedido cambio : producto.modificaciones()) {	Crea una nueva instancia de HashSet y la usa o guarda para continuar el proceso.
195	ReglaIngrediente regla = receta.get(cambio.idProductoIngrediente());	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
196	if (regla == null regla.idIngrediente != cambio.idIngrediente()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	!cambiosUnicos.add(cambio.tipo() + ":" + cambio.idProductoIngrediente()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
198	throw new IllegalArgumentException(producto.nombre() + ": una modificación ya no es válida.");	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
199	if ("SIN".equals(cambio.tipo())) {	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
200	if (conExtra.contains(cambio.idProductoIngrediente()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
201	throw new IllegalArgumentException("No puedes quitar y agregar extra del mismo ingrediente."	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
202	);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
203	if (!regla.quitar cambio.cantidad().compareTo(regla.cantidadDefault) != 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
204	cambio.precioExtra().signum() != 0)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	throw new IllegalArgumentException("Ya no se permite quitar " + regla.nombre + ".");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
206	quitados.add(cambio.idProductoIngrediente());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
207	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
208	if (quitados.contains(cambio.idProductoIngrediente()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
209	throw new IllegalArgumentException("No puedes quitar y agregar extra del mismo ingrediente."	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
210	);	
211	BigDecimal cantidad = regla.cantidadExtra.multiply(BigDecimal.valueOf(cambio.veces()));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
212	BigDecimal precio = regla.precioExtra.multiply(BigDecimal.valueOf(cambio.veces()));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
213	if (!regla.extra cambio.veces() > regla.maxExtras cambio.veces() < 1	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
214	cambio.cantidad().compareTo(cantidad) != 0 cambio.precioExtra().compareTo(precio)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	!= 0)	
216	throw new IllegalArgumentException("Cambió la regla del extra " + regla.nombre + ".");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
217	consumo.ingredientes.merge(regla.idIngrediente,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	cambio.cantidad().multiply(multiplicador), BigDecimal::add);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219	conExtra.add(cambio.idProductoIngrediente());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220	}	Cierra el bloque abierto anteriormente.
	}	Cierra el bloque abierto anteriormente.
	for (Map.Entry<Integer,ReglaIngrediente> entrada : receta.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
	if (!quitados.contains(entrada.getKey())) consumo.ingredientes.merge(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

221	entrada.getValue().idIngrediente,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222	entrada.getValue().cantidadDefault.multiply(multiplicador), BigDecimal::add);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223	}	Cierra el bloque abierto anteriormente.
224	}	Cierra el bloque abierto anteriormente.
225		Línea en blanco: separa visualmente bloques para que el código sea más legible.
226	private void bloquearInventario(Connection c, Consumo consumo) throws SQLException {	Declara el método bloquearInventario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
227	for (Map.Entry<Integer,BigDecimal> entrada : consumo.productos.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
228	try (PreparedStatement s = c.prepareStatement("SELECT nombre,stock_actual,estado FROM producto WHERE id_producto=? FOR UPDATE")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
229	s.setInt(1, entrada.getKey());	Configura una propiedad mediante setInt en el objeto indicado.
230	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
231	if (!r.next() !r.getBoolean("estado"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
232	BigDecimal.valueOf(r.getInt("stock_actual")).compareTo(entrada.getValue()) < 0)	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
233	throw new IllegalArgumentException("No hay existencias suficientes de un producto del pedido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
234	}	Cierra el bloque abierto anteriormente.
235	}	Cierra el bloque abierto anteriormente.
236	}	Cierra el bloque abierto anteriormente.
237	for (Map.Entry<Integer,BigDecimal> entrada : consumo.ingredientes.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
238	try (PreparedStatement s = c.prepareStatement("SELECT nombre,stock_actual,estado FROM ingrediente WHERE id_ingredient=? FOR UPDATE")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
239	s.setInt(1, entrada.getKey());	Configura una propiedad mediante setInt en el objeto indicado.
240	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
241	if (!r.next() !r.getBoolean("estado"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
242	r.getBigDecimal("stock_actual").compareTo(entrada.getValue()) < 0)	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
243	throw new IllegalArgumentException("No hay ingredientes suficientes para preparar el pedido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
244	}	Cierra el bloque abierto anteriormente.
245	}	Cierra el bloque abierto anteriormente.
246	}	Cierra el bloque abierto anteriormente.
247	}	Cierra el bloque abierto anteriormente.
248		Línea en blanco: separa visualmente bloques para que el código sea más legible.
249	private void guardarLinea(Connection c, int idPedido, LineaPedido linea) throws SQLException {	Declara el método guardarLinea. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
250	int idDetalle = insertarId(c, "INSERT INTO pedido_detalle(id_pedido,id_presentacion,nombre,presentacion,cantidad,precio_unitario,subtotal) VALUES(?,?,?,?,,?,?)",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
251	idPedido, linea.idPresentacion(), linea.nombre(), linea.presentacion(), linea.cantidad(), linea.precio(), linea.subtotal());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
252	for (OpcionPedido opcion : linea.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
253	int idPedidoOpcion = insertarId(c, "INSERT INTO pedido_opcion(id_detalle,id_grupo,id_opcion,numero,grupo,opcion,precio_extra) VALUES(?,?,?,?,,?,?)",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
254	idDetalle,opcion.idGrupo(),opcion.idOpcion(),opcion.numero(),opcion.grupo(),opcion.opcion(),opcion.precioExtra());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	for (ProductoPedido producto : opcion.productos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
256	int idPedidoProducto = insertarId(c, "INSERT INTO pedido_producto(id_pedido_opcion,id_producto,nombre,cantidad) VALUES(?,?,?,?)",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
257	idPedidoOpcion,producto.idProducto(),producto.nombre(),producto.cantidad());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
258	for (ModificacionPedido cambio : producto.modificaciones()) ejecutar(c,	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.

259	"INSERT INTO pedido_modificacion(id_pedido_producto,id_producto_ingredient, tipo,ingrediente,cantidad,precio_extra) VALUES(?,?,?,?,,?)",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	idPedidoProducto,cambio.idProductoIngrediente(),cambio.tipo(),cambio.ingrediente(),cambio.cantidad(),cambio.precioExtra());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
261	}	Cierra el bloque abierto anteriormente.
262	}	Cierra el bloque abierto anteriormente.
263	}	Cierra el bloque abierto anteriormente.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	private void descontarInventario(Connection c, int idPedido, Consumo consumo) throws SQLException {	Declara el método descontarInventario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
266	for (Map.Entry<Integer,BigDecimal> entrada : consumo.productos.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
267	ejecutar(c,"UPDATE producto SET stock_actual=stock_actual-? WHERE id_producto=?",entrada.getValue().intValueExact(),entrada.getKey());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
268	ejecutar(c,"INSERT INTO movimiento_inventario(id_producto,id_pedido,tipo_movimiento,cantidad,motivo) VALUES(?,?, 'SALIDA',?, 'Venta en caja')",entrada.getKey(),idPedido,entrada.getValue());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	}	Cierra el bloque abierto anteriormente.
270	for (Map.Entry<Integer,BigDecimal> entrada : consumo.ingredientes.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
271	ejecutar(c,"UPDATE ingrediente SET stock_actual=stock_actual-? WHERE id_ingrediente=?",entrada.getValue(),entrada.getKey());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
272	ejecutar(c,"INSERT INTO movimiento_inventario(id_ingrediente,id_pedido,tipo_movimiento,cantidad,motivo) VALUES(?,?, 'SALIDA',?, 'Preparación de pedido')",entrada.getKey(),idPedido,entrada.getValue());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
273	}	Cierra el bloque abierto anteriormente.
274	}	Cierra el bloque abierto anteriormente.
275		Línea en blanco: separa visualmente bloques para que el código sea más legible.
276	private static int insertarId(Connection c, String sql, Object... args) throws SQLException {	Declara el método insertarId. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
277	try (PreparedStatement s = c.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
278	for (int i=0;i<args.length;i++) s.setObject(i+1,args[i]);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
279	s.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
280	try (ResultSet r=s.getGeneratedKeys()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
281	if (!r.next()) throw new SQLException("No se obtuvo el registro creado.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
282	return r.getInt(1);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
283	}	Cierra el bloque abierto anteriormente.
284	}	Cierra el bloque abierto anteriormente.
285	}	Cierra el bloque abierto anteriormente.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	private static final class Consumo {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
288	final Map<Integer,BigDecimal> productos = new TreeMap<>();	Crea una nueva instancia de TreeMap y la usa o guarda para continuar el proceso.
289	final Map<Integer,BigDecimal> ingredientes = new TreeMap<>();	Crea una nueva instancia de TreeMap y la usa o guarda para continuar el proceso.
290	}	Cierra el bloque abierto anteriormente.
291	private record ReglaGrupo(int minimo,int maximo,boolean repetir) {}	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
292	private record ReglaIngrediente(int idIngrediente,String nombre,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
293	BigDecimal cantidadDefault,boolean quitar,boolean extra,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
294	BigDecimal cantidadExtra,BigDecimal precioExtra,int maxExtras) {}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
295		Línea en blanco: separa visualmente bloques para que el código sea más legible.
296	private static void ejecutar(Connection c, String sql, Object... args) throws SQLException {	Declara el método ejecutar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
297	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
298	for (int i = 0; i < args.length; i++) s.setObject(i + 1, args[i]);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.

299	s.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
300	}	Cierra el bloque abierto anteriormente.
301	}	Cierra el bloque abierto anteriormente.
302		Línea en blanco: separa visualmente bloques para que el código sea más legible.
303	private static String comprobante(int id, SolicitudPago p) {	Declara el método comprobante. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
304	StringBuilder b = new StringBuilder("WALDONALD'S\nComprobante de venta\nPedido #" + id + "\n" + LocalDateTime.now().withNano(0)	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
305	+ "\nCajero: " + p.usuario() + "\n" + (p.servicio().equals("COMER_AQUI") ? "Comer aquí" : "Para llevar") + "\n\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306	for (LineaPedido l : p.lineas()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
307	b.append(l.cantidad()).append(" x ").append(l.nombre()).append(" ")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
308	.append(l.presentacion()).append(" Q").append(l.subtotal().toString()).append('\n');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	for (OpcionPedido o : l.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
310	if (!o.grupo().startsWith("__")) b.append(" - ").append(o.grupo()).append(": ").append(o.opcion()).append('\n');	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
311	for (ProductoPedido pr : o.productos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
312	b.append(" ").append(pr.nombre()).append('\n');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313	for (ModificacionPedido m : pr.modificaciones())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
314	b.append(" ").append(m.tipo().equals("SIN") ? "SIN " : "EXTRA ")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	.append(m.ingrediente()).append(m.veces()>1 ? " x"+m.veces() : "").append('\n');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
316	}	Cierra el bloque abierto anteriormente.
317	}	Cierra el bloque abierto anteriormente.
318	}	Cierra el bloque abierto anteriormente.
319	b.append("\nTotal: Q").append(p.total().toString()).append("\nMétodo: ").append(p.metodo())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
320	.append("\nRecibido: Q").append(p.recibido().toString()).append("\nCambio: Q").append(p.cam bio().toString());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321	if (!p.referencia().isBlank()) b.append("\nReferencia: ").append(p.referencia());	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
322	return b.append("\n\nGracias por su compra.").toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
323	}	Cierra el bloque abierto anteriormente.
324	}	Cierra el bloque abierto anteriormente.

16.51 src/DAO/ProductoDAO.java

Consulta los productos visibles para el cajero por categoría, subcategoría y horario.

Tipo: java | Líneas: 381 | Archivo 51 de 157

Clases/tipos detectados: ProductoDAO

Línea	Constructor o método detectado
20	public List<Productos> obtenerPorCategoria(int idCategoria) {
30	public List<Productos> obtenerPorCategoria(int idCategoria, LocalTime hora) {
110	public Integer obtenerIdCategoriaPorNombre(String nombreCategoria) {
141	public List<String> obtenerSubcategorias(int idCategoria) {
173	public List<Productos> obtenerPorCategoriaYSubcategoria(
225	private List<Productos> filtrarPorTexto(
241	private String sqlSubcategorias(Connection conexion) throws SQLException {
278	private String sqlProductosSubcategoria(Connection conexion) throws SQLException {
327	private boolean tieneRelacionSubcategoria(Connection conexion) throws SQLException {
334	private boolean tablaExiste(Connection conexion, String tabla) throws SQLException {
346	private boolean columnaExiste(Connection conexion, String tabla, String columna) throws SQLException {
358	private String normalizar(String texto) {
369	private boolean coincideNombreCategoria(String nombreBase, String nombreBuscado) {
376	private String quitarPlural(String texto) {

1	package DAO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Productos;	Importa Modelos.Productos para poder usar esa clase sin escribir su nombre completo.
5		Línea en blanco: separa visualmente bloques para que el código sea más legible.
6	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
7	import java.sql.DatabaseMetaData;	Importa java.sql.DatabaseMetaData para poder usar esa clase sin escribir su nombre completo.
8	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
9	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.
10	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	import java.time.LocalTime;	Importa java.time.LocalTime: representa una hora sin fecha.
13	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
14	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	import Utilidades.HorarioMenu;	Importa Utilidades.HorarioMenu para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.

18	public class ProductoDAO {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	public List<Productos> obtenerPorCategoria(int idCategoria) {	Declara el método obtenerPorCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
21	return obtenerPorCategoria(idCategoria, LocalTime.now());	Finaliza el método y devuelve el valor indicado a quien lo llamó.
22	}	Cierra el bloque abierto anteriormente.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	* Busca todos los productos activos de la categoría y marca cuáles están	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
26	* disponibles para la hora indicada. Los productos de todo el día siempre	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
27	* aparecen como disponibles.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
28	* La hora explícita permite comprobar el filtro sin insertar productos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
29	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
30	public List<Productos> obtenerPorCategoria(int idCategoria, LocalTime hora) {	Declara el método obtenerPorCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	List<Productos> productos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
35	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
36	return productos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
37	}	Cierra el bloque abierto anteriormente.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	String sql;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
40	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
41	sql = tablaExiste(conexion, "presentacion_menu") ? ""	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
42	SELECT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	id_producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	id_categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	COALESCE((SELECT pm.precio FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
49	ORDER BY pm.predeterminada DESC,pm.id_presentacion LIMIT 1), precio_base) AS preci	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	o_base,	
51	imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	WHERE p.id_categoria = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
55	AND p.estado = TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
56	AND EXISTS (SELECT 1 FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58	"" : ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	SELECT id_producto, id_categoria, nombre, descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
	precio_base, imagen, disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

60	FROM producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	WHERE id_categoria = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
62	AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
63	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
65	System.out.println("Error verificando presentaciones: " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	return productos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
67	}	Cierra el bloque abierto anteriormente.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69	try (	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
70	conexion; PreparedStatement ps = conexion.prepareStatement(sql) {	Prepara una sentencia SQL parametrizada para enviarla a MySQL de forma segura.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	ps.setInt(1, idCategoria);	Configura una propiedad mediante setInt en el objeto indicado.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	Productos producto = new Productos(	Crea una nueva instancia de Productos y la usa o guarda para continuar el proceso.
79	rs.getInt("id_producto"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
80	rs.getInt("id_categoria"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
81	rs.getString("nombre"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
82	rs.getString("descripcion"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
83	rs.getDouble("precio_base"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	rs.getString("imagen"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
85	HorarioMenu.estaDisponible(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	rs.getString("disponibilidad_menu"), hora)	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
87	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	productos.add(producto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	}	Cierra el bloque abierto anteriormente.
91	}	Cierra el bloque abierto anteriormente.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	System.out.println(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	"Error cargando productos: "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	+ e.getMessage()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
98	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	e.printStackTrace();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	}	Cierra el bloque abierto anteriormente.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	return productos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

104	}	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
107	* Busca el id real de una categoría por su nombre. Así los botones del	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
108	* cajero no dependen de ids fijos de la base de datos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
109	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
110	public Integer obtenerIdCategoriaPorNombre(String nombreCategoria) {	Declara el método obtenerIdCategoriaPorNombre. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
111	String sql = "SELECT id_categoria, nombre FROM categoria WHERE estado = TRUE";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	try (Connection conexion = Conexion.conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
114	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
115	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
116	}	Cierra el bloque abierto anteriormente.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	try (PreparedStatement ps = conexion.prepareStatement(sql);	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
119	ResultSet rs = ps.executeQuery()) {	Ejecuta una consulta SELECT y recibe sus filas en un ResultSet.
120	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
121	if (coincideNombreCategoria(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
122	rs.getString("nombre"), nombreCategoria)) {	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
123	return rs.getInt("id_categoria");	Finaliza el método y devuelve el valor indicado a quien lo llamó.
124	}	Cierra el bloque abierto anteriormente.
125	}	Cierra el bloque abierto anteriormente.
126	}	Cierra el bloque abierto anteriormente.
127	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
128	System.out.println("Error buscando categoría: " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129	}	Cierra el bloque abierto anteriormente.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
132	}	Cierra el bloque abierto anteriormente.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
135	* Obtiene las subcategorías almacenadas en la base de datos. Se admiten	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
136	* los dos esquemas habituales del proyecto: producto.subcategoria o la	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
137	* relación producto.id_subcategoria -> subcategoria.nombre. Si no existe	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
138	* una relación de subcategorías, devuelve una lista vacía y la UI muestra	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
139	* solo "Todos", sin inventar datos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
140	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
141	public List<String> obtenerSubcategorias(int idCategoria) {	Declara el método obtenerSubcategorias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
142	List<String> subcategorias = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
144	try (Connection conexion = Conexion.conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
145	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
146	return subcategorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

147	}	Cierra el bloque abierto anteriormente.
148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
149	String sql = sqlSubcategorias(conexion);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
150	if (sql == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
151	return subcategorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
152	}	Cierra el bloque abierto anteriormente.
153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154	try (PreparedStatement ps = conexion.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
155	ps.setInt(1, idCategoria);	Configura una propiedad mediante setInt en el objeto indicado.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
158	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
159	String nombre = rs.getString("subcategoria");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
160	if (nombre != null && !nombre.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
161	subcategorias.add(nombre.trim());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	}	Cierra el bloque abierto anteriormente.
163	}	Cierra el bloque abierto anteriormente.
164	}	Cierra el bloque abierto anteriormente.
165	}	Cierra el bloque abierto anteriormente.
166	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
167	System.out.println("Error cargando subcategorias: " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	}	Cierra el bloque abierto anteriormente.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	return subcategorias;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
171	}	Cierra el bloque abierto anteriormente.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	public List<Productos> obtenerPorCategoriaYSubcategoria(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
174	int idCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	String nombreSubcategoria) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	if (nombreSubcategoria == null nombreSubcategoria.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
178	return obtenerPorCategoria(idCategoria);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
179	}	Cierra el bloque abierto anteriormente.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	List<Productos> productos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
182	try (Connection conexion = Conexion.conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
183	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
184	return productos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
185	}	Cierra el bloque abierto anteriormente.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	String sql = sqlProductosSubcategoria(conexion);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
188	if (sql == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
189	return filtrarPorTexto(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
190	obtenerPorCategoria(idCategoria), nombreSubcategoria);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

191	}	Cierra el bloque abierto anteriormente.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	try (PreparedStatement ps = conexion.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
194	ps.setInt(1, idCategoria);	Configura una propiedad mediante setInt en el objeto indicado.
195	ps.setString(2, nombreSubcategoria);	Configura una propiedad mediante setString en el objeto indicado.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
198	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
199	productos.add(new Productos(	Crea una nueva instancia de Productos y la usa o guarda para continuar el proceso.
200	rs.getInt("id_producto"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
201	rs.getInt("id_categoria"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
202	rs.getString("nombre"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
203	rs.getString("descripcion"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
204	rs.getDouble("precio_base"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	rs.getString("imagen"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
206	HorarioMenu.estaDisponible(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
207	rs.getString("disponibilidad_menu"),	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
208	LocalTime.now())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
210	}	Cierra el bloque abierto anteriormente.
211	}	Cierra el bloque abierto anteriormente.
212	}	Cierra el bloque abierto anteriormente.
213	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
214	System.out.println("Error filtrando subcategoria: " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215	}	Cierra el bloque abierto anteriormente.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	return productos;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
218	}	Cierra el bloque abierto anteriormente.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
221	* Compatibilidad para el esquema actual, que no tiene una columna de	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
222	* subcategoria. Se filtra únicamente dentro de la categoría ya consultada	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
223	* usando nombre y descripción del producto, sin crear ni alterar datos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
224	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
225	private List<Productos> filtrarPorTexto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	List<Productos> productos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	String nombreSubcategoria) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	List<Productos> filtrados = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
229	String subcategoria = normalizar(nombreSubcategoria);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	for (Productos producto : productos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
232	String texto = normalizar(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
233	producto.getNombre() + " " + producto.getDescripcion());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
234	if (texto.contains(subcategoria)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

235	filtrados.add(producto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	}	Cierra el bloque abierto anteriormente.
237	}	Cierra el bloque abierto anteriormente.
238	return filtrados;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
239	}	Cierra el bloque abierto anteriormente.
240		Línea en blanco: separa visualmente bloques para que el código sea más legible.
241	private String sqlSubcategorias(Connection conexion) throws SQLException {	Declara el método sqlSubcategorias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
242	if (columnaExiste(conexion, "producto", "subcategoria")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
243	if (!tablaExiste(conexion, "presentacion_menu")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
244	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
245	SELECT DISTINCT subcategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246	FROM producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	WHERE id_categoria = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
248	AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
249	AND subcategoria IS NOT NULL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
250	ORDER BY subcategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
251	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
252	}	Cierra el bloque abierto anteriormente.
253	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
254	SELECT DISTINCT subcategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
255	FROM producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
256	WHERE id_categoria = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
257	AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
258	AND EXISTS (SELECT 1 FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	WHERE pm.id_producto_principal=producto.id_producto AND pm.estado=TRUE)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
260	AND subcategoria IS NOT NULL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
261	ORDER BY subcategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
262	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
263	}	Cierra el bloque abierto anteriormente.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	if (tieneRelacionSubcategoria(conexion)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
266	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
267	SELECT DISTINCT s.nombre AS subcategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
268	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
269	INNER JOIN subcategoria s ON s.id_subcategoria = p.id_subcategoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
270	WHERE p.id_categoria = ?	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
271	AND p.estado = TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
272	ORDER BY s.nombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
274	}	Cierra el bloque abierto anteriormente.
275	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
276	}	Cierra el bloque abierto anteriormente.
277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
278	private String sqlProductosSubcategoria(Connection conexion) throws SQLException {	Declara el método sqlProductosSubcategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

279	if (columnaExiste(conexion, "producto", "subcategoria")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
280	if (!tablaExiste(conexion, "presentacion_menu")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
281	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
282	SELECT id_producto, id_categoria, nombre, descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	precio_base, imagen, disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
284	FROM producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
285	WHERE id_categoria = ? AND subcategoria = ? AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
286	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
287	}	Cierra el bloque abierto anteriormente.
288	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
289	SELECT p.id_producto, p.id_categoria, p.nombre, p.descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290	COALESCE((SELECT pm.precio FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
291	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
292	ORDER BY pm.predeterminada DESC,pm.id_presentacion LIMIT 1),p.precio_base)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
293	AS precio_base,	
294	p.imagen, p.disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
295	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
296	WHERE p.id_categoria = ? AND p.subcategoria = ? AND p.estado = TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
297	AND EXISTS (SELECT 1 FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
298	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
299	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
300	}	Cierra el bloque abierto anteriormente.
301	if (tieneRelacionSubcategoria(conexion)) {	Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	if (!tablaExiste(conexion, "presentacion_menu")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
303	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
304	SELECT p.id_producto, p.id_categoria, p.nombre, p.descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
305	p.precio_base, p.imagen, p.disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
307	INNER JOIN subcategoria s ON s.id_subcategoria = p.id_subcategoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
308	WHERE p.id_categoria = ? AND s.nombre = ? AND p.estado = TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
309	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
310	}	Cierra el bloque abierto anteriormente.
311	return ""	Finaliza el método y devuelve el valor indicado a quien lo llamó.
312	SELECT p.id_producto, p.id_categoria, p.nombre, p.descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
313	COALESCE((SELECT pm.precio FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
314	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
315	ORDER BY pm.predeterminada DESC,pm.id_presentacion LIMIT 1),p.precio_base)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	AS precio_base,	
317	p.imagen, p.disponibilidad_menu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	FROM producto p	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
319	INNER JOIN subcategoria s ON s.id_subcategoria = p.id_subcategoria	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
320	WHERE p.id_categoria = ? AND s.nombre = ? AND p.estado = TRUE	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
321	AND EXISTS (SELECT 1 FROM presentacion_menu pm	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
322	WHERE pm.id_producto_principal=p.id_producto AND pm.estado=TRUE)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

322	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
323	}	Cierra el bloque abierto anteriormente.
324	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
325	}	Cierra el bloque abierto anteriormente.
326		Línea en blanco: separa visualmente bloques para que el código sea más legible.
327	private boolean tieneRelacionSubcategoria(Connection conexion) throws SQLException {	Declara el método tieneRelacionSubcategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
328	return tablaExiste(conexion, "subcategoria")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
329	&& columnaExiste(conexion, "producto", "id_subcategoria")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	&& columnaExiste(conexion, "subcategoria", "id_subcategoria")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
331	&& columnaExiste(conexion, "subcategoria", "nombre");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332	}	Cierra el bloque abierto anteriormente.
333		Línea en blanco: separa visualmente bloques para que el código sea más legible.
334	private boolean tablaExiste(Connection conexion, String tabla) throws SQLException {	Declara el método tablaExiste. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
335	DatabaseMetaData metadata = conexion.getMetaData();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
336	try (ResultSet rs = metadata.getTables(conexion.getCatalog(), null, "%", new String[]{"TABLE"})) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
337	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
338	if (tabla.equalsIgnoreCase(rs.getString("TABLE_NAME"))) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
339	return true;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
340	}	Cierra el bloque abierto anteriormente.
341	}	Cierra el bloque abierto anteriormente.
342	}	Cierra el bloque abierto anteriormente.
343	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
344	}	Cierra el bloque abierto anteriormente.
345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
346	private boolean columnaExiste(Connection conexion, String tabla, String columna) throws SQLException {	Declara el método columnaExiste. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
347	DatabaseMetaData metadata = conexion.getMetaData();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
348	try (ResultSet rs = metadata.getColumns(conexion.getCatalog(), null, tabla, "%")) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
349	while (rs.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
350	if (columna.equalsIgnoreCase(rs.getString("COLUMN_NAME"))) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
351	return true;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
352	}	Cierra el bloque abierto anteriormente.
353	}	Cierra el bloque abierto anteriormente.
354	}	Cierra el bloque abierto anteriormente.
355	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
356	}	Cierra el bloque abierto anteriormente.
357		Línea en blanco: separa visualmente bloques para que el código sea más legible.
358	private String normalizar(String texto) {	Declara el método normalizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
359	if (texto == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
360	return "";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
361	}	Cierra el bloque abierto anteriormente.
362		Línea en blanco: separa visualmente bloques para que el código sea más legible.
363	return java.text.Normalizer.normalize(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
364	texto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	java.text.Normalizer.Form.NFD	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

366	).replaceAll("\\p{M}", "").toLowerCase(java.util.Locale.ROOT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
367	}	Cierra el bloque abierto anteriormente.
368		Línea en blanco: separa visualmente bloques para que el código sea más legible.
369	private boolean coincideNombreCategoria(String nombreBase, String nombreBuscado) {	Declara el método coincideNombreCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
370	String base = normalizar(nombreBase);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
371	String buscado = normalizar(nombreBuscado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
372	return base.equals(buscado)	Finaliza el método y devuelve el valor indicado a quien lo llamó.
373	quitarPlural(base).equals(quitarPlural(buscado));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
374	}	Cierra el bloque abierto anteriormente.
375		Línea en blanco: separa visualmente bloques para que el código sea más legible.
376	private String quitarPlural(String texto) {	Declara el método quitarPlural. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
377	return texto.endsWith("s")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
378	? texto.substring(0, texto.length() - 1)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
379	: texto;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
380	}	Cierra el bloque abierto anteriormente.
381	}	Cierra el bloque abierto anteriormente.

16.52 src/DAO/ReporteDAO.java

Obtiene indicadores y ventas del día para el panel de reportes.

Tipo: java | Líneas: 136 | Archivo 52 de 157

Clases/tipos detectados: ReporteDAO, Resumen

Línea	Constructor o método detectado
12	public record Resumen(BigDecimal ventas, long pedidos, BigDecimal ticket,
17	public Resumen(BigDecimal ventas, long pedidos, BigDecimal ticket,
26	public Resumen cargar(LocalDate fecha) throws SQLException {
105	private BigDecimal totalPorMetodo(
116	private List<Object[]> consultar(Connection c, String sql, LocalDate fecha) throws SQLException {

ReporteDAO.java	
1 package DAO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
4 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
5 import java.sql.*;	Importa java.sql.* para poder usar esa clase sin escribir su nombre completo.
6 import java.time.LocalDate;	Importa java.time.LocalDate: representa una fecha sin hora.
7 import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
8 import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
9	Línea en blanco: separa visualmente bloques para que el código sea más legible.
10 /** Consultas de solo lectura; las ventas se reconocen al pagar el pedido. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11 public final class ReporteDAO {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12 public record Resumen(BigDecimal ventas, long pedidos, BigDecimal ticket,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
13 long cancelados, long stockBajo, List<Object[]> recientes, List<Object[]> productos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14 List<Object[]> alertas, List<Object[]> ventasDetalle,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15 BigDecimal efectivo, BigDecimal tarjeta, BigDecimal otros) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	Línea en blanco: separa visualmente bloques para que el código sea más legible.
17 public Resumen(BigDecimal ventas, long pedidos, BigDecimal ticket,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18 long cancelados, long stockBajo, List<Object[]> recientes,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19 List<Object[]> productos, List<Object[]> alertas) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20 this(ventas, pedidos, ticket, cancelados, stockBajo, recientes,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21 productos, alertas, List.of(), BigDecimal.ZERO,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22 BigDecimal.ZERO, BigDecimal.ZERO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
23 }	Cierra el bloque abierto anteriormente.
24 }	Cierra el bloque abierto anteriormente.
25	Línea en blanco: separa visualmente bloques para que el código sea más legible.
26 public Resumen cargar(LocalDate fecha) throws SQLException {	Declara el método cargar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
27 try (Connection c = Conexion.conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
28 if (c == null) throw new SQLException("No se pudo conectar con la base de datos.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
29 return cargar(c, fecha);	Finaliza el método y devuelve el valor indicado a quien lo llamó.

30	}	Cierra el bloque abierto anteriormente.
31	}	Cierra el bloque abierto anteriormente.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	Resumen cargar(Connection c, LocalDate fecha) throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	c.setReadOnly(true);	Configura una propiedad mediante setReadOnly en el objeto indicado.
35	c.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);	Configura una propiedad mediante setTransactionIsolation en el objeto indicado.
36	c.setAutoCommit(false);	Configura una propiedad mediante setAutoCommit en el objeto indicado.
37	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
38	List<Object[]> indicadores = consultar(c,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
39	"SELECT COALESCE(SUM(CASE WHEN estado='PAGADO' THEN total ELSE 0 END),0), "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
40	+ "COALESCE(SUM(estado='PAGADO'),0), COALESCE(AVG(CASE WHEN estado='PAGADO' THEN total END),	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
41	0), "	
42	+ "COALESCE(SUM(estado='CANCELADO'),0) "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
43	+ "FROM pedido WHERE fecha_hora>=? AND fecha_hora<?", fecha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	List<Object[]> stock = consultar(c,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	"SELECT COUNT(*) FROM producto WHERE estado=1 AND tipo_stock='DIRECTO' AND stock_actual<=sto	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	ck_minimo", null);	
47	List<Object[]> pedidos = consultar(c,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
48	"SELECT p.numero_orden, CONCAT(u.nombre, ' ',u.apellido), DATE_FORMAT(p.fecha_hora, '%H:%i'),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	ar' ELSE 'A domicilio' END, "	
50	+ "CASE p.tipo_servicio WHEN 'COMER_AQUI' THEN 'En salón' WHEN 'PARA_LLEVAR' THEN 'Para llev	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	ar' ELSE 'A domicilio' END, "	
52	+ "p.estado, p.total FROM pedido p JOIN usuario u ON u.id_usuario=p.id_usuario "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
53	+ "WHERE p.fecha_hora>=? AND p.fecha_hora<? ORDER BY p.fecha_hora DESC,p.id_pedido DESC", fe	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	cha);	
55	List<Object[]> ventasDetalle;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
56	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
57	ventasDetalle = consultar(c,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58	"SELECT p.numero_orden, DATE_FORMAT(p.fecha_hora, '%d/%m/%Y %H:%i'), "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	+ "CONCAT(u.nombre, ' ',u.apellido), "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	+ "CASE p.tipo_servicio WHEN 'COMER_AQUI' THEN 'En el local' "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	+ "WHEN 'PARA_LLEVAR' THEN 'Para llevar' ELSE 'A domicilio' END, "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	+ "p.estado, COALESCE(p.metodo_pago, '-'), "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	+ "COALESCE(p.monto_recibido,0), p.total, "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	+ "COALESCE(SUM(d.cantidad),0) "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	+ "FROM pedido p JOIN usuario u ON u.id_usuario=p.id_usuario "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
66	+ "LEFT JOIN pedido_detalle d ON d.id_pedido=p.id_pedido "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
67	+ "WHERE p.fecha_hora>=? AND p.fecha_hora<? "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	+ "GROUP BY p.id_pedido,p.numero_orden,p.fecha_hora,u.nombre,u.apellido,"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	+ "p.tipo_servicio,p.estado,p.metodo_pago,p.monto_recibido,p.total "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	+ "ORDER BY p.fecha_hora DESC,p.id_pedido DESC", fecha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	} catch (SQLException esquemaAnterior) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
72	// Mantiene compatibilidad con bases antiguas sin datos de pago.	Comentario para humanos; Java no ejecuta este texto. Mantiene compatibilidad con bases antiguas sin datos de pago.
73	ventasDetalle = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
74	}	Cierra el bloque abierto anteriormente.

70	// La presentación es la línea vendida; sus productos internos no duplican la venta.	Comentario para humanos; Java no ejecuta este texto. La presentación es la línea vendida; sus productos internos no duplican la venta.
71	List<Object[]> productos = consultar(c,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
72	"SELECT pr.nombre, ca.nombre, SUM(d.cantidad), "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	+ "CASE WHEN pr.estado=0 THEN 'Inactivo' WHEN pr.stock_actual<=0 THEN 'Agotado' ELSE 'Dispon	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
74	ible' END "	
74	+ "FROM pedido_detalle d JOIN pedido p ON p.id_pedido=d.id_pedido "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
75	+ "JOIN presentacion_menu pm ON pm.id_presentacion=d.id_presentacion "	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
76	+ "JOIN producto pr ON pr.id_producto=pm.id_producto_principal JOIN categoria ca ON ca.id_ca	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
76	tegoria=pr.id_categoria "	
77	+ "WHERE p.estado='PAGADO' AND p.fecha_hora>=? AND p.fecha_hora<? "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	+ "GROUP BY pr.id_producto,pr.nombre,ca.nombre,pr.estado,pr.stock_actual "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	+ "ORDER BY SUM(d.cantidad) DESC,pr.nombre LIMIT 10", fecha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	List<Object[]> alertas = consultar(c,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	"SELECT 'Producto',nombre,stock_actual,stock_minimo,'unidades', "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	+ "CASE WHEN stock_actual<=0 THEN 'Agotado' ELSE 'Stock bajo' END "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	+ "FROM producto WHERE estado=1 AND tipo_stock='DIRECTO' AND stock_actual<=stock_minimo "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	+ "UNION ALL SELECT 'Ingrediente',nombre,stock_actual,stock_minimo,unidad_medida, "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
85	+ "CASE WHEN stock_actual<=0 THEN 'Agotado' ELSE 'Stock bajo' END "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	+ "FROM ingrediente WHERE estado=1 AND stock_actual<=stock_minimo ORDER BY 6,2", null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	Object[] k = indicadores.get(0);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
88	BigDecimal efectivo = totalPorMetodo(ventasDetalle, "EFECTIVO");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
89	BigDecimal tarjeta = totalPorMetodo(ventasDetalle, "TARJETA");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
90	BigDecimal ventasPagadas = new BigDecimal(k[0].toString());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
91	BigDecimal otros = ventasPagadas.subtract(efectivo).subtract(tarjeta);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	if (otros.signum() < 0) otros = BigDecimal.ZERO;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
93	Resumen resultado = new Resumen(new BigDecimal(k[0].toString()), ((Number) k[1]).longValue(),	Crea una nueva instancia de Resumen y la usa o guarda para continuar el proceso.
94	new BigDecimal(k[2].toString()), ((Number) k[3]).longValue(),	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
95	((Number) stock.get(0)[0]).longValue(), pedidos, productos, alertas,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	ventasDetalle, efectivo, tarjeta, otros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	c.commit();	Confirma la transacción: todos los cambios pendientes quedan guardados juntos.
98	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
99	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
100	c.rollback();	Revierte la transacción porque algo falló, evitando guardar un pedido o inventario incompleto.
101	throw ex;	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
102	}	Cierra el bloque abierto anteriormente.
103	}	Cierra el bloque abierto anteriormente.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	private BigDecimal totalPorMetodo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
106	List<Object[]> ventas, String metodo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	BigDecimal total = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
108	for (Object[] fila : ventas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
109	if ("PAGADO".equals(fila[4]) && metodo.equals(fila[5])) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
110	total = total.add(new BigDecimal(fila[7].toString()));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
111	}	Cierra el bloque abierto anteriormente.

112	}	Cierra el bloque abierto anteriormente.
113	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
114	}	Cierra el bloque abierto anteriormente.
115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
116	private List<Object[]> consultar(Connection c, String sql, LocalDate fecha) throws SQLException {	Declara el método consultar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
117	try (PreparedStatement s = c.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
118	s.setQueryTimeout(20);	Configura una propiedad mediante setQueryTimeout en el objeto indicado.
119	if (fecha != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
120	// Límites inclusivo/exclusivo: incluye todo el día y permite usar índices.	Comentario para humanos; Java no ejecuta este texto. Límites inclusivo/exclusivo: incluye todo el día y permite usar índices.
121	s.setObject(1, fecha.atStartOfDay());	Configura una propiedad mediante setObject en el objeto indicado.
122	s.setObject(2, fecha.plusDays(1).atStartOfDay());	Configura una propiedad mediante setObject en el objeto indicado.
123	}	Cierra el bloque abierto anteriormente.
124	try (ResultSet r = s.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
125	List<Object[]> filas = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
126	int columnas = r.getMetaData().getColumnCount();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
127	while (r.next()) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
128	Object[] fila = new Object[columnas];	Crea una nueva instancia de Object y la usa o guarda para continuar el proceso.
129	for (int i = 0; i < columnas; i++) fila[i] = r.getObject(i + 1);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
130	filas.add(fila);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	}	Cierra el bloque abierto anteriormente.
132	return filas;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
133	}	Cierra el bloque abierto anteriormente.
134	}	Cierra el bloque abierto anteriormente.
135	}	Cierra el bloque abierto anteriormente.
136	}	Cierra el bloque abierto anteriormente.

16.53 src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.form

Metadatos visuales de NetBeans para diseñar CatalogoIngredientesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 248 | Archivo 53 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<Dimension value="[1530, 932]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</Property>	Cierra la sección XML abierta anteriormente.
11	</Properties>	Cierra la sección XML abierta anteriormente.
12	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,115,0,0,6,-1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	</AuxValues>	Cierra la sección XML abierta anteriormente.
24		Línea en blanco para organizar el XML del formulario.
25	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
26	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
27	</Layout>	Cierra la sección XML abierta anteriormente.
28	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<Component class="javax.swing.JLabel" name="labelTitulo">	Declara un componente o contenedor visual y su clase Java.
30	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
31	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="text" type="java.lang.String" value="Catálogo de Ingredientes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	</Properties>	Cierra la sección XML abierta anteriormente.
36	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

37	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
38	<AbsoluteConstraints x="60" y="16" width="650" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
39	</Constraint>	Cierra la sección XML abierta anteriormente.
40	</Constraints>	Cierra la sección XML abierta anteriormente.
41	</Component>	Cierra la sección XML abierta anteriormente.
42	<Component class="javax.swing.JLabel" name="labelSubtitulo">	Declara un componente o contenedor visual y su clase Java.
43	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
44	<Property name="text" type="java.lang.String" value="Define los insumos, su unidad y el nivel mínimo de reposición"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
45	</Properties>	Cierra la sección XML abierta anteriormente.
46	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
47	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
48	<AbsoluteConstraints x="60" y="76" width="690" height="36"/>	Define la posición y el tamaño del componente dentro del diseño.
49	</Constraint>	Cierra la sección XML abierta anteriormente.
50	</Constraints>	Cierra la sección XML abierta anteriormente.
51	</Component>	Cierra la sección XML abierta anteriormente.
52	<Component class="Componentes.BotonDerretido" name="botonAgregar">	Declara un componente o contenedor visual y su clase Java.
53	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
54	<Property name="text" type="java.lang.String" value="Agregar ingrediente"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
55	</Properties>	Cierra la sección XML abierta anteriormente.
56	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
57	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
58	<AbsoluteConstraints x="1290" y="42" width="220" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
59	</Constraint>	Cierra la sección XML abierta anteriormente.
60	</Constraints>	Cierra la sección XML abierta anteriormente.
61	</Component>	Cierra la sección XML abierta anteriormente.
62	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
63	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
65	<Color blue="ed" green="e6" red="e1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
66	</Property>	Cierra la sección XML abierta anteriormente.
67	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
69	</Properties>	Cierra la sección XML abierta anteriormente.
70	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
71	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
72	<AbsoluteConstraints x="60" y="175" width="1470" height="95"/>	Define la posición y el tamaño del componente dentro del diseño.
73	</Constraint>	Cierra la sección XML abierta anteriormente.
74	</Constraints>	Cierra la sección XML abierta anteriormente.
75		Línea en blanco para organizar el XML del formulario.
76	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
77	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

78	</Layout>	Cierra la sección XML abierta anteriormente.
79	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
80	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.
81	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
82	<Property name="placeholder" type="java.lang.String" value="Buscar ingrediente o unidad..."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
83	</Properties>	Cierra la sección XML abierta anteriormente.
84	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
85	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
86	<AbsoluteConstraints x="20" y="20" width="440" height="55"/>	Define la posición y el tamaño del componente dentro del diseño.
87	</Constraint>	Cierra la sección XML abierta anteriormente.
88	</Constraints>	Cierra la sección XML abierta anteriormente.
89	</Component>	Cierra la sección XML abierta anteriormente.
90	<Component class="Componentes.BotonDesplegable" name="filtroStock">	Declara un componente o contenedor visual y su clase Java.
91	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
92	<Property name="text" type="java.lang.String" value="Todos los niveles"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
93	<Property name="textoDesplegable" type="java.lang.String" value="Todos los niveles;Stock bajo;Stock normal"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
94	</Properties>	Cierra la sección XML abierta anteriormente.
95	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
96	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
97	<AbsoluteConstraints x="480" y="22" width="280" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
98	</Constraint>	Cierra la sección XML abierta anteriormente.
99	</Constraints>	Cierra la sección XML abierta anteriormente.
100	</Component>	Cierra la sección XML abierta anteriormente.
101	<Component class="Componentes.BotonDesplegable" name="filtroEstado">	Declara un componente o contenedor visual y su clase Java.
102	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	<Property name="text" type="java.lang.String" value="Todos los estados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
104	<Property name="textoDesplegable" type="java.lang.String" value="Todos los estados;Activos;Inactivos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
105	</Properties>	Cierra la sección XML abierta anteriormente.
106	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
107	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
108	<AbsoluteConstraints x="780" y="22" width="280" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
109	</Constraint>	Cierra la sección XML abierta anteriormente.
110	</Constraints>	Cierra la sección XML abierta anteriormente.
111	</Component>	Cierra la sección XML abierta anteriormente.
112	<Component class="Componentes.BotonRedondeado" name="botonLimpiar">	Declara un componente o contenedor visual y su clase Java.
113	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
114	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
115	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
116	</Property>	Cierra la sección XML abierta anteriormente.
117	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	

118	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
119	</Property>	Cierra la sección XML abierta anteriormente.
120	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
121	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
122	</Property>	Cierra la sección XML abierta anteriormente.
123	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
124	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
125	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
126	</Property>	Cierra la sección XML abierta anteriormente.
127	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
128	</Properties>	Cierra la sección XML abierta anteriormente.
129	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
130	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
131	<AbsoluteConstraints x="1190" y="22" width="250" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
132	</Constraint>	Cierra la sección XML abierta anteriormente.
133	</Constraints>	Cierra la sección XML abierta anteriormente.
134	</Component>	Cierra la sección XML abierta anteriormente.
135	</SubComponents>	Cierra la sección XML abierta anteriormente.
136	</Container>	Cierra la sección XML abierta anteriormente.
137	<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
138	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
139	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
140	<Color blue="ed" green="e6" red="e1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
141	</Property>	Cierra la sección XML abierta anteriormente.
142	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
143	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
144	</Properties>	Cierra la sección XML abierta anteriormente.
145	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
146	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
147	<AbsoluteConstraints x="60" y="285" width="1470" height="640"/>	Define la posición y el tamaño del componente dentro del diseño.
148	</Constraint>	Cierra la sección XML abierta anteriormente.
149	</Constraints>	Cierra la sección XML abierta anteriormente.
150		Línea en blanco para organizar el XML del formulario.
151	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
152	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
153	</Layout>	Cierra la sección XML abierta anteriormente.
154	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
155	<Container class="javax.swing.JScrollPane" name="scrollTabla">	Declara un componente o contenedor visual y su clase Java.
156	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
157	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
158	<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

159	</Property>	Cierra la sección XML abierta anteriormente.
160	<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
161	<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
162	</Properties>	Cierra la sección XML abierta anteriormente.
163	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
164	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
165	<AbsoluteConstraints x="14" y="14" width="1442" height="555"/>	Define la posición y el tamaño del componente dentro del diseño.
166	</Constraint>	Cierra la sección XML abierta anteriormente.
167	</Constraints>	Cierra la sección XML abierta anteriormente.
168		Línea en blanco para organizar el XML del formulario.
169	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
170	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
171	<Component class="Componentes.TablaAdministrativa" name="tablaIngredientes">	Declara un componente o contenedor visual y su clase Java.
172	</Component>	Cierra la sección XML abierta anteriormente.
173	</SubComponents>	Cierra la sección XML abierta anteriormente.
174	</Container>	Cierra la sección XML abierta anteriormente.
175	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
176	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
177	<Property name="text" type="java.lang.String" value="Mostrando ingredientes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
178	</Properties>	Cierra la sección XML abierta anteriormente.
179	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
180	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
181	<AbsoluteConstraints x="28" y="581" width="430" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
182	</Constraint>	Cierra la sección XML abierta anteriormente.
183	</Constraints>	Cierra la sección XML abierta anteriormente.
184	</Component>	Cierra la sección XML abierta anteriormente.
185	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
186	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
187	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
188	</Properties>	Cierra la sección XML abierta anteriormente.
189	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
190	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
191	<AbsoluteConstraints x="1165" y="581" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
192	</Constraint>	Cierra la sección XML abierta anteriormente.
193	</Constraints>	Cierra la sección XML abierta anteriormente.
194		Línea en blanco para organizar el XML del formulario.
195	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
196	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
197	</Layout>	Cierra la sección XML abierta anteriormente.
198	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
199	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
200	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

201	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
202	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
203	</Constraint>	Cierra la sección XML abierta anteriormente.
204	</Constraints>	Cierra la sección XML abierta anteriormente.
205	</Component>	Cierra la sección XML abierta anteriormente.
206	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
207	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
208	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
209	</Properties>	Cierra la sección XML abierta anteriormente.
210	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
211	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
212	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
213	</Constraint>	Cierra la sección XML abierta anteriormente.
214	</Constraints>	Cierra la sección XML abierta anteriormente.
215	</Component>	Cierra la sección XML abierta anteriormente.
216	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
217	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
218	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
219	</Properties>	Cierra la sección XML abierta anteriormente.
220	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
221	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
222	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
223	</Constraint>	Cierra la sección XML abierta anteriormente.
224	</Constraints>	Cierra la sección XML abierta anteriormente.
225	</Component>	Cierra la sección XML abierta anteriormente.
226	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
227	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
228	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
229	</Properties>	Cierra la sección XML abierta anteriormente.
230	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
231	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
232	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
233	</Constraint>	Cierra la sección XML abierta anteriormente.
234	</Constraints>	Cierra la sección XML abierta anteriormente.
235	</Component>	Cierra la sección XML abierta anteriormente.
236	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
237	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
238	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
239	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
240	</Constraint>	Cierra la sección XML abierta anteriormente.
241	</Constraints>	Cierra la sección XML abierta anteriormente.

242	</Component>	Cierra la sección XML abierta anteriormente.
243	</SubComponents>	Cierra la sección XML abierta anteriormente.
244	</Container>	Cierra la sección XML abierta anteriormente.
245	</SubComponents>	Cierra la sección XML abierta anteriormente.
246	</Container>	Cierra la sección XML abierta anteriormente.
247	</SubComponents>	Cierra la sección XML abierta anteriormente.
248	</Form>	Cierra la sección XML abierta anteriormente.

16.54 src/GUI_ADMINISTRADOR/CatalogoIngredientesPanel.java

Interfaz del módulo administrador: CatalogoIngredientesPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 568 | Archivo 54 de 157

Clases/tipos detectados: CatalogoIngredientesPanel, ModeloTabla, RenderNivel, RenderEstado, RenderAcciones, EditorAcciones

Línea	Constructor o método detectado
59	public CatalogoIngredientesPanel() {
75	public void recargarDatos() {
79	private void prepararVista() {
95	private void configurarTabla() {
115	private void configurarFiltros() {
156	private void prepararSelector(BotonDesplegable selector) {
168	private void configurarPaginacion() {
182	private void prepararBotonPagina(JButton boton, IconosUsuarios.Tipo icono) {
199	private void cargarPagina() {
238	private void actualizarPaginacion(int total) {
266	private void cambiarPagina(int pagina) {
272	private void abrirFormulario(Ingrediente ingrediente) {
279	private void editar(Ingrediente ingrediente) {
293	private void desactivar(Ingrediente ingrediente) {
307	private void mostrarError(String accion, Throwable error) {
345	private JPanel etiquetaEstado(String texto, Color color, Color fondo,
381	private JPanel panelAcciones(Color fondo) {
387	private BotonRedondeado botonAccion(IconosUsuarios.Tipo icono,
422	private final BotonRedondeado editar = botonAccion(
424	private final BotonRedondeado baja = botonAccion(
453	private void initComponents() {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.IngredienteCRUD;	Importa CRUD.IngredienteCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
5	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
7	import Modelos.PaginaIngredientes;	Importa Modelos.PaginaIngredientes para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.

9	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
16	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
17	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
18	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.AbstractCellEditor;	Importa javax.swing.AbstractCellEditor para poder usar esa clase sin escribir su nombre completo.
20	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
21	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
22	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
23	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
24	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
25	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
26	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
27	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
28	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
29	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
30	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
31	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
32	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
33	import javax.swing.table.TableCellEditor;	Importa javax.swing.table.TableCellEditor para poder usar esa clase sin escribir su nombre completo.
34	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	/** Catálogo de ingredientes integrado al módulo Inventario. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
37	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
38	public class CatalogoIngredientesPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	private static final int POR_PAGINA = 10;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
41	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
42	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
43	private static final Color BORDE = new Color(225, 230, 237);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
44	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
45	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	private final IngredienteCRUD crud = new IngredienteCRUD();	Crea una nueva instancia de IngredienteCRUD y la usa o guarda para continuar el proceso.
48	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
49	private final ModeloTabla modelo = new ModeloTabla();	Crea una nueva instancia de ModeloTabla y la usa o guarda para continuar el proceso.
50	private final JButton[] botonesPagina;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
51	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
52	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

53	private int totalPaginas = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
54	private int secuenciaCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55	private Boolean stockBajoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
56	private Boolean estadoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	private boolean errorMostrado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	public CatalogoIngredientesPanel() {	Constructor de CatalogoIngredientesPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
60	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	botonesPagina = new JButton[]{botonPagina1, botonPagina2, botonPagina3};	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
62	temporizadorBusqueda = new Timer(350, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
63	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
64	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
67	prepararVista();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	configurarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	configurarPaginacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	botonAgregar.addActionListener(e -> abrirFormulario(null));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
72	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73	}	Cierra el bloque abierto anteriormente.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	public void recargarDatos() {	Declara el método recargarDatos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
76	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	}	Cierra el bloque abierto anteriormente.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	private void prepararVista() {	Declara el método prepararVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
80	labelTitulo.setFont(tema.negrita(42f));	Configura una propiedad mediante setFont en el objeto indicado.
81	labelSubtitulo.setFont(tema.regular(16f));	Configura una propiedad mediante setFont en el objeto indicado.
82	labelSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
83	botonAgregar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
84	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
85	botonLimpiar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
86	botonLimpiar.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
87	IconosUsuarios.Tipo.FILTRO, ROJO, 19));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	botonLimpiar.setIconTextGap(9);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
89	scrollTabla.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
90	scrollTabla.getViewport().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
91	etiquetaRango.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
92	etiquetaRango.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
93	}	Cierra el bloque abierto anteriormente.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
96	tablaIngredientes.setModel(modelo);	Configura una propiedad mediante setModel en el objeto indicado.

97	<code>tablaIngredientes.aplicarEstilo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	<code>tablaIngredientes.setRowHeight(53);</code>	Configura una propiedad mediante <code>setRowHeight</code> en el objeto indicado.
99	<code>tablaIngredientes.setColumnsCentradas("0,2,3,4,5,6,7");</code>	Configura una propiedad mediante <code>setColumnsCentradas</code> en el objeto indicado.
100	<code>tablaIngredientes.getColumnModel().getColumn(5)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	<code>.setCellRenderer(new RenderNivel());</code>	Crea una nueva instancia de <code>RenderNivel</code> y la usa o guarda para continuar el proceso.
102	<code>tablaIngredientes.getColumnModel().getColumn(6)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
103	<code>.setCellRenderer(new RenderEstado());</code>	Crea una nueva instancia de <code>RenderEstado</code> y la usa o guarda para continuar el proceso.
104	<code>tablaIngredientes.getColumnModel().getColumn(7)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
105	<code>.setCellRenderer(new RenderAcciones());</code>	Crea una nueva instancia de <code>RenderAcciones</code> y la usa o guarda para continuar el proceso.
106	<code>tablaIngredientes.getColumnModel().getColumn(7)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	<code>.setCellEditor(new EditorAcciones());</code>	Crea una nueva instancia de <code>EditorAcciones</code> y la usa o guarda para continuar el proceso.
108	<code>int[] anchos = {80, 330, 155, 135, 135, 115, 115, 125};</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
109	<code>for (int i = 0; i < anchos.length; i++) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
110	<code>tablaIngredientes.getColumnModel().getColumn(i)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
111	<code>.setPreferredWidth(anchos[i]);</code>	Configura una propiedad mediante <code>setPreferredWidth</code> en el objeto indicado.
112	<code>}</code>	Cierra el bloque abierto anteriormente.
113	<code>}</code>	Cierra el bloque abierto anteriormente.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	<code>private void configurarFiltros() {</code>	Declara el método <code>configurarFiltros</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
116	<code>prepararSelector(filtroStock);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	<code>filtroStock.addMenuOpcionListener(e -> {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
118	<code>filtroStock.setText(e.getActionCommand());</code>	Cambia el texto visible de un componente de la interfaz.
119	<code>stockBajoActual = switch (e.getActionCommand()) {</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
120	<code>case "Stock bajo" -> true;</code>	Define una de las rutas posibles dentro del switch.
121	<code>case "Stock normal" -> false;</code>	Define una de las rutas posibles dentro del switch.
122	<code>default -> null;</code>	Define una de las rutas posibles dentro del switch.
123	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
124	<code>paginaActual = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
125	<code>cargarPagina();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	<code>prepararSelector(filtroEstado);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
128	<code>filtroEstado.addMenuOpcionListener(e -> {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	<code>filtroEstado.setText(e.getActionCommand());</code>	Cambia el texto visible de un componente de la interfaz.
130	<code>estadoActual = switch (e.getActionCommand()) {</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
131	<code>case "Activos" -> true;</code>	Define una de las rutas posibles dentro del switch.
132	<code>case "Inactivos" -> false;</code>	Define una de las rutas posibles dentro del switch.
133	<code>default -> null;</code>	Define una de las rutas posibles dentro del switch.
134	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
135	<code>paginaActual = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
136	<code>cargarPagina();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	<code>campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {</code>	Crea una nueva instancia de <code>DocumentListener</code> y la usa o guarda para continuar el proceso.
139	<code>@Override public void insertUpdate(DocumentEvent e) { temporizadorBusqueda.restart(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
140	<code>@Override public void removeUpdate(DocumentEvent e) { temporizadorBusqueda.restart(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

141	@Override public void changedUpdate(DocumentEvent e) { temporizadorBusqueda.restart(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
142	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	botonLimpiar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
144	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
145	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.
146	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	stockBajoActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
148	estadoActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
149	filtroStock.setText("Todos los niveles");	Cambia el texto visible de un componente de la interfaz.
150	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
151	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
152	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
153	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154	}	Cierra el bloque abierto anteriormente.
155		Línea en blanco: separa visualmente bloques para que el código sea más legible.
156	private void prepararSelector(BotonDesplegable selector) {	Declara el método prepararSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
157	selector.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
158	selector.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
159	selector.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
160	selector.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
161	selector.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
162	selector.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
163	selector.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
164	selector.setAnchoMenu(280);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
165	selector.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
166	}	Cierra el bloque abierto anteriormente.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	private void configurarPaginacion() {	Declara el método configurarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
169	prepararBotonPagina(botonAnterior, IconosUsuarios.Tipo.ANTERIOR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	botonAnterior.addActionListener(e -> cambiarPagina(paginaActual - 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
171	for (JButton boton : botonesPagina) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
172	prepararBotonPagina(boton, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173	boton.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
174	Object numero = boton.getClientProperty("pagina");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
175	if (numero instanceof Integer pagina) cambiarPagina(pagina);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
176	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177	}	Cierra el bloque abierto anteriormente.
178	prepararBotonPagina(botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	botonSiguiente.addActionListener(e -> cambiarPagina(paginaActual + 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
180	}	Cierra el bloque abierto anteriormente.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	private void prepararBotonPagina(JButton boton, IconosUsuarios.Tipo icono) {	Declara el método prepararBotonPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
183	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

184	redondeado.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
185	redondeado.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
186	redondeado.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
187	redondeado.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
188	redondeado.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
189	redondeado.setRadio(10);	Configura una propiedad mediante setRadio en el objeto indicado.
190	}	Cierra el bloque abierto anteriormente.
191	boton.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
192	boton.setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
193	if (icono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
194	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
195	boton.setIcon(IconosUsuarios.crear(icono, AZUL, 16));	Configura una propiedad mediante setIcon en el objeto indicado.
196	}	Cierra el bloque abierto anteriormente.
197	}	Cierra el bloque abierto anteriormente.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199	private void cargarPagina() {	Declara el método cargarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
200	final int solicitud = ++secuenciaCarga;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
201	tablaIngredientes.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
202	etiquetaRango.setText("Cargando ingredientes...");	Cambia el texto visible de un componente de la interfaz.
203	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
204	final int pagina = paginaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
205	final Boolean nivel = stockBajoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
206	final Boolean estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
207	new SwingWorker<PaginaIngredientes, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
208	@Override protected PaginaIngredientes doInBackground() throws SQLException {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
209	return crud.listarPagina(busqueda, nivel, estado, pagina, POR_PAGINA);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
210	}	Cierra el bloque abierto anteriormente.
211	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
212	if (solicitud != secuenciaCarga) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
213	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
214	PaginaIngredientes datos = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
215	totalPaginas = Math.max(1,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
216	(datos.totalRegistros() + POR_PAGINA - 1) / POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
217	if (paginaActual > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
218	paginaActual = totalPaginas;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
219	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
221	}	Cierra el bloque abierto anteriormente.
222	modelo.setIngredientes(datos.ingredientes());	Configura una propiedad mediante setIngredientes en el objeto indicado.
223	actualizarPaginacion(datos.totalRegistros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
224	errorMostrado = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
225	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
226	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

228	modelo.setIngredientes(List.of());	Configura una propiedad mediante setIngredientes en el objeto indicado.
229	actualizarPaginacion(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
230	mostrarError("cargar los ingredientes", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
231	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
232	tablaIngredientes.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
233	}	Cierra el bloque abierto anteriormente.
234	}	Cierra el bloque abierto anteriormente.
235	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	}	Cierra el bloque abierto anteriormente.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	private void actualizarPaginacion(int total) {	Declara el método actualizarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
239	if (total == 0) etiquetaRango.setText("No se encontraron ingredientes");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
240	else {	Ruta alternativa que se ejecuta cuando la condición anterior fue falsa.
241	int inicio = (paginaActual - 1) * POR_PAGINA + 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
242	int fin = Math.min(inicio + POR_PAGINA - 1, total);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
243	etiquetaRango.setText(String.format(	Cambia el texto visible de un componente de la interfaz.
244	"Mostrando %d-%d de %d ingredientes", inicio, fin, total));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245	}	Cierra el bloque abierto anteriormente.
246	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
247	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
248	int inicio = Math.max(1, Math.min(paginaActual - 1, totalPaginas - 2));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
249	for (int i = 0; i < botonesPagina.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
250	JButton boton = botonesPagina[i];	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
251	int pagina = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
252	boton.setVisible(pagina <= totalPaginas);	Muestra u oculta una ventana o componente según el valor booleano enviado.
253	if (pagina > totalPaginas) continue;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
254	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
256	boolean seleccionada = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
257	boton.setFont(seleccionada ? tema.negrita(13f) : tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
258	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
259	redondeado.setColorInicio(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
260	redondeado.setColorFinal(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
261	redondeado.setColorBorde(seleccionada ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
262	}	Cierra el bloque abierto anteriormente.
263	}	Cierra el bloque abierto anteriormente.
264	}	Cierra el bloque abierto anteriormente.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	private void cambiarPagina(int pagina) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
267	if (pagina < 1 pagina > totalPaginas pagina == paginaActual) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
268	paginaActual = pagina;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
269	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
270	}	Cierra el bloque abierto anteriormente.
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.

272	private void abrirFormulario(Ingrediente ingrediente) {	Declara el método abrirFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
273	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
274	if (IngredienteFormDialog.mostrar(propietario, crud, ingrediente)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
275	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276	}	Cierra el bloque abierto anteriormente.
277	}	Cierra el bloque abierto anteriormente.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	private void editar(Ingrediente ingrediente) {	Declara el método editar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
280	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
281	Ingrediente actual = crud.obtenerPorId(ingrediente.getIdIngrediente());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
282	if (actual == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
283	JOptionPane.showMessageDialog(this, "No se encontró el ingrediente.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
284	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
285	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
286	}	Cierra el bloque abierto anteriormente.
287	abrirFormulario(actual);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
288	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
289	mostrarError("obtener el ingrediente", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
290	}	Cierra el bloque abierto anteriormente.
291	}	Cierra el bloque abierto anteriormente.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	private void desactivar(Ingrediente ingrediente) {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
294	if (!ingrediente.isActivo()) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
295	int opcion = JOptionPane.showConfirmDialog(this,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
296	"¿Desactivar «" + ingrediente.getNombre() + "»?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
297	"Confirmar desactivación", JOptionPane.YES_NO_OPTION);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	if (opcion != JOptionPane.YES_OPTION) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
299	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
300	crud.cambiarEstado(ingrediente.getIdIngrediente(), false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
301	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
303	mostrarError("desactivar el ingrediente", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
304	}	Cierra el bloque abierto anteriormente.
305	}	Cierra el bloque abierto anteriormente.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	private void mostrarError(String accion, Throwable error) {	Declara el método mostrarError. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
308	if (errorMostrado) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
309	errorMostrado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
310	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	"No fue posible " + accion + ".\n"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	+ (error == null ? "Error no especificado." : error.getMessage()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
313	"Error de base de datos", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
314	}	Cierra el bloque abierto anteriormente.
315		Línea en blanco: separa visualmente bloques para que el código sea más legible.

316	private final class ModeloTabla extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
317	private final String[] columnas = {"ID", "Ingrediente", "Unidad",	Declara una constante o referencia que no puede reasignarse después de inicializarse.
318	"Stock actual", "Stock mínimo", "Nivel", "Estado", "Acciones"};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
319	private List<Ingrediente> ingredientes = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
320	void setIngredientes(List<Ingrediente> nuevos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
321	ingredientes = List.copyOf(nuevos);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
322	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
323	}	Cierra el bloque abierto anteriormente.
324	@Override public int getRowCount() { return ingredientes.size(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
325	@Override public int getColumnCount() { return columnas.length; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
326	@Override public String getColumnName(int columna) { return columnas[columna]; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
327	@Override public boolean isCellEditable(int fila, int columna) { return columna == 7; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
328	@Override public Class<?> getColumnClass(int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
329	return columna >= 5 ? Ingrediente.class : String.class;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
330	}	Cierra el bloque abierto anteriormente.
331	@Override public Object getValueAt(int fila, int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
332	Ingrediente item = ingredientes.get(fila);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
333	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
334	case 0 -> String.format("#%04d", item.getIdIngrediente());	Define una de las rutas posibles dentro del switch.
335	case 1 -> item.getNombre();	Define una de las rutas posibles dentro del switch.
336	case 2 -> item.getUnidadMedida();	Define una de las rutas posibles dentro del switch.
337	case 3 -> item.getStockActual().stripTrailingZeros().toString();	Define una de las rutas posibles dentro del switch.
338	case 4 -> item.getStockMinimo().stripTrailingZeros().toString();	Define una de las rutas posibles dentro del switch.
339	case 5, 6, 7 -> item;	Define una de las rutas posibles dentro del switch.
340	default -> "";	Define una de las rutas posibles dentro del switch.
341	};	Cierra el bloque abierto anteriormente y termina la declaración.
342	}	Cierra el bloque abierto anteriormente.
343	}	Cierra el bloque abierto anteriormente.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
345	private JPanel etiquetaEstado(String texto, Color color, Color fondo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
346	JTable tabla, boolean seleccionado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
347	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 13));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
348	panel.setBackground(seleccionado ? tabla.getSelectionBackground() : Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
349	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
350	etiqueta.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
351	etiqueta.setFont(tema.media(12f));	Configura una propiedad mediante setFont en el objeto indicado.
352	etiqueta.setForeground(color);	Configura una propiedad mediante setForeground en el objeto indicado.
353	etiqueta.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
354	etiqueta.setBorder(new EmptyBorder(4, 11, 4, 11));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
355	panel.add(etiqueta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
356	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
357	}	Cierra el bloque abierto anteriormente.
358		Línea en blanco: separa visualmente bloques para que el código sea más legible.
359	private final class RenderNivel implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.

360	@Override public Component getTableCellRendererComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
361	Object valor, boolean seleccionado, boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
362	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
363	return etiquetaEstado(item.tieneStockBajo() ? "Bajo" : "Normal",	Finaliza el método y devuelve el valor indicado a quien lo llamó.
364	item.tieneStockBajo() ? ROJO : new Color(34, 139, 71),	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
365	item.tieneStockBajo() ? new Color(252, 233, 233)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
366	: new Color(230, 245, 234), tabla, seleccionado);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
367	}	Cierra el bloque abierto anteriormente.
368	}	Cierra el bloque abierto anteriormente.
369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
370	private final class RenderEstado implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
371	@Override public Component getTableCellRendererComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
372	Object valor, boolean seleccionado, boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
373	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
374	return etiquetaEstado(item.isActive() ? "Activo" : "Inactivo",	Finaliza el método y devuelve el valor indicado a quien lo llamó.
375	item.isActive() ? new Color(34, 139, 71) : SECUNDARIO,	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
376	item.isActive() ? new Color(230, 245, 234)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
377	: new Color(237, 239, 243), tabla, seleccionado);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
378	}	Cierra el bloque abierto anteriormente.
379	}	Cierra el bloque abierto anteriormente.
380		Línea en blanco: separa visualmente bloques para que el código sea más legible.
381	private JPanel panelAcciones(Color fondo) {	Declara el método panelAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
382	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 11));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
383	panel.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
384	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
385	}	Cierra el bloque abierto anteriormente.
386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
387	private BotonRedondeado botonAccion(IconosUsuarios.Tipo icono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
388	Color color, String texto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
389	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
390	boton.setIcon(IconosUsuarios.crear(icono, color, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
391	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
392	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
393	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
394	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
395	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
396	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
397	boton.setRadio(9);	Configura una propiedad mediante setRadio en el objeto indicado.
398	boton.setPreferredSize(new Dimension(34, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
399	boton.setToolTipText(texto);	Configura una propiedad mediante setToolTipText en el objeto indicado.
400	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
401	}	Cierra el bloque abierto anteriormente.
402		Línea en blanco: separa visualmente bloques para que el código sea más legible.
403	private final class RenderAcciones implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.

404	@Override public Component getTableCellRendererComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
405	Object valor, boolean seleccionado, boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
406	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
407	JPanel panel = panelAcciones(seleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
408	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	panel.add(botonAccion(IconosUsuarios.Tipo.EDITAR,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
410	new Color(214, 151, 0), "Editar ingrediente"));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
411	BotonRedondeado baja = botonAccion(IconosUsuarios.Tipo.ELIMINAR,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
412	ROJO, "Desactivar ingrediente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413	baja.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
414	panel.add(baja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
415	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
416	}	Cierra el bloque abierto anteriormente.
417	}	Cierra el bloque abierto anteriormente.
418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
419	private final class EditorAcciones extends AbstractCellEditor	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
420	implements TableCellEditor {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
421	private final JPanel panel = panelAcciones(Color.WHITE);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
422	private final BotonRedondeado editar = botonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
423	IconosUsuarios.Tipo.EDITAR, new Color(214, 151, 0), "Editar ingrediente");	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
424	private final BotonRedondeado baja = botonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
425	IconosUsuarios.Tipo.ELIMINAR, ROJO, "Desactivar ingrediente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
426	private Ingrediente item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
427	EditorAcciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
428	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429	panel.add(baja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430	editar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
431	Ingrediente seleccionado = item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
432	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433	editar(seleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
434	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
435	baja.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
436	Ingrediente seleccionado = item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
437	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
438	desactivar(seleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
439	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
440	}	Cierra el bloque abierto anteriormente.
441	@Override public Component getTableCellEditorComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
442	Object valor, boolean seleccionado, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	item = (Ingrediente) valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
444	baja.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
445	panel.setBackground(tabla.getSelectionBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
446	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
447	}	Cierra el bloque abierto anteriormente.

448	@Override public Object getCellEditorValue() { return item; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
449	}	Cierra el bloque abierto anteriormente.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
452	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
453	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
454		Línea en blanco: separa visualmente bloques para que el código sea más legible.
455	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
456	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
457	botonAgregar = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
458	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
459	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
460	filtroStock = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
461	filtroEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
462	botonLimpiar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
463	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
464	scrollTabla = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
465	tablaIngredientes = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
466	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
467	panelPaginacion = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
468	botonAnterior = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
469	botonPagina1 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
470	botonPagina2 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
471	botonPagina3 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
472	botonSiguiente = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
473		Línea en blanco: separa visualmente bloques para que el código sea más legible.
474	setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
475	setPreferredSize(new java.awt.Dimension(1530, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
476	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
477		Línea en blanco: separa visualmente bloques para que el código sea más legible.
478	labelTitulo.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
479	labelTitulo.setText("Catálogo de Ingredientes");	Cambia el texto visible de un componente de la interfaz.
480	add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 16, 650, 64));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
481		Línea en blanco: separa visualmente bloques para que el código sea más legible.
482	labelSubtitulo.setText("Define los insumos, su unidad y el nivel mínimo de reposición");	Cambia el texto visible de un componente de la interfaz.
483	add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 76, 690, 36));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
484		Línea en blanco: separa visualmente bloques para que el código sea más legible.
485	botonAgregar.setText("Agregar ingrediente");	Cambia el texto visible de un componente de la interfaz.
486	add(botonAgregar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1290, 42, 220, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
487		Línea en blanco: separa visualmente bloques para que el código sea más legible.
488	panelFiltros.setColorBorde(new java.awt.Color(225, 230, 237));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
489	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
490	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
491	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.

492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
493	<code>campoBusqueda.setPlaceholder("Buscar ingrediente o unidad...");</code>	Configura una propiedad mediante <code>setPlaceholder</code> en el objeto indicado.
494	<code>panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 20, 440, 55));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
495		Línea en blanco: separa visualmente bloques para que el código sea más legible.
496	<code>filtroStock.setText("Todos los niveles");</code>	Cambia el texto visible de un componente de la interfaz.
497	<code>filtroStock.setTextoDesplegable("Todos los niveles;Stock bajo;Stock normal");</code>	Configura una propiedad mediante <code>setTextoDesplegable</code> en el objeto indicado.
498	<code>panelFiltros.add(filtroStock, new org.netbeans.lib.awtextra.AbsoluteConstraints(480, 22, 280, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
499		Línea en blanco: separa visualmente bloques para que el código sea más legible.
500	<code>filtroEstado.setText("Todos los estados");</code>	Cambia el texto visible de un componente de la interfaz.
501	<code>filtroEstado.setTextoDesplegable("Todos los estados;Activos;Inactivos");</code>	Configura una propiedad mediante <code>setTextoDesplegable</code> en el objeto indicado.
502	<code>panelFiltros.add(filtroEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 22, 280, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
503		Línea en blanco: separa visualmente bloques para que el código sea más legible.
504	<code>botonLimpiar.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
505	<code>botonLimpiar.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
506	<code>botonLimpiar.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
507	<code>botonLimpiar.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.
508	<code>botonLimpiar.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
509	<code>botonLimpiar.setText("Limpiar filtros");</code>	Cambia el texto visible de un componente de la interfaz.
510	<code>panelFiltros.add(botonLimpiar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1190, 22, 250, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
511		Línea en blanco: separa visualmente bloques para que el código sea más legible.
512	<code>add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 175, 1470, 95));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
513		Línea en blanco: separa visualmente bloques para que el código sea más legible.
514	<code>panelTabla.setColorBorde(new java.awt.Color(225, 230, 237));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
515	<code>panelTabla.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
516	<code>panelTabla.setSombra(false);</code>	Configura una propiedad mediante <code>setSombra</code> en el objeto indicado.
517	<code>panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
518		Línea en blanco: separa visualmente bloques para que el código sea más legible.
519	<code>scrollTabla.setBorder(null);</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
520	<code>scrollTabla.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setVerticalScrollBarPolicy</code> en el objeto indicado.
521	<code>scrollTabla.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setHorizontalScrollBarPolicy</code> en el objeto indicado.
522	<code>scrollTabla.setViewportView(tablaIngredientes);</code>	Configura una propiedad mediante <code>setViewportView</code> en el objeto indicado.
523		Línea en blanco: separa visualmente bloques para que el código sea más legible.
524	<code>panelTabla.add(scrollTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 555));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
525		Línea en blanco: separa visualmente bloques para que el código sea más legible.
526	<code>etiquetaRango.setText("Mostrando ingredientes");</code>	Cambia el texto visible de un componente de la interfaz.
527	<code>panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 581, 430, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
528		Línea en blanco: separa visualmente bloques para que el código sea más legible.
529	<code>panelPaginacion.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
530	<code>panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
531	<code>panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
532		Línea en blanco: separa visualmente bloques para que el código sea más legible.
533	<code>botonPagina1.setText("1");</code>	Cambia el texto visible de un componente de la interfaz.
534	<code>panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
535		Línea en blanco: separa visualmente bloques para que el código sea más legible.

536	botonPagina2.setText("2");	Cambia el texto visible de un componente de la interfaz.
537	panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
538		Línea en blanco: separa visualmente bloques para que el código sea más legible.
539	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
540	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
541	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
542		Línea en blanco: separa visualmente bloques para que el código sea más legible.
543	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 581, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
544		Línea en blanco: separa visualmente bloques para que el código sea más legible.
545	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 285, 1470, 640));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
546	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
547		Línea en blanco: separa visualmente bloques para que el código sea más legible.
548	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
549	private Componentes.BotonDerretido botonAgregar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
550	private Componentes.BotonRedondeado botonAnterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
551	private Componentes.BotonRedondeado botonLimpiar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
552	private Componentes.BotonRedondeado botonPagina1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
553	private Componentes.BotonRedondeado botonPagina2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
554	private Componentes.BotonRedondeado botonPagina3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
555	private Componentes.BotonRedondeado botonSiguiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
556	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
557	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
558	private Componentes.BotonDesplegable filtroEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
559	private Componentes.BotonDesplegable filtroStock;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
560	private javax.swing.JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
561	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
562	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
563	private javax.swing.JPanel panelPaginacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
564	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
565	private javax.swing.JScrollPane scrollTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
566	private Componentes.TablaAdministrativa tablaIngredientes;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
567	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
568	}	Cierra el bloque abierto anteriormente.

16.55 src/GUI_ADMINISTRADOR/CombosPanel.form

Metadatos visuales de NetBeans para diseñar CombosPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 33 | Archivo 55 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	</AuxValues>	Cierra la sección XML abierta anteriormente.
20		Línea en blanco para organizar el XML del formulario.
21	<Layout>	Indica el administrador de distribución que organiza los componentes hijos.
22	<DimensionLayout dim="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
24	<EmptySpace min="0" pref="1580" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	</Group>	Cierra la sección XML abierta anteriormente.
26	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
27	<DimensionLayout dim="1">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<EmptySpace min="0" pref="980" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	</Group>	Cierra la sección XML abierta anteriormente.
31	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
32	</Layout>	Cierra la sección XML abierta anteriormente.
33	</Form>	Cierra la sección XML abierta anteriormente.

16.56 src/GUI_ADMINISTRADOR/CombosPanel.java

Interfaz del módulo administrador: CombosPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 46 | Archivo 56 de 157

Clases/tipos detectados: CombosPanel

Línea	Constructor o método detectado
16	public CombosPanel() {
27	private void initComponents() {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JPanel.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author Humberto Alvarado	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class CombosPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	* Creates new form CombosPanel	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	public CombosPanel() {	Constructor de CombosPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
17	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18	}	Cierra el bloque abierto anteriormente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
26	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
27	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	javax.swing.GroupLayout layout = new javax.swing.GroupLayout(this);	Crea una nueva instancia de javax.swing.GroupLayout y la usa o guarda para continuar el proceso.
32	this.setLayout(layout);	Configura una propiedad mediante setLayout en el objeto indicado.

33	layout.setHorizontalGroup(	Configura una propiedad mediante setHorizontalGroup en el objeto indicado.
34	layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	.addGap(0, 1580, Short.MAX_VALUE)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	layout.setVerticalGroup(	Configura una propiedad mediante setVerticalGroup en el objeto indicado.
38	layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	.addGap(0, 980, Short.MAX_VALUE)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
45	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
46	}	Cierra el bloque abierto anteriormente.

16.57 src/GUI_ADMINISTRADOR/ConfiguracionMenuAdminDialog.java

Editor administrativo de presentaciones, grupos, opciones, componentes y receta del producto.

Tipo: java | Líneas: 266 | Archivo 57 de 157

Clases/tipos detectados: ConfiguracionMenuAdminDialog, Item

Línea	Constructor o método detectado
33	public ConfiguracionMenuAdminDialog(Window propietario, Producto producto) {
49	private JComponent crearContenido() {
76	private JComponent crearPresentaciones() {
100	private JComponent crearIngredientes() {
123	private JPanel columna(String titulo, JList<?> lista, JButton... botones) {
139	private JLabel etiquetaLista(String texto, boolean seleccionado) {
147	private BotonRedondeado boton(String texto, Color fondo, Color frente) {
153	private JButton botonAccion(String texto, java.awt.event.ActionListener accion) {
157	private void conectarEventos() {
163	private void recargar() {
171	private void cargarGrupos() {
177	private void cargarOpciones() {
182	private void cargarComponentes() {
187	private void editarPresentacion(PresentacionMenu original) {
197	private void editarGrupo(GrupoMenu original) {
209	private void editarOpcion(OpcionMenu original) {
218	private void agregarComponente() {
228	private void quitarComponente() {
234	private void editarIngrediente(IngredienteProducto original) {
250	private void desactivarPresentacion(){PresentacionMenu p=presentaciones.getSelectedValue();if(p!=null)desactivar("presentacion_menu","id_presentacion",p.idPresentacion())
251	private void desactivarGrupo(){GrupoMenu g=grupos.getSelectedValue();if(g!=null)desactivar("grupo_presentacion","id_grupo",g.idGrupo());}
252	private void desactivarOpcion(){OpcionMenu o=opciones.getSelectedValue();if(o!=null)desactivar("opcion_grupo","id_opcion",o.idOpcion());}
253	private void desactivar(String tabla,String campo,int id){try{dao.desactivar(tabla,campo,id);recargar();}catch(Exception ex){error(ex);}}
254	private void desactivarIngrediente(){IngredienteProducto i=ingredientes.getSelectedValue();if(i==null)return;try{dao.desactivarIngredienteProducto(i.idProductoIngrediente
256	private boolean confirmarFormulario(String titulo,String[] etiquetas,JComponent...campos){
261	private BigDecimal decimal(String texto){try{BigDecimal d=new BigDecimal(texto.trim().replace(",",""));if(d.signum()<0 d.stripTrailingZeros().scale()>2)throw new Number
262	private void validarTexto(String texto,String campo){if(texto==null texto.isBlank())throw new IllegalArgumentException("Ingresa el "+campo+").;}
263	private void aviso(String texto){JOptionPane.showMessageDialog(this,texto,"Configuración",JOptionPane.INFORMATION_MESSAGE);}
264	private void error(Exception ex){JOptionPane.showMessageDialog(this,ex.getMessage(),"No se pudo guardar",JOptionPane.ERROR_MESSAGE);}

Línea	Constructor o método detectado
265	private record Item(int id,String nombre){@Override public String toString(){return nombre;}}

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
4	import DAO.ConfiguracionMenuDAO;	Importa DAO.ConfiguracionMenuDAO para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.*;	Importa Modelos.* para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.*;	Importa java.awt.* para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
9	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
10	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
11	import javax.swing.*;	Importa javax.swing.* para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	/** Editor administrativo del producto: receta y presentaciones del menú. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	public final class ConfiguracionMenuAdminDialog extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
16	private static final Color AZUL = new Color(0,20,43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
17	private static final Color AMARILLO = new Color(255,188,0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
18	private static final Color BORDE = new Color(225,229,235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
19	private final Producto producto;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
20	private final ConfiguracionMenuDAO dao = new ConfiguracionMenuDAO();	Crea una nueva instancia de ConfiguracionMenuDAO y la usa o guarda para continuar el proceso.
21	private ConfiguracionProducto configuracion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	private final DefaultListModel<PresentacionMenu> modeloPresentaciones = new DefaultListModel<>();	Crea una nueva instancia de DefaultListModel y la usa o guarda para continuar el proceso.
23	private final DefaultListModel<GrupoMenu> modeloGrupos = new DefaultListModel<>();	Crea una nueva instancia de DefaultListModel y la usa o guarda para continuar el proceso.
24	private final DefaultListModel<OpcionMenu> modeloOpciones = new DefaultListModel<>();	Crea una nueva instancia de DefaultListModel y la usa o guarda para continuar el proceso.
25	private final DefaultListModel<ComponenteMenu> modeloComponentes = new DefaultListModel<>();	Crea una nueva instancia de DefaultListModel y la usa o guarda para continuar el proceso.
26	private final DefaultListModel<IngredienteProducto> modeloIngredientes = new DefaultListModel<>();	Crea una nueva instancia de DefaultListModel y la usa o guarda para continuar el proceso.
27	private final JList<PresentacionMenu> presentaciones = new JList<>(modeloPresentaciones);	Crea una nueva instancia de JList y la usa o guarda para continuar el proceso.
28	private final JList<GrupoMenu> grupos = new JList<>(modeloGrupos);	Crea una nueva instancia de JList y la usa o guarda para continuar el proceso.
29	private final JList<OpcionMenu> opciones = new JList<>(modeloOpciones);	Crea una nueva instancia de JList y la usa o guarda para continuar el proceso.
30	private final JList<ComponenteMenu> componentes = new JList<>(modeloComponentes);	Crea una nueva instancia de JList y la usa o guarda para continuar el proceso.
31	private final JList<IngredienteProducto> ingredientes = new JList<>(modeloIngredientes);	Crea una nueva instancia de JList y la usa o guarda para continuar el proceso.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	public ConfiguracionMenuAdminDialog(Window propietario, Producto producto) {	Constructor de ConfiguracionMenuAdminDialog: prepara un objeto nuevo y deja sus componentes listos para usarse.
34	super(proprietario, "Opciones y receta", ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	this.producto = producto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
36	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	setMinimumSize(new Dimension(1050, 700));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
38	setContentPane(crearContenido());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	conectarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

41	dao.asegurarPresentacionIndividual(producto.getIdProducto());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
43	error(ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	}	Cierra el bloque abierto anteriormente.
45	recargar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	private JComponent crearContenido() {	Declara el método crearContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
50	JPanel raiz = new JPanel(new BorderLayout(0,12));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
51	raiz.setBackground(new Color(247,248,250));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
52	raiz.setBorder(new EmptyBorder(18,18,18,18));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
53	JPanel cabecera = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
54	cabecera.setBackground(AZUL);	Configura una propiedad mediante setBackground en el objeto indicado.
55	cabecera.setBorder(new EmptyBorder(17,20,17,20));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
56	JLabel titulo = new JLabel("Configurar " + producto.getNombre());	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
57	titulo.setFont(new Font("SansSerif",Font.BOLD,24)); titulo.setForeground(Color.WHITE);	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
58	JLabel ayuda = new JLabel("Crea Individual, Menú, Infantil o Combo; luego define sus elecciones y productos internos.");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
59	ayuda.setForeground(new Color(219,226,233));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
60	JPanel textos = new JPanel(); textos.setOpaque(false); textos.setLayout(new BoxLayout(textos,BoxLayout.Y_AXIS));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
61	textos.add(titulo); textos.add(ayuda); cabecera.add(textos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	raiz.add(cabecera,BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	JTabbedPane pestanas = new JTabbedPane();	Crea una nueva instancia de JTabbedPane y la usa o guarda para continuar el proceso.
65	pestanas.setFont(new Font("SansSerif",Font.BOLD,14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
66	pestanas.addTab("Presentaciones y opciones", crearPresentaciones());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	pestanas.addTab("Ingredientes y personalización", crearIngredientes());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68	raiz.add(pestanas,BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	BotonRedondeado cerrar = boton("Cerrar",AMARILLO,AZUL);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
70	cerrar.addActionListener(e -> dispose());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
71	JPanel pie = new JPanel(new FlowLayout(FlowLayout.RIGHT)); pie.setOpaque(false); pie.add(cerrar);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
72	raiz.add(pie,BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73	return raiz;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
74	}	Cierra el bloque abierto anteriormente.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	private JComponent crearPresentaciones() {	Declara el método crearPresentaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
77	JPanel panel = new JPanel(new GridLayout(1,4,10,0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
78	panel.setBackground(Color.WHITE); panel.setBorder(new EmptyBorder(12,12,12,12));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
79	panel.add(columna("1. Presentaciones",presentaciones,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	botonAccion("Nueva",e->editarPresentacion(null)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
81	botonAccion("Editar",e->editarPresentacion(presentaciones.getSelectedValue())),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	botonAccion("Desactivar",e->desactivarPresentacion()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.

83	panel.add(columna("2. Grupos de elección",grupos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	botonAccion("Nuevo",e->editarGrupo(null)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
85	botonAccion("Editar",e->editarGrupo(grupos.getSelectedValue())),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	botonAccion("Desactivar",e->desactivarGrupo()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	panel.add(columna("3. Opciones",opciones,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	botonAccion("Nueva",e->editarOpcion(null)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
89	botonAccion("Editar",e->editarOpcion(opciones.getSelectedValue())),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
90	botonAccion("Desactivar",e->desactivarOpcion()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	panel.add(columna("4. Productos internos",componentes,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
92	botonAccion("Agregar",e->agregarComponente()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
93	botonAccion("Quitar",e->quitarComponente()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	componentes.setCellRenderer((list,value,index,selected,focus) -> etiquetaLista(	Configura una propiedad mediante setCellRenderer en el objeto indicado.
95	value.nombre()+" x"+value.cantidad(),selected));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	opciones.setCellRenderer((list,value,index,selected,focus) -> etiquetaLista(value.toString(),selected));	Configura una propiedad mediante setCellRenderer en el objeto indicado.
97	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
98	}	Cierra el bloque abierto anteriormente.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	private JComponent crearIngredientes() {	Declara el método crearIngredientes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
101	JPanel panel = new JPanel(new BorderLayout(0,12));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
102	panel.setBackground(Color.WHITE); panel.setBorder(new EmptyBorder(18,18,18,18));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
103	JLabel texto = new JLabel("Estos ingredientes controlan inventario y generan las opciones Sin / Extra de l cajero.");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
104	texto.setFont(new Font("SansSerif",Font.PLAIN,14)); texto.setForeground(new Color(92,103,124));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
105	panel.add(texto,BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106	ingredientes.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);	Configura una propiedad mediante setSelectionMode en el objeto indicado.
107	ingredientes.setFixedCellHeight(52);	Configura una propiedad mediante setFixedCellHeight en el objeto indicado.
108	ingredientes.setCellRenderer((list,value,index,selected,focus) -> etiquetaLista(	Configura una propiedad mediante setCellRenderer en el objeto indicado.
109	value.nombre()+" · base "+value.cantidadDefault()+	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
110	(value.permiteQuitar())?" · se puede quitar":""")+	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
111	(value.permiteExtra())?" · extra Q"+value.precioExtra():"",selected));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	JScrollPane scrollIngredientes = new JScrollPane(ingredientes);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
113	Componentes.DesplazamientoSuave.ocultarBarras(scrollIngredientes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	panel.add(scrollIngredientes,BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
115	JPanel acciones = new JPanel(new FlowLayout(FlowLayout.RIGHT,8,0)); acciones.setOpaque(false);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
116	acciones.add(botonAccion("Agregar ingrediente",e->editarIngrediente(null));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	acciones.add(botonAccion("Editar",e->editarIngrediente(ingredientes.getSelectedValue())));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	acciones.add(botonAccion("Desactivar",e->desactivarIngrediente()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	panel.add(acciones,BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
121	}	Cierra el bloque abierto anteriormente.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	private JPanel columna(String titulo, JList<?> lista, JButton... botones) {	Declara el método columna. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
124	JPanel panel = new JPanel(new BorderLayout(0,8)); panel.setOpaque(false);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.

125	JLabel etiqueta = new JLabel(titulo); etiqueta.setFont(new Font("SansSerif",Font.BOLD,14)); etiqueta.setForeground(AZUL);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
126	panel.add(etiqueta,BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	lista.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);	Configura una propiedad mediante setSelectionMode en el objeto indicado.
128	lista.setFixedCellHeight(44); lista.setFont(new Font("SansSerif",Font.PLAIN,13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
129	JScrollPane scroll = new JScrollPane(lista);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
130	Componentes.DesplazamientoSuave.ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	scroll.setBorder(BorderFactory.createLineBorder(BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
132	panel.add(scroll,BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	JPanel acciones = new JPanel(new GridLayout(0,1,0,5)); acciones.setOpaque(false);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
134	for(JButton boton:botones) acciones.add(boton);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
135	panel.add(acciones,BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
137	}	Cierra el bloque abierto anteriormente.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	private JLabel etiquetaLista(String texto, boolean seleccionado) {	Declara el método etiquetaLista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
140	JLabel etiqueta = new JLabel(texto); etiqueta.setOpaque(true);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
141	etiqueta.setBorder(new EmptyBorder(0,9,0,9));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
142	etiqueta.setBackground(seleccionado?new Color(255,243,203):Color.WHITE);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
143	etiqueta.setForeground(AZUL); etiqueta.setFont(new Font("SansSerif",Font.PLAIN,13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
144	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
145	}	Cierra el bloque abierto anteriormente.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	private BotonRedondeado boton(String texto, Color fondo, Color frente) {	Declara el método boton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
148	BotonRedondeado b=new BotonRedondeado(); b.setText(texto); b.setDegradado(false);	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
149	b.setColorInicio(fondo); b.setColorFinal(fondo); b.setColorBorde(fondo.equals(Color.WHITE)?BORDE:fondo);	Configura una propiedad mediante setColorInicio en el objeto indicado.
150	b.setGrosorBorde(1f); b.setRadio(14); b.setForeground(frente);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
151	b.setFont(new Font("SansSerif",Font.BOLD,13)); b.setPreferredSize(new Dimension(180,42)); return b;	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
152	}	Cierra el bloque abierto anteriormente.
153	private JButton botonAccion(String texto, java.awt.event.ActionListener accion) {	Declara el método botonAccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
154	BotonRedondeado b=boton(texto,Color.WHITE,AZUL); b.addActionListener(accion); return b;	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
155	}	Cierra el bloque abierto anteriormente.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	private void conectarEventos() {	Declara el método conectarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
158	presentaciones.addListSelectionListener(e->{ if(!e.getValueIsAdjusting()) cargarGrupos(); });	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	grupos.addListSelectionListener(e->{ if(!e.getValueIsAdjusting()) cargarOpciones(); });	Ejecuta una instrucción Java completa; el punto y coma marca su final.
160	opciones.addListSelectionListener(e->{ if(!e.getValueIsAdjusting()) cargarComponentes(); });	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	private void recargar() {	Declara el método recargar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
164	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
165	configuracion=dao.cargarProductoAdmin(producto.getIdProducto());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
166	modeloPresentaciones.clear(); for(PresentacionMenu p:configuracion.presentaciones()) modeloPresentaciones.addElement(p);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	if(!modeloPresentaciones.isEmpty()) presentaciones.setSelectedIndex(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

168	modeloIngredientes.clear(); for(IngredienteProducto i:dao.cargarIngredientes(producto.getIdProducto())) modeloIngredientes.addElement(i);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	} catch(Exception ex) { error(ex); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
170	}	Cierra el bloque abierto anteriormente.
171	private void cargarGrupos() {	Declara el método cargarGrupos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
172	modeloGrupos.clear(); modeloOpciones.clear(); modeloComponentes.clear();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173	PresentacionMenu p=presentaciones.getSelectedValue(); if(p==null)return;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
174	for(GrupoMenu g:p.grupos()) modeloGrupos.addElement(g);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
175	if(!modeloGrupos.isEmpty()) grupos.setSelectedIndex(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
176	}	Cierra el bloque abierto anteriormente.
177	private void cargarOpciones() {	Declara el método cargarOpciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
178	modeloOpciones.clear(); modeloComponentes.clear(); GrupoMenu g=grupos.getSelectedValue(); if(g==null)ret	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	urn;	
179	for(OpcionMenu o:g.opciones()) modeloOpciones.addElement(o);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
180	if(!modeloOpciones.isEmpty()) opciones.setSelectedIndex(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
181	}	Cierra el bloque abierto anteriormente.
182	private void cargarComponentes() {	Declara el método cargarComponentes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
183	modeloComponentes.clear(); OpcionMenu o=opciones.getSelectedValue(); if(o==null)return;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
184	for(ComponenteMenu c:o.componentes()) modeloComponentes.addElement(c);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
185	}	Cierra el bloque abierto anteriormente.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	private void editarPresentacion(PresentacionMenu original) {	Declara el método editarPresentacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
188	TextField nombre=new TextField(original==null?"".original.nombre());	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
189	JComboBox<String> tipo=new JComboBox<>(new String[]{"INDIVIDUAL","MENU","INFANTIL","COMBO"});	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
190	TextField precio=new TextField(original==null?producto.getPrecioBase().toString():original.precio).toString());	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
191	JCheckBox determinada=new JCheckBox("Presentación predeterminada",original!=null&&original.predetermi nada());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
192	if(original!=null)tipo.setSelectedItem(original.tipo());	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
193	if(!confirmarFormulario("Presentación",new String[]{"Nombre","Tipo","Precio (Q)",""},nombre,tipo,precio, determinada))return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
194	try { validarTexto(nombre.getText(),"nombre"); dao.guardarPresentacion(original==null?null:original.idPr esentacion(),producto.getIdProducto(),nombre.getText().trim(),(String)tipo.getSelectedItem(),decimal(precio.getT ext()),predeterminada.isSelected()); recargar(); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
195	catch(Exception ex){error(ex);}	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
196	}	Cierra el bloque abierto anteriormente.
197	private void editarGrupo(GrupoMenu original) {	Declara el método editarGrupo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
198	PresentacionMenu p=presentaciones.getSelectedValue(); if(p==null){aviso("Selecciona una presentación."); return;}	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
199	TextField nombre=new TextField(original==null?"".original.nombre());	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
200	JSpinner minimo=new JSpinner(new SpinnerNumberModel(original==null?1:original.minimo(),0,20,1));	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
201	JSpinner maximo=new JSpinner(new SpinnerNumberModel(original==null?1:original.maximo(),1,20,1));	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
202	JCheckBox repetir=new JCheckBox("Permitir repetir opciones",original!=null&&original.permiteRepetir());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
203	JCheckBox visible=new JCheckBox("Mostrar al cajero",original==null original.visible());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
204	JCheckBox personalizar=new JCheckBox("Permitir personalizar productos internos",original==null original .permitePersonalizar());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.

205	<code>if(!confirmarFormulario("Grupo de elección",new String[]{"Nombre","Mínimo","Máximo","","",""},nombre,minimo,maximo,repertir,visible,personalizar))return;</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
206	<code>try { int min=(Integer)minimo.getValue(),max=(Integer)maximo.getValue(); if(min>max)throw new IllegalArgumentException("El mínimo no puede superar al máximo."); validarTexto(nombre.getText(),"nombre"); dao.guardarGrupo(original==null?null:original.idGrupo(),p.idPresentacion(),nombre.getText().trim(),min,max,repertir.isSelected(),visible.isSelected(),personalizar.isSelected()); recargar(); }</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
207	<code>catch(Exception ex){error(ex);}</code>	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
208	<code>}</code>	Cierra el bloque abierto anteriormente.
209	<code>private void editarOpcion(OpcionMenu original) {</code>	Declara el método editarOpcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
210	<code>GrupoMenu g=grupos.getSelectedValue(); if(g==null){aviso("Selecciona un grupo.");return;}</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
211	<code>TextField nombre=new TextField(original==null?"":original.nombre());</code>	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
212	<code>TextField precio=new TextField(original==null?"0.00":original.incrementoPrecio().toString());</code>	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
213	<code>JCheckBox predeterminada=new JCheckBox("Seleccionada inicialmente",original!=null&&original.predeterminada());</code>	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
214	<code>if(!confirmarFormulario("Opción",new String[]{"Nombre","Incremento (Q)",""},nombre,precio,predeterminada))return;</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
215	<code>try { validarTexto(nombre.getText(),"nombre"); dao.guardarOpcion(original==null?null:original.idOpcion(),g.idGrupo(),nombre.getText().trim(),decimal(precio.getText()),predeterminada.isSelected()); recargar(); }</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
216	<code>catch(Exception ex){error(ex);}</code>	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
217	<code>}</code>	Cierra el bloque abierto anteriormente.
218	<code>private void agregarComponente() {</code>	Declara el método agregarComponente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
219	<code>OpcionMenu o=opciones.getSelectedValue(); if(o==null){aviso("Selecciona una opción.");return;}</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
220	<code>try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
221	<code>JComboBox<Item> productos=new JComboBox<>(); for(var e:dao.listarProductos().entrySet())productos.addItem(new Item(e.getKey(),e.getValue()));</code>	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
222	<code>JSpinner cantidad=new JSpinner(new SpinnerNumberModel(1,1,20,1));</code>	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
223	<code>if(!confirmarFormulario("Producto interno",new String[]{"Producto","Cantidad"},productos,cantidad))return;</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
224	<code>Item elegido=(Item)productos.getSelectedItem(); if(elegido==null)return;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
225	<code>dao.agregarComponente(o.idOpcion(),elegido.id(),(Integer)cantidad.getValue()); recargar();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
226	<code>}catch(Exception ex){error(ex);}</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
227	<code>}</code>	Cierra el bloque abierto anteriormente.
228	<code>private void quitarComponente() {</code>	Declara el método quitarComponente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
229	<code>OpcionMenu o=opciones.getSelectedValue(); ComponenteMenu c=componentes.getSelectedValue();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
230	<code>if(o==null c==null){aviso("Selecciona un producto interno.");return;}</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
231	<code>try{dao.quitarComponente(o.idOpcion(),c.idProducto());recargar();}catch(Exception ex){error(ex);}</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
232	<code>}</code>	Cierra el bloque abierto anteriormente.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	<code>private void editarIngrediente(IngredienteProducto original) {</code>	Declara el método editarIngrediente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
235	<code>try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
236	<code>JComboBox<Item> lista=new JComboBox<>(); for(var e:dao.listarIngredientes().entrySet())lista.addItem(new Item(e.getKey(),e.getValue()));</code>	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
237	<code>if(original!=null)for(int i=0;i<lista.getItemCount();i++)if(lista.getItemAt(i).id()==original.idIngrediente())lista.setSelectedIndex(i);</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
238	<code>TextField base=new TextField(original==null?"1":original.cantidadDefault().toString());</code>	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
239	<code>JCheckBox quitar=new JCheckBox("Se puede quitar",original!=null&&original.permiteQuitar());</code>	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
240	<code>JCheckBox extra=new JCheckBox("Se puede agregar extra",original!=null&&original.permiteExtra());</code>	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.

241	JTextField cantidadExtra=new JTextField(original==null?"1":original.cantidadExtra()).toPlainString());	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
242	JTextField precioExtra=new JTextField(original==null?"0.00":original.precioExtra()).toPlainString());	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
243	JSpinner maximo=new JSpinner(new SpinnerNumberModel(original==null?1:original.maxExtras(),1,20,1));	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
244	if(!confirmarFormulario("Ingrediente del producto",new String[]{"Ingrediente","Cantidad normal","", "","Cantidad por extra","Precio por extra (Q)","Máximo de extras"},lista,base,quitar,extra,cantidadExtra,precioEx tra,maximo))return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
245	Item elegido=(Item)lista.getSelectedItem();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
246	dao.guardarIngredienteProducto(original==null?null:original.idProductoIngrediente(),producto.getIdPr oducto(),elegido.id(),decimal(base.getText()),quitar.isSelected(),extra.isSelected(),decimal(cantidadExtra.getTe xt()),decimal(precioExtra.getText()),(Integer)maximo.getValue()); recargar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247	}catch(Exception ex){error(ex);}	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
248	}	Cierra el bloque abierto anteriormente.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	private void desactivarPresentacion(){PresentacionMenu p=presentaciones.getSelectedValue();if(p!=null)desact ivar("presentacion_menu","id_presentacion",p.idPresentacion());}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
251	private void desactivarGrupo(){GrupoMenu g=grupos.getSelectedValue();if(g!=null)desactivar("grupo_presentaci on","id_grupo",g.idGrupo());}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
252	private void desactivarOpcion(){OpcionMenu o=opciones.getSelectedValue();if(o!=null)desactivar("opcion_grupo ","id_opcion",o.idOpcion());}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	private void desactivar(String tabla,String campo,int id){try{dao.desactivar(tabla,campo,id);recargar();}cat ch(Exception ex){error(ex);}}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	private void desactivarIngrediente(){IngredienteProducto i=ingredientes.getSelectedValue();if(i==null)return ;try{dao.desactivarIngredienteProducto(i.idProductoIngrediente());recargar();}catch(Exception ex){error(ex);}}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	private boolean confirmarFormulario(String titulo,String[] etiquetas,JComponent...campos){	Declara el método confirmarFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
257	JPanel panel=new JPanel(new GridLayout(0,1,3,5));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
258	for(int i=0;i<campos.length;i++){if(i<etiquetas.length&&!etiquetas[i].isBlank())panel.add(new JLabel(eti quetas[i]));panel.add(campos[i]);}	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
259	return JOptionPane.showConfirmDialog(this,panel,titulo,JOptionPane.OK_CANCEL_OPTION,JOptionPane.PLAIN_ME SSAGE)==JOptionPane.OK_OPTION;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
260	}	Cierra el bloque abierto anteriormente.
261	private BigDecimal decimal(String texto){try{BigDecimal d=new BigDecimal(texto.trim().replace(',','.'));if(d .signum()<0 d.stripTrailingZeros().scale()>2)throw new NumberFormatException();return d.setScale(2);}catch(Exce ption ex){throw new IllegalArgumentException("Ingresa cantidades válidas con hasta dos decimales.");}}	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
262	private void validarTexto(String texto,String campo){if(texto==null texto.isBlank())throw new IllegalArgume ntException("Ingresa el "+campo+".");}	Crea una nueva instancia de IllegalArgumentException y la usa o guarda para continuar el proceso.
263	private void aviso(String texto){JOptionPane.showMessageDialog(this,texto,"Configuración",JOptionPane.INFORM ATION_MESSAGE);}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
264	private void error(Exception ex){JOptionPane.showMessageDialog(this,ex.getMessage(),"No se pudo guardar",JOp tionPane.ERROR_MESSAGE);}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	private record Item(int id,String nombre){@Override public String toString(){return nombre;}}	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
266	}	Cierra el bloque abierto anteriormente.

16.58 src/GUI_ADMINISTRADOR/DashboardPanel.form

Metadatos visuales de NetBeans para diseñar DashboardPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 410 | Archivo 58 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<NonVisualComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Component class="javax.swing.JLabel" name="labelTitulo">	Declara un componente o contenedor visual y su clase Java.
6	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
8		Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	</Property>	Cierra la sección XML abierta anteriormente.
10	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
11	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	</Property>	Cierra la sección XML abierta anteriormente.
13	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
14	<Property name="text" type="java.lang.String" value="Inicio de Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
15	</Properties>	Cierra la sección XML abierta anteriormente.
16	</Component>	Cierra la sección XML abierta anteriormente.
17	</NonVisualComponents>	Cierra la sección XML abierta anteriormente.
18	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
20	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	</Property>	Cierra la sección XML abierta anteriormente.
22	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
23	<Dimension value="[1580, 932]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
24	</Property>	Cierra la sección XML abierta anteriormente.
25	</Properties>	Cierra la sección XML abierta anteriormente.
26	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
27	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
31	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
34	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
35	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
36	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84, -19, 0, 5, 115, 114, 0, 18, 106, 97, 118, 97, 46, 97, 119, 116, 46, 68, 105, 109, 101, 110, 115, 105, 111, 110, 65, -114, -39, -41, -84, 95, 68, 20, 2, 0, 2, 73, 0, 6, 104, 101, 105, 103, 104, 116, 73, 0, 5, 119, 105, 100, 116, 104, 120, 112, 0, 0, 3, -44, 0, 0, 6, 44"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
37	</AuxValues>	Cierra la sección XML abierta anteriormente.

38		Línea en blanco para organizar el XML del formulario.
39	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
40	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
41	</Layout>	Cierra la sección XML abierta anteriormente.
42	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
43	<Container class="Componentes.PanelFlotante" name="panelFlotante1">	Declara un componente o contenedor visual y su clase Java.
44	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
45	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
46	<AbsoluteConstraints x="60" y="30" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
47	</Constraint>	Cierra la sección XML abierta anteriormente.
48	</Constraints>	Cierra la sección XML abierta anteriormente.
49		Línea en blanco para organizar el XML del formulario.
50	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
51	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	</Layout>	Cierra la sección XML abierta anteriormente.
53	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
54	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
55	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
56	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
57	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
58	</Constraint>	Cierra la sección XML abierta anteriormente.
59	</Constraints>	Cierra la sección XML abierta anteriormente.
60		Línea en blanco para organizar el XML del formulario.
61	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
62	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
63	</Layout>	Cierra la sección XML abierta anteriormente.
64	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
65	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
66	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	<Image iconType="3" name="/Imagenes/quetzal_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
69	</Property>	Cierra la sección XML abierta anteriormente.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
72	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
73	<AbsoluteConstraints x="8" y="13" width="65" height="65"/>	Define la posición y el tamaño del componente dentro del diseño.
74	</Constraint>	Cierra la sección XML abierta anteriormente.
75	</Constraints>	Cierra la sección XML abierta anteriormente.
76	</Component>	Cierra la sección XML abierta anteriormente.
77	</SubComponents>	Cierra la sección XML abierta anteriormente.
78	</Container>	Cierra la sección XML abierta anteriormente.
79	<Component class="javax.swing.JLabel" name="labelTitulo3">	Declara un componente o contenedor visual y su clase Java.

80	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
81	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
82		Metadato de NetBeans que conserva el estado editable del diseñador visual.
83	</Property>	Cierra la sección XML abierta anteriormente.
84	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
85	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
86	</Property>	Cierra la sección XML abierta anteriormente.
87	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
88	<Property name="text" type="java.lang.String" value="Ventas del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
89	</Properties>	Cierra la sección XML abierta anteriormente.
90	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
91	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
92	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
93	</Constraint>	Cierra la sección XML abierta anteriormente.
94	</Constraints>	Cierra la sección XML abierta anteriormente.
95	</Component>	Cierra la sección XML abierta anteriormente.
96	<Component class="javax.swing.JLabel" name="labelTitulo2">	Declara un componente o contenedor visual y su clase Java.
97	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
98	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
99		Metadato de NetBeans que conserva el estado editable del diseñador visual.
100	</Property>	Cierra la sección XML abierta anteriormente.
101	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
102	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
105	<Property name="text" type="java.lang.String" value="Ganancias del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
106	</Properties>	Cierra la sección XML abierta anteriormente.
107	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
108	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
109	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
110	</Constraint>	Cierra la sección XML abierta anteriormente.
111	</Constraints>	Cierra la sección XML abierta anteriormente.
112	</Component>	Cierra la sección XML abierta anteriormente.
113	<Component class="javax.swing.JLabel" name="labelTitulo5">	Declara un componente o contenedor visual y su clase Java.
114	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
115	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
116		Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	</Property>	Cierra la sección XML abierta anteriormente.
118	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
119	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
120	</Property>	Cierra la sección XML abierta anteriormente.

121	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
122	<Property name="text" type="java.lang.String" value="Q100.00"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
123	</Properties>	Cierra la sección XML abierta anteriormente.
124	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
125	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
126	<AbsoluteConstraints x="140" y="40" width="200" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
127	</Constraint>	Cierra la sección XML abierta anteriormente.
128	</Constraints>	Cierra la sección XML abierta anteriormente.
129	</Component>	Cierra la sección XML abierta anteriormente.
130	</SubComponents>	Cierra la sección XML abierta anteriormente.
131	</Container>	Cierra la sección XML abierta anteriormente.
132	<Container class="Componentes.PanelFlotante" name="panelFlotante2">	Declara un componente o contenedor visual y su clase Java.
133	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
134	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
135	<AbsoluteConstraints x="430" y="30" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
136	</Constraint>	Cierra la sección XML abierta anteriormente.
137	</Constraints>	Cierra la sección XML abierta anteriormente.
138		Línea en blanco para organizar el XML del formulario.
139	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
140	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
141	</Layout>	Cierra la sección XML abierta anteriormente.
142	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
143	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
144	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
145	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
146	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
147	</Property>	Cierra la sección XML abierta anteriormente.
148	</Properties>	Cierra la sección XML abierta anteriormente.
149	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
150	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
151	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
152	</Constraint>	Cierra la sección XML abierta anteriormente.
153	</Constraints>	Cierra la sección XML abierta anteriormente.
154		Línea en blanco para organizar el XML del formulario.
155	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
156	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
157	</Layout>	Cierra la sección XML abierta anteriormente.
158	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
159	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
160	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
161	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

162	<Image iconType="3" name="/Imagenes/bolsa_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
163	</Property>	Cierra la sección XML abierta anteriormente.
164	</Properties>	Cierra la sección XML abierta anteriormente.
165	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
166	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
167	<AbsoluteConstraints x="11" y="13" width="58" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
168	</Constraint>	Cierra la sección XML abierta anteriormente.
169	</Constraints>	Cierra la sección XML abierta anteriormente.
170	</Component>	Cierra la sección XML abierta anteriormente.
171	</SubComponents>	Cierra la sección XML abierta anteriormente.
172	</Container>	Cierra la sección XML abierta anteriormente.
173	<Component class="javax.swing.JLabel" name="labelTitulo7">	Declara un componente o contenedor visual y su clase Java.
174	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
175	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
176		Metadato de NetBeans que conserva el estado editable del diseñador visual.
177	</Property>	Cierra la sección XML abierta anteriormente.
178	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
179	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
180	</Property>	Cierra la sección XML abierta anteriormente.
181	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
182	<Property name="text" type="java.lang.String" value="Pedidos del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
183	</Properties>	Cierra la sección XML abierta anteriormente.
184	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
185	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
186	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
187	</Constraint>	Cierra la sección XML abierta anteriormente.
188	</Constraints>	Cierra la sección XML abierta anteriormente.
189	</Component>	Cierra la sección XML abierta anteriormente.
190	<Component class="javax.swing.JLabel" name="labelTitulo8">	Declara un componente o contenedor visual y su clase Java.
191	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
192	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
193		Metadato de NetBeans que conserva el estado editable del diseñador visual.
194	</Property>	Cierra la sección XML abierta anteriormente.
195	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
196	<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
197	</Property>	Cierra la sección XML abierta anteriormente.
198	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
199	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
200	</Properties>	Cierra la sección XML abierta anteriormente.
201	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
202	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

203	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
204	</Constraint>	Cierra la sección XML abierta anteriormente.
205	</Constraints>	Cierra la sección XML abierta anteriormente.
206	</Component>	Cierra la sección XML abierta anteriormente.
207	<Component class="javax.swing.JLabel" name="labelTitulo9">	Declara un componente o contenedor visual y su clase Java.
208	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
209	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
210		Metadato de NetBeans que conserva el estado editable del diseñador visual.
211	</Property>	Cierra la sección XML abierta anteriormente.
212	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
213	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
214	</Property>	Cierra la sección XML abierta anteriormente.
215	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
216	<Property name="text" type="java.lang.String" value="Pedidos realizados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
217	</Properties>	Cierra la sección XML abierta anteriormente.
218	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
219	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
220	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
221	</Constraint>	Cierra la sección XML abierta anteriormente.
222	</Constraints>	Cierra la sección XML abierta anteriormente.
223	</Component>	Cierra la sección XML abierta anteriormente.
224	</SubComponents>	Cierra la sección XML abierta anteriormente.
225	</Container>	Cierra la sección XML abierta anteriormente.
226	<Container class="Componentes.PanelFlotante" name="panelFlotante4">	Declara un componente o contenedor visual y su clase Java.
227	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
228	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
229	<AbsoluteConstraints x="800" y="30" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
230	</Constraint>	Cierra la sección XML abierta anteriormente.
231	</Constraints>	Cierra la sección XML abierta anteriormente.
232		Línea en blanco para organizar el XML del formulario.
233	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
234	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
235	</Layout>	Cierra la sección XML abierta anteriormente.
236	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
237	<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
238	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
239	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
240	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
241	</Constraint>	Cierra la sección XML abierta anteriormente.
242	</Constraints>	Cierra la sección XML abierta anteriormente.
243		Línea en blanco para organizar el XML del formulario.
244	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.

245	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
246	</Layout>	Cierra la sección XML abierta anteriormente.
247	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
248	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
249	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
250	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251	<Image iconType="3" name="/Imagenes/ticket_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
252	</Property>	Cierra la sección XML abierta anteriormente.
253	</Properties>	Cierra la sección XML abierta anteriormente.
254	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
255	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
256	<AbsoluteConstraints x="11" y="15" width="60" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
257	</Constraint>	Cierra la sección XML abierta anteriormente.
258	</Constraints>	Cierra la sección XML abierta anteriormente.
259	</Component>	Cierra la sección XML abierta anteriormente.
260	</SubComponents>	Cierra la sección XML abierta anteriormente.
261	</Container>	Cierra la sección XML abierta anteriormente.
262	<Component class="javax.swing.JLabel" name="labelTitulo13">	Declara un componente o contenedor visual y su clase Java.
263	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
264	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
265		Metadato de NetBeans que conserva el estado editable del diseñador visual.
266	</Property>	Cierra la sección XML abierta anteriormente.
267	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
268	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
269	</Property>	Cierra la sección XML abierta anteriormente.
270	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
271	<Property name="text" type="java.lang.String" value="Ticket Promedio"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
272	</Properties>	Cierra la sección XML abierta anteriormente.
273	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
274	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
275	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
276	</Constraint>	Cierra la sección XML abierta anteriormente.
277	</Constraints>	Cierra la sección XML abierta anteriormente.
278	</Component>	Cierra la sección XML abierta anteriormente.
279	<Component class="javax.swing.JLabel" name="labelTitulo15">	Declara un componente o contenedor visual y su clase Java.
280	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
281	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
282		Metadato de NetBeans que conserva el estado editable del diseñador visual.
283	</Property>	Cierra la sección XML abierta anteriormente.
284	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
285	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

286	</Property>	Cierra la sección XML abierta anteriormente.
287	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
288	<Property name="text" type="java.lang.String" value="Disponible para uso"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
289	</Properties>	Cierra la sección XML abierta anteriormente.
290	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
291	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
292	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
293	</Constraint>	Cierra la sección XML abierta anteriormente.
294	</Constraints>	Cierra la sección XML abierta anteriormente.
295	</Component>	Cierra la sección XML abierta anteriormente.
296	<Component class="javax.swing.JLabel" name="labelTitulo4">	Declara un componente o contenedor visual y su clase Java.
297	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
298	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
299		Metadato de NetBeans que conserva el estado editable del diseñador visual.
300	</Property>	Cierra la sección XML abierta anteriormente.
301	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
302	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
303	</Property>	Cierra la sección XML abierta anteriormente.
304	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
305	<Property name="text" type="java.lang.String" value="Q100.00"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
306	</Properties>	Cierra la sección XML abierta anteriormente.
307	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
308	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
309	<AbsoluteConstraints x="140" y="40" width="200" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
310	</Constraint>	Cierra la sección XML abierta anteriormente.
311	</Constraints>	Cierra la sección XML abierta anteriormente.
312	</Component>	Cierra la sección XML abierta anteriormente.
313	</SubComponents>	Cierra la sección XML abierta anteriormente.
314	</Container>	Cierra la sección XML abierta anteriormente.
315	<Container class="Componentes.PanelFlotante" name="panelFlotante3">	Declara un componente o contenedor visual y su clase Java.
316	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
317	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
318	<AbsoluteConstraints x="1170" y="30" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
319	</Constraint>	Cierra la sección XML abierta anteriormente.
320	</Constraints>	Cierra la sección XML abierta anteriormente.
321		Línea en blanco para organizar el XML del formulario.
322	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
323	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
324	</Layout>	Cierra la sección XML abierta anteriormente.
325	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
326	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
327	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

328	>	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
329		<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
330		</Property>	Cierra la sección XML abierta anteriormente.
331		</Properties>	Cierra la sección XML abierta anteriormente.
332		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
333		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
334		<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
335		</Constraint>	Cierra la sección XML abierta anteriormente.
336		</Constraints>	Cierra la sección XML abierta anteriormente.
337			Línea en blanco para organizar el XML del formulario.
338		<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
339		<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
340		</Layout>	Cierra la sección XML abierta anteriormente.
341		<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
342		<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
343		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
344	tor">	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
345		<Image iconType="3" name="/Imágenes/caja_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
346		</Property>	Cierra la sección XML abierta anteriormente.
347		</Properties>	Cierra la sección XML abierta anteriormente.
348		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
349		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
350		<AbsoluteConstraints x="11" y="15" width="60" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
351		</Constraint>	Cierra la sección XML abierta anteriormente.
352		</Constraints>	Cierra la sección XML abierta anteriormente.
353		</Component>	Cierra la sección XML abierta anteriormente.
354		</SubComponents>	Cierra la sección XML abierta anteriormente.
355		</Container>	Cierra la sección XML abierta anteriormente.
356		<Component class="javax.swing.JLabel" name="labelTitulo10">	Declara un componente o contenedor visual y su clase Java.
357		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
358		<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
359			Metadato de NetBeans que conserva el estado editable del diseñador visual.
360		</Property>	Cierra la sección XML abierta anteriormente.
361	>	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
362		<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
363		</Property>	Cierra la sección XML abierta anteriormente.
364		<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
365		<Property name="text" type="java.lang.String" value="Productos con stock bajo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
366		</Properties>	Cierra la sección XML abierta anteriormente.
367		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

368	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
369	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
370	</Constraint>	Cierra la sección XML abierta anteriormente.
371	</Constraints>	Cierra la sección XML abierta anteriormente.
372	</Component>	Cierra la sección XML abierta anteriormente.
373	<Component class="javax.swing.JLabel" name="labelTitulo11">	Declara un componente o contenedor visual y su clase Java.
374	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
375	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
376		Metadato de NetBeans que conserva el estado editable del diseñador visual.
377	</Property>	Cierra la sección XML abierta anteriormente.
378	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
379	<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
380	</Property>	Cierra la sección XML abierta anteriormente.
381	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
382	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
383	</Properties>	Cierra la sección XML abierta anteriormente.
384	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
385	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
386	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
387	</Constraint>	Cierra la sección XML abierta anteriormente.
388	</Constraints>	Cierra la sección XML abierta anteriormente.
389	</Component>	Cierra la sección XML abierta anteriormente.
390	<Component class="javax.swing.JLabel" name="labelTitulo12">	Declara un componente o contenedor visual y su clase Java.
391	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
392	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
393		Metadato de NetBeans que conserva el estado editable del diseñador visual.
394	</Property>	Cierra la sección XML abierta anteriormente.
395	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
396	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
397	</Property>	Cierra la sección XML abierta anteriormente.
398	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
399	<Property name="text" type="java.lang.String" value="Requieren atención"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
400	</Properties>	Cierra la sección XML abierta anteriormente.
401	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
402	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
403	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
404	</Constraint>	Cierra la sección XML abierta anteriormente.
405	</Constraints>	Cierra la sección XML abierta anteriormente.
406	</Component>	Cierra la sección XML abierta anteriormente.
407	</SubComponents>	Cierra la sección XML abierta anteriormente.
408	</Container>	Cierra la sección XML abierta anteriormente.

409	</SubComponents>	Cierra la sección XML abierta anteriormente.
410	</Form>	Cierra la sección XML abierta anteriormente.

16.59 src/GUI_ADMINISTRADOR/DashboardPanel.java

Interfaz del módulo administrador: DashboardPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 211 | Archivo 59 de 157

Clases/tipos detectados: DashboardPanel

Línea	Constructor o método detectado	
16	public DashboardPanel() {	
27	private void initComponents() {	
1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JPanel.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author Humberto Alvarado	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class DashboardPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	* Creates new form DashboardPanel	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	public DashboardPanel() {	Constructor de DashboardPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
17	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18	}	Cierra el bloque abierto anteriormente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
26	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:inicializarComponentes	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
27	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
30	panelFlotante1 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
31	panelCircular2 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
32	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.

33	labelTitulo3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
34	labelTitulo2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
35	labelTitulo5 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
36	panelFlotante2 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
37	panelCircular3 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
38	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
39	labelTitulo7 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
40	labelTitulo8 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
41	labelTitulo9 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
42	panelFlotante4 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
43	panelCircular5 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
44	labelEscalable5 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
45	labelTitulo13 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
46	labelTitulo15 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
47	labelTitulo4 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
48	panelFlotante3 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
49	panelCircular4 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
50	labelEscalable4 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
51	labelTitulo10 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
52	labelTitulo11 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
53	labelTitulo12 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.
55	labelTitulo.setFont(new java.awt.Font("Dialog", 1, 59)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
56	labelTitulo.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
57	labelTitulo.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
58	labelTitulo.setText("Inicio de Administrador");	Cambia el texto visible de un componente de la interfaz.
59		Línea en blanco: separa visualmente bloques para que el código sea más legible.
60	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
61	setPreferredSize(new java.awt.Dimension(1580, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
62	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	panelFlotante1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/quetzal_ícono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
69	panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(8, 13, 65, 65));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	panelFlotante1.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	labelTitulo3.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
74	labelTitulo3.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
75	labelTitulo3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
76	labelTitulo3.setText("Ventas del día");	Cambia el texto visible de un componente de la interfaz.

77	panelFlotante1.add(labelTitulo3, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	labelTitulo2.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
80	labelTitulo2.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
81	labelTitulo2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
82	labelTitulo2.setText("Ganancias del día");	Cambia el texto visible de un componente de la interfaz.
83	panelFlotante1.add(labelTitulo2, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	labelTitulo5.setFont(new java.awt.Font("Dialog", 1, 35)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
86	labelTitulo5.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
87	labelTitulo5.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
88	labelTitulo5.setText("Q100.00");	Cambia el texto visible de un componente de la interfaz.
89	panelFlotante1.add(labelTitulo5, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 200, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	add(panelFlotante1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 30, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	panelFlotante2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
96	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/bolsa_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
99	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(11, 13, 58, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	panelFlotante2.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	labelTitulo7.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
104	labelTitulo7.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
105	labelTitulo7.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
106	labelTitulo7.setText("Pedidos del día");	Cambia el texto visible de un componente de la interfaz.
107	panelFlotante2.add(labelTitulo7, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	labelTitulo8.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
110	labelTitulo8.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
111	labelTitulo8.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
112	labelTitulo8.setText("48");	Cambia el texto visible de un componente de la interfaz.
113	panelFlotante2.add(labelTitulo8, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	labelTitulo9.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
116	labelTitulo9.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
117	labelTitulo9.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
118	labelTitulo9.setText("Pedidos realizados");	Cambia el texto visible de un componente de la interfaz.
119	panelFlotante2.add(labelTitulo9, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.

121	<code>add(panelFlotante2, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 30, 360, 160));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	<code>panelFlotante4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	<code>panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	<code>labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/ticket_icono.png"))); // NOI18N</code>	Crea una nueva instancia de <code>javax.swing.ImageIcon</code> y la usa o guarda para continuar el proceso.
128	<code>panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(11, 15, 60, 60));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
130	<code>panelFlotante4.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	<code>labelTitulo13.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
133	<code>labelTitulo13.setForeground(new java.awt.Color(92, 103, 124));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
134	<code>labelTitulo13.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);</code>	Configura una propiedad mediante <code>setHorizontalAlignment</code> en el objeto indicado.
135	<code>labelTitulo13.setText("Ticket Promedio");</code>	Cambia el texto visible de un componente de la interfaz.
136	<code>panelFlotante4.add(labelTitulo13, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	<code>labelTitulo15.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
139	<code>labelTitulo15.setForeground(new java.awt.Color(127, 137, 154));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
140	<code>labelTitulo15.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);</code>	Configura una propiedad mediante <code>setHorizontalAlignment</code> en el objeto indicado.
141	<code>labelTitulo15.setText("Disponible para uso");</code>	Cambia el texto visible de un componente de la interfaz.
142	<code>panelFlotante4.add(labelTitulo15, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
144	<code>labelTitulo4.setFont(new java.awt.Font("Dialog", 1, 35)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
145	<code>labelTitulo4.setForeground(new java.awt.Color(13, 17, 23));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
146	<code>labelTitulo4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);</code>	Configura una propiedad mediante <code>setHorizontalAlignment</code> en el objeto indicado.
147	<code>labelTitulo4.setText("Q100.00");</code>	Cambia el texto visible de un componente de la interfaz.
148	<code>panelFlotante4.add(labelTitulo4, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 200, 76));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	<code>add(panelFlotante4, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 30, 360, 160));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	<code>panelFlotante3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154	<code>panelCircular4.setColorFondo(new java.awt.Color(252, 233, 233));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
155	<code>panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	<code>labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/caja_icono.png"))); // NOI18N</code>	Crea una nueva instancia de <code>javax.swing.ImageIcon</code> y la usa o guarda para continuar el proceso.
158	<code>panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(11, 15, 60, 60));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
159		Línea en blanco: separa visualmente bloques para que el código sea más legible.
160	<code>panelFlotante3.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	<code>labelTitulo10.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
163	<code>labelTitulo10.setForeground(new java.awt.Color(92, 103, 124));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.

164	labelTitulo10.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
165	labelTitulo10.setText("Productos con stock bajo");	Cambia el texto visible de un componente de la interfaz.
166	panelFlotante3.add(labelTitulo10, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	labelTitulo11.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
169	labelTitulo11.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
170	labelTitulo11.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
171	labelTitulo11.setText("48");	Cambia el texto visible de un componente de la interfaz.
172	panelFlotante3.add(labelTitulo11, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	labelTitulo12.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
175	labelTitulo12.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
176	labelTitulo12.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
177	labelTitulo12.setText("Requieren atención");	Cambia el texto visible de un componente de la interfaz.
178	panelFlotante3.add(labelTitulo12, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
179		Línea en blanco: separa visualmente bloques para que el código sea más legible.
180	add(panelFlotante3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 30, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
181	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
185	private javax.swing.JLabel labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
186	private javax.swing.JLabel labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
187	private javax.swing.JLabel labelEscalable4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
188	private javax.swing.JLabel labelEscalable5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
189	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
190	private javax.swing.JLabel labelTitulo10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
191	private javax.swing.JLabel labelTitulo11;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
192	private javax.swing.JLabel labelTitulo12;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
193	private javax.swing.JLabel labelTitulo13;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
194	private javax.swing.JLabel labelTitulo15;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
195	private javax.swing.JLabel labelTitulo2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
196	private javax.swing.JLabel labelTitulo3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	private javax.swing.JLabel labelTitulo4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
198	private javax.swing.JLabel labelTitulo5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
199	private javax.swing.JLabel labelTitulo7;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
200	private javax.swing.JLabel labelTitulo8;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
201	private javax.swing.JLabel labelTitulo9;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
202	private javax.swing.JPanel panelCircular2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
203	private javax.swing.JPanel panelCircular3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
204	private javax.swing.JPanel panelCircular4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
205	private javax.swing.JPanel panelCircular5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
206	private javax.swing.JPanel panelFlotante1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
207	private javax.swing.JPanel panelFlotante2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

208	<code>private Componentes.PanelFlotante panelFlotante3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
209	<code>private Componentes.PanelFlotante panelFlotante4;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
210	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
211	<code>}</code>	Cierra el bloque abierto anteriormente.

16.60 src/GUI_ADMINISTRADOR/GestionInventarioPanel.form

Metadatos visuales de NetBeans para diseñar GestionInventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 813 | Archivo 60 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<Dimension value="[1530, 932]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</Property>	Cierra la sección XML abierta anteriormente.
11	</Properties>	Cierra la sección XML abierta anteriormente.
12	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	</AuxValues>	Cierra la sección XML abierta anteriormente.
23		Línea en blanco para organizar el XML del formulario.
24	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
25	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
26	</Layout>	Cierra la sección XML abierta anteriormente.
27	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Component class="javax.swing.JLabel" name="labelTitulo3">	Declara un componente o contenedor visual y su clase Java.
29	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
31		Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	</Property>	Cierra la sección XML abierta anteriormente.
33	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
34	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
35	</Property>	Cierra la sección XML abierta anteriormente.
36	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
37	<Property name="text" type="java.lang.String" value="Gestión de Inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
38	</Properties>	Cierra la sección XML abierta anteriormente.

39	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
40	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
41	<AbsoluteConstraints x="60" y="16" width="650" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
42	</Constraint>	Cierra la sección XML abierta anteriormente.
43	</Constraints>	Cierra la sección XML abierta anteriormente.
44	</Component>	Cierra la sección XML abierta anteriormente.
45	<Component class="javax.swing.JLabel" name="labelTitulo2">	Declara un componente o contenedor visual y su clase Java.
46	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
47	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
48		Metadato de NetBeans que conserva el estado editable del diseñador visual.
49	</Property>	Cierra la sección XML abierta anteriormente.
50	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
51	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
52	</Property>	Cierra la sección XML abierta anteriormente.
53	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
54	<Property name="text" type="java.lang.String" value="Administra existencias, entradas y salidas"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
55	</Properties>	Cierra la sección XML abierta anteriormente.
56	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
57	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
58	<AbsoluteConstraints x="60" y="76" width="620" height="36"/>	Define la posición y el tamaño del componente dentro del diseño.
59	</Constraint>	Cierra la sección XML abierta anteriormente.
60	</Constraints>	Cierra la sección XML abierta anteriormente.
61	</Component>	Cierra la sección XML abierta anteriormente.
62	<Component class="Componentes.BotonDerretido" name="botonAgregarProducto">	Declara un componente o contenedor visual y su clase Java.
63	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
65	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
66	</Property>	Cierra la sección XML abierta anteriormente.
67	<Property name="text" type="java.lang.String" value="Registrar entrada"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	</Properties>	Cierra la sección XML abierta anteriormente.
69	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
70	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
71	<AbsoluteConstraints x="780" y="42" width="175" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
72	</Constraint>	Cierra la sección XML abierta anteriormente.
73	</Constraints>	Cierra la sección XML abierta anteriormente.
74	</Component>	Cierra la sección XML abierta anteriormente.
75	<Component class="Componentes.BotonRedondeado" name="botonRegistrarSalida">	Declara un componente o contenedor visual y su clase Java.
76	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
77	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
78	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
79	</Property>	Cierra la sección XML abierta anteriormente.
80	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

81	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
82	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
83	</Property>	Cierra la sección XML abierta anteriormente.
84	<Property name="text" type="java.lang.String" value="Registrar salida"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
85	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
87	</Property>	Cierra la sección XML abierta anteriormente.
88	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
89	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
90	</Property>	Cierra la sección XML abierta anteriormente.
91	<Property name="grosorBorde" type="float" value="1.3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
92	<Property name="radio" type="int" value="13"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
93	</Properties>	Cierra la sección XML abierta anteriormente.
94	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
95	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
96	<AbsoluteConstraints x="965" y="45" width="175" height="54"/>	Define la posición y el tamaño del componente dentro del diseño.
97	</Constraint>	Cierra la sección XML abierta anteriormente.
98	</Constraints>	Cierra la sección XML abierta anteriormente.
99	</Component>	Cierra la sección XML abierta anteriormente.
100	<Component class="Componentes.BotonRedondeado" name="botonMovimientos">	Declara un componente o contenedor visual y su clase Java.
101	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
102	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
103	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
104	</Property>	Cierra la sección XML abierta anteriormente.
105	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
106	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
107	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
108	</Property>	Cierra la sección XML abierta anteriormente.
109	<Property name="text" type="java.lang.String" value="Ajustar stock"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
110	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
111	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
112	</Property>	Cierra la sección XML abierta anteriormente.
113	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
114	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
115	</Property>	Cierra la sección XML abierta anteriormente.
116	<Property name="grosorBorde" type="float" value="1.3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
117	<Property name="radio" type="int" value="13"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
118	</Properties>	Cierra la sección XML abierta anteriormente.
119	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
120	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
121	<AbsoluteConstraints x="1150" y="45" width="175" height="54"/>	Define la posición y el tamaño del componente dentro del diseño.
122	</Constraint>	Cierra la sección XML abierta anteriormente.
123	</Constraints>	Cierra la sección XML abierta anteriormente.

124	</Component>	Cierra la sección XML abierta anteriormente.
125	<Container class="Componentes.PanelFlotante" name="panelFlotante1">	Declara un componente o contenedor visual y su clase Java.
126	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
127	<Property name="visible" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
128	</Properties>	Cierra la sección XML abierta anteriormente.
129	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
130	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
131	<AbsoluteConstraints x="60" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
132	</Constraint>	Cierra la sección XML abierta anteriormente.
133	</Constraints>	Cierra la sección XML abierta anteriormente.
134		Línea en blanco para organizar el XML del formulario.
135	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
136	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
137	</Layout>	Cierra la sección XML abierta anteriormente.
138	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
139	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
140	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
141	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
142	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
143	</Constraint>	Cierra la sección XML abierta anteriormente.
144	</Constraints>	Cierra la sección XML abierta anteriormente.
145		Línea en blanco para organizar el XML del formulario.
146	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
147	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
148	</Layout>	Cierra la sección XML abierta anteriormente.
149	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
150	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
151	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
152	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
153	<Image iconType="3" name="/Imagenes/lista_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
154	</Property>	Cierra la sección XML abierta anteriormente.
155	</Properties>	Cierra la sección XML abierta anteriormente.
156	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
157	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
158	<AbsoluteConstraints x="15" y="20" width="50" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
159	</Constraint>	Cierra la sección XML abierta anteriormente.
160	</Constraints>	Cierra la sección XML abierta anteriormente.
161	</Component>	Cierra la sección XML abierta anteriormente.
162	</SubComponents>	Cierra la sección XML abierta anteriormente.
163	</Container>	Cierra la sección XML abierta anteriormente.
164	<Component class="javax.swing.JLabel" name="labelTitulo4">	Declara un componente o contenedor visual y su clase Java.
165	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

166	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
167		Metadato de NetBeans que conserva el estado editable del diseñador visual.
168	</Property>	Cierra la sección XML abierta anteriormente.
169	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
170	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
171	</Property>	Cierra la sección XML abierta anteriormente.
172	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
173	<Property name="text" type="java.lang.String" value="Productos totales"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
174	</Properties>	Cierra la sección XML abierta anteriormente.
175	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
176	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
177	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
178	</Constraint>	Cierra la sección XML abierta anteriormente.
179	</Constraints>	Cierra la sección XML abierta anteriormente.
180	</Component>	Cierra la sección XML abierta anteriormente.
181	<Component class="javax.swing.JLabel" name="labelTotalProductos">	Declara un componente o contenedor visual y su clase Java.
182	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
183	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
184		Metadato de NetBeans que conserva el estado editable del diseñador visual.
185	</Property>	Cierra la sección XML abierta anteriormente.
186	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
187	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
188	</Property>	Cierra la sección XML abierta anteriormente.
189	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
190	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
191	</Properties>	Cierra la sección XML abierta anteriormente.
192	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
193	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
194	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
195	</Constraint>	Cierra la sección XML abierta anteriormente.
196	</Constraints>	Cierra la sección XML abierta anteriormente.
197	</Component>	Cierra la sección XML abierta anteriormente.
198	<Component class="javax.swing.JLabel" name="labelTitulo6">	Declara un componente o contenedor visual y su clase Java.
199	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
200	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
201		Metadato de NetBeans que conserva el estado editable del diseñador visual.
202	</Property>	Cierra la sección XML abierta anteriormente.
203	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
204	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
205	</Property>	Cierra la sección XML abierta anteriormente.
206	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

207	<Property name="text" type="java.lang.String" value="En el inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
208	</Properties>	Cierra la sección XML abierta anteriormente.
209	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
210	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
211	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
212	</Constraint>	Cierra la sección XML abierta anteriormente.
213	</Constraints>	Cierra la sección XML abierta anteriormente.
214	</Component>	Cierra la sección XML abierta anteriormente.
215	</SubComponents>	Cierra la sección XML abierta anteriormente.
216	</Container>	Cierra la sección XML abierta anteriormente.
217	<Container class="Componentes.PanelFlotante" name="panelFlotante2">	Declara un componente o contenedor visual y su clase Java.
218	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
219	<Property name="visible" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
220	</Properties>	Cierra la sección XML abierta anteriormente.
221	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
222	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
223	<AbsoluteConstraints x="430" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
224	</Constraint>	Cierra la sección XML abierta anteriormente.
225	</Constraints>	Cierra la sección XML abierta anteriormente.
226		Línea en blanco para organizar el XML del formulario.
227	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
228	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
229	</Layout>	Cierra la sección XML abierta anteriormente.
230	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
231	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
232	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
233	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
234	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
235	</Property>	Cierra la sección XML abierta anteriormente.
236	</Properties>	Cierra la sección XML abierta anteriormente.
237	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
238	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
239	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
240	</Constraint>	Cierra la sección XML abierta anteriormente.
241	</Constraints>	Cierra la sección XML abierta anteriormente.
242		Línea en blanco para organizar el XML del formulario.
243	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
244	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
245	</Layout>	Cierra la sección XML abierta anteriormente.
246	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
247	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
248	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

249	tor">	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
250		<Image iconType="3" name="/Imagenes/advertencia_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
251		</Property>	Cierra la sección XML abierta anteriormente.
252		</Properties>	Cierra la sección XML abierta anteriormente.
253		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
254		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
255		<AbsoluteConstraints x="19" y="22" width="43" height="43"/>	Define la posición y el tamaño del componente dentro del diseño.
256		</Constraint>	Cierra la sección XML abierta anteriormente.
257		</Constraints>	Cierra la sección XML abierta anteriormente.
258		</Component>	Cierra la sección XML abierta anteriormente.
259		</SubComponents>	Cierra la sección XML abierta anteriormente.
260		</Container>	Cierra la sección XML abierta anteriormente.
261		<Component class="javax.swing.JLabel" name="labelTitulo7">	Declara un componente o contenedor visual y su clase Java.
262		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
263		<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
264			Metadato de NetBeans que conserva el estado editable del diseñador visual.
265		</Property>	Cierra la sección XML abierta anteriormente.
266		<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
267		<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
268		</Property>	Cierra la sección XML abierta anteriormente.
269		<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
270		<Property name="text" type="java.lang.String" value="Stock bajo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
271		</Properties>	Cierra la sección XML abierta anteriormente.
272		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
273		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
274		<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
275		</Constraint>	Cierra la sección XML abierta anteriormente.
276		</Constraints>	Cierra la sección XML abierta anteriormente.
277		</Component>	Cierra la sección XML abierta anteriormente.
278		<Component class="javax.swing.JLabel" name="labelStockBajo">	Declara un componente o contenedor visual y su clase Java.
279		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
280		<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
281			Metadato de NetBeans que conserva el estado editable del diseñador visual.
282		</Property>	Cierra la sección XML abierta anteriormente.
283		<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
284		<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
285		</Property>	Cierra la sección XML abierta anteriormente.
286		<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
287		<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
288		</Properties>	Cierra la sección XML abierta anteriormente.
289		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

290	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
291	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
292	</Constraint>	Cierra la sección XML abierta anteriormente.
293	</Constraints>	Cierra la sección XML abierta anteriormente.
294	</Component>	Cierra la sección XML abierta anteriormente.
295	<Component class="javax.swing.JLabel" name="labelTitulo9">	Declara un componente o contenedor visual y su clase Java.
296	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
297	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
298		Metadato de NetBeans que conserva el estado editable del diseñador visual.
299	</Property>	Cierra la sección XML abierta anteriormente.
300	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
301	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
302	</Property>	Cierra la sección XML abierta anteriormente.
303	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
304	<Property name="text" type="java.lang.String" value="Requieren reposición"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
305	</Properties>	Cierra la sección XML abierta anteriormente.
306	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
307	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
308	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
309	</Constraint>	Cierra la sección XML abierta anteriormente.
310	</Constraints>	Cierra la sección XML abierta anteriormente.
311	</Component>	Cierra la sección XML abierta anteriormente.
312	</SubComponents>	Cierra la sección XML abierta anteriormente.
313	</Container>	Cierra la sección XML abierta anteriormente.
314	<Container class="Componentes.PanelFlotante" name="panelFlotante4">	Declara un componente o contenedor visual y su clase Java.
315	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
316	<Property name="visible" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
317	</Properties>	Cierra la sección XML abierta anteriormente.
318	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
319	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
320	<AbsoluteConstraints x="800" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
321	</Constraint>	Cierra la sección XML abierta anteriormente.
322	</Constraints>	Cierra la sección XML abierta anteriormente.
323		Línea en blanco para organizar el XML del formulario.
324	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
325	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
326	</Layout>	Cierra la sección XML abierta anteriormente.
327	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
328	<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
329	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
330	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		

331	<Color blue="ea" green="f5" red="e6" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
332	</Property>	Cierra la sección XML abierta anteriormente.
333	</Properties>	Cierra la sección XML abierta anteriormente.
334	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
335	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
336	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
337	</Constraint>	Cierra la sección XML abierta anteriormente.
338	</Constraints>	Cierra la sección XML abierta anteriormente.
339		Línea en blanco para organizar el XML del formulario.
340	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
341	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
342	</Layout>	Cierra la sección XML abierta anteriormente.
343	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
344	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
345	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
346	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
347	<Image iconType="3" name="/Imágenes/cheque.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
348	</Property>	Cierra la sección XML abierta anteriormente.
349	</Properties>	Cierra la sección XML abierta anteriormente.
350	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
351	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
352	<AbsoluteConstraints x="18" y="23" width="45" height="45"/>	Define la posición y el tamaño del componente dentro del diseño.
353	</Constraint>	Cierra la sección XML abierta anteriormente.
354	</Constraints>	Cierra la sección XML abierta anteriormente.
355	</Component>	Cierra la sección XML abierta anteriormente.
356	</SubComponents>	Cierra la sección XML abierta anteriormente.
357	</Container>	Cierra la sección XML abierta anteriormente.
358	<Component class="javax.swing.JLabel" name="labelTitulo13">	Declara un componente o contenedor visual y su clase Java.
359	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
360	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
361		Metadato de NetBeans que conserva el estado editable del diseñador visual.
362	</Property>	Cierra la sección XML abierta anteriormente.
363	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
364	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
365	</Property>	Cierra la sección XML abierta anteriormente.
366	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
367	<Property name="text" type="java.lang.String" value="Activos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
368	</Properties>	Cierra la sección XML abierta anteriormente.
369	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
370	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
371	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.

372	</Constraint>	Cierra la sección XML abierta anteriormente.
373	</Constraints>	Cierra la sección XML abierta anteriormente.
374	</Component>	Cierra la sección XML abierta anteriormente.
375	<Component class="javax.swing.JLabel" name="labelActivos">	Declara un componente o contenedor visual y su clase Java.
376	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
377	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
378		Metadato de NetBeans que conserva el estado editable del diseñador visual.
379	</Property>	Cierra la sección XML abierta anteriormente.
380	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
381	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
382	</Property>	Cierra la sección XML abierta anteriormente.
383	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
384	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
385	</Properties>	Cierra la sección XML abierta anteriormente.
386	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
387	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
388	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
389	</Constraint>	Cierra la sección XML abierta anteriormente.
390	</Constraints>	Cierra la sección XML abierta anteriormente.
391	</Component>	Cierra la sección XML abierta anteriormente.
392	<Component class="javax.swing.JLabel" name="labelTitulo15">	Declara un componente o contenedor visual y su clase Java.
393	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
394	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
395		Metadato de NetBeans que conserva el estado editable del diseñador visual.
396	</Property>	Cierra la sección XML abierta anteriormente.
397	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
398	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	</Property>	Cierra la sección XML abierta anteriormente.
400	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
401	<Property name="text" type="java.lang.String" value="Disponible para uso"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
402	</Properties>	Cierra la sección XML abierta anteriormente.
403	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
404	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
405	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
406	</Constraint>	Cierra la sección XML abierta anteriormente.
407	</Constraints>	Cierra la sección XML abierta anteriormente.
408	</Component>	Cierra la sección XML abierta anteriormente.
409	</SubComponents>	Cierra la sección XML abierta anteriormente.
410	</Container>	Cierra la sección XML abierta anteriormente.
411	<Container class="Componentes.PanelFlotante" name="panelFlotante3">	Declara un componente o contenedor visual y su clase Java.
412	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
413	<Property name="visible" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

414	</Properties>	Cierra la sección XML abierta anteriormente.
415	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
416	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
417	<AbsoluteConstraints x="1170" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
418	</Constraint>	Cierra la sección XML abierta anteriormente.
419	</Constraints>	Cierra la sección XML abierta anteriormente.
420		Línea en blanco para organizar el XML del formulario.
421	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
422	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
423	</Layout>	Cierra la sección XML abierta anteriormente.
424	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
425	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
426	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
427	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
428	<Color blue="f3" green="ef" red="ed" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
429	</Property>	Cierra la sección XML abierta anteriormente.
430	</Properties>	Cierra la sección XML abierta anteriormente.
431	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
432	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
433	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
434	</Constraint>	Cierra la sección XML abierta anteriormente.
435	</Constraints>	Cierra la sección XML abierta anteriormente.
436		Línea en blanco para organizar el XML del formulario.
437	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
438	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
439	</Layout>	Cierra la sección XML abierta anteriormente.
440	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
441	<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
442	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
443	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
444	<Image iconType="3" name="/Imágenes/x_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
445	</Property>	Cierra la sección XML abierta anteriormente.
446	</Properties>	Cierra la sección XML abierta anteriormente.
447	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
448	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
449	<AbsoluteConstraints x="23" y="27" width="35" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
450	</Constraint>	Cierra la sección XML abierta anteriormente.
451	</Constraints>	Cierra la sección XML abierta anteriormente.
452	</Component>	Cierra la sección XML abierta anteriormente.
453	</SubComponents>	Cierra la sección XML abierta anteriormente.
454	</Container>	Cierra la sección XML abierta anteriormente.

455	<Component class="javax.swing.JLabel" name="labelTitulo10">	Declara un componente o contenedor visual y su clase Java.
456	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
457	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
458		Metadato de NetBeans que conserva el estado editable del diseñador visual.
459	</Property>	Cierra la sección XML abierta anteriormente.
460	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
461	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
462	</Property>	Cierra la sección XML abierta anteriormente.
463	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
464	<Property name="text" type="java.lang.String" value="Inactivos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
465	</Properties>	Cierra la sección XML abierta anteriormente.
466	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
467	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
468	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
469	</Constraint>	Cierra la sección XML abierta anteriormente.
470	</Constraints>	Cierra la sección XML abierta anteriormente.
471	</Component>	Cierra la sección XML abierta anteriormente.
472	<Component class="javax.swing.JLabel" name="labelInactivos">	Declara un componente o contenedor visual y su clase Java.
473	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
474	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
475		Metadato de NetBeans que conserva el estado editable del diseñador visual.
476	</Property>	Cierra la sección XML abierta anteriormente.
477	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
478	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
479	</Property>	Cierra la sección XML abierta anteriormente.
480	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
481	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
482	</Properties>	Cierra la sección XML abierta anteriormente.
483	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
484	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
485	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
486	</Constraint>	Cierra la sección XML abierta anteriormente.
487	</Constraints>	Cierra la sección XML abierta anteriormente.
488	</Component>	Cierra la sección XML abierta anteriormente.
489	<Component class="javax.swing.JLabel" name="labelTitulo12">	Declara un componente o contenedor visual y su clase Java.
490	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
491	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
492		Metadato de NetBeans que conserva el estado editable del diseñador visual.
493	</Property>	Cierra la sección XML abierta anteriormente.
494	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
495	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

496	</Property>	Cierra la sección XML abierta anteriormente.
497	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
498	<Property name="text" type="java.lang.String" value="No disponibles"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
499	</Properties>	Cierra la sección XML abierta anteriormente.
500	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
501	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
502	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
503	</Constraint>	Cierra la sección XML abierta anteriormente.
504	</Constraints>	Cierra la sección XML abierta anteriormente.
505	</Component>	Cierra la sección XML abierta anteriormente.
506	</SubComponents>	Cierra la sección XML abierta anteriormente.
507	</Container>	Cierra la sección XML abierta anteriormente.
508	<Component class="Componentes.BotonRedondeado" name="botonExportar">	Declara un componente o contenedor visual y su clase Java.
509	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
510	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
511	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
512	</Property>	Cierra la sección XML abierta anteriormente.
513	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
514	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
515	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
516	</Property>	Cierra la sección XML abierta anteriormente.
517	<Property name="radio" type="int" value="20"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
518	<Property name="text" type="java.lang.String" value="Exportar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
519	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
520	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
521	</Property>	Cierra la sección XML abierta anteriormente.
522	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
523	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
524	</Property>	Cierra la sección XML abierta anteriormente.
525	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
526		Metadato de NetBeans que conserva el estado editable del diseñador visual.
527	</Property>	Cierra la sección XML abierta anteriormente.
528	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
529	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
530	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
531	</Property>	Cierra la sección XML abierta anteriormente.
532	</Properties>	Cierra la sección XML abierta anteriormente.
533	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
534	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
535	<AbsoluteConstraints x="1335" y="45" width="175" height="54"/>	Define la posición y el tamaño del componente dentro del diseño.
536	</Constraint>	Cierra la sección XML abierta anteriormente.
537	</Constraints>	Cierra la sección XML abierta anteriormente.
538	</Component>	Cierra la sección XML abierta anteriormente.

539	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
540	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
541	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
542	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
543	</Property>	Cierra la sección XML abierta anteriormente.
544	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
545	<Color blue="ed" green="e6" red="e1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
546	</Property>	Cierra la sección XML abierta anteriormente.
547	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
548	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
549	</Properties>	Cierra la sección XML abierta anteriormente.
550	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
551	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
552	<AbsoluteConstraints x="60" y="175" width="1470" height="95"/>	Define la posición y el tamaño del componente dentro del diseño.
553	</Constraint>	Cierra la sección XML abierta anteriormente.
554	</Constraints>	Cierra la sección XML abierta anteriormente.
555		Línea en blanco para organizar el XML del formulario.
556	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
557	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
558	</Layout>	Cierra la sección XML abierta anteriormente.
559	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
560	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.
561	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
562	<Property name="placeholder" type="java.lang.String" value="Buscar producto o ingrediente..."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
563	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
564		Metadato de NetBeans que conserva el estado editable del diseñador visual.
565	</Property>	Cierra la sección XML abierta anteriormente.
566	</Properties>	Cierra la sección XML abierta anteriormente.
567	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
568	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="campoBusquedaActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
569	</Events>	Cierra la sección XML abierta anteriormente.
570	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
571	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
572	<AbsoluteConstraints x="20" y="20" width="420" height="55"/>	Define la posición y el tamaño del componente dentro del diseño.
573	</Constraint>	Cierra la sección XML abierta anteriormente.
574	</Constraints>	Cierra la sección XML abierta anteriormente.
575	</Component>	Cierra la sección XML abierta anteriormente.
576	<Component class="Componentes.BotonDesplegable" name="filtroCategoria">	Declara un componente o contenedor visual y su clase Java.
577	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
578	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
579	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
580	</Property>	Cierra la sección XML abierta anteriormente.

581	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
582	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
583	</Property>	Cierra la sección XML abierta anteriormente.
584	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
585	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
586	</Property>	Cierra la sección XML abierta anteriormente.
587	<Property name="text" type="java.lang.String" value="Todos los tipos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
588	<Property name="textoDesplegable" type="java.lang.String" value="Todos los tipos;Ingredientes;Produc	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
589	</Properties>	Cierra la sección XML abierta anteriormente.
590	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
591	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.	Asocia un evento visual con el método Java que lo atenderá.
592	</Events>	Cierra la sección XML abierta anteriormente.
593	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
594	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.	Define la posición y el tamaño del componente dentro del diseño.
595	<AbsoluteConstraints x="460" y="22" width="280" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
596	</Constraint>	Cierra la sección XML abierta anteriormente.
597	</Constraints>	Cierra la sección XML abierta anteriormente.
598	</Component>	Cierra la sección XML abierta anteriormente.
599	<Component class="Componentes.BotonDesplegable" name="filtroEstado">	Declara un componente o contenedor visual y su clase Java.
600	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
601	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
602	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
603	</Property>	Cierra la sección XML abierta anteriormente.
604	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
605	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
606	</Property>	Cierra la sección XML abierta anteriormente.
607	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
608	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
609	</Property>	Cierra la sección XML abierta anteriormente.
610	<Property name="text" type="java.lang.String" value="Todos los estados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
611	<Property name="textoDesplegable" type="java.lang.String" value="Todos los estados;Disponible;Stock	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
612	</Properties>	Cierra la sección XML abierta anteriormente.
613	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
614	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.	Define la posición y el tamaño del componente dentro del diseño.
615	<AbsoluteConstraints x="760" y="22" width="280" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
616	</Constraint>	Cierra la sección XML abierta anteriormente.
617	</Constraints>	Cierra la sección XML abierta anteriormente.
618	</Component>	Cierra la sección XML abierta anteriormente.

619	<Component class="Componentes.BotonRedondeado" name="botonLimpiarFiltros">	Declara un componente o contenedor visual y su clase Java.
620	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
621	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
622	>	
622	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
623	</Property>	Cierra la sección XML abierta anteriormente.
624	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
625	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
626	>	
626	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
627	</Property>	Cierra la sección XML abierta anteriormente.
628	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
629	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
630	>	
630	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
631	</Property>	Cierra la sección XML abierta anteriormente.
632	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
633	>	
633	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
634	</Property>	Cierra la sección XML abierta anteriormente.
635	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
636		Metadato de NetBeans que conserva el estado editable del diseñador visual.
637	</Property>	Cierra la sección XML abierta anteriormente.
638	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
639	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
640	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
641	</Property>	Cierra la sección XML abierta anteriormente.
642	</Properties>	Cierra la sección XML abierta anteriormente.
643	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
644	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
645	<AbsoluteConstraints x="1060" y="22" width="380" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
646	</Constraint>	Cierra la sección XML abierta anteriormente.
647	</Constraints>	Cierra la sección XML abierta anteriormente.
648	</Component>	Cierra la sección XML abierta anteriormente.
649	</SubComponents>	Cierra la sección XML abierta anteriormente.
650	</Container>	Cierra la sección XML abierta anteriormente.
651	<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
652	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
653	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
654	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
655	</Property>	Cierra la sección XML abierta anteriormente.
656	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
657	<Color blue="ed" green="e6" red="e1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
658	</Property>	Cierra la sección XML abierta anteriormente.
659	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

660	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
661	</Properties>	Cierra la sección XML abierta anteriormente.
662	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
663	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
664	<AbsoluteConstraints x="60" y="285" width="1470" height="640"/>	Define la posición y el tamaño del componente dentro del diseño.
665	</Constraint>	Cierra la sección XML abierta anteriormente.
666	</Constraints>	Cierra la sección XML abierta anteriormente.
667		Línea en blanco para organizar el XML del formulario.
668	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
669	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
670	</Layout>	Cierra la sección XML abierta anteriormente.
671	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
672	<Container class="javax.swing.JScrollPane" name="scrollUsuarios">	Declara un componente o contenedor visual y su clase Java.
673	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
674	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
675	<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
676	</Property>	Cierra la sección XML abierta anteriormente.
677	<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
678	<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
679	<Property name="autoscrolls" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
680	</Properties>	Cierra la sección XML abierta anteriormente.
681	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
682	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
683	<AbsoluteConstraints x="14" y="14" width="1442" height="555"/>	Define la posición y el tamaño del componente dentro del diseño.
684	</Constraint>	Cierra la sección XML abierta anteriormente.
685	</Constraints>	Cierra la sección XML abierta anteriormente.
686		Línea en blanco para organizar el XML del formulario.
687	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
688	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
689	<Component class="Componentes.TablaAdministrativa" name="tablaProductos">	Declara un componente o contenedor visual y su clase Java.
690	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
691	<Property name="model" type="javax.swing.table.TableModel" editor="org.netbeans.modules.form.editors2.TableModelEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
692	<Table columnCount="10" rowCount="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
693	<Column editable="false" title="ID" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
694	<Column editable="false" title="Producto / ingrediente" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
695	<Column editable="false" title="Tipo" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
696	<Column editable="false" title="Categoría" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
697	<Column editable="false" title="Unidad" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
698	<Column editable="false" title="Stock actual" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
699	<Column editable="false" title="Stock máximo" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
700	<Column editable="false" title="Último movimiento" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
701	<Column editable="false" title="Estado" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

702	<Column editable="false" title="Acción" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
703	</Table>	Cierra la sección XML abierta anteriormente.
704	</Property>	Cierra la sección XML abierta anteriormente.
705	<Property name="autoscrolls" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
706	<Property name="columnasCentradas" type="java.lang.String" value="0,2,4,5,6,8,9"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
707	</Properties>	Cierra la sección XML abierta anteriormente.
708	</Component>	Cierra la sección XML abierta anteriormente.
709	</SubComponents>	Cierra la sección XML abierta anteriormente.
710	</Container>	Cierra la sección XML abierta anteriormente.
711	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
712	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
713	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
714	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
715	</Property>	Cierra la sección XML abierta anteriormente.
716	<Property name="text" type="java.lang.String" value="Mostrando inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
717	</Properties>	Cierra la sección XML abierta anteriormente.
718	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
719	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
720	<AbsoluteConstraints x="28" y="581" width="420" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
721	</Constraint>	Cierra la sección XML abierta anteriormente.
722	</Constraints>	Cierra la sección XML abierta anteriormente.
723	</Component>	Cierra la sección XML abierta anteriormente.
724	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
725	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
726	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
727	</Properties>	Cierra la sección XML abierta anteriormente.
728	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
729	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
730	<AbsoluteConstraints x="1165" y="581" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
731	</Constraint>	Cierra la sección XML abierta anteriormente.
732	</Constraints>	Cierra la sección XML abierta anteriormente.
733		Línea en blanco para organizar el XML del formulario.
734	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
735	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
736	</Layout>	Cierra la sección XML abierta anteriormente.
737	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
738	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
739	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
740	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
741	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
742	</Property>	Cierra la sección XML abierta anteriormente.
743	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

744	<Property name="text" type="java.lang.String" value="#x2039;"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
745	</Properties>	Cierra la sección XML abierta anteriormente.
746	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
747	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
748	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
749	</Constraint>	Cierra la sección XML abierta anteriormente.
750	</Constraints>	Cierra la sección XML abierta anteriormente.
751	</Component>	Cierra la sección XML abierta anteriormente.
752	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
753	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
754	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
755	<Color blue="0" green="bc" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
756	</Property>	Cierra la sección XML abierta anteriormente.
757	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
758	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
759	</Properties>	Cierra la sección XML abierta anteriormente.
760	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
761	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
762	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
763	</Constraint>	Cierra la sección XML abierta anteriormente.
764	</Constraints>	Cierra la sección XML abierta anteriormente.
765	</Component>	Cierra la sección XML abierta anteriormente.
766	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
767	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
768	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
769	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
770	</Property>	Cierra la sección XML abierta anteriormente.
771	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
772	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
773	</Properties>	Cierra la sección XML abierta anteriormente.
774	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
775	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
776	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
777	</Constraint>	Cierra la sección XML abierta anteriormente.
778	</Constraints>	Cierra la sección XML abierta anteriormente.
779	</Component>	Cierra la sección XML abierta anteriormente.
780	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
781	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
782	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
783	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

784	</Property>	Cierra la sección XML abierta anteriormente.
785	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
786	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
787	</Properties>	Cierra la sección XML abierta anteriormente.
788	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
789	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
790	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
791	</Constraint>	Cierra la sección XML abierta anteriormente.
792	</Constraints>	Cierra la sección XML abierta anteriormente.
793	</Component>	Cierra la sección XML abierta anteriormente.
794	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
795	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
796	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
797	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
798	</Property>	Cierra la sección XML abierta anteriormente.
799	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
800	<Property name="text" type="java.lang.String" value="#x203a;/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
801	</Properties>	Cierra la sección XML abierta anteriormente.
802	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
803	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
804	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
805	</Constraint>	Cierra la sección XML abierta anteriormente.
806	</Constraints>	Cierra la sección XML abierta anteriormente.
807	</Component>	Cierra la sección XML abierta anteriormente.
808	</SubComponents>	Cierra la sección XML abierta anteriormente.
809	</Container>	Cierra la sección XML abierta anteriormente.
810	</SubComponents>	Cierra la sección XML abierta anteriormente.
811	</Container>	Cierra la sección XML abierta anteriormente.
812	</SubComponents>	Cierra la sección XML abierta anteriormente.
813	</Form>	Cierra la sección XML abierta anteriormente.

16.61 src/GUI_ADMINISTRADOR/GestionInventarioPanel.java

Pantalla compuesta de inventario con pestañas Existencias, Ingredientes y Movimientos.

Tipo: java | Líneas: 1257 | Archivo 61 de 157

Clases/tipos detectados: GestionInventarioPanel, ModeloTabla, RenderEstado, RenderAccion, EditorAccion, DialogoMovimiento

Línea	Constructor o método detectado
90	public GestionInventarioPanel() {
108	private void prepararVista() {
129	private BotonRedondeado crearBotonSuperior(String texto, Color color) {
143	private void configurarFiltros() {
200	private void prepararSelector(BotonDesplegable boton) {
212	private void configurarTabla() {
231	private void configurarPaginacion() {
245	private void prepararBotonPagina(JButton boton,
265	private void configurarEventos() {
275	private void cargarPagina() {
317	private void actualizarPaginacion(int total) {
348	private void cambiarPagina(int pagina) {
354	private ArticuloInventario articuloSeleccionado() {
360	private void abrirMovimientoSeleccionado(String tipoMovimiento) {
371	private void abrirMovimiento(ArticuloInventario articulo,
380	private void mostrarHistorial() {
457	private String cantidadVisible(MovimientoInventario movimiento) {
466	private void exportarInventario() {
490	private void guardarCsv(List<ArticuloInventario> articulos) {
525	private void filaCsv(StringBuilder csv, Object... valores) {
537	private void mostrarError(String operacion, Throwable error) {
658	private JPanel panelAccion(Color fondo) {
664	private BotonRedondeado botonMovimiento() {
682	private final JComboBox<String> tipo = new JComboBox<>(new String[]{
689	private final BotonRedondeado guardar = botonDialogo(
711	private PanelFlotante crearContenido() {
780	private void agregarCampo(JPanel panel, int fila, String nombre,
799	private void actualizarInstruccion() {
805	private void guardar() {

Línea	Constructor o método detectado
833	private void estilizarCampo(JTextField campo) {
841	private BotonRedondeado botonDialogo(String texto, boolean principal) {
862	private void initComponents() {
1094	public void actionPerformed(java.awt.event.ActionEvent evt) {
1106	public void actionPerformed(java.awt.event.ActionEvent evt) {
1154	public boolean isCellEditable(int rowIndex, int columnIndex) {
1201	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_campoBusquedaActionPerformed
1205	private void filtroCategoriaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroCategoriaActionPerformed

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JPanel.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
8	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
9	import Componentes.PanelFlotante;	Importa Componentes.PanelFlotante para poder usar esa clase sin escribir su nombre completo.
10	import DAO.InventarioDAO;	Importa DAO.InventarioDAO para poder usar esa clase sin escribir su nombre completo.
11	import Modelos.ArticuloInventario;	Importa Modelos.ArticuloInventario para poder usar esa clase sin escribir su nombre completo.
12	import Modelos.MovimientoInventario;	Importa Modelos.MovimientoInventario para poder usar esa clase sin escribir su nombre completo.
13	import Modelos.PaginaInventario;	Importa Modelos.PaginaInventario para poder usar esa clase sin escribir su nombre completo.
14	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.
15	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
18	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
19	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
20	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
21	import java.awt.GridBagConstraints;	Importa java.awt.GridBagConstraints para poder usar esa clase sin escribir su nombre completo.
22	import java.awt.GridBagLayout;	Importa java.awt.GridBagLayout para poder usar esa clase sin escribir su nombre completo.
23	import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
24	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
25	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
26	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
27	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
28	import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
29	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
30	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
31	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.

32	import java.time.format.DateTimeFormatter;	Importa java.time.format.DateTimeFormatter para poder usar esa clase sin escribir su nombre completo.
33	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
34	import java.util.Locale;	Importa java.util.Locale para poder usar esa clase sin escribir su nombre completo.
35	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
36	import javax.swing.AbstractCellEditor;	Importa javax.swing.AbstractCellEditor para poder usar esa clase sin escribir su nombre completo.
37	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
38	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
39	import javax.swing.JComboBox;	Importa javax.swing.JComboBox para poder usar esa clase sin escribir su nombre completo.
40	import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
41	import javax.swing.JFileChooser;	Importa javax.swing.JFileChooser para poder usar esa clase sin escribir su nombre completo.
42	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
43	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
44	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
45	import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
46	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
47	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
48	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
49	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
50	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
51	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
52	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
53	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
54	import javax.swing.filechooser.FileNameExtensionFilter;	Importa javax.swing.filechooser.FileNameExtensionFilter para poder usar esa clase sin escribir su nombre completo.
55	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
56	import javax.swing.table.DefaultTableModel;	Importa javax.swing.table.DefaultTableModel para poder usar esa clase sin escribir su nombre completo.
57	import javax.swing.table.TableCellEditor;	Importa javax.swing.table.TableCellEditor para poder usar esa clase sin escribir su nombre completo.
58	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
59		Línea en blanco: separa visualmente bloques para que el código sea más legible.
60	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
61	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
62	* @author Humberto Alvarado	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
63	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
64	public class GestionInventarioPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
67	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
68	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
69	private static final Color BORDE = new Color(225, 230, 237);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
70	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
71	private static final int POR_PAGINA = 10;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
72	private static final DateTimeFormatter FECHA =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
73	DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.

76	private final InventarioDAO inventarioDAO = new InventarioDAO();	Crea una nueva instancia de InventarioDAO y la usa o guarda para continuar el proceso.
77	private final ModeloTabla modeloTabla = new ModeloTabla();	Crea una nueva instancia de ModeloTabla y la usa o guarda para continuar el proceso.
78	private final JButton[] botonesPagina = new JButton[3];	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
79	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
80	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	private int totalPaginas = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
82	private int secuenciaCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	private String tipoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
84	private String estadoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
85	private boolean errorConexionMostrado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
88	* Creates new form GestionInventarioPanel	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
89	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
90	public GestionInventarioPanel() {	Constructor de GestionInventarioPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
91	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	temporizadorBusqueda = new Timer(350, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
93	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
94	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
97	botonesPagina[0] = botonPagina1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
98	botonesPagina[1] = botonPagina2;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
99	botonesPagina[2] = botonPagina3;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
100	prepararVista();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	configurarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	configurarPaginacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	configurarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
105	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106	}	Cierra el bloque abierto anteriormente.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	private void prepararVista() {	Declara el método prepararVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
109	labelTitulo3.setFont(tema.negrita(42f));	Configura una propiedad mediante setFont en el objeto indicado.
110	labelTitulo2.setFont(tema.regular(16f));	Configura una propiedad mediante setFont en el objeto indicado.
111	botonAgregarProducto.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
112	botonRegistrarSalida.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
113	botonMovimientos.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
114	botonExportar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
115	botonExportar.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
116	IconosUsuarios.Tipo.EXPORTAR, ROJO, 19));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	botonExportar.setIconTextGap(9);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
118	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
119	botonLimpiarFiltros.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.

120	botonLimpiarFiltros.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
121	IconosUsuarios.Tipo.FILTRO, ROJO, 19));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	botonLimpiarFiltros.setIconTextGap(9);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
123	scrollUsuarios.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
124	scrollUsuarios.getViewport().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
125	etiquetaRango.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
126	etiquetaRango.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
127	}	Cierra el bloque abierto anteriormente.
128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	private BotonRedondeado crearBotonSuperior(String texto, Color color) {	Declara el método crearBotonSuperior. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
130	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
131	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
132	boton.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
133	boton.setForeground(color);	Configura una propiedad mediante setForeground en el objeto indicado.
134	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
135	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
136	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
137	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
138	boton.setGrosorBorde(1.3f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
139	boton.setRadio(13);	Configura una propiedad mediante setRadio en el objeto indicado.
140	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
141	}	Cierra el bloque abierto anteriormente.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	private void configurarFiltros() {	Declara el método configurarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
144	filtroCategoria.setText("Todos los tipos");	Cambia el texto visible de un componente de la interfaz.
145	filtroCategoria.setTextoDesplegable(	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
146	"Todos los tipos;Ingredientes;Productos directos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	prepararSelector(filtroCategoria);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	filtroCategoria.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	filtroCategoria.setText(e.getActionCommand());	Cambia el texto visible de un componente de la interfaz.
150	tipoActual = switch (e.getActionCommand()) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
151	case "Ingredientes" -> "INGREDIENTE";	Define una de las rutas posibles dentro del switch.
152	case "Productos directos" -> "PRODUCTO";	Define una de las rutas posibles dentro del switch.
153	default -> null;	Define una de las rutas posibles dentro del switch.
154	};	Cierra el bloque abierto anteriormente y termina la declaración.
155	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
156	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
157	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
160	filtroEstado.setTextoDesplegable(	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
161	"Todos los estados;Disponible;Stock bajo;Agotado;Inactivo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	prepararSelector(filtroEstado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	filtroEstado.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

164	<code>filtroEstado.setText(e.getActionCommand());</code>	Cambia el texto visible de un componente de la interfaz.
165	<code>estadoActual = switch (e.getActionCommand()) {</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
166	<code>case "Disponible" -> "DISPONIBLE";</code>	Define una de las rutas posibles dentro del switch.
167	<code>case "Stock bajo" -> "BAJO";</code>	Define una de las rutas posibles dentro del switch.
168	<code>case "Agotado" -> "AGOTADO";</code>	Define una de las rutas posibles dentro del switch.
169	<code>case "Inactivo" -> "INACTIVO";</code>	Define una de las rutas posibles dentro del switch.
170	<code>default -> null;</code>	Define una de las rutas posibles dentro del switch.
171	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
172	<code>paginaActual = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
173	<code>cargarPagina();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
174	<code>}};</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
175		Línea en blanco: separa visualmente bloques para que el código sea más legible.
176	<code>campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {</code>	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
177	<code>@Override public void insertUpdate(DocumentEvent e) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
178	<code>temporizadorBusqueda.restart();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	<code>}</code>	Cierra el bloque abierto anteriormente.
180	<code>@Override public void removeUpdate(DocumentEvent e) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
181	<code>temporizadorBusqueda.restart();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
182	<code>}</code>	Cierra el bloque abierto anteriormente.
183	<code>@Override public void changedUpdate(DocumentEvent e) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
184	<code>temporizadorBusqueda.restart();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	<code>}</code>	Cierra el bloque abierto anteriormente.
186	<code>}};</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	<code>botonLimpiarFiltros.addActionListener(e -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
188	<code>temporizadorBusqueda.stop();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189	<code>campoBusqueda.setText("");</code>	Cambia el texto visible de un componente de la interfaz.
190	<code>temporizadorBusqueda.stop();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
191	<code>tipoActual = null;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
192	<code>estadoActual = null;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
193	<code>filtroCategoria.setText("Todos los tipos");</code>	Cambia el texto visible de un componente de la interfaz.
194	<code>filtroEstado.setText("Todos los estados");</code>	Cambia el texto visible de un componente de la interfaz.
195	<code>paginaActual = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
196	<code>cargarPagina();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	<code>}};</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198	<code>}</code>	Cierra el bloque abierto anteriormente.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	<code>private void prepararSelector(BotonDesplegable boton) {</code>	Declara el método prepararSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
201	<code>boton.setFont(tema.regular(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
202	<code>boton.setForeground(AZUL);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
203	<code>boton.setColorFondo(Color.WHITE);</code>	Configura una propiedad mediante setColorFondo en el objeto indicado.
204	<code>boton.setColorHover(new Color(248, 249, 251));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
205	<code>boton.setColorDesplegado(new Color(255, 247, 222));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
206	<code>boton.setColorTextoOpcion(AZUL);</code>	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
207	<code>boton.setColorBordeMenu(BORDE);</code>	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.

208	boton.setAnchoMenu(280);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
209	boton.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
210	}	Cierra el bloque abierto anteriormente.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
213	tablaProductos.setModel(modeloTabla);	Configura una propiedad mediante setModel en el objeto indicado.
214	tablaProductos.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215	tablaProductos.setRowHeight(53);	Configura una propiedad mediante setRowHeight en el objeto indicado.
216	tablaProductos.setAutoCreateRowSorter(false);	Configura una propiedad mediante setAutoCreateRowSorter en el objeto indicado.
217	tablaProductos.setColumnsCentradas("0,2,4,5,6,8,9");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
218	tablaProductos.getColumnModel().getColumn(8)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
219	.setCellRenderer(new RenderEstado());	Crea una nueva instancia de RenderEstado y la usa o guarda para continuar el proceso.
220	tablaProductos.getColumnModel().getColumn(9)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
221	.setCellRenderer(new RenderAccion());	Crea una nueva instancia de RenderAccion y la usa o guarda para continuar el proceso.
222	tablaProductos.getColumnModel().getColumn(9)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	.setCellEditor(new EditorAccion());	Crea una nueva instancia de EditorAccion y la usa o guarda para continuar el proceso.
224	int[] anchos = {78, 220, 128, 125, 82, 95, 95, 160, 105, 125};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
225	for (int i = 0; i < anchos.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
226	tablaProductos.getColumnModel().getColumn(i)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	.setPreferredWidth(anchos[i]);	Configura una propiedad mediante setPreferredWidth en el objeto indicado.
228	}	Cierra el bloque abierto anteriormente.
229	}	Cierra el bloque abierto anteriormente.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	private void configurarPaginacion() {	Declara el método configurarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
232	prepararBotonPagina(botonAnterior, IconosUsuarios.Tipo.ANTERIOR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
233	botonAnterior.addActionListener(e -> cambiarPagina(paginaActual - 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
234	for (JButton boton : botonesPagina) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
235	prepararBotonPagina(boton, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	boton.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
237	Object numero = boton.getClientProperty("pagina");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
238	if (numero instanceof Integer pagina) cambiarPagina(pagina);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
239	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240	}	Cierra el bloque abierto anteriormente.
241	prepararBotonPagina(botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
242	botonSiguiente.addActionListener(e -> cambiarPagina(paginaActual + 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
243	}	Cierra el bloque abierto anteriormente.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	private void prepararBotonPagina(JButton boton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246	IconosUsuarios.Tipo icono) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
248	redondeado.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
249	redondeado.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
250	redondeado.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
251	redondeado.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.

252	redondeado.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
253	redondeado.setRadio(10);	Configura una propiedad mediante setRadio en el objeto indicado.
254	}	Cierra el bloque abierto anteriormente.
255	boton.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
256	boton.setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
257	boton.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
258	boton.setIconTextGap(0);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
259	if (icono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
260	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
261	boton.setIcon(IconosUsuarios.crear(icono, AZUL, 16));	Configura una propiedad mediante setIcon en el objeto indicado.
262	}	Cierra el bloque abierto anteriormente.
263	}	Cierra el bloque abierto anteriormente.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	private void configurarEventos() {	Declara el método configurarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
266	botonAgregarProducto.addActionListener(e ->	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
267	abrirMovimientoSeleccionado("ENTRADA"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268	botonRegistrarSalida.addActionListener(e ->	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
269	abrirMovimientoSeleccionado("SALIDA"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
270	botonMovimientos.addActionListener(e ->	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
271	abrirMovimientoSeleccionado("AJUSTE"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
272	botonExportar.addActionListener(e -> exportarInventario());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
273	}	Cierra el bloque abierto anteriormente.
274		Línea en blanco: separa visualmente bloques para que el código sea más legible.
275	private void cargarPagina() {	Declara el método cargarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
276	final int solicitud = ++secuenciaCarga;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
277	final int paginaSolicitada = paginaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
278	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
279	final String tipo = tipoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
280	final String estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
281	tablaProductos.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
282	etiquetaRango.setText("Cargando inventario...");	Cambia el texto visible de un componente de la interfaz.
283	new SwingWorker<PaginaInventario, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
284	@Override protected PaginaInventario doInBackground()	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
285	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
286	return inventarioDAO.listarPagina(busqueda, tipo, estado,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
287	paginaSolicitada, POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
288	}	Cierra el bloque abierto anteriormente.
289		Línea en blanco: separa visualmente bloques para que el código sea más legible.
290	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
291	if (solicitud != secuenciaCarga) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
292	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
293	PaginaInventario datos = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
294	totalPaginas = Math.max(1, (datos.totalRegistros()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
295	+ POR_PAGINA - 1) / POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

296	if (paginaActual > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
297	paginaActual = totalPaginas;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
298	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
299	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
300	}	Cierra el bloque abierto anteriormente.
301	modeloTabla.setArticulos(datos.articulos());	Configura una propiedad mediante setArticulos en el objeto indicado.
302	actualizarPaginacion(datos.totalRegistros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
303	errorConexionMostrado = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
304	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
305	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
307	modeloTabla.setArticulos(List.of());	Configura una propiedad mediante setArticulos en el objeto indicado.
308	actualizarPaginacion(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	mostrarError("cargar el inventario", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
310	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
311	tablaProductos.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
312	}	Cierra el bloque abierto anteriormente.
313	}	Cierra el bloque abierto anteriormente.
314	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
315	}	Cierra el bloque abierto anteriormente.
316		Línea en blanco: separa visualmente bloques para que el código sea más legible.
317	private void actualizarPaginacion(int total) {	Declara el método actualizarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
318	if (total == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
319	etiquetaRango.setText("No se encontraron articulos");	Cambia el texto visible de un componente de la interfaz.
320	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
321	int inicio = (paginaActual - 1) * POR_PAGINA + 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
322	int fin = Math.min(inicio + POR_PAGINA - 1, total);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
323	etiquetaRango.setText(String.format(	Cambia el texto visible de un componente de la interfaz.
324	"Mostrando %d-%d de %d articulos", inicio, fin, total));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
325	}	Cierra el bloque abierto anteriormente.
326	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
327	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
328	int inicio = Math.max(1, Math.min(paginaActual - 1,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
329	totalPaginas - 2));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
330	for (int i = 0; i < botonesPagina.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
331	JButton boton = botonesPagina[i];	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
332	int pagina = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
333	boton.setVisible(pagina <= totalPaginas);	Muestra u oculta una ventana o componente según el valor booleano enviado.
334	if (pagina > totalPaginas) continue;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
335	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
336	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
337	boolean seleccionada = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
338	boton.setFont(seleccionada	Configura una propiedad mediante setFont en el objeto indicado.
339	? tema.negrita(13f) : tema.media(13f));	Ejecuta una instrucción Java completa; el punto y coma marca su final.

340	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
341	redondeado.setColorInicio(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
342	redondeado.setColorFinal(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
343	redondeado.setColorBorde(seleccionada ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
344	}	Cierra el bloque abierto anteriormente.
345	}	Cierra el bloque abierto anteriormente.
346	}	Cierra el bloque abierto anteriormente.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348	private void cambiarPagina(int pagina) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
349	if (pagina < 1 pagina > totalPaginas pagina == paginaActual) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
350	paginaActual = pagina;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
351	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
352	}	Cierra el bloque abierto anteriormente.
353		Línea en blanco: separa visualmente bloques para que el código sea más legible.
354	private ArtículoInventario articuloSeleccionado() {	Declara el método articuloSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
355	int filaVista = tablaProductos.getSelectedRow();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
356	if (filaVista < 0) return null;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
357	return modeloTabla.obtener(tablaProductos.convertRowIndexToModel(filaVista));	Finaliza el método y devuelve el valor indicado a quien lo llamó.
358	}	Cierra el bloque abierto anteriormente.
359		Línea en blanco: separa visualmente bloques para que el código sea más legible.
360	private void abrirMovimientoSeleccionado(String tipoMovimiento) {	Declara el método abrirMovimientoSeleccionado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
361	ArticuloInventario articulo = articuloSeleccionado();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
362	if (articulo == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
363	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
364	"Selecciona primero un producto o ingrediente de la tabla.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	"Selecciona un artículo", JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
366	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
367	}	Cierra el bloque abierto anteriormente.
368	abrirMovimiento(articulo, tipoMovimiento);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
369	}	Cierra el bloque abierto anteriormente.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	private void abrirMovimiento(ArticuloInventario articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
372	String tipoMovimiento) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
373	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
374	if (MovimientoInventarioFormDialog.mostrar(propietario,	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
375	inventarioDAO, articulo, tipoMovimiento)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
376	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377	}	Cierra el bloque abierto anteriormente.
378	}	Cierra el bloque abierto anteriormente.
379		Línea en blanco: separa visualmente bloques para que el código sea más legible.
380	private void mostrarHistorial() {	Declara el método mostrarHistorial. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
381	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
382	List<MovimientoInventario> movimientos =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

383	inventarioDAO.listarMovimientos(200);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
384	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
385	JDialog dialogo = new JDialog(propietario,	Crea una nueva instancia de JDialog y la usa o guarda para continuar el proceso.
386	"Movimientos de inventario",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	java.awt.Dialog.ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
388	dialogo.setUndecorated(true);	Configura una propiedad mediante setUndecorated en el objeto indicado.
389	dialogo.setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
390	dialogo.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);	Configura una propiedad mediante setDefaultCloseOperation en el objeto indicado.
391		Línea en blanco: separa visualmente bloques para que el código sea más legible.
392	PanelFlotante contenido = new PanelFlotante();	Crea una nueva instancia de PanelFlotante y la usa o guarda para continuar el proceso.
393	contenido.setLayout(new BorderLayout(0, 16));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
394	contenido.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
395	contenido.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
396	contenido.setRadio(24);	Configura una propiedad mediante setRadio en el objeto indicado.
397	contenido.setBorder(new EmptyBorder(24, 28, 22, 28));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
398	JPanel cabecera = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
399	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
400	JLabel titulo = new JLabel("Movimientos de inventario");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
401	titulo.setFont(tema.negrita(24f));	Configura una propiedad mediante setFont en el objeto indicado.
402	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
403	JLabel detalle = new JLabel{	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
404	"Últimos " + movimientos.size() + " movimientos registrados";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
405	detalle.setFont(tema.regular(13f));	Configura una propiedad mediante setFont en el objeto indicado.
406	detalle.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
407	cabecera.add(titulo, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
408	cabecera.add(detalle, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	contenido.add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
410		Línea en blanco: separa visualmente bloques para que el código sea más legible.
411	DefaultTableModel modelo = new DefaultTableModel(new String[]{	Crea una nueva instancia de DefaultTableModel y la usa o guarda para continuar el proceso.
412	"ID", "Fecha", "Artículo", "Tipo", "Movimiento",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
413	"Cantidad", "Motivo", "Pedido"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
414	}, 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	@Override public boolean isCellEditable(int f, int c) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
416	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
417	}	Cierra el bloque abierto anteriormente.
418	};	Cierra el bloque abierto anteriormente y termina la declaración.
419	for (MovimientoInventario movimiento : movimientos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
420	modelo.addRow(new Object[]{	Crea una nueva instancia de Object y la usa o guarda para continuar el proceso.
421	"#" + movimiento.id(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
422	movimiento.fecha() == null ? "-" : FECHA.format(movimiento.fecha()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
423	movimiento.articulo(), movimiento.tipoArticulo(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
424	movimiento.tipoMovimiento(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
425	cantidadVisible(movimiento),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
426	movimiento.motivo() == null ? "-" : movimiento.motivo(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

427	movimiento.idPedido() == null ? "-" : "#" + movimiento.idPedido()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
428	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429	}	Cierra el bloque abierto anteriormente.
430	Componentes.TablaAdministrativa tabla =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
431	new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
432	tabla.setModel(modelo);	Configura una propiedad mediante setModel en el objeto indicado.
433	tabla.setRowHeight(43);	Configura una propiedad mediante setRowHeight en el objeto indicado.
434	tabla.setColumnsCentradas("0,1,3,4,5,7");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
435	tabla.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
436	JScrollPane scroll = new JScrollPane(tabla);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
437	Componentes.DesplazamientoSuave.ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
438	scroll.setBorder(BorderFactory.createLineBorder(BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
439	contenido.add(scroll, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
441	JPanel pie = new JPanel(new FlowLayout(FlowLayout.RIGHT));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
442	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
443	BotonRedondeado cerrar = crearBotonSuperior("Cerrar", AZUL);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
444	cerrar.setPreferredSize(new Dimension(130, 42));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
445	cerrar.addActionListener(e -> dialogo.dispose());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
446	pie.add(cerrar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
447	contenido.add(pie, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448	dialogo.setContentPane(contenido);	Configura una propiedad mediante setContentPane en el objeto indicado.
449	dialogo.setSize(1120, 650);	Configura una propiedad mediante setSize en el objeto indicado.
450	dialogo.setLocationRelativeTo(propietario);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
451	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
452	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
453	mostrarError("cargar los movimientos", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
454	}	Cierra el bloque abierto anteriormente.
455	}	Cierra el bloque abierto anteriormente.
456		Línea en blanco: separa visualmente bloques para que el código sea más legible.
457	private String cantidadVisible(MovimientoInventario movimiento) {	Declara el método cantidadVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
458	BigDecimal cantidad = movimiento.cantidad();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
459	if ("SALIDA".equals(movimiento.tipoMovimiento())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
460	cantidad = cantidad.abs().negate();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
461	}	Cierra el bloque abierto anteriormente.
462	String prefijo = cantidad.signum() > 0 ? "+" : "";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
463	return prefijo + cantidad.stripTrailingZeros().toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
464	}	Cierra el bloque abierto anteriormente.
465		Línea en blanco: separa visualmente bloques para que el código sea más legible.
466	private void exportarInventario() {	Declara el método exportarInventario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
467	botonExportar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
468	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
469	final String tipo = tipoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
470	final String estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.

471	new SwingWorker<List<ArticuloInventario>, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
472	@Override protected List<ArticuloInventario> doInBackground()	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
473	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
474	return inventarioDAO.listarParaExportar(búsqueda, tipo, estado);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
475	}	Cierra el bloque abierto anteriormente.
476		Línea en blanco: separa visualmente bloques para que el código sea más legible.
477	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
478	botonExportar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
479	try {	Inicio una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
480	guardarCsv(get());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
481	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
482	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
483	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
484	mostrarError("exportar el inventario", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
485	}	Cierra el bloque abierto anteriormente.
486	}	Cierra el bloque abierto anteriormente.
487	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
488	}	Cierra el bloque abierto anteriormente.
489		Línea en blanco: separa visualmente bloques para que el código sea más legible.
490	private void guardarCsv(List<ArticuloInventario> articulos) {	Declara el método guardarCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
491	JFileChooser selector = new JFileChooser();	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
492	selector.setDialogTitle("Guardar inventario");	Configura una propiedad mediante setDialogTitle en el objeto indicado.
493	selector.setFileFilter(new FileNameExtensionFilter(	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.
494	"Archivo CSV", "csv"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
495	selector.setSelectedFile(new File("inventario.csv"));	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
496	if (selector.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
497	Path archivo = selector.getSelectedFile().toPath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
498	if (!archivo.toString().toLowerCase(Locale.ROOT).endsWith(".csv")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
499	archivo = archivo.resolveSibling(archivo.getFileName() + ".csv");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
500	}	Cierra el bloque abierto anteriormente.
501	StringBuilder csv = new StringBuilder("\uFEFF");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
502	filaCsv(csv, "ID", "Nombre", "Tipo", "Categoría", "Unidad",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
503	"Stock actual", "Stock mínimo", "Último movimiento", "Estado");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
504	for (ArticuloInventario articulo : articulos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
505	filaCsv(csv, articulo.getCodigoVisible(), articulo.getNombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
506	articulo.getTipoVisible(), articulo.getCategoría(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
507	articulo.getUnidad(), articulo.getStockActual(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
508	articulo.getStockMinimo(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
509	articulo.getUltimoMovimiento() == null ? ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
510	: FECHA.format(articulo.getUltimoMovimiento()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
511	articulo.getEstadoVisible());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
512	}	Cierra el bloque abierto anteriormente.
513	try {	Inicio una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
514	Files.writeString(archivo, csv.toString(), StandardCharsets.UTF_8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

515	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
516	"Inventario guardado en:\n" + archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
517	"Exportación completada", JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
518	} catch (IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
519	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
520	"No se pudo guardar el archivo.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
521	"Exportación", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
522	}	Cierra el bloque abierto anteriormente.
523	}	Cierra el bloque abierto anteriormente.
524		Línea en blanco: separa visualmente bloques para que el código sea más legible.
525	private void filaCsv(StringBuilder csv, Object... valores) {	Declara el método filaCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
526	for (int i = 0; i < valores.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
527	if (i > 0) csv.append(',');	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
528	String texto = valores[i] == null ? "" : valores[i].toString();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
529	String inicio = texto.stripLeading();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
530	if (!(valores[i] instanceof Number) && !inicio.isEmpty()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
531	&& "+-@".indexOf(inicio.charAt(0)) >= 0) texto = "" + texto;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
532	csv.append('').append(texto.replace("\n", "\\n")).append('');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
533	}	Cierra el bloque abierto anteriormente.
534	csv.append("\r\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
535	}	Cierra el bloque abierto anteriormente.
536		Línea en blanco: separa visualmente bloques para que el código sea más legible.
537	private void mostrarError(String operacion, Throwable error) {	Declara el método mostrarError. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
538	if (errorConexionMostrado) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
539	errorConexionMostrado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
540	String detalle = error == null error.getMessage() == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
541	? "Error no especificado." : error.getMessage();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
542	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
543	"No fue posible " + operacion + ".\n" + detalle,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
544	"Error de base de datos", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
545	}	Cierra el bloque abierto anteriormente.
546		Línea en blanco: separa visualmente bloques para que el código sea más legible.
547	private final class ModeloTabla extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
548	private final String[] columnas = {"ID", "Producto / ingrediente",	Declara una constante o referencia que no puede reasignarse después de inicializarse.
549	"Tipo", "Categoría", "Unidad", "Stock actual", "Stock mínimo",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
550	"Último movimiento", "Estado", "Acción"};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
551	private List<ArticuloInventario> articulos = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
552		Línea en blanco: separa visualmente bloques para que el código sea más legible.
553	void setArticulos(List<ArticuloInventario> nuevos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
554	articulos = List.copyOf(nuevos);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
555	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
556	}	Cierra el bloque abierto anteriormente.
557		Línea en blanco: separa visualmente bloques para que el código sea más legible.
558	ArticuloInventario obtener(int fila) { return articulos.get(fila); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

559	<code>@Override public int getRowCount() { return articulos.size(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
560	<code>@Override public int getColumnCount() { return columnas.length; }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
561	<code>@Override public String getColumnName(int columna) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
562	<code>return columnas[columna];</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
563	<code>}</code>	Cierra el bloque abierto anteriormente.
564	<code>@Override public boolean isCellEditable(int fila, int columna) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
565	<code>return columna == 9;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
566	<code>}</code>	Cierra el bloque abierto anteriormente.
567	<code>@Override public Class<?> getColumnClass(int columna) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
568	<code>return columna >= 8 ? ArtículoInventario.class : String.class;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
569	<code>}</code>	Cierra el bloque abierto anteriormente.
570	<code>@Override public Object getValueAt(int fila, int columna) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
571	<code>ArticuloInventario item = articulos.get(fila);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
572	<code>return switch (columna) {</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
573	<code>case 0 -> item.getCodigoVisible();</code>	Define una de las rutas posibles dentro del switch.
574	<code>case 1 -> item.getNombre();</code>	Define una de las rutas posibles dentro del switch.
575	<code>case 2 -> item.getTipoVisible();</code>	Define una de las rutas posibles dentro del switch.
576	<code>case 3 -> item.getCategoria();</code>	Define una de las rutas posibles dentro del switch.
577	<code>case 4 -> item.getUnidad();</code>	Define una de las rutas posibles dentro del switch.
578	<code>case 5 -> item.getStockActual().stripTrailingZeros().toString();</code>	Define una de las rutas posibles dentro del switch.
579	<code>case 6 -> item.getStockMinimo().stripTrailingZeros().toString();</code>	Define una de las rutas posibles dentro del switch.
580	<code>case 7 -> item.getUltimoMovimiento() == null ? "Sin movimientos"</code>	Define una de las rutas posibles dentro del switch.
581	<code>: FECHA.format(item.getUltimoMovimiento());</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
582	<code>case 8, 9 -> item;</code>	Define una de las rutas posibles dentro del switch.
583	<code>default -> "";</code>	Define una de las rutas posibles dentro del switch.
584	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
585	<code>}</code>	Cierra el bloque abierto anteriormente.
586	<code>}</code>	Cierra el bloque abierto anteriormente.
587		Línea en blanco: separa visualmente bloques para que el código sea más legible.
588	<code>private final class RenderEstado implements TableCellRenderer {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
589	<code>@Override public Component getTableCellRendererComponent(JTable tabla,</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
590	<code>Object valor, boolean seleccionado, boolean foco,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
591	<code>int fila, int columna) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
592	<code>ArticuloInventario item = (ArticuloInventario) valor;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
593	<code>String estado = item.getEstadoVisible();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
594	<code>Color texto = switch (estado) {</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
595	<code>case "Disponible" -> new Color(34, 139, 71);</code>	Define una de las rutas posibles dentro del switch.
596	<code>case "Stock bajo" -> new Color(187, 123, 0);</code>	Define una de las rutas posibles dentro del switch.
597	<code>case "Agotado" -> ROJO;</code>	Define una de las rutas posibles dentro del switch.
598	<code>default -> SECUNDARIO;</code>	Define una de las rutas posibles dentro del switch.
599	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
600	<code>Color fondo = switch (estado) {</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
601	<code>case "Disponible" -> new Color(230, 245, 234);</code>	Define una de las rutas posibles dentro del switch.
602	<code>case "Stock bajo" -> new Color(255, 244, 210);</code>	Define una de las rutas posibles dentro del switch.

603	case "Agotado" -> new Color(252, 233, 233);	Define una de las rutas posibles dentro del switch.
604	default -> new Color(237, 239, 243);	Define una de las rutas posibles dentro del switch.
605	};	Cierra el bloque abierto anteriormente y termina la declaración.
606	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 13));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
607	panel.setBackground(seleccionado	Configura una propiedad mediante setBackground en el objeto indicado.
608	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
609	JLabel etiqueta = new JLabel(estado);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
610	etiqueta.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
611	etiqueta.setFont(tema.media(12f));	Configura una propiedad mediante setFont en el objeto indicado.
612	etiqueta.setForeground(texto);	Configura una propiedad mediante setForeground en el objeto indicado.
613	etiqueta.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
614	etiqueta.setBorder(new EmptyBorder(4, 10, 4, 10));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
615	panel.add(etiqueta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
616	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
617	}	Cierra el bloque abierto anteriormente.
618	}	Cierra el bloque abierto anteriormente.
619		Línea en blanco: separa visualmente bloques para que el código sea más legible.
620	private final class RenderAccion implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
621	@Override public Component getTableCellRendererComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
622	Object valor, boolean seleccionado, boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
623	int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
624	JPanel panel = panelAccion(seleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
625	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
626	ArticuloInventario item = (ArticuloInventario) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
627	BotonRedondeado boton = botonMovimiento();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
628	boton.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
629	panel.add(boton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
630	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
631	}	Cierra el bloque abierto anteriormente.
632	}	Cierra el bloque abierto anteriormente.
633		Línea en blanco: separa visualmente bloques para que el código sea más legible.
634	private final class EditorAccion extends AbstractCellEditor	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
635	implements TableCellEditor {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
636	private final JPanel panel = panelAccion(Color.WHITE);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
637	private final BotonRedondeado boton = botonMovimiento();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
638	private ArticuloInventario item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
639		Línea en blanco: separa visualmente bloques para que el código sea más legible.
640	EditorAccion() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
641	panel.add(boton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
642	boton.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
643	ArticuloInventario seleccionado = item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
644	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
645	abrirMovimiento(seleccionado, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
646	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.

647	}	Cierra el bloque abierto anteriormente.
648	@Override public Component getTableCellEditorComponent(JTable tabla,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
649	Object valor, boolean seleccionado, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
650	item = (ArticuloInventario) valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
651	boton.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
652	panel.setBackground(tabla.getSelectionBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
653	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
654	}	Cierra el bloque abierto anteriormente.
655	@Override public Object getCellEditorValue() { return item; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
656	}	Cierra el bloque abierto anteriormente.
657		Línea en blanco: separa visualmente bloques para que el código sea más legible.
658	private JPanel panelAccion(Color fondo) {	Declara el método panelAccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
659	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 10));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
660	panel.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
661	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
662	}	Cierra el bloque abierto anteriormente.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	private BotonRedondeado botonMovimiento() {	Declara el método botonMovimiento. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
665	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
666	boton.setText("± Stock");	Cambia el texto visible de un componente de la interfaz.
667	boton.setFont(tema.negrita(12f));	Configura una propiedad mediante setFont en el objeto indicado.
668	boton.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
669	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
670	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
671	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
672	boton.setColorBorde(AMARILLO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
673	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
674	boton.setRadio(9);	Configura una propiedad mediante setRadio en el objeto indicado.
675	boton.setPreferredSize(new Dimension(92, 31));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
676	boton.setToolTipText("Registrar entrada, salida o ajuste");	Configura una propiedad mediante setToolTipText en el objeto indicado.
677	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
678	}	Cierra el bloque abierto anteriormente.
679		Línea en blanco: separa visualmente bloques para que el código sea más legible.
680	private final class DialogoMovimiento extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
681	private final ArticuloInventario articulo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
682	private final JComboBox<String> tipo = new JComboBox<>(new String[]{	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
683	"Entrada", "Salida", "Ajuste"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
684	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
685	private final JTextField cantidad = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
686	private final JTextField motivo = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
687	private final JLabel instruccion = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
688	private final JLabel error = new JLabel(" ");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
689	private final BotonRedondeado guardar = botonDialogo(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
690	"Guardar movimiento", true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

691	private boolean guardado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
692		Línea en blanco: separa visualmente bloques para que el código sea más legible.
693	DialogoMovimiento(Window propietario, ArtículoInventario articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
694	String tipoFijo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
695	super(proprietario, ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
696	this.articulo = articulo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
697	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
698	setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
699	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
700	setContentPane(crearContenido());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
701	if ("ENTRADA".equals(tipoFijo)) tipo.setSelectedIndex(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
702	if ("SALIDA".equals(tipoFijo)) tipo.setSelectedIndex(1);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
703	if (tipoFijo != null) tipo.setEnabled(false);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
704	tipo.addActionListener(e -> actualizarInstruccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
705	actualizarInstruccion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
706	guardar.addActionListener(e -> guardar());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
707	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
708	setLocationRelativeTo(proprietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
709	}	Cierra el bloque abierto anteriormente.
710		Línea en blanco: separa visualmente bloques para que el código sea más legible.
711	private PanelFlotante crearContenido() {	Declara el método crearContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
712	PanelFlotante panel = new PanelFlotante();	Crea una nueva instancia de PanelFlotante y la usa o guarda para continuar el proceso.
713	panel.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
714	panel.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
715	panel.setRadio(26);	Configura una propiedad mediante setRadio en el objeto indicado.
716	panel.setPreferredSize(new Dimension(650, 445));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
717	panel.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
718		Línea en blanco: separa visualmente bloques para que el código sea más legible.
719	JPanel cabecera = new JPanel(new BorderLayout(0, 4));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
720	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
721	cabecera.setBorder(new EmptyBorder(27, 32, 15, 32));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
722	JLabel titulo = new JLabel("Movimiento de inventario");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
723	titulo.setFont(tema.negrita(24f));	Configura una propiedad mediante setFont en el objeto indicado.
724	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
725	JLabel subtítulo = new JLabel(String.format(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
726	"%s · %s · Stock actual: %s %s",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
727	articulo.getCodigoVisible(), articulo.getNombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
728	articulo.getStockActual().stripTrailingZeros().toString(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
729	articulo.getUnidad());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
730	subtitulo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
731	subtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
732	cabecera.add(titulo, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
733	cabecera.add(subtitulo, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
734	panel.add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

735		Línea en blanco: separa visualmente bloques para que el código sea más legible.
736	JPanel campos = new JPanel(new GridBagLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
737	campos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
738	campos.setBorder(new EmptyBorder(0, 30, 0, 30));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
739	tipo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
740	tipo.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
741	tipo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
742	tipo.setBorder(BorderFactory.createLineBorder(BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
743	estilizarCampo(cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
744	estilizarCampo(motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
745	agregarCampo(campos, 0, "Tipo de movimiento", tipo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
746	agregarCampo(campos, 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
747	"Cantidad (" + articulo.getUnidad() + ")", cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
748	agregarCampo(campos, 2, "Motivo (opcional)", motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
749	instruccion.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
750	instruccion.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
751	GridBagConstraints c = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
752	c.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
753	c.gridy = 3;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
754	c.weightx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
755	c.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
756	c.insets = new Insets(0, 4, 0, 4);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
757	campos.add(instruccion, c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
758	panel.add(campos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
759		Línea en blanco: separa visualmente bloques para que el código sea más legible.
760	JPanel pie = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
761	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
762	pie.setPreferredSize(new Dimension(650, 96));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
763	pie.setBorder(new EmptyBorder(0, 32, 20, 32));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
764	error.setFont(tema.media(12f));	Configura una propiedad mediante setFont en el objeto indicado.
765	error.setForeground(R0J0);	Configura una propiedad mediante setForeground en el objeto indicado.
766	pie.add(error, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
767	JPanel botones = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
768	botones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
769	BotonRedondeado cancelar = botonDialogo("Cancelar", false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
770	cancelar.setPreferredSize(new Dimension(128, 45));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
771	guardar.setPreferredSize(new Dimension(195, 45));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
772	cancelar.addActionListener(e -> dispose());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
773	botones.add(cancelar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
774	botones.add(guardar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
775	pie.add(botones, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
776	panel.add(pie, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
777	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
778	}	Cierra el bloque abierto anteriormente.

779		Línea en blanco: separa visualmente bloques para que el código sea más legible.
780	private void agregarCampo(JPanel panel, int fila, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
781	Component control) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
782	JPanel filaPanel = new JPanel(new BorderLayout(0, 5));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
783	filaPanel.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
784	JLabel etiqueta = new JLabel(nombre);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
785	etiqueta.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
786	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
787	filaPanel.add(etiqueta, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
788	control.setPreferredSize(new Dimension(565, 40));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
789	filaPanel.add(control, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
790	GridBagConstraints c = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
791	c.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
792	c.gridy = fila;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
793	c.weightx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
794	c.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
795	c.insets = new Insets(0, 4, 10, 4);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
796	panel.add(filaPanel, c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
797	}	Cierra el bloque abierto anteriormente.
798		Línea en blanco: separa visualmente bloques para que el código sea más legible.
799	private void actualizarInstruccion() {	Declara el método actualizarInstruccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
800	instruccion.setText(tipo.getSelectedIndex() == 2	Cambia el texto visible de un componente de la interfaz.
801	? "En Ajuste, escribe el stock final que debe quedar."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
802	: "En Entrada o Salida, escribe cuánto se agrega o retira.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
803	}	Cierra el bloque abierto anteriormente.
804		Línea en blanco: separa visualmente bloques para que el código sea más legible.
805	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
806	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
807	BigDecimal valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
808	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
809	valor = new BigDecimal(cantidad.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
810	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
811	error.setText("Ingresa una cantidad válida con punto decimal.");	Cambia el texto visible de un componente de la interfaz.
812	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
813	}	Cierra el bloque abierto anteriormente.
814	String codigo = switch (tipo.getSelectedIndex()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
815	case 0 -> "ENTRADA";	Define una de las rutas posibles dentro del switch.
816	case 1 -> "SALIDA";	Define una de las rutas posibles dentro del switch.
817	default -> "AJUSTE";	Define una de las rutas posibles dentro del switch.
818	};	Cierra el bloque abierto anteriormente y termina la declaración.
819	guardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
820	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
821	inventarioDAO.registrarMovimiento(articulo, codigo, valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
822	motivo.getText().trim());	Ejecuta una instrucción Java completa; el punto y coma marca su final.

823	guardado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
824	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
825	} catch (SQLException IllegalArgumentException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
826	error.setText("No fue posible registrar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
827	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
828	guardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
829	}	Cierra el bloque abierto anteriormente.
830	}	Cierra el bloque abierto anteriormente.
831	}	Cierra el bloque abierto anteriormente.
832		Línea en blanco: separa visualmente bloques para que el código sea más legible.
833	private void estilizarCampo(JTextField campo) {	Declara el método estilizarCampo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
834	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
835	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
836	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
837	BorderFactory.createLineBorder(BORDE),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
838	new EmptyBorder(0, 12, 0, 12));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
839	}	Cierra el bloque abierto anteriormente.
840		Línea en blanco: separa visualmente bloques para que el código sea más legible.
841	private BotonRedondeado botonDialogo(String texto, boolean principal) {	Declara el método botonDialogo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
842	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
843	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
844	boton.setFont(tema.negrita(13f));	Configura una propiedad mediante setFont en el objeto indicado.
845	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
846	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
847	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
848	boton.setColorInicio(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
849	boton.setColorFinal(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
850	boton.setColorBorde(principal ? AMARILLO : ROJO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
851	boton.setForeground(principal ? AZUL : ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
852	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
853	}	Cierra el bloque abierto anteriormente.
854		Línea en blanco: separa visualmente bloques para que el código sea más legible.
855	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
856	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
857	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
858	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
859	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
860	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
861	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
862	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
863		Línea en blanco: separa visualmente bloques para que el código sea más legible.
864	labelTitulo3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
865	labelTitulo2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
866	botonAgregarProducto = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.

867	botonRegistrarSalida = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
868	botonMovimientos = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
869	panelFlotante1 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
870	panelCircular2 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
871	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
872	labelTitulo4 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
873	labelTotalProductos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
874	labelTitulo6 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
875	panelFlotante2 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
876	panelCircular3 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
877	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
878	labelTitulo7 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
879	labelStockBajo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
880	labelTitulo9 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
881	panelFlotante4 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
882	panelCircular5 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
883	labelEscalable5 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
884	labelTitulo13 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
885	labelActivos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
886	labelTitulo15 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
887	panelFlotante3 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
888	panelCircular4 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
889	labelEscalable4 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
890	labelTitulo10 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
891	labelInactivos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
892	labelTitulo12 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
893	botonExportar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
894	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
895	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
896	filtroCategoria = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
897	filtroEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
898	botonLimpiarFiltros = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
899	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
900	scrollUsuarios = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
901	tablaProductos = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
902	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
903	panelPaginacion = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
904	botonAnterior = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
905	botonPagina1 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
906	botonPagina2 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
907	botonPagina3 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
908	botonSiguiente = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
909		Línea en blanco: separa visualmente bloques para que el código sea más legible.
910	setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.

911	setPreferredSize(new java.awt.Dimension(1530, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
912	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
913		Línea en blanco: separa visualmente bloques para que el código sea más legible.
914	labelTitulo3.setFont(new java.awt.Font("Dialog", 1, 42)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
915	labelTitulo3.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
916	labelTitulo3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
917	labelTitulo3.setText("Gestión de Inventario");	Cambia el texto visible de un componente de la interfaz.
918	add(labelTitulo3, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 16, 650, 64));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
919		Línea en blanco: separa visualmente bloques para que el código sea más legible.
920	labelTitulo2.setFont(new java.awt.Font("Dialog", 0, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
921	labelTitulo2.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
922	labelTitulo2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
923	labelTitulo2.setText("Administra existencias, entradas y salidas");	Cambia el texto visible de un componente de la interfaz.
924	add(labelTitulo2, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 76, 620, 36));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
925		Línea en blanco: separa visualmente bloques para que el código sea más legible.
926	botonAgregarProducto.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
927	botonAgregarProducto.setText("Registrar entrada");	Cambia el texto visible de un componente de la interfaz.
928	add(botonAgregarProducto, new org.netbeans.lib.awtextra.AbsoluteConstraints(780, 42, 175, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
929		Línea en blanco: separa visualmente bloques para que el código sea más legible.
930	botonRegistrarSalida.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
931	botonRegistrarSalida.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
932	botonRegistrarSalida.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
933	botonRegistrarSalida.setText("Registrar salida");	Cambia el texto visible de un componente de la interfaz.
934	botonRegistrarSalida.setColorBorde(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
935	botonRegistrarSalida.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
936	botonRegistrarSalida.setGrosorBorde(1.3F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
937	botonRegistrarSalida.setRadio(13);	Configura una propiedad mediante setRadio en el objeto indicado.
938	add(botonRegistrarSalida, new org.netbeans.lib.awtextra.AbsoluteConstraints(965, 45, 175, 54));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
939		Línea en blanco: separa visualmente bloques para que el código sea más legible.
940	botonMovimientos.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
941	botonMovimientos.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
942	botonMovimientos.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
943	botonMovimientos.setText("Ajustar stock");	Cambia el texto visible de un componente de la interfaz.
944	botonMovimientos.setColorBorde(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
945	botonMovimientos.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
946	botonMovimientos.setGrosorBorde(1.3F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
947	botonMovimientos.setRadio(13);	Configura una propiedad mediante setRadio en el objeto indicado.
948	add(botonMovimientos, new org.netbeans.lib.awtextra.AbsoluteConstraints(1150, 45, 175, 54));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
949		Línea en blanco: separa visualmente bloques para que el código sea más legible.
950	panelFlotante1.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
951	panelFlotante1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
952		Línea en blanco: separa visualmente bloques para que el código sea más legible.
953	panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
954		Línea en blanco: separa visualmente bloques para que el código sea más legible.

955	labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/lista_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
956	panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(15, 20, 50, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
957		Línea en blanco: separa visualmente bloques para que el código sea más legible.
958	panelFlotante1.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
959		Línea en blanco: separa visualmente bloques para que el código sea más legible.
960	labelTitulo4.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
961	labelTitulo4.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
962	labelTitulo4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
963	labelTitulo4.setText("Productos totales");	Cambia el texto visible de un componente de la interfaz.
964	panelFlotante1.add(labelTitulo4, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
965		Línea en blanco: separa visualmente bloques para que el código sea más legible.
966	labelTotalProductos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
967	labelTotalProductos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
968	labelTotalProductos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
969	labelTotalProductos.setText("48");	Cambia el texto visible de un componente de la interfaz.
970	panelFlotante1.add(labelTotalProductos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
971		Línea en blanco: separa visualmente bloques para que el código sea más legible.
972	labelTitulo6.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
973	labelTitulo6.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
974	labelTitulo6.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
975	labelTitulo6.setText("En el inventario");	Cambia el texto visible de un componente de la interfaz.
976	panelFlotante1.add(labelTitulo6, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
977		Línea en blanco: separa visualmente bloques para que el código sea más legible.
978	add(panelFlotante1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
979		Línea en blanco: separa visualmente bloques para que el código sea más legible.
980	panelFlotante2.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
981	panelFlotante2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
982		Línea en blanco: separa visualmente bloques para que el código sea más legible.
983	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
984	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
985		Línea en blanco: separa visualmente bloques para que el código sea más legible.
986	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/advertencia_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
987	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(19, 22, 43, 43));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
988		Línea en blanco: separa visualmente bloques para que el código sea más legible.
989	panelFlotante2.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
990		Línea en blanco: separa visualmente bloques para que el código sea más legible.
991	labelTitulo7.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
992	labelTitulo7.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
993	labelTitulo7.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
994	labelTitulo7.setText("Stock bajo");	Cambia el texto visible de un componente de la interfaz.
995	panelFlotante2.add(labelTitulo7, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
996		Línea en blanco: separa visualmente bloques para que el código sea más legible.

997	labelStockBajo.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
998	labelStockBajo.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
999	labelStockBajo.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1000	labelStockBajo.setText("48");	Cambia el texto visible de un componente de la interfaz.
1001	panelFlotante2.add(labelStockBajo, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1002		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1003	labelTitulo9.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1004	labelTitulo9.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1005	labelTitulo9.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1006	labelTitulo9.setText("Requieren reposición");	Cambia el texto visible de un componente de la interfaz.
1007	panelFlotante2.add(labelTitulo9, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1008		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1009	add(panelFlotante2, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1010		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1011	panelFlotante4.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
1012	panelFlotante4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1013		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1014	panelCircular5.setColorFondo(new java.awt.Color(230, 245, 234));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1015	panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1016		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1017	labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/cheque.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
1018	panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(18, 23, 45, 45));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1019		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1020	panelFlotante4.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1021		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1022	labelTitulo13.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1023	labelTitulo13.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1024	labelTitulo13.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1025	labelTitulo13.setText("Activos");	Cambia el texto visible de un componente de la interfaz.
1026	panelFlotante4.add(labelTitulo13, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1027		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1028	labelActivos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1029	labelActivos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1030	labelActivos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1031	labelActivos.setText("48");	Cambia el texto visible de un componente de la interfaz.
1032	panelFlotante4.add(labelActivos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1033		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1034	labelTitulo15.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1035	labelTitulo15.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1036	labelTitulo15.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1037	labelTitulo15.setText("Disponible para uso");	Cambia el texto visible de un componente de la interfaz.
1038	panelFlotante4.add(labelTitulo15, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1039		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1040	add(panelFlotante4, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

1041		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1042	panelFlotante3.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
1043	panelFlotante3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1044		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1045	panelCircular4.setColorFondo(new java.awt.Color(237, 239, 243));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1046	panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1047		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1048	labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/x_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
1049	panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(23, 27, 35, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1050		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1051	panelFlotante3.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1052		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1053	labelTitulo10.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1054	labelTitulo10.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1055	labelTitulo10.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1056	labelTitulo10.setText("Inactivos");	Cambia el texto visible de un componente de la interfaz.
1057	panelFlotante3.add(labelTitulo10, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1058		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1059	labelInactivos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1060	labelInactivos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1061	labelInactivos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1062	labelInactivos.setText("48");	Cambia el texto visible de un componente de la interfaz.
1063	panelFlotante3.add(labelInactivos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1064		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1065	labelTitulo12.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1066	labelTitulo12.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1067	labelTitulo12.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1068	labelTitulo12.setText("No disponibles");	Cambia el texto visible de un componente de la interfaz.
1069	panelFlotante3.add(labelTitulo12, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1070		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1071	add(panelFlotante3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1072		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1073	botonExportar.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1074	botonExportar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1075	botonExportar.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1076	botonExportar.setRadio(20);	Configura una propiedad mediante setRadio en el objeto indicado.
1077	botonExportar.setText("Exportar");	Cambia el texto visible de un componente de la interfaz.
1078	botonExportar.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1079	botonExportar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1080	botonExportar.setFont(new java.awt.Font("Arial", 1, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1081	botonExportar.setGrosorBorde(1.6F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1082	botonExportar.setMargin(new java.awt.Insets(5, 14, 3, 14));	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
1083	add(botonExportar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1335, 45, 175, 54));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1084		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1085	panelFiltros.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1086	panelFiltros.setColorBorde(new java.awt.Color(225, 230, 237));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1087	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
1088	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
1089	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1090		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1091	campoBusqueda.setPlaceholder("Buscar producto o ingrediente...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
1092	campoBusqueda.setFont(new java.awt.Font("Dialog", 0, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1093	campoBusqueda.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
1094	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1095	campoBusquedaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1096	}	Cierra el bloque abierto anteriormente.
1097	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1098	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 20, 420, 55));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1099		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1100	filtroCategoria.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1101	filtroCategoria.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1102	filtroCategoria.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1103	filtroCategoria.setText("Todos los tipos");	Cambia el texto visible de un componente de la interfaz.
1104	filtroCategoria.setTextoDesplegable("Todos los tipos;Ingredientes;Productos directos");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
1105	filtroCategoria.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
1106	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1107	filtroCategoriaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1108	}	Cierra el bloque abierto anteriormente.
1109	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1110	panelFiltros.add(filtroCategoria, new org.netbeans.lib.awtextra.AbsoluteConstraints(460, 22, 280, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1112	filtroEstado.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1113	filtroEstado.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1114	filtroEstado.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1115	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
1116	filtroEstado.setTextoDesplegable("Todos los estados;Disponible;Stock bajo;Agotado;Inactivo");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
1117	panelFiltros.add(filtroEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(760, 22, 280, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1119	botonLimpiarFiltros.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1120	botonLimpiarFiltros.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1121	botonLimpiarFiltros.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1122	botonLimpiarFiltros.setText("Limpiar filtros");	Cambia el texto visible de un componente de la interfaz.
1123	botonLimpiarFiltros.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1124	botonLimpiarFiltros.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1125	botonLimpiarFiltros.setFont(new java.awt.Font("Arial", 1, 15)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
1126	botonLimpiarFiltros.setGrosorBorde(1.6F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1127	botonLimpiarFiltros.setMargin(new java.awt.Insets(5, 14, 3, 14));	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
1128	panelFiltros.add(botonLimpiarFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(1060, 22, 380, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

1129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1130	<code>add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 175, 1470, 95));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
1131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1132	<code>panelTabla.setColorFondo(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
1133	<code>panelTabla.setColorBorde(new java.awt.Color(225, 230, 237));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
1134	<code>panelTabla.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
1135	<code>panelTabla.setSombra(false);</code>	Configura una propiedad mediante <code>setSombra</code> en el objeto indicado.
1136	<code>panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
1137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1138	<code>scrollUsuarios.setBorder(null);</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
1139	<code>scrollUsuarios.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setVerticalScrollBarPolicy</code> en el objeto indicado.
1140	<code>scrollUsuarios.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setHorizontalScrollBarPolicy</code> en el objeto indicado.
1141	<code>scrollUsuarios.setAutoscrolls(true);</code>	Configura una propiedad mediante <code>setAutoscrolls</code> en el objeto indicado.
1142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1143	<code>tablaProductos.setModel(new javax.swing.table.DefaultTableModel(</code>	Crea una nueva instancia de <code>javax.swing.table.DefaultTableModel</code> y la usa o guarda para continuar el proceso.
1144	<code>new Object [][] {</code>	Crea una nueva instancia de <code>Object</code> y la usa o guarda para continuar el proceso.
1145	<code>},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1146	<code>new String [] {</code>	Crea una nueva instancia de <code>String</code> y la usa o guarda para continuar el proceso.
1147	<code>"ID", "Producto / ingrediente", "Tipo", "Categoría", "Unidad", "Stock actual", "Stock mínimo", "Último movimiento", "Estado", "Acción"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1148	<code>}</code>	Cierra el bloque abierto anteriormente.
1149	<code>) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1150	<code>boolean[] canEdit = new boolean [] {</code>	Crea una nueva instancia de <code>boolean</code> y la usa o guarda para continuar el proceso.
1151	<code>false, false, false, false, false, false, false, false, false, false</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1152	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
1153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1154	<code>public boolean isCellEditable(int rowIndex, int columnIndex) {</code>	Declara el método <code>isCellEditable</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1155	<code>return canEdit [columnIndex];</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1156	<code>}</code>	Cierra el bloque abierto anteriormente.
1157	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1158	<code>tablaProductos.setAutoscrolls(false);</code>	Configura una propiedad mediante <code>setAutoscrolls</code> en el objeto indicado.
1159	<code>tablaProductos.setColumnsCentradas("0,2,4,5,6,8,9");</code>	Configura una propiedad mediante <code>setColumnsCentradas</code> en el objeto indicado.
1160	<code>scrollUsuarios.setViewportView(tablaProductos);</code>	Configura una propiedad mediante <code>setViewportView</code> en el objeto indicado.
1161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1162	<code>panelTabla.add(scrollUsuarios, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 555));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
1163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1164	<code>etiquetaRango.setForeground(new java.awt.Color(92, 103, 124));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
1165	<code>etiquetaRango.setText("Mostrando inventario");</code>	Cambia el texto visible de un componente de la interfaz.
1166	<code>panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 581, 420, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
1167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1168	<code>panelPaginacion.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
1169	<code>panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
1170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1171	<code>botonAnterior.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
1172	<code>botonAnterior.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.

1173	botonAnterior.setText("<");	Cambia el texto visible de un componente de la interfaz.
1174	panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1175		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1176	botonPagina1.setColorInicio(new java.awt.Color(255, 188, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1177	botonPagina1.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1178	botonPagina1.setText("1");	Cambia el texto visible de un componente de la interfaz.
1179	panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1181	botonPagina2.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1182	botonPagina2.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1183	botonPagina2.setText("2");	Cambia el texto visible de un componente de la interfaz.
1184	panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1185		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1186	botonPagina3.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1187	botonPagina3.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1188	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
1189	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1191	botonSiguiente.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
1192	botonSiguiente.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1193	botonSiguiente.setText(">");	Cambia el texto visible de un componente de la interfaz.
1194	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1196	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 581, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1198	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 285, 1470, 640));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
1199	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1201	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_campoBusquedaAc tionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1202	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
1203	}//GEN-LAST:event_campoBusquedaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1204		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1205	private void filtroCategoriaActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_filtroCategor iaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1206	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
1207	}//GEN-LAST:event_filtroCategoriaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1208		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1210	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1211	private Componentes.BotonDerretido botonAgregarProducto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1212	private Componentes.BotonRedondeado botonAnterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1213	private Componentes.BotonRedondeado botonExportar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1214	private Componentes.BotonRedondeado botonLimpiarFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1215	private Componentes.BotonRedondeado botonMovimientos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1216	private Componentes.BotonRedondeado botonPagina1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1217	private Componentes.BotonRedondeado botonPagina2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1218	private Componentes.BotonRedondeado botonPagina3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1219	private Componentes.BotonRedondeado botonRegistrarSalida;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1220	private Componentes.BotonRedondeado botonSiguiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1221	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1222	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1223	private Componentes.BotonDesplegable filtroCategoria;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1224	private Componentes.BotonDesplegable filtroEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1225	private javax.swing.JLabel labelActivos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1226	private Labels.LabelEscalable labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1227	private Labels.LabelEscalable labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1228	private Labels.LabelEscalable labelEscalable4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1229	private Labels.LabelEscalable labelEscalable5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1230	private javax.swing.JLabel labelInactivos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1231	private javax.swing.JLabel labelStockBajo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1232	private javax.swing.JLabel labelTitulo10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1233	private javax.swing.JLabel labelTitulo12;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1234	private javax.swing.JLabel labelTitulo13;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1235	private javax.swing.JLabel labelTitulo15;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1236	private javax.swing.JLabel labelTitulo2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1237	private javax.swing.JLabel labelTitulo3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1238	private javax.swing.JLabel labelTitulo4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1239	private javax.swing.JLabel labelTitulo6;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1240	private javax.swing.JLabel labelTitulo7;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1241	private javax.swing.JLabel labelTitulo9;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1242	private javax.swing.JLabel labelTotalProductos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1243	private Componentes.PanelCircular panelCircular2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1244	private Componentes.PanelCircular panelCircular3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1245	private Componentes.PanelCircular panelCircular4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1246	private Componentes.PanelCircular panelCircular5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1247	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1248	private Componentes.PanelFlotante panelFlotante1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1249	private Componentes.PanelFlotante panelFlotante2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1250	private Componentes.PanelFlotante panelFlotante3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1251	private Componentes.PanelFlotante panelFlotante4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1252	private javax.swing.JPanel panelPaginacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1253	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1254	private javax.swing.JScrollPane scrollUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1255	private Componentes.TablaAdministrativa tablaProductos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1256	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1257	}	Cierra el bloque abierto anteriormente.

16.62 src/GUI_ADMINISTRADOR/GestionMenuPanel.form

Metadatos visuales de NetBeans para diseñar GestionMenuPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 739 | Archivo 62 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,-44,0,0,6,44"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="javax.swing.JLabel" name="labelTitulo3">	Declara un componente o contenedor visual y su clase Java.
27	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
29		Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	</Property>	Cierra la sección XML abierta anteriormente.
31	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	<Property name="text" type="java.lang.String" value="Gestión del Menú"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
36	</Properties>	Cierra la sección XML abierta anteriormente.
37	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

38	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
39	<AbsoluteConstraints x="70" y="30" width="460" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
40	</Constraint>	Cierra la sección XML abierta anteriormente.
41	</Constraints>	Cierra la sección XML abierta anteriormente.
42	</Component>	Cierra la sección XML abierta anteriormente.
43	<Component class="javax.swing.JLabel" name="labelTitulo2">	Declara un componente o contenedor visual y su clase Java.
44	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
45	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
46		Metadato de NetBeans que conserva el estado editable del diseñador visual.
47	</Property>	Cierra la sección XML abierta anteriormente.
48	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
49	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
50	</Property>	Cierra la sección XML abierta anteriormente.
51	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	<Property name="text" type="java.lang.String" value="Administra productos y combos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
53	</Properties>	Cierra la sección XML abierta anteriormente.
54	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
55	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
56	<AbsoluteConstraints x="70" y="80" width="320" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
57	</Constraint>	Cierra la sección XML abierta anteriormente.
58	</Constraints>	Cierra la sección XML abierta anteriormente.
59	</Component>	Cierra la sección XML abierta anteriormente.
60	<Component class="Componentes.BotonDerretido" name="botonAgregarProducto">	Declara un componente o contenedor visual y su clase Java.
61	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
62	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
63	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	</Property>	Cierra la sección XML abierta anteriormente.
65	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
66	<Image iconType="3" name="/Imagenes/icono_agregar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	</Property>	Cierra la sección XML abierta anteriormente.
68	<Property name="text" type="java.lang.String" value="Agregar producto"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
69	<Property name="iconTextGap" type="int" value="15"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
72	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
73	<AbsoluteConstraints x="1040" y="60" width="230" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
74	</Constraint>	Cierra la sección XML abierta anteriormente.
75	</Constraints>	Cierra la sección XML abierta anteriormente.
76	</Component>	Cierra la sección XML abierta anteriormente.
77	<Container class="Componentes.PanelFlotante" name="panelFlotante1">	Declara un componente o contenedor visual y su clase Java.
78	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
79	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

80	<AbsoluteConstraints x="60" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
81	</Constraint>	Cierra la sección XML abierta anteriormente.
82	</Constraints>	Cierra la sección XML abierta anteriormente.
83		Línea en blanco para organizar el XML del formulario.
84	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
85	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	</Layout>	Cierra la sección XML abierta anteriormente.
87	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
88	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
89	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
90	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
91	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
92	</Constraint>	Cierra la sección XML abierta anteriormente.
93	</Constraints>	Cierra la sección XML abierta anteriormente.
94		Línea en blanco para organizar el XML del formulario.
95	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
96	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
97	</Layout>	Cierra la sección XML abierta anteriormente.
98	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
99	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
100	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
101	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102	<Image iconType="3" name="/Imagenes/lista_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	</Properties>	Cierra la sección XML abierta anteriormente.
105	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
106	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
107	<AbsoluteConstraints x="15" y="20" width="50" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
108	</Constraint>	Cierra la sección XML abierta anteriormente.
109	</Constraints>	Cierra la sección XML abierta anteriormente.
110	</Component>	Cierra la sección XML abierta anteriormente.
111	</SubComponents>	Cierra la sección XML abierta anteriormente.
112	</Container>	Cierra la sección XML abierta anteriormente.
113	<Component class="javax.swing.JLabel" name="labelTitulo4">	Declara un componente o contenedor visual y su clase Java.
114	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
115	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
116		Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	</Property>	Cierra la sección XML abierta anteriormente.
118	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
119	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
120	</Property>	Cierra la sección XML abierta anteriormente.
121	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

122	<Property name="text" type="java.lang.String" value="Productos totales"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
123	</Properties>	Cierra la sección XML abierta anteriormente.
124	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
125	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
126	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
127	</Constraint>	Cierra la sección XML abierta anteriormente.
128	</Constraints>	Cierra la sección XML abierta anteriormente.
129	</Component>	Cierra la sección XML abierta anteriormente.
130	<Component class="javax.swing.JLabel" name="labelTotalProductos">	Declara un componente o contenedor visual y su clase Java.
131	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
132	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
133		Metadato de NetBeans que conserva el estado editable del diseñador visual.
134	</Property>	Cierra la sección XML abierta anteriormente.
135	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
136	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
137	</Property>	Cierra la sección XML abierta anteriormente.
138	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
139	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
140	</Properties>	Cierra la sección XML abierta anteriormente.
141	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
142	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
143	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
144	</Constraint>	Cierra la sección XML abierta anteriormente.
145	</Constraints>	Cierra la sección XML abierta anteriormente.
146	</Component>	Cierra la sección XML abierta anteriormente.
147	<Component class="javax.swing.JLabel" name="labelTitulo6">	Declara un componente o contenedor visual y su clase Java.
148	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
149	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
150		Metadato de NetBeans que conserva el estado editable del diseñador visual.
151	</Property>	Cierra la sección XML abierta anteriormente.
152	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
153	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
154	</Property>	Cierra la sección XML abierta anteriormente.
155	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
156	<Property name="text" type="java.lang.String" value="En el inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
157	</Properties>	Cierra la sección XML abierta anteriormente.
158	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
159	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
160	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
161	</Constraint>	Cierra la sección XML abierta anteriormente.
162	</Constraints>	Cierra la sección XML abierta anteriormente.

163	</Component>	Cierra la sección XML abierta anteriormente.
164	</SubComponents>	Cierra la sección XML abierta anteriormente.
165	</Container>	Cierra la sección XML abierta anteriormente.
166	<Container class="Componentes.PanelFlotante" name="panelFlotante2">	Declara un componente o contenedor visual y su clase Java.
167	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
168	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
169	<AbsoluteConstraints x="430" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
170	</Constraint>	Cierra la sección XML abierta anteriormente.
171	</Constraints>	Cierra la sección XML abierta anteriormente.
172		Línea en blanco para organizar el XML del formulario.
173	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
174	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
175	</Layout>	Cierra la sección XML abierta anteriormente.
176	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
177	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
178	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
179	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
180	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
181	</Property>	Cierra la sección XML abierta anteriormente.
182	</Properties>	Cierra la sección XML abierta anteriormente.
183	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
184	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
185	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
186	</Constraint>	Cierra la sección XML abierta anteriormente.
187	</Constraints>	Cierra la sección XML abierta anteriormente.
188		Línea en blanco para organizar el XML del formulario.
189	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
190	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
191	</Layout>	Cierra la sección XML abierta anteriormente.
192	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
193	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
194	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
195	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
196	<Image iconType="3" name="/Imágenes/advertencia_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
197	</Property>	Cierra la sección XML abierta anteriormente.
198	</Properties>	Cierra la sección XML abierta anteriormente.
199	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
200	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
201	<AbsoluteConstraints x="19" y="22" width="43" height="43"/>	Define la posición y el tamaño del componente dentro del diseño.
202	</Constraint>	Cierra la sección XML abierta anteriormente.
203	</Constraints>	Cierra la sección XML abierta anteriormente.

204	</Component>	Cierra la sección XML abierta anteriormente.
205	</SubComponents>	Cierra la sección XML abierta anteriormente.
206	</Container>	Cierra la sección XML abierta anteriormente.
207	<Component class="javax.swing.JLabel" name="labelTitulo7">	Declara un componente o contenedor visual y su clase Java.
208	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
209	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
210		Metadato de NetBeans que conserva el estado editable del diseñador visual.
211	</Property>	Cierra la sección XML abierta anteriormente.
212	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
213	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
214	</Property>	Cierra la sección XML abierta anteriormente.
215	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
216	<Property name="text" type="java.lang.String" value="Stock bajo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
217	</Properties>	Cierra la sección XML abierta anteriormente.
218	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
219	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
220	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
221	</Constraint>	Cierra la sección XML abierta anteriormente.
222	</Constraints>	Cierra la sección XML abierta anteriormente.
223	</Component>	Cierra la sección XML abierta anteriormente.
224	<Component class="javax.swing.JLabel" name="labelStockBajo">	Declara un componente o contenedor visual y su clase Java.
225	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
226	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
227		Metadato de NetBeans que conserva el estado editable del diseñador visual.
228	</Property>	Cierra la sección XML abierta anteriormente.
229	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
230	<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
231	</Property>	Cierra la sección XML abierta anteriormente.
232	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
233	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
234	</Properties>	Cierra la sección XML abierta anteriormente.
235	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
236	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
237	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
238	</Constraint>	Cierra la sección XML abierta anteriormente.
239	</Constraints>	Cierra la sección XML abierta anteriormente.
240	</Component>	Cierra la sección XML abierta anteriormente.
241	<Component class="javax.swing.JLabel" name="labelTitulo9">	Declara un componente o contenedor visual y su clase Java.
242	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
243	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
244		Metadato de NetBeans que conserva el estado editable del diseñador visual.
245	</Property>	Cierra la sección XML abierta anteriormente.

246	>	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
247		<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
248		</Property>	Cierra la sección XML abierta anteriormente.
249		<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
250		<Property name="text" type="java.lang.String" value="Requieren reposición"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251		</Properties>	Cierra la sección XML abierta anteriormente.
252		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
253		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
254		<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
255		</Constraint>	Cierra la sección XML abierta anteriormente.
256		</Constraints>	Cierra la sección XML abierta anteriormente.
257		</Component>	Cierra la sección XML abierta anteriormente.
258		</SubComponents>	Cierra la sección XML abierta anteriormente.
259		</Container>	Cierra la sección XML abierta anteriormente.
260		<Container class="Componentes.PanelFlotante" name="panelFlotante4">	Declara un componente o contenedor visual y su clase Java.
261		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
262		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
263		<AbsoluteConstraints x="800" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
264		</Constraint>	Cierra la sección XML abierta anteriormente.
265		</Constraints>	Cierra la sección XML abierta anteriormente.
266			Línea en blanco para organizar el XML del formulario.
267		<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
268		<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
269		</Layout>	Cierra la sección XML abierta anteriormente.
270		<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
271		<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
272		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
273	>	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
274		<Color blue="ea" green="f5" red="e6" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
275		</Property>	Cierra la sección XML abierta anteriormente.
276		</Properties>	Cierra la sección XML abierta anteriormente.
277		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
278		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
279		<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
280		</Constraint>	Cierra la sección XML abierta anteriormente.
281		</Constraints>	Cierra la sección XML abierta anteriormente.
282			Línea en blanco para organizar el XML del formulario.
283		<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
284		<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
285		</Layout>	Cierra la sección XML abierta anteriormente.
286		<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

287	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
288	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
289	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
290	<Image iconType="3" name="/Imagenes/cheque.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
291	</Property>	Cierra la sección XML abierta anteriormente.
292	</Properties>	Cierra la sección XML abierta anteriormente.
293	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
294	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
295	<AbsoluteConstraints x="18" y="23" width="45" height="45"/>	Define la posición y el tamaño del componente dentro del diseño.
296	</Constraint>	Cierra la sección XML abierta anteriormente.
297	</Constraints>	Cierra la sección XML abierta anteriormente.
298	</Component>	Cierra la sección XML abierta anteriormente.
299	</SubComponents>	Cierra la sección XML abierta anteriormente.
300	</Container>	Cierra la sección XML abierta anteriormente.
301	<Component class="javax.swing.JLabel" name="labelTitulo13">	Declara un componente o contenedor visual y su clase Java.
302	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
303	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
304		Metadato de NetBeans que conserva el estado editable del diseñador visual.
305	</Property>	Cierra la sección XML abierta anteriormente.
306	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
307	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
308	</Property>	Cierra la sección XML abierta anteriormente.
309	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
310	<Property name="text" type="java.lang.String" value="Activos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
311	</Properties>	Cierra la sección XML abierta anteriormente.
312	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
313	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
314	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
315	</Constraint>	Cierra la sección XML abierta anteriormente.
316	</Constraints>	Cierra la sección XML abierta anteriormente.
317	</Component>	Cierra la sección XML abierta anteriormente.
318	<Component class="javax.swing.JLabel" name="labelActivos">	Declara un componente o contenedor visual y su clase Java.
319	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
320	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
321		Metadato de NetBeans que conserva el estado editable del diseñador visual.
322	</Property>	Cierra la sección XML abierta anteriormente.
323	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
324	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
325	</Property>	Cierra la sección XML abierta anteriormente.
326	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
327	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

328	</Properties>	Cierra la sección XML abierta anteriormente.
329	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
330	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
331	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
332	</Constraint>	Cierra la sección XML abierta anteriormente.
333	</Constraints>	Cierra la sección XML abierta anteriormente.
334	</Component>	Cierra la sección XML abierta anteriormente.
335	<Component class="javax.swing.JLabel" name="labelTitulo15">	Declara un componente o contenedor visual y su clase Java.
336	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
337	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
338		Metadato de NetBeans que conserva el estado editable del diseñador visual.
339	</Property>	Cierra la sección XML abierta anteriormente.
340	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
341	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
342	</Property>	Cierra la sección XML abierta anteriormente.
343	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
344	<Property name="text" type="java.lang.String" value="Disponible para uso"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
345	</Properties>	Cierra la sección XML abierta anteriormente.
346	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
347	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
348	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
349	</Constraint>	Cierra la sección XML abierta anteriormente.
350	</Constraints>	Cierra la sección XML abierta anteriormente.
351	</Component>	Cierra la sección XML abierta anteriormente.
352	</SubComponents>	Cierra la sección XML abierta anteriormente.
353	</Container>	Cierra la sección XML abierta anteriormente.
354	<Container class="Componentes.PanelFlotante" name="panelFlotante3">	Declara un componente o contenedor visual y su clase Java.
355	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
356	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
357	<AbsoluteConstraints x="1170" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
358	</Constraint>	Cierra la sección XML abierta anteriormente.
359	</Constraints>	Cierra la sección XML abierta anteriormente.
360		Línea en blanco para organizar el XML del formulario.
361	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
362	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
363	</Layout>	Cierra la sección XML abierta anteriormente.
364	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
365	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
366	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
367	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
368	<Color blue="f3" green="ef" red="ed" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

369	</Property>	Cierra la sección XML abierta anteriormente.
370	</Properties>	Cierra la sección XML abierta anteriormente.
371	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
372	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
373	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
374	</Constraint>	Cierra la sección XML abierta anteriormente.
375	</Constraints>	Cierra la sección XML abierta anteriormente.
376		Línea en blanco para organizar el XML del formulario.
377	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
378	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
379	</Layout>	Cierra la sección XML abierta anteriormente.
380	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
381	<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
382	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
383	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
384	<Image iconType="3" name="/Imagenes/x_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
385	</Property>	Cierra la sección XML abierta anteriormente.
386	</Properties>	Cierra la sección XML abierta anteriormente.
387	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
388	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
389	<AbsoluteConstraints x="23" y="27" width="35" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
390	</Constraint>	Cierra la sección XML abierta anteriormente.
391	</Constraints>	Cierra la sección XML abierta anteriormente.
392	</Component>	Cierra la sección XML abierta anteriormente.
393	</SubComponents>	Cierra la sección XML abierta anteriormente.
394	</Container>	Cierra la sección XML abierta anteriormente.
395	<Component class="javax.swing.JLabel" name="labelTitulo10">	Declara un componente o contenedor visual y su clase Java.
396	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
397	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
398		Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	</Property>	Cierra la sección XML abierta anteriormente.
400	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
401	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
402	</Property>	Cierra la sección XML abierta anteriormente.
403	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
404	<Property name="text" type="java.lang.String" value="Inactivos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
405	</Properties>	Cierra la sección XML abierta anteriormente.
406	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
407	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
408	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
409	</Constraint>	Cierra la sección XML abierta anteriormente.

410	</Constraints>	Cierra la sección XML abierta anteriormente.
411	</Component>	Cierra la sección XML abierta anteriormente.
412	<Component class="javax.swing.JLabel" name="labelInactivos">	Declara un componente o contenedor visual y su clase Java.
413	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
414	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
415		Metadato de NetBeans que conserva el estado editable del diseñador visual.
416	</Property>	Cierra la sección XML abierta anteriormente.
417	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
418	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
419	</Property>	Cierra la sección XML abierta anteriormente.
420	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
421	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
422	</Properties>	Cierra la sección XML abierta anteriormente.
423	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
424	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
425	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
426	</Constraint>	Cierra la sección XML abierta anteriormente.
427	</Constraints>	Cierra la sección XML abierta anteriormente.
428	</Component>	Cierra la sección XML abierta anteriormente.
429	<Component class="javax.swing.JLabel" name="labelTitulo12">	Declara un componente o contenedor visual y su clase Java.
430	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
431	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
432		Metadato de NetBeans que conserva el estado editable del diseñador visual.
433	</Property>	Cierra la sección XML abierta anteriormente.
434	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
435	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
436	</Property>	Cierra la sección XML abierta anteriormente.
437	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
438	<Property name="text" type="java.lang.String" value="No disponibles"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
439	</Properties>	Cierra la sección XML abierta anteriormente.
440	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
441	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
442	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
443	</Constraint>	Cierra la sección XML abierta anteriormente.
444	</Constraints>	Cierra la sección XML abierta anteriormente.
445	</Component>	Cierra la sección XML abierta anteriormente.
446	</SubComponents>	Cierra la sección XML abierta anteriormente.
447	</Container>	Cierra la sección XML abierta anteriormente.
448	<Component class="Componentes.BotonRedondeado" name="botonExportar">	Declara un componente o contenedor visual y su clase Java.
449	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
450	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
451	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

452	</Property>	Cierra la sección XML abierta anteriormente.
453	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
454	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
455	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
456	</Property>	Cierra la sección XML abierta anteriormente.
457	<Property name="radio" type="int" value="20"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
458	<Property name="text" type="java.lang.String" value="Exportar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
459	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
460	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
461	</Property>	Cierra la sección XML abierta anteriormente.
462	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
463	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
464	</Property>	Cierra la sección XML abierta anteriormente.
465	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
466		Metadato de NetBeans que conserva el estado editable del diseñador visual.
467	</Property>	Cierra la sección XML abierta anteriormente.
468	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
469	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
470	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
471	</Property>	Cierra la sección XML abierta anteriormente.
472	</Properties>	Cierra la sección XML abierta anteriormente.
473	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
474	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
475	<AbsoluteConstraints x="1290" y="60" width="230" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
476	</Constraint>	Cierra la sección XML abierta anteriormente.
477	</Constraints>	Cierra la sección XML abierta anteriormente.
478	</Component>	Cierra la sección XML abierta anteriormente.
479	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
480	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
481	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
482	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
483	</Properties>	Cierra la sección XML abierta anteriormente.
484	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
485	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
486	<AbsoluteConstraints x="60" y="320" width="1470" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
487	</Constraint>	Cierra la sección XML abierta anteriormente.
488	</Constraints>	Cierra la sección XML abierta anteriormente.
489		Línea en blanco para organizar el XML del formulario.
490	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
491	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
492	</Layout>	Cierra la sección XML abierta anteriormente.
493	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
494	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.

495	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
496	<Property name="placeholder" type="java.lang.String" value="Buscar producto o categor-a..." />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
497	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
498		Metadato de NetBeans que conserva el estado editable del diseñador visual.
499	</Property>	Cierra la sección XML abierta anteriormente.
500	</Properties>	Cierra la sección XML abierta anteriormente.
501	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
502	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="campoBusquedaActionPerformed" />	Asocia un evento visual con el método Java que lo atenderá.
503	</Events>	Cierra la sección XML abierta anteriormente.
504	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
505	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
506	<AbsoluteConstraints x="20" y="22" width="455" height="56" />	Define la posición y el tamaño del componente dentro del diseño.
507	</Constraint>	Cierra la sección XML abierta anteriormente.
508	</Constraints>	Cierra la sección XML abierta anteriormente.
509	</Component>	Cierra la sección XML abierta anteriormente.
510	<Component class="Componentes.BotonDesplegable" name="filtroCategoria">	Declara un componente o contenedor visual y su clase Java.
511	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
512	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
513	<Color blue="ff" green="ff" red="ff" type="rgb" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
514	</Property>	Cierra la sección XML abierta anteriormente.
515	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
516	<Color blue="fb" green="f9" red="f8" type="rgb" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
517	</Property>	Cierra la sección XML abierta anteriormente.
518	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
519	<Color blue="2b" green="14" red="0" type="rgb" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
520	</Property>	Cierra la sección XML abierta anteriormente.
521	<Property name="text" type="java.lang.String" value="Todas las categor-as" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
522	<Property name="textoDesplegable" type="java.lang.String" value="Todos los roles;ADMINISTRADOR;CAJERO" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
523	</Properties>	Cierra la sección XML abierta anteriormente.
524	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
525	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="filtroCategoriaActionPerformed" />	Asocia un evento visual con el método Java que lo atenderá.
526	</Events>	Cierra la sección XML abierta anteriormente.
527	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
528	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
529	<AbsoluteConstraints x="495" y="24" width="330" height="52" />	Define la posición y el tamaño del componente dentro del diseño.
530	</Constraint>	Cierra la sección XML abierta anteriormente.
531	</Constraints>	Cierra la sección XML abierta anteriormente.
532	</Component>	Cierra la sección XML abierta anteriormente.
533	<Component class="Componentes.BotonDesplegable" name="filtroEstado">	Declara un componente o contenedor visual y su clase Java.

534	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
535	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
536	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
537	</Property>	Cierra la sección XML abierta anteriormente.
538	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
539	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
540	</Property>	Cierra la sección XML abierta anteriormente.
541	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
542	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
543	</Property>	Cierra la sección XML abierta anteriormente.
544	<Property name="text" type="java.lang.String" value="Todos los estados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
545	<Property name="textoDesplegable" type="java.lang.String" value="Todos los estados;Activo;Inactivo"/	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
546	</Properties>	Cierra la sección XML abierta anteriormente.
547	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
548	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
549	<AbsoluteConstraints x="850" y="24" width="330" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
550	</Constraint>	Cierra la sección XML abierta anteriormente.
551	</Constraints>	Cierra la sección XML abierta anteriormente.
552	</Component>	Cierra la sección XML abierta anteriormente.
553	<Component class="Componentes.BotonRedondeado" name="botonLimpiarFiltros">	Declara un componente o contenedor visual y su clase Java.
554	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
555	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
">		
556	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
557	</Property>	Cierra la sección XML abierta anteriormente.
558	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
559	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
560	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
561	</Property>	Cierra la sección XML abierta anteriormente.
562	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
563	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
564	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
565	</Property>	Cierra la sección XML abierta anteriormente.
566	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
567	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
568	</Property>	Cierra la sección XML abierta anteriormente.
569	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
570		Metadato de NetBeans que conserva el estado editable del diseñador visual.
571	</Property>	Cierra la sección XML abierta anteriormente.

572	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
573	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
574	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
575	</Property>	Cierra la sección XML abierta anteriormente.
576	</Properties>	Cierra la sección XML abierta anteriormente.
577	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
578	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
579	<AbsoluteConstraints x="1210" y="24" width="230" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
580	</Constraint>	Cierra la sección XML abierta anteriormente.
581	</Constraints>	Cierra la sección XML abierta anteriormente.
582	</Component>	Cierra la sección XML abierta anteriormente.
583	</SubComponents>	Cierra la sección XML abierta anteriormente.
584	</Container>	Cierra la sección XML abierta anteriormente.
585	<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
586	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
587	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
588	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
589	</Properties>	Cierra la sección XML abierta anteriormente.
590	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
591	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
592	<AbsoluteConstraints x="60" y="420" width="1470" height="510"/>	Define la posición y el tamaño del componente dentro del diseño.
593	</Constraint>	Cierra la sección XML abierta anteriormente.
594	</Constraints>	Cierra la sección XML abierta anteriormente.
595		Línea en blanco para organizar el XML del formulario.
596	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
597	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
598	</Layout>	Cierra la sección XML abierta anteriormente.
599	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
600	<Container class="javax.swing.JScrollPane" name="scrollUsuarios">	Declara un componente o contenedor visual y su clase Java.
601	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
602	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
603	<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
604	</Property>	Cierra la sección XML abierta anteriormente.
605	<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
606	<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
607	<Property name="autoscrolls" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
608	</Properties>	Cierra la sección XML abierta anteriormente.
609	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
610	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
611	<AbsoluteConstraints x="14" y="14" width="1442" height="428"/>	Define la posición y el tamaño del componente dentro del diseño.
612	</Constraint>	Cierra la sección XML abierta anteriormente.
613	</Constraints>	Cierra la sección XML abierta anteriormente.

614		Línea en blanco para organizar el XML del formulario.
615	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
616	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
617	<Component class="Componentes.TablaAdministrativa" name="tablaProductos">	Declara un componente o contenedor visual y su clase Java.
618	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
619	<Property name="model" type="javax.swing.table.TableModel" editor="org.netbeans.modules.form.editors2.TableModelEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
620	<Table columnCount="8" rowCount="10">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
621	<Column editable="false" title="ID" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
622	<Column editable="false" title="Nombre" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
623	<Column editable="false" title="Usuario" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
624	<Column editable="false" title="Rol" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
625	<Column editable="false" title="Correo" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
626	<Column editable="false" title="Estado" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
627	<Column editable="false" title="Fecha creación" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
628	<Column editable="false" title="Acciones" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
629	</Table>	Cierra la sección XML abierta anteriormente.
630	</Property>	Cierra la sección XML abierta anteriormente.
631	<Property name="autoscrolls" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
632	<Property name="columnasCentradas" type="java.lang.String" value="5,7"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
633	</Properties>	Cierra la sección XML abierta anteriormente.
634	</Component>	Cierra la sección XML abierta anteriormente.
635	</SubComponents>	Cierra la sección XML abierta anteriormente.
636	</Container>	Cierra la sección XML abierta anteriormente.
637	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
638	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
639	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"> >	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
640	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
641	</Property>	Cierra la sección XML abierta anteriormente.
642	<Property name="text" type="java.lang.String" value="Mostrando 1–10 de 10 productos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
643	</Properties>	Cierra la sección XML abierta anteriormente.
644	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
645	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
646	<AbsoluteConstraints x="28" y="452" width="330" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
647	</Constraint>	Cierra la sección XML abierta anteriormente.
648	</Constraints>	Cierra la sección XML abierta anteriormente.
649	</Component>	Cierra la sección XML abierta anteriormente.
650	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
651	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
652	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
653	</Properties>	Cierra la sección XML abierta anteriormente.
654	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
655	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

656	<AbsoluteConstraints x="1165" y="452" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
657	</Constraint>	Cierra la sección XML abierta anteriormente.
658	</Constraints>	Cierra la sección XML abierta anteriormente.
659		Línea en blanco para organizar el XML del formulario.
660	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
661	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
662	</Layout>	Cierra la sección XML abierta anteriormente.
663	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
664	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
665	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
666	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
667	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
668	</Property>	Cierra la sección XML abierta anteriormente.
669	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
670	<Property name="text" type="java.lang.String" value="#x2039;/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
671	</Properties>	Cierra la sección XML abierta anteriormente.
672	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
673	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
674	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
675	</Constraint>	Cierra la sección XML abierta anteriormente.
676	</Constraints>	Cierra la sección XML abierta anteriormente.
677	</Component>	Cierra la sección XML abierta anteriormente.
678	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
679	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
680	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
681	<Color blue="0" green="bc" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
682	</Property>	Cierra la sección XML abierta anteriormente.
683	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
684	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
685	</Properties>	Cierra la sección XML abierta anteriormente.
686	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
687	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
688	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
689	</Constraint>	Cierra la sección XML abierta anteriormente.
690	</Constraints>	Cierra la sección XML abierta anteriormente.
691	</Component>	Cierra la sección XML abierta anteriormente.
692	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
693	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
694	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
695	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
696	</Property>	Cierra la sección XML abierta anteriormente.

697	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
698	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
699	</Properties>	Cierra la sección XML abierta anteriormente.
700	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
701	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
702	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
703	</Constraint>	Cierra la sección XML abierta anteriormente.
704	</Constraints>	Cierra la sección XML abierta anteriormente.
705	</Component>	Cierra la sección XML abierta anteriormente.
706	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
707	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
708	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
709	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
710	</Property>	Cierra la sección XML abierta anteriormente.
711	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
712	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
713	</Properties>	Cierra la sección XML abierta anteriormente.
714	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
715	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
716	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
717	</Constraint>	Cierra la sección XML abierta anteriormente.
718	</Constraints>	Cierra la sección XML abierta anteriormente.
719	</Component>	Cierra la sección XML abierta anteriormente.
720	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
721	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
722	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
723	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
724	</Property>	Cierra la sección XML abierta anteriormente.
725	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
726	<Property name="text" type="java.lang.String" value="#x203a"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
727	</Properties>	Cierra la sección XML abierta anteriormente.
728	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
729	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
730	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
731	</Constraint>	Cierra la sección XML abierta anteriormente.
732	</Constraints>	Cierra la sección XML abierta anteriormente.
733	</Component>	Cierra la sección XML abierta anteriormente.
734	</SubComponents>	Cierra la sección XML abierta anteriormente.
735	</Container>	Cierra la sección XML abierta anteriormente.
736	</SubComponents>	Cierra la sección XML abierta anteriormente.
737	</Container>	Cierra la sección XML abierta anteriormente.

738	</SubComponents>	Cierra la sección XML abierta anteriormente.
739	</Form>	Cierra la sección XML abierta anteriormente.

16.63 src/GUI_ADMINISTRADOR/GestionMenuPanel.java

Mantenimiento de productos del menú: filtros, resumen, alta, edición, estado y configuración comercial.

Tipo: java | Líneas: 1759 | Archivo 63 de 157

Clases/tipos detectados: GestionMenuPanel, ModeloTablaProductos, RenderProducto, RenderEstado, RenderAcciones, EditorAcciones, FondoOscuro

Línea	Constructor o método detectado
103	public GestionMenuPanel() {
126	private void agregarBotonConfiguracion() {
152	private void initComponents() {
357	public void actionPerformed(java.awt.event.ActionEvent evt) {
369	public void actionPerformed(java.awt.event.ActionEvent evt) {
424	public boolean isCellEditable(int rowIndex, int columnIndex) {
471	private void filtroCategoriaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroCategoriaActionPerformed
475	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_campoBusquedaActionPerformed
482	private void configurarApariencia() {
490	private void configurarFiltros() {
541	private void configurarSelector(BotonDesplegable selector, int ancho) {
557	private void configurarTabla() {
604	private void configurarBotonPagina(BotonRedondeado boton,
619	private void configurarEventos() {
622	public void insertUpdate(DocumentEvent e) {
627	public void removeUpdate(DocumentEvent e) {
632	public void changedUpdate(DocumentEvent e) {
638	private void cargarCategorias() {
641	protected Map<Integer, String> doInBackground()
647	protected void done() {
664	private void cargarResumen() {
667	protected ResumenProductos doInBackground()
673	protected void done() {
690	private void cargarDatos() {
700	protected PaginaProductos doInBackground()
707	protected void done() {
736	private void actualizarPaginacion(int totalRegistros) {
765	private void cambiarPagina(int pagina) {
773	private void mostrarErrorDatos(String operacion, Throwable causa) {

Línea	Constructor o método detectado
792	private Producto productoEn(int fila) { return productos.get(fila); }
794	private void setProductos(List<Producto> nuevos) {
800	public int getRowCount() {
805	public int getColumnCount() {
810	public String getColumnName(int c) {
815	public Class<?> getColumnClass(int c) {
820	public boolean isCellEditable(int r, int c) {
825	public Object getValueAt(int fila, int columna) {
848	private String nombreDisponibilidad(String valor) {
867	public Component getTableCellRendererComponent(
906	public Component getTableCellRendererComponent(
929	public Component getTableCellRendererComponent(
950	private final BotonRedondeado editar = crearBotonAccion(
952	private final BotonRedondeado desactivar = crearBotonAccion(
956	private EditorAcciones() {
972	public Component getTableCellEditorComponent(
982	public Object getCellEditorValue() {
987	private JPanel crearPanelAcciones(Color fondo) {
993	private BotonRedondeado crearBotonAccion(IconosUsuarios.Tipo icono,
1010	private void editarProducto(int id) {
1024	private boolean desactivarProducto(Producto seleccionado, Component padre) {
1048	private void exportarCsv() {
1075	protected Integer doInBackground() throws Exception {
1090	protected void done() {
1106	private void escribirCsv(Path archivo, List<Producto> productos)
1134	private String csv(String valor) {
1138	private ImagenIcono cargarImagen(String ruta, int tamano) {
1172	private String copiarImagenProducto(File archivo) throws IOException {
1187	private void mostrarError(String mensaje, Throwable causa) {
1194	public void removeNotify() {
1201	private FondoOscuro() {
1208	protected void paintComponent(Graphics graphics) {
1217	private void mostrarFormularioProducto(Producto original) {

Línea	Constructor o método detectado
1269	private JPanel crearFormulario(JDialog dialogo, Producto original,
1492	private void escucharRuedaEnFormulario(Component componente,
1502	private JPanel crearCampo(String titulo, javax.swing.JComponent control) {
1517	private void agregarFila(JPanel panel, int fila,
1532	private void agregarFilaCompleta(JPanel panel, int fila, JPanel contenido) {
1543	private void aplicarEstiloCampo(JTextField campo) {
1551	private BotonDesplegable nuevoSelector(String texto, String opciones) {
1561	private void estilizarSpinner(JSpinner spinner) {
1572	private BotonRedondeado botonFormulario(String texto, boolean principal) {
1586	private void mostrarVistaPrevia(JLabel etiqueta, String ruta) {
1596	private String nombreTipoStock(String codigo) {
1608	private void guardarProducto(JDialog dialogo, Producto original,

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.ProductoCRUD;	Importa CRUD.ProductoCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
5	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.PaginaProductos;	Importa Modelos.PaginaProductos para poder usar esa clase sin escribir su nombre completo.
7	import Modelos.Producto;	Importa Modelos.Producto para poder usar esa clase sin escribir su nombre completo.
8	import Modelos.ResumenProductos;	Importa Modelos.ResumenProductos para poder usar esa clase sin escribir su nombre completo.
9	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.
10	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
18	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
19	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
20	import java.awt.GridBagConstraints;	Importa java.awt.GridBagConstraints para poder usar esa clase sin escribir su nombre completo.
21	import java.awt.GridBagLayout;	Importa java.awt.GridBagLayout para poder usar esa clase sin escribir su nombre completo.
22	import java.awt.Image;	Importa java.awt.Image para poder usar esa clase sin escribir su nombre completo.
23	import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
24	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
25	import java.awt.event.ActionEvent;	Importa java.awt.event.ActionEvent para poder usar esa clase sin escribir su nombre completo.
26	import java.io.BufferedWriter;	Importa java.io.BufferedWriter para poder usar esa clase sin escribir su nombre completo.

27	<code>import java.io.File;</code>	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
28	<code>import java.io.IOException;</code>	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
29	<code>import java.math.BigDecimal;</code>	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
30	<code>import java.nio.charset.StandardCharsets;</code>	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
31	<code>import java.nio.file.Files;</code>	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
32	<code>import java.nio.file.Path;</code>	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
33	<code>import java.nio.file.Paths;</code>	Importa java.nio.file.Paths para poder usar esa clase sin escribir su nombre completo.
34	<code>import java.nio.file.StandardCopyOption;</code>	Importa java.nio.file.StandardCopyOption para poder usar esa clase sin escribir su nombre completo.
35	<code>import java.sql.SQLException;</code>	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
36	<code>import java.util.ArrayList;</code>	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
37	<code>import java.util.HashMap;</code>	Importa java.util.HashMap para poder usar esa clase sin escribir su nombre completo.
38	<code>import java.util.LinkedHashMap;</code>	Importa java.util.LinkedHashMap para poder usar esa clase sin escribir su nombre completo.
39	<code>import java.util.List;</code>	Importa java.util.List: colección ordenada que admite varios elementos.
40	<code>import java.util.Locale;</code>	Importa java.util.Locale para poder usar esa clase sin escribir su nombre completo.
41	<code>import java.util.Map;</code>	Importa java.util.Map: colección de pares clave-valor.
42	<code>import java.util.concurrent.ExecutionException;</code>	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
43	<code>import javax.swing.AbstractCellEditor;</code>	Importa javax.swing.AbstractCellEditor para poder usar esa clase sin escribir su nombre completo.
44	<code>import javax.swing.BorderFactory;</code>	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
45	<code>import javax.swing.ImageIcon;</code>	Importa javax.swing.ImageIcon para poder usar esa clase sin escribir su nombre completo.
46	<code>import javax.swing.JButton;</code>	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
47	<code>import javax.swing.JCheckBox;</code>	Importa javax.swing.JCheckBox para poder usar esa clase sin escribir su nombre completo.
48	<code>import javax.swing.JDialog;</code>	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
49	<code>import javax.swing.JFileChooser;</code>	Importa javax.swing.JFileChooser para poder usar esa clase sin escribir su nombre completo.
50	<code>import javax.swing.JLabel;</code>	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
51	<code>import javax.swing.JOptionPane;</code>	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
52	<code>import javax.swing.JPanel;</code>	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
53	<code>import javax.swing.JRootPane;</code>	Importa javax.swing.JRootPane para poder usar esa clase sin escribir su nombre completo.
54	<code>import javax.swing.JScrollPane;</code>	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
55	<code>import javax.swing.JSpinner;</code>	Importa javax.swing.JSpinner para poder usar esa clase sin escribir su nombre completo.
56	<code>import javax.swing.JTextArea;</code>	Importa javax.swing.JTextArea para poder usar esa clase sin escribir su nombre completo.
57	<code>import javax.swing.JTextField;</code>	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
58	<code>import javax.swing.SpinnerNumberModel;</code>	Importa javax.swing.SpinnerNumberModel para poder usar esa clase sin escribir su nombre completo.
59	<code>import javax.swing.SwingConstants;</code>	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
60	<code>import javax.swing.SwingUtilities;</code>	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
61	<code>import javax.swing.SwingWorker;</code>	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
62	<code>import javax.swing.Timer;</code>	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
63	<code>import javax.swing.WindowConstants;</code>	Importa javax.swing.WindowConstants para poder usar esa clase sin escribir su nombre completo.
64	<code>import javax.swing.border.EmptyBorder;</code>	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
65	<code>import javax.swing.event.DocumentEvent;</code>	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
66	<code>import javax.swing.event.DocumentListener;</code>	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
67	<code>import javax.swing.filechooser.FileNameExtensionFilter;</code>	Importa javax.swing.filechooser.FileNameExtensionFilter para poder usar esa clase sin escribir su nombre completo.
68	<code>import javax.swing.table.AbstractTableModel;</code>	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
69	<code>import javax.swing.table.TableCellEditor;</code>	Importa javax.swing.table.TableCellEditor para poder usar esa clase sin escribir su nombre completo.
70	<code>import javax.swing.table.TableCellRenderer;</code>	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.

71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
73	* Gestión de productos con la misma presentación que UsuariosPanel.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
74	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
75	@SuppressWarnings("serial")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
76	public class GestionMenuPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	private static final int PRODUCTOS_POR_PAGINA = 10;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
79	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
80	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
81	private static final Color BORDE = new Color(225, 229, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
82	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
83	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
84	private static final String CARPETA_IMAGENES = "src/Imagenes/productos/";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
85	private static final String RECURSO_IMAGENES = "/Imagenes/productos/";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
88	private final ProductoCRUD productoCRUD = new ProductoCRUD();	Crea una nueva instancia de ProductoCRUD y la usa o guarda para continuar el proceso.
89	private final ModeloTablaProductos modeloTabla = new ModeloTablaProductos();	Crea una nueva instancia de ModeloTablaProductos y la usa o guarda para continuar el proceso.
90	private final List<BotonRedondeado> botonesPagina = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
91	private final Map<Integer, String> categorias = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
92	private final Map<String, ImageIcon> cacheImagenes = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
93	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
94	private final BotonRedondeado botonConfigurar = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
97	private int totalPaginas = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
98	private int secuenciaCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
99	private Integer categoriaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
100	private Boolean estadoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
101	private boolean mostrandoErrorConexion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	public GestionMenuPanel() {	Constructor de GestionMenuPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
104	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
105	agregarBotonConfiguracion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106	temporizadorBusqueda = new Timer(300, evento -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
107	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
108	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
111	configurarApariencia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	configurarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	configurarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

115	cargarCategorias();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	scrollUsuarios.setVerticalScrollBarPolicy(	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
119	javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
120	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	scrollUsuarios.setHorizontalScrollBarPolicy(	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
122	javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
123	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	}	Cierra el bloque abierto anteriormente.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	private void agregarBotonConfiguracion() {	Declara el método agregarBotonConfiguracion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
127	botonConfigurar.setText("Opciones y receta");	Cambia el texto visible de un componente de la interfaz.
128	botonConfigurar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
129	botonConfigurar.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
130	botonConfigurar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
131	botonConfigurar.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
132	botonConfigurar.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
133	botonConfigurar.setColorBorde(AMARILLO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
134	botonConfigurar.setGrosorBorde(1.5f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
135	botonConfigurar.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
136	add(botonConfigurar, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 75, 225, 54));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
137	botonConfigurar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
138	int fila = tablaProductos.getSelectedRow();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
139	if (fila < 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
140	JOptionPane.showMessageDialog(this, "Selecciona primero un producto de la tabla.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
141	"Configurar menú", JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
143	}	Cierra el bloque abierto anteriormente.
144	Producto producto = modeloTabla.productoEn(tablaProductos.convertRowIndexToModel(fila));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
145	new ConfiguracionMenuAdminDialog(SwingUtilities.getWindowAncestor(this), producto).setVisible(true);	Crea una nueva instancia de ConfiguracionMenuAdminDialog y la usa o guarda para continuar el proceso.
146	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	}	Cierra el bloque abierto anteriormente.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
151	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
152	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154	labelTitulo3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
155	labelTitulo2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
156	botonAgregarProducto = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
157	panelFlotante1 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.

158	panelCircular2 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
159	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
160	labelTitulo4 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
161	labelTotalProductos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
162	labelTitulo6 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
163	panelFlotante2 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
164	panelCircular3 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
165	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
166	labelTitulo7 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
167	labelStockBajo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
168	labelTitulo9 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
169	panelFlotante4 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
170	panelCircular5 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
171	labelEscalable5 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
172	labelTitulo13 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
173	labelActivos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
174	labelTitulo15 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
175	panelFlotante3 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
176	panelCircular4 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
177	labelEscalable4 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
178	labelTitulo10 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
179	labelInactivos = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
180	labelTitulo12 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
181	botonExportar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
182	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
183	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
184	filtroCategoria = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
185	filtroEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
186	botonLimpiarFiltros = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
187	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
188	scrollUsuarios = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
189	tablaProductos = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
190	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
191	panelPaginacion = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
192	botonAnterior = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
193	botonPagina1 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
194	botonPagina2 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
195	botonPagina3 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
196	botonSiguiente = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
198	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
199	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	labelTitulo3.setFont(new java.awt.Font("Dialog", 1, 55)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.

202	labelTitulo3.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
203	labelTitulo3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
204	labelTitulo3.setText("Gestión del Menú");	Cambia el texto visible de un componente de la interfaz.
205	add(labelTitulo3, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 30, 460, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	labelTitulo2.setFont(new java.awt.Font("Dialog", 1, 18)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
208	labelTitulo2.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
209	labelTitulo2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
210	labelTitulo2.setText("Administra productos y combos");	Cambia el texto visible de un componente de la interfaz.
211	add(labelTitulo2, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 80, 320, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	botonAgregarProducto.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
214	botonAgregarProducto.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/icono_agregar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
215	botonAgregarProducto.setText("Agregar producto");	Cambia el texto visible de un componente de la interfaz.
216	botonAgregarProducto.setIconTextGap(15);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
217	add(botonAgregarProducto, new org.netbeans.lib.awtextra.AbsoluteConstraints(1040, 60, 230, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
218		Línea en blanco: separa visualmente bloques para que el código sea más legible.
219	panelFlotante1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/lista_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
224	panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(15, 20, 50, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
225		Línea en blanco: separa visualmente bloques para que el código sea más legible.
226	panelFlotante1.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	labelTitulo4.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
229	labelTitulo4.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
230	labelTitulo4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
231	labelTitulo4.setText("Productos totales");	Cambia el texto visible de un componente de la interfaz.
232	panelFlotante1.add(labelTitulo4, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	labelTotalProductos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
235	labelTotalProductos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
236	labelTotalProductos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
237	labelTotalProductos.setText("48");	Cambia el texto visible de un componente de la interfaz.
238	panelFlotante1.add(labelTotalProductos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
239		Línea en blanco: separa visualmente bloques para que el código sea más legible.
240	labelTitulo6.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
241	labelTitulo6.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
242	labelTitulo6.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
243	labelTitulo6.setText("En el inventario");	Cambia el texto visible de un componente de la interfaz.

244	panelFlotante1.add(labelTitulo6, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	add(panelFlotante1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
247		Línea en blanco: separa visualmente bloques para que el código sea más legible.
248	panelFlotante2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
251	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
252		Línea en blanco: separa visualmente bloques para que el código sea más legible.
253	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/advertencia_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
254	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(19, 22, 43, 43));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	panelFlotante2.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
257		Línea en blanco: separa visualmente bloques para que el código sea más legible.
258	labelTitulo7.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
259	labelTitulo7.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
260	labelTitulo7.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
261	labelTitulo7.setText("Stock bajo");	Cambia el texto visible de un componente de la interfaz.
262	panelFlotante2.add(labelTitulo7, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
263		Línea en blanco: separa visualmente bloques para que el código sea más legible.
264	labelStockBajo.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
265	labelStockBajo.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
266	labelStockBajo.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
267	labelStockBajo.setText("48");	Cambia el texto visible de un componente de la interfaz.
268	panelFlotante2.add(labelStockBajo, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
270	labelTitulo9.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
271	labelTitulo9.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
272	labelTitulo9.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
273	labelTitulo9.setText("Requieren reposición");	Cambia el texto visible de un componente de la interfaz.
274	panelFlotante2.add(labelTitulo9, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
275		Línea en blanco: separa visualmente bloques para que el código sea más legible.
276	add(panelFlotante2, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
278	panelFlotante4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
279		Línea en blanco: separa visualmente bloques para que el código sea más legible.
280	panelCircular5.setColorFondo(new java.awt.Color(230, 245, 234));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
281	panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
282		Línea en blanco: separa visualmente bloques para que el código sea más legible.
283	labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/cheque.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
284	panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(18, 23, 45, 45));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	panelFlotante4.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	labelTitulo13.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
289	labelTitulo13.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
290	labelTitulo13.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
291	labelTitulo13.setText("Activos");	Cambia el texto visible de un componente de la interfaz.
292	panelFlotante4.add(labelTitulo13, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
293		Línea en blanco: separa visualmente bloques para que el código sea más legible.
294	labelActivos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
295	labelActivos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
296	labelActivos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
297	labelActivos.setText("48");	Cambia el texto visible de un componente de la interfaz.
298	panelFlotante4.add(labelActivos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
299		Línea en blanco: separa visualmente bloques para que el código sea más legible.
300	labelTitulo15.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
301	labelTitulo15.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
302	labelTitulo15.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
303	labelTitulo15.setText("Disponible para uso");	Cambia el texto visible de un componente de la interfaz.
304	panelFlotante4.add(labelTitulo15, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
305		Línea en blanco: separa visualmente bloques para que el código sea más legible.
306	add(panelFlotante4, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
307		Línea en blanco: separa visualmente bloques para que el código sea más legible.
308	panelFlotante3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
309		Línea en blanco: separa visualmente bloques para que el código sea más legible.
310	panelCircular4.setColorFondo(new java.awt.Color(237, 239, 243));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
311	panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
312		Línea en blanco: separa visualmente bloques para que el código sea más legible.
313	labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/x_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
314	panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(23, 27, 35, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
315		Línea en blanco: separa visualmente bloques para que el código sea más legible.
316	panelFlotante3.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318	labelTitulo10.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
319	labelTitulo10.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
320	labelTitulo10.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
321	labelTitulo10.setText("Inactivos");	Cambia el texto visible de un componente de la interfaz.
322	panelFlotante3.add(labelTitulo10, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
323		Línea en blanco: separa visualmente bloques para que el código sea más legible.
324	labelInactivos.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
325	labelInactivos.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
326	labelInactivos.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
327	labelInactivos.setText("48");	Cambia el texto visible de un componente de la interfaz.
328	panelFlotante3.add(labelInactivos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
330	labelTitulo12.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.

331	labelTitulo12.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
332	labelTitulo12.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
333	labelTitulo12.setText("No disponibles");	Cambia el texto visible de un componente de la interfaz.
334	panelFlotante3.add(labelTitulo12, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
335		Línea en blanco: separa visualmente bloques para que el código sea más legible.
336	add(panelFlotante3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	botonExportar.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
339	botonExportar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
340	botonExportar.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
341	botonExportar.setRadio(20);	Configura una propiedad mediante setRadio en el objeto indicado.
342	botonExportar.setText("Exportar");	Cambia el texto visible de un componente de la interfaz.
343	botonExportar.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
344	botonExportar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
345	botonExportar.setFont(new java.awt.Font("Arial", 1, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
346	botonExportar.setGrosorBorde(1.6F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
347	botonExportar.setMargin(new java.awt.Insets(5, 14, 3, 14));	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
348	add(botonExportar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1290, 60, 230, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
349		Línea en blanco: separa visualmente bloques para que el código sea más legible.
350	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
351	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
352	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
353		Línea en blanco: separa visualmente bloques para que el código sea más legible.
354	campoBusqueda.setPlaceholder("Buscar producto o categoría...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
355	campoBusqueda.setFont(new java.awt.Font("Dialog", 0, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
356	campoBusqueda.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
357	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
358	campoBusquedaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
359	}	Cierra el bloque abierto anteriormente.
360	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
361	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 22, 455, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
362		Línea en blanco: separa visualmente bloques para que el código sea más legible.
363	filtroCategoria.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
364	filtroCategoria.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
365	filtroCategoria.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
366	filtroCategoria.setText("Todas las categorías");	Cambia el texto visible de un componente de la interfaz.
367	filtroCategoria.setTextoDesplegable("Todos los roles;ADMINISTRADOR;CAJERO");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
368	filtroCategoria.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
369	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
370	filtroCategoriaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
371	}	Cierra el bloque abierto anteriormente.
372	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
373	panelFiltros.add(filtroCategoria, new org.netbeans.lib.awtextra.AbsoluteConstraints(495, 24, 330, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.

375	<code>filtroEstado.setColorFondo(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
376	<code>filtroEstado.setColorHover(new java.awt.Color(248, 249, 251));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
377	<code>filtroEstado.setForeground(new java.awt.Color(0, 20, 43));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
378	<code>filtroEstado.setText("Todos los estados");</code>	Cambia el texto visible de un componente de la interfaz.
379	<code>filtroEstado.setTextoDesplegable("Todos los estados;Activo;Inactivo");</code>	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
380	<code>panelFiltros.add(filtroEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(850, 24, 330, 52));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
381		Línea en blanco: separa visualmente bloques para que el código sea más legible.
382	<code>botonLimpiarFiltros.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
383	<code>botonLimpiarFiltros.setDegradado(false);</code>	Configura una propiedad mediante setDegradado en el objeto indicado.
384	<code>botonLimpiarFiltros.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
385	<code>botonLimpiarFiltros.setText("Limpiar filtros");</code>	Cambia el texto visible de un componente de la interfaz.
386	<code>botonLimpiarFiltros.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
387	<code>botonLimpiarFiltros.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
388	<code>botonLimpiarFiltros.setFont(new java.awt.Font("Arial", 1, 15)); // NOI18N</code>	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
389	<code>botonLimpiarFiltros.setGrosorBorde(1.6F);</code>	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
390	<code>botonLimpiarFiltros.setMargin(new java.awt.Insets(5, 14, 3, 14));</code>	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
391	<code>panelFiltros.add(botonLimpiarFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 24, 230, 52));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
392		Línea en blanco: separa visualmente bloques para que el código sea más legible.
393	<code>add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 320, 1470, 100));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
394		Línea en blanco: separa visualmente bloques para que el código sea más legible.
395	<code>panelTabla.setRadio(18);</code>	Configura una propiedad mediante setRadio en el objeto indicado.
396	<code>panelTabla.setSombra(false);</code>	Configura una propiedad mediante setSombra en el objeto indicado.
397	<code>panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
398		Línea en blanco: separa visualmente bloques para que el código sea más legible.
399	<code>scrollUsuarios.setBorder(null);</code>	Configura una propiedad mediante setBorder en el objeto indicado.
400	<code>scrollUsuarios.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
401	<code>scrollUsuarios.setAutoscrolls(true);</code>	Configura una propiedad mediante setAutoscrolls en el objeto indicado.
402		Línea en blanco: separa visualmente bloques para que el código sea más legible.
403	<code>tablaProductos.setModel(new javax.swing.table.DefaultTableModel(</code>	Crea una nueva instancia de javax.swing.table.DefaultTableModel y la usa o guarda para continuar el proceso.
404	<code>new Object [][] {</code>	Crea una nueva instancia de Object y la usa o guarda para continuar el proceso.
405	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
406	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
407	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
408	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
409	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
410	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
411	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
412	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
413	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
414	<code>{null, null, null, null, null, null, null, null}</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	<code>},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
416	<code>new String [] {</code>	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
417	<code>"ID", "Nombre", "Usuario", "Rol", "Correo", "Estado", "Fecha creación", "Acciones"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	<code>}</code>	Cierra el bloque abierto anteriormente.

419	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
420	boolean[] canEdit = new boolean [] {	Crea una nueva instancia de boolean y la usa o guarda para continuar el proceso.
421	false, false, false, false, false, false, false, false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
422	};	Cierra el bloque abierto anteriormente y termina la declaración.
423		Línea en blanco: separa visualmente bloques para que el código sea más legible.
424	public boolean isCellEditable(int rowIndex, int columnIndex) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
425	return canEdit [columnIndex];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
426	}	Cierra el bloque abierto anteriormente.
427	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
428	tablaProductos.setAutoscrolls(false);	Configura una propiedad mediante setAutoscrolls en el objeto indicado.
429	tablaProductos.setColumnsCentradas("5,7");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
430	scrollUsuarios.setViewportView(tablaProductos);	Configura una propiedad mediante setViewportView en el objeto indicado.
431		Línea en blanco: separa visualmente bloques para que el código sea más legible.
432	panelTabla.add(scrollUsuarios, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 428));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
433		Línea en blanco: separa visualmente bloques para que el código sea más legible.
434	etiquetaRango.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
435	etiquetaRango.setText("Mostrando 1-10 de 10 productos");	Cambia el texto visible de un componente de la interfaz.
436	panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 452, 330, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
437		Línea en blanco: separa visualmente bloques para que el código sea más legible.
438	panelPaginacion.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
439	panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
441	botonAnterior.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
442	botonAnterior.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
443	botonAnterior.setText("<");	Cambia el texto visible de un componente de la interfaz.
444	panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
445		Línea en blanco: separa visualmente bloques para que el código sea más legible.
446	botonPagina1.setColorInicio(new java.awt.Color(255, 188, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
447	botonPagina1.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
448	botonPagina1.setText("1");	Cambia el texto visible de un componente de la interfaz.
449	panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	botonPagina2.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
452	botonPagina2.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
453	botonPagina2.setText("2");	Cambia el texto visible de un componente de la interfaz.
454	panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	botonPagina3.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
457	botonPagina3.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
458	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
459	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	botonSiguiente.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
462	botonSiguiente.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.

463	botonSiguiente.setText(",");	Cambia el texto visible de un componente de la interfaz.
464	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
465		Línea en blanco: separa visualmente bloques para que el código sea más legible.
466	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 452, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
467		Línea en blanco: separa visualmente bloques para que el código sea más legible.
468	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 420, 1470, 510));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
469	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
470		Línea en blanco: separa visualmente bloques para que el código sea más legible.
471	private void filtroCategoriaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroCategor	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
472	iaActionPerformed	
473	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
474	}//GEN-LAST:event_filtroCategoriaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
475		Línea en blanco: separa visualmente bloques para que el código sea más legible.
476	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_campoBusquedaAc	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
477	tionPerformed	
478	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
479	}//GEN-LAST:event_campoBusquedaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
480		Línea en blanco: separa visualmente bloques para que el código sea más legible.
481	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
482	* Sólo cambia propiedades fuera del bloque generado por NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
483	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
484	private void configurarApariencia() {	Declara el método configurarApariencia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
485		
486	botonAgregarProducto.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
487	evento -> mostrarFormularioProducto(null));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
488		Línea en blanco: separa visualmente bloques para que el código sea más legible.
489	botonExportar.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
490	evento -> exportarCsv());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
491	}	Cierra el bloque abierto anteriormente.
492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
493	private void configurarFiltros() {	Declara el método configurarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
494	panelFiltros.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
495	panelFiltros.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
496	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
497	configurarSelector(filtroCategoria, 330);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
498	configurarSelector(filtroEstado, 330);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
499	filtroCategoria.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
500	String opcion = evento.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
501	filtroCategoria.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
502	categoriaActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
503	if (!"Todas las categorías".equals(opcion)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
504	for (Map.Entry<Integer, String> entrada : categorias.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
505	if (entrada.getValue().equals(opcion)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
506	categoriaActual = entrada.getKey();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
507	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.

505	}	Cierra el bloque abierto anteriormente.
506	}	Cierra el bloque abierto anteriormente.
507	}	Cierra el bloque abierto anteriormente.
508	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
509	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
510	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
511	filtroEstado.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
512	String opcion = evento.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
513	filtroEstado.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
514	estadoActual = switch (opcion) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
515	case "Activo" ->	Define una de las rutas posibles dentro del switch.
516	Boolean.TRUE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
517	case "Inactivo" ->	Define una de las rutas posibles dentro del switch.
518	Boolean.FALSE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
519	default ->	Define una de las rutas posibles dentro del switch.
520	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
521	};	Cierra el bloque abierto anteriormente y termina la declaración.
522	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
523	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
524	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
525	botonLimpiarFiltros.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
526	botonLimpiarFiltros.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
527	IconosUsuarios.Tipo.FILTRO, ROJO, 21));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
528	botonLimpiarFiltros.setIconTextGap(10);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
529	botonLimpiarFiltros.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
530	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
531	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.
532	categoriaActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
533	estadoActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
534	filtroCategoria.setText("Todas las categorías");	Cambia el texto visible de un componente de la interfaz.
535	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
536	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
537	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
538	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
539	}	Cierra el bloque abierto anteriormente.
540		Línea en blanco: separa visualmente bloques para que el código sea más legible.
541	private void configurarSelector(BotonDesplegable selector, int ancho) {	Declara el método configurarSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
542	selector.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
543	selector.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
544	selector.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
545	selector.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
546	selector.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
547	selector.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
548	selector.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.

549	selector.setAnchoMenu(ancho);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
550	selector.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
551	selector.setBorderPainted(true);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
552	selector.setBorder(BorderFactory.createCompoundBorder(553 BorderFactory.createLineBorder(BORDE, 1, true), 554 new EmptyBorder(0, 14, 0, 34)));	Configura una propiedad mediante setBorder en el objeto indicado. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
555	}	Cierra el bloque abierto anteriormente.
556		Línea en blanco: separa visualmente bloques para que el código sea más legible.
557	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
558	panelTabla.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
559	panelTabla.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
560	tablaProductos.setModel(modeloTabla);	Configura una propiedad mediante setModel en el objeto indicado.
561	tablaProductos.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
562	tablaProductos.setRowHeight(60);	Configura una propiedad mediante setRowHeight en el objeto indicado.
563	tablaProductos.setAutoCreateRowSorter(false);	Configura una propiedad mediante setAutoCreateRowSorter en el objeto indicado.
564	tablaProductos.getColumnModel().getColumn(1)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
565	.setCellRenderer(new RenderProducto());	Crea una nueva instancia de RenderProducto y la usa o guarda para continuar el proceso.
566	tablaProductos.getColumnModel().getColumn(6)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
567	.setCellRenderer(new RenderEstado());	Crea una nueva instancia de RenderEstado y la usa o guarda para continuar el proceso.
568	tablaProductos.getColumnModel().getColumn(7)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
569	.setCellRenderer(new RenderAcciones());	Crea una nueva instancia de RenderAcciones y la usa o guarda para continuar el proceso.
570	tablaProductos.getColumnModel().getColumn(7)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
571	.setCellEditor(new EditorAcciones());	Crea una nueva instancia de EditorAcciones y la usa o guarda para continuar el proceso.
572	int[] anchos = {75, 315, 170, 115, 165, 90, 110, 130};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
573	for (int i = 0; i < anchos.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
574	tablaProductos.getColumnModel().getColumn(i)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
575	.setPreferredWidth(anchos[i]);	Configura una propiedad mediante setPreferredWidth en el objeto indicado.
576	}	Cierra el bloque abierto anteriormente.
577	scrollUsuarios.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
578	scrollUsuarios.getViewPort().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
579	scrollUsuarios.setVerticalScrollBarPolicy(580 JScrollPane.VERTICAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado. Ejecuta una instrucción Java completa; el punto y coma marca su final.
581	scrollUsuarios.setHorizontalScrollBarPolicy(582 JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado. Ejecuta una instrucción Java completa; el punto y coma marca su final.
583	etiquetaRango.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
584	etiquetaRango.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
585		Línea en blanco: separa visualmente bloques para que el código sea más legible.
586	configurarBotonPagina(botonAnterior, IconosUsuarios.Tipo.ANTERIOR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
587	botonAnterior.addActionListener(evento -> cambiarPagina(paginaActual - 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
588	botonesPagina.add(botonPagina1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
589	botonesPagina.add(botonPagina2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
590	botonesPagina.add(botonPagina3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
591	for (BotonRedondeado boton : botonesPagina) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
592	configurarBotonPagina(boton, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

593	boton.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
594	Object pagina = boton.getClientProperty("pagina");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
595	if (pagina instanceof Integer numero) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
596	cambiarPagina(numero);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	}	Cierra el bloque abierto anteriormente.
598	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
599	}	Cierra el bloque abierto anteriormente.
600	configurarBotonPagina(botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
601	botonSiguiente.addActionListener(evento -> cambiarPagina(paginaActual + 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
602	}	Cierra el bloque abierto anteriormente.
603		Línea en blanco: separa visualmente bloques para que el código sea más legible.
604	private void configurarBotonPagina(BotonRedondeado boton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
605	IconosUsuarios.Tipo tipoIcono) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
606	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
607	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
608	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
609	boton.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
610	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
611	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
612	boton.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
613	if (tipoIcono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
614	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
615	boton.setIcon(IconosUsuarios.crear(tipoIcono, AZUL, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
616	}	Cierra el bloque abierto anteriormente.
617	}	Cierra el bloque abierto anteriormente.
618		Línea en blanco: separa visualmente bloques para que el código sea más legible.
619	private void configurarEventos() {	Declara el método configurarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
620	campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
621	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
622	public void insertUpdate(DocumentEvent e) {	Declara el método insertUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
623	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
624	}	Cierra el bloque abierto anteriormente.
625		Línea en blanco: separa visualmente bloques para que el código sea más legible.
626	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
627	public void removeUpdate(DocumentEvent e) {	Declara el método removeUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
628	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
629	}	Cierra el bloque abierto anteriormente.
630		Línea en blanco: separa visualmente bloques para que el código sea más legible.
631	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
632	public void changedUpdate(DocumentEvent e) {	Declara el método changedUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
633	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
634	}	Cierra el bloque abierto anteriormente.
635	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
636	}	Cierra el bloque abierto anteriormente.

637		Línea en blanco: separa visualmente bloques para que el código sea más legible.
638	private void cargarCategorias() {	Declara el método cargarCategorias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
639	new SwingWorker<Map<Integer, String>, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
640	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
641	protected Map<Integer, String> doInBackground()	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
642	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
643	return productoCRUD.listarCategoriasActivas();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
644	}	Cierra el bloque abierto anteriormente.
645		Línea en blanco: separa visualmente bloques para que el código sea más legible.
646	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
647	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
648	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
649	categorias.clear();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
650	categorias.putAll(get());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
651	List<String> opciones = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
652	opciones.add("Todas las categorías");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
653	opciones.addAll(categorias.values());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
654	filtroCategoria.setTextoDesplegable(String.join(";", opciones));	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
655	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
656	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
657	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
658	mostrarErrorDatos("cargar las categorías", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
659	}	Cierra el bloque abierto anteriormente.
660	}	Cierra el bloque abierto anteriormente.
661	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
662	}	Cierra el bloque abierto anteriormente.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	private void cargarResumen() {	Declara el método cargarResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
665	new SwingWorker<ResumenProductos, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
666	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
667	protected ResumenProductos doInBackground()	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
668	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
669	return productoCRUD.obtenerResumen();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
670	}	Cierra el bloque abierto anteriormente.
671		Línea en blanco: separa visualmente bloques para que el código sea más legible.
672	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
673	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
674	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
675	ResumenProductos resumen = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
676	labelTotalProductos.setText(String.valueOf(resumen.total()));	Cambia el texto visible de un componente de la interfaz.
677	labelStockBajo.setText(String.valueOf(resumen.stockBajo()));	Cambia el texto visible de un componente de la interfaz.
678	labelActivos.setText(String.valueOf(resumen.activos()));	Cambia el texto visible de un componente de la interfaz.
679	labelInactivos.setText(String.valueOf(resumen.inactivos()));	Cambia el texto visible de un componente de la interfaz.
680	mostrandoErrorConexion = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

681	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
682	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
683	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
684	mostrarErrorDatos("cargar el resumen", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
685	}	Cierra el bloque abierto anteriormente.
686	}	Cierra el bloque abierto anteriormente.
687	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
688	}	Cierra el bloque abierto anteriormente.
689		Línea en blanco: separa visualmente bloques para que el código sea más legible.
690	private void cargarDatos() {	Declara el método cargarDatos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
691	final int solicitud = ++secuenciaCarga;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
692	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
693	final Integer categoria = categoriaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
694	final Boolean estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
695	final int paginaSolicitada = paginaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
696	tablaProductos.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
697	etiquetaRango.setText("Cargando productos...");	Cambia el texto visible de un componente de la interfaz.
698	new SwingWorker<PaginaProductos, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
699	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
700	protected PaginaProductos doInBackground()	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
701	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
702	return productoCRUD.listarPagina(busqueda, categoria, null,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
703	estado, paginaSolicitada, PRODUCTOS_POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
704	}	Cierra el bloque abierto anteriormente.
705		Línea en blanco: separa visualmente bloques para que el código sea más legible.
706	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
707	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
708	if (solicitud != secuenciaCarga) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
709	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
710	}	Cierra el bloque abierto anteriormente.
711	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
712	PaginaProductos resultado = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
713	totalPaginas = Math.max(1, (int) Math.ceil(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
714	resultado.totalRegistros() / (double) PRODUCTOS_POR_PAGINA));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
715	if (paginaActual > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
716	paginaActual = totalPaginas;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
717	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
718	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
719	}	Cierra el bloque abierto anteriormente.
720	modeloTabla.setProductos(resultado.productos());	Configura una propiedad mediante setProductos en el objeto indicado.
721	actualizarPaginacion(resultado.totalRegistros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
722	mostrandoErrorConexion = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
723	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
724	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

725	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
726	modeloTabla.setProductos(List.of());	Configura una propiedad mediante setProductos en el objeto indicado.
727	actualizarPaginacion(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
728	mostrarErrorDatos("cargar los productos", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
729	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
730	tablaProductos.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
731	}	Cierra el bloque abierto anteriormente.
732	}	Cierra el bloque abierto anteriormente.
733	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
734	}	Cierra el bloque abierto anteriormente.
735		Línea en blanco: separa visualmente bloques para que el código sea más legible.
736	private void actualizarPaginacion(int totalRegistros) {	Declara el método actualizarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
737	if (totalRegistros == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
738	etiquetaRango.setText("No se encontraron productos");	Cambia el texto visible de un componente de la interfaz.
739	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
740	int primero = (paginaActual - 1) * PRODUCTOS_POR_PAGINA + 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
741	int ultimo = Math.min(primero + PRODUCTOS_POR_PAGINA - 1, totalRegistros);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
742	etiquetaRango.setText(String.format("Mostrando %d-%d de %d productos",	Cambia el texto visible de un componente de la interfaz.
743	primero, ultimo, totalRegistros));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
744	}	Cierra el bloque abierto anteriormente.
745	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
746	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
747	int inicio = Math.max(1, Math.min(paginaActual - 1, totalPaginas - 2));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
748	for (int i = 0; i < botonesPagina.size(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
749	BotonRedondeado boton = botonesPagina.get(i);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
750	int pagina = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
751	boton.setVisible(pagina <= totalPaginas);	Muestra u oculta una ventana o componente según el valor booleano enviado.
752	if (pagina > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
753	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
754	}	Cierra el bloque abierto anteriormente.
755	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
756	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
757	boolean actual = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
758	boton.setColorInicio(actual ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
759	boton.setColorFinal(actual ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
760	boton.setColorBorde(actual ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
761	boton.setFont(actual ? tema.negrita(13f) : tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
762	}	Cierra el bloque abierto anteriormente.
763	}	Cierra el bloque abierto anteriormente.
764		Línea en blanco: separa visualmente bloques para que el código sea más legible.
765	private void cambiarPagina(int pagina) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
766	if (pagina < 1 pagina > totalPaginas pagina == paginaActual) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
767	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
768	}	Cierra el bloque abierto anteriormente.

769	paginaActual = pagina;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
770	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
771	}	Cierra el bloque abierto anteriormente.
772		Línea en blanco: separa visualmente bloques para que el código sea más legible.
773	private void mostrarErrorDatos(String operacion, Throwable causa) {	Declara el método mostrarErrorDatos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
774	if (mostrandoErrorConexion) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
775	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
776	}	Cierra el bloque abierto anteriormente.
777	mostrandoErrorConexion = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
778	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
779	"No fue posible " + operacion + ".\n"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
780	+ (causa == null ? "Error desconocido" : causa.getMessage()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
781	"Error de base de datos", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
782	}	Cierra el bloque abierto anteriormente.
783		Línea en blanco: separa visualmente bloques para que el código sea más legible.
784	private final class ModeloTablaProductos extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
785		Línea en blanco: separa visualmente bloques para que el código sea más legible.
786	private final String[] columnas = {	Declara una constante o referencia que no puede reasignarse después de inicializarse.
787	"ID", "Producto", "Categoría", "Precio", "Disponibilidad",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
788	"Stock", "Estado", "Acciones"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
789	};	Cierra el bloque abierto anteriormente y termina la declaración.
790	private List<Producto> productos = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
791		Línea en blanco: separa visualmente bloques para que el código sea más legible.
792	private Producto productoEn(int fila) { return productos.get(fila); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
793		Línea en blanco: separa visualmente bloques para que el código sea más legible.
794	private void setProductos(List<Producto> nuevos) {	Declara el método setProductos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
795	productos = List.copyOf(nuevos);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
796	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
797	}	Cierra el bloque abierto anteriormente.
798		Línea en blanco: separa visualmente bloques para que el código sea más legible.
799	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
800	public int getRowCount() {	Declara el método getRowCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
801	return productos.size();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
802	}	Cierra el bloque abierto anteriormente.
803		Línea en blanco: separa visualmente bloques para que el código sea más legible.
804	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
805	public int getColumnCount() {	Declara el método getColumnCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
806	return columnas.length;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
807	}	Cierra el bloque abierto anteriormente.
808		Línea en blanco: separa visualmente bloques para que el código sea más legible.
809	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
810	public String getColumnName(int c) {	Declara el método getColumnName. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
811	return columnas[c];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
812	}	Cierra el bloque abierto anteriormente.

813		Línea en blanco: separa visualmente bloques para que el código sea más legible.
814	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
815	public Class<?> getColumnClass(int c) {	Declara el método getColumnClass. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
816	return c == 1 c == 6 c == 7 ? Producto.class : String.class;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
817	}	Cierra el bloque abierto anteriormente.
818		Línea en blanco: separa visualmente bloques para que el código sea más legible.
819	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
820	public boolean isCellEditable(int r, int c) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
821	return c == 7;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
822	}	Cierra el bloque abierto anteriormente.
823		Línea en blanco: separa visualmente bloques para que el código sea más legible.
824	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
825	public Object getValueAt(int fila, int columna) {	Declara el método getValueAt. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
826	Producto producto = productos.get(fila);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
827	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
828	case 0 ->	Define una de las rutas posibles dentro del switch.
829	String.format("#%04d", producto.getIdProducto());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
830	case 1, 6, 7 ->	Define una de las rutas posibles dentro del switch.
831	producto;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
832	case 2 ->	Define una de las rutas posibles dentro del switch.
833	producto.getCategoria();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
834	case 3 ->	Define una de las rutas posibles dentro del switch.
835	producto.getPrecioBase() == null ? "Q0.00"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
836	: "Q" + producto.getPrecioBase().setScale(2,	Configura una propiedad mediante setScale en el objeto indicado.
837	java.math.RoundingMode.HALF_UP).toString();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
838	case 4 ->	Define una de las rutas posibles dentro del switch.
839	nombreDisponibilidad(producto.getDisponibilidadMenu());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
840	case 5 ->	Define una de las rutas posibles dentro del switch.
841	String.valueOf(producto.getStockActual());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
842	default ->	Define una de las rutas posibles dentro del switch.
843	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
844	};	Cierra el bloque abierto anteriormente y termina la declaración.
845	}	Cierra el bloque abierto anteriormente.
846	}	Cierra el bloque abierto anteriormente.
847		Línea en blanco: separa visualmente bloques para que el código sea más legible.
848	private String nombreDisponibilidad(String valor) {	Declara el método nombreDisponibilidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
849	if (valor == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
850	return "";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
851	}	Cierra el bloque abierto anteriormente.
852	return switch (valor) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
853	case "DESAYUNO" ->	Define una de las rutas posibles dentro del switch.
854	"Desayuno";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
855	case "ALMUERZO" ->	Define una de las rutas posibles dentro del switch.
856	"Almuerzo";	Ejecuta una instrucción Java completa; el punto y coma marca su final.

857	case "TODO_DIA" ->	Define una de las rutas posibles dentro del switch.
858	"Todo el día";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
859	default ->	Define una de las rutas posibles dentro del switch.
860	valor;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
861	};	Cierra el bloque abierto anteriormente y termina la declaración.
862	}	Cierra el bloque abierto anteriormente.
863		Línea en blanco: separa visualmente bloques para que el código sea más legible.
864	private final class RenderProducto implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
865		Línea en blanco: separa visualmente bloques para que el código sea más legible.
866	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
867	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
868	javax.swing.JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
869	boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
870	Producto producto = (Producto) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
871	JPanel panel = new JPanel(new BorderLayout(12, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
872	panel.setBorder(new EmptyBorder(3, 10, 3, 8));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
873	panel.setBackground(seleccionado	Configura una propiedad mediante setBackground en el objeto indicado.
874	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
875	JLabel imagen = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
876	imagen.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
877	imagen.setPreferredSize(new Dimension(40, 40));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
878	imagen.setIcon(cargarImagen(producto.getImagen(), 40));	Configura una propiedad mediante setIcon en el objeto indicado.
879	if (imagen.getIcon() == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
880	imagen.setText("-");	Cambia el texto visible de un componente de la interfaz.
881	}	Cierra el bloque abierto anteriormente.
882	JPanel textos = new JPanel(new java.awt.GridLayout(2, 1, 0, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
883	textos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
884	JLabel nombre = new JLabel(producto.getNombre());	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
885	nombre.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
886	nombre.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
887	String subtipo = producto.getTipo();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
888	if (producto.getSubCategoria() != null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
889	&& !producto.getSubCategoria().isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
890	subtipo += " - " + producto.getSubCategoria();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
891	}	Cierra el bloque abierto anteriormente.
892	JLabel tipo = new JLabel(subtipo);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
893	tipo.setFont(tema.regular(11f));	Configura una propiedad mediante setFont en el objeto indicado.
894	tipo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
895	textos.add(nombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
896	textos.add(tipo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
897	panel.add(imagen, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
898	panel.add(textos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
899	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
900	}	Cierra el bloque abierto anteriormente.

901	}	Cierra el bloque abierto anteriormente.
902		Línea en blanco: separa visualmente bloques para que el código sea más legible.
903	private final class RenderEstado implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
904		Línea en blanco: separa visualmente bloques para que el código sea más legible.
905	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
906	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
907	javax.swing.JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
908	boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
909	Producto producto = (Producto) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
910	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 16));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
911	panel.setBackground(seleccionado	Configura una propiedad mediante setBackground en el objeto indicado.
912	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
913	JLabel estado = new JLabel(producto.isActive() ? "Activo" : "Inactivo");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
914	estado.setFont(tema.media(11f));	Configura una propiedad mediante setFont en el objeto indicado.
915	estado.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
916	estado.setForeground(producto.isActive()	Configura una propiedad mediante setForeground en el objeto indicado.
917	? new Color(34, 139, 71) : SECUNDARIO);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
918	estado.setBackground(producto.isActive()	Configura una propiedad mediante setBackground en el objeto indicado.
919	? new Color(230, 245, 234) : new Color(237, 239, 243));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
920	estado.setBorder(new EmptyBorder(3, 13, 3, 13));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
921	panel.add(estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
922	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
923	}	Cierra el bloque abierto anteriormente.
924	}	Cierra el bloque abierto anteriormente.
925		Línea en blanco: separa visualmente bloques para que el código sea más legible.
926	private final class RenderAcciones implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
927		Línea en blanco: separa visualmente bloques para que el código sea más legible.
928	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
929	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
930	javax.swing.JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
931	boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
932	Producto producto = (Producto) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
933	JPanel panel = crearPanelAcciones(seleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
934	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
935	BotonRedondeado editar = crearBotonAccion(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
936	IconosUsuarios.Tipo.EDITAR, new Color(214, 151, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
937	BotonRedondeado desactivar = crearBotonAccion(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
938	IconosUsuarios.Tipo.ELIMINAR, ROJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
939	desactivar.setEnabled(producto.isActive());	Habilita o deshabilita la interacción con el componente.
940	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
941	panel.add(desactivar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
942	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
943	}	Cierra el bloque abierto anteriormente.
944	}	Cierra el bloque abierto anteriormente.

945		Línea en blanco: separa visualmente bloques para que el código sea más legible.
946	private final class EditorAcciones extends AbstractCellEditor	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
947	implements TableCellEditor {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
948		Línea en blanco: separa visualmente bloques para que el código sea más legible.
949	private final JPanel panel = crearPanelAcciones(Color.WHITE);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
950	private final BotonRedondeado editar = crearBotonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
951	IconosUsuarios.Tipo.EDITAR, new Color(214, 151, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
952	private final BotonRedondeado desactivar = crearBotonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
953	IconosUsuarios.Tipo.ELIMINAR, ROJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
954	private Producto producto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
955		Línea en blanco: separa visualmente bloques para que el código sea más legible.
956	private EditorAcciones() {	Declara el método EditorAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
957	editar.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
958	Producto seleccionado = producto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
959	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
960	editarProducto(seleccionado.getIdProducto());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
961	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
962	desactivar.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
963	Producto seleccionado = producto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
964	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
965	desactivarProducto(seleccionado, GestionMenuPanel.this);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
966	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
967	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
968	panel.add(desactivar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
969	}	Cierra el bloque abierto anteriormente.
970		Línea en blanco: separa visualmente bloques para que el código sea más legible.
971	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
972	public Component getTableCellEditorComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
973	javax.swing.JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
974	int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
975	producto = (Producto) valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
976	desactivar.setEnabled(producto.isActivo());	Habilita o deshabilita la interacción con el componente.
977	panel.setBackground(tabla.getSelectionBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
978	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
979	}	Cierra el bloque abierto anteriormente.
980		Línea en blanco: separa visualmente bloques para que el código sea más legible.
981	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
982	public Object getCellEditorValue() {	Declara el método getCellEditorValue. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
983	return producto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
984	}	Cierra el bloque abierto anteriormente.
985	}	Cierra el bloque abierto anteriormente.
986		Línea en blanco: separa visualmente bloques para que el código sea más legible.
987	private JPanel crearPanelAcciones(Color fondo) {	Declara el método crearPanelAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

988	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 7, 14));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
989	panel.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
990	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
991	}	Cierra el bloque abierto anteriormente.
992		Línea en blanco: separa visualmente bloques para que el código sea más legible.
993	private BotonRedondeado crearBotonAccion(IconosUsuarios.Tipo icono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
994	Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
995	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
996	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
997	boton.setIcon(IconosUsuarios.crear(icono, color, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
998	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
999	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
1000	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
1001	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1002	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1003	boton.setRadio(9);	Configura una propiedad mediante setRadio en el objeto indicado.
1004	boton.setPreferredSize(new Dimension(33, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1005	boton.setToolTipText(icono == IconosUsuarios.Tipo.EDITAR	Configura una propiedad mediante setToolTipText en el objeto indicado.
1006	? "Editar producto" : "Desactivar producto");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1007	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1008	}	Cierra el bloque abierto anteriormente.
1009		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1010	private void editarProducto(int id) {	Declara el método editarProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1011	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1012	Producto producto = productoCRUD.obtenerPorId(id);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1013	if (producto == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1014	JOptionPane.showMessageDialog(this, "No se encontró el producto.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1015	"Producto", JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1016	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1017	}	Cierra el bloque abierto anteriormente.
1018	mostrarFormularioProducto(producto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1019	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1020	mostrarError("No fue posible obtener el producto", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1021	}	Cierra el bloque abierto anteriormente.
1022	}	Cierra el bloque abierto anteriormente.
1023		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1024	private boolean desactivarProducto(Producto seleccionado, Component padre) {	Declara el método desactivarProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1025	if (!seleccionado.isActivo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1026	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1027	}	Cierra el bloque abierto anteriormente.
1028	int respuesta = JOptionPane.showConfirmDialog(padre,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1029	"¿Desactivar «" + seleccionado.getNombre() + "»?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1030	"Confirmar desactivación", JOptionPane.YES_NO_OPTION,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1031	JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

1032	if (respuesta != JOptionPane.YES_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1033	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1034	}	Cierra el bloque abierto anteriormente.
1035	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1036	productoCRUD.desactivar(seleccionado.getIdProducto());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1037	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1038	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1039	return true;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1040	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1041	JOptionPane.showMessageDialog(padre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1042	"No fue posible desactivar el producto.\n" + ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1043	"Error", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1044	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1045	}	Cierra el bloque abierto anteriormente.
1046	}	Cierra el bloque abierto anteriormente.
1047		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1048	private void exportarCsv() {	Declara el método exportarCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1049	JFileChooser selector = new JFileChooser();	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
1050	selector.setDialogTitle("Exportar productos");	Configura una propiedad mediante setDialogTitle en el objeto indicado.
1051	selector.setSelectedFile(new File("productos.csv"));	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
1052	selector.setFileFilter(new FileNameExtensionFilter("Archivo CSV (*.csv)", "csv"));	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.
1053	if (selector.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1054	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1055	}	Cierra el bloque abierto anteriormente.
1056	Path destino = selector.getSelectedFile().toPath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1057	if (!destino.getFileName().toString().toLowerCase(Locale.ROOT)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1058	.endsWith(".csv")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1059	destino = destino.resolveSibling(destino.getFileName() + ".csv");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1060	}	Cierra el bloque abierto anteriormente.
1061	if (Files.exists(destino) && JOptionPane.showConfirmDialog(this,	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1062	"El archivo ya existe. ¿Deseas reemplazarlo?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1063	"Confirmar exportación", JOptionPane.YES_NO_OPTION)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1064	!= JOptionPane.YES_OPTION) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1065	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1066	}	Cierra el bloque abierto anteriormente.
1067	final Path archivo = destino;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1068	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1069	final Integer categoria = categoriaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1070	final Boolean estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1071	setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1072	botonExportar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
1073	new SwingWorker<Integer, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
1074	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1075	protected Integer doInBackground() throws Exception {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

1076	List<Producto> productos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
1077	PaginaProductos primera = productoCRUD.listarPagina(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1078	busqueda, categoria, null, estado, 1, 100);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1079	productos.addAll(primer.productos());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1080	int paginas = (int) Math.ceil(primer.totalRegistros() / 100.0);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1081	for (int pagina = 2; pagina <= paginas; pagina++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1082	productos.addAll(productoCRUD.listarPagina(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1083	busqueda, categoria, null, estado, pagina, 100).productos());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1084	}	Cierra el bloque abierto anteriormente.
1085	escribirCsv(archivo, productos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1086	return productos.size();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1087	}	Cierra el bloque abierto anteriormente.
1088		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1089	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1090	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1091	botonExportar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
1092	setCursor(Cursor.getDefaultCursor());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1093	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1094	JOptionPane.showMessageDialog(GestionMenuPanel.this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1095	"Se exportaron " + get() + " productos en:\n" + archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1096	"Exportación completada", JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1097	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1098	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1099	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1100	mostrarError("No fue posible exportar", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1101	}	Cierra el bloque abierto anteriormente.
1102	}	Cierra el bloque abierto anteriormente.
1103	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1104	}	Cierra el bloque abierto anteriormente.
1105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1106	private void escribirCsv(Path archivo, List<Producto> productos)	Declara el método escribirCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1107	throws IOException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1108	try (BufferedWriter escritor = Files.newBufferedWriter(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1109	archivo, StandardCharsets.UTF_8)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1110	escritor.write('\u000a');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1111	escritor.write("ID,Nombre,Categoria,Subcategoria,Descripción,Precio base,Disponibilidad,"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1112	+ "Tipo stock,Personalizable,Stock,Stock mínimo,Estado,Imagen");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1113	escritor.newLine();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1114	for (Producto producto : productos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1115	escritor.write(String.join(", ",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1116	csv(String.valueOf(producto.getIdProducto()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1117	csv(producto.getNombre()), csv(producto.getCategoria()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1118	csv(producto.getSubCategoria()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1119	csv(producto.getDescripcion()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1120	csv(producto.getPrecioBase() == null ? ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1121	: producto.getPrecioBase().toString()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1122	csv(producto.getDisponibilidadMenu()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1123	csv(producto.getTipoStock()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1124	csv(producto.isPersonalizable() ? "Sí" : "No"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1125	csv(String.valueOf(producto.getStockActual()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1126	csv(String.valueOf(producto.getStockMinimo()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1127	csv(producto.isActivo() ? "Activo" : "Inactivo"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1128	csv(producto.getImagen()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1129	escritor.newLine();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1130	}	Cierra el bloque abierto anteriormente.
1131	}	Cierra el bloque abierto anteriormente.
1132	}	Cierra el bloque abierto anteriormente.
1133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1134	private String csv(String valor) {	Declara el método csv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1135	return "\"" + (valor == null ? "" : valor).replace("\n", "\\n") + "\"";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1136	}	Cierra el bloque abierto anteriormente.
1137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1138	private ImageIcon cargarImagen(String ruta, int tamano) {	Declara el método cargarImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1139	if (ruta == null ruta.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1140	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1141	}	Cierra el bloque abierto anteriormente.
1142	String clave = ruta + "#" + tamano;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1143	if (cacheImagenes.containsKey(clave)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1144	return cacheImagenes.get(clave);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1145	}	Cierra el bloque abierto anteriormente.
1146	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1147	File archivo = new File(ruta);	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
1148	if (!archivo.isFile()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1149	archivo = new File("src/" + ruta.replaceFirst("^/", ""));	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
1150	}	Cierra el bloque abierto anteriormente.
1151	ImageIcon original = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1152	if (archivo.isFile()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1153	original = new ImageIcon(archivo.getAbsolutePath());	Crea una nueva instancia de ImageIcon y la usa o guarda para continuar el proceso.
1154	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1155	java.net.URL recurso = getClass().getResource(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1156	ruta.startsWith("/") ? ruta : "/" + ruta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1157	if (recurso != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1158	original = new ImageIcon(recurso);	Crea una nueva instancia de ImageIcon y la usa o guarda para continuar el proceso.
1159	}	Cierra el bloque abierto anteriormente.
1160	}	Cierra el bloque abierto anteriormente.
1161	if (original != null && original.getIconWidth() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1162	Image escalada = original.getImage().getScaledInstance(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1163	tamano, tamano, Image.SCALE_SMOOTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

1164	cacheImágenes.put(clave, new ImageIcon(escalada));	Crea una nueva instancia de ImageIcon y la usa o guarda para continuar el proceso.
1165	}	Cierra el bloque abierto anteriormente.
1166	} catch (RuntimeException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1167	// Se muestra el producto sin miniatura si el archivo falta.	Comentario para humanos; Java no ejecuta este texto. Se muestra el producto sin miniatura si el archivo falta.
1168	}	Cierra el bloque abierto anteriormente.
1169	return cacheImágenes.get(clave);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1170	}	Cierra el bloque abierto anteriormente.
1171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1172	private String copiarImagenProducto(File archivo) throws IOException {	Declara el método copiarImagenProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1173	Path carpeta = Paths.get(CARPETA_IMAGENES);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1174	Files.createDirectories(carpeta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1175	String nombre = archivo.getName();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1176	int punto = nombre.lastIndexOf('.');	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1177	String extension = punto < 0 ? ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1178	: nombre.substring(punto).toLowerCase(Locale.ROOT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1179	String base = (punto < 0 ? nombre : nombre.substring(0, punto))	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1180	.replaceAll("[a-zA-Z0-9_-]", "_");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1181	String nombreFinal = base + "_" + System.currentTimeMillis() + extension;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1182	Files.copy(archivo.toPath(), carpeta.resolve(nombreFinal),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1183	StandardCopyOption.REPLACE_EXISTING);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1184	return RECURSO_IMAGENES + nombreFinal;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1185	}	Cierra el bloque abierto anteriormente.
1186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1187	private void mostrarError(String mensaje, Throwable causa) {	Declara el método mostrarError. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1188	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1189	mensaje + ".\n" + (causa == null ? "" : causa.getMessage()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1190	"Error", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1191	}	Cierra el bloque abierto anteriormente.
1192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1193	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1194	public void removeNotify() {	Declara el método removeNotify. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1195	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1196	super.removeNotify();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1197	}	Cierra el bloque abierto anteriormente.
1198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1199	private static final class FondoOscuro extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1201	private FondoOscuro() {	Declara el método FondoOscuro. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1202	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1203	addMouseListener(new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
1204	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1205	}	Cierra el bloque abierto anteriormente.
1206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1207	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

1208	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1209	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1210	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1211	g2.setColor(new Color(0, 12, 28, 90));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1212	g2.fillRect(0, 0, getWidth(), getHeight());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1213	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1214	}	Cierra el bloque abierto anteriormente.
1215	}	Cierra el bloque abierto anteriormente.
1216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1217	private void mostrarFormularioProducto(Producto original) {	Declara el método mostrarFormularioProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1218	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1219	JRootPane raiz = SwingUtilities.getRootPane(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1220	Component cristalAnterior = raiz == null ? null : raiz.getGlassPane();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1221	boolean cristalVisible = cristalAnterior != null && cristalAnterior.isVisible();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1222	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1223	Map<Integer, String> disponibles = productoCRUD	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1224	.listarCategoriasParaFormulario(original == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1225	? null : original.getIdCategoria());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1226	if (disponibles.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1227	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1228	"Primero registra una categoría activa.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1229	"Sin categorías", JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1230	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1231	}	Cierra el bloque abierto anteriormente.
1232	JDialog dialogo = new JDialog(proprietario,	Crea una nueva instancia de JDialog y la usa o guarda para continuar el proceso.
1233	java.awt.Dialog.ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1234	dialogo.setUndecorated(true);	Configura una propiedad mediante setUndecorated en el objeto indicado.
1235	dialogo.setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1236	dialogo.setDefaultCloseOperation(WindowConstants.DISPOSE_ON_CLOSE);	Configura una propiedad mediante setDefaultCloseOperation en el objeto indicado.
1237	final boolean[] guardado = {false};	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1238	dialogo.setContentPane(crearFormulario(dialogo, original,	Configura una propiedad mediante setContentPane en el objeto indicado.
1239	disponibles, guardado));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1240	dialogo.pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1241	dialogo.setLocationRelativeTo(proprietario);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
1242	if (raiz != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1243	FondoOscuro fondo = new FondoOscuro();	Crea una nueva instancia de FondoOscuro y la usa o guarda para continuar el proceso.
1244	raiz.setGlassPane(fondo);	Configura una propiedad mediante setGlassPane en el objeto indicado.
1245	fondo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
1246	}	Cierra el bloque abierto anteriormente.
1247	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1248	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
1249	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1250	if (raiz != null && cristalAnterior != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1251	raiz.setGlassPane(cristalAnterior);	Configura una propiedad mediante setGlassPane en el objeto indicado.

1252	cristalAnterior.setVisible(cristalVisible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
1253	}	Cierra el bloque abierto anteriormente.
1254	}	Cierra el bloque abierto anteriormente.
1255	if (guardado[0]) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1256	cacheImagenes.clear();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1257	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1258	cargarDatos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1259	JOptionPane.showMessageDialog(this, original == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1260	? "Producto agregado correctamente."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1261	: "Producto actualizado correctamente.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1262	"Gestión del menú", JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1263	}	Cierra el bloque abierto anteriormente.
1264	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1265	mostrarError("No fue posible abrir el formulario", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1266	}	Cierra el bloque abierto anteriormente.
1267	}	Cierra el bloque abierto anteriormente.
1268		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1269	private JPanel crearFormulario(JDialog dialogo, Producto original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1270	Map<Integer, String> categoriasDisponibles, boolean[] guardado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1271	boolean editando = original != null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1272	Componentes.PanelFlotante tarjeta = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
1273	tarjeta.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
1274	tarjeta.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1275	tarjeta.setRadio(26);	Configura una propiedad mediante setRadio en el objeto indicado.
1276	tarjeta.setPreferredSize(new Dimension(790, 660));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1277	tarjeta.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
1278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1279	JPanel cabecera = new JPanel(null);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1280	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1281	cabecera.setPreferredSize(new Dimension(790, 112));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1282	Componentes.PanelCircular icono = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
1283	icono.setColorFondo(new Color(255, 244, 211));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1284	icono.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
1285	java.net.URL urlIcono = getClass().getResource("/Imágenes/lista_icono.png");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1286	if (urlIcono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1287	JLabel imagen = new JLabel(new ImageIcon(new ImageIcon(urlIcono)	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1288	.getImage().getScaledInstance(39, 39, Image.SCALE_SMOOTH));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1289	imagen.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1290	icono.add(imagen, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1291	}	Cierra el bloque abierto anteriormente.
1292	cabecera.add(icono);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1293	icono.setBounds(30, 20, 80, 80);	Configura una propiedad mediante setBounds en el objeto indicado.
1294	JLabel titulo = new JLabel(editando ? "Editar producto" : "Agregar producto");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1295	titulo.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.

1296	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1297	titulo.setBounds(125, 27, 595, 36);	Configura una propiedad mediante setBounds en el objeto indicado.
1298	cabecera.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1299	JLabel subtitulo = new JLabel(editando	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1300	? "Actualiza la información de "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1301	+ String.format("#%04d", original.getIdProducto())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1302	: "Completa los datos del nuevo producto");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1303	subtitulo.setFont(tema.regular(15f));	Configura una propiedad mediante setFont en el objeto indicado.
1304	subtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
1305	subtitulo.setBounds(125, 62, 600, 28);	Configura una propiedad mediante setBounds en el objeto indicado.
1306	cabecera.add(subtitulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1307	tarjeta.add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1308		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1309	JTextField nombre = new JTextField(editando ? original.getNombre() : "");	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1310	JTextField subCategoria = new JTextField(editando	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1311	&& original.getSubCategoria() != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1312	? original.getSubCategoria() : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1313	JTextField precio = new JTextField(editando && original.getPrecioBase() != null	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1314	? original.getPrecioBase().toString() : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1315	JTextArea descripcion = new JTextArea(editando	Crea una nueva instancia de JTextArea y la usa o guarda para continuar el proceso.
1316	&& original.getDescripcion() != null ? original.getDescripcion() : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1317	descripcion.setLineWrap(true);	Configura una propiedad mediante setLineWrap en el objeto indicado.
1318	descripcion.setWrapStyleWord(true);	Configura una propiedad mediante setWrapStyleWord en el objeto indicado.
1319	descripcion.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1320	descripcion.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1321	descripcion.setBorder(new EmptyBorder(8, 12, 8, 12));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1322	JScrollPane scrollDescripcion = new JScrollPane(descripcion);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
1323	Componentes.DesplazamientoSuave.ocultarBarras(scrollDescripcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1324	scrollDescripcion.setBorder(BorderFactory.createLineBorder(BORDE, 1, true));	Configura una propiedad mediante setBorder en el objeto indicado.
1325	scrollDescripcion.setPreferredSize(new Dimension(330, 78));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1326	scrollDescripcion.setWheelScrollingEnabled(false);	Configura una propiedad mediante setWheelScrollingEnabled en el objeto indicado.
1327	aplicarEstiloCampo(nombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1328	aplicarEstiloCampo(subCategoria);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1329	aplicarEstiloCampo(precio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1330		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1331	BotonDesplegable categoria = nuevoSelector("Seleccionar categoría",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1332	String.join(", ", categoriasDisponibles.values()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1333	BotonDesplegable disponibilidad = nuevoSelector("Todo el día",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1334	"Desayuno;Almuerzo;Todo el día");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1335	BotonDesplegable tamano = nuevoSelector("Stock directo",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1336	"Por receta;Stock directo;Sin control de stock");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1337	BotonDesplegable estado = nuevoSelector("Activo", "Activo;Inactivo");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1338	if (editando) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1339	categoria.setText(original.getCategoria());	Cambia el texto visible de un componente de la interfaz.

1340	disponibilidad.setText(nombreDisponibilidad(original.getDisponibilidadMenu()));	Cambia el texto visible de un componente de la interfaz.
1341	tamano.setText(nombreTipoStock(original.getTipoStock()));	Cambia el texto visible de un componente de la interfaz.
1342	estado.setText(original.isActivo() ? "Activo" : "Inactivo");	Cambia el texto visible de un componente de la interfaz.
1343	}	Cierra el bloque abierto anteriormente.
1344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1345	JSpinner stock = new JSpinner(new SpinnerNumberModel(	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
1346	editando ? original.getStockActual() : 0, 0, 999999999, 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1347	JSpinner stockMinimo = new JSpinner(new SpinnerNumberModel(	Crea una nueva instancia de JSpinner y la usa o guarda para continuar el proceso.
1348	editando ? original.getStockMinimo() : 0, 0, 999999999, 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1349	estilizarSpinner(stock);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1350	estilizarSpinner(stockMinimo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1351	JCheckBox esCombo = new JCheckBox("Permitir personalización",	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
1352	!editando original.isPersonalizable());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1353	esCombo.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1354	esCombo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1355	esCombo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1356		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1357	JPanel campos = new JPanel(new GridBagLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1358	campos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1359	campos.setBorder(new EmptyBorder(3, 28, 12, 28));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1360	agregarFila(campos, 0, crearCampo("Nombre", nombre),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1361	crearCampo("Categoría", categoria));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1362	agregarFila(campos, 1, crearCampo("Subcategoría (opcional)", subCategoria),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1363	crearCampo("Precio individual (Q)", precio));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1364	agregarFila(campos, 2, crearCampo("Disponibilidad", disponibilidad),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1365	crearCampo("Control de inventario", tamano));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1366	agregarFila(campos, 3, crearCampo("Stock actual", stock),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1367	crearCampo("Stock mínimo", stockMinimo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1368	agregarFila(campos, 4, crearCampo("Estado", estado),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1369	crearCampo("Preparación", esCombo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1370	agregarFilaCompleta(campos, 5, crearCampo("Descripción", scrollDescripcion));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1371		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1372	final String[] rutaImagen = {editando ? original.getImagen() : null};	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1373	final File[] imagenNueva = {null};	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1374	JLabel vistaPrevia = new JLabel("Sin imagen", SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1375	vistaPrevia.setPreferredSize(new Dimension(84, 84));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1376	vistaPrevia.setBorder(BorderFactory.createLineBorder(BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
1377	mostrarVistaPrevia(vistaPrevia, rutaImagen[0]);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1378	JLabel nombreImagen = new JLabel(rutaImagen[0] == null	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1379	? "Sin imagen seleccionada" : new File(rutaImagen[0]).getName());	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
1380	nombreImagen.setFont(tema.regular(13f));	Configura una propiedad mediante setFont en el objeto indicado.
1381	nombreImagen.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
1382	BotonRedondeado elegir = botonFormulario("Seleccionar imagen", false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1383	BotonRedondeado quitar = botonFormulario("Quitar imagen", false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1384	<code>elegir.setPreferredSize(new Dimension(210, 38));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1385	<code>quitar.setPreferredSize(new Dimension(175, 38));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1386	<code>JPanel controlesImagen = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1387	<code>controlesImagen.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1388	<code>controlesImagen.add(elegir);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1389	<code>controlesImagen.add(quitar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1390	<code>JPanel detallesImagen = new JPanel(new BorderLayout(0, 9));</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1391	<code>detallesImagen.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1392	<code>detallesImagen.add(nombreImagen, BorderLayout.NORTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1393	<code>detallesImagen.add(controlesImagen, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1394	<code>JPanel panelImagen = new JPanel(new BorderLayout(18, 0));</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1395	<code>panelImagen.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1396	<code>panelImagen.setBorder(new EmptyBorder(10, 10, 10, 10));</code>	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1397	<code>panelImagen.add(vistaPrevia, BorderLayout.WEST);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1398	<code>panelImagen.add(detallesImagen, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1399	<code>agregarFilaCompleta(campos, 6, crearCampo("Imagen del producto", panelImagen));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1400		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1401	<code>elegir.addActionListener(evento -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1402	<code> JFileChooser selector = new JFileChooser();</code>	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
1403	<code> selector.setFileFilter(new FileNameExtensionFilter(</code>	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.
1404	<code> "Imágenes PNG, JPG y JPEG", "png", "jpg", "jpeg"));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1405	<code> if (selector.showOpenDialog(dialogo) == JFileChooser.APPROVE_OPTION) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1406	<code> imagenNueva[0] = selector.getSelectedFile();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1407	<code> rutaImagen[0] = imagenNueva[0].getAbsolutePath();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1408	<code> nombreImagen.setText(imagenNueva[0].getName());</code>	Cambia el texto visible de un componente de la interfaz.
1409	<code> mostrarVistaPrevia(vistaPrevia, rutaImagen[0]);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1410	<code> }</code>	Cierra el bloque abierto anteriormente.
1411	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1412	<code>quitar.addActionListener(evento -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1413	<code> imagenNueva[0] = null;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1414	<code> rutaImagen[0] = null;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1415	<code> nombreImagen.setText("Sin imagen seleccionada");</code>	Cambia el texto visible de un componente de la interfaz.
1416	<code> mostrarVistaPrevia(vistaPrevia, null);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1417	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1419	<code>JScrollPane desplazamiento = new JScrollPane(campos);</code>	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
1420	<code>Componentes.DesplazamientoSuave.ocultarBarras(desplazamiento);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1421	<code>desplazamiento.setBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante setBorder en el objeto indicado.
1422	<code>desplazamiento.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1423	<code>desplazamiento.getViewport().setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1424	<code>// La rueda se dirige aquí desde todo el formulario.</code>	Comentario para humanos; Java no ejecuta este texto. La rueda se dirige aquí desde todo el formulario.
1425	<code>desplazamiento.setWheelScrollingEnabled(false);</code>	Configura una propiedad mediante setWheelScrollingEnabled en el objeto indicado.
1426	<code>javax.swing.JScrollBar barraVertical</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1427	<code> = desplazamiento.getVerticalScrollBar();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

1428	<code>barraVertical.setPreferredSize(new Dimension(0, 0));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1429	<code>barraVertical.setUnitIncrement(36);</code>	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
1430	<code>barraVertical.setBlockIncrement(180);</code>	Configura una propiedad mediante setBlockIncrement en el objeto indicado.
1431	<code>tarjeta.add(desplazamiento, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1432		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1433	<code>JPanel pie = new JPanel(new BorderLayout());</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1434	<code>pie.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1435	<code>pie.setBorder(new EmptyBorder(0, 30, 22, 30));</code>	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1436	<code>pie.setPreferredSize(new Dimension(790, 105));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1437	<code>JLabel error = new JLabel(" ");</code>	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1438	<code>error.setFont(tema.media(13f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
1439	<code>error.setForeground(R0J0);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
1440	<code>pie.add(error, BorderLayout.NORTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1441	<code>JPanel filaBotones = new JPanel(new BorderLayout());</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1442	<code>filaBotones.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1443	<code>BotonRedondeado desactivar</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1444	<code> = botonFormulario("Desactivar producto", false);</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1445	<code>desactivar.setPreferredSize(new Dimension(220, 48));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1446		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1447	<code>JPanel botonesIzquierda</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1448	<code> = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1449	<code>botonesIzquierda.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1450	<code>botonesIzquierda.add(desactivar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1451	<code>botonesIzquierda.setVisible(editando && original.isActivo());</code>	Muestra u oculta una ventana o componente según el valor booleano enviado.
1452	<code>filaBotones.add(botonesIzquierda, BorderLayout.WEST);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1453		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1454	<code>JPanel botonesDerecha</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1455	<code> = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1456	<code>botonesDerecha.setOpaque(false);</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
1457		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1458	<code>BotonRedondeado cancelar = botonFormulario("Cancelar", false);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1459	<code>cancelar.setPreferredSize(new Dimension(130, 48));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1461	<code>BotonRedondeado guardar</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1462	<code> = botonFormulario("Guardar producto", true);</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1463	<code>guardar.setPreferredSize(new Dimension(210, 48));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1464		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1465	<code>botonesDerecha.add(cancelar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1466	<code>botonesDerecha.add(guardar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1467	<code>filaBotones.add(botonesDerecha, BorderLayout.EAST);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1468	<code>pie.add(filaBotones, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1469	<code>tarjeta.add(pie, BorderLayout.SOUTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1470	<code>cancelar.addActionListener(evento -> dialogo.dispose());</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1471	<code>desactivar.addActionListener(evento -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.

1472	if (desactivarProducto(original, dialogo)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1473	dialogo.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1474	}	Cierra el bloque abierto anteriormente.
1475	}};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1476	guardar.addActionListener(evento -> guardarProducto(dialogo, original,	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1477	categoriasDisponibles, nombre, subCategoria, precio, descripcion, categoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1478	disponibilidad, tamano, estado, stock, stockMinimo, esCombo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1479	rutaImagen[0], imagenNueva[0], error, guardar, guardado));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1480	java.awt.event.MouseWheelListener ruedaFormulario = evento -> {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1481	int avance = (int) Math.round(evento.getPreciseWheelRotation()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1482	* 3 * barraVertical.getUnitIncrement());	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1483	if (avance != 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1484	barraVertical.setValue(barraVertical.getValue() + avance);	Configura una propiedad mediante setValue en el objeto indicado.
1485	evento.consume();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1486	}	Cierra el bloque abierto anteriormente.
1487	};	Cierra el bloque abierto anteriormente y termina la declaración.
1488	escucharRuedaEnFormulario(tarjeta, ruedaFormulario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1489	return tarjeta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1490	}	Cierra el bloque abierto anteriormente.
1491		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1492	private void escucharRuedaEnFormulario(Component componente,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1493	java.awt.event.MouseWheelListener oyente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1494	componente.addMouseListener(oyente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1495	if (componente instanceof java.awt.Container contenedor) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1496	for (Component hijo : contenedor.getComponents()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1497	escucharRuedaEnFormulario(hijo, oyente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1498	}	Cierra el bloque abierto anteriormente.
1499	}	Cierra el bloque abierto anteriormente.
1500	}	Cierra el bloque abierto anteriormente.
1501		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1502	private JPanel crearCampo(String titulo, javax.swing.JComponent control) {	Declara el método crearCampo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1503	JPanel campo = new JPanel(new BorderLayout(0, 5));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1504	campo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1505	JLabel etiqueta = new JLabel(titulo);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1506	etiqueta.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1507	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1508	campo.add(etiqueta, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1509	if (control instanceof JTextField control instanceof BotonDesplegable	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1510	control instanceof JSpinner control instanceof JCheckBox) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1511	control.setPreferredSize(new Dimension(320, 46));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1512	}	Cierra el bloque abierto anteriormente.
1513	campo.add(control, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1514	return campo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1515	}	Cierra el bloque abierto anteriormente.

1516		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1517	private void agregarFila(JPanel panel, int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1518	JPanel izquierda, JPanel derecha) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1519	GridBagConstraints gbc = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1520	gbc.gridy = fila;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1521	gbc.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1522	gbc.weightx = 0.5;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1523	gbc.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1524	gbc.anchor = GridBagConstraints.NORTH;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1525	gbc.insets = new Insets(5, 8, 8, 12);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1526	panel.add(izquierda, gbc);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1527	gbc.gridx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1528	gbc.insets = new Insets(5, 12, 8, 8);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1529	panel.add(derecha, gbc);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1530	}	Cierra el bloque abierto anteriormente.
1531		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1532	private void agregarFilaCompleta(JPanel panel, int fila, JPanel contenido) {	Declara el método agregarFilaCompleta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1533	GridBagConstraints gbc = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1534	gbc.gridy = fila;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1535	gbc.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1536	gbc.gridwidth = 2;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1537	gbc.weightx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1538	gbc.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1539	gbc.insets = new Insets(5, 8, 8, 8);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1540	panel.add(contenido, gbc);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1541	}	Cierra el bloque abierto anteriormente.
1542		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1543	private void aplicarEstiloCampo(JTextField campo) {	Declara el método aplicarEstiloCampo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1544	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1545	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1546	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
1547	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1548	new EmptyBorder(0, 13, 0, 13)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1549	}	Cierra el bloque abierto anteriormente.
1550		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1551	private BotonDesplegable nuevoSelector(String texto, String opciones) {	Declara el método nuevoSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1552	BotonDesplegable selector = new BotonDesplegable();	Crea una nueva instancia de BotonDesplegable y la usa o guarda para continuar el proceso.
1553	configurarSelector(selector, 330);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1554	selector.setTextoDesplegable(opciones);	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
1555	selector.setText(texto);	Cambia el texto visible de un componente de la interfaz.
1556	selector.addMenuOpcionListener(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1557	evento -> selector.setText(evento.getActionCommand()));	Cambia el texto visible de un componente de la interfaz.
1558	return selector;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

1559	}	Cierra el bloque abierto anteriormente.
1560		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1561	private void estilizarSpinner(JSpinner spinner) {	Declara el método estilizarSpinner. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1562	spinner.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1563	spinner.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1564	spinner.setBorder(BorderFactory.createLineBorder(BORDE, 1, true));	Configura una propiedad mediante setBorder en el objeto indicado.
1565	if (spinner.getEditor() instanceof JSpinner.DefaultEditor editor) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1566	editor.getTextField().setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1567	editor.getTextField().setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1568	editor.getTextField().setBorder(new EmptyBorder(0, 13, 0, 13));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1569	}	Cierra el bloque abierto anteriormente.
1570	}	Cierra el bloque abierto anteriormente.
1571		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1572	private BotonRedondeado botonFormulario(String texto, boolean principal) {	Declara el método botonFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1573	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
1574	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
1575	boton.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1576	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1577	boton.setColorInicio(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
1578	boton.setColorFinal(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
1579	boton.setColorBorde(principal ? AMARILLO : ROJO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1580	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1581	boton.setForeground(principal ? AZUL : ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
1582	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
1583	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1584	}	Cierra el bloque abierto anteriormente.
1585		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1586	private void mostrarVistaPrevio(JLabel etiqueta, String ruta) {	Declara el método mostrarVistaPrevio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1587	etiqueta.setIcon(null);	Configura una propiedad mediante setIcon en el objeto indicado.
1588	etiqueta.setText("Sin imagen");	Cambia el texto visible de un componente de la interfaz.
1589	ImageIcon icono = cargarImagen(ruta, 75);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1590	if (icono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1591	etiqueta.setIcon(icono);	Configura una propiedad mediante setIcon en el objeto indicado.
1592	etiqueta.setText("");	Cambia el texto visible de un componente de la interfaz.
1593	}	Cierra el bloque abierto anteriormente.
1594	}	Cierra el bloque abierto anteriormente.
1595		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1596	private String nombreTipoStock(String codigo) {	Declara el método nombreTipoStock. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1597	if (codigo == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1598	return "Stock directo";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1599	}	Cierra el bloque abierto anteriormente.
1600	return switch (codigo) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1601	case "RECETA" -> "Por receta";	Define una de las rutas posibles dentro del switch.

1602	case "NINGUNO" -> "Sin control de stock";	Define una de las rutas posibles dentro del switch.
1603	default ->	Define una de las rutas posibles dentro del switch.
1604	"Stock directo";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1605	};	Cierra el bloque abierto anteriormente y termina la declaración.
1606	}	Cierra el bloque abierto anteriormente.
1607		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1608	private void guardarProducto(JDialog dialogo, Producto original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1609	Map<Integer, String> categoriasDisponibles,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1610	JTextField nombre, JTextField subCategoria, JTextField precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1611	JTextArea descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1612	BotonDesplegable categoria, BotonDesplegable disponibilidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1613	BotonDesplegable tamano, BotonDesplegable estado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1614	JSpinner stock, JSpinner stockMinimo, JCheckBox esCombo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1615	String rutaImagen, File imagenNueva, JLabel error,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1616	BotonRedondeado guardar, boolean[] guardado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1617	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
1618	String nombreValor = nombre.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1619	String subCategoriaValor = subCategoria.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1620	String descripcionValor = descripcion.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1621	if (nombreValor.isBlank() nombreValor.length() > 100) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1622	error.setText("Ingresa un nombre de hasta 100 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1623	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1624	}	Cierra el bloque abierto anteriormente.
1625	if (descripcionValor.length() > 255) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1626	error.setText("La descripción admite hasta 255 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1627	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1628	}	Cierra el bloque abierto anteriormente.
1629	if (subCategoriaValor.length() > 50) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1630	error.setText("La subcategoría admite hasta 50 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1631	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1632	}	Cierra el bloque abierto anteriormente.
1633	Integer idCategoria = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1634	for (Map.Entry<Integer, String> entrada : categoriasDisponibles.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1635	if (entrada.getValue().equals(categoria.getText())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1636	idCategoria = entrada.getKey();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1637	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1638	}	Cierra el bloque abierto anteriormente.
1639	}	Cierra el bloque abierto anteriormente.
1640	if (idCategoria == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1641	error.setText("Selecciona una categoría.");	Cambia el texto visible de un componente de la interfaz.
1642	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1643	}	Cierra el bloque abierto anteriormente.
1644	BigDecimal precioValor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1645	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

1646	precioValor = new BigDecimal(precio.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1647	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1648	error.setText("Ingresa un precio válido.");	Cambia el texto visible de un componente de la interfaz.
1649	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1650	}	Cierra el bloque abierto anteriormente.
1651	if (precioValor.compareTo(BigDecimal.ZERO) < 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1652	precioValor.compareTo(new BigDecimal("99999999.99")) > 0	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1653	precioValor.stripTrailingZeros().scale() > 2) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1654	error.setText("El precio debe ser positivo y tener hasta dos decimales.");	Cambia el texto visible de un componente de la interfaz.
1655	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1656	}	Cierra el bloque abierto anteriormente.
1657	String codigoDisponibilidad = switch (disponibilidad.getText()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1658	case "Desayuno" ->	Define una de las rutas posibles dentro del switch.
1659	"DESAYUNO";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1660	case "Almuerzo" ->	Define una de las rutas posibles dentro del switch.
1661	"ALMUERZO";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1662	default ->	Define una de las rutas posibles dentro del switch.
1663	"TODO_DIA";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1664	};	Cierra el bloque abierto anteriormente y termina la declaración.
1665	String codigoTamano = switch (tamano.getText()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1666	case "Por receta" -> "RECETA";	Define una de las rutas posibles dentro del switch.
1667	case "Sin control de stock" -> "NINGUNO";	Define una de las rutas posibles dentro del switch.
1668	default -> "DIRECTO";	Define una de las rutas posibles dentro del switch.
1669	};	Cierra el bloque abierto anteriormente y termina la declaración.
1670	String imagenCopiada = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1671	guardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
1672	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1673	if (imagenNueva != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1674	imagenCopiada = copiarImagenProducto(imagenNueva);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1675	rutaImagen = imagenCopiada;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1676	}	Cierra el bloque abierto anteriormente.
1677	Producto producto = original == null ? new Producto() : original;	Crea una nueva instancia de Producto y la usa o guarda para continuar el proceso.
1678	producto.setIdCategoria(idCategoria);	Configura una propiedad mediante setIdCategoria en el objeto indicado.
1679	producto.setCategoria(categoria.getText());	Configura una propiedad mediante setCategoria en el objeto indicado.
1680	producto.setSubCategoria(subCategoriaValor.isBlank()	Configura una propiedad mediante setSubCategoria en el objeto indicado.
1681	? null : subCategoriaValor);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1682	producto.setNombre(nombreValor);	Configura una propiedad mediante setNombre en el objeto indicado.
1683	producto.setDescripcion(descripcionValor.isBlank()	Configura una propiedad mediante setDescripcion en el objeto indicado.
1684	? null : descripcionValor);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1685	producto.setPrecioBase(precioValor);	Configura una propiedad mediante setPrecioBase en el objeto indicado.
1686	producto.setDisponibilidadMenu(codigoDisponibilidad);	Configura una propiedad mediante setDisponibilidadMenu en el objeto indicado.
1687	producto.setTipoStock(codigoTamano);	Configura una propiedad mediante setTipoStock en el objeto indicado.
1688	producto.setPersonalizable(esCombo.isSelected());	Configura una propiedad mediante setPersonalizable en el objeto indicado.
1689	producto.setStockActual(((Number) stock.getValue()).intValue());	Configura una propiedad mediante setStockActual en el objeto indicado.

1690	<code>producto.setStockMinimo(((Number) stockMinimo.getValue()).intValue());</code>	Configura una propiedad mediante <code>setStockMinimo</code> en el objeto indicado.
1691	<code>producto.setActivo("Activo".equals(estado.getText()));</code>	Configura una propiedad mediante <code>setActivo</code> en el objeto indicado.
1692	<code>producto.setImagen(rutaImagen);</code>	Configura una propiedad mediante <code>setImagen</code> en el objeto indicado.
1693	<code>if (original == null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1694	<code> productoCRUD.insertar(producto);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1695	<code> } else {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1696	<code> productoCRUD.actualizar(producto);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1697	<code> }</code>	Cierra el bloque abierto anteriormente.
1698	<code> guardado[0] = true;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1699	<code> dialogo.dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1700	<code> } catch (SQLException IOException ex) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1701	<code> if (imagenCopiada != null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1702	<code> try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1703	<code> Files.deleteIfExists(Paths.get("src", imagenCopiada.substring(1)));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1704	<code> } catch (IOException ignorada) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1705	<code> /* Se conserva el error original. */ }</code>	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1706	<code> }</code>	Cierra el bloque abierto anteriormente.
1707	<code> error.setText("No fue posible guardar: " + ex.getMessage());</code>	Cambia el texto visible de un componente de la interfaz.
1708	<code> } finally {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1709	<code> guardar.setEnabled(true);</code>	Habilita o deshabilita la interacción con el componente.
1710	<code> }</code>	Cierra el bloque abierto anteriormente.
1711	<code>}</code>	Cierra el bloque abierto anteriormente.
1712		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1713		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1714	<code>// Variables declaration - do not modify//GEN-BEGIN:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1715	<code>private Componentes.BotonDerretido botonAgregarProducto;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1716	<code>private Componentes.BotonRedondeado botonAnterior;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1717	<code>private Componentes.BotonRedondeado botonExportar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1718	<code>private Componentes.BotonRedondeado botonLimpiarFiltros;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1719	<code>private Componentes.BotonRedondeado botonPagina1;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1720	<code>private Componentes.BotonRedondeado botonPagina2;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1721	<code>private Componentes.BotonRedondeado botonPagina3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1722	<code>private Componentes.BotonRedondeado botonSiguiente;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1723	<code>private Componentes.CampoBusquedaAdmin campoBusqueda;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1724	<code>private javax.swing.JLabel etiquetaRango;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1725	<code>private Componentes.BotonDesplegable filtroCategoria;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1726	<code>private Componentes.BotonDesplegable filtroEstado;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1727	<code>private javax.swing.JLabel labelActivos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1728	<code>private Labels.LabelEscalable labelEscalable2;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1729	<code>private Labels.LabelEscalable labelEscalable3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1730	<code>private Labels.LabelEscalable labelEscalable4;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1731	<code>private Labels.LabelEscalable labelEscalable5;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1732	<code>private javax.swing.JLabel labelInactivos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1733	<code>private javax.swing.JLabel labelStockBajo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1734	<code>private javax.swing.JLabel labelTitulo10;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1735	<code>private javax.swing.JLabel labelTitulo12;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1736	<code>private javax.swing.JLabel labelTitulo13;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1737	<code>private javax.swing.JLabel labelTitulo15;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1738	<code>private javax.swing.JLabel labelTitulo2;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1739	<code>private javax.swing.JLabel labelTitulo3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1740	<code>private javax.swing.JLabel labelTitulo4;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1741	<code>private javax.swing.JLabel labelTitulo6;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1742	<code>private javax.swing.JLabel labelTitulo7;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1743	<code>private javax.swing.JLabel labelTitulo9;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1744	<code>private javax.swing.JLabel labelTotalProductos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1745	<code>private Componentes.PanelCircular panelCircular2;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1746	<code>private Componentes.PanelCircular panelCircular3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1747	<code>private Componentes.PanelCircular panelCircular4;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1748	<code>private Componentes.PanelCircular panelCircular5;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1749	<code>private Componentes.PanelFlotante panelFiltros;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1750	<code>private Componentes.PanelFlotante panelFlotante1;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1751	<code>private Componentes.PanelFlotante panelFlotante2;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1752	<code>private Componentes.PanelFlotante panelFlotante3;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1753	<code>private Componentes.PanelFlotante panelFlotante4;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1754	<code>private javax.swing.JPanel panelPaginacion;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1755	<code>private Componentes.PanelFlotante panelTabla;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1756	<code>private javax.swing.JScrollPane scrollUsuarios;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1757	<code>private Componentes.TablaAdministrativa tablaProductos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1758	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1759	<code>}</code>	Cierra el bloque abierto anteriormente.

16.64 src/GUI_ADMINISTRADOR/IconoMenuAdmin.java

Interfaz del módulo administrador: IconoMenuAdmin. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 29 | Archivo 64 de 157

Clases/tipos detectados: IconoMenuAdmin

Línea	Constructor o método detectado
16	public IconoMenuAdmin() {
23	protected void paintComponent(Graphics graphics) {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Utilidades.IconosAdmin;	Importa Utilidades.IconosAdmin para poder usar esa clase sin escribir su nombre completo.
4	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
8	import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	/** Icono vectorial del menú utilizado por el JFrame Form administrativo. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
12	public final class IconoMenuAdmin extends JComponent {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	private final TemaAdmin tema;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	public IconoMenuAdmin() {	Constructor de IconoMenuAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
17	tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
18	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
19	setPreferredSize(new Dimension(180, 180));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
20	}	Cierra el bloque abierto anteriormente.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
23	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
24	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
25	TemaAdmin.aplicarCalidad(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
26	IconosAdmin.pintarTicket(g2, getWidth() / 2, getHeight() / 2, tema);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	}	Cierra el bloque abierto anteriormente.
29	}	Cierra el bloque abierto anteriormente.

16.65 src/GUI_ADMINISTRADOR/IngredienteFormDialog.java

Interfaz del módulo administrador: IngredienteFormDialog. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 46 | Archivo 65 de 157

Clases/tipos detectados: IngredienteFormDialog

Línea	Constructor o método detectado
15	private IngredienteFormDialog(Window propietario, IngredienteCRUD crud,
39	public static boolean mostrar(Window propietario, IngredienteCRUD crud,

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.IngredienteCRUD;	Importa CRUD.IngredienteCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	/** Ventana modal; el contenido editable vive en IngredienteFormPanel.form. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	@SuppressWarnings("serial")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
11	public final class IngredienteFormDialog extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	private boolean guardado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	private IngredienteFormDialog(Window propietario, IngredienteCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	Ingrediente ingrediente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	super(propietario, ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
19	setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
20	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	IngredienteFormPanel formulario =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	new IngredienteFormPanel(crud, ingrediente);	Crea una nueva instancia de IngredienteFormPanel y la usa o guarda para continuar el proceso.
24	formulario.addPropertyChangeListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25	if (IngredienteFormPanel.EVENTO_GUARDADO.equals(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
26	evento.getPropertyName()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	guardado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
28	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	} else if (IngredienteFormPanel.EVENTO_CANCELAR.equals(	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
30	evento.getPropertyName()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	}	Cierra el bloque abierto anteriormente.
33	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.

34	setContentPane(formulario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	}	Cierra el bloque abierto anteriormente.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	public static boolean mostrar(Window propietario, IngredienteCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	Ingrediente ingrediente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	IngredienteFormDialog dialogo = new IngredienteFormDialog(	Crea una nueva instancia de IngredienteFormDialog y la usa o guarda para continuar el proceso.
42	propietario, crud, ingrediente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
44	return dialogo.guardado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
45	}	Cierra el bloque abierto anteriormente.
46	}	Cierra el bloque abierto anteriormente.

16.66 src/GUI_ADMINISTRADOR/IngredienteFormPanel.form

Metadatos visuales de NetBeans para diseñar IngredienteFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 60 | Archivo 66 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor"><Dimension value="[760, 515]"/></Property>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
7	</Properties>	Cierra la sección XML abierta anteriormente.
8	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	</AuxValues>	Cierra la sección XML abierta anteriormente.
19	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
20	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<Container class="Componentes.PanelFlotante" name="panelTarjeta">	Declara un componente o contenedor visual y su clase Java.
22	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="ea" green="e3" red="de" type="rgb"/></Property>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	<Property name="radio" type="int" value="26"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
25	</Properties>	Cierra la sección XML abierta anteriormente.
26	<Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="0" y="0" width="760" height="515"/></Constraint></Constraints>	Define la posición y el tamaño del componente dentro del diseño.
27	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
28	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<Container class="Componentes.PanelCircular" name="panelIcono">	Declara un componente o contenedor visual y su clase Java.
30	<Properties><Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="d3" green="f4" red="ff" type="rgb"/></Property></Properties>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
31	<Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="34" y="26" width="70" height="70"/></Constraint></Constraints>	Define la posición y el tamaño del componente dentro del diseño.
32	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

33	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
34	<Component class="Labels.LabelEscala" name="labelIcono">	Declara un componente o contenedor visual y su clase Java.
35	<Properties><Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editor.s2.IconEditor"><Image iconType="3" name="/Imagenes/lista_icono.png"/></Property><Property name="mantenerProporcion" type="boolean" value="true"/></Properties>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
36	<Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="15" y="15" width="40" height="40"/></Constraint></Constraints>	Define la posición y el tamaño del componente dentro del diseño.
37	</Component>	Cierra la sección XML abierta anteriormente.
38	</SubComponents>	Cierra la sección XML abierta anteriormente.
39	</Container>	Cierra la sección XML abierta anteriormente.
40	<Component class="javax.swing.JLabel" name="labelTitulo"><Properties><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="2b" green="14" red="0" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Agregar ingrediente"/></Properties><Constraint><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="125" y="27" width="590" height="34"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
41	<Component class="javax.swing.JLabel" name="labelSubtitulo"><Properties><Property name="text" type="java.lang.String" value="Completa los datos del nuevo ingrediente"/></Properties><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="125" y="62" width="590" height="26"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
42	<Component class="javax.swing.JLabel" name="labelNombre"><Properties><Property name="text" type="java.lang.String" value="Nombre"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="120" width="325" height="24"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
43	<Component class="javax.swing.JTextField" name="campoNombre"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="148" width="325" height="46"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
44	<Component class="javax.swing.JLabel" name="labelUnidad"><Properties><Property name="text" type="java.lang.String" value="Unidad de medida"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="395" y="120" width="325" height="24"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
45	<Component class="javax.swing.JTextField" name="campoUnidad"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="395" y="148" width="325" height="46"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
46	<Component class="javax.swing.JLabel" name="labelStockActual"><Properties><Property name="text" type="java.lang.String" value="Stock actual (solo lectura)/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="220" width="325" height="24"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
47	<Component class="javax.swing.JTextField" name="campoStockActual"><Properties><Property name="editable" type="boolean" value="false"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="248" width="325" height="46"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
48	<Component class="javax.swing.JLabel" name="labelStockMinimo"><Properties><Property name="text" type="java.lang.String" value="Stock mínimo"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="395" y="220" width="325" height="24"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
49	<Component class="javax.swing.JTextField" name="campoStockMinimo"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="395" y="248" width="325" height="46"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.

50	<pre><Component class="javax.swing.JLabel" name="labelEstado"><Properties><Property name="text" type="java.lang.String" value="Estado"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="320" width="325" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
51	<pre><Component class="Componentes.BotonDesplegable" name="selectorEstado"><Properties><Property name="text" type="java.lang.String" value="Activo"/><Property name="textoDesplegable" type="java.lang.String" value="Activo; Inactivo"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="348" width="325" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
52	<pre><Component class="javax.swing.JLabel" name="labelNota"><Properties><Property name="text" type="java.lang.String" value="<html>El stock se modifica desde Existencias, mediante una entrada, salida o ajuste.</html>"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="395" y="345" width="325" height="52"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
53	<pre><Component class="javax.swing.JLabel" name="labelError"><Properties><Property name="text" type="java.lang.String" value=""/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="410" width="680" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
54	<pre><Component class="Componentes.BotonRedondeado" name="botonDesactivar"><Properties><Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="degradado" type="boolean" value="false"/><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Desactivar ingrediente"/></Properties><Events><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonDesactivarActionPerformed"/></Events><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="40" y="449" width="230" height="46"/></Constraint></Component></pre>	Declara un componente o contenedor visual y su clase Java.
55	<pre><Component class="Componentes.BotonRedondeado" name="botonCancelar"><Properties><Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="degradado" type="boolean" value="false"/><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Cancelar"/></Properties><Events><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonCancelarActionPerformed"/></Events><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="365" y="449" width="140" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
56	<pre><Component class="Componentes.BotonDerretido" name="botonGuardar"><Properties><Property name="text" type="java.lang.String" value="Guardar ingrediente"/></Properties><Events><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonGuardarActionPerformed"/></Events><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="520" y="449" width="200" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
57	<pre></SubComponents></pre>	Cierra la sección XML abierta anteriormente.
58	<pre></Container></pre>	Cierra la sección XML abierta anteriormente.
59	<pre></SubComponents></pre>	Cierra la sección XML abierta anteriormente.
60	<pre></Form></pre>	Cierra la sección XML abierta anteriormente.

16.67 src/GUI_ADMINISTRADOR/IngredienteFormPanel.java

Interfaz del módulo administrador: IngredienteFormPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 295 | Archivo 67 de 157

Clases/tipos detectados: IngredienteFormPanel

Línea	Constructor o método detectado
32	public IngredienteFormPanel() {
36	public IngredienteFormPanel(IngredienteCRUD crud, Ingrediente original) {
45	private void configurarAspecto() {
74	private void configurarSelector() {
90	private void cargarModo() {
112	private void guardar() {
158	private void desactivar() {
174	private void initComponents() {
261	private void botonDesactivarActionPerformed(java.awt.event.ActionEvent evt) {
265	private void botonCancelarActionPerformed(java.awt.event.ActionEvent evt) {
269	private void botonGuardarActionPerformed(java.awt.event.ActionEvent evt) {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.IngredienteCRUD;	Importa CRUD.IngredienteCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
5	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
9	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
10	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	/** Formulario visual editable desde Design para crear o editar ingredientes. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
16	public class IngredienteFormPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	public static final String EVENTO_GUARDADO = "ingredienteGuardado";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	public static final String EVENTO_CANCELAR = "cancelarIngrediente";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

22	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
23	private static final Color BORDE = new Color(222, 227, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
24	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
25	private static final BigDecimal MAXIMO = new BigDecimal("99999999.99");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
28	private IngredienteCRUD crud;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
29	private Ingrediente original;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	private boolean activo = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	public IngredienteFormPanel() {	Constructor de IngredienteFormPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
33	this(null, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	}	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	public IngredienteFormPanel(IngredienteCRUD crud, Ingrediente original) {	Constructor de IngredienteFormPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
37	this.crud = crud;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	this.original = original;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
39	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	configurarAspecto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	configurarSelector();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	cargarModo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	private void configurarAspecto() {	Declara el método configurarAspecto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
46	labelTitulo.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
47	labelSubtitulo.setFont(tema.regular(15f));	Configura una propiedad mediante setFont en el objeto indicado.
48	labelSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
49	labelError.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
50	labelError.setForeground(ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
51	labelNota.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
52	labelNota.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	for (javax.swing.JLabel etiqueta : new javax.swing.JLabel[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
55	labelNombre, labelUnidad, labelStockActual, labelStockMinimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	labelEstado}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	etiqueta.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
58	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
59	}	Cierra el bloque abierto anteriormente.
60	for (JTextField campo : new JTextField[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
61	campoNombre, campoUnidad, campoStockActual, campoStockMinimo}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
63	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
64	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
65	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

66	<code>new EmptyBorder(0, 13, 0, 13));</code>	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
67	<code>}</code>	Cierra el bloque abierto anteriormente.
68	<code>campoStockActual.setBackground(new Color(248, 249, 251));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
69	<code>botonGuardar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
70	<code>botonCancelar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
71	<code>botonDesactivar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
72	<code>}</code>	Cierra el bloque abierto anteriormente.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	<code>private void configurarSelector() {</code>	Declara el método configurarSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
75	<code>selectorEstado.setFont(tema.regular(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
76	<code>selectorEstado.setForeground(AZUL);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
77	<code>selectorEstado.setColorFondo(Color.WHITE);</code>	Configura una propiedad mediante setColorFondo en el objeto indicado.
78	<code>selectorEstado.setColorHover(new Color(248, 249, 251));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
79	<code>selectorEstado.setColorDesplegado(new Color(255, 247, 222));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
80	<code>selectorEstado.setColorTextoOpcion(AZUL);</code>	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
81	<code>selectorEstado.setColorBordeMenu(BORDE);</code>	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
82	<code>selectorEstado.setAnchoMenu(325);</code>	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
83	<code>selectorEstado.setAltoOpcion(42);</code>	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
84	<code>selectorEstado.addMenuOpcionListener(evento -> {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
85	<code>activo = "Activo".equals(evento.getActionCommand());</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
86	<code>selectorEstado.setText(evento.getActionCommand());</code>	Cambia el texto visible de un componente de la interfaz.
87	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	<code>}</code>	Cierra el bloque abierto anteriormente.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	<code>private void cargarModo() {</code>	Declara el método cargarModo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
91	<code>boolean editando = original != null;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	<code>labelTitulo.setText(editando ? "Editar ingrediente" : "Agregar ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
93	<code>labelSubtitulo.setText(editando</code>	Cambia el texto visible de un componente de la interfaz.
94	<code> ? "Actualiza la información de "</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	<code> + String.format("#%04d", original.getIdIngrediente())</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	<code> : "Completa los datos del nuevo ingrediente");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	<code>botonDesactivar.setVisible(editando && original.isActivo());</code>	Muestra u oculta una ventana o componente según el valor booleano enviado.
98	<code>if (!editando) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
99	<code> campoStockActual.setText("0.00");</code>	Cambia el texto visible de un componente de la interfaz.
100	<code> campoStockMinimo.setText("0.00");</code>	Cambia el texto visible de un componente de la interfaz.
101	<code> selectorEstado.setText("Activo");</code>	Cambia el texto visible de un componente de la interfaz.
102	<code>return;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	<code>}</code>	Cierra el bloque abierto anteriormente.
104	<code>campoNombre.setText(original.getNombre());</code>	Cambia el texto visible de un componente de la interfaz.
105	<code>campoUnidad.setText(original.getUnidadMedida());</code>	Cambia el texto visible de un componente de la interfaz.
106	<code>campoStockActual.setText(original.getStockActual().toString());</code>	Cambia el texto visible de un componente de la interfaz.
107	<code>campoStockMinimo.setText(original.getStockMinimo().toString());</code>	Cambia el texto visible de un componente de la interfaz.
108	<code>activo = original.isActivo();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
109	<code>selectorEstado.setText(activo ? "Activo" : "Inactivo");</code>	Cambia el texto visible de un componente de la interfaz.

110	}	Cierra el bloque abierto anteriormente.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
113	labelError.setText(" ");	Cambia el texto visible de un componente de la interfaz.
114	if (crud == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
115	labelError.setText("El formulario no tiene conexión CRUD configurada.");	Cambia el texto visible de un componente de la interfaz.
116	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
117	}	Cierra el bloque abierto anteriormente.
118	String nombre = campoNombre.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119	String unidad = campoUnidad.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
120	if (nombre.isEmpty() nombre.length() > 100) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
121	labelError.setText("Ingresa un nombre de hasta 100 caracteres.");	Cambia el texto visible de un componente de la interfaz.
122	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
123	}	Cierra el bloque abierto anteriormente.
124	if (unidad.isEmpty() unidad.length() > 20) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
125	labelError.setText("Ingresa una unidad de hasta 20 caracteres.");	Cambia el texto visible de un componente de la interfaz.
126	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
127	}	Cierra el bloque abierto anteriormente.
128	BigDecimal minimo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
129	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
130	minimo = new BigDecimal(campoStockMinimo.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
131	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
132	labelError.setText("Ingresa un stock mínimo válido. Usa punto decimal.");	Cambia el texto visible de un componente de la interfaz.
133	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
134	}	Cierra el bloque abierto anteriormente.
135	if (minimo.signum() < 0 minimo.scale() > 2	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
136	minimo.compareTo(MAXIMO) > 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	labelError.setText("El mínimo debe ser positivo y tener hasta dos decimales.");	Cambia el texto visible de un componente de la interfaz.
138	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
139	}	Cierra el bloque abierto anteriormente.
140	botonGuardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
141	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
142	Ingrediente ingrediente = original == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
143	? new Ingrediente() : original;	Crea una nueva instancia de Ingrediente y la usa o guarda para continuar el proceso.
144	ingrediente.setNombre(nombre);	Configura una propiedad mediante setNombre en el objeto indicado.
145	ingrediente.setUnidadMedida(unidad);	Configura una propiedad mediante setUnidadMedida en el objeto indicado.
146	ingrediente.setStockMinimo(minimo);	Configura una propiedad mediante setStockMinimo en el objeto indicado.
147	ingrediente.setActivo(activo);	Configura una propiedad mediante setActivo en el objeto indicado.
148	if (original == null) crud.insertar(ingrediente);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
149	else crud.actualizar(ingrediente);	Ruta alternativa que se ejecuta cuando la condición anterior fue falsa.
150	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
152	labelError.setText("No fue posible guardar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
153	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

154	botonGuardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
155	}	Cierra el bloque abierto anteriormente.
156	}	Cierra el bloque abierto anteriormente.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	private void desactivar() {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
159	if (original == null) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
160	int respuesta = JOptionPane.showConfirmDialog(this,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
161	"¿Desactivar «" + original.getNombre() + "»?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
162	"Confirmar", JOptionPane.YES_NO_OPTION);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	if (respuesta != JOptionPane.YES_OPTION) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
164	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
165	crud.cambiarEstado(original.getIdIngrediente(), false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
166	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
168	labelError.setText("No fue posible desactivar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
169	}	Cierra el bloque abierto anteriormente.
170	}	Cierra el bloque abierto anteriormente.
171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
173	// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
174	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
175	panelTarjeta = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
176	panelIcono = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
177	labelIcono = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
178	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
179	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
180	labelNombre = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
181	campoNombre = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
182	labelUnidad = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
183	campoUnidad = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
184	labelStockActual = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
185	campoStockActual = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
186	labelStockMinimo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
187	campoStockMinimo = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
188	labelEstado = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
189	selectorEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
190	labelNota = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
191	labelError = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
192	botonDesactivar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
193	botonCancelar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
194	botonGuardar = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	setPreferredSize(new java.awt.Dimension(760, 515));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.

198	<code>setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	<code>panelTarjeta.setColorBorde(new java.awt.Color(222, 227, 234));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
201	<code>panelTarjeta.setRadio(26);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
202	<code>panelTarjeta.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
203	<code>panelIcono.setColorFondo(new java.awt.Color(255, 244, 211));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
204	<code>panelIcono.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
205	<code>labelIcono.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/lista_icono.png")));</code>	Crea una nueva instancia de <code>javax.swing.ImageIcon</code> y la usa o guarda para continuar el proceso.
206	<code>labelIcono.setMantenerProporcion(true);</code>	Configura una propiedad mediante <code>setMantenerProporcion</code> en el objeto indicado.
207	<code>panelIcono.add(labelIcono, new org.netbeans.lib.awtextra.AbsoluteConstraints(15, 15, 40, 40));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
208	<code>panelTarjeta.add(panelIcono, new org.netbeans.lib.awtextra.AbsoluteConstraints(34, 26, 70, 70));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
209	<code>labelTitulo.setForeground(new java.awt.Color(0, 20, 43));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
210	<code>labelTitulo.setText("Agregar ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
211	<code>panelTarjeta.add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(125, 27, 590, 34));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
212	<code>labelSubtitulo.setText("Completa los datos del nuevo ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
213	<code>panelTarjeta.add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(125, 62, 590, 26));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	<code>labelNombre.setText("Nombre");</code>	Cambia el texto visible de un componente de la interfaz.
216	<code>panelTarjeta.add(labelNombre, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 120, 325, 24));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
217	<code>panelTarjeta.add(campoNombre, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 148, 325, 46));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
218	<code>labelUnidad.setText("Unidad de medida");</code>	Cambia el texto visible de un componente de la interfaz.
219	<code>panelTarjeta.add(labelUnidad, new org.netbeans.lib.awtextra.AbsoluteConstraints(395, 120, 325, 24));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
220	<code>panelTarjeta.add(campoUnidad, new org.netbeans.lib.awtextra.AbsoluteConstraints(395, 148, 325, 46));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
221	<code>labelStockActual.setText("Stock actual (solo lectura)");</code>	Cambia el texto visible de un componente de la interfaz.
222	<code>panelTarjeta.add(labelStockActual, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 220, 325, 24));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
223	<code>campoStockActual.setEditable(false);</code>	Configura una propiedad mediante <code>setEditable</code> en el objeto indicado.
224	<code>panelTarjeta.add(campoStockActual, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 248, 325, 46));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
225	<code>labelStockMinimo.setText("Stock mínimo");</code>	Cambia el texto visible de un componente de la interfaz.
226	<code>panelTarjeta.add(labelStockMinimo, new org.netbeans.lib.awtextra.AbsoluteConstraints(395, 220, 325, 24))</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
227	<code>panelTarjeta.add(campoStockMinimo, new org.netbeans.lib.awtextra.AbsoluteConstraints(395, 248, 325, 46))</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
228	<code>labelEstado.setText("Estado");</code>	Cambia el texto visible de un componente de la interfaz.
229	<code>panelTarjeta.add(labelEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 320, 325, 24));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
230	<code>selectorEstado.setText("Activo");</code>	Cambia el texto visible de un componente de la interfaz.
231	<code>selectorEstado.setTextoDesplegable("Activo;Inactivo");</code>	Configura una propiedad mediante <code>setTextoDesplegable</code> en el objeto indicado.
232	<code>panelTarjeta.add(selectorEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 348, 325, 46));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
233	<code>labelNota.setText("<html>El stock se modifica desde Existencias, "</code>	Cambia el texto visible de un componente de la interfaz.
234	<code>+ "mediante una entrada, salida o ajuste.</html>");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
235	<code>panelTarjeta.add(labelNota, new org.netbeans.lib.awtextra.AbsoluteConstraints(395, 345, 325, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
236	<code>labelError.setText(" ");</code>	Cambia el texto visible de un componente de la interfaz.
237	<code>panelTarjeta.add(labelError, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 410, 680, 24));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	<code>botonDesactivar.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
240	<code>botonDesactivar.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.

241	<code>botonDesactivar.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
242	<code>botonDesactivar.setDegradado(false);</code>	Configura una propiedad mediante setDegradado en el objeto indicado.
243	<code>botonDesactivar.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
244	<code>botonDesactivar.setText("Desactivar ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
245	<code>botonDesactivar.addActionListener(e -> botonDesactivarActionPerformed(e));</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
246	<code>panelTarjeta.add(botonDesactivar, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 449, 230, 46));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
247	<code>botonCancelar.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
248	<code>botonCancelar.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
249	<code>botonCancelar.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
250	<code>botonCancelar.setDegradado(false);</code>	Configura una propiedad mediante setDegradado en el objeto indicado.
251	<code>botonCancelar.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
252	<code>botonCancelar.setText("Cancelar");</code>	Cambia el texto visible de un componente de la interfaz.
253	<code>botonCancelar.addActionListener(e -> botonCancelarActionPerformed(e));</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
254	<code>panelTarjeta.add(botonCancelar, new org.netbeans.lib.awtextra.AbsoluteConstraints(365, 449, 140, 46));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
255	<code>botonGuardar.setText("Guardar ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
256	<code>botonGuardar.addActionListener(e -> botonGuardarActionPerformed(e));</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
257	<code>panelTarjeta.add(botonGuardar, new org.netbeans.lib.awtextra.AbsoluteConstraints(520, 449, 200, 46));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
258	<code>add(panelTarjeta, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 760, 515));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
259	<code>}</code>	Cierra el bloque abierto anteriormente.
260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	<code>private void botonDesactivarActionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método botonDesactivarActionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
262	<code> desactivar();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
263	<code>}</code>	Cierra el bloque abierto anteriormente.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	<code>private void botonCancelarActionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método botonCancelarActionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
266	<code> firePropertyChange(EVENTO_CANCELAR, false, true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
267	<code>}</code>	Cierra el bloque abierto anteriormente.
268		Línea en blanco: separa visualmente bloques para que el código sea más legible.
269	<code>private void botonGuardarActionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método botonGuardarActionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
270	<code> guardar();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
271	<code>}// </editor-fold>//GEN-END:initComponents</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	<code>// Variables declaration - do not modify//GEN-BEGIN:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
274	<code>private Componentes.BotonRedondeado botonCancelar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
275	<code>private Componentes.BotonRedondeado botonDesactivar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
276	<code>private Componentes.BotonDerretido botonGuardar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
277	<code>private javax.swing.JTextField campoNombre;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
278	<code>private javax.swing.JTextField campoStockActual;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
279	<code>private javax.swing.JTextField campoStockMinimo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
280	<code>private javax.swing.JTextField campoUnidad;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
281	<code>private javax.swing.JLabel labelError;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
282	<code>private Labels.LabelEscalable labelIcono;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

283	private javax.swing.JLabel labelEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
284	private javax.swing.JLabel labelNombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
285	private javax.swing.JLabel labelNota;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
286	private javax.swing.JLabel labelStockActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
287	private javax.swing.JLabel labelStockMinimo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
288	private javax.swing.JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
289	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
290	private javax.swing.JLabel labelUnidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
291	private Componentes.PanelCircular panelIcono;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
292	private Componentes.PanelFlotante panelTarjeta;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
293	private Componentes.BotonDesplegable selectorEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
294	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
295	}	Cierra el bloque abierto anteriormente.

16.68 src/GUI_ADMINISTRADOR/IngredientesPanel.form

Metadatos visuales de NetBeans para diseñar IngredientesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 708 | Archivo 68 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,-44,0,0,6,44"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="Componentes.BotonDerretido" name="botonAgregarIngrediente">	Declara un componente o contenedor visual y su clase Java.
27	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
29	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	</Property>	Cierra la sección XML abierta anteriormente.
31	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Image iconType="3" name="/Imagenes/icono_agregar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="text" type="java.lang.String" value="Agregar ingrediente"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	<Property name="iconTextGap" type="int" value="15"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
36	</Properties>	Cierra la sección XML abierta anteriormente.
37	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

38	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
39	<AbsoluteConstraints x="1290" y="50" width="230" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
40	</Constraint>	Cierra la sección XML abierta anteriormente.
41	</Constraints>	Cierra la sección XML abierta anteriormente.
42	</Component>	Cierra la sección XML abierta anteriormente.
43	<Container class="Componentes.PanelFlotante" name="panelFlotante1">	Declara un componente o contenedor visual y su clase Java.
44	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
45	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
46	<AbsoluteConstraints x="60" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
47	</Constraint>	Cierra la sección XML abierta anteriormente.
48	</Constraints>	Cierra la sección XML abierta anteriormente.
49		Línea en blanco para organizar el XML del formulario.
50	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
51	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	</Layout>	Cierra la sección XML abierta anteriormente.
53	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
54	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
55	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
56	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
57	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
58	</Constraint>	Cierra la sección XML abierta anteriormente.
59	</Constraints>	Cierra la sección XML abierta anteriormente.
60		Línea en blanco para organizar el XML del formulario.
61	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
62	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
63	</Layout>	Cierra la sección XML abierta anteriormente.
64	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
65	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
66	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	<Image iconType="3" name="/Imágenes/lista_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
69	</Property>	Cierra la sección XML abierta anteriormente.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
72	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
73	<AbsoluteConstraints x="15" y="20" width="50" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
74	</Constraint>	Cierra la sección XML abierta anteriormente.
75	</Constraints>	Cierra la sección XML abierta anteriormente.
76	</Component>	Cierra la sección XML abierta anteriormente.
77	</SubComponents>	Cierra la sección XML abierta anteriormente.
78	</Container>	Cierra la sección XML abierta anteriormente.

79	<Component class="javax.swing.JLabel" name="labelTitulo3">	Declara un componente o contenedor visual y su clase Java.
80	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
81	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
82		Metadato de NetBeans que conserva el estado editable del diseñador visual.
83	</Property>	Cierra la sección XML abierta anteriormente.
84	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
85	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
86	</Property>	Cierra la sección XML abierta anteriormente.
87	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
88	<Property name="text" type="java.lang.String" value="Ingredientes totales"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
89	</Properties>	Cierra la sección XML abierta anteriormente.
90	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
91	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
92	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
93	</Constraint>	Cierra la sección XML abierta anteriormente.
94	</Constraints>	Cierra la sección XML abierta anteriormente.
95	</Component>	Cierra la sección XML abierta anteriormente.
96	<Component class="javax.swing.JLabel" name="labelTitulo4">	Declara un componente o contenedor visual y su clase Java.
97	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
98	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
99		Metadato de NetBeans que conserva el estado editable del diseñador visual.
100	</Property>	Cierra la sección XML abierta anteriormente.
101	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
102	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
105	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
106	</Properties>	Cierra la sección XML abierta anteriormente.
107	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
108	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
109	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
110	</Constraint>	Cierra la sección XML abierta anteriormente.
111	</Constraints>	Cierra la sección XML abierta anteriormente.
112	</Component>	Cierra la sección XML abierta anteriormente.
113	<Component class="javax.swing.JLabel" name="labelTitulo2">	Declara un componente o contenedor visual y su clase Java.
114	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
115	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
116		Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	</Property>	Cierra la sección XML abierta anteriormente.
118	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
119	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

120	</Property>	Cierra la sección XML abierta anteriormente.
121	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
122	<Property name="text" type="java.lang.String" value="En el inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
123	</Properties>	Cierra la sección XML abierta anteriormente.
124	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
125	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
126	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
127	</Constraint>	Cierra la sección XML abierta anteriormente.
128	</Constraints>	Cierra la sección XML abierta anteriormente.
129	</Component>	Cierra la sección XML abierta anteriormente.
130	</SubComponents>	Cierra la sección XML abierta anteriormente.
131	</Container>	Cierra la sección XML abierta anteriormente.
132	<Component class="javax.swing.JLabel" name="labelTitulo5">	Declara un componente o contenedor visual y su clase Java.
133	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
134	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
135		Metadato de NetBeans que conserva el estado editable del diseñador visual.
136	</Property>	Cierra la sección XML abierta anteriormente.
137	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
138	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
139	</Property>	Cierra la sección XML abierta anteriormente.
140	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
141	<Property name="text" type="java.lang.String" value="Gestión de Ingredientes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
142	</Properties>	Cierra la sección XML abierta anteriormente.
143	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
144	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
145	<AbsoluteConstraints x="70" y="30" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
146	</Constraint>	Cierra la sección XML abierta anteriormente.
147	</Constraints>	Cierra la sección XML abierta anteriormente.
148	</Component>	Cierra la sección XML abierta anteriormente.
149	<Component class="javax.swing.JLabel" name="labelTitulo6">	Declara un componente o contenedor visual y su clase Java.
150	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
151	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
152		Metadato de NetBeans que conserva el estado editable del diseñador visual.
153	</Property>	Cierra la sección XML abierta anteriormente.
154	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
155	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
156	</Property>	Cierra la sección XML abierta anteriormente.
157	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
158	<Property name="text" type="java.lang.String" value="Administra el inventario de ingredientes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
159	</Properties>	Cierra la sección XML abierta anteriormente.
160	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
161	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

162	<AbsoluteConstraints x="70" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
163	</Constraint>	Cierra la sección XML abierta anteriormente.
164	</Constraints>	Cierra la sección XML abierta anteriormente.
165	</Component>	Cierra la sección XML abierta anteriormente.
166	<Container class="Componentes.PanelFlotante" name="panelFlotante2">	Declara un componente o contenedor visual y su clase Java.
167	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
168	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
169	<AbsoluteConstraints x="430" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
170	</Constraint>	Cierra la sección XML abierta anteriormente.
171	</Constraints>	Cierra la sección XML abierta anteriormente.
172		Línea en blanco para organizar el XML del formulario.
173	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
174	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
175	</Layout>	Cierra la sección XML abierta anteriormente.
176	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
177	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
178	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
179	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
180	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
181	</Property>	Cierra la sección XML abierta anteriormente.
182	</Properties>	Cierra la sección XML abierta anteriormente.
183	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
184	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
185	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
186	</Constraint>	Cierra la sección XML abierta anteriormente.
187	</Constraints>	Cierra la sección XML abierta anteriormente.
188		Línea en blanco para organizar el XML del formulario.
189	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
190	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
191	</Layout>	Cierra la sección XML abierta anteriormente.
192	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
193	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
194	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
195	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
196	<Image iconType="3" name="/Imágenes/advertencia_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
197	</Property>	Cierra la sección XML abierta anteriormente.
198	</Properties>	Cierra la sección XML abierta anteriormente.
199	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
200	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
201	<AbsoluteConstraints x="19" y="22" width="43" height="43"/>	Define la posición y el tamaño del componente dentro del diseño.
202	</Constraint>	Cierra la sección XML abierta anteriormente.

203	</Constraints>	Cierra la sección XML abierta anteriormente.
204	</Component>	Cierra la sección XML abierta anteriormente.
205	</SubComponents>	Cierra la sección XML abierta anteriormente.
206	</Container>	Cierra la sección XML abierta anteriormente.
207	<Component class="javax.swing.JLabel" name="labelTitulo7">	Declara un componente o contenedor visual y su clase Java.
208	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
209	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
210		Metadato de NetBeans que conserva el estado editable del diseñador visual.
211	</Property>	Cierra la sección XML abierta anteriormente.
212	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
213	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
214	</Property>	Cierra la sección XML abierta anteriormente.
215	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
216	<Property name="text" type="java.lang.String" value="Stock bajo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
217	</Properties>	Cierra la sección XML abierta anteriormente.
218	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
219	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
220	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
221	</Constraint>	Cierra la sección XML abierta anteriormente.
222	</Constraints>	Cierra la sección XML abierta anteriormente.
223	</Component>	Cierra la sección XML abierta anteriormente.
224	<Component class="javax.swing.JLabel" name="labelTitulo8">	Declara un componente o contenedor visual y su clase Java.
225	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
226	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
227		Metadato de NetBeans que conserva el estado editable del diseñador visual.
228	</Property>	Cierra la sección XML abierta anteriormente.
229	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
230	<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
231	</Property>	Cierra la sección XML abierta anteriormente.
232	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
233	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
234	</Properties>	Cierra la sección XML abierta anteriormente.
235	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
236	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
237	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
238	</Constraint>	Cierra la sección XML abierta anteriormente.
239	</Constraints>	Cierra la sección XML abierta anteriormente.
240	</Component>	Cierra la sección XML abierta anteriormente.
241	<Component class="javax.swing.JLabel" name="labelTitulo9">	Declara un componente o contenedor visual y su clase Java.
242	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
243	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
244		Metadato de NetBeans que conserva el estado editable del diseñador visual.

245	</Property>	Cierra la sección XML abierta anteriormente.
246	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
247	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
248	</Property>	Cierra la sección XML abierta anteriormente.
249	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
250	<Property name="text" type="java.lang.String" value="Requieren reposición"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251	</Properties>	Cierra la sección XML abierta anteriormente.
252	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
253	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
254	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
255	</Constraint>	Cierra la sección XML abierta anteriormente.
256	</Constraints>	Cierra la sección XML abierta anteriormente.
257	</Component>	Cierra la sección XML abierta anteriormente.
258	</SubComponents>	Cierra la sección XML abierta anteriormente.
259	</Container>	Cierra la sección XML abierta anteriormente.
260	<Container class="Componentes.PanelFlotante" name="panelFlotante4">	Declara un componente o contenedor visual y su clase Java.
261	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
262	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
263	<AbsoluteConstraints x="800" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
264	</Constraint>	Cierra la sección XML abierta anteriormente.
265	</Constraints>	Cierra la sección XML abierta anteriormente.
266		Línea en blanco para organizar el XML del formulario.
267	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
268	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
269	</Layout>	Cierra la sección XML abierta anteriormente.
270	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
271	<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
272	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
273	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
274	<Color blue="ea" green="f5" red="e6" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
275	</Property>	Cierra la sección XML abierta anteriormente.
276	</Properties>	Cierra la sección XML abierta anteriormente.
277	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
278	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
279	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
280	</Constraint>	Cierra la sección XML abierta anteriormente.
281	</Constraints>	Cierra la sección XML abierta anteriormente.
282		Línea en blanco para organizar el XML del formulario.
283	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
284	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
285	</Layout>	Cierra la sección XML abierta anteriormente.

286	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
287	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
288	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
289	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
290	<Image iconType="3" name="/Imágenes/cheque.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
291	</Property>	Cierra la sección XML abierta anteriormente.
292	</Properties>	Cierra la sección XML abierta anteriormente.
293	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
294	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
295	<AbsoluteConstraints x="18" y="23" width="45" height="45"/>	Define la posición y el tamaño del componente dentro del diseño.
296	</Constraint>	Cierra la sección XML abierta anteriormente.
297	</Constraints>	Cierra la sección XML abierta anteriormente.
298	</Component>	Cierra la sección XML abierta anteriormente.
299	</SubComponents>	Cierra la sección XML abierta anteriormente.
300	</Container>	Cierra la sección XML abierta anteriormente.
301	<Component class="javax.swing.JLabel" name="labelTitulo13">	Declara un componente o contenedor visual y su clase Java.
302	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
303	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
304		Metadato de NetBeans que conserva el estado editable del diseñador visual.
305	</Property>	Cierra la sección XML abierta anteriormente.
306	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
307	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
308	</Property>	Cierra la sección XML abierta anteriormente.
309	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
310	<Property name="text" type="java.lang.String" value="Activos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
311	</Properties>	Cierra la sección XML abierta anteriormente.
312	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
313	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
314	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
315	</Constraint>	Cierra la sección XML abierta anteriormente.
316	</Constraints>	Cierra la sección XML abierta anteriormente.
317	</Component>	Cierra la sección XML abierta anteriormente.
318	<Component class="javax.swing.JLabel" name="labelTitulo14">	Declara un componente o contenedor visual y su clase Java.
319	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
320	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
321		Metadato de NetBeans que conserva el estado editable del diseñador visual.
322	</Property>	Cierra la sección XML abierta anteriormente.
323	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
324	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
325	</Property>	Cierra la sección XML abierta anteriormente.
326	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

327	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
328	</Properties>	Cierra la sección XML abierta anteriormente.
329	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
330	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
331	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
332	</Constraint>	Cierra la sección XML abierta anteriormente.
333	</Constraints>	Cierra la sección XML abierta anteriormente.
334	</Component>	Cierra la sección XML abierta anteriormente.
335	<Component class="javax.swing.JLabel" name="labelTitulo15">	Declara un componente o contenedor visual y su clase Java.
336	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
337	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
338		Metadato de NetBeans que conserva el estado editable del diseñador visual.
339	</Property>	Cierra la sección XML abierta anteriormente.
340	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
341	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
342	</Property>	Cierra la sección XML abierta anteriormente.
343	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
344	<Property name="text" type="java.lang.String" value="Disponible para uso"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
345	</Properties>	Cierra la sección XML abierta anteriormente.
346	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
347	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
348	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
349	</Constraint>	Cierra la sección XML abierta anteriormente.
350	</Constraints>	Cierra la sección XML abierta anteriormente.
351	</Component>	Cierra la sección XML abierta anteriormente.
352	</SubComponents>	Cierra la sección XML abierta anteriormente.
353	</Container>	Cierra la sección XML abierta anteriormente.
354	<Container class="Componentes.PanelFlotante" name="panelFlotante3">	Declara un componente o contenedor visual y su clase Java.
355	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
356	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
357	<AbsoluteConstraints x="1170" y="140" width="360" height="160"/>	Define la posición y el tamaño del componente dentro del diseño.
358	</Constraint>	Cierra la sección XML abierta anteriormente.
359	</Constraints>	Cierra la sección XML abierta anteriormente.
360		Línea en blanco para organizar el XML del formulario.
361	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
362	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
363	</Layout>	Cierra la sección XML abierta anteriormente.
364	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
365	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
366	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
367	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		

368	<Color blue="f3" green="ef" red="ed" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
369	</Property>	Cierra la sección XML abierta anteriormente.
370	</Properties>	Cierra la sección XML abierta anteriormente.
371	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
372	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
373	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
374	</Constraint>	Cierra la sección XML abierta anteriormente.
375	</Constraints>	Cierra la sección XML abierta anteriormente.
376		Línea en blanco para organizar el XML del formulario.
377	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
378	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
379	</Layout>	Cierra la sección XML abierta anteriormente.
380	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
381	<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
382	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
383	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
384	<Image iconType="3" name="/Imagenes/x_ico.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
385	</Property>	Cierra la sección XML abierta anteriormente.
386	</Properties>	Cierra la sección XML abierta anteriormente.
387	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
388	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
389	<AbsoluteConstraints x="23" y="27" width="35" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
390	</Constraint>	Cierra la sección XML abierta anteriormente.
391	</Constraints>	Cierra la sección XML abierta anteriormente.
392	</Component>	Cierra la sección XML abierta anteriormente.
393	</SubComponents>	Cierra la sección XML abierta anteriormente.
394	</Container>	Cierra la sección XML abierta anteriormente.
395	<Component class="javax.swing.JLabel" name="labelTitulo10">	Declara un componente o contenedor visual y su clase Java.
396	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
397	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
398		Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	</Property>	Cierra la sección XML abierta anteriormente.
400	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
401	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
402	</Property>	Cierra la sección XML abierta anteriormente.
403	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
404	<Property name="text" type="java.lang.String" value="Inactivos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
405	</Properties>	Cierra la sección XML abierta anteriormente.
406	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
407	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
408	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.

409	</Constraint>	Cierra la sección XML abierta anteriormente.
410	</Constraints>	Cierra la sección XML abierta anteriormente.
411	</Component>	Cierra la sección XML abierta anteriormente.
412	<Component class="javax.swing.JLabel" name="labelTitulo11">	Declara un componente o contenedor visual y su clase Java.
413	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
414	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
415		Metadato de NetBeans que conserva el estado editable del diseñador visual.
416	</Property>	Cierra la sección XML abierta anteriormente.
417	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
418	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
419	</Property>	Cierra la sección XML abierta anteriormente.
420	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
421	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
422	</Properties>	Cierra la sección XML abierta anteriormente.
423	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
424	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
425	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
426	</Constraint>	Cierra la sección XML abierta anteriormente.
427	</Constraints>	Cierra la sección XML abierta anteriormente.
428	</Component>	Cierra la sección XML abierta anteriormente.
429	<Component class="javax.swing.JLabel" name="labelTitulo12">	Declara un componente o contenedor visual y su clase Java.
430	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
431	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
432		Metadato de NetBeans que conserva el estado editable del diseñador visual.
433	</Property>	Cierra la sección XML abierta anteriormente.
434	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
435	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
436	</Property>	Cierra la sección XML abierta anteriormente.
437	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
438	<Property name="text" type="java.lang.String" value="No disponibles"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
439	</Properties>	Cierra la sección XML abierta anteriormente.
440	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
441	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
442	<AbsoluteConstraints x="140" y="80" width="460" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
443	</Constraint>	Cierra la sección XML abierta anteriormente.
444	</Constraints>	Cierra la sección XML abierta anteriormente.
445	</Component>	Cierra la sección XML abierta anteriormente.
446	</SubComponents>	Cierra la sección XML abierta anteriormente.
447	</Container>	Cierra la sección XML abierta anteriormente.
448	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
449	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
450	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

451	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
452	</Properties>	Cierra la sección XML abierta anteriormente.
453	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
454	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
455	<AbsoluteConstraints x="60" y="320" width="1470" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
456	</Constraint>	Cierra la sección XML abierta anteriormente.
457	</Constraints>	Cierra la sección XML abierta anteriormente.
458		Línea en blanco para organizar el XML del formulario.
459	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
460	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
461	</Layout>	Cierra la sección XML abierta anteriormente.
462	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
463	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.
464	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
465	<Property name="placeholder" type="java.lang.String" value="Buscar producto o categoría..."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
466	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
467		Metadato de NetBeans que conserva el estado editable del diseñador visual.
468	</Property>	Cierra la sección XML abierta anteriormente.
469	</Properties>	Cierra la sección XML abierta anteriormente.
470	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
471	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="campoBusquedaActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
472	</Events>	Cierra la sección XML abierta anteriormente.
473	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
474	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
475	<AbsoluteConstraints x="20" y="22" width="455" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
476	</Constraint>	Cierra la sección XML abierta anteriormente.
477	</Constraints>	Cierra la sección XML abierta anteriormente.
478	</Component>	Cierra la sección XML abierta anteriormente.
479	<Component class="Componentes.BotonDesplegable" name="filtroStock">	Declara un componente o contenedor visual y su clase Java.
480	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
481	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
482	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
483	</Property>	Cierra la sección XML abierta anteriormente.
484	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
485	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
486	</Property>	Cierra la sección XML abierta anteriormente.
487	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
488	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
489	</Property>	Cierra la sección XML abierta anteriormente.
490	<Property name="text" type="java.lang.String" value="Stock"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

491	<Property name="textoDesplegable" type="java.lang.String" value=""/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
492	</Properties>	Cierra la sección XML abierta anteriormente.
493	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
494	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="filtroStockActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
495	</Events>	Cierra la sección XML abierta anteriormente.
496	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
497	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
498	<AbsoluteConstraints x="495" y="24" width="330" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
499	</Constraint>	Cierra la sección XML abierta anteriormente.
500	</Constraints>	Cierra la sección XML abierta anteriormente.
501	</Component>	Cierra la sección XML abierta anteriormente.
502	<Component class="Componentes.BotonDesplegable" name="filtroEstado">	Declara un componente o contenedor visual y su clase Java.
503	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
504	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
505	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
506	</Property>	Cierra la sección XML abierta anteriormente.
507	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
508	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
509	</Property>	Cierra la sección XML abierta anteriormente.
510	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
511	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
512	</Property>	Cierra la sección XML abierta anteriormente.
513	<Property name="text" type="java.lang.String" value="Todos los estados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
514	<Property name="textoDesplegable" type="java.lang.String" value="Todos los estados;Activo;Inactivo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
515	</Properties>	Cierra la sección XML abierta anteriormente.
516	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
517	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
518	<AbsoluteConstraints x="850" y="24" width="330" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
519	</Constraint>	Cierra la sección XML abierta anteriormente.
520	</Constraints>	Cierra la sección XML abierta anteriormente.
521	</Component>	Cierra la sección XML abierta anteriormente.
522	<Component class="Componentes.BotonRedondeado" name="botonLimpiarFiltros">	Declara un componente o contenedor visual y su clase Java.
523	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
524	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
525	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
526	</Property>	Cierra la sección XML abierta anteriormente.
527	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
528	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

529	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
530	</Property>	Cierra la sección XML abierta anteriormente.
531	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
532	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
533	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
534	</Property>	Cierra la sección XML abierta anteriormente.
535	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
536	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
537	</Property>	Cierra la sección XML abierta anteriormente.
538	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
539		Metadato de NetBeans que conserva el estado editable del diseñador visual.
540	</Property>	Cierra la sección XML abierta anteriormente.
541	<Property name=" grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
542	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
543	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
544	</Property>	Cierra la sección XML abierta anteriormente.
545	</Properties>	Cierra la sección XML abierta anteriormente.
546	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
547	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
548	<AbsoluteConstraints x="1210" y="24" width="230" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
549	</Constraint>	Cierra la sección XML abierta anteriormente.
550	</Constraints>	Cierra la sección XML abierta anteriormente.
551	</Component>	Cierra la sección XML abierta anteriormente.
552	</SubComponents>	Cierra la sección XML abierta anteriormente.
553	</Container>	Cierra la sección XML abierta anteriormente.
554	<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
555	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
556	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
557	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
558	</Properties>	Cierra la sección XML abierta anteriormente.
559	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
560	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
561	<AbsoluteConstraints x="60" y="420" width="1470" height="540"/>	Define la posición y el tamaño del componente dentro del diseño.
562	</Constraint>	Cierra la sección XML abierta anteriormente.
563	</Constraints>	Cierra la sección XML abierta anteriormente.
564		Línea en blanco para organizar el XML del formulario.
565	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
566	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
567	</Layout>	Cierra la sección XML abierta anteriormente.
568	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
569	<Container class="javax.swing.JScrollPane" name="scrollUsuarios">	Declara un componente o contenedor visual y su clase Java.
570	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

571	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
572	<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
573	</Property>	Cierra la sección XML abierta anteriormente.
574	<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
575	<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
576	<Property name="autoscrolls" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
577	</Properties>	Cierra la sección XML abierta anteriormente.
578	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
579	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
580	<AbsoluteConstraints x="14" y="14" width="1442" height="458"/>	Define la posición y el tamaño del componente dentro del diseño.
581	</Constraint>	Cierra la sección XML abierta anteriormente.
582	</Constraints>	Cierra la sección XML abierta anteriormente.
583		Línea en blanco para organizar el XML del formulario.
584	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
585	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
586	<Component class="Componentes.TablaAdministrativa" name="tablaProductos">	Declara un componente o contenedor visual y su clase Java.
587	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
588	<Property name="model" type="javax.swing.table.TableModel" editor="org.netbeans.modules.form.editors2.TableModelEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
589	<Table columnCount="8" rowCount="10">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
590	<Column editable="false" title="ID" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
591	<Column editable="false" title="Nombre" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
592	<Column editable="false" title="Usuario" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
593	<Column editable="false" title="Rol" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
594	<Column editable="false" title="Correo" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
595	<Column editable="false" title="Estado" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
596	<Column editable="false" title="Fecha creación" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
597	<Column editable="false" title="Acciones" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
598	</Table>	Cierra la sección XML abierta anteriormente.
599	</Property>	Cierra la sección XML abierta anteriormente.
600	<Property name="autoscrolls" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
601	<Property name="columnasCentradas" type="java.lang.String" value="5,7"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
602	</Properties>	Cierra la sección XML abierta anteriormente.
603	</Component>	Cierra la sección XML abierta anteriormente.
604	</SubComponents>	Cierra la sección XML abierta anteriormente.
605	</Container>	Cierra la sección XML abierta anteriormente.
606	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
607	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
608	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
609	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
610	</Property>	Cierra la sección XML abierta anteriormente.
611	<Property name="text" type="java.lang.String" value="Mostrando 1–10 de 10 productos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
612	</Properties>	Cierra la sección XML abierta anteriormente.

613	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
614	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
615	<AbsoluteConstraints x="28" y="482" width="330" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
616	</Constraint>	Cierra la sección XML abierta anteriormente.
617	</Constraints>	Cierra la sección XML abierta anteriormente.
618	</Component>	Cierra la sección XML abierta anteriormente.
619	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
620	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
621	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
622	</Properties>	Cierra la sección XML abierta anteriormente.
623	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
624	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
625	<AbsoluteConstraints x="1165" y="482" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
626	</Constraint>	Cierra la sección XML abierta anteriormente.
627	</Constraints>	Cierra la sección XML abierta anteriormente.
628		Línea en blanco para organizar el XML del formulario.
629	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
630	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
631	</Layout>	Cierra la sección XML abierta anteriormente.
632	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
633	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
634	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
635	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
636	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
637	</Property>	Cierra la sección XML abierta anteriormente.
638	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
639	<Property name="text" type="java.lang.String" value="#x2039;/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
640	</Properties>	Cierra la sección XML abierta anteriormente.
641	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
642	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
643	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
644	</Constraint>	Cierra la sección XML abierta anteriormente.
645	</Constraints>	Cierra la sección XML abierta anteriormente.
646	</Component>	Cierra la sección XML abierta anteriormente.
647	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
648	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
649	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
650	<Color blue="0" green="bc" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
651	</Property>	Cierra la sección XML abierta anteriormente.
652	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
653	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

654	</Properties>	Cierra la sección XML abierta anteriormente.
655	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
656	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
657	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
658	</Constraint>	Cierra la sección XML abierta anteriormente.
659	</Constraints>	Cierra la sección XML abierta anteriormente.
660	</Component>	Cierra la sección XML abierta anteriormente.
661	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
662	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
663	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
664	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
665	</Property>	Cierra la sección XML abierta anteriormente.
666	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
667	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
668	</Properties>	Cierra la sección XML abierta anteriormente.
669	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
670	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
671	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
672	</Constraint>	Cierra la sección XML abierta anteriormente.
673	</Constraints>	Cierra la sección XML abierta anteriormente.
674	</Component>	Cierra la sección XML abierta anteriormente.
675	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
676	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
677	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
678	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
679	</Property>	Cierra la sección XML abierta anteriormente.
680	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
681	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
682	</Properties>	Cierra la sección XML abierta anteriormente.
683	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
684	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
685	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
686	</Constraint>	Cierra la sección XML abierta anteriormente.
687	</Constraints>	Cierra la sección XML abierta anteriormente.
688	</Component>	Cierra la sección XML abierta anteriormente.
689	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
690	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
691	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
692	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
693	</Property>	Cierra la sección XML abierta anteriormente.

694	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
695	<Property name="text" type="java.lang.String" value="›"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
696	</Properties>	Cierra la sección XML abierta anteriormente.
697	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
698	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
699	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
700	</Constraint>	Cierra la sección XML abierta anteriormente.
701	</Constraints>	Cierra la sección XML abierta anteriormente.
702	</Component>	Cierra la sección XML abierta anteriormente.
703	</SubComponents>	Cierra la sección XML abierta anteriormente.
704	</Container>	Cierra la sección XML abierta anteriormente.
705	</SubComponents>	Cierra la sección XML abierta anteriormente.
706	</Container>	Cierra la sección XML abierta anteriormente.
707	</SubComponents>	Cierra la sección XML abierta anteriormente.
708	</Form>	Cierra la sección XML abierta anteriormente.

16.69 src/GUI_ADMINISTRADOR/IngredientesPanel.java

Mantenimiento del catálogo de ingredientes, stock mínimo, estado y movimientos.

Tipo: java | Líneas: 1463 | Archivo 69 de 157

Clases/tipos detectados: IngredientesPanel, ModeloTabla, RenderNivel, RenderEstado, RenderAcciones, EditorAcciones, PanelFormularioIngrediente, DialogoMovimientoIngrediente

Línea	Constructor o método detectado
72	public IngredientesPanel() {
90	private void initComponents() {
282	public void actionPerformed(java.awt.event.ActionEvent evt) {
294	public void actionPerformed(java.awt.event.ActionEvent evt) {
350	public boolean isCellEditable(int rowIndex, int columnIndex) {
399	private void configurarApariencia() {
415	private void configurarFiltros() {
452	public void insertUpdate(DocumentEvent e) {
457	public void removeUpdate(DocumentEvent e) {
462	public void changedUpdate(DocumentEvent e) {
479	private void prepararSelector(BotonDesplegable boton) {
491	private void configurarTabla() {
508	private void configurarPaginacion() {
524	private void prepararBotonPagina(JButton boton, IconosUsuarios.Tipo icono) {
540	private void cargarTodo() {
545	private void cargarResumen() {
548	protected ResumenIngredientes doInBackground() throws SQLException {
553	protected void done() {
570	private void cargarPagina() {
580	protected PaginalIngredientes doInBackground() throws SQLException {
586	protected void done() {
616	private void actualizarPaginacion(int total) {
649	private void cambiarPagina(int numero) {
657	private void abrirFormulario(Ingrediente ingrediente) {
663	private void editar(int id) {
677	private void movimiento(int id) {
693	private void desactivar(Ingrediente ingrediente) {
711	private void mostrarErrorDatos(String operacion, Throwable ex) {
733	public int getRowCount() {

Línea	Constructor o método detectado
738	public int getColumnCount() {
743	public String getColumnName(int columna) {
748	public Class<?> getColumnClass(int columna) {
753	public boolean isCellEditable(int fila, int columna) {
758	public Object getValueAt(int fila, int columna) {
782	public Component getTableCellRendererComponent(javax.swing.JTable t,
796	public Component getTableCellRendererComponent(javax.swing.JTable t,
807	private JPanel etiquetaEstado(String texto, Color primerPlano,
825	public Component getTableCellRendererComponent(javax.swing.JTable t,
848	private final BotonRedondeado editar = botonAccion(IconosUsuarios.Tipo.EDITAR,
850	private final BotonRedondeado mover = botonAccion(null, AZUL,
852	private final BotonRedondeado baja = botonAccion(IconosUsuarios.Tipo.ELIMINAR,
878	public Component getTableCellEditorComponent(javax.swing.JTable t,
888	public Object getCellEditorValue() {
893	private JPanel panelAcciones(Color fondo) {
899	private BotonRedondeado botonAccion(IconosUsuarios.Tipo icono,
923	private boolean mostrarDialogoIngrediente(Ingrediente ingrediente) {
949	private void mostrarDialogoConFondo(JDialog dialogo) {
956	protected void paintComponent(Graphics grafico) {
991	private PanelFormularioIngrediente(Ingrediente original) {
1009	private void construirCabecera() {
1043	private void construirCampos() {
1075	private JPanel campo(String texto, JComponent control) {
1087	private void fila(JPanel panel, int numero, JPanel izquierda, JPanel derecha) {
1101	private void construirPie() {
1135	private void guardar() {
1179	private void desactivar() {
1194	private void estilizar(JTextField campo) {
1201	private BotonRedondeado boton(String texto, boolean principal) {
1218	private boolean mostrarDialogoMovimiento(Ingrediente ingrediente) {
1229	private final JComboBox<String> tipo = new JComboBox<>(new String[]{
1239	private DialogoMovimientoIngrediente(Window propietario,
1254	private PanelFlotante crearContenido() {

Línea	Constructor o método detectado
1321	private void agregarCampo(JPanel panel, int fila, String nombre,
1340	private void actualizarInstruccion() {
1346	private void guardarMovimiento() {
1387	private void estilizar(JTextField campo) {
1394	private BotonRedondeado crearBoton(String texto, boolean principal) {
1410	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_campoBusquedaActionPerformed
1414	private void filtroStockActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroStockActionPerformed

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.IngredienteCRUD;	Importa CRUD.IngredienteCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
5	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
6	import Componentes.PanelCircular;	Importa Componentes.PanelCircular para poder usar esa clase sin escribir su nombre completo.
7	import Componentes.PanelFlotante;	Importa Componentes.PanelFlotante para poder usar esa clase sin escribir su nombre completo.
8	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
9	import Modelos.PaginaIngredientes;	Importa Modelos.PaginaIngredientes para poder usar esa clase sin escribir su nombre completo.
10	import Modelos.ResumenIngredientes;	Importa Modelos.ResumenIngredientes para poder usar esa clase sin escribir su nombre completo.
11	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.
12	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
18	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
19	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
20	import java.awt.GridBagConstraints;	Importa java.awt.GridBagConstraints para poder usar esa clase sin escribir su nombre completo.
21	import java.awt.GridBagLayout;	Importa java.awt.GridBagLayout para poder usar esa clase sin escribir su nombre completo.
22	import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
23	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
24	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
25	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
26	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
27	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
28	import javax.swing.AbstractCellEditor;	Importa javax.swing.AbstractCellEditor para poder usar esa clase sin escribir su nombre completo.
29	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
30	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
31	import javax.swing.JComboBox;	Importa javax.swing.JComboBox para poder usar esa clase sin escribir su nombre completo.
32	import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.

33	import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
34	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
35	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
36	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
37	import javax.swing.JRootPane;	Importa javax.swing.JRootPane para poder usar esa clase sin escribir su nombre completo.
38	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
39	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
40	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
41	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
42	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
43	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
44	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
45	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
46	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
47	import javax.swing.table.TableCellEditor;	Importa javax.swing.table.TableCellEditor para poder usar esa clase sin escribir su nombre completo.
48	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	public class IngredientesPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	private static final int POR_PAGINA = 10;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
53	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
54	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
55	private static final Color BORDE = new Color(225, 229, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
56	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
57	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	private final IngredienteCRUD ingredienteCRUD = new IngredienteCRUD();	Crea una nueva instancia de IngredienteCRUD y la usa o guarda para continuar el proceso.
60	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
61	private final ModeloTabla modeloTabla = new ModeloTabla();	Crea una nueva instancia de ModeloTabla y la usa o guarda para continuar el proceso.
62	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
63	private final JButton[] botonesPagina;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
64	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	private int totalPaginas = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	private int secuenciaCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	private Boolean stockBajoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	private Boolean estadoActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
69	private boolean errorConexionMostrado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	// Sustituye SOLO el constructor actual; deja initComponents() intacto:	Comentario para humanos; Java no ejecuta este texto. Sustituye SOLO el constructor actual; deja initComponents() intacto:
72	public IngredientesPanel() {	Constructor de IngredientesPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
73	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	botonesPagina = new JButton[]{botonPagina1, botonPagina2, botonPagina3};	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
75	temporizadorBusqueda = new Timer(300, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
76	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

77	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
80	configurarApariencia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	configurarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	configurarPaginacion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	botonAgregarIngrediente.addActionListener(e -> abrirFormulario(null));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
85	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	}	Cierra el bloque abierto anteriormente.
87		Línea en blanco: separa visualmente bloques para que el código sea más legible.
88	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
89	// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
90	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	botonAgregarIngrediente = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
93	panelFlotante1 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
94	panelCircular2 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
95	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
96	labelTitulo3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
97	labelTitulo4 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
98	labelTitulo2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
99	labelTitulo5 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
100	labelTitulo6 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
101	panelFlotante2 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
102	panelCircular3 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
103	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
104	labelTitulo7 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
105	labelTitulo8 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
106	labelTitulo9 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
107	panelFlotante4 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
108	panelCircular5 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
109	labelEscalable5 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
110	labelTitulo13 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
111	labelTitulo14 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
112	labelTitulo15 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
113	panelFlotante3 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
114	panelCircular4 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
115	labelEscalable4 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
116	labelTitulo10 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
117	labelTitulo11 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
118	labelTitulo12 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
119	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
120	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.

121	<code>filtroStock = new Componentes.BotonDesplegable();</code>	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
122	<code>filtroEstado = new Componentes.BotonDesplegable();</code>	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
123	<code>botonLimpiarFiltros = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
124	<code>panelTabla = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
125	<code>scrollUsuarios = new javax.swing.JScrollPane();</code>	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
126	<code>tablaProductos = new Componentes.TablaAdministrativa();</code>	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
127	<code>etiquetaRango = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
128	<code>panelPaginacion = new javax.swing.JPanel();</code>	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
129	<code>botonAnterior = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
130	<code>botonPagina1 = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
131	<code>botonPagina2 = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
132	<code>botonPagina3 = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
133	<code>botonSiguiente = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	<code>setBackground(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
136	<code>setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	<code>botonAgregarIngrediente.setForeground(new java.awt.Color(0, 0, 0));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
139	<code>botonAgregarIngrediente.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/icono_agrega r.png"))); // NOI18N</code>	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
140	<code>botonAgregarIngrediente.setText("Agregar ingrediente");</code>	Cambia el texto visible de un componente de la interfaz.
141	<code>botonAgregarIngrediente.setIconTextGap(15);</code>	Configura una propiedad mediante setIconTextGap en el objeto indicado.
142	<code>add(botonAgregarIngrediente, new org.netbeans.lib.awtextra.AbsoluteConstraints(1290, 50, 230, 80));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
144	<code>panelFlotante1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	<code>panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.
148	<code>labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/lista_icono.png"))); // NOI18N</code>	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
149	<code>panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(15, 20, 50, 50));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	<code>panelFlotante1.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	<code>labelTitulo3.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N</code>	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
154	<code>labelTitulo3.setForeground(new java.awt.Color(92, 103, 124));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
155	<code>labelTitulo3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);</code>	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
156	<code>labelTitulo3.setText("Ingredientes totales");</code>	Cambia el texto visible de un componente de la interfaz.
157	<code>panelFlotante1.add(labelTitulo3, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	<code>labelTitulo4.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N</code>	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
160	<code>labelTitulo4.setForeground(new java.awt.Color(13, 17, 23));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
161	<code>labelTitulo4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);</code>	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
162	<code>labelTitulo4.setText("48");</code>	Cambia el texto visible de un componente de la interfaz.
163	<code>panelFlotante1.add(labelTitulo4, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));</code>	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	labelTitulo2.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
166	labelTitulo2.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
167	labelTitulo2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
168	labelTitulo2.setText("En el inventario");	Cambia el texto visible de un componente de la interfaz.
169	panelFlotante1.add(labelTitulo2, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	add(panelFlotante1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	labelTitulo5.setFont(new java.awt.Font("Dialog", 1, 55)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
174	labelTitulo5.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
175	labelTitulo5.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
176	labelTitulo5.setText("Gestión de Ingredientes");	Cambia el texto visible de un componente de la interfaz.
177	add(labelTitulo5, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 30, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
179	labelTitulo6.setFont(new java.awt.Font("Dialog", 1, 18)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
180	labelTitulo6.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
181	labelTitulo6.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
182	labelTitulo6.setText("Administra el inventario de ingredientes");	Cambia el texto visible de un componente de la interfaz.
183	add(labelTitulo6, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	panelFlotante2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
188	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
189		Línea en blanco: separa visualmente bloques para que el código sea más legible.
190	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/advertencia_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
191	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(19, 22, 43, 43));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	panelFlotante2.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
194		Línea en blanco: separa visualmente bloques para que el código sea más legible.
195	labelTitulo7.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
196	labelTitulo7.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
197	labelTitulo7.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
198	labelTitulo7.setText("Stock bajo");	Cambia el texto visible de un componente de la interfaz.
199	panelFlotante2.add(labelTitulo7, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	labelTitulo8.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
202	labelTitulo8.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
203	labelTitulo8.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
204	labelTitulo8.setText("48");	Cambia el texto visible de un componente de la interfaz.
205	panelFlotante2.add(labelTitulo8, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	labelTitulo9.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.

208	labelTitulo9.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
209	labelTitulo9.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
210	labelTitulo9.setText("Requieren reposición");	Cambia el texto visible de un componente de la interfaz.
211	panelFlotante2.add(labelTitulo9, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	add(panelFlotante2, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	panelFlotante4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	panelCircular5.setColorFondo(new java.awt.Color(230, 245, 234));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
218	panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/cheque.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
221	panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(18, 23, 45, 45));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	panelFlotante4.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	labelTitulo13.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
226	labelTitulo13.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
227	labelTitulo13.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
228	labelTitulo13.setText("Activos");	Cambia el texto visible de un componente de la interfaz.
229	panelFlotante4.add(labelTitulo13, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	labelTitulo14.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
232	labelTitulo14.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
233	labelTitulo14.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
234	labelTitulo14.setText("48");	Cambia el texto visible de un componente de la interfaz.
235	panelFlotante4.add(labelTitulo14, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	labelTitulo15.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
238	labelTitulo15.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
239	labelTitulo15.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
240	labelTitulo15.setText("Disponible para uso");	Cambia el texto visible de un componente de la interfaz.
241	panelFlotante4.add(labelTitulo15, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
242		Línea en blanco: separa visualmente bloques para que el código sea más legible.
243	add(panelFlotante4, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	panelFlotante3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
246		Línea en blanco: separa visualmente bloques para que el código sea más legible.
247	panelCircular4.setColorFondo(new java.awt.Color(237, 239, 243));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
248	panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
249		Línea en blanco: separa visualmente bloques para que el código sea más legible.
250	labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/x_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.

251	panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(23, 27, 35, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
252		Línea en blanco: separa visualmente bloques para que el código sea más legible.
253	panelFlotante3.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255	labelTitulo10.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
256	labelTitulo10.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
257	labelTitulo10.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
258	labelTitulo10.setText("Inactivos");	Cambia el texto visible de un componente de la interfaz.
259	panelFlotante3.add(labelTitulo10, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	labelTitulo11.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
262	labelTitulo11.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
263	labelTitulo11.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
264	labelTitulo11.setText("48");	Cambia el texto visible de un componente de la interfaz.
265	panelFlotante3.add(labelTitulo11, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267	labelTitulo12.setFont(new java.awt.Font("Dialog", 1, 12)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
268	labelTitulo12.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
269	labelTitulo12.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
270	labelTitulo12.setText("No disponibles");	Cambia el texto visible de un componente de la interfaz.
271	panelFlotante3.add(labelTitulo12, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 80, 460, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	add(panelFlotante3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 140, 360, 160));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
274		Línea en blanco: separa visualmente bloques para que el código sea más legible.
275	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
276	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
277	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	campoBusqueda.setPlaceholder("Buscar producto o categoría...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
280	campoBusqueda.setFont(new java.awt.Font("Dialog", 0, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
281	campoBusqueda.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
282	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
283	campoBusquedaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
284	}	Cierra el bloque abierto anteriormente.
285	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
286	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 22, 455, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	filtroStock.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
289	filtroStock.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
290	filtroStock.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
291	filtroStock.setText("Stock");	Cambia el texto visible de un componente de la interfaz.
292	filtroStock.setTextoDesplegable("");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
293	filtroStock.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
294	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

295	filtroStockActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
296	}	Cierra el bloque abierto anteriormente.
297	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	panelFiltros.add(filtroStock, new org.netbeans.lib.awtextra.AbsoluteConstraints(495, 24, 330, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
299		Línea en blanco: separa visualmente bloques para que el código sea más legible.
300	filtroEstado.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
301	filtroEstado.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
302	filtroEstado.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
303	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
304	filtroEstado.setTextoDesplegable("Todos los estados;Activo;Inactivo");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
305	panelFiltros.add(filtroEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(850, 24, 330, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	botonLimpiarFiltros.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
308	botonLimpiarFiltros.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
309	botonLimpiarFiltros.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
310	botonLimpiarFiltros.setText("Limpiar filtros");	Cambia el texto visible de un componente de la interfaz.
311	botonLimpiarFiltros.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
312	botonLimpiarFiltros.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
313	botonLimpiarFiltros.setFont(new java.awt.Font("Arial", 1, 15)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
314	botonLimpiarFiltros.setGrosorBorde(1.6F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
315	botonLimpiarFiltros.setMargin(new java.awt.Insets(5, 14, 3, 14));	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
316	panelFiltros.add(botonLimpiarFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 24, 230, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318	add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 320, 1470, 100));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
319		Línea en blanco: separa visualmente bloques para que el código sea más legible.
320	panelTabla.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
321	panelTabla.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
322	panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
323		Línea en blanco: separa visualmente bloques para que el código sea más legible.
324	scrollUsuarios.setBorder(null);	Configura una propiedad mediante setBorder en el objeto indicado.
325	scrollUsuarios.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
326	scrollUsuarios.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
327	scrollUsuarios.setAutoscrolls(true);	Configura una propiedad mediante setAutoscrolls en el objeto indicado.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	tablaProductos.setModel(new javax.swing.table.DefaultTableModel(	Crea una nueva instancia de javax.swing.table.DefaultTableModel y la usa o guarda para continuar el proceso.
330	new Object [][] {	Crea una nueva instancia de Object y la usa o guarda para continuar el proceso.
331	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
332	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
333	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
334	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
335	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
337	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

339	{null, null, null, null, null, null, null, null},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
340	{null, null, null, null, null, null, null, null}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341	},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
342	new String [] {	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
343	"ID", "Nombre", "Usuario", "Rol", "Correo", "Estado", "Fecha creación", "Acciones"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
344	}	Cierra el bloque abierto anteriormente.
345	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
346	boolean[] canEdit = new boolean [] {	Crea una nueva instancia de boolean y la usa o guarda para continuar el proceso.
347	false, false, false, false, false, false, false, false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
348	};	Cierra el bloque abierto anteriormente y termina la declaración.
349		Línea en blanco: separa visualmente bloques para que el código sea más legible.
350	public boolean isCellEditable(int rowIndex, int columnIndex) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
351	return canEdit [columnIndex];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
352	}	Cierra el bloque abierto anteriormente.
353	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
354	tablaProductos.setAutoscrolls(false);	Configura una propiedad mediante setAutoscrolls en el objeto indicado.
355	tablaProductos.setColumnsCentradas("5,7");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
356	scrollUsuarios.setViewportView(tablaProductos);	Configura una propiedad mediante setViewportView en el objeto indicado.
357		Línea en blanco: separa visualmente bloques para que el código sea más legible.
358	panelTabla.add(scrollUsuarios, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 458));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
359		Línea en blanco: separa visualmente bloques para que el código sea más legible.
360	etiquetaRango.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
361	etiquetaRango.setText("Mostrando 1-10 de 10 productos");	Cambia el texto visible de un componente de la interfaz.
362	panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 482, 330, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
363		Línea en blanco: separa visualmente bloques para que el código sea más legible.
364	panelPaginacion.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
365	panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
366		Línea en blanco: separa visualmente bloques para que el código sea más legible.
367	botonAnterior.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
368	botonAnterior.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
369	botonAnterior.setText("<");	Cambia el texto visible de un componente de la interfaz.
370	panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
371		Línea en blanco: separa visualmente bloques para que el código sea más legible.
372	botonPagina1.setColorInicio(new java.awt.Color(255, 188, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
373	botonPagina1.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
374	botonPagina1.setText("1");	Cambia el texto visible de un componente de la interfaz.
375	panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
376		Línea en blanco: separa visualmente bloques para que el código sea más legible.
377	botonPagina2.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
378	botonPagina2.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
379	botonPagina2.setText("2");	Cambia el texto visible de un componente de la interfaz.
380	panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
381		Línea en blanco: separa visualmente bloques para que el código sea más legible.
382	botonPagina3.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.

383	botonPagina3.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
384	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
385	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
387	botonSiguiente.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
388	botonSiguiente.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
389	botonSiguiente.setText(">");	Cambia el texto visible de un componente de la interfaz.
390	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
391		Línea en blanco: separa visualmente bloques para que el código sea más legible.
392	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 482, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
393		Línea en blanco: separa visualmente bloques para que el código sea más legible.
394	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 420, 1470, 540));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
395	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397	// Pega estos métodos dentro de IngredientesPanel, después de initComponents()	Comentario para humanos; Java no ejecuta este texto. Pega estos métodos dentro de IngredientesPanel, después de initComponents()
398	// y antes de «Variables declaration - do not modify».	Comentario para humanos; Java no ejecuta este texto. y antes de «Variables declaration - do not modify».
399	private void configurarApariencia() {	Declara el método configurarApariencia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
400	setBackground(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
401	campoBusqueda.setPlaceholder("Buscar ingrediente o unidad...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
402	panelFiltros.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
403	panelFiltros.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
404	panelTabla.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
405	panelTabla.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
406	scrollUsuarios.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
407	scrollUsuarios.getViewPort().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
408	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
409	botonAgregarIngrediente.setText("Agregar ingrediente");	Cambia el texto visible de un componente de la interfaz.
410	botonLimpiarFiltros.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
411	botonLimpiarFiltros.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
412	IconosUsuarios.Tipo.FILTRO, ROJO, 20));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413	}	Cierra el bloque abierto anteriormente.
414		Línea en blanco: separa visualmente bloques para que el código sea más legible.
415	private void configurarFiltros() {	Declara el método configurarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
416	filtroStock.setText("Todos los niveles");	Cambia el texto visible de un componente de la interfaz.
417	filtroStock.setTextoDesplegable("Todos los niveles;Stock bajo;Stock normal");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
418	prepararSelector(filtroStock);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
419	filtroStock.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
420	String opcion = e.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
421	filtroStock.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
422	stockBajoActual = switch (opcion) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
423	case "Stock bajo" ->	Define una de las rutas posibles dentro del switch.
424	Boolean.TRUE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
425	case "Stock normal" ->	Define una de las rutas posibles dentro del switch.

426	Boolean.FALSE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
427	default ->	Define una de las rutas posibles dentro del switch.
428	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429	};	Cierra el bloque abierto anteriormente y termina la declaración.
430	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
431	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
432	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
434	filtroEstado.setTextoDesplegable("Todos los estados;Activo;Inactivo");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
435	prepararSelector(filtroEstado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
436	filtroEstado.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
437	String opcion = e.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
438	filtroEstado.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
439	estadoActual = switch (opcion) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
440	case "Activo" ->	Define una de las rutas posibles dentro del switch.
441	Boolean.TRUE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
442	case "Inactivo" ->	Define una de las rutas posibles dentro del switch.
443	Boolean.FALSE;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
444	default ->	Define una de las rutas posibles dentro del switch.
445	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
446	};	Cierra el bloque abierto anteriormente y termina la declaración.
447	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
448	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
449	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
450	campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
451	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
452	public void insertUpdate(DocumentEvent e) {	Declara el método insertUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
453	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
454	}	Cierra el bloque abierto anteriormente.
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
457	public void removeUpdate(DocumentEvent e) {	Declara el método removeUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
458	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
459	}	Cierra el bloque abierto anteriormente.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
462	public void changedUpdate(DocumentEvent e) {	Declara el método changedUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
463	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
464	}	Cierra el bloque abierto anteriormente.
465	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
466	botonLimpiarFiltros.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
467	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
468	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.
469	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

470	stockBajoActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
471	estadoActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
472	filtroStock.setText("Todos los niveles");	Cambia el texto visible de un componente de la interfaz.
473	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
474	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
475	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
476	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
477	}	Cierra el bloque abierto anteriormente.
478		Línea en blanco: separa visualmente bloques para que el código sea más legible.
479	private void prepararSelector(BotonDesplegable boton) {	Declara el método prepararSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
480	boton.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
481	boton.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
482	boton.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
483	boton.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
484	boton.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
485	boton.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
486	boton.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
487	boton.setAnchoMenu(275);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
488	boton.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
489	}	Cierra el bloque abierto anteriormente.
490		Línea en blanco: separa visualmente bloques para que el código sea más legible.
491	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
492	tablaProductos.setModel(modeloTabla);	Configura una propiedad mediante setModel en el objeto indicado.
493	tablaProductos.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
494	tablaProductos.setRowHeight(53);	Configura una propiedad mediante setRowHeight en el objeto indicado.
495	tablaProductos.setAutoCreateRowSorter(false);	Configura una propiedad mediante setAutoCreateRowSorter en el objeto indicado.
496	tablaProductos.getColumnModel().getColumn(5).setCellRenderer(new RenderNivel());	Crea una nueva instancia de RenderNivel y la usa o guarda para continuar el proceso.
497	tablaProductos.getColumnModel().getColumn(6).setCellRenderer(new RenderEstado());	Crea una nueva instancia de RenderEstado y la usa o guarda para continuar el proceso.
498	tablaProductos.getColumnModel().getColumn(7).setCellRenderer(new RenderAcciones());	Crea una nueva instancia de RenderAcciones y la usa o guarda para continuar el proceso.
499	tablaProductos.getColumnModel().getColumn(7).setCellEditor(new EditorAcciones());	Crea una nueva instancia de EditorAcciones y la usa o guarda para continuar el proceso.
500	int[] anchos = {80, 270, 130, 130, 130, 115, 105, 150};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
501	for (int i = 0; i < anchos.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
502	tablaProductos.getColumnModel().getColumn(i).setPreferredWidth(anchos[i]);	Configura una propiedad mediante setPreferredWidth en el objeto indicado.
503	}	Cierra el bloque abierto anteriormente.
504	etiquetaRango.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
505	etiquetaRango.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
506	}	Cierra el bloque abierto anteriormente.
507		Línea en blanco: separa visualmente bloques para que el código sea más legible.
508	private void configurarPaginacion() {	Declara el método configurarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
509	prepararBotonPagina(botonAnterior, IconosUsuarios.Tipo.ANTERIOR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
510	botonAnterior.addActionListener(e -> cambiarPagina(paginaActual - 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
511	for (JButton boton : botonesPagina) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
512	prepararBotonPagina(boton, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
513	boton.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.

514	Object numero = boton.getClientProperty("pagina");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
515	if (numero instanceof Integer pagina) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
516	cambiarPagina(pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
517	}	Cierra el bloque abierto anteriormente.
518	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
519	}	Cierra el bloque abierto anteriormente.
520	prepararBotonPagina(botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
521	botonSiguiente.addActionListener(e -> cambiarPagina(paginaActual + 1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
522	}	Cierra el bloque abierto anteriormente.
523		Línea en blanco: separa visualmente bloques para que el código sea más legible.
524	private void prepararBotonPagina(JButton boton, IconosUsuarios.Tipo icono) {	Declara el método prepararBotonPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
525	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
526	redondeado.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
527	redondeado.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
528	redondeado.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
529	redondeado.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
530	redondeado.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
531	redondeado.setRadio(10);	Configura una propiedad mediante setRadio en el objeto indicado.
532	}	Cierra el bloque abierto anteriormente.
533	boton.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
534	if (icono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
535	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
536	boton.setIcon(IconosUsuarios.crear(icono, AZUL, 16));	Configura una propiedad mediante setIcon en el objeto indicado.
537	}	Cierra el bloque abierto anteriormente.
538	}	Cierra el bloque abierto anteriormente.
539		Línea en blanco: separa visualmente bloques para que el código sea más legible.
540	private void cargarTodo() {	Declara el método cargarTodo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
541	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
542	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
543	}	Cierra el bloque abierto anteriormente.
544		Línea en blanco: separa visualmente bloques para que el código sea más legible.
545	private void cargarResumen() {	Declara el método cargarResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
546	new SwingWorker<ResumenIngredientes, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
547	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
548	protected ResumenIngredientes doInBackground() throws SQLException {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
549	return ingredienteCRUD.obtenerResumen();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
550	}	Cierra el bloque abierto anteriormente.
551		Línea en blanco: separa visualmente bloques para que el código sea más legible.
552	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
553	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
554	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
555	ResumenIngredientes datos = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
556	labelTitulo4.setText(String.valueOf(datos.total()));	Cambia el texto visible de un componente de la interfaz.
557	labelTitulo8.setText(String.valueOf(datos.stockBajo()));	Cambia el texto visible de un componente de la interfaz.

558	labelTitulo14.setText(String.valueOf(datos.activos()));	Cambia el texto visible de un componente de la interfaz.
559	labelTitulo11.setText(String.valueOf(datos.inactivos()));	Cambia el texto visible de un componente de la interfaz.
560	errorConexionMostrado = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
561	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
562	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
563	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
564	mostrarErrorDatos("cargar el resumen", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
565	}	Cierra el bloque abierto anteriormente.
566	}	Cierra el bloque abierto anteriormente.
567	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
568	}	Cierra el bloque abierto anteriormente.
569		Línea en blanco: separa visualmente bloques para que el código sea más legible.
570	private void cargarPagina() {	Declara el método cargarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
571	final int solicitud = ++secuenciaCarga;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
572	final int paginaSolicitada = paginaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
573	final String busqueda = campoBusqueda.getText().trim();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
574	final Boolean stock = stockBajoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
575	final Boolean estado = estadoActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
576	tablaProductos.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
577	etiquetaRango.setText("Cargando ingredientes...");	Cambia el texto visible de un componente de la interfaz.
578	new SwingWorker<PaginaIngredientes, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
579	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
580	protected PaginaIngredientes doInBackground() throws SQLException {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
581	return ingredienteCRUD.listarPagina(busqueda, stock, estado,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
582	paginaSolicitada, POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
583	}	Cierra el bloque abierto anteriormente.
584		Línea en blanco: separa visualmente bloques para que el código sea más legible.
585	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
586	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
587	if (solicitud != secuenciaCarga) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
588	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
589	}	Cierra el bloque abierto anteriormente.
590	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
591	PaginaIngredientes datos = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
592	totalPaginas = Math.max(1, (datos.totalRegistros()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
593	+ POR_PAGINA - 1) / POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
594	if (paginaActual > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
595	paginaActual = totalPaginas;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
596	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
598	}	Cierra el bloque abierto anteriormente.
599	modeloTabla.setIngredientes(datos.ingredientes());	Configura una propiedad mediante setIngredientes en el objeto indicado.
600	actualizarPaginacion(datos.totalRegistros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
601	errorConexionMostrado = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

602	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
603	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
604	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
605	modeloTabla.setIngredientes(List.of());	Configura una propiedad mediante setIngredientes en el objeto indicado.
606	totalPaginas = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
607	actualizarPaginacion(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
608	mostrarErrorDatos("cargar ingredientes", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
609	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
610	tablaProductos.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
611	}	Cierra el bloque abierto anteriormente.
612	}	Cierra el bloque abierto anteriormente.
613	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
614	}	Cierra el bloque abierto anteriormente.
615		Línea en blanco: separa visualmente bloques para que el código sea más legible.
616	private void actualizarPaginacion(int total) {	Declara el método actualizarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
617	if (total == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
618	etiquetaRango.setText("No se encontraron ingredientes");	Cambia el texto visible de un componente de la interfaz.
619	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
620	int inicio = (paginaActual - 1) * POR_PAGINA + 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
621	int fin = Math.min(inicio + POR_PAGINA - 1, total);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
622	etiquetaRango.setText(String.format("Mostrando %d-%d de %d ingredientes",	Cambia el texto visible de un componente de la interfaz.
623	inicio, fin, total));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
624	}	Cierra el bloque abierto anteriormente.
625	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
626	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
627	int inicio = Math.max(1, Math.min(paginaActual - 1, totalPaginas - 2));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
628	for (int i = 0; i < botonesPagina.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
629	JButton boton = botonesPagina[i];	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
630	int pagina = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
631	boton.setVisible(pagina <= totalPaginas);	Muestra u oculta una ventana o componente según el valor booleano enviado.
632	if (pagina > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
633	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
634	}	Cierra el bloque abierto anteriormente.
635	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
636	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
637	boolean seleccionada = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
638	boton.setFont(seleccionada ? tema.negrita(13f) : tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
639	if (boton instanceof BotonRedondeado redondeado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
640	redondeado.setColorInicio(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
641	redondeado.setColorFinal(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
642	redondeado.setColorBorde(seleccionada ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
643	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
644	boton.setBackground(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
645	}	Cierra el bloque abierto anteriormente.

646	}	Cierra el bloque abierto anteriormente.
647	}	Cierra el bloque abierto anteriormente.
648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
649	private void cambiarPagina(int numero) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
650	if (numero < 1 numero > totalPaginas numero == paginaActual) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
651	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
652	}	Cierra el bloque abierto anteriormente.
653	paginaActual = numero;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
654	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
655	}	Cierra el bloque abierto anteriormente.
656		Línea en blanco: separa visualmente bloques para que el código sea más legible.
657	private void abrirFormulario(Ingrediente ingrediente) {	Declara el método abrirFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
658	if (mostrarDialogoIngrediente(ingrediente)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
659	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
660	}	Cierra el bloque abierto anteriormente.
661	}	Cierra el bloque abierto anteriormente.
662		Línea en blanco: separa visualmente bloques para que el código sea más legible.
663	private void editar(int id) {	Declara el método editar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
664	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
665	Ingrediente ingrediente = ingredienteCRUD.obtenerPorId(id);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
666	if (ingrediente == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
667	JOptionPane.showMessageDialog(this, "No se encontró el ingrediente.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
668	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
669	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
670	}	Cierra el bloque abierto anteriormente.
671	abrirFormulario(ingrediente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
672	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
673	mostrarErrorDatos("obtener el ingrediente", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
674	}	Cierra el bloque abierto anteriormente.
675	}	Cierra el bloque abierto anteriormente.
676		Línea en blanco: separa visualmente bloques para que el código sea más legible.
677	private void movimiento(int id) {	Declara el método movimiento. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
678	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
679	Ingrediente ingrediente = ingredienteCRUD.obtenerPorId(id);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
680	if (ingrediente == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
681	JOptionPane.showMessageDialog(this, "No se encontró el ingrediente.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
682	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
683	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
684	}	Cierra el bloque abierto anteriormente.
685	if (mostrarDialogoMovimiento(ingrediente)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
686	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
687	}	Cierra el bloque abierto anteriormente.
688	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
689	mostrarErrorDatos("obtener el ingrediente", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

690	}	Cierra el bloque abierto anteriormente.
691	}	Cierra el bloque abierto anteriormente.
692		Línea en blanco: separa visualmente bloques para que el código sea más legible.
693	private void desactivar(Ingrediente ingrediente) {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
694	if (!ingrediente.isActivo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
695	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
696	}	Cierra el bloque abierto anteriormente.
697	int opcion = JOptionPane.showConfirmDialog(this,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
698	"¿Desactivar «" + ingrediente.getNombre() + "»?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
699	"Confirmar desactivación", JOptionPane.YES_NO_OPTION);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
700	if (opcion != JOptionPane.YES_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
701	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
702	}	Cierra el bloque abierto anteriormente.
703	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
704	ingredienteCRUD.cambiarEstado(ingrediente.getIdIngrediente(), false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
705	cargarTodo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
706	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
707	mostrarErrorDatos("desactivar el ingrediente", ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
708	}	Cierra el bloque abierto anteriormente.
709	}	Cierra el bloque abierto anteriormente.
710		Línea en blanco: separa visualmente bloques para que el código sea más legible.
711	private void mostrarErrorDatos(String operacion, Throwable ex) {	Declara el método mostrarErrorDatos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
712	if (errorConexionMostrado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
713	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
714	}	Cierra el bloque abierto anteriormente.
715	errorConexionMostrado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
716	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
717	"No fue posible " + operacion + ".\n" + ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
718	"Error de base de datos", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
719	}	Cierra el bloque abierto anteriormente.
720		Línea en blanco: separa visualmente bloques para que el código sea más legible.
721	private final class ModeloTabla extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
722		Línea en blanco: separa visualmente bloques para que el código sea más legible.
723	private final String[] columnas = {"ID", "Ingrediente", "Unidad",	Declara una constante o referencia que no puede reasignarse después de inicializarse.
724	"Stock actual", "Stock mínimo", "Nivel", "Estado", "Acciones"};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
725	private List<Ingrediente> ingredientes = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
726		Línea en blanco: separa visualmente bloques para que el código sea más legible.
727	void setIngredientes(List<Ingrediente> nuevos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
728	ingredientes = List.copyOf(nuevos);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
729	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
730	}	Cierra el bloque abierto anteriormente.
731		Línea en blanco: separa visualmente bloques para que el código sea más legible.
732	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
733	public int getRowCount() {	Declara el método getRowCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

734	return ingredientes.size();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
735	}	Cierra el bloque abierto anteriormente.
736		Línea en blanco: separa visualmente bloques para que el código sea más legible.
737	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
738	public int getColumnCount() {	Declara el método getColumnCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
739	return columnas.length;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
740	}	Cierra el bloque abierto anteriormente.
741		Línea en blanco: separa visualmente bloques para que el código sea más legible.
742	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
743	public String getColumnName(int columna) {	Declara el método getColumnName. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
744	return columnas[columna];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
745	}	Cierra el bloque abierto anteriormente.
746		Línea en blanco: separa visualmente bloques para que el código sea más legible.
747	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
748	public Class<?> getColumnClass(int columna) {	Declara el método getColumnClass. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
749	return columna >= 5 ? Ingrediente.class : String.class;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
750	}	Cierra el bloque abierto anteriormente.
751		Línea en blanco: separa visualmente bloques para que el código sea más legible.
752	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
753	public boolean isCellEditable(int fila, int columna) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
754	return columna == 7;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
755	}	Cierra el bloque abierto anteriormente.
756		Línea en blanco: separa visualmente bloques para que el código sea más legible.
757	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
758	public Object getValueAt(int fila, int columna) {	Declara el método getValueAt. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
759	Ingrediente item = ingredientes.get(fila);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
760	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
761	case 0 ->	Define una de las rutas posibles dentro del switch.
762	String.format("#%04d", item.getIdIngrediente());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
763	case 1 ->	Define una de las rutas posibles dentro del switch.
764	item.getNombre();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
765	case 2 ->	Define una de las rutas posibles dentro del switch.
766	item.getUnidadMedida();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
767	case 3 ->	Define una de las rutas posibles dentro del switch.
768	item.getStockActual().toPlainString();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
769	case 4 ->	Define una de las rutas posibles dentro del switch.
770	item.getStockMinimo().toPlainString();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
771	case 5, 6, 7 ->	Define una de las rutas posibles dentro del switch.
772	item;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
773	default ->	Define una de las rutas posibles dentro del switch.
774	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
775	};	Cierra el bloque abierto anteriormente y termina la declaración.
776	}	Cierra el bloque abierto anteriormente.
777	}	Cierra el bloque abierto anteriormente.

778		Línea en blanco: separa visualmente bloques para que el código sea más legible.
779	private final class RenderNivel implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
780		Línea en blanco: separa visualmente bloques para que el código sea más legible.
781	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
782	public Component getTableCellRendererComponent(javax.swing.JTable t,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
783	Object valor, boolean seleccionado, boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
784	int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
785	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
786	return etiquetaEstado(item.tieneStockBajo() ? "Bajo" : "Normal",	Finaliza el método y devuelve el valor indicado a quien lo llamó.
787	item.tieneStockBajo() ? ROJO : new Color(34, 139, 71),	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
788	item.tieneStockBajo() ? new Color(252, 233, 233)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
789	: new Color(230, 245, 234), t, seleccionado);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
790	}	Cierra el bloque abierto anteriormente.
791	}	Cierra el bloque abierto anteriormente.
792		Línea en blanco: separa visualmente bloques para que el código sea más legible.
793	private final class RenderEstado implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
794		Línea en blanco: separa visualmente bloques para que el código sea más legible.
795	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
796	public Component getTableCellRendererComponent(javax.swing.JTable t,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
797	Object valor, boolean seleccionado, boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
798	int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
799	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
800	return etiquetaEstado(item.isActive() ? "Activo" : "Inactivo",	Finaliza el método y devuelve el valor indicado a quien lo llamó.
801	item.isActive() ? new Color(34, 139, 71) : SECUNDARIO,	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
802	item.isActive() ? new Color(230, 245, 234)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
803	: new Color(237, 239, 243), t, seleccionado);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
804	}	Cierra el bloque abierto anteriormente.
805	}	Cierra el bloque abierto anteriormente.
806		Línea en blanco: separa visualmente bloques para que el código sea más legible.
807	private JPanel etiquetaEstado(String texto, Color primerPlano,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
808	Color fondoEtiqueta, javax.swing.JTable tablaActual, boolean seleccionado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
809	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 0, 13));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
810	panel.setBackground(seleccionado	Configura una propiedad mediante setBackground en el objeto indicado.
811	? tablaActual.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
812	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
813	etiqueta.setFont(tema.media(12f));	Configura una propiedad mediante setFont en el objeto indicado.
814	etiqueta.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
815	etiqueta.setForeground(primerPlano);	Configura una propiedad mediante setForeground en el objeto indicado.
816	etiqueta.setBackground(fondoEtiqueta);	Configura una propiedad mediante setBackground en el objeto indicado.
817	etiqueta.setBorder(new EmptyBorder(4, 11, 4, 11));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
818	panel.add(etiqueta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
819	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
820	}	Cierra el bloque abierto anteriormente.
821		Línea en blanco: separa visualmente bloques para que el código sea más legible.

822	private final class RenderAcciones implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
823		Línea en blanco: separa visualmente bloques para que el código sea más legible.
824	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
825	public Component getTableCellRendererComponent(javax.swing.JTable t,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
826	Object valor, boolean seleccionado, boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
827	int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
828	Ingrediente item = (Ingrediente) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
829	JPanel panel = panelAcciones(seleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
830	? t.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
831	panel.add(botonAccion(IconosUsuarios.Tipo.EDITAR,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
832	new Color(214, 151, 0), "Editar ingrediente"));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
833	BotonRedondeado mover = botonAccion(null, AZUL, "Registrar movimiento");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
834	mover.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
835	panel.add(mover);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
836	BotonRedondeado baja = botonAccion(IconosUsuarios.Tipo.ELIMINAR,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
837	ROJO, "Desactivar ingrediente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
838	baja.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
839	panel.add(baja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
840	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
841	}	Cierra el bloque abierto anteriormente.
842	}	Cierra el bloque abierto anteriormente.
843		Línea en blanco: separa visualmente bloques para que el código sea más legible.
844	private final class EditorAcciones extends AbstractCellEditor	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
845	implements TableCellEditor {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
846		Línea en blanco: separa visualmente bloques para que el código sea más legible.
847	private final JPanel panel = panelAcciones(Color.WHITE);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
848	private final BotonRedondeado editar = botonAccion(IconosUsuarios.Tipo.EDITAR,	Declara una constante o referencia que no puede reasignarse después de inicializarse.
849	new Color(214, 151, 0), "Editar ingrediente");	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
850	private final BotonRedondeado mover = botonAccion(null, AZUL,	Declara una constante o referencia que no puede reasignarse después de inicializarse.
851	"Registrar movimiento");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
852	private final BotonRedondeado baja = botonAccion(IconosUsuarios.Tipo.ELIMINAR,	Declara una constante o referencia que no puede reasignarse después de inicializarse.
853	ROJO, "Desactivar ingrediente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
854	private Ingrediente item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
855		Línea en blanco: separa visualmente bloques para que el código sea más legible.
856	EditorAcciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
857	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
858	panel.add(mover);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
859	panel.add(baja);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
860	editar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
861	int id = item.getIdIngrediente();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
862	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
863	editar(id);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
864	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
865	mover.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.

866	int id = item.getIdIngrediente();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
867	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
868	movimiento(id);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
869	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
870	baja.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
871	Ingrediente seleccionado = item;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
872	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
873	desactivar(seleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
874	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
875	}	Cierra el bloque abierto anteriormente.
876		Línea en blanco: separa visualmente bloques para que el código sea más legible.
877	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
878	public Component getTableCellEditorComponent(javax.swing.JTable t,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
879	Object valor, boolean seleccionado, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
880	item = (Ingrediente) valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
881	mover.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
882	baja.setEnabled(item.isActivo());	Habilita o deshabilita la interacción con el componente.
883	panel.setBackground(t.getSelectionBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
884	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
885	}	Cierra el bloque abierto anteriormente.
886		Línea en blanco: separa visualmente bloques para que el código sea más legible.
887	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
888	public Object getCellEditorValue() {	Declara el método getCellEditorValue. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
889	return item;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
890	}	Cierra el bloque abierto anteriormente.
891	}	Cierra el bloque abierto anteriormente.
892		Línea en blanco: separa visualmente bloques para que el código sea más legible.
893	private JPanel panelAcciones(Color fondo) {	Declara el método panelAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
894	JPanel panel = new JPanel(new FlowLayout(FlowLayout.CENTER, 5, 11));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
895	panel.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
896	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
897	}	Cierra el bloque abierto anteriormente.
898		Línea en blanco: separa visualmente bloques para que el código sea más legible.
899	private BotonRedondeado botonAccion(IconosUsuarios.Tipo icono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
900	Color color, String descripcion) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
901	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
902	if (icono == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
903	boton.setText("±");	Cambia el texto visible de un componente de la interfaz.
904	boton.setFont(tema.negrita(16f));	Configura una propiedad mediante setFont en el objeto indicado.
905	boton.setForeground(color);	Configura una propiedad mediante setForeground en el objeto indicado.
906	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
907	boton.setIcon(IconosUsuarios.crear(icono, color, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
908	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
909	}	Cierra el bloque abierto anteriormente.

910	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
911	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
912	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
913	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
914	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
915	boton.setRadio(9);	Configura una propiedad mediante setRadio en el objeto indicado.
916	boton.setPreferredSize(new Dimension(32, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
917	boton.setToolTipText(descripcion);	Configura una propiedad mediante setToolTipText en el objeto indicado.
918	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
919	}	Cierra el bloque abierto anteriormente.
920		Línea en blanco: separa visualmente bloques para que el código sea más legible.
921	// Pega estos métodos y la clase interna dentro de IngredientesPanel,	Comentario para humanos; Java no ejecuta este texto. Pega estos métodos y la clase interna dentro de IngredientesPanel,
922	// fuera de initComponents() y fuera del bloque generado.	Comentario para humanos; Java no ejecuta este texto. fuera de initComponents() y fuera del bloque generado.
923	private boolean mostrarDialogoIngrediente(Ingrediente ingrediente) {	Declara el método mostrarDialogoIngrediente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
924	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
925	JDialog dialogo = new JDialog(proprietario,	Crea una nueva instancia de JDialog y la usa o guarda para continuar el proceso.
926	java.awt.Dialog.ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
927	dialogo.setUndecorated(true);	Configura una propiedad mediante setUndecorated en el objeto indicado.
928	dialogo.setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
929	dialogo.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);	Configura una propiedad mediante setDefaultCloseOperation en el objeto indicado.
930	boolean[] guardado = {false};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
931	PanelFormularioIngrediente formulario = new PanelFormularioIngrediente(ingrediente);	Crea una nueva instancia de PanelFormularioIngrediente y la usa o guarda para continuar el proceso.
932	formulario.addPropertyChangeListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
933	if (PanelFormularioIngrediente.EVENTO_GUARDADO.equals(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
934	evento.getPropertyName())) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
935	guardado[0] = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
936	dialogo.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
937	} else if (PanelFormularioIngrediente.EVENTO_CANCELAR.equals(	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
938	evento.getPropertyName())) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
939	dialogo.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
940	}	Cierra el bloque abierto anteriormente.
941	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
942	dialogo.setContentPane(formulario);	Configura una propiedad mediante setContentPane en el objeto indicado.
943	dialogo.pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
944	dialogo.setLocationRelativeTo(proprietario);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
945	mostrarDialogoConFondo(dialogo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
946	return guardado[0];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
947	}	Cierra el bloque abierto anteriormente.
948		Línea en blanco: separa visualmente bloques para que el código sea más legible.
949	private void mostrarDialogoConFondo(JDialog dialogo) {	Declara el método mostrarDialogoConFondo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
950	JRootPane raiz = SwingUtilities.getRootPane(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
951	Component anterior = raiz == null ? null : raiz.getGlassPane();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
952	boolean eraVisible = anterior != null && anterior.isVisible();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

953	if (raiz != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
954	JPanel fondo = new JPanel() {	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
955	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
956	protected void paintComponent(Graphics grafico) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
957	Graphics2D g = (Graphics2D) grafico.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
958	g.setColor(new Color(0, 12, 28, 90));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
959	g.fillRect(0, 0, getWidth(), getHeight());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
960	g.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
961	}	Cierra el bloque abierto anteriormente.
962	};	Cierra el bloque abierto anteriormente y termina la declaración.
963	fondo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
964	fondo.addMouseListener(new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
965	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
966	raiz.setGlassPane(fondo);	Configura una propiedad mediante setGlassPane en el objeto indicado.
967	fondo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
968	}	Cierra el bloque abierto anteriormente.
969	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
970	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
971	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
972	if (raiz != null && anterior != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
973	raiz.setGlassPane(anterior);	Configura una propiedad mediante setGlassPane en el objeto indicado.
974	anterior.setVisible(eraVisible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
975	}	Cierra el bloque abierto anteriormente.
976	}	Cierra el bloque abierto anteriormente.
977	}	Cierra el bloque abierto anteriormente.
978		Línea en blanco: separa visualmente bloques para que el código sea más legible.
979	private final class PanelFormularioIngrediente extends PanelFlotante {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
980		Línea en blanco: separa visualmente bloques para que el código sea más legible.
981	public static final String EVENTO_GUARDADO = "ingredienteGuardado";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
982	public static final String EVENTO_CANCELAR = "ingredienteCancelado";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
983	private final Ingrediente original;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
984	private final JTextField campoNombre = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
985	private final JTextField campoUnidadMedida = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
986	private final JTextField campoStockMinimo = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
987	private final BotonDesplegable selectorEstado = new BotonDesplegable();	Crea una nueva instancia de BotonDesplegable y la usa o guarda para continuar el proceso.
988	private final JLabel labelError = new JLabel(" ");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
989	private final BotonRedondeado botonGuardar = boton("Guardar ingrediente", true);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
990		Línea en blanco: separa visualmente bloques para que el código sea más legible.
991	private PanelFormularioIngrediente(Ingrediente original) {	Declara el método PanelFormularioIngrediente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
992	this.original = original;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
993	setColorFondo(Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
994	setColorBorde(BORDE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
995	setRadio(26);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
996	setPreferredSize(new Dimension(760, 480));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.

997	setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
998	construirCabecera();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
999	construirCampos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1000	construirPie();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1001	if (original != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1002	campoNombre.setText(original.getNombre());	Cambia el texto visible de un componente de la interfaz.
1003	campoUnidadMedida.setText(original.getUnidadMedida());	Cambia el texto visible de un componente de la interfaz.
1004	campoStockMinimo.setText(original.getStockMinimo().toString());	Cambia el texto visible de un componente de la interfaz.
1005	selectorEstado.setText(original.isActivo() ? "Activo" : "Inactivo");	Cambia el texto visible de un componente de la interfaz.
1006	}	Cierra el bloque abierto anteriormente.
1007	}	Cierra el bloque abierto anteriormente.
1008		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1009	private void construirCabecera() {	Declara el método construirCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1010	JPanel cabecera = new JPanel(null);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1011	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1012	cabecera.setPreferredSize(new Dimension(760, 112));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1013	PanelCircular circulo = new PanelCircular();	Crea una nueva instancia de PanelCircular y la usa o guarda para continuar el proceso.
1014	circulo.setColorFondo(new Color(255, 244, 211));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1015	circulo.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
1016	circulo.setBounds(30, 19, 80, 80);	Configura una propiedad mediante setBounds en el objeto indicado.
1017	java.net.URL urlIcono = getClass().getResource("/Imagenes/lista_icono.png");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1018	if (urlIcono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1019	JLabel imagen = new JLabel(new javax.swing.ImageIcon(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1020	new javax.swing.ImageIcon(urlIcono).getImage()	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
1021	.getScaledInstance(39, 39, java.awt.Image.SCALE_SMOOTH));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1022	imagen.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
1023	circulo.add(imagen, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1024	}	Cierra el bloque abierto anteriormente.
1025	cabecera.add(circulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1026	JLabel titulo = new JLabel(original == null	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1027	? "Agregar ingrediente" : "Editar ingrediente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1028	titulo.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
1029	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1030	titulo.setBounds(125, 27, 595, 36);	Configura una propiedad mediante setBounds en el objeto indicado.
1031	cabecera.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1032	JLabel subtitulo = new JLabel(original == null	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1033	? "Completa los datos del nuevo ingrediente"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1034	: "Actualiza la información de "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1035	+ String.format("#%04d", original.getIdIngrediente());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1036	subtitulo.setFont(tema.regular(15f));	Configura una propiedad mediante setFont en el objeto indicado.
1037	subtitulo.setForeground(new Color(92, 103, 124));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1038	subtitulo.setBounds(125, 62, 595, 27);	Configura una propiedad mediante setBounds en el objeto indicado.
1039	cabecera.add(subtitulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1040	add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

1041	}	Cierra el bloque abierto anteriormente.
1042		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1043	private void construirCampos() {	Declara el método construirCampos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1044	estilizar(campoNombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1045	estilizar(campoUnidadMedida);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1046	estilizar(campoStockMinimo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1047	selectorEstado.setText("Activo");	Cambia el texto visible de un componente de la interfaz.
1048	selectorEstado.setTextoDesplegable("Activo;Inactivo");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
1049	selectorEstado.addMenuOpcionListener(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1050	evento -> selectorEstado.setText(evento.getActionCommand());	Cambia el texto visible de un componente de la interfaz.
1051	selectorEstado.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1052	selectorEstado.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1053	selectorEstado.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
1054	selectorEstado.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
1055	JLabel stock = new JLabel(original == null ? "0.00"	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1056	: original.getStockActual().toString());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1057	stock.setFont(tema.media(15f));	Configura una propiedad mediante setFont en el objeto indicado.
1058	stock.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1059	stock.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
1060	BorderFactory.createLineBorder(BORDE), new EmptyBorder(0, 14, 0, 14));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1061		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1062	JPanel campos = new JPanel(new GridBagLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1063	campos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1064	campos.setBorder(new EmptyBorder(4, 28, 4, 28));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1065	fila(campos, 0, campo("Nombre", campoNombre),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1066	campo("Unidad de medida", campoUnidadMedida));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1067	fila(campos, 1, campo("Stock actual (solo lectura)", stock),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1068	campo("Stock mínimo", campoStockMinimo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1069	fila(campos, 2, campo("Estado", selectorEstado),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1070	campo("Ejemplos de unidad: gramos, mililitros, unidades",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1071	new JLabel("El stock se cambia con un movimiento."));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1072	add(campos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1073	}	Cierra el bloque abierto anteriormente.
1074		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1075	private JPanel campo(String texto, JComponent control) {	Declara el método campo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1076	JPanel panel = new JPanel(new BorderLayout(0, 6));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1077	panel.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1078	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1079	etiqueta.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
1080	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1081	panel.add(etiqueta, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1082	control.setPreferredSize(new Dimension(310, 46));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1083	panel.add(control, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1084	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

1085	}	Cierra el bloque abierto anteriormente.
1086		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1087	private void fila(JPanel panel, int numero, JPanel izquierda, JPanel derecha) {	Declara el método fila. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1088	GridBagConstraints c = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1089	c.gridy = numero;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1090	c.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1091	c.weightx = 0.5;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1092	c.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1093	c.anchor = GridBagConstraints.NORTH;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1094	c.insets = new Insets(6, 10, 12, 13);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1095	panel.add(izquierda, c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1096	c.gridx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1097	c.insets = new Insets(6, 13, 12, 10);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1098	panel.add(derecha, c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1099	}	Cierra el bloque abierto anteriormente.
1100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1101	private void construirPie() {	Declara el método construirPie. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1102	JPanel pie = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1103	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1104	pie.setPreferredSize(new Dimension(760, 104));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1105	pie.setBorder(new EmptyBorder(0, 30, 20, 30));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1106	labelError.setForeground(ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
1107	labelError.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
1108	pie.add(labelError, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1109	JPanel botones = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1110	botones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1111	if (original != null && original.isActivo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1112	BotonRedondeado desactivar = boton("Desactivar ingrediente", false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1113	desactivar.setPreferredSize(new Dimension(230, 48));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1114	desactivar.addActionListener(evento -> desactivar());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1116	JPanel izquierda = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1117	izquierda.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1118	izquierda.add(desactivar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1119	botones.add(izquierda, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1120	}	Cierra el bloque abierto anteriormente.
1121	JPanel derecha = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1122	derecha.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1123	BotonRedondeado cancelar = boton("Cancelar", false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1124	cancelar.setPreferredSize(new Dimension(130, 48));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1125	botonGuardar.setPreferredSize(new Dimension(220, 48));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1126	cancelar.addActionListener(evento -> firePropertyChange(EVENTO_CANCELAR, false, true));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1127	botonGuardar.addActionListener(evento -> guardar());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1128	derecha.add(cancelar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

1129	derecha.add(botonGuardar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1130	botones.add(derecha, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1131	pie.add(botones, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1132	add(pie, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1133	}	Cierra el bloque abierto anteriormente.
1134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1135	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1136	labelError.setText(" ");	Cambia el texto visible de un componente de la interfaz.
1137	String nombre = campoNombre.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1138	String unidad = campoUnidadMedida.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1139	if (nombre.isEmpty() nombre.length() > 100) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1140	labelError.setText("Ingresa un nombre de hasta 100 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1141	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1142	}	Cierra el bloque abierto anteriormente.
1143	if (unidad.isEmpty() unidad.length() > 20) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1144	labelError.setText("Ingresa una unidad de hasta 20 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1145	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1146	}	Cierra el bloque abierto anteriormente.
1147	BigDecimal minimo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1148	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1149	minimo = new BigDecimal(campoStockMinimo.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1150	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1151	labelError.setText("Ingresa un stock mínimo válido. Usa punto decimal.");	Cambia el texto visible de un componente de la interfaz.
1152	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1153	}	Cierra el bloque abierto anteriormente.
1154	if (minimo.signum() < 0 minimo.scale() > 2	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1155	minimo.compareTo(new BigDecimal("99999999.99")) > 0) {	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1156	labelError.setText("El mínimo debe ser positivo y tener hasta dos decimales.");	Cambia el texto visible de un componente de la interfaz.
1157	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1158	}	Cierra el bloque abierto anteriormente.
1159	botonGuardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
1160	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1161	Ingrediente ingrediente = original == null ? new Ingrediente() : original;	Crea una nueva instancia de Ingrediente y la usa o guarda para continuar el proceso.
1162	ingrediente.setNombre(nombre);	Configura una propiedad mediante setNombre en el objeto indicado.
1163	ingrediente.setUnidadMedida(unidad);	Configura una propiedad mediante setUnidadMedida en el objeto indicado.
1164	ingrediente.setStockMinimo(minimo);	Configura una propiedad mediante setStockMinimo en el objeto indicado.
1165	ingrediente.setActivo("Activo".equals(selectorEstado.getText()));	Configura una propiedad mediante setActivo en el objeto indicado.
1166	if (original == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1167	ingredienteCRUD.insertar(ingrediente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1168	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1169	ingredienteCRUD.actualizar(ingrediente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1170	}	Cierra el bloque abierto anteriormente.
1171	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1172	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

1173	labelError.setText("No fue posible guardar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
1174	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1175	botonGuardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
1176	}	Cierra el bloque abierto anteriormente.
1177	}	Cierra el bloque abierto anteriormente.
1178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1179	private void desactivar() {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1180	int respuesta = JOptionPane.showConfirmDialog(this,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1181	"¿Desactivar «" + original.getNombre() + "»?",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1182	"Confirmar", JOptionPane.YES_NO_OPTION);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1183	if (respuesta != JOptionPane.YES_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1184	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1185	}	Cierra el bloque abierto anteriormente.
1186	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1187	ingredienteCRUD.cambiarEstado(original.getIdIngrediente(), false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1188	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1189	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1190	labelError.setText("No fue posible desactivar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
1191	}	Cierra el bloque abierto anteriormente.
1192	}	Cierra el bloque abierto anteriormente.
1193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1194	private void estilizar(JTextField campo) {	Declara el método estilizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1195	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1196	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1197	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
1198	BorderFactory.createLineBorder(BORDE), new EmptyBorder(0, 13, 0, 13));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1199	}	Cierra el bloque abierto anteriormente.
1200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1201	private BotonRedondeado boton(String texto, boolean principal) {	Declara el método boton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1202	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
1203	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
1204	boton.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1205	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1206	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
1207	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1208	boton.setColorInicio(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
1209	boton.setColorFinal(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
1210	boton.setColorBorde(principal ? AMARILLO : ROJO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1211	boton.setForeground(principal ? AZUL : ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
1212	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1213	}	Cierra el bloque abierto anteriormente.
1214	}	Cierra el bloque abierto anteriormente.
1215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1216	// Formulario para registrar movimientos de inventario.	Comentario para humanos; Java no ejecuta este texto. Formulario para registrar movimientos de inventario.

1217	// fuera de initComponents() y fuera del bloque generado.	Comentario para humanos; Java no ejecuta este texto. fuera de initComponents() y fuera del bloque generado.
1218	private boolean mostrarDialogoMovimiento(Ingrediente ingrediente) {	Declara el método mostrarDialogoMovimiento. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1219	Window propietario = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1220	DialogoMovimientoIngrediente dialogo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1221	= new DialogoMovimientoIngrediente(propietario, ingrediente);	Crea una nueva instancia de DialogoMovimientoIngrediente y la usa o guarda para continuar el proceso.
1222	mostrarDialogoConFondo(dialogo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1223	return dialogo.guardado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1224	}	Cierra el bloque abierto anteriormente.
1225		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1226	private final class DialogoMovimientoIngrediente extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1228	private final Ingrediente ingrediente;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1229	private final JComboBox<String> tipo = new JComboBox<>(new String[]{	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
1230	"Entrada", "Salida", "Ajuste"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1231	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1232	private final JTextField cantidad = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1233	private final JTextField motivo = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1234	private final JLabel instruccion = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1235	private final JLabel error = new JLabel(" ");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1236	private final BotonRedondeado guardar = crearBoton("Guardar movimiento", true);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1237	private boolean guardado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1239	private DialogoMovimientoIngrediente(Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1240	Ingrediente ingrediente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1241	super(propietario, ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1242	this.ingrediente = ingrediente;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
1243	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1244	setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1245	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1246	setContentPane(crearContenido());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1247	tipo.addActionListener(e -> actualizarInstruccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1248	actualizarInstruccion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1249	guardar.addActionListener(e -> guardarMovimiento());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1250	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1251	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1252	}	Cierra el bloque abierto anteriormente.
1253		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1254	private PanelFlotante crearContenido() {	Declara el método crearContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1255	PanelFlotante panel = new PanelFlotante();	Crea una nueva instancia de PanelFlotante y la usa o guarda para continuar el proceso.
1256	panel.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
1257	panel.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1258	panel.setRadio(26);	Configura una propiedad mediante setRadio en el objeto indicado.
1259	panel.setPreferredSize(new Dimension(650, 435));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1260	panel.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.

1261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1262	JPanel cabecera = new JPanel(new BorderLayout(0, 4));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1263	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1264	cabecera.setBorder(new EmptyBorder(27, 32, 15, 32));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1265	JLabel titulo = new JLabel("Movimiento de inventario");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1266	titulo.setFont(tema.negrita(24f));	Configura una propiedad mediante setFont en el objeto indicado.
1267	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1268	JLabel subtitulo = new JLabel(String.format("#%04d · %s · Stock actual: %s %s",	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1269	ingrediente.getIdIngrediente(), ingrediente.getNombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1270	ingrediente.getStockActual().toPlainString(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1271	ingrediente.getUnidadMedida()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1272	subtitulo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1273	subtitulo.setForeground(new Color(92, 103, 124));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1274	cabecera.add(titulo, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1275	cabecera.add(subtitulo, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1276	panel.add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1278	JPanel campos = new JPanel(new GridBagLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1279	campos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1280	campos.setBorder(new EmptyBorder(0, 30, 0, 30));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1281	tipo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1282	tipo.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
1283	tipo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1284	tipo.setBorder(BorderFactory.createLineBorder(BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
1285	estilizar(cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1286	estilizar(motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1287	agregarCampo(campos, 0, "Tipo de movimiento", tipo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1288	agregarCampo(campos, 1, "Cantidad (" + ingrediente.getUnidadMedida() + ")", cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1289	agregarCampo(campos, 2, "Motivo (opcional)", motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1290	instruccion.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
1291	instruccion.setForeground(new Color(92, 103, 124));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1292	GridBagConstraints c = new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1293	c.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1294	c.gridy = 3;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1295	c.weightx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1296	c.fill = GridBagConstraints.HORIZONTAL;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1297	c.insets = new Insets(0, 4, 0, 4);	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1298	campos.add(instruccion, c);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1299	panel.add(campos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1301	JPanel pie = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1302	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
1303	pie.setPreferredSize(new Dimension(650, 95));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1304	pie.setBorder(new EmptyBorder(0, 32, 20, 32));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.

1305	<code>error.setFont(tema.media(12f));</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
1306	<code>error.setForeground(R0J0);</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
1307	<code>pie.add(error, BorderLayout.NORTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1308	<code>JPanel botones = new JPanel(new FlowLayout(FlowLayout.RIGHT, 14, 0));</code>	Crea una nueva instancia de <code>JPanel</code> y la usa o guarda para continuar el proceso.
1309	<code>botones.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
1310	<code>BotonRedondeado cancelar = crearBoton("Cancelar", false);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1311	<code>cancelar.setPreferredSize(new Dimension(128, 45));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
1312	<code>guardar.setPreferredSize(new Dimension(195, 45));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
1313	<code>cancelar.addActionListener(e -> dispose());</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1314	<code>botones.add(cancelar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1315	<code>botones.add(guardar);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1316	<code>pie.add(botones, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1317	<code>panel.add(pie, BorderLayout.SOUTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1318	<code>return panel;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1319	<code>}</code>	Cierra el bloque abierto anteriormente.
1320		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1321	<code>private void agregarCampo(JPanel panel, int fila, String nombre,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1322	<code>java.awt.Component control) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1323	<code>JPanel filaPanel = new JPanel(new BorderLayout(0, 5));</code>	Crea una nueva instancia de <code>JPanel</code> y la usa o guarda para continuar el proceso.
1324	<code>filaPanel.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
1325	<code>JLabel etiqueta = new JLabel(nombre);</code>	Crea una nueva instancia de <code>JLabel</code> y la usa o guarda para continuar el proceso.
1326	<code>etiqueta.setFont(tema.media(13f));</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
1327	<code>etiqueta.setForeground(AZUL);</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
1328	<code>filaPanel.add(etiqueta, BorderLayout.NORTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1329	<code>control.setPreferredSize(new Dimension(565, 40));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
1330	<code>filaPanel.add(control, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1331	<code>GridBagConstraints c = new GridBagConstraints();</code>	Crea una nueva instancia de <code>GridBagConstraints</code> y la usa o guarda para continuar el proceso.
1332	<code>c.gridx = 0;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1333	<code>c.gridy = fila;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1334	<code>c.weightx = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1335	<code>c.fill = GridBagConstraints.HORIZONTAL;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1336	<code>c.insets = new Insets(0, 4, 10, 4);</code>	Crea una nueva instancia de <code>Insets</code> y la usa o guarda para continuar el proceso.
1337	<code>panel.add(filaPanel, c);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1338	<code>}</code>	Cierra el bloque abierto anteriormente.
1339		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1340	<code>private void actualizarInstruccion() {</code>	Declara el método <code>actualizarInstruccion</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1341	<code>instruccion.setText(tipo.getSelectedIndex() == 2</code>	Cambia el texto visible de un componente de la interfaz.
1342	<code>? "En Ajuste, escribe el stock final que debe quedar."</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1343	<code>: "En Entrada o Salida, escribe cuánto se agrega o retira.");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1344	<code>}</code>	Cierra el bloque abierto anteriormente.
1345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1346	<code>private void guardarMovimiento() {</code>	Declara el método <code>guardarMovimiento</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1347	<code>error.setText(" ");</code>	Cambia el texto visible de un componente de la interfaz.

1348	BigDecimal valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1349	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1350	valor = new BigDecimal(cantidad.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1351	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1352	error.setText("Ingresa una cantidad válida con punto decimal.");	Cambia el texto visible de un componente de la interfaz.
1353	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1354	}	Cierra el bloque abierto anteriormente.
1355	boolean ajuste = tipo.getSelectedIndex() == 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1356	if (valor.scale() > 2 valor.compareTo(new BigDecimal("99999999.99")) > 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1357	(ajuste ? valor.signum() < 0 : valor.signum() <= 0)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1358	error.setText("Cantidad inválida; usa hasta dos decimales.");	Cambia el texto visible de un componente de la interfaz.
1359	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1360	}	Cierra el bloque abierto anteriormente.
1361	String textoMotivo = motivo.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1362	if (textoMotivo.length() > 150) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1363	error.setText("El motivo admite hasta 150 caracteres.");	Cambia el texto visible de un componente de la interfaz.
1364	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1365	}	Cierra el bloque abierto anteriormente.
1366	String codigo = switch (tipo.getSelectedIndex()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1367	case 0 ->	Define una de las rutas posibles dentro del switch.
1368	"ENTRADA";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1369	case 1 ->	Define una de las rutas posibles dentro del switch.
1370	"SALIDA";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1371	default ->	Define una de las rutas posibles dentro del switch.
1372	"AJUSTE";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1373	};	Cierra el bloque abierto anteriormente y termina la declaración.
1374	guardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
1375	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1376	ingredienteCRUD.registrarMovimiento(ingrediente.getIdIngrediente(), codigo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1377	valor, textoMotivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1378	guardado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1379	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1380	} catch (SQLException IllegalArgumentException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1381	error.setText("No fue posible registrar el movimiento: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
1382	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1383	guardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
1384	}	Cierra el bloque abierto anteriormente.
1385	}	Cierra el bloque abierto anteriormente.
1386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1387	private void estilizar(JTextField campo) {	Declara el método estilizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1388	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
1389	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1390	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
1391	BorderFactory.createLineBorder(BORDE), new EmptyBorder(0, 12, 0, 12)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.

1392	}	Cierra el bloque abierto anteriormente.
1393		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1394	private BotonRedondeado crearBoton(String texto, boolean principal) {	Declara el método crearBoton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1395	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
1396	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
1397	boton.setFont(tema.negrita(13f));	Configura una propiedad mediante setFont en el objeto indicado.
1398	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1399	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
1400	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1401	boton.setColorInicio(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
1402	boton.setColorFinal(principal ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
1403	boton.setColorBorde(principal ? AMARILLO : ROJO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1404	boton.setForeground(principal ? AZUL : ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
1405	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1406	}	Cierra el bloque abierto anteriormente.
1407	}	Cierra el bloque abierto anteriormente.
1408		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1410	private void campoBusquedaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_campoBusquedaAc tionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1411	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
1412	}GEN-LAST:event_campoBusquedaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1414	private void filtroStockActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroStockAction Performed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1415	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
1416	}GEN-LAST:event_filtroStockActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1417		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1419	// Variables declaration - do not modifyGEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1420	private Componentes.BotonDerretido botonAgregarIngrediente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1421	private Componentes.BotonRedondeado botonAnterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1422	private Componentes.BotonRedondeado botonLimpiarFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1423	private Componentes.BotonRedondeado botonPagina1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1424	private Componentes.BotonRedondeado botonPagina2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1425	private Componentes.BotonRedondeado botonPagina3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1426	private Componentes.BotonRedondeado botonSiguiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1427	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1428	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1429	private Componentes.BotonDesplegable filtroEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1430	private Componentes.BotonDesplegable filtroStock;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1431	private Labels.LabelEscalable labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1432	private Labels.LabelEscalable labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1433	private Labels.LabelEscalable labelEscalable4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1434	private Labels.LabelEscalable labelEscalable5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1435	private javax.swing.JLabel labelTitulo10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1436	private javax.swing.JLabel labelTitulo11;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1437	private javax.swing.JLabel labelTitulo12;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1438	private javax.swing.JLabel labelTitulo13;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1439	private javax.swing.JLabel labelTitulo14;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1440	private javax.swing.JLabel labelTitulo15;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1441	private javax.swing.JLabel labelTitulo2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1442	private javax.swing.JLabel labelTitulo3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1443	private javax.swing.JLabel labelTitulo4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1444	private javax.swing.JLabel labelTitulo5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1445	private javax.swing.JLabel labelTitulo6;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1446	private javax.swing.JLabel labelTitulo7;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1447	private javax.swing.JLabel labelTitulo8;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1448	private javax.swing.JLabel labelTitulo9;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1449	private Componentes.PanelCircular panelCircular2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1450	private Componentes.PanelCircular panelCircular3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1451	private Componentes.PanelCircular panelCircular4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1452	private Componentes.PanelCircular panelCircular5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1453	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1454	private Componentes.PanelFlotante panelFlotante1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1455	private Componentes.PanelFlotante panelFlotante2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1456	private Componentes.PanelFlotante panelFlotante3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1457	private Componentes.PanelFlotante panelFlotante4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1458	private javax.swing.JPanel panelPaginacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1459	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1460	private javax.swing.JScrollPane scrollUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1461	private Componentes.TablaAdministrativa tablaProductos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1462	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1463	}	Cierra el bloque abierto anteriormente.

16.70 src/GUI_ADMINISTRADOR/InicioAdminForm.form

Metadatos visuales de NetBeans para diseñar InicioAdminForm. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 417 | Archivo 70 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.8" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JFrameFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="defaultCloseOperation" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Property name="title" type="java.lang.String" value="Inicio de Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
7	<Property name="minimumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
8	<Dimension value="[1024, 650]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	</Property>	Cierra la sección XML abierta anteriormente.
10	</Properties>	Cierra la sección XML abierta anteriormente.
11	<SyntheticProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<SyntheticProperty name="formSizePolicy" type="int" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<SyntheticProperty name="generateCenter" type="boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	</SyntheticProperties>	Cierra la sección XML abierta anteriormente.
15	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
24	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84, -19, 0, 5, 115, 114, 0, 18, 106, 97, 118, 97, 46, 97, 119, 116, 46, 68, 105, 109, 101, 110, 115, 105, 111, 110, 65, -114, -39, -41, -84, 95, 68, 20, 2, 0, 2, 73, 0, 6, 104, 101, 105, 103, 104, 116, 73, 0, 5, 119, 105, 100, 116, 104, 120, 112, 0, 0, 4, 56, 0, 0, 7, -128"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	</AuxValues>	Cierra la sección XML abierta anteriormente.
27		Línea en blanco para organizar el XML del formulario.
28	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
29	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
30	</Layout>	Cierra la sección XML abierta anteriormente.
31	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	<Container class="javax.swing.JPanel" name="panelRaiz">	Declara un componente o contenedor visual y su clase Java.
33	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
34	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	<Color blue="fd" green="fc" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
36	</Property>	Cierra la sección XML abierta anteriormente.
37	</Properties>	Cierra la sección XML abierta anteriormente.

38	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
39	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
40	<AbsoluteConstraints x="0" y="0" width="1920" height="1080"/>	Define la posición y el tamaño del componente dentro del diseño.
41	</Constraint>	Cierra la sección XML abierta anteriormente.
42	</Constraints>	Cierra la sección XML abierta anteriormente.
43		Línea en blanco para organizar el XML del formulario.
44	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
45	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
46	</Layout>	Cierra la sección XML abierta anteriormente.
47	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
48	<Container class="javax.swing.JPanel" name="panelCabecera">	Declara un componente o contenedor visual y su clase Java.
49	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
50	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
51	<Color blue="1e" green="f" red="1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
52	</Property>	Cierra la sección XML abierta anteriormente.
53	</Properties>	Cierra la sección XML abierta anteriormente.
54	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
55	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
56	<AbsoluteConstraints x="0" y="0" width="1920" height="135"/>	Define la posición y el tamaño del componente dentro del diseño.
57	</Constraint>	Cierra la sección XML abierta anteriormente.
58	</Constraints>	Cierra la sección XML abierta anteriormente.
59		Línea en blanco para organizar el XML del formulario.
60	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
61	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
62	</Layout>	Cierra la sección XML abierta anteriormente.
63	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	<Component class="Labels.LabelEscalable" name="labelLogo">	Declara un componente o contenedor visual y su clase Java.
65	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
66	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
67	<Image iconType="3" name="/Imagenes/LogoW.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
68	</Property>	Cierra la sección XML abierta anteriormente.
69	<Property name="mantenerProporcion" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
72	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
73	<AbsoluteConstraints x="65" y="25" width="92" height="82"/>	Define la posición y el tamaño del componente dentro del diseño.
74	</Constraint>	Cierra la sección XML abierta anteriormente.
75	</Constraints>	Cierra la sección XML abierta anteriormente.
76	</Component>	Cierra la sección XML abierta anteriormente.
77	<Component class="javax.swing.JLabel" name="labelMarca">	Declara un componente o contenedor visual y su clase Java.
78	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

79	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
80		Metadato de NetBeans que conserva el estado editable del diseñador visual.
81	</Property>	Cierra la sección XML abierta anteriormente.
82	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
83	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
84	</Property>	Cierra la sección XML abierta anteriormente.
85	<Property name="text" type="java.lang.String" value="Waldonald’s"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	</Properties>	Cierra la sección XML abierta anteriormente.
87	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
88	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
89	<AbsoluteConstraints x="197" y="28" width="360" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
90	</Constraint>	Cierra la sección XML abierta anteriormente.
91	</Constraints>	Cierra la sección XML abierta anteriormente.
92	</Component>	Cierra la sección XML abierta anteriormente.
93	<Component class="javax.swing.JLabel" name="labelRol">	Declara un componente o contenedor visual y su clase Java.
94	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
95	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
96		Metadato de NetBeans que conserva el estado editable del diseñador visual.
97	</Property>	Cierra la sección XML abierta anteriormente.
98	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
99	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
100	</Property>	Cierra la sección XML abierta anteriormente.
101	<Property name="text" type="java.lang.String" value="Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102	</Properties>	Cierra la sección XML abierta anteriormente.
103	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
104	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
105	<AbsoluteConstraints x="198" y="76" width="300" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
106	</Constraint>	Cierra la sección XML abierta anteriormente.
107	</Constraints>	Cierra la sección XML abierta anteriormente.
108	</Component>	Cierra la sección XML abierta anteriormente.
109	<Component class="Labels.LabelEscalable" name="labelIconoUsuario">	Declara un componente o contenedor visual y su clase Java.
110	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
111	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
112	<Image iconType="3" name="/Imagenes/IconoUsuario.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
113	</Property>	Cierra la sección XML abierta anteriormente.
114	<Property name="mantenerProporcion" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
115	</Properties>	Cierra la sección XML abierta anteriormente.
116	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
117	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
118	<AbsoluteConstraints x="1618" y="34" width="54" height="54"/>	Define la posición y el tamaño del componente dentro del diseño.

119	</Constraint>	Cierra la sección XML abierta anteriormente.
120	</Constraints>	Cierra la sección XML abierta anteriormente.
121	</Component>	Cierra la sección XML abierta anteriormente.
122	<Component class="Componentes.BotonDesplegable" name="botonUsuario">	Declara un componente o contenedor visual y su clase Java.
123	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
124	<Property name="textoDesplegable" type="java.lang.String" value="Cerrar sesión"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
125	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
126		Metadato de NetBeans que conserva el estado editable del diseñador visual.
127	</Property>	Cierra la sección XML abierta anteriormente.
128	</Properties>	Cierra la sección XML abierta anteriormente.
129	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
130	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonUsuarioActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
131	</Events>	Cierra la sección XML abierta anteriormente.
132	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
133	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
134	<AbsoluteConstraints x="1680" y="30" width="215" height="62"/>	Define la posición y el tamaño del componente dentro del diseño.
135	</Constraint>	Cierra la sección XML abierta anteriormente.
136	</Constraints>	Cierra la sección XML abierta anteriormente.
137	</Component>	Cierra la sección XML abierta anteriormente.
138	</SubComponents>	Cierra la sección XML abierta anteriormente.
139	</Container>	Cierra la sección XML abierta anteriormente.
140	<Component class="javax.swing.JLabel" name="labelTitulo">	Declara un componente o contenedor visual y su clase Java.
141	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
142	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
143		Metadato de NetBeans que conserva el estado editable del diseñador visual.
144	</Property>	Cierra la sección XML abierta anteriormente.
145	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"> >	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
146	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
147	</Property>	Cierra la sección XML abierta anteriormente.
148	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
149	<Property name="text" type="java.lang.String" value="Inicio de Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
150	</Properties>	Cierra la sección XML abierta anteriormente.
151	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
152	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
153	<AbsoluteConstraints x="0" y="190" width="1920" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
154	</Constraint>	Cierra la sección XML abierta anteriormente.
155	</Constraints>	Cierra la sección XML abierta anteriormente.
156	</Component>	Cierra la sección XML abierta anteriormente.
157	<Component class="javax.swing.JLabel" name="labelSubtitulo">	Declara un componente o contenedor visual y su clase Java.
158	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
159	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
160		Metadato de NetBeans que conserva el estado editable del diseñador visual.

161	</Property>	Cierra la sección XML abierta anteriormente.
162	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
163	<Color blue="68" green="58" red="50" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
164	</Property>	Cierra la sección XML abierta anteriormente.
165	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
166	<Property name="text" type="java.lang.String" value="Selecciona una opción para continuar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
167	</Properties>	Cierra la sección XML abierta anteriormente.
168	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
169	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
170	<AbsoluteConstraints x="0" y="276" width="1920" height="44"/>	Define la posición y el tamaño del componente dentro del diseño.
171	</Constraint>	Cierra la sección XML abierta anteriormente.
172	</Constraints>	Cierra la sección XML abierta anteriormente.
173	</Component>	Cierra la sección XML abierta anteriormente.
174	<Container class="Componentes.PanelFlotante" name="tarjetaMenu">	Declara un componente o contenedor visual y su clase Java.
175	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
176	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
177	<Color blue="0" green="bc" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
178	</Property>	Cierra la sección XML abierta anteriormente.
179	<Property name="desplazamientoSombraY" type="int" value="6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
180	<Property name="grosorBorde" type="float" value="1.5"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
181	<Property name="radio" type="int" value="25"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
182	</Properties>	Cierra la sección XML abierta anteriormente.
183	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
184	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
185	<AbsoluteConstraints x="380" y="375" width="549" height="594"/>	Define la posición y el tamaño del componente dentro del diseño.
186	</Constraint>	Cierra la sección XML abierta anteriormente.
187	</Constraints>	Cierra la sección XML abierta anteriormente.
188		Línea en blanco para organizar el XML del formulario.
189	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
190	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
191	</Layout>	Cierra la sección XML abierta anteriormente.
192	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
193	<Container class="Componentes.PanelCircular" name="circuloMenu">	Declara un componente o contenedor visual y su clase Java.
194	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
195	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
196	<AbsoluteConstraints x="172" y="35" width="206" height="206"/>	Define la posición y el tamaño del componente dentro del diseño.
197	</Constraint>	Cierra la sección XML abierta anteriormente.
198	</Constraints>	Cierra la sección XML abierta anteriormente.
199		Línea en blanco para organizar el XML del formulario.
200	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
201	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

202	</Layout>	Cierra la sección XML abierta anteriormente.
203	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
204	<Component class="GUI_ADMINISTRADOR.IconoMenuAdmin" name="iconoMenu">	Declara un componente o contenedor visual y su clase Java.
205	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
206	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
207	<AbsoluteConstraints x="0" y="0" width="206" height="206"/>	Define la posición y el tamaño del componente dentro del diseño.
208	</Constraint>	Cierra la sección XML abierta anteriormente.
209	</Constraints>	Cierra la sección XML abierta anteriormente.
210	</Component>	Cierra la sección XML abierta anteriormente.
211	</SubComponents>	Cierra la sección XML abierta anteriormente.
212	</Container>	Cierra la sección XML abierta anteriormente.
213	<Component class="javax.swing.JLabel" name="labelTituloMenu">	Declara un componente o contenedor visual y su clase Java.
214	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
215	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
216		Metadato de NetBeans que conserva el estado editable del diseñador visual.
217	</Property>	Cierra la sección XML abierta anteriormente.
218	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
219	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
220	</Property>	Cierra la sección XML abierta anteriormente.
221	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
222	<Property name="text" type="java.lang.String" value="Ver menú"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
223	</Properties>	Cierra la sección XML abierta anteriormente.
224	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
225	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
226	<AbsoluteConstraints x="34" y="268" width="480" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
227	</Constraint>	Cierra la sección XML abierta anteriormente.
228	</Constraints>	Cierra la sección XML abierta anteriormente.
229	</Component>	Cierra la sección XML abierta anteriormente.
230	<Component class="javax.swing.JLabel" name="labelDescripcionMenu1">	Declara un componente o contenedor visual y su clase Java.
231	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
232	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
233		Metadato de NetBeans que conserva el estado editable del diseñador visual.
234	</Property>	Cierra la sección XML abierta anteriormente.
235	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
236	<Color blue="68" green="58" red="50" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
237	</Property>	Cierra la sección XML abierta anteriormente.
238	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
239	<Property name="text" type="java.lang.String" value="Accede al menú de alimentos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
240	</Properties>	Cierra la sección XML abierta anteriormente.
241	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
242	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

243	<AbsoluteConstraints x="29" y="340" width="491" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
244	</Constraint>	Cierra la sección XML abierta anteriormente.
245	</Constraints>	Cierra la sección XML abierta anteriormente.
246	</Component>	Cierra la sección XML abierta anteriormente.
247	<Component class="javax.swing.JLabel" name="labelDescripcionMenu2">	Declara un componente o contenedor visual y su clase Java.
248	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
249	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
250		Metadato de NetBeans que conserva el estado editable del diseñador visual.
251	</Property>	Cierra la sección XML abierta anteriormente.
252	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
253	<Color blue="68" green="58" red="50" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
254	</Property>	Cierra la sección XML abierta anteriormente.
255	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
256	<Property name="text" type="java.lang.String" value="y bebidas disponible."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
257	</Properties>	Cierra la sección XML abierta anteriormente.
258	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
259	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
260	<AbsoluteConstraints x="29" y="375" width="491" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
261	</Constraint>	Cierra la sección XML abierta anteriormente.
262	</Constraints>	Cierra la sección XML abierta anteriormente.
263	</Component>	Cierra la sección XML abierta anteriormente.
264	<Component class="Componentes.BotonDerretido" name="botonMenu">	Declara un componente o contenedor visual y su clase Java.
265	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
266	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
267	<Image iconType="3" name="/Imagenes/flecha_boton.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
268	</Property>	Cierra la sección XML abierta anteriormente.
269	<Property name="text" type="java.lang.String" value="Entrar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
270	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
271		Metadato de NetBeans que conserva el estado editable del diseñador visual.
272	</Property>	Cierra la sección XML abierta anteriormente.
273	<Property name="horizontalTextPosition" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
274	<Property name="iconTextGap" type="int" value="30"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
275	</Properties>	Cierra la sección XML abierta anteriormente.
276	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
277	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonMenuActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
278	</Events>	Cierra la sección XML abierta anteriormente.
279	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
280	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
281	<AbsoluteConstraints x="40" y="460" width="470" height="110"/>	Define la posición y el tamaño del componente dentro del diseño.
282	</Constraint>	Cierra la sección XML abierta anteriormente.
283	</Constraints>	Cierra la sección XML abierta anteriormente.

284	</Component>	Cierra la sección XML abierta anteriormente.
285	</SubComponents>	Cierra la sección XML abierta anteriormente.
286	</Container>	Cierra la sección XML abierta anteriormente.
287	<Container class="Componentes.PanelFlotante" name="tarjetaAdmin">	Declara un componente o contenedor visual y su clase Java.
288	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
289	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
290	<Color blue="a" green="0" red="c6" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
291	</Property>	Cierra la sección XML abierta anteriormente.
292	<Property name="desplazamientoSombraY" type="int" value="6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
293	<Property name="grosorBorde" type="float" value="1.5"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
294	<Property name="radio" type="int" value="25"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
295	</Properties>	Cierra la sección XML abierta anteriormente.
296	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
297	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
298	<AbsoluteConstraints x="991" y="375" width="549" height="594"/>	Define la posición y el tamaño del componente dentro del diseño.
299	</Constraint>	Cierra la sección XML abierta anteriormente.
300	</Constraints>	Cierra la sección XML abierta anteriormente.
301		Línea en blanco para organizar el XML del formulario.
302	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
303	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
304	</Layout>	Cierra la sección XML abierta anteriormente.
305	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
306	<Container class="Componentes.PanelCircular" name="circuloAdmin">	Declara un componente o contenedor visual y su clase Java.
307	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
308	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
309	<Color blue="e9" green="e8" red="fe" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
310	</Property>	Cierra la sección XML abierta anteriormente.
311	</Properties>	Cierra la sección XML abierta anteriormente.
312	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
313	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
314	<AbsoluteConstraints x="172" y="35" width="206" height="206"/>	Define la posición y el tamaño del componente dentro del diseño.
315	</Constraint>	Cierra la sección XML abierta anteriormente.
316	</Constraints>	Cierra la sección XML abierta anteriormente.
317		Línea en blanco para organizar el XML del formulario.
318	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
319	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
320	</Layout>	Cierra la sección XML abierta anteriormente.
321	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
322	<Component class="Labels.LabelEscalable" name="labelIconoAdmin">	Declara un componente o contenedor visual y su clase Java.
323	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
324	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

325	<Image iconType="3" name="/Imagenes/IconoAdmin.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
326	</Property>	Cierra la sección XML abierta anteriormente.
327	<Property name="mantenerProporcion" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
328	</Properties>	Cierra la sección XML abierta anteriormente.
329	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
330	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
331	<AbsoluteConstraints x="20" y="30" width="170" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
332	</Constraint>	Cierra la sección XML abierta anteriormente.
333	</Constraints>	Cierra la sección XML abierta anteriormente.
334	</Component>	Cierra la sección XML abierta anteriormente.
335	</SubComponents>	Cierra la sección XML abierta anteriormente.
336	</Container>	Cierra la sección XML abierta anteriormente.
337	<Component class="javax.swing.JLabel" name="labelTituloAdmin">	Declara un componente o contenedor visual y su clase Java.
338	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
339	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
340		Metadato de NetBeans que conserva el estado editable del diseñador visual.
341	</Property>	Cierra la sección XML abierta anteriormente.
342	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
343	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
344	</Property>	Cierra la sección XML abierta anteriormente.
345	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
346	<Property name="text" type="java.lang.String" value="Menú Admin"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
347	</Properties>	Cierra la sección XML abierta anteriormente.
348	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
349	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
350	<AbsoluteConstraints x="34" y="268" width="480" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
351	</Constraint>	Cierra la sección XML abierta anteriormente.
352	</Constraints>	Cierra la sección XML abierta anteriormente.
353	</Component>	Cierra la sección XML abierta anteriormente.
354	<Component class="javax.swing.JLabel" name="labelDescripcionAdmin1">	Declara un componente o contenedor visual y su clase Java.
355	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
356	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
357		Metadato de NetBeans que conserva el estado editable del diseñador visual.
358	</Property>	Cierra la sección XML abierta anteriormente.
359	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
360	<Color blue="68" green="58" red="50" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
361	</Property>	Cierra la sección XML abierta anteriormente.
362	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
363	<Property name="text" type="java.lang.String" value="Gestiona productos, inventario,"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
364	</Properties>	Cierra la sección XML abierta anteriormente.
365	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

366	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
367	<AbsoluteConstraints x="29" y="340" width="491" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
368	</Constraint>	Cierra la sección XML abierta anteriormente.
369	</Constraints>	Cierra la sección XML abierta anteriormente.
370	</Component>	Cierra la sección XML abierta anteriormente.
371	<Component class="javax.swing.JLabel" name="labelDescripcionAdmin2">	Declara un componente o contenedor visual y su clase Java.
372	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
373	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
374		Metadato de NetBeans que conserva el estado editable del diseñador visual.
375	</Property>	Cierra la sección XML abierta anteriormente.
376	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
377	<Color blue="68" green="58" red="50" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
378	</Property>	Cierra la sección XML abierta anteriormente.
379	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
380	<Property name="text" type="java.lang.String" value="usuarios y reportes."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
381	</Properties>	Cierra la sección XML abierta anteriormente.
382	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
383	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
384	<AbsoluteConstraints x="29" y="375" width="491" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
385	</Constraint>	Cierra la sección XML abierta anteriormente.
386	</Constraints>	Cierra la sección XML abierta anteriormente.
387	</Component>	Cierra la sección XML abierta anteriormente.
388	<Component class="Componentes.BotonDerretido" name="botonAdmin">	Declara un componente o contenedor visual y su clase Java.
389	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
390	<Property name="colorBoton" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
391	<Color blue="9" green="0" red="c8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
392	</Property>	Cierra la sección XML abierta anteriormente.
393	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
394	<Image iconType="3" name="/Imagenes/flecha_boton.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
395	</Property>	Cierra la sección XML abierta anteriormente.
396	<Property name="text" type="java.lang.String" value="Entrar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
397	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
398		Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	</Property>	Cierra la sección XML abierta anteriormente.
400	<Property name="horizontalTextPosition" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
401	<Property name="iconTextGap" type="int" value="30"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
402	</Properties>	Cierra la sección XML abierta anteriormente.
403	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
404	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonAdminActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
405	</Events>	Cierra la sección XML abierta anteriormente.

406	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
407	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
408	<AbsoluteConstraints x="40" y="460" width="470" height="110"/>	Define la posición y el tamaño del componente dentro del diseño.
409	</Constraint>	Cierra la sección XML abierta anteriormente.
410	</Constraints>	Cierra la sección XML abierta anteriormente.
411	</Component>	Cierra la sección XML abierta anteriormente.
412	</SubComponents>	Cierra la sección XML abierta anteriormente.
413	</Container>	Cierra la sección XML abierta anteriormente.
414	</SubComponents>	Cierra la sección XML abierta anteriormente.
415	</Container>	Cierra la sección XML abierta anteriormente.
416	</SubComponents>	Cierra la sección XML abierta anteriormente.
417	</Form>	Cierra la sección XML abierta anteriormente.

16.71 src/GUI_ADMINISTRADOR/InicioAdminForm.java

Ventana contenedora del área administrativa y su barra de título.

Tipo: java | Líneas: 314 | Archivo 71 de 157

Clases/tipos detectados: InicioAdminForm

Línea	Constructor o método detectado
23	public InicioAdminForm() {
39	private void aplicarTipografia() {
55	public void setAccionMenuAdmin(Runnable accionMenuAdmin) {
61	private void procesarOpcionUsuario(ActionEvent evento) {
71	private void abrirMenu() {
78	private void abrirMenuAdmin() {
85	private void cerrarSesion() {
97	private void initComponents() {
155	public void actionPerformed(java.awt.event.ActionEvent evt) {
210	public void actionPerformed(java.awt.event.ActionEvent evt) {
258	public void actionPerformed(java.awt.event.ActionEvent evt) {
273	private void botonAdminActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonAdminActionPerformed
277	private void botonMenuActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonMenuActionPerformed
281	private void botonUsuarioActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonUsuarioActionPerformed
285	public static void main(String[] args) {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* JFrame Form editable desde el diseñador visual de NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
5		Línea en blanco: separa visualmente bloques para que el código sea más legible.
6	import GUI_CAJERO.Cajero;	Importa GUI_CAJERO.Cajero para poder usar esa clase sin escribir su nombre completo.
7	import Login.Login;	Importa Login.Login para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.EventQueue;	Importa java.awt.EventQueue para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.event.ActionEvent;	Importa java.awt.event.ActionEvent para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
12	import Componentes.BarraTituloForm;	Importa Componentes.BarraTituloForm para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	* Pantalla administrativa construida como JFrame Form de NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

17	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
18	public class InicioAdminForm extends javax.swing.JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	private final TemaAdmin tema;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
21	private Runnable accionMenuAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	public InicioAdminForm() {	Constructor de InicioAdminForm: prepara un objeto nuevo y deja sus componentes listos para usarse.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
26	tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
27	accionMenuAdmin = this::abrirMenuAdmin;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
28	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	BarraTituloForm.instalar(this, "Inicio de Administrador");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	aplicarTipografia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	botonUsuario.addMenuOpcionListener(this::procesarOpcionUsuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	}	Cierra el bloque abierto anteriormente.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	private void aplicarTipografia() {	Declara el método aplicarTipografia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
40	labelMarca.setFont(tema.negrita(34f));	Configura una propiedad mediante setFont en el objeto indicado.
41	labelRol.setFont(tema.regular(23f));	Configura una propiedad mediante setFont en el objeto indicado.
42	botonUsuario.setFont(tema.media(17f));	Configura una propiedad mediante setFont en el objeto indicado.
43	labelTitulo.setFont(tema.negrita(52f));	Configura una propiedad mediante setFont en el objeto indicado.
44	labelSubtitulo.setFont(tema.regular(27f));	Configura una propiedad mediante setFont en el objeto indicado.
45	labelTituloMenu.setFont(tema.negrita(36f));	Configura una propiedad mediante setFont en el objeto indicado.
46	labelTituloAdmin.setFont(tema.negrita(36f));	Configura una propiedad mediante setFont en el objeto indicado.
47	labelDescripcionMenu1.setFont(tema.regular(22f));	Configura una propiedad mediante setFont en el objeto indicado.
48	labelDescripcionMenu2.setFont(tema.regular(22f));	Configura una propiedad mediante setFont en el objeto indicado.
49	labelDescripcionAdmin1.setFont(tema.regular(22f));	Configura una propiedad mediante setFont en el objeto indicado.
50	labelDescripcionAdmin2.setFont(tema.regular(22f));	Configura una propiedad mediante setFont en el objeto indicado.
51	botonMenu.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
52	botonAdmin.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
53	}	Cierra el bloque abierto anteriormente.
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.
55	public void setAccionMenuAdmin(Runnable accionMenuAdmin) {	Declara el método setAccionMenuAdmin. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
56	this.accionMenuAdmin = accionMenuAdmin != null	Accede al atributo del objeto actual para asignarlo o utilizarlo.
57	? accionMenuAdmin	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
58	: this::abrirMenuAdmin;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	}	Cierra el bloque abierto anteriormente.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.

61	private void procesarOpcionUsuario(ActionEvent evento) {	Declara el método procesarOpcionUsuario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
62	switch (botonUsuario.getIndiceOpcionSeleccionada()) {	Selecciona una ruta entre varias según el valor evaluado.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	case 0 ->	Define una de las rutas posibles dentro del switch.
65	cerrarSesion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	default -> {	Define una de las rutas posibles dentro del switch.
67	}	Cierra el bloque abierto anteriormente.
68	}	Cierra el bloque abierto anteriormente.
69	}	Cierra el bloque abierto anteriormente.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	private void abrirMenu() {	Declara el método abrirMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
72	Cajero menu = new Cajero();	Crea una nueva instancia de Cajero y la usa o guarda para continuar el proceso.
73	menu.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
74	menu.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
75	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76	}	Cierra el bloque abierto anteriormente.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	private void abrirMenuAdmin() {	Declara el método abrirMenuAdmin. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
79	MenuAdmin menuAdmin = new MenuAdmin();	Crea una nueva instancia de MenuAdmin y la usa o guarda para continuar el proceso.
80	menuAdmin.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
81	menuAdmin.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
82	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	private void cerrarSesion() {	Declara el método cerrarSesion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
86	Login login = new Login();	Crea una nueva instancia de Login y la usa o guarda para continuar el proceso.
87	login.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
88	login.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
89	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	}	Cierra el bloque abierto anteriormente.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
93	* Este bloque lo administra el diseñador visual de NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
94	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
95	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
96	// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
97	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	panelRaiz = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
100	panelCabecera = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
101	labelLogo = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
102	labelMarca = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
103	labelRol = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.

104	labelIconoUsuario = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
105	botonUsuario = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
106	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
107	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
108	tarjetaMenu = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
109	circuloMenu = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
110	iconoMenu = new GUI_ADMINISTRADOR.IconoMenuAdmin();	Crea una nueva instancia de GUI_ADMINISTRADOR.IconoMenuAdmin y la usa o guarda para continuar el proceso.
111	labelTituloMenu = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
112	labelDescripcionMenu1 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
113	labelDescripcionMenu2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
114	botonMenu = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
115	tarjetaAdmin = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
116	circuloAdmin = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
117	labelIconoAdmin = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
118	labelTituloAdmin = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
119	labelDescripcionAdmin1 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
120	labelDescripcionAdmin2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
121	botonAdmin = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	setTitle("Inicio de Administrador");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	setMinimumSize(new java.awt.Dimension(1024, 650));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
126	getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	panelRaiz.setBackground(new java.awt.Color(252, 252, 253));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
129	panelRaiz.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	panelCabecera.setBackground(new java.awt.Color(1, 15, 30));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
132	panelCabecera.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	labelLogo.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/Logow.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
135	labelLogo.setMantenerProporcion(true);	Configura una propiedad mediante setMantenerProporcion en el objeto indicado.
136	panelCabecera.add(labelLogo, new org.netbeans.lib.awtextra.AbsoluteConstraints(65, 25, 92, 82));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	labelMarca.setFont(new java.awt.Font("Dialog", 1, 39)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
139	labelMarca.setForeground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
140	labelMarca.setText("Waldonald's");	Cambia el texto visible de un componente de la interfaz.
141	panelCabecera.add(labelMarca, new org.netbeans.lib.awtextra.AbsoluteConstraints(197, 28, 360, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	labelRol.setFont(new java.awt.Font("Dialog", 0, 26)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
144	labelRol.setForeground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
145	labelRol.setText("Administrador");	Cambia el texto visible de un componente de la interfaz.
146	panelCabecera.add(labelRol, new org.netbeans.lib.awtextra.AbsoluteConstraints(198, 76, 300, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.

148	labelIconoUsuario.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/IconoUsuario.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
149	labelIconoUsuario.setMantenerProporcion(true);	Configura una propiedad mediante setMantenerProporcion en el objeto indicado.
150	panelCabecera.add(labelIconoUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(1618, 34, 54, 54));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	botonUsuario.setTextoDesplegable("Cerrar sesión");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
153	botonUsuario.setFont(new java.awt.Font("Dialog", 1, 19)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
154	botonUsuario.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
155	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
156	botonUsuarioActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
157	}	Cierra el bloque abierto anteriormente.
158	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	panelCabecera.add(botonUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(1680, 30, 215, 62));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
160		Línea en blanco: separa visualmente bloques para que el código sea más legible.
161	panelRaiz.add(panelCabecera, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1920, 135));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	labelTitulo.setFont(new java.awt.Font("Dialog", 1, 59)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
164	labelTitulo.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
165	labelTitulo.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
166	labelTitulo.setText("Inicio de Administrador");	Cambia el texto visible de un componente de la interfaz.
167	panelRaiz.add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 190, 1920, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	labelSubtitulo.setFont(new java.awt.Font("Dialog", 0, 31)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
170	labelSubtitulo.setForeground(new java.awt.Color(80, 88, 104));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
171	labelSubtitulo.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
172	labelSubtitulo.setText("Selecciona una opción para continuar");	Cambia el texto visible de un componente de la interfaz.
173	panelRaiz.add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 276, 1920, 44));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
175	tarjetaMenu.setColorBorde(new java.awt.Color(255, 188, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
176	tarjetaMenu.setDesplazamientoSombraY(6);	Configura una propiedad mediante setDesplazamientoSombraY en el objeto indicado.
177	tarjetaMenu.setGrosorBorde(1.5F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
178	tarjetaMenu.setRadio(25);	Configura una propiedad mediante setRadio en el objeto indicado.
179	tarjetaMenu.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	circuloMenu.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
182	circuloMenu.add(iconoMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 206, 206));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	tarjetaMenu.add(circuloMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(172, 35, 206, 206));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
185		Línea en blanco: separa visualmente bloques para que el código sea más legible.
186	labelTituloMenu.setFont(new java.awt.Font("Dialog", 1, 41)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
187	labelTituloMenu.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
188	labelTituloMenu.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
189	labelTituloMenu.setText("Ver menú");	Cambia el texto visible de un componente de la interfaz.
190	tarjetaMenu.add(labelTituloMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(34, 268, 480, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	labelDescripcionMenu1.setFont(new java.awt.Font("Dialog", 0, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
193	labelDescripcionMenu1.setForeground(new java.awt.Color(80, 88, 104));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
194	labelDescripcionMenu1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
195	labelDescripcionMenu1.setText("Accede al menú de alimentos");	Cambia el texto visible de un componente de la interfaz.
196	tarjetaMenu.add(labelDescripcionMenu1, new org.netbeans.lib.awtextra.AbsoluteConstraints(29, 340, 491, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
198	labelDescripcionMenu2.setFont(new java.awt.Font("Dialog", 0, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
199	labelDescripcionMenu2.setForeground(new java.awt.Color(80, 88, 104));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
200	labelDescripcionMenu2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
201	labelDescripcionMenu2.setText("y bebidas disponible.");	Cambia el texto visible de un componente de la interfaz.
202	tarjetaMenu.add(labelDescripcionMenu2, new org.netbeans.lib.awtextra.AbsoluteConstraints(29, 375, 491, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	botonMenu.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/flecha_boton.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
205	botonMenu.setText("Entrar");	Cambia el texto visible de un componente de la interfaz.
206	botonMenu.setFont(new java.awt.Font("Arial", 1, 30)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
207	botonMenu.setHorizontalTextPosition(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalTextPosition en el objeto indicado.
208	botonMenu.setIconTextGap(30);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
209	botonMenu.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
210	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
211	botonMenuActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
212	}	Cierra el bloque abierto anteriormente.
213	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
214	tarjetaMenu.add(botonMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 460, 470, 110));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
216	panelRaiz.add(tarjetaMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 375, 549, 594));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	tarjetaAdmin.setColorBorde(new java.awt.Color(198, 0, 10));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
219	tarjetaAdmin.setDesplazamientoSombraY(6);	Configura una propiedad mediante setDesplazamientoSombraY en el objeto indicado.
220	tarjetaAdmin.setGrosorBorde(1.5F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
221	tarjetaAdmin.setRadio(25);	Configura una propiedad mediante setRadio en el objeto indicado.
222	tarjetaAdmin.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	circuloAdmin.setColorFondo(new java.awt.Color(254, 232, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
225	circuloAdmin.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
226		Línea en blanco: separa visualmente bloques para que el código sea más legible.
227	labelIconoAdmin.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/IconoAdmin.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
228	labelIconoAdmin.setMantenerProporcion(true);	Configura una propiedad mediante setMantenerProporcion en el objeto indicado.
229	circuloAdmin.add(labelIconoAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 30, 170, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	tarjetaAdmin.add(circuloAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(172, 35, 206, 206));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.

233	labelTituloAdmin.setFont(new java.awt.Font("Dialog", 1, 41)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
234	labelTituloAdmin.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
235	labelTituloAdmin.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
236	labelTituloAdmin.setText("Menú Admin");	Cambia el texto visible de un componente de la interfaz.
237	tarjetaAdmin.add(labelTituloAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(34, 268, 480, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	labelDescripcionAdmin1.setFont(new java.awt.Font("Dialog", 0, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
240	labelDescripcionAdmin1.setForeground(new java.awt.Color(80, 88, 104));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
241	labelDescripcionAdmin1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
242	labelDescripcionAdmin1.setText("Gestiona productos, inventario,");	Cambia el texto visible de un componente de la interfaz.
243	tarjetaAdmin.add(labelDescripcionAdmin1, new org.netbeans.lib.awtextra.AbsoluteConstraints(29, 340, 491, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	labelDescripcionAdmin2.setFont(new java.awt.Font("Dialog", 0, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
246	labelDescripcionAdmin2.setForeground(new java.awt.Color(80, 88, 104));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
247	labelDescripcionAdmin2.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
248	labelDescripcionAdmin2.setText("usuarios y reportes.");	Cambia el texto visible de un componente de la interfaz.
249	tarjetaAdmin.add(labelDescripcionAdmin2, new org.netbeans.lib.awtextra.AbsoluteConstraints(29, 375, 491, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	botonAdmin.setColorBoton(new java.awt.Color(200, 0, 9));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
252	botonAdmin.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/flecha_boton.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
253	botonAdmin.setText("Entrar");	Cambia el texto visible de un componente de la interfaz.
254	botonAdmin.setFont(new java.awt.Font("Arial", 1, 30)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
255	botonAdmin.setHorizontalTextPosition(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalTextPosition en el objeto indicado.
256	botonAdmin.setIconTextGap(30);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
257	botonAdmin.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
258	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
259	botonAdminActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
260	}	Cierra el bloque abierto anteriormente.
261	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
262	tarjetaAdmin.add(botonAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 460, 470, 110));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
263		Línea en blanco: separa visualmente bloques para que el código sea más legible.
264	panelRaiz.add(tarjetaAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(991, 375, 549, 594));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	getContentPane().add(panelRaiz, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1920, 1080));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
268	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
270	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	private void botonAdminActionPerformed(java.awt.event.ActionEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	accionMenuAdmin.run();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

275	}//GEN-LAST:event_botonAdminActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	private void botonMenuActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_botonMenuActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
278	abrirMenu();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
279	}//GEN-LAST:event_botonMenuActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281	private void botonUsuarioActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_botonUsuarioActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
283	}//GEN-LAST:event_botonUsuarioActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
284		Línea en blanco: separa visualmente bloques para que el código sea más legible.
285	public static void main(String[] args) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
286	EventQueue.invokeLater(() -> new InicioAdminForm().setVisible(true));	Crea una nueva instancia de InicioAdminForm y la usa o guarda para continuar el proceso.
287	}	Cierra el bloque abierto anteriormente.
288		Línea en blanco: separa visualmente bloques para que el código sea más legible.
289	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
290	private Componentes.BotonDerretido botonAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
291	private Componentes.BotonDerretido botonMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
292	private Componentes.BotonDesplegable botonUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
293	private Componentes.PanelCircular circuloAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
294	private Componentes.PanelCircular circuloMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
295	private GUI_ADMINISTRADOR.IconoMenuAdmin iconoMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
296	private javax.swing.JLabel labelDescripcionAdmin1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
297	private javax.swing.JLabel labelDescripcionAdmin2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
298	private javax.swing.JLabel labelDescripcionMenu1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
299	private javax.swing.JLabel labelDescripcionMenu2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
300	private Labels.LabelEscalable labelIconoAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
301	private Labels.LabelEscalable labelIconoUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
302	private Labels.LabelEscalable labelLogo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
303	private javax.swing.JLabel labelMarca;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
304	private javax.swing.JLabel labelRol;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
305	private javax.swing.JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
306	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
307	private javax.swing.JLabel labelTituloAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
308	private javax.swing.JLabel labelTituloMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
309	private javax.swing.JPanel panelCabecera;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
310	private javax.swing.JPanel panelRaiz;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
311	private Componentes.PanelFlotante tarjetaAdmin;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
312	private Componentes.PanelFlotante tarjetaMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
313	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
314	}	Cierra el bloque abierto anteriormente.

16.72 src/GUI_ADMINISTRADOR/InventarioPanel.form

Metadatos visuales de NetBeans para diseñar InventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 87 | Archivo 72 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.9" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<Dimension value="[1530, 932]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</Property>	Cierra la sección XML abierta anteriormente.
11	</Properties>	Cierra la sección XML abierta anteriormente.
12	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,16,0,0,5,-44"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	</AuxValues>	Cierra la sección XML abierta anteriormente.
24		Línea en blanco para organizar el XML del formulario.
25	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
26	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
27	</Layout>	Cierra la sección XML abierta anteriormente.
28	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<Container class="javax.swing.JLayeredPane" name="panelCapas">	Declara un componente o contenedor visual y su clase Java.
30	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
31	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
32	<AbsoluteConstraints x="0" y="0" width="1530" height="932"/>	Define la posición y el tamaño del componente dentro del diseño.
33	</Constraint>	Cierra la sección XML abierta anteriormente.
34	</Constraints>	Cierra la sección XML abierta anteriormente.
35		Línea en blanco para organizar el XML del formulario.
36	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.

37	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
38	</Layout>	Cierra la sección XML abierta anteriormente.
39	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
40	<Container class="javax.swing.JPanel" name="panelVistas">	Declara un componente o contenedor visual y su clase Java.
41	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
42	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
43	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
44	</Property>	Cierra la sección XML abierta anteriormente.
45	</Properties>	Cierra la sección XML abierta anteriormente.
46	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
47	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
48	<AbsoluteConstraints x="0" y="0" width="1530" height="932"/>	Define la posición y el tamaño del componente dentro del diseño.
49	</Constraint>	Cierra la sección XML abierta anteriormente.
50	</Constraints>	Cierra la sección XML abierta anteriormente.
51		Línea en blanco para organizar el XML del formulario.
52	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignCardLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
53	</Container>	Cierra la sección XML abierta anteriormente.
54	<Component class="Componentes.BotonRedondeado" name="botonExistencias">	Declara un componente o contenedor visual y su clase Java.
55	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
56	<Property name="text" type="java.lang.String" value="Existencias"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
57	</Properties>	Cierra la sección XML abierta anteriormente.
58	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
59	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
60	<AbsoluteConstraints x="60" y="120" width="190" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
61	</Constraint>	Cierra la sección XML abierta anteriormente.
62	</Constraints>	Cierra la sección XML abierta anteriormente.
63	</Component>	Cierra la sección XML abierta anteriormente.
64	<Component class="Componentes.BotonRedondeado" name="botonIngredientes">	Declara un componente o contenedor visual y su clase Java.
65	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
66	<Property name="text" type="java.lang.String" value="Ingredientes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
67	</Properties>	Cierra la sección XML abierta anteriormente.
68	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
69	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
70	<AbsoluteConstraints x="265" y="120" width="190" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
71	</Constraint>	Cierra la sección XML abierta anteriormente.
72	</Constraints>	Cierra la sección XML abierta anteriormente.
73	</Component>	Cierra la sección XML abierta anteriormente.
74	<Component class="Componentes.BotonRedondeado" name="botonMovimientos">	Declara un componente o contenedor visual y su clase Java.
75	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
76	<Property name="text" type="java.lang.String" value="Movimientos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
77	</Properties>	Cierra la sección XML abierta anteriormente.
78	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

79	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
80	<AbsoluteConstraints x="470" y="120" width="190" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
81	</Constraint>	Cierra la sección XML abierta anteriormente.
82	</Constraints>	Cierra la sección XML abierta anteriormente.
83	</Component>	Cierra la sección XML abierta anteriormente.
84	</SubComponents>	Cierra la sección XML abierta anteriormente.
85	</Container>	Cierra la sección XML abierta anteriormente.
86	</SubComponents>	Cierra la sección XML abierta anteriormente.
87	</Form>	Cierra la sección XML abierta anteriormente.

16.73 src/GUI_ADMINISTRADOR/InventarioPanel.java

Vista de existencias, búsqueda, filtros, paginación, entradas/salidas y exportación.

Tipo: java | Líneas: 133 | Archivo 73 de 157

Clases/tipos detectados: InventarioPanel

Línea	Constructor o método detectado
29	public InventarioPanel() {
36	private void configurarVistas() {
50	private void traerPestanasAlFrente() {
61	private void configurarBotones() {
75	private void mostrar(String vista, BotonRedondeado seleccionado) {
87	private void estilizar(BotonRedondeado boton, boolean seleccionado) {
96	private void initComponents() {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
4	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.CardLayout;	Importa java.awt.CardLayout para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	/** Módulo único de inventario con vistas internas. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
10	public class InventarioPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	private static final String EXISTENCIAS = "existencias";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13	private static final String INGREDIENTES = "ingredientes";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
14	private static final String MOVIMIENTOS = "movimientos";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
16	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
17	private static final Color BORDE = new Color(225, 230, 237);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
20	private CardLayout navegador;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
21	private BotonRedondeado botonActivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	private final GestionInventarioPanel existencias =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
23	new GestionInventarioPanel();	Crea una nueva instancia de GestionInventarioPanel y la usa o guarda para continuar el proceso.
24	private final CatalogoIngredientesPanel ingredientes =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
25	new CatalogoIngredientesPanel();	Crea una nueva instancia de CatalogoIngredientesPanel y la usa o guarda para continuar el proceso.
26	private final MovimientosInventarioPanel movimientos =	Declara una constante o referencia que no puede reasignarse después de inicializarse.

27	new MovimientosInventarioPanel();	Crea una nueva instancia de MovimientosInventarioPanel y la usa o guarda para continuar el proceso.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	public InventarioPanel() {	Constructor de InventarioPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
30	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	configurarVistas();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	configurarBotones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	mostrar(EXISTENCIAS, botonExistencias);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	}	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	private void configurarVistas() {	Declara el método configurarVistas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
37	navegador = new CardLayout();	Crea una nueva instancia de CardLayout y la usa o guarda para continuar el proceso.
38	panelVistas.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	panelVistas.setLayout(navegador);	Configura una propiedad mediante setLayout en el objeto indicado.
40	panelVistas.add(existencias, EXISTENCIAS);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	panelVistas.add(ingredientes, INGREDIENTES);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	panelVistas.add(movimientos, MOVIMIENTOS);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	panelCapas.setLayer(panelVistas, javax.swing.JLayeredPane.DEFAULT_LAYER);	Configura una propiedad mediante setLayer en el objeto indicado.
44	panelCapas.setLayer(botonExistencias, javax.swing.JLayeredPane.PALETTE_LAYER);	Configura una propiedad mediante setLayer en el objeto indicado.
45	panelCapas.setLayer(botonIngredientes, javax.swing.JLayeredPane.PALETTE_LAYER);	Configura una propiedad mediante setLayer en el objeto indicado.
46	panelCapas.setLayer(botonMovimientos, javax.swing.JLayeredPane.PALETTE_LAYER);	Configura una propiedad mediante setLayer en el objeto indicado.
47	traerPestanasAlFrente();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	}	Cierra el bloque abierto anteriormente.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	private void traerPestanasAlFrente() {	Declara el método traerPestanasAlFrente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
51	// Las vistas viven en la capa base y la navegación en una capa	Comentario para humanos; Java no ejecuta este texto. Las vistas viven en la capa base y la navegación en una capa
52	// superior, por lo que cambiar de CardLayout no puede cubrirla.	Comentario para humanos; Java no ejecuta este texto. superior, por lo que cambiar de CardLayout no puede cubrirla.
53	panelCapas.moveToFront(botonExistencias);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	panelCapas.moveToFront(botonIngredientes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	panelCapas.moveToFront(botonMovimientos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	botonExistencias.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
57	botonIngredientes.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
58	botonMovimientos.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
59	}	Cierra el bloque abierto anteriormente.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	private void configurarBotones() {	Declara el método configurarBotones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
62	for (BotonRedondeado boton : new BotonRedondeado[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
63	botonExistencias, botonIngredientes, botonMovimientos}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	boton.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
65	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
66	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
67	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
68	estilizar(boton, false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	}	Cierra el bloque abierto anteriormente.

70	botonExistencias.addActionListener(e -> mostrar(EXISTENCIAS, botonExistencias));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
71	botonIngredientes.addActionListener(e -> mostrar(INGREDIENTES, botonIngredientes));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
72	botonMovimientos.addActionListener(e -> mostrar(MOVIMIENTOS, botonMovimientos));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
73	}	Cierra el bloque abierto anteriormente.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	private void mostrar(String vista, BotonRedondeado seleccionado) {	Declara el método mostrar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
76	if (botonActivo != null) estilizar(botonActivo, false);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
77	estilizar(seleccionado, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	botonActivo = seleccionado;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
79	if (INGREDIENTES.equals(vista)) ingredientes.recargarDatos();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
80	if (MOVIMIENTOS.equals(vista)) movimientos.recargarDatos();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
81	navegador.show(panelVistas, vista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	traerPestanasAlFrente();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	panelCapas.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	panelCapas.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	}	Cierra el bloque abierto anteriormente.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	private void estilizar(BotonRedondeado boton, boolean seleccionado) {	Declara el método estilizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
88	boton.setColorInicio(seleccionado ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
89	boton.setColorFinal(seleccionado ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
90	boton.setColorBorde(seleccionado ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
91	boton.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
95	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
96	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	panelCapas = new javax.swing.JLayeredPane();	Crea una nueva instancia de javax.swing.JLayeredPane y la usa o guarda para continuar el proceso.
99	panelVistas = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
100	botonExistencias = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
101	botonIngredientes = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
102	botonMovimientos = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
105	setPreferredSize(new java.awt.Dimension(1530, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
106	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	panelCapas.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
109		Línea en blanco: separa visualmente bloques para que el código sea más legible.
110	panelVistas.setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
111	panelVistas.setLayout(new java.awt.CardLayout());	Crea una nueva instancia de java.awt.CardLayout y la usa o guarda para continuar el proceso.
112	panelCapas.add(panelVistas, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1530, 932));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.

114	<code>botonExistencias.setText("Existencias");</code>	Cambia el texto visible de un componente de la interfaz.
115	<code>panelCapas.add(botonExistencias, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 120, 190, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	<code>botonIngredientes.setText("Ingredientes");</code>	Cambia el texto visible de un componente de la interfaz.
118	<code>panelCapas.add(botonIngredientes, new org.netbeans.lib.awtextra.AbsoluteConstraints(265, 120, 190, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	<code>botonMovimientos.setText("Movimientos");</code>	Cambia el texto visible de un componente de la interfaz.
121	<code>panelCapas.add(botonMovimientos, new org.netbeans.lib.awtextra.AbsoluteConstraints(470, 120, 190, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	<code>add(panelCapas, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1530, 932));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
124	<code>}// </editor-fold>//GEN-END:initComponents</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	<code>// Variables declaration - do not modify//GEN-BEGIN:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
127	<code>private Componentes.BotonRedondeado botonExistencias;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
128	<code>private Componentes.BotonRedondeado botonIngredientes;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
129	<code>private Componentes.BotonRedondeado botonMovimientos;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
130	<code>private javax.swing.JLayeredPane panelCapas;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	<code>private javax.swing.JPanel panelVistas;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
132	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
133	<code>}</code>	Cierra el bloque abierto anteriormente.

16.74 src/GUI_ADMINISTRADOR/MenuAdmin.form

Metadatos visuales de NetBeans para diseñar MenuAdmin. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 338 | Archivo 74 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.5" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JFrameFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="defaultCloseOperation" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Property name="title" type="java.lang.String" value="Panel de Administración"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
7	<Property name="minimumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
8	<Dimension value="[1024, 650]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	</Property>	Cierra la sección XML abierta anteriormente.
10	<Property name="size" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
11	<Dimension value="[1920, 1080]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	</Property>	Cierra la sección XML abierta anteriormente.
13	</Properties>	Cierra la sección XML abierta anteriormente.
14	<SyntheticProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<SyntheticProperty name="formSizePolicy" type="int" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<SyntheticProperty name="generateCenter" type="boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	</SyntheticProperties>	Cierra la sección XML abierta anteriormente.
18	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
24	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
27	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84, -19, 0, 5, 115, 114, 0, 18, 106, 97, 118, 97, 46, 97, 119, 116, 46, 68, 105, 109, 101, 110, 115, 105, 111, 110, 65, -114, -39, -41, -84, 95, 68, 20, 2, 0, 2, 73, 0, 6, 104, 101, 105, 103, 104, 116, 73, 0, 5, 119, 105, 100, 116, 104, 120, 112, 0, 0, 4, 80, 0, 0, 7, -124"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	</AuxValues>	Cierra la sección XML abierta anteriormente.
30		Línea en blanco para organizar el XML del formulario.
31	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
32	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
33	</Layout>	Cierra la sección XML abierta anteriormente.
34	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
35	<Container class="javax.swing.JPanel" name="panelRaiz">	Declara un componente o contenedor visual y su clase Java.
36	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
37	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

38	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
39	</Property>	Cierra la sección XML abierta anteriormente.
40	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
41	<Dimension value="[1920, 1032]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
42	</Property>	Cierra la sección XML abierta anteriormente.
43	</Properties>	Cierra la sección XML abierta anteriormente.
44	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
45	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
46	<AbsoluteConstraints x="0" y="0" width="1920" height="1032"/>	Define la posición y el tamaño del componente dentro del diseño.
47	</Constraint>	Cierra la sección XML abierta anteriormente.
48	</Constraints>	Cierra la sección XML abierta anteriormente.
49		Línea en blanco para organizar el XML del formulario.
50	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
51	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	</Layout>	Cierra la sección XML abierta anteriormente.
53	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
54	<Container class="javax.swing.JPanel" name="panelLateral">	Declara un componente o contenedor visual y su clase Java.
55	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
56	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
57	<Color blue="1e" green="f" red="1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
58	</Property>	Cierra la sección XML abierta anteriormente.
59	</Properties>	Cierra la sección XML abierta anteriormente.
60	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
61	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
62	<AbsoluteConstraints x="0" y="0" width="340" height="1032"/>	Define la posición y el tamaño del componente dentro del diseño.
63	</Constraint>	Cierra la sección XML abierta anteriormente.
64	</Constraints>	Cierra la sección XML abierta anteriormente.
65		Línea en blanco para organizar el XML del formulario.
66	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
67	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
68	</Layout>	Cierra la sección XML abierta anteriormente.
69	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
70	<Component class="Labels.LabelEscalable" name="labelLogo">	Declara un componente o contenedor visual y su clase Java.
71	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
72	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
73	<Image iconType="3" name="/Imagenes/LogoW.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
74	</Property>	Cierra la sección XML abierta anteriormente.
75	<Property name="mantenerProporcion" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
76	</Properties>	Cierra la sección XML abierta anteriormente.
77	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

78	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
79	<AbsoluteConstraints x="27" y="25" width="68" height="68"/>	Define la posición y el tamaño del componente dentro del diseño.
80	</Constraint>	Cierra la sección XML abierta anteriormente.
81	</Constraints>	Cierra la sección XML abierta anteriormente.
82	</Component>	Cierra la sección XML abierta anteriormente.
83	<Component class="javax.swing.JLabel" name="labelMarca">	Declara un componente o contenedor visual y su clase Java.
84	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
85	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86		Metadato de NetBeans que conserva el estado editable del diseñador visual.
87	</Property>	Cierra la sección XML abierta anteriormente.
88	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
89	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
90	</Property>	Cierra la sección XML abierta anteriormente.
91	<Property name="text" type="java.lang.String" value="Waldonald's"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
92	</Properties>	Cierra la sección XML abierta anteriormente.
93	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
94	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
95	<AbsoluteConstraints x="110" y="25" width="205" height="39"/>	Define la posición y el tamaño del componente dentro del diseño.
96	</Constraint>	Cierra la sección XML abierta anteriormente.
97	</Constraints>	Cierra la sección XML abierta anteriormente.
98	</Component>	Cierra la sección XML abierta anteriormente.
99	<Component class="javax.swing.JLabel" name="labelRol">	Declara un componente o contenedor visual y su clase Java.
100	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
101	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102		Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
105	<Color blue="0" green="be" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
106	</Property>	Cierra la sección XML abierta anteriormente.
107	<Property name="text" type="java.lang.String" value="Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
108	</Properties>	Cierra la sección XML abierta anteriormente.
109	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
110	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
111	<AbsoluteConstraints x="110" y="62" width="190" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
112	</Constraint>	Cierra la sección XML abierta anteriormente.
113	</Constraints>	Cierra la sección XML abierta anteriormente.
114	</Component>	Cierra la sección XML abierta anteriormente.
115	<Component class="Componentes.BotonMenuLateral" name="botonDashboard">	Declara un componente o contenedor visual y su clase Java.
116	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	<Property name="text" type="java.lang.String" value="Dashboard"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
118	<Property name="seleccionado" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

119	</Properties>	Cierra la sección XML abierta anteriormente.
120	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
121	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonDashboardActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
122	</Events>	Cierra la sección XML abierta anteriormente.
123	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
124	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
125	<AbsoluteConstraints x="22" y="145" width="296" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
126	</Constraint>	Cierra la sección XML abierta anteriormente.
127	</Constraints>	Cierra la sección XML abierta anteriormente.
128	</Component>	Cierra la sección XML abierta anteriormente.
129	<Component class="Componentes.BotonMenuLateral" name="botonUsuarios">	Declara un componente o contenedor visual y su clase Java.
130	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
131	<Property name="text" type="java.lang.String" value="Gestión de Usuarios"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
132	<Property name="tipoIcono" type="java.lang.String" value="USUARIOS"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
133	</Properties>	Cierra la sección XML abierta anteriormente.
134	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
135	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonUsuariosActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
136	</Events>	Cierra la sección XML abierta anteriormente.
137	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
138	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
139	<AbsoluteConstraints x="22" y="236" width="296" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
140	</Constraint>	Cierra la sección XML abierta anteriormente.
141	</Constraints>	Cierra la sección XML abierta anteriormente.
142	</Component>	Cierra la sección XML abierta anteriormente.
143	<Component class="Componentes.BotonMenuLateral" name="botonGestionMenu">	Declara un componente o contenedor visual y su clase Java.
144	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
145	<Property name="text" type="java.lang.String" value="Gestión del Menú" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
146	<Property name="tipoIcono" type="java.lang.String" value="MENU"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
147	</Properties>	Cierra la sección XML abierta anteriormente.
148	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
149	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonGestionMenuActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
150	</Events>	Cierra la sección XML abierta anteriormente.
151	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
152	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
153	<AbsoluteConstraints x="22" y="315" width="296" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
154	</Constraint>	Cierra la sección XML abierta anteriormente.
155	</Constraints>	Cierra la sección XML abierta anteriormente.
156	</Component>	Cierra la sección XML abierta anteriormente.
157	<Component class="Componentes.BotonMenuLateral" name="botonInventario">	Declara un componente o contenedor visual y su clase Java.
158	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

159	<Property name="text" type="java.lang.String" value="Inventario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
160	<Property name="tipoIcono" type="java.lang.String" value="INVENTARIO"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
161	</Properties>	Cierra la sección XML abierta anteriormente.
162	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
163	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonInventarioActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
164	</Events>	Cierra la sección XML abierta anteriormente.
165	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
166	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
167	<AbsoluteConstraints x="22" y="400" width="296" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
168	</Constraint>	Cierra la sección XML abierta anteriormente.
169	</Constraints>	Cierra la sección XML abierta anteriormente.
170	</Component>	Cierra la sección XML abierta anteriormente.
171	<Component class="Componentes.BotonMenuLateral" name="botonReportes">	Declara un componente o contenedor visual y su clase Java.
172	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
173	<Property name="text" type="java.lang.String" value="Reportes"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
174	<Property name="tipoIcono" type="java.lang.String" value="REPORTES"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
175	</Properties>	Cierra la sección XML abierta anteriormente.
176	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
177	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonReportesActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
178	</Events>	Cierra la sección XML abierta anteriormente.
179	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
180	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
181	<AbsoluteConstraints x="22" y="485" width="296" height="64"/>	Define la posición y el tamaño del componente dentro del diseño.
182	</Constraint>	Cierra la sección XML abierta anteriormente.
183	</Constraints>	Cierra la sección XML abierta anteriormente.
184	</Component>	Cierra la sección XML abierta anteriormente.
185	<Component class="Componentes.BotonMenuLateral" name="botonCerrarSesion">	Declara un componente o contenedor visual y su clase Java.
186	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
187	<Property name="text" type="java.lang.String" value="Volver Al Menú"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
188	<Property name="mostrarBorde" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
189	<Property name="tipoIcono" type="java.lang.String" value="SALIR"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
190	</Properties>	Cierra la sección XML abierta anteriormente.
191	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
192	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonCerrarSesionActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
193	</Events>	Cierra la sección XML abierta anteriormente.
194	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
195	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
196	<AbsoluteConstraints x="22" y="955" width="296" height="62"/>	Define la posición y el tamaño del componente dentro del diseño.
197	</Constraint>	Cierra la sección XML abierta anteriormente.
198	</Constraints>	Cierra la sección XML abierta anteriormente.

199	</Component>	Cierra la sección XML abierta anteriormente.
200	</SubComponents>	Cierra la sección XML abierta anteriormente.
201	</Container>	Cierra la sección XML abierta anteriormente.
202	<Container class="javax.swing.JPanel" name="panelCabecera">	Declara un componente o contenedor visual y su clase Java.
203	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
204	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
205	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
206	</Property>	Cierra la sección XML abierta anteriormente.
207	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
208	<Border info="org.netbeans.modules.form.compat2.border.LineBorderInfo">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
209	<LineBorder>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
210	<Color PropertyName="color" blue="cc" green="cc" red="cc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
211	</LineBorder>	Cierra la sección XML abierta anteriormente.
212	</Border>	Cierra la sección XML abierta anteriormente.
213	</Property>	Cierra la sección XML abierta anteriormente.
214	</Properties>	Cierra la sección XML abierta anteriormente.
215	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
216	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
217	<AbsoluteConstraints x="340" y="0" width="1500" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
218	</Constraint>	Cierra la sección XML abierta anteriormente.
219	</Constraints>	Cierra la sección XML abierta anteriormente.
220		Línea en blanco para organizar el XML del formulario.
221	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
222	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
223	</Layout>	Cierra la sección XML abierta anteriormente.
224	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
225	<Component class="javax.swing.JLabel" name="labelHamburguesa">	Declara un componente o contenedor visual y su clase Java.
226	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
227	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
228		Metadato de NetBeans que conserva el estado editable del diseñador visual.
229	</Property>	Cierra la sección XML abierta anteriormente.
230	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
231	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
232	</Property>	Cierra la sección XML abierta anteriormente.
233	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
234	<Property name="text" type="java.lang.String" value="☰"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
235	</Properties>	Cierra la sección XML abierta anteriormente.
236	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
237	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
238	<AbsoluteConstraints x="25" y="21" width="55" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
239	</Constraint>	Cierra la sección XML abierta anteriormente.

240	</Constraints>	Cierra la sección XML abierta anteriormente.
241	</Component>	Cierra la sección XML abierta anteriormente.
242	<Component class="javax.swing.JLabel" name="labelTituloCabecera">	Declara un componente o contenedor visual y su clase Java.
243	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
244	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
245		Metadato de NetBeans que conserva el estado editable del diseñador visual.
246	</Property>	Cierra la sección XML abierta anteriormente.
247	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
248	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
249	</Property>	Cierra la sección XML abierta anteriormente.
250	<Property name="text" type="java.lang.String" value="PANEL DE ADMINISTRACION"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251	</Properties>	Cierra la sección XML abierta anteriormente.
252	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
253	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
254	<AbsoluteConstraints x="100" y="25" width="510" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
255	</Constraint>	Cierra la sección XML abierta anteriormente.
256	</Constraints>	Cierra la sección XML abierta anteriormente.
257	</Component>	Cierra la sección XML abierta anteriormente.
258	<Component class="Labels.LabelEscalable" name="labelIconoUsuario">	Declara un componente o contenedor visual y su clase Java.
259	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
260	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
261	<Image iconType="3" name="/Imagenes/IconoUsuario.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
262	</Property>	Cierra la sección XML abierta anteriormente.
263	<Property name="mantenerProporcion" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
264	</Properties>	Cierra la sección XML abierta anteriormente.
265	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
266	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
267	<AbsoluteConstraints x="1270" y="22" width="56" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
268	</Constraint>	Cierra la sección XML abierta anteriormente.
269	</Constraints>	Cierra la sección XML abierta anteriormente.
270	</Component>	Cierra la sección XML abierta anteriormente.
271	<Component class="javax.swing.JLabel" name="labelBienvenido">	Declara un componente o contenedor visual y su clase Java.
272	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
273	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
274		Metadato de NetBeans que conserva el estado editable del diseñador visual.
275	</Property>	Cierra la sección XML abierta anteriormente.
276	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
277	<Color blue="3f" green="25" red="12" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
278	</Property>	Cierra la sección XML abierta anteriormente.
279	<Property name="text" type="java.lang.String" value="Bienvenido,"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
280	</Properties>	Cierra la sección XML abierta anteriormente.

281	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
282	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
283	<AbsoluteConstraints x="1340" y="32" width="105" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
284	</Constraint>	Cierra la sección XML abierta anteriormente.
285	</Constraints>	Cierra la sección XML abierta anteriormente.
286	</Component>	Cierra la sección XML abierta anteriormente.
287	<Component class="javax.swing.JLabel" name="labelUsuario">	Declara un componente o contenedor visual y su clase Java.
288	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
289	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
290		Metadato de NetBeans que conserva el estado editable del diseñador visual.
291	</Property>	Cierra la sección XML abierta anteriormente.
292	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
293	<Color blue="3f" green="25" red="12" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
294	</Property>	Cierra la sección XML abierta anteriormente.
295	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
296	<Property name="text" type="java.lang.String" value="Administrador"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
297	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
298	</Properties>	Cierra la sección XML abierta anteriormente.
299	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
300	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
301	<AbsoluteConstraints x="1440" y="15" width="123" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
302	</Constraint>	Cierra la sección XML abierta anteriormente.
303	</Constraints>	Cierra la sección XML abierta anteriormente.
304	</Component>	Cierra la sección XML abierta anteriormente.
305	<Component class="javax.swing.JSeparator" name="separadorCabecera">	Declara un componente o contenedor visual y su clase Java.
306	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
307	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
308	<Color blue="eb" green="e5" red="e1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
309	</Property>	Cierra la sección XML abierta anteriormente.
310	</Properties>	Cierra la sección XML abierta anteriormente.
311	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
312	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
313	<AbsoluteConstraints x="0" y="99" width="1580" height="1"/>	Define la posición y el tamaño del componente dentro del diseño.
314	</Constraint>	Cierra la sección XML abierta anteriormente.
315	</Constraints>	Cierra la sección XML abierta anteriormente.
316	</Component>	Cierra la sección XML abierta anteriormente.
317	</SubComponents>	Cierra la sección XML abierta anteriormente.
318	</Container>	Cierra la sección XML abierta anteriormente.
319	<Container class="javax.swing.JPanel" name="panelContenido">	Declara un componente o contenedor visual y su clase Java.
320	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

321	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
322	>	
322	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
323	</Property>	Cierra la sección XML abierta anteriormente.
324	</Properties>	Cierra la sección XML abierta anteriormente.
325	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
326	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
327	<AbsoluteConstraints x="340" y="100" width="1580" height="932"/>	Define la posición y el tamaño del componente dentro del diseño.
328	</Constraint>	Cierra la sección XML abierta anteriormente.
329	</Constraints>	Cierra la sección XML abierta anteriormente.
330		Línea en blanco para organizar el XML del formulario.
331	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
332	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
333	</Layout>	Cierra la sección XML abierta anteriormente.
334	</Container>	Cierra la sección XML abierta anteriormente.
335	</SubComponents>	Cierra la sección XML abierta anteriormente.
336	</Container>	Cierra la sección XML abierta anteriormente.
337	</SubComponents>	Cierra la sección XML abierta anteriormente.
338	</Form>	Cierra la sección XML abierta anteriormente.

16.75 src/GUI_ADMINISTRADOR/MenuAdmin.java

Menú lateral del administrador. Cambia entre dashboard, usuarios, menú, inventario y reportes.

Tipo: java | Líneas: 750 | Archivo 75 de 157

Clases/tipos detectados: MenuAdmin

Línea	Constructor o método detectado
79	public MenuAdmin() {
100	private void configurarMenuDesplegable() {
125	public void mouseClicked(MouseEvent e) {
143	private void guardarGeometriaMenu() {
198	private void alternarMenuLateral() {
252	private void aplicarAperturaMenu(double apertura) {
415	private void actualizarBotonCompacto(
457	private int interpolar(
472	private void aplicarTipografia() {
486	private void configurarSecciones() {
497	private void agregarSeccion(JPanel seccion, String nombre) {
505	private void mostrarSeccion(String nombre, BotonMenuLateral botonSeleccionado) {
519	private String nombreSeccion(String seccion) {
530	private void cerrarSesion() {
539	private void initComponents() {
591	public void actionPerformed(java.awt.event.ActionEvent evt) {
600	public void actionPerformed(java.awt.event.ActionEvent evt) {
609	public void actionPerformed(java.awt.event.ActionEvent evt) {
618	public void actionPerformed(java.awt.event.ActionEvent evt) {
627	public void actionPerformed(java.awt.event.ActionEvent evt) {
637	public void actionPerformed(java.awt.event.ActionEvent evt) {
690	private void botonDashboardActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonDashboardActionPerformed
694	private void botonUsuariosActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonUsuariosActionPerformed
698	private void botonGestionMenuActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonGestionMenuActionPerformed
702	private void botonInventarioActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonInventarioActionPerformed
706	private void botonReportesActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonReportesActionPerformed
710	private void botonCerrarSesionActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonCerrarSesionActionPerformed
714	public static void main(String args[]) {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* JFrame Form editable desde el diseñador visual de NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
5		Línea en blanco: separa visualmente bloques para que el código sea más legible.
6	import Componentes.BotonMenuLateral;	Importa Componentes.BotonMenuLateral para poder usar esa clase sin escribir su nombre completo.
7	import Login.Login;	Importa Login.Login para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.CardLayout;	Importa java.awt.CardLayout para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.GridBagLayout;	Importa java.awt.GridBagLayout para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
13	import GUI_ADMINISTRADOR.InicioAdminForm;	Importa GUI_ADMINISTRADOR.InicioAdminForm para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.Rectangle;	Importa java.awt.Rectangle para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
20	import Componentes.BarraTituloForm;	Importa Componentes.BarraTituloForm para poder usar esa clase sin escribir su nombre completo.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	* Pantalla contenedora del modulo administrativo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
26	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
27	public class MenuAdmin extends javax.swing.JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	private static final String DASHBOARD = "dashboard";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
30	private static final String USUARIOS = "usuarios";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
31	private static final String GESTION_MENU = "gestionMenu";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
32	private static final String INVENTARIO = "inventario";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
33	private static final String REPORTES = "reportes";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
34	private final TemaAdmin tema;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
35	private CardLayout navegador;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36	private BotonMenuLateral botonActivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	private BarraTituloForm barraTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38	// =====	Comentario para humanos; Java no ejecuta este texto. =====
39	// MENÚ LATERAL DESPLEGABLE	Comentario para humanos; Java no ejecuta este texto. MENÚ LATERAL DESPLEGABLE
40	// =====	Comentario para humanos; Java no ejecuta este texto. =====
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	private boolean menuLateralAbierto = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.

44	// 1.0 = completamente abierto	Comentario para humanos; Java no ejecuta este texto. 1.0 = completamente abierto
45	// 0.0 = completamente cerrado	Comentario para humanos; Java no ejecuta este texto. 0.0 = completamente cerrado
46	private double aperturaMenu = 1.0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	private double aperturaInicial;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
49	private double aperturaObjetivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	private Timer timerMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52	private long tiempoInicioAnimacionNanos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	private long duracionAnimacionNanos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
54	// Duración de la animación (300 ms).	Comentario para humanos; Java no ejecuta este texto. Duración de la animación (300 ms).
55	private static final long DURACION_MENU_NANOS = 300_000_000L;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	// Medidas originales del menú lateral.	Comentario para humanos; Java no ejecuta este texto. Medidas originales del menú lateral.
58	private int anchoLateralAbierto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	private int anchoLateralCerrado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	// Bounds originales	Comentario para humanos; Java no ejecuta este texto. Bounds originales
62	private Rectangle boundsLogoAbierto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	private Rectangle boundsBotonDashboard;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	private Rectangle boundsBotonUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	private Rectangle boundsBotonGestionMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	private Rectangle boundsBotonInventario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	private Rectangle boundsBotonReportes;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
69	private Rectangle boundsBotonCerrarSesion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	private Rectangle boundsCabeceraAbierta;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
72	private Rectangle boundsContenidoAbierto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
73	private Rectangle boundsSeparadorAbierto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	private Rectangle boundsIconoUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	private Rectangle boundsBienvenido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
77	private Rectangle boundsUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	public MenuAdmin() {	Constructor de MenuAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
80	tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	barraTitulo = BarraTituloForm.instalar(this, "Inicio de Administrador");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
87	// La barra personalizada ocupa 48 px dentro de una ventana de 1080 px.	Comentario para humanos; Java no ejecuta este texto. La barra personalizada ocupa 48 px dentro de una ventana de 1080 px.

88	// El contenido administrativo dispone de los 1032 px restantes.	Comentario para humanos; Java no ejecuta este texto. El contenido administrativo dispone de los 1032 px restantes.
89	setSize(1920, 1080);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	aplicarTipografia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	configurarSecciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	mostrarSeccion(DASHBOARD, botonDashboard);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	// Conserva la animación del menú con las medidas del formulario.	Comentario para humanos; Java no ejecuta este texto. Conserva la animación del menú con las medidas del formulario.
98	configurarMenuDesplegable();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99	}	Cierra el bloque abierto anteriormente.
100	private void configurarMenuDesplegable() {	Declara el método configurarMenuDesplegable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
103	* Guardamos las dimensiones actuales.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
104	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
105	* Se usan directamente las dimensiones diseñadas en el formulario.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
106	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	guardarGeometriaMenu();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109		Línea en blanco: separa visualmente bloques para que el código sea más legible.
110	// =====	Comentario para humanos; Java no ejecuta este texto. =====
111	// HACER CLICKEABLE EL ICONO ■	Comentario para humanos; Java no ejecuta este texto. HACER CLICKEABLE EL ICONO ■
112	// =====	Comentario para humanos; Java no ejecuta este texto. =====
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	labelHamburguesa.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
115	Cursor.getPredefinedCursor(Cursor.HAND_CURSOR)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
116	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	labelHamburguesa.setToolTipText(	Configura una propiedad mediante setToolTipText en el objeto indicado.
119	"Abrir / cerrar menú"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
120	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	labelHamburguesa.addMouseListener(new MouseAdapter() {	Crea una nueva instancia de MouseAdapter y la usa o guarda para continuar el proceso.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
125	public void mouseClicked(MouseEvent e) {	Declara el método mouseClicked. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
126	alternarMenuLateral();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	}	Cierra el bloque abierto anteriormente.
128	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129	labelHamburguesa.setFocusable(true);	Configura una propiedad mediante setFocusable en el objeto indicado.
130	labelHamburguesa.getInputMap(javax.swing.JComponent.WHEN_FOCUSED).put(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	javax.swing.KeyStroke.getKeyStroke("SPACE"), "alternarMenu");	Ejecuta una instrucción Java completa; el punto y coma marca su final.

132	labelHamburguesa.getInputMap(javax.swing.JComponent.WHEN_FOCUSED).put(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	javax.swing.KeyStroke.getKeyStroke("ENTER"), "alternarMenu");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	labelHamburguesa.getActionMap().put("alternarMenu", new javax.swing.AbstractAction() {	Crea una nueva instancia de javax.swing.AbstractAction y la usa o guarda para continuar el proceso.
135	@Override public void actionPerformed(java.awt.event.ActionEvent e) { alternarMenuLateral(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
136	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	for (BotonMenuLateral boton : new BotonMenuLateral[]{botonDashboard, botonUsuarios,	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
138	botonGestionMenu, botonInventario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	botonReportes, botonCerrarSesion}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140	boton.setToolTipText(boton.getText());	Configura una propiedad mediante setToolTipText en el objeto indicado.
141	}	Cierra el bloque abierto anteriormente.
142	}	Cierra el bloque abierto anteriormente.
143	private void guardarGeometriaMenu() {	Declara el método guardarGeometriaMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
144	anchoLateralAbierto = panelLateral.getWidth();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
147	* Tu menú original mide 340.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
148	* Cerrado queremos que sea aproximadamente 90.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
149	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
150	* El ancho cerrado conserva la proporción del diseño original.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
151	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
152	anchoLateralCerrado =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
153	(int) Math.round(anchoLateralAbierto * (90.0 / 340.0));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	// Guardamos posiciones originales	Comentario para humanos; Java no ejecuta este texto. Guardamos posiciones originales
156	boundsLogoAbierto = new Rectangle(labelLogo.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	boundsBotonDashboard =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
159	new Rectangle(botonDashboard.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
160		Línea en blanco: separa visualmente bloques para que el código sea más legible.
161	boundsBotonUsuarios =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
162	new Rectangle(botonUsuarios.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	boundsBotonGestionMenu =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
165	new Rectangle(botonGestionMenu.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
166		Línea en blanco: separa visualmente bloques para que el código sea más legible.
167	boundsBotonInventario =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
168	new Rectangle(botonInventario.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	boundsBotonReportes =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
171	new Rectangle(botonReportes.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	boundsBotonCerrarSesion =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
174	new Rectangle(botonCerrarSesion.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
175		Línea en blanco: separa visualmente bloques para que el código sea más legible.

176	boundsCabeceraAbierta =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
177	new Rectangle(panelCabecera.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
179	boundsContenidoAbierto =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
180	new Rectangle(panelContenido.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	boundsSeparadorAbierto =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
183	new Rectangle(separadorCabecera.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	// Elementos que están pegados al lado derecho	Comentario para humanos; Java no ejecuta este texto. Elementos que están pegados al lado derecho
186	// de la cabecera.	Comentario para humanos; Java no ejecuta este texto. de la cabecera.
187	boundsIconoUsuario =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
188	new Rectangle(labelIconoUsuario.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
189		Línea en blanco: separa visualmente bloques para que el código sea más legible.
190	boundsBienvenido =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
191	new Rectangle(labelBienvenido.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	boundsUsuario =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
194	new Rectangle(labelUsuario.getBounds());	Crea una nueva instancia de Rectangle y la usa o guarda para continuar el proceso.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	}	Cierra el bloque abierto anteriormente.
198	private void alternarMenuLateral() {	Declara el método alternarMenuLateral. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	// Cambiamos el estado deseado	Comentario para humanos; Java no ejecuta este texto. Cambiamos el estado deseado
201	menuLateralAbierto = !menuLateralAbierto;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
203	// Partimos desde donde se encuentre actualmente.	Comentario para humanos; Java no ejecuta este texto. Partimos desde donde se encuentre actualmente.
204	// Esto permite incluso pulsar el botón mientras	Comentario para humanos; Java no ejecuta este texto. Esto permite incluso pulsar el botón mientras
205	// todavía se está animando.	Comentario para humanos; Java no ejecuta este texto. todavía se está animando.
206	aperturaInicial = aperturaMenu;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	aperturaObjetivo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
209	menuLateralAbierto ? 1.0 : 0.0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211	tiempoInicioAnimacionNanos =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
212	System.nanoTime();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
213		Línea en blanco: separa visualmente bloques para que el código sea más legible.
214		Línea en blanco: separa visualmente bloques para que el código sea más legible.
215	// Si ya existe una animación, la detenemos.	Comentario para humanos; Java no ejecuta este texto. Si ya existe una animación, la detenemos.
216	if (timerMenu != null && timerMenu.isRunning()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
217	timerMenu.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218	}	Cierra el bloque abierto anteriormente.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.

220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	duracionAnimacionNanos = Math.max(80_000_000L,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
222	(long) (DURACION_MENU_NANOS * Math.abs(aperturaObjetivo - aperturaInicial)));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223	labelHamburguesa.setToolTipText(menuLateralAbierto ? "Contraer menú" : "Expandir menú");	Configura una propiedad mediante setToolTipText en el objeto indicado.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	// Cerca de 60 FPS. Swing agrupa los repintados si el EDT se ocupa.	Comentario para humanos; Java no ejecuta este texto. Cerca de 60 FPS. Swing agrupa los repintados si el EDT se ocupa.
226	timerMenu = new Timer(16, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
227	long transcurrido = System.nanoTime() - tiempoInicioAnimacionNanos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
228	double progreso = Math.min(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
229	1.0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
230	(double) transcurrido / duracionAnimacionNanos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	// Aceleración y frenado graduales para evitar el tirón inicial.	Comentario para humanos; Java no ejecuta este texto. Aceleración y frenado graduales para evitar el tirón inicial.
233	double suavizado = progreso * progreso * progreso	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
234	* (progreso * (6.0 * progreso - 15.0) + 10.0);	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
235		Línea en blanco: separa visualmente bloques para que el código sea más legible.
236	aperturaMenu = aperturaInicial	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
237	+ (aperturaObjetivo - aperturaInicial) * suavizado;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238		Línea en blanco: separa visualmente bloques para que el código sea más legible.
239	aplicarAperturaMenu(aperturaMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240		Línea en blanco: separa visualmente bloques para que el código sea más legible.
241	if (progreso >= 1.0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
242	timerMenu.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
243	aperturaMenu = aperturaObjetivo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
244	aplicarAperturaMenu(aperturaMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246		Línea en blanco: separa visualmente bloques para que el código sea más legible.
247	}	Cierra el bloque abierto anteriormente.
248	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
249	timerMenu.setCoalesce(true);	Configura una propiedad mediante setCoalesce en el objeto indicado.
250	timerMenu.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
251	}	Cierra el bloque abierto anteriormente.
252	private void aplicarAperturaMenu(double apertura) {	Declara el método aplicarAperturaMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
253		Línea en blanco: separa visualmente bloques para que el código sea más legible.
254	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
255	* apertura:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
256	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
257	* 0.0 = cerrado	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
258	* 1.0 = abierto	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
259	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
260		Línea en blanco: separa visualmente bloques para que el código sea más legible.
261	int anchoActual = interpolar(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
262	anchoLateralCerrado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
263	anchoLateralAbierto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

264	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
268	// =====	Comentario para humanos; Java no ejecuta este texto. =====
269	// PANEL LATERAL	Comentario para humanos; Java no ejecuta este texto. PANEL LATERAL
270	// =====	Comentario para humanos; Java no ejecuta este texto. =====
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.
272	panelLateral.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
273	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	anchoActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
276	panelLateral.getHeight()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
277	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279		Línea en blanco: separa visualmente bloques para que el código sea más legible.
280	// =====	Comentario para humanos; Java no ejecuta este texto. =====
281	// PANEL CABECERA	Comentario para humanos; Java no ejecuta este texto. PANEL CABECERA
282	// =====	Comentario para humanos; Java no ejecuta este texto. =====
283		Línea en blanco: separa visualmente bloques para que el código sea más legible.
284	int anchoCabecera =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
285	panelRaiz.getWidth() - anchoActual;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	panelCabecera.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
288	anchoActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
289	boundsCabeceraAbierta.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290	anchoCabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
291	boundsCabeceraAbierta.height	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
292	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
293		Línea en blanco: separa visualmente bloques para que el código sea más legible.
294		Línea en blanco: separa visualmente bloques para que el código sea más legible.
295	// =====	Comentario para humanos; Java no ejecuta este texto. =====
296	// PANEL CONTENIDO	Comentario para humanos; Java no ejecuta este texto. PANEL CONTENIDO
297	// =====	Comentario para humanos; Java no ejecuta este texto. =====
298		Línea en blanco: separa visualmente bloques para que el código sea más legible.
299	int anchoContenido =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
300	panelRaiz.getWidth() - anchoActual;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
301		Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	panelContenido.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
303	anchoActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
304	boundsContenidoAbierto.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
305	anchoContenido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	boundsContenidoAbierto.height	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
307	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

308		Línea en blanco: separa visualmente bloques para que el código sea más legible.
309		Línea en blanco: separa visualmente bloques para que el código sea más legible.
310	// =====	Comentario para humanos; Java no ejecuta este texto. =====
311	// SEPARADOR SUPERIOR	Comentario para humanos; Java no ejecuta este texto. SEPARADOR SUPERIOR
312	// =====	Comentario para humanos; Java no ejecuta este texto. =====
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	separadorCabecera.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
315	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	boundsSeparadorAbierto.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
317	anchoCabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	boundsSeparadorAbierto.height	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
319	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
320		Línea en blanco: separa visualmente bloques para que el código sea más legible.
321		Línea en blanco: separa visualmente bloques para que el código sea más legible.
322	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
323	* Como la cabecera aumenta de ancho hacia	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
324	* la izquierda, movemos el usuario hacia	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
325	* la derecha para que permanezca en el mismo	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
326	* lugar visual.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
327	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	int diferenciaCabecera =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
330	anchoCabecera - boundsCabeceraAbierta.width;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
331		Línea en blanco: separa visualmente bloques para que el código sea más legible.
332		Línea en blanco: separa visualmente bloques para que el código sea más legible.
333	labelIconoUsuario.setLocation(	Configura una propiedad mediante setLocation en el objeto indicado.
334	boundsIconoUsuario.x + diferenciaCabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
335	boundsIconoUsuario.y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	labelBienvenido.setLocation(	Configura una propiedad mediante setLocation en el objeto indicado.
339	boundsBienvenido.x + diferenciaCabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
340	boundsBienvenido.y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
342		Línea en blanco: separa visualmente bloques para que el código sea más legible.
343	labelUsuario.setLocation(	Configura una propiedad mediante setLocation en el objeto indicado.
344	boundsUsuario.x + diferenciaCabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
345	boundsUsuario.y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
346	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349	// =====	Comentario para humanos; Java no ejecuta este texto. =====
350	// LOGO	Comentario para humanos; Java no ejecuta este texto. LOGO
351	// =====	Comentario para humanos; Java no ejecuta este texto. =====

352		Línea en blanco: separa visualmente bloques para que el código sea más legible.
353	int xLogoCerrado =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
354	(anchoLateralCerrado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	- boundsLogoAbierto.width) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
356		Línea en blanco: separa visualmente bloques para que el código sea más legible.
357	int xLogoActual = interpolat(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
358	xLogoCerrado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359	boundsLogoAbierto.x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
360	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
361	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
362		Línea en blanco: separa visualmente bloques para que el código sea más legible.
363	labelLogo.setLocation(	Configura una propiedad mediante setLocation en el objeto indicado.
364	xLogoActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	boundsLogoAbierto.y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
366	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
367		Línea en blanco: separa visualmente bloques para que el código sea más legible.
368		Línea en blanco: separa visualmente bloques para que el código sea más legible.
369	// =====	Comentario para humanos; Java no ejecuta este texto. =====
370	// BOTONES	Comentario para humanos; Java no ejecuta este texto. BOTONES
371	// =====	Comentario para humanos; Java no ejecuta este texto. =====
372		Línea en blanco: separa visualmente bloques para que el código sea más legible.
373	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
374	botonDashboard,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
375	boundsBotonDashboard,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
376	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
377	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
380	botonUsuarios,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
381	boundsBotonUsuarios,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
382	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
383	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
384		Línea en blanco: separa visualmente bloques para que el código sea más legible.
385	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
386	botonGestionMenu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	boundsBotonGestionMenu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
388	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
389	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
390		Línea en blanco: separa visualmente bloques para que el código sea más legible.
391	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	botonInventario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
393	boundsBotonInventario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
394	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
395	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
398	botonReportes,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
399	boundsBotonReportes,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
400	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
401	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
402		Línea en blanco: separa visualmente bloques para que el código sea más legible.
403	actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
404	botonCerrarSesion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
405	boundsBotonCerrarSesion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
406	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
407	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
408		Línea en blanco: separa visualmente bloques para que el código sea más legible.
409	int alpha = (int) Math.round(255 * Math.max(0.0, Math.min(1.0, (apertura - 0.35) / 0.65)));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
410	labelMarca.setForeground(new Color(255, 255, 255, alpha));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
411	labelRol.setForeground(new Color(255, 190, 0, alpha));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
412	panelContenido.doLayout();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413	panelRaiz.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
414	}	Cierra el bloque abierto anteriormente.
415	private void actualizarBotonCompacto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
416	BotonMenuLateral boton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
417	Rectangle abierto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	double apertura) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
419		Línea en blanco: separa visualmente bloques para que el código sea más legible.
420	// Margen proporcional al ancho original del menú.	Comentario para humanos; Java no ejecuta este texto. Margen proporcional al ancho original del menú.
421	int margenCerrado =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
422	Math.max(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
423	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
424	(int) Math.round(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
425	anchoLateralAbierto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
426	* (12.0 / 340.0)	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
427	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
428	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429		Línea en blanco: separa visualmente bloques para que el código sea más legible.
430		Línea en blanco: separa visualmente bloques para que el código sea más legible.
431	int anchoBotonCerrado =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
432	anchoLateralCerrado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
433	- (margenCerrado * 2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
434		Línea en blanco: separa visualmente bloques para que el código sea más legible.
435		Línea en blanco: separa visualmente bloques para que el código sea más legible.
436	int xActual = interpolar(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
437	margenCerrado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
438	abierto.x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

440	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
441		Línea en blanco: separa visualmente bloques para que el código sea más legible.
442	int anchoActual = interpolar(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
443	anchoBotonCerrado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	abierto.width,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
447		Línea en blanco: separa visualmente bloques para que el código sea más legible.
448		Línea en blanco: separa visualmente bloques para que el código sea más legible.
449	boton.setAperturaMenu(apertura);	Configura una propiedad mediante setAperturaMenu en el objeto indicado.
450	boton.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
451	xActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
452	abierto.y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	anchoActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
454	abierto.height	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
455	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
456	}	Cierra el bloque abierto anteriormente.
457	private int interpolar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
458	int cerrado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459	int abierto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
460	double apertura) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461		Línea en blanco: separa visualmente bloques para que el código sea más legible.
462	return (int) Math.round(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
463	cerrado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
464	+ (abierto - cerrado) * apertura	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
465	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
466	}	Cierra el bloque abierto anteriormente.
467	@Override public void dispose() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
468	if (timerMenu != null) timerMenu.stop();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
469	super.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
470	}	Cierra el bloque abierto anteriormente.
471		Línea en blanco: separa visualmente bloques para que el código sea más legible.
472	private void aplicarTipografia() {	Declara el método aplicarTipografia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
473	labelMarca.setFont(tema.negrita(28f));	Configura una propiedad mediante setFont en el objeto indicado.
474	labelRol.setFont(tema.media(18f));	Configura una propiedad mediante setFont en el objeto indicado.
475	labelTituloCabecera.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
476	labelBienvenido.setFont(tema.regular(16f));	Configura una propiedad mediante setFont en el objeto indicado.
477	labelUsuario.setFont(tema.negrita(16f));	Configura una propiedad mediante setFont en el objeto indicado.
478	botonDashboard.setFont(tema.negrita(17f));	Configura una propiedad mediante setFont en el objeto indicado.
479	botonUsuarios.setFont(tema.media(17f));	Configura una propiedad mediante setFont en el objeto indicado.
480	botonGestionMenu.setFont(tema.media(17f));	Configura una propiedad mediante setFont en el objeto indicado.
481	botonInventario.setFont(tema.media(17f));	Configura una propiedad mediante setFont en el objeto indicado.
482	botonReportes.setFont(tema.media(17f));	Configura una propiedad mediante setFont en el objeto indicado.
483	botonCerrarSesion.setFont(tema.negrita(17f));	Configura una propiedad mediante setFont en el objeto indicado.

484	}	Cierra el bloque abierto anteriormente.
485		Línea en blanco: separa visualmente bloques para que el código sea más legible.
486	private void configurarSecciones() {	Declara el método configurarSecciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
487	navegador = new CardLayout();	Crea una nueva instancia de CardLayout y la usa o guarda para continuar el proceso.
488	panelContenido.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
489	panelContenido.setLayout(navegador);	Configura una propiedad mediante setLayout en el objeto indicado.
490	agregarSeccion(new DashboardPanel(), DASHBOARD);	Crea una nueva instancia de DashboardPanel y la usa o guarda para continuar el proceso.
491	agregarSeccion(new UsuariosPanel(), USUARIOS);	Crea una nueva instancia de UsuariosPanel y la usa o guarda para continuar el proceso.
492	agregarSeccion(new GestionMenuPanel(), GESTION_MENU);	Crea una nueva instancia de GestionMenuPanel y la usa o guarda para continuar el proceso.
493	agregarSeccion(new InventarioPanel(), INVENTARIO);	Crea una nueva instancia de InventarioPanel y la usa o guarda para continuar el proceso.
494	agregarSeccion(new ReportesPanel(), REPORTES);	Crea una nueva instancia de ReportesPanel y la usa o guarda para continuar el proceso.
495	}	Cierra el bloque abierto anteriormente.
496		Línea en blanco: separa visualmente bloques para que el código sea más legible.
497	private void agregarSeccion(JPanel seccion, String nombre) {	Declara el método agregarSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
498	seccion.setBackground(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
499	JPanel contenedor = new JPanel(new GridBagLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
500	contenedor.setBackground(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
501	contenedor.add(seccion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
502	panelContenido.add(contenedor, nombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
503	}	Cierra el bloque abierto anteriormente.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	private void mostrarSeccion(String nombre, BotonMenuLateral botonSeleccionado) {	Declara el método mostrarSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
506	if (botonActivo != null && botonActivo != botonSeleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
507	botonActivo.setSeleccionado(false);	Configura una propiedad mediante setSeleccionado en el objeto indicado.
508	}	Cierra el bloque abierto anteriormente.
509	botonSeleccionado.setSeleccionado(true);	Configura una propiedad mediante setSeleccionado en el objeto indicado.
510	botonActivo = botonSeleccionado;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
511	navegador.show(panelContenido, nombre);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
512	panelContenido.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
513	panelContenido.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
514	if (barraTitulo != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
515	barraTitulo.setSeccion(nombreSeccion(nombre));	Configura una propiedad mediante setSeccion en el objeto indicado.
516	}	Cierra el bloque abierto anteriormente.
517	}	Cierra el bloque abierto anteriormente.
518		Línea en blanco: separa visualmente bloques para que el código sea más legible.
519	private String nombreSeccion(String seccion) {	Declara el método nombreSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
520	return switch (seccion) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
521	case DASHBOARD -> "Inicio de Administrador";	Define una de las rutas posibles dentro del switch.
522	case USUARIOS -> "Gestión de Usuarios";	Define una de las rutas posibles dentro del switch.
523	case GESTION_MENU -> "Gestión de Productos";	Define una de las rutas posibles dentro del switch.
524	case INVENTARIO -> "Inventario";	Define una de las rutas posibles dentro del switch.
525	case REPORTES -> "Reportes";	Define una de las rutas posibles dentro del switch.
526	default -> "Administración";	Define una de las rutas posibles dentro del switch.
527	};	Cierra el bloque abierto anteriormente y termina la declaración.

528	}	Cierra el bloque abierto anteriormente.
529		Línea en blanco: separa visualmente bloques para que el código sea más legible.
530	private void cerrarSesion() {	Declara el método cerrarSesion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
531	InicioAdminForm inicio = new InicioAdminForm();	Crea una nueva instancia de InicioAdminForm y la usa o guarda para continuar el proceso.
532	inicio.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
533	inicio.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
534	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
535	}	Cierra el bloque abierto anteriormente.
536		Línea en blanco: separa visualmente bloques para que el código sea más legible.
537	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
538	// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
539	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
540		Línea en blanco: separa visualmente bloques para que el código sea más legible.
541	panelRaiz = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
542	panelLateral = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
543	labelLogo = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
544	labelMarca = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
545	labelRol = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
546	botonDashboard = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
547	botonUsuarios = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
548	botonGestionMenu = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
549	botonInventario = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
550	botonReportes = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
551	botonCerrarSesion = new Componentes.BotonMenuLateral();	Crea una nueva instancia de Componentes.BotonMenuLateral y la usa o guarda para continuar el proceso.
552	panelCabecera = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
553	labelHamburguesa = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
554	labelTituloCabecera = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
555	labelIconoUsuario = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
556	labelBienvenido = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
557	labelUsuario = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
558	separadorCabecera = new javax.swing.JSeparator();	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
559	panelContenido = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
560		Línea en blanco: separa visualmente bloques para que el código sea más legible.
561	setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
562	setTitle("Panel de Administración");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
563	setMinimumSize(new java.awt.Dimension(1024, 650));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
564	setSize(new java.awt.Dimension(1920, 1080));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
565	getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
566		Línea en blanco: separa visualmente bloques para que el código sea más legible.
567	panelRaiz.setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
568	panelRaiz.setPreferredSize(new java.awt.Dimension(1920, 1032));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
569	panelRaiz.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
570		Línea en blanco: separa visualmente bloques para que el código sea más legible.
571	panelLateral.setBackground(new java.awt.Color(1, 15, 30));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.

572	<code>panelLateral.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
573		Línea en blanco: separa visualmente bloques para que el código sea más legible.
574	<code>labelLogo.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/LogoW.png"))); // NOI18N</code>	Crea una nueva instancia de <code>javax.swing.ImageIcon</code> y la usa o guarda para continuar el proceso.
575	<code>labelLogo.setMantenerProporcion(true);</code>	Configura una propiedad mediante <code>setMantenerProporcion</code> en el objeto indicado.
576	<code>panelLateral.add(labelLogo, new org.netbeans.lib.awtextra.AbsoluteConstraints(27, 25, 68, 68));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
577		Línea en blanco: separa visualmente bloques para que el código sea más legible.
578	<code>labelMarca.setFont(new java.awt.Font("Dialog", 1, 28)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
579	<code>labelMarca.setForeground(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
580	<code>labelMarca.setText("Waldonald's");</code>	Cambia el texto visible de un componente de la interfaz.
581	<code>panelLateral.add(labelMarca, new org.netbeans.lib.awtextra.AbsoluteConstraints(110, 25, 205, 39));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
582		Línea en blanco: separa visualmente bloques para que el código sea más legible.
583	<code>labelRol.setFont(new java.awt.Font("Dialog", 0, 18)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
584	<code>labelRol.setForeground(new java.awt.Color(255, 190, 0));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
585	<code>labelRol.setText("Administrador");</code>	Cambia el texto visible de un componente de la interfaz.
586	<code>panelLateral.add(labelRol, new org.netbeans.lib.awtextra.AbsoluteConstraints(110, 62, 190, 30));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
587		Línea en blanco: separa visualmente bloques para que el código sea más legible.
588	<code>botonDashboard.setText("Dashboard");</code>	Cambia el texto visible de un componente de la interfaz.
589	<code>botonDashboard.setSelected(true);</code>	Configura una propiedad mediante <code>setSelected</code> en el objeto indicado.
590	<code>botonDashboard.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
591	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
592	<code> botonDashboardActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
593	<code> }</code>	Cierra el bloque abierto anteriormente.
594	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
595	<code>panelLateral.add(botonDashboard, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 145, 296, 64));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
596		Línea en blanco: separa visualmente bloques para que el código sea más legible.
597	<code>botonUsuarios.setText("Gestión de Usuarios");</code>	Cambia el texto visible de un componente de la interfaz.
598	<code>botonUsuarios.setTipoIcono("USUARIOS");</code>	Configura una propiedad mediante <code>setTipoIcono</code> en el objeto indicado.
599	<code>botonUsuarios.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
600	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
601	<code> botonUsuariosActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
602	<code> }</code>	Cierra el bloque abierto anteriormente.
603	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
604	<code>panelLateral.add(botonUsuarios, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 230, 296, 64));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
605		Línea en blanco: separa visualmente bloques para que el código sea más legible.
606	<code>botonGestionMenu.setText("Gestión del Menú");</code>	Cambia el texto visible de un componente de la interfaz.
607	<code>botonGestionMenu.setTipoIcono("MENU");</code>	Configura una propiedad mediante <code>setTipoIcono</code> en el objeto indicado.
608	<code>botonGestionMenu.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
609	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
610	<code> botonGestionMenuActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
611	<code> }</code>	Cierra el bloque abierto anteriormente.
612	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
613	<code>panelLateral.add(botonGestionMenu, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 315, 296, 64));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
614		Línea en blanco: separa visualmente bloques para que el código sea más legible.
615	<code>botonInventario.setText("Inventario");</code>	Cambia el texto visible de un componente de la interfaz.

616	botonInventario.setTipoIcono("INVENTARIO");	Configura una propiedad mediante setTipolcono en el objeto indicado.
617	botonInventario.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
618	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
619	botonInventarioActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
620	}	Cierra el bloque abierto anteriormente.
621	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
622	panelLateral.add(botonInventario, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 400, 296, 64));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
623		Línea en blanco: separa visualmente bloques para que el código sea más legible.
624	botonReportes.setText("Reportes");	Cambia el texto visible de un componente de la interfaz.
625	botonReportes.setTipoIcono("REPORTES");	Configura una propiedad mediante setTipolcono en el objeto indicado.
626	botonReportes.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
627	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
628	botonReportesActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
629	}	Cierra el bloque abierto anteriormente.
630	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
631	panelLateral.add(botonReportes, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 485, 296, 64));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
632		Línea en blanco: separa visualmente bloques para que el código sea más legible.
633	botonCerrarSesion.setText("Volver Al Menú");	Cambia el texto visible de un componente de la interfaz.
634	botonCerrarSesion.setMostrarBorde(true);	Configura una propiedad mediante setMostrarBorde en el objeto indicado.
635	botonCerrarSesion.setTipoIcono("SALIR");	Configura una propiedad mediante setTipolcono en el objeto indicado.
636	botonCerrarSesion.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
637	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
638	botonCerrarSesionActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
639	}	Cierra el bloque abierto anteriormente.
640	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
641	panelLateral.add(botonCerrarSesion, new org.netbeans.lib.awtextra.AbsoluteConstraints(22, 955, 296, 62))	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
	;	
642		Línea en blanco: separa visualmente bloques para que el código sea más legible.
643	panelRaiz.add(panelLateral, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 340, 1032));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
644		Línea en blanco: separa visualmente bloques para que el código sea más legible.
645	panelCabecera.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
646	panelCabecera.setBorder(javax.swing.BorderFactory.createLineBorder(new java.awt.Color(204, 204, 204)));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
647	panelCabecera.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
649	labelHamburguesa.setFont(new java.awt.Font("Dialog", 0, 32)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
650	labelHamburguesa.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
651	labelHamburguesa.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
652	labelHamburguesa.setText("■");	Cambia el texto visible de un componente de la interfaz.
653	panelCabecera.add(labelHamburguesa, new org.netbeans.lib.awtextra.AbsoluteConstraints(25, 21, 55, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
654		Línea en blanco: separa visualmente bloques para que el código sea más legible.
655	labelTituloCabecera.setFont(new java.awt.Font("Dialog", 1, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
656	labelTituloCabecera.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
657	labelTituloCabecera.setText("PANEL DE ADMINISTRACION");	Cambia el texto visible de un componente de la interfaz.
658	panelCabecera.add(labelTituloCabecera, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 25, 510, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

659		Línea en blanco: separa visualmente bloques para que el código sea más legible.
660	labelIconoUsuario.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/IconoUsuario.png"))); // NOI18N	Crea una nueva instancia de javax.swing.Imagelcon y la usa o guarda para continuar el proceso.
661	labelIconoUsuario.setMantenerProporcion(true);	Configura una propiedad mediante setMantenerProporcion en el objeto indicado.
662	panelCabecera.add(labelIconoUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(1270, 22, 56, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	labelBienvenido.setFont(new java.awt.Font("Dialog", 0, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
665	labelBienvenido.setForeground(new java.awt.Color(18, 37, 63));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
666	labelBienvenido.setText("Bienvenido,");	Cambia el texto visible de un componente de la interfaz.
667	panelCabecera.add(labelBienvenido, new org.netbeans.lib.awtextra.AbsoluteConstraints(1340, 32, 105, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
668		Línea en blanco: separa visualmente bloques para que el código sea más legible.
669	labelUsuario.setFont(new java.awt.Font("Dialog", 1, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
670	labelUsuario.setForeground(new java.awt.Color(18, 37, 63));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
671	labelUsuario.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
672	labelUsuario.setText("Administrador");	Cambia el texto visible de un componente de la interfaz.
673	labelUsuario.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
674	panelCabecera.add(labelUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(1440, 15, 123, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
675		Línea en blanco: separa visualmente bloques para que el código sea más legible.
676	separadorCabecera.setForeground(new java.awt.Color(225, 229, 235));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
677	panelCabecera.add(separadorCabecera, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 99, 1580, 1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
678		Línea en blanco: separa visualmente bloques para que el código sea más legible.
679	panelRaiz.add(panelCabecera, new org.netbeans.lib.awtextra.AbsoluteConstraints(340, 0, 1580, 100));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
680		Línea en blanco: separa visualmente bloques para que el código sea más legible.
681	panelContenido.setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
682	panelContenido.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
683	panelRaiz.add(panelContenido, new org.netbeans.lib.awtextra.AbsoluteConstraints(340, 100, 1580, 932));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
684		Línea en blanco: separa visualmente bloques para que el código sea más legible.
685	getContentPane().add(panelRaiz, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1920, 1032));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
686		Línea en blanco: separa visualmente bloques para que el código sea más legible.
687	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
688	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
689		Línea en blanco: separa visualmente bloques para que el código sea más legible.
690	private void botonDashboardActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonDashboardActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
691	mostrarSeccion(DASHBOARD, botonDashboard);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
692	}//GEN-LAST:event_botonDashboardActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
693		Línea en blanco: separa visualmente bloques para que el código sea más legible.
694	private void botonUsuariosActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonUsuariosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
695	mostrarSeccion(USUARIOS, botonUsuarios);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
696	}//GEN-LAST:event_botonUsuariosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
697		Línea en blanco: separa visualmente bloques para que el código sea más legible.
698	private void botonGestionMenuActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonGestionMenuActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

699	mostrarSeccion(GESTION_MENU, botonGestionMenu);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
700	}//GEN-LAST:event_botonGestionMenuActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
701		Línea en blanco: separa visualmente bloques para que el código sea más legible.
702	private void botonInventarioActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_botonInventar ioActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
703	mostrarSeccion(INVENTARIO, botonInventario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
704	}//GEN-LAST:event_botonInventarioActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
705		Línea en blanco: separa visualmente bloques para que el código sea más legible.
706	private void botonReportesActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_botonReportesAc tionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
707	mostrarSeccion(REPORTES, botonReportes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
708	}//GEN-LAST:event_botonReportesActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
709		Línea en blanco: separa visualmente bloques para que el código sea más legible.
710	private void botonCerrarSesionActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_botonCerrar SesionActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
711	cerrarSesion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
712	}//GEN-LAST:event_botonCerrarSesionActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
713		Línea en blanco: separa visualmente bloques para que el código sea más legible.
714	public static void main(String args[]) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
715	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
716	for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
717	{	
717	if ("Nimbus".equals(info.getName())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
718	javax.swing.UIManager.setLookAndFeel(info.getClassName());	Configura una propiedad mediante setLookAndFeel en el objeto indicado.
719	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
720	}	Cierra el bloque abierto anteriormente.
721	}	Cierra el bloque abierto anteriormente.
722	} catch (ReflectiveOperationException javax.swing.UnsupportedLookAndFeelException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
723	java.util.logging.Logger.getLogger(MenuAdmin.class.getName())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
724	.log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
725	}	Cierra el bloque abierto anteriormente.
726	java.awt.EventQueue.invokeLater(() -> new MenuAdmin().setVisible(true));	Crea una nueva instancia de MenuAdmin y la usa o guarda para continuar el proceso.
727	}	Cierra el bloque abierto anteriormente.
728		Línea en blanco: separa visualmente bloques para que el código sea más legible.
729	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
730	private Componentes.BotonMenuLateral botonCerrarSesion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
731	private Componentes.BotonMenuLateral botonDashboard;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
732	private Componentes.BotonMenuLateral botonGestionMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
733	private Componentes.BotonMenuLateral botonInventario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
734	private Componentes.BotonMenuLateral botonReportes;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
735	private Componentes.BotonMenuLateral botonUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
736	private javax.swing.JLabel labelBienvenido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
737	private javax.swing.JLabel labelHamburguesa;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
738	private Labels.LabelEscalable labelIconoUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
739	private Labels.LabelEscalable labelLogo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

740	<code>private javax.swing.JLabel labelMarca;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
741	<code>private javax.swing.JLabel labelRol;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
742	<code>private javax.swing.JLabel labelTituloCabecera;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
743	<code>private javax.swing.JLabel labelUsuario;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
744	<code>private javax.swing.JPanel panelCabecera;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
745	<code>private javax.swing.JPanel panelContenido;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
746	<code>private javax.swing.JPanel panelLateral;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
747	<code>private javax.swing.JPanel panelRaiz;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
748	<code>private javax.swing.JSeparator separadorCabecera;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
749	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
750	<code>}</code>	Cierra el bloque abierto anteriormente.

16.76 src/GUI_ADMINISTRADOR/MovimientoInventarioFormDialog.java

Interfaz del módulo administrador: MovimientoInventarioFormDialog. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 46 | Archivo 76 de 157

Clases/tipos detectados: MovimientoInventarioFormDialog

Línea	Constructor o método detectado
15	private MovimientoInventarioFormDialog(Window propietario,
38	public static boolean mostrar(Window propietario, InventarioDAO dao,

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import DAO.InventarioDAO;	Importa DAO.InventarioDAO para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.ArticuloInventario;	Importa Modelos.ArticuloInventario para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	/** Ventana modal; su diseño está en MovimientoInventarioFormPanel.form. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	@SuppressWarnings("serial")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
11	public final class MovimientoInventarioFormDialog extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	private boolean guardado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	private MovimientoInventarioFormDialog(Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	InventarioDAO dao, ArticuloInventario articulo, String tipoFijo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	super(proprietario, ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
19	setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
20	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
21	MovimientoInventarioFormPanel formulario =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	new MovimientoInventarioFormPanel(dao, articulo, tipoFijo);	Crea una nueva instancia de MovimientoInventarioFormPanel y la usa o guarda para continuar el proceso.
23	formulario.addPropertyChangeListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	if (MovimientoInventarioFormPanel.EVENTO_GUARDADO.equals(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
25	evento.getPropertyName()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26	guardado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
27	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	} else if (MovimientoInventarioFormPanel.EVENTO_CANCELAR.equals(	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
29	evento.getPropertyName()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	}	Cierra el bloque abierto anteriormente.
32	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	setContentPane(formulario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

34	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	public static boolean mostrar(Window propietario, InventarioDAO dao,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	ArticuloInventario articulo, String tipoFijo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	MovimientoInventarioFormDialog dialogo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
41	new MovimientoInventarioFormDialog(	Crea una nueva instancia de MovimientoInventarioFormDialog y la usa o guarda para continuar el proceso.
42	propietario, dao, articulo, tipoFijo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
44	return dialogo.guardado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
45	}	Cierra el bloque abierto anteriormente.
46	}	Cierra el bloque abierto anteriormente.

16.77 src/GUI_ADMINISTRADOR/MovimientoInventarioFormPanel.form

Metadatos visuales de NetBeans para diseñar MovimientoInventarioFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 33 | Archivo 77 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties><Property name="opaque" type="boolean" value="false"/><Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor"><Dimension value="[700, 500]"/></Property></Properties>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
5	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
6	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/><AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/><AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/><AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/><AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</AuxValues>	Cierra la sección XML abierta anteriormente.
8	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<Container class="Componentes.PanelFlotante" name="panelTarjeta">	Declara un componente o contenedor visual y su clase Java.
11	<Properties><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="ea" green="e3" red="de" type="rgb"/></Property><Property name="radio" type="int" value="26"/></Properties>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
12	<Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="0" y="0" width="700" height="500"/></Constraint></Constraints>	Define la posición y el tamaño del componente dentro del diseño.
13	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
14	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<Container class="Componentes.PanelCircular" name="panelIcono"><Properties><Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="d3" green="f4" red="ff" type="rgb"/></Property></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="34" y="26" width="70" height="70"/></Constraint></Constraints><Layout class="org.netbeans.modules.form.compat2.layouts.DesignBorderLayout"><SubComponents><Component class="javax.swing.JLabel" name="labelIcono"><Properties><Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor"></Property><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="2b" green="14" red="0" type="rgb"/></Property><Property name="horizontalAlignment" type="int" value="0"/><Property name="text" type="java.lang.String" value="±"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignBorderLayout" value="org.netbeans.modules.form.compat2.layouts.DesignBorderLayout\$BorderConstraintsDescription"><BorderConstraints direction="Center"/></Constraint></Constraints></SubComponents></Container>	Declara un componente o contenedor visual y su clase Java.
16	<Component class="javax.swing.JLabel" name="labelTitulo"><Properties><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="2b" green="14" red="0" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Movimiento de inventario"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="125" y="27" width="535" height="34"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.

17	<pre><Component class="javax.swing.JLabel" name="labelSubtitulo"><Properties><Property name="text" type="java.lang.String" value="Art&#x26;#xed;culo seleccionado"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="125" y="62" width="535" height="26"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
18	<pre><Component class="javax.swing.JLabel" name="labelTipo"><Properties><Property name="text" type="java.lang.String" value="Tipo de movimiento"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="120" width="315" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
19	<pre><Component class="Componentes.BotonDesplegable" name="selectorTipo"><Properties><Property name="text" type="java.lang.String" value="Entrada"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="148" width="315" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
20	<pre><Component class="javax.swing.JLabel" name="labelStock"><Properties><Property name="text" type="java.lang.String" value="Stock actual"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="380" y="120" width="285" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
21	<pre><Component class="javax.swing.JTextField" name="campoStock"><Properties><Property name="editable" type="boolean" value="false"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="380" y="148" width="285" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
22	<pre><Component class="javax.swing.JLabel" name="labelCantidad"><Properties><Property name="text" type="java.lang.String" value="Cantidad"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="220" width="315" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
23	<pre><Component class="javax.swing.JTextField" name="campoCantidad"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="248" width="315" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
24	<pre><Component class="javax.swing.JLabel" name="labelMotivo"><Properties><Property name="text" type="java.lang.String" value="Motivo (opcional)"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="380" y="220" width="285" height="24"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
25	<pre><Component class="javax.swing.JTextField" name="campoMotivo"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="380" y="248" width="285" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
26	<pre><Component class="javax.swing.JLabel" name="labelInstruccion"><Properties><Property name="text" type="java.lang.String" value="Escribe cu&#x26;#xed;to se agrega del inventario."/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="315" width="630" height="28"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
27	<pre><Component class="javax.swing.JLabel" name="labelError"><Properties><Property name="text" type="java.lang.String" value=""/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="35" y="365" width="630" height="30"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.

28	<pre><Component class="Componentes.BotonRedondeado" name="botonCancelar"><Properties><Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editor.s.ColorEditor"><Color blue="fff" green="fff" red="fff" type="rgb"/></Property><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="degradado" type="boolean" value="false"/><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="41" green="37" red="e7" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Cancelar"/></Properties><Events><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonCancelarActionPerformed"/></Events><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="305" y="425" width="130" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
29	<pre><Component class="Componentes.BotonDerretido" name="botonGuardar"><Properties><Property name="text" type="java.lang.String" value="Guardar movimiento"/></Properties><Events><EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonGuardarActionPerformed"/></Events><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="450" y="425" width="215" height="46"/></Constraint></Constraints></Component></pre>	Declara un componente o contenedor visual y su clase Java.
30	<pre></SubComponents></pre>	Cierra la sección XML abierta anteriormente.
31	<pre></Container></pre>	Cierra la sección XML abierta anteriormente.
32	<pre></SubComponents></pre>	Cierra la sección XML abierta anteriormente.
33	<pre></Form></pre>	Cierra la sección XML abierta anteriormente.

16.78 src/GUI_ADMINISTRADOR/MovimientoInventarioFormPanel.java

Interfaz del módulo administrador: MovimientoInventarioFormPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 265 | Archivo 78 de 157

Clases/tipos detectados: MovimientoInventarioFormPanel

Línea	Constructor o método detectado
31	public MovimientoInventarioFormPanel() {
35	public MovimientoInventarioFormPanel(InventarioDAO dao,
45	private void configurarAspecto() {
71	private void configurarSelector(String tipoFijo) {
92	private void seleccionarTipo(String etiqueta) {
106	private String etiquetaTipo(String tipo) {
114	private void cargarArticulo() {
128	private void guardar() {
165	private void initComponents() {
238	private void botonCancelarActionPerformed(java.awt.event.ActionEvent evt) {
242	private void botonGuardarActionPerformed(java.awt.event.ActionEvent evt) {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import DAO.InventarioDAO;	Importa DAO.InventarioDAO para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.ArticuloInventario;	Importa Modelos.ArticuloInventario para poder usar esa clase sin escribir su nombre completo.
5	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
9	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
10	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** Formulario visual para registrar entradas, salidas y ajustes. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
15	public class MovimientoInventarioFormPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
16		Línea en blanco: separa visualmente bloques para que el código sea más legible.
17	public static final String EVENTO_GUARDADO = "movimientoGuardado";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18	public static final String EVENTO_CANCELAR = "cancelarMovimiento";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
21	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

22	private static final Color BORDE = new Color(222, 227, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
23	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
24	private static final BigDecimal MAXIMO = new BigDecimal("99999999.99");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
27	private InventarioDAO dao;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	private ArtículoInventario articulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
29	private String tipoSeleccionado = "ENTRADA";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	public MovimientoInventarioFormPanel() {	Constructor de MovimientoInventarioFormPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
32	this(null, null, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	}	Cierra el bloque abierto anteriormente.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	public MovimientoInventarioFormPanel(InventarioDAO dao,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	ArtículoInventario articulo, String tipoFijo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	this.dao = dao;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	this.articulo = articulo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
39	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	configurarAspecto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	configurarSelector(tipoFijo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	cargarArticulo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	private void configurarAspecto() {	Declara el método configurarAspecto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
46	labelTitulo.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
47	labelSubtitulo.setFont(tema.regular(15f));	Configura una propiedad mediante setFont en el objeto indicado.
48	labelSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
49	labelError.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
50	labelError.setForeground(ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
51	labelInstruccion.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
52	labelInstruccion.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
53	for (javax.swing.JLabel etiqueta : new javax.swing.JLabel[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
54	labelTipo, labelCantidad, labelMotivo, labelStock}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	etiqueta.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
56	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
57	}	Cierra el bloque abierto anteriormente.
58	for (JTextField campo : new JTextField[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
59	campoCantidad, campoMotivo, campoStock}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
61	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
62	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
63	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	new EmptyBorder(0, 13, 0, 13)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
65	}	Cierra el bloque abierto anteriormente.

66	<code>campoStock.setBackground(new Color(248, 249, 251));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
67	<code>botonCancelar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
68	<code>botonGuardar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
69	<code>}</code>	Cierra el bloque abierto anteriormente.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	<code>private void configurarSelector(String tipoFijo) {</code>	Declara el método configurarSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
72	<code>selectorTipo.setFont(tema.regular(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
73	<code>selectorTipo.setForeground(AZUL);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
74	<code>selectorTipo.setColorFondo(Color.WHITE);</code>	Configura una propiedad mediante setColorFondo en el objeto indicado.
75	<code>selectorTipo.setColorHover(new Color(248, 249, 251));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
76	<code>selectorTipo.setColorDesplegado(new Color(255, 247, 222));</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
77	<code>selectorTipo.setColorTextoOpcion(AZUL);</code>	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
78	<code>selectorTipo.setColorBordeMenu(BORDE);</code>	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
79	<code>selectorTipo.setAnchoMenu(315);</code>	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
80	<code>selectorTipo.setAltoOpcion(42);</code>	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
81	<code>selectorTipo.addMenuOpcionListener(evento -> {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	<code>seleccionarTipo(evento.getActionCommand());</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	<code>if (tipoFijo != null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
85	<code>seleccionarTipo(etiquetaTipo(tipoFijo));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	<code>selectorTipo.setEnabled(false);</code>	Habilita o deshabilita la interacción con el componente.
87	<code>} else {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
88	<code>seleccionarTipo("Entrada");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	<code>}</code>	Cierra el bloque abierto anteriormente.
90	<code>}</code>	Cierra el bloque abierto anteriormente.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	<code>private void seleccionarTipo(String etiqueta) {</code>	Declara el método seleccionarTipo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
93	<code>selectorTipo.setText(etiqueta);</code>	Cambia el texto visible de un componente de la interfaz.
94	<code>tipoSeleccionado = switch (etiqueta) {</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
95	<code>case "Salida" -> "SALIDA";</code>	Define una de las rutas posibles dentro del switch.
96	<code>case "Ajuste" -> "AJUSTE";</code>	Define una de las rutas posibles dentro del switch.
97	<code>default -> "ENTRADA";</code>	Define una de las rutas posibles dentro del switch.
98	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
99	<code>labelInstruccion.setText("AJUSTE".equals(tipoSeleccionado)</code>	Cambia el texto visible de un componente de la interfaz.
100	<code>? "En Ajuste, escribe el stock final que debe quedar."</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	<code>: "Escribe cuánto se "</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
102	<code>+ ("SALIDA".equals(tipoSeleccionado) ? "retira" : "agrega")</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
103	<code>+ " del inventario.");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	<code>}</code>	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	<code>private String etiquetaTipo(String tipo) {</code>	Declara el método etiquetaTipo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
107	<code>return switch (tipo) {</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
108	<code>case "SALIDA" -> "Salida";</code>	Define una de las rutas posibles dentro del switch.
109	<code>case "AJUSTE" -> "Ajuste";</code>	Define una de las rutas posibles dentro del switch.

110	default -> "Entrada";	Define una de las rutas posibles dentro del switch.
111	};	Cierra el bloque abierto anteriormente y termina la declaración.
112	}	Cierra el bloque abierto anteriormente.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	private void cargarArticulo() {	Declara el método cargarArticulo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
115	if (articulo == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
116	labelSubtitulo.setText("Selecciona un artículo desde Existencias");	Cambia el texto visible de un componente de la interfaz.
117	campoStock.setText("-");	Cambia el texto visible de un componente de la interfaz.
118	botonGuardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
119	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
120	}	Cierra el bloque abierto anteriormente.
121	labelSubtitulo.setText(articulo.getCodigoVisible() + " . "	Cambia el texto visible de un componente de la interfaz.
122	+ articulo.getNombre());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	campoStock.setText(articulo.getStockActual().stripTrailingZeros()	Cambia el texto visible de un componente de la interfaz.
124	.toString() + " " + articulo.getUnidad());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	labelCantidad.setText("Cantidad (" + articulo.getUnidad() + ")");	Cambia el texto visible de un componente de la interfaz.
126	}	Cierra el bloque abierto anteriormente.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
129	labelError.setText(" ");	Cambia el texto visible de un componente de la interfaz.
130	if (dao == null articulo == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
131	labelError.setText("No hay un artículo seleccionado.");	Cambia el texto visible de un componente de la interfaz.
132	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
133	}	Cierra el bloque abierto anteriormente.
134	BigDecimal cantidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
135	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
136	cantidad = new BigDecimal(campoCantidad.getText().trim());	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
137	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
138	labelError.setText("Ingresa una cantidad válida. Usa punto decimal.");	Cambia el texto visible de un componente de la interfaz.
139	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
140	}	Cierra el bloque abierto anteriormente.
141	boolean ajuste = "AJUSTE".equals(tipoSeleccionado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
142	if (cantidad.scale() > 2 cantidad.compareTo(MAXIMO) > 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
143	(ajuste ? cantidad.signum() < 0 : cantidad.signum() <= 0)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
144	labelError.setText("Cantidad inválida; usa hasta dos decimales.");	Cambia el texto visible de un componente de la interfaz.
145	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
146	}	Cierra el bloque abierto anteriormente.
147	String motivo = campoMotivo.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
148	if (motivo.length() > 150) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
149	labelError.setText("El motivo admite hasta 150 caracteres.");	Cambia el texto visible de un componente de la interfaz.
150	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
151	}	Cierra el bloque abierto anteriormente.
152	botonGuardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
153	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

154	dao.registrarMovimiento(articulo, tipoSeleccionado, cantidad, motivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156	} catch (SQLException IllegalArgumentException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
157	labelError.setText("No fue posible registrar: " + ex.getMessage());	Cambia el texto visible de un componente de la interfaz.
158	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
159	botonGuardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
160	}	Cierra el bloque abierto anteriormente.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
164	// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
165	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
166	panelTarjeta = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
167	panelIcono = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
168	labelIcono = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
169	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
170	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
171	labelTipo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
172	selectorTipo = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
173	labelStock = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
174	campoStock = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
175	labelCantidad = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
176	campoCantidad = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
177	labelMotivo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
178	campoMotivo = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
179	labelInstruccion = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
180	labelError = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
181	botonCancelar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
182	botonGuardar = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	setPreferredSize(new java.awt.Dimension(700, 500));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
186	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
187	panelTarjeta.setColorBorde(new java.awt.Color(222, 227, 234));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
188	panelTarjeta.setRadio(26);	Configura una propiedad mediante setRadio en el objeto indicado.
189	panelTarjeta.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
190	panelIcono.setColorFondo(new java.awt.Color(255, 244, 211));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
191	panelIcono.setLayout(new java.awt.BorderLayout());	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
192	labelIcono.setFont(new java.awt.Font("Dialog", 1, 31));	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
193	labelIcono.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
194	labelIcono.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
195	labelIcono.setText("±");	Cambia el texto visible de un componente de la interfaz.
196	panelIcono.add(labelIcono, java.awt.BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	panelTarjeta.add(panelIcono, new org.netbeans.lib.awtextra.AbsoluteConstraints(34, 26, 70, 70));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

198	labelTitulo.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
199	labelTitulo.setText("Movimiento de inventario");	Cambia el texto visible de un componente de la interfaz.
200	panelTarjeta.add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(125, 27, 535, 34));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
201	labelSubtitulo.setText("Artículo seleccionado");	Cambia el texto visible de un componente de la interfaz.
202	panelTarjeta.add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(125, 62, 535, 26));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	labelTipo.setText("Tipo de movimiento");	Cambia el texto visible de un componente de la interfaz.
205	panelTarjeta.add(labelTipo, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 120, 315, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
206	selectorTipo.setText("Entrada");	Cambia el texto visible de un componente de la interfaz.
207	selectorTipo.setTextoDesplegable("Entrada;Salida;Ajuste");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
208	panelTarjeta.add(selectorTipo, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 148, 315, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
209	labelStock.setText("Stock actual");	Cambia el texto visible de un componente de la interfaz.
210	panelTarjeta.add(labelStock, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 120, 285, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
211	campoStock.setEditable(false);	Configura una propiedad mediante setEditable en el objeto indicado.
212	panelTarjeta.add(campoStock, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 148, 285, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
213	labelCantidad.setText("Cantidad");	Cambia el texto visible de un componente de la interfaz.
214	panelTarjeta.add(labelCantidad, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 220, 315, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
215	panelTarjeta.add(campoCantidad, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 248, 315, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
216	labelMotivo.setText("Motivo (opcional)");	Cambia el texto visible de un componente de la interfaz.
217	panelTarjeta.add(labelMotivo, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 220, 285, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
218	panelTarjeta.add(campoMotivo, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 248, 285, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
219	labelInstruccion.setText("Escribe cuánto se agrega del inventario.");	Cambia el texto visible de un componente de la interfaz.
220	panelTarjeta.add(labelInstruccion, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 315, 630, 28));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
221	labelError.setText(" ");	Cambia el texto visible de un componente de la interfaz.
222	panelTarjeta.add(labelError, new org.netbeans.lib.awtextra.AbsoluteConstraints(35, 365, 630, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	botonCancelar.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
225	botonCancelar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
226	botonCancelar.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
227	botonCancelar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
228	botonCancelar.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
229	botonCancelar.setText("Cancelar");	Cambia el texto visible de un componente de la interfaz.
230	botonCancelar.addActionListener(e -> botonCancelarActionPerformed(e));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
231	panelTarjeta.add(botonCancelar, new org.netbeans.lib.awtextra.AbsoluteConstraints(305, 425, 130, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
232	botonGuardar.setText("Guardar movimiento");	Cambia el texto visible de un componente de la interfaz.
233	botonGuardar.addActionListener(e -> botonGuardarActionPerformed(e));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
234	panelTarjeta.add(botonGuardar, new org.netbeans.lib.awtextra.AbsoluteConstraints(450, 425, 215, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
235	add(panelTarjeta, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 700, 500));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
236	}	Cierra el bloque abierto anteriormente.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	private void botonCancelarActionPerformed(java.awt.event.ActionEvent evt) {	Declara el método botonCancelarActionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
239	firePropertyChange(EVENTO_CANCELAR, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240	}	Cierra el bloque abierto anteriormente.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.

242	<code>private void botonGuardarActionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>botonGuardarActionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
243	<code> guardar();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
244	<code>}// </editor-fold>//GEN-END initComponents</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	<code>// Variables declaration - do not modify//GEN-BEGIN:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
247	<code>private Componentes.BotonRedondeado botonCancelar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
248	<code>private Componentes.BotonDerretido botonGuardar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
249	<code>private javax.swing.JTextField campoCantidad;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
250	<code>private javax.swing.JTextField campoMotivo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
251	<code>private javax.swing.JTextField campoStock;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
252	<code>private javax.swing.JLabel labelCantidad;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
253	<code>private javax.swing.JLabel labelError;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
254	<code>private javax.swing.JLabel labelIcono;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
255	<code>private javax.swing.JLabel labelInstruccion;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
256	<code>private javax.swing.JLabel labelMotivo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
257	<code>private javax.swing.JLabel labelStock;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
258	<code>private javax.swing.JLabel labelSubtitulo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
259	<code>private javax.swing.JLabel labelTipo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
260	<code>private javax.swing.JLabel labelTitulo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
261	<code>private Componentes.PanelCircular panelIcono;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
262	<code>private Componentes.PanelFlotante panelTarjeta;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
263	<code>private Componentes.BotonDesplegable selectorTipo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
264	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
265	<code>}</code>	Cierra el bloque abierto anteriormente.

16.79 src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.form

Metadatos visuales de NetBeans para diseñar MovimientosInventarioPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 20 | Archivo 79 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties><Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="fb" green="f9" red="f8" type="rgb"/></Property><Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor"><Dimension value="[1530, 932]"/></Property></Properties>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
5	<AuxValues><AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="6"/><AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/><AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/><AuxValue name="FormSettings_listenGenerationStyle" type="java.lang.Integer" value="0"/><AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/><AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/></AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
6	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
7	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	<Component class="javax.swing.JLabel" name="labelTitulo"><Properties><Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="2b" green="14" red="0" type="rgb"/></Property><Property name="text" type="java.lang.String" value="Movimientos de Inventario"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="60" y="16" width="700" height="64"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
9	<Component class="javax.swing.JLabel" name="labelSubtitulo"><Properties><Property name="text" type="java.lang.String" value="Consulta entradas, salidas y ajustes registrados"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="60" y="76" width="650" height="36"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
10	<Component class="Componentes.BotonDerretido" name="botonActualizar"><Properties><Property name="text" type="java.lang.String" value="Actualizar historial"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="1290" y="42" width="220" height="60"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
11	<Container class="Componentes.PanelFlotante" name="panelFiltros"><Properties><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="ed" green="e6" red="e1" type="rgb"/></Property><Property name="radio" type="int" value="18"/><Property name="sombra" type="boolean" value="false"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="60" y="175" width="1470" height="95"/></Constraint></Constraints><Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout><SubComponents>	Declara un componente o contenedor visual y su clase Java.
12	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda"><Properties><Property name="placeholder" type="java.lang.String" value="Buscar artículo, motivo o ID..."/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="20" y="20" width="750" height="55"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
13	<Component class="Componentes.BotonDesplegable" name="filtroTipo"><Properties><Property name="text" type="java.lang.String" value="Todos los movimientos"/><Property name="textoDesplegable" type="java.lang.String" value="Todos los movimientos;Entradas;Salidas;Ajustes"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="1160" y="22" width="280" height="52"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.

14	</SubComponents></Container>	Cierra la sección XML abierta anteriormente.
15	<Container class="Componentes.PanelFlotante" name="panelTabla"><Properties><Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"><Color blue="ed" green="e6" red="e1" type="rgb"/></Property><Property name="radio" type="int" value="18"/><Property name="sombra" type="boolean" value="false"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="60" y="285" width="1470" height="640"/></Constraint></Constraints><Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout"><Property name="useNullLayout" type="boolean" value="false"/></Layout><SubComponents>	Declara un componente o contenedor visual y su clase Java.
16	<Container class="javax.swing.JScrollPane" name="scrollTabla"><Properties><Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor"><Border info="null"/></Property><Property name="verticalScrollBarPolicy" type="int" value="21"/><Property name="horizontalScrollBarPolicy" type="int" value="31"/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="14" y="14" width="1442" height="555"/></Constraint></Constraints><Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/><SubComponents><Component class="Componentes.TablaAdministrativa" name="tablaMovimientos"><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout" value="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout\$JScrollPaneConstraintsDescription"/></Constraints></Component></SubComponents></Container>	Declara un componente o contenedor visual y su clase Java.
17	<Component class="javax.swing.JLabel" name="etiquetaRango"><Properties><Property name="text" type="java.lang.String" value="Cargando movimientos..."/></Properties><Constraints><Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription"><AbsoluteConstraints x="28" y="581" width="800" height="42"/></Constraint></Constraints></Component>	Declara un componente o contenedor visual y su clase Java.
18	</SubComponents></Container>	Cierra la sección XML abierta anteriormente.
19	</SubComponents>	Cierra la sección XML abierta anteriormente.
20	</Form>	Cierra la sección XML abierta anteriormente.

16.80 src/GUI_ADMINISTRADOR/MovimientosInventarioPanel.java

Historial de entradas, salidas y ajustes de inventario.

Tipo: java | Líneas: 257 | Archivo 80 de 157

Clases/tipos detectados: MovimientosInventarioPanel, ModeloMovimientos

Línea	Constructor o método detectado
39	public MovimientosInventarioPanel() {
48	public void recargarDatos() {
52	private void prepararVista() {
73	private void configurarFiltros() {
92	private void prepararSelector(BotonDesplegable selector) {
104	private void cargarMovimientos() {
132	private void aplicarFiltros() {
149	private boolean contiene(String valor, String texto) {
153	private String cantidadVisible(MovimientoInventario movimiento) {
197	private void initComponents() {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
4	import DAO.InventarioDAO;	Importa DAO.InventarioDAO para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.MovimientoInventario;	Importa Modelos.MovimientoInventario para poder usar esa clase sin escribir su nombre completo.
6	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
8	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
9	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
10	import java.time.format.DateTimeFormatter;	Importa java.time.format.DateTimeFormatter para poder usar esa clase sin escribir su nombre completo.
11	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
12	import java.util.Locale;	Importa java.util.Locale para poder usar esa clase sin escribir su nombre completo.
13	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
17	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
18	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
20	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	/** Historial de movimientos integrado al módulo Inventario. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

24	<code>public class MovimientosInventarioPanel extends javax.swing.JPanel {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	<code>private static final Color AZUL = new Color(0, 20, 43);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
27	<code>private static final Color SECUNDARIO = new Color(92, 103, 124);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
28	<code>private static final Color BORDE = new Color(225, 230, 237);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
29	<code>private static final DateFormatter FECHA =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
30	<code> DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	<code>private final TemaAdmin tema = new TemaAdmin();</code>	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
33	<code>private final InventarioDAO dao = new InventarioDAO();</code>	Crea una nueva instancia de InventarioDAO y la usa o guarda para continuar el proceso.
34	<code>private final ModeloMovimientos modelo = new ModeloMovimientos();</code>	Crea una nueva instancia de ModeloMovimientos y la usa o guarda para continuar el proceso.
35	<code>private final Timer temporizadorFiltro;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
36	<code>private List<MovimientoInventario> movimientos = List.of();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	<code>private String tipoActual;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	<code>public MovimientosInventarioPanel() {</code>	Constructor de MovimientosInventarioPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
40	<code> initComponents();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	<code> temporizadorFiltro = new Timer(250, e -> aplicarFiltros());</code>	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
42	<code> temporizadorFiltro.setRepeats(false);</code>	Configura una propiedad mediante setRepeats en el objeto indicado.
43	<code> prepararVista();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	<code> configurarFiltros();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	<code> cargarMovimientos();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	<code>}</code>	Cierra el bloque abierto anteriormente.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	<code>public void recargarDatos() {</code>	Declara el método recargarDatos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
49	<code> cargarMovimientos();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	<code>}</code>	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	<code>private void prepararVista() {</code>	Declara el método prepararVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	<code> labelTitulo.setFont(tema.negrita(42f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
54	<code> labelSubtitulo.setFont(tema.regular(16f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
55	<code> labelSubtitulo.setForeground(SECUNDARIO);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
56	<code> botonActualizar.setFont(tema.negrita(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
57	<code> campoBusqueda.setFont(tema.regular(14f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
58	<code> etiquetaRango.setFont(tema.regular(12f));</code>	Configura una propiedad mediante setFont en el objeto indicado.
59	<code> etiquetaRango.setForeground(SECUNDARIO);</code>	Configura una propiedad mediante setForeground en el objeto indicado.
60	<code> scrollTabla.setBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante setBorder en el objeto indicado.
61	<code> scrollTabla.getViewport().setBackground(Color.WHITE);</code>	Configura una propiedad mediante setBackground en el objeto indicado.
62	<code> tablaMovimientos.setModel(modelo);</code>	Configura una propiedad mediante setModel en el objeto indicado.
63	<code> tablaMovimientos.aplicarEstilo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	<code> tablaMovimientos.setRowHeight(49);</code>	Configura una propiedad mediante setRowHeight en el objeto indicado.
65	<code> tablaMovimientos.setColumnsCentradas("0,1,3,4,5,7");</code>	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
66	<code> int[] anchos = {75, 145, 275, 145, 125, 105, 360, 85};</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	<code> for (int i = 0; i < anchos.length; i++) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.

68	tablaMovimientos.getColumnModel().getColumn(i).setPreferredWidth(anchos[i]);	Configura una propiedad mediante setPreferredWidth en el objeto indicado.
69	}	Cierra el bloque abierto anteriormente.
70	prepararSelector(filtroTipo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	}	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	private void configurarFiltros() {	Declara el método configurarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
74	filtroTipo.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75	filtroTipo.setText(e.getActionCommand());	Cambia el texto visible de un componente de la interfaz.
76	tipoActual = switch (e.getActionCommand()) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
77	case "Entradas" -> "ENTRADA";	Define una de las rutas posibles dentro del switch.
78	case "Salidas" -> "SALIDA";	Define una de las rutas posibles dentro del switch.
79	case "Ajustes" -> "AJUSTE";	Define una de las rutas posibles dentro del switch.
80	default -> null;	Define una de las rutas posibles dentro del switch.
81	};	Cierra el bloque abierto anteriormente y termina la declaración.
82	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
85	@Override public void insertUpdate(DocumentEvent e) { temporizadorFiltro.restart(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
86	@Override public void removeUpdate(DocumentEvent e) { temporizadorFiltro.restart(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
87	@Override public void changedUpdate(DocumentEvent e) { temporizadorFiltro.restart(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
88	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	botonActualizar.addActionListener(e -> cargarMovimientos());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
90	}	Cierra el bloque abierto anteriormente.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	private void prepararSelector(BotonDesplegable selector) {	Declara el método prepararSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
93	selector.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
94	selector.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
95	selector.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
96	selector.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
97	selector.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
98	selector.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
99	selector.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
100	selector.setAnchoMenu(280);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
101	selector.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
102	}	Cierra el bloque abierto anteriormente.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	private void cargarMovimientos() {	Declara el método cargarMovimientos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
105	botonActualizar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
106	etiquetaRango.setText("Cargando movimientos...");	Cambia el texto visible de un componente de la interfaz.
107	new SwingWorker<List<MovimientoInventario>, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
108	@Override protected List<MovimientoInventario> doInBackground() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
109	throws SQLException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
110	return dao.listarMovimientos(300);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
111	}	Cierra el bloque abierto anteriormente.

112	<code>@Override protected void done() {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
113	<code> botonActualizar.setEnabled(true);</code>	Habilita o deshabilita la interacción con el componente.
114	<code> try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
115	<code> movimientos = List.copyOf(get());</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
116	<code> aplicarFiltros();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	<code> } catch (InterruptedException ex) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
118	<code> Thread.currentThread().interrupt();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	<code> } catch (ExecutionException ex) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
120	<code> movimientos = List.of();</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
121	<code> modelo.setMovimientos(movimientos);</code>	Configura una propiedad mediante setMovimientos en el objeto indicado.
122	<code> etiquetaRango.setText("No fue posible cargar el historial");</code>	Cambia el texto visible de un componente de la interfaz.
123	<code> Throwable causa = ex.getCause();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
124	<code> JOptionPane.showMessageDialog(MovimientosInventarioPanel.this,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125	<code> causa == null ? "Error no especificado." : causa.getMessage(),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	<code> "Movimientos de inventario", JOptionPane.ERROR_MESSAGE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	<code> }</code>	Cierra el bloque abierto anteriormente.
128	<code>}</code>	Cierra el bloque abierto anteriormente.
129	<code>}.execute();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	<code>}</code>	Cierra el bloque abierto anteriormente.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	<code>private void aplicarFiltros() {</code>	Declara el método aplicarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
133	<code> String texto = campoBusqueda.getText().trim().toLowerCase(Locale.ROOT);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
134	<code> List<MovimientoInventario> filtrados = movimientos.stream()</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
135	<code> .filter(m -> tipoActual == null</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
136	<code> tipoActual.equals(m.tipoMovimiento()))</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	<code> .filter(m -> texto.isEmpty()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
138	<code> contiene(m.articulo(), texto)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	<code> contiene(m.tipoArticulo(), texto)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140	<code> contiene(m.motivo(), texto)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
141	<code> String.valueOf(m.id()).contains(texto))</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
142	<code> .toList();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	<code> modelo.setMovimientos(filtrados);</code>	Configura una propiedad mediante setMovimientos en el objeto indicado.
144	<code> etiquetaRango.setText(filtrados.size() + " movimiento"</code>	Cambia el texto visible de un componente de la interfaz.
145	<code> + (filtrados.size() == 1 ? "" : "s") + " mostrado"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
146	<code> + (filtrados.size() == 1 ? "" : "s"));</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	<code> }</code>	Cierra el bloque abierto anteriormente.
148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
149	<code>private boolean contiene(String valor, String texto) {</code>	Declara el método contiene. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
150	<code> return valor != null && valor.toLowerCase(Locale.ROOT).contains(texto);</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
151	<code>}</code>	Cierra el bloque abierto anteriormente.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	<code>private String cantidadVisible(MovimientoInventario movimiento) {</code>	Declara el método cantidadVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
154	<code> BigDecimal cantidad = movimiento.cantidad();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
155	<code> if ("SALIDA".equals(movimiento.tipoMovimiento())) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

156	cantidad = cantidad.abs().negate();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
157	}	Cierra el bloque abierto anteriormente.
158	return (cantidad.signum() > 0 ? "+" : "")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
159	+ cantidad.stripTrailingZeros().toString();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
160	}	Cierra el bloque abierto anteriormente.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	private final class ModeloMovimientos extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
163	private final String[] columnas = {"ID", "Fecha", "Artículo", "Tipo",	Declara una constante o referencia que no puede reasignarse después de inicializarse.
164	"Movimiento", "Cantidad", "Motivo", "Pedido"};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165	private List<MovimientoInventario> datos = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
166	void setMovimientos(List<MovimientoInventario> nuevos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	datos = List.copyOf(nuevos);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
168	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	}	Cierra el bloque abierto anteriormente.
170	@Override public int getRowCount() { return datos.size(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
171	@Override public int getColumnCount() { return columnas.length; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
172	@Override public String getColumnName(int columna) { return columnas[columna]; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
173	@Override public boolean isCellEditable(int fila, int columna) { return false; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
174	@Override public Object getValueAt(int fila, int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
175	MovimientoInventario item = datos.get(fila);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
176	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
177	case 0 -> "#" + item.id();	Define una de las rutas posibles dentro del switch.
178	case 1 -> item.fecha() == null ? "-" : FECHA.format(item.fecha());	Define una de las rutas posibles dentro del switch.
179	case 2 -> item.articulo();	Define una de las rutas posibles dentro del switch.
180	case 3 -> item.tipoArticulo();	Define una de las rutas posibles dentro del switch.
181	case 4 -> switch (item.tipoMovimiento()) {	Define una de las rutas posibles dentro del switch.
182	case "ENTRADA" -> "Entrada";	Define una de las rutas posibles dentro del switch.
183	case "SALIDA" -> "Salida";	Define una de las rutas posibles dentro del switch.
184	default -> "Ajuste";	Define una de las rutas posibles dentro del switch.
185	};	Cierra el bloque abierto anteriormente y termina la declaración.
186	case 5 -> cantidadVisible(item);	Define una de las rutas posibles dentro del switch.
187	case 6 -> item.motivo() == null item.motivo().isBlank()	Define una de las rutas posibles dentro del switch.
188	? "-" : item.motivo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189	case 7 -> item.idPedido() == null ? "-" : "#" + item.idPedido();	Define una de las rutas posibles dentro del switch.
190	default -> "";	Define una de las rutas posibles dentro del switch.
191	};	Cierra el bloque abierto anteriormente y termina la declaración.
192	}	Cierra el bloque abierto anteriormente.
193	}	Cierra el bloque abierto anteriormente.
194		Línea en blanco: separa visualmente bloques para que el código sea más legible.
195	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
196	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
197	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
198	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
199	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.

200	botonActualizar = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
201	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
202	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
203	filtroTipo = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
204	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
205	scrollTabla = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
206	tablaMovimientos = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
207	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
208		Línea en blanco: separa visualmente bloques para que el código sea más legible.
209	setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
210	setPreferredSize(new java.awt.Dimension(1530, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
211	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
212	labelTitulo.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
213	labelTitulo.setText("Movimientos de Inventario");	Cambia el texto visible de un componente de la interfaz.
214	add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 16, 700, 64));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
215	labelSubtitulo.setText("Consulta entradas, salidas y ajustes registrados");	Cambia el texto visible de un componente de la interfaz.
216	add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 76, 650, 36));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
217	botonActualizar.setText("Actualizar historial");	Cambia el texto visible de un componente de la interfaz.
218	add(botonActualizar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1290, 42, 220, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	panelFiltros.setColorBorde(new java.awt.Color(225, 230, 237));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
221	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
222	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
223	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
224	campoBusqueda.setPlaceholder("Buscar artículo, motivo o ID...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
225	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 20, 750, 55));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
226	filtroTipo.setText("Todos los movimientos");	Cambia el texto visible de un componente de la interfaz.
227	filtroTipo.setTextoDesplegable("Todos los movimientos;Entradas;Salidas;Ajustes");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
228	panelFiltros.add(filtroTipo, new org.netbeans.lib.awtextra.AbsoluteConstraints(1160, 22, 280, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
229	add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 175, 1470, 95));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	panelTabla.setColorBorde(new java.awt.Color(225, 230, 237));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
232	panelTabla.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
233	panelTabla.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
234	panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
235	scrollTabla.setBorder(null);	Configura una propiedad mediante setBorder en el objeto indicado.
236	scrollTabla.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
237	scrollTabla.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
238	scrollTabla.setViewportViewView(tablaMovimientos);	Configura una propiedad mediante setViewportView en el objeto indicado.
239	panelTabla.add(scrollTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 555));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
240	etiquetaRango.setText("Cargando movimientos...");	Cambia el texto visible de un componente de la interfaz.
241	panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 581, 800, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
242	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 285, 1470, 640));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
243	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
246	private Componentes.BotonDerretido botonActualizar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
247	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
248	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
249	private Componentes.BotonDesplegable filtroTipo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
250	private javax.swing.JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
251	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
252	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
253	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
254	private javax.swing.JScrollPane scrollTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
255	private Componentes.TablaAdministrativa tablaMovimientos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
256	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
257	}	Cierra el bloque abierto anteriormente.

16.81 src/GUI_ADMINISTRADOR/ProductoFormDialog.java

Formulario emergente para crear o editar productos, categorías, precios, stock, horario e imagen.

Tipo: java | Líneas: 1798 | Archivo 81 de 157

Clases/tipos detectados: ProductoFormDialog, CategoricalItem

Línea	Constructor o método detectado
145	private ProductoFormDialog(
208	public static boolean mostrar(
231	private void construirInterfaz() {
717	private JPanel crearPanellImagen() {
836	private void configurarEventos() {
864	private void configurarNuevoProducto() {
900	private void cargarProducto() {
984	private void cargarCategorias() {
1028	private void seleccionarCategoria(
1055	private void seleccionarImagen() {
1117	private void quitarImagen() {
1128	private void actualizarVistaPrevia() {
1220	private void actualizarTipoProducto() {
1229	private void guardar() {
1346	private Producto obtenerProductoFormulario(
1374	private boolean validarFormulario() {
1537	private void advertencia(
1552	private JTextField crearCampoTexto() {
1603	private void configurarCombo(
1626	private void agregarEtiqueta(
1677	private void agregarComponente(
1708	private JButton crearBotonPrincipal(
1746	private JButton crearBotonSecundario(
1788	private record CategoricalItem(
1793	public String toString() {

Importaciones de paquetes	
1 package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import CRUD.ProductoCRUD;	Importa CRUD.ProductoCRUD para poder usar esa clase sin escribir su nombre completo.
4	Línea en blanco: separa visualmente bloques para que el código sea más legible.
5 import Modelos.Producto;	Importa Modelos.Producto para poder usar esa clase sin escribir su nombre completo.
6	Línea en blanco: separa visualmente bloques para que el código sea más legible.
7 import Utilidades.GestorImagenProducto;	Importa Utilidades.GestorImagenProducto para poder usar esa clase sin escribir su nombre completo.
8	Línea en blanco: separa visualmente bloques para que el código sea más legible.
9 import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
10 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
11 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
12 import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
13 import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
14 import java.awt.GridBagConstraints;	Importa java.awt.GridBagConstraints para poder usar esa clase sin escribir su nombre completo.
15 import java.awt.GridBagLayout;	Importa java.awt.GridBagLayout para poder usar esa clase sin escribir su nombre completo.
16 import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
17 import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
18	Línea en blanco: separa visualmente bloques para que el código sea más legible.
19 import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
20	Línea en blanco: separa visualmente bloques para que el código sea más legible.
21 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
22	Línea en blanco: separa visualmente bloques para que el código sea más legible.
23 import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
24	Línea en blanco: separa visualmente bloques para que el código sea más legible.
25 import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
26	Línea en blanco: separa visualmente bloques para que el código sea más legible.
27 import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
28 import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
29 import javax.swing.JComboBox;	Importa javax.swing.JComboBox para poder usar esa clase sin escribir su nombre completo.
30 import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
31 import javax.swing.JFileChooser;	Importa javax.swing.JFileChooser para poder usar esa clase sin escribir su nombre completo.
32 import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
33 import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
34 import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
35 import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
36 import javax.swing.JTextArea;	Importa javax.swing.JTextArea para poder usar esa clase sin escribir su nombre completo.
37 import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
38 import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
39	Línea en blanco: separa visualmente bloques para que el código sea más legible.
40 import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
41	Línea en blanco: separa visualmente bloques para que el código sea más legible.
42 import javax.swing.filechooser.FileNameExtensionFilter;	Importa javax.swing.filechooser.FileNameExtensionFilter para poder usar esa clase sin escribir su nombre completo.
43	Línea en blanco: separa visualmente bloques para que el código sea más legible.

44	<code>public class ProductoFormDialog</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
45	<code>extends JDialog {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
48	<code>// COLORES</code>	Comentario para humanos; Java no ejecuta este texto. COLORES
49	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	<code>private static final Color AZUL =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
52	<code>new Color(</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
53	<code>0,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	<code>20,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	<code>43</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	<code>private static final Color AMARILLO =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
59	<code>new Color(</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
60	<code>255,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	<code>188,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	<code>0</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	<code>private static final Color ROJO =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
66	<code>new Color(</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
67	<code>231,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	<code>55,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	<code>65</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	<code>private static final Color GRIS =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
73	<code>new Color(</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
74	<code>92,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75	<code>103,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
76	<code>124</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
77	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	<code>private static final Color BORDE =</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
80	<code>new Color(</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
81	<code>218,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
82	<code>223,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	<code>231</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.

86	// =====	Comentario para humanos; Java no ejecuta este texto. =====
87	// DATOS	Comentario para humanos; Java no ejecuta este texto. DATOS
88	// =====	Comentario para humanos; Java no ejecuta este texto. =====
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	private final ProductoCRUD crud;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	private final Producto productoOriginal;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	private boolean modificado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96	private File archivoImagenSeleccionada;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	private String rutaImagenActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	private boolean quitarImagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	// =====	Comentario para humanos; Java no ejecuta este texto. =====
103	// COMPONENTES	Comentario para humanos; Java no ejecuta este texto. COMPONENTES
104	// =====	Comentario para humanos; Java no ejecuta este texto. =====
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	private JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107	private JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	private JTextField campoNombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
110	private JTextField campoPrecio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	private JTextField campoStockActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
113	private JTextField campoStockMinimo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	private JTextArea campoDescripcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	private JComboBox<CategoriaItem>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
118	comboCategoria;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	private JComboBox<String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
121	comboDisponibilidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	private JComboBox<String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
124	comboTamano;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	private JComboBox<String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127	comboTipo;	Ejecuta una instrucción Java completa; el punto y coma marca su final.

128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	private JComboBox<String>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	comboEstado;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	private JLabel labelVistaPrevia;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
133	private JLabel labelNombreImagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	private JButton botonSeleccionarImagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136	private JButton botonQuitarImagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	private JButton botonCancelar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
139	private JButton botonGuardar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	// =====	Comentario para humanos; Java no ejecuta este texto. =====
142	// CONSTRUCTOR	Comentario para humanos; Java no ejecuta este texto. CONSTRUCTOR
143	// =====	Comentario para humanos; Java no ejecuta este texto. =====
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	private ProductoFormDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
146	Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
147	ProductoCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
148	Producto producto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	super(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
151	propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
152	producto == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
153	? "Agregar producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	: "Editar producto",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	ModalityType.APPLICATION_MODAL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
156	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	this.crud = crud;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
159	this.productoOriginal =	Accede al atributo del objeto actual para asignarlo o utilizarlo.
160	producto;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	construirInterfaz();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	configurarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	cargarCategorias();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	if (productoOriginal != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	cargarProducto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
172	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	configurarNuevoProducto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
175	}	Cierra el bloque abierto anteriormente.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	setDefaultCloseOperation(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	DISPOSE_ON_CLOSE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	setResizable(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.
183	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	setMinimumSize(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
187	780,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	760	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	setSize(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
194	780,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
195	760	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
196	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199	setLocationRelativeTo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
200	propietario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
201	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202	}	Cierra el bloque abierto anteriormente.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	// =====	Comentario para humanos; Java no ejecuta este texto. =====
205	// MOSTRAR	Comentario para humanos; Java no ejecuta este texto. MOSTRAR
206	// =====	Comentario para humanos; Java no ejecuta este texto. =====
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	public static boolean mostrar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210	ProductoCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	Producto producto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	ProductoFormDialog dialogo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

214	new ProductoFormDialog(	Crea una nueva instancia de ProductoFormDialog y la usa o guarda para continuar el proceso.
215	propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
218	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	dialogo.setVisible(	Muestra u oculta una ventana o componente según el valor booleano enviado.
221	true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	return dialogo.modificado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
225	}	Cierra el bloque abierto anteriormente.
226		Línea en blanco: separa visualmente bloques para que el código sea más legible.
227	// =====	Comentario para humanos; Java no ejecuta este texto. =====
228	// INTERFAZ	Comentario para humanos; Java no ejecuta este texto. INTERFAZ
229	// =====	Comentario para humanos; Java no ejecuta este texto. =====
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	private void construirInterfaz() {	Declara el método construirInterfaz. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	JPanel principal =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
234	new JPanel(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
235	new BorderLayout()	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
236	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	principal.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
239	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.
242	principal.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
243	new EmptyBorder(	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
244	25,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
245	30,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246	20,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	30	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
248	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
249	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	// CABECERA	Comentario para humanos; Java no ejecuta este texto. CABECERA
252	JPanel cabecera =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
253	new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255	cabecera.setOpaque(	Configura una propiedad mediante setOpaque en el objeto indicado.
256	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

257	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.
259	cabecera.setLayout({	Configura una propiedad mediante setLayout en el objeto indicado.
260	new javax.swing.BoxLayout({	Crea una nueva instancia de javax.swing.BoxLayout y la usa o guarda para continuar el proceso.
261	cabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
262	javax.swing.BoxLayout.Y_AXIS	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
263	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
264	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	labelTitulo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
267	new JLabel({	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
268	productoOriginal == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
269	? "Agregar producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
270	: "Editar producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
271	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	labelTitulo.setForeground({	Configura una propiedad mediante setForeground en el objeto indicado.
274	AZUL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	labelTitulo.setFont({	Configura una propiedad mediante setFont en el objeto indicado.
278	new Font({	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
279	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
280	Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
281	27	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
282	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
284		Línea en blanco: separa visualmente bloques para que el código sea más legible.
285	labelSubtitulo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
286	new JLabel({	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
287	productoOriginal == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
288	? "Completa los datos del nuevo producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
289	: "Actualiza la información del producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
291		Línea en blanco: separa visualmente bloques para que el código sea más legible.
292	labelSubtitulo.setForeground({	Configura una propiedad mediante setForeground en el objeto indicado.
293	GRIS	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
294	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
295		Línea en blanco: separa visualmente bloques para que el código sea más legible.
296	labelSubtitulo.setFont({	Configura una propiedad mediante setFont en el objeto indicado.
297	new Font({	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
298	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
299	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
300	16	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

301	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
302	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
303		Línea en blanco: separa visualmente bloques para que el código sea más legible.
304	cabecera.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
305	labelTitulo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
307		Línea en blanco: separa visualmente bloques para que el código sea más legible.
308	cabecera.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
309	javax.swing.Box.createVerticalStrut(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
310	5	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	cabecera.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	labelSubtitulo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318	principal.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
319	cabecera,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
320	BorderLayout.NORTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
321	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
322		Línea en blanco: separa visualmente bloques para que el código sea más legible.
323	// FORMULARIO	Comentario para humanos; Java no ejecuta este texto. FORMULARIO
324	JPanel formulario =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
325	new JPanel(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
326	new GridBagLayout()	Crea una nueva instancia de GridBagLayout y la usa o guarda para continuar el proceso.
327	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	formulario.setOpaque(	Configura una propiedad mediante setOpaque en el objeto indicado.
330	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
331	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332		Línea en blanco: separa visualmente bloques para que el código sea más legible.
333	formulario.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
334	new EmptyBorder(	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
335	22,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
337	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338	0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
340	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
341		Línea en blanco: separa visualmente bloques para que el código sea más legible.
342	GridBagConstraints gbc =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
343	new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.

345	<code>gbc.fill =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
346	<code>GridBagConstraints.HORIZONTAL;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348	<code>gbc.anchor =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
349	<code>GridBagConstraints.NORTHWEST;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
350		Línea en blanco: separa visualmente bloques para que el código sea más legible.
351	<code>gbc.insets =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
352	<code>new Insets(</code>	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
353	<code>5,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
354	<code>5,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	<code>5,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
356	<code>15</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
358		Línea en blanco: separa visualmente bloques para que el código sea más legible.
359	<code>gbc.weightx = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
360		Línea en blanco: separa visualmente bloques para que el código sea más legible.
361	<code>int fila = 0;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
362		Línea en blanco: separa visualmente bloques para que el código sea más legible.
363	<code>// NOMBRE / CATEGORÍA</code>	Comentario para humanos; Java no ejecuta este texto. NOMBRE / CATEGORÍA
364	<code>agregarEtiqueta(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	<code>formulario,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
366	<code>"Nombre",</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
367	<code>0,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
368	<code>fila</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
369	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	<code>agregarEtiqueta(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
372	<code>formulario,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
373	<code>"Categoría",</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
374	<code>1,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
375	<code>fila</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
376	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377		Línea en blanco: separa visualmente bloques para que el código sea más legible.
378	<code>fila++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
379		Línea en blanco: separa visualmente bloques para que el código sea más legible.
380	<code>campoNombre =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
381	<code>crearCampoTexto();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
383	<code>comboCategoria =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
384	<code>crearCombo();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
385		Línea en blanco: separa visualmente bloques para que el código sea más legible.
386	<code>agregarComponente(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	<code>formulario,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
388	<code>campoNombre,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

389	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
390	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
391	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
392		Línea en blanco: separa visualmente bloques para que el código sea más legible.
393	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
394	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
395	comboCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
396	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
397	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
398	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
399		Línea en blanco: separa visualmente bloques para que el código sea más legible.
400	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
401		Línea en blanco: separa visualmente bloques para que el código sea más legible.
402	// PRECIO / DISPONIBILIDAD	Comentario para humanos; Java no ejecuta este texto. PRECIO / DISPONIBILIDAD
403	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
404	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
405	"Precio base",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
406	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
407	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
408	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
410	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
411	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
412	"Disponibilidad",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
413	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
414	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
419	campoPrecio =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
420	crearCampoTexto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
421		Línea en blanco: separa visualmente bloques para que el código sea más legible.
422	comboDisponibilidad =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
423	new JComboBox<>(new String[]{	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
424	new String[]{	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
425	"TODO_DIA",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
426	"DESAYUNO",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
427	"ALMUERZO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
428	}	Cierra el bloque abierto anteriormente.
429	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430		Línea en blanco: separa visualmente bloques para que el código sea más legible.
431	configurarCombo(comboDisponibilidad	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
432		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

433	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
434		Línea en blanco: separa visualmente bloques para que el código sea más legible.
435	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
436	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
437	campoPrecio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
438	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
440	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
441		Línea en blanco: separa visualmente bloques para que el código sea más legible.
442	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	comboDisponibilidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448		Línea en blanco: separa visualmente bloques para que el código sea más legible.
449	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	// STOCK	Comentario para humanos; Java no ejecuta este texto. STOCK
452	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
454	"Stock actual",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
455	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
456	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
457	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
458		Línea en blanco: separa visualmente bloques para que el código sea más legible.
459	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
460	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461	"Stock mínimo",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
462	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
463	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
464	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
465		Línea en blanco: separa visualmente bloques para que el código sea más legible.
466	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
467		Línea en blanco: separa visualmente bloques para que el código sea más legible.
468	campoStockActual =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
469	crearCampoTexto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
470		Línea en blanco: separa visualmente bloques para que el código sea más legible.
471	campoStockMinimo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
472	crearCampoTexto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
473		Línea en blanco: separa visualmente bloques para que el código sea más legible.
474	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
475	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
476	campoStockActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

477	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
478	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
479	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
480		Línea en blanco: separa visualmente bloques para que el código sea más legible.
481	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
482	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
483	campoStockMinimo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
484	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
485	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
486	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
487		Línea en blanco: separa visualmente bloques para que el código sea más legible.
488	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
489		Línea en blanco: separa visualmente bloques para que el código sea más legible.
490	// INVENTARIO / PERSONALIZACIÓN	Comentario para humanos; Java no ejecuta este texto. INVENTARIO / PERSONALIZACIÓN
491	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
492	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
493	"Control de inventario",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
494	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
495	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
496	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
497		Línea en blanco: separa visualmente bloques para que el código sea más legible.
498	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
499	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
500	"Personalización",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
501	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
502	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
503	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
506		Línea en blanco: separa visualmente bloques para que el código sea más legible.
507	comboTipo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
508	new JComboBox<>{	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
509	new String[]{	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
510	"RECETA",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
511	"DIRECTO",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
512	"NINGUNO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
513	}	Cierra el bloque abierto anteriormente.
514	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
515		Línea en blanco: separa visualmente bloques para que el código sea más legible.
516	configurarCombo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
517	comboTipo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
518	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
519		Línea en blanco: separa visualmente bloques para que el código sea más legible.
520	comboTamano =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

521	new JComboBox<>({	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
522	new String[]{	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
523	"PERMITIDA",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
524	"NO_PERMITIDA"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
525	}	Cierra el bloque abierto anteriormente.
526	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
527		Línea en blanco: separa visualmente bloques para que el código sea más legible.
528	configurarCombo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
529	comboTamano	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
530	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
531		Línea en blanco: separa visualmente bloques para que el código sea más legible.
532	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
533	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
534	comboTipo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
535	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
536	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
537	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
538		Línea en blanco: separa visualmente bloques para que el código sea más legible.
539	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
540	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
541	comboTamano,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
542	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
543	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
544	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
545		Línea en blanco: separa visualmente bloques para que el código sea más legible.
546	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
547		Línea en blanco: separa visualmente bloques para que el código sea más legible.
548	// ESTADO	Comentario para humanos; Java no ejecuta este texto. ESTADO
549	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
550	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
551	"Estado",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
552	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
553	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
554	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
555		Línea en blanco: separa visualmente bloques para que el código sea más legible.
556	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
557		Línea en blanco: separa visualmente bloques para que el código sea más legible.
558	comboEstado =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
559	new JComboBox<>({	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
560	new String[]{	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
561	"ACTIVO",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
562	"INACTIVO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
563	}	Cierra el bloque abierto anteriormente.
564	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.

565		Línea en blanco: separa visualmente bloques para que el código sea más legible.
566	configurarCombo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
567	comboEstado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
568	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
569		Línea en blanco: separa visualmente bloques para que el código sea más legible.
570	agregarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
571	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
572	comboEstado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
573	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
574	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
575	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
576		Línea en blanco: separa visualmente bloques para que el código sea más legible.
577	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
578		Línea en blanco: separa visualmente bloques para que el código sea más legible.
579	// DESCRIPCIÓN	Comentario para humanos; Java no ejecuta este texto. DESCRIPCIÓN
580	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
581	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
582	"Descripción",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
583	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
584	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
585	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
586		Línea en blanco: separa visualmente bloques para que el código sea más legible.
587	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
588		Línea en blanco: separa visualmente bloques para que el código sea más legible.
589	campoDescripcion =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
590	new JTextArea(	Crea una nueva instancia de JTextArea y la usa o guarda para continuar el proceso.
591	3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
592	20	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
593	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
594		Línea en blanco: separa visualmente bloques para que el código sea más legible.
595	campoDescripcion.setLineWrap(	Configura una propiedad mediante setLineWrap en el objeto indicado.
596	true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
597	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
598		Línea en blanco: separa visualmente bloques para que el código sea más legible.
599	campoDescripcion.setWrapStyleWord(	Configura una propiedad mediante setWrapStyleWord en el objeto indicado.
600	true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
601	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
602		Línea en blanco: separa visualmente bloques para que el código sea más legible.
603	campoDescripcion.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
604	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
605	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
606	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
607	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
608	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

609	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
610		Línea en blanco: separa visualmente bloques para que el código sea más legible.
611	campoDescripcion.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
612	new EmptyBorder(	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
613	8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
614	8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
615	8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
616	8	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
617	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
618	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
619		Línea en blanco: separa visualmente bloques para que el código sea más legible.
620	JScrollPane scrollDescripcion =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
621	new JScrollPane(	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
622	campoDescripcion	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
623	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
624	Componentes.DesplazamientoSuave.ocultarBarras(scrollDescripcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
625		Línea en blanco: separa visualmente bloques para que el código sea más legible.
626	scrollDescripcion.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
627	BorderFactory.createLineBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
628	BORDE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
629	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
630	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
631		Línea en blanco: separa visualmente bloques para que el código sea más legible.
632	gbc.gridx = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
633	gbc.gridy = fila;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
634	gbc.gridwidth = 2;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
635	gbc.weightx = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
636		Línea en blanco: separa visualmente bloques para que el código sea más legible.
637	formulario.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
638	scrollDescripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
639	gbc	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
640	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
641		Línea en blanco: separa visualmente bloques para que el código sea más legible.
642	gbc.gridwidth = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
643		Línea en blanco: separa visualmente bloques para que el código sea más legible.
644	fila++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
645		Línea en blanco: separa visualmente bloques para que el código sea más legible.
646	// IMAGEN	Comentario para humanos; Java no ejecuta este texto. IMAGEN
647	agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
648	formulario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
649	"Imagen del producto",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
650	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
651	fila	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
652	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

653		Línea en blanco: separa visualmente bloques para que el código sea más legible.
654	<code>fila++;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
655		Línea en blanco: separa visualmente bloques para que el código sea más legible.
656	<code>JPanel panelImagen =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
657	<code>crearPanelImagen();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
658		Línea en blanco: separa visualmente bloques para que el código sea más legible.
659	<code>gbc.gridx = 0;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
660	<code>gbc.gridy = fila;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
661	<code>gbc.gridwidth = 2;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
662		Línea en blanco: separa visualmente bloques para que el código sea más legible.
663	<code>formulario.add(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
664	<code>panelImagen,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
665	<code>gbc</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
666	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
667		Línea en blanco: separa visualmente bloques para que el código sea más legible.
668	<code>principal.add(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
669	<code>formulario,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
670	<code>BorderLayout.CENTER</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
671	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
672		Línea en blanco: separa visualmente bloques para que el código sea más legible.
673	<code>// BOTONES INFERIORES</code>	Comentario para humanos; Java no ejecuta este texto. BOTONES INFERIORES
674	<code>JPanel acciones =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
675	<code>new JPanel(</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
676	<code>new FlowLayout(</code>	Crea una nueva instancia de FlowLayout y la usa o guarda para continuar el proceso.
677	<code>FlowLayout.RIGHT,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
678	<code>15,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
679	<code>0</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
680	<code>)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
681	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
682		Línea en blanco: separa visualmente bloques para que el código sea más legible.
683	<code>acciones.setOpaque(</code>	Configura una propiedad mediante setOpaque en el objeto indicado.
684	<code>false</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
685	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
686		Línea en blanco: separa visualmente bloques para que el código sea más legible.
687	<code>botonCancelar =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
688	<code>crearBotonSecundario(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
689	<code>"Cancelar"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
690	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
691		Línea en blanco: separa visualmente bloques para que el código sea más legible.
692	<code>botonGuardar =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
693	<code>crearBotonPrincipal(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
694	<code>productoOriginal == null</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
695	<code>? "Guardar producto"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
696	<code>: "Guardar cambios"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

697	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
698		Línea en blanco: separa visualmente bloques para que el código sea más legible.
699	acciones.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
700	botonCancelar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
701	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
702		Línea en blanco: separa visualmente bloques para que el código sea más legible.
703	acciones.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
704	botonGuardar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
705	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
706		Línea en blanco: separa visualmente bloques para que el código sea más legible.
707	principal.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
708	acciones,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
709	BorderLayout.SOUTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
710	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
711		Línea en blanco: separa visualmente bloques para que el código sea más legible.
712	setContentPane(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
713	principal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
714	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
715	}	Cierra el bloque abierto anteriormente.
716		Línea en blanco: separa visualmente bloques para que el código sea más legible.
717	private JPanel crearPanelImagen() {	Declara el método crearPanelImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
718		Línea en blanco: separa visualmente bloques para que el código sea más legible.
719	JPanel panel =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
720	new JPanel(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
721	new BorderLayout(	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
722	15,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
723	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
724	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
725	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
726		Línea en blanco: separa visualmente bloques para que el código sea más legible.
727	panel.setOpaque(	Configura una propiedad mediante setOpaque en el objeto indicado.
728	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
729	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
730		Línea en blanco: separa visualmente bloques para que el código sea más legible.
731	labelVistaPrevia =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
732	new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
733	"Sin imagen",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
734	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
735	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
736		Línea en blanco: separa visualmente bloques para que el código sea más legible.
737	labelVistaPrevia.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
738	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
739	220,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
740	145	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

741	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
742	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
743		Línea en blanco: separa visualmente bloques para que el código sea más legible.
744	labelVistaPrevia.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
745	BorderFactory.createLineBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
746	BORDE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
747	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
748	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
749		Línea en blanco: separa visualmente bloques para que el código sea más legible.
750	labelVistaPrevia.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
751	GRIS	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
752	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
753		Línea en blanco: separa visualmente bloques para que el código sea más legible.
754	JPanel derecha =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
755	new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
756		Línea en blanco: separa visualmente bloques para que el código sea más legible.
757	derecha.setOpaque(	Configura una propiedad mediante setOpaque en el objeto indicado.
758	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
759	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
760		Línea en blanco: separa visualmente bloques para que el código sea más legible.
761	derecha.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
762	new javax.swing.BoxLayout(	Crea una nueva instancia de javax.swing.BoxLayout y la usa o guarda para continuar el proceso.
763	derecha,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
764	javax.swing.BoxLayout.Y_AXIS	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
765	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
766	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
767		Línea en blanco: separa visualmente bloques para que el código sea más legible.
768	botonSeleccionarImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
769	crearBotonPrincipal(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
770	"Seleccionar imagen"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
771	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
772		Línea en blanco: separa visualmente bloques para que el código sea más legible.
773	botonQuitarImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
774	crearBotonSecundario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
775	"Quitar imagen"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
776	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
777		Línea en blanco: separa visualmente bloques para que el código sea más legible.
778	labelNombreImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
779	new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
780	"Ninguna imagen seleccionada"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
781	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
782		Línea en blanco: separa visualmente bloques para que el código sea más legible.
783	labelNombreImagen.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
784	GRIS	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

785	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
786		Línea en blanco: separa visualmente bloques para que el código sea más legible.
787	labelNombreImagen.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
788	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
789	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
790	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
791	12	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
792	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
793	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
794		Línea en blanco: separa visualmente bloques para que el código sea más legible.
795	derecha.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
796	botonSeleccionarImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
797	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
798		Línea en blanco: separa visualmente bloques para que el código sea más legible.
799	derecha.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
800	javax.swing.Box.createVerticalStrut(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
801	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
802	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
803	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
804		Línea en blanco: separa visualmente bloques para que el código sea más legible.
805	derecha.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
806	botonQuitarImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
807	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
808		Línea en blanco: separa visualmente bloques para que el código sea más legible.
809	derecha.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
810	javax.swing.Box.createVerticalStrut(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
811	12	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
812	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
813	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
814		Línea en blanco: separa visualmente bloques para que el código sea más legible.
815	derecha.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
816	labelNombreImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
817	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
818		Línea en blanco: separa visualmente bloques para que el código sea más legible.
819	panel.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
820	labelVistaPrevia,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
821	BorderLayout.WEST	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
822	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
823		Línea en blanco: separa visualmente bloques para que el código sea más legible.
824	panel.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
825	derecha,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
826	BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
827	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
828		Línea en blanco: separa visualmente bloques para que el código sea más legible.

829	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
830	}	Cierra el bloque abierto anteriormente.
831		Línea en blanco: separa visualmente bloques para que el código sea más legible.
832	// =====	Comentario para humanos; Java no ejecuta este texto. =====
833	// EVENTOS	Comentario para humanos; Java no ejecuta este texto. EVENTOS
834	// =====	Comentario para humanos; Java no ejecuta este texto. =====
835		Línea en blanco: separa visualmente bloques para que el código sea más legible.
836	private void configurarEventos() {	Declara el método configurarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
837		Línea en blanco: separa visualmente bloques para que el código sea más legible.
838	botonCancelar.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
839	evento -> dispose()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
840	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
841		Línea en blanco: separa visualmente bloques para que el código sea más legible.
842	botonGuardar.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
843	evento -> guardar()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
844	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
845		Línea en blanco: separa visualmente bloques para que el código sea más legible.
846	botonSeleccionarImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
847	.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
848	evento ->	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
849	seleccionarImagen()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
850	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
851		Línea en blanco: separa visualmente bloques para que el código sea más legible.
852	botonQuitarImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
853	.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
854	evento ->	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
855	quitarImagen()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
856	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
857		Línea en blanco: separa visualmente bloques para que el código sea más legible.
858	}	Cierra el bloque abierto anteriormente.
859		Línea en blanco: separa visualmente bloques para que el código sea más legible.
860	// =====	Comentario para humanos; Java no ejecuta este texto. =====
861	// NUEVO PRODUCTO	Comentario para humanos; Java no ejecuta este texto. NUEVO PRODUCTO
862	// =====	Comentario para humanos; Java no ejecuta este texto. =====
863		Línea en blanco: separa visualmente bloques para que el código sea más legible.
864	private void configurarNuevoProducto() {	Declara el método configurarNuevoProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
865		Línea en blanco: separa visualmente bloques para que el código sea más legible.
866	campoStockActual.setText(	Cambia el texto visible de un componente de la interfaz.
867	"0"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
868	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

869		Línea en blanco: separa visualmente bloques para que el código sea más legible.
870	campoStockMinimo.setText(	Cambia el texto visible de un componente de la interfaz.
871	"0"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
872	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
873		Línea en blanco: separa visualmente bloques para que el código sea más legible.
874	comboDisponibilidad.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
875	"TODO_DIA"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
876	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
877		Línea en blanco: separa visualmente bloques para que el código sea más legible.
878	comboTipo.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
879	"DIRECTO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
880	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
881		Línea en blanco: separa visualmente bloques para que el código sea más legible.
882	comboTamano.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
883	"PERMITIDA"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
884	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
885		Línea en blanco: separa visualmente bloques para que el código sea más legible.
886	comboEstado.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
887	"ACTIVO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
888	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
889		Línea en blanco: separa visualmente bloques para que el código sea más legible.
890	rutaImagenActual =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
891	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
892		Línea en blanco: separa visualmente bloques para que el código sea más legible.
893	actualizarVistaPrevio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
894	}	Cierra el bloque abierto anteriormente.
895		Línea en blanco: separa visualmente bloques para que el código sea más legible.
896	// =====	Comentario para humanos; Java no ejecuta este texto. =====
897	// CARGAR PRODUCTO	Comentario para humanos; Java no ejecuta este texto. CARGAR PRODUCTO
898	// =====	Comentario para humanos; Java no ejecuta este texto. =====
899		Línea en blanco: separa visualmente bloques para que el código sea más legible.
900	private void cargarProducto() {	Declara el método cargarProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
901		Línea en blanco: separa visualmente bloques para que el código sea más legible.
902	labelSubtitulo.setText(	Cambia el texto visible de un componente de la interfaz.
903	String.format(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
904	"Actualiza la información de #%04d",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
905	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
906	.getIdProducto()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
907	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
908	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
909		Línea en blanco: separa visualmente bloques para que el código sea más legible.
910	campoNombre.setText(	Cambia el texto visible de un componente de la interfaz.
911	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

912	.getNombre()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
913	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
914		Línea en blanco: separa visualmente bloques para que el código sea más legible.
915	campoDescripcion.setText(	Cambia el texto visible de un componente de la interfaz.
916	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
917	.getDescripcion() == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
918	? ""	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
919	: productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
920	.getDescripcion()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
921	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
922		Línea en blanco: separa visualmente bloques para que el código sea más legible.
923	campoPrecio.setText(	Cambia el texto visible de un componente de la interfaz.
924	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
925	.getPrecioBase()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
926	.toString()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
927	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
928		Línea en blanco: separa visualmente bloques para que el código sea más legible.
929	campoStockActual.setText(	Cambia el texto visible de un componente de la interfaz.
930	String.valueOf(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
931	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
932	.getStockActual()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
933	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
934	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
935		Línea en blanco: separa visualmente bloques para que el código sea más legible.
936	campoStockMinimo.setText(	Cambia el texto visible de un componente de la interfaz.
937	String.valueOf(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
938	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
939	.getStockMinimo()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
940	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
941	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
942		Línea en blanco: separa visualmente bloques para que el código sea más legible.
943	comboDisponibilidad	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
944	.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
945	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
946	.getDisponibilidadMenu()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
947	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
948		Línea en blanco: separa visualmente bloques para que el código sea más legible.
949	comboTipo.setSelectedItem(productoOriginal.getTipoStock());	Configura una propiedad mediante setSelectedItem en el objeto indicado.
950	comboTamano.setSelectedItem(productoOriginal.isPersonalizable())	Configura una propiedad mediante setSelectedItem en el objeto indicado.
951	? "PERMITIDA" : "NO_PERMITIDA");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
952		Línea en blanco: separa visualmente bloques para que el código sea más legible.
953	comboEstado.setSelectedItem(	Configura una propiedad mediante setSelectedItem en el objeto indicado.
954	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
955	.isActivo()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

956	? "ACTIVO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
957	: "INACTIVO"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
958	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
959		Línea en blanco: separa visualmente bloques para que el código sea más legible.
960	seleccionarCategoria(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
961	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
962	.getIdCategoria()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
963	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
964		Línea en blanco: separa visualmente bloques para que el código sea más legible.
965	rutaImagenActual =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
966	productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
967	.getImagen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
968		Línea en blanco: separa visualmente bloques para que el código sea más legible.
969	archivoImagenSeleccionada =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
970	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
971		Línea en blanco: separa visualmente bloques para que el código sea más legible.
972	quitarImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
973	false;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
974		Línea en blanco: separa visualmente bloques para que el código sea más legible.
975	actualizarTipoProducto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
976		Línea en blanco: separa visualmente bloques para que el código sea más legible.
977	actualizarVistaPrevia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
978	}	Cierra el bloque abierto anteriormente.
979		Línea en blanco: separa visualmente bloques para que el código sea más legible.
980	// =====	Comentario para humanos; Java no ejecuta este texto. =====
981	// CATEGORÍAS	Comentario para humanos; Java no ejecuta este texto. CATEGORÍAS
982	// =====	Comentario para humanos; Java no ejecuta este texto. =====
983		Línea en blanco: separa visualmente bloques para que el código sea más legible.
984	private void cargarCategorias() {	Declara el método cargarCategorias. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
985		Línea en blanco: separa visualmente bloques para que el código sea más legible.
986	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
987		Línea en blanco: separa visualmente bloques para que el código sea más legible.
988	Integer categoriaActual =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
989	productoOriginal == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
990	? null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
991	: productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
992	.getIdCategoria();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
993		Línea en blanco: separa visualmente bloques para que el código sea más legible.
994	Map<Integer, String> categorias =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
995	crud.listarCategoriasParaFormulario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
996	categoriaActual	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
997	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
998		Línea en blanco: separa visualmente bloques para que el código sea más legible.

999	comboCategoria.removeAllItems();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1000		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1001	for (Map.Entry<Integer, String> entrada	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1002	: categorias.entrySet()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1003		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1004	comboCategoria.addItem(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1005	new CategoriaItem(	Crea una nueva instancia de CategoriaItem y la usa o guarda para continuar el proceso.
1006	entrada.getKey(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1007	entrada.getValue()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1008	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1009	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1010	}	Cierra el bloque abierto anteriormente.
1011		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1012	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1013		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1014	botonGuardar.setEnabled(	Habilita o deshabilita la interacción con el componente.
1015	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1016	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1017		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1018	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1019	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1020	"No fue posible cargar las categorías.\n"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1021	+ ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1022	"Error de base de datos",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1023	JOptionPane.ERROR_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1024	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1025	}	Cierra el bloque abierto anteriormente.
1026	}	Cierra el bloque abierto anteriormente.
1027		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1028	private void seleccionarCategoria(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1029	int idCategoria) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1030		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1031	for (int i = 0;	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
1032	i < comboCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1033	.getItemCount();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1034	i++) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1035		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1036	CategoriaItem item =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1037	comboCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1038	.getItemAt(i);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1039		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1040	if (item.id()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1041	== idCategoria) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1042		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1043	comboCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1044	.setSelectedIndex(i);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
1045		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1046	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1047	}	Cierra el bloque abierto anteriormente.
1048	}	Cierra el bloque abierto anteriormente.
1049	}	Cierra el bloque abierto anteriormente.
1050		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1051	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1052	// IMÁGENES	Comentario para humanos; Java no ejecuta este texto. IMÁGENES
1053	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1054		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1055	private void seleccionarImagen() {	Declara el método seleccionarImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1056		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1057	JFileChooser selector =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1058	new JFileChooser();	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
1059		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1060	selector.setDialogTitle(	Configura una propiedad mediante setDialogTitle en el objeto indicado.
1061	"Seleccionar imagen del producto"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1062	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1063		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1064	selector.setAcceptAllFileFilterUsed(	Configura una propiedad mediante setAcceptAllFileFilterUsed en el objeto indicado.
1065	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1066	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1067		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1068	selector.setFileFilter(	Configura una propiedad mediante setFileFilter en el objeto indicado.
1069	new FileNameExtensionFilter(	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.
1070	"Imágenes PNG, JPG o JPEG",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1071	"png",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1072	"jpg",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1073	"jpeg"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1074	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1075	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1076		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1077	int resultado =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1078	selector.showOpenDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1079	this	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1080	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1081		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1082	if (resultado	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1083	!= JFileChooser.APPROVE_OPTION) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1084		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1085	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1086	}	Cierra el bloque abierto anteriormente.
1087		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1088	File archivo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1089	selector.getSelectedFile();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1090		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1091	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1092		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1093	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1094	.validarImagen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1095	archivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1096	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1097		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1098	archivoImagenSeleccionada =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1099	archivo;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1101	quitarImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1102	false;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1104	actualizarVistaPrevia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1106	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1108	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1109	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1110	ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1111	"Imagen no válida",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1112	JOptionPane.WARNING_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1113	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1114	}	Cierra el bloque abierto anteriormente.
1115	}	Cierra el bloque abierto anteriormente.
1116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1117	private void quitarImagen() {	Declara el método quitarImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1119	archivoImagenSeleccionada =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1120	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1122	quitarImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1123	true;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1125	actualizarVistaPrevia();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1126	}	Cierra el bloque abierto anteriormente.
1127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1128	private void actualizarVistaPrevia() {	Declara el método actualizarVistaPrevia. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

1129		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1130	if (archivoImagenSeleccionada	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1131	!= null) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1133	var icono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1134	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1135	.cargarIcono(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1136	archivoImagenSeleccionada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1137	205,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1138	130	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1139	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1141	labelVistaPrevia	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1142	.setText("");	Cambia el texto visible de un componente de la interfaz.
1143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1144	labelVistaPrevia	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1145	.setIcon(	Configura una propiedad mediante setIcon en el objeto indicado.
1146	icono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1147	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1149	labelNombreImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1150	.setText(	Cambia el texto visible de un componente de la interfaz.
1151	archivoImagenSeleccionada	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1152	.getName()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1153	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1155	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1156	}	Cierra el bloque abierto anteriormente.
1157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1158	if (!quitarImagen	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1159	&& rutaImagenActual != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1160	&& !rutaImagenActual.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1162	var icono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1163	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1164	.cargarIcono(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1165	rutaImagenActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1166	205,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1167	130	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1168	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1170	if (icono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1171		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1172	labelVistaPrevia	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1173	.setText("");	Cambia el texto visible de un componente de la interfaz.
1174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1175	labelVistaPrevia	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1176	.setIcon(	Configura una propiedad mediante setIcon en el objeto indicado.
1177	icono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1178	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1179		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1180	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1182	labelNombreImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1183	.setText(	Cambia el texto visible de un componente de la interfaz.
1184	Path.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1185	rutaImagenActual	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1186	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1187	.getFileName()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1188	.toString()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1189	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1190		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1191	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1193	labelNombreImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1194	.setText(	Cambia el texto visible de un componente de la interfaz.
1195	rutaImagenActual	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1196	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1197	}	Cierra el bloque abierto anteriormente.
1198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1199	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1200	}	Cierra el bloque abierto anteriormente.
1201	}	Cierra el bloque abierto anteriormente.
1202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1203	labelVistaPrevia.setIcon(	Configura una propiedad mediante setIcon en el objeto indicado.
1204	null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1205	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1207	labelVistaPrevia.setText(	Cambia el texto visible de un componente de la interfaz.
1208	"Sin imagen"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1209	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1211	labelNombreImagen.setText(	Cambia el texto visible de un componente de la interfaz.
1212	"Ninguna imagen seleccionada"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1213	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1214	}	Cierra el bloque abierto anteriormente.
1215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1216	// =====	Comentario para humanos; Java no ejecuta este texto. =====

1217	// CONTROL DE INVENTARIO	Comentario para humanos; Java no ejecuta este texto. CONTROL DE INVENTARIO
1218	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1220	private void actualizarTipoProducto() {	Declara el método actualizarTipoProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1221		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1222	comboTamano.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
1223	}	Cierra el bloque abierto anteriormente.
1224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1225	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1226	// GUARDAR	Comentario para humanos; Java no ejecuta este texto. GUARDAR
1227	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1228		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1229	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1231	if (!validarFormulario()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1232	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1233	}	Cierra el bloque abierto anteriormente.
1234		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1235	String nuevaRutaImagen =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1236	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1238	String imagenAnterior =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1239	productoOriginal == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1240	? null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1241	: productoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1242	.getImagen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1243		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1244	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1246	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1247	* Solo copiamos la nueva imagen	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1248	* cuando el usuario presiona Guardar.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1249	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1250	if (archivoImagenSeleccionada	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1251	!= null) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1252		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1253	nuevaRutaImagen =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1254	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1255	.guardarImagen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1256	archivoImagenSeleccionada	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1257	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1258	}	Cierra el bloque abierto anteriormente.

1259		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1260	String imagenFinal;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1262	if (archivoImagenSeleccionada	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1263	!= null) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1265	imagenFinal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1266	nuevaRutaImagen;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1268	} else if (quitarImagen) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1270	imagenFinal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1271	null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1273	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1274		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1275	imagenFinal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1276	rutaImagenActual;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1277	}	Cierra el bloque abierto anteriormente.
1278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1279	Producto producto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1280	obtenerProductoFormulario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1281	imagenFinal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1282	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1283		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1284	if (productoOriginal == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1286	crud.insertar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1287	producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1288	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1289		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1290	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1291		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1292	crud.actualizar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1293	producto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1294	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1295	}	Cierra el bloque abierto anteriormente.
1296		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1297	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1298	* MySQL ya confirmó el UPDATE.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1299	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1300	* Ahora podemos eliminar una imagen antigua	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1301	* creada por el propio programa.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1302	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

1303	if (productoOriginal != null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1304	&& imagenAnterior != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1305	&& (archivoImagenSeleccionada != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1306	quitarImagen)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1307	&& !imagenAnterior.equals(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1308	imagenFinal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1309	)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1311	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1312	.eliminarImagenGestionada(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1313	imagenAnterior	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1314	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1315	}	Cierra el bloque abierto anteriormente.
1316		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1317	modificado =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1318	true;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1319		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1320	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1321		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1322	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1323		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1324	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1325	* Si ya habíamos copiado una nueva imagen pero	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1326	* MySQL falló, no dejamos un archivo huérfano.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1327	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1328	if (nuevaRutaImagen != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1330	GestorImagenProducto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1331	.eliminarImagenGestionada(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1332	nuevaRutaImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1333	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1334	}	Cierra el bloque abierto anteriormente.
1335		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1336	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1337	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1338	"No fue posible guardar el producto.\n"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1339	+ ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1340	"Error",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1341	JOptionPane.ERROR_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1342	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1343	}	Cierra el bloque abierto anteriormente.
1344	}	Cierra el bloque abierto anteriormente.
1345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1346	private Producto obtenerProductoFormulario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1347	String imagen) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1349	CategoriaItem categoria =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1350	(CategoriaItem)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1351	comboCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1352	.getSelectedItem();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1353		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1354	Producto resultado = productoOriginal == null ? new Producto() : productoOriginal;	Crea una nueva instancia de Producto y la usa o guarda para continuar el proceso.
1355	resultado.setIdCategoria(categoria.id());	Configura una propiedad mediante setIdCategoria en el objeto indicado.
1356	resultado.setCategoria(categoria.nombre());	Configura una propiedad mediante setCategoria en el objeto indicado.
1357	resultado.setNombre(campoNombre.getText().trim());	Configura una propiedad mediante setNombre en el objeto indicado.
1358	resultado.setDescripcion(campoDescripcion.getText().trim());	Configura una propiedad mediante setDescripcion en el objeto indicado.
1359	resultado.setPrecioBase(new BigDecimal(campoPrecio.getText().trim()));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1360	resultado.setImagen(imagen);	Configura una propiedad mediante setImagen en el objeto indicado.
1361	resultado.setDisponibilidadMenu((String) comboDisponibilidad.getSelectedItem());	Configura una propiedad mediante setDisponibilidadMenu en el objeto indicado.
1362	resultado.setTipoStock((String) comboTipo.getSelectedItem());	Configura una propiedad mediante setTipoStock en el objeto indicado.
1363	resultado.setPersonalizable("PERMITIDA".equals(comboTamano.getSelectedItem()));	Configura una propiedad mediante setPersonalizable en el objeto indicado.
1364	resultado.setStockActual(Integer.parseInt(campoStockActual.getText().trim()));	Configura una propiedad mediante setStockActual en el objeto indicado.
1365	resultado.setStockMinimo(Integer.parseInt(campoStockMinimo.getText().trim()));	Configura una propiedad mediante setStockMinimo en el objeto indicado.
1366	resultado.setActivo("ACTIVO".equals(comboEstado.getSelectedItem()));	Configura una propiedad mediante setActivo en el objeto indicado.
1367	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1368	}	Cierra el bloque abierto anteriormente.
1369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1370	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1371	// VALIDACIONES	Comentario para humanos; Java no ejecuta este texto. VALIDACIONES
1372	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1373		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1374	private boolean validarFormulario() {	Declara el método validarFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1375		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1376	String nombre =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1377	campoNombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1378	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1379	.trim();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1380		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1381	if (nombre.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1383	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1384	"Ingresa el nombre del producto."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1385	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1387	campoNombre.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1388		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1389	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

1390	}	Cierra el bloque abierto anteriormente.
1391		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1392	if (nombre.length()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1393	> 100) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1394		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1395	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1396	"El nombre no puede superar los 100 caracteres."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1397	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1398		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1399	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1400	}	Cierra el bloque abierto anteriormente.
1401		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1402	if (comboCategoria	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1403	.getSelectedItem()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1404	== null) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1405		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1406	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1407	"Selecciona una categoría."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1408	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1410	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1411	}	Cierra el bloque abierto anteriormente.
1412		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1413	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1414		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1415	BigDecimal precio =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1416	new BigDecimal(	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
1417	campoPrecio	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1418	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1419	.trim()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1420	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1421		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1422	if (precio.compareTo(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1423	BigDecimal.ZERO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1424	) < 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1425		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1426	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1427	"El precio no puede ser negativo."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1428	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1429		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1430	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1431	}	Cierra el bloque abierto anteriormente.
1432		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1433	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

1434		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1435	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1436	"Ingresa un precio válido. Ejemplo: 35.00"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1437	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1438		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1439	campoPrecio.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1441	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1442	}	Cierra el bloque abierto anteriormente.
1443		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1444	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1445		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1446	int stock =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1447	Integer.parseInt(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1448	campoStockActual	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1449	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1450	.trim()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1451	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1452		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1453	if (stock < 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1454		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1455	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1456	"El stock actual no puede ser negativo."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1457	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1458		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1459	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1460	}	Cierra el bloque abierto anteriormente.
1461		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1462	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1463		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1464	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1465	"El stock actual debe ser un número entero."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1466	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1467		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1468	campoStockActual.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1469		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1470	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1471	}	Cierra el bloque abierto anteriormente.
1472		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1473	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
1474		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1475	int minimo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1476	Integer.parseInt(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1477	campoStockMinimo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1478	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1479	.trim()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1480	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1481		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1482	if (minimo < 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1483		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1484	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1485	"El stock mínimo no puede ser negativo."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1486	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1487		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1488	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1489	}	Cierra el bloque abierto anteriormente.
1490		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1491	} catch (NumberFormatException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1493	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1494	"El stock mínimo debe ser un número entero."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1495	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1496		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1497	campoStockMinimo.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1498		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1499	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1500	}	Cierra el bloque abierto anteriormente.
1501		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1502	if (campoDescripcion	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1503	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1504	.trim()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1505	.length()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1506	> 255) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1507		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1508	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1509	"La descripción no puede superar los 255 caracteres."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1510	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1511		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1512	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1513	}	Cierra el bloque abierto anteriormente.
1514		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1515	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1516	* Exigimos que todos los productos del menú	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1517	* tengan imagen.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1518	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1519	boolean tieneImagen =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1520	archivoImagenSeleccionada != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1521	(!quitarImagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1522	&& rutaImagenActual != null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1523	&& !rutaImagenActual.isBlank());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1524		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1525	if (!tieneImagen) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1526		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1527	advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1528	"Selecciona una imagen para el producto."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1529	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1530		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1531	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1532	}	Cierra el bloque abierto anteriormente.
1533		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1534	return true;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1535	}	Cierra el bloque abierto anteriormente.
1536		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1537	private void advertencia(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1538	String mensaje) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1539		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1540	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1541	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1542	mensaje,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1543	"Verifica los datos",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1544	JOptionPane.WARNING_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1545	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1546	}	Cierra el bloque abierto anteriormente.
1547		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1548	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1549	// COMPONENTES AUXILIARES	Comentario para humanos; Java no ejecuta este texto. COMPONENTES AUXILIARES
1550	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1551		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1552	private JTextField crearCampoTexto() {	Declara el método crearCampoTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1553		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1554	JTextField campo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1555	new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
1556		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1557	campo.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
1558	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1559	310,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1560	42	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1561	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1562	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1563		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1564	campo.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
1565	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
1566	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1567	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1568	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1569	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1570	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1571		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1572	campo.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
1573	BorderFactory.createCompoundBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1574	BorderFactory	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1575	.createLineBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1576	BORDE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1577	),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1578	new EmptyBorder(	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1579	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1580	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1581	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1582	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1583	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1584	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1585	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1586		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1587	return campo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1588	}	Cierra el bloque abierto anteriormente.
1589		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1590	private <T> JComboBox<T>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1591	crearCombo() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1592		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1593	JComboBox<T> combo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1594	new JComboBox<>();	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
1595		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1596	configurarCombo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1597	combo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1598	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1599		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1600	return combo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1601	}	Cierra el bloque abierto anteriormente.
1602		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1603	private void configurarCombo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1604	JComboBox<?> combo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1605		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1606	combo.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
1607	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.

1608	310,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1609	42	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1610	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1611	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1612		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1613	combo.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
1614	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
1615	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1616	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1617	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1618	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1619	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1620		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1621	combo.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
1622	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1623	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1624	}	Cierra el bloque abierto anteriormente.
1625		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1626	private void agregarEtiqueta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1627	JPanel panel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1628	String texto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1629	int x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1630	int y) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1631		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1632	JLabel label =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1633	new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1634	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1635	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1636		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1637	label.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
1638	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
1639	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1640	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1641	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1642	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1643	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1644		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1645	label.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
1646	AZUL	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1647	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1649	GridBagConstraints gbc =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1650	new GridBagConstraints();	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1651		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1652	<code>gbc.gridx = x;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1653	<code>gbc.gridy = y;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1654		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1655	<code>gbc.weightx = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1656		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1657	<code>gbc.fill =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1658	<code>GridBagConstraints.HORIZONTAL;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1659		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1660	<code>gbc.anchor =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1661	<code>GridBagConstraints.WEST;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1662		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1663	<code>gbc.insets =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1664	<code>new Insets(</code>	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
1665	<code>5,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1666	<code>5,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1667	<code>2,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1668	<code>15</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1669	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1670		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1671	<code>panel.add(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1672	<code>label,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1673	<code>gbc</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1674	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1675	<code>}</code>	Cierra el bloque abierto anteriormente.
1676		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1677	<code>private void agregarComponente(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1678	<code>JPanel panel,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1679	<code>java.awt.Component componente,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1680	<code>int x,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1681	<code>int y) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1682		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1683	<code>GridBagConstraints gbc =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1684	<code>new GridBagConstraints();</code>	Crea una nueva instancia de GridBagConstraints y la usa o guarda para continuar el proceso.
1685		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1686	<code>gbc.gridx = x;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1687	<code>gbc.gridy = y;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1688		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1689	<code>gbc.weightx = 1;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1690		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1691	<code>gbc.fill =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1692	<code>GridBagConstraints.HORIZONTAL;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1693		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1694	<code>gbc.insets =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1695	<code>new Insets(</code>	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.

1696	2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1697	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1698	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1699	15	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1700	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1701		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1702	panel.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1703	componente,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1704	gbc	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1705	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1706	}	Cierra el bloque abierto anteriormente.
1707		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1708	private JButton crearBotonPrincipal(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1709	String texto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1710		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1711	JButton boton =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1712	new JButton(	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
1713	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1714	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1715		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1716	boton.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
1717	AMARILLO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1718	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1719		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1720	boton.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
1721	Color.BLACK	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1722	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1723		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1724	boton.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
1725	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
1726	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1727	Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1728	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1729	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1730	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1731		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1732	boton.setFocusPainted(	Configura una propiedad mediante setFocusPainted en el objeto indicado.
1733	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1734	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1735		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1736	boton.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
1737	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1738	165,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1739	45	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1740	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1741	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1742		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1743	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1744	}	Cierra el bloque abierto anteriormente.
1745		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1746	private JButton crearBotonSecundario(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1747	String texto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1748		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1749	JButton boton =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1750	new JButton(	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
1751	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1752	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1753		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1754	boton.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
1755	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1756	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1757		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1758	boton.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
1759	ROJO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1760	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1761		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1762	boton.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
1763	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
1764	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1765	Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1766	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1767	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1768	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1769		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1770	boton.setFocusPainted(	Configura una propiedad mediante setFocusPainted en el objeto indicado.
1771	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1772	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1773		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1774	boton.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
1775	new Dimension(	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1776	150,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1777	45	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1778	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1779	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1780		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1781	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1782	}	Cierra el bloque abierto anteriormente.
1783		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1784	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1785	// ITEM DE CATEGORÍA	Comentario para humanos; Java no ejecuta este texto. ITEM DE CATEGORÍA
1786	// =====	Comentario para humanos; Java no ejecuta este texto. =====
1787		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1788	private record CategoriaItem(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1789	int id,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1790	String nombre) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1791		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1792	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1793	public String toString() {	Declara el método toString. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1794		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1795	return nombre;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1796	}	Cierra el bloque abierto anteriormente.
1797	}	Cierra el bloque abierto anteriormente.
1798	}	Cierra el bloque abierto anteriormente.

16.82 src/GUI_ADMINISTRADOR/ReportesPanel.form

Metadatos visuales de NetBeans para diseñar ReportesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 807 | Archivo 82 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.5" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
9	<Dimension value="[1580, 932]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</Property>	Cierra la sección XML abierta anteriormente.
11	</Properties>	Cierra la sección XML abierta anteriormente.
12	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,126,0,0,5,126"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	</AuxValues>	Cierra la sección XML abierta anteriormente.
23		Línea en blanco para organizar el XML del formulario.
24	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
25	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
26	</Layout>	Cierra la sección XML abierta anteriormente.
27	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Component class="javax.swing.JLabel" name="labelTitulo">	Declara un componente o contenedor visual y su clase Java.
29	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
31		Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	</Property>	Cierra la sección XML abierta anteriormente.
33	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
34	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
35	</Property>	Cierra la sección XML abierta anteriormente.
36	<Property name="text" type="java.lang.String" value="Reportes - Ventas del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
37	</Properties>	Cierra la sección XML abierta anteriormente.

38	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
39	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
40	<AbsoluteConstraints x="70" y="25" width="690" height="65"/>	Define la posición y el tamaño del componente dentro del diseño.
41	</Constraint>	Cierra la sección XML abierta anteriormente.
42	</Constraints>	Cierra la sección XML abierta anteriormente.
43	</Component>	Cierra la sección XML abierta anteriormente.
44	<Component class="javax.swing.JLabel" name="labelSubtitulo">	Declara un componente o contenedor visual y su clase Java.
45	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
47	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
48	</Property>	Cierra la sección XML abierta anteriormente.
49	<Property name="text" type="java.lang.String" value="Resumen detallado de todas las ventas realizadas en la fecha seleccionada"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
50	</Properties>	Cierra la sección XML abierta anteriormente.
51	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
52	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
53	<AbsoluteConstraints x="70" y="88" width="760" height="28"/>	Define la posición y el tamaño del componente dentro del diseño.
54	</Constraint>	Cierra la sección XML abierta anteriormente.
55	</Constraints>	Cierra la sección XML abierta anteriormente.
56	</Component>	Cierra la sección XML abierta anteriormente.
57	<Component class="Componentes.BotonDerretido" name="botonActualizar">	Declara un componente o contenedor visual y su clase Java.
58	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
59	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
60	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
61	</Property>	Cierra la sección XML abierta anteriormente.
62	<Property name="text" type="java.lang.String" value="Actualizar reporte"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
63	</Properties>	Cierra la sección XML abierta anteriormente.
64	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
65	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
66	<AbsoluteConstraints x="950" y="60" width="220" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
67	</Constraint>	Cierra la sección XML abierta anteriormente.
68	</Constraints>	Cierra la sección XML abierta anteriormente.
69	</Component>	Cierra la sección XML abierta anteriormente.
70	<Component class="Componentes.BotonRedondeado" name="botonExportar">	Declara un componente o contenedor visual y su clase Java.
71	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
72	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
73	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
74	</Property>	Cierra la sección XML abierta anteriormente.
75	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
76	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
77	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
78	</Property>	Cierra la sección XML abierta anteriormente.
79	<Property name="radio" type="int" value="20"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

80	<Property name="text" type="java.lang.String" value="Exportar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
81	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
82	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
83	</Property>	Cierra la sección XML abierta anteriormente.
84	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
85	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
86	</Property>	Cierra la sección XML abierta anteriormente.
87	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
88		Metadato de NetBeans que conserva el estado editable del diseñador visual.
89	</Property>	Cierra la sección XML abierta anteriormente.
90	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
91	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
92	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
93	</Property>	Cierra la sección XML abierta anteriormente.
94	<Property name="maximumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
95	<Dimension value="[96, 29]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
96	</Property>	Cierra la sección XML abierta anteriormente.
97	<Property name="minimumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
98	<Dimension value="[96, 29]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
99	</Property>	Cierra la sección XML abierta anteriormente.
100	</Properties>	Cierra la sección XML abierta anteriormente.
101	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
102	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
103	<AbsoluteConstraints x="1185" y="60" width="180" height="58"/>	Define la posición y el tamaño del componente dentro del diseño.
104	</Constraint>	Cierra la sección XML abierta anteriormente.
105	</Constraints>	Cierra la sección XML abierta anteriormente.
106	</Component>	Cierra la sección XML abierta anteriormente.
107	<Component class="Componentes.BotonRedondeado" name="botonImprimir">	Declara un componente o contenedor visual y su clase Java.
108	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
109	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
110	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
111	</Property>	Cierra la sección XML abierta anteriormente.
112	<Property name="text" type="java.lang.String" value="Imprimir"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
113	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
114	</Properties>	Cierra la sección XML abierta anteriormente.
115	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
116	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
117	<AbsoluteConstraints x="1375" y="60" width="145" height="58"/>	Define la posición y el tamaño del componente dentro del diseño.
118	</Constraint>	Cierra la sección XML abierta anteriormente.
119	</Constraints>	Cierra la sección XML abierta anteriormente.
120	</Component>	Cierra la sección XML abierta anteriormente.
121	<Container class="Componentes.PanelFlotante" name="panelVentas">	Declara un componente o contenedor visual y su clase Java.

122	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
123	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
124	</Properties>	Cierra la sección XML abierta anteriormente.
125	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
126	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
127	<AbsoluteConstraints x="60" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
128	</Constraint>	Cierra la sección XML abierta anteriormente.
129	</Constraints>	Cierra la sección XML abierta anteriormente.
130		Línea en blanco para organizar el XML del formulario.
131	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
132	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
133	</Layout>	Cierra la sección XML abierta anteriormente.
134	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
135	<Component class="javax.swing.JLabel" name="labelTituloVentas">	Declara un componente o contenedor visual y su clase Java.
136	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
137	<Property name="text" type="java.lang.String" value="Ventas del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
138	</Properties>	Cierra la sección XML abierta anteriormente.
139	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
140	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
141	<AbsoluteConstraints x="140" y="20" width="195" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
142	</Constraint>	Cierra la sección XML abierta anteriormente.
143	</Constraints>	Cierra la sección XML abierta anteriormente.
144	</Component>	Cierra la sección XML abierta anteriormente.
145	<Component class="javax.swing.JLabel" name="labelVentas">	Declara un componente o contenedor visual y su clase Java.
146	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
147	<Property name="text" type="java.lang.String" value="—"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
148	</Properties>	Cierra la sección XML abierta anteriormente.
149	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
150	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
151	<AbsoluteConstraints x="140" y="52" width="195" height="48"/>	Define la posición y el tamaño del componente dentro del diseño.
152	</Constraint>	Cierra la sección XML abierta anteriormente.
153	</Constraints>	Cierra la sección XML abierta anteriormente.
154	</Component>	Cierra la sección XML abierta anteriormente.
155	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
156	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
157	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
158	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
159	</Constraint>	Cierra la sección XML abierta anteriormente.
160	</Constraints>	Cierra la sección XML abierta anteriormente.
161		Línea en blanco para organizar el XML del formulario.
162	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
163	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

164	</Layout>	Cierra la sección XML abierta anteriormente.
165	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
166	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
167	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
168	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
169	<Image iconType="3" name="/Imagenes/quetzal_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
170	</Property>	Cierra la sección XML abierta anteriormente.
171	</Properties>	Cierra la sección XML abierta anteriormente.
172	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
173	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
174	<AbsoluteConstraints x="8" y="13" width="65" height="65"/>	Define la posición y el tamaño del componente dentro del diseño.
175	</Constraint>	Cierra la sección XML abierta anteriormente.
176	</Constraints>	Cierra la sección XML abierta anteriormente.
177	</Component>	Cierra la sección XML abierta anteriormente.
178	</SubComponents>	Cierra la sección XML abierta anteriormente.
179	</Container>	Cierra la sección XML abierta anteriormente.
180	</SubComponents>	Cierra la sección XML abierta anteriormente.
181	</Container>	Cierra la sección XML abierta anteriormente.
182	<Container class="Componentes.PanelFlotante" name="panelPedidos">	Declara un componente o contenedor visual y su clase Java.
183	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
184	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
185	</Properties>	Cierra la sección XML abierta anteriormente.
186	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
187	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
188	<AbsoluteConstraints x="430" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
189	</Constraint>	Cierra la sección XML abierta anteriormente.
190	</Constraints>	Cierra la sección XML abierta anteriormente.
191		Línea en blanco para organizar el XML del formulario.
192	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
193	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
194	</Layout>	Cierra la sección XML abierta anteriormente.
195	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
196	<Component class="javax.swing.JLabel" name="labelTituloPedidos">	Declara un componente o contenedor visual y su clase Java.
197	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
198	<Property name="text" type="java.lang.String" value="Pedidos cobrados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
199	</Properties>	Cierra la sección XML abierta anteriormente.
200	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
201	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
202	<AbsoluteConstraints x="140" y="20" width="195" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
203	</Constraint>	Cierra la sección XML abierta anteriormente.
204	</Constraints>	Cierra la sección XML abierta anteriormente.
205	</Component>	Cierra la sección XML abierta anteriormente.

206	<Component class="javax.swing.JLabel" name="labelPedidos">	Declara un componente o contenedor visual y su clase Java.
207	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
208	<Property name="text" type="java.lang.String" value="—" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
209	</Properties>	Cierra la sección XML abierta anteriormente.
210	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
211	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
212	<AbsoluteConstraints x="140" y="52" width="195" height="48" />	Define la posición y el tamaño del componente dentro del diseño.
213	</Constraint>	Cierra la sección XML abierta anteriormente.
214	</Constraints>	Cierra la sección XML abierta anteriormente.
215	</Component>	Cierra la sección XML abierta anteriormente.
216	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
217	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
218	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
219	<Color blue="e9" green="e9" red="fc" type="rgb" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
220	</Property>	Cierra la sección XML abierta anteriormente.
221	</Properties>	Cierra la sección XML abierta anteriormente.
222	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
223	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
224	<AbsoluteConstraints x="40" y="30" width="80" height="90" />	Define la posición y el tamaño del componente dentro del diseño.
225	</Constraint>	Cierra la sección XML abierta anteriormente.
226	</Constraints>	Cierra la sección XML abierta anteriormente.
227		Línea en blanco para organizar el XML del formulario.
228	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
229	<Property name="useNullLayout" type="boolean" value="false" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
230	</Layout>	Cierra la sección XML abierta anteriormente.
231	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
232	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
233	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
234	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
235	<Image iconType="3" name="/Imagenes/bolsa_icono.png" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
236	</Property>	Cierra la sección XML abierta anteriormente.
237	</Properties>	Cierra la sección XML abierta anteriormente.
238	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
239	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
240	<AbsoluteConstraints x="11" y="13" width="58" height="60" />	Define la posición y el tamaño del componente dentro del diseño.
241	</Constraint>	Cierra la sección XML abierta anteriormente.
242	</Constraints>	Cierra la sección XML abierta anteriormente.
243	</Component>	Cierra la sección XML abierta anteriormente.
244	</SubComponents>	Cierra la sección XML abierta anteriormente.
245	</Container>	Cierra la sección XML abierta anteriormente.
246	</SubComponents>	Cierra la sección XML abierta anteriormente.

247	</Container>	Cierra la sección XML abierta anteriormente.
248	<Container class="Componentes.PanelFlotante" name="panelTicket">	Declara un componente o contenedor visual y su clase Java.
249	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
250	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251	</Properties>	Cierra la sección XML abierta anteriormente.
252	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
253	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
254	<AbsoluteConstraints x="800" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
255	</Constraint>	Cierra la sección XML abierta anteriormente.
256	</Constraints>	Cierra la sección XML abierta anteriormente.
257		Línea en blanco para organizar el XML del formulario.
258	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
259	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
260	</Layout>	Cierra la sección XML abierta anteriormente.
261	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
262	<Component class="javax.swing.JLabel" name="labelTituloTicket">	Declara un componente o contenedor visual y su clase Java.
263	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
264	<Property name="text" type="java.lang.String" value="Ticket promedio"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
265	</Properties>	Cierra la sección XML abierta anteriormente.
266	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
267	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
268	<AbsoluteConstraints x="140" y="20" width="195" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
269	</Constraint>	Cierra la sección XML abierta anteriormente.
270	</Constraints>	Cierra la sección XML abierta anteriormente.
271	</Component>	Cierra la sección XML abierta anteriormente.
272	<Component class="javax.swing.JLabel" name="labelTicket">	Declara un componente o contenedor visual y su clase Java.
273	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
274	<Property name="text" type="java.lang.String" value="—"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
275	</Properties>	Cierra la sección XML abierta anteriormente.
276	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
277	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
278	<AbsoluteConstraints x="140" y="52" width="195" height="48"/>	Define la posición y el tamaño del componente dentro del diseño.
279	</Constraint>	Cierra la sección XML abierta anteriormente.
280	</Constraints>	Cierra la sección XML abierta anteriormente.
281	</Component>	Cierra la sección XML abierta anteriormente.
282	<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
283	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
284	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
285	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
286	</Constraint>	Cierra la sección XML abierta anteriormente.
287	</Constraints>	Cierra la sección XML abierta anteriormente.
288		Línea en blanco para organizar el XML del formulario.

289	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
290	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
291	</Layout>	Cierra la sección XML abierta anteriormente.
292	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
293	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
294	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
295	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
296	<Image iconType="3" name="/Imagenes/ticket_icono.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
297	</Property>	Cierra la sección XML abierta anteriormente.
298	</Properties>	Cierra la sección XML abierta anteriormente.
299	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
300	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
301	<AbsoluteConstraints x="11" y="15" width="60" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
302	</Constraint>	Cierra la sección XML abierta anteriormente.
303	</Constraints>	Cierra la sección XML abierta anteriormente.
304	</Component>	Cierra la sección XML abierta anteriormente.
305	</SubComponents>	Cierra la sección XML abierta anteriormente.
306	</Container>	Cierra la sección XML abierta anteriormente.
307	</SubComponents>	Cierra la sección XML abierta anteriormente.
308	</Container>	Cierra la sección XML abierta anteriormente.
309	<Container class="Componentes.PanelFlotante" name="panelCancelados">	Declara un componente o contenedor visual y su clase Java.
310	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
311	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
312	</Properties>	Cierra la sección XML abierta anteriormente.
313	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
314	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
315	<AbsoluteConstraints x="1170" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
316	</Constraint>	Cierra la sección XML abierta anteriormente.
317	</Constraints>	Cierra la sección XML abierta anteriormente.
318		Línea en blanco para organizar el XML del formulario.
319	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
320	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
321	</Layout>	Cierra la sección XML abierta anteriormente.
322	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
323	<Component class="javax.swing.JLabel" name="labelTituloCancelados">	Declara un componente o contenedor visual y su clase Java.
324	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
325	<Property name="text" type="java.lang.String" value="Cancelados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
326	</Properties>	Cierra la sección XML abierta anteriormente.
327	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
328	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
329	<AbsoluteConstraints x="140" y="20" width="195" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
330	</Constraint>	Cierra la sección XML abierta anteriormente.

331	</Constraints>	Cierra la sección XML abierta anteriormente.
332	</Component>	Cierra la sección XML abierta anteriormente.
333	<Component class="javax.swing.JLabel" name="labelCancelados">	Declara un componente o contenedor visual y su clase Java.
334	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
335	<Property name="text" type="java.lang.String" value="—" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
336	</Properties>	Cierra la sección XML abierta anteriormente.
337	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
338	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
339	<AbsoluteConstraints x="140" y="52" width="195" height="48" />	Define la posición y el tamaño del componente dentro del diseño.
340	</Constraint>	Cierra la sección XML abierta anteriormente.
341	</Constraints>	Cierra la sección XML abierta anteriormente.
342	</Component>	Cierra la sección XML abierta anteriormente.
343	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
344	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
345	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
346	<Color blue="f3" green="ef" red="ed" type="rgb" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
347	</Property>	Cierra la sección XML abierta anteriormente.
348	</Properties>	Cierra la sección XML abierta anteriormente.
349	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
350	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
351	<AbsoluteConstraints x="40" y="30" width="80" height="90" />	Define la posición y el tamaño del componente dentro del diseño.
352	</Constraint>	Cierra la sección XML abierta anteriormente.
353	</Constraints>	Cierra la sección XML abierta anteriormente.
354		Línea en blanco para organizar el XML del formulario.
355	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
356	<Property name="useNullLayout" type="boolean" value="false" />	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
357	</Layout>	Cierra la sección XML abierta anteriormente.
358	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
359	<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
360	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
361	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
362	<Image iconType="3" name="/Imagenes/x_icono.png" />	Metadato de NetBeans que conserva el estado editable del diseñador visual.
363	</Property>	Cierra la sección XML abierta anteriormente.
364	</Properties>	Cierra la sección XML abierta anteriormente.
365	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
366	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
367	<AbsoluteConstraints x="23" y="27" width="35" height="35" />	Define la posición y el tamaño del componente dentro del diseño.
368	</Constraint>	Cierra la sección XML abierta anteriormente.
369	</Constraints>	Cierra la sección XML abierta anteriormente.
370	</Component>	Cierra la sección XML abierta anteriormente.
371	</SubComponents>	Cierra la sección XML abierta anteriormente.

372	</Container>	Cierra la sección XML abierta anteriormente.
373	</SubComponents>	Cierra la sección XML abierta anteriormente.
374	</Container>	Cierra la sección XML abierta anteriormente.
375	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
376	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
377	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
378	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
379	</Properties>	Cierra la sección XML abierta anteriormente.
380	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
381	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
382	<AbsoluteConstraints x="60" y="320" width="1470" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
383	</Constraint>	Cierra la sección XML abierta anteriormente.
384	</Constraints>	Cierra la sección XML abierta anteriormente.
385		Línea en blanco para organizar el XML del formulario.
386	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
387	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
388	</Layout>	Cierra la sección XML abierta anteriormente.
389	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
390	<Component class="javax.swing.JSpinner" name="fechaReporte">	Declara un componente o contenedor visual y su clase Java.
391	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
392	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
393	<AbsoluteConstraints x="20" y="24" width="180" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
394	</Constraint>	Cierra la sección XML abierta anteriormente.
395	</Constraints>	Cierra la sección XML abierta anteriormente.
396	</Component>	Cierra la sección XML abierta anteriormente.
397	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.
398	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	<Property name="placeholder" type="java.lang.String" value="Buscar por número de orden..."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
400	</Properties>	Cierra la sección XML abierta anteriormente.
401	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
402	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
403	<AbsoluteConstraints x="215" y="22" width="330" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
404	</Constraint>	Cierra la sección XML abierta anteriormente.
405	</Constraints>	Cierra la sección XML abierta anteriormente.
406	</Component>	Cierra la sección XML abierta anteriormente.
407	<Component class="Componentes.BotonDesplegable" name="filtroCajero">	Declara un componente o contenedor visual y su clase Java.
408	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
409	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
410	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
411	</Property>	Cierra la sección XML abierta anteriormente.
412	<Property name="text" type="java.lang.String" value="Todos los cajeros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
413	<Property name="textoDesplegable" type="java.lang.String" value="Todos los cajeros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

414	</Properties>	Cierra la sección XML abierta anteriormente.
415	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
416	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
417	<AbsoluteConstraints x="560" y="24" width="235" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
418	</Constraint>	Cierra la sección XML abierta anteriormente.
419	</Constraints>	Cierra la sección XML abierta anteriormente.
420	</Component>	Cierra la sección XML abierta anteriormente.
421	<Component class="Componentes.BotonDesplegable" name="filtroMetodo">	Declara un componente o contenedor visual y su clase Java.
422	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
423	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
424	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
425	</Property>	Cierra la sección XML abierta anteriormente.
426	<Property name="text" type="java.lang.String" value="Todos los métodos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
427	<Property name="textoDesplegable" type="java.lang.String" value="Todos los métodos;Efectivo;Tarjeta;Otros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
428	</Properties>	Cierra la sección XML abierta anteriormente.
429	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
430	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
431	<AbsoluteConstraints x="810" y="24" width="200" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
432	</Constraint>	Cierra la sección XML abierta anteriormente.
433	</Constraints>	Cierra la sección XML abierta anteriormente.
434	</Component>	Cierra la sección XML abierta anteriormente.
435	<Component class="Componentes.BotonDesplegable" name="filtroServicio">	Declara un componente o contenedor visual y su clase Java.
436	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
437	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
438	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
439	</Property>	Cierra la sección XML abierta anteriormente.
440	<Property name="text" type="java.lang.String" value="Todos los servicios"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
441	<Property name="textoDesplegable" type="java.lang.String" value="Todos los servicios;En el local;Para llevar;A domicilio"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
442	</Properties>	Cierra la sección XML abierta anteriormente.
443	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
444	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
445	<AbsoluteConstraints x="1025" y="24" width="205" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
446	</Constraint>	Cierra la sección XML abierta anteriormente.
447	</Constraints>	Cierra la sección XML abierta anteriormente.
448	</Component>	Cierra la sección XML abierta anteriormente.
449	<Component class="Componentes.BotonRedondeado" name="botonLimpiarFiltros">	Declara un componente o contenedor visual y su clase Java.
450	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
451	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
452	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

453	</Property>	Cierra la sección XML abierta anteriormente.
454	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
455	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
456	</Properties>	Cierra la sección XML abierta anteriormente.
457	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
458	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
459	<AbsoluteConstraints x="1245" y="24" width="200" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
460	</Constraint>	Cierra la sección XML abierta anteriormente.
461	</Constraints>	Cierra la sección XML abierta anteriormente.
462	</Component>	Cierra la sección XML abierta anteriormente.
463	</SubComponents>	Cierra la sección XML abierta anteriormente.
464	</Container>	Cierra la sección XML abierta anteriormente.
465	<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
466	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
467	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
468	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
469	</Properties>	Cierra la sección XML abierta anteriormente.
470	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
471	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
472	<AbsoluteConstraints x="60" y="435" width="1470" height="340"/>	Define la posición y el tamaño del componente dentro del diseño.
473	</Constraint>	Cierra la sección XML abierta anteriormente.
474	</Constraints>	Cierra la sección XML abierta anteriormente.
475		Línea en blanco para organizar el XML del formulario.
476	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
477	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
478	</Layout>	Cierra la sección XML abierta anteriormente.
479	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
480	<Container class="javax.swing.JScrollPane" name="scrollPedidos">	Declara un componente o contenedor visual y su clase Java.
481	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
482	<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
483	<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
484	</Property>	Cierra la sección XML abierta anteriormente.
485	<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
486	<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
487	</Properties>	Cierra la sección XML abierta anteriormente.
488	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
489	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
490	<AbsoluteConstraints x="14" y="14" width="1442" height="250"/>	Define la posición y el tamaño del componente dentro del diseño.
491	</Constraint>	Cierra la sección XML abierta anteriormente.
492	</Constraints>	Cierra la sección XML abierta anteriormente.
493		Línea en blanco para organizar el XML del formulario.
494	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.

495	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
496	<Component class="Componentes.TablaAdministrativa" name="tablaPedidos">	Declara un componente o contenedor visual y su clase Java.
497	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
498	<Property name="model" type="javax.swing.table.TableModel" editor="org.netbeans.modules.form.editors2.TableModelEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
499	<Table columnCount="9" rowCount="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
500	<Column editable="false" title="N° orden" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
501	<Column editable="false" title="Fecha/Hora" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
502	<Column editable="false" title="Cajero" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
503	<Column editable="false" title="Tipo servicio" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
504	<Column editable="false" title="Estado" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
505	<Column editable="false" title="Método de pago" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
506	<Column editable="false" title="Monto recibido" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
507	<Column editable="false" title="Total" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
508	<Column editable="false" title="Cantidad de ítems" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
509	</Table>	Cierra la sección XML abierta anteriormente.
510	</Property>	Cierra la sección XML abierta anteriormente.
511	</Properties>	Cierra la sección XML abierta anteriormente.
512	</Component>	Cierra la sección XML abierta anteriormente.
513	</SubComponents>	Cierra la sección XML abierta anteriormente.
514	</Container>	Cierra la sección XML abierta anteriormente.
515	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
516	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
517	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
518	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
519	</Property>	Cierra la sección XML abierta anteriormente.
520	<Property name="text" type="java.lang.String" value="No se encontraron ventas"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
521	</Properties>	Cierra la sección XML abierta anteriormente.
522	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
523	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
524	<AbsoluteConstraints x="28" y="278" width="420" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
525	</Constraint>	Cierra la sección XML abierta anteriormente.
526	</Constraints>	Cierra la sección XML abierta anteriormente.
527	</Component>	Cierra la sección XML abierta anteriormente.
528	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
529	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
530	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
531	</Properties>	Cierra la sección XML abierta anteriormente.
532	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
533	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
534	<AbsoluteConstraints x="1165" y="278" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
535	</Constraint>	Cierra la sección XML abierta anteriormente.
536	</Constraints>	Cierra la sección XML abierta anteriormente.

537		Línea en blanco para organizar el XML del formulario.
538	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
539	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
540	</Layout>	Cierra la sección XML abierta anteriormente.
541	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
542	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
543	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
544	<Property name="text" type="java.lang.String" value="#x2039;"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
545	</Properties>	Cierra la sección XML abierta anteriormente.
546	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
547	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
548	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
549	</Constraint>	Cierra la sección XML abierta anteriormente.
550	</Constraints>	Cierra la sección XML abierta anteriormente.
551	</Component>	Cierra la sección XML abierta anteriormente.
552	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
553	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
554	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
555	</Properties>	Cierra la sección XML abierta anteriormente.
556	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
557	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
558	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
559	</Constraint>	Cierra la sección XML abierta anteriormente.
560	</Constraints>	Cierra la sección XML abierta anteriormente.
561	</Component>	Cierra la sección XML abierta anteriormente.
562	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
563	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
564	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
565	</Properties>	Cierra la sección XML abierta anteriormente.
566	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
567	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
568	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
569	</Constraint>	Cierra la sección XML abierta anteriormente.
570	</Constraints>	Cierra la sección XML abierta anteriormente.
571	</Component>	Cierra la sección XML abierta anteriormente.
572	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
573	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
574	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
575	</Properties>	Cierra la sección XML abierta anteriormente.
576	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
577	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
578	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.

579	</Constraint>	Cierra la sección XML abierta anteriormente.
580	</Constraints>	Cierra la sección XML abierta anteriormente.
581	</Component>	Cierra la sección XML abierta anteriormente.
582	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
583	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
584	<Property name="text" type="java.lang.String" value="›"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
585	</Properties>	Cierra la sección XML abierta anteriormente.
586	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
587	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
588	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
589	</Constraint>	Cierra la sección XML abierta anteriormente.
590	</Constraints>	Cierra la sección XML abierta anteriormente.
591	</Component>	Cierra la sección XML abierta anteriormente.
592	</SubComponents>	Cierra la sección XML abierta anteriormente.
593	</Container>	Cierra la sección XML abierta anteriormente.
594	</SubComponents>	Cierra la sección XML abierta anteriormente.
595	</Container>	Cierra la sección XML abierta anteriormente.
596	<Container class="Componentes.PanelFlotante" name="panelResumen">	Declara un componente o contenedor visual y su clase Java.
597	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
598	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
599	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
600	</Properties>	Cierra la sección XML abierta anteriormente.
601	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
602	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
603	<AbsoluteConstraints x="60" y="790" width="1470" height="110"/>	Define la posición y el tamaño del componente dentro del diseño.
604	</Constraint>	Cierra la sección XML abierta anteriormente.
605	</Constraints>	Cierra la sección XML abierta anteriormente.
606		Línea en blanco para organizar el XML del formulario.
607	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
608	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
609	</Layout>	Cierra la sección XML abierta anteriormente.
610	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
611	<Component class="javax.swing.JLabel" name="labelResumen">	Declara un componente o contenedor visual y su clase Java.
612	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
613	<Property name="text" type="java.lang.String" value="Resumen de caja"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
614	</Properties>	Cierra la sección XML abierta anteriormente.
615	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
616	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
617	<AbsoluteConstraints x="28" y="18" width="180" height="28"/>	Define la posición y el tamaño del componente dentro del diseño.
618	</Constraint>	Cierra la sección XML abierta anteriormente.
619	</Constraints>	Cierra la sección XML abierta anteriormente.
620	</Component>	Cierra la sección XML abierta anteriormente.

621	<Component class="javax.swing.JLabel" name="labelResumenSubtitulo">	Declara un componente o contenedor visual y su clase Java.
622	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
623	<Property name="text" type="java.lang.String" value="Totales del día"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
624	</Properties>	Cierra la sección XML abierta anteriormente.
625	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
626	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
627	<AbsoluteConstraints x="28" y="50" width="180" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
628	</Constraint>	Cierra la sección XML abierta anteriormente.
629	</Constraints>	Cierra la sección XML abierta anteriormente.
630	</Component>	Cierra la sección XML abierta anteriormente.
631	<Component class="javax.swing.JSeparator" name="separadorResumen1">	Declara un componente o contenedor visual y su clase Java.
632	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
633	<Property name="orientation" type="int" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
634	</Properties>	Cierra la sección XML abierta anteriormente.
635	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
636	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
637	<AbsoluteConstraints x="225" y="16" width="1" height="78"/>	Define la posición y el tamaño del componente dentro del diseño.
638	</Constraint>	Cierra la sección XML abierta anteriormente.
639	</Constraints>	Cierra la sección XML abierta anteriormente.
640	</Component>	Cierra la sección XML abierta anteriormente.
641	<Component class="javax.swing.JLabel" name="efectivoTitulo">	Declara un componente o contenedor visual y su clase Java.
642	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
643	<Property name="text" type="java.lang.String" value="Efectivo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
644	</Properties>	Cierra la sección XML abierta anteriormente.
645	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
646	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
647	<AbsoluteConstraints x="250" y="14" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
648	</Constraint>	Cierra la sección XML abierta anteriormente.
649	</Constraints>	Cierra la sección XML abierta anteriormente.
650	</Component>	Cierra la sección XML abierta anteriormente.
651	<Component class="javax.swing.JLabel" name="labelEfectivo">	Declara un componente o contenedor visual y su clase Java.
652	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
653	<Property name="text" type="java.lang.String" value="—"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
654	</Properties>	Cierra la sección XML abierta anteriormente.
655	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
656	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
657	<AbsoluteConstraints x="250" y="38" width="250" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
658	</Constraint>	Cierra la sección XML abierta anteriormente.
659	</Constraints>	Cierra la sección XML abierta anteriormente.
660	</Component>	Cierra la sección XML abierta anteriormente.
661	<Component class="javax.swing.JLabel" name="labelPorcentajeEfectivo">	Declara un componente o contenedor visual y su clase Java.
662	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

663	<Property name="text" type="java.lang.String" value="0% del total"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
664	</Properties>	Cierra la sección XML abierta anteriormente.
665	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
666	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
667	<AbsoluteConstraints x="250" y="73" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
668	</Constraint>	Cierra la sección XML abierta anteriormente.
669	</Constraints>	Cierra la sección XML abierta anteriormente.
670	</Component>	Cierra la sección XML abierta anteriormente.
671	<Component class="javax.swing.JSeparator" name="separadorResumen2">	Declara un componente o contenedor visual y su clase Java.
672	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
673	<Property name="orientation" type="int" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
674	</Properties>	Cierra la sección XML abierta anteriormente.
675	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
676	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
677	<AbsoluteConstraints x="530" y="16" width="1" height="78"/>	Define la posición y el tamaño del componente dentro del diseño.
678	</Constraint>	Cierra la sección XML abierta anteriormente.
679	</Constraints>	Cierra la sección XML abierta anteriormente.
680	</Component>	Cierra la sección XML abierta anteriormente.
681	<Component class="javax.swing.JLabel" name="tarjetaTitulo">	Declara un componente o contenedor visual y su clase Java.
682	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
683	<Property name="text" type="java.lang.String" value="Tarjeta"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
684	</Properties>	Cierra la sección XML abierta anteriormente.
685	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
686	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
687	<AbsoluteConstraints x="555" y="14" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
688	</Constraint>	Cierra la sección XML abierta anteriormente.
689	</Constraints>	Cierra la sección XML abierta anteriormente.
690	</Component>	Cierra la sección XML abierta anteriormente.
691	<Component class="javax.swing.JLabel" name="labelTarjeta">	Declara un componente o contenedor visual y su clase Java.
692	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
693	<Property name="text" type="java.lang.String" value="—"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
694	</Properties>	Cierra la sección XML abierta anteriormente.
695	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
696	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
697	<AbsoluteConstraints x="555" y="38" width="250" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
698	</Constraint>	Cierra la sección XML abierta anteriormente.
699	</Constraints>	Cierra la sección XML abierta anteriormente.
700	</Component>	Cierra la sección XML abierta anteriormente.
701	<Component class="javax.swing.JLabel" name="labelPorcentajeTarjeta">	Declara un componente o contenedor visual y su clase Java.
702	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
703	<Property name="text" type="java.lang.String" value="0% del total"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
704	</Properties>	Cierra la sección XML abierta anteriormente.

705	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
706	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
707	<AbsoluteConstraints x="555" y="73" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
708	</Constraint>	Cierra la sección XML abierta anteriormente.
709	</Constraints>	Cierra la sección XML abierta anteriormente.
710	</Component>	Cierra la sección XML abierta anteriormente.
711	<Component class="javax.swing.JSeparator" name="separadorResumen3">	Declara un componente o contenedor visual y su clase Java.
712	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
713	<Property name="orientation" type="int" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
714	</Properties>	Cierra la sección XML abierta anteriormente.
715	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
716	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
717	<AbsoluteConstraints x="835" y="16" width="1" height="78"/>	Define la posición y el tamaño del componente dentro del diseño.
718	</Constraint>	Cierra la sección XML abierta anteriormente.
719	</Constraints>	Cierra la sección XML abierta anteriormente.
720	</Component>	Cierra la sección XML abierta anteriormente.
721	<Component class="javax.swing.JLabel" name="otrosTitulo">	Declara un componente o contenedor visual y su clase Java.
722	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
723	<Property name="text" type="java.lang.String" value="Otros métodos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
724	</Properties>	Cierra la sección XML abierta anteriormente.
725	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
726	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
727	<AbsoluteConstraints x="860" y="14" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
728	</Constraint>	Cierra la sección XML abierta anteriormente.
729	</Constraints>	Cierra la sección XML abierta anteriormente.
730	</Component>	Cierra la sección XML abierta anteriormente.
731	<Component class="javax.swing.JLabel" name="labelOtros">	Declara un componente o contenedor visual y su clase Java.
732	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
733	<Property name="text" type="java.lang.String" value="—"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
734	</Properties>	Cierra la sección XML abierta anteriormente.
735	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
736	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
737	<AbsoluteConstraints x="860" y="38" width="250" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
738	</Constraint>	Cierra la sección XML abierta anteriormente.
739	</Constraints>	Cierra la sección XML abierta anteriormente.
740	</Component>	Cierra la sección XML abierta anteriormente.
741	<Component class="javax.swing.JLabel" name="labelPorcentajeOtros">	Declara un componente o contenedor visual y su clase Java.
742	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
743	<Property name="text" type="java.lang.String" value="0% del total"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
744	</Properties>	Cierra la sección XML abierta anteriormente.
745	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

746	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
747	<AbsoluteConstraints x="860" y="73" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
748	</Constraint>	Cierra la sección XML abierta anteriormente.
749	</Constraints>	Cierra la sección XML abierta anteriormente.
750	</Component>	Cierra la sección XML abierta anteriormente.
751	<Component class="javax.swing.JSeparator" name="separadorResumen4">	Declara un componente o contenedor visual y su clase Java.
752	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
753	<Property name="orientation" type="int" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
754	</Properties>	Cierra la sección XML abierta anteriormente.
755	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
756	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
757	<AbsoluteConstraints x="1140" y="16" width="1" height="78"/>	Define la posición y el tamaño del componente dentro del diseño.
758	</Constraint>	Cierra la sección XML abierta anteriormente.
759	</Constraints>	Cierra la sección XML abierta anteriormente.
760	</Component>	Cierra la sección XML abierta anteriormente.
761	<Component class="javax.swing.JLabel" name="cancelacionTitulo">	Declara un componente o contenedor visual y su clase Java.
762	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
763	<Property name="text" type="java.lang.String" value="Cancelaciones"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
764	</Properties>	Cierra la sección XML abierta anteriormente.
765	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
766	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
767	<AbsoluteConstraints x="1165" y="14" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
768	</Constraint>	Cierra la sección XML abierta anteriormente.
769	</Constraints>	Cierra la sección XML abierta anteriormente.
770	</Component>	Cierra la sección XML abierta anteriormente.
771	<Component class="javax.swing.JLabel" name="labelCancelacionesCaja">	Declara un componente o contenedor visual y su clase Java.
772	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
773	<Property name="text" type="java.lang.String" value="0 pedidos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
774	</Properties>	Cierra la sección XML abierta anteriormente.
775	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
776	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
777	<AbsoluteConstraints x="1165" y="38" width="250" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
778	</Constraint>	Cierra la sección XML abierta anteriormente.
779	</Constraints>	Cierra la sección XML abierta anteriormente.
780	</Component>	Cierra la sección XML abierta anteriormente.
781	<Component class="javax.swing.JLabel" name="cancelacionDetalle">	Declara un componente o contenedor visual y su clase Java.
782	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
783	<Property name="text" type="java.lang.String" value="Pedidos cancelados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
784	</Properties>	Cierra la sección XML abierta anteriormente.
785	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
786	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

787	<AbsoluteConstraints x="1165" y="73" width="250" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
788	</Constraint>	Cierra la sección XML abierta anteriormente.
789	</Constraints>	Cierra la sección XML abierta anteriormente.
790	</Component>	Cierra la sección XML abierta anteriormente.
791	</SubComponents>	Cierra la sección XML abierta anteriormente.
792	</Container>	Cierra la sección XML abierta anteriormente.
793	<Component class="javax.swing.JLabel" name="estadoCarga">	Declara un componente o contenedor visual y su clase Java.
794	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
795	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
796	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
797	</Property>	Cierra la sección XML abierta anteriormente.
798	<Property name="text" type="java.lang.String" value="Selecciona una fecha y actualiza el reporte."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
799	</Properties>	Cierra la sección XML abierta anteriormente.
800	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
801	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
802	<AbsoluteConstraints x="60" y="904" width="1450" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
803	</Constraint>	Cierra la sección XML abierta anteriormente.
804	</Constraints>	Cierra la sección XML abierta anteriormente.
805	</Component>	Cierra la sección XML abierta anteriormente.
806	</SubComponents>	Cierra la sección XML abierta anteriormente.
807	</Form>	Cierra la sección XML abierta anteriormente.

16.83 src/GUI_ADMINISTRADOR/ReportesPanel.java

Dashboard de reportes con indicadores, filtros, tabla, paginación, exportación e impresión.

Tipo: java | Líneas: 1239 | Archivo 83 de 157

Clases/tipos detectados: ReportesPanel, ModeloTablaVentas, RenderEstado, RenderMoneda

Línea	Constructor o método detectado
69	private final DecimalFormat moneda = new DecimalFormat(
86	public ReportesPanel() {
100	private void configurarVista() {
153	private void configurarTarjeta(PanelFlotante panel,
165	private void configurarResumen(
175	private void configurarBotonAccion(
188	private void configurarFiltros() {
231	private void configurarSelectorFecha() {
246	private void configurarFiltro(BotonDesplegable boton, int ancho) {
262	private void configurarTabla() {
305	private void configurarAnchosColumnas() {
313	private void configurarBotonPagina(
330	private void configurarEventos() {
360	public void actualizarReporte() {
418	private void mostrarResumen(ReporteDAO.Resumen resultado) {
435	private String porcentaje(BigDecimal parte, BigDecimal total) {
442	private void cargarOpciones(List<Object[]> ventas) {
463	private void aplicarFiltros() {
496	private void limpiarFiltros() {
509	private int totalPaginas() {
514	private void mostrarPagina() {
525	private void actualizarPaginacion(
553	private void cambiarPagina(int pagina) {
560	private void habilitarConsulta(boolean habilitada) {
565	private void limpiarResultados() {
582	private void exportarReporte() {
613	private void imprimirReporte() {
631	public void removeNotify() {
642	private void initComponents() {

Línea	Constructor o método detectado
867	public boolean isCellEditable(int rowIndex, int columnIndex) {
966	private org.netbeans.lib.awtextra.AbsoluteConstraints posicion(
972	private JLabel etiqueta(String texto, int tamaño, boolean negrita) {
980	private BotonRedondeado boton(String texto) {
989	private PanelFlotante panel() {
998	private PanelFlotante tarjeta() {
1004	private PanelCircular icono(PanelFlotante tarjeta, String ruta) {
1014	private JLabel tituloTarjeta(PanelFlotante panel, String texto) {
1020	private JLabel valorTarjeta(PanelFlotante panel) {
1026	private BotonDesplegable filtro(String texto, String opciones) {
1034	private javax.swing.JSeparator separador(PanelFlotante panel, int x) {
1041	private JLabel tituloResumen(
1048	private JLabel valorResumen(PanelFlotante panel, int x) {
1054	private JLabel detalleResumen(
1069	private void setFilas(List<Object[]> nuevasFilas) {
1097	private String metodoVisible(Object valor) {
1109	public Component getTableCellRendererComponent(
1148	private RenderMoneda(boolean resaltar) {
1153	public Component getTableCellRendererComponent(
1169	private Color fondoFila(

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
5	import Componentes.PanelCircular;	Importa Componentes.PanelCircular para poder usar esa clase sin escribir su nombre completo.
6	import Componentes.PanelFlotante;	Importa Componentes.PanelFlotante para poder usar esa clase sin escribir su nombre completo.
7	import DAO.ReporteDAO;	Importa DAO.ReporteDAO para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.
9	import Utilidades.ReporteCsv;	Importa Utilidades.ReporteCsv para poder usar esa clase sin escribir su nombre completo.
10	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
15	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.

16	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
17	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
18	import java.math.RoundingMode;	Importa java.math.RoundingMode para poder usar esa clase sin escribir su nombre completo.
19	import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
20	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
21	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
22	import java.text.DecimalFormat;	Importa java.text.DecimalFormat para poder usar esa clase sin escribir su nombre completo.
23	import java.text.DecimalFormatSymbols;	Importa java.text.DecimalFormatSymbols para poder usar esa clase sin escribir su nombre completo.
24	import java.text.MessageFormat;	Importa java.text.MessageFormat para poder usar esa clase sin escribir su nombre completo.
25	import java.time.LocalDate;	Importa java.time.LocalDate: representa una fecha sin hora.
26	import java.time.ZoneId;	Importa java.time.ZoneId para poder usar esa clase sin escribir su nombre completo.
27	import java.time.format.DateTimeFormatter;	Importa java.time.format.DateTimeFormatter para poder usar esa clase sin escribir su nombre completo.
28	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
29	import java.util.Date;	Importa java.util.Date para poder usar esa clase sin escribir su nombre completo.
30	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
31	import java.util.Locale;	Importa java.util.Locale para poder usar esa clase sin escribir su nombre completo.
32	import java.util.Set;	Importa java.util.Set: colección que evita elementos repetidos.
33	import java.util.TreeSet;	Importa java.util.TreeSet para poder usar esa clase sin escribir su nombre completo.
34	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
35	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
36	import javax.swing.JFileChooser;	Importa javax.swing.JFileChooser para poder usar esa clase sin escribir su nombre completo.
37	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
38	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
39	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
40	import javax.swing.JSpinner;	Importa javax.swing.JSpinner para poder usar esa clase sin escribir su nombre completo.
41	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
42	import javax.swing.ListSelectionModel;	Importa javax.swing.ListSelectionModel para poder usar esa clase sin escribir su nombre completo.
43	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
44	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
45	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
46	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
47	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
48	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
49	import javax.swing.filechooser.FileNameExtensionFilter;	Importa javax.swing.filechooser.FileNameExtensionFilter para poder usar esa clase sin escribir su nombre completo.
50	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
51	import javax.swing.table.DefaultTableCellRenderer;	Importa javax.swing.table.DefaultTableCellRenderer para poder usar esa clase sin escribir su nombre completo.
52	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	/** Panel administrativo de ventas diarias. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
55	public class ReportesPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	private static final int VENTAS_POR_PAGINA = 8;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
58	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
59	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

60	private static final Color BORDE = new Color(225, 229, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
61	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
62	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
63	private static final Color VERDE = new Color(34, 139, 71);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
64	private static final Color NARANJA = new Color(235, 79, 35);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
67	private final ReporteDAO reporteDAO = new ReporteDAO();	Crea una nueva instancia de ReporteDAO y la usa o guarda para continuar el proceso.
68	private final ModeloTablaVentas modeloTabla = new ModeloTablaVentas();	Crea una nueva instancia de ModeloTablaVentas y la usa o guarda para continuar el proceso.
69	private final DecimalFormat moneda = new DecimalFormat(	Crea una nueva instancia de DecimalFormat y la usa o guarda para continuar el proceso.
70	"'Q'#,##0.00",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	DecimalFormatSymbols.getInstance(Locale.US));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	private final DateTimeFormatter fechaVisible =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
73	DateTimeFormatter.ofPattern("dd/MM/yyyy");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	private final List<BotonRedondeado> botonesPagina = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
75	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	private SwingWorker<ReporteDAO.Resumen, Void> carga;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	private ReporteDAO.Resumen ultimoReporte;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	private LocalDate fechaCargada;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
80	private List<Object[]> ventasFiltradas = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	private String cajeroSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
82	private String metodoSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	private String servicioSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
84	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	public ReportesPanel() {	Constructor de ReportesPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
87	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	temporizadorBusqueda = new Timer(250, evento -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
89	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
90	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
93	configurarVista();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	configurarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	configurarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	limpiarResultados();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	}	Cierra el bloque abierto anteriormente.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	private void configurarVista() {	Declara el método configurarVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
101	setBackground(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
102	labelTitulo.setFont(tema.negrita(36f));	Configura una propiedad mediante setFont en el objeto indicado.
103	labelSubtitulo.setFont(tema.regular(16f));	Configura una propiedad mediante setFont en el objeto indicado.

104	labelSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	botonActualizar.setFont(tema.negrita(15f));	Configura una propiedad mediante setFont en el objeto indicado.
107	botonActualizar.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
108	botonActualizar.addActionListener(e -> actualizarReporte());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
109	configurarTarjeta(panelVentas,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
110	labelTituloVentas, labelVentas, false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
111	configurarTarjeta(panelPedidos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
112	labelTituloPedidos, labelPedidos, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113	configurarTarjeta(panelTicket,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	labelTituloTicket, labelTicket, false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
115	configurarTarjeta(panelCancelados,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
116	labelTituloCancelados, labelCancelados, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	labelCancelados.setForeground(ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
118	configurarBotonAccion(botonExportar, ROJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	botonExportar.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
120	IconosUsuarios.Tipo.EXPORTAR, ROJO, 20));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	botonExportar.setIconTextGap(10);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
122	botonExportar.addActionListener(e -> exportarReporte());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	configurarBotonAccion(botonImprimir, AZUL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	botonImprimir.addActionListener(e -> imprimirReporte());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
126	labelCancelados.setForeground(ROJO);	Configura una propiedad mediante setForeground en el objeto indicado.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	panelResumen.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
129	panelResumen.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
130	for (javax.swing.JSeparator separador : new javax.swing.JSeparator[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
131	separadorResumen1, separadorResumen2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	separadorResumen3, separadorResumen4	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	}) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
134	separador.setForeground(BORDE);	Configura una propiedad mediante setForeground en el objeto indicado.
135	separador.setBackground(BORDE);	Configura una propiedad mediante setBackground en el objeto indicado.
136	}	Cierra el bloque abierto anteriormente.
137	labelResumen.setFont(tema.negrita(17f));	Configura una propiedad mediante setFont en el objeto indicado.
138	labelResumenSubtitulo.setFont(tema.regular(13f));	Configura una propiedad mediante setFont en el objeto indicado.
139	labelResumenSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
140	configurarResumen(efectivoTitulo, labelEfectivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
141	labelPorcentajeEfectivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	configurarResumen(tarjetaTitulo, labelTarjeta,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
143	labelPorcentajeTarjeta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	configurarResumen(otrosTitulo, labelOtros,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	labelPorcentajeOtros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146	configurarResumen(cancelacionTitulo, labelCancelacionesCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
147	cancelacionDetalle);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

148	labelCancelacionesCaja.setForeground(R0J0);	Configura una propiedad mediante setForeground en el objeto indicado.
149	estadoCarga.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
150	estadoCarga.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
151	}	Cierra el bloque abierto anteriormente.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	private void configurarTarjeta(PanelFlotante panel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	JLabel titulo, JLabel valor, boolean roja) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	panel.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
156	panel.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
157	panel.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
158	panel.setSombra(true);	Configura una propiedad mediante setSombra en el objeto indicado.
159	panel.setTamanoSombra(9);	Configura una propiedad mediante setTamanoSombra en el objeto indicado.
160	titulo.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
161	titulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
162	valor.setFont(tema.negrita(28f));	Configura una propiedad mediante setFont en el objeto indicado.
163	}	Cierra el bloque abierto anteriormente.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	private void configurarResumen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	JLabel titulo, JLabel valor, JLabel detalle) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	titulo.setFont(tema.regular(13f));	Configura una propiedad mediante setFont en el objeto indicado.
168	titulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
169	valor.setFont(tema.negrita(21f));	Configura una propiedad mediante setFont en el objeto indicado.
170	valor.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
171	detalle.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
172	detalle.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
173	}	Cierra el bloque abierto anteriormente.
174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
175	private void configurarBotonAccion(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	BotonRedondeado boton, Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	boton.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
178	boton.setForeground(color);	Configura una propiedad mediante setForeground en el objeto indicado.
179	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
180	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
181	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
182	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
183	boton.setGrosorBorde(1.5f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
184	boton.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
185	boton.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
186	}	Cierra el bloque abierto anteriormente.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	private void configurarFiltros() {	Declara el método configurarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
189	panelFiltros.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
190	panelFiltros.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
191	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.

192	configurarSelectorFecha();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
194	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
195	campoBusqueda.setPlaceholder("Buscar por número de orden...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
196	configurarFiltro(filtroCajero, 225);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	configurarFiltro(filtroMetodo, 190);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198	configurarFiltro(filtroServicio, 195);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	filtroCajero.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
201	String opcion = e.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
202	cajeroSeleccionado = opcion.startsWith("Todos") ? null : opcion;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
203	filtroCajero.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
204	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
205	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
207	filtroMetodo.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
208	String opcion = e.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
209	metodoSeleccionado = "Todos los métodos".equals(opcion)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
210	? null : opcion;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
211	filtroMetodo.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
212	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
213	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
214	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215	filtroServicio.addMenuOpcionListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	String opcion = e.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
217	servicioSeleccionado = "Todos los servicios".equals(opcion)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
218	? null : opcion;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219	filtroServicio.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
220	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
221	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
222	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	configurarBotonAccion(botonLimpiarFiltros, R0J0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
225	botonLimpiarFiltros.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
226	IconosUsuarios.Tipo.FILTRO, R0J0, 20));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227	botonLimpiarFiltros.setIconTextGap(9);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
228	botonLimpiarFiltros.addActionListener(e -> limpiarFiltros());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
229	}	Cierra el bloque abierto anteriormente.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	private void configurarSelectorFecha() {	Declara el método configurarSelectorFecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
232	fechaReporte.setModel(new javax.swing.SpinnerDateModel(	Crea una nueva instancia de javax.swing.SpinnerDateModel y la usa o guarda para continuar el proceso.
233	new Date(), null, null, java.util.Calendar.DAY_OF_MONTH));	Crea una nueva instancia de Date y la usa o guarda para continuar el proceso.
234	JSpinner.DateEditor editor =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
235	new JSpinner.DateEditor(fechaReporte, "dd/MM/yyyy");	Crea una nueva instancia de JSpinner.DateEditor y la usa o guarda para continuar el proceso.

236	editor.getFormat().setLenient(false);	Configura una propiedad mediante setLenient en el objeto indicado.
237	editor.getTextField().setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
238	editor.getTextField().setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
239	editor.getTextField().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
240	editor.getTextField().setBorder(new EmptyBorder(0, 14, 0, 8));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
241	fechaReporte.setEditor(editor);	Configura una propiedad mediante setEditor en el objeto indicado.
242	fechaReporte.setBorder(BorderFactory.createLineBorder(BORDE, 1, true));	Configura una propiedad mediante setBorder en el objeto indicado.
243	fechaReporte.setToolTipText("Fecha del reporte");	Configura una propiedad mediante setToolTipText en el objeto indicado.
244	}	Cierra el bloque abierto anteriormente.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	private void configurarFiltro(BotonDesplegable boton, int ancho) {	Declara el método configurarFiltro. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
247	boton.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
248	boton.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
249	boton.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
250	boton.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
251	boton.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
252	boton.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
253	boton.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
254	boton.setAnchoMenu(ancho);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
255	boton.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
256	boton.setBorderPainted(true);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
257	boton.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
258	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	new EmptyBorder(0, 14, 0, 34)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
260	}	Cierra el bloque abierto anteriormente.
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
263	panelTabla.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
264	panelTabla.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
265	panelTabla.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
266	tablaPedidos.setModel(modeloTabla);	Configura una propiedad mediante setModel en el objeto indicado.
267	tablaPedidos.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268	tablaPedidos.setSelectionMode(ListSelectionMode.SINGLE_SELECTION);	Configura una propiedad mediante setSelectionMode en el objeto indicado.
269	tablaPedidos.setRowHeight(32);	Configura una propiedad mediante setRowHeight en el objeto indicado.
270	tablaPedidos.setFilasAlternadas(true);	Configura una propiedad mediante setFilasAlternadas en el objeto indicado.
271	tablaPedidos.setAltoCabecera(42);	Configura una propiedad mediante setAltoCabecera en el objeto indicado.
272	tablaPedidos.setColumnsCentradas("4,8");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
273	tablaPedidos.setColumnsDerecha("6,7");	Configura una propiedad mediante setColumnsDerecha en el objeto indicado.
274	tablaPedidos.getColumnModel().getColumn(4)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	.setCellRenderer(new RenderEstado());	Crea una nueva instancia de RenderEstado y la usa o guarda para continuar el proceso.
276	tablaPedidos.getColumnModel().getColumn(6)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
277	.setCellRenderer(new RenderMoneda(false));	Crea una nueva instancia de RenderMoneda y la usa o guarda para continuar el proceso.
278	tablaPedidos.getColumnModel().getColumn(7)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
279	.setCellRenderer(new RenderMoneda(true));	Crea una nueva instancia de RenderMoneda y la usa o guarda para continuar el proceso.

280	<code>configurarAnchosColumnas();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
281		Línea en blanco: separa visualmente bloques para que el código sea más legible.
282	<code>scrollPedidos.setBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
283	<code>scrollPedidos.getViewPort().setBackground(Color.WHITE);</code>	Configura una propiedad mediante <code>setBackground</code> en el objeto indicado.
284	<code>etiquetaRango.setFont(tema.regular(12f));</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
285	<code>etiquetaRango.setForeground(SECUNDARIO);</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
286		Línea en blanco: separa visualmente bloques para que el código sea más legible.
287	<code>configurarBotonPagina(botonAnterior, IconosUsuarios.Tipo.ANTERIOR);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
288	<code>botonAnterior.addActionListener(e -> cambiarPagina(paginaActual - 1));</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
289		Línea en blanco: separa visualmente bloques para que el código sea más legible.
290	<code>botonesPagina.add(botonPagina1);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
291	<code>botonesPagina.add(botonPagina2);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
292	<code>botonesPagina.add(botonPagina3);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
293	<code>for (BotonRedondeado boton : botonesPagina) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
294	<code> configurarBotonPagina(boton, null);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
295	<code> boton.addActionListener(e -> {</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
296	<code> Object valor = boton.getClientProperty("pagina");</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
297	<code> if (valor instanceof Integer pagina) cambiarPagina(pagina);</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
298	<code> });</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
299	<code>}</code>	Cierra el bloque abierto anteriormente.
300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
301	<code>configurarBotonPagina(botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	<code>botonSiguiente.addActionListener(e -> cambiarPagina(paginaActual + 1));</code>	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
303	<code>}</code>	Cierra el bloque abierto anteriormente.
304		Línea en blanco: separa visualmente bloques para que el código sea más legible.
305	<code>private void configurarAnchosColumnas() {</code>	Declara el método <code>configurarAnchosColumnas</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
306	<code> int[] anchos = {78, 145, 165, 135, 115, 145, 130, 120, 105};</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
307	<code> for (int i = 0; i < anchos.length; i++) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
308	<code> tablaPedidos.getColumnModel().getColumn(i)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
309	<code> .setPreferredWidth(anchos[i]);</code>	Configura una propiedad mediante <code>setPreferredWidth</code> en el objeto indicado.
310	<code> }</code>	Cierra el bloque abierto anteriormente.
311	<code>}</code>	Cierra el bloque abierto anteriormente.
312		Línea en blanco: separa visualmente bloques para que el código sea más legible.
313	<code>private void configurarBotonPagina(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
314	<code> BotonRedondeado boton, IconosUsuarios.Tipo tipoIcono) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	<code> boton.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.
316	<code> boton.setColorInicio(Color.WHITE);</code>	Configura una propiedad mediante <code>setColorInicio</code> en el objeto indicado.
317	<code> boton.setColorFinal(Color.WHITE);</code>	Configura una propiedad mediante <code>setColorFinal</code> en el objeto indicado.
318	<code> boton.setColorBorde(BORDE);</code>	Configura una propiedad mediante <code>setColorBorde</code> en el objeto indicado.
319	<code> boton.setGrosorBorde(1f);</code>	Configura una propiedad mediante <code>setGrosorBorde</code> en el objeto indicado.
320	<code> boton.setRadio(12);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
321	<code> boton.setPreferredSize(new Dimension(40, 38));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
322	<code> boton.setFont(tema.media(13f));</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
323	<code> boton.setFocusPainted(false);</code>	Configura una propiedad mediante <code>setFocusPainted</code> en el objeto indicado.

324	if (tipoIcono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
325	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
326	boton.setIcon(IconosUsuarios.crear(tipoIcono, AZUL, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
327	}	Cierra el bloque abierto anteriormente.
328	}	Cierra el bloque abierto anteriormente.
329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
330	private void configurarEventos() {	Declara el método configurarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
331	campoBusqueda.getDocument().addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
332	@Override public void insertUpdate(DocumentEvent e) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
333	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
334	}	Cierra el bloque abierto anteriormente.
335	@Override public void removeUpdate(DocumentEvent e) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
336	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337	}	Cierra el bloque abierto anteriormente.
338	@Override public void changedUpdate(DocumentEvent e) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
339	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
340	}	Cierra el bloque abierto anteriormente.
341	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
342		Línea en blanco: separa visualmente bloques para que el código sea más legible.
343	fechaReporte.addChangeListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
344	botonExportar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
345	botonImprimir.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
346	estadoCarga.setText(	Cambia el texto visible de un componente de la interfaz.
347	"Fecha modificada. Pulsa Actualizar reporte.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
348	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
349		Línea en blanco: separa visualmente bloques para que el código sea más legible.
350	addHierarchyListener(e -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
351	boolean cambioVisibilidad = (e.getChangeFlags()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
352	& java.awt.event.HierarchyEvent.SHOWING_CHANGED) != 0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
353	if (cambioVisibilidad && isShowing() && ultimoReporte == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
354	&& carga == null && !java.beans.Beans.isDesignTime()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	actualizarReporte();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
356	}	Cierra el bloque abierto anteriormente.
357	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
358	}	Cierra el bloque abierto anteriormente.
359		Línea en blanco: separa visualmente bloques para que el código sea más legible.
360	public void actualizarReporte() {	Declara el método actualizarReporte. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
361	if (carga != null) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
362	final LocalDate fecha;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
363	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
364	fechaReporte.commitEdit();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
365	fecha = ((Date) fechaReporte.getValue()).toInstant()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
366	.atZone(ZoneId.systemDefault()).toLocalDate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

367	if (fecha.getYear() < 1000 fecha.getYear() > 9998) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
368	throw new IllegalArgumentException();	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
369	}	Cierra el bloque abierto anteriormente.
370	} catch (java.text.ParseException IllegalArgumentException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
371	estadoCarga.setText("Fecha inválida. Usa dd/MM/aaaa.");	Cambia el texto visible de un componente de la interfaz.
372	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
373	}	Cierra el bloque abierto anteriormente.
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.
375	habilitarConsulta(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
376	limpiarResultados();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377	estadoCarga.setText("Consultando "	Cambia el texto visible de un componente de la interfaz.
378	+ fechaVisible.format(fecha) + "...");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
379		Línea en blanco: separa visualmente bloques para que el código sea más legible.
380	carga = new SwingWorker<>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
381	@Override protected ReporteDAO.Resumen doInBackground()	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
382	throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
383	return reporteDAO.cargar(fecha);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
384	}	Cierra el bloque abierto anteriormente.
385		Línea en blanco: separa visualmente bloques para que el código sea más legible.
386	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
387	if (isCancelled()) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
388	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
389	ReporteDAO.Resumen resultado = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
390	ultimoReporte = resultado;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
391	fechaCargada = fecha;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
392	mostrarResumen(resultado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
393	cargarOpciones(resultado.ventasDetalle());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
394	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
395	estadoCarga.setText(fechaVisible.format(fecha)	Cambia el texto visible de un componente de la interfaz.
396	+ " . Reporte actualizado");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
397	botonExportar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
398	botonImprimir.setEnabled(	Habilita o deshabilita la interacción con el componente.
399	!resultado.ventasDetalle().isEmpty());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
400	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
401	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
402	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
403	estadoCarga.setText(	Cambia el texto visible de un componente de la interfaz.
404	"No se pudo cargar el reporte. Comprueba MySQL.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
405	java.util.logging.Logger.getLogger(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
406	ReportesPanel.class.getName()).log(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
407	java.util.logging.Level.WARNING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
408	"Error al consultar reportes", ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
410	carga = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

411	habilitarConsulta(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
412	}	Cierra el bloque abierto anteriormente.
413	}	Cierra el bloque abierto anteriormente.
414	};	Cierra el bloque abierto anteriormente y termina la declaración.
415	carga.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416	}	Cierra el bloque abierto anteriormente.
417		Línea en blanco: separa visualmente bloques para que el código sea más legible.
418	private void mostrarResumen(ReporteDAO.Resumen resultado) {	Declara el método mostrarResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
419	labelVentas.setText(moneda.format(resultado.ventas()));	Cambia el texto visible de un componente de la interfaz.
420	labelPedidos.setText(Long.toString(resultado.pedidos()));	Cambia el texto visible de un componente de la interfaz.
421	labelTicket.setText(moneda.format(resultado.ticket()));	Cambia el texto visible de un componente de la interfaz.
422	labelCancelados.setText(Long.toString(resultado.cancelados()));	Cambia el texto visible de un componente de la interfaz.
423	labelEfectivo.setText(moneda.format(resultado.efectivo()));	Cambia el texto visible de un componente de la interfaz.
424	labelTarjeta.setText(moneda.format(resultado.tarjeta()));	Cambia el texto visible de un componente de la interfaz.
425	labelOtros.setText(moneda.format(resultado.otros()));	Cambia el texto visible de un componente de la interfaz.
426	labelCancelacionesCaja.setText(resultado.cancelados() + " pedidos");	Cambia el texto visible de un componente de la interfaz.
427	labelPorcentajeEfectivo.setText(	Cambia el texto visible de un componente de la interfaz.
428	porcentaje(resultado.efectivo(), resultado.ventas()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429	labelPorcentajeTarjeta.setText(	Cambia el texto visible de un componente de la interfaz.
430	porcentaje(resultado.tarjeta(), resultado.ventas()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
431	labelPorcentajeOtros.setText(	Cambia el texto visible de un componente de la interfaz.
432	porcentaje(resultado.otros(), resultado.ventas()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433	}	Cierra el bloque abierto anteriormente.
434		Línea en blanco: separa visualmente bloques para que el código sea más legible.
435	private String porcentaje(BigDecimal parte, BigDecimal total) {	Declara el método porcentaje. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
436	if (total == null total.signum() == 0) return "% del total";	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
437	return parte.multiply(BigDecimal.valueOf(100))	Finaliza el método y devuelve el valor indicado a quien lo llamó.
438	.divide(total, 1, RoundingMode.HALF_UP)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439	.stripTrailingZeros().toString() + "% del total";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
440	}	Cierra el bloque abierto anteriormente.
441		Línea en blanco: separa visualmente bloques para que el código sea más legible.
442	private void cargarOpciones(List<Object[]> ventas) {	Declara el método cargarOpciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
443	Set<String> cajeros = new TreeSet<>();	Crea una nueva instancia de TreeSet y la usa o guarda para continuar el proceso.
444	for (Object[] fila : ventas) cajeros.add(String.valueOf(fila[2]));	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
445	StringBuilder opciones = new StringBuilder("Todos los cajeros");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
446	for (String cajero : cajeros) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
447	opciones.append(';').append(cajero.replace(';', ' '));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448	}	Cierra el bloque abierto anteriormente.
449	filtroCajero.setTextoDesplegable(opciones.toString());	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
450	filtroCajero.setText("Todos los cajeros");	Cambia el texto visible de un componente de la interfaz.
451	filtroMetodo.setTextoDesplegable(	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
452	"Todos los métodos;Efectivo;Tarjeta;Otros");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
453	filtroMetodo.setText("Todos los métodos");	Cambia el texto visible de un componente de la interfaz.
454	filtroServicio.setTextoDesplegable(	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.

455	"Todos los servicios;En el local;Para llevar;A domicilio");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
456	filtroServicio.setText("Todos los servicios");	Cambia el texto visible de un componente de la interfaz.
457	cajeroSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
458	metodoSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
459	servicioSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
460	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.
461	}	Cierra el bloque abierto anteriormente.
462		Línea en blanco: separa visualmente bloques para que el código sea más legible.
463	private void aplicarFiltros() {	Declara el método aplicarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
464	if (ultimoReporte == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
465	ventasFiltradas = List.of();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
466	mostrarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
467	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
468	}	Cierra el bloque abierto anteriormente.
469	String busqueda = campoBusqueda.getText().trim()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
470	.replace("#", "").toLowerCase(Locale.ROOT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
471	List<Object[]> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
472	for (Object[] fila : ultimoReporte.ventasDetalle()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
473	String metodo = String.valueOf(fila[5]);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
474	boolean coincideCajero = cajeroSeleccionado == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
475	cajeroSeleccionado.equals(String.valueOf(fila[2]));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
476	boolean coincideMetodo = metodoSeleccionado == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
477	("Otros".equals(metodoSeleccionado)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
478	? !"EFECTIVO".equalsIgnoreCase(metodo)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
479	&& !"TARJETA".equalsIgnoreCase(metodo)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
480	: metodoSeleccionado.equalsIgnoreCase(metodo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
481	boolean coincideServicio = servicioSeleccionado == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
482	servicioSeleccionado.equals(String.valueOf(fila[3]));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
483	boolean coincideBusqueda = busqueda.isEmpty()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
484	String.valueOf(fila[0]).toLowerCase(Locale.ROOT)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
485	.contains(busqueda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
486	if (coincideCajero && coincideMetodo	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
487	&& coincideServicio && coincideBusqueda) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
488	resultado.add(fila);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
489	}	Cierra el bloque abierto anteriormente.
490	}	Cierra el bloque abierto anteriormente.
491	ventasFiltradas = List.copyOf(resultado);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
492	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
493	mostrarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
494	}	Cierra el bloque abierto anteriormente.
495		Línea en blanco: separa visualmente bloques para que el código sea más legible.
496	private void limpiarFiltros() {	Declara el método limpiarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
497	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
498	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.

499	cajeroSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
500	metodoSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
501	servicioSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
502	filtroCajero.setText("Todos los cajeros");	Cambia el texto visible de un componente de la interfaz.
503	filtroMetodo.setText("Todos los métodos");	Cambia el texto visible de un componente de la interfaz.
504	filtroServicio.setText("Todos los servicios");	Cambia el texto visible de un componente de la interfaz.
505	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
506	aplicarFiltros();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
507	}	Cierra el bloque abierto anteriormente.
508		Línea en blanco: separa visualmente bloques para que el código sea más legible.
509	private int totalPaginas() {	Declara el método totalPaginas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
510	return Math.max(1, (int) Math.ceil(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
511	ventasFiltradas.size() / (double) VENTAS_POR_PAGINA));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
512	}	Cierra el bloque abierto anteriormente.
513		Línea en blanco: separa visualmente bloques para que el código sea más legible.
514	private void mostrarPagina() {	Declara el método mostrarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
515	int totalPaginas = totalPaginas();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
516	paginaActual = Math.max(1, Math.min(paginaActual, totalPaginas));	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
517	int desde = Math.min((paginaActual - 1) * VENTAS_POR_PAGINA,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
518	ventasFiltradas.size());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
519	int hasta = Math.min(desde + VENTAS_POR_PAGINA,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
520	ventasFiltradas.size());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
521	modeloTabla.setFilas(ventasFiltradas.subList(desde, hasta));	Configura una propiedad mediante setFilas en el objeto indicado.
522	actualizarPaginacion(desde, hasta, totalPaginas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
523	}	Cierra el bloque abierto anteriormente.
524		Línea en blanco: separa visualmente bloques para que el código sea más legible.
525	private void actualizarPaginacion(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
526	int desde, int hasta, int totalPaginas) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
527	etiquetaRango.setText(ventasFiltradas.isEmpty()	Cambia el texto visible de un componente de la interfaz.
528	? "No se encontraron ventas"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
529	: String.format("Mostrando %-d de %-d ventas",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
530	desde + 1, hasta, ventasFiltradas.size()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
531	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
532	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
533	int inicio = Math.max(1,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
534	Math.min(paginaActual - 1, totalPaginas - 2));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
535		Línea en blanco: separa visualmente bloques para que el código sea más legible.
536	for (int i = 0; i < botonesPagina.size(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
537	int pagina = inicio + i;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
538	BotonRedondeado boton = botonesPagina.get(i);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
539	boolean visible = pagina <= totalPaginas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
540	boton.setVisible(visible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
541	if (!visible) continue;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
542	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

543	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
544	boolean seleccionada = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
545	boton.setColorInicio(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
546	boton.setColorFinal(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
547	boton.setColorBorde(seleccionada ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
548	boton.setFont(seleccionada	Configura una propiedad mediante setFont en el objeto indicado.
549	? tema.negrita(13f) : tema.media(13f));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
550	}	Cierra el bloque abierto anteriormente.
551	}	Cierra el bloque abierto anteriormente.
552		Línea en blanco: separa visualmente bloques para que el código sea más legible.
553	private void cambiarPagina(int pagina) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
554	if (pagina < 1 pagina > totalPaginas())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
555	pagina == paginaActual) return;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
556	paginaActual = pagina;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
557	mostrarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
558	}	Cierra el bloque abierto anteriormente.
559		Línea en blanco: separa visualmente bloques para que el código sea más legible.
560	private void habilitarConsulta(boolean habilitada) {	Declara el método habilitarConsulta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
561	botonActualizar.setEnabled(habilitada);	Habilita o deshabilita la interacción con el componente.
562	fechaReporte.setEnabled(habilitada);	Habilita o deshabilita la interacción con el componente.
563	}	Cierra el bloque abierto anteriormente.
564		Línea en blanco: separa visualmente bloques para que el código sea más legible.
565	private void limpiarResultados() {	Declara el método limpiarResultados. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
566	ultimoReporte = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
567	ventasFiltradas = List.of();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
568	for (JLabel etiqueta : new JLabel[]{	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
569	labelVentas, labelPedidos, labelTicket, labelCancelados,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
570	labelEfectivo, labelTarjeta, labelOtros	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
571	}) etiqueta.setText("-");	Cambia el texto visible de un componente de la interfaz.
572	labelCancelacionesCaja.setText("0 pedidos");	Cambia el texto visible de un componente de la interfaz.
573	labelPorcentajeEfectivo.setText("0% del total");	Cambia el texto visible de un componente de la interfaz.
574	labelPorcentajeTarjeta.setText("0% del total");	Cambia el texto visible de un componente de la interfaz.
575	labelPorcentajeOtros.setText("0% del total");	Cambia el texto visible de un componente de la interfaz.
576	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
577	mostrarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
578	botonExportar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
579	botonImprimir.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
580	}	Cierra el bloque abierto anteriormente.
581		Línea en blanco: separa visualmente bloques para que el código sea más legible.
582	private void exportarReporte() {	Declara el método exportarReporte. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
583	if (ultimoReporte == null fechaCargada == null) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
584	JFileChooser selector = new JFileChooser();	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
585	selector.setDialogTitle("Guardar reporte de ventas");	Configura una propiedad mediante setDialogTitle en el objeto indicado.
586	selector.setFileFilter(new FileNameExtensionFilter(	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.

587	"Archivo CSV", "csv"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
588	selector.setSelectedFile(new File(	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
589	"reporte-ventas-" + fechaCargada + ".csv"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
590	if (selector.showSaveDialog(this)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
591	!= JFileChooser.APPROVE_OPTION) return;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
592		Línea en blanco: separa visualmente bloques para que el código sea más legible.
593	Path archivo = selector.getSelectedFile().toPath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
594	if (!archivo.toString().toLowerCase(Locale.ROOT).endsWith(".csv")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
595	archivo = archivo.resolveSibling(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
596	archivo.getFileName() + ".csv");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	}	Cierra el bloque abierto anteriormente.
598	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
599	Files.writeString(archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
600	ReporteCsv.generar(fechaCargada, ultimoReporte),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
601	StandardCharsets.UTF_8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
602	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
603	"Reporte guardado en:\n" + archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
604	"Exportación completada",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
605	JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
606	} catch (IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
607	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
608	"No se pudo guardar el archivo.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
609	"Exportación", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
610	}	Cierra el bloque abierto anteriormente.
611	}	Cierra el bloque abierto anteriormente.
612		Línea en blanco: separa visualmente bloques para que el código sea más legible.
613	private void imprimirReporte() {	Declara el método imprimirReporte. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
614	if (ventasFiltradas.isEmpty() fechaCargada == null) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
615	modeloTabla.setFilas(ventasFiltradas);	Configura una propiedad mediante setFilas en el objeto indicado.
616	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
617	tablaPedidos.print(JTable.PrintMode.FIT_WIDTH,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
618	new MessageFormat("Reporte de ventas · "	Crea una nueva instancia de MessageFormat y la usa o guarda para continuar el proceso.
619	+ fechaVisible.format(fechaCargada)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
620	new MessageFormat("Página {0}"));	Crea una nueva instancia de MessageFormat y la usa o guarda para continuar el proceso.
621	} catch (java.awt.print.PrinterException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
622	JOptionPane.showMessageDialog(this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
623	"No se pudo imprimir el reporte.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
624	"Impresión", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
625	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
626	mostrarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
627	}	Cierra el bloque abierto anteriormente.
628	}	Cierra el bloque abierto anteriormente.
629		Línea en blanco: separa visualmente bloques para que el código sea más legible.
630	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

631	<code>public void removeNotify() {</code>	Declara el método removeNotify. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
632	<code> temporizadorBusqueda.stop();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
633	<code> if (carga != null) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
634	<code> carga.cancel(true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
635	<code> carga = null;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
636	<code> }</code>	Cierra el bloque abierto anteriormente.
637	<code> super.removeNotify();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
638	<code>}</code>	Cierra el bloque abierto anteriormente.
639		Línea en blanco: separa visualmente bloques para que el código sea más legible.
640	<code>@SuppressWarnings("unchecked")</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
641	<code>// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
642	<code>private void initComponents() {</code>	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
643		Línea en blanco: separa visualmente bloques para que el código sea más legible.
644	<code> labelTitulo = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
645	<code> labelSubtitulo = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
646	<code> botonActualizar = new Componentes.BotonDerretido();</code>	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
647	<code> botonExportar = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
648	<code> botonImprimir = new Componentes.BotonRedondeado();</code>	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
649	<code> panelVentas = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
650	<code> labelTituloVentas = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
651	<code> labelVentas = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
652	<code> panelCircular2 = new Componentes.PanelCircular();</code>	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
653	<code> labelEscalable2 = new Labels.LabelEscalable();</code>	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
654	<code> panelPedidos = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
655	<code> labelTituloPedidos = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
656	<code> labelPedidos = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
657	<code> panelCircular3 = new Componentes.PanelCircular();</code>	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
658	<code> labelEscalable3 = new Labels.LabelEscalable();</code>	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
659	<code> panelTicket = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
660	<code> labelTituloTicket = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
661	<code> labelTicket = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
662	<code> panelCircular5 = new Componentes.PanelCircular();</code>	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
663	<code> labelEscalable5 = new Labels.LabelEscalable();</code>	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
664	<code> panelCancelados = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
665	<code> labelTituloCancelados = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
666	<code> labelCancelados = new javax.swing.JLabel();</code>	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
667	<code> panelCircular4 = new Componentes.PanelCircular();</code>	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
668	<code> labelEscalable4 = new Labels.LabelEscalable();</code>	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
669	<code> panelFiltros = new Componentes.PanelFlotante();</code>	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
670	<code> fechaReporte = new javax.swing.JSpinner();</code>	Crea una nueva instancia de javax.swing.JSpinner y la usa o guarda para continuar el proceso.
671	<code> campoBusqueda = new Componentes.CampoBusquedaAdmin();</code>	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
672	<code> filtroCajero = new Componentes.BotonDesplegable();</code>	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
673	<code> filtroMetodo = new Componentes.BotonDesplegable();</code>	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
674	<code> filtroServicio = new Componentes.BotonDesplegable();</code>	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.

675	botonLimpiarFiltros = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
676	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
677	scrollPedidos = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
678	tablaPedidos = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
679	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
680	panelPaginacion = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
681	botonAnterior = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
682	botonPagina1 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
683	botonPagina2 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
684	botonPagina3 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
685	botonSiguiente = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
686	panelResumen = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
687	labelResumen = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
688	labelResumenSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
689	separadorResumen1 = new javax.swing.JSeparator();	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
690	efectivoTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
691	labelEfectivo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
692	labelPorcentajeEfectivo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
693	separadorResumen2 = new javax.swing.JSeparator();	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
694	tarjetaTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
695	labelTarjeta = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
696	labelPorcentajeTarjeta = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
697	separadorResumen3 = new javax.swing.JSeparator();	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
698	otrosTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
699	labelOtros = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
700	labelPorcentajeOtros = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
701	separadorResumen4 = new javax.swing.JSeparator();	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
702	cancelacionTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
703	labelCancelacionesCaja = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
704	cancelacionDetalle = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
705	estadoCarga = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
706		Línea en blanco: separa visualmente bloques para que el código sea más legible.
707	setBackground(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
708	setPreferredSize(new java.awt.Dimension(1580, 932));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
709	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
710		Línea en blanco: separa visualmente bloques para que el código sea más legible.
711	labelTitulo.setFont(new java.awt.Font("Dialog", 1, 42)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
712	labelTitulo.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
713	labelTitulo.setText("Reportes - Ventas del día");	Cambia el texto visible de un componente de la interfaz.
714	add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 25, 690, 65));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
715		Línea en blanco: separa visualmente bloques para que el código sea más legible.
716	labelSubtitulo.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
717	labelSubtitulo.setText("Resumen detallado de todas las ventas realizadas en la fecha seleccionada");	Cambia el texto visible de un componente de la interfaz.
718	add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 88, 760, 28));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

719		Línea en blanco: separa visualmente bloques para que el código sea más legible.
720	<code>botonActualizar.setForeground(new java.awt.Color(0, 20, 43));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
721	<code>botonActualizar.setText("Actualizar reporte");</code>	Cambia el texto visible de un componente de la interfaz.
722	<code>add(botonActualizar, new org.netbeans.lib.awtextra.AbsoluteConstraints(950, 60, 220, 80));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
723		Línea en blanco: separa visualmente bloques para que el código sea más legible.
724	<code>botonExportar.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
725	<code>botonExportar.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.
726	<code>botonExportar.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
727	<code>botonExportar.setRadio(20);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
728	<code>botonExportar.setText("Exportar");</code>	Cambia el texto visible de un componente de la interfaz.
729	<code>botonExportar.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
730	<code>botonExportar.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
731	<code>botonExportar.setFont(new java.awt.Font("Arial", 1, 16)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
732	<code>botonExportar.setGrosorBorde(1.6F);</code>	Configura una propiedad mediante <code>setGrosorBorde</code> en el objeto indicado.
733	<code>botonExportar.setMargin(new java.awt.Insets(5, 14, 3, 14));</code>	Crea una nueva instancia de <code>java.awt.Insets</code> y la usa o guarda para continuar el proceso.
734	<code>botonExportar.setMaximumSize(new java.awt.Dimension(96, 29));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
735	<code>botonExportar.setMinimumSize(new java.awt.Dimension(96, 29));</code>	Crea una nueva instancia de <code>java.awt.Dimension</code> y la usa o guarda para continuar el proceso.
736	<code>add(botonExportar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1185, 60, 180, 58));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
737		Línea en blanco: separa visualmente bloques para que el código sea más legible.
738	<code>botonImprimir.setForeground(new java.awt.Color(0, 20, 43));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
739	<code>botonImprimir.setText("Imprimir");</code>	Cambia el texto visible de un componente de la interfaz.
740	<code>botonImprimir.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.
741	<code>add(botonImprimir, new org.netbeans.lib.awtextra.AbsoluteConstraints(1375, 60, 145, 58));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
742		Línea en blanco: separa visualmente bloques para que el código sea más legible.
743	<code>panelVentas.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
744	<code>panelVentas.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
745		Línea en blanco: separa visualmente bloques para que el código sea más legible.
746	<code>labelTituloVentas.setText("Ventas del día");</code>	Cambia el texto visible de un componente de la interfaz.
747	<code>panelVentas.add(labelTituloVentas, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 20, 195, 30));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
748		Línea en blanco: separa visualmente bloques para que el código sea más legible.
749	<code>labelVentas.setText("-");</code>	Cambia el texto visible de un componente de la interfaz.
750	<code>panelVentas.add(labelVentas, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 52, 195, 48));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
751		Línea en blanco: separa visualmente bloques para que el código sea más legible.
752	<code>panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
753		Línea en blanco: separa visualmente bloques para que el código sea más legible.
754	<code>labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/quetzal_ícono.png"))); // NOI18N</code>	Crea una nueva instancia de <code>javax.swing.ImageIcon</code> y la usa o guarda para continuar el proceso.
755	<code>panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(8, 13, 65, 65));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
756		Línea en blanco: separa visualmente bloques para que el código sea más legible.
757	<code>panelVentas.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
758		Línea en blanco: separa visualmente bloques para que el código sea más legible.
759	<code>add(panelVentas, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 150, 360, 150));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
760		Línea en blanco: separa visualmente bloques para que el código sea más legible.
761	<code>panelPedidos.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
762	<code>panelPedidos.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.

763		Línea en blanco: separa visualmente bloques para que el código sea más legible.
764	labelTituloPedidos.setText("Pedidos cobrados");	Cambia el texto visible de un componente de la interfaz.
765	panelPedidos.add(labelTituloPedidos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 20, 195, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
766		Línea en blanco: separa visualmente bloques para que el código sea más legible.
767	labelPedidos.setText("-");	Cambia el texto visible de un componente de la interfaz.
768	panelPedidos.add(labelPedidos, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 52, 195, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
769		Línea en blanco: separa visualmente bloques para que el código sea más legible.
770	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
771	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
772		Línea en blanco: separa visualmente bloques para que el código sea más legible.
773	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/bolsa_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
774	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(11, 13, 58, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
775		Línea en blanco: separa visualmente bloques para que el código sea más legible.
776	panelPedidos.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
777		Línea en blanco: separa visualmente bloques para que el código sea más legible.
778	add(panelPedidos, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
779		Línea en blanco: separa visualmente bloques para que el código sea más legible.
780	panelTicket.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
781	panelTicket.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
782		Línea en blanco: separa visualmente bloques para que el código sea más legible.
783	labelTituloTicket.setText("Ticket promedio");	Cambia el texto visible de un componente de la interfaz.
784	panelTicket.add(labelTituloTicket, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 20, 195, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
785		Línea en blanco: separa visualmente bloques para que el código sea más legible.
786	labelTicket.setText("-");	Cambia el texto visible de un componente de la interfaz.
787	panelTicket.add(labelTicket, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 52, 195, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
788		Línea en blanco: separa visualmente bloques para que el código sea más legible.
789	panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
790		Línea en blanco: separa visualmente bloques para que el código sea más legible.
791	labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/ticket_icono.png"))); ; // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
792	panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(11, 15, 60, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
793		Línea en blanco: separa visualmente bloques para que el código sea más legible.
794	panelTicket.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
795		Línea en blanco: separa visualmente bloques para que el código sea más legible.
796	add(panelTicket, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
797		Línea en blanco: separa visualmente bloques para que el código sea más legible.
798	panelCancelados.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
799	panelCancelados.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
800		Línea en blanco: separa visualmente bloques para que el código sea más legible.
801	labelTituloCancelados.setText("Cancelados");	Cambia el texto visible de un componente de la interfaz.
802	panelCancelados.add(labelTituloCancelados, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 20, 195, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
803		Línea en blanco: separa visualmente bloques para que el código sea más legible.
804	labelCancelados.setText("-");	Cambia el texto visible de un componente de la interfaz.

805	panelCancelados.add(labelCancelados, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 52, 195, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
806		Línea en blanco: separa visualmente bloques para que el código sea más legible.
807	panelCircular4.setColorFondo(new java.awt.Color(237, 239, 243));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
808	panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
809		Línea en blanco: separa visualmente bloques para que el código sea más legible.
810	labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/x_icono.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
811	panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(23, 27, 35, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
812		Línea en blanco: separa visualmente bloques para que el código sea más legible.
813	panelCancelados.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
814		Línea en blanco: separa visualmente bloques para que el código sea más legible.
815	add(panelCancelados, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
816		Línea en blanco: separa visualmente bloques para que el código sea más legible.
817	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
818	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
819	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
820	panelFiltros.add(fechaReporte, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 24, 180, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
821		Línea en blanco: separa visualmente bloques para que el código sea más legible.
822	campoBusqueda.setPlaceholder("Buscar por número de orden...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
823	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(215, 22, 330, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
824		Línea en blanco: separa visualmente bloques para que el código sea más legible.
825	filtroCajero.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
826	filtroCajero.setText("Todos los cajeros");	Cambia el texto visible de un componente de la interfaz.
827	filtroCajero.setTextoDesplegable("Todos los cajeros");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
828	panelFiltros.add(filtroCajero, new org.netbeans.lib.awtextra.AbsoluteConstraints(560, 24, 235, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
829		Línea en blanco: separa visualmente bloques para que el código sea más legible.
830	filtroMetodo.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
831	filtroMetodo.setText("Todos los métodos");	Cambia el texto visible de un componente de la interfaz.
832	filtroMetodo.setTextoDesplegable("Todos los métodos;Efectivo;Tarjeta;Otros");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
833	panelFiltros.add(filtroMetodo, new org.netbeans.lib.awtextra.AbsoluteConstraints(810, 24, 200, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
834		Línea en blanco: separa visualmente bloques para que el código sea más legible.
835	filtroServicio.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
836	filtroServicio.setText("Todos los servicios");	Cambia el texto visible de un componente de la interfaz.
837	filtroServicio.setTextoDesplegable("Todos los servicios;En el local;Para llevar;A domicilio");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
838	panelFiltros.add(filtroServicio, new org.netbeans.lib.awtextra.AbsoluteConstraints(1025, 24, 205, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
839		Línea en blanco: separa visualmente bloques para que el código sea más legible.
840	botonLimpiarFiltros.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
841	botonLimpiarFiltros.setText("Limpiar filtros");	Cambia el texto visible de un componente de la interfaz.
842	botonLimpiarFiltros.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
843	panelFiltros.add(botonLimpiarFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(1245, 24, 200, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
844		Línea en blanco: separa visualmente bloques para que el código sea más legible.
845	add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 320, 1470, 100));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
846		Línea en blanco: separa visualmente bloques para que el código sea más legible.

847	<code>panelTabla.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
848	<code>panelTabla.setSombra(false);</code>	Configura una propiedad mediante <code>setSombra</code> en el objeto indicado.
849	<code>panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
850		Línea en blanco: separa visualmente bloques para que el código sea más legible.
851	<code>scrollPedidos.setBorder(null);</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
852	<code>scrollPedidos.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setVerticalScrollBarPolicy</code> en el objeto indicado.
853	<code>scrollPedidos.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setHorizontalScrollBarPolicy</code> en el objeto indicado.
854		Línea en blanco: separa visualmente bloques para que el código sea más legible.
855	<code>tablaPedidos.setModel(new javax.swing.table.DefaultTableModel(</code>	Crea una nueva instancia de <code>javax.swing.table.DefaultTableModel</code> y la usa o guarda para continuar el proceso.
856	<code>new Object [][] {</code>	Crea una nueva instancia de <code>Object</code> y la usa o guarda para continuar el proceso.
857		Línea en blanco: separa visualmente bloques para que el código sea más legible.
858	<code>},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
859	<code>new String [] {</code>	Crea una nueva instancia de <code>String</code> y la usa o guarda para continuar el proceso.
860	<code>"N° orden", "Fecha/Hora", "Cajero", "Tipo servicio", "Estado", "Método de pago", "Monto recibido", "Total", "Cantidad de ítems"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
861	<code>}</code>	Cierra el bloque abierto anteriormente.
862	<code>) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
863	<code>boolean[] canEdit = new boolean [] {</code>	Crea una nueva instancia de <code>boolean</code> y la usa o guarda para continuar el proceso.
864	<code>false, false, false, false, false, false, false, false, false</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
865	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
866		Línea en blanco: separa visualmente bloques para que el código sea más legible.
867	<code>public boolean isCellEditable(int rowIndex, int columnIndex) {</code>	Declara el método <code>isCellEditable</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
868	<code>return canEdit [columnIndex];</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
869	<code>}</code>	Cierra el bloque abierto anteriormente.
870	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
871	<code>scrollPedidos.setViewportViewView(tablaPedidos);</code>	Configura una propiedad mediante <code>setViewportView</code> en el objeto indicado.
872		Línea en blanco: separa visualmente bloques para que el código sea más legible.
873	<code>panelTabla.add(scrollPedidos, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 250));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
874		Línea en blanco: separa visualmente bloques para que el código sea más legible.
875	<code>etiquetaRango.setForeground(new java.awt.Color(92, 103, 124));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
876	<code>etiquetaRango.setText("No se encontraron ventas");</code>	Cambia el texto visible de un componente de la interfaz.
877	<code>panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 278, 420, 42));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
878		Línea en blanco: separa visualmente bloques para que el código sea más legible.
879	<code>panelPaginacion.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
880	<code>panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
881		Línea en blanco: separa visualmente bloques para que el código sea más legible.
882	<code>botonAnterior.setText("<");</code>	Cambia el texto visible de un componente de la interfaz.
883	<code>panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
884		Línea en blanco: separa visualmente bloques para que el código sea más legible.
885	<code>botonPagina1.setText("1");</code>	Cambia el texto visible de un componente de la interfaz.
886	<code>panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
887		Línea en blanco: separa visualmente bloques para que el código sea más legible.
888	<code>botonPagina2.setText("2");</code>	Cambia el texto visible de un componente de la interfaz.
889	<code>panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
890		Línea en blanco: separa visualmente bloques para que el código sea más legible.

891	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
892	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
893		Línea en blanco: separa visualmente bloques para que el código sea más legible.
894	botonSiguiente.setText(">");	Cambia el texto visible de un componente de la interfaz.
895	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
896		Línea en blanco: separa visualmente bloques para que el código sea más legible.
897	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 278, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
898		Línea en blanco: separa visualmente bloques para que el código sea más legible.
899	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 435, 1470, 340));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
900		Línea en blanco: separa visualmente bloques para que el código sea más legible.
901	panelResumen.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
902	panelResumen.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
903	panelResumen.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
904		Línea en blanco: separa visualmente bloques para que el código sea más legible.
905	labelResumen.setText("Resumen de caja");	Cambia el texto visible de un componente de la interfaz.
906	panelResumen.add(labelResumen, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 18, 180, 28));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
907		Línea en blanco: separa visualmente bloques para que el código sea más legible.
908	labelResumenSubtitulo.setText("Totales del día");	Cambia el texto visible de un componente de la interfaz.
909	panelResumen.add(labelResumenSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 50, 180, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
910		Línea en blanco: separa visualmente bloques para que el código sea más legible.
911	separadorResumen1.setOrientation(javax.swing.SwingConstants.VERTICAL);	Configura una propiedad mediante setOrientation en el objeto indicado.
912	panelResumen.add(separadorResumen1, new org.netbeans.lib.awtextra.AbsoluteConstraints(225, 16, 1, 78));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
913		Línea en blanco: separa visualmente bloques para que el código sea más legible.
914	efectivoTitulo.setText("Efectivo");	Cambia el texto visible de un componente de la interfaz.
915	panelResumen.add(efectivoTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(250, 14, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
916		Línea en blanco: separa visualmente bloques para que el código sea más legible.
917	labelEfectivo.setText("-");	Cambia el texto visible de un componente de la interfaz.
918	panelResumen.add(labelEfectivo, new org.netbeans.lib.awtextra.AbsoluteConstraints(250, 38, 250, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
919		Línea en blanco: separa visualmente bloques para que el código sea más legible.
920	labelPorcentajeEfectivo.setText("% del total");	Cambia el texto visible de un componente de la interfaz.
921	panelResumen.add(labelPorcentajeEfectivo, new org.netbeans.lib.awtextra.AbsoluteConstraints(250, 73, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
922		Línea en blanco: separa visualmente bloques para que el código sea más legible.
923	separadorResumen2.setOrientation(javax.swing.SwingConstants.VERTICAL);	Configura una propiedad mediante setOrientation en el objeto indicado.
924	panelResumen.add(separadorResumen2, new org.netbeans.lib.awtextra.AbsoluteConstraints(530, 16, 1, 78));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
925		Línea en blanco: separa visualmente bloques para que el código sea más legible.
926	tarjetaTitulo.setText("Tarjeta");	Cambia el texto visible de un componente de la interfaz.
927	panelResumen.add(tarjetaTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(555, 14, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
928		Línea en blanco: separa visualmente bloques para que el código sea más legible.
929	labelTarjeta.setText("-");	Cambia el texto visible de un componente de la interfaz.
930	panelResumen.add(labelTarjeta, new org.netbeans.lib.awtextra.AbsoluteConstraints(555, 38, 250, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
931		Línea en blanco: separa visualmente bloques para que el código sea más legible.
932	labelPorcentajeTarjeta.setText("% del total");	Cambia el texto visible de un componente de la interfaz.

933	panelResumen.add(labelPorcentajeTarjeta, new org.netbeans.lib.awtextra.AbsoluteConstraints(555, 73, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
934		Línea en blanco: separa visualmente bloques para que el código sea más legible.
935	separadorResumen3.setOrientation(javax.swing.SwingConstants.VERTICAL);	Configura una propiedad mediante setOrientation en el objeto indicado.
936	panelResumen.add(separadorResumen3, new org.netbeans.lib.awtextra.AbsoluteConstraints(835, 16, 1, 78));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
937		Línea en blanco: separa visualmente bloques para que el código sea más legible.
938	otrosTitulo.setText("Otros métodos");	Cambia el texto visible de un componente de la interfaz.
939	panelResumen.add(otrosTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(860, 14, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
940		Línea en blanco: separa visualmente bloques para que el código sea más legible.
941	labelOtros.setText("-");	Cambia el texto visible de un componente de la interfaz.
942	panelResumen.add(labelOtros, new org.netbeans.lib.awtextra.AbsoluteConstraints(860, 38, 250, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
943		Línea en blanco: separa visualmente bloques para que el código sea más legible.
944	labelPorcentajeOtros.setText("0% del total");	Cambia el texto visible de un componente de la interfaz.
945	panelResumen.add(labelPorcentajeOtros, new org.netbeans.lib.awtextra.AbsoluteConstraints(860, 73, 250, 2));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
946		Línea en blanco: separa visualmente bloques para que el código sea más legible.
947	separadorResumen4.setOrientation(javax.swing.SwingConstants.VERTICAL);	Configura una propiedad mediante setOrientation en el objeto indicado.
948	panelResumen.add(separadorResumen4, new org.netbeans.lib.awtextra.AbsoluteConstraints(1140, 16, 1, 78));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
949		Línea en blanco: separa visualmente bloques para que el código sea más legible.
950	cancelacionTitulo.setText("Cancelaciones");	Cambia el texto visible de un componente de la interfaz.
951	panelResumen.add(cancelacionTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 14, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
952		Línea en blanco: separa visualmente bloques para que el código sea más legible.
953	labelCancelacionesCaja.setText("0 pedidos");	Cambia el texto visible de un componente de la interfaz.
954	panelResumen.add(labelCancelacionesCaja, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 38, 250, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
955		Línea en blanco: separa visualmente bloques para que el código sea más legible.
956	cancelacionDetalle.setText("Pedidos cancelados");	Cambia el texto visible de un componente de la interfaz.
957	panelResumen.add(cancelacionDetalle, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 73, 250, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
958		Línea en blanco: separa visualmente bloques para que el código sea más legible.
959	add(panelResumen, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 790, 1470, 110));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
960		Línea en blanco: separa visualmente bloques para que el código sea más legible.
961	estadoCarga.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
962	estadoCarga.setText("Selecciona una fecha y actualiza el reporte.");	Cambia el texto visible de un componente de la interfaz.
963	add(estadoCarga, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 904, 1450, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
964	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
965		Línea en blanco: separa visualmente bloques para que el código sea más legible.
966	private org.netbeans.lib.awtextra.AbsoluteConstraints posicion(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
967	int x, int y, int ancho, int alto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
968	return new org.netbeans.lib.awtextra.AbsoluteConstraints(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
969	x, y, ancho, alto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
970	}	Cierra el bloque abierto anteriormente.
971		Línea en blanco: separa visualmente bloques para que el código sea más legible.
972	private JLabel etiqueta(String texto, int tamano, boolean negrita) {	Declara el método etiqueta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
973	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.

974	etiqueta.setFont(new java.awt.Font(	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
975	"Dialog", negrita ? 1 : 0, tamaño));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
976	etiqueta.setForeground(negrita ? AZUL : SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
977	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
978	}	Cierra el bloque abierto anteriormente.
979		Línea en blanco: separa visualmente bloques para que el código sea más legible.
980	private BotonRedondeado boton(String texto) {	Declara el método boton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
981	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
982	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
983	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
984	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
985	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
986	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
987	}	Cierra el bloque abierto anteriormente.
988		Línea en blanco: separa visualmente bloques para que el código sea más legible.
989	private PanelFlotante panel() {	Declara el método panel. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
990	PanelFlotante panel = new PanelFlotante();	Crea una nueva instancia de PanelFlotante y la usa o guarda para continuar el proceso.
991	panel.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
992	panel.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
993	panel.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
994	panel.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
995	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
996	}	Cierra el bloque abierto anteriormente.
997		Línea en blanco: separa visualmente bloques para que el código sea más legible.
998	private PanelFlotante tarjeta() {	Declara el método tarjeta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
999	PanelFlotante panel = panel();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1000	panel.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1001	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1002	}	Cierra el bloque abierto anteriormente.
1003		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1004	private PanelCircular icono(PanelFlotante tarjeta, String ruta) {	Declara el método icono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1005	PanelCircular panelIcono = new PanelCircular();	Crea una nueva instancia de PanelCircular y la usa o guarda para continuar el proceso.
1006	panelIcono.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
1007	Labels.LabelEscalable imagen = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
1008	imagen.setIcon(new javax.swing.ImageIcon(getClass().getResource(ruta)));	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
1009	panelIcono.add(imagen, posicion(14, 18, 52, 52));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1010	tarjeta.add(panelIcono, posicion(40, 30, 80, 90));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1011	return panelIcono;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1012	}	Cierra el bloque abierto anteriormente.
1013		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1014	private JLabel tituloTarjeta(PanelFlotante panel, String texto) {	Declara el método tituloTarjeta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1015	JLabel etiqueta = etiqueta(texto, 14, false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1016	panel.add(etiqueta, posicion(140, 20, 195, 30));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1017	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

1018	}	Cierra el bloque abierto anteriormente.
1019		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1020	private JLabel valorTarjeta(PanelFlotante panel) {	Declara el método valorTarjeta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1021	JLabel etiqueta = etiqueta("-", 28, true);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1022	panel.add(etiqueta, posicion(140, 52, 195, 48));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1023	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1024	}	Cierra el bloque abierto anteriormente.
1025		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1026	private BotonDesplegable filtro(String texto, String opciones) {	Declara el método filtro. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1027	BotonDesplegable filtro = new BotonDesplegable();	Crea una nueva instancia de BotonDesplegable y la usa o guarda para continuar el proceso.
1028	filtro.setText(texto);	Cambia el texto visible de un componente de la interfaz.
1029	filtro.setTextoDesplegable(opciones);	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
1030	filtro.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
1031	return filtro;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1032	}	Cierra el bloque abierto anteriormente.
1033		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1034	private javax.swing.JSeparator separador(PanelFlotante panel, int x) {	Declara el método separador. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1035	javax.swing.JSeparator separador =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1036	new javax.swing.JSeparator(SwingConstants.VERTICAL);	Crea una nueva instancia de javax.swing.JSeparator y la usa o guarda para continuar el proceso.
1037	panel.add(separador, posicion(x, 16, 1, 78));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1038	return separador;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1039	}	Cierra el bloque abierto anteriormente.
1040		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1041	private JLabel tituloResumen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1042	PanelFlotante panel, String texto, int x) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1043	JLabel etiqueta = etiqueta(texto, 13, false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1044	panel.add(etiqueta, posicion(x, 14, 250, 22));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1045	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1046	}	Cierra el bloque abierto anteriormente.
1047		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1048	private JLabel valorResumen(PanelFlotante panel, int x) {	Declara el método valorResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1049	JLabel etiqueta = etiqueta("-", 21, true);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1050	panel.add(etiqueta, posicion(x, 38, 250, 30));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1051	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1052	}	Cierra el bloque abierto anteriormente.
1053		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1054	private JLabel detalleResumen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1055	PanelFlotante panel, String texto, int x) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1056	JLabel etiqueta = etiqueta(texto, 12, false);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1057	panel.add(etiqueta, posicion(x, 73, 250, 22));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1058	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1059	}	Cierra el bloque abierto anteriormente.
1060		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1061	private final class ModeloTablaVentas extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.

1062	private final String[] columnas = {	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1063	"N° orden", "Fecha/Hora", "Cajero", "Tipo servicio",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1064	"Estado", "Método de pago", "Monto recibido", "Total",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1065	"Cantidad de ítems"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1066	};	Cierra el bloque abierto anteriormente y termina la declaración.
1067	private List<Object[]> filas = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1068		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1069	private void setFilas(List<Object[]> nuevasFilas) {	Declara el método setFilas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1070	filas = List.copyOf(nuevasFilas);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1071	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1072	}	Cierra el bloque abierto anteriormente.
1073	@Override public int getRowCount() { return filas.size(); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1074	@Override public int getColumnCount() { return columnas.length; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1075	@Override public String getColumnName(int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1076	return columnas[columna];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1077	}	Cierra el bloque abierto anteriormente.
1078	@Override public Object getValueAt(int fila, int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1079	Object valor = filas.get(fila)[columna];	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1080	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1081	case 0 -> "#" + valor;	Define una de las rutas posibles dentro del switch.
1082	case 5 -> metodoVisible(valor);	Define una de las rutas posibles dentro del switch.
1083	default -> valor;	Define una de las rutas posibles dentro del switch.
1084	};	Cierra el bloque abierto anteriormente y termina la declaración.
1085	}	Cierra el bloque abierto anteriormente.
1086	@Override public Class<?> getColumnClass(int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1087	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1088	case 6, 7 -> BigDecimal.class;	Define una de las rutas posibles dentro del switch.
1089	default -> String.class;	Define una de las rutas posibles dentro del switch.
1090	};	Cierra el bloque abierto anteriormente y termina la declaración.
1091	}	Cierra el bloque abierto anteriormente.
1092	@Override public boolean isCellEditable(int fila, int columna) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1093	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1094	}	Cierra el bloque abierto anteriormente.
1095	}	Cierra el bloque abierto anteriormente.
1096		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1097	private String metodoVisible(Object valor) {	Declara el método metodoVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1098	String metodo = valor == null ? "" : valor.toString();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1099	return switch (metodo.toUpperCase(Locale.ROOT)) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1100	case "EFECTIVO" -> "Efectivo";	Define una de las rutas posibles dentro del switch.
1101	case "TARJETA" -> "Tarjeta";	Define una de las rutas posibles dentro del switch.
1102	case "", "-" -> "-";	Define una de las rutas posibles dentro del switch.
1103	default -> metodo;	Define una de las rutas posibles dentro del switch.
1104	};	Cierra el bloque abierto anteriormente y termina la declaración.
1105	}	Cierra el bloque abierto anteriormente.

1106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1107	private final class RenderEstado implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1108	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1109	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1110	JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1111	boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1112	String visible;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1113	Color texto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1114	Color fondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1115	switch (String.valueOf(valor)) {	Selecciona una ruta entre varias según el valor evaluado.
1116	case "PAGADO" -> {	Define una de las rutas posibles dentro del switch.
1117	visible = "Completado";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1118	texto = VERDE;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1119	fondo = new Color(230, 245, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1120	}	Cierra el bloque abierto anteriormente.
1121	case "CANCELADO" -> {	Define una de las rutas posibles dentro del switch.
1122	visible = "Cancelado";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1123	texto = ROJO;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1124	fondo = new Color(252, 233, 233);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1125	}	Cierra el bloque abierto anteriormente.
1126	default -> {	Define una de las rutas posibles dentro del switch.
1127	visible = "Pendiente";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1128	texto = new Color(176, 117, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1129	fondo = new Color(255, 244, 211);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1130	}	Cierra el bloque abierto anteriormente.
1131	}	Cierra el bloque abierto anteriormente.
1132	JPanel panel = new JPanel(new FlowLayout(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1133	FlowLayout.CENTER, 0, 5));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1134	panel.setBackground(fondoFila(tabla, seleccionado, fila));	Configura una propiedad mediante setBackground en el objeto indicado.
1135	JLabel etiqueta = new JLabel(visible);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
1136	etiqueta.setFont(tema.media(11f));	Configura una propiedad mediante setFont en el objeto indicado.
1137	etiqueta.setForeground(texto);	Configura una propiedad mediante setForeground en el objeto indicado.
1138	etiqueta.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
1139	etiqueta.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
1140	etiqueta.setBorder(new EmptyBorder(3, 11, 3, 11));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1141	panel.add(etiqueta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1142	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1143	}	Cierra el bloque abierto anteriormente.
1144	}	Cierra el bloque abierto anteriormente.
1145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1146	private final class RenderMoneda extends DefaultTableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1147	private final boolean resaltar;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1148	private RenderMoneda(boolean resaltar) {	Declara el método RenderMoneda. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1149	this.resaltar = resaltar;	Accede al atributo del objeto actual para asignarlo o utilizarlo.

1150	setHorizontalAlignment(SwingConstants.RIGHT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1151	}	Cierra el bloque abierto anteriormente.
1152	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1153	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1154	JTable tabla, Object valor, boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1155	boolean foco, int fila, int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1156	super.getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1157	tabla, valor, seleccionado, foco, fila, columna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1158	setText(valor instanceof Number ? moneda.format(valor) : "Q0.00");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1159	setBackground(fondoFila(tabla, seleccionado, fila));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1160	setForeground(seleccionado	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1161	? tabla.getSelectionForeground()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1162	: resaltar ? NARANJA : AZUL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1163	setFont(resaltar ? tema.negrita(12f) : tema.regular(12f));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1164	setBorder(new EmptyBorder(0, 12, 0, 12));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
1165	return this;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1166	}	Cierra el bloque abierto anteriormente.
1167	}	Cierra el bloque abierto anteriormente.
1168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1169	private Color fondoFila(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1170	JTable tabla, boolean seleccionado, int fila) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1171	if (seleccionado) return tabla.getSelectionBackground();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1172	return fila % 2 == 0 ? Color.WHITE : new Color(250, 251, 252);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1173	}	Cierra el bloque abierto anteriormente.
1174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1175	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1176	private Componentes.BotonDerretido botonActualizar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1177	private Componentes.BotonRedondeado botonAnterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1178	private Componentes.BotonRedondeado botonExportar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1179	private Componentes.BotonRedondeado botonImprimir;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1180	private Componentes.BotonRedondeado botonLimpiarFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1181	private Componentes.BotonRedondeado botonPagina1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1182	private Componentes.BotonRedondeado botonPagina2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1183	private Componentes.BotonRedondeado botonPagina3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1184	private Componentes.BotonRedondeado botonSiguiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1185	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1186	private javax.swing.JLabel cancelacionDetalle;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1187	private javax.swing.JLabel cancelacionTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1188	private javax.swing.JLabel efectivoTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1189	private javax.swing.JLabel estadoCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1190	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1191	private javax.swing.JSpinner fechaReporte;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1192	private Componentes.BotonDesplegable filtroCajero;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1193	private Componentes.BotonDesplegable filtroMetodo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1194	private Componentes.BotonDesplegable filtroServicio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1195	private javax.swing.JLabel labelCancelacionesCaja;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1196	private javax.swing.JLabel labelCancelados;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1197	private javax.swing.JLabel labelEfectivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1198	private Labels.LabelEscalable labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1199	private Labels.LabelEscalable labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1200	private Labels.LabelEscalable labelEscalable4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1201	private Labels.LabelEscalable labelEscalable5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1202	private javax.swing.JLabel labelOtros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1203	private javax.swing.JLabel labelPedidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1204	private javax.swing.JLabel labelPorcentajeEfectivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1205	private javax.swing.JLabel labelPorcentajeOtros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1206	private javax.swing.JLabel labelPorcentajeTarjeta;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1207	private javax.swing.JLabel labelResumen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1208	private javax.swing.JLabel labelResumenSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1209	private javax.swing.JLabel labelSubtitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1210	private javax.swing.JLabel labelTarjeta;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1211	private javax.swing.JLabel labelTicket;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1212	private javax.swing.JLabel labelTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1213	private javax.swing.JLabel labelTituloCancelados;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1214	private javax.swing.JLabel labelTituloPedidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1215	private javax.swing.JLabel labelTituloTicket;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1216	private javax.swing.JLabel labelTituloVentas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1217	private javax.swing.JLabel labelVentas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1218	private javax.swing.JLabel otrosTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1219	private Componentes.PanelFlotante panelCancelados;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1220	private Componentes.PanelCircular panelCircular2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1221	private Componentes.PanelCircular panelCircular3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1222	private Componentes.PanelCircular panelCircular4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1223	private Componentes.PanelCircular panelCircular5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1224	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1225	private javax.swing.JPanel panelPaginacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1226	private Componentes.PanelFlotante panelPedidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1227	private Componentes.PanelFlotante panelResumen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1228	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1229	private Componentes.PanelFlotante panelTicket;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1230	private Componentes.PanelFlotante panelVentas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1231	private javax.swing.JScrollPane scrollPedidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1232	private javax.swing.JSeparator separadorResumen1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1233	private javax.swing.JSeparator separadorResumen2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1234	private javax.swing.JSeparator separadorResumen3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1235	private javax.swing.JSeparator separadorResumen4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1236	private Componentes.TablaAdministrativa tablaPedidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1237	private javax.swing.JLabel tarjetaTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1238	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1239	}	Cierra el bloque abierto anteriormente.

16.84 src/GUI_ADMINISTRADOR/UsuarioFormDialog.java

Interfaz del módulo administrador: UsuarioFormDialog. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 53 | Archivo 84 de 157

Clases/tipos detectados: UsuarioFormDialog

Línea	Constructor o método detectado
18	private UsuarioFormDialog(
44	public static boolean mostrar(

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.UsuarioCRUD;	Importa CRUD.UsuarioCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Usuario;	Importa Modelos.Usuario para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
7	import javax.swing.JDialog;	Importa javax.swing.JDialog para poder usar esa clase sin escribir su nombre completo.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	* Ventana modal mínima. Todo el contenido visible está en	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	* UsuarioFormPanel.form para poder moverlo desde Design en NetBeans.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
12	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	@SuppressWarnings("serial")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
14	public final class UsuarioFormDialog extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	private boolean guardado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	private UsuarioFormDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19	Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	UsuarioCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21	Usuario usuario) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	super(propietario, ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
23	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
24	setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
25	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	UsuarioFormPanel formulario = new UsuarioFormPanel(crud, usuario);	Crea una nueva instancia de UsuarioFormPanel y la usa o guarda para continuar el proceso.
28	formulario.addPropertyChangeListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	if (UsuarioFormPanel.EVENTO_GUARDADO.equals(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
30	evento.getPropertyName()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	guardado = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
32	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	} else if (UsuarioFormPanel.EVENTO_CANCELAR.equals(	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

34	evento.getPropertyName())) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	}	Cierra el bloque abierto anteriormente.
37	}};	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	setContentPane(formulario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	}	Cierra el bloque abierto anteriormente.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	public static boolean mostrar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	UsuarioCRUD crud,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	Usuario usuario) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	UsuarioFormDialog dialogo = new UsuarioFormDialog(	Crea una nueva instancia de UsuarioFormDialog y la usa o guarda para continuar el proceso.
49	propietario, crud, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
51	return dialogo.guardado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
52	}	Cierra el bloque abierto anteriormente.
53	}	Cierra el bloque abierto anteriormente.

16.85 src/GUI_ADMINISTRADOR/UsuarioFormPanel.form

Metadatos visuales de NetBeans para diseñar UsuarioFormPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 435 | Archivo 85 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
7	<Dimension value="[790, 710]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	</Property>	Cierra la sección XML abierta anteriormente.
9	</Properties>	Cierra la sección XML abierta anteriormente.
10	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20		Línea en blanco para organizar el XML del formulario.
21		Línea en blanco para organizar el XML del formulario.
22	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84, -19, 0, 5, 115, 114, 0, 18, 106, 97, 118, 97, 46, 97, 119, 116, 46, 68, 105, 109, 101, 110, 115, 105, 111, 110, 65, -114, -39, -41, -84, 95, 68, 20, 2, 0, 2, 73, 0, 6, 104, 101, 105, 103, 104, 116, 73, 0, 5, 119, 105, 100, 116, 104, 120, 112, 0, 0, 2, -40, 0, 0, 3, 16"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	</AuxValues>	Cierra la sección XML abierta anteriormente.
24		Línea en blanco para organizar el XML del formulario.
25	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
26	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
27	</Layout>	Cierra la sección XML abierta anteriormente.
28	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
29	<Container class="Componentes.PanelFlotante" name="panelTarjeta">	Declara un componente o contenedor visual y su clase Java.
30	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
31	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Color blue="ea" green="e3" red="de" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="radio" type="int" value="26"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	</Properties>	Cierra la sección XML abierta anteriormente.
36	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

37	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
38	<AbsoluteConstraints x="0" y="0" width="790" height="730"/>	Define la posición y el tamaño del componente dentro del diseño.
39	</Constraint>	Cierra la sección XML abierta anteriormente.
40	</Constraints>	Cierra la sección XML abierta anteriormente.
41		Línea en blanco para organizar el XML del formulario.
42	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
43	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
44	</Layout>	Cierra la sección XML abierta anteriormente.
45	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Container class="Componentes.PanelCircular" name="panelIcono">	Declara un componente o contenedor visual y su clase Java.
47	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
48	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
49	<AbsoluteConstraints x="40" y="35" width="80" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
50	</Constraint>	Cierra la sección XML abierta anteriormente.
51	</Constraints>	Cierra la sección XML abierta anteriormente.
52		Línea en blanco para organizar el XML del formulario.
53	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
54	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
55	</Layout>	Cierra la sección XML abierta anteriormente.
56	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
57	<Component class="Labels.LabelEscalable" name="labelIconoAgregar">	Declara un componente o contenedor visual y su clase Java.
58	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
59	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
60	<Image iconType="3" name="/Imagenes/icono_agregarusuario.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
61	</Property>	Cierra la sección XML abierta anteriormente.
62	</Properties>	Cierra la sección XML abierta anteriormente.
63	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
64	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
65	<AbsoluteConstraints x="0" y="0" width="80" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
66	</Constraint>	Cierra la sección XML abierta anteriormente.
67	</Constraints>	Cierra la sección XML abierta anteriormente.
68	</Component>	Cierra la sección XML abierta anteriormente.
69	<Component class="Labels.LabelEscalable" name="labelIconoEditar">	Declara un componente o contenedor visual y su clase Java.
70	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
71	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
72	<Image iconType="3" name="/Imagenes/icono_editarUsuario.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
73	</Property>	Cierra la sección XML abierta anteriormente.
74	<Property name="visible" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
75	</Properties>	Cierra la sección XML abierta anteriormente.
76	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

77	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
78	<AbsoluteConstraints x="0" y="0" width="80" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
79	</Constraint>	Cierra la sección XML abierta anteriormente.
80	</Constraints>	Cierra la sección XML abierta anteriormente.
81	</Component>	Cierra la sección XML abierta anteriormente.
82	</SubComponents>	Cierra la sección XML abierta anteriormente.
83	</Container>	Cierra la sección XML abierta anteriormente.
84	<Component class="javax.swing.JLabel" name="labelTitulo">	Declara un componente o contenedor visual y su clase Java.
85	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
86	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
87		Metadato de NetBeans que conserva el estado editable del diseñador visual.
88	</Property>	Cierra la sección XML abierta anteriormente.
89	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
90	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
91	</Property>	Cierra la sección XML abierta anteriormente.
92	<Property name="text" type="java.lang.String" value="Agregar usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
93	</Properties>	Cierra la sección XML abierta anteriormente.
94	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
95	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
96	<AbsoluteConstraints x="135" y="35" width="560" height="35"/>	Define la posición y el tamaño del componente dentro del diseño.
97	</Constraint>	Cierra la sección XML abierta anteriormente.
98	</Constraints>	Cierra la sección XML abierta anteriormente.
99	</Component>	Cierra la sección XML abierta anteriormente.
100	<Component class="javax.swing.JLabel" name="labelSubtitulo">	Declara un componente o contenedor visual y su clase Java.
101	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
102	<Property name="text" type="java.lang.String" value="Completa los datos del nuevo usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
103	</Properties>	Cierra la sección XML abierta anteriormente.
104	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
105	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
106	<AbsoluteConstraints x="135" y="68" width="560" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
107	</Constraint>	Cierra la sección XML abierta anteriormente.
108	</Constraints>	Cierra la sección XML abierta anteriormente.
109	</Component>	Cierra la sección XML abierta anteriormente.
110	<Component class="javax.swing.JLabel" name="labelNombre">	Declara un componente o contenedor visual y su clase Java.
111	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
112	<Property name="text" type="java.lang.String" value="Nombre"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
113	</Properties>	Cierra la sección XML abierta anteriormente.
114	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
115	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
116	<AbsoluteConstraints x="40" y="130" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
117	</Constraint>	Cierra la sección XML abierta anteriormente.

118	</Constraints>	Cierra la sección XML abierta anteriormente.
119	</Component>	Cierra la sección XML abierta anteriormente.
120	<Component class="javax.swing.JTextField" name="campoNombre">	Declara un componente o contenedor visual y su clase Java.
121	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
122	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
123	<AbsoluteConstraints x="40" y="158" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
124	</Constraint>	Cierra la sección XML abierta anteriormente.
125	</Constraints>	Cierra la sección XML abierta anteriormente.
126	</Component>	Cierra la sección XML abierta anteriormente.
127	<Component class="javax.swing.JLabel" name="labelApellido">	Declara un componente o contenedor visual y su clase Java.
128	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
129	<Property name="text" type="java.lang.String" value="Apellido"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
130	</Properties>	Cierra la sección XML abierta anteriormente.
131	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
132	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
133	<AbsoluteConstraints x="420" y="130" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
134	</Constraint>	Cierra la sección XML abierta anteriormente.
135	</Constraints>	Cierra la sección XML abierta anteriormente.
136	</Component>	Cierra la sección XML abierta anteriormente.
137	<Component class="javax.swing.JTextField" name="campoApellido">	Declara un componente o contenedor visual y su clase Java.
138	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
139	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
140	<AbsoluteConstraints x="420" y="158" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
141	</Constraint>	Cierra la sección XML abierta anteriormente.
142	</Constraints>	Cierra la sección XML abierta anteriormente.
143	</Component>	Cierra la sección XML abierta anteriormente.
144	<Component class="javax.swing.JLabel" name="labelUsuario">	Declara un componente o contenedor visual y su clase Java.
145	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
146	<Property name="text" type="java.lang.String" value="Usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
147	</Properties>	Cierra la sección XML abierta anteriormente.
148	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
149	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
150	<AbsoluteConstraints x="40" y="220" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
151	</Constraint>	Cierra la sección XML abierta anteriormente.
152	</Constraints>	Cierra la sección XML abierta anteriormente.
153	</Component>	Cierra la sección XML abierta anteriormente.
154	<Component class="javax.swing.JTextField" name="campoUsuario">	Declara un componente o contenedor visual y su clase Java.
155	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
156	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
157	<AbsoluteConstraints x="40" y="248" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
158	</Constraint>	Cierra la sección XML abierta anteriormente.

159	</Constraints>	Cierra la sección XML abierta anteriormente.
160	</Component>	Cierra la sección XML abierta anteriormente.
161	<Component class="javax.swing.JLabel" name="labelCorreo">	Declara un componente o contenedor visual y su clase Java.
162	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
163	<Property name="text" type="java.lang.String" value="Correo electrónico"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
164	</Properties>	Cierra la sección XML abierta anteriormente.
165	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
166	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
167	<AbsoluteConstraints x="420" y="220" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
168	</Constraint>	Cierra la sección XML abierta anteriormente.
169	</Constraints>	Cierra la sección XML abierta anteriormente.
170	</Component>	Cierra la sección XML abierta anteriormente.
171	<Component class="javax.swing.JTextField" name="campoCorreo">	Declara un componente o contenedor visual y su clase Java.
172	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
173	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
174	<AbsoluteConstraints x="420" y="248" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
175	</Constraint>	Cierra la sección XML abierta anteriormente.
176	</Constraints>	Cierra la sección XML abierta anteriormente.
177	</Component>	Cierra la sección XML abierta anteriormente.
178	<Component class="javax.swing.JLabel" name="labelRol">	Declara un componente o contenedor visual y su clase Java.
179	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
180	<Property name="text" type="java.lang.String" value="Rol"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
181	</Properties>	Cierra la sección XML abierta anteriormente.
182	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
183	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
184	<AbsoluteConstraints x="40" y="310" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
185	</Constraint>	Cierra la sección XML abierta anteriormente.
186	</Constraints>	Cierra la sección XML abierta anteriormente.
187	</Component>	Cierra la sección XML abierta anteriormente.
188	<Component class="Componentes.BotonDesplegable" name="selectorRol">	Declara un componente o contenedor visual y su clase Java.
189	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
190	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
191	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
192	</Property>	Cierra la sección XML abierta anteriormente.
193	<Property name="text" type="java.lang.String" value="Seleccionar rol"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
194	<Property name="textoDesplegable" type="java.lang.String" value="ADMINISTRADOR;CAJERO"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
195	</Properties>	Cierra la sección XML abierta anteriormente.
196	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
197	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
198	<AbsoluteConstraints x="40" y="338" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
199	</Constraint>	Cierra la sección XML abierta anteriormente.

200	</Constraints>	Cierra la sección XML abierta anteriormente.
201	</Component>	Cierra la sección XML abierta anteriormente.
202	<Component class="javax.swing.JLabel" name="labelEstado">	Declara un componente o contenedor visual y su clase Java.
203	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
204	<Property name="text" type="java.lang.String" value="Estado"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
205	</Properties>	Cierra la sección XML abierta anteriormente.
206	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
207	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
208	<AbsoluteConstraints x="420" y="310" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
209	</Constraint>	Cierra la sección XML abierta anteriormente.
210	</Constraints>	Cierra la sección XML abierta anteriormente.
211	</Component>	Cierra la sección XML abierta anteriormente.
212	<Component class="Componentes.BotonDesplegable" name="selectorEstado">	Declara un componente o contenedor visual y su clase Java.
213	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
214	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
215	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
216	</Property>	Cierra la sección XML abierta anteriormente.
217	<Property name="text" type="java.lang.String" value="Activo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
218	<Property name="textoDesplegable" type="java.lang.String" value="Activo;Inactivo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
219	</Properties>	Cierra la sección XML abierta anteriormente.
220	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
221	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
222	<AbsoluteConstraints x="420" y="338" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
223	</Constraint>	Cierra la sección XML abierta anteriormente.
224	</Constraints>	Cierra la sección XML abierta anteriormente.
225	</Component>	Cierra la sección XML abierta anteriormente.
226	<Component class="javax.swing.JLabel" name="labelTurno">	Declara un componente o contenedor visual y su clase Java.
227	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
228	<Property name="text" type="java.lang.String" value="Turno del cajero"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
229	</Properties>	Cierra la sección XML abierta anteriormente.
230	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
231	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
232	<AbsoluteConstraints x="420" y="400" width="330" height="24"/>	Define la posición y el tamaño del componente dentro del diseño.
233	</Constraint>	Cierra la sección XML abierta anteriormente.
234	</Constraints>	Cierra la sección XML abierta anteriormente.
235	</Component>	Cierra la sección XML abierta anteriormente.
236	<Component class="Componentes.BotonDesplegable" name="selectorTurno">	Declara un componente o contenedor visual y su clase Java.
237	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
238	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
239	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
240	</Property>	Cierra la sección XML abierta anteriormente.

241	<Property name="text" type="java.lang.String" value="Sin turno"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
242	<Property name="textoDesplegable" type="java.lang.String" value="Mañana (06:00–14:00);Tarde (14:00–22:00);Sin turno"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
243	</Properties>	Cierra la sección XML abierta anteriormente.
244	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
245	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
246	<AbsoluteConstraints x="420" y="428" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
247	</Constraint>	Cierra la sección XML abierta anteriormente.
248	</Constraints>	Cierra la sección XML abierta anteriormente.
249	</Component>	Cierra la sección XML abierta anteriormente.
250	<Component class="javax.swing.JCheckBox" name="cambiarContrasena">	Declara un componente o contenedor visual y su clase Java.
251	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
252	<Property name="text" type="java.lang.String" value="Cambiar contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
253	</Properties>	Cierra la sección XML abierta anteriormente.
254	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
255	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="cambiarContrasenaActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
256	</Events>	Cierra la sección XML abierta anteriormente.
257	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
258	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
259	<AbsoluteConstraints x="40" y="490" width="250" height="30"/>	Define la posición y el tamaño del componente dentro del diseño.
260	</Constraint>	Cierra la sección XML abierta anteriormente.
261	</Constraints>	Cierra la sección XML abierta anteriormente.
262	</Component>	Cierra la sección XML abierta anteriormente.
263	<Container class="javax.swing.JPanel" name="panelContrasenas">	Declara un componente o contenedor visual y su clase Java.
264	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
265	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
266	</Properties>	Cierra la sección XML abierta anteriormente.
267	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
268	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
269	<AbsoluteConstraints x="40" y="528" width="710" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
270	</Constraint>	Cierra la sección XML abierta anteriormente.
271	</Constraints>	Cierra la sección XML abierta anteriormente.
272		Línea en blanco para organizar el XML del formulario.
273	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
274	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
275	</Layout>	Cierra la sección XML abierta anteriormente.
276	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
277	<Component class="javax.swing.JLabel" name="labelContrasena">	Declara un componente o contenedor visual y su clase Java.
278	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
279	<Property name="text" type="java.lang.String" value="Contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
280	</Properties>	Cierra la sección XML abierta anteriormente.
281	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

282	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
283	<AbsoluteConstraints x="0" y="0" width="330" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
284	</Constraint>	Cierra la sección XML abierta anteriormente.
285	</Constraints>	Cierra la sección XML abierta anteriormente.
286	</Component>	Cierra la sección XML abierta anteriormente.
287	<Component class="javax.swing.JButton" name="botonVerContrasena">	Declara un componente o contenedor visual y su clase Java.
288	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
289	<Property name="contentAreaFilled" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
290	<Property name="focusPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
291	<Property name="borderPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
292	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
293	<Image iconType="3" name="/Imagenes/visualizar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
294	</Property>	Cierra la sección XML abierta anteriormente.
295	<Property name="toolTipText" type="java.lang.String" value="Mostrar contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
296	</Properties>	Cierra la sección XML abierta anteriormente.
297	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
298	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonVerContrasenaActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
299	</Events>	Cierra la sección XML abierta anteriormente.
300	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
301	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
302	<AbsoluteConstraints x="285" y="28" width="40" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
303	</Constraint>	Cierra la sección XML abierta anteriormente.
304	</Constraints>	Cierra la sección XML abierta anteriormente.
305	</Component>	Cierra la sección XML abierta anteriormente.
306	<Component class="javax.swing.JPasswordField" name="campoContrasena">	Declara un componente o contenedor visual y su clase Java.
307	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
308	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
309	<AbsoluteConstraints x="0" y="26" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
310	</Constraint>	Cierra la sección XML abierta anteriormente.
311	</Constraints>	Cierra la sección XML abierta anteriormente.
312	</Component>	Cierra la sección XML abierta anteriormente.
313	<Component class="javax.swing.JLabel" name="labelConfirmar">	Declara un componente o contenedor visual y su clase Java.
314	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
315	<Property name="text" type="java.lang.String" value="Confirmar contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
316	</Properties>	Cierra la sección XML abierta anteriormente.
317	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
318	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
319	<AbsoluteConstraints x="380" y="0" width="330" height="22"/>	Define la posición y el tamaño del componente dentro del diseño.
320	</Constraint>	Cierra la sección XML abierta anteriormente.
321	</Constraints>	Cierra la sección XML abierta anteriormente.

322	</Component>	Cierra la sección XML abierta anteriormente.
323	<Component class="javax.swing.JButton" name="botonVerConfirmacion">	Declara un componente o contenedor visual y su clase Java.
324	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
325	<Property name="contentAreaFilled" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
326	<Property name="focusPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
327	<Property name="borderPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
328	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
329	<Image iconType="3" name="/Imagenes/visualizar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
330	</Property>	Cierra la sección XML abierta anteriormente.
331	<Property name="toolTipText" type="java.lang.String" value="Mostrar contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
332	</Properties>	Cierra la sección XML abierta anteriormente.
333	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
334	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonVerConfirmacionActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
335	</Events>	Cierra la sección XML abierta anteriormente.
336	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
337	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
338	<AbsoluteConstraints x="665" y="28" width="40" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
339	</Constraint>	Cierra la sección XML abierta anteriormente.
340	</Constraints>	Cierra la sección XML abierta anteriormente.
341	</Component>	Cierra la sección XML abierta anteriormente.
342	<Component class="javax.swing.JPasswordField" name="campoConfirmar">	Declara un componente o contenedor visual y su clase Java.
343	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
344	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
345	<AbsoluteConstraints x="380" y="26" width="330" height="46"/>	Define la posición y el tamaño del componente dentro del diseño.
346	</Constraint>	Cierra la sección XML abierta anteriormente.
347	</Constraints>	Cierra la sección XML abierta anteriormente.
348	</Component>	Cierra la sección XML abierta anteriormente.
349	</SubComponents>	Cierra la sección XML abierta anteriormente.
350	</Container>	Cierra la sección XML abierta anteriormente.
351	<Component class="javax.swing.JLabel" name="labelError">	Declara un componente o contenedor visual y su clase Java.
352	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
353	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
354	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
355	</Property>	Cierra la sección XML abierta anteriormente.
356	<Property name="text" type="java.lang.String" value=" "/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
357	</Properties>	Cierra la sección XML abierta anteriormente.
358	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
359	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
360	<AbsoluteConstraints x="40" y="612" width="710" height="26"/>	Define la posición y el tamaño del componente dentro del diseño.
361	</Constraint>	Cierra la sección XML abierta anteriormente.

362	</Constraints>	Cierra la sección XML abierta anteriormente.
363	</Component>	Cierra la sección XML abierta anteriormente.
364	<Component class="Componentes.BotonRedondeado" name="botonDesactivar">	Declara un componente o contenedor visual y su clase Java.
365	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
366	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
367	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
368	</Property>	Cierra la sección XML abierta anteriormente.
369	<Property name="text" type="java.lang.String" value="Desactivar usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
370	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
">		
371	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
372	</Property>	Cierra la sección XML abierta anteriormente.
373	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
374	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
375	</Property>	Cierra la sección XML abierta anteriormente.
376	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
377	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
378	</Property>	Cierra la sección XML abierta anteriormente.
379	<Property name="grosorBorde" type="float" value="1.0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
380	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
381	</Properties>	Cierra la sección XML abierta anteriormente.
382	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
383	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonDesactivarActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
384	</Events>	Cierra la sección XML abierta anteriormente.
385	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
386	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
387	<AbsoluteConstraints x="40" y="650" width="180" height="48"/>	Define la posición y el tamaño del componente dentro del diseño.
388	</Constraint>	Cierra la sección XML abierta anteriormente.
389	</Constraints>	Cierra la sección XML abierta anteriormente.
390	</Component>	Cierra la sección XML abierta anteriormente.
391	<Component class="Componentes.BotonRedondeado" name="botonCancelar">	Declara un componente o contenedor visual y su clase Java.
392	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
393	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
394	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
395	</Property>	Cierra la sección XML abierta anteriormente.
396	<Property name="text" type="java.lang.String" value="Cancelar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
397	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
">		
398	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
399	</Property>	Cierra la sección XML abierta anteriormente.

400	>	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
401		<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
402		</Property>	Cierra la sección XML abierta anteriormente.
403	>	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
404		<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
405		</Property>	Cierra la sección XML abierta anteriormente.
406		<Property name="grosorBorde" type="float" value="1.0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
407		<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
408		</Properties>	Cierra la sección XML abierta anteriormente.
409		<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
410		<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonCancelarActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
411		</Events>	Cierra la sección XML abierta anteriormente.
412		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
413		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
414		<AbsoluteConstraints x="440" y="650" width="130" height="48"/>	Define la posición y el tamaño del componente dentro del diseño.
415		</Constraint>	Cierra la sección XML abierta anteriormente.
416		</Constraints>	Cierra la sección XML abierta anteriormente.
417		</Component>	Cierra la sección XML abierta anteriormente.
418		<Component class="Componentes.BotonRedondeado" name="botonGuardar">	Declara un componente o contenedor visual y su clase Java.
419		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
420		<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
421		<Property name="text" type="java.lang.String" value="Guardar usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
422		</Properties>	Cierra la sección XML abierta anteriormente.
423		<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
424		<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonGuardarActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
425		</Events>	Cierra la sección XML abierta anteriormente.
426		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
427		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
428		<AbsoluteConstraints x="585" y="650" width="165" height="48"/>	Define la posición y el tamaño del componente dentro del diseño.
429		</Constraint>	Cierra la sección XML abierta anteriormente.
430		</Constraints>	Cierra la sección XML abierta anteriormente.
431		</Component>	Cierra la sección XML abierta anteriormente.
432		</SubComponents>	Cierra la sección XML abierta anteriormente.
433		</Container>	Cierra la sección XML abierta anteriormente.
434		</SubComponents>	Cierra la sección XML abierta anteriormente.
435		</Form>	Cierra la sección XML abierta anteriormente.

16.86 src/GUI_ADMINISTRADOR/UsuarioFormPanel.java

Interfaz del módulo administrador: UsuarioFormPanel. Combina componentes visuales con acciones del usuario.

Tipo: java | Líneas: 674 | Archivo 86 de 157

Clases/tipos detectados: UsuarioFormPanel

Línea	Constructor o método detectado
35	private static final Pattern CORREO_VALIDO = Pattern.compile(
51	public UsuarioFormPanel() {
55	public UsuarioFormPanel(UsuarioCRUD crud, Usuario original) {
66	private void configurarAspecto() {
126	private void configurarSelector(Componentes.BotonDesplegable selector) {
141	private void configurarSelectores() {
157	private void cargarModo() {
192	private void actualizarTurnoSegunRol() {
205	private void guardar() {
330	private boolean dejaAlSistemaSinAdministrador() throws SQLException {
341	private void desactivar() {
371	private void limpiarErrores() {
384	private void marcarError(JTextField campo) {
390	private void mostrarError(String mensaje) {
394	private void alternarContrasena(JPasswordField campo, boolean confirmacion) {
414	private void cambiarContrasenaActionPerformed(
421	private void botonVerContrasenaActionPerformed(
426	private void botonVerConfirmacionActionPerformed(
431	private void botonDesactivarActionPerformed(
436	private void botonCancelarActionPerformed(
441	private void botonGuardarActionPerformed(
448	private void initComponents() {
552	public void actionPerformed(java.awt.event.ActionEvent evt) {
570	public void actionPerformed(java.awt.event.ActionEvent evt) {
586	public void actionPerformed(java.awt.event.ActionEvent evt) {
607	public void actionPerformed(java.awt.event.ActionEvent evt) {
621	public void actionPerformed(java.awt.event.ActionEvent evt) {
630	public void actionPerformed(java.awt.event.ActionEvent evt) {

1	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import CRUD.UsuarioCRUD;	Importa CRUD.UsuarioCRUD para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.Usuario;	Importa Modelos.Usuario para poder usar esa clase sin escribir su nombre completo.
5	import Utilidades.SeguridadContrasena;	Importa Utilidades.SeguridadContrasena para poder usar esa clase sin escribir su nombre completo.
6	import Utilidades.SesionUsuario;	Importa Utilidades.SesionUsuario para poder usar esa clase sin escribir su nombre completo.
7	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
8	import Utilidades.HorarioTurno;	Importa Utilidades.HorarioTurno para poder usar esa clase sin escribir su nombre completo.
9	import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa una hora sin fecha.
10	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
12	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
13	import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa fecha y hora juntas.
14	import java.util.regex.Pattern;	Importa java.util.regex.Pattern para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.JPasswordField;	Importa javax.swing.JPasswordField para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	* Contenido visual editable desde Design para agregar y editar usuarios.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	* UsuarioFormDialog sólo lo coloca dentro de una ventana modal.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
26	public class UsuarioFormPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	public static final String EVENTO_GUARDADO = "usuarioGuardado";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
29	public static final String EVENTO_CANCELAR = "cancelarFormulario";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
32	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
33	private static final Color BORDE = new Color(222, 227, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
34	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
35	private static final Pattern CORREO_VALIDO = Pattern.compile(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
36	"^[A-Z0-9._%+-]+@[A-Z0-9.-]+\\.[A-Z]{2,}\$",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	Pattern.CASE_INSENSITIVE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
40	private UsuarioCRUD crud;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
41	private Usuario original;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42	private String rolSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43	private String turnoSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

44	private boolean estadoSeleccionado = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	private boolean contrasenaVisible;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
46	private boolean confirmacionVisible;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	private char ecoContrasena;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
48	private char ecoConfirmacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	/** Constructor vacío requerido por el diseñador de NetBeans. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
51	public UsuarioFormPanel() {	Constructor de UsuarioFormPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
52	this(null, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	}	Cierra el bloque abierto anteriormente.
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.
55	public UsuarioFormPanel(UsuarioCRUD crud, Usuario original) {	Constructor de UsuarioFormPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
56	this.crud = crud;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
57	this.original = original;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
58	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	ecoContrasena = campoContrasena.getEchoChar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
60	ecoConfirmacion = campoConfirmar.getEchoChar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
61	configurarAspecto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	configurarSelectores();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	cargarModo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	}	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	private void configurarAspecto() {	Declara el método configurarAspecto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
67	labelTitulo.setFont(tema.negrita(25f));	Configura una propiedad mediante setFont en el objeto indicado.
68	labelSubtitulo.setFont(tema.regular(15f));	Configura una propiedad mediante setFont en el objeto indicado.
69	labelSubtitulo.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
70	labelError.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	javax.swing.JLabel[] etiquetas = {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
73	labelNombre, labelApellido, labelUsuario, labelCorreo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
74	labelRol, labelEstado, labelTurno,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75	labelContrasena, labelConfirmar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
76	};	Cierra el bloque abierto anteriormente y termina la declaración.
77	for (javax.swing.JLabel etiqueta : etiquetas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
78	etiqueta.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
79	etiqueta.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
80	}	Cierra el bloque abierto anteriormente.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	JTextField[] campos = {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	campoNombre, campoApellido, campoUsuario, campoCorreo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	campoContrasena, campoConfirmar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
85	};	Cierra el bloque abierto anteriormente y termina la declaración.
86	for (JTextField campo : campos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
87	campo.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.

88	campo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
89	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
90	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
91	new EmptyBorder(0, 13, 0, 13)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
92	}	Cierra el bloque abierto anteriormente.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	// Los botones del ojo se superponen al extremo derecho de los campos.	Comentario para humanos; Java no ejecuta este texto. Los botones del ojo se superponen al extremo derecho de los campos.
95	// Este espacio evita que la contraseña quede debajo del icono.	Comentario para humanos; Java no ejecuta este texto. Este espacio evita que la contraseña quede debajo del icono.
96	campoContrasena.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
97	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
98	new EmptyBorder(0, 13, 0, 52)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
99	campoConfirmar.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
100	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	new EmptyBorder(0, 13, 0, 52)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	// En AbsoluteLayout el botón y el campo ocupan parte de la misma zona.	Comentario para humanos; Java no ejecuta este texto. En AbsoluteLayout el botón y el campo ocupan parte de la misma zona.
104	// Los colocamos al frente para que Swing no los oculte tras el campo.	Comentario para humanos; Java no ejecuta este texto. Los colocamos al frente para que Swing no los oculte tras el campo.
105	panelContrasenas.setComponentZOrder(botonVerConfirmacion, 0);	Configura una propiedad mediante setComponentZOrder en el objeto indicado.
106	panelContrasenas.setComponentZOrder(botonVerContrasena, 0);	Configura una propiedad mediante setComponentZOrder en el objeto indicado.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	selectorRol.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
109	selectorEstado.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
110	selectorTurno.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
111	configurarSelector(selectorRol);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	configurarSelector(selectorEstado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113	configurarSelector(selectorTurno);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	cambiarContrasena.setFont(tema.media(14f));	Configura una propiedad mediante setFont en el objeto indicado.
115	cambiarContrasena.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
116	botonGuardar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
117	botonCancelar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
118	botonDesactivar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	botonVerContrasena.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
121	Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	botonVerConfirmacion.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
123	Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	}	Cierra el bloque abierto anteriormente.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	private void configurarSelector(Componentes.BotonDesplegable selector) {	Declara el método configurarSelector. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
127	selector.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
128	selector.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
129	selector.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

130	selector.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
131	selector.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
132	selector.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
133	selector.setAnchoMenu(330);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
134	selector.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
135	selector.setBorderPainted(true);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
136	selector.setBorder(BorderFactory.createCompoundBorder(137 BorderFactory.createLineBorder(BORDE, 1, true), 138 new EmptyBorder(0, 13, 0, 34)));	Configura una propiedad mediante setBorder en el objeto indicado.
139	}	Cierra el bloque abierto anteriormente.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	private void configurarSelectores() {	Declara el método configurarSelectores. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
142	selectorRol.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
143	rolSeleccionado = evento.getActionCommand();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
144	selectorRol.setText(rolSeleccionado);	Cambia el texto visible de un componente de la interfaz.
145	actualizarTurnoSegunRol();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	selectorTurno.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
148	turnoSeleccionado = evento.getActionCommand();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
149	selectorTurno.setText(turnoSeleccionado);	Cambia el texto visible de un componente de la interfaz.
150	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	selectorEstado.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
152	estadoSeleccionado = "Activo".equals(evento.getActionCommand());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
153	selectorEstado.setText(evento.getActionCommand());	Cambia el texto visible de un componente de la interfaz.
154	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	private void cargarModo() {	Declara el método cargarModo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
158	boolean editando = original != null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
159	// Los iconos provienen del archivo .form. Aquí solamente elegimos	Comentario para humanos; Java no ejecuta este texto. Los iconos provienen del archivo .form. Aquí solamente elegimos
160	// cuál de los dos JLabel diseñados en NetBeans debe mostrarse.	Comentario para humanos; Java no ejecuta este texto. cuál de los dos JLabel diseñados en NetBeans debe mostrarse.
161	labelIconoAgregar.setVisible(!editando);	Muestra u oculta una ventana o componente según el valor booleano enviado.
162	labelIconoEditar.setVisible(editando);	Muestra u oculta una ventana o componente según el valor booleano enviado.
163	labelTitulo.setText(editando ? "Editar usuario" : "Agregar usuario");	Cambia el texto visible de un componente de la interfaz.
164	labelSubtitulo.setText(editando	Cambia el texto visible de un componente de la interfaz.
165	? "Actualiza la información de "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	+ String.format("#%04d", original.getIdUsuario())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	: "Completa los datos del nuevo usuario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	cambiarContrasena.setVisible(editando);	Muestra u oculta una ventana o componente según el valor booleano enviado.
170	panelContrasenas.setVisible(!editando);	Muestra u oculta una ventana o componente según el valor booleano enviado.
171	botonDesactivar.setVisible(editando && original.isActivo());	Muestra u oculta una ventana o componente según el valor booleano enviado.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	if (!editando) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

174	selectorTurno.setText("Sin turno");	Cambia el texto visible de un componente de la interfaz.
175	turnoSeleccionado = "Sin turno";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
176	actualizarTurnoSegunRol();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
178	}	Cierra el bloque abierto anteriormente.
179	campoNombre.setText(original.getNombre());	Cambia el texto visible de un componente de la interfaz.
180	campoApellido.setText(original.getApellido());	Cambia el texto visible de un componente de la interfaz.
181	campoUsuario.setText(original.getNombreUsuario());	Cambia el texto visible de un componente de la interfaz.
182	campoCorreo.setText(original.getCorreo());	Cambia el texto visible de un componente de la interfaz.
183	rolSeleccionado = original.getRol();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
184	estadoSeleccionado = original.isActive();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
185	selectorRol.setText(rolSeleccionado);	Cambia el texto visible de un componente de la interfaz.
186	selectorEstado.setText(estadoSeleccionado ? "Activo" : "Inactivo");	Cambia el texto visible de un componente de la interfaz.
187	turnoSeleccionado = HorarioTurno.etiqueta(original.getTurno());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
188	selectorTurno.setText(turnoSeleccionado);	Cambia el texto visible de un componente de la interfaz.
189	actualizarTurnoSegunRol();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	}	Cierra el bloque abierto anteriormente.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	private void actualizarTurnoSegunRol() {	Declara el método actualizarTurnoSegunRol. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
193	boolean esCajero = "CAJERO".equals(rolSeleccionado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
194	selectorTurno.setEnabled(esCajero);	Habilita o deshabilita la interacción con el componente.
195	if (!esCajero) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
196	turnoSeleccionado = "Sin turno";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
197	selectorTurno.setText(turnoSeleccionado);	Cambia el texto visible de un componente de la interfaz.
198	} else if (turnoSeleccionado == null	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
199	turnoSeleccionado.equals("Sin turno")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
200	turnoSeleccionado = "Seleccionar turno";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
201	selectorTurno.setText(turnoSeleccionado);	Cambia el texto visible de un componente de la interfaz.
202	}	Cierra el bloque abierto anteriormente.
203	}	Cierra el bloque abierto anteriormente.
204		Línea en blanco: separa visualmente bloques para que el código sea más legible.
205	private void guardar() {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
206	limpiarErrores();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
207	if (crud == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
208	mostrarError("El formulario no tiene conexión CRUD configurada.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
210	}	Cierra el bloque abierto anteriormente.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	String nombre = campoNombre.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
213	String apellido = campoApellido.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
214	String usuario = campoUsuario.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
215	String correo = campoCorreo.getText().trim().toLowerCase();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	if (nombre.isBlank() apellido.isBlank() usuario.isBlank())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

218	correo.isBlank() rolSeleccionado == null) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
219	mostrarError("Completa todos los campos obligatorios.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
220	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
221	}	Cierra el bloque abierto anteriormente.
222	String turno = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
223	LocalTime horaInicio = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
224	LocalTime horaFin = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
225	if ("CAJERO".equals(rolSeleccionado)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
226	if (turnoSeleccionado == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
227	turnoSeleccionado.startsWith("Seleccionar")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	mostrarError("Asigna un turno al cajero.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
229	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
230	}	Cierra el bloque abierto anteriormente.
231	if (turnoSeleccionado.startsWith("Mañana")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
232	turno = HorarioTurno.MANANA;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
233	horaInicio = HorarioTurno.INICIO_MANANA;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
234	horaFin = HorarioTurno.FIN_MANANA;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
235	} else if (turnoSeleccionado.startsWith("Tarde")) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
236	turno = HorarioTurno.TARDE;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
237	horaInicio = HorarioTurno.INICIO_TARDE;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
238	horaFin = HorarioTurno.FIN_TARDE;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
239	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
240	mostrarError("Asigna un turno al cajero.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
241	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
242	}	Cierra el bloque abierto anteriormente.
243	}	Cierra el bloque abierto anteriormente.
244	if (nombre.length() > 50 apellido.length() > 50	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
245	usuario.length() > 50 correo.length() > 120) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
246	mostrarError("Uno de los campos supera la longitud permitida.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
248	}	Cierra el bloque abierto anteriormente.
249	if (!usuario.matches("[A-Za-z0-9._-]{4,50}")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
250	marcarError(campoUsuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
251	mostrarError("El usuario debe tener 4 caracteres o más y no llevar espacios.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
252	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
253	}	Cierra el bloque abierto anteriormente.
254	if (!CORREO_VALIDO.matcher(correo).matches()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
255	marcarError(campoCorreo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
256	mostrarError("Ingresa un correo electrónico válido.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
257	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
258	}	Cierra el bloque abierto anteriormente.
259		Línea en blanco: separa visualmente bloques para que el código sea más legible.
260	boolean requiereContrasena = original == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
261	cambiarContrasena.isSelected();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

262	String contrasena = new String(campoContrasena.getPassword());	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
263	String confirmacion = new String(campoConfirmar.getPassword());	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
264	if (requiereContrasena) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
265	String errorContrasena = SeguridadContrasena.validarPolitica(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
266	contrasena, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
267	if (errorContrasena != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
268	marcarError(campoContrasena);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	mostrarError(errorContrasena);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
270	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
271	}	Cierra el bloque abierto anteriormente.
272	}	Cierra el bloque abierto anteriormente.
273	if (requiereContrasena && !contrasena.equals(confirmacion)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
274	marcarError(campoConfirmar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
275	mostrarError("Las contraseñas no coinciden.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
277	}	Cierra el bloque abierto anteriormente.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	int id = original == null ? 0 : original.getIdUsuario();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
280	botonGuardar.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
281	setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
282	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
283	if (crud.existeNombreUsuario(usuario, id)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
284	marcarError(campoUsuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
285	mostrarError("Ese nombre de usuario ya está registrado.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
286	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
287	}	Cierra el bloque abierto anteriormente.
288	if (crud.existeCorreo(correo, id)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
289	marcarError(campoCorreo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
290	mostrarError("Ese correo electrónico ya está registrado.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
291	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
292	}	Cierra el bloque abierto anteriormente.
293	if (dejaAlSistemaSinAdministrador()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
294	mostrarError("Debe existir al menos un administrador activo.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
295	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
296	}	Cierra el bloque abierto anteriormente.
297	// Esta protección sólo aplica al editar una cuenta existente.	Comentario para humanos; Java no ejecuta este texto. Esta protección sólo aplica al editar una cuenta existente.
298	// Durante un alta, tanto el ID temporal como una sesión no	Comentario para humanos; Java no ejecuta este texto. Durante un alta, tanto el ID temporal como una sesión no
299	// iniciada pueden valer 0 y no representan a la misma persona.	Comentario para humanos; Java no ejecuta este texto. iniciada pueden valer 0 y no representan a la misma persona.
300	if (original != null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
301	&& id == SesionUsuario.getIdUsuario()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
302	&& !estadoSeleccionado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
303	mostrarError("No puedes desactivar tu propia cuenta.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
304	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
305	}	Cierra el bloque abierto anteriormente.

306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	Usuario datos = new Usuario(	Crea una nueva instancia de Usuario y la usa o guarda para continuar el proceso.
308	id, nombre, apellido, usuario, correo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
309	rolSeleccionado, estadoSeleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
310	original == null ? LocalDateTime.now()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	: original.getFechaCreacion(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	turno, horaInicio, horaFin);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313	String hash = requiereContrasena	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
314	? SeguridadContrasena.hash(contrasena) : null;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
315		Línea en blanco: separa visualmente bloques para que el código sea más legible.
316	if (original == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
317	crud.insertar(datos, hash);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
318	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
319	crud.actualizar(datos, hash);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
320	}	Cierra el bloque abierto anteriormente.
321	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
322	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
323	mostrarError("No fue posible guardar: " + ex.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
324	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
325	botonGuardar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
326	setCursor(Cursor.getDefaultCursor());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
327	}	Cierra el bloque abierto anteriormente.
328	}	Cierra el bloque abierto anteriormente.
329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
330	private boolean dejaAlSistemaSinAdministrador() throws SQLException {	Declara el método dejaAlSistemaSinAdministrador. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
331	if (original == null !original.isActivo()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
332	!"ADMINISTRADOR".equals(original.getRol())) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
333	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
334	}	Cierra el bloque abierto anteriormente.
335	boolean dejaDeSerAdministrador = !estadoSeleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
336	!"ADMINISTRADOR".equals(rolSeleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337	return dejaDeSerAdministrador	Finaliza el método y devuelve el valor indicado a quien lo llamó.
338	&& crud.contarAdministradoresActivos() <= 1;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
339	}	Cierra el bloque abierto anteriormente.
340		Línea en blanco: separa visualmente bloques para que el código sea más legible.
341	private void desactivar() {	Declara el método desactivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
342	if (original == null crud == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
343	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
344	}	Cierra el bloque abierto anteriormente.
345	if (original.getIdUsuario() == SesionUsuario.getIdUsuario()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
346	mostrarError("No puedes desactivar tu propia cuenta.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
347	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
348	}	Cierra el bloque abierto anteriormente.
349	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

350	if ("ADMINISTRADOR".equals(original.getRol()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
351	&& crud.contarAdministradoresActivos() <= 1) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
352	mostrarError("No puedes desactivar al último administrador.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
353	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
354	}	Cierra el bloque abierto anteriormente.
355	int respuesta = JOptionPane.showConfirmDialog(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
356	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	"¿Desactivar a " + original.getNombreCompleto()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
358	+ "?\nSus pedidos se conservarán.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359	"Confirmar desactivación",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
360	JOptionPane.YES_NO_OPTION,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
361	JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
362	if (respuesta == JOptionPane.YES_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
363	crud.desactivar(original.getIdUsuario());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
364	firePropertyChange(EVENTO_GUARDADO, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
365	}	Cierra el bloque abierto anteriormente.
366	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
367	mostrarError("No fue posible desactivar: " + ex.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368	}	Cierra el bloque abierto anteriormente.
369	}	Cierra el bloque abierto anteriormente.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	private void limpiarErrores() {	Declara el método limpiarErrores. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
372	TextField[] campos = {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
373	campoNombre, campoApellido, campoUsuario, campoCorreo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
374	campoContraseña, campoConfirmar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
375	};	Cierra el bloque abierto anteriormente y termina la declaración.
376	for (TextField campo : campos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
377	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
378	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
379	new EmptyBorder(0, 13, 0, 13)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
380	}	Cierra el bloque abierto anteriormente.
381	labelError.setText(" ");	Cambia el texto visible de un componente de la interfaz.
382	}	Cierra el bloque abierto anteriormente.
383		Línea en blanco: separa visualmente bloques para que el código sea más legible.
384	private void marcarError(TextField campo) {	Declara el método marcarError. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
385	campo.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
386	BorderFactory.createLineBorder(ROJO, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	new EmptyBorder(0, 13, 0, 13)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
388	}	Cierra el bloque abierto anteriormente.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	private void mostrarError(String mensaje) {	Declara el método mostrarError. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
391	labelError.setText(mensaje);	Cambia el texto visible de un componente de la interfaz.
392	}	Cierra el bloque abierto anteriormente.
393		Línea en blanco: separa visualmente bloques para que el código sea más legible.

394	<code>private void alternarContrasena(JPasswordField campo, boolean confirmacion) {</code>	Declara el método <code>alternarContrasena</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
395	<code>boolean visible;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
396	<code>char ecoOculto;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
397	<code>if (confirmacion) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
398	<code>confirmacionVisible = !confirmacionVisible;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
399	<code>visible = confirmacionVisible;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
400	<code>ecoOculto = ecoConfirmacion;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
401	<code>botonVerConfirmacion.setToolTipText(</code>	Configura una propiedad mediante <code>setToolTipText</code> en el objeto indicado.
402	<code>visible ? "Ocultar contraseña" : "Mostrar contraseña");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
403	<code>} else {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
404	<code>contrasenaVisible = !contrasenaVisible;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
405	<code>visible = contrasenaVisible;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
406	<code>ecoOculto = ecoContrasena;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
407	<code>botonVerContrasena.setToolTipText(</code>	Configura una propiedad mediante <code>setToolTipText</code> en el objeto indicado.
408	<code>visible ? "Ocultar contraseña" : "Mostrar contraseña");</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	<code>}</code>	Cierra el bloque abierto anteriormente.
410	<code>campo.setEchoChar(visible ? (char) 0 : ecoOculto);</code>	Configura una propiedad mediante <code>setEchoChar</code> en el objeto indicado.
411	<code>campo.requestFocusInWindow();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
412	<code>}</code>	Cierra el bloque abierto anteriormente.
413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
414	<code>private void cambiarContrasenaActionPerformed(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	<code>java.awt.event.ActionEvent evt) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
416	<code>panelContrasenas.setVisible(cambiarContrasena.isSelected());</code>	Muestra u oculta una ventana o componente según el valor booleano enviado.
417	<code>revalidate();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
418	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
419	<code>}</code>	Cierra el bloque abierto anteriormente.
420		Línea en blanco: separa visualmente bloques para que el código sea más legible.
421	<code>private void botonVerContrasenaActionPerformed(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
422	<code>java.awt.event.ActionEvent evt) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
423	<code>alternarContrasena(campoContrasena, false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
424	<code>}</code>	Cierra el bloque abierto anteriormente.
425		Línea en blanco: separa visualmente bloques para que el código sea más legible.
426	<code>private void botonVerConfirmacionActionPerformed(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
427	<code>java.awt.event.ActionEvent evt) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
428	<code>alternarContrasena(campoConfirmar, true);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
429	<code>}</code>	Cierra el bloque abierto anteriormente.
430		Línea en blanco: separa visualmente bloques para que el código sea más legible.
431	<code>private void botonDesactivarActionPerformed(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
432	<code>java.awt.event.ActionEvent evt) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
433	<code>desactivar();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
434	<code>}</code>	Cierra el bloque abierto anteriormente.
435		Línea en blanco: separa visualmente bloques para que el código sea más legible.
436	<code>private void botonCancelarActionPerformed(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
437	<code>java.awt.event.ActionEvent evt) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

438	firePropertyChange(EVENTO_CANCELAR, false, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
439	}	Cierra el bloque abierto anteriormente.
440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
441	private void botonGuardarActionPerformed(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
442	java.awt.event.ActionEvent evt) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	guardar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
444	}	Cierra el bloque abierto anteriormente.
445		Línea en blanco: separa visualmente bloques para que el código sea más legible.
446	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
447	// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
448	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
449		Línea en blanco: separa visualmente bloques para que el código sea más legible.
450	panelTarjeta = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
451	panelIcono = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
452	labelIconoAgregar = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
453	labelIconoEditar = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
454	labelTitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
455	labelSubtitulo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
456	labelNombre = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
457	campoNombre = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
458	labelApellido = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
459	campoApellido = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
460	labelUsuario = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
461	campoUsuario = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
462	labelCorreo = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
463	campoCorreo = new javax.swing.JTextField();	Crea una nueva instancia de javax.swing.JTextField y la usa o guarda para continuar el proceso.
464	labelRol = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
465	selectorRol = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
466	labelEstado = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
467	selectorEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
468	labelTurno = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
469	selectorTurno = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
470	cambiarContrasena = new javax.swing.JCheckBox();	Crea una nueva instancia de javax.swing.JCheckBox y la usa o guarda para continuar el proceso.
471	panelContrasenas = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
472	labelContrasena = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
473	botonVerContrasena = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
474	campoContrasena = new javax.swing.JPasswordField();	Crea una nueva instancia de javax.swing.JPasswordField y la usa o guarda para continuar el proceso.
475	labelConfirmar = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
476	botonVerConfirmacion = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
477	campoConfirmar = new javax.swing.JPasswordField();	Crea una nueva instancia de javax.swing.JPasswordField y la usa o guarda para continuar el proceso.
478	labelError = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
479	botonDesactivar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
480	botonCancelar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
481	botonGuardar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.

482		Línea en blanco: separa visualmente bloques para que el código sea más legible.
483	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
484	setPreferredSize(new java.awt.Dimension(790, 710));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
485	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
486		Línea en blanco: separa visualmente bloques para que el código sea más legible.
487	panelTarjeta.setColorBorde(new java.awt.Color(222, 227, 234));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
488	panelTarjeta.setRadio(26);	Configura una propiedad mediante setRadio en el objeto indicado.
489	panelTarjeta.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
490		Línea en blanco: separa visualmente bloques para que el código sea más legible.
491	panelIcono.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
493	labelIconoAgregar.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/icono_agregarusuario.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
494	panelIcono.add(labelIconoAgregar, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 80, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
495		Línea en blanco: separa visualmente bloques para que el código sea más legible.
496	labelIconoEditar.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/icono_editarUsuario.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
497	labelIconoEditar.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
498	panelIcono.add(labelIconoEditar, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 80, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
499		Línea en blanco: separa visualmente bloques para que el código sea más legible.
500	panelTarjeta.add(panelIcono, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 35, 80, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
501		Línea en blanco: separa visualmente bloques para que el código sea más legible.
502	labelTitulo.setFont(new java.awt.Font("Dialog", 1, 25)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
503	labelTitulo.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
504	labelTitulo.setText("Agregar usuario");	Cambia el texto visible de un componente de la interfaz.
505	panelTarjeta.add(labelTitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(135, 35, 560, 35));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
506		Línea en blanco: separa visualmente bloques para que el código sea más legible.
507	labelSubtitulo.setText("Completa los datos del nuevo usuario");	Cambia el texto visible de un componente de la interfaz.
508	panelTarjeta.add(labelSubtitulo, new org.netbeans.lib.awtextra.AbsoluteConstraints(135, 68, 560, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
509		Línea en blanco: separa visualmente bloques para que el código sea más legible.
510	labelNombre.setText("Nombre");	Cambia el texto visible de un componente de la interfaz.
511	panelTarjeta.add(labelNombre, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 130, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
512	panelTarjeta.add(campoNombre, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 158, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
513		Línea en blanco: separa visualmente bloques para que el código sea más legible.
514	labelApellido.setText("Apellido");	Cambia el texto visible de un componente de la interfaz.
515	panelTarjeta.add(labelApellido, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 130, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
516	panelTarjeta.add(campoApellido, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 158, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
517		Línea en blanco: separa visualmente bloques para que el código sea más legible.
518	labelUsuario.setText("Usuario");	Cambia el texto visible de un componente de la interfaz.
519	panelTarjeta.add(labelUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 220, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
520	panelTarjeta.add(campoUsuario, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 248, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
521		Línea en blanco: separa visualmente bloques para que el código sea más legible.
522	labelCorreo.setText("Correo electrónico");	Cambia el texto visible de un componente de la interfaz.
523	panelTarjeta.add(labelCorreo, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 220, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
524	panelTarjeta.add(campoCorreo, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 248, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

525		Línea en blanco: separa visualmente bloques para que el código sea más legible.
526	labelRol.setText("Rol");	Cambia el texto visible de un componente de la interfaz.
527	panelTarjeta.add(labelRol, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 310, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
528		Línea en blanco: separa visualmente bloques para que el código sea más legible.
529	selectorRol.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
530	selectorRol.setText("Seleccionar rol");	Cambia el texto visible de un componente de la interfaz.
531	selectorRol.setTextoDesplegable("ADMINISTRADOR;CAJERO");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
532	panelTarjeta.add(selectorRol, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 338, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
533		Línea en blanco: separa visualmente bloques para que el código sea más legible.
534	labelEstado.setText("Estado");	Cambia el texto visible de un componente de la interfaz.
535	panelTarjeta.add(labelEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 310, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
536		Línea en blanco: separa visualmente bloques para que el código sea más legible.
537	selectorEstado.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
538	selectorEstado.setText("Activo");	Cambia el texto visible de un componente de la interfaz.
539	selectorEstado.setTextoDesplegable("Activo;Inactivo");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
540	panelTarjeta.add(selectorEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 338, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
541		Línea en blanco: separa visualmente bloques para que el código sea más legible.
542	labelTurno.setText("Turno del cajero");	Cambia el texto visible de un componente de la interfaz.
543	panelTarjeta.add(labelTurno, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 400, 330, 24));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
544		Línea en blanco: separa visualmente bloques para que el código sea más legible.
545	selectorTurno.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
546	selectorTurno.setText("Sin turno");	Cambia el texto visible de un componente de la interfaz.
547	selectorTurno.setTextoDesplegable("Mañana (06:00-14:00);Tarde (14:00-22:00);Sin turno");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
548	panelTarjeta.add(selectorTurno, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 428, 330, 46));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
549		Línea en blanco: separa visualmente bloques para que el código sea más legible.
550	cambiarContrasena.setText("Cambiar contraseña");	Cambia el texto visible de un componente de la interfaz.
551	cambiarContrasena.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
552	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
553	cambiarContrasenaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
554	}	Cierra el bloque abierto anteriormente.
555	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
556	panelTarjeta.add(cambiarContrasena, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 490, 250, 30))	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
557	;	Línea en blanco: separa visualmente bloques para que el código sea más legible.
558	panelContrasenas.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
559	panelContrasenas.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
560		Línea en blanco: separa visualmente bloques para que el código sea más legible.
561	labelContrasena.setText("Contraseña");	Cambia el texto visible de un componente de la interfaz.
562	panelContrasenas.add(labelContrasena, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 330, 22));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
563		Línea en blanco: separa visualmente bloques para que el código sea más legible.
564	botonVerContrasena.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
565	botonVerContrasena.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
566	botonVerContrasena.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
567	botonVerContrasena.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/visualizar.png"))	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
	); // NOI18N	

568	botonVerContrasena.setToolTipText("Mostrar contraseña");	Configura una propiedad mediante setToolTipText en el objeto indicado.
569	botonVerContrasena.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
570	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
571	botonVerContrasenaActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
572	}	Cierra el bloque abierto anteriormente.
573	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
574	panelContrasenas.add(botonVerContrasena, new org.netbeans.lib.awtextra.AbsoluteConstraints(285, 28, 40,	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
575	42));	
576	panelContrasenas.add(campoContrasena, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 26, 330, 46))	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
577	;	Línea en blanco: separa visualmente bloques para que el código sea más legible.
578	labelConfirmar.setText("Confirmar contraseña");	Cambia el texto visible de un componente de la interfaz.
579	panelContrasenas.add(labelConfirmar, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 0, 330, 22))	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
580	;	Línea en blanco: separa visualmente bloques para que el código sea más legible.
581	botonVerConfirmacion.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
582	botonVerConfirmacion.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
583	botonVerConfirmacion.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
584	botonVerConfirmacion.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/visualizar.png"	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
585	))); // NOI18N	
586	botonVerConfirmacion.setToolTipText("Mostrar contraseña");	Configura una propiedad mediante setToolTipText en el objeto indicado.
587	botonVerConfirmacion.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
588	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
589	botonVerConfirmacionActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
590	}	Cierra el bloque abierto anteriormente.
591	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
592	panelContrasenas.add(botonVerConfirmacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(665, 28, 40	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
593	, 42));	
594	panelContrasenas.add(campoConfirmar, new org.netbeans.lib.awtextra.AbsoluteConstraints(380, 26, 330, 46)	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
595	);	Línea en blanco: separa visualmente bloques para que el código sea más legible.
596	panelTarjeta.add(panelContrasenas, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 528, 710, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
597	labelError.setForeground(new java.awt.Color(231, 55, 65));	Línea en blanco: separa visualmente bloques para que el código sea más legible.
598	labelError.setText(" ");	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
599	panelTarjeta.add(labelError, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 612, 710, 26));	Cambia el texto visible de un componente de la interfaz.
600	;	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
601	botonDesactivar.setForeground(new java.awt.Color(231, 55, 65));	Línea en blanco: separa visualmente bloques para que el código sea más legible.
602	botonDesactivar.setText("Desactivar usuario");	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
603	botonDesactivar.setColorInicio(new java.awt.Color(255, 255, 255));	Cambia el texto visible de un componente de la interfaz.
604	botonDesactivar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
605	botonDesactivar.setColorBorde(new java.awt.Color(231, 55, 65));	Línea en blanco: separa visualmente bloques para que el código sea más legible.
606	botonDesactivar.setGrosorBorde(1.0F);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
607	botonDesactivar.setDegradado(false);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
608	botonDesactivar.addActionListener(new java.awt.event.ActionListener() {	Configura una propiedad mediante setDegradado en el objeto indicado.
609	public void actionPerformed(java.awt.event.ActionEvent evt) {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
610		Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

608	botonDesactivarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
609	}	Cierra el bloque abierto anteriormente.
610	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
611	panelTarjeta.add(botonDesactivar, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 650, 180, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
612		Línea en blanco: separa visualmente bloques para que el código sea más legible.
613	botonCancelar.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
614	botonCancelar.setText("Cancelar");	Cambia el texto visible de un componente de la interfaz.
615	botonCancelar.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
616	botonCancelar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
617	botonCancelar.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
618	botonCancelar.setGrosorBorde(1.0F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
619	botonCancelar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
620	botonCancelar.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
621	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
622	botonCancelarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
623	}	Cierra el bloque abierto anteriormente.
624	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
625	panelTarjeta.add(botonCancelar, new org.netbeans.lib.awtextra.AbsoluteConstraints(440, 650, 130, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
626		Línea en blanco: separa visualmente bloques para que el código sea más legible.
627	botonGuardar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
628	botonGuardar.setText("Guardar usuario");	Cambia el texto visible de un componente de la interfaz.
629	botonGuardar.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
630	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
631	botonGuardarActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
632	}	Cierra el bloque abierto anteriormente.
633	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
634	panelTarjeta.add(botonGuardar, new org.netbeans.lib.awtextra.AbsoluteConstraints(585, 650, 165, 48));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
635		Línea en blanco: separa visualmente bloques para que el código sea más legible.
636	add(panelTarjeta, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 790, 730));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
637		Línea en blanco: separa visualmente bloques para que el código sea más legible.
638	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
639		Línea en blanco: separa visualmente bloques para que el código sea más legible.
640	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
641	private Componentes.BotonRedondeado botonCancelar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
642	private Componentes.BotonRedondeado botonDesactivar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
643	private Componentes.BotonRedondeado botonGuardar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
644	private javax.swing.JButton botonVerConfirmacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
645	private javax.swing.JButton botonVerContrasena;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
646	private javax.swing.JCheckBox cambiarContrasena;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
647	private javax.swing.JTextField campoApellido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
648	private javax.swing.JPasswordField campoConfirmar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
649	private javax.swing.JPasswordField campoContrasena;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
650	private javax.swing.JTextField campoCorreo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
651	private javax.swing.JTextField campoNombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

652	<code>private javax.swing.JTextField campoUsuario;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
653	<code>private javax.swing.JLabel labelApellido;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
654	<code>private javax.swing.JLabel labelConfirmar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
655	<code>private javax.swing.JLabel labelContrasena;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
656	<code>private javax.swing.JLabel labelCorreo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
657	<code>private javax.swing.JLabel labelError;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
658	<code>private javax.swing.JLabel labelEstado;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
659	<code>private Labels.LabelEscalable labelIconoAgregar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
660	<code>private Labels.LabelEscalable labelIconoEditar;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
661	<code>private javax.swing.JLabel labelNombre;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
662	<code>private javax.swing.JLabel labelRol;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
663	<code>private javax.swing.JLabel labelSubtitulo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
664	<code>private javax.swing.JLabel labelTitulo;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
665	<code>private javax.swing.JLabel labelTurno;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
666	<code>private javax.swing.JLabel labelUsuario;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
667	<code>private javax.swing.JPanel panelContrasenas;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
668	<code>private Componentes.PanelCircular panelIcono;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
669	<code>private Componentes.PanelFlotante panelTarjeta;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
670	<code>private Componentes.BotonDesplegable selectorEstado;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
671	<code>private Componentes.BotonDesplegable selectorRol;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
672	<code>private Componentes.BotonDesplegable selectorTurno;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
673	<code>// End of variables declaration//GEN-END:variables</code>	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
674	<code>}</code>	Cierra el bloque abierto anteriormente.

16.87 src/GUI_ADMINISTRADOR/UsuariosPanel.form

Metadatos visuales de NetBeans para diseñar UsuariosPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 663 | Archivo 87 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,3,-44,0,0,6,44"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="javax.swing.JLabel" name="labelTitulo2">	Declara un componente o contenedor visual y su clase Java.
27	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
29		Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	</Property>	Cierra la sección XML abierta anteriormente.
31	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Color blue="9a" green="89" red="7f" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	<Property name="text" type="java.lang.String" value="Administra los usuarios del sistema"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
36	</Properties>	Cierra la sección XML abierta anteriormente.
37	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

38	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
39	<AbsoluteConstraints x="70" y="80" width="320" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
40	</Constraint>	Cierra la sección XML abierta anteriormente.
41	</Constraints>	Cierra la sección XML abierta anteriormente.
42	</Component>	Cierra la sección XML abierta anteriormente.
43	<Component class="javax.swing.JLabel" name="labelTitulo3">	Declara un componente o contenedor visual y su clase Java.
44	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
45	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
46		Metadato de NetBeans que conserva el estado editable del diseñador visual.
47	</Property>	Cierra la sección XML abierta anteriormente.
48	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
49	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
50	</Property>	Cierra la sección XML abierta anteriormente.
51	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	<Property name="text" type="java.lang.String" value="Gestión de Usuarios"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
53	</Properties>	Cierra la sección XML abierta anteriormente.
54	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
55	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
56	<AbsoluteConstraints x="70" y="30" width="540" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
57	</Constraint>	Cierra la sección XML abierta anteriormente.
58	</Constraints>	Cierra la sección XML abierta anteriormente.
59	</Component>	Cierra la sección XML abierta anteriormente.
60	<Component class="Componentes.BotonDerretido" name="botonDerretido1">	Declara un componente o contenedor visual y su clase Java.
61	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
62	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
63	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	</Property>	Cierra la sección XML abierta anteriormente.
65	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
66	<Image iconType="3" name="/Imagenes/icono_agregar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	</Property>	Cierra la sección XML abierta anteriormente.
68	<Property name="text" type="java.lang.String" value="Agregar usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
69	<Property name="iconTextGap" type="int" value="15"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
72	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
73	<AbsoluteConstraints x="1050" y="60" width="230" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
74	</Constraint>	Cierra la sección XML abierta anteriormente.
75	</Constraints>	Cierra la sección XML abierta anteriormente.
76	</Component>	Cierra la sección XML abierta anteriormente.
77	<Container class="Componentes.PanelFlotante" name="panelFlotante1">	Declara un componente o contenedor visual y su clase Java.
78	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
79	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

80	<AbsoluteConstraints x="60" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
81	</Constraint>	Cierra la sección XML abierta anteriormente.
82	</Constraints>	Cierra la sección XML abierta anteriormente.
83		Línea en blanco para organizar el XML del formulario.
84	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
85	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	</Layout>	Cierra la sección XML abierta anteriormente.
87	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
88	<Container class="Componentes.PanelCircular" name="panelCircular2">	Declara un componente o contenedor visual y su clase Java.
89	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
90	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
91	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
92	</Constraint>	Cierra la sección XML abierta anteriormente.
93	</Constraints>	Cierra la sección XML abierta anteriormente.
94		Línea en blanco para organizar el XML del formulario.
95	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
96	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
97	</Layout>	Cierra la sección XML abierta anteriormente.
98	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
99	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
100	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
101	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102	<Image iconType="3" name="/Imagenes/icono_usuariostotal.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	</Properties>	Cierra la sección XML abierta anteriormente.
105	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
106	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
107	<AbsoluteConstraints x="6" y="20" width="70" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
108	</Constraint>	Cierra la sección XML abierta anteriormente.
109	</Constraints>	Cierra la sección XML abierta anteriormente.
110	</Component>	Cierra la sección XML abierta anteriormente.
111	</SubComponents>	Cierra la sección XML abierta anteriormente.
112	</Container>	Cierra la sección XML abierta anteriormente.
113	<Component class="javax.swing.JLabel" name="labelTitulo4">	Declara un componente o contenedor visual y su clase Java.
114	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
115	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
116		Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	</Property>	Cierra la sección XML abierta anteriormente.
118	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
119	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
120	</Property>	Cierra la sección XML abierta anteriormente.
121	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

122	<Property name="text" type="java.lang.String" value="Total de usuarios"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
123	</Properties>	Cierra la sección XML abierta anteriormente.
124	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
125	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
126	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
127	</Constraint>	Cierra la sección XML abierta anteriormente.
128	</Constraints>	Cierra la sección XML abierta anteriormente.
129	</Component>	Cierra la sección XML abierta anteriormente.
130	<Component class="javax.swing.JLabel" name="labelTitulo5">	Declara un componente o contenedor visual y su clase Java.
131	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
132	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
133		Metadato de NetBeans que conserva el estado editable del diseñador visual.
134	</Property>	Cierra la sección XML abierta anteriormente.
135	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
136	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
137	</Property>	Cierra la sección XML abierta anteriormente.
138	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
139	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
140	</Properties>	Cierra la sección XML abierta anteriormente.
141	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
142	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
143	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
144	</Constraint>	Cierra la sección XML abierta anteriormente.
145	</Constraints>	Cierra la sección XML abierta anteriormente.
146	</Component>	Cierra la sección XML abierta anteriormente.
147	</SubComponents>	Cierra la sección XML abierta anteriormente.
148	</Container>	Cierra la sección XML abierta anteriormente.
149	<Container class="Componentes.PanelFlotante" name="panelFlotante2">	Declara un componente o contenedor visual y su clase Java.
150	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
151	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
152	<AbsoluteConstraints x="430" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
153	</Constraint>	Cierra la sección XML abierta anteriormente.
154	</Constraints>	Cierra la sección XML abierta anteriormente.
155		Línea en blanco para organizar el XML del formulario.
156	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
157	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
158	</Layout>	Cierra la sección XML abierta anteriormente.
159	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
160	<Container class="Componentes.PanelCircular" name="panelCircular3">	Declara un componente o contenedor visual y su clase Java.
161	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
162	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		

163	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
164	</Property>	Cierra la sección XML abierta anteriormente.
165	</Properties>	Cierra la sección XML abierta anteriormente.
166	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
167	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
168	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
169	</Constraint>	Cierra la sección XML abierta anteriormente.
170	</Constraints>	Cierra la sección XML abierta anteriormente.
171		Línea en blanco para organizar el XML del formulario.
172	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
173	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
174	</Layout>	Cierra la sección XML abierta anteriormente.
175	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
176	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
177	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
178	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
179	<Image iconType="3" name="/Imagenes/icono_admin.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
180	</Property>	Cierra la sección XML abierta anteriormente.
181	</Properties>	Cierra la sección XML abierta anteriormente.
182	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
183	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
184	<AbsoluteConstraints x="8" y="15" width="65" height="65"/>	Define la posición y el tamaño del componente dentro del diseño.
185	</Constraint>	Cierra la sección XML abierta anteriormente.
186	</Constraints>	Cierra la sección XML abierta anteriormente.
187	</Component>	Cierra la sección XML abierta anteriormente.
188	</SubComponents>	Cierra la sección XML abierta anteriormente.
189	</Container>	Cierra la sección XML abierta anteriormente.
190	<Component class="javax.swing.JLabel" name="labelTitulo7">	Declara un componente o contenedor visual y su clase Java.
191	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
192	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
193		Metadato de NetBeans que conserva el estado editable del diseñador visual.
194	</Property>	Cierra la sección XML abierta anteriormente.
195	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
196	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
197	</Property>	Cierra la sección XML abierta anteriormente.
198	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
199	<Property name="text" type="java.lang.String" value="Administradores"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
200	</Properties>	Cierra la sección XML abierta anteriormente.
201	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
202	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
203	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.

204	</Constraint>	Cierra la sección XML abierta anteriormente.
205	</Constraints>	Cierra la sección XML abierta anteriormente.
206	</Component>	Cierra la sección XML abierta anteriormente.
207	<Component class="javax.swing.JLabel" name="labelTitulo8">	Declara un componente o contenedor visual y su clase Java.
208	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
209	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
210		Metadato de NetBeans que conserva el estado editable del diseñador visual.
211	</Property>	Cierra la sección XML abierta anteriormente.
212	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
213	<Color blue="42" green="3d" red="c3" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
214	</Property>	Cierra la sección XML abierta anteriormente.
215	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
216	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
217	</Properties>	Cierra la sección XML abierta anteriormente.
218	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
219	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
220	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
221	</Constraint>	Cierra la sección XML abierta anteriormente.
222	</Constraints>	Cierra la sección XML abierta anteriormente.
223	</Component>	Cierra la sección XML abierta anteriormente.
224	</SubComponents>	Cierra la sección XML abierta anteriormente.
225	</Container>	Cierra la sección XML abierta anteriormente.
226	<Container class="Componentes.PanelFlotante" name="panelFlotante4">	Declara un componente o contenedor visual y su clase Java.
227	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
228	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
229	<AbsoluteConstraints x="800" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
230	</Constraint>	Cierra la sección XML abierta anteriormente.
231	</Constraints>	Cierra la sección XML abierta anteriormente.
232		Línea en blanco para organizar el XML del formulario.
233	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
234	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
235	</Layout>	Cierra la sección XML abierta anteriormente.
236	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
237	<Container class="Componentes.PanelCircular" name="panelCircular5">	Declara un componente o contenedor visual y su clase Java.
238	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
239	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
240	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
241	</Constraint>	Cierra la sección XML abierta anteriormente.
242	</Constraints>	Cierra la sección XML abierta anteriormente.
243		Línea en blanco para organizar el XML del formulario.
244	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
245	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

246	</Layout>	Cierra la sección XML abierta anteriormente.
247	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
248	<Component class="Labels.LabelEscalable" name="labelEscalable5">	Declara un componente o contenedor visual y su clase Java.
249	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
250	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
251	<Image iconType="3" name="/Imagenes/icono_usuario.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
252	</Property>	Cierra la sección XML abierta anteriormente.
253	</Properties>	Cierra la sección XML abierta anteriormente.
254	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
255	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
256	<AbsoluteConstraints x="3" y="9" width="75" height="75"/>	Define la posición y el tamaño del componente dentro del diseño.
257	</Constraint>	Cierra la sección XML abierta anteriormente.
258	</Constraints>	Cierra la sección XML abierta anteriormente.
259	</Component>	Cierra la sección XML abierta anteriormente.
260	</SubComponents>	Cierra la sección XML abierta anteriormente.
261	</Container>	Cierra la sección XML abierta anteriormente.
262	<Component class="javax.swing.JLabel" name="labelTitulo13">	Declara un componente o contenedor visual y su clase Java.
263	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
264	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
265		Metadato de NetBeans que conserva el estado editable del diseñador visual.
266	</Property>	Cierra la sección XML abierta anteriormente.
267	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
268	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
269	</Property>	Cierra la sección XML abierta anteriormente.
270	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
271	<Property name="text" type="java.lang.String" value="Cajeros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
272	</Properties>	Cierra la sección XML abierta anteriormente.
273	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
274	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
275	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
276	</Constraint>	Cierra la sección XML abierta anteriormente.
277	</Constraints>	Cierra la sección XML abierta anteriormente.
278	</Component>	Cierra la sección XML abierta anteriormente.
279	<Component class="javax.swing.JLabel" name="labelTitulo14">	Declara un componente o contenedor visual y su clase Java.
280	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
281	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
282		Metadato de NetBeans que conserva el estado editable del diseñador visual.
283	</Property>	Cierra la sección XML abierta anteriormente.
284	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
285	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
286	</Property>	Cierra la sección XML abierta anteriormente.

287	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
288	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
289	</Properties>	Cierra la sección XML abierta anteriormente.
290	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
291	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
292	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
293	</Constraint>	Cierra la sección XML abierta anteriormente.
294	</Constraints>	Cierra la sección XML abierta anteriormente.
295	</Component>	Cierra la sección XML abierta anteriormente.
296	</SubComponents>	Cierra la sección XML abierta anteriormente.
297	</Container>	Cierra la sección XML abierta anteriormente.
298	<Container class="Componentes.PanelFlotante" name="panelFlotante3">	Declara un componente o contenedor visual y su clase Java.
299	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
300	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
301	<AbsoluteConstraints x="1170" y="150" width="360" height="150"/>	Define la posición y el tamaño del componente dentro del diseño.
302	</Constraint>	Cierra la sección XML abierta anteriormente.
303	</Constraints>	Cierra la sección XML abierta anteriormente.
304		Línea en blanco para organizar el XML del formulario.
305	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
306	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
307	</Layout>	Cierra la sección XML abierta anteriormente.
308	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
309	<Container class="Componentes.PanelCircular" name="panelCircular4">	Declara un componente o contenedor visual y su clase Java.
310	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
311	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
312	<Color blue="e9" green="e9" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
313	</Property>	Cierra la sección XML abierta anteriormente.
314	</Properties>	Cierra la sección XML abierta anteriormente.
315	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
316	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
317	<AbsoluteConstraints x="40" y="30" width="80" height="90"/>	Define la posición y el tamaño del componente dentro del diseño.
318	</Constraint>	Cierra la sección XML abierta anteriormente.
319	</Constraints>	Cierra la sección XML abierta anteriormente.
320		Línea en blanco para organizar el XML del formulario.
321	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
322	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
323	</Layout>	Cierra la sección XML abierta anteriormente.
324	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
325	<Component class="Labels.LabelEscalable" name="labelEscalable4">	Declara un componente o contenedor visual y su clase Java.
326	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
327	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

328	<Image iconType="3" name="/Imagenes/icono_usuario_inactivo.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
329	</Property>	Cierra la sección XML abierta anteriormente.
330	</Properties>	Cierra la sección XML abierta anteriormente.
331	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
332	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
333	<AbsoluteConstraints x="5" y="12" width="70" height="70"/>	Define la posición y el tamaño del componente dentro del diseño.
334	</Constraint>	Cierra la sección XML abierta anteriormente.
335	</Constraints>	Cierra la sección XML abierta anteriormente.
336	</Component>	Cierra la sección XML abierta anteriormente.
337	</SubComponents>	Cierra la sección XML abierta anteriormente.
338	</Container>	Cierra la sección XML abierta anteriormente.
339	<Component class="javax.swing.JLabel" name="labelTitulo10">	Declara un componente o contenedor visual y su clase Java.
340	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
341	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
342		Metadato de NetBeans que conserva el estado editable del diseñador visual.
343	</Property>	Cierra la sección XML abierta anteriormente.
344	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
345	>	
345	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
346	</Property>	Cierra la sección XML abierta anteriormente.
347	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
348	<Property name="text" type="java.lang.String" value="Inactivos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
349	</Properties>	Cierra la sección XML abierta anteriormente.
350	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
351	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
352	<AbsoluteConstraints x="140" y="0" width="650" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
353	</Constraint>	Cierra la sección XML abierta anteriormente.
354	</Constraints>	Cierra la sección XML abierta anteriormente.
355	</Component>	Cierra la sección XML abierta anteriormente.
356	<Component class="javax.swing.JLabel" name="labelTitulo11">	Declara un componente o contenedor visual y su clase Java.
357	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
358	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
359		Metadato de NetBeans que conserva el estado editable del diseñador visual.
360	</Property>	Cierra la sección XML abierta anteriormente.
361	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
362	>	
362	<Color blue="17" green="11" red="d" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
363	</Property>	Cierra la sección XML abierta anteriormente.
364	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
365	<Property name="text" type="java.lang.String" value="48"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
366	</Properties>	Cierra la sección XML abierta anteriormente.
367	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
368	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

369	<AbsoluteConstraints x="140" y="40" width="90" height="76"/>	Define la posición y el tamaño del componente dentro del diseño.
370	</Constraint>	Cierra la sección XML abierta anteriormente.
371	</Constraints>	Cierra la sección XML abierta anteriormente.
372	</Component>	Cierra la sección XML abierta anteriormente.
373	</SubComponents>	Cierra la sección XML abierta anteriormente.
374	</Container>	Cierra la sección XML abierta anteriormente.
375	<Component class="Componentes.BotonRedondeado" name="botonExportar">	Declara un componente o contenedor visual y su clase Java.
376	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
377	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
378	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
379	</Property>	Cierra la sección XML abierta anteriormente.
380	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
381	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
382	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
383	</Property>	Cierra la sección XML abierta anteriormente.
384	<Property name="radio" type="int" value="20"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
385	<Property name="text" type="java.lang.String" value="Exportar"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
386	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
387	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
388	</Property>	Cierra la sección XML abierta anteriormente.
389	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
390	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
391	</Property>	Cierra la sección XML abierta anteriormente.
392	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
393		Metadato de NetBeans que conserva el estado editable del diseñador visual.
394	</Property>	Cierra la sección XML abierta anteriormente.
395	<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
396	<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
397	<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
398	</Property>	Cierra la sección XML abierta anteriormente.
399	</Properties>	Cierra la sección XML abierta anteriormente.
400	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
401	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
402	<AbsoluteConstraints x="1290" y="60" width="230" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
403	</Constraint>	Cierra la sección XML abierta anteriormente.
404	</Constraints>	Cierra la sección XML abierta anteriormente.
405	</Component>	Cierra la sección XML abierta anteriormente.
406	<Container class="Componentes.PanelFlotante" name="panelFiltros">	Declara un componente o contenedor visual y su clase Java.
407	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
408	<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
409	<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
410	</Properties>	Cierra la sección XML abierta anteriormente.
411	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

412	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
413	<AbsoluteConstraints x="60" y="320" width="1470" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
414	</Constraint>	Cierra la sección XML abierta anteriormente.
415	</Constraints>	Cierra la sección XML abierta anteriormente.
416		Línea en blanco para organizar el XML del formulario.
417	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
418	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
419	</Layout>	Cierra la sección XML abierta anteriormente.
420	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
421	<Component class="Componentes.CampoBusquedaAdmin" name="campoBusqueda">	Declara un componente o contenedor visual y su clase Java.
422	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
423	<Property name="placeholder" type="java.lang.String" value="Buscar usuario, nombre o correo..."/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
424	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
425		Metadato de NetBeans que conserva el estado editable del diseñador visual.
426	</Property>	Cierra la sección XML abierta anteriormente.
427	</Properties>	Cierra la sección XML abierta anteriormente.
428	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
429	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
430	<AbsoluteConstraints x="20" y="22" width="455" height="56"/>	Define la posición y el tamaño del componente dentro del diseño.
431	</Constraint>	Cierra la sección XML abierta anteriormente.
432	</Constraints>	Cierra la sección XML abierta anteriormente.
433	</Component>	Cierra la sección XML abierta anteriormente.
434	<Component class="Componentes.BotonDesplegable" name="filtroRol">	Declara un componente o contenedor visual y su clase Java.
435	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
436	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
437	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
438	</Property>	Cierra la sección XML abierta anteriormente.
439	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
440	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
441	</Property>	Cierra la sección XML abierta anteriormente.
442	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
443	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
444	</Property>	Cierra la sección XML abierta anteriormente.
445	<Property name="text" type="java.lang.String" value="Todos los roles"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
446	<Property name="textoDesplegable" type="java.lang.String" value="Todos los roles;ADMINISTRADOR;CAJERO"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
447	</Properties>	Cierra la sección XML abierta anteriormente.
448	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
449	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="filtroRolActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
450	</Events>	Cierra la sección XML abierta anteriormente.
451	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.

452	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
453	<AbsoluteConstraints x="495" y="24" width="330" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
454	</Constraint>	Cierra la sección XML abierta anteriormente.
455	</Constraints>	Cierra la sección XML abierta anteriormente.
456	</Component>	Cierra la sección XML abierta anteriormente.
457	<Component class="Componentes.BotonDesplegable" name="filtroEstado">	Declara un componente o contenedor visual y su clase Java.
458	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
459	<Property name="colorFondo" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
460	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
461	</Property>	Cierra la sección XML abierta anteriormente.
462	<Property name="colorHover" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
463	<Color blue="fb" green="f9" red="f8" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
464	</Property>	Cierra la sección XML abierta anteriormente.
465	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
466	<Color blue="2b" green="14" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
467	</Property>	Cierra la sección XML abierta anteriormente.
468	<Property name="text" type="java.lang.String" value="Todos los estados"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
469	<Property name="textoDesplegable" type="java.lang.String" value="Todos los estados;Activo;Inactivo"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
470	</Properties>	Cierra la sección XML abierta anteriormente.
471	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
472	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
473	<AbsoluteConstraints x="850" y="24" width="330" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
474	</Constraint>	Cierra la sección XML abierta anteriormente.
475	</Constraints>	Cierra la sección XML abierta anteriormente.
476	</Component>	Cierra la sección XML abierta anteriormente.
477	<Component class="Componentes.BotonRedondeado" name="botonLimpiarFiltros">	Declara un componente o contenedor visual y su clase Java.
478	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
479	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
">		
480	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
481	</Property>	Cierra la sección XML abierta anteriormente.
482	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
483	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
484	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
485	</Property>	Cierra la sección XML abierta anteriormente.
486	<Property name="text" type="java.lang.String" value="Limpiar filtros"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
487	<Property name="colorBorde" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
488	<Color blue="41" green="37" red="e7" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
489	</Property>	Cierra la sección XML abierta anteriormente.

490	>	<Property name="colorFinal" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
491		<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
492		</Property>	Cierra la sección XML abierta anteriormente.
493		<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
494			Metadato de NetBeans que conserva el estado editable del diseñador visual.
495		</Property>	Cierra la sección XML abierta anteriormente.
496		<Property name="grosorBorde" type="float" value="1.6"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
497		<Property name="margin" type="java.awt.Insets" editor="org.netbeans.beaninfo.editors.InsetsEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
498		<Insets value="[5, 14, 3, 14]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
499		</Property>	Cierra la sección XML abierta anteriormente.
500		</Properties>	Cierra la sección XML abierta anteriormente.
501		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
502		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
503		<AbsoluteConstraints x="1210" y="24" width="230" height="52"/>	Define la posición y el tamaño del componente dentro del diseño.
504		</Constraint>	Cierra la sección XML abierta anteriormente.
505		</Constraints>	Cierra la sección XML abierta anteriormente.
506		</Component>	Cierra la sección XML abierta anteriormente.
507		</SubComponents>	Cierra la sección XML abierta anteriormente.
508		</Container>	Cierra la sección XML abierta anteriormente.
509		<Container class="Componentes.PanelFlotante" name="panelTabla">	Declara un componente o contenedor visual y su clase Java.
510		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
511		<Property name="radio" type="int" value="18"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
512		<Property name="sombra" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
513		</Properties>	Cierra la sección XML abierta anteriormente.
514		<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
515		<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
516		<AbsoluteConstraints x="60" y="420" width="1470" height="510"/>	Define la posición y el tamaño del componente dentro del diseño.
517		</Constraint>	Cierra la sección XML abierta anteriormente.
518		</Constraints>	Cierra la sección XML abierta anteriormente.
519			Línea en blanco para organizar el XML del formulario.
520		<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
521		<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
522		</Layout>	Cierra la sección XML abierta anteriormente.
523		<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
524		<Container class="javax.swing.JScrollPane" name="scrollUsuarios">	Declara un componente o contenedor visual y su clase Java.
525		<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
526		<Property name="border" type="javax.swing.border.Border" editor="org.netbeans.modules.form.editors2.BorderEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
527		<Border info="null"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
528		</Property>	Cierra la sección XML abierta anteriormente.
529		<Property name="verticalScrollBarPolicy" type="int" value="21"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
530		<Property name="horizontalScrollBarPolicy" type="int" value="31"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
531		<Property name="autoscrolls" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

532	</Properties>	Cierra la sección XML abierta anteriormente.
533	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
534	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
535	<AbsoluteConstraints x="14" y="14" width="1442" height="428"/>	Define la posición y el tamaño del componente dentro del diseño.
536	</Constraint>	Cierra la sección XML abierta anteriormente.
537	</Constraints>	Cierra la sección XML abierta anteriormente.
538		Línea en blanco para organizar el XML del formulario.
539	<Layout class="org.netbeans.modules.form.compat2.layouts.support.JScrollPaneSupportLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
540	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
541	<Component class="Componentes.TablaAdministrativa" name="tablaUsuarios">	Declara un componente o contenedor visual y su clase Java.
542	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
543	<Property name="model" type="javax.swing.table.TableModel" editor="org.netbeans.modules.form.editors2.TableModelEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
544	<Table columnCount="8" rowCount="10">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
545	<Column editable="false" title="ID" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
546	<Column editable="false" title="Nombre" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
547	<Column editable="false" title="Usuario" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
548	<Column editable="false" title="Rol" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
549	<Column editable="false" title="Correo" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
550	<Column editable="false" title="Estado" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
551	<Column editable="false" title="Fecha creación" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
552	<Column editable="false" title="Acciones" type="java.lang.Object"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
553	</Table>	Cierra la sección XML abierta anteriormente.
554	</Property>	Cierra la sección XML abierta anteriormente.
555	<Property name="autoscrolls" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
556	<Property name="columnasCentradas" type="java.lang.String" value="5,7"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
557	</Properties>	Cierra la sección XML abierta anteriormente.
558	</Component>	Cierra la sección XML abierta anteriormente.
559	</SubComponents>	Cierra la sección XML abierta anteriormente.
560	</Container>	Cierra la sección XML abierta anteriormente.
561	<Component class="javax.swing.JLabel" name="etiquetaRango">	Declara un componente o contenedor visual y su clase Java.
562	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
563	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
564	<Color blue="7c" green="67" red="5c" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
565	</Property>	Cierra la sección XML abierta anteriormente.
566	<Property name="text" type="java.lang.String" value="Mostrando 1–10 de 10 usuarios"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
567	</Properties>	Cierra la sección XML abierta anteriormente.
568	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
569	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
570	<AbsoluteConstraints x="28" y="452" width="330" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
571	</Constraint>	Cierra la sección XML abierta anteriormente.
572	</Constraints>	Cierra la sección XML abierta anteriormente.
573	</Component>	Cierra la sección XML abierta anteriormente.

574	<Container class="javax.swing.JPanel" name="panelPaginacion">	Declara un componente o contenedor visual y su clase Java.
575	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
576	<Property name="opaque" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
577	</Properties>	Cierra la sección XML abierta anteriormente.
578	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
579	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
580	<AbsoluteConstraints x="1165" y="452" width="270" height="42"/>	Define la posición y el tamaño del componente dentro del diseño.
581	</Constraint>	Cierra la sección XML abierta anteriormente.
582	</Constraints>	Cierra la sección XML abierta anteriormente.
583		Línea en blanco para organizar el XML del formulario.
584	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
585	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
586	</Layout>	Cierra la sección XML abierta anteriormente.
587	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
588	<Component class="Componentes.BotonRedondeado" name="botonAnterior">	Declara un componente o contenedor visual y su clase Java.
589	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
590	<Property name="text" type="java.lang.String" value="#x2039;"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
591	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
592	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
593	</Property>	Cierra la sección XML abierta anteriormente.
594	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
595	</Properties>	Cierra la sección XML abierta anteriormente.
596	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
597	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
598	<AbsoluteConstraints x="0" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
599	</Constraint>	Cierra la sección XML abierta anteriormente.
600	</Constraints>	Cierra la sección XML abierta anteriormente.
601	</Component>	Cierra la sección XML abierta anteriormente.
602	<Component class="Componentes.BotonRedondeado" name="botonPagina1">	Declara un componente o contenedor visual y su clase Java.
603	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
604	<Property name="text" type="java.lang.String" value="1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
605	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
606	<Color blue="0" green="bc" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
607	</Property>	Cierra la sección XML abierta anteriormente.
608	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
609	</Properties>	Cierra la sección XML abierta anteriormente.
610	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
611	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
612	<AbsoluteConstraints x="50" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
613	</Constraint>	Cierra la sección XML abierta anteriormente.
614	</Constraints>	Cierra la sección XML abierta anteriormente.

615	</Component>	Cierra la sección XML abierta anteriormente.
616	<Component class="Componentes.BotonRedondeado" name="botonPagina2">	Declara un componente o contenedor visual y su clase Java.
617	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
618	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
619	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
620	</Property>	Cierra la sección XML abierta anteriormente.
621	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
622	<Property name="text" type="java.lang.String" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
623	</Properties>	Cierra la sección XML abierta anteriormente.
624	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
625	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
626	<AbsoluteConstraints x="100" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
627	</Constraint>	Cierra la sección XML abierta anteriormente.
628	</Constraints>	Cierra la sección XML abierta anteriormente.
629	</Component>	Cierra la sección XML abierta anteriormente.
630	<Component class="Componentes.BotonRedondeado" name="botonPagina3">	Declara un componente o contenedor visual y su clase Java.
631	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
632	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
633	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
634	</Property>	Cierra la sección XML abierta anteriormente.
635	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
636	<Property name="text" type="java.lang.String" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
637	</Properties>	Cierra la sección XML abierta anteriormente.
638	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
639	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
640	<AbsoluteConstraints x="150" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.
641	</Constraint>	Cierra la sección XML abierta anteriormente.
642	</Constraints>	Cierra la sección XML abierta anteriormente.
643	</Component>	Cierra la sección XML abierta anteriormente.
644	<Component class="Componentes.BotonRedondeado" name="botonSiguiente">	Declara un componente o contenedor visual y su clase Java.
645	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
646	<Property name="colorInicio" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
647	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
648	</Property>	Cierra la sección XML abierta anteriormente.
649	<Property name="degradado" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
650	<Property name="text" type="java.lang.String" value="›/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
651	</Properties>	Cierra la sección XML abierta anteriormente.
652	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
653	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
654	<AbsoluteConstraints x="200" y="2" width="40" height="38"/>	Define la posición y el tamaño del componente dentro del diseño.

655	</Constraint>	Cierra la sección XML abierta anteriormente.
656	</Constraints>	Cierra la sección XML abierta anteriormente.
657	</Component>	Cierra la sección XML abierta anteriormente.
658	</SubComponents>	Cierra la sección XML abierta anteriormente.
659	</Container>	Cierra la sección XML abierta anteriormente.
660	</SubComponents>	Cierra la sección XML abierta anteriormente.
661	</Container>	Cierra la sección XML abierta anteriormente.
662	</SubComponents>	Cierra la sección XML abierta anteriormente.
663	</Form>	Cierra la sección XML abierta anteriormente.

16.88 src/GUI_ADMINISTRADOR/UsuariosPanel.java

Gestión paginada de usuarios con búsqueda, filtros, resumen, alta, edición y activación.

Tipo: java | Líneas: 1152 | Archivo 88 de 157

Clases/tipos detectados: UsuariosPanel, ModeloTablaUsuarios, RenderEstado, RenderAcciones, EditorAcciones, FondoOscuro

Línea	Constructor o método detectado
87	public UsuariosPanel() {
107	private void configurarElementosExistentes() {
128	private void configurarFiltrosVisuales() {
165	private void configurarTabla() {
210	private void configurarAnchosColumnas() {
218	private void configurarEventos() {
222	public void insertUpdate(DocumentEvent evento) {
227	public void removeUpdate(DocumentEvent evento) {
232	public void changedUpdate(DocumentEvent evento) {
238	private void configurarFiltro(
257	private void cargarPagina() {
269	protected PaginaUsuarios dolnBackground() throws Exception {
276	protected void done() {
307	private void cargarResumen() {
310	protected ResumenUsuarios dolnBackground() throws Exception {
315	protected void done() {
335	private void actualizarPaginacion(int totalRegistros) {
371	private void cambiarPagina(int pagina) {
379	private void limpiarFiltros() {
390	private void abrirFormulario(Usuario usuario) {
427	private void desactivarUsuario(Usuario usuario) {
474	private void exportarCsv() {
499	protected Integer dolnBackground() throws Exception {
507	protected void done() {
531	private void escribirCsv(Path archivo, List<Usuario> usuarios)
557	private String csv(String valor) {
561	private void mostrarErrorDatos(Throwable causa) {
576	private void configurarBotonPagina(
595	public void removeNotify() {

Línea	Constructor o método detectado
607	private void initComponents() {
790	public void actionPerformed(java.awt.event.ActionEvent evt) {
846	public boolean isCellEditable(int rowIndex, int columnIndex) {
893	private void filtroRolActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_filtroRolActionPerformed
906	private void setUsuarios(List<Usuario> nuevosUsuarios) {
912	public int getRowCount() {
917	public int getColumnCount() {
922	public String getColumnName(int columna) {
927	public Object getValueAt(int fila, int columna) {
944	public Class<?> getColumnClass(int columna) {
950	public boolean isCellEditable(int fila, int columna) {
959	public Component getTableCellRendererComponent(
991	public Component getTableCellRendererComponent(
1021	private final BotonRedondeado editar = crearBotonAccion(
1023	private final BotonRedondeado eliminar = crearBotonAccion(
1027	private EditorAcciones() {
1035	public Component getTableCellEditorComponent(
1048	public Object getCellEditorValue() {
1052	private void editar(ActionEvent evento) {
1058	private void eliminar(ActionEvent evento) {
1065	private JPanel crearPanelAcciones(Color fondo) {
1072	private BotonRedondeado crearBotonAccion(
1093	private FondoOscuro() {
1101	protected void paintComponent(Graphics graphics) {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JPanel.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package GUI_ADMINISTRADOR;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	import CRUD.UsuarioCRUD;	Importa CRUD.UsuarioCRUD para poder usar esa clase sin escribir su nombre completo.
8	import Componentes.BotonDesplegable;	Importa Componentes.BotonDesplegable para poder usar esa clase sin escribir su nombre completo.
9	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
10	import Modelos.PaginaUsuarios;	Importa Modelos.PaginaUsuarios para poder usar esa clase sin escribir su nombre completo.

11	import Modelos.ResumenUsuarios;	Importa Modelos.ResumenUsuarios para poder usar esa clase sin escribir su nombre completo.
12	import Modelos.Usuario;	Importa Modelos.Usuario para poder usar esa clase sin escribir su nombre completo.
13	import Utilidades.IconosUsuarios;	Importa Utilidades.IconosUsuarios para poder usar esa clase sin escribir su nombre completo.
14	import Utilidades.SesionUsuario;	Importa Utilidades.SesionUsuario para poder usar esa clase sin escribir su nombre completo.
15	import Utilidades.TemaAdmin;	Importa Utilidades.TemaAdmin para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
18	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
19	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
20	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
21	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
22	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
23	import java.awt.Window;	Importa java.awt.Window para poder usar esa clase sin escribir su nombre completo.
24	import java.awt.event.ActionEvent;	Importa java.awt.event.ActionEvent para poder usar esa clase sin escribir su nombre completo.
25	import java.io.BufferedWriter;	Importa java.io.BufferedWriter para poder usar esa clase sin escribir su nombre completo.
26	import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
27	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
28	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
29	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
30	import java.time.format.DateTimeFormatter;	Importa java.time.format.DateTimeFormatter para poder usar esa clase sin escribir su nombre completo.
31	import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
32	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
33	import java.util.concurrent.ExecutionException;	Importa java.util.concurrent.ExecutionException para poder usar esa clase sin escribir su nombre completo.
34	import javax.swing.AbstractCellEditor;	Importa javax.swing.AbstractCellEditor para poder usar esa clase sin escribir su nombre completo.
35	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
36	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
37	import javax.swing.JFileChooser;	Importa javax.swing.JFileChooser para poder usar esa clase sin escribir su nombre completo.
38	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
39	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
40	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
41	import javax.swing.JRootPane;	Importa javax.swing.JRootPane para poder usar esa clase sin escribir su nombre completo.
42	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
43	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
44	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
45	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
46	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
47	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
48	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
49	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
50	import javax.swing.filechooser.FileNameExtensionFilter;	Importa javax.swing.filechooser.FileNameExtensionFilter para poder usar esa clase sin escribir su nombre completo.
51	import javax.swing.table.AbstractTableModel;	Importa javax.swing.table.AbstractTableModel para poder usar esa clase sin escribir su nombre completo.
52	import javax.swing.table.TableCellEditor;	Importa javax.swing.table.TableCellEditor para poder usar esa clase sin escribir su nombre completo.
53	import javax.swing.table.TableCellRenderer;	Importa javax.swing.table.TableCellRenderer para poder usar esa clase sin escribir su nombre completo.
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.

55	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
56	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
57	* @author Humberto Alvarado	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
58	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
59	public class UsuariosPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	private static final int USUARIOS_POR_PAGINA = 10;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
62	private static final Color AZUL = new Color(0, 20, 43);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
63	private static final Color SECUNDARIO = new Color(92, 103, 124);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
64	private static final Color BORDE = new Color(225, 229, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
65	private static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
66	private static final Color ROJO = new Color(231, 55, 65);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
67	private static final DateFormatter FORMATO_FECHA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	= DateFormatter.ofPattern("dd/MM/yyyy HH:mm");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	private final TemaAdmin tema = new TemaAdmin();	Crea una nueva instancia de TemaAdmin y la usa o guarda para continuar el proceso.
71	private final UsuarioCRUD crud = new UsuarioCRUD();	Crea una nueva instancia de UsuarioCRUD y la usa o guarda para continuar el proceso.
72	private final ModeloTablaUsuarios modeloTabla = new ModeloTablaUsuarios();	Crea una nueva instancia de ModeloTablaUsuarios y la usa o guarda para continuar el proceso.
73	private final Timer temporizadorBusqueda;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	private final List<BotonRedondeado> botonesPagina = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	private String rolSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
78	private Boolean estadoSeleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	private int paginaActual = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
80	private int totalPaginas = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
81	private int secuenciaCarga;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
82	private boolean mostrandoErrorConexion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
85	* Creates new form UsuariosPanel	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
86	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
87	public UsuariosPanel() {	Constructor de UsuariosPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
88	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	// Agrupa pulsaciones rápidas: la consulta se ejecuta 300 ms después	Comentario para humanos; Java no ejecuta este texto. Agrupa pulsaciones rápidas: la consulta se ejecuta 300 ms después
91	// de la última tecla, no una vez por cada carácter.	Comentario para humanos; Java no ejecuta este texto. de la última tecla, no una vez por cada carácter.
92	temporizadorBusqueda = new Timer(300, evento -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
93	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
94	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	temporizadorBusqueda.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	configurarElementosExistentes();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

99	configurarFiltrosVisuales();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	configurarTabla();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	configurarEventos();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	}	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	/** Corrige las tarjetas que estaban copiadas del panel de ingredientes. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
107	private void configurarElementosExistentes() {	Declara el método configurarElementosExistentes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
108	setBackground(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
109	labelTitulo2.setFont(tema.regular(17f));	Configura una propiedad mediante setFont en el objeto indicado.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	botonDerretido1.setFont(tema.negrita(16f));	Configura una propiedad mediante setFont en el objeto indicado.
112	botonDerretido1.addActionListener(evento -> abrirFormulario(null));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	panelCircular2.setColorFondo(new Color(255, 244, 211));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
115	panelCircular3.setColorFondo(new Color(252, 233, 233));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
116	panelCircular5.setColorFondo(new Color(255, 244, 211));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
117	panelCircular4.setColorFondo(new Color(252, 233, 233));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	botonExportar.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
120	botonExportar.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
121	IconosUsuarios.Tipo.EXPORTAR, ROJO, 22));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	botonExportar.setIconTextGap(12);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
123	botonExportar.addActionListener(evento -> exportarCsv());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
124	}	Cierra el bloque abierto anteriormente.
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	/** Configura los controles que ya existen visualmente en el .form. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
128	private void configurarFiltrosVisuales() {	Declara el método configurarFiltrosVisuales. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
129	panelFiltros.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
130	panelFiltros.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
131	campoBusqueda.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	configurarFiltro(filtroRol, 270);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	filtroRol.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	String opcion = evento.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
136	rolSeleccionado = "Todos los roles".equals(opcion)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
137	? null : opcion;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	filtroRol.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
139	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
140	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.

142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143	configurarFiltro(filtroEstado, 270);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	filtroEstado.addMenuOpcionListener(evento -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	String opcion = evento.getActionCommand();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
146	estadoSeleccionado = switch (opcion) {	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
147	case "Activo" -> Boolean.TRUE;	Define una de las rutas posibles dentro del switch.
148	case "Inactivo" -> Boolean.FALSE;	Define una de las rutas posibles dentro del switch.
149	default -> null;	Define una de las rutas posibles dentro del switch.
150	};	Cierra el bloque abierto anteriormente y termina la declaración.
151	filtroEstado.setText(opcion);	Cambia el texto visible de un componente de la interfaz.
152	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
153	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155		Línea en blanco: separa visualmente bloques para que el código sea más legible.
156	botonLimpiarFiltros.setFont(tema.negrita(14f));	Configura una propiedad mediante setFont en el objeto indicado.
157	botonLimpiarFiltros.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
158	IconosUsuarios.Tipo.FILTRO, ROJO, 21));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	botonLimpiarFiltros.setIconTextGap(10);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
160	botonLimpiarFiltros.addActionListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
161	evento -> limpiarFiltros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	/** Aplica comportamiento a la JTable dibujada en UsuariosPanel.form. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
165	private void configurarTabla() {	Declara el método configurarTabla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
166	panelTabla.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
167	panelTabla.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
168	tablaUsuarios.setModel(modeloTabla);	Configura una propiedad mediante setModel en el objeto indicado.
169	tablaUsuarios.aplicarEstilo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	tablaUsuarios.getColumnModel().getColumn(6)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	.setCellRenderer(new RenderEstado());	Crea una nueva instancia de RenderEstado y la usa o guarda para continuar el proceso.
172	tablaUsuarios.getColumnModel().getColumn(8)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	.setCellRenderer(new RenderAcciones());	Crea una nueva instancia de RenderAcciones y la usa o guarda para continuar el proceso.
174	tablaUsuarios.getColumnModel().getColumn(8)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175	.setCellEditor(new EditorAcciones());	Crea una nueva instancia de EditorAcciones y la usa o guarda para continuar el proceso.
176	configurarAnchosColumnas();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	scrollUsuarios.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
179	scrollUsuarios.getViewPort().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
180	etiquetaRango.setFont(tema.regular(12f));	Configura una propiedad mediante setFont en el objeto indicado.
181	etiquetaRango.setForeground(SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.
183	configurarBotonPagina(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184	botonAnterior, IconosUsuarios.Tipo.ANTERIOR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	botonAnterior.addActionListener(evento -> cambiarPagina(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.

186	paginaActual - 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	botonesPagina.add(botonPagina1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189	botonesPagina.add(botonPagina2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	botonesPagina.add(botonPagina3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
191	for (int indice = 0; indice < botonesPagina.size(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
192	BotonRedondeado boton = botonesPagina.get(indice);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
193	configurarBotonPagina(boton, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
194	final int posicion = indice;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
195	boton.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
196	Object valor = botonesPagina.get(posicion)	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	.getClientProperty("pagina");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
198	if (valor instanceof Integer) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
199	cambiarPagina(pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
200	}	Cierra el bloque abierto anteriormente.
201	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202	}	Cierra el bloque abierto anteriormente.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	configurarBotonPagina(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	botonSiguiente, IconosUsuarios.Tipo.SIGUIENTE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	botonSiguiente.addActionListener(evento -> cambiarPagina(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
207	paginaActual + 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
208	}	Cierra el bloque abierto anteriormente.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	private void configurarAnchosColumnas() {	Declara el método configurarAnchosColumnas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
211	int[] anchos = {65, 155, 115, 125, 185, 230, 95, 165, 110};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
212	for (int indice = 0; indice < anchos.length; indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
213	tablaUsuarios.getColumnModel().getColumn(indice)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	.setPreferredWidth(anchos[indice]);	Configura una propiedad mediante setPreferredWidth en el objeto indicado.
215	}	Cierra el bloque abierto anteriormente.
216	}	Cierra el bloque abierto anteriormente.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	private void configurarEventos() {	Declara el método configurarEventos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
219	campoBusqueda.getDocument()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
220	.addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
221	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
222	public void insertUpdate(DocumentEvent evento) {	Declara el método insertUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
223	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
224	}	Cierra el bloque abierto anteriormente.
225		Línea en blanco: separa visualmente bloques para que el código sea más legible.
226	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
227	public void removeUpdate(DocumentEvent evento) {	Declara el método removeUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
228	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

229	}	Cierra el bloque abierto anteriormente.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
232	public void changedUpdate(DocumentEvent evento) {	Declara el método changedUpdate. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
233	temporizadorBusqueda.restart();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
234	}	Cierra el bloque abierto anteriormente.
235	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	}	Cierra el bloque abierto anteriormente.
237		Línea en blanco: separa visualmente bloques para que el código sea más legible.
238	private void configurarFiltro(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
239	BotonDesplegable boton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240	int ancho) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
241	boton.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
242	boton.setFont(tema.regular(14f));	Configura una propiedad mediante setFont en el objeto indicado.
243	boton.setColorFondo(Color.WHITE);	Configura una propiedad mediante setColorFondo en el objeto indicado.
244	boton.setColorHover(new Color(248, 249, 251));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
245	boton.setColorDesplegado(new Color(255, 247, 222));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
246	boton.setColorTextoOpcion(AZUL);	Configura una propiedad mediante setColorTextoOpcion en el objeto indicado.
247	boton.setColorBordeMenu(BORDE);	Configura una propiedad mediante setColorBordeMenu en el objeto indicado.
248	boton.setAnchoMenu(ancho);	Configura una propiedad mediante setAnchoMenu en el objeto indicado.
249	boton.setAltoOpcion(42);	Configura una propiedad mediante setAltoOpcion en el objeto indicado.
250	boton.setBorderPainted(true);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
251	boton.setBorder(BorderFactory.createCompoundBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
252	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	new EmptyBorder(0, 14, 0, 34)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
254	}	Cierra el bloque abierto anteriormente.
255		Línea en blanco: separa visualmente bloques para que el código sea más legible.
256	/** Ejecuta la consulta fuera del hilo gráfico para no congelar Swing. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
257	private void cargarPagina() {	Declara el método cargarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
258	final int solicitud = ++secuenciaCarga;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
259	final String busqueda = campoBusqueda.getText();	Declara una constante o referencia que no puede reasignarse después de inicializarse.
260	final String rol = rolSeleccionado;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
261	final Boolean estado = estadoSeleccionado;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
262	final int paginaSolicitada = paginaActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
263		Línea en blanco: separa visualmente bloques para que el código sea más legible.
264	tablaUsuarios.setEnabled(false);	Habilita o deshabilita la interacción con el componente.
265	etiquetaRango.setText("Cargando usuarios...");	Cambia el texto visible de un componente de la interfaz.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267	new SwingWorker<PaginaUsuarios, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
268	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
269	protected PaginaUsuarios doInBackground() throws Exception {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
270	return crud.listarPagina(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
271	busqueda, rol, estado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272	paginaSolicitada, USUARIOS_POR_PAGINA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

273	}	Cierra el bloque abierto anteriormente.
274		Línea en blanco: separa visualmente bloques para que el código sea más legible.
275	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
276	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
277	if (solicitud != secuenciaCarga) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
278	return; // Ignora respuestas anteriores a un filtro nuevo.	Finaliza el método y devuelve el valor indicado a quien lo llamó.
279	}	Cierra el bloque abierto anteriormente.
280	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
281	PaginaUsuarios resultado = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
282	totalPaginas = Math.max(1, (int) Math.ceil(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
283	resultado.totalRegistros()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
284	/ (double) USUARIOS_POR_PAGINA));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
285	if (paginaActual > totalPaginas) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
286	paginaActual = totalPaginas;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
287	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
288	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
289	}	Cierra el bloque abierto anteriormente.
290		Línea en blanco: separa visualmente bloques para que el código sea más legible.
291	modeloTabla.setUsuarios(resultado.usuarios());	Configura una propiedad mediante setUsuarios en el objeto indicado.
292	actualizarPaginacion(resultado.totalRegistros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
293	mostrandoErrorConexion = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
294	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
295	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
296	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
297	mostrarErrorDatos(ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	modeloTabla.setUsuarios(List.of());	Configura una propiedad mediante setUsuarios en el objeto indicado.
299	actualizarPaginacion(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
300	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
301	tablaUsuarios.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
302	}	Cierra el bloque abierto anteriormente.
303	}	Cierra el bloque abierto anteriormente.
304	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
305	}	Cierra el bloque abierto anteriormente.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	private void cargarResumen() {	Declara el método cargarResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
308	new SwingWorker<ResumenUsuarios, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
309	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
310	protected ResumenUsuarios doInBackground() throws Exception {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
311	return crud.obtenerResumen();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
312	}	Cierra el bloque abierto anteriormente.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
315	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
316	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

317	ResumenUsuarios resumen = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
318	labelTitulo5.setText(String.valueOf(resumen.total()));	Cambia el texto visible de un componente de la interfaz.
319	labelTitulo8.setText(String.valueOf(	Cambia el texto visible de un componente de la interfaz.
320	resumen.administradoresActivos()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321	labelTitulo14.setText(String.valueOf(	Cambia el texto visible de un componente de la interfaz.
322	resumen.cajerosActivos()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
323	labelTitulo11.setText(String.valueOf(	Cambia el texto visible de un componente de la interfaz.
324	resumen.inactivos()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
325	mostrandoErrorConexion = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
326	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
327	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
328	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
329	mostrarErrorDatos(ex.getCause());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
330	}	Cierra el bloque abierto anteriormente.
331	}	Cierra el bloque abierto anteriormente.
332	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
333	}	Cierra el bloque abierto anteriormente.
334		Línea en blanco: separa visualmente bloques para que el código sea más legible.
335	private void actualizarPaginacion(int totalRegistros) {	Declara el método actualizarPaginacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
336	if (totalRegistros == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
337	etiquetaRango.setText("No se encontraron usuarios");	Cambia el texto visible de un componente de la interfaz.
338	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
339	int primero = (paginaActual - 1) * USUARIOS_POR_PAGINA + 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
340	int ultimo = Math.min(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
341	primero + USUARIOS_POR_PAGINA - 1, totalRegistros);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
342	etiquetaRango.setText(String.format(	Cambia el texto visible de un componente de la interfaz.
343	"Mostrando %d-%d de %d usuarios",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
344	primero, ultimo, totalRegistros));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
345	}	Cierra el bloque abierto anteriormente.
346		Línea en blanco: separa visualmente bloques para que el código sea más legible.
347	botonAnterior.setEnabled(paginaActual > 1);	Habilita o deshabilita la interacción con el componente.
348	botonSiguiente.setEnabled(paginaActual < totalPaginas);	Habilita o deshabilita la interacción con el componente.
349	int inicio = Math.max(1,	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
350	Math.min(paginaActual - 1, totalPaginas - 2));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
351		Línea en blanco: separa visualmente bloques para que el código sea más legible.
352	for (int indice = 0; indice < botonesPagina.size(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
353	int pagina = inicio + indice;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
354	BotonRedondeado boton = botonesPagina.get(indice);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
355	boolean visible = pagina <= totalPaginas;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
356	boton.setVisible(visible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
357	if (!visible) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
358	continue;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
359	}	Cierra el bloque abierto anteriormente.
360	boton.putClientProperty("pagina", pagina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

361	boton.setText(String.valueOf(pagina));	Cambia el texto visible de un componente de la interfaz.
362	boolean seleccionada = pagina == paginaActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
363	boton.setColorInicio(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
364	boton.setColorFinal(seleccionada ? AMARILLO : Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
365	boton.setColorBorde(seleccionada ? AMARILLO : BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
366	boton.setFont(seleccionada	Configura una propiedad mediante setFont en el objeto indicado.
367	? tema.negrita(13f) : tema.media(13f));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368	}	Cierra el bloque abierto anteriormente.
369	}	Cierra el bloque abierto anteriormente.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	private void cambiarPagina(int pagina) {	Declara el método cambiarPagina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
372	if (pagina < 1 pagina > totalPaginas pagina == paginaActual) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
373	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
374	}	Cierra el bloque abierto anteriormente.
375	paginaActual = pagina;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
376	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377	}	Cierra el bloque abierto anteriormente.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379	private void limpiarFiltros() {	Declara el método limpiarFiltros. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
380	campoBusqueda.setText("");	Cambia el texto visible de un componente de la interfaz.
381	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
382	rolSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
383	estadoSeleccionado = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
384	filtroRol.setText("Todos los roles");	Cambia el texto visible de un componente de la interfaz.
385	filtroEstado.setText("Todos los estados");	Cambia el texto visible de un componente de la interfaz.
386	paginaActual = 1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
387	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
388	}	Cierra el bloque abierto anteriormente.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	private void abrirFormulario(Usuario usuario) {	Declara el método abrirFormulario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
391	Window ventana = SwingUtilities.getWindowAncestor(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
392	JRootPane raiz = SwingUtilities.getRootPane(this);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
393	Component cristalAnterior = raiz != null ? raiz.getGlassPane() : null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
394	boolean cristalVisible = cristalAnterior != null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
395	&& cristalAnterior.isVisible();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397	if (raiz != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
398	FondoOscuro fondo = new FondoOscuro();	Crea una nueva instancia de FondoOscuro y la usa o guarda para continuar el proceso.
399	raiz.setGlassPane(fondo);	Configura una propiedad mediante setGlassPane en el objeto indicado.
400	fondo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
401	}	Cierra el bloque abierto anteriormente.
402		Línea en blanco: separa visualmente bloques para que el código sea más legible.
403	boolean modificado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
404	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

405	modificado = UsuarioFormDialog.mostrar(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
406	ventana, crud, usuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
407	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
408	if (raiz != null && cristalAnterior != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
409	raiz.setGlassPane(cristalAnterior);	Configura una propiedad mediante setGlassPane en el objeto indicado.
410	cristalAnterior.setVisible(cristalVisible);	Muestra u oculta una ventana o componente según el valor booleano enviado.
411	}	Cierra el bloque abierto anteriormente.
412	}	Cierra el bloque abierto anteriormente.
413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
414	if (modificado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
415	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
417	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
419	usuario == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
420	? "Usuario agregado correctamente."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
421	: "Usuario actualizado correctamente.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
422	"Gestión de usuarios",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
423	JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
424	}	Cierra el bloque abierto anteriormente.
425	}	Cierra el bloque abierto anteriormente.
426		Línea en blanco: separa visualmente bloques para que el código sea más legible.
427	private void desactivarUsuario(Usuario usuario) {	Declara el método desactivarUsuario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
428	if (!usuario.isActivo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
429	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
430	}	Cierra el bloque abierto anteriormente.
431	if (usuario.getIdUsuario() == SesionUsuario.getIdUsuario()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
432	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
433	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
434	"No puedes desactivar tu propia cuenta.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
435	"Acción no permitida",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
436	JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
437	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
438	}	Cierra el bloque abierto anteriormente.
439		Línea en blanco: separa visualmente bloques para que el código sea más legible.
440	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
441	if ("ADMINISTRADOR".equals(usuario.getRol()))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
442	&& crud.contarAdministradoresActivos() <= 1) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	"Debe existir al menos un administrador activo.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	"Acción no permitida",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

449	}	Cierra el bloque abierto anteriormente.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	int respuesta = JOptionPane.showConfirmDialog(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
452	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	"¿Desactivar a " + usuario.getNombreCompleto()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
454	+ "?\nYa no podrá iniciar sesión, "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
455	+ "pero sus pedidos se conservarán.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
456	"Confirmar desactivación",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
457	JOptionPane.YES_NO_OPTION,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
458	JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
459	if (respuesta == JOptionPane.YES_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
460	crud.desactivar(usuario.getIdUsuario());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
461	cargarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
462	cargarPagina();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
463	}	Cierra el bloque abierto anteriormente.
464	} catch (SQLException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
465	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
466	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
467	"No fue posible desactivar: " + ex.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
468	"Error",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
469	JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
470	}	Cierra el bloque abierto anteriormente.
471	}	Cierra el bloque abierto anteriormente.
472		Línea en blanco: separa visualmente bloques para que el código sea más legible.
473	/** Exporta todos los resultados del filtro, no sólo la página visible. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
474	private void exportarCsv() {	Declara el método exportarCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
475	JFileChooser selector = new JFileChooser();	Crea una nueva instancia de JFileChooser y la usa o guarda para continuar el proceso.
476	selector.setDialogTitle("Exportar usuarios");	Configura una propiedad mediante setDialogTitle en el objeto indicado.
477	selector.setSelectedFile(new java.io.File("usuarios.csv"));	Crea una nueva instancia de java.io.File y la usa o guarda para continuar el proceso.
478	selector.setFileFilter(new FileNameExtensionFilter(	Crea una nueva instancia de FileNameExtensionFilter y la usa o guarda para continuar el proceso.
479	"Archivo CSV (*.csv)", "csv"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
480	if (selector.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
481	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
482	}	Cierra el bloque abierto anteriormente.
483		Línea en blanco: separa visualmente bloques para que el código sea más legible.
484	Path destino = selector.getSelectedFile().toPath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
485	if (!destino.getFileName().toString().toLowerCase()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
486	.endsWith(".csv")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
487	destino = destino.resolveSibling(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
488	destino.getFileName().toString() + ".csv");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
489	}	Cierra el bloque abierto anteriormente.
490		Línea en blanco: separa visualmente bloques para que el código sea más legible.
491	final Path archivo = destino;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
492	final String busqueda = campoBusqueda.getText();	Declara una constante o referencia que no puede reasignarse después de inicializarse.

493	final String rol = rolSeleccionado;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
494	final Boolean estado = estadoSeleccionado;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
495	setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
496		Línea en blanco: separa visualmente bloques para que el código sea más legible.
497	new SwingWorker<Integer, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
498	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
499	protected Integer doInBackground() throws Exception {	Declara el método doInBackground. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
500	List<Usuario> usuarios = crud.listarParaExportar(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
501	busqueda, rol, estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
502	escribirCsv(archivo, usuarios);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
503	return usuarios.size();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
504	}	Cierra el bloque abierto anteriormente.
505		Línea en blanco: separa visualmente bloques para que el código sea más legible.
506	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
507	protected void done() {	Declara el método done. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
508	setCursor(Cursor.getDefaultCursor());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
509	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
510	int cantidad = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
511	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
512	UsuariosPanel.this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
513	"Se exportaron " + cantidad + " usuarios en:\n"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
514	+ archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
515	"Exportación completada",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
516	JOptionPane.INFORMATION_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
517	} catch (InterruptedException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
518	Thread.currentThread().interrupt();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
519	} catch (ExecutionException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
520	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
521	UsuariosPanel.this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
522	"No fue posible exportar: "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
523	+ ex.getCause().getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
524	"Error de exportación",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
525	JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
526	}	Cierra el bloque abierto anteriormente.
527	}	Cierra el bloque abierto anteriormente.
528	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
529	}	Cierra el bloque abierto anteriormente.
530		Línea en blanco: separa visualmente bloques para que el código sea más legible.
531	private void escribirCsv(Path archivo, List<Usuario> usuarios)	Declara el método escribirCsv. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
532	throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
533	try (BufferedWriter escritor = Files.newBufferedWriter(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
534	archivo, StandardCharsets.UTF_8)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
535	escritor.write('\u000d'); // BOM: conserva tildes al abrir en Excel.	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
536	escritor.write(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

537	"ID,Nombre,Apellido,Usuario,Correo,Rol,Turno,Estado,Fecha creaci3n");	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
538	escritor.newLine();	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
539	for (Usuario usuario : usuarios) {	Bucle for: repite el bloque para una secuencia, rango o colecci3n de elementos.
540	escritor.write(String.join(", ",	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
541	csv(String.valueOf(usuario.getIdUsuario()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
542	csv(usuario.getNombre()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
543	csv(usuario.getApellido()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
544	csv(usuario.getNombreUsuario()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
545	csv(usuario.getCorreo()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
546	csv(usuario.getRol()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
547	csv(usuario.getTurnoEtiqueta()),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
548	csv(usuario.isActivo() ? "Activo" : "Inactivo"),	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
549	csv(usuario.getFechaCreacion() == null ? ""	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
550	: usuario.getFechaCreacion()	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
551	.format(FORMATO_FECHA)));	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
552	escritor.newLine();	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
553	}	Cierra el bloque abierto anteriormente.
554	}	Cierra el bloque abierto anteriormente.
555	}	Cierra el bloque abierto anteriormente.
556		L3nea en blanco: separa visualmente bloques para que el c3digo sea m3s legible.
557	private String csv(String valor) {	Declara el m3todo csv. Sus par3metros son datos de entrada y el tipo anterior al nombre indica qu3 devuelve.
558	return "\"" + valor.replace("\"", "\\\"") + "\"";	Finaliza el m3todo y devuelve el valor indicado a quien lo llam3.
559	}	Cierra el bloque abierto anteriormente.
560		L3nea en blanco: separa visualmente bloques para que el c3digo sea m3s legible.
561	private void mostrarErrorDatos(Throwable causa) {	Declara el m3todo mostrarErrorDatos. Sus par3metros son datos de entrada y el tipo anterior al nombre indica qu3 devuelve.
562	if (mostrandoErrorConexion) {	Eval3a una condici3n; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
563	return;	Finaliza el m3todo y devuelve el valor indicado a quien lo llam3.
564	}	Cierra el bloque abierto anteriormente.
565	mostrandoErrorConexion = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
566	String mensaje = causa == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
567	? "Error desconocido."	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
568	: causa.getMessage();	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
569	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
570	this,	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
571	"No fue posible cargar los usuarios.\n" + mensaje,	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
572	"Error de base de datos",	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
573	JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucci3n Java completa; el punto y coma marca su final.
574	}	Cierra el bloque abierto anteriormente.
575		L3nea en blanco: separa visualmente bloques para que el c3digo sea m3s legible.
576	private void configurarBotonPagina(	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
577	BotonRedondeado boton,	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
578	IconosUsuarios.Tipo tipoIcono) {	Forma parte del bloque actual y completa la l3gica indicada por las l3neas cercanas.
579	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
580	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.

581	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
582	boton.setColorBorde(BORDE);	Configura una propiedad mediante setColorBorde en el objeto indicado.
583	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
584	boton.setRadio(12);	Configura una propiedad mediante setRadio en el objeto indicado.
585	boton.setPreferredSize(new Dimension(40, 38));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
586	boton.setFont(tema.media(13f));	Configura una propiedad mediante setFont en el objeto indicado.
587	if (tipoIcono != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
588	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
589	boton.setIcon(IconosUsuarios.crear(	Configura una propiedad mediante setIcon en el objeto indicado.
590	tipoIcono, AZUL, 17));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
591	}	Cierra el bloque abierto anteriormente.
592	}	Cierra el bloque abierto anteriormente.
593		Línea en blanco: separa visualmente bloques para que el código sea más legible.
594	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
595	public void removeNotify() {	Declara el método removeNotify. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
596	temporizadorBusqueda.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	super.removeNotify();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
598	}	Cierra el bloque abierto anteriormente.
599		Línea en blanco: separa visualmente bloques para que el código sea más legible.
600	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
601	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
602	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
603	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
604	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
605	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
606	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
607	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
608		Línea en blanco: separa visualmente bloques para que el código sea más legible.
609	labelTitulo2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
610	labelTitulo3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
611	botonDerretido1 = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
612	panelFlotante1 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
613	panelCircular2 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
614	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
615	labelTitulo4 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
616	labelTitulo5 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
617	panelFlotante2 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
618	panelCircular3 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
619	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
620	labelTitulo7 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
621	labelTitulo8 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
622	panelFlotante4 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
623	panelCircular5 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
624	labelEscalable5 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.

625	labelTitulo13 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
626	labelTitulo14 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
627	panelFlotante3 = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
628	panelCircular4 = new Componentes.PanelCircular();	Crea una nueva instancia de Componentes.PanelCircular y la usa o guarda para continuar el proceso.
629	labelEscalable4 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
630	labelTitulo10 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
631	labelTitulo11 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
632	botonExportar = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
633	panelFiltros = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
634	campoBusqueda = new Componentes.CampoBusquedaAdmin();	Crea una nueva instancia de Componentes.CampoBusquedaAdmin y la usa o guarda para continuar el proceso.
635	filtroRol = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
636	filtroEstado = new Componentes.BotonDesplegable();	Crea una nueva instancia de Componentes.BotonDesplegable y la usa o guarda para continuar el proceso.
637	botonLimpiarFiltros = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
638	panelTabla = new Componentes.PanelFlotante();	Crea una nueva instancia de Componentes.PanelFlotante y la usa o guarda para continuar el proceso.
639	scrollUsuarios = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
640	tablaUsuarios = new Componentes.TablaAdministrativa();	Crea una nueva instancia de Componentes.TablaAdministrativa y la usa o guarda para continuar el proceso.
641	etiquetaRango = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
642	panelPaginacion = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
643	botonAnterior = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
644	botonPagina1 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
645	botonPagina2 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
646	botonPagina3 = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
647	botonSiguiente = new Componentes.BotonRedondeado();	Crea una nueva instancia de Componentes.BotonRedondeado y la usa o guarda para continuar el proceso.
648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
649	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
650	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
651		Línea en blanco: separa visualmente bloques para que el código sea más legible.
652	labelTitulo2.setFont(new java.awt.Font("Dialog", 1, 18)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
653	labelTitulo2.setForeground(new java.awt.Color(127, 137, 154));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
654	labelTitulo2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
655	labelTitulo2.setText("Administra los usuarios del sistema");	Cambia el texto visible de un componente de la interfaz.
656	add(labelTitulo2, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 80, 320, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
657		Línea en blanco: separa visualmente bloques para que el código sea más legible.
658	labelTitulo3.setFont(new java.awt.Font("Dialog", 1, 55)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
659	labelTitulo3.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
660	labelTitulo3.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
661	labelTitulo3.setText("Gestión de Usuarios");	Cambia el texto visible de un componente de la interfaz.
662	add(labelTitulo3, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 30, 540, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	botonDerretido1.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
665	botonDerretido1.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/icono_agregar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
666	botonDerretido1.setText("Agregar usuario");	Cambia el texto visible de un componente de la interfaz.
667	botonDerretido1.setIconTextGap(15);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
668	add(botonDerretido1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1050, 60, 230, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

669		Línea en blanco: separa visualmente bloques para que el código sea más legible.
670	panelFlotante1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
671		Línea en blanco: separa visualmente bloques para que el código sea más legible.
672	panelCircular2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
673		Línea en blanco: separa visualmente bloques para que el código sea más legible.
674	labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/icono_usuariostotal.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
675	panelCircular2.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(6, 20, 70, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
676		Línea en blanco: separa visualmente bloques para que el código sea más legible.
677	panelFlotante1.add(panelCircular2, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
678		Línea en blanco: separa visualmente bloques para que el código sea más legible.
679	labelTitulo4.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
680	labelTitulo4.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
681	labelTitulo4.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
682	labelTitulo4.setText("Total de usuarios");	Cambia el texto visible de un componente de la interfaz.
683	panelFlotante1.add(labelTitulo4, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
684		Línea en blanco: separa visualmente bloques para que el código sea más legible.
685	labelTitulo5.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
686	labelTitulo5.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
687	labelTitulo5.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
688	labelTitulo5.setText("48");	Cambia el texto visible de un componente de la interfaz.
689	panelFlotante1.add(labelTitulo5, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
690		Línea en blanco: separa visualmente bloques para que el código sea más legible.
691	add(panelFlotante1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
692		Línea en blanco: separa visualmente bloques para que el código sea más legible.
693	panelFlotante2.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
694		Línea en blanco: separa visualmente bloques para que el código sea más legible.
695	panelCircular3.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
696	panelCircular3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
697		Línea en blanco: separa visualmente bloques para que el código sea más legible.
698	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imagenes/icono_admin.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
699	panelCircular3.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(8, 15, 65, 65));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
700		Línea en blanco: separa visualmente bloques para que el código sea más legible.
701	panelFlotante2.add(panelCircular3, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
702		Línea en blanco: separa visualmente bloques para que el código sea más legible.
703	labelTitulo7.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
704	labelTitulo7.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
705	labelTitulo7.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
706	labelTitulo7.setText("Administradores");	Cambia el texto visible de un componente de la interfaz.
707	panelFlotante2.add(labelTitulo7, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
708		Línea en blanco: separa visualmente bloques para que el código sea más legible.
709	labelTitulo8.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
710	labelTitulo8.setForeground(new java.awt.Color(195, 61, 66));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
711	labelTitulo8.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.

712	labelTitulo8.setText("48");	Cambia el texto visible de un componente de la interfaz.
713	panelFlotante2.add(labelTitulo8, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
714		Línea en blanco: separa visualmente bloques para que el código sea más legible.
715	add(panelFlotante2, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
716		Línea en blanco: separa visualmente bloques para que el código sea más legible.
717	panelFlotante4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
718		Línea en blanco: separa visualmente bloques para que el código sea más legible.
719	panelCircular5.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
720		Línea en blanco: separa visualmente bloques para que el código sea más legible.
721	labelEscalable5.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/icono_usuario.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
722	panelCircular5.add(labelEscalable5, new org.netbeans.lib.awtextra.AbsoluteConstraints(3, 9, 75, 75));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
723		Línea en blanco: separa visualmente bloques para que el código sea más legible.
724	panelFlotante4.add(panelCircular5, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
725		Línea en blanco: separa visualmente bloques para que el código sea más legible.
726	labelTitulo13.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
727	labelTitulo13.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
728	labelTitulo13.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
729	labelTitulo13.setText("Cajeros");	Cambia el texto visible de un componente de la interfaz.
730	panelFlotante4.add(labelTitulo13, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
731		Línea en blanco: separa visualmente bloques para que el código sea más legible.
732	labelTitulo14.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
733	labelTitulo14.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
734	labelTitulo14.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
735	labelTitulo14.setText("48");	Cambia el texto visible de un componente de la interfaz.
736	panelFlotante4.add(labelTitulo14, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
737		Línea en blanco: separa visualmente bloques para que el código sea más legible.
738	add(panelFlotante4, new org.netbeans.lib.awtextra.AbsoluteConstraints(800, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
739		Línea en blanco: separa visualmente bloques para que el código sea más legible.
740	panelFlotante3.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
741		Línea en blanco: separa visualmente bloques para que el código sea más legible.
742	panelCircular4.setColorFondo(new java.awt.Color(252, 233, 233));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
743	panelCircular4.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
744		Línea en blanco: separa visualmente bloques para que el código sea más legible.
745	labelEscalable4.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/icono_usuario_inacti vo.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
746	panelCircular4.add(labelEscalable4, new org.netbeans.lib.awtextra.AbsoluteConstraints(5, 12, 70, 70));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
747		Línea en blanco: separa visualmente bloques para que el código sea más legible.
748	panelFlotante3.add(panelCircular4, new org.netbeans.lib.awtextra.AbsoluteConstraints(40, 30, 80, 90));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
749		Línea en blanco: separa visualmente bloques para que el código sea más legible.
750	labelTitulo10.setFont(new java.awt.Font("Arial", 1, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
751	labelTitulo10.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
752	labelTitulo10.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
753	labelTitulo10.setText("Inactivos");	Cambia el texto visible de un componente de la interfaz.
754	panelFlotante3.add(labelTitulo10, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 0, 650, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

755		Línea en blanco: separa visualmente bloques para que el código sea más legible.
756	labelTitulo11.setFont(new java.awt.Font("Dialog", 1, 40)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
757	labelTitulo11.setForeground(new java.awt.Color(13, 17, 23));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
758	labelTitulo11.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
759	labelTitulo11.setText("48");	Cambia el texto visible de un componente de la interfaz.
760	panelFlotante3.add(labelTitulo11, new org.netbeans.lib.awtextra.AbsoluteConstraints(140, 40, 90, 76));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
761		Línea en blanco: separa visualmente bloques para que el código sea más legible.
762	add(panelFlotante3, new org.netbeans.lib.awtextra.AbsoluteConstraints(1170, 150, 360, 150));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
763		Línea en blanco: separa visualmente bloques para que el código sea más legible.
764	botonExportar.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
765	botonExportar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
766	botonExportar.setForeground(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
767	botonExportar.setRadio(20);	Configura una propiedad mediante setRadio en el objeto indicado.
768	botonExportar.setText("Exportar");	Cambia el texto visible de un componente de la interfaz.
769	botonExportar.setColorBorde(new java.awt.Color(231, 55, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
770	botonExportar.setColorFinal(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
771	botonExportar.setFont(new java.awt.Font("Arial", 1, 16)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
772	botonExportar.setGrosorBorde(1.6F);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
773	botonExportar.setMargin(new java.awt.Insets(5, 14, 3, 14));	Crea una nueva instancia de java.awt.Insets y la usa o guarda para continuar el proceso.
774	add(botonExportar, new org.netbeans.lib.awtextra.AbsoluteConstraints(1290, 60, 230, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
775		Línea en blanco: separa visualmente bloques para que el código sea más legible.
776	panelFiltros.setRadio(18);	Configura una propiedad mediante setRadio en el objeto indicado.
777	panelFiltros.setSombra(false);	Configura una propiedad mediante setSombra en el objeto indicado.
778	panelFiltros.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
779		Línea en blanco: separa visualmente bloques para que el código sea más legible.
780	campoBusqueda.setPlaceholder("Buscar usuario, nombre o correo...");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
781	campoBusqueda.setFont(new java.awt.Font("Dialog", 0, 14)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
782	panelFiltros.add(campoBusqueda, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 22, 455, 56));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
783		Línea en blanco: separa visualmente bloques para que el código sea más legible.
784	filtroRol.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
785	filtroRol.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
786	filtroRol.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
787	filtroRol.setText("Todos los roles");	Cambia el texto visible de un componente de la interfaz.
788	filtroRol.setTextoDesplegable("Todos los roles;ADMINISTRADOR;CAJERO");	Configura una propiedad mediante setTextoDesplegable en el objeto indicado.
789	filtroRol.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
790	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
791	filtroRolActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
792	}	Cierra el bloque abierto anteriormente.
793	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
794	panelFiltros.add(filtroRol, new org.netbeans.lib.awtextra.AbsoluteConstraints(495, 24, 330, 52));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
795		Línea en blanco: separa visualmente bloques para que el código sea más legible.
796	filtroEstado.setColorFondo(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
797	filtroEstado.setColorHover(new java.awt.Color(248, 249, 251));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
798	filtroEstado.setForeground(new java.awt.Color(0, 20, 43));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.

799	<code>filtroEstado.setText("Todos los estados");</code>	Cambia el texto visible de un componente de la interfaz.
800	<code>filtroEstado.setTextoDesplegable("Todos los estados;Activo;Inactivo");</code>	Configura una propiedad mediante <code>setTextoDesplegable</code> en el objeto indicado.
801	<code>panelFiltros.add(filtroEstado, new org.netbeans.lib.awtextra.AbsoluteConstraints(850, 24, 330, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
802		Línea en blanco: separa visualmente bloques para que el código sea más legible.
803	<code>botonLimpiarFiltros.setColorInicio(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
804	<code>botonLimpiarFiltros.setDegradado(false);</code>	Configura una propiedad mediante <code>setDegradado</code> en el objeto indicado.
805	<code>botonLimpiarFiltros.setForeground(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
806	<code>botonLimpiarFiltros.setText("Limpiar filtros");</code>	Cambia el texto visible de un componente de la interfaz.
807	<code>botonLimpiarFiltros.setColorBorde(new java.awt.Color(231, 55, 65));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
808	<code>botonLimpiarFiltros.setColorFinal(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de <code>java.awt.Color</code> y la usa o guarda para continuar el proceso.
809	<code>botonLimpiarFiltros.setFont(new java.awt.Font("Arial", 1, 15)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
810	<code>botonLimpiarFiltros.setGrosorBorde(1.6F);</code>	Configura una propiedad mediante <code>setGrosorBorde</code> en el objeto indicado.
811	<code>botonLimpiarFiltros.setMargin(new java.awt.Insets(5, 14, 3, 14));</code>	Crea una nueva instancia de <code>java.awt.Insets</code> y la usa o guarda para continuar el proceso.
812	<code>panelFiltros.add(botonLimpiarFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 24, 230, 52));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
813		Línea en blanco: separa visualmente bloques para que el código sea más legible.
814	<code>add(panelFiltros, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 320, 1470, 100));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
815		Línea en blanco: separa visualmente bloques para que el código sea más legible.
816	<code>panelTabla.setRadio(18);</code>	Configura una propiedad mediante <code>setRadio</code> en el objeto indicado.
817	<code>panelTabla.setSombra(false);</code>	Configura una propiedad mediante <code>setSombra</code> en el objeto indicado.
818	<code>panelTabla.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteLayout</code> y la usa o guarda para continuar el proceso.
819		Línea en blanco: separa visualmente bloques para que el código sea más legible.
820	<code>scrollUsuarios.setBorder(null);</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
821	<code>scrollUsuarios.setVerticalScrollBarPolicy(javax.swing.ScrollPaneConstants.VERTICAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setVerticalScrollBarPolicy</code> en el objeto indicado.
822	<code>scrollUsuarios.setHorizontalScrollBarPolicy(javax.swing.ScrollPaneConstants.HORIZONTAL_SCROLLBAR_NEVER);</code>	Configura una propiedad mediante <code>setHorizontalScrollBarPolicy</code> en el objeto indicado.
823	<code>scrollUsuarios.setAutoscrolls(true);</code>	Configura una propiedad mediante <code>setAutoscrolls</code> en el objeto indicado.
824		Línea en blanco: separa visualmente bloques para que el código sea más legible.
825	<code>tablaUsuarios.setModel(new javax.swing.table.DefaultTableModel(</code>	Crea una nueva instancia de <code>javax.swing.table.DefaultTableModel</code> y la usa o guarda para continuar el proceso.
826	<code>new Object [][] {</code>	Crea una nueva instancia de <code>Object</code> y la usa o guarda para continuar el proceso.
827	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
828	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
829	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
830	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
831	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
832	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
833	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
834	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
835	<code>{null, null, null, null, null, null, null, null},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
836	<code>{null, null, null, null, null, null, null, null}</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
837	<code>},</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
838	<code>new String [] {</code>	Crea una nueva instancia de <code>String</code> y la usa o guarda para continuar el proceso.
839	<code>"ID", "Nombre", "Usuario", "Rol", "Correo", "Estado", "Fecha creación", "Acciones"</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
840	<code>}</code>	Cierra el bloque abierto anteriormente.
841	<code>) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
842	<code>boolean[] canEdit = new boolean [] {</code>	Crea una nueva instancia de <code>boolean</code> y la usa o guarda para continuar el proceso.

843	false, false, false, false, false, false, false, false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
844	};	Cierra el bloque abierto anteriormente y termina la declaración.
845		Línea en blanco: separa visualmente bloques para que el código sea más legible.
846	public boolean isCellEditable(int rowIndex, int columnIndex) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
847	return canEdit [columnIndex];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
848	}	Cierra el bloque abierto anteriormente.
849	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
850	tablaUsuarios.setAutoscrolls(false);	Configura una propiedad mediante setAutoscrolls en el objeto indicado.
851	tablaUsuarios.setColumnsCentradas("5,7");	Configura una propiedad mediante setColumnsCentradas en el objeto indicado.
852	scrollUsuarios.setViewportView(tablaUsuarios);	Configura una propiedad mediante setViewportView en el objeto indicado.
853		Línea en blanco: separa visualmente bloques para que el código sea más legible.
854	panelTabla.add(scrollUsuarios, new org.netbeans.lib.awtextra.AbsoluteConstraints(14, 14, 1442, 428));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
855		Línea en blanco: separa visualmente bloques para que el código sea más legible.
856	etiquetaRango.setForeground(new java.awt.Color(92, 103, 124));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
857	etiquetaRango.setText("Mostrando 1-10 de 10 usuarios");	Cambia el texto visible de un componente de la interfaz.
858	panelTabla.add(etiquetaRango, new org.netbeans.lib.awtextra.AbsoluteConstraints(28, 452, 330, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
859		Línea en blanco: separa visualmente bloques para que el código sea más legible.
860	panelPaginacion.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
861	panelPaginacion.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
862		Línea en blanco: separa visualmente bloques para que el código sea más legible.
863	botonAnterior.setText("<");	Cambia el texto visible de un componente de la interfaz.
864	botonAnterior.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
865	botonAnterior.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
866	panelPaginacion.add(botonAnterior, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
867		Línea en blanco: separa visualmente bloques para que el código sea más legible.
868	botonPagina1.setText("1");	Cambia el texto visible de un componente de la interfaz.
869	botonPagina1.setColorInicio(new java.awt.Color(255, 188, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
870	botonPagina1.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
871	panelPaginacion.add(botonPagina1, new org.netbeans.lib.awtextra.AbsoluteConstraints(50, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
872		Línea en blanco: separa visualmente bloques para que el código sea más legible.
873	botonPagina2.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
874	botonPagina2.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
875	botonPagina2.setText("2");	Cambia el texto visible de un componente de la interfaz.
876	panelPaginacion.add(botonPagina2, new org.netbeans.lib.awtextra.AbsoluteConstraints(100, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
877		Línea en blanco: separa visualmente bloques para que el código sea más legible.
878	botonPagina3.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
879	botonPagina3.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
880	botonPagina3.setText("3");	Cambia el texto visible de un componente de la interfaz.
881	panelPaginacion.add(botonPagina3, new org.netbeans.lib.awtextra.AbsoluteConstraints(150, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
882		Línea en blanco: separa visualmente bloques para que el código sea más legible.
883	botonSiguiente.setColorInicio(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
884	botonSiguiente.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
885	botonSiguiente.setText(">");	Cambia el texto visible de un componente de la interfaz.
886	panelPaginacion.add(botonSiguiente, new org.netbeans.lib.awtextra.AbsoluteConstraints(200, 2, 40, 38));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.

887		Línea en blanco: separa visualmente bloques para que el código sea más legible.
888	panelTabla.add(panelPaginacion, new org.netbeans.lib.awtextra.AbsoluteConstraints(1165, 452, 270, 42));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
889		Línea en blanco: separa visualmente bloques para que el código sea más legible.
890	add(panelTabla, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 420, 1470, 510));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
891	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
892		Línea en blanco: separa visualmente bloques para que el código sea más legible.
893	private void filtroRolActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_filtroRolActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
894	// TODO add your handling code here:	Comentario para humanos; Java no ejecuta este texto. TODO add your handling code here:
895	//GEN-LAST:event_filtroRolActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
896		Línea en blanco: separa visualmente bloques para que el código sea más legible.
897	/** Modelo no editable salvo por la celda que contiene las acciones. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
898	private final class ModeloTablaUsuarios extends AbstractTableModel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
899		Línea en blanco: separa visualmente bloques para que el código sea más legible.
900	private final String[] columnas = {	Declara una constante o referencia que no puede reasignarse después de inicializarse.
901	"ID", "Nombre", "Usuario", "Rol", "Turno", "Correo",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
902	"Estado", "Fecha creación", "Acciones"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
903	};	Cierra el bloque abierto anteriormente y termina la declaración.
904	private List<Usuario> usuarios = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
905		Línea en blanco: separa visualmente bloques para que el código sea más legible.
906	private void setUsuarios(List<Usuario> nuevosUsuarios) {	Declara el método setUsuarios. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
907	usuarios = List.copyOf(nuevosUsuarios);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
908	fireTableDataChanged();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
909	}	Cierra el bloque abierto anteriormente.
910		Línea en blanco: separa visualmente bloques para que el código sea más legible.
911	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
912	public int getRowCount() {	Declara el método getRowCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
913	return usuarios.size();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
914	}	Cierra el bloque abierto anteriormente.
915		Línea en blanco: separa visualmente bloques para que el código sea más legible.
916	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
917	public int getColumnCount() {	Declara el método getColumnCount. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
918	return columnas.length;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
919	}	Cierra el bloque abierto anteriormente.
920		Línea en blanco: separa visualmente bloques para que el código sea más legible.
921	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
922	public String getColumnName(int columna) {	Declara el método getColumnName. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
923	return columnas[columna];	Finaliza el método y devuelve el valor indicado a quien lo llamó.
924	}	Cierra el bloque abierto anteriormente.
925		Línea en blanco: separa visualmente bloques para que el código sea más legible.
926	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
927	public Object getValueAt(int fila, int columna) {	Declara el método getValueAt. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
928	Usuario usuario = usuarios.get(fila);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
929	return switch (columna) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
930	case 0 -> String.format("#%04d", usuario.getIdUsuario());	Define una de las rutas posibles dentro del switch.

931	case 1 -> usuario.getNombreCompleto();	Define una de las rutas posibles dentro del switch.
932	case 2 -> usuario.getNombreUsuario();	Define una de las rutas posibles dentro del switch.
933	case 3 -> usuario.getRol();	Define una de las rutas posibles dentro del switch.
934	case 4 -> usuario.getTurnoEtiqueta();	Define una de las rutas posibles dentro del switch.
935	case 5 -> usuario.getCorreo();	Define una de las rutas posibles dentro del switch.
936	case 6, 8 -> usuario;	Define una de las rutas posibles dentro del switch.
937	case 7 -> usuario.getFechaCreacion() == null	Define una de las rutas posibles dentro del switch.
938	? "" : usuario.getFechaCreacion().format(FORMATO_FECHA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
939	default -> "";	Define una de las rutas posibles dentro del switch.
940	};	Cierra el bloque abierto anteriormente y termina la declaración.
941	}	Cierra el bloque abierto anteriormente.
942		Línea en blanco: separa visualmente bloques para que el código sea más legible.
943	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
944	public Class<?> getColumnClass(int columna) {	Declara el método getColumnClass. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
945	return columna == 6 columna == 8	Finaliza el método y devuelve el valor indicado a quien lo llamó.
946	? Usuario.class : String.class;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
947	}	Cierra el bloque abierto anteriormente.
948		Línea en blanco: separa visualmente bloques para que el código sea más legible.
949	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
950	public boolean isCellEditable(int fila, int columna) {	Declara el método isCellEditable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
951	return columna == 8;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
952	}	Cierra el bloque abierto anteriormente.
953	}	Cierra el bloque abierto anteriormente.
954		Línea en blanco: separa visualmente bloques para que el código sea más legible.
955	/** Dibuja Activo/Inactivo como una insignia, no como texto plano. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
956	private final class RenderEstado implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
957		Línea en blanco: separa visualmente bloques para que el código sea más legible.
958	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
959	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
960	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
961	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
962	boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
963	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
964	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
965	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
966	Usuario usuario = (Usuario) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
967	JPanel panel = new JPanel(new FlowLayout(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
968	FlowLayout.CENTER, 0, 7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
969	panel.setBackground(seleccionado	Configura una propiedad mediante setBackground en el objeto indicado.
970	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
971		Línea en blanco: separa visualmente bloques para que el código sea más legible.
972	JLabel estado = new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
973	usuario.isActivo() ? "Activo" : "Inactivo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
974	estado.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.

975	estado.setFont(tema.media(11f));	Configura una propiedad mediante setFont en el objeto indicado.
976	estado.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
977	estado.setForeground(usuario.isActivo())	Configura una propiedad mediante setForeground en el objeto indicado.
978	? new Color(34, 139, 71) : SECUNDARIO);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
979	estado.setBackground(usuario.isActivo())	Configura una propiedad mediante setBackground en el objeto indicado.
980	? new Color(230, 245, 234)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
981	: new Color(237, 239, 243));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
982	estado.setBorder(new EmptyBorder(3, 13, 3, 13));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
983	panel.add(estado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
984	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
985	}	Cierra el bloque abierto anteriormente.
986	}	Cierra el bloque abierto anteriormente.
987		Línea en blanco: separa visualmente bloques para que el código sea más legible.
988	private final class RenderAcciones implements TableCellRenderer {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
989		Línea en blanco: separa visualmente bloques para que el código sea más legible.
990	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
991	public Component getTableCellRendererComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
992	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
993	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
994	boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
995	boolean foco,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
996	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
997	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
998	Usuario usuario = (Usuario) valor;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
999	JPanel panel = crearPanelAcciones(seleccionado	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1000	? tabla.getSelectionBackground() : Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1001	BotonRedondeado editar = crearBotonAccion(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1002	IconosUsuarios.Tipo.EDITAR,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1003	new Color(214, 151, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1004	BotonRedondeado eliminar = crearBotonAccion(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1005	IconosUsuarios.Tipo.ELIMINAR, ROJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1006	eliminar.setEnabled(usuario.isActivo());	Habilita o deshabilita la interacción con el componente.
1007	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1008	panel.add(eliminar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1009	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1010	}	Cierra el bloque abierto anteriormente.
1011	}	Cierra el bloque abierto anteriormente.
1012		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1013	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1014	* El renderer sólo dibuja. Este editor es el que convierte los iconos en	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1015	* botones reales y llama las operaciones de la fila correspondiente.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1016	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1017	private final class EditorAcciones extends AbstractCellEditor	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1018	implements TableCellEditor {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

1019		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1020	private final JPanel panel = crearPanelAcciones(Color.WHITE);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1021	private final BotonRedondeado editar = crearBotonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1022	IconosUsuarios.Tipo.EDITAR, new Color(214, 151, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1023	private final BotonRedondeado eliminar = crearBotonAccion(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
1024	IconosUsuarios.Tipo.ELIMINAR, ROJO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1025	private Usuario usuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1026		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1027	private EditorAcciones() {	Declara el método EditorAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1028	editar.addActionListener(this::editar);	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1029	eliminar.addActionListener(this::eliminar);	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
1030	panel.add(editar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1031	panel.add(eliminar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1032	}	Cierra el bloque abierto anteriormente.
1033		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1034	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1035	public Component getTableCellEditorComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1036	JTable tabla,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1037	Object valor,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1038	boolean seleccionado,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1039	int fila,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1040	int columna) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1041	usuario = (Usuario) valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1042	panel.setBackground(tabla.getSelectionBackground());	Configura una propiedad mediante setBackground en el objeto indicado.
1043	eliminar.setEnabled(usuario.isActivo());	Habilita o deshabilita la interacción con el componente.
1044	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1045	}	Cierra el bloque abierto anteriormente.
1046		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1047	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1048	public Object getCellEditorValue() {	Declara el método getCellEditorValue. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1049	return usuario;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1050	}	Cierra el bloque abierto anteriormente.
1051		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1052	private void editar(ActionEvent evento) {	Declara el método editar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1053	Usuario seleccionado = usuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1054	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1055	abrirFormulario(seleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1056	}	Cierra el bloque abierto anteriormente.
1057		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1058	private void eliminar(ActionEvent evento) {	Declara el método eliminar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1059	Usuario seleccionado = usuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1060	fireEditingStopped();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1061	desactivarUsuario(seleccionado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1062	}	Cierra el bloque abierto anteriormente.

1063	}	Cierra el bloque abierto anteriormente.
1064		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1065	private JPanel crearPanelAcciones(Color fondo) {	Declara el método crearPanelAcciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1066	JPanel panel = new JPanel(new FlowLayout(	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
1067	FlowLayout.CENTER, 7, 4));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1068	panel.setBackground(fondo);	Configura una propiedad mediante setBackground en el objeto indicado.
1069	return panel;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1070	}	Cierra el bloque abierto anteriormente.
1071		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1072	private BotonRedondeado crearBotonAccion(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1073	IconosUsuarios.Tipo icono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1074	Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1075	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
1076	boton.setText("");	Cambia el texto visible de un componente de la interfaz.
1077	boton.setIcon(IconosUsuarios.crear(icono, color, 17));	Configura una propiedad mediante setIcon en el objeto indicado.
1078	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
1079	boton.setColorInicio(Color.WHITE);	Configura una propiedad mediante setColorInicio en el objeto indicado.
1080	boton.setColorFinal(Color.WHITE);	Configura una propiedad mediante setColorFinal en el objeto indicado.
1081	boton.setColorBorde(color);	Configura una propiedad mediante setColorBorde en el objeto indicado.
1082	boton.setGrosorBorde(1f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
1083	boton.setRadio(9);	Configura una propiedad mediante setRadio en el objeto indicado.
1084	boton.setPreferredSize(new Dimension(33, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
1085	boton.setToolTipText(icono == IconosUsuarios.Tipo.EDITAR	Configura una propiedad mediante setToolTipText en el objeto indicado.
1086	? "Editar usuario" : "Desactivar usuario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1087	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1088	}	Cierra el bloque abierto anteriormente.
1089		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1090	/** Capa temporal que oscurece el panel mientras el JDialog está abierto. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
1091	private static final class FondoOscuro extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1092		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1093	private FondoOscuro() {	Declara el método FondoOscuro. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1094	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1095	// Consume clics para que el fondo no pueda modificarse.	Comentario para humanos; Java no ejecuta este texto. Consume clics para que el fondo no pueda modificarse.
1096	addMouseListener(new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
1097	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1098	}	Cierra el bloque abierto anteriormente.
1099		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1100	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1101	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1102	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1103	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1104	g2.setColor(new Color(0, 12, 28, 90));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1105	g2.fillRect(0, 0, getWidth(), getHeight());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1106	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

1107	}	Cierra el bloque abierto anteriormente.
1108	}	Cierra el bloque abierto anteriormente.
1109		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1111	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1112	private Componentes.BotonRedondeado botonAnterior;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1113	private Componentes.BotonDerretido botonDerretido1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1114	private Componentes.BotonRedondeado botonExportar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1115	private Componentes.BotonRedondeado botonLimpiarFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1116	private Componentes.BotonRedondeado botonPagina1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1117	private Componentes.BotonRedondeado botonPagina2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1118	private Componentes.BotonRedondeado botonPagina3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1119	private Componentes.BotonRedondeado botonSiguiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1120	private Componentes.CampoBusquedaAdmin campoBusqueda;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1121	private javax.swing.JLabel etiquetaRango;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1122	private Componentes.BotonDesplegable filtroEstado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1123	private Componentes.BotonDesplegable filtroRol;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1124	private Labels.LabelEscalable labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1125	private Labels.LabelEscalable labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1126	private Labels.LabelEscalable labelEscalable4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1127	private Labels.LabelEscalable labelEscalable5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1128	private javax.swing.JLabel labelTitulo10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1129	private javax.swing.JLabel labelTitulo11;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1130	private javax.swing.JLabel labelTitulo13;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1131	private javax.swing.JLabel labelTitulo14;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1132	private javax.swing.JLabel labelTitulo2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1133	private javax.swing.JLabel labelTitulo3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1134	private javax.swing.JLabel labelTitulo4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1135	private javax.swing.JLabel labelTitulo5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1136	private javax.swing.JLabel labelTitulo7;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1137	private javax.swing.JLabel labelTitulo8;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1138	private Componentes.PanelCircular panelCircular2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1139	private Componentes.PanelCircular panelCircular3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1140	private Componentes.PanelCircular panelCircular4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1141	private Componentes.PanelCircular panelCircular5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1142	private Componentes.PanelFlotante panelFiltros;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1143	private Componentes.PanelFlotante panelFlotante1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1144	private Componentes.PanelFlotante panelFlotante2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1145	private Componentes.PanelFlotante panelFlotante3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1146	private Componentes.PanelFlotante panelFlotante4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1147	private javax.swing.JPanel panelPaginacion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1148	private Componentes.PanelFlotante panelTabla;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1149	private javax.swing.JScrollPane scrollUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1150	private Componentes.TablaAdministrativa tablaUsuarios;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

1151	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
1152	}	Cierra el bloque abierto anteriormente.

16.89 src/GUI_CAJERO/Cajero.form

Metadatos visuales de NetBeans para diseñar Cajero. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 72 | Archivo 89 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.8" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JFrameFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="defaultCloseOperation" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	</Properties>	Cierra la sección XML abierta anteriormente.
7	<SyntheticProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	<SyntheticProperty name="formSizePolicy" type="int" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	<SyntheticProperty name="generateCenter" type="boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</SyntheticProperties>	Cierra la sección XML abierta anteriormente.
11	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,2,-48,0,0,5,80"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	</AuxValues>	Cierra la sección XML abierta anteriormente.
23		Línea en blanco para organizar el XML del formulario.
24	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
25	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
26	</Layout>	Cierra la sección XML abierta anteriormente.
27	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Container class="javax.swing.JPanel" name="jPanel1">	Declara un componente o contenedor visual y su clase Java.
29	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
31	<Color blue="fd" green="fc" red="fc" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	</Property>	Cierra la sección XML abierta anteriormente.
33	</Properties>	Cierra la sección XML abierta anteriormente.
34	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
35	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
36	<AbsoluteConstraints x="0" y="0" width="1360" height="720"/>	Define la posición y el tamaño del componente dentro del diseño.
37	</Constraint>	Cierra la sección XML abierta anteriormente.

38	</Constraints>	Cierra la sección XML abierta anteriormente.
39		Línea en blanco para organizar el XML del formulario.
40	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
41	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
42	</Layout>	Cierra la sección XML abierta anteriormente.
43	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
44	<Container class="javax.swing.JPanel" name="jPanel2">	Declara un componente o contenedor visual y su clase Java.
45	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
47	<Color blue="1e" green="f" red="1" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
48	</Property>	Cierra la sección XML abierta anteriormente.
49	</Properties>	Cierra la sección XML abierta anteriormente.
50	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
51	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
52	<AbsoluteConstraints x="0" y="0" width="1360" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
53	</Constraint>	Cierra la sección XML abierta anteriormente.
54	</Constraints>	Cierra la sección XML abierta anteriormente.
55		Línea en blanco para organizar el XML del formulario.
56	<Layout>	Indica el administrador de distribución que organiza los componentes hijos.
57	<DimensionLayout dim="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
58	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
59	<EmptySpace min="0" pref="1360" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
60	</Group>	Cierra la sección XML abierta anteriormente.
61	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
62	<DimensionLayout dim="1">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
63	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	<EmptySpace min="0" pref="80" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
65	</Group>	Cierra la sección XML abierta anteriormente.
66	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
67	</Layout>	Cierra la sección XML abierta anteriormente.
68	</Container>	Cierra la sección XML abierta anteriormente.
69	</SubComponents>	Cierra la sección XML abierta anteriormente.
70	</Container>	Cierra la sección XML abierta anteriormente.
71	</SubComponents>	Cierra la sección XML abierta anteriormente.
72	</Form>	Cierra la sección XML abierta anteriormente.

16.90 src/GUI_CAJERO/Cajero.java

Ventana principal del cajero. Coordina menú, pedido, carrito y barra de título.

Tipo: java | Líneas: 873 | Archivo 90 de 157

Clases/tipos detectados: Cajero

Línea	Constructor o método detectado
70	public Cajero() {
90	private void configurarInterfaz() {
239	private void mostrarPanel(
262	private void configurarBarra() {
490	private javax.swing.JButton crearBotonBarra(
499	protected void paintComponent(
644	public void mouseEntered(
661	public void mouseExited(
679	private void seleccionarBoton(
711	private void abrirPedido() {
726	private void cerrarPedido() {
741	private void crearPanelPedido() {
747	public void agregarProducto(int idProducto) {
773	private void configurarHoverSutilCancelar(javax.swing.JButton boton) {
779	public void mouseEntered(java.awt.event.MouseEvent evento) {
786	public void mouseExited(java.awt.event.MouseEvent evento) {
802	private void initComponents() {
836	public static void main(String args[]) {
863	public void run() {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	import Componentes.BarraTituloForm;	Importa Componentes.BarraTituloForm para poder usar esa clase sin escribir su nombre completo.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	* @author Computacion	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

12	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	public class Cajero extends javax.swing.JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14	// =====	Comentario para humanos; Java no ejecuta este texto. =====
15	// COLORES	Comentario para humanos; Java no ejecuta este texto. COLORES
16	// =====	Comentario para humanos; Java no ejecuta este texto. =====
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	private final java.awt.Color AZUL_BARRA =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	new java.awt.Color(1, 20, 36);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private final java.awt.Color AMARILLO =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
22	new java.awt.Color(255, 188, 13);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	private final java.awt.Color AMARILLO_HOVER_SUAVE =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
25	new java.awt.Color(255, 196, 28);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	private final java.awt.Color GRIS_TEXTO =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
28	new java.awt.Color(110, 110, 110);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	// =====	Comentario para humanos; Java no ejecuta este texto. =====
32	// CARD LAYOUT	Comentario para humanos; Java no ejecuta este texto. CARD LAYOUT
33	// =====	Comentario para humanos; Java no ejecuta este texto. =====
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	private java.awt.CardLayout cardLayout;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	private javax.swing.JPanel panelContenido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	private MenuCajeroPanel menuCajeroPanel;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	private promocionesPanel promocionesPanel;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	// =====	Comentario para humanos; Java no ejecuta este texto. =====
46	// ZONA CENTRAL Y PEDIDO	Comentario para humanos; Java no ejecuta este texto. ZONA CENTRAL Y PEDIDO
47	// =====	Comentario para humanos; Java no ejecuta este texto. =====
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	private javax.swing.JPanel zonaCentral;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
50	private javax.swing.JPanel panelPedido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
51	private boolean configurandoProducto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	// =====	Comentario para humanos; Java no ejecuta este texto. =====
55	// BOTONES	Comentario para humanos; Java no ejecuta este texto. BOTONES

56	// =====	Comentario para humanos; Java no ejecuta este texto. =====
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	private javax.swing.JButton btnMenu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	private javax.swing.JButton btnPromociones;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	private javax.swing.JButton btnOrdenar;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	private javax.swing.JButton botonActivo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
63	private BarraTituloForm barraTitulo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	// =====	Comentario para humanos; Java no ejecuta este texto. =====
67	// CONSTRUCTOR	Comentario para humanos; Java no ejecuta este texto. CONSTRUCTOR
68	// =====	Comentario para humanos; Java no ejecuta este texto. =====
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	public Cajero() {	Constructor de Cajero: prepara un objeto nuevo y deja sus componentes listos para usarse.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	configurarInterfaz();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	barraTitulo = BarraTituloForm.instalar(this, "Menú");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
78	setExtendedState(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	javax.swing.JFrame.MAXIMIZED_BOTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	// =====	Comentario para humanos; Java no ejecuta este texto. =====
87	// CONFIGURAR INTERFAZ PRINCIPAL	Comentario para humanos; Java no ejecuta este texto. CONFIGURAR INTERFAZ PRINCIPAL
88	// =====	Comentario para humanos; Java no ejecuta este texto. =====
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	private void configurarInterfaz() {	Declara el método configurarInterfaz. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	// Limpiar JFrame	Comentario para humanos; Java no ejecuta este texto. Limpiar JFrame
93	getContentPane().removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	getContentPane().setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
96	new java.awt.BorderLayout()	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
97	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	getContentPane().add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

100	jPanel1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	java.awt.BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
102	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	// Panel principal	Comentario para humanos; Java no ejecuta este texto. Panel principal
106	jPanel1.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108	jPanel1.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
109	new java.awt.BorderLayout()	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
110	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	// =====	Comentario para humanos; Java no ejecuta este texto. =====
114	// CARD LAYOUT	Comentario para humanos; Java no ejecuta este texto. CARD LAYOUT
115	// =====	Comentario para humanos; Java no ejecuta este texto. =====
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	cardLayout =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
118	new java.awt.CardLayout();	Crea una nueva instancia de java.awt.CardLayout y la usa o guarda para continuar el proceso.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	panelContenido =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
121	new javax.swing.JPanel(cardLayout);	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	panelContenido.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
124	new java.awt.Color(	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
125	252,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	252,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127	253	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	// =====	Comentario para humanos; Java no ejecuta este texto. =====
133	// CREAR LOS JPanel Form	Comentario para humanos; Java no ejecuta este texto. CREAR LOS JPanel Form
134	// =====	Comentario para humanos; Java no ejecuta este texto. =====
135		Línea en blanco: separa visualmente bloques para que el código sea más legible.
136	menuCajeroPanel =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
137	new MenuCajeroPanel();	Crea una nueva instancia de MenuCajeroPanel y la usa o guarda para continuar el proceso.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	promocionesPanel =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
140	new promocionesPanel();	Crea una nueva instancia de promocionesPanel y la usa o guarda para continuar el proceso.
141		Línea en blanco: separa visualmente bloques para que el código sea más legible.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.

144	// =====	Comentario para humanos; Java no ejecuta este texto. =====
145	// AGREGAR AL CARD LAYOUT	Comentario para humanos; Java no ejecuta este texto. AGREGAR AL CARD LAYOUT
146	// =====	Comentario para humanos; Java no ejecuta este texto. =====
147		Línea en blanco: separa visualmente bloques para que el código sea más legible.
148	panelContenido.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	menuCajeroPanel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
150	"MENU"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
151	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	panelContenido.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	promocionesPanel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	"PROMOCIONES"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
156	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	// =====	Comentario para humanos; Java no ejecuta este texto. =====
159	// CREAR PEDIDO	Comentario para humanos; Java no ejecuta este texto. CREAR PEDIDO
160	// =====	Comentario para humanos; Java no ejecuta este texto. =====
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	crearPanelPedido();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	// =====	Comentario para humanos; Java no ejecuta este texto. =====
166	// ZONA CENTRAL	Comentario para humanos; Java no ejecuta este texto. ZONA CENTRAL
167	// =====	Comentario para humanos; Java no ejecuta este texto. =====
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	zonaCentral =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
170	new javax.swing.JPanel(	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
171	new java.awt.BorderLayout()	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
172	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	zonaCentral.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
175	new java.awt.Color(	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
176	252,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	252,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	253	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182		Línea en blanco: separa visualmente bloques para que el código sea más legible.
183	// Los JPanel Form van en el centro	Comentario para humanos; Java no ejecuta este texto. Los JPanel Form van en el centro
184	zonaCentral.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
185	panelContenido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186	java.awt.BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

188		Línea en blanco: separa visualmente bloques para que el código sea más legible.
189		Línea en blanco: separa visualmente bloques para que el código sea más legible.
190	// El pedido va a la derecha	Comentario para humanos; Java no ejecuta este texto. El pedido va a la derecha
191	zonaCentral.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	panelPedido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	java.awt.BorderLayout.EAST	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196		Línea en blanco: separa visualmente bloques para que el código sea más legible.
197	// =====	Comentario para humanos; Java no ejecuta este texto. =====
198	// BARRA SUPERIOR	Comentario para humanos; Java no ejecuta este texto. BARRA SUPERIOR
199	// =====	Comentario para humanos; Java no ejecuta este texto. =====
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	configurarBarra();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202		Línea en blanco: separa visualmente bloques para que el código sea más legible.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	// =====	Comentario para humanos; Java no ejecuta este texto. =====
205	// AGREGAR AL JFrame	Comentario para humanos; Java no ejecuta este texto. AGREGAR AL JFrame
206	// =====	Comentario para humanos; Java no ejecuta este texto. =====
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	jPanel1.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
209	jPanel2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
210	java.awt.BorderLayout.NORTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	jPanel1.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	zonaCentral,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	java.awt.BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218		Línea en blanco: separa visualmente bloques para que el código sea más legible.
219	// =====	Comentario para humanos; Java no ejecuta este texto. =====
220	// MENÚ AL INICIAR	Comentario para humanos; Java no ejecuta este texto. MENÚ AL INICIAR
221	// =====	Comentario para humanos; Java no ejecuta este texto. =====
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	mostrarPanel("MENU");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	seleccionarBoton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	btnMenu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
228		Línea en blanco: separa visualmente bloques para que el código sea más legible.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230	jPanel1.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
231	jPanel1.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

232 }	Cierra el bloque abierto anteriormente.
233	Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	Línea en blanco: separa visualmente bloques para que el código sea más legible.
235 // =====	Comentario para humanos; Java no ejecuta este texto. =====
236 // MOSTRAR UN PANEL	Comentario para humanos; Java no ejecuta este texto. MOSTRAR UN PANEL
237 // =====	Comentario para humanos; Java no ejecuta este texto. =====
238	Línea en blanco: separa visualmente bloques para que el código sea más legible.
239 private void mostrarPanel(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240 String nombrePanel) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
241	Línea en blanco: separa visualmente bloques para que el código sea más legible.
242 cardLayout.show(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
243 panelContenido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244 nombrePanel	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
245);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
246	Línea en blanco: separa visualmente bloques para que el código sea más legible.
247 panelContenido.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
248 panelContenido.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
249	Línea en blanco: separa visualmente bloques para que el código sea más legible.
250 if (barraTitulo != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
251 barraTitulo.setSeccion(	Configura una propiedad mediante setSeccion en el objeto indicado.
252 "PROMOCIONES".equals(nombrePanel) ? "Promociones" : "Menú"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254 }	Cierra el bloque abierto anteriormente.
255 }	Cierra el bloque abierto anteriormente.
256	Línea en blanco: separa visualmente bloques para que el código sea más legible.
257	Línea en blanco: separa visualmente bloques para que el código sea más legible.
258 // =====	Comentario para humanos; Java no ejecuta este texto. =====
259 // BARRA SUPERIOR	Comentario para humanos; Java no ejecuta este texto. BARRA SUPERIOR
260 // =====	Comentario para humanos; Java no ejecuta este texto. =====
261	Línea en blanco: separa visualmente bloques para que el código sea más legible.
262 private void configurarBarra() {	Declara el método configurarBarra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
263	Línea en blanco: separa visualmente bloques para que el código sea más legible.
264 jPanel2.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
265	Línea en blanco: separa visualmente bloques para que el código sea más legible.
266 jPanel2.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
267 AZUL_BARRA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
268);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	Línea en blanco: separa visualmente bloques para que el código sea más legible.
270 jPanel2.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
271 new java.awt.Dimension(	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
272 0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273 80	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	jPanel2.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
278	new java.awt.BorderLayout()	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
279	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281		Línea en blanco: separa visualmente bloques para que el código sea más legible.
282	// =====	Comentario para humanos; Java no ejecuta este texto. =====
283	// IZQUIERDA	Comentario para humanos; Java no ejecuta este texto. IZQUIERDA
284	// =====	Comentario para humanos; Java no ejecuta este texto. =====
285		Línea en blanco: separa visualmente bloques para que el código sea más legible.
286	javax.swing.JPanel panelIzquierdo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
287	new javax.swing.JPanel(	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
288	new java.awt.FlowLayout(	Crea una nueva instancia de java.awt.FlowLayout y la usa o guarda para continuar el proceso.
289	java.awt.FlowLayout.LEFT,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290	18,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
291	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
292	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
293	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
294		Línea en blanco: separa visualmente bloques para que el código sea más legible.
295	panelIzquierdo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
296		Línea en blanco: separa visualmente bloques para que el código sea más legible.
297		Línea en blanco: separa visualmente bloques para que el código sea más legible.
298	// =====	Comentario para humanos; Java no ejecuta este texto. =====
299	// ESPACIO PARA LOGO	Comentario para humanos; Java no ejecuta este texto. ESPACIO PARA LOGO
300	// =====	Comentario para humanos; Java no ejecuta este texto. =====
301		Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	javax.swing.JPanel espacioLogo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
303	new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
304		Línea en blanco: separa visualmente bloques para que el código sea más legible.
305	espacioLogo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	espacioLogo.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
308	new java.awt.Dimension(	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
309	95,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
310	60	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314	panelIzquierdo.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	espacioLogo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318		Línea en blanco: separa visualmente bloques para que el código sea más legible.
319	// =====	Comentario para humanos; Java no ejecuta este texto. =====

320	// BOTONES	Comentario para humanos; Java no ejecuta este texto. BOTONES
321	// =====	Comentario para humanos; Java no ejecuta este texto. =====
322		Línea en blanco: separa visualmente bloques para que el código sea más legible.
323	btnMenu =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
324	crearBotonBarra(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
325	"Menú"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
326	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
327		Línea en blanco: separa visualmente bloques para que el código sea más legible.
328	btnPromociones =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
329	crearBotonBarra(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	"Promociones"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
331	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332		Línea en blanco: separa visualmente bloques para que el código sea más legible.
333	panelIzquierdo.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
334	btnMenu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
335	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
336		Línea en blanco: separa visualmente bloques para que el código sea más legible.
337	panelIzquierdo.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338	btnPromociones	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
340		Línea en blanco: separa visualmente bloques para que el código sea más legible.
341		Línea en blanco: separa visualmente bloques para que el código sea más legible.
342	// =====	Comentario para humanos; Java no ejecuta este texto. =====
343	// ORDENAR	Comentario para humanos; Java no ejecuta este texto. ORDENAR
344	// =====	Comentario para humanos; Java no ejecuta este texto. =====
345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
346	btnOrdenar = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
347	btnOrdenar.setText("Ordenar");	Cambia el texto visible de un componente de la interfaz.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349		Línea en blanco: separa visualmente bloques para que el código sea más legible.
350	btnOrdenar.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
351	new java.awt.Dimension(	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
352	180,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
353	60	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
354	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
356		Línea en blanco: separa visualmente bloques para que el código sea más legible.
357		Línea en blanco: separa visualmente bloques para que el código sea más legible.
358	btnOrdenar.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
359	new java.awt.Font(	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
360	"Arial",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
361	java.awt.Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
362	16	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
363	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

364	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
365		Línea en blanco: separa visualmente bloques para que el código sea más legible.
366		Línea en blanco: separa visualmente bloques para que el código sea más legible.
367	btnOrdenar.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
368	java.awt.Color.BLACK	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
369	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	btnOrdenar.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
372		Línea en blanco: separa visualmente bloques para que el código sea más legible.
373	btnOrdenar.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.
375	btnOrdenar.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
376		Línea en blanco: separa visualmente bloques para que el código sea más legible.
377	btnOrdenar.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379	// Conserva la posición del texto del botón Ordenar original.	Comentario para humanos; Java no ejecuta este texto. Conserva la posición del texto del botón Ordenar original.
380	btnOrdenar.setHorizontalAlignment(	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
381	javax.swing.SwingConstants.RIGHT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
382	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
383	btnOrdenar.setVerticalAlignment(	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
384	javax.swing.SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
385	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
387	btnOrdenar.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
388	javax.swing.BorderFactory.createEmptyBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
389	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
390	45,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
391	14,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	20	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
393	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
394	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
395		Línea en blanco: separa visualmente bloques para que el código sea más legible.
396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397	btnOrdenar.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
398	new java.awt.Cursor(	Crea una nueva instancia de java.awt.Cursor y la usa o guarda para continuar el proceso.
399	java.awt.Cursor.HAND_CURSOR	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
400	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
401	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
402		Línea en blanco: separa visualmente bloques para que el código sea más legible.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	// =====	Comentario para humanos; Java no ejecuta este texto. =====
405	// DERECHA	Comentario para humanos; Java no ejecuta este texto. DERECHA
406	// =====	Comentario para humanos; Java no ejecuta este texto. =====
407		Línea en blanco: separa visualmente bloques para que el código sea más legible.

408	javax.swing.JPanel panelDerecho =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
409	new javax.swing.JPanel(	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
410	new java.awt.FlowLayout(	Crea una nueva instancia de java.awt.FlowLayout y la usa o guarda para continuar el proceso.
411	java.awt.FlowLayout.RIGHT,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
412	25,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
413	15	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
414	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417	panelDerecho.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
419	panelDerecho.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
420	btnOrdenar	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
421	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
422		Línea en blanco: separa visualmente bloques para que el código sea más legible.
423		Línea en blanco: separa visualmente bloques para que el código sea más legible.
424	// =====	Comentario para humanos; Java no ejecuta este texto. =====
425	// AGREGAR A BARRA	Comentario para humanos; Java no ejecuta este texto. AGREGAR A BARRA
426	// =====	Comentario para humanos; Java no ejecuta este texto. =====
427		Línea en blanco: separa visualmente bloques para que el código sea más legible.
428	jPanel2.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
429	panelIzquierdo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
430	java.awt.BorderLayout.WEST	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
431	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
432		Línea en blanco: separa visualmente bloques para que el código sea más legible.
433	jPanel2.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
434	panelDerecho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
435	java.awt.BorderLayout.EAST	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
436	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
437		Línea en blanco: separa visualmente bloques para que el código sea más legible.
438		Línea en blanco: separa visualmente bloques para que el código sea más legible.
439	// =====	Comentario para humanos; Java no ejecuta este texto. =====
440	// MENÚ	Comentario para humanos; Java no ejecuta este texto. MENÚ
441	// =====	Comentario para humanos; Java no ejecuta este texto. =====
442		Línea en blanco: separa visualmente bloques para que el código sea más legible.
443	btnMenu.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
444		Línea en blanco: separa visualmente bloques para que el código sea más legible.
445	mostrarPanel(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	"MENU"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448		Línea en blanco: separa visualmente bloques para que el código sea más legible.
449	seleccionarBoton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
450	btnMenu	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
451	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

452	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
453		Línea en blanco: separa visualmente bloques para que el código sea más legible.
454		Línea en blanco: separa visualmente bloques para que el código sea más legible.
455	// =====	Comentario para humanos; Java no ejecuta este texto. =====
456	// PROMOCIONES	Comentario para humanos; Java no ejecuta este texto. PROMOCIONES
457	// =====	Comentario para humanos; Java no ejecuta este texto. =====
458		Línea en blanco: separa visualmente bloques para que el código sea más legible.
459	btnPromociones.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	mostrarPanel(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
462	"PROMOCIONES"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
463	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
464		Línea en blanco: separa visualmente bloques para que el código sea más legible.
465	seleccionarBoton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
466	btnPromociones	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
467	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
468	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
469		Línea en blanco: separa visualmente bloques para que el código sea más legible.
470	// =====	Comentario para humanos; Java no ejecuta este texto. =====
471	// ORDENAR	Comentario para humanos; Java no ejecuta este texto. ORDENAR
472	// =====	Comentario para humanos; Java no ejecuta este texto. =====
473		Línea en blanco: separa visualmente bloques para que el código sea más legible.
474	btnOrdenar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
475		Línea en blanco: separa visualmente bloques para que el código sea más legible.
476	abrirPedido();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
477		Línea en blanco: separa visualmente bloques para que el código sea más legible.
478	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
479		Línea en blanco: separa visualmente bloques para que el código sea más legible.
480		Línea en blanco: separa visualmente bloques para que el código sea más legible.
481	jPanel2.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
482	jPanel2.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
483	}	Cierra el bloque abierto anteriormente.
484		Línea en blanco: separa visualmente bloques para que el código sea más legible.
485		Línea en blanco: separa visualmente bloques para que el código sea más legible.
486	// =====	Comentario para humanos; Java no ejecuta este texto. =====
487	// CREAR BOTONES DE LA BARRA	Comentario para humanos; Java no ejecuta este texto. CREAR BOTONES DE LA BARRA
488	// =====	Comentario para humanos; Java no ejecuta este texto. =====
489		Línea en blanco: separa visualmente bloques para que el código sea más legible.
490	private javax.swing.JButton crearBotonBarra(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
491	String texto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
493	javax.swing.JButton boton =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
494	new javax.swing.JButton(	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
495	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

496	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
497		Línea en blanco: separa visualmente bloques para que el código sea más legible.
498	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
499	protected void paintComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
500	java.awt.Graphics g) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
501		Línea en blanco: separa visualmente bloques para que el código sea más legible.
502	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
503		Línea en blanco: separa visualmente bloques para que el código sea más legible.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	// Línea amarilla	Comentario para humanos; Java no ejecuta este texto. Línea amarilla
506	if (this == botonActivo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
507		Línea en blanco: separa visualmente bloques para que el código sea más legible.
508	java.awt.Graphics2D g2 =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
509	(java.awt.Graphics2D)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
510	g.create();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
511		Línea en blanco: separa visualmente bloques para que el código sea más legible.
512	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
513	java.awt.RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
514	java.awt.RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
515	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
516		Línea en blanco: separa visualmente bloques para que el código sea más legible.
517	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
518	AMARILLO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
519	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
520		Línea en blanco: separa visualmente bloques para que el código sea más legible.
521	int anchoLinea =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
522	35;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
523		Línea en blanco: separa visualmente bloques para que el código sea más legible.
524	int altoLinea =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
525	Math.max(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
526	3,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
527	3	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
528	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
529		Línea en blanco: separa visualmente bloques para que el código sea más legible.
530	int x =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
531	(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
532	getWidth()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
533	- anchoLinea	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
534	) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
535		Línea en blanco: separa visualmente bloques para que el código sea más legible.
536	int y =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
537	getHeight()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
538	- 6;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
539		Línea en blanco: separa visualmente bloques para que el código sea más legible.

540	g2.fillRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
541	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
542	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
543	anchoLinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
544	altoLinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
545	altoLinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
546	altoLinea	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
547	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
548		Línea en blanco: separa visualmente bloques para que el código sea más legible.
549	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
550	}	Cierra el bloque abierto anteriormente.
551	}	Cierra el bloque abierto anteriormente.
552	};	Cierra el bloque abierto anteriormente y termina la declaración.
553		Línea en blanco: separa visualmente bloques para que el código sea más legible.
554		Línea en blanco: separa visualmente bloques para que el código sea más legible.
555	// =====	Comentario para humanos; Java no ejecuta este texto. =====
556	// FUENTE	Comentario para humanos; Java no ejecuta este texto. FUENTE
557	// =====	Comentario para humanos; Java no ejecuta este texto. =====
558		Línea en blanco: separa visualmente bloques para que el código sea más legible.
559	java.awt.Font fuente =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
560	new java.awt.Font(	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
561	"Arial",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
562	java.awt.Font.BOLD,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
563	14	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
564	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
565		Línea en blanco: separa visualmente bloques para que el código sea más legible.
566	boton.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
567	fuentes	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
568	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
569		Línea en blanco: separa visualmente bloques para que el código sea más legible.
570		Línea en blanco: separa visualmente bloques para que el código sea más legible.
571	// =====	Comentario para humanos; Java no ejecuta este texto. =====
572	// ANCHO SEGÚN TEXTO	Comentario para humanos; Java no ejecuta este texto. ANCHO SEGÚN TEXTO
573	// =====	Comentario para humanos; Java no ejecuta este texto. =====
574		Línea en blanco: separa visualmente bloques para que el código sea más legible.
575	java.awt.FontMetrics fm =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
576	boton.getFontMetrics(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
577	fuentes	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
578	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
579		Línea en blanco: separa visualmente bloques para que el código sea más legible.
580	int anchoTexto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
581	fm.stringWidth(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
582	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
583	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

584		Línea en blanco: separa visualmente bloques para que el código sea más legible.
585	int anchoBoton =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
586	anchoTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
587	+ 45;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
588		Línea en blanco: separa visualmente bloques para que el código sea más legible.
589		Línea en blanco: separa visualmente bloques para que el código sea más legible.
590	boton.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
591	new java.awt.Dimension(	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
592	anchoBoton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
593	55	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
594	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
595	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
596		Línea en blanco: separa visualmente bloques para que el código sea más legible.
597		Línea en blanco: separa visualmente bloques para que el código sea más legible.
598	boton.setMinimumSize(	Configura una propiedad mediante setMinimumSize en el objeto indicado.
599	new java.awt.Dimension(	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
600	anchoBoton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
601	55	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
602	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
603	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
604		Línea en blanco: separa visualmente bloques para que el código sea más legible.
605		Línea en blanco: separa visualmente bloques para que el código sea más legible.
606	// =====	Comentario para humanos; Java no ejecuta este texto. =====
607	// ESTILO	Comentario para humanos; Java no ejecuta este texto. ESTILO
608	// =====	Comentario para humanos; Java no ejecuta este texto. =====
609		Línea en blanco: separa visualmente bloques para que el código sea más legible.
610	boton.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
611	java.awt.Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
612	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
613		Línea en blanco: separa visualmente bloques para que el código sea más legible.
614	boton.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
615		Línea en blanco: separa visualmente bloques para que el código sea más legible.
616	boton.setContentAreaFilled(	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
617	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
618	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
619		Línea en blanco: separa visualmente bloques para que el código sea más legible.
620	boton.setBorderPainted(	Configura una propiedad mediante setBorderPainted en el objeto indicado.
621	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
622	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
623		Línea en blanco: separa visualmente bloques para que el código sea más legible.
624	boton.setFocusPainted(	Configura una propiedad mediante setFocusPainted en el objeto indicado.
625	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
626	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
627		Línea en blanco: separa visualmente bloques para que el código sea más legible.

628		Línea en blanco: separa visualmente bloques para que el código sea más legible.
629	boton.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
630	new java.awt.Cursor(	Crea una nueva instancia de java.awt.Cursor y la usa o guarda para continuar el proceso.
631	java.awt.Cursor.HAND_CURSOR	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
632	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
633	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
634		Línea en blanco: separa visualmente bloques para que el código sea más legible.
635		Línea en blanco: separa visualmente bloques para que el código sea más legible.
636	// =====	Comentario para humanos; Java no ejecuta este texto. =====
637	// EFECTO MOUSE	Comentario para humanos; Java no ejecuta este texto. EFECTO MOUSE
638	// =====	Comentario para humanos; Java no ejecuta este texto. =====
639		Línea en blanco: separa visualmente bloques para que el código sea más legible.
640	boton.addMouseListener(	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
641	new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
642		Línea en blanco: separa visualmente bloques para que el código sea más legible.
643	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
644	public void mouseEntered(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
645	java.awt.event.MouseEvent e) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
646		Línea en blanco: separa visualmente bloques para que el código sea más legible.
647	if (boton != botonActivo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
649	boton.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
650	new java.awt.Color(	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
651	220,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
652	220,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
653	220	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
654	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
655	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
656	}	Cierra el bloque abierto anteriormente.
657	}	Cierra el bloque abierto anteriormente.
658		Línea en blanco: separa visualmente bloques para que el código sea más legible.
659		Línea en blanco: separa visualmente bloques para que el código sea más legible.
660	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
661	public void mouseExited(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
662	java.awt.event.MouseEvent e) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	boton.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
665	java.awt.Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
666	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
667	}	Cierra el bloque abierto anteriormente.
668	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
669		Línea en blanco: separa visualmente bloques para que el código sea más legible.
670		Línea en blanco: separa visualmente bloques para que el código sea más legible.
671	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

672 }	Cierra el bloque abierto anteriormente.
673	Línea en blanco: separa visualmente bloques para que el código sea más legible.
674	Línea en blanco: separa visualmente bloques para que el código sea más legible.
675 // =====	Comentario para humanos; Java no ejecuta este texto. =====
676 // BOTÓN ACTIVO	Comentario para humanos; Java no ejecuta este texto. BOTÓN ACTIVO
677 // =====	Comentario para humanos; Java no ejecuta este texto. =====
678	Línea en blanco: separa visualmente bloques para que el código sea más legible.
679 private void seleccionarBoton(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
680 javax.swing.JButton boton) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
681	Línea en blanco: separa visualmente bloques para que el código sea más legible.
682 botonActivo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
683 boton;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
684	Línea en blanco: separa visualmente bloques para que el código sea más legible.
685	Línea en blanco: separa visualmente bloques para que el código sea más legible.
686 if (btnMenu != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
687	Línea en blanco: separa visualmente bloques para que el código sea más legible.
688 btnMenu.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
689 java.awt.Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
690);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
691	Línea en blanco: separa visualmente bloques para que el código sea más legible.
692 btnMenu.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
693 }	Cierra el bloque abierto anteriormente.
694	Línea en blanco: separa visualmente bloques para que el código sea más legible.
695	Línea en blanco: separa visualmente bloques para que el código sea más legible.
696 if (btnPromociones != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
697	Línea en blanco: separa visualmente bloques para que el código sea más legible.
698 btnPromociones.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
699 java.awt.Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
700);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
701	Línea en blanco: separa visualmente bloques para que el código sea más legible.
702 btnPromociones.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
703 }	Cierra el bloque abierto anteriormente.
704 }	Cierra el bloque abierto anteriormente.
705	Línea en blanco: separa visualmente bloques para que el código sea más legible.
706	Línea en blanco: separa visualmente bloques para que el código sea más legible.
707 // =====	Comentario para humanos; Java no ejecuta este texto. =====
708 // ABRIR PEDIDO	Comentario para humanos; Java no ejecuta este texto. ABRIR PEDIDO
709 // =====	Comentario para humanos; Java no ejecuta este texto. =====
710	Línea en blanco: separa visualmente bloques para que el código sea más legible.
711 private void abrirPedido() {	Declara el método abrirPedido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
712	Línea en blanco: separa visualmente bloques para que el código sea más legible.
713 panelPedido.setVisible(	Muestra u oculta una ventana o componente según el valor booleano enviado.
714 true	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
715);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

716		Línea en blanco: separa visualmente bloques para que el código sea más legible.
717	zonaCentral.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
718	zonaCentral.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
719	}	Cierra el bloque abierto anteriormente.
720		Línea en blanco: separa visualmente bloques para que el código sea más legible.
721		Línea en blanco: separa visualmente bloques para que el código sea más legible.
722	// =====	Comentario para humanos; Java no ejecuta este texto. =====
723	// CERRAR PEDIDO	Comentario para humanos; Java no ejecuta este texto. CERRAR PEDIDO
724	// =====	Comentario para humanos; Java no ejecuta este texto. =====
725		Línea en blanco: separa visualmente bloques para que el código sea más legible.
726	private void cerrarPedido() {	Declara el método cerrarPedido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
727		Línea en blanco: separa visualmente bloques para que el código sea más legible.
728	panelPedido.setVisible(	Muestra u oculta una ventana o componente según el valor booleano enviado.
729	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
730	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
731		Línea en blanco: separa visualmente bloques para que el código sea más legible.
732	zonaCentral.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
733	zonaCentral.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
734	}	Cierra el bloque abierto anteriormente.
735		Línea en blanco: separa visualmente bloques para que el código sea más legible.
736		Línea en blanco: separa visualmente bloques para que el código sea más legible.
737	// =====	Comentario para humanos; Java no ejecuta este texto. =====
738	// CREAR PANEL PEDIDO	Comentario para humanos; Java no ejecuta este texto. CREAR PANEL PEDIDO
739	// =====	Comentario para humanos; Java no ejecuta este texto. =====
740		Línea en blanco: separa visualmente bloques para que el código sea más legible.
741	private void crearPanelPedido() {	Declara el método crearPanelPedido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
742	panelPedido = new PedidoPanel();	Crea una nueva instancia de PedidoPanel y la usa o guarda para continuar el proceso.
743	panelPedido.setPreferredSize(new java.awt.Dimension(390, 0));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
744	panelPedido.setVisible(false);	Muestra u oculta una ventana o componente según el valor booleano enviado.
745	}	Cierra el bloque abierto anteriormente.
746		Línea en blanco: separa visualmente bloques para que el código sea más legible.
747	public void agregarProducto(int idProducto) {	Declara el método agregarProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
748	if (configurandoProducto) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
749	configurandoProducto = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
750	setCursor(java.awt.Cursor.getPredefinedCursor(java.awt.Cursor.WAIT_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
751	new javax.swing.SwingWorker<Modelos.ConfiguracionProducto, Void>() {	Crea una nueva instancia de javax.swing.SwingWorker y la usa o guarda para continuar el proceso.
752	@Override protected Modelos.ConfiguracionProducto doInBackground() throws Exception {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
753	return new DAO.ConfiguracionMenuDAO().cargarProducto(idProducto);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
754	}	Cierra el bloque abierto anteriormente.
755	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
756	setCursor(java.awt.Cursor.getDefaultCursor());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
757	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
758	Modelos.ConfiguracionProducto configuracion = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
759	abrirPedido();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

760	((PedidoPanel) panelPedido).mostrarConfigurador(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
761	configuracion, () -> configurandoProducto = false);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
762	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
763	Throwable causa = ex instanceof java.util.concurrent.ExecutionException	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
764	&& ex.getCause() != null ? ex.getCause() : ex;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
765	javax.swing.JOptionPane.showMessageDialog(Cajero.this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
766	causa.getMessage() == null ? "No se pudo configurar el producto." : causa.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
767	"Producto no disponible", javax.swing.JOptionPane.WARNING_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
768	configurandoProducto = false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
769	}	Cierra el bloque abierto anteriormente.
770	}	Cierra el bloque abierto anteriormente.
771	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
772	}	Cierra el bloque abierto anteriormente.
773	private void configurarHoverSutilCancelar(javax.swing.JButton boton) {	Declara el método configurarHoverSutilCancelar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
774	java.awt.Color bordeNormal = new java.awt.Color(190, 190, 190);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
775	java.awt.Color bordeHover = new java.awt.Color(170, 170, 170);	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
776		Línea en blanco: separa visualmente bloques para que el código sea más legible.
777	boton.addMouseListener(new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
778	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
779	public void mouseEntered(java.awt.event.MouseEvent evento) {	Declara el método mouseEntered. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
780	boton.setForeground(new java.awt.Color(65, 65, 65));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
781	boton.setBorder(javax.swing.BorderFactory.createLineBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
782	bordeHover, 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
783	}	Cierra el bloque abierto anteriormente.
784		Línea en blanco: separa visualmente bloques para que el código sea más legible.
785	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
786	public void mouseExited(java.awt.event.MouseEvent evento) {	Declara el método mouseExited. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
787	boton.setForeground(new java.awt.Color(80, 80, 80));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
788	boton.setBorder(javax.swing.BorderFactory.createLineBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
789	bordeNormal, 1));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
790	}	Cierra el bloque abierto anteriormente.
791	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
792	}	Cierra el bloque abierto anteriormente.
793		Línea en blanco: separa visualmente bloques para que el código sea más legible.
794		Línea en blanco: separa visualmente bloques para que el código sea más legible.
795	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
796	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
797	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
798	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
799	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
800	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
801	// <editor-fold defaultstate="collapsed" desc="Generated Code">>//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
802	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

803		Línea en blanco: separa visualmente bloques para que el código sea más legible.
804	jPanel1 = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
805	jPanel2 = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
806		Línea en blanco: separa visualmente bloques para que el código sea más legible.
807	setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
808	getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
809		Línea en blanco: separa visualmente bloques para que el código sea más legible.
810	jPanel1.setBackground(new java.awt.Color(252, 252, 253));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
811	jPanel1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
812		Línea en blanco: separa visualmente bloques para que el código sea más legible.
813	jPanel2.setBackground(new java.awt.Color(1, 15, 30));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
814		Línea en blanco: separa visualmente bloques para que el código sea más legible.
815	javax.swing.GroupLayout jPanel2Layout = new javax.swing.GroupLayout(jPanel2);	Crea una nueva instancia de javax.swing.GroupLayout y la usa o guarda para continuar el proceso.
816	jPanel2.setLayout(jPanel2Layout);	Configura una propiedad mediante setLayout en el objeto indicado.
817	jPanel2Layout.setHorizontalGroup(	Configura una propiedad mediante setHorizontalGroup en el objeto indicado.
818	jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
819	.addGap(0, 1360, Short.MAX_VALUE)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
820	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
821	jPanel2Layout.setVerticalGroup(	Configura una propiedad mediante setVerticalGroup en el objeto indicado.
822	jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
823	.addGap(0, 80, Short.MAX_VALUE)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
824	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
825		Línea en blanco: separa visualmente bloques para que el código sea más legible.
826	jPanel1.add(jPanel2, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1360, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
827		Línea en blanco: separa visualmente bloques para que el código sea más legible.
828	getContentPane().add(jPanel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1360, 720));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
829		Línea en blanco: separa visualmente bloques para que el código sea más legible.
830	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
831	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
832		Línea en blanco: separa visualmente bloques para que el código sea más legible.
833	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
834	* @param args the command line arguments	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
835	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
836	public static void main(String args[]) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
837	/* Set the Nimbus look and feel */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
838	//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">	Comentario para humanos; Java no ejecuta este texto. <editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
839	/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
840	* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
841	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
842	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
843	for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
844	{	
844	if ("Nimbus".equals(info.getName())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
845	javax.swing.UIManager.setLookAndFeel(info.getClassName());	Configura una propiedad mediante setLookAndFeel en el objeto indicado.

846	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
847	}	Cierra el bloque abierto anteriormente.
848	}	Cierra el bloque abierto anteriormente.
849	} catch (ClassNotFoundException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
850	java.util.logging.Logger.getLogger(Cajero.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
851	} catch (InstantiationException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
852	java.util.logging.Logger.getLogger(Cajero.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
853	} catch (IllegalAccessException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
854	java.util.logging.Logger.getLogger(Cajero.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
855	} catch (javax.swing.UnsupportedLookAndFeelException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
856	java.util.logging.Logger.getLogger(Cajero.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
857	}	Cierra el bloque abierto anteriormente.
858	//</editor-fold>	Comentario para humanos; Java no ejecuta este texto. </editor-fold>
859	//</editor-fold>	Comentario para humanos; Java no ejecuta este texto. </editor-fold>
860		Línea en blanco: separa visualmente bloques para que el código sea más legible.
861	/* Create and display the form */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
862	java.awt.EventQueue.invokeLater(new Runnable() {	Crea una nueva instancia de Runnable y la usa o guarda para continuar el proceso.
863	public void run() {	Declara el método run. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
864	new Cajero().setVisible(true);	Crea una nueva instancia de Cajero y la usa o guarda para continuar el proceso.
865	}	Cierra el bloque abierto anteriormente.
866	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
867	}	Cierra el bloque abierto anteriormente.
868		Línea en blanco: separa visualmente bloques para que el código sea más legible.
869	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
870	private javax.swing.JPanel jPanel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
871	private javax.swing.JPanel jPanel2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
872	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
873	}	Cierra el bloque abierto anteriormente.

16.91 src/GUI_CAJERO/CategoriasPanel.form

Metadatos visuales de NetBeans para diseñar CategoriasPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 153 | Archivo 91 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,0,-66,0,0,6,-2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="Componentes.BotonCategoria" name="btnCajita">	Declara un componente o contenedor visual y su clase Java.
27	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Property name="text" type="java.lang.String" value="Cajita Feliz"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
29	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
30		Metadato de NetBeans que conserva el estado editable del diseñador visual.
31	</Property>	Cierra la sección XML abierta anteriormente.
32	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
33	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
34	</Properties>	Cierra la sección XML abierta anteriormente.
35	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
36	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnCajitaActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
37	</Events>	Cierra la sección XML abierta anteriormente.

38	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
39	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
40	<AbsoluteConstraints x="1220" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
41	</Constraint>	Cierra la sección XML abierta anteriormente.
42	</Constraints>	Cierra la sección XML abierta anteriormente.
43	</Component>	Cierra la sección XML abierta anteriormente.
44	<Component class="Componentes.BotonCategoria" name="btnDesayuno1">	Declara un componente o contenedor visual y su clase Java.
45	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Property name="text" type="java.lang.String" value="Desayuno"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
47	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
48		Metadato de NetBeans que conserva el estado editable del diseñador visual.
49	</Property>	Cierra la sección XML abierta anteriormente.
50	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
51	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	</Properties>	Cierra la sección XML abierta anteriormente.
53	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
54	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnDesayuno1ActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
55	</Events>	Cierra la sección XML abierta anteriormente.
56	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
57	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
58	<AbsoluteConstraints x="10" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
59	</Constraint>	Cierra la sección XML abierta anteriormente.
60	</Constraints>	Cierra la sección XML abierta anteriormente.
61	</Component>	Cierra la sección XML abierta anteriormente.
62	<Component class="Componentes.BotonCategoria" name="btnAlmuerzos">	Declara un componente o contenedor visual y su clase Java.
63	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	<Property name="text" type="java.lang.String" value="Almuerzos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
65	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
66		Metadato de NetBeans que conserva el estado editable del diseñador visual.
67	</Property>	Cierra la sección XML abierta anteriormente.
68	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
69	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
72	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnAlmuerzosActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
73	</Events>	Cierra la sección XML abierta anteriormente.
74	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
75	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
76	<AbsoluteConstraints x="210" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
77	</Constraint>	Cierra la sección XML abierta anteriormente.
78	</Constraints>	Cierra la sección XML abierta anteriormente.

79	</Component>	Cierra la sección XML abierta anteriormente.
80	<Component class="Componentes.BotonCategoria" name="btnPostres">	Declara un componente o contenedor visual y su clase Java.
81	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
82	<Property name="text" type="java.lang.String" value="Postres"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
83	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
84		Metadato de NetBeans que conserva el estado editable del diseñador visual.
85	</Property>	Cierra la sección XML abierta anteriormente.
86	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
87	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
88	</Properties>	Cierra la sección XML abierta anteriormente.
89	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
90	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnPostresActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
91	</Events>	Cierra la sección XML abierta anteriormente.
92	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
93	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
94	<AbsoluteConstraints x="410" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
95	</Constraint>	Cierra la sección XML abierta anteriormente.
96	</Constraints>	Cierra la sección XML abierta anteriormente.
97	</Component>	Cierra la sección XML abierta anteriormente.
98	<Component class="Componentes.BotonCategoria" name="btnCafe">	Declara un componente o contenedor visual y su clase Java.
99	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
100	<Property name="text" type="java.lang.String" value="WlCafé"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
101	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102		Metadato de NetBeans que conserva el estado editable del diseñador visual.
103	</Property>	Cierra la sección XML abierta anteriormente.
104	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
105	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
106	</Properties>	Cierra la sección XML abierta anteriormente.
107	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
108	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnCafeActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
109	</Events>	Cierra la sección XML abierta anteriormente.
110	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
111	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
112	<AbsoluteConstraints x="610" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
113	</Constraint>	Cierra la sección XML abierta anteriormente.
114	</Constraints>	Cierra la sección XML abierta anteriormente.
115	</Component>	Cierra la sección XML abierta anteriormente.
116	<Component class="Componentes.BotonCategoria" name="btnBebidos">	Declara un componente o contenedor visual y su clase Java.
117	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
118	<Property name="text" type="java.lang.String" value="Bebidas"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
119	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
120		Metadato de NetBeans que conserva el estado editable del diseñador visual.

121	</Property>	Cierra la sección XML abierta anteriormente.
122	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
123	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
124	</Properties>	Cierra la sección XML abierta anteriormente.
125	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
126	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnBebidosActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
127	</Events>	Cierra la sección XML abierta anteriormente.
128	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
129	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
130	<AbsoluteConstraints x="810" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
131	</Constraint>	Cierra la sección XML abierta anteriormente.
132	</Constraints>	Cierra la sección XML abierta anteriormente.
133	</Component>	Cierra la sección XML abierta anteriormente.
134	<Component class="Componentes.BotonCategoria" name="btnAntojos">	Declara un componente o contenedor visual y su clase Java.
135	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
136	<Property name="text" type="java.lang.String" value="Antojos"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
137	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
138		Metadato de NetBeans que conserva el estado editable del diseñador visual.
139	</Property>	Cierra la sección XML abierta anteriormente.
140	<Property name="hideActionText" type="boolean" value="true"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
141	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
142	</Properties>	Cierra la sección XML abierta anteriormente.
143	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
144	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="btnAntojosActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
145	</Events>	Cierra la sección XML abierta anteriormente.
146	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
147	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
148	<AbsoluteConstraints x="1010" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
149	</Constraint>	Cierra la sección XML abierta anteriormente.
150	</Constraints>	Cierra la sección XML abierta anteriormente.
151	</Component>	Cierra la sección XML abierta anteriormente.
152	</SubComponents>	Cierra la sección XML abierta anteriormente.
153	</Form>	Cierra la sección XML abierta anteriormente.

16.92 src/GUI_CAJERO/CategoriasPanel.java

Interfaz del módulo cajero: CategoriasPanel. Participa en la selección y venta de productos.

Tipo: java | Líneas: 197 | Archivo 92 de 157

Clases/tipos detectados: CategoriasPanel

Línea	Constructor o método detectado
8	public CategoriasPanel() {
17	public CategoriasPanel(
34	public CategoriasPanel(
43	private void initComponents() {
61	public void actionPerformed(java.awt.event.ActionEvent evt) {
72	public void actionPerformed(java.awt.event.ActionEvent evt) {
83	public void actionPerformed(java.awt.event.ActionEvent evt) {
94	public void actionPerformed(java.awt.event.ActionEvent evt) {
105	public void actionPerformed(java.awt.event.ActionEvent evt) {
116	public void actionPerformed(java.awt.event.ActionEvent evt) {
127	public void actionPerformed(java.awt.event.ActionEvent evt) {
134	private void btnCajitaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnCajitaActionPerformed
140	private void btnDesayunoActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnDesayunoActionPerformed
146	private void btnAlmuerzosActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnAlmuerzosActionPerformed
152	private void btnPostresActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnPostresActionPerformed
158	private void btnCafeActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnCafeActionPerformed
164	private void btnBebidosActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnBebidosActionPerformed
170	private void btnAntojosActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnAntojosActionPerformed
176	private void mostrarCategoriaDinamica(

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	public class CategoriasPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	private MenuCajeroPanel menuCajeroPanel;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
6	private java.util.function.Consumer<javax.swing.JPanel> panelDestino;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	public CategoriasPanel() {	Constructor de CategoriasPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.

12	setPreferredSize(13 new java.awt.Dimension(1790, 200) 14); 15 }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso. Ejecuta una instrucción Java completa; el punto y coma marca su final.
16		Cierra el bloque abierto anteriormente.
17	public CategoriasPanel(18 MenuCajeroPanel menuCajeroPanel) {	Línea en blanco: separa visualmente bloques para que el código sea más legible. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	initComponents();	Línea en blanco: separa visualmente bloques para que el código sea más legible. Ejecuta una instrucción Java completa; el punto y coma marca su final.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	this.menuCajeroPanel	Accede al atributo del objeto actual para asignarlo o utilizarlo.
23	= menuCajeroPanel;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	setPreferredSize(26 new java.awt.Dimension(27 1900, 28 170 29) 30); 31 }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.
32		Cierra el bloque abierto anteriormente.
33	/** Constructor reutilizable para vistas que quieran el mismo selector. */	Línea en blanco: separa visualmente bloques para que el código sea más legible. Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
34	public CategoriasPanel(35 java.util.function.Consumer<javax.swing.JPanel> panelDestino) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	initComponents();	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	this.panelDestino = panelDestino;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	setPreferredSize(new java.awt.Dimension(1900, 170));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
39	}	Cierra el bloque abierto anteriormente.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
42	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
43	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	btnCajita = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
46	btnDesayuno1 = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
47	btnAlmuerzos = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
48	btnPostres = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
49	btnCafe = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
50	btnBebidos = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
51	btnAntojos = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
54	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.

56	<code>btnCajita.setText("Cajita Feliz");</code>	Cambia el texto visible de un componente de la interfaz.
57	<code>btnCajita.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
58	<code>btnCajita.setHideActionText(true);</code>	Configura una propiedad mediante <code>setHideActionText</code> en el objeto indicado.
59	<code>btnCajita.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);</code>	Configura una propiedad mediante <code>setVerticalAlignment</code> en el objeto indicado.
60	<code>btnCajita.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
61	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
62	<code> btnCajitaActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	<code> }</code>	Cierra el bloque abierto anteriormente.
64	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65	<code>add(btnCajita, new org.netbeans.lib.awtextra.AbsoluteConstraints(1220, 10, 180, 170));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	<code>btnDesayuno1.setText("Desayuno");</code>	Cambia el texto visible de un componente de la interfaz.
68	<code>btnDesayuno1.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
69	<code>btnDesayuno1.setHideActionText(true);</code>	Configura una propiedad mediante <code>setHideActionText</code> en el objeto indicado.
70	<code>btnDesayuno1.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);</code>	Configura una propiedad mediante <code>setVerticalAlignment</code> en el objeto indicado.
71	<code>btnDesayuno1.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
72	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
73	<code> btnDesayuno1ActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	<code> }</code>	Cierra el bloque abierto anteriormente.
75	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76	<code>add(btnDesayuno1, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 10, 180, 170));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	<code>btnAlmuerzos.setText("Almuerzos");</code>	Cambia el texto visible de un componente de la interfaz.
79	<code>btnAlmuerzos.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
80	<code>btnAlmuerzos.setHideActionText(true);</code>	Configura una propiedad mediante <code>setHideActionText</code> en el objeto indicado.
81	<code>btnAlmuerzos.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);</code>	Configura una propiedad mediante <code>setVerticalAlignment</code> en el objeto indicado.
82	<code>btnAlmuerzos.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
83	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
84	<code> btnAlmuerzosActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	<code> }</code>	Cierra el bloque abierto anteriormente.
86	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	<code>add(btnAlmuerzos, new org.netbeans.lib.awtextra.AbsoluteConstraints(210, 10, 180, 170));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	<code>btnPostres.setText("Postres");</code>	Cambia el texto visible de un componente de la interfaz.
90	<code>btnPostres.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N</code>	Crea una nueva instancia de <code>java.awt.Font</code> y la usa o guarda para continuar el proceso.
91	<code>btnPostres.setHideActionText(true);</code>	Configura una propiedad mediante <code>setHideActionText</code> en el objeto indicado.
92	<code>btnPostres.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);</code>	Configura una propiedad mediante <code>setVerticalAlignment</code> en el objeto indicado.
93	<code>btnPostres.addActionListener(new java.awt.event.ActionListener() {</code>	Crea una nueva instancia de <code>java.awt.event.ActionListener</code> y la usa o guarda para continuar el proceso.
94	<code> public void actionPerformed(java.awt.event.ActionEvent evt) {</code>	Declara el método <code>actionPerformed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
95	<code> btnPostresActionPerformed(evt);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	<code> }</code>	Cierra el bloque abierto anteriormente.
97	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	<code>add(btnPostres, new org.netbeans.lib.awtextra.AbsoluteConstraints(410, 10, 180, 170));</code>	Crea una nueva instancia de <code>org.netbeans.lib.awtextra.AbsoluteConstraints</code> y la usa o guarda para continuar el proceso.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.

100	btnCafe.setText("WlCafé");	Cambia el texto visible de un componente de la interfaz.
101	btnCafe.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
102	btnCafe.setHideActionText(true);	Configura una propiedad mediante setHideActionText en el objeto indicado.
103	btnCafe.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
104	btnCafe.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
105	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
106	btnCafeActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107	}	Cierra el bloque abierto anteriormente.
108	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	add(btnCafe, new org.netbeans.lib.awtextra.AbsoluteConstraints(610, 10, 180, 170));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	btnBebidos.setText("Bebidas");	Cambia el texto visible de un componente de la interfaz.
112	btnBebidos.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
113	btnBebidos.setHideActionText(true);	Configura una propiedad mediante setHideActionText en el objeto indicado.
114	btnBebidos.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
115	btnBebidos.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
116	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
117	btnBebidosActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	}	Cierra el bloque abierto anteriormente.
119	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	add(btnBebidos, new org.netbeans.lib.awtextra.AbsoluteConstraints(810, 10, 180, 170));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	btnAntojos.setText("Antojos");	Cambia el texto visible de un componente de la interfaz.
123	btnAntojos.setFont(new java.awt.Font("Arial", 1, 24)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
124	btnAntojos.setHideActionText(true);	Configura una propiedad mediante setHideActionText en el objeto indicado.
125	btnAntojos.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
126	btnAntojos.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
127	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
128	btnAntojosActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129	}	Cierra el bloque abierto anteriormente.
130	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	add(btnAntojos, new org.netbeans.lib.awtextra.AbsoluteConstraints(1010, 10, 180, 170));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
132	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	private void btnCajitaActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnCajitaActionPerf	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	ormed	
136	mostrarCategoriaDinamica("Cajita Feliz", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	"Hamburguesas", "McNuggets", "Acompañamientos",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
138	"Bebidas", "Postres", "Juguetes"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
139	}//GEN-LAST:event_btnCajitaActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	private void btnDesayuno1ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_btnDesayuno1Acti	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
142	onPerformed	
	mostrarCategoriaDinamica("Desayunos", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
	"Sándwiches", "McMuffin", "Hot Cakes", "Bebidas",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

143	"Por tiempo limitado"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	}//GEN-LAST:event_btnDesayuno1ActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145		Línea en blanco: separa visualmente bloques para que el código sea más legible.
146	private void btnAlmuerzosActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_btnAlmuerzosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
147	mostrarCategoriaDinamica("Almuerzos", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
148	"Hamburguesas", "McNuggets", "Combos", "Papas",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	"Ensaladas", "Bebidas"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
150	}//GEN-LAST:event_btnAlmuerzosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	private void btnPostresActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_btnPostresActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
153	mostrarCategoriaDinamica("Postres", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
154	"McFlurry", "Sundae", "Conos", "Pasteles", "Galletas",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	"Por tiempo limitado"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156	}//GEN-LAST:event_btnPostresActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157		Línea en blanco: separa visualmente bloques para que el código sea más legible.
158	private void btnCafeActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_btnCafeActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
159	mostrarCategoriaDinamica("WlCafé", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
160	"Café", "Frappés", "Chocolate", "Té", "Bebidas frías",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
161	"Repostería"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	}//GEN-LAST:event_btnCafeActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	private void btnBebidosActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_btnBebidosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165	mostrarCategoriaDinamica("Bebidas", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	"Gaseosas", "Jugos", "Agua", "Café", "Bebidas frías",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	"Batidos"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	}//GEN-LAST:event_btnBebidosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	private void btnAntojosActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_btnAntojosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
171	mostrarCategoriaDinamica("Antojos", java.util.List.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	"Papas", "McNuggets", "Snacks", "Acompañamientos",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173	"Compartir"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
174	}//GEN-LAST:event_btnAntojosActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
175		Línea en blanco: separa visualmente bloques para que el código sea más legible.
176	private void mostrarCategoriaDinamica(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	String nombreCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	java.util.List<String> subcategorias) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	if (panelDestino != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
180	panelDestino.accept(new SubCategoriasPanel(nombreCategoria, subcategorias));	Crea una nueva instancia de SubCategoriasPanel y la usa o guarda para continuar el proceso.
181	} else if (menuCajeroPanel != null) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
182	menuCajeroPanel.mostrarPanelCategoria(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
183	new SubCategoriasPanel(nombreCategoria, subcategorias)	Crea una nueva instancia de SubCategoriasPanel y la usa o guarda para continuar el proceso.

184	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	}	Cierra el bloque abierto anteriormente.
186	}	Cierra el bloque abierto anteriormente.
187		Línea en blanco: separa visualmente bloques para que el código sea más legible.
188	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
189	private Componentes.BotonCategoria btnAlmuerzos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
190	private Componentes.BotonCategoria btnAntojos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
191	private Componentes.BotonCategoria btnBebidos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
192	private Componentes.BotonCategoria btnCafe;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
193	private Componentes.BotonCategoria btnCajita;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
194	private Componentes.BotonCategoria btnDesayuno1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
195	private Componentes.BotonCategoria btnPostres;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
196	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
197	}	Cierra el bloque abierto anteriormente.

16.93 src/GUI_CAJERO/ConfiguradorProductoDialog.java

Diálogo que aloja el configurador de producto y devuelve la línea terminada al carrito.

Tipo: java | Líneas: 469 | Archivo 93 de 157

Clases/tipos detectados: ConfiguradorProductoDialog, VistaPresentacion, SelectorGrupo, PersonalizacionProducto, Seleccion

Línea	Constructor o método detectado
25	public static LineaPedido configurar(Window propietario, int idProducto) throws Exception {
30	public static LineaPedido configurar(Window propietario,
37	private ConfiguradorProductoDialog(Window propietario, ConfiguracionProducto producto) {
48	private JComponent crearContenido() {
98	private BotonRedondeado boton(String texto, Color fondo, Color frente) {
113	private VistaPresentacion vistaActual() {
117	private void actualizarPrecio() {
123	private void confirmar() {
221	private void crearControles() {
266	private void seleccionarPredeterminada(JComboBox<OpcionMenu> lista) {
274	private void cambioSeleccion() {
279	private List<Seleccion> seleccionadas() {
299	private void reconstruirPersonalizaciones() {
325	private boolean tienePersonalizaciones() {
468	private record Seleccion(int numero, OpcionMenu opcion) {}

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
4	import DAO.ConfiguracionMenuDAO;	Importa DAO.ConfiguracionMenuDAO para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.*;	Importa Modelos.* para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.*;	Importa java.awt.* para poder usar esa clase sin escribir su nombre completo.
7	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
8	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
9	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
10	import javax.swing.*;	Importa javax.swing.* para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** Configura presentaciones, elecciones y cada producto interno del combo. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	public final class ConfiguradorProductoDialog extends JDialog {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
15	private static final Color AZUL = new Color(1, 20, 36);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
16	private static final Color AMARILLO = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

17	private static final Color BORDE = new Color(226, 229, 234);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
18	private static final Color TEXTO_SECUNDARIO = new Color(102, 108, 118);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
19	private final ConfiguracionProducto producto;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
20	private final JTabbedPane presentaciones = new JTabbedPane();	Crea una nueva instancia de JTabbedPane y la usa o guarda para continuar el proceso.
21	private final Map<Integer, VistaPresentacion> vistas = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
22	private final JLabel precio = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
23	private LineaPedido resultado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	public static LineaPedido configurar(Window propietario, int idProducto) throws Exception {	Declara el método configurar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
26	ConfiguracionProducto producto = new ConfiguracionMenuDAO().cargarProducto(idProducto);	Crea una nueva instancia de ConfiguracionMenuDAO y la usa o guarda para continuar el proceso.
27	return configurar(propietario, producto);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
28	}	Cierra el bloque abierto anteriormente.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	public static LineaPedido configurar(Window propietario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	ConfiguracionProducto producto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	ConfiguradorProductoDialog dialogo = new ConfiguradorProductoDialog(propietario, producto);	Crea una nueva instancia de ConfiguradorProductoDialog y la usa o guarda para continuar el proceso.
33	dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
34	return dialogo.resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
35	}	Cierra el bloque abierto anteriormente.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	private ConfiguradorProductoDialog(Window propietario, ConfiguracionProducto producto) {	Constructor de ConfiguradorProductoDialog: prepara un objeto nuevo y deja sus componentes listos para usarse.
38	super(propietario, "Configurar " + producto.nombre(), ModalityType.APPLICATION_MODAL);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	this.producto = producto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
40	setDefaultCloseOperation(DISPOSE_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	setMinimumSize(new Dimension(760, 690));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
42	setPreferredSize(new Dimension(880, 780));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
43	setContentPane(crearContenido());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	setLocationRelativeTo(propietario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	}	Cierra el bloque abierto anteriormente.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	private JComponent crearContenido() {	Declara el método crearContenido. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
49	JPanel raiz = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
50	raiz.setBackground(new Color(247, 248, 250));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
51	raiz.setBorder(new EmptyBorder(16, 16, 16, 16));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	JPanel cabecera = new JPanel(new BorderLayout(12, 4));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
54	cabecera.setBackground(AZUL);	Configura una propiedad mediante setBackground en el objeto indicado.
55	cabecera.setBorder(new EmptyBorder(18, 22, 18, 22));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
56	JPanel textos = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
57	textos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
58	textos.setLayout(new BoxLayout(textos, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
59	JLabel titulo = new JLabel(producto.nombre());	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
60	titulo.setForeground(Color.WHITE);	Configura una propiedad mediante setForeground en el objeto indicado.

61	titulo.setFont(new Font("SansSerif", Font.BOLD, 25));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
62	JLabel ayuda = new JLabel(producto.descripcion() == null producto.descripcion().isBlank()	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
63	? "Elige cómo preparar este producto" : producto.descripcion());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	ayuda.setForeground(new Color(220, 226, 233));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
65	ayuda.setFont(new Font("SansSerif", Font.PLAIN, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
66	textos.add(titulo); textos.add(Box.createVerticalStrut(4)); textos.add(ayuda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	precio.setForeground(AMARILLO);	Configura una propiedad mediante setForeground en el objeto indicado.
68	precio.setFont(new Font("SansSerif", Font.BOLD, 24));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
69	cabecera.add(textos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	cabecera.add(precio, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	raiz.add(cabecera, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	presentaciones.setFont(new Font("SansSerif", Font.BOLD, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
74	presentaciones.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
75	for (PresentacionMenu presentacion : producto.presentaciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
76	VistaPresentacion vista = new VistaPresentacion(presentacion);	Crea una nueva instancia de VistaPresentacion y la usa o guarda para continuar el proceso.
77	vistas.put(presentacion.idPresentacion(), vista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	presentaciones.addTab(presentacion.nombre(), vista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	if (presentacion.predeterminada()) presentaciones.setSelectedComponent(vista);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
80	}	Cierra el bloque abierto anteriormente.
81	presentaciones.addChangeListener(e -> actualizarPrecio());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	raiz.add(presentaciones, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	JPanel pie = new JPanel(new BorderLayout(12, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
85	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
86	pie.setBorder(new EmptyBorder(14, 0, 0, 0));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
87	BotonRedondeado cancelar = boton("Cancelar", Color.WHITE, new Color(196, 30, 42));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
88	BotonRedondeado agregar = boton("Agregar al pedido", AMARILLO, AZUL);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
89	cancelar.addActionListener(e -> dispose());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
90	agregar.addActionListener(e -> confirmar());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
91	pie.add(cancelar, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	pie.add(agregar, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	raiz.add(pie, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	SwingUtilities.invokeLater(this::actualizarPrecio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	return raiz;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
96	}	Cierra el bloque abierto anteriormente.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	private BotonRedondeado boton(String texto, Color fondo, Color frente) {	Declara el método boton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
99	BotonRedondeado boton = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
100	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
101	boton.setFont(new Font("SansSerif", Font.BOLD, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
102	boton.setForeground(frente);	Configura una propiedad mediante setForeground en el objeto indicado.
103	boton.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
104	boton.setColorInicio(fondo);	Configura una propiedad mediante setColorInicio en el objeto indicado.

105	<code>boton.setColorFinal(fondo);</code>	Configura una propiedad mediante setColorFinal en el objeto indicado.
106	<code>boton.setColorBorde(fondo.equals(Color.WHITE) ? new Color(196,30,42) : fondo);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
107	<code>boton.setGrosorBorde(1f);</code>	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
108	<code>boton.setRadio(18);</code>	Configura una propiedad mediante setRadio en el objeto indicado.
109	<code>boton.setPreferredSize(new Dimension(210, 48));</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
110	<code>return boton;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
111	<code>}</code>	Cierra el bloque abierto anteriormente.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113	<code>private VistaPresentacion vistaActual() {</code>	Declara el método vistaActual. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
114	<code>return (VistaPresentacion) presentaciones.getSelectedComponent();</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
115	<code>}</code>	Cierra el bloque abierto anteriormente.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	<code>private void actualizarPrecio() {</code>	Declara el método actualizarPrecio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
118	<code>VistaPresentacion vista = vistaActual();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
119	<code>if (vista == null) return;</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
120	<code>precio.setText("Q" + vista.precioActual().setScale(2).toPlainString());</code>	Cambia el texto visible de un componente de la interfaz.
121	<code>}</code>	Cierra el bloque abierto anteriormente.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	<code>private void confirmar() {</code>	Declara el método confirmar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
124	<code>try {</code>	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
125	<code>VistaPresentacion vista = vistaActual();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
126	<code>List<OpcionPedido> opciones = vista.crearOpciones();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
127	<code>resultado = new LineaPedido(UUID.randomUUID().toString(), producto.idProducto(),</code>	Crea una nueva instancia de LineaPedido y la usa o guarda para continuar el proceso.
128	<code>vista.presentacion.idPresentacion(), producto.nombre(),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	<code>vista.presentacion.nombre(), vista.presentacion.precio(), opciones, 1);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	<code>dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	<code>} catch (IllegalArgumentException ex) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
132	<code>JOptionPane.showMessageDialog(this, ex.getMessage(),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	<code>"Completa el producto", JOptionPane.WARNING_MESSAGE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	<code>}</code>	Cierra el bloque abierto anteriormente.
135	<code>}</code>	Cierra el bloque abierto anteriormente.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	<code>private final class VistaPresentacion extends JPanel {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
138	<code>private final PresentacionMenu presentacion;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
139	<code>private final List<SelectorGrupo> selectores = new ArrayList<>();</code>	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	<code>VistaPresentacion(PresentacionMenu presentacion) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
142	<code>super(new BorderLayout());</code>	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
143	<code>this.presentacion = presentacion;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
144	<code>setBackground(Color.WHITE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
145	<code>JPanel contenido = new JPanel();</code>	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
146	<code>contenido.setBackground(Color.WHITE);</code>	Configura una propiedad mediante setBackground en el objeto indicado.
147	<code>contenido.setLayout(new BoxLayout(contenido, BoxLayout.Y_AXIS));</code>	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
148	<code>contenido.setBorder(new EmptyBorder(18, 22, 18, 22));</code>	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.

149	for (GrupoMenu grupo : presentacion.grupos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
150	SelectorGrupo selector = new SelectorGrupo(grupo);	Crea una nueva instancia de SelectorGrupo y la usa o guarda para continuar el proceso.
151	selectores.add(selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152	if (grupo.visible() selector.tienePersonalizaciones()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
153	contenido.add(selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154	contenido.add(Box.createVerticalStrut(14));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156	}	Cierra el bloque abierto anteriormente.
157	JScrollPane scroll = new JScrollPane(contenido);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
158	Componentes.DesplazamientoSuave.ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	scroll.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
160	scroll.getVerticalScrollBar().setUnitIncrement(22);	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
161	scroll.getViewport().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
162	add(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	}	Cierra el bloque abierto anteriormente.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165	BigDecimal precioActual() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	BigDecimal total = presentacion.precio();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
167	for (SelectorGrupo selector : selectores) total = total.add(selector.precioExtra());	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
168	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
169	}	Cierra el bloque abierto anteriormente.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	List<OpcionPedido> crearOpciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	List<OpcionPedido> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
173	for (SelectorGrupo selector : selectores) resultado.addAll(selector.crearOpciones());	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
174	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
175	}	Cierra el bloque abierto anteriormente.
176	}	Cierra el bloque abierto anteriormente.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	private final class SelectorGrupo extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
179	private final GrupoMenu grupo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
180	private final JPanel opciones = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
181	private final JPanel personalizaciones = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
182	private final List<AbstractButton> botones = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
183	private final List<JComboBox<OpcionMenu>> listas = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
184	private final Map<String, PersonalizacionProducto> panelesProducto = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
185		Línea en blanco: separa visualmente bloques para que el código sea más legible.
186	SelectorGrupo(GrupoMenu grupo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	super(new BorderLayout(0, 9));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
188	this.grupo = grupo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
189	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
190	setBorder(BorderFactory.createCompoundBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	BorderFactory.createLineBorder(BORDE, 1, true),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	new EmptyBorder(14, 16, 14, 16)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.

193	JLabel titulo = new JLabel(grupo.nombre().startsWith("__") ? "" : grupo.nombre());	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
194	titulo.setFont(new Font("SansSerif", Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
195	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
196	String regla = grupo.minimo() == grupo.maximo()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
197	? grupo.minimo() + (grupo.minimo() == 1 ? " requerido" : " requeridos")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198	: "Elige de " + grupo.minimo() + " a " + grupo.maximo();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
199	JLabel detalle = new JLabel(regla);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
200	detalle.setForeground(TEXT0_SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
201	detalle.setFont(new Font("SansSerif", Font.PLAIN, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
202	JPanel encabezado = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
203	encabezado.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
204	encabezado.setLayout(new BorderLayout(encabezado, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
205	encabezado.add(titulo); encabezado.add(detalle);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	if (grupo.visible()) add(encabezado, BorderLayout.NORTH);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	opciones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
209	opciones.setLayout(new BorderLayout(opciones, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
210	crearControles();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
211	personalizaciones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
212	personalizaciones.setLayout(new BorderLayout(personalizaciones, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
213	JPanel centro = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
214	centro.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
215	centro.setLayout(new BorderLayout(centro, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
216	centro.add(opciones); centro.add(personalizaciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
217	add(centro, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218	reconstruirPersonalizaciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219	}	Cierra el bloque abierto anteriormente.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	private void crearControles() {	Declara el método crearControles. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
222	if (!grupo.visible()) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
223	if (grupo.permiteRepetir() && grupo.maximo() > 1) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
224	for (int i=0; i<grupo.maximo(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
225	JPanel fila = new JPanel(new BorderLayout(10,0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
226	fila.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
227	JLabel etiqueta = new JLabel("Producto " + (i+1));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
228	etiqueta.setPreferredSize(new Dimension(95, 34));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
229	JComboBox<OpcionMenu> lista = new JComboBox<>();	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
230	if (i >= grupo.minimo()) lista.addItem(null);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
231	for (OpcionMenu opcion : grupo.opciones()) lista.addItem(opcion);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
232	if (i < grupo.minimo()) seleccionarPredeterminada(lista);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
233	else lista.setSelectedIndex(0);	Ruta alternativa que se ejecuta cuando la condición anterior fue falsa.
234	lista.addActionListener(e -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
235	listas.add(lista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236	fila.add(etiqueta, BorderLayout.WEST); fila.add(lista, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

237	opciones.add(fila); opciones.add(Box.createVerticalStrut(7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238	}	Cierra el bloque abierto anteriormente.
239	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
240	ButtonGroup radios = grupo.maximo() == 1 ? new ButtonGroup() : null;	Crea una nueva instancia de ButtonGroup y la usa o guarda para continuar el proceso.
241	if (radios != null && grupo.minimo() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
242	JRadioButton ninguna = new JRadioButton("No agregar");	Crea una nueva instancia de JRadioButton y la usa o guarda para continuar el proceso.
243	ninguna.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
244	ninguna.setSelected(true);	Configura una propiedad mediante setSelected en el objeto indicado.
245	ninguna.addActionListener(e -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
246	radios.add(ninguna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247	opciones.add(ninguna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
248	}	Cierra el bloque abierto anteriormente.
249	for (OpcionMenu opcion : grupo.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
250	AbstractButton control = radios == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
251	? new JCheckBox(opcion.toString()) : new JRadioButton(opcion.toString());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
252	control.putClientProperty("opcion", opcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
253	control.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
254	control.setFont(new Font("SansSerif", Font.PLAIN, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
255	control.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
256	control.setSelected(opcion.predeterminada());	Configura una propiedad mediante setSelected en el objeto indicado.
257	control.addActionListener(e -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
258	if (radios != null) radios.add(control);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
259	botones.add(control); opciones.add(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
260	}	Cierra el bloque abierto anteriormente.
261	if (grupo.minimo() > 0 && botones.stream().noneMatch(AbstractButton::isSelected)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
262	&& !botones.isEmpty()) botones.get(0).setSelected(true);	Configura una propiedad mediante setSelected en el objeto indicado.
263	}	Cierra el bloque abierto anteriormente.
264	}	Cierra el bloque abierto anteriormente.
265		Línea en blanco: separa visualmente bloques para que el código sea más legible.
266	private void seleccionarPredeterminada(JComboBox<OpcionMenu> lista) {	Declara el método seleccionarPredeterminada. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
267	for (int i=0; i<lista.getItemCount(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
268	OpcionMenu opcion = lista.getItemAt(i);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
269	if (opcion != null && opcion.predeterminada()) { lista.setSelectedIndex(i); return; }	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
270	}	Cierra el bloque abierto anteriormente.
271	if (lista.getItemCount() > 0 && grupo.minimo() > 0) lista.setSelectedIndex(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
272	}	Cierra el bloque abierto anteriormente.
273		Línea en blanco: separa visualmente bloques para que el código sea más legible.
274	private void cambioSeleccion() {	Declara el método cambioSeleccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
275	reconstruirPersonalizaciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
276	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
277	}	Cierra el bloque abierto anteriormente.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	private List<Seleccion> seleccionadas() {	Declara el método seleccionadas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
280	List<Seleccion> seleccionadas = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.

281	if (!grupo.visible()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
282	int numero = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
283	List<OpcionMenu> defaults = grupo.opciones().stream().filter(OpcionMenu::predeterminada).toList();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
284	});	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
285	if (defaults.isEmpty() && !grupo.opciones().isEmpty()) defaults = List.of(grupo.opciones().get(0));	
286	for (OpcionMenu opcion : defaults) seleccionadas.add(new Seleccion(numero++, opcion));	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
287	} else if (!listas.isEmpty()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
288	for (int i=0; i<listas.size(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
289	OpcionMenu opcion = (OpcionMenu) listas.get(i).getSelectedItem();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
290	if (opcion != null) seleccionadas.add(new Seleccion(i+1, opcion));	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
291	}	Cierra el bloque abierto anteriormente.
292	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
293	int numero = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
294	for (AbstractButton boton : botones) if (boton.isSelected())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
295	seleccionadas.add(new Seleccion(numero++, (OpcionMenu) boton.getClientProperty("opcion")));	Crea una nueva instancia de Seleccion y la usa o guarda para continuar el proceso.
296	}	Cierra el bloque abierto anteriormente.
297	return seleccionadas;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
298	}	Cierra el bloque abierto anteriormente.
299	private void reconstruirPersonalizaciones() {	Línea en blanco: separa visualmente bloques para que el código sea más legible.
300	personalizaciones.removeAll();	Declara el método reconstruirPersonalizaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
301	if (grupo.permitePersonalizar()) {	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	for (Seleccion seleccion : seleccionadas()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
303	int indiceComponente = 0;	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
304	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
305	for (int unidad=1; unidad<=componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
306	String clave = seleccion.numero + ":" + seleccion.opcion.idOpcion()	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
307	+ ":" + indiceComponente + ":" + unidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
308	PersonalizacionProducto panel = panelesProducto.get(clave);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
309	if (panel == null) {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
310	panel = new PersonalizacionProducto(componente, unidad, componente.cantidad());	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
311	panelesProducto.put(clave, panel);	Crea una nueva instancia de PersonalizacionProducto y la usa o guarda para continuar el proceso.
312	}	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313	if (panel.tieneControles()) {	Cierra el bloque abierto anteriormente.
314	personalizaciones.add(Box.createVerticalStrut(8));	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
315	personalizaciones.add(panel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
316	}	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317	}	Cierra el bloque abierto anteriormente.
318	indiceComponente++;	Cierra el bloque abierto anteriormente.
319	}	Ejecuta una instrucción Java completa; el punto y coma marca su final.
320	}	Cierra el bloque abierto anteriormente.
321	}	Cierra el bloque abierto anteriormente.
322	personalizaciones.revalidate(); personalizaciones.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

323	}	Cierra el bloque abierto anteriormente.
324		Línea en blanco: separa visualmente bloques para que el código sea más legible.
325	private boolean tienePersonalizaciones() {	Declara el método tienePersonalizaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
326	return personalizaciones.getComponentCount() > 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
327	}	Cierra el bloque abierto anteriormente.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	BigDecimal precioExtra() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	BigDecimal total = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
331	for (Seleccion seleccion : seleccionadas()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
332	total = total.add(seleccion.opcion.incrementoPrecio());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
333	int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
334	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
335	for (int unidad=1; unidad<=componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
336	PersonalizacionProducto panel = panelesProducto.get(seleccion.numero + ":"	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
337	+ seleccion.opcion.idOpcion() + ":" + indice + ":" + unidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
338	if (panel != null) total = total.add(panel.precioExtra());	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
339	}	Cierra el bloque abierto anteriormente.
340	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
341	}	Cierra el bloque abierto anteriormente.
342	}	Cierra el bloque abierto anteriormente.
343	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
344	}	Cierra el bloque abierto anteriormente.
345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
346	List<OpcionPedido> crearOpciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
347	List<Seleccion> elegidas = seleccionadas();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
348	if (elegidas.size() < grupo.minimo() elegidas.size() > grupo.maximo())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
349	throw new IllegalArgumentException("En " + grupo.nombre() + " debes elegir de "	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
350	+ grupo.minimo() + " a " + grupo.maximo() + " opciones.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
351	if (!grupo.permiteRepetir()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
352	long diferentes = elegidas.stream().map(s -> s.opcion.idOpcion()).distinct().count();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
353	if (diferentes != elegidas.size())	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
354	throw new IllegalArgumentException("No puedes repetir una opción en " + grupo.nombre() + " "	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
355	.");	
356	}	Cierra el bloque abierto anteriormente.
357	List<OpcionPedido> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
358	for (Seleccion seleccion : elegidas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
359	List<ProductoPedido> productos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
360	int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
361	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
362	for (int unidad=1; unidad<=componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
363	String clave = seleccion.numero + ":" + seleccion.opcion.idOpcion()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
364	+ ":" + indice + ":" + unidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
365	PersonalizacionProducto panel = panelesProducto.get(clave);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
	if (panel == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

366	panel = new PersonalizacionProducto(componente, unidad, componente.cantidad());	Crea una nueva instancia de PersonalizacionProducto y la usa o guarda para continuar el proceso.
367	panelesProducto.put(clave, panel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368	}	Cierra el bloque abierto anteriormente.
369	productos.add(new ProductoPedido(UUID.randomUUID().toString(),	Crea una nueva instancia de ProductoPedido y la usa o guarda para continuar el proceso.
370	componente.idProducto(), componente.nombre(), 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
371	componente.tipoStock(), panel.modificaciones()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
372	}	Cierra el bloque abierto anteriormente.
373	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
374	}	Cierra el bloque abierto anteriormente.
375	BigDecimal precioModificaciones = productos.stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
376	.flatMap(p -> p.modificaciones().stream())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
377	.map(ModificacionPedido::precioExtra)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
378	.reduce(BigDecimal.ZERO, BigDecimal::add);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
379	resultado.add(new OpcionPedido(grupo.idGrupo(), seleccion.opcion.idOpcion(),	Crea una nueva instancia de OpcionPedido y la usa o guarda para continuar el proceso.
380	seleccion.numero, grupo.visible() ? grupo.nombre() : "__Producto principal",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
381	seleccion.opcion.nombre(), seleccion.opcion.incrementoPrecio(), productos));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
382	}	Cierra el bloque abierto anteriormente.
383	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
384	}	Cierra el bloque abierto anteriormente.
385	}	Cierra el bloque abierto anteriormente.
386		Línea en blanco: separa visualmente bloques para que el código sea más legible.
387	private final class PersonalizacionProducto extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
388	private final ComponenteMenu componente;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
389	private final Map<IngredienteProducto, JCheckBox> quitar = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
390	private final Map<IngredienteProducto, JSpinner> extras = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
391		Línea en blanco: separa visualmente bloques para que el código sea más legible.
392	PersonalizacionProducto(ComponenteMenu componente, int unidad, int total) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
393	this.componente = componente;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
394	setOpaque(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
395	setBackground(new Color(249, 250, 252));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
396	setBorder(BorderFactory.createCompoundBorder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
397	BorderFactory.createLineBorder(new Color(235,237,241),1,true),	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
398	new EmptyBorder(10,12,10,12)));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
399	setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
400	JLabel titulo = new JLabel(componente.nombre()	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
401	+ (total > 1 ? " " + unidad : "") + " - personalización");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
402	titulo.setFont(new Font("SansSerif", Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
403	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
404	add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
405	for (IngredienteProducto ingrediente : componente.ingredientes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
406	if (!ingrediente.permiteQuitar() && !ingrediente.permiteExtra()) continue;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
407	JPanel fila = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 2));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
408	fila.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
409	JCheckBox sinControl = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

410	JSeparator extraControl = null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
411	if (ingrediente.permiteQuitar()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
412	JCheckBox sin = new JCheckBox("Sin " + ingrediente.nombre());	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
413	sin.setOpaque(false); quitar.put(ingrediente, sin); fila.add(sin);	Configura una propiedad mediante setOpaque en el objeto indicado.
414	sinControl = sin;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
415	}	Cierra el bloque abierto anteriormente.
416	if (ingrediente.permiteExtra()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
417	fila.add(new JLabel("Extra " + ingrediente.nombre()));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
418	JSeparator veces = new JSeparator(new SpinnerNumberModel(0,0,ingrediente.maxExtras(),1));	Crea una nueva instancia de JSeparator y la usa o guarda para continuar el proceso.
419	veces.setPreferredSize(new Dimension(62,28));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
420	veces.addChangeListener(e -> actualizarPrecio());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
421	extras.put(ingrediente, veces); fila.add(veces);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
422	extraControl = veces;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
423	if (ingrediente.precioExtra().signum() > 0)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
424	fila.add(new JLabel("+ Q" + ingrediente.precioExtra().setScale(2).toString() + " c/u"));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
425	}	Cierra el bloque abierto anteriormente.
426	if (sinControl != null && extraControl != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
427	JCheckBox sin = sinControl;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
428	JSeparator veces = extraControl;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
429	sin.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
430	if (sin.isSelected()) veces.setValue(0);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
431	veces.setEnabled(!sin.isSelected());	Habilita o deshabilita la interacción con el componente.
432	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433	}	Cierra el bloque abierto anteriormente.
434	add(fila);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
435	}	Cierra el bloque abierto anteriormente.
436	}	Cierra el bloque abierto anteriormente.
437		Línea en blanco: separa visualmente bloques para que el código sea más legible.
438	boolean tieneControles() { return !quitar.isEmpty() !extras.isEmpty(); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439		Línea en blanco: separa visualmente bloques para que el código sea más legible.
440	BigDecimal precioExtra() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
441	BigDecimal total = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
442	for (Map.Entry<IngredienteProducto, JSeparator> entrada : extras.entrySet())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
443	total = total.add(entrada.getKey().precioExtra()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
444	.multiply(BigDecimal.valueOf(((Number) entrada.getValue().getValue()).intValue())));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
445	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
446	}	Cierra el bloque abierto anteriormente.
447		Línea en blanco: separa visualmente bloques para que el código sea más legible.
448	List<ModificacionPedido> modificaciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
449	List<ModificacionPedido> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
450	for (Map.Entry<IngredienteProducto, JCheckBox> entrada : quitar.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
451	IngredienteProducto i = entrada.getKey();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
452	if (entrada.getValue().isSelected()) resultado.add(new ModificacionPedido(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
453	i.idProductoIngrediente(), i.idIngrediente(), "SIN", i.nombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

454	i.cantidadDefault(), 1, BigDecimal.ZERO));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
455	}	Cierra el bloque abierto anteriormente.
456	for (Map.Entry<IngredienteProducto, JSpinner> entrada : extras.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
457	IngredienteProducto i = entrada.getKey();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
458	int veces = ((Number) entrada.getValue()).intValue();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
459	if (veces > 0) resultado.add(new ModificacionPedido(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
460	i.idProductoIngrediente(), i.idIngrediente(), "EXTRA", i.nombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461	i.cantidadExtra().multiply(BigDecimal.valueOf(veces)), veces,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
462	i.precioExtra().multiply(BigDecimal.valueOf(veces))));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
463	}	Cierra el bloque abierto anteriormente.
464	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
465	}	Cierra el bloque abierto anteriormente.
466	}	Cierra el bloque abierto anteriormente.
467		Línea en blanco: separa visualmente bloques para que el código sea más legible.
468	private record Seleccion(int numero, OpcionMenu opcion) {}	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
469	}	Cierra el bloque abierto anteriormente.

16.94 src/GUI_CAJERO/ConfiguradorProductoPanel.java

Constructor visual de un producto: presentación Individual/WIMenú/Combo, grupos de opciones y cambios internos por ingrediente.

Tipo: java | Líneas: 1045 | Archivo 94 de 157

Clases/tipos detectados: ConfiguradorProductoPanel, BotonPresentacion, VistaPresentacion, SelectorGrupo, PersonalizacionProducto, SelectorExtra, IconoSeleccion, BotonPaso, Seleccion

Línea	Constructor o método detectado
87	public ConfiguradorProductoPanel(ConfiguracionProducto producto,
102	private void construirInterfaz() {
116	private JComponent crearCabecera() {
154	private JComponent crearBarraPresentaciones() {
182	private int ordenPresentacion(PresentacionMenu presentacion) {
192	private String tituloPresentacion(PresentacionMenu presentacion) {
202	private String clave(PresentacionMenu presentacion) {
206	private void seleccionarPresentacion(PresentacionMenu presentacion) {
217	private JComponent crearPie() {
257	private JButton botonCantidad(String texto) {
262	private void cambiarCantidad(int cambio) {
268	private VistaPresentacion vistaActual() {
273	private void actualizarPrecio() {
283	private void confirmar() {
301	private static String escaparHtml(String texto) {
330	protected void paintComponent(Graphics graphics) {
429	public Dimension getMaximumSize() {
434	private JComponent crearEncabezadoGrupo() {
453	private String textoRegla(GrupeMenu grupo) {
465	private void crearControles() {
505	private void crearListasRepetibles() {
539	private void seleccionarPredeterminada(JComboBox<OpcionMenu> lista) {
552	private void cambioSeleccion() {
558	private List<Seleccion> seleccionadas() {
589	private void reconstruirPersonalizaciones() {
616	private String llave(Seleccion seleccion, int indiceComponente, int unidad) {
621	private boolean tienePersonalizaciones() {
761	public Dimension getMaximumSize() {
766	private JLabel tituloSeccion(String texto) {

Línea	Constructor o método detectado
774	private JLabel descripcionSeccion(String texto) {
864	private JButton botonExtra(String texto) {
869	private void cambiar(int valor) {
877	private void reiniciar() {
882	private void setControlesHabilitados(boolean habilitado) {
888	private void actualizarEstado() {
895	private JCheckBox casilla(String texto, boolean seleccionada) {
903	private JRadioButton radio(String texto) {
911	private void prepararControl(AbstractButton control) {
931	public int getIconWidth() {
936	public int getIconHeight() {
941	public void paintIcon(Component componente, Graphics graphics, int x, int y) {
1003	protected void paintComponent(Graphics graphics) {
1034	private static Color aclarar(Color color, int cantidad) {
1043	private record Seleccion(int numero, OpcionMenu opcion) {

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.ComponenteMenu;	Importa Modelos.ComponenteMenu para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.ConfiguracionProducto;	Importa Modelos.ConfiguracionProducto para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.GrupoMenu;	Importa Modelos.GrupoMenu para poder usar esa clase sin escribir su nombre completo.
7	import Modelos.IngredienteProducto;	Importa Modelos.IngredienteProducto para poder usar esa clase sin escribir su nombre completo.
8	import Modelos.LineaPedido;	Importa Modelos.LineaPedido para poder usar esa clase sin escribir su nombre completo.
9	import Modelos.ModificacionPedido;	Importa Modelos.ModificacionPedido para poder usar esa clase sin escribir su nombre completo.
10	import Modelos.OpcionMenu;	Importa Modelos.OpcionMenu para poder usar esa clase sin escribir su nombre completo.
11	import Modelos.OpcionPedido;	Importa Modelos.OpcionPedido para poder usar esa clase sin escribir su nombre completo.
12	import Modelos.PresentacionMenu;	Importa Modelos.PresentacionMenu para poder usar esa clase sin escribir su nombre completo.
13	import Modelos.ProductoPedido;	Importa Modelos.ProductoPedido para poder usar esa clase sin escribir su nombre completo.
14	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
15	import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
16	import java.awt.CardLayout;	Importa java.awt.CardLayout para poder usar esa clase sin escribir su nombre completo.
17	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
18	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
19	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
20	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
21	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
22	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.

23 import java.awt.FontMetrics;	Importa java.awt.FontMetrics para poder usar esa clase sin escribir su nombre completo.
24 import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
25 import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
26 import java.awt.GridLayout;	Importa java.awt.GridLayout para poder usar esa clase sin escribir su nombre completo.
27 import java.awt.Insets;	Importa java.awt.Insets para poder usar esa clase sin escribir su nombre completo.
28 import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
29 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
30 import java.util.ArrayList;	Importa java.util.ArrayList para poder usar esa clase sin escribir su nombre completo.
31 import java.util.Comparator;	Importa java.util.Comparator para poder usar esa clase sin escribir su nombre completo.
32 import java.util.LinkedHashMap;	Importa java.util.LinkedHashMap para poder usar esa clase sin escribir su nombre completo.
33 import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
34 import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
35 import java.util.UUID;	Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.
36 import java.util.function.Consumer;	Importa java.util.function.Consumer para poder usar esa clase sin escribir su nombre completo.
37 import javax.swing.AbstractButton;	Importa javax.swing.AbstractButton para poder usar esa clase sin escribir su nombre completo.
38 import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
39 import javax.swing.Box;	Importa javax.swing.Box para poder usar esa clase sin escribir su nombre completo.
40 import javax.swing.BoxLayout;	Importa javax.swing.BoxLayout para poder usar esa clase sin escribir su nombre completo.
41 import javax.swing.ButtonGroup;	Importa javax.swing.ButtonGroup para poder usar esa clase sin escribir su nombre completo.
42 import javax.swing.Icon;	Importa javax.swing.Icon para poder usar esa clase sin escribir su nombre completo.
43 import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
44 import javax.swing.JCheckBox;	Importa javax.swing.JCheckBox para poder usar esa clase sin escribir su nombre completo.
45 import javax.swing.JComboBox;	Importa javax.swing.JComboBox para poder usar esa clase sin escribir su nombre completo.
46 import javax.swing.JComponent;	Importa javax.swing.JComponent para poder usar esa clase sin escribir su nombre completo.
47 import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
48 import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
49 import javax.swing.JRadioButton;	Importa javax.swing.JRadioButton para poder usar esa clase sin escribir su nombre completo.
50 import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
51 import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
52 import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
53 import javax.swing.border.EmptyBorder;	Importa javax.swing.border.EmptyBorder para poder usar esa clase sin escribir su nombre completo.
54	Línea en blanco: separa visualmente bloques para que el código sea más legible.
55 /**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
56 * Configurador incrustable para el panel derecho del cajero. Las	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
57 * presentaciones, grupos y opciones siguen viniendo completamente de la base	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
58 * de datos; este componente solo controla su aspecto y construye la línea que	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
59 * se agregará al pedido.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
60 */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
61 public final class ConfiguradorProductoPanel extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
62	Línea en blanco: separa visualmente bloques para que el código sea más legible.
63 private static final Color AZUL = new Color(1, 20, 36);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
64 private static final Color AMARILLO = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
65 private static final Color AMARILLO_SUAVE = new Color(255, 248, 225);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
66 private static final Color TEXT0 = new Color(27, 30, 34);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

67	private static final Color TEXTO_SECUNDARIO = new Color(112, 116, 123);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
68	private static final Color BORDE = new Color(229, 231, 235);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
69	private static final Color FONDO_CONTROL = new Color(248, 249, 251);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
70	private static final Color ERROR = new Color(196, 30, 42);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
71	private static final String FUENTE = "Arial";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	private final ConfiguracionProducto producto;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
74	private final Consumer<LineaPedido> alAgregar;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
75	private final Runnable alCancelar;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
76	private final CardLayout tarjetasPresentacion = new CardLayout();	Crea una nueva instancia de CardLayout y la usa o guarda para continuar el proceso.
77	private final JPanel contenidoPresentacion = new JPanel(tarjetasPresentacion);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
78	private final JPanel barraPresentaciones = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
79	private final Map<Integer, VistaPresentacion> vistas = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
80	private final Map<Integer, BotonPresentacion> botonesPresentacion = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
81	private final JLabel cantidadTexto = new JLabel("1", SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
82	private final JLabel error = new JLabel(" ");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
83	private final BotonRedondeado agregar = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
84	private PresentacionMenu presentacionActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
85	private int cantidad = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	public ConfiguradorProductoPanel(ConfiguracionProducto producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	Consumer<LineaPedido> alAgregar, Runnable alCancelar) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
89	super(new BorderLayout(0, 12));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
90	if (producto == null producto.presentaciones().isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
91	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
92	"El producto no tiene una presentación activa para venderse.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	}	Cierra el bloque abierto anteriormente.
94	this.producto = producto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
95	this.alAgregar = alAgregar;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
96	this.alCancelar = alCancelar;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
97	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	setBorder(new EmptyBorder(18, 16, 18, 16));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
99	construirInterfaz();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	private void construirInterfaz() {	Declara el método construirInterfaz. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
103	add(crearCabecera(), BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	JPanel centro = new JPanel(new BorderLayout(0, 6));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
106	centro.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
107	centro.add(crearBarraPresentaciones(), BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
108	contenidoPresentacion.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
109	centro.add(contenidoPresentacion, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	add(centro, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	add(crearPie(), BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
113	SwingUtilities.invokeLater(this::actualizarPrecio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	}	Cierra el bloque abierto anteriormente.
115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
116	private JComponent crearCabecera() {	Declara el método crearCabecera. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
117	JPanel cabecera = new JPanel(new BorderLayout(8, 8));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
118	cabecera.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	JButton volver = new JButton("< Volver al pedido");	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
121	volver.setFont(new Font(FUENTE, Font.BOLD, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
122	volver.setForeground(TEXTO_SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
123	volver.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
124	volver.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
125	volver.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
126	volver.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
127	volver.setHorizontalAlignment(SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
128	volver.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Configura una propiedad mediante setCursor en el objeto indicado.
129	volver.setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
130	volver.addActionListener(evento -> alCancelar.run());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
131	cabecera.add(volver, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	JPanel textos = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
134	textos.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
135	textos.setLayout(new BoxLayout(textos, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
136	JLabel titulo = new JLabel(producto.nombre());	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
137	titulo.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
138	titulo.setFont(new Font(FUENTE, Font.BOLD, 21));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
139	titulo.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
140	JLabel ayuda = new JLabel("<html>" + escaparHtml(producto.descripcion() == null producto.descripcion().isBlank() ? "Personaliza el producto antes de añadirlo." : producto.descripcion()) + "</html>");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
141		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
142		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
143		Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	ayuda.setForeground(TEXTO_SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
145	ayuda.setFont(new Font(FUENTE, Font.PLAIN, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
146	ayuda.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
147	textos.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	textos.add(Box.createVerticalStrut(4));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	textos.add(ayuda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
150	cabecera.add(textos, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151	return cabecera;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
152	}	Cierra el bloque abierto anteriormente.
153		Línea en blanco: separa visualmente bloques para que el código sea más legible.
154	private JComponent crearBarraPresentaciones() {	Declara el método crearBarraPresentaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

155	List<PresentacionMenu> ordenadas = new ArrayList<>(producto.presentaciones());	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
156	ordenadas.sort(Comparator.comparingInt(this::ordenPresentacion)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	.thenComparingInt(PresentacionMenu::idPresentacion));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	barraPresentaciones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
160	barraPresentaciones.setLayout(new GridLayout(1, ordenadas.size(), 0, 0));	Crea una nueva instancia de GridLayout y la usa o guarda para continuar el proceso.
161	barraPresentaciones.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0, BORDE));	Configura una propiedad mediante setBorder en el objeto indicado.
162	barraPresentaciones.setPreferredSize(new Dimension(0, 50));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	PresentacionMenu predeterminada = ordenadas.stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
165	.filter(PresentacionMenu::predeterminada)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	.findFirst().orElse(ordenadas.get(0));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	for (PresentacionMenu presentacion : ordenadas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
169	VistaPresentacion vista = new VistaPresentacion(presentacion);	Crea una nueva instancia de VistaPresentacion y la usa o guarda para continuar el proceso.
170	vistas.put(presentacion.idPresentacion(), vista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
171	contenidoPresentacion.add(vista, clave(presentacion));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	BotonPresentacion boton = new BotonPresentacion(	Crea una nueva instancia de BotonPresentacion y la usa o guarda para continuar el proceso.
174	tituloPresentacion(presentacion), presentacion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
175	botonesPresentacion.put(presentacion.idPresentacion(), boton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
176	barraPresentaciones.add(boton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
177	}	Cierra el bloque abierto anteriormente.
178	seleccionarPresentacion(predeterminada);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	return barraPresentaciones;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
180	}	Cierra el bloque abierto anteriormente.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	private int ordenPresentacion(PresentacionMenu presentacion) {	Declara el método ordenPresentacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
183	return switch (presentacion.tipo().toUpperCase()) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
184	case "MENU" -> 0;	Define una de las rutas posibles dentro del switch.
185	case "INDIVIDUAL" -> 1;	Define una de las rutas posibles dentro del switch.
186	case "INFANTIL" -> 2;	Define una de las rutas posibles dentro del switch.
187	case "COMBO" -> 3;	Define una de las rutas posibles dentro del switch.
188	default -> 4;	Define una de las rutas posibles dentro del switch.
189	};	Cierra el bloque abierto anteriormente y termina la declaración.
190	}	Cierra el bloque abierto anteriormente.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	private String tituloPresentacion(PresentacionMenu presentacion) {	Declara el método tituloPresentacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
193	if ("MENU".equalsIgnoreCase(presentacion.tipo())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
194	return "WlMenú";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
195	}	Cierra el bloque abierto anteriormente.
196	if ("INDIVIDUAL".equalsIgnoreCase(presentacion.tipo())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
197	return "Individual";	Finaliza el método y devuelve el valor indicado a quien lo llamó.

198	}	Cierra el bloque abierto anteriormente.
199	return presentacion.nombre();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
200	}	Cierra el bloque abierto anteriormente.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	private String clave(PresentacionMenu presentacion) {	Declara el método clave. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
203	return "PRESENTACION_" + presentacion.idPresentacion();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
204	}	Cierra el bloque abierto anteriormente.
205		Línea en blanco: separa visualmente bloques para que el código sea más legible.
206	private void seleccionarPresentacion(PresentacionMenu presentacion) {	Declara el método seleccionarPresentacion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
207	presentacionActual = presentacion;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
208	tarjetasPresentacion.show(contenidoPresentacion, clave(presentacion));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209	for (BotonPresentacion boton : botonesPresentacion.values()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
210	boton.setSeleccionado(boton.presentacion.idPresentacion()	Configura una propiedad mediante setSeleccionado en el objeto indicado.
211	== presentacion.idPresentacion());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
212	}	Cierra el bloque abierto anteriormente.
213	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
214	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215	}	Cierra el bloque abierto anteriormente.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	private JComponent crearPie() {	Declara el método crearPie. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
218	JPanel pie = new JPanel(new BorderLayout(0, 7));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
219	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	error.setForeground(ERROR);	Configura una propiedad mediante setForeground en el objeto indicado.
222	error.setFont(new Font(FUENTE, Font.BOLD, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
223	pie.add(error, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
224		Línea en blanco: separa visualmente bloques para que el código sea más legible.
225	JPanel acciones = new JPanel(new BorderLayout(12, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
226	acciones.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	JPanel selectorCantidad = new JPanel(new FlowLayout(FlowLayout.LEFT, 7, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
229	selectorCantidad.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
230	JButton menos = botonCantidad("-");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
231	JButton mas = botonCantidad("+");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
232	cantidadTexto.setFont(new Font(FUENTE, Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
233	cantidadTexto.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
234	cantidadTexto.setPreferredSize(new Dimension(22, 42));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
235	menos.addActionListener(evento -> cambiarCantidad(-1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
236	mas.addActionListener(evento -> cambiarCantidad(1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
237	selectorCantidad.add(menos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238	selectorCantidad.add(cantidadTexto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
239	selectorCantidad.add(mas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240	acciones.add(selectorCantidad, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.

242	agregar.setFont(new Font(FUENTE, Font.BOLD, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
243	agregar.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
244	agregar.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
245	agregar.setColorInicio(AMARILLO);	Configura una propiedad mediante setColorInicio en el objeto indicado.
246	agregar.setColorFinal(AMARILLO);	Configura una propiedad mediante setColorFinal en el objeto indicado.
247	agregar.setColorBorde(AMARILLO);	Configura una propiedad mediante setColorBorde en el objeto indicado.
248	agregar.setGrosorBorde(0f);	Configura una propiedad mediante setGrosorBorde en el objeto indicado.
249	agregar.setRadio(17);	Configura una propiedad mediante setRadio en el objeto indicado.
250	agregar.setPreferredSize(new Dimension(250, 48));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
251	agregar.addActionListener(evento -> confirmar());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
252	acciones.add(agregar, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
253	pie.add(acciones, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254	return pie;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
255	}	Cierra el bloque abierto anteriormente.
256		Línea en blanco: separa visualmente bloques para que el código sea más legible.
257	private JButton botonCantidad(String texto) {	Declara el método botonCantidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
258	return new BotonPaso(texto, AMARILLO_SUAVE,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
259	new Color(244, 223, 157), AZUL, 42, 38, 12, 22);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
260	}	Cierra el bloque abierto anteriormente.
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	private void cambiarCantidad(int cambio) {	Declara el método cambiarCantidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
263	cantidad = Math.max(1, Math.min(99, cantidad + cambio));	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
264	cantidadTexto.setText(String.valueOf(cantidad));	Cambia el texto visible de un componente de la interfaz.
265	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266	}	Cierra el bloque abierto anteriormente.
267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
268	private VistaPresentacion vistaActual() {	Declara el método vistaActual. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
269	return presentacionActual == null	Finaliza el método y devuelve el valor indicado a quien lo llamó.
270	? null : vistas.get(presentacionActual.idPresentacion());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
271	}	Cierra el bloque abierto anteriormente.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	private void actualizarPrecio() {	Declara el método actualizarPrecio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
274	VistaPresentacion vista = vistaActual();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
275	if (vista == null agregar == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
276	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
277	}	Cierra el bloque abierto anteriormente.
278	BigDecimal total = vista.precioActual()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
279	.multiply(BigDecimal.valueOf(cantidad)).setScale(2);	Configura una propiedad mediante setScale en el objeto indicado.
280	agregar.setText("Añadir al carrito Q" + total.toPlainString());	Cambia el texto visible de un componente de la interfaz.
281	}	Cierra el bloque abierto anteriormente.
282		Línea en blanco: separa visualmente bloques para que el código sea más legible.
283	private void confirmar() {	Declara el método confirmar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
284	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
285	VistaPresentacion vista = vistaActual();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

286	if (vista == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
287	throw new IllegalArgumentException("Selecciona una presentación.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
288	}	Cierra el bloque abierto anteriormente.
289	List<OpcionPedido> opciones = vista.crearOpciones();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
290	LineaPedido linea = new LineaPedido(UUID.randomUUID().toString(),	Crea una nueva instancia de LineaPedido y la usa o guarda para continuar el proceso.
291	producto.idProducto(), vista.presentacion.idPresentacion(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
292	producto.nombre(), vista.presentacion.nombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
293	vista.presentacion.precio(), opciones, cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
294	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
295	alAgregar.accept(linea);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
296	} catch (IllegalArgumentException excepcion) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
297	error.setText("<html>" + escaparHtml(excepcion.getMessage()) + "</html>");	Cambia el texto visible de un componente de la interfaz.
298	}	Cierra el bloque abierto anteriormente.
299	}	Cierra el bloque abierto anteriormente.
300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
301	private static String escaparHtml(String texto) {	Declara el método escaparHtml. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
302	return texto == null ? "" : texto.replace("&", "&")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
303	.replace("<", "<").replace(">", ">");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
304	}	Cierra el bloque abierto anteriormente.
305		Línea en blanco: separa visualmente bloques para que el código sea más legible.
306	private final class BotonPresentacion extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
307	private final PresentacionMenu presentacion;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
308	private boolean seleccionado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
309		Línea en blanco: separa visualmente bloques para que el código sea más legible.
310	BotonPresentacion(String texto, PresentacionMenu presentacion) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	super(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
312	this.presentacion = presentacion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
313	setFont(new Font(FUENTE, Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
314	setForeground(TEXT0_SECUNDARIO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
315	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
316	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
318	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
319	setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
320	addActionListener(evento -> seleccionarPresentacion(presentacion));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321	}	Cierra el bloque abierto anteriormente.
322		Línea en blanco: separa visualmente bloques para que el código sea más legible.
323	void setSeleccionado(boolean valor) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
324	seleccionado = valor;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
325	setForeground(valor ? TEXT0 : new Color(165, 166, 169));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
326	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
327	}	Cierra el bloque abierto anteriormente.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.

330	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
331	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
332	if (!seleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
333	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
334	}	Cierra el bloque abierto anteriormente.
335	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
336	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
337	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
338	g2.setColor(AMARILLO);	Configura una propiedad mediante setColor en el objeto indicado.
339	g2.fillRoundRect(0, getHeight() - 4, getWidth(), 4, 4, 4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
340	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
341	}	Cierra el bloque abierto anteriormente.
342	}	Cierra el bloque abierto anteriormente.
343		Línea en blanco: separa visualmente bloques para que el código sea más legible.
344	private final class VistaPresentacion extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
345	private final PresentacionMenu presentacion;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
346	private final List<SelectorGrupo> selectores = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348	VistaPresentacion(PresentacionMenu presentacion) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
349	super(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
350	this.presentacion = presentacion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
351	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
352		Línea en blanco: separa visualmente bloques para que el código sea más legible.
353	JPanel contenido = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
354	contenido.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
355	contenido.setLayout(new BoxLayout(contenido, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
356	contenido.setBorder(new EmptyBorder(16, 8, 18, 8));	Crea una nueva instancia de EmptyBorder y la usa o guarda para continuar el proceso.
357		Línea en blanco: separa visualmente bloques para que el código sea más legible.
358	for (GrupoMenu grupo : presentacion.grupos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
359	SelectorGrupo selector = new SelectorGrupo(grupo);	Crea una nueva instancia de SelectorGrupo y la usa o guarda para continuar el proceso.
360	selectores.add(selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
361	if (grupo.visible() selector.tienePersonalizaciones()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
362	selector.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
363	contenido.add(selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
364	contenido.add(Box.createVerticalStrut(18));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
365	}	Cierra el bloque abierto anteriormente.
366	}	Cierra el bloque abierto anteriormente.
367	contenido.add(Box.createVerticalGlue());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368		Línea en blanco: separa visualmente bloques para que el código sea más legible.
369	JScrollPane scroll = new JScrollPane(contenido);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
370	Componentes.DesplazamientoSuave.ocultarBarras(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
371	scroll.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
372	scroll.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
373	scroll.getViewport().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.

374	<code>scroll.setVerticalScrollBar().setUnitIncrement(22);</code>	Configura una propiedad mediante <code>setUnitIncrement</code> en el objeto indicado.
375	<code>add(scroll, BorderLayout.CENTER);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
376	<code>}</code>	Cierra el bloque abierto anteriormente.
377		Línea en blanco: separa visualmente bloques para que el código sea más legible.
378	<code>BigDecimal precioActual() {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
379	<code>BigDecimal total = presentacion.precio();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
380	<code>for (SelectorGrupo selector : selectores) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
381	<code>total = total.add(selector.precioExtra());</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
382	<code>}</code>	Cierra el bloque abierto anteriormente.
383	<code>return total;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
384	<code>}</code>	Cierra el bloque abierto anteriormente.
385		Línea en blanco: separa visualmente bloques para que el código sea más legible.
386	<code>List<OpcionPedido> crearOpciones() {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	<code>List<OpcionPedido> resultado = new ArrayList<>();</code>	Crea una nueva instancia de <code>ArrayList</code> y la usa o guarda para continuar el proceso.
388	<code>for (SelectorGrupo selector : selectores) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
389	<code>resultado.addAll(selector.crearOpciones());</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
390	<code>}</code>	Cierra el bloque abierto anteriormente.
391	<code>return resultado;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
392	<code>}</code>	Cierra el bloque abierto anteriormente.
393	<code>}</code>	Cierra el bloque abierto anteriormente.
394		Línea en blanco: separa visualmente bloques para que el código sea más legible.
395	<code>private final class SelectorGrupo extends JPanel {</code>	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
396	<code>private final GrupoMenu grupo;</code>	Declara una constante o referencia que no puede reasignarse después de inicializarse.
397	<code>private final JPanel opciones = new JPanel();</code>	Crea una nueva instancia de <code>JPanel</code> y la usa o guarda para continuar el proceso.
398	<code>private final JPanel personalizaciones = new JPanel();</code>	Crea una nueva instancia de <code>JPanel</code> y la usa o guarda para continuar el proceso.
399	<code>private final List<AbstractButton> botones = new ArrayList<>();</code>	Crea una nueva instancia de <code>ArrayList</code> y la usa o guarda para continuar el proceso.
400	<code>private final List<JComboBox<OpcionMenu>> listas = new ArrayList<>();</code>	Crea una nueva instancia de <code>ArrayList</code> y la usa o guarda para continuar el proceso.
401	<code>private final Map<String, PersonalizacionProducto> panelesProducto</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
402	<code>= new LinkedHashMap<>();</code>	Crea una nueva instancia de <code>LinkedHashMap</code> y la usa o guarda para continuar el proceso.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	<code>SelectorGrupo(GrupoMenu grupo) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
405	<code>super(new BorderLayout(0, 10));</code>	Crea una nueva instancia de <code>BorderLayout</code> y la usa o guarda para continuar el proceso.
406	<code>this.grupo = grupo;</code>	Accede al atributo del objeto actual para asignarlo o utilizarlo.
407	<code>setOpaque(false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
408	<code>setBorder(BorderFactory.createEmptyBorder());</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409		Línea en blanco: separa visualmente bloques para que el código sea más legible.
410	<code>if (grupo.visible()) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
411	<code>add(crearEncabezadoGrupo(), BorderLayout.NORTH);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
412	<code>}</code>	Cierra el bloque abierto anteriormente.
413	<code>opciones.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
414	<code>opciones.setLayout(new BoxLayout(opciones, BoxLayout.Y_AXIS));</code>	Crea una nueva instancia de <code>BoxLayout</code> y la usa o guarda para continuar el proceso.
415	<code>crearControles();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417	<code>personalizaciones.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.

418	personalizaciones.setLayout(new BorderLayout(personalizaciones, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
419	JPanel centro = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
420	centro.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
421	centro.setLayout(new BorderLayout(centro, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
422	centro.add(opciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
423	centro.add(personalizaciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
424	add(centro, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
425	reconstruirPersonalizaciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
426	}	Cierra el bloque abierto anteriormente.
427		Línea en blanco: separa visualmente bloques para que el código sea más legible.
428	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
429	public Dimension getMaximumSize() {	Declara el método getMaximumSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
430	Dimension preferido = getPreferredSize();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
431	return new Dimension(Integer.MAX_VALUE, preferido.height);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
432	}	Cierra el bloque abierto anteriormente.
433		Línea en blanco: separa visualmente bloques para que el código sea más legible.
434	private JComponent crearEncabezadoGrupo() {	Declara el método crearEncabezadoGrupo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
435	JPanel encabezado = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
436	encabezado.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
437	encabezado.setLayout(new BorderLayout(encabezado, BorderLayout.Y_AXIS));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
438	JLabel titulo = new JLabel(grupo.nombre().startsWith("__")	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
439	? "" : grupo.nombre());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
440	titulo.setFont(new Font(FUENTE, Font.BOLD, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
441	titulo.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
442	titulo.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
443	JLabel detalle = new JLabel(textoRegla(grupo));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
444	detalle.setForeground(TEXT0_SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
445	detalle.setFont(new Font(FUENTE, Font.PLAIN, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
446	detalle.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
447	encabezado.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448	encabezado.add(Box.createVerticalStrut(3));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
449	encabezado.add(detalle);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
450	return encabezado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
451	}	Cierra el bloque abierto anteriormente.
452		Línea en blanco: separa visualmente bloques para que el código sea más legible.
453	private String textoRegla(GruposMenu grupo) {	Declara el método textoRegla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
454	if (grupo.minimo() == grupo.maximo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
455	return grupo.minimo() + (grupo.minimo() == 1	Finaliza el método y devuelve el valor indicado a quien lo llamó.
456	? " requerido" : " requeridos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
457	}	Cierra el bloque abierto anteriormente.
458	if (grupo.minimo() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
459	return "Selecciona hasta " + grupo.maximo()	Finaliza el método y devuelve el valor indicado a quien lo llamó.
460	+ (grupo.maximo() == 1 ? " opción" : " opciones");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
461	}	Cierra el bloque abierto anteriormente.

462	return "Selecciona de " + grupo.minimo() + " a " + grupo.maximo();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
463	}	Cierra el bloque abierto anteriormente.
464		Línea en blanco: separa visualmente bloques para que el código sea más legible.
465	private void crearControles() {	Declara el método crearControles. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
466	if (!grupo.visible()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
467	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
468	}	Cierra el bloque abierto anteriormente.
469	if (grupo.permiteRepetir() && grupo.maximo() > 1) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
470	crearListasRepetibles();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
471	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
472	}	Cierra el bloque abierto anteriormente.
473		Línea en blanco: separa visualmente bloques para que el código sea más legible.
474	ButtonGroup radios = grupo.maximo() == 1 ? new ButtonGroup() : null;	Crea una nueva instancia de ButtonGroup y la usa o guarda para continuar el proceso.
475	if (radios != null && grupo.minimo() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
476	JRadioButton ninguna = radio("No agregar");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
477	ninguna.setSelected(true);	Configura una propiedad mediante setSelected en el objeto indicado.
478	ninguna.addActionListener(evento -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
479	radios.add(ninguna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
480	opciones.add(ninguna);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
481	opciones.add(Box.createVerticalStrut(7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
482	}	Cierra el bloque abierto anteriormente.
483		Línea en blanco: separa visualmente bloques para que el código sea más legible.
484	for (OpcionMenu opcion : grupo.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
485	AbstractButton control = radios == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
486	? casilla(opcion.toString(), false) : radio(opcion.toString());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
487	control.putClientProperty("opcion", opcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
488	control.setSelected(opcion.predeterminada());	Configura una propiedad mediante setSelected en el objeto indicado.
489	control.addActionListener(evento -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
490	if (radios != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
491	radios.add(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
492	}	Cierra el bloque abierto anteriormente.
493	botones.add(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
494	opciones.add(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
495	opciones.add(Box.createVerticalStrut(7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
496	}	Cierra el bloque abierto anteriormente.
497		Línea en blanco: separa visualmente bloques para que el código sea más legible.
498	if (grupo.minimo() > 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
499	&& botones.stream().noneMatch(AbstractButton::isSelected)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
500	&& !botones.isEmpty()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
501	botones.get(0).setSelected(true);	Configura una propiedad mediante setSelected en el objeto indicado.
502	}	Cierra el bloque abierto anteriormente.
503	}	Cierra el bloque abierto anteriormente.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	private void crearListasRepetibles() {	Declara el método crearListasRepetibles. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

506	for (int indice = 0; indice < grupo.maximo(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
507	JPanel fila = new JPanel(new BorderLayout(10, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
508	fila.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
509	fila.setMaximumSize(new Dimension(Integer.MAX_VALUE, 42));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
510	JLabel etiqueta = new JLabel("Producto " + (indice + 1));	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
511	etiqueta.setFont(new Font(FUENTE, Font.BOLD, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
512	etiqueta.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
513	etiqueta.setPreferredSize(new Dimension(82, 38));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
514		Línea en blanco: separa visualmente bloques para que el código sea más legible.
515	JComboBox<OpcionMenu> lista = new JComboBox<>();	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
516	lista.setFont(new Font(FUENTE, Font.PLAIN, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
517	lista.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
518	lista.setBorder(BorderFactory.createLineBorder(BORDE, 1, true));	Configura una propiedad mediante setBorder en el objeto indicado.
519	if (indice >= grupo.minimo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
520	lista.addItem(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
521	}	Cierra el bloque abierto anteriormente.
522	for (OpcionMenu opcion : grupo.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
523	lista.addItem(opcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
524	}	Cierra el bloque abierto anteriormente.
525	if (indice < grupo.minimo()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
526	seleccionarPredeterminada(lista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
527	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
528	lista.setSelectedIndex(0);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
529	}	Cierra el bloque abierto anteriormente.
530	lista.addActionListener(evento -> cambioSeleccion());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
531	listas.add(lista);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
532	fila.add(etiqueta, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
533	fila.add(lista, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
534	opciones.add(fila);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
535	opciones.add(Box.createVerticalStrut(8));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
536	}	Cierra el bloque abierto anteriormente.
537	}	Cierra el bloque abierto anteriormente.
538		Línea en blanco: separa visualmente bloques para que el código sea más legible.
539	private void seleccionarPredeterminada(JComboBox<OpcionMenu> lista) {	Declara el método seleccionarPredeterminada. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
540	for (int indice = 0; indice < lista.getItemCount(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
541	OpcionMenu opcion = lista.getItemAt(indice);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
542	if (opcion != null && opcion.predeterminada()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
543	lista.setSelectedIndex(indice);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
544	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
545	}	Cierra el bloque abierto anteriormente.
546	}	Cierra el bloque abierto anteriormente.
547	if (lista.getItemCount() > 0 && grupo.minimo() > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
548	lista.setSelectedIndex(0);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
549	}	Cierra el bloque abierto anteriormente.

550	}	Cierra el bloque abierto anteriormente.
551		Línea en blanco: separa visualmente bloques para que el código sea más legible.
552	private void cambioSeleccion() {	Declara el método cambioSeleccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
553	reconstruirPersonalizaciones();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
554	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
555	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
556	}	Cierra el bloque abierto anteriormente.
557		Línea en blanco: separa visualmente bloques para que el código sea más legible.
558	private List<Seleccion> seleccionadas() {	Declara el método seleccionadas. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
559	List<Seleccion> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
560	if (!grupo.visible()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
561	List<OpcionMenu> predeterminadas = grupo.opciones().stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
562	.filter(OpcionMenu::predeterminada).toList();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
563	if (predeterminadas.isEmpty() && !grupo.opciones().isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
564	predeterminadas = List.of(grupo.opciones().get(0));	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
565	}	Cierra el bloque abierto anteriormente.
566	int numero = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
567	for (OpcionMenu opcion : predeterminadas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
568	resultado.add(new Seleccion(numero++, opcion));	Crea una nueva instancia de Seleccion y la usa o guarda para continuar el proceso.
569	}	Cierra el bloque abierto anteriormente.
570	} else if (!listas.isEmpty()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
571	for (int indice = 0; indice < listas.size(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
572	OpcionMenu opcion = (OpcionMenu) listas.get(indice).getSelectedItem();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
573	if (opcion != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
574	resultado.add(new Seleccion(indice + 1, opcion));	Crea una nueva instancia de Seleccion y la usa o guarda para continuar el proceso.
575	}	Cierra el bloque abierto anteriormente.
576	}	Cierra el bloque abierto anteriormente.
577	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
578	int numero = 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
579	for (AbstractButton boton : botones) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
580	if (boton.isSelected()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
581	resultado.add(new Seleccion(numero++,	Crea una nueva instancia de Seleccion y la usa o guarda para continuar el proceso.
582	(OpcionMenu) boton.getClientProperty("opcion"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
583	}	Cierra el bloque abierto anteriormente.
584	}	Cierra el bloque abierto anteriormente.
585	}	Cierra el bloque abierto anteriormente.
586	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
587	}	Cierra el bloque abierto anteriormente.
588		Línea en blanco: separa visualmente bloques para que el código sea más legible.
589	private void reconstruirPersonalizaciones() {	Declara el método reconstruirPersonalizaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
590	personalizaciones.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
591	if (grupo.permitePersonalizar()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
592	for (Seleccion seleccion : seleccionadas()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
593	int indiceComponente = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

594	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
595	for (int unidad = 1; unidad <= componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
596	String llave = llave(seleccion, indiceComponente, unidad);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
597	PersonalizacionProducto panel = panelesProducto.get(llave);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
598	if (panel == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
599	panel = new PersonalizacionProducto(componente, unidad,	Crea una nueva instancia de PersonalizacionProducto y la usa o guarda para continuar el proceso.
600	componente.cantidad());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
601	panelesProducto.put(llave, panel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
602	}	Cierra el bloque abierto anteriormente.
603	if (panel.tieneControles()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
604	personalizaciones.add(Box.createVerticalStrut(12));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
605	personalizaciones.add(panel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
606	}	Cierra el bloque abierto anteriormente.
607	}	Cierra el bloque abierto anteriormente.
608	indiceComponente++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
609	}	Cierra el bloque abierto anteriormente.
610	}	Cierra el bloque abierto anteriormente.
611	}	Cierra el bloque abierto anteriormente.
612	personalizaciones.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
613	personalizaciones.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
614	}	Cierra el bloque abierto anteriormente.
615		Línea en blanco: separa visualmente bloques para que el código sea más legible.
616	private String llave(Seleccion seleccion, int indiceComponente, int unidad) {	Declara el método llave. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
617	return seleccion.numero + ":" + seleccion.opcion.idOpcion()	Finaliza el método y devuelve el valor indicado a quien lo llamó.
618	+ ":" + indiceComponente + ":" + unidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
619	}	Cierra el bloque abierto anteriormente.
620		Línea en blanco: separa visualmente bloques para que el código sea más legible.
621	private boolean tienePersonalizaciones() {	Declara el método tienePersonalizaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
622	return personalizaciones.getComponentCount() > 0;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
623	}	Cierra el bloque abierto anteriormente.
624		Línea en blanco: separa visualmente bloques para que el código sea más legible.
625	BigDecimal precioExtra() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
626	BigDecimal total = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
627	for (Seleccion seleccion : seleccionadas()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
628	total = total.add(seleccion.opcion.incrementoPrecio());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
629	int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
630	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
631	for (int unidad = 1; unidad <= componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
632	PersonalizacionProducto panel = panelesProducto.get(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
633	llave(seleccion, indice, unidad));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
634	if (panel != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
635	total = total.add(panel.precioExtra());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
636	}	Cierra el bloque abierto anteriormente.
637	}	Cierra el bloque abierto anteriormente.

638	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
639	}	Cierra el bloque abierto anteriormente.
640	}	Cierra el bloque abierto anteriormente.
641	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
642	}	Cierra el bloque abierto anteriormente.
643		Línea en blanco: separa visualmente bloques para que el código sea más legible.
644	List<OpcionPedido> crearOpciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
645	List<Seleccion> elegidas = seleccionadas();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
646	if (elegidas.size() < grupo.minimo()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
647	elegidas.size() > grupo.maximo()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
648	throw new IllegalArgumentException("En " + grupo.nombre()	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
649	+ "" debes elegir de " + grupo.minimo() + " a "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
650	+ grupo.maximo() + " opciones.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
651	}	Cierra el bloque abierto anteriormente.
652	if (!grupo.permiteRepetir()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
653	long diferentes = elegidas.stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
654	.map(seleccion -> seleccion.opcion.idOpcion())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
655	.distinct().count();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
656	if (diferentes != elegidas.size()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
657	throw new IllegalArgumentException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
658	"No puedes repetir una opción en " + grupo.nombre() + "."");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
659	}	Cierra el bloque abierto anteriormente.
660	}	Cierra el bloque abierto anteriormente.
661		Línea en blanco: separa visualmente bloques para que el código sea más legible.
662	List<OpcionPedido> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
663	for (Seleccion seleccion : elegidas) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
664	List<ProductoPedido> productos = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
665	int indice = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
666	for (ComponenteMenu componente : seleccion.opcion.componentes()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
667	for (int unidad = 1; unidad <= componente.cantidad(); unidad++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
668	String llave = llave(seleccion, indice, unidad);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
669	PersonalizacionProducto panel = panelesProducto.get(llave);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
670	if (panel == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
671	panel = new PersonalizacionProducto(componente, unidad,	Crea una nueva instancia de PersonalizacionProducto y la usa o guarda para continuar el proceso.
672	componente.cantidad());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
673	panelesProducto.put(llave, panel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
674	}	Cierra el bloque abierto anteriormente.
675	productos.add(new ProductoPedido(UUID.randomUUID().toString(),	Crea una nueva instancia de ProductoPedido y la usa o guarda para continuar el proceso.
676	componente.idProducto(), componente.nombre(), 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
677	componente.tipoStock(), panel.modificaciones());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
678	}	Cierra el bloque abierto anteriormente.
679	indice++;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
680	}	Cierra el bloque abierto anteriormente.
681	resultado.add(new OpcionPedido(grupo.idGrupo(),	Crea una nueva instancia de OpcionPedido y la usa o guarda para continuar el proceso.

682	seleccion.opcion.idOpcion(), seleccion.numero,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
683	grupo.visible() ? grupo.nombre() : "__Producto principal",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
684	seleccion.opcion.nombre(), seleccion.opcion.incrementoPrecio(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
685	productos));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
686	}	Cierra el bloque abierto anteriormente.
687	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
688	}	Cierra el bloque abierto anteriormente.
689	}	Cierra el bloque abierto anteriormente.
690		Línea en blanco: separa visualmente bloques para que el código sea más legible.
691	private final class PersonalizacionProducto extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
692	private final ComponenteMenu componente;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
693	private final Map<IngredienteProducto, JCheckBox> incluidos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
694	= new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
695	private final Map<IngredienteProducto, SelectorExtra> extras	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
696	= new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
697		Línea en blanco: separa visualmente bloques para que el código sea más legible.
698	PersonalizacionProducto(ComponenteMenu componente, int unidad, int total) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
699	this.componente = componente;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
700	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
701	setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
702	setAlignmentX(Component.LEFT_ALIGNMENT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
703		Línea en blanco: separa visualmente bloques para que el código sea más legible.
704	if (total > 1) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
705	JLabel productoNumero = new JLabel(componente.nombre() + " " + unidad);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
706	productoNumero.setFont(new Font(FUENTE, Font.BOLD, 14));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
707	productoNumero.setForeground(AZUL);	Configura una propiedad mediante setForeground en el objeto indicado.
708	productoNumero.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
709	add(productoNumero);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
710	add(Box.createVerticalStrut(10));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
711	}	Cierra el bloque abierto anteriormente.
712		Línea en blanco: separa visualmente bloques para que el código sea más legible.
713	List<IngredienteProducto> removibles = componente.ingredientes().stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
714	.filter(IngredienteProducto::permiteQuitar).toList();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
715	if (!removibles.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
716	add(tituloSeccion("¿Deseas eliminar algún ingrediente?"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
717	add(descripcionSeccion("Desmarca lo que deseas eliminar"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
718	add(Box.createVerticalStrut(6));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
719	for (IngredienteProducto ingrediente : removibles) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
720	JCheckBox incluido = casilla(ingrediente.nombre(), true);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
721	incluido.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
722	incluidos.put(ingrediente, incluido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
723	add(incluido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
724	add(Box.createVerticalStrut(5));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
725	}	Cierra el bloque abierto anteriormente.

726	}	Cierra el bloque abierto anteriormente.
727		Línea en blanco: separa visualmente bloques para que el código sea más legible.
728	List<IngredienteProducto> agregables = componente.ingredientes().stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
729	.filter(IngredienteProducto::permiteExtra).toList();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
730	if (!agregables.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
731	if (!removibles.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
732	add(Box.createVerticalStrut(13));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
733	}	Cierra el bloque abierto anteriormente.
734	add(tituloSeccion("Elige tu complemento"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
735	int maximo = agregables.stream()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
736	.mapToInt(IngredienteProducto::maxExtras).max().orElse(1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
737	add(descripcionSeccion("Selecciona hasta " + maximo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
738	+ (maximo == 1 ? " opción" : " opciones por ingrediente")));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
739	add(Box.createVerticalStrut(6));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
740	for (IngredienteProducto ingrediente : agregables) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
741	SelectorExtra selector = new SelectorExtra(ingrediente);	Crea una nueva instancia de SelectorExtra y la usa o guarda para continuar el proceso.
742	selector.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
743	extras.put(ingrediente, selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
744	JCheckBox incluido = incluidos.get(ingrediente);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
745	if (incluido != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
746	incluido.addActionListener(evento -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
747	if (!incluido.isSelected()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
748	selector.reiniciar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
749	}	Cierra el bloque abierto anteriormente.
750	selector.setControlesHabilitados(incluido.isSelected());	Configura una propiedad mediante setControlesHabilitados en el objeto indicado.
751	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
752	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
753	}	Cierra el bloque abierto anteriormente.
754	add(selector);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
755	add(Box.createVerticalStrut(7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
756	}	Cierra el bloque abierto anteriormente.
757	}	Cierra el bloque abierto anteriormente.
758	}	Cierra el bloque abierto anteriormente.
759		Línea en blanco: separa visualmente bloques para que el código sea más legible.
760	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
761	public Dimension getMaximumSize() {	Declara el método getMaximumSize. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
762	Dimension preferido = getPreferredSize();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
763	return new Dimension(Integer.MAX_VALUE, preferido.height);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
764	}	Cierra el bloque abierto anteriormente.
765		Línea en blanco: separa visualmente bloques para que el código sea más legible.
766	private JLabel tituloSeccion(String texto) {	Declara el método tituloSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
767	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
768	etiqueta.setFont(new Font(FUENTE, Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
769	etiqueta.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.

770	etiqueta.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
771	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
772	}	Cierra el bloque abierto anteriormente.
773		Línea en blanco: separa visualmente bloques para que el código sea más legible.
774	private JLabel descripcionSeccion(String texto) {	Declara el método descripcionSeccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
775	JLabel etiqueta = new JLabel(texto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
776	etiqueta.setFont(new Font(FUENTE, Font.PLAIN, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
777	etiqueta.setForeground(TEXT0_SECUNDARIO);	Configura una propiedad mediante setForeground en el objeto indicado.
778	etiqueta.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
779	return etiqueta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
780	}	Cierra el bloque abierto anteriormente.
781		Línea en blanco: separa visualmente bloques para que el código sea más legible.
782	boolean tieneControles() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
783	return !incluidos.isEmpty() !extras.isEmpty();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
784	}	Cierra el bloque abierto anteriormente.
785		Línea en blanco: separa visualmente bloques para que el código sea más legible.
786	BigDecimal precioExtra() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
787	BigDecimal total = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
788	for (Map.Entry<IngredienteProducto, SelectorExtra> entrada	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
789	: extras.entrySet()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
790	total = total.add(entrada.getKey().precioExtra()	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
791	.multiply(BigDecimal.valueOf(entrada.getValue().cantidad)));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
792	}	Cierra el bloque abierto anteriormente.
793	return total;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
794	}	Cierra el bloque abierto anteriormente.
795		Línea en blanco: separa visualmente bloques para que el código sea más legible.
796	List<ModificacionPedido> modificaciones() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
797	List<ModificacionPedido> resultado = new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
798	for (Map.Entry<IngredienteProducto, JCheckBox> entrada	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
799	: incluidos.entrySet()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
800	IngredienteProducto ingrediente = entrada.getKey();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
801	if (!entrada.getValue().isSelected()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
802	resultado.add(new ModificacionPedido(	Crea una nueva instancia de ModificacionPedido y la usa o guarda para continuar el proceso.
803	ingrediente.idProductoIngrediente(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
804	ingrediente.idIngrediente(), "SIN",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
805	ingrediente.nombre(), ingrediente.cantidadDefault(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
806	1, BigDecimal.ZERO));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
807	}	Cierra el bloque abierto anteriormente.
808	}	Cierra el bloque abierto anteriormente.
809	for (Map.Entry<IngredienteProducto, SelectorExtra> entrada	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
810	: extras.entrySet()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
811	IngredienteProducto ingrediente = entrada.getKey();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
812	int veces = entrada.getValue().cantidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
813	if (veces > 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

814	resultado.add(new ModificacionPedido(	Crea una nueva instancia de ModificacionPedido y la usa o guarda para continuar el proceso.
815	ingrediente.idProductoIngrediente(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
816	ingrediente.idIngrediente(), "EXTRA",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
817	ingrediente.nombre(), ingrediente.cantidadExtra()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
818	.multiply(BigDecimal.valueOf(veces)), veces,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
819	ingrediente.precioExtra()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
820	.multiply(BigDecimal.valueOf(veces)));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
821	}	Cierra el bloque abierto anteriormente.
822	}	Cierra el bloque abierto anteriormente.
823	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
824	}	Cierra el bloque abierto anteriormente.
825	}	Cierra el bloque abierto anteriormente.
826		Línea en blanco: separa visualmente bloques para que el código sea más legible.
827	private final class SelectorExtra extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
828	private final IngredienteProducto ingrediente;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
829	private final JButton menos;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
830	private final JButton mas;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
831	private final JLabel numero = new JLabel("0", SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
832	private int cantidad;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
833		Línea en blanco: separa visualmente bloques para que el código sea más legible.
834	SelectorExtra(IngredienteProducto ingrediente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
835	super(new BorderLayout(8, 0));	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
836	this.ingrediente = ingrediente;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
837	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
838	setMaximumSize(new Dimension(Integer.MAX_VALUE, 38));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
839		Línea en blanco: separa visualmente bloques para que el código sea más legible.
840	String precioTexto = ingrediente.precioExtra().signum() > 0	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
841	? " + Q" + ingrediente.precioExtra().setScale(2).toPlainString()	Configura una propiedad mediante setScale en el objeto indicado.
842	: "";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
843	JLabel nombre = new JLabel("Extra " + ingrediente.nombre() + precioTexto);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
844	nombre.setFont(new Font(FUENTE, Font.PLAIN, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
845	nombre.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
846	add(nombre, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
847		Línea en blanco: separa visualmente bloques para que el código sea más legible.
848	JPanel controles = new JPanel(new FlowLayout(FlowLayout.RIGHT, 5, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
849	controles.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
850	menos = botonExtra("-");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
851	mas = botonExtra("+");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
852	numero.setFont(new Font(FUENTE, Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
853	numero.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
854	numero.setPreferredSize(new Dimension(20, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
855	menos.addActionListener(evento -> cambiar(-1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
856	mas.addActionListener(evento -> cambiar(1));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
857	controles.add(menos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

858	controles.add(numero);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
859	controles.add(mas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
860	add(controles, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
861	actualizarEstado();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
862	}	Cierra el bloque abierto anteriormente.
863		Línea en blanco: separa visualmente bloques para que el código sea más legible.
864	private JButton botonExtra(String texto) {	Declara el método botonExtra. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
865	return new BotonPaso(texto, FONDO_CONTROL,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
866	BORDE, AZUL, 36, 30, 10, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
867	}	Cierra el bloque abierto anteriormente.
868		Línea en blanco: separa visualmente bloques para que el código sea más legible.
869	private void cambiar(int valor) {	Declara el método cambiar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
870	cantidad = Math.max(0,	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
871	Math.min(ingrediente.maxExtras(), cantidad + valor));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
872	actualizarEstado();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
873	error.setText(" ");	Cambia el texto visible de un componente de la interfaz.
874	actualizarPrecio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
875	}	Cierra el bloque abierto anteriormente.
876		Línea en blanco: separa visualmente bloques para que el código sea más legible.
877	private void reiniciar() {	Declara el método reiniciar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
878	cantidad = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
879	actualizarEstado();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
880	}	Cierra el bloque abierto anteriormente.
881		Línea en blanco: separa visualmente bloques para que el código sea más legible.
882	private void setControlesHabilitados(boolean habilitado) {	Declara el método setControlesHabilitados. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
883	menos.setEnabled(habilitado && cantidad > 0);	Habilita o deshabilita la interacción con el componente.
884	mas.setEnabled(habilitado && cantidad < ingrediente.maxExtras());	Habilita o deshabilita la interacción con el componente.
885	numero.setEnabled(habilitado);	Habilita o deshabilita la interacción con el componente.
886	}	Cierra el bloque abierto anteriormente.
887		Línea en blanco: separa visualmente bloques para que el código sea más legible.
888	private void actualizarEstado() {	Declara el método actualizarEstado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
889	numero.setText(String.valueOf(cantidad));	Cambia el texto visible de un componente de la interfaz.
890	menos.setEnabled(cantidad > 0);	Habilita o deshabilita la interacción con el componente.
891	mas.setEnabled(cantidad < ingrediente.maxExtras());	Habilita o deshabilita la interacción con el componente.
892	}	Cierra el bloque abierto anteriormente.
893	}	Cierra el bloque abierto anteriormente.
894		Línea en blanco: separa visualmente bloques para que el código sea más legible.
895	private JCheckBox casilla(String texto, boolean seleccionada) {	Declara el método casilla. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
896	JCheckBox control = new JCheckBox(texto, seleccionada);	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
897	prepararControl(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
898	control.setIcon(new IconoSeleccion(false, false));	Crea una nueva instancia de IconoSeleccion y la usa o guarda para continuar el proceso.
899	control.setSelectedIcon(new IconoSeleccion(false, true));	Crea una nueva instancia de IconoSeleccion y la usa o guarda para continuar el proceso.
900	return control;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
901	}	Cierra el bloque abierto anteriormente.

902		Línea en blanco: separa visualmente bloques para que el código sea más legible.
903	private JRadioButton radio(String texto) {	Declara el método radio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
904	JRadioButton control = new JRadioButton(texto);	Crea una nueva instancia de JRadioButton y la usa o guarda para continuar el proceso.
905	prepararControl(control);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
906	control.setIcon(new IconoSeleccion(true, false));	Crea una nueva instancia de IconoSeleccion y la usa o guarda para continuar el proceso.
907	control.setSelectedIcon(new IconoSeleccion(true, true));	Crea una nueva instancia de IconoSeleccion y la usa o guarda para continuar el proceso.
908	return control;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
909	}	Cierra el bloque abierto anteriormente.
910		Línea en blanco: separa visualmente bloques para que el código sea más legible.
911	private void prepararControl(AbstractButton control) {	Declara el método prepararControl. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
912	control.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
913	control.setFont(new Font(FUENTE, Font.PLAIN, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
914	control.setForeground(TEXT0);	Configura una propiedad mediante setForeground en el objeto indicado.
915	control.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
916	control.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Configura una propiedad mediante setCursor en el objeto indicado.
917	control.setIconTextGap(10);	Configura una propiedad mediante setIconTextGap en el objeto indicado.
918	control.setAlignmentX(Component.LEFT_ALIGNMENT);	Configura una propiedad mediante setAlignmentX en el objeto indicado.
919	}	Cierra el bloque abierto anteriormente.
920		Línea en blanco: separa visualmente bloques para que el código sea más legible.
921	private static final class IconoSeleccion implements Icon {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
922	private final boolean circular;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
923	private final boolean seleccionado;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
924		Línea en blanco: separa visualmente bloques para que el código sea más legible.
925	IconoSeleccion(boolean circular, boolean seleccionado) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
926	this.circular = circular;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
927	this.seleccionado = seleccionado;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
928	}	Cierra el bloque abierto anteriormente.
929		Línea en blanco: separa visualmente bloques para que el código sea más legible.
930	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
931	public int getIconWidth() {	Declara el método getIconWidth. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
932	return 18;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
933	}	Cierra el bloque abierto anteriormente.
934		Línea en blanco: separa visualmente bloques para que el código sea más legible.
935	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
936	public int getIconHeight() {	Declara el método getIconHeight. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
937	return 18;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
938	}	Cierra el bloque abierto anteriormente.
939		Línea en blanco: separa visualmente bloques para que el código sea más legible.
940	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
941	public void paintIcon(Component componente, Graphics graphics, int x, int y) {	Declara el método paintIcon. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
942	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
943	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
944	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
945	g2.setStroke(new BasicStroke(1.4f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.

946	g2.setColor(seleccionado ? AMARILLO : new Color(145, 149, 155));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
947	if (circular) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
948	if (seleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
949	g2.fillOval(x + 1, y + 1, 16, 16);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
950	g2.setColor(AZUL);	Configura una propiedad mediante setColor en el objeto indicado.
951	g2.fillOval(x + 6, y + 6, 6, 6);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
952	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
953	g2.drawOval(x + 1, y + 1, 15, 15);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
954	}	Cierra el bloque abierto anteriormente.
955	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
956	if (seleccionado) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
957	g2.fillRoundRect(x + 1, y + 1, 16, 16, 3, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
958	g2.setColor(AZUL);	Configura una propiedad mediante setColor en el objeto indicado.
959	g2.setStroke(new BasicStroke(1.8f, BasicStroke.CAP_ROUND,	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
960	BasicStroke.JOIN_ROUND));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
961	g2.drawLine(x + 5, y + 9, x + 8, y + 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
962	g2.drawLine(x + 8, y + 12, x + 14, y + 6);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
963	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
964	g2.drawRoundRect(x + 1, y + 1, 15, 15, 3, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
965	}	Cierra el bloque abierto anteriormente.
966	}	Cierra el bloque abierto anteriormente.
967	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
968	}	Cierra el bloque abierto anteriormente.
969	}	Cierra el bloque abierto anteriormente.
970		Línea en blanco: separa visualmente bloques para que el código sea más legible.
971	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
972	* Botón compacto que pinta el símbolo directamente. De esta forma ningún	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
973	* Look & Feel puede sustituir los signos + y - por puntos suspensivos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
974	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
975	private static final class BotonPaso extends JButton {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
976	private final Color fondo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
977	private final Color borde;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
978	private final Color simbolo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
979	private final int radio;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
980		Línea en blanco: separa visualmente bloques para que el código sea más legible.
981	BotonPaso(String texto, Color fondo, Color borde, Color simbolo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
982	int ancho, int alto, int radio, int tamanoFuente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
983	super(texto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
984	this.fondo = fondo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
985	this.borde = borde;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
986	this.simbolo = simbolo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
987	this.radio = radio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
988	setFont(new Font(FUENTE, Font.BOLD, tamanoFuente));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
989	setPreferredSize(new Dimension(ancho, alto));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.

990	setMinimumSize(new Dimension(ancho, alto));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
991	setMaximumSize(new Dimension(ancho, alto));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
992	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
993	setContentAreaFilled(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
994	setBorderPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
995	setFocusPainted(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
996	setRolloverEnabled(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
997	setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
998	setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
999	setBorder(BorderFactory.createEmptyBorder());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1000	}	Cierra el bloque abierto anteriormente.
1001		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1002	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
1003	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1004	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1005	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
1006	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1007	Color colorFondo = fondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1008	if (!isEnabled()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
1009	colorFondo = new Color(239, 240, 242);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1010	} else if (getModel().isPressed()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1011	colorFondo = fondo.darker();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1012	} else if (getModel().isRollover()) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
1013	colorFondo = aclarar(fondo, 10);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
1014	}	Cierra el bloque abierto anteriormente.
1015	g2.setColor(colorFondo);	Configura una propiedad mediante setColor en el objeto indicado.
1016	g2.fillRoundRect(0, 0, Math.max(1, getWidth() - 1),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1017	Math.max(1, getHeight() - 1), radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1018	g2.setColor(isEnabled() ? borde : new Color(220, 222, 226));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1019	g2.setStroke(new BasicStroke(1f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
1020	g2.drawRoundRect(0, 0, Math.max(1, getWidth() - 1),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1021	Math.max(1, getHeight() - 1), radio, radio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1022		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1023	String texto = getText();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1024	g2.setFont(getFont());	Configura una propiedad mediante setFont en el objeto indicado.
1025	FontMetrics medidas = g2.getFontMetrics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1026	int x = (getWidth() - medidas.stringWidth(texto)) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1027	int y = (getHeight() - medidas.getHeight()) / 2	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
1028	+ medidas.getAscent() - 1;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1029	g2.setColor(isEnabled() ? simbolo : new Color(150, 154, 160));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
1030	g2.drawString(texto, x, y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1031	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1032	}	Cierra el bloque abierto anteriormente.
1033		Línea en blanco: separa visualmente bloques para que el código sea más legible.

1034	private static Color aclarar(Color color, int cantidad) {	Declara el método aclarar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
1035	return new Color(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
1036	Math.min(255, color.getRed() + cantidad),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1037	Math.min(255, color.getGreen() + cantidad),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1038	Math.min(255, color.getBlue() + cantidad),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
1039	color.getAlpha());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
1040	}	Cierra el bloque abierto anteriormente.
1041	}	Cierra el bloque abierto anteriormente.
1042		Línea en blanco: separa visualmente bloques para que el código sea más legible.
1043	private record Seleccion(int numero, OpcionMenu opcion) {	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
1044	}	Cierra el bloque abierto anteriormente.
1045	}	Cierra el bloque abierto anteriormente.

16.95 src/GUI_CAJERO/MenuCajeroPanel.form

Metadatos visuales de NetBeans para diseñar MenuCajeroPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 64 | Archivo 95 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,4,56,0,0,7,-128"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="Componentes.Scroll_Categorias" name="panelCategoria">	Declara un componente o contenedor visual y su clase Java.
27	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
28	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
29	<AbsoluteConstraints x="60" y="140" width="1810" height="250"/>	Define la posición y el tamaño del componente dentro del diseño.
30	</Constraint>	Cierra la sección XML abierta anteriormente.
31	</Constraints>	Cierra la sección XML abierta anteriormente.
32	</Component>	Cierra la sección XML abierta anteriormente.
33	<Component class="javax.swing.JLabel" name="jLabel1">	Declara un componente o contenedor visual y su clase Java.
34	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
35	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
36		Metadato de NetBeans que conserva el estado editable del diseñador visual.
37	</Property>	Cierra la sección XML abierta anteriormente.

38	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
39	<Color blue="1b" green="d" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
40	</Property>	Cierra la sección XML abierta anteriormente.
41	<Property name="text" type="java.lang.String" value="Menú">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
42	</Properties>	Cierra la sección XML abierta anteriormente.
43	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
44	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
45	<AbsoluteConstraints x="60" y="30" width="310" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
46	</Constraint>	Cierra la sección XML abierta anteriormente.
47	</Constraints>	Cierra la sección XML abierta anteriormente.
48	</Component>	Cierra la sección XML abierta anteriormente.
49	<Container class="javax.swing.JPanel" name="panelContenido">	Declara un componente o contenedor visual y su clase Java.
50	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
51	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
52	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
53	</Property>	Cierra la sección XML abierta anteriormente.
54	</Properties>	Cierra la sección XML abierta anteriormente.
55	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
56	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
57	<AbsoluteConstraints x="60" y="410" width="1810" height="660"/>	Define la posición y el tamaño del componente dentro del diseño.
58	</Constraint>	Cierra la sección XML abierta anteriormente.
59	</Constraints>	Cierra la sección XML abierta anteriormente.
60		Línea en blanco para organizar el XML del formulario.
61	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignFlowLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
62	</Container>	Cierra la sección XML abierta anteriormente.
63	</SubComponents>	Cierra la sección XML abierta anteriormente.
64	</Form>	Cierra la sección XML abierta anteriormente.

16.96 src/GUI_CAJERO/MenuCajeroPanel.java

Interfaz del módulo cajero: MenuCajeroPanel. Participa en la selección y venta de productos.

Tipo: java | Líneas: 109 | Archivo 96 de 157

Clases/tipos detectados: MenuCajeroPanel

Línea	Constructor o método detectado
5	public MenuCajeroPanel() {
18	public void mostrarPanelCategoria(javax.swing.JPanel panel) {
38	private void cargarProductos(int idCategoria) {
84	private void initComponents() {

1 package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 public class MenuCajeroPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4	Línea en blanco: separa visualmente bloques para que el código sea más legible.
5 public MenuCajeroPanel() {	Constructor de MenuCajeroPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
6	Línea en blanco: separa visualmente bloques para que el código sea más legible.
7 initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
8	Línea en blanco: separa visualmente bloques para que el código sea más legible.
9 CategoriasPanel categoriasPanel	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10 = new CategoriasPanel(this);	Crea una nueva instancia de CategoriasPanel y la usa o guarda para continuar el proceso.
11	Línea en blanco: separa visualmente bloques para que el código sea más legible.
12 panelCategoria.setPanelCategorias(	Configura una propiedad mediante setPanelCategorias en el objeto indicado.
13 categoriasPanel	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
15	Línea en blanco: separa visualmente bloques para que el código sea más legible.
16 }	Cierra el bloque abierto anteriormente.
17	Línea en blanco: separa visualmente bloques para que el código sea más legible.
18 public void mostrarPanelCategoria(javax.swing.JPanel panel) {	Declara el método mostrarPanelCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
19 panelContenido.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
20	Línea en blanco: separa visualmente bloques para que el código sea más legible.
21 panelContenido.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
22 new java.awt.BorderLayout()	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
23);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
24	Línea en blanco: separa visualmente bloques para que el código sea más legible.
25 panelContenido.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26 panel,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27 java.awt.BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	Línea en blanco: separa visualmente bloques para que el código sea más legible.
30 panel.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.

31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	panelContenido.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	panelContenido.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	}	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	private javax.swing.JPanel jPanel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	private void cargarProductos(int idCategoria) {	Declara el método cargarProductos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	// Eliminar productos anteriores	Comentario para humanos; Java no ejecuta este texto. Eliminar productos anteriores
41	jPanel1.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	// 4 productos por cada fila	Comentario para humanos; Java no ejecuta este texto. 4 productos por cada fila
44	jPanel1.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
45	new java.awt.FlowLayout(	Crea una nueva instancia de java.awt.FlowLayout y la usa o guarda para continuar el proceso.
46	java.awt.FlowLayout.LEFT,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	15,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	15	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	// Consultar base de datos	Comentario para humanos; Java no ejecuta este texto. Consultar base de datos
53	DAO.ProductoDAO productoDAO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	= new DAO.ProductoDAO();	Crea una nueva instancia de DAO.ProductoDAO y la usa o guarda para continuar el proceso.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	java.util.List<Modelos.Productos> productos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	= productoDAO.obtenerPorCategoria(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58	idCategoria	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	// Crear una tarjeta por cada producto	Comentario para humanos; Java no ejecuta este texto. Crear una tarjeta por cada producto
62	for (Modelos.Productos producto : productos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	Componentes.TarjetaProducto tarjeta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	= new Componentes.TarjetaProducto(	Crea una nueva instancia de Componentes.TarjetaProducto y la usa o guarda para continuar el proceso.
66	producto.getIdProducto(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	producto.getNombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	producto.getDescripcion(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	producto.getPrecio(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	producto.getImagen(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	producto.isDisponibleHorario()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	jPanel1.add(tarjeta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

75	}	Cierra el bloque abierto anteriormente.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77	// Actualizar el panel	Comentario para humanos; Java no ejecuta este texto. Actualizar el panel
78	jPanel1.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	jPanel1.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	}	Cierra el bloque abierto anteriormente.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
83	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
84	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	panelCategoria = new Componentes.Scroll_Categorias();	Crea una nueva instancia de Componentes.Scroll_Categorias y la usa o guarda para continuar el proceso.
87	jLabel1 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
88	panelContenido = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
91	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
92	add(panelCategoria, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 140, 1810, 250));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	jLabel1.setFont(new java.awt.Font("DM Sans 18pt", 1, 110)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
95	jLabel1.setForeground(new java.awt.Color(0, 13, 27));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
96	jLabel1.setText("Menú");	Cambia el texto visible de un componente de la interfaz.
97	add(jLabel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 30, 310, 100));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	panelContenido.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
100	add(panelContenido, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 410, 1810, 660));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
101	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
105	private javax.swing.JLabel jLabel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
106	private Componentes.Scroll_Categorias panelCategoria;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
107	private javax.swing.JPanel panelContenido;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
108	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
109	}	Cierra el bloque abierto anteriormente.

16.97 src/GUI_CAJERO/PedidoPanel.java

Carrito de compra. Mantiene líneas del pedido, cantidades, total, pago y acceso al último comprobante.

Tipo: java | Líneas: 660 | Archivo 97 de 157

Clases/tipos detectados: PedidoPanel, BarraPedidoMinimalista

Línea	Constructor o método detectado
29	private final DefaultTableModel modelo = new DefaultTableModel(new String[]{"Producto", "Cant.", "Importe"}, 0) {
32	private final TablaAdministrativa tabla = new TablaAdministrativa() {
74	private void actualizarHover(java.awt.event.MouseEvent evento) {
97	private final JLabel contador = new JLabel("0 productos") {
119	public PedidoPanel() {
366	private static JButton boton(String texto, Color fondo, Color textoColor) {
411	private static JButton botonInvisible() {
425	protected void paintComponent(Graphics graphics) {
436	public void agregar(LineaPedido linea) {
449	public void mostrarConfigurador(ConfiguracionProducto producto,
475	private void mostrarResumen() {
485	private void cambiar(int incremento) {
494	private boolean bloqueado() { return ocupado pendiente != null recuperacionFallida; }
495	private BigDecimal suma() { return lineas.values().stream().map(LineaPedido::subtotal).reduce(new BigDecimal("0.00"), BigDecimal::add); }
496	private void actualizar() {
509	private void abrirPago() {
530	public void insertUpdate(DocumentEvent e) { refrescar.run(); }
531	public void removeUpdate(DocumentEvent e) { refrescar.run(); }
532	public void changedUpdate(DocumentEvent e) { refrescar.run(); }
595	public static BigDecimal monto(String texto) {
599	private void mostrarComprobante() {
619	private void imprimir(JTextArea texto) {
625	private static String crearTicketCocina(SolicitudPago solicitud) {
655	private static String mensaje(Exception ex) {
659	private void aviso(String mensaje) { JOptionPane.showMessageDialog(this, mensaje, "Pedido", JOptionPane.WARNING_MESSAGE); }

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonRedondeado;	Importa Componentes.BotonRedondeado para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.TablaAdministrativa;	Importa Componentes.TablaAdministrativa para poder usar esa clase sin escribir su nombre completo.
5	import DAO.PagoDAO;	Importa DAO.PagoDAO para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.*;	Importa Modelos.* para poder usar esa clase sin escribir su nombre completo.
7	import Utilidades.SesionUsuario;	Importa Utilidades.SesionUsuario para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.*;	Importa java.awt.* para poder usar esa clase sin escribir su nombre completo.
9	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
10	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.*;	Importa javax.swing.* para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.event.DocumentEvent;	Importa javax.swing.event.DocumentEvent para poder usar esa clase sin escribir su nombre completo.
13	import javax.swing.event.DocumentListener;	Importa javax.swing.event.DocumentListener para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.table.DefaultTableModel;	Importa javax.swing.table.DefaultTableModel para poder usar esa clase sin escribir su nombre completo.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	/** Carrito de una caja; conserva la operación pendiente ante errores de conexión. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	public class PedidoPanel extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
18	private static final Color AMARILLO = new Color(255, 188, 13);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
19	private static final Color ROJO = new Color(196, 30, 42);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
20	private static final Color ROJO_HEADER = new Color(1, 20, 36);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
21	private static final Color TINTA = new Color(20, 27, 35);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
22	private static final Color FONDO = new Color(246, 246, 248);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
23	private static final Color BORDE_SUAVE = new Color(231, 226, 213);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
24	private static final String FUENTE = "Arial";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
25	private final CardLayout vistasPanel = new CardLayout();	Crea una nueva instancia de CardLayout y la usa o guarda para continuar el proceso.
26	private final JPanel resumenPedido = new JPanel(new BorderLayout(8, 16));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
27	private ConfiguratorProductoPanel configuradorActual;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	private final Map<String, LineaPedido> lineas = new LinkedHashMap<>();	Crea una nueva instancia de LinkedHashMap y la usa o guarda para continuar el proceso.
29	private final DefaultTableModel modelo = new DefaultTableModel(new String[]{"Producto", "Cant.", "Importe"}, 0) {	Crea una nueva instancia de DefaultTableModel y la usa o guarda para continuar el proceso.
30	@Override public boolean isCellEditable(int r, int c) { return false; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
31	};	Cierra el bloque abierto anteriormente y termina la declaración.
32	private final TablaAdministrativa tabla = new TablaAdministrativa() {	Crea una nueva instancia de TablaAdministrativa y la usa o guarda para continuar el proceso.
33	private int filaHover = -1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	@Override protected void paintComponent(Graphics g) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
36	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
38	for (int fila = 0; fila < getRowCount(); fila++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
39	Rectangle area = getCellRect(fila, 0, true);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
40	g2.setColor(getSelectionModel().isSelectedIndex(fila)	Configura una propiedad mediante setColor en el objeto indicado.
41	? new Color(255, 235, 170)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
42	: fila == filaHover ? new Color(255, 251, 239) : Color.WHITE);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

43	g2.fillRoundRect(4, area.y + 3, getWidth() - 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	Math.max(1, area.height - 6), 22, 22);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	}	Cierra el bloque abierto anteriormente.
46	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	if (getRowCount() == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
49	Graphics2D emptyG2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
50	emptyG2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALI	Configura una propiedad mediante setRenderingHint en el objeto indicado.
AS_ON);		
51	int y = Math.max(48, getHeight() / 2 - 10);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52	emptyG2.setFont(new Font("SansSerif", Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
53	emptyG2.setColor(TINTA);	Configura una propiedad mediante setColor en el objeto indicado.
54	String titulo = "El pedido está vacío";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55	emptyG2.drawString(titulo, (getWidth() - emptyG2.getFontMetrics().stringWidth(titulo)) / 2, y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	emptyG2.setFont(new Font("SansSerif", Font.PLAIN, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
57	emptyG2.setColor(new Color(105, 109, 115));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
58	String ayuda = "Agrega productos desde el menú";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	emptyG2.drawString(ayuda, (getWidth() - emptyG2.getFontMetrics().stringWidth(ayuda)) / 2, y + 24	Ejecuta una instrucción Java completa; el punto y coma marca su final.
);		
60	emptyG2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	}	Cierra el bloque abierto anteriormente.
62	}	Cierra el bloque abierto anteriormente.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	@Override public Component prepareRenderer(javax.swing.table.TableCellRenderer renderer,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
65	int row, int column) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	Component componente = super.prepareRenderer(renderer, row, column);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	if (componente instanceof JComponent componenteSwing) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
68	componenteSwing.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
69	componenteSwing.setBorder(BorderFactory.createEmptyBorder(0, 10, 0, 10));	Configura una propiedad mediante setBorder en el objeto indicado.
70	}	Cierra el bloque abierto anteriormente.
71	return componente;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
72	}	Cierra el bloque abierto anteriormente.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	private void actualizarHover(java.awt.event.MouseEvent evento) {	Declara el método actualizarHover. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
75	int anterior = filaHover;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	filaHover = rowAtPoint(evento.getPoint());	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
77	if (anterior != filaHover) repaint();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
78	}	Cierra el bloque abierto anteriormente.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	{	Abre un bloque de instrucciones perteneciente a la declaración anterior.
81	addMouseListener(new java.awt.event.MouseMotionAdapter() {	Crea una nueva instancia de java.awt.event.MouseMotionAdapter y la usa o guarda para continuar el proceso.
82	@Override public void mouseMoved(java.awt.event.MouseEvent evento) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
83	actualizarHover(evento);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.

86	addMouseListener(new java.awt.event.MouseAdapter() {	Crea una nueva instancia de java.awt.event.MouseAdapter y la usa o guarda para continuar el proceso.
87	@Override public void mouseExited(java.awt.event.MouseEvent evento) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
88	if (filaHover != -1) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
89	filaHover = -1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
90	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	}	Cierra el bloque abierto anteriormente.
92	}	Cierra el bloque abierto anteriormente.
93	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	}	Cierra el bloque abierto anteriormente.
95	};	Cierra el bloque abierto anteriormente y termina la declaración.
96	private final JLabel total = new JLabel("Total: Q0.00");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
97	private final JLabel contador = new JLabel("0 productos") {	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
98	@Override protected void paintComponent(Graphics graphics) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
99	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
100	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
101	g2.setColor(new Color(239, 241, 244));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
102	g2.fillRoundRect(0, 0, getWidth(), getHeight(), 18, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
105	}	Cierra el bloque abierto anteriormente.
106	};	Cierra el bloque abierto anteriormente y termina la declaración.
107	private final JButton cobrar = boton("Continuar al pago", AMARILLO, TINTA);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
108	private final JButton mas = boton("+", AMARILLO, TINTA), menos = boton("-", new Color(255, 246, 217), TINTA) , quitar = boton("Quitar", new Color(255, 237, 238), ROJO);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
109	private final JButton cancelar = boton("Cancelar pedido", ROJO, Color.WHITE), ultimo = boton("Último comprob ante", new Color(245, 246, 248), TINTA);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
110	private boolean ocupado;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	private SolicitudPago pendiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
112	private String comprobante;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
113	private String ticketCocina;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	private final Utilidades.PagoPendienteStore respaldo = new Utilidades.PagoPendienteStore(SesionUsuario.getId Usuario());	Crea una nueva instancia de Utilidades.PagoPendienteStore y la usa o guarda para continuar el proceso.
115	private final Utilidades.UltimoComprobanteStore ultimoComprobanteStore =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
116	new Utilidades.UltimoComprobanteStore(SesionUsuario.getIdUsuario());	Crea una nueva instancia de Utilidades.UltimoComprobanteStore y la usa o guarda para continuar el proceso.
117	private boolean recuperacionFallida;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	public PedidoPanel() {	Constructor de PedidoPanel; prepara un objeto nuevo y deja sus componentes listos para usarse.
120	super();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	setLayout(vistasPanel);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	resumenPedido.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
124	resumenPedido.setBorder(BorderFactory.createEmptyBorder(22, 16, 22, 16));	Configura una propiedad mediante setBorder en el objeto indicado.
125	JPanel encabezadoPedido = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 0)) {	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
126	@Override protected void paintComponent(Graphics graphics) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
127	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

128	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
129	g2.setColor(ROJO_HEADER);	Configura una propiedad mediante setColor en el objeto indicado.
130	g2.fillRoundRect(0, 0, getWidth(), getHeight(), 22, 22);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	}	Cierra el bloque abierto anteriormente.
134	};	Cierra el bloque abierto anteriormente y termina la declaración.
135	encabezadoPedido.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
136	JPanel indicador = new JPanel() {	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
137	@Override protected void paintComponent(Graphics graphics) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
138	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
139	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
140	g2.setColor(AMARILLO);	Configura una propiedad mediante setColor en el objeto indicado.
141	g2.fillOval(1, 1, getWidth() - 2, getHeight() - 2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	}	Cierra el bloque abierto anteriormente.
144	};	Cierra el bloque abierto anteriormente y termina la declaración.
145	indicador.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
146	indicador.setPreferredSize(new Dimension(48, 48));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
147	JLabel marca = new JLabel("W", SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
148	marca.setFont(new Font(FUENTE, Font.BOLD, 25));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
149	marca.setForeground(ROJO_HEADER);	Configura una propiedad mediante setForeground en el objeto indicado.
150	indicador.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
151	indicador.add(marca, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152	JPanel textosHeader = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
153	textosHeader.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
154	textosHeader.setLayout(new BoxLayout(textosHeader, BoxLayout.Y_AXIS));	Crea una nueva instancia de BoxLayout y la usa o guarda para continuar el proceso.
155	JLabel titulo = new JLabel("PEDIDO");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
156	titulo.setFont(new Font(FUENTE, Font.BOLD, 21));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
157	titulo.setForeground(Color.WHITE);	Configura una propiedad mediante setForeground en el objeto indicado.
158	JLabel subtitulo = new JLabel("RESUMEN DEL PEDIDO");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
159	subtitulo.setFont(new Font(FUENTE, Font.BOLD, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
160	subtitulo.setForeground(new Color(255, 235, 192));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
161	textosHeader.add(titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	textosHeader.add(subtitulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	encabezadoPedido.add(indicador);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
164	encabezadoPedido.add(textosHeader);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
165	encabezadoPedido.setBorder(BorderFactory.createEmptyBorder(10, 14, 10, 14));	Configura una propiedad mediante setBorder en el objeto indicado.
166	resumenPedido.add(encabezadoPedido, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	tabla.setModel(modelo);	Configura una propiedad mediante setModel en el objeto indicado.
169	tabla.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
170	tabla.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
171	tabla.setBackground(new Color(247, 247, 249));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

172	<code>tabla.setRowHeight(76);</code>	Configura una propiedad mediante <code>setRowHeight</code> en el objeto indicado.
173	<code>tabla.setShowHorizontalLines(false);</code>	Configura una propiedad mediante <code>setShowHorizontalLines</code> en el objeto indicado.
174	<code>tabla.setShowVerticalLines(false);</code>	Configura una propiedad mediante <code>setShowVerticalLines</code> en el objeto indicado.
175	<code>tabla.setIntercellSpacing(new Dimension(0, 4));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
176	<code>tabla.setFont(new Font(FUENTE, Font.PLAIN, 13));</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
177	<code>tabla.setSelectionMode(ListSelectionMode.SINGLE_SELECTION);</code>	Configura una propiedad mediante <code>setSelectionMode</code> en el objeto indicado.
178	<code>tabla.setColorCabecera(AMARILLO);</code>	Configura una propiedad mediante <code>setColorCabecera</code> en el objeto indicado.
179	<code>tabla.setColorTextoCabecera(TINTA);</code>	Configura una propiedad mediante <code>setColorTextoCabecera</code> en el objeto indicado.
180	<code>tabla.setFuenteCabecera(new Font(FUENTE, Font.BOLD, 12));</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
181	<code>tabla.setAltoCabecera(42);</code>	Configura una propiedad mediante <code>setAltoCabecera</code> en el objeto indicado.
182	<code>tabla.setColorFilas(Color.WHITE);</code>	Configura una propiedad mediante <code>setColorFilas</code> en el objeto indicado.
183	<code>tabla.setFilasAlternadas(true);</code>	Configura una propiedad mediante <code>setFilasAlternadas</code> en el objeto indicado.
184	<code>tabla.setColorFilaAlterna(new Color(255, 251, 239));</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
185	<code>tabla.setColorSeleccion(new Color(255, 235, 170));</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
186	<code>tabla.setColorLineas(new Color(241, 236, 220));</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
187	<code>tabla.setPaddingHorizontal(10);</code>	Configura una propiedad mediante <code>setPaddingHorizontal</code> en el objeto indicado.
188	<code>tabla.setColumnsCentradas("1");</code>	Configura una propiedad mediante <code>setColumnsCentradas</code> en el objeto indicado.
189	<code>tabla.setColumnsDerecha("2");</code>	Configura una propiedad mediante <code>setColumnsDerecha</code> en el objeto indicado.
190	<code>tabla.getColumnModel().getColumn(0).setCellRenderer(</code>	Configura una propiedad mediante <code>setCellRenderer</code> en el objeto indicado.
191	<code>new javax.swing.table.DefaultTableCellRenderer() {</code>	Crea una nueva instancia de <code>javax.swing.table.DefaultTableCellRenderer</code> y la usa o guarda para continuar el proceso.
192	<code>@Override public Component getTableCellRendererComponent(JTable table,</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
193	<code>Object value, boolean selected, boolean focused, int row, int column) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	<code>JTextArea area = new JTextArea(String.valueOf(value).replace(" ", "\n • "));</code>	Crea una nueva instancia de <code>JTextArea</code> y la usa o guarda para continuar el proceso.
195	<code>area.setLineWrap(true); area.setWrapStyleWord(true); area.setEditable(false);</code>	Configura una propiedad mediante <code>setLineWrap</code> en el objeto indicado.
196	<code>area.setOpaque(false); area.setForeground(TINTA);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
197	<code>area.setFont(new Font(FUENTE, Font.PLAIN, 12));</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
198	<code>area.setBorder(BorderFactory.createEmptyBorder(7,10,5,7));</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
199	<code>return area;</code>	Finaliza el método y devuelve el valor indicado a quien lo llamó.
200	<code>}</code>	Cierra el bloque abierto anteriormente.
201	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202	<code>tabla.getColumnModel().getColumn(0).setPreferredWidth(180);</code>	Configura una propiedad mediante <code>setPreferredWidth</code> en el objeto indicado.
203	<code>tabla.getColumnModel().getColumn(1).setPreferredWidth(52);</code>	Configura una propiedad mediante <code>setPreferredWidth</code> en el objeto indicado.
204	<code>tabla.getColumnModel().getColumn(1).setMaxWidth(65);</code>	Configura una propiedad mediante <code>setMaxWidth</code> en el objeto indicado.
205	<code>tabla.getColumnModel().getColumn(2).setPreferredWidth(110);</code>	Configura una propiedad mediante <code>setPreferredWidth</code> en el objeto indicado.
206	<code>tabla.getColumnModel().getColumn(2).setMinWidth(95);</code>	Configura una propiedad mediante <code>setMinWidth</code> en el objeto indicado.
207	<code>JScrollPane scroll = new JScrollPane(tabla);</code>	Crea una nueva instancia de <code>JScrollPane</code> y la usa o guarda para continuar el proceso.
208	<code>Componentes.DesplazamientoSuave.ocultarBarras(scroll);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
209	<code>scroll.setColumnHeaderView(tabla.getTableHeader());</code>	Configura una propiedad mediante <code>setColumnHeaderView</code> en el objeto indicado.
210	<code>scroll.setPreferredSize(new Dimension(320, 180));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
211	<code>scroll.setBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
212	<code>scroll.setViewportBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante <code>setViewportBorder</code> en el objeto indicado.
213	<code>scroll.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
214	<code>scroll.getViewport().setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
215	<code>scroll.getViewport().setBackground(new Color(247, 247, 249));</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.

216	<code>javax.swing.table.JTableHeader encabezado = new javax.swing.table.JTableHeader(tabla.getColumnModel()) {</code>	Crea una nueva instancia de <code>javax.swing.table.JTableHeader</code> y la usa o guarda para continuar el proceso.
217	<code>@Override protected void paintComponent(Graphics graphics) {</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
218	<code>Graphics2D g2 = (Graphics2D) graphics.create();</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
219	<code>g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);</code>	Configura una propiedad mediante <code>setRenderingHint</code> en el objeto indicado.
220	<code>g2.setColor(AMARILLO);</code>	Configura una propiedad mediante <code>setColor</code> en el objeto indicado.
221	<code>g2.fillRoundRect(0, 0, Math.max(1, getWidth() - 1),</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
222	<code>Math.max(1, getHeight() - 1), 18, 18);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
223	<code>g2.setColor(TINTA);</code>	Configura una propiedad mediante <code>setColor</code> en el objeto indicado.
224	<code>g2.setFont(new Font(FUENTE, Font.BOLD, 12));</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
225	<code>for (int column = 0; column < getColumnModel().getColumnCount(); column++) {</code>	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
226	<code>Rectangle area = getHeaderRect(column);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
227	<code>String texto = tabla洗getColumnName(column);</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
228	<code>int x;</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
229	<code>if (column == 1) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
230	<code>x = area.x + (area.width - g2.getFontMetrics().stringWidth(texto)) / 2;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
231	<code>} else if (column == 2) {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
232	<code>x = area.x + area.width - g2.getFontMetrics().stringWidth(texto) - 10;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
233	<code>} else {</code>	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
234	<code>x = area.x + 10;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
235	<code>}</code>	Cierra el bloque abierto anteriormente.
236	<code>int y = (getHeight() - g2.getFontMetrics().getHeight()) / 2</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
237	<code>+ g2.getFontMetrics().getAscent();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
238	<code>g2.drawString(texto, x, y);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
239	<code>}</code>	Cierra el bloque abierto anteriormente.
240	<code>g2.dispose();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
241	<code>}</code>	Cierra el bloque abierto anteriormente.
242	<code>};</code>	Cierra el bloque abierto anteriormente y termina la declaración.
243	<code>encabezado.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
244	<code>encabezado.setBackground(Color.WHITE);</code>	Configura una propiedad mediante <code>setBackground</code> en el objeto indicado.
245	<code>encabezado.setBorder(BorderFactory.createEmptyBorder());</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
246	<code>encabezado.setPreferredSize(new Dimension(0, 36));</code>	Crea una nueva instancia de <code>Dimension</code> y la usa o guarda para continuar el proceso.
247	<code>encabezado.setReorderingAllowed(false);</code>	Configura una propiedad mediante <code>setReorderingAllowed</code> en el objeto indicado.
248	<code>encabezado.setResizingAllowed(false);</code>	Configura una propiedad mediante <code>setResizingAllowed</code> en el objeto indicado.
249	<code>encabezado.setFocusable(false);</code>	Configura una propiedad mediante <code>setFocusable</code> en el objeto indicado.
250	<code>encabezado.addMouseListener(new java.awt.event.MouseAdapter() {</code>	Crea una nueva instancia de <code>java.awt.event.MouseAdapter</code> y la usa o guarda para continuar el proceso.
251	<code>@Override public void mousePressed(java.awt.event.MouseEvent evento) { evento.consume(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
252	<code>@Override public void mouseReleased(java.awt.event.MouseEvent evento) { evento.consume(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
253	<code>@Override public void mouseClicked(java.awt.event.MouseEvent evento) { evento.consume(); }</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
254	<code>});</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
255	<code>encabezado.setDefaultRenderer(new javax.swing.table.DefaultTableCellRenderer() {</code>	Crea una nueva instancia de <code>javax.swing.table.DefaultTableCellRenderer</code> y la usa o guarda para continuar el proceso.
256	<code>@Override public Component getTableCellRendererComponent(javax.swing.JTable table,</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
257	<code>Object value, boolean selected, boolean focused, int row, int column) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	<code>JLabel label = (JLabel) super.getTableCellRendererComponent(</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
259	<code>table, value, selected, focused, row, column);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.

260	label.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
261	label.setForeground(TINTA);	Configura una propiedad mediante setForeground en el objeto indicado.
262	label.setFont(new Font(FUENTE, Font.BOLD, 12));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
263	label.setBorder(BorderFactory.createEmptyBorder(0, 10, 0, 10));	Configura una propiedad mediante setBorder en el objeto indicado.
264	label.setHorizontalAlignment(column == 1 ? SwingConstants.CENTER	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
265	: column == 2 ? SwingConstants.RIGHT : SwingConstants.LEFT);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266	return label;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
267	}	Cierra el bloque abierto anteriormente.
268	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269	tabla.setTableHeader(encabezado);	Configura una propiedad mediante setTableHeader en el objeto indicado.
270	scroll.setColumnHeaderView(encabezado);	Configura una propiedad mediante setColumnHeaderView en el objeto indicado.
271	scroll.getVerticalScrollBar().setUnitIncrement(24);	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
272	javax.swing.JScrollBar barraVertical = scroll.getVerticalScrollBar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
273	barraVertical.setUI(new BarraPedidoMinimalista());	Crea una nueva instancia de BarraPedidoMinimalista y la usa o guarda para continuar el proceso.
274	barraVertical.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
275	barraVertical.setPreferredSize(new Dimension(10, 0));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
276	barraVertical.setBackground(new Color(0, 0, 0, 0));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
277	JPanel esquinaAmarilla = new JPanel();	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
278	esquinaAmarilla.setOpaque(true);	Configura una propiedad mediante setOpaque en el objeto indicado.
279	esquinaAmarilla.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
280	scroll.setCorner(ScrollPaneConstants.UPPER_RIGHT_CORNER, esquinaAmarilla);	Configura una propiedad mediante setCorner en el objeto indicado.
281	JPanel centro = new JPanel(new BorderLayout(0, 10));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
282	centro.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
283	JPanel tablaContenedor = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
284	tablaContenedor.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
285	tablaContenedor.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
286	tablaContenedor.add(scroll);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
287	JPanel encabezadoTabla = new JPanel(new BorderLayout());	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
288	encabezadoTabla.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
289	JLabel tituloProductos = new JLabel("Productos seleccionados");	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
290	tituloProductos.setFont(new Font(FUENTE, Font.BOLD, 16));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
291	tituloProductos.setForeground(TINTA);	Configura una propiedad mediante setForeground en el objeto indicado.
292	contador.setFont(new Font(FUENTE, Font.BOLD, 11));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
293	contador.setForeground(new Color(31, 41, 55));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
294	contador.setHorizontalAlignment(SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
295	contador.setBorder(BorderFactory.createEmptyBorder(5, 11, 5, 11));	Configura una propiedad mediante setBorder en el objeto indicado.
296	contador.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
297	encabezadoTabla.add(tituloProductos, BorderLayout.WEST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	encabezadoTabla.add(contador, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
299	JPanel contenidoTabla = new JPanel(new BorderLayout(0, 8));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
300	contenidoTabla.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
301	contenidoTabla.add(encabezadoTabla, BorderLayout.NORTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
302	contenidoTabla.add(tablaContenedor, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
303	centro.add(contenidoTabla);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

304	JPanel cantidades = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
305	cantidades.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
306	menos.setPreferredSize(new Dimension(42, 36));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
307	menos.setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
308	mas.setMargin(new Insets(0, 0, 0, 0));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
309	mas.setPreferredSize(new Dimension(42, 36));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
310	quitar.setPreferredSize(new Dimension(92, 36));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
311	menos.setFont(new Font(FUENTE, Font.BOLD, 20));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
312	mas.setFont(new Font(FUENTE, Font.BOLD, 20));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
313	menos.setToolTipText("Reducir cantidad del producto seleccionado");	Configura una propiedad mediante setToolTipText en el objeto indicado.
314	mas.setToolTipText("Aumentar cantidad del producto seleccionado");	Configura una propiedad mediante setToolTipText en el objeto indicado.
315	quitar.setToolTipText("Quitar el producto seleccionado");	Configura una propiedad mediante setToolTipText en el objeto indicado.
316	cantidades.add(menos); cantidades.add(mas); cantidades.add(quitar);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
317	centro.add(cantidades, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
318	resumenPedido.add(centro, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
319		Línea en blanco: separa visualmente bloques para que el código sea más legible.
320	JPanel pie = new JPanel(new GridLayout(0, 1, 0, 10));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
321	pie.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
322	pie.setPreferredSize(new Dimension(320, 202));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
323	total.setFont(new Font(FUENTE, Font.BOLD, 23));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
324	total.setForeground(TIANTA);	Configura una propiedad mediante setForeground en el objeto indicado.
325	total.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, new Color(238, 231, 211)));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
326	pie.add(total); pie.add(cobrar); pie.add(cancelar); pie.add(ultimo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
327	resumenPedido.add(pie, BorderLayout.SOUTH);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
328	mas.addActionListener(e -> cambiar(1)); menos.addActionListener(e -> cambiar(-1)); quitar.addActionListener(e -> cambiar(-999));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
329	cancelar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
330	// Un cobro en curso conserva su solicitud; cerrar la vista no lo cancela.	Comentario para humanos; Java no ejecuta este texto. Un cobro en curso conserva su solicitud; cerrar la vista no lo cancela.
331	if (!bloqueado()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
332	if (!lineas.isEmpty() && JOptionPane.showConfirmDialog(this,	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
333	"¿Cancelar todo el pedido y volver al menú?", "Cancelar pedido",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
334	JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
335	lineas.clear();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
336	actualizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337	}	Cierra el bloque abierto anteriormente.
338	setVisible(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
339	if (getParent() != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
340	getParent().revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
341	getParent().repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
342	}	Cierra el bloque abierto anteriormente.
343	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
344	cobrar.addActionListener(e -> abrirPago()); ultimo.addActionListener(e -> mostrarComprobante());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
345	add(resumenPedido, "RESUMEN");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
346	vistasPanel.show(this, "RESUMEN");	Ejecuta una instrucción Java completa; el punto y coma marca su final.

347	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
348	pendiente = respaldo.cargar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
349	if (pendiente != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
350	if (pendiente.usuario() != SesionUsuario.getIdUsuario()) throw new java.io.IOException("El cobro pendiente pertenece a otro usuario.");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
351	for (LineaPedido l : pendiente.lineas()) lineas.put(l.idLinea(), l);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
352	}	Cierra el bloque abierto anteriormente.
353	} catch (java.io.IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
354	recuperacionFallida = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
355	SwingUtilities.invokeLater(() -> aviso(ex.getMessage()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
356	}	Cierra el bloque abierto anteriormente.
357	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
358	comprobante = ultimoComprobanteStore.cargar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
359	} catch (java.io.IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
360	SwingUtilities.invokeLater(() -> aviso(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
361	"No se pudo recuperar el último comprobante: " + ex.getMessage()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
362	}	Cierra el bloque abierto anteriormente.
363	actualizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
364	}	Cierra el bloque abierto anteriormente.
365		Línea en blanco: separa visualmente bloques para que el código sea más legible.
366	private static JButton boton(String texto, Color fondo, Color textoColor) {	Declara el método boton. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
367	BotonRedondeado b = new BotonRedondeado();	Crea una nueva instancia de BotonRedondeado y la usa o guarda para continuar el proceso.
368	b.setText(texto);	Cambia el texto visible de un componente de la interfaz.
369	b.setFont(new Font("SansSerif", Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
370	b.setDegradado(false);	Configura una propiedad mediante setDegradado en el objeto indicado.
371	b.setColorInicio(fondo);	Configura una propiedad mediante setColorInicio en el objeto indicado.
372	b.setForeground(textoColor);	Configura una propiedad mediante setForeground en el objeto indicado.
373	b.setRadio(20);	Configura una propiedad mediante setRadio en el objeto indicado.
374	b.setFocusPainted(true);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
375	b.setRolloverEnabled(true);	Configura una propiedad mediante setRolloverEnabled en el objeto indicado.
376	b.setMargin(new Insets(8, 12, 8, 12));	Crea una nueva instancia de Insets y la usa o guarda para continuar el proceso.
377	b.setPreferredSize(new Dimension(220, 44));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
378	return b;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
379	}	Cierra el bloque abierto anteriormente.
380		Línea en blanco: separa visualmente bloques para que el código sea más legible.
381	/** Scroll minimalista: solo muestra el pulgar, sin flechas ni riel gris. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
382	private static final class BarraPedidoMinimalista	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
383	extends javax.swing.plaf.basic.BasicScrollBarUI {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
384		Línea en blanco: separa visualmente bloques para que el código sea más legible.
385	@Override protected JButton createDecreaseButton(int orientation) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
386	return botonInvisible();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
387	}	Cierra el bloque abierto anteriormente.
388		Línea en blanco: separa visualmente bloques para que el código sea más legible.
389	@Override protected JButton createIncreaseButton(int orientation) {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
390	return botonInvisible();	Finaliza el método y devuelve el valor indicado a quien lo llamó.

391	}	Cierra el bloque abierto anteriormente.
392		Línea en blanco: separa visualmente bloques para que el código sea más legible.
393	@Override protected void paintTrack(Graphics graphics, JComponent componente,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
394	Rectangle limites) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
395	// El riel queda transparente para mostrar únicamente la barra.	Comentario para humanos; Java no ejecuta este texto. El riel queda transparente para mostrar únicamente la barra.
396	}	Cierra el bloque abierto anteriormente.
397		Línea en blanco: separa visualmente bloques para que el código sea más legible.
398	@Override protected void paintThumb(Graphics graphics, JComponent componente,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
399	Rectangle limites) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
400	if (limites.isEmpty()) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
401	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
402	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
403	g2.setColor(new Color(110, 119, 132, 170));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
404	int ancho = Math.min(6, Math.max(4, limites.width - 3));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
405	int x = limites.x + (limites.width - ancho) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
406	g2.fillRoundRect(x, limites.y + 2, ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
407	Math.max(8, limites.height - 4), ancho, ancho);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
408	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
409	}	Cierra el bloque abierto anteriormente.
410		Línea en blanco: separa visualmente bloques para que el código sea más legible.
411	private static JButton botonInvisible() {	Declara el método botonInvisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
412	JButton boton = new JButton();	Crea una nueva instancia de JButton y la usa o guarda para continuar el proceso.
413	boton.setPreferredSize(new Dimension(0, 0));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
414	boton.setMinimumSize(new Dimension(0, 0));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
415	boton.setMaximumSize(new Dimension(0, 0));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
416	boton.setBorder(BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
417	boton.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
418	boton.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
419	boton.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
420	return boton;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
421	}	Cierra el bloque abierto anteriormente.
422	}	Cierra el bloque abierto anteriormente.
423		Línea en blanco: separa visualmente bloques para que el código sea más legible.
424	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
425	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
426	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
427	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);	Configura una propiedad mediante setRenderingHint en el objeto indicado.
428	g2.setColor(new Color(0, 0, 0, 10));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
429	g2.fillRoundRect(2, 4, Math.max(1, getWidth() - 4), Math.max(1, getHeight() - 6), 30, 30);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430	g2.setColor(Color.WHITE);	Configura una propiedad mediante setColor en el objeto indicado.
431	g2.fillRoundRect(0, 0, getWidth(), Math.max(1, getHeight() - 2), 30, 30);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
432	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
434	}	Cierra el bloque abierto anteriormente.

435		Línea en blanco: separa visualmente bloques para que el código sea más legible.
436	public void agregar(LineaPedido linea) {	Declara el método agregar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
437	if (bloqueado()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
438	aviso("Termine o reintente el cobro pendiente antes de modificar el pedido.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
439	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
440	}	Cierra el bloque abierto anteriormente.
441	lineas.put(linea.idLinea(), linea);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
442	actualizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
443	}	Cierra el bloque abierto anteriormente.
444		Línea en blanco: separa visualmente bloques para que el código sea más legible.
445	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
446	* Sustituye temporalmente el resumen por la personalización del producto.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
447	* Al agregar o volver se recupera el mismo carrito, sin perder sus líneas.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
448	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
449	public void mostrarConfigurador(ConfiguracionProducto producto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
450	Runnable alFinalizar) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
451	if (bloqueado()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
452	aviso("Termine o reintente el cobro pendiente antes de agregar productos.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
453	if (alFinalizar != null) alFinalizar.run();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
454	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
455	}	Cierra el bloque abierto anteriormente.
456	if (configuradorActual != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
457	remove(configuradorActual);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
458	}	Cierra el bloque abierto anteriormente.
459	configuradorActual = new ConfiguradorProductoPanel(producto,	Crea una nueva instancia de ConfiguradorProductoPanel y la usa o guarda para continuar el proceso.
460	linea -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461	agregar(linea);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
462	mostrarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
463	if (alFinalizar != null) alFinalizar.run();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
464	},	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
465	() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
466	mostrarResumen();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
467	if (alFinalizar != null) alFinalizar.run();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
468	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
469	add(configuradorActual, "CONFIGURADOR");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
470	vistasPanel.show(this, "CONFIGURADOR");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
471	revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
472	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
473	}	Cierra el bloque abierto anteriormente.
474		Línea en blanco: separa visualmente bloques para que el código sea más legible.
475	private void mostrarResumen() {	Declara el método mostrarResumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
476	vistasPanel.show(this, "RESUMEN");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
477	if (configuradorActual != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
478	remove(configuradorActual);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

479	configuradorActual = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
480	}	Cierra el bloque abierto anteriormente.
481	revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
482	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
483	}	Cierra el bloque abierto anteriormente.
484		Línea en blanco: separa visualmente bloques para que el código sea más legible.
485	private void cambiar(int incremento) {	Declara el método cambiar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
486	if (bloqueado() tabla.getSelectedRow() < 0) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
487	LineaPedido l = new ArrayList<>(lineas.values()).get(tabla.getSelectedRow());	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
488	int n = l.cantidad() + incremento;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
489	if (n <= 0) lineas.remove(l.idLinea());	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
490	else if (n <= 999) lineas.put(l.idLinea(), l.conCantidad(n));	Prueba otra condición únicamente cuando la condición anterior no se cumplió.
491	actualizar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
492	}	Cierra el bloque abierto anteriormente.
493		Línea en blanco: separa visualmente bloques para que el código sea más legible.
494	private boolean bloqueado() { return ocupado pendiente != null recuperacionFallida; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
495	private BigDecimal suma() { return lineas.values().stream().map(LineaPedido::subtotal).reduce(new BigDecimal("0.00"), BigDecimal::add); }	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
496	private void actualizar() {	Declara el método actualizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
497	int seleccion = tabla.getSelectedRow(); modelo.setRowCount(0);	Configura una propiedad mediante setRowCount en el objeto indicado.
498	for (LineaPedido l : lineas.values()) modelo.addRow(new Object[]{l.resumen(), l.cantidad(), "Q" + l.subtotal().toString()});	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
499	if (seleccion >= 0 && seleccion < modelo.getRowCount()) tabla.setRowSelectionInterval(seleccion, seleccion);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
500	total.setText("Total: Q" + suma().toString());	Cambia el texto visible de un componente de la interfaz.
501	contador.setText(modelo.getRowCount() + (modelo.getRowCount() == 1 ? " producto" : " productos"));	Cambia el texto visible de un componente de la interfaz.
502	cobrar.setText(pendiente == null ? "Continuar al pago" : "Reintentar cobro pendiente");	Cambia el texto visible de un componente de la interfaz.
503	cobrar.setEnabled(!ocupado && !recuperacionFallida && !lineas.isEmpty());	Habilita o deshabilita la interacción con el componente.
504	for (JButton b : new JButton[]{mas, menos, quitar}) b.setEnabled(!bloqueado() && !lineas.isEmpty());	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
505	cancelar.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
506	ultimo.setEnabled(comprobante != null);	Habilita o deshabilita la interacción con el componente.
507	}	Cierra el bloque abierto anteriormente.
508		Línea en blanco: separa visualmente bloques para que el código sea más legible.
509	private void abrirPago() {	Declara el método abrirPago. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
510	if (ocupado recuperacionFallida lineas.isEmpty()) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
511	JDialog dialogo = new JDialog(SwingUtilities.getWindowAncestor(this), "Cobrar pedido", Dialog.ModalityType.APPLICATION_MODAL);	Crea una nueva instancia de JDialog y la usa o guarda para continuar el proceso.
512	JPanel campos = new JPanel(new GridLayout(0, 1, 5, 8)); campos.setBorder(BorderFactory.createEmptyBorder(20,20,20,20)); campos.setBackground(Color.WHITE);	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
513	JComboBox<String> servicio = new JComboBox<>(new String[]{"Comer aquí", "Para llevar"});	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
514	JComboBox<String> metodo = new JComboBox<>(new String[]{"Efectivo", "Tarjeta (terminal externa)"});	Crea una nueva instancia de JComboBox y la usa o guarda para continuar el proceso.
515	TextField recibido = new TextField(suma().toString(), referencia = new TextField());	Crea una nueva instancia de TextField y la usa o guarda para continuar el proceso.
516	JCheckBox confirmado = new JCheckBox("Pago aprobado en la terminal externa");	Crea una nueva instancia de JCheckBox y la usa o guarda para continuar el proceso.
517	JLabel cambio = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
518	JButton guardar = boton("Confirmar pago", AMARILLO, TINTA), volver = boton("Volver al pedido", new Color(245, 246, 248), TINTA);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

519	campos.add(new JLabel("Total a pagar: Q" + suma().toString())); campos.add(servicio); campos.add(me	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
520	todo); campos.add(new JLabel("Monto recibido")); campos.add(recibido); campos.add(cambio);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
521	campos.add(new JLabel("Referencia de terminal (sin datos de tarjeta)")); campos.add(referencia); campos.	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
522	add(confirmado); campos.add(guardar); campos.add(volver);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
523	Runnable refrescar = () -> {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
524	boolean tarjeta = metodo.getSelectedIndex() == 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
525	recibido.setEnabled(!tarjeta && pendiente == null); referencia.setEnabled(tarjeta && pendiente == nu	Habilita o deshabilita la interacción con el componente.
526	ll); confirmado.setEnabled(tarjeta && pendiente == null);	
526	try { BigDecimal valor = monto(recibido.getText()).subtract(suma()); cambio.setText(valor.signum() <	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
527	0 ? "Faltan: Q" + valor.negate() : "Cambio: Q" + valor); }	
527	catch (IllegalArgumentException ex) { cambio.setText("Ingrese un monto con hasta dos decimales."); }	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
528	};	Cierra el bloque abierto anteriormente y termina la declaración.
529	recibido.getDocument().addDocumentListener(new DocumentListener() {	Crea una nueva instancia de DocumentListener y la usa o guarda para continuar el proceso.
530	public void insertUpdate(DocumentEvent e) { refrescar.run(); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
531	public void removeUpdate(DocumentEvent e) { refrescar.run(); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
532	public void changedUpdate(DocumentEvent e) { refrescar.run(); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
533	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
534	metodo.addActionListener(e -> { if (metodo.getSelectedIndex() == 1) recibido.setText(suma().toString	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
535	g()); refrescar.run(); });	
535	if (pendiente != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
536	servicio.setSelectedIndex(pendiente.servicio().equals("COMER_AQUI") ? 0 : 1);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
537	metodo.setSelectedIndex(pendiente.metodo().equals("EFECTIVO") ? 0 : 1);	Configura una propiedad mediante setSelectedIndex en el objeto indicado.
538	recibido.setText(pendiente.recibido().toString()); referencia.setText(pendiente.referencia());	Cambia el texto visible de un componente de la interfaz.
539	confirmado.setSelected(true);	
539	servicio.setEnabled(false); metodo.setEnabled(false); guardar.setText("Reintentar misma operación");	Cambia el texto visible de un componente de la interfaz.
540	}	Cierra el bloque abierto anteriormente.
541	refrescar.run();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
542	volver.addActionListener(e -> dialogo.dispose());	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
543	guardar.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
544	boolean nueva = pendiente == null;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
545	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
546	if (pendiente == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
547	boolean tarjeta = metodo.getSelectedIndex() == 1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
548	if (tarjeta && !confirmado.isSelected()) throw new IllegalArgumentException("Confirme primer	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
549	o la aprobación en la terminal externa.");	
549	pendiente = new SolicitudPago(UUID.randomUUID().toString(), SesionUsuario.getIdUsuario(), ne	Crea una nueva instancia de SolicitudPago y la usa o guarda para continuar el proceso.
550	w ArrayList<>(lineas.values()),	
550	servicio.getSelectedIndex() == 0 ? "COMER_AQUI" : "PARA_LLEVAR", tarjeta ? "TARJETA"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
551	: "EFECTIVO",	
551	tarjeta ? suma() : monto(recibido.getText()), tarjeta ? referencia.getText() : "");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
552	}	Cierra el bloque abierto anteriormente.
553	if (nueva) respaldo.guardar(pendiente);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
554	} catch (java.io.IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
555	pendiente = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
556	aviso("No se pudo guardar el cobro pendiente. No se ha enviado a MySQL: " + ex.getMessage()); re	Ejecuta una instrucción Java completa; el punto y coma marca su final.
	turn;	

557	} catch (IllegalArgumentException ex) { aviso(ex.getMessage()); return; }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
558	ocupado = true; actualizar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
559	guardar.setEnabled(false); volver.setEnabled(false); servicio.setEnabled(false); metodo.setEnabled(false); refrescar.run();	Habilita o deshabilita la interacción con el componente.
560	dialogo.setDefaultCloseOperation(JDialog.DO_NOTHING_ON_CLOSE);	Configura una propiedad mediante setDefaultCloseOperation en el objeto indicado.
561	SolicitudPago solicitud = pendiente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
562	new SwingWorker<String, Void>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
563	@Override protected String doInBackground() throws Exception { return new PagoDAO().cobrar(solicitud); }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
564	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
565	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
566	comprobante = get();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
567	ticketCocina = crearTicketCocina(solicitud);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
568	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
569	ultimoComprobanteStore.guardar(comprobante);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
570	} catch (java.io.IOException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
571	aviso("El pago se confirmó, pero no se pudo guardar el último comprobante: "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
572	+ ex.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
573	}	Cierra el bloque abierto anteriormente.
574	respaldo.eliminar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
575	pendiente = null; lineas.clear(); dialogo.dispose(); mostrarComprobante();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
576	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
577	Throwable causa = ex instanceof java.util.concurrent.ExecutionException ? ex.getCause()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
578	: ex;	
579	if (causa instanceof IllegalArgumentException) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
580	try { respaldo.eliminar(); pendiente = null; dialogo.dispose(); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
581	catch (java.io.IOException fallo) { aviso("No se pudo retirar el respaldo pendiente: "	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
582	+ fallo.getMessage()); }	
583	}	Cierra el bloque abierto anteriormente.
584	JOptionPane.showMessageDialog(dialogo.isDisplayable() ? dialogo : PedidoPanel.this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
585	mensaje(ex) + (pendiente != null ? "\nEl pedido se conserva. Reintente esta operación; no repita el cobro externo." : ""), "No se completó el registro", JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
586	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
587	ocupado = false; actualizar(); guardar.setEnabled(true); volver.setEnabled(true);	Habilita o deshabilita la interacción con el componente.
588	guardar.setText("Reintentar misma operación"); dialogo.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);	Cambia el texto visible de un componente de la interfaz.
589	}	Cierra el bloque abierto anteriormente.
590	}	Cierra el bloque abierto anteriormente.
591	}.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
592	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
593	dialogo.setContentPane(campos); dialogo.pack(); dialogo.setMinimumSize(new Dimension(440, dialogo.getHeight()));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
594	dialogo.setLocationRelativeTo(this); dialogo.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
595	}	Cierra el bloque abierto anteriormente.
596		Línea en blanco: separa visualmente bloques para que el código sea más legible.
597	public static BigDecimal monto(String texto) {	Declara el método monto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

596	if (texto == null !texto.trim().matches("[0-9]{1,8}([.][0-9]{1,2})?")) throw new IllegalArgumentExceptionExce	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
597	ption("Monto inválido.");	
597	return new BigDecimal(texto.trim().replace(',', ' ').setScale(2);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
598	}	Cierra el bloque abierto anteriormente.
599	private void mostrarComprobante() {	Declara el método mostrarComprobante. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
600	if (comprobante == null) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
601	JTextArea texto = new JTextArea(comprobante, 20, 38); texto.setEditable(false); texto.setFont(new Font(F	Crea una nueva instancia de JTextArea y la usa o guarda para continuar el proceso.
601	ont.MONOSPACED, Font.PLAIN, 13));	
602	Object[] opciones = {"Cerrar", "Imprimir comprobante", "Ver / imprimir cocina"};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
603	JScrollPane scrollComprobante = new JScrollPane(texto);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
604	Componentes.DesplazamientoSuave.ocultarBarras(scrollComprobante);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
605	int eleccion = JOptionPane.showOptionDialog(this, scrollComprobante, "Comprobante",	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
606	JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE, null, opciones, opciones[0]);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
607	if (eleccion == 1) imprimir(texto);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
608	if (eleccion == 2 && ticketCocina != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
609	JTextArea cocina = new JTextArea(ticketCocina, 24, 42);	Crea una nueva instancia de JTextArea y la usa o guarda para continuar el proceso.
610	cocina.setEditable(false); cocina.setFont(new Font(Font.MONOSPACED, Font.BOLD, 13));	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
611	Object[] acciones = {"Cerrar", "Imprimir cocina"};	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
612	JScrollPane scrollCocina = new JScrollPane(cocina);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
613	Componentes.DesplazamientoSuave.ocultarBarras(scrollCocina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
614	if (JOptionPane.showOptionDialog(this, scrollCocina, "Ticket de cocina",	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
615	JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE, null, acciones, acciones[0]) == 1)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
616	imprimir(cocina);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
617	}	Cierra el bloque abierto anteriormente.
618	}	Cierra el bloque abierto anteriormente.
619	private void imprimir(JTextArea texto) {	Declara el método imprimir. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
620	try { texto.print(); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
621	catch (java.awt.print.PrinterException ex) {	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
622	aviso("No se pudo imprimir. Puedes reabrir el comprobante. " + ex.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
623	}	Cierra el bloque abierto anteriormente.
624	}	Cierra el bloque abierto anteriormente.
625	private static String crearTicketCocina(SolicitudPago solicitud) {	Declara el método crearTicketCocina. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
626	StringBuilder b = new StringBuilder("WALDONALD'S - COCINA\n")	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
627	.append(solicitud.servicio().equals("COMER_AQUI") ? "COMER AQUI" : "PARA LLEVAR")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
628	.append("\n=====\\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
629	for (LineaPedido linea : solicitud.lineas()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
630	b.append('\\n').append(linea.cantidad()).append(" x ").append(linea.nombre())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
631	.append(" [").append(linea.presentacion()).append("]\\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
632	Map<String,Integer> repetidos = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
633	Map<String,Long> totalesNombre = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
634	linea.opciones().stream().flatMap(o -> o.productos().stream()).forEach(p ->	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
635	totalesNombre.merge(p.nombre(), 1L, Long::sum));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
636	for (OpcionPedido opcion : linea.opciones()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
637	if (!opcion.grupo().startsWith("__"))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

638	b.append(" ").append(opcion.grupo()).append(": ").append(opcion.opcion()).append('\n');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
639	for (ProductoPedido producto : opcion.productos()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
640	int numero = repetidos.merge(producto.nombre(), 1, Integer::sum);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
641	b.append(" > ").append(producto.nombre());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
642	if (totalesNombre.getDefault(producto.nombre(), 0L) > 1)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
643	b.append(" #").append(numero);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
644	if (producto.modificaciones().isEmpty()) b.append(" - CON TODO");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
645	b.append('\n');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
646	for (ModificacionPedido cambio : producto.modificaciones())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
647	b.append(" ** ").append(cambio.tipo().equals("SIN") ? "SIN " : "EXTRA ")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
648	.append(cambio.ingrediente())	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
649	.append(cambio.veces() > 1 ? " x" + cambio.veces() : "").append(" **\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
650	}	Cierra el bloque abierto anteriormente.
651	}	Cierra el bloque abierto anteriormente.
652	}	Cierra el bloque abierto anteriormente.
653	return b.append("\n===== \n").toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
654	}	Cierra el bloque abierto anteriormente.
655	private static String mensaje(Exception ex) {	Declara el método mensaje. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
656	Throwable causa = ex instanceof java.util.concurrent.ExecutionException ? ex.getCause() : ex;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
657	return causa.getMessage() == null ? "No se pudo completar la operación." : causa.getMessage();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
658	}	Cierra el bloque abierto anteriormente.
659	private void aviso(String mensaje) { JOptionPane.showMessageDialog(this, mensaje, "Pedido", JOptionPane.WARNING_MESSAGE); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
660	}	Cierra el bloque abierto anteriormente.

16.98 src/GUI_CAJERO/promoCategoriasPanel.form

Metadatos visuales de NetBeans para diseñar promoCategoriasPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 53 | Archivo 98 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
6	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,0,-66,0,0,3,97"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	</AuxValues>	Cierra la sección XML abierta anteriormente.
16		Línea en blanco para organizar el XML del formulario.
17	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
18	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
19	</Layout>	Cierra la sección XML abierta anteriormente.
20	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<Container class="javax.swing.JPanel" name="jPanel1">	Declara un componente o contenedor visual y su clase Java.
22	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
23	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	</Property>	Cierra la sección XML abierta anteriormente.
26	</Properties>	Cierra la sección XML abierta anteriormente.
27	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
28	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
29	<AbsoluteConstraints x="0" y="0" width="1790" height="190"/>	Define la posición y el tamaño del componente dentro del diseño.
30	</Constraint>	Cierra la sección XML abierta anteriormente.
31	</Constraints>	Cierra la sección XML abierta anteriormente.
32		Línea en blanco para organizar el XML del formulario.
33	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
34	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	</Layout>	Cierra la sección XML abierta anteriormente.
36	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
37	<Component class="Componentes.BotonCategoria" name="botonCategoria1">	Declara un componente o contenedor visual y su clase Java.

38	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
39	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
40		Metadato de NetBeans que conserva el estado editable del diseñador visual.
41	</Property>	Cierra la sección XML abierta anteriormente.
42	<Property name="verticalAlignment" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
43	</Properties>	Cierra la sección XML abierta anteriormente.
44	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
45	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
46	<AbsoluteConstraints x="10" y="10" width="180" height="170"/>	Define la posición y el tamaño del componente dentro del diseño.
47	</Constraint>	Cierra la sección XML abierta anteriormente.
48	</Constraints>	Cierra la sección XML abierta anteriormente.
49	</Component>	Cierra la sección XML abierta anteriormente.
50	</SubComponents>	Cierra la sección XML abierta anteriormente.
51	</Container>	Cierra la sección XML abierta anteriormente.
52	</SubComponents>	Cierra la sección XML abierta anteriormente.
53	</Form>	Cierra la sección XML abierta anteriormente.

16.99 src/GUI_CAJERO/promoCategoriasPanel.java

Interfaz del módulo cajero: promoCategoriasPanel. Participa en la selección y venta de productos.

Tipo: java | Líneas: 51 | Archivo 99 de 157

Clases/tipos detectados: promoCategoriasPanel

Línea	Constructor o método detectado
14	public promoCategoriasPanel() { this(null); }
16	public promoCategoriasPanel(Consumer<JPanel> panelDestino) {
21	private void inicializarVista() {
38	private void agregarCategoria(String nombre, int x, String categoriaProductos, List<String> subcategorias) {
48	private void mostrarCategoria(String nombre, List<String> subcategorias) {

1 package GUI_CAJERO;			Declara el paquete o carpeta lógica a la que pertenece esta clase.
2			Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.Dimension;			Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
4 import java.util.List;			Importa java.util.List: colección ordenada que admite varios elementos.
5 import java.util.function.Consumer;			Importa java.util.function.Consumer para poder usar esa clase sin escribir su nombre completo.
6 import javax.swing.JPanel;			Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
7			Línea en blanco: separa visualmente bloques para que el código sea más legible.
8 /** Selector de promociones con el mismo patrón visual del menú principal. */			Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9 public class promoCategoriasPanel extends JPanel {			Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
10			Línea en blanco: separa visualmente bloques para que el código sea más legible.
11 private final Consumer<JPanel> panelDestino;			Declara una constante o referencia que no puede reasignarse después de inicializarse.
12 private final JPanel panelCategorias = new JPanel();			Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
13			Línea en blanco: separa visualmente bloques para que el código sea más legible.
14 public promoCategoriasPanel() { this(null); }			Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15			Línea en blanco: separa visualmente bloques para que el código sea más legible.
16 public promoCategoriasPanel(Consumer<JPanel> panelDestino) {			Constructor de promoCategoriasPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
17 this.panelDestino = panelDestino;			Accede al atributo del objeto actual para asignarlo o utilizarlo.
18 inicializarVista();			Ejecuta una instrucción Java completa; el punto y coma marca su final.
19 }			Cierra el bloque abierto anteriormente.
20			Línea en blanco: separa visualmente bloques para que el código sea más legible.
21 private void inicializarVista() {			Declara el método inicializarVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
22 setBackground(java.awt.Color.WHITE);			Ejecuta una instrucción Java completa; el punto y coma marca su final.
23 setPreferredSize(new Dimension(1900, 170));			Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
24 setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());			Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
25 panelCategorias.setBackground(java.awt.Color.WHITE);			Configura una propiedad mediante setBackground en el objeto indicado.
26 panelCategorias.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());			Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
27 add(panelCategorias, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 1900, 170));			Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
28			Línea en blanco: separa visualmente bloques para que el código sea más legible.

29	agregarCategoria("Combos", 10, "Almuerzos", List.of("Hamburguesas", "McNuggets", "Combos", "Papas", "Bebidas"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	agregarCategoria("Familia", 210, "Cajita Feliz", List.of("Hamburguesas", "McNuggets", "Acompañamientos", "Bebidas", "Postres"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	agregarCategoria("Descuentos", 410, "Antojos", List.of("Papas", "McNuggets", "Snacks", "Acompañamientos", "Compartir"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	agregarCategoria("2x1", 610, "Bebidas", List.of("Gaseosas", "Jugos", "Agua", "Café", "Bebidas frías"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	agregarCategoria("Temporada", 810, "Postres", List.of("McFlurry", "Sundae", "Conos", "Pasteles", "Galletas"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34	agregarCategoria("Cajita Feliz", 1010, "Cajita Feliz", List.of("Hamburguesas", "McNuggets", "Acompañamientos", "Bebidas", "Postres", "Juguetes"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	agregarCategoria("McCafé", 1210, "WlCafé", List.of("Café", "Frappés", "Chocolate", "Té", "Bebidas frías", "Repostería"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	private void agregarCategoria(String nombre, int x, String categoriaProductos, List<String> subcategorias) {	Declara el método agregarCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
39	Componentes.BotonCategoria boton = new Componentes.BotonCategoria();	Crea una nueva instancia de Componentes.BotonCategoria y la usa o guarda para continuar el proceso.
40	boton.setText(nombre);	Cambia el texto visible de un componente de la interfaz.
41	boton.setFont(new java.awt.Font("Arial", java.awt.Font.BOLD, 24));	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
42	boton.setHideActionText(true);	Configura una propiedad mediante setHideActionText en el objeto indicado.
43	boton.setVerticalAlignment(javax.swing.SwingConstants.BOTTOM);	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
44	boton.addActionListener(e -> mostrarCategoria(categoriaProductos, subcategorias));	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
45	panelCategorias.add(boton, new org.netbeans.lib.awtextra.AbsoluteConstraints(x, 10, 180, 170));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
46	}	Cierra el bloque abierto anteriormente.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	private void mostrarCategoria(String nombre, List<String> subcategorias) {	Declara el método mostrarCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
49	if (panelDestino != null) panelDestino.accept(new SubCategoriasPanel(nombre, subcategorias));	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
50	}	Cierra el bloque abierto anteriormente.
51	}	Cierra el bloque abierto anteriormente.

16.100 src/GUI_CAJERO/promocionesPanel.form

Metadatos visuales de NetBeans para diseñar promocionesPanel. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 71 | Archivo 100 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JPanelFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
7	</Property>	Cierra la sección XML abierta anteriormente.
8	</Properties>	Cierra la sección XML abierta anteriormente.
9	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
11	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,4,56,0,0,7,-128"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	</AuxValues>	Cierra la sección XML abierta anteriormente.
21		Línea en blanco para organizar el XML del formulario.
22	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
23	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
24	</Layout>	Cierra la sección XML abierta anteriormente.
25	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component class="javax.swing.JLabel" name="jLabel1">	Declara un componente o contenedor visual y su clase Java.
27	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
29		Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	</Property>	Cierra la sección XML abierta anteriormente.
31	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
32	<Color blue="1b" green="d" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
33	</Property>	Cierra la sección XML abierta anteriormente.
34	<Property name="text" type="java.lang.String" value="Promociones"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
35	</Properties>	Cierra la sección XML abierta anteriormente.
36	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
37	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.

38	<AbsoluteConstraints x="70" y="20" width="760" height="100"/>	Define la posición y el tamaño del componente dentro del diseño.
39	</Constraint>	Cierra la sección XML abierta anteriormente.
40	</Constraints>	Cierra la sección XML abierta anteriormente.
41	</Component>	Cierra la sección XML abierta anteriormente.
42	<Container class="javax.swing.JPanel" name="panelCategoriaPromo">	Declara un componente o contenedor visual y su clase Java.
43	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
44	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
45	<Color blue="fff" green="fff" red="fff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	</Property>	Cierra la sección XML abierta anteriormente.
47	</Properties>	Cierra la sección XML abierta anteriormente.
48	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
49	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
50	<AbsoluteConstraints x="70" y="150" width="1790" height="190"/>	Define la posición y el tamaño del componente dentro del diseño.
51	</Constraint>	Cierra la sección XML abierta anteriormente.
52	</Constraints>	Cierra la sección XML abierta anteriormente.
53		Línea en blanco para organizar el XML del formulario.
54	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignFlowLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
55	</Container>	Cierra la sección XML abierta anteriormente.
56	<Container class="javax.swing.JPanel" name="panelContenidoPromo">	Declara un componente o contenedor visual y su clase Java.
57	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
58	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
59	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
60	</Property>	Cierra la sección XML abierta anteriormente.
61	</Properties>	Cierra la sección XML abierta anteriormente.
62	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
63	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
64	<AbsoluteConstraints x="70" y="410" width="1810" height="660"/>	Define la posición y el tamaño del componente dentro del diseño.
65	</Constraint>	Cierra la sección XML abierta anteriormente.
66	</Constraints>	Cierra la sección XML abierta anteriormente.
67		Línea en blanco para organizar el XML del formulario.
68	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignFlowLayout"/>	Indica el administrador de distribución que organiza los componentes hijos.
69	</Container>	Cierra la sección XML abierta anteriormente.
70	</SubComponents>	Cierra la sección XML abierta anteriormente.
71	</Form>	Cierra la sección XML abierta anteriormente.

16.101 src/GUI_CAJERO/promocionesPanel.java

Interfaz del módulo cajero: promocionesPanel. Participa en la selección y venta de productos.

Tipo: java | Líneas: 123 | Archivo 101 de 157

Clases/tipos detectados: promocionesPanel

Línea	Constructor o método detectado
14	public promocionesPanel() {
87	private void mostrarPanelPromocion(javax.swing.JPanel panel) {
97	private void initComponents() {

1 package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.awt.BorderLayout;	Importa java.awt.BorderLayout para poder usar esa clase sin escribir su nombre completo.
4 import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5 import java.awt.ComponentOrientation;	Importa java.awt.ComponentOrientation para poder usar esa clase sin escribir su nombre completo.
6 import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7 import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
8	Línea en blanco: separa visualmente bloques para que el código sea más legible.
9 public class promocionesPanel extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
10	Línea en blanco: separa visualmente bloques para que el código sea más legible.
11 private Componentes.Scroll_Categorias	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
12 scrollCategorias;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
13	Línea en blanco: separa visualmente bloques para que el código sea más legible.
14 public promocionesPanel() {	Constructor de promocionesPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
15	Línea en blanco: separa visualmente bloques para que el código sea más legible.
16 initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
17	Línea en blanco: separa visualmente bloques para que el código sea más legible.
18 // NO usar setLayout(null)	Comentario para humanos; Java no ejecuta este texto. NO usar setLayout(null)
19 // NetBeans ya utiliza GroupLayout	Comentario para humanos; Java no ejecuta este texto. NetBeans ya utiliza GroupLayout
20	Línea en blanco: separa visualmente bloques para que el código sea más legible.
21 // =====	Comentario para humanos; Java no ejecuta este texto. =====
22 // CREAR PANEL DE CATEGORÍAS	Comentario para humanos; Java no ejecuta este texto. CREAR PANEL DE CATEGORÍAS
23 // =====	Comentario para humanos; Java no ejecuta este texto. =====
24	Línea en blanco: separa visualmente bloques para que el código sea más legible.
25 promoCategoriasPanel categorias =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
26 new promoCategoriasPanel(this::mostrarPanelPromocion);	Crea una nueva instancia de promoCategoriasPanel y la usa o guarda para continuar el proceso.
27	Línea en blanco: separa visualmente bloques para que el código sea más legible.
28 categorias.setPreferredSize(	Configura una propiedad mediante setPreferredSize en el objeto indicado.
29 new Dimension(1900, 170)	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
30);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	Línea en blanco: separa visualmente bloques para que el código sea más legible.
32 categorias.setComponentOrientation(	Configura una propiedad mediante setComponentOrientation en el objeto indicado.

33	ComponentOrientation.LEFT_TO_RIGHT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	// =====	Comentario para humanos; Java no ejecuta este texto. =====
37	// CREAR SCROLL	Comentario para humanos; Java no ejecuta este texto. CREAR SCROLL
38	// =====	Comentario para humanos; Java no ejecuta este texto. =====
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	scrollCategorias =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
41	new Componentes.Scroll_Categorias();	Crea una nueva instancia de Componentes.Scroll_Categorias y la usa o guarda para continuar el proceso.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	scrollCategorias.setComponentOrientation(	Configura una propiedad mediante setComponentOrientation en el objeto indicado.
44	ComponentOrientation.LEFT_TO_RIGHT	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	scrollCategorias.setPanelCategorias(	Configura una propiedad mediante setPanelCategorias en el objeto indicado.
48	categorias	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	// =====	Comentario para humanos; Java no ejecuta este texto. =====
52	// AGREGAR SCROLL AL PANEL	Comentario para humanos; Java no ejecuta este texto. AGREGAR SCROLL AL PANEL
53	// =====	Comentario para humanos; Java no ejecuta este texto. =====
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.
55	panelCategoriaPromo.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	panelCategoriaPromo.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
58	new BorderLayout()	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
59	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	panelCategoriaPromo.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
62	Color.WHITE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	panelCategoriaPromo.add(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	scrollCategorias,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	BorderLayout.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	panelCategoriaPromo.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	panelCategoriaPromo.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	// =====	Comentario para humanos; Java no ejecuta este texto. =====
74	// INICIAR DESDE LA IZQUIERDA	Comentario para humanos; Java no ejecuta este texto. INICIAR DESDE LA IZQUIERDA
75	// =====	Comentario para humanos; Java no ejecuta este texto. =====
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.

77	SwingUtilities.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	scrollCategorias.irAlInicio();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	scrollCategorias.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	scrollCategorias.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	}	Cierra el bloque abierto anteriormente.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	private void mostrarPanelPromocion(javax.swing.JPanel panel) {	Declara el método mostrarPanelPromocion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
88	panelContenidoPromo.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	panelContenidoPromo.setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.
90	panelContenidoPromo.add(panel, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	panelContenidoPromo.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	panelContenidoPromo.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	}	Cierra el bloque abierto anteriormente.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
96	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
97	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	jLabel1 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
100	panelCategoriaPromo = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
101	panelContenidoPromo = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
104	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	jLabel1.setFont(new java.awt.Font("DM Sans 18pt", 1, 100)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
107	jLabel1.setForeground(new java.awt.Color(0, 13, 27));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
108	jLabel1.setText("Promociones");	Cambia el texto visible de un componente de la interfaz.
109	add(jLabel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 20, 760, 100));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	panelCategoriaPromo.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
112	add(panelCategoriaPromo, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 150, 1790, 190));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	panelContenidoPromo.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
115	add(panelContenidoPromo, new org.netbeans.lib.awtextra.AbsoluteConstraints(70, 410, 1810, 660));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
116	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117		Línea en blanco: separa visualmente bloques para que el código sea más legible.
118	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.

119	private javax.swing.JLabel jLabel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
120	private javax.swing.JPanel panelCategoriaPromo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
121	private javax.swing.JPanel panelContenidoPromo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
122	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
123	}	Cierra el bloque abierto anteriormente.

16.102 src/GUI_CAJERO/SubCatDesayunos.java

Interfaz del módulo cajero: SubCatDesayunos. Participa en la selección y venta de productos.

Tipo: java | Líneas: 119 | Archivo 102 de 157

Clases/tipos detectados: SubCatDesayunos

Línea	Constructor o método detectado
5	public SubCatDesayunos() {
51	private void initComponents() {

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	public class SubCatDesayunos extends javax.swing.JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public SubCatDesayunos() {	Constructor de SubCatDesayunos: prepara un objeto nuevo y deja sus componentes listos para usarse.
6	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
7	// 1 = id de la categoría Desayunos	Comentario para humanos; Java no ejecuta este texto. 1 = id de la categoría Desayunos
8	cargarProductos(1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
9	}	Cierra el bloque abierto anteriormente.
10		Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	private void	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
12	cargarProductos(int idCategoria) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	jPanel1.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	jPanel1.setLayout(	Configura una propiedad mediante setLayout en el objeto indicado.
17	new java.awt.FlowLayout(	Crea una nueva instancia de java.awt.FlowLayout y la usa o guarda para continuar el proceso.
18	java.awt.FlowLayout.LEFT,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	12	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	DAO.ProductoDAO productoDAO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25	= new DAO.ProductoDAO();	Crea una nueva instancia de DAO.ProductoDAO y la usa o guarda para continuar el proceso.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	java.util.List<Modelos.Productos> productos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	= productoDAO.obtenerPorCategoria(idCategoria);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	for (Modelos.Productos producto : productos) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	Componentes.TarjetaProducto tarjeta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	= new Componentes.TarjetaProducto(	Crea una nueva instancia de Componentes.TarjetaProducto y la usa o guarda para continuar el proceso.

34	producto.getIdProducto(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	producto.getNombre(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	producto.getDescripcion(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	producto.getPrecio(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	producto.getImagen(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	producto.isDisponibleHorario()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	jPanel1.add(tarjeta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	jPanel1.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	jPanel1.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
50	// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
51	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	jScrollPane1 = new javax.swing.JScrollPane();	Crea una nueva instancia de javax.swing.JScrollPane y la usa o guarda para continuar el proceso.
54	jPanel1 = new Componentes.PanelProductosVertical();	Crea una nueva instancia de Componentes.PanelProductosVertical y la usa o guarda para continuar el proceso.
55	botonLetras1 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
56	botonLetras2 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
57	botonLetras3 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
58	botonLetras4 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
59	botonLetras5 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
60	botonLetras6 = new Componentes.BotonLetras();	Crea una nueva instancia de Componentes.BotonLetras y la usa o guarda para continuar el proceso.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
63	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	jScrollPane1.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
66	jScrollPane1.setForeground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
67	jScrollPane1.setHorizontalScrollBarPolicy(	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
68	javax.swing.JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	jScrollPane1.setVerticalScrollBarPolicy(	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
70	javax.swing.JScrollPane.VERTICAL_SCROLLBAR_NEVER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	jScrollPane1.getHorizontalScrollBar().setEnabled(false);	Habilita o deshabilita la interacción con el componente.
72	jScrollPane1.setBorder(javax.swing.BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
73	java.awt.event.MouseWheelListener rueda = evento -> {	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
74	javax.swing.JScrollBar barra = jScrollPane1.getVerticalScrollBar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
75	int paso = evento.getUnitsToScroll() * barra.getUnitIncrement();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	barra.setValue(barra.getValue() + paso);	Configura una propiedad mediante setValue en el objeto indicado.
77	evento.consume();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

78	};	Cierra el bloque abierto anteriormente y termina la declaración.
79	jScrollPane1.addMouseWheelListener(rueda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	jScrollPane1.getViewPort().addMouseWheelListener(rueda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	jPanel1.addMouseWheelListener(rueda);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	jPanel1.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
84	jPanel1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
85	jScrollPane1.setViewportView(jPanel1);	Configura una propiedad mediante.setViewportViewView en el objeto indicado.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	add(jScrollPane1, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 50, 1810, 680));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	botonLetras1.setText("Por tiempo limitado");	Cambia el texto visible de un componente de la interfaz.
90	add(botonLetras1, new org.netbeans.lib.awtextra.AbsoluteConstraints(860, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
91		Línea en blanco: separa visualmente bloques para que el código sea más legible.
92	botonLetras2.setText("Todos");	Cambia el texto visible de un componente de la interfaz.
93	add(botonLetras2, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95	botonLetras3.setText("Sándwiches");	Cambia el texto visible de un componente de la interfaz.
96	add(botonLetras3, new org.netbeans.lib.awtextra.AbsoluteConstraints(180, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
97		Línea en blanco: separa visualmente bloques para que el código sea más legible.
98	botonLetras4.setText("McMuffin");	Cambia el texto visible de un componente de la interfaz.
99	add(botonLetras4, new org.netbeans.lib.awtextra.AbsoluteConstraints(350, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	botonLetras5.setText("Hot Cakes");	Cambia el texto visible de un componente de la interfaz.
102	add(botonLetras5, new org.netbeans.lib.awtextra.AbsoluteConstraints(520, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
103		Línea en blanco: separa visualmente bloques para que el código sea más legible.
104	botonLetras6.setText("Bebidas");	Cambia el texto visible de un componente de la interfaz.
105	add(botonLetras6, new org.netbeans.lib.awtextra.AbsoluteConstraints(690, 10, 160, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
106	}// </editor-fold>//GEN-END:initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107		Línea en blanco: separa visualmente bloques para que el código sea más legible.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
110	private Componentes.BotonLetras botonLetras1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	private Componentes.BotonLetras botonLetras2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
112	private Componentes.BotonLetras botonLetras3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
113	private Componentes.BotonLetras botonLetras4;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	private Componentes.BotonLetras botonLetras5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	private Componentes.BotonLetras botonLetras6;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
116	private javax.swing.JPanel jPanel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
117	private javax.swing.JScrollPane jScrollPane1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
118	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
119	}	Cierra el bloque abierto anteriormente.

16.103 src/GUI_CAJERO/SubCategoriasPanel.java

Interfaz del módulo cajero: SubCategoriasPanel. Participa en la selección y venta de productos.

Tipo: java | Líneas: 199 | Archivo 103 de 157

Clases/tipos detectados: SubCategoriasPanel, CategoriaDatos

Línea	Constructor o método detectado
35	private record CategoriaDatos(Integer id, List<String> subcategorias) { }
37	public SubCategoriasPanel(String nombreCategoria, List<String> subcategoriasConfiguradas) {
44	private void inicializarVista() {
63	private void configurarRuedaVertical() {
72	private void cargarCategoria() {
106	private void crearBoton(String texto, Integer idCategoria, String subcategoria,
121	private void mostrarProductos(int idCategoria, String subcategoria, Color colorFondo) {
155	private void cargarSiguienteLote(int version) {
179	private void mostrarEstado(String texto) {
189	private Color colorDeSubcategoria(String subcategoria) {

1	package GUI_CAJERO;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.BotonLetras;	Importa Componentes.BotonLetras para poder usar esa clase sin escribir su nombre completo.
4	import Componentes.TarjetaProducto;	Importa Componentes.TarjetaProducto para poder usar esa clase sin escribir su nombre completo.
5	import DAO.ProductoDAO;	Importa DAO.ProductoDAO para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.Productos;	Importa Modelos.Productos para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.FlowLayout;	Importa java.awt.FlowLayout para poder usar esa clase sin escribir su nombre completo.
10	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
11	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
12	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
13	import javax.swing.JScrollPane;	Importa javax.swing.JScrollPane para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.SwingWorker;	Importa javax.swing.SwingWorker: ejecuta trabajo lento fuera del hilo visual y publica el resultado después.
16	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	/** Vista de subcategorías con consulta asíncrona y renderizado por lotes. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	public class SubCategoriasPanel extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private static final int PRODUCTOS_POR_LOTE = 8;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
22	private final ProductoDAO productoDAO = new ProductoDAO();	Crea una nueva instancia de ProductoDAO y la usa o guarda para continuar el proceso.
23	private final String nombreCategoria;	Declara una constante o referencia que no puede reasignarse después de inicializarse.

24	private final List<String> subcategoriasConfiguradas;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
25	private final JPanel barraSubcategorias = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));	Crea una nueva instancia de JPanel y la usa o guarda para continuar el proceso.
26	private final JPanel panelProductos = new Componentes.PanelProductosVertical();	Crea una nueva instancia de Componentes.PanelProductosVertical y la usa o guarda para continuar el proceso.
27	private final JScrollPane scrollProductos = new JScrollPane(panelProductos);	Crea una nueva instancia de JScrollPane y la usa o guarda para continuar el proceso.
28	private SwingWorker<CategoriaDatos, Void> cargaCategoria;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	private SwingWorker<List<Productos>, Void> cargaProductos;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	private Timer temporizadorLotes;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31	private int versionVista;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
32	private List<Productos> productosPendientes = List.of();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33	private int indiceLote;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	private record CategoriaDatos(Integer id, List<String> subcategorias) { }	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	public SubCategoriasPanel(String nombreCategoria, List<String> subcategoriasConfiguradas) {	Constructor de SubCategoriasPanel: prepara un objeto nuevo y deja sus componentes listos para usarse.
38	this.nombreCategoria = nombreCategoria;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
39	this.subcategoriasConfiguradas = List.copyOf(subcategoriasConfiguradas);	Accede al atributo del objeto actual para asignarlo o utilizarlo.
40	inicializarVista();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	cargarCategoria();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	}	Cierra el bloque abierto anteriormente.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	private void inicializarVista() {	Declara el método inicializarVista. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
45	setBackground(Color.WHITE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
47	barraSubcategorias.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
48	add(barraSubcategorias, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 10, 1810, 30));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
49	panelProductos.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
50	scrollProductos.setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
51	scrollProductos.getViewPort().setBackground(Color.WHITE);	Configura una propiedad mediante setBackground en el objeto indicado.
52	scrollProductos.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setHorizontalScrollBarPolicy en el objeto indicado.
53	scrollProductos.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_NEVER);	Configura una propiedad mediante setVerticalScrollBarPolicy en el objeto indicado.
54	scrollProductos.setBorder(javax.swing.BorderFactory.createEmptyBorder());	Configura una propiedad mediante setBorder en el objeto indicado.
55	scrollProductos.setVerticalScrollBar().setUnitIncrement(32);	Configura una propiedad mediante setUnitIncrement en el objeto indicado.
56	scrollProductos.setVerticalScrollBar().setBlockIncrement(180);	Configura una propiedad mediante setBlockIncrement en el objeto indicado.
57	configurarRuedaVertical();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	// El área visible queda limitada; las filas restantes continúan abajo.	Comentario para humanos; Java no ejecuta este texto. El área visible queda limitada; las filas restantes continúan abajo.
59	add(scrollProductos, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 50, 1810, 610));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
60	mostrarEstado("Cargando productos...");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	}	Cierra el bloque abierto anteriormente.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	private void configurarRuedaVertical() {	Declara el método configurarRuedaVertical. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
64	Componentes.DesplazamientoSuave.instalarVertical(scrollProductos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65	Componentes.DesplazamientoSuave.instalarSobreVista(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	panelProductos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	scrollProductos.setVerticalScrollBar(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

68	false	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	}	Cierra el bloque abierto anteriormente.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	private void cargarCategoria() {	Declara el método cargarCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
73	if (cargaCategoria != null && !cargaCategoria.isDone()) cargaCategoria.cancel(true);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
74	final int version = ++versionVista;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
75	cargaCategoria = new SwingWorker<>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
76	@Override protected CategoriaDatos doInBackground() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
77	Integer id = productoDAO.obtenerIdCategoriaPorNombre(nombreCategoria);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
78	List<String> subcategorias = id == null	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	? subcategoriasConfiguradas	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	: productoDAO.obtenerSubcategorias(id);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	return new CategoriaDatos(id,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
82	subcategorias.isEmpty() ? subcategoriasConfiguradas : subcategorias);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
86	if (isCancelled() version != versionVista) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
87	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
88	CategoriaDatos datos = get();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
89	barraSubcategorias.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	crearBoton("Todos", datos.id(), null, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	for (String subcategoria : datos.subcategorias()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
92	crearBoton(subcategoria, datos.id(), subcategoria, false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	}	Cierra el bloque abierto anteriormente.
94	barraSubcategorias.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	barraSubcategorias.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	if (datos.id() != null) mostrarProductos(datos.id(), null, Color.WHITE);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
97	else mostrarEstado("No hay productos disponibles en esta categoría.");	Ruta alternativa que se ejecuta cuando la condición anterior fue falsa.
98	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
99	mostrarEstado("No se pudieron cargar las categorías.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101	}	Cierra el bloque abierto anteriormente.
102	};	Cierra el bloque abierto anteriormente y termina la declaración.
103	cargaCategoria.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104	}	Cierra el bloque abierto anteriormente.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	private void crearBoton(String texto, Integer idCategoria, String subcategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	boolean seleccionadoInicialmente) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
108	BotonLetras boton = new BotonLetras();	Crea una nueva instancia de BotonLetras y la usa o guarda para continuar el proceso.
109	boton.setText(texto);	Cambia el texto visible de un componente de la interfaz.
110	boton.setPreferredSize(new Dimension(160, 30));	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
111	boton.setSelected(selectedadoInicialmente);	Configura una propiedad mediante setSelected en el objeto indicado.

112	boton.addActionListener(e -> {	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
113	if (idCategoria != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
114	mostrarProductos(idCategoria, subcategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
115	subcategoria == null ? Color.WHITE : colorDeSubcategoria(subcategoria));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	}	Cierra el bloque abierto anteriormente.
117	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	barraSubcategorias.add(boton);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	}	Cierra el bloque abierto anteriormente.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	private void mostrarProductos(int idCategoria, String subcategoria, Color colorFondo) {	Declara el método mostrarProductos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
122	if (cargaProductos != null && !cargaProductos.isDone()) cargaProductos.cancel(true);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
123	if (temporizadorLotes != null) temporizadorLotes.stop();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
124	final int version = ++versionVista;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
125	panelProductos.setBackground(colorFondo);	Configura una propiedad mediante setBackground en el objeto indicado.
126	scrollProductos.getViewport().setBackground(colorFondo);	Configura una propiedad mediante setBackground en el objeto indicado.
127	mostrarEstado("Cargando productos...");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
128	cargaProductos = new SwingWorker<>() {	Crea una nueva instancia de SwingWorker y la usa o guarda para continuar el proceso.
129	@Override protected List<Productos> doInBackground() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
130	return subcategoria == null	Finaliza el método y devuelve el valor indicado a quien lo llamó.
131	? productoDAO.obtenerPorCategoria(idCategoria)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	: productoDAO.obtenerPorCategoriaYSubcategoria(idCategoria, subcategoria);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
133	}	Cierra el bloque abierto anteriormente.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	@Override protected void done() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
136	if (isCancelled() version != versionVista) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
137	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
138	productosPendientes = get();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
139	indiceLote = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
140	panelProductos.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	panelProductos.setLayout(new FlowLayout(FlowLayout.LEFT, 40, 15));	Crea una nueva instancia de FlowLayout y la usa o guarda para continuar el proceso.
142	if (productosPendientes.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
143	mostrarEstado("No hay productos disponibles.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
145	cargarSiguienteLote(version);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146	}	Cierra el bloque abierto anteriormente.
147	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
148	mostrarEstado("No se pudieron cargar los productos.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	}	Cierra el bloque abierto anteriormente.
150	}	Cierra el bloque abierto anteriormente.
151	};	Cierra el bloque abierto anteriormente y termina la declaración.
152	cargaProductos.execute();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
153	}	Cierra el bloque abierto anteriormente.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.

155	private void cargarSiguienteLote(int version) {	Declara el método cargarSiguienteLote. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
156	if (version != versionVista) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
157	int fin = Math.min(indiceLote + PRODUCTOS_POR_LOTE, productosPendientes.size());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
158	for (int i = indiceLote; i < fin; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
159	Productos producto = productosPendientes.get(i);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
160	panelProductos.add(new TarjetaProducto(producto.getIdProducto(), producto.getNombre(),	Crea una nueva instancia de TarjetaProducto y la usa o guarda para continuar el proceso.
161	producto.getDescripcion(), producto.getPrecio(), producto.getImagen(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
162	producto.isDisponibleHorario()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	}	Cierra el bloque abierto anteriormente.
164	indiceLote = fin;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
165	panelProductos.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
166	panelProductos.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	scrollProductos.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	scrollProductos.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	if (indiceLote < productosPendientes.size()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
170	temporizadorLotes = new Timer(20, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
171	((Timer) e.getSource()).stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
172	cargarSiguienteLote(version);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
174	temporizadorLotes.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
175	temporizadorLotes.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
176	}	Cierra el bloque abierto anteriormente.
177	}	Cierra el bloque abierto anteriormente.
178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
179	private void mostrarEstado(String texto) {	Declara el método mostrarEstado. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
180	panelProductos.removeAll();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181	JLabel estado = new JLabel(texto, SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
182	estado.setForeground(new Color(95, 95, 95));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
183	panelProductos.setLayout(new java.awt.BorderLayout());	Crea una nueva instancia de java.awt.BorderLayout y la usa o guarda para continuar el proceso.
184	panelProductos.add(estado, java.awt.BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
185	panelProductos.revalidate();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
186	panelProductos.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	}	Cierra el bloque abierto anteriormente.
188		Línea en blanco: separa visualmente bloques para que el código sea más legible.
189	private Color colorDeSubcategoria(String subcategoria) {	Declara el método colorDeSubcategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
190	return switch (subcategoria) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
191	case "Sándwiches" -> new Color(255, 244, 230);	Define una de las rutas posibles dentro del switch.
192	case "McMuffin" -> new Color(235, 246, 255);	Define una de las rutas posibles dentro del switch.
193	case "Hot Cakes" -> new Color(255, 240, 246);	Define una de las rutas posibles dentro del switch.
194	case "Bebidas" -> new Color(235, 250, 243);	Define una de las rutas posibles dentro del switch.
195	case "Por tiempo limitado" -> new Color(248, 241, 255);	Define una de las rutas posibles dentro del switch.
196	default -> new Color(250, 250, 250);	Define una de las rutas posibles dentro del switch.
197	};	Cierra el bloque abierto anteriormente y termina la declaración.

198	}	Cierra el bloque abierto anteriormente.
199	}	Cierra el bloque abierto anteriormente.

16.104 src/Imagenes/Imagen.java

Archivo de apoyo del proyecto: src/Imagenes/Imagen.java.

Tipo: java | Líneas: 13 | Archivo 104 de 157

Clases/tipos detectados: Imagen

		1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		3	* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		5	package Imagenes;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
		6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
		7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		9	* @author Computacion	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
		11	public class Imagen {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
		12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
		13	}	Cierra el bloque abierto anteriormente.

16.105 src/Labels/LabelEscalable.java

Control de entrada o etiqueta personalizado LabelEscalable, diseñado para reutilizar estilos y comportamiento.

Tipo: java | Líneas: 151 | Archivo 105 de 157

Clases/tipos detectados: LabelEscalable

Línea	Constructor o método detectado
24	public void setIcon(javax.swing.Icon icon) {
37	public boolean isMantenerProporcion() {
41	public void setMantenerProporcion(boolean mantenerProporcion) {
50	protected void paintComponent(Graphics graphics) {
90	private BufferedImage convertir(Imagelcon icono) {
105	private BufferedImage reducirProgresivamente(
124	private BufferedImage escalarEtapa(Image origen, int ancho, int alto) {
145	private void invalidarCache() {

1	package Labels;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.AlphaComposite;	Importa java.awt.AlphaComposite para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Image;	Importa java.awt.Image para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
9	import java.beans.BeanProperty;	Importa java.beans.BeanProperty para poder usar esa clase sin escribir su nombre completo.
10	import javax.swing.ImageIcon;	Importa javax.swing.ImageIcon para poder usar esa clase sin escribir su nombre completo.
11	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** JLabel que escala imágenes con alta calidad y caché por tamaño. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	@SuppressWarnings({"serial", "this-escape"})	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
15	public class LabelEscalable extends JLabel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
16		Línea en blanco: separa visualmente bloques para que el código sea más legible.
17	private BufferedImage imagenOriginal;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
18	private BufferedImage imagenEscalada;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
19	private boolean mantenerProporcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
20	private int anchoCache = -1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
21	private int altoCache = -1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
24	public void setIcon(javax.swing.Icon icon) {	Declara el método setIcon. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
25	super.setIcon(icon);	Configura una propiedad mediante setIcon en el objeto indicado.

26	if (icon instanceof ImageIcon) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
27	imagenOriginal = convertir((ImageIcon) icon);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
28	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
29	imagenOriginal = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
30	}	Cierra el bloque abierto anteriormente.
31	invalidarCache();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	}	Cierra el bloque abierto anteriormente.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	@BeanProperty(preferred = true,	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
36	description = "Conserva la proporción original de la imagen.")	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
37	public boolean isMantenerProporcion() {	Declara el método isMantenerProporcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
38	return mantenerProporcion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
39	}	Cierra el bloque abierto anteriormente.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	public void setMantenerProporcion(boolean mantenerProporcion) {	Declara el método setMantenerProporcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	boolean anterior = this.mantenerProporcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43	this.mantenerProporcion = mantenerProporcion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
44	invalidarCache();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	firePropertyChange("mantenerProporcion", anterior, mantenerProporcion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
50	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
51	if (imagenOriginal == null getWidth() <= 0 getHeight() <= 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
52	super.paintComponent(graphics);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	int anchoObjetivo = getWidth();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	int altoObjetivo = getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58	if (mantenerProporcion) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
59	double escala = Math.min(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	getWidth() / (double) imagenOriginal.getWidth(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	getHeight() / (double) imagenOriginal.getHeight()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	anchoObjetivo = Math.max(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
64	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	(int) Math.round(imagenOriginal.getWidth() * escala)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	altoObjetivo = Math.max(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
68	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

69	(int) Math.round(imagenOriginal.getHeight() * escala)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	}	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	if (imagenEscalada == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
74	anchoCache != anchoObjetivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
75	altoCache != altoObjetivo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
76	imagenEscalada = reducirProgresivamente(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
77	imagenOriginal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
78	anchoObjetivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	altoObjetivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
80	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	anchoCache = anchoObjetivo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
82	altoCache = altoObjetivo;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
83	}	Cierra el bloque abierto anteriormente.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	int x = (getWidth() - imagenEscalada.getWidth()) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
86	int y = (getHeight() - imagenEscalada.getHeight()) / 2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
87	graphics.drawImage(imagenEscalada, x, y, this);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	}	Cierra el bloque abierto anteriormente.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	private BufferedImage convertir(ImageIcon icono) {	Declara el método convertir. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
91	int ancho = Math.max(1, icono.getWidth());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	int alto = Math.max(1, icono.getHeight());	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
93	BufferedImage resultado = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
94	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	alto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	BufferedImage.TYPE_INT_ARGB_PRE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	Graphics2D g2 = resultado.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
99	g2.setComposite(AlphaComposite.Src);	Configura una propiedad mediante setComposite en el objeto indicado.
100	g2.drawImage(icono.getImage(), 0, 0, ancho, alto, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	return resultado;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	}	Cierra el bloque abierto anteriormente.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	private BufferedImage reducirProgresivamente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
106	BufferedImage origen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	int anchoObjetivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
108	int altoObjetivo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
109		Línea en blanco: separa visualmente bloques para que el código sea más legible.
110	BufferedImage actual = origen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	while (actual.getWidth() > anchoObjetivo * 2	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
112	actual.getHeight() > altoObjetivo * 2) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

113	int anchoSiguiente = Math.max(anchoObjetivo, actual.getWidth() / 2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	int altoSiguiente = Math.max(altoObjetivo, actual.getHeight() / 2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	actual = escalarEtapa(actual, anchoSiguiente, altoSiguiente);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
116	}	Cierra el bloque abierto anteriormente.
117	if (actual.getWidth() != anchoObjetivo	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
118	actual.getHeight() != altoObjetivo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
119	actual = escalarEtapa(actual, anchoObjetivo, altoObjetivo);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
120	}	Cierra el bloque abierto anteriormente.
121	return actual;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
122	}	Cierra el bloque abierto anteriormente.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	private BufferedImage escalarEtapa(Image origen, int ancho, int alto) {	Declara el método escalarEtapa. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
125	BufferedImage destino = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
126	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127	alto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	BufferedImage.TYPE_INT_ARGB_PRE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	Graphics2D g2 = destino.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	g2.setComposite(AlphaComposite.Src);	Configura una propiedad mediante setComposite en el objeto indicado.
132	g2.setRenderingHint(RenderingHints.KEY_INTERPOLATION,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
133	RenderingHints.VALUE_INTERPOLATION_BICUBIC);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	g2.setRenderingHint(RenderingHints.KEY_ALPHA_INTERPOLATION,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
135	RenderingHints.VALUE_ALPHA_INTERPOLATION_QUALITY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	g2.setRenderingHint(RenderingHints.KEY_COLOR_RENDERING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
137	RenderingHints.VALUE_COLOR_RENDER_QUALITY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	g2.setRenderingHint(RenderingHints.KEY_RENDERING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
139	RenderingHints.VALUE_RENDER_QUALITY);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	g2.drawImage(origen, 0, 0, ancho, alto, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	return destino;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
143	}	Cierra el bloque abierto anteriormente.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	private void invalidarCache() {	Declara el método invalidarCache. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
146	imagenEscalada = null;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
147	anchoCache = -1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
148	altoCache = -1;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
149	}	Cierra el bloque abierto anteriormente.
150	}	Cierra el bloque abierto anteriormente.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.106 src/Labels/Prueba.form

Metadatos visuales de NetBeans para diseñar Prueba. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 76 | Archivo 106 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JFrameFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="defaultCloseOperation" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	</Properties>	Cierra la sección XML abierta anteriormente.
7	<SyntheticProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	<SyntheticProperty name="formSizePolicy" type="int" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	<SyntheticProperty name="generateCenter" type="boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</SyntheticProperties>	Cierra la sección XML abierta anteriormente.
11	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	</AuxValues>	Cierra la sección XML abierta anteriormente.
22		Línea en blanco para organizar el XML del formulario.
23	<Layout>	Indica el administrador de distribución que organiza los componentes hijos.
24	<DimensionLayout dim="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
25	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
26	<Component id="jPanel1" alignment="0" max="32767" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
27	</Group>	Cierra la sección XML abierta anteriormente.
28	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
29	<DimensionLayout dim="1">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
31	<Component id="jPanel1" alignment="0" max="32767" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
32	</Group>	Cierra la sección XML abierta anteriormente.
33	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
34	</Layout>	Cierra la sección XML abierta anteriormente.
35	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
36	<Container class="javax.swing.JPanel" name="jPanel1">	Declara un componente o contenedor visual y su clase Java.
37	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
38	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

39	<Color blue="ff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
40	</Property>	Cierra la sección XML abierta anteriormente.
41	</Properties>	Cierra la sección XML abierta anteriormente.
42		Línea en blanco para organizar el XML del formulario.
43	<Layout>	Indica el administrador de distribución que organiza los componentes hijos.
44	<DimensionLayout dim="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
45	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Group type="102" alignment="1" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
47	<EmptySpace pref="98" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
48	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
49	<Component id="textBox_Contrasena1" min="-2" max="-2" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
50	<Component id="textBox_Login1" min="-2" max="-2" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
51	</Group>	Cierra la sección XML abierta anteriormente.
52	<EmptySpace min="-2" pref="90" max="-2" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
53	</Group>	Cierra la sección XML abierta anteriormente.
54	</Group>	Cierra la sección XML abierta anteriormente.
55	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
56	<DimensionLayout dim="1">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
57	<Group type="103" groupAlignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
58	<Group type="102" alignment="0" attributes="0">	Metadato de NetBeans que conserva el estado editable del diseñador visual.
59	<EmptySpace min="-2" pref="35" max="-2" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
60	<Component id="textBox_Login1" min="-2" max="-2" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
61	<EmptySpace min="-2" pref="47" max="-2" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
62	<Component id="textBox_Contrasena1" min="-2" max="-2" attributes="0"/>	Declara un componente o contenedor visual y su clase Java.
63	<EmptySpace pref="252" max="32767" attributes="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
64	</Group>	Cierra la sección XML abierta anteriormente.
65	</Group>	Cierra la sección XML abierta anteriormente.
66	</DimensionLayout>	Cierra la sección XML abierta anteriormente.
67	</Layout>	Cierra la sección XML abierta anteriormente.
68	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
69	<Component class="Labels.TextBox_Login" name="textBox_Login1">	Declara un componente o contenedor visual y su clase Java.
70	</Component>	Cierra la sección XML abierta anteriormente.
71	<Component class="Labels.Textbox_Contrasena" name="textBox_Contrasena1">	Declara un componente o contenedor visual y su clase Java.
72	</Component>	Cierra la sección XML abierta anteriormente.
73	</SubComponents>	Cierra la sección XML abierta anteriormente.
74	</Container>	Cierra la sección XML abierta anteriormente.
75	</SubComponents>	Cierra la sección XML abierta anteriormente.
76	</Form>	Cierra la sección XML abierta anteriormente.

16.107 src/Labels/Prueba.java

Control de entrada o etiqueta personalizado Prueba, diseñado para reutilizar estilos y comportamiento.

Tipo: java | Líneas: 112 | Archivo 107 de 157

Clases/tipos detectados: Prueba

Línea	Constructor o método detectado
16	public Prueba() {
27	private void initComponents() {
75	public static void main(String args[]) {
101	public void run() {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package Labels;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	* @author HP	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class Prueba extends javax.swing.JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	* Creates new form Prueba	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	public Prueba() {	Constructor de Prueba: prepara un objeto nuevo y deja sus componentes listos para usarse.
17	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18	}	Cierra el bloque abierto anteriormente.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
21	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
22	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
26	// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
27	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	jPanel1 = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
30	textBox_Login1 = new Labels.TextBox_Login();	Crea una nueva instancia de Labels.TextBox_Login y la usa o guarda para continuar el proceso.

31	<code>textbox_Contrasena1 = new Labels.Textbox_Contrasena();</code>	Crea una nueva instancia de Labels.Textbox_Contrasena y la usa o guarda para continuar el proceso.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	<code>setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	<code>jPanel1.setBackground(new java.awt.Color(255, 255, 255));</code>	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
36		Línea en blanco: separa visualmente bloques para que el código sea más legible.
37	<code>javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);</code>	Crea una nueva instancia de javax.swing.GroupLayout y la usa o guarda para continuar el proceso.
38	<code>jPanel1.setLayout(jPanel1Layout);</code>	Configura una propiedad mediante setLayout en el objeto indicado.
39	<code>jPanel1Layout.setHorizontalGroup(</code>	Configura una propiedad mediante setHorizontalGroup en el objeto indicado.
40	<code> jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	<code> .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, jPanel1Layout.createSequentialGroup()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	<code> .addContainerGap(98, Short.MAX_VALUE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	<code> .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	<code> .addComponent(textbox_Contrasena1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
45	<code> .addComponent(textBox_Login1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	<code> .addGap(90, 90, 90))</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	<code>jPanel1Layout.setVerticalGroup(</code>	Configura una propiedad mediante setVerticalGroup en el objeto indicado.
49	<code> jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	<code> .addGroup(jPanel1Layout.createSequentialGroup()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	<code> .addGap(35, 35, 35)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	<code> .addComponent(textBox_Login1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	<code> .addGap(47, 47, 47)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	<code> .addComponent(textbox_Contrasena1, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	<code> .addContainerGap(252, Short.MAX_VALUE))</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
57		Línea en blanco: separa visualmente bloques para que el código sea más legible.
58	<code>javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());</code>	Crea una nueva instancia de javax.swing.GroupLayout y la usa o guarda para continuar el proceso.
59	<code>getContentPane().setLayout(layout);</code>	Configura una propiedad mediante setLayout en el objeto indicado.
60	<code>layout.setHorizontalGroup(</code>	Configura una propiedad mediante setHorizontalGroup en el objeto indicado.
61	<code> layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	<code> .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	<code>layout.setVerticalGroup(</code>	Configura una propiedad mediante setVerticalGroup en el objeto indicado.
65	<code> layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	<code> .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
68		Línea en blanco: separa visualmente bloques para que el código sea más legible.
69	<code>pack();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	<code>}// </editor-fold>//GEN-END: initComponents</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
73	* @param args the command line arguments	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
74	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
75	public static void main(String args[]) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
76	/* Set the Nimbus look and feel */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
77	//<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">	Comentario para humanos; Java no ejecuta este texto. <editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
78	/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
79	* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
80	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
81	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
82	for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
83	{	
84	if ("Nimbus".equals(info.getName())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
85	javax.swing.UIManager.setLookAndFeel(info.getClassName());	Configura una propiedad mediante setLookAndFeel en el objeto indicado.
86	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	}	Cierra el bloque abierto anteriormente.
88	} catch (ClassNotFoundException ex) {	Cierra el bloque abierto anteriormente.
89	java.util.logging.Logger.getLogger(Prueba.class.getName()).log(java.util.logging.Level.SEVERE, null,	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
90	ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	} catch (InstantiationException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
92	java.util.logging.Logger.getLogger(Prueba.class.getName()).log(java.util.logging.Level.SEVERE, null,	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	ex);	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
94	} catch (IllegalAccessException ex) {	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	java.util.logging.Logger.getLogger(Prueba.class.getName()).log(java.util.logging.Level.SEVERE, null,	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	ex);	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
97	} catch (javax.swing.UnsupportedLookAndFeelException ex) {	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	java.util.logging.Logger.getLogger(Prueba.class.getName()).log(java.util.logging.Level.SEVERE, null,	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
99	ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101	} //</editor-fold>	Comentario para humanos; Java no ejecuta este texto. </editor-fold>
102	/* Create and display the form */	Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	java.awt.EventQueue.invokeLater(new Runnable() {	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
104	public void run() {	Crea una nueva instancia de Runnable y la usa o guarda para continuar el proceso.
105	new Prueba().setVisible(true);	Declara el método run. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
106	}	Crea una nueva instancia de Prueba y la usa o guarda para continuar el proceso.
107	});	Cierra el bloque abierto anteriormente.
108	}	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109		Cierra el bloque abierto anteriormente.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
112	private javax.swing.JPanel jPanel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
113	private javax.swing.JLabel loginLabel;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
114	private javax.swing.JTextField loginField;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	private javax.swing.JPasswordField passwordField;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

111	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
112	}	Cierra el bloque abierto anteriormente.

16.108 src/Labels/TextBoxPassword.java

Control de entrada o etiqueta personalizado TextBoxPassword, diseñado para reutilizar estilos y comportamiento.

Tipo: java | Líneas: 225 | Archivo 108 de 157

Clases/tipos detectados: TextBoxPassword

Línea	Constructor o método detectado
21	public TextBoxPassword() {
36	protected void paintComponent(Graphics g) {
104	public void mousePressed(MouseEvent e) {
129	public void focusLost(FocusEvent e) {
171	protected void paintComponent(Graphics g) {
204	public String getPassword() {
213	public void setPlaceholder(String placeholder) {
218	public String getPlaceholder() {
222	public void setText(String string) {

1	package Labels;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import javax.swing.*;	Importa javax.swing.* para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.*;	Importa java.awt.* para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.event.FocusAdapter;	Importa java.awt.event.FocusAdapter para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.event.FocusEvent;	Importa java.awt.event.FocusEvent para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.event.MouseAdapter;	Importa java.awt.event.MouseAdapter para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.event.MouseEvent;	Importa java.awt.event.MouseEvent para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	public class TextBoxPassword extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	private JPasswordField txtInput;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
13	private JLabel lblIcono;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	private int radioEsquinas = 20;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
16	private String placeholder = "Ingresa tu contraseña";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	// Controla si se muestra el placeholder	Comentario para humanos; Java no ejecuta este texto. Controla si se muestra el placeholder
19	private boolean mostrarPlaceholder = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	public TextBoxPassword() {	Constructor de TextBoxPassword: prepara un objeto nuevo y deja sus componentes listos para usarse.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
24	setLayout(new BorderLayout());	Crea una nueva instancia de BorderLayout y la usa o guarda para continuar el proceso.

25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	// Espacio interno	Comentario para humanos; Java no ejecuta este texto. Espacio interno
27	setBorder(BorderFactory.createEmptyBorder(5, 15, 5, 12));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	// =====	Comentario para humanos; Java no ejecuta este texto. =====
30	// CAMPO DE CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. CAMPO DE CONTRASEÑA
31	// =====	Comentario para humanos; Java no ejecuta este texto. =====
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	txtInput = new JPasswordField() {	Crea una nueva instancia de JPasswordField y la usa o guarda para continuar el proceso.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
36	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	// Dibujar normalmente el campo	Comentario para humanos; Java no ejecuta este texto. Dibujar normalmente el campo
39	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	// Mostrar placeholder si está vacío	Comentario para humanos; Java no ejecuta este texto. Mostrar placeholder si está vacío
42	if (mostrarPlaceholder && getPassword().length == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
47	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	g2.setColor(new Color(140, 140, 145));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
52	g2.setFont(getFont());	Configura una propiedad mediante setFont en el objeto indicado.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	FontMetrics fm = g2.getFontMetrics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	int y = (getHeight() - fm.getHeight()) / 2	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	+ fm.getAscent();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	g2.drawString(placeholder, 0, y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	}	Cierra el bloque abierto anteriormente.
63	}	Cierra el bloque abierto anteriormente.
64	};	Cierra el bloque abierto anteriormente y termina la declaración.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	// =====	Comentario para humanos; Java no ejecuta este texto. =====
67	// ESTILO	Comentario para humanos; Java no ejecuta este texto. ESTILO
68	// =====	Comentario para humanos; Java no ejecuta este texto. =====

69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	<code>txtInput.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	<code>txtInput.setBackground(</code>	Configura una propiedad mediante <code>setBackground</code> en el objeto indicado.
73	<code>new Color(0, 0, 0, 0)</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
74	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	<code>txtInput.setBorder(null);</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	<code>txtInput.setFont(</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
79	<code>new Font("Segoe UI", Font.PLAIN, 15)</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
80	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81		Línea en blanco: separa visualmente bloques para que el código sea más legible.
82	<code>txtInput.setForeground(</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
83	<code>new Color(50, 50, 50)</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
84	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	<code>txtInput.setCaretColor(</code>	Configura una propiedad mediante <code>setCaretColor</code> en el objeto indicado.
87	<code>new Color(50, 50, 50)</code>	Crea una nueva instancia de <code>Color</code> y la usa o guarda para continuar el proceso.
88	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89		Línea en blanco: separa visualmente bloques para que el código sea más legible.
90	<code>// Caracter que aparecerá al escribir contraseña</code>	Comentario para humanos; Java no ejecuta este texto. Caracter que aparecerá al escribir contraseña
91	<code>txtInput.setEchoChar('*');</code>	Configura una propiedad mediante <code>setEchoChar</code> en el objeto indicado.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	<code>// Evita que aparezca seleccionado al abrir la ventana</code>	Comentario para humanos; Java no ejecuta este texto. Evita que aparezca seleccionado al abrir la ventana
94	<code>txtInput.setFocusable(false);</code>	Configura una propiedad mediante <code>setFocusable</code> en el objeto indicado.
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
98	<code>// AL HACER CLIC</code>	Comentario para humanos; Java no ejecuta este texto. AL HACER CLIC
99	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	<code>txtInput.addMouseListener(new MouseAdapter() {</code>	Crea una nueva instancia de <code>MouseListener</code> y la usa o guarda para continuar el proceso.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
104	<code>public void mousePressed(MouseEvent e) {</code>	Declara el método <code>mousePressed</code> . Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	<code>// Ahora puede recibir el foco</code>	Comentario para humanos; Java no ejecuta este texto. Ahora puede recibir el foco
107	<code>txtInput.setFocusable(true);</code>	Configura una propiedad mediante <code>setFocusable</code> en el objeto indicado.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	<code>// Ocultar placeholder</code>	Comentario para humanos; Java no ejecuta este texto. Ocultar placeholder
110	<code>mostrarPlaceholder = false;</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
111		Línea en blanco: separa visualmente bloques para que el código sea más legible.
112	<code>// Colocar cursor dentro</code>	Comentario para humanos; Java no ejecuta este texto. Colocar cursor dentro

113	SwingUtilities.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	txtInput.requestFocusInWindow();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
115	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	txtInput.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	}	Cierra el bloque abierto anteriormente.
119	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	// =====	Comentario para humanos; Java no ejecuta este texto. =====
123	// CUANDO SALE DEL CAMPO	Comentario para humanos; Java no ejecuta este texto. CUANDO SALE DEL CAMPO
124	// =====	Comentario para humanos; Java no ejecuta este texto. =====
125		Línea en blanco: separa visualmente bloques para que el código sea más legible.
126	txtInput.addFocusListener(new FocusAdapter() {	Crea una nueva instancia de FocusAdapter y la usa o guarda para continuar el proceso.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
129	public void focusLost(FocusEvent e) {	Declara el método focusLost. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	// Si no escribió contraseña	Comentario para humanos; Java no ejecuta este texto. Si no escribió contraseña
132	if (txtInput.getPassword().length == 0) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	mostrarPlaceholder = true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
135		Línea en blanco: separa visualmente bloques para que el código sea más legible.
136	// Evita que tome foco automáticamente	Comentario para humanos; Java no ejecuta este texto. Evita que tome foco automáticamente
137	txtInput.setFocusable(false);	Configura una propiedad mediante setFocusable en el objeto indicado.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	txtInput.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	}	Cierra el bloque abierto anteriormente.
141	}	Cierra el bloque abierto anteriormente.
142	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	// =====	Comentario para humanos; Java no ejecuta este texto. =====
146	// ICONO	Comentario para humanos; Java no ejecuta este texto. ICONO
147	// =====	Comentario para humanos; Java no ejecuta este texto. =====
148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
149	lblIcono = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	lblIcono.setHorizontalAlignment(	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
152	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
153	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	lblIcono.setCursor(	Configura una propiedad mediante setCursor en el objeto indicado.
156	new Cursor(Cursor.HAND_CURSOR)	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.

157	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159		Línea en blanco: separa visualmente bloques para que el código sea más legible.
160	// Agregar componentes	Comentario para humanos; Java no ejecuta este texto. Agregar componentes
161	add(txtInput, BorderLayout.CENTER);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	add(lblIcono, BorderLayout.EAST);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163	}	Cierra el bloque abierto anteriormente.
164		Línea en blanco: separa visualmente bloques para que el código sea más legible.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	// =====	Comentario para humanos; Java no ejecuta este texto. =====
167	// FONDO REDONDEADO	Comentario para humanos; Java no ejecuta este texto. FONDO REDONDEADO
168	// =====	Comentario para humanos; Java no ejecuta este texto. =====
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
171	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
172		Línea en blanco: separa visualmente bloques para que el código sea más legible.
173	Graphics2D g2 = (Graphics2D) g.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
175	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
176	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
177	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
178	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179		Línea en blanco: separa visualmente bloques para que el código sea más legible.
180	// Fondo gris #EAEAEA	Comentario para humanos; Java no ejecuta este texto. Fondo gris #EAEAEA
181	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
182	new Color(234, 234, 234)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
183	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	getWidth(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	getHeight(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	radioEsquinas,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191	radioEsquinas	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
192	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
193		Línea en blanco: separa visualmente bloques para que el código sea más legible.
194	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	}	Cierra el bloque abierto anteriormente.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	// =====	Comentario para humanos; Java no ejecuta este texto. =====

201	// OBTENER CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. OBTENER CONTRASEÑA
202	// =====	Comentario para humanos; Java no ejecuta este texto. =====
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	public String getPassword() {	Declara el método getPassword. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
205	return new String(txtInput.getPassword());	Finaliza el método y devuelve el valor indicado a quien lo llamó.
206	}	Cierra el bloque abierto anteriormente.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208		Línea en blanco: separa visualmente bloques para que el código sea más legible.
209	// =====	Comentario para humanos; Java no ejecuta este texto. =====
210	// PLACEHOLDER	Comentario para humanos; Java no ejecuta este texto. PLACEHOLDER
211	// =====	Comentario para humanos; Java no ejecuta este texto. =====
212		Línea en blanco: separa visualmente bloques para que el código sea más legible.
213	public void setPlaceholder(String placeholder) {	Declara el método setPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
214	this.placeholder = placeholder;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
215	txtInput.repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
216	}	Cierra el bloque abierto anteriormente.
217		Línea en blanco: separa visualmente bloques para que el código sea más legible.
218	public String getPlaceholder() {	Declara el método getPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
219	return placeholder;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
220	}	Cierra el bloque abierto anteriormente.
221		Línea en blanco: separa visualmente bloques para que el código sea más legible.
222	public void setText(String string) {	Declara el método setText. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
223	throw new UnsupportedOperationException("Not supported yet."); // Generated from nbfs://nbhost/SystemFiles/System/Templates/Classes/Code/GeneratedMethodBody	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
224	}	Cierra el bloque abierto anteriormente.
225	}	Cierra el bloque abierto anteriormente.

16.109 src/Labels/Textbox_Contrasena.java

Control de entrada o etiqueta personalizado Textbox_Contrasena, diseñado para reutilizar estilos y comportamiento.

Tipo: java | Líneas: 604 | Archivo 109 de 157

Clases/tipos detectados: Textbox_Contrasena

Línea	Constructor o método detectado
62	public Textbox_Contrasena() {
148	public void focusGained(
171	public void focusLost(
203	public void doLayout() {
246	protected void paintComponent(
457	public String getTexto() {
472	public void setTexto(
515	public void setPlaceholder(String placeholder) {
532	public String getPlaceholder() {
540	public void setIcono(
551	public Icon getIcono() {
559	public JPasswordField getPasswordField() {
567	public void mostrarPassword(
589	public void setText(String string) {
593	public String getPassword() {
597	public void setEchoChar(char character) {
601	public char getEchoChar() {

1	package Labels;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.event.FocusAdapter;	Importa java.awt.event.FocusAdapter para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.event.FocusEvent;	Importa java.awt.event.FocusEvent para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.

15	import javax.swing.Icon;	Importa javax.swing.Icon para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.JPasswordField;	Importa javax.swing.JPasswordField para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	public class Textbox_Contrasena extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	// =====	Comentario para humanos; Java no ejecuta este texto. =====
24	// COMPONENTES	Comentario para humanos; Java no ejecuta este texto. COMPONENTES
25	// =====	Comentario para humanos; Java no ejecuta este texto. =====
26	private JPasswordField txtPassword;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27	private JLabel lblIcono;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	// =====	Comentario para humanos; Java no ejecuta este texto. =====
30	// PLACEHOLDER	Comentario para humanos; Java no ejecuta este texto. PLACEHOLDER
31	// =====	Comentario para humanos; Java no ejecuta este texto. =====
32	private String placeholder = "Ingresa contraseña";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	private boolean mostrandoPlaceholder = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	// =====	Comentario para humanos; Java no ejecuta este texto. =====
37	// COLORES	Comentario para humanos; Java no ejecuta este texto. COLORES
38	// =====	Comentario para humanos; Java no ejecuta este texto. =====
39	private final Color rojo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	= new Color(215, 25, 25);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	private final Color rojoClaro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	= new Color(255, 205, 205);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	private final Color fondoCampo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	= new Color(248, 248, 248);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	private final Color colorTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	= new Color(35, 35, 35);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	private final Color colorPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	= new Color(145, 145, 145);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	// =====	Comentario para humanos; Java no ejecuta este texto. =====
55	// CARÁCTER DE CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. CARÁCTER DE CONTRASEÑA
56	// =====	Comentario para humanos; Java no ejecuta este texto. =====
57	private char caracterPassword;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.

59	// =====	Comentario para humanos; Java no ejecuta este texto. =====
60	// CONSTRUCTOR	Comentario para humanos; Java no ejecuta este texto. CONSTRUCTOR
61	// =====	Comentario para humanos; Java no ejecuta este texto. =====
62	public Textbox_Contrasena() {	Constructor de Textbox_Contrasena: prepara un objeto nuevo y deja sus componentes listos para usarse.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	setOpaque(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	setLayout(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	setPreferredSize(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	new Dimension(600, 90)	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
70	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	setMinimumSize(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	new Dimension(300, 70)	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
74	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	// =====	Comentario para humanos; Java no ejecuta este texto. =====
77	// ICONO	Comentario para humanos; Java no ejecuta este texto. ICONO
78	// =====	Comentario para humanos; Java no ejecuta este texto. =====
79	lblIcono = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	lblIcono.setHorizontalAlignment(	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
82	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	lblIcono.setVerticalAlignment(	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
86	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
87	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88		Línea en blanco: separa visualmente bloques para que el código sea más legible.
89	lblIcono.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	add(lblIcono);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92		Línea en blanco: separa visualmente bloques para que el código sea más legible.
93	// =====	Comentario para humanos; Java no ejecuta este texto. =====
94	// CAMPO DE CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. CAMPO DE CONTRASEÑA
95	// =====	Comentario para humanos; Java no ejecuta este texto. =====
96	txtPassword	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	= new JPasswordField();	Crea una nueva instancia de JPasswordField y la usa o guarda para continuar el proceso.
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.
99	characterPassword	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
100	= txtPassword.getEchoChar();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	// Placeholder visible	Comentario para humanos; Java no ejecuta este texto. Placeholder visible

103	<code>txtPassword.setEchoChar(</code>	Configura una propiedad mediante <code>setEchoChar</code> en el objeto indicado.
104	<code>(char) 0</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
105	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106		Línea en blanco: separa visualmente bloques para que el código sea más legible.
107	<code>txtPassword.setText(</code>	Cambia el texto visible de un componente de la interfaz.
108	<code>placeholder</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
109	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	<code>txtPassword.setForeground(</code>	Configura una propiedad mediante <code>setForeground</code> en el objeto indicado.
112	<code>colorPlaceholder</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	<code>txtPassword.setFont(</code>	Configura una propiedad mediante <code>setFont</code> en el objeto indicado.
116	<code>new Font(</code>	Crea una nueva instancia de <code>Font</code> y la usa o guarda para continuar el proceso.
117	<code>"Dialog",</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
118	<code>Font.PLAIN,</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
119	<code>16</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
120	<code>)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
121	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	<code>txtPassword.setBorder(</code>	Configura una propiedad mediante <code>setBorder</code> en el objeto indicado.
124	<code>BorderFactory.createEmptyBorder()</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	<code>txtPassword.setOpaque(false);</code>	Configura una propiedad mediante <code>setOpaque</code> en el objeto indicado.
128		Línea en blanco: separa visualmente bloques para que el código sea más legible.
129	<code>txtPassword.setCaretColor(</code>	Configura una propiedad mediante <code>setCaretColor</code> en el objeto indicado.
130	<code>colorText</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	<code>txtPassword.setCursor(</code>	Configura una propiedad mediante <code>setCursor</code> en el objeto indicado.
134	<code>new Cursor(</code>	Crea una nueva instancia de <code>Cursor</code> y la usa o guarda para continuar el proceso.
135	<code>Cursor.TEXT_CURSOR</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
136	<code>)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	<code>add(txtPassword);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
142	<code>// PLACEHOLDER</code>	Comentario para humanos; Java no ejecuta este texto. PLACEHOLDER
143	<code>// =====</code>	Comentario para humanos; Java no ejecuta este texto. =====
144	<code>txtPassword.addFocusListener(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	<code>new FocusAdapter() {</code>	Crea una nueva instancia de <code>FocusAdapter</code> y la usa o guarda para continuar el proceso.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.

147	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
148	public void focusGained(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	FocusEvent e) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	txtPassword.setText("");	Cambia el texto visible de un componente de la interfaz.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
156	characterPassword	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	txtPassword.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
160	colorTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
161	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	mostrandoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
164	= false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
165	}	Cierra el bloque abierto anteriormente.
166		Línea en blanco: separa visualmente bloques para que el código sea más legible.
167	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	}	Cierra el bloque abierto anteriormente.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
171	public void focusLost(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	FocusEvent e) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	if (txtPassword	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
175	.getPassword().length == 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
178	(char) 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
179	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	txtPassword.setText(	Cambia el texto visible de un componente de la interfaz.
182	placeholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
183	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	txtPassword.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
186	colorPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
187	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188		Línea en blanco: separa visualmente bloques para que el código sea más legible.
189	mostrandoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	= true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

191	}	Cierra el bloque abierto anteriormente.
192		Línea en blanco: separa visualmente bloques para que el código sea más legible.
193	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
194	}	Cierra el bloque abierto anteriormente.
195	}	Cierra el bloque abierto anteriormente.
196	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
197	}	Cierra el bloque abierto anteriormente.
198		Línea en blanco: separa visualmente bloques para que el código sea más legible.
199	// =====	Comentario para humanos; Java no ejecuta este texto. =====
200	// POSICIONAR COMPONENTES	Comentario para humanos; Java no ejecuta este texto. POSICIONAR COMPONENTES
201	// =====	Comentario para humanos; Java no ejecuta este texto. =====
202	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
203	public void doLayout() {	Declara el método doLayout. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
204		Línea en blanco: separa visualmente bloques para que el código sea más legible.
205	super.doLayout();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	int alto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
208	= getHeight();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	int tamanoCuadro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
211	= Math.min(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
212	alto - 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	90	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
215		Línea en blanco: separa visualmente bloques para que el código sea más legible.
216	int y	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	= (alto - tamanoCuadro)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
218	/ 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	// Icono	Comentario para humanos; Java no ejecuta este texto. Icono
221	lblIcono.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
222	4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	y,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	tamanoCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	tamanoCuadro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
227		Línea en blanco: separa visualmente bloques para que el código sea más legible.
228	// Campo password	Comentario para humanos; Java no ejecuta este texto. Campo password
229	int inicioTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
230	= tamanoCuadro + 25;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	txtPassword.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
233	inicioTexto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
234	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

235	getWidth()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
236	- inicioTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
237	- 30,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
238	alto - 20	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
239	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240	}	Cierra el bloque abierto anteriormente.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.
242	// =====	Comentario para humanos; Java no ejecuta este texto. =====
243	// DIBUJAR COMPONENTE	Comentario para humanos; Java no ejecuta este texto. DIBUJAR COMPONENTE
244	// =====	Comentario para humanos; Java no ejecuta este texto. =====
245	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
246	protected void paintComponent(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
247	Graphics g) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
248		Línea en blanco: separa visualmente bloques para que el código sea más legible.
249	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
250		Línea en blanco: separa visualmente bloques para que el código sea más legible.
251	Graphics2D g2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
252	= (Graphics2D) g.create();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
253		Línea en blanco: separa visualmente bloques para que el código sea más legible.
254	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
255	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
256	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.
259	int ancho	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	= getWidth();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	int alto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
263	= getHeight();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	int tamanoCuadro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
266	= Math.min(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
267	alto - 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
268	90	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
269	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
270		Línea en blanco: separa visualmente bloques para que el código sea más legible.
271	// =====	Comentario para humanos; Java no ejecuta este texto. =====
272	// CAJA PRINCIPAL	Comentario para humanos; Java no ejecuta este texto. CAJA PRINCIPAL
273	// =====	Comentario para humanos; Java no ejecuta este texto. =====
274	int xCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
275	= tamanoCuadro / 2;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	int yCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
278	= 10;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

279		Línea en blanco: separa visualmente bloques para que el código sea más legible.
280	int anchoCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
281	= ancho	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
282	- xCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
283	- 3;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
284		Línea en blanco: separa visualmente bloques para que el código sea más legible.
285	int altoCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
286	= alto - 20;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	// Fondo gris claro	Comentario para humanos; Java no ejecuta este texto. Fondo gris claro
289	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
290	fondoCampo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
291	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
294	xCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
295	yCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
296	anchoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
297	altoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
298	35,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
299	35	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
300	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
301		Línea en blanco: separa visualmente bloques para que el código sea más legible.
302	// Borde rojo	Comentario para humanos; Java no ejecuta este texto. Borde rojo
303	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
304	rojo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
305	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
308	new BasicStroke(2.5f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
309	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
311	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	xCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
313	yCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
314	anchoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
315	altoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
316	35,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
317	35	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
319		Línea en blanco: separa visualmente bloques para que el código sea más legible.
320	// =====	Comentario para humanos; Java no ejecuta este texto. =====
321	// CUADRO IZQUIERDO	Comentario para humanos; Java no ejecuta este texto. CUADRO IZQUIERDO
322	// =====	Comentario para humanos; Java no ejecuta este texto. =====

323	int xCuadro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
324	= 3;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
325		Línea en blanco: separa visualmente bloques para que el código sea más legible.
326	int yCuadro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
327	= (alto - tamanoCuadro)	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
328	/ 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
329		Línea en blanco: separa visualmente bloques para que el código sea más legible.
330	// Fondo rojo claro	Comentario para humanos; Java no ejecuta este texto. Fondo rojo claro
331	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
332	rojoClaro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
333	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
334		Línea en blanco: separa visualmente bloques para que el código sea más legible.
335	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	xCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
337	yCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
338	tamanoCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339	tamanoCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
340	28,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341	28	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
342	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
343		Línea en blanco: separa visualmente bloques para que el código sea más legible.
344	// Borde rojo	Comentario para humanos; Java no ejecuta este texto. Borde rojo
345	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
346	rojo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
347	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
350	new BasicStroke(3f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
351	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
352		Línea en blanco: separa visualmente bloques para que el código sea más legible.
353	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
354	xCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
355	yCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
356	tamanoCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	tamanoCuadro,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
358	28,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359	28	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
360	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
361		Línea en blanco: separa visualmente bloques para que el código sea más legible.
362	// =====	Comentario para humanos; Java no ejecuta este texto. =====
363	// LÍNEAS DECORATIVAS	Comentario para humanos; Java no ejecuta este texto. LÍNEAS DECORATIVAS
364	// =====	Comentario para humanos; Java no ejecuta este texto. =====
365	int inicioLinea	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
366	= tamanoCuadro + 15;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

367		Línea en blanco: separa visualmente bloques para que el código sea más legible.
368	int centro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
369	= (int) (ancho * 0.70);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
370		Línea en blanco: separa visualmente bloques para que el código sea más legible.
371	int finallinea	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
372	= ancho - 45;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
373		Línea en blanco: separa visualmente bloques para que el código sea más legible.
374	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
375	new BasicStroke(2f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
376	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
377		Línea en blanco: separa visualmente bloques para que el código sea más legible.
378	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
379	rojo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
380	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
381		Línea en blanco: separa visualmente bloques para que el código sea más legible.
382	// Línea superior izquierda	Comentario para humanos; Java no ejecuta este texto. Línea superior izquierda
383	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
384	inicioLinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
385	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
386	centro - 50,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
387	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
388	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
389		Línea en blanco: separa visualmente bloques para que el código sea más legible.
390	// Línea superior derecha	Comentario para humanos; Java no ejecuta este texto. Línea superior derecha
391	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	centro + 55,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
393	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
394	finallinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
395	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
396	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
397		Línea en blanco: separa visualmente bloques para que el código sea más legible.
398	// Línea inferior izquierda	Comentario para humanos; Java no ejecuta este texto. Línea inferior izquierda
399	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
400	inicioLinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
401	alto - 10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
402	centro - 50,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
403	alto - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
404	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
405		Línea en blanco: separa visualmente bloques para que el código sea más legible.
406	// Línea inferior derecha	Comentario para humanos; Java no ejecuta este texto. Línea inferior derecha
407	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
408	centro + 55,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
409	alto - 10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
410	finallinea,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

411	alto - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
412	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
413		Línea en blanco: separa visualmente bloques para que el código sea más legible.
414	// =====	Comentario para humanos; Java no ejecuta este texto. =====
415	// PUNTOS DECORATIVOS	Comentario para humanos; Java no ejecuta este texto. PUNTOS DECORATIVOS
416	// =====	Comentario para humanos; Java no ejecuta este texto. =====
417	int cantidad	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	= 6;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
419		Línea en blanco: separa visualmente bloques para que el código sea más legible.
420	int separacion	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
421	= 11;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
422		Línea en blanco: separa visualmente bloques para que el código sea más legible.
423	int inicioPuntos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
424	= centro - 28;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
425		Línea en blanco: separa visualmente bloques para que el código sea más legible.
426	for (int i = 0;	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
427	i < cantidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
428	i++) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
429		Línea en blanco: separa visualmente bloques para que el código sea más legible.
430	int x	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
431	= inicioPuntos	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
432	+ (i * separacion);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
433		Línea en blanco: separa visualmente bloques para que el código sea más legible.
434	// Puntos superiores	Comentario para humanos; Java no ejecuta este texto. Puntos superiores
435	g2.fillOval(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
436	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
437	7,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
438	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
439	5	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
440	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
441		Línea en blanco: separa visualmente bloques para que el código sea más legible.
442	// Puntos inferiores	Comentario para humanos; Java no ejecuta este texto. Puntos inferiores
443	g2.fillOval(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	alto - 12,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	5	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
448	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
449	}	Cierra el bloque abierto anteriormente.
450		Línea en blanco: separa visualmente bloques para que el código sea más legible.
451	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
452	}	Cierra el bloque abierto anteriormente.
453		Línea en blanco: separa visualmente bloques para que el código sea más legible.
454	// =====	Comentario para humanos; Java no ejecuta este texto. =====

455	// OBTENER CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. OBTENER CONTRASEÑA
456	// =====	Comentario para humanos; Java no ejecuta este texto. =====
457	public String getTexto() {	Declara el método getTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
458		Línea en blanco: separa visualmente bloques para que el código sea más legible.
459	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
460		Línea en blanco: separa visualmente bloques para que el código sea más legible.
461	return "";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
462	}	Cierra el bloque abierto anteriormente.
463		Línea en blanco: separa visualmente bloques para que el código sea más legible.
464	return new String(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
465	txtPassword.getPassword()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
466	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
467	}	Cierra el bloque abierto anteriormente.
468		Línea en blanco: separa visualmente bloques para que el código sea más legible.
469	// =====	Comentario para humanos; Java no ejecuta este texto. =====
470	// CAMBIAR CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. CAMBIAR CONTRASEÑA
471	// =====	Comentario para humanos; Java no ejecuta este texto. =====
472	public void setTexto(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
473	String texto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
474		Línea en blanco: separa visualmente bloques para que el código sea más legible.
475	if (texto == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
476	texto.isEmpty()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
477		Línea en blanco: separa visualmente bloques para que el código sea más legible.
478	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
479	(char) 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
480	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
481		Línea en blanco: separa visualmente bloques para que el código sea más legible.
482	txtPassword.setText(	Cambia el texto visible de un componente de la interfaz.
483	placeholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
484	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
485		Línea en blanco: separa visualmente bloques para que el código sea más legible.
486	txtPassword.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
487	colorPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
488	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
489		Línea en blanco: separa visualmente bloques para que el código sea más legible.
490	mostrandoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
491	= true;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
492		Línea en blanco: separa visualmente bloques para que el código sea más legible.
493	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
494		Línea en blanco: separa visualmente bloques para que el código sea más legible.
495	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
496	characterPassword	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
497	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
498		Línea en blanco: separa visualmente bloques para que el código sea más legible.

499	txtPassword.setText(	Cambia el texto visible de un componente de la interfaz.
500	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
501	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
502		Línea en blanco: separa visualmente bloques para que el código sea más legible.
503	txtPassword.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
504	colorTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
505	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
506		Línea en blanco: separa visualmente bloques para que el código sea más legible.
507	mostrandoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
508	= false;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
509	}	Cierra el bloque abierto anteriormente.
510	}	Cierra el bloque abierto anteriormente.
511		Línea en blanco: separa visualmente bloques para que el código sea más legible.
512	// =====	Comentario para humanos; Java no ejecuta este texto. =====
513	// CAMBIAR PLACEHOLDER	Comentario para humanos; Java no ejecuta este texto. CAMBIAR PLACEHOLDER
514	// =====	Comentario para humanos; Java no ejecuta este texto. =====
515	public void setPlaceholder(String placeholder) {	Declara el método setPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
516		Línea en blanco: separa visualmente bloques para que el código sea más legible.
517	this.placeholder = placeholder;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
518		Línea en blanco: separa visualmente bloques para que el código sea más legible.
519	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
520		Línea en blanco: separa visualmente bloques para que el código sea más legible.
521	txtPassword.setEchoChar((char) 0);	Configura una propiedad mediante setEchoChar en el objeto indicado.
522		Línea en blanco: separa visualmente bloques para que el código sea más legible.
523	txtPassword.setText(placeholder);	Cambia el texto visible de un componente de la interfaz.
524		Línea en blanco: separa visualmente bloques para que el código sea más legible.
525	txtPassword.setForeground(colorPlaceholder);	Configura una propiedad mediante setForeground en el objeto indicado.
526	}	Cierra el bloque abierto anteriormente.
527	}	Cierra el bloque abierto anteriormente.
528		Línea en blanco: separa visualmente bloques para que el código sea más legible.
529	// =====	Comentario para humanos; Java no ejecuta este texto. =====
530	// OBTENER PLACEHOLDER	Comentario para humanos; Java no ejecuta este texto. OBTENER PLACEHOLDER
531	// =====	Comentario para humanos; Java no ejecuta este texto. =====
532	public String getPlaceholder() {	Declara el método getPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
533		Línea en blanco: separa visualmente bloques para que el código sea más legible.
534	return placeholder;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
535	}	Cierra el bloque abierto anteriormente.
536		Línea en blanco: separa visualmente bloques para que el código sea más legible.
537	// =====	Comentario para humanos; Java no ejecuta este texto. =====
538	// COLOCAR ICONO	Comentario para humanos; Java no ejecuta este texto. COLOCAR ICONO
539	// =====	Comentario para humanos; Java no ejecuta este texto. =====
540	public void setIcono(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
541	Icon icono) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
542		Línea en blanco: separa visualmente bloques para que el código sea más legible.

543	lblIcono.setIcon(	Configura una propiedad mediante setIcon en el objeto indicado.
544	icono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
545	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
546	}	Cierra el bloque abierto anteriormente.
547		Línea en blanco: separa visualmente bloques para que el código sea más legible.
548	// =====	Comentario para humanos; Java no ejecuta este texto. =====
549	// OBTENER ICONO	Comentario para humanos; Java no ejecuta este texto. OBTENER ICONO
550	// =====	Comentario para humanos; Java no ejecuta este texto. =====
551	public Icon getIcon() {	Declara el método getIcon. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
552		Línea en blanco: separa visualmente bloques para que el código sea más legible.
553	return lblIcono.getIcon();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
554	}	Cierra el bloque abierto anteriormente.
555		Línea en blanco: separa visualmente bloques para que el código sea más legible.
556	// =====	Comentario para humanos; Java no ejecuta este texto. =====
557	// OBTENER JPASSWORDFIELD	Comentario para humanos; Java no ejecuta este texto. OBTENER JPASSWORDFIELD
558	// =====	Comentario para humanos; Java no ejecuta este texto. =====
559	public JPasswordField getPasswordField() {	Declara el método getPasswordField. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
560		Línea en blanco: separa visualmente bloques para que el código sea más legible.
561	return txtPassword;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
562	}	Cierra el bloque abierto anteriormente.
563		Línea en blanco: separa visualmente bloques para que el código sea más legible.
564	// =====	Comentario para humanos; Java no ejecuta este texto. =====
565	// MOSTRAR / OCULTAR CONTRASEÑA	Comentario para humanos; Java no ejecuta este texto. MOSTRAR / OCULTAR CONTRASEÑA
566	// =====	Comentario para humanos; Java no ejecuta este texto. =====
567	public void mostrarPassword(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
568	boolean mostrar) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
569		Línea en blanco: separa visualmente bloques para que el código sea más legible.
570	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
571		Línea en blanco: separa visualmente bloques para que el código sea más legible.
572	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
573	}	Cierra el bloque abierto anteriormente.
574		Línea en blanco: separa visualmente bloques para que el código sea más legible.
575	if (mostrar) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
576		Línea en blanco: separa visualmente bloques para que el código sea más legible.
577	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
578	(char) 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
579	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
580		Línea en blanco: separa visualmente bloques para que el código sea más legible.
581	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
582		Línea en blanco: separa visualmente bloques para que el código sea más legible.
583	txtPassword.setEchoChar(	Configura una propiedad mediante setEchoChar en el objeto indicado.
584	characterPassword	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
585	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
586	}	Cierra el bloque abierto anteriormente.

587	}	Cierra el bloque abierto anteriormente.
588		Línea en blanco: separa visualmente bloques para que el código sea más legible.
589	public void setText(String string) {	Declara el método setText. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
590	setTexto(string);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
591	}	Cierra el bloque abierto anteriormente.
592		Línea en blanco: separa visualmente bloques para que el código sea más legible.
593	public String getPassword() {	Declara el método getPassword. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
594	return getTexto();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
595	}	Cierra el bloque abierto anteriormente.
596		Línea en blanco: separa visualmente bloques para que el código sea más legible.
597	public void setEchoChar(char caracter) {	Declara el método setEchoChar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
598	txtPassword.setEchoChar(caracter);	Configura una propiedad mediante setEchoChar en el objeto indicado.
599	}	Cierra el bloque abierto anteriormente.
600		Línea en blanco: separa visualmente bloques para que el código sea más legible.
601	public char getEchoChar() {	Declara el método getEchoChar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
602	return txtPassword.getEchoChar();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
603	}	Cierra el bloque abierto anteriormente.
604	}	Cierra el bloque abierto anteriormente.

16.110 src/Labels/TextBox_Login.java

Control de entrada o etiqueta personalizado TextBox_Login, diseñado para reutilizar estilos y comportamiento.

Tipo: java | Líneas: 710 | Archivo 110 de 157

Clases/tipos detectados: TextBox_Login, Tema

Línea	Constructor o método detectado
81	public TextBox_Login() {
163	public void focusGained(
182	public void focusLost(
215	public void doLayout() {
263	protected void paintComponent(Graphics g) {
536	public void setTema(Tema tema) {
544	public Tema getTema() {
554	public String getText() {
567	public void setText(String textoNuevo) {
604	public String getTexto() {
610	public void setTexto(String textoNuevo) {
619	public void enfocar() {
628	public void setPlaceholder(
647	public String getPlaceholder() {
656	public void setIcono(Icon icono) {
664	public Icon getIcono() {
673	public JTextField getTextField() {
682	public void setAmarilloPrincipal(
692	public void setRojoPrincipal(
702	public void setFondoCampo(

1	package Labels;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Cursor;	Importa java.awt.Cursor para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Dimension;	Importa java.awt.Dimension para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.

11	import java.awt.event.FocusAdapter;	Importa java.awt.event.FocusAdapter para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.event.FocusEvent;	Importa java.awt.event.FocusEvent para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.geom.RoundRectangle2D;	Importa java.awt.geom.RoundRectangle2D para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.BorderFactory;	Importa javax.swing.BorderFactory para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.Icon;	Importa javax.swing.Icon para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.JTextField;	Importa javax.swing.JTextField para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	public class TextBox_Login extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	// TEMAS DISPONIBLES	Comentario para humanos; Java no ejecuta este texto. TEMAS DISPONIBLES
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	public enum Tema {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
27	AMARILLO,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	ROJO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	}	Cierra el bloque abierto anteriormente.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	private Tema tema = Tema.AMARILLO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	// COMPONENTES	Comentario para humanos; Java no ejecuta este texto. COMPONENTES
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	private JTextField txtCampo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
37	private JLabel lblIcono;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	// CONFIGURACIÓN	Comentario para humanos; Java no ejecuta este texto. CONFIGURACIÓN
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	private String placeholder = "Ingresa texto";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	private boolean mostrandoPlaceholder = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	private int radioCaja = 35;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	private int radioIcono = 28;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	// COLORES	Comentario para humanos; Java no ejecuta este texto. COLORES
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	// Amarillo Waldonald's	Comentario para humanos; Java no ejecuta este texto. Amarillo Waldonald's
54	private Color amarilloPrincipal =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

55	<code>new Color(255, 183, 0);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	<code>private Color amarilloClaro =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
58	<code>new Color(255, 239, 190);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
59		Línea en blanco: separa visualmente bloques para que el código sea más legible.
60	<code>// Rojo Waldonald's</code>	Comentario para humanos; Java no ejecuta este texto. Rojo Waldonald's
61	<code>private Color rojoPrincipal =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
62	<code>new Color(215, 25, 25);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	<code>private Color rojoClaro =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	<code>new Color(255, 205, 205);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	<code>// Caja donde se escribe</code>	Comentario para humanos; Java no ejecuta este texto. Caja donde se escribe
68	<code>private Color fondoCampo =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
69	<code>new Color(248, 248, 248);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
70		Línea en blanco: separa visualmente bloques para que el código sea más legible.
71	<code>private Color texto =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
72	<code>new Color(35, 35, 35);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	<code>private Color textoPlaceholder =</code>	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
75	<code>new Color(145, 145, 145);</code>	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
76		Línea en blanco: separa visualmente bloques para que el código sea más legible.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	<code>// CONSTRUCTOR</code>	Comentario para humanos; Java no ejecuta este texto. CONSTRUCTOR
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.
81	<code>public TextBox_Login() {</code>	Constructor de TextBox_Login: prepara un objeto nuevo y deja sus componentes listos para usarse.
82		Línea en blanco: separa visualmente bloques para que el código sea más legible.
83	<code>setOpaque(false);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	<code>setLayout(null);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86		Línea en blanco: separa visualmente bloques para que el código sea más legible.
87	<code>setPreferredSize(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	<code>new Dimension(600, 90)</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
89	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	<code>setMinimumSize(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
92	<code>new Dimension(300, 70)</code>	Crea una nueva instancia de Dimension y la usa o guarda para continuar el proceso.
93	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94		Línea en blanco: separa visualmente bloques para que el código sea más legible.
95		Línea en blanco: separa visualmente bloques para que el código sea más legible.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	<code>// ICONO</code>	Comentario para humanos; Java no ejecuta este texto. ICONO
98		Línea en blanco: separa visualmente bloques para que el código sea más legible.

99	lblIcono = new JLabel();	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
100		Línea en blanco: separa visualmente bloques para que el código sea más legible.
101	lblIcono.setHorizontalAlignment(	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
102	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
103	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	lblIcono.setVerticalAlignment(	Configura una propiedad mediante setVerticalAlignment en el objeto indicado.
106	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	lblIcono.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
110		Línea en blanco: separa visualmente bloques para que el código sea más legible.
111	add(lblIcono);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112		Línea en blanco: separa visualmente bloques para que el código sea más legible.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114		Línea en blanco: separa visualmente bloques para que el código sea más legible.
115	// CAMPO DE TEXTO	Comentario para humanos; Java no ejecuta este texto. CAMPO DE TEXTO
116		Línea en blanco: separa visualmente bloques para que el código sea más legible.
117	txtCampo = new JTextField();	Crea una nueva instancia de JTextField y la usa o guarda para continuar el proceso.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	txtCampo.setText(placeholder);	Cambia el texto visible de un componente de la interfaz.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	txtCampo.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
122	textoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
123	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124		Línea en blanco: separa visualmente bloques para que el código sea más legible.
125	txtCampo.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
126	new Font(	Crea una nueva instancia de Font y la usa o guarda para continuar el proceso.
127	"Dialog",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	Font.PLAIN,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	16	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132		Línea en blanco: separa visualmente bloques para que el código sea más legible.
133	txtCampo.setBorder(	Configura una propiedad mediante setBorder en el objeto indicado.
134	BorderFactory.createEmptyBorder()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136		Línea en blanco: separa visualmente bloques para que el código sea más legible.
137	txtCampo.setOpaque(false);	Configura una propiedad mediante setOpaque en el objeto indicado.
138		Línea en blanco: separa visualmente bloques para que el código sea más legible.
139	txtCampo.setBackground(	Configura una propiedad mediante setBackground en el objeto indicado.
140	new Color(0, 0, 0, 0)	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
141	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142		Línea en blanco: separa visualmente bloques para que el código sea más legible.

143	<code>txtCampo.setCaretColor(</code>	Configura una propiedad mediante setCaretColor en el objeto indicado.
144	<code>texto</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
145	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	<code>txtCampo.setCursor(</code>	Configura una propiedad mediante setCursor en el objeto indicado.
148	<code>new Cursor(</code>	Crea una nueva instancia de Cursor y la usa o guarda para continuar el proceso.
149	<code>Cursor.TEXT_CURSOR</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
150	<code>)</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
151	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152		Línea en blanco: separa visualmente bloques para que el código sea más legible.
153	<code>add(txtCampo);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154		Línea en blanco: separa visualmente bloques para que el código sea más legible.
155		Línea en blanco: separa visualmente bloques para que el código sea más legible.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	<code>// PLACEHOLDER</code>	Comentario para humanos; Java no ejecuta este texto. PLACEHOLDER
158		Línea en blanco: separa visualmente bloques para que el código sea más legible.
159	<code>txtCampo.addFocusListener(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
160	<code>new FocusAdapter() {</code>	Crea una nueva instancia de FocusAdapter y la usa o guarda para continuar el proceso.
161		Línea en blanco: separa visualmente bloques para que el código sea más legible.
162	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
163	<code>public void focusGained(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
164	<code>FocusEvent e) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165		Línea en blanco: separa visualmente bloques para que el código sea más legible.
166	<code>if (mostrandoPlaceholder) {</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
167		Línea en blanco: separa visualmente bloques para que el código sea más legible.
168	<code>txtCampo.setText("");</code>	Cambia el texto visible de un componente de la interfaz.
169		Línea en blanco: separa visualmente bloques para que el código sea más legible.
170	<code>txtCampo.setForeground(</code>	Configura una propiedad mediante setForeground en el objeto indicado.
171	<code>texto</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
172	<code>);</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	<code>mostrandoPlaceholder =</code>	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
175	<code>false;</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
176	<code>}</code>	Cierra el bloque abierto anteriormente.
177		Línea en blanco: separa visualmente bloques para que el código sea más legible.
178	<code>repaint();</code>	Ejecuta una instrucción Java completa; el punto y coma marca su final.
179	<code>}</code>	Cierra el bloque abierto anteriormente.
180		Línea en blanco: separa visualmente bloques para que el código sea más legible.
181	<code>@Override</code>	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
182	<code>public void focusLost(</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
183	<code>FocusEvent e) {</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	<code>if (</code>	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
186	<code>txtCampo</code>	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

187	.getText()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
188	.trim()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
189	.isEmpty()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
190	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	txtCampo.setText(	Cambia el texto visible de un componente de la interfaz.
193	placeholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
194	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
195		Línea en blanco: separa visualmente bloques para que el código sea más legible.
196	txtCampo.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
197	textoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
198	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	mostrandoPlaceholder =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
201	true;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
202	}	Cierra el bloque abierto anteriormente.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
205	}	Cierra el bloque abierto anteriormente.
206	}	Cierra el bloque abierto anteriormente.
207	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
208	}	Cierra el bloque abierto anteriormente.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210		Línea en blanco: separa visualmente bloques para que el código sea más legible.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	// POSICIONAR COMPONENTES	Comentario para humanos; Java no ejecuta este texto. POSICIONAR COMPONENTES
213		Línea en blanco: separa visualmente bloques para que el código sea más legible.
214	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
215	public void doLayout() {	Declara el método doLayout. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	super.doLayout();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218		Línea en blanco: separa visualmente bloques para que el código sea más legible.
219	int alto = getHeight();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
220		Línea en blanco: separa visualmente bloques para que el código sea más legible.
221	int tamanoIcono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
222	Math.min(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
223	alto - 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
224	90	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
226		Línea en blanco: separa visualmente bloques para que el código sea más legible.
227	int yIcono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
228	(alto - tamanoIcono) / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
229		Línea en blanco: separa visualmente bloques para que el código sea más legible.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.

231		Línea en blanco: separa visualmente bloques para que el código sea más legible.
232	// CUADRO DEL ICONO	Comentario para humanos; Java no ejecuta este texto. CUADRO DEL ICONO
233		Línea en blanco: separa visualmente bloques para que el código sea más legible.
234	lblIcono.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
235	4,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
236	yIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
237	tamanoIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
238	tamanoIcono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
239	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
240		Línea en blanco: separa visualmente bloques para que el código sea más legible.
241		Línea en blanco: separa visualmente bloques para que el código sea más legible.
242		Línea en blanco: separa visualmente bloques para que el código sea más legible.
243	// TEXTBOX	Comentario para humanos; Java no ejecuta este texto. TEXTBOX
244		Línea en blanco: separa visualmente bloques para que el código sea más legible.
245	int inicioTexto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
246	tamanoIcono + 30;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
247		Línea en blanco: separa visualmente bloques para que el código sea más legible.
248	txtCampo.setBounds(	Configura una propiedad mediante setBounds en el objeto indicado.
249	inicioTexto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
250	5,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
251	getWidth()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
252	- inicioTexto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	- 30,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
254	alto - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
255	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
256	}	Cierra el bloque abierto anteriormente.
257		Línea en blanco: separa visualmente bloques para que el código sea más legible.
258		Línea en blanco: separa visualmente bloques para que el código sea más legible.
259		Línea en blanco: separa visualmente bloques para que el código sea más legible.
260	// PINTAR DISEÑO	Comentario para humanos; Java no ejecuta este texto. PINTAR DISEÑO
261		Línea en blanco: separa visualmente bloques para que el código sea más legible.
262	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
263	protected void paintComponent(Graphics g) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
264		Línea en blanco: separa visualmente bloques para que el código sea más legible.
265	super.paintComponent(g);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267	Graphics2D g2 =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
268	(Graphics2D) g.create();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
270	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
271	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
272	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
274		Línea en blanco: separa visualmente bloques para que el código sea más legible.

275		Línea en blanco: separa visualmente bloques para que el código sea más legible.
276		Línea en blanco: separa visualmente bloques para que el código sea más legible.
277	// SELECCIONAR COLORES	Comentario para humanos; Java no ejecuta este texto. SELECCIONAR COLORES
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	Color colorPrincipal;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281	Color colorClaro;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
282		Línea en blanco: separa visualmente bloques para que el código sea más legible.
283	if (tema == Tema.ROJO) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
284		Línea en blanco: separa visualmente bloques para que el código sea más legible.
285	colorPrincipal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
286	rojoPrincipal;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	colorClaro =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
289	rojoClaro;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
290		Línea en blanco: separa visualmente bloques para que el código sea más legible.
291	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	colorPrincipal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
294	amarilloPrincipal;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
295		Línea en blanco: separa visualmente bloques para que el código sea más legible.
296	colorClaro =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
297	amarilloClaro;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
298	}	Cierra el bloque abierto anteriormente.
299		Línea en blanco: separa visualmente bloques para que el código sea más legible.
300		Línea en blanco: separa visualmente bloques para que el código sea más legible.
301	int ancho =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
302	getWidth();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
303		Línea en blanco: separa visualmente bloques para que el código sea más legible.
304	int alto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
305	getHeight();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
306		Línea en blanco: separa visualmente bloques para que el código sea más legible.
307		Línea en blanco: separa visualmente bloques para que el código sea más legible.
308	int tamanoIcono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
309	Math.min(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
310	alto - 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
311	90	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
313		Línea en blanco: separa visualmente bloques para que el código sea más legible.
314		Línea en blanco: separa visualmente bloques para que el código sea más legible.
315		Línea en blanco: separa visualmente bloques para que el código sea más legible.
316	// CAJA PRINCIPAL	Comentario para humanos; Java no ejecuta este texto. CAJA PRINCIPAL
317		Línea en blanco: separa visualmente bloques para que el código sea más legible.
318		Línea en blanco: separa visualmente bloques para que el código sea más legible.

319	int xCaja =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
320	tamanoIcono / 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
321		Línea en blanco: separa visualmente bloques para que el código sea más legible.
322	int yCaja = 10;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
323		Línea en blanco: separa visualmente bloques para que el código sea más legible.
324	int anchoCaja =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
325	ancho	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
326	- xCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
327	- 3;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	int altoCaja =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
330	alto - 20;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
331		Línea en blanco: separa visualmente bloques para que el código sea más legible.
332		Línea en blanco: separa visualmente bloques para que el código sea más legible.
333	// Fondo gris/blanco	Comentario para humanos; Java no ejecuta este texto. Fondo gris/blanco
334	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
335	fondoCampo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
336	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	g2.fillRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339	xCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
340	yCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341	anchoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
342	altoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
343	radioCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
344	radioCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
345	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
346		Línea en blanco: separa visualmente bloques para que el código sea más legible.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348		Línea en blanco: separa visualmente bloques para que el código sea más legible.
349	// BORDE COLOREADO	Comentario para humanos; Java no ejecuta este texto. BORDE COLOREADO
350		Línea en blanco: separa visualmente bloques para que el código sea más legible.
351	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
352	colorPrincipal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
353	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
354		Línea en blanco: separa visualmente bloques para que el código sea más legible.
355	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
356	new BasicStroke(2.5f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
357	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
358		Línea en blanco: separa visualmente bloques para que el código sea más legible.
359		Línea en blanco: separa visualmente bloques para que el código sea más legible.
360	RoundRectangle2D borde =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
361	new RoundRectangle2D.Double(	Crea una nueva instancia de RoundRectangle2D.Double y la usa o guarda para continuar el proceso.
362	xCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

363	yCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
364	anchoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	altoCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
366	radioCaja,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
367	radioCaja	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
368	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
370	g2.draw(borde);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
371		Línea en blanco: separa visualmente bloques para que el código sea más legible.
372		Línea en blanco: separa visualmente bloques para que el código sea más legible.
373		Línea en blanco: separa visualmente bloques para que el código sea más legible.
374	// CUADRO IZQUIERDO	Comentario para humanos; Java no ejecuta este texto. CUADRO IZQUIERDO
375		Línea en blanco: separa visualmente bloques para que el código sea más legible.
376		Línea en blanco: separa visualmente bloques para que el código sea más legible.
377	int xIcono = 3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
378		Línea en blanco: separa visualmente bloques para que el código sea más legible.
379	int yIcono =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
380	(alto - tamanoIcono)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
381	/ 2;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
383		Línea en blanco: separa visualmente bloques para que el código sea más legible.
384	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
385	colorClaro	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
386	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
387		Línea en blanco: separa visualmente bloques para que el código sea más legible.
388	g2.fillRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
389	xIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
390	yIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
391	tamanoIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
392	tamanoIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
393	radioIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
394	radioIcono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
395	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
396		Línea en blanco: separa visualmente bloques para que el código sea más legible.
397		Línea en blanco: separa visualmente bloques para que el código sea más legible.
398	// BORDE DEL CUADRO	Comentario para humanos; Java no ejecuta este texto. BORDE DEL CUADRO
399		Línea en blanco: separa visualmente bloques para que el código sea más legible.
400	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
401	colorPrincipal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
402	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
403		Línea en blanco: separa visualmente bloques para que el código sea más legible.
404	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
405	new BasicStroke(3f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
406	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

407		Línea en blanco: separa visualmente bloques para que el código sea más legible.
408	g2.drawRoundRect(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
409	xIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
410	yIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
411	tamanoIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
412	tamanoIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
413	radioIcono,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
414	radioIcono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
415	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417		Línea en blanco: separa visualmente bloques para que el código sea más legible.
418		Línea en blanco: separa visualmente bloques para que el código sea más legible.
419	// LÍNEAS DECORATIVAS	Comentario para humanos; Java no ejecuta este texto. LÍNEAS DECORATIVAS
420		Línea en blanco: separa visualmente bloques para que el código sea más legible.
421		Línea en blanco: separa visualmente bloques para que el código sea más legible.
422	int inicioDecoracion =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
423	tamanoIcono + 15;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
424		Línea en blanco: separa visualmente bloques para que el código sea más legible.
425	int centroDecoracion =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
426	(int) (ancho * 0.68);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
427		Línea en blanco: separa visualmente bloques para que el código sea más legible.
428	int finalDecoracion =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
429	ancho - 45;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
430		Línea en blanco: separa visualmente bloques para que el código sea más legible.
431		Línea en blanco: separa visualmente bloques para que el código sea más legible.
432	g2.setStroke(	Configura una propiedad mediante setStroke en el objeto indicado.
433	new BasicStroke(2f)	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
434	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
435		Línea en blanco: separa visualmente bloques para que el código sea más legible.
436	g2.setColor(	Configura una propiedad mediante setColor en el objeto indicado.
437	colorPrincipal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
438	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
439		Línea en blanco: separa visualmente bloques para que el código sea más legible.
440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
441	// Línea superior izquierda	Comentario para humanos; Java no ejecuta este texto. Línea superior izquierda
442	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443	inicioDecoracion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
444	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
445	centroDecoracion - 50,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448		Línea en blanco: separa visualmente bloques para que el código sea más legible.
449		Línea en blanco: separa visualmente bloques para que el código sea más legible.
450	// Línea superior derecha	Comentario para humanos; Java no ejecuta este texto. Línea superior derecha

451	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
452	centroDecoracion + 55,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
453	10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
454	finalDecoracion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
455	10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
456	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
457		Línea en blanco: separa visualmente bloques para que el código sea más legible.
458		Línea en blanco: separa visualmente bloques para que el código sea más legible.
459	// Línea inferior izquierda	Comentario para humanos; Java no ejecuta este texto. Línea inferior izquierda
460	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
461	inicioDecoracion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
462	alto - 10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
463	centroDecoracion - 50,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
464	alto - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
465	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
466		Línea en blanco: separa visualmente bloques para que el código sea más legible.
467		Línea en blanco: separa visualmente bloques para que el código sea más legible.
468	// Línea inferior derecha	Comentario para humanos; Java no ejecuta este texto. Línea inferior derecha
469	g2.drawLine(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
470	centroDecoracion + 55,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
471	alto - 10,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
472	finalDecoracion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
473	alto - 10	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
474	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
475		Línea en blanco: separa visualmente bloques para que el código sea más legible.
476		Línea en blanco: separa visualmente bloques para que el código sea más legible.
477		Línea en blanco: separa visualmente bloques para que el código sea más legible.
478	// PUNTOS DECORATIVOS	Comentario para humanos; Java no ejecuta este texto. PUNTOS DECORATIVOS
479		Línea en blanco: separa visualmente bloques para que el código sea más legible.
480		Línea en blanco: separa visualmente bloques para que el código sea más legible.
481	int cantidadPuntos = 6;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
482		Línea en blanco: separa visualmente bloques para que el código sea más legible.
483	int separacion = 12;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
484		Línea en blanco: separa visualmente bloques para que el código sea más legible.
485	int tamanoPunto = 5;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
486		Línea en blanco: separa visualmente bloques para que el código sea más legible.
487		Línea en blanco: separa visualmente bloques para que el código sea más legible.
488	int inicioPuntos =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
489	centroDecoracion	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
490	- (	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
491	(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
492	cantidadPuntos - 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
493	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
494	* separacion	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

495	/ 2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
496	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
497		Línea en blanco: separa visualmente bloques para que el código sea más legible.
498		Línea en blanco: separa visualmente bloques para que el código sea más legible.
499	for (	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
500	int i = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
501	i < cantidadPuntos;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
502	i++	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
503	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
504		Línea en blanco: separa visualmente bloques para que el código sea más legible.
505	int x =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
506	inicioPuntos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
507	+ i * separacion;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
508		Línea en blanco: separa visualmente bloques para que el código sea más legible.
509		Línea en blanco: separa visualmente bloques para que el código sea más legible.
510	// Superior	Comentario para humanos; Java no ejecuta este texto. Superior
511	g2.fillOval(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
512	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
513	8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
514	tamanoPunto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
515	tamanoPunto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
516	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
517		Línea en blanco: separa visualmente bloques para que el código sea más legible.
518		Línea en blanco: separa visualmente bloques para que el código sea más legible.
519	// Inferior	Comentario para humanos; Java no ejecuta este texto. Inferior
520	g2.fillOval(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
521	x,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
522	alto - 13,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
523	tamanoPunto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
524	tamanoPunto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
525	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
526	}	Cierra el bloque abierto anteriormente.
527		Línea en blanco: separa visualmente bloques para que el código sea más legible.
528		Línea en blanco: separa visualmente bloques para que el código sea más legible.
529	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
530	}	Cierra el bloque abierto anteriormente.
531		Línea en blanco: separa visualmente bloques para que el código sea más legible.
532		Línea en blanco: separa visualmente bloques para que el código sea más legible.
533		Línea en blanco: separa visualmente bloques para que el código sea más legible.
534	// CAMBIAR TEMA	Comentario para humanos; Java no ejecuta este texto. CAMBIAR TEMA
535		Línea en blanco: separa visualmente bloques para que el código sea más legible.
536	public void setTema(Tema tema) {	Declara el método setTema. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
537		Línea en blanco: separa visualmente bloques para que el código sea más legible.
538	this.tema = tema;	Accede al atributo del objeto actual para asignarlo o utilizarlo.

539		Línea en blanco: separa visualmente bloques para que el código sea más legible.
540	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
541	}	Cierra el bloque abierto anteriormente.
542		Línea en blanco: separa visualmente bloques para que el código sea más legible.
543		Línea en blanco: separa visualmente bloques para que el código sea más legible.
544	public Tema getTema() {	Declara el método getTema. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
545		Línea en blanco: separa visualmente bloques para que el código sea más legible.
546	return tema;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
547	}	Cierra el bloque abierto anteriormente.
548		Línea en blanco: separa visualmente bloques para que el código sea más legible.
549		Línea en blanco: separa visualmente bloques para que el código sea más legible.
550		Línea en blanco: separa visualmente bloques para que el código sea más legible.
551	// GET TEXT	Comentario para humanos; Java no ejecuta este texto. GET TEXT
552	// ESTE ES EL MÉTODO IMPORTANTE PARA EL LOGIN	Comentario para humanos; Java no ejecuta este texto. ESTE ES EL MÉTODO IMPORTANTE PARA EL LOGIN
553		Línea en blanco: separa visualmente bloques para que el código sea más legible.
554	public String getText() {	Declara el método getText. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
555		Línea en blanco: separa visualmente bloques para que el código sea más legible.
556	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
557	return "";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
558	}	Cierra el bloque abierto anteriormente.
559		Línea en blanco: separa visualmente bloques para que el código sea más legible.
560	return txtCampo.getText();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
561	}	Cierra el bloque abierto anteriormente.
562		Línea en blanco: separa visualmente bloques para que el código sea más legible.
563		Línea en blanco: separa visualmente bloques para que el código sea más legible.
564		Línea en blanco: separa visualmente bloques para que el código sea más legible.
565	// SET TEXT	Comentario para humanos; Java no ejecuta este texto. SET TEXT
566		Línea en blanco: separa visualmente bloques para que el código sea más legible.
567	public void setText(String textoNuevo) {	Declara el método setText. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
568		Línea en blanco: separa visualmente bloques para que el código sea más legible.
569	if (	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
570	textoNuevo == null	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
571	textoNuevo.isEmpty()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
572	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
573		Línea en blanco: separa visualmente bloques para que el código sea más legible.
574	txtCampo.setText(	Cambia el texto visible de un componente de la interfaz.
575	placeholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
576	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
577		Línea en blanco: separa visualmente bloques para que el código sea más legible.
578	txtCampo.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
579	textoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
580	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
581		Línea en blanco: separa visualmente bloques para que el código sea más legible.
582	mostrandoPlaceholder =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

583	true;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
584		Línea en blanco: separa visualmente bloques para que el código sea más legible.
585	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
586		Línea en blanco: separa visualmente bloques para que el código sea más legible.
587	txtCampo.setText(	Cambia el texto visible de un componente de la interfaz.
588	textoNuevo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
589	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
590		Línea en blanco: separa visualmente bloques para que el código sea más legible.
591	txtCampo.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
592	texto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
593	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
594		Línea en blanco: separa visualmente bloques para que el código sea más legible.
595	mostrandoPlaceholder =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
596	false;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
597	}	Cierra el bloque abierto anteriormente.
598	}	Cierra el bloque abierto anteriormente.
599		Línea en blanco: separa visualmente bloques para que el código sea más legible.
600		Línea en blanco: separa visualmente bloques para que el código sea más legible.
601		Línea en blanco: separa visualmente bloques para que el código sea más legible.
602	// MÉTODOS ANTERIORES	Comentario para humanos; Java no ejecuta este texto. MÉTODOS ANTERIORES
603		Línea en blanco: separa visualmente bloques para que el código sea más legible.
604	public String getTexto() {	Declara el método getTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
605		Línea en blanco: separa visualmente bloques para que el código sea más legible.
606	return getText();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
607	}	Cierra el bloque abierto anteriormente.
608		Línea en blanco: separa visualmente bloques para que el código sea más legible.
609		Línea en blanco: separa visualmente bloques para que el código sea más legible.
610	public void setTexto(String textoNuevo) {	Declara el método setTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
611		Línea en blanco: separa visualmente bloques para que el código sea más legible.
612	setText(textoNuevo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
613	}	Cierra el bloque abierto anteriormente.
614		Línea en blanco: separa visualmente bloques para que el código sea más legible.
615		Línea en blanco: separa visualmente bloques para que el código sea más legible.
616		Línea en blanco: separa visualmente bloques para que el código sea más legible.
617	// DAR FOCO AL CAMPO	Comentario para humanos; Java no ejecuta este texto. DAR FOCO AL CAMPO
618		Línea en blanco: separa visualmente bloques para que el código sea más legible.
619	public void enfocar() {	Declara el método enfocar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
620		Línea en blanco: separa visualmente bloques para que el código sea más legible.
621	txtCampo.requestFocusInWindow();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
622	}	Cierra el bloque abierto anteriormente.
623		Línea en blanco: separa visualmente bloques para que el código sea más legible.
624		Línea en blanco: separa visualmente bloques para que el código sea más legible.
625		Línea en blanco: separa visualmente bloques para que el código sea más legible.
626	// PLACEHOLDER	Comentario para humanos; Java no ejecuta este texto. PLACEHOLDER

627		Línea en blanco: separa visualmente bloques para que el código sea más legible.
628	public void setPlaceholder(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
629	String placeholder) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
630		Línea en blanco: separa visualmente bloques para que el código sea más legible.
631	this.placeholder =	Accede al atributo del objeto actual para asignarlo o utilizarlo.
632	placeholder;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
633		Línea en blanco: separa visualmente bloques para que el código sea más legible.
634	if (mostrandoPlaceholder) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
635		Línea en blanco: separa visualmente bloques para que el código sea más legible.
636	txtCampo.setText(	Cambia el texto visible de un componente de la interfaz.
637	placeholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
638	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
639		Línea en blanco: separa visualmente bloques para que el código sea más legible.
640	txtCampo.setForeground(	Configura una propiedad mediante setForeground en el objeto indicado.
641	textoPlaceholder	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
642	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
643	}	Cierra el bloque abierto anteriormente.
644	}	Cierra el bloque abierto anteriormente.
645		Línea en blanco: separa visualmente bloques para que el código sea más legible.
646		Línea en blanco: separa visualmente bloques para que el código sea más legible.
647	public String getPlaceholder() {	Declara el método getPlaceholder. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
648		Línea en blanco: separa visualmente bloques para que el código sea más legible.
649	return placeholder;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
650	}	Cierra el bloque abierto anteriormente.
651		Línea en blanco: separa visualmente bloques para que el código sea más legible.
652		Línea en blanco: separa visualmente bloques para que el código sea más legible.
653		Línea en blanco: separa visualmente bloques para que el código sea más legible.
654	// ICONO	Comentario para humanos; Java no ejecuta este texto. ICONO
655		Línea en blanco: separa visualmente bloques para que el código sea más legible.
656	public void setIcono(Icon icono) {	Declara el método setIcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
657		Línea en blanco: separa visualmente bloques para que el código sea más legible.
658	lblIcono.setIcon(	Configura una propiedad mediante setIcon en el objeto indicado.
659	icono	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
660	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
661	}	Cierra el bloque abierto anteriormente.
662		Línea en blanco: separa visualmente bloques para que el código sea más legible.
663		Línea en blanco: separa visualmente bloques para que el código sea más legible.
664	public Icon getIcono() {	Declara el método getIcono. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
665		Línea en blanco: separa visualmente bloques para que el código sea más legible.
666	return lblIcono.getIcon();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
667	}	Cierra el bloque abierto anteriormente.
668		Línea en blanco: separa visualmente bloques para que el código sea más legible.
669		Línea en blanco: separa visualmente bloques para que el código sea más legible.
670		Línea en blanco: separa visualmente bloques para que el código sea más legible.

671	// ACCESO AL JTextField	Comentario para humanos; Java no ejecuta este texto. ACCESO AL JTextField
672		Línea en blanco: separa visualmente bloques para que el código sea más legible.
673	public JTextField getTextField() {	Declara el método getTextField. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
674		Línea en blanco: separa visualmente bloques para que el código sea más legible.
675	return txtCampo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
676	}	Cierra el bloque abierto anteriormente.
677		Línea en blanco: separa visualmente bloques para que el código sea más legible.
678		Línea en blanco: separa visualmente bloques para que el código sea más legible.
679		Línea en blanco: separa visualmente bloques para que el código sea más legible.
680	// CAMBIAR COLORES MANUALMENTE	Comentario para humanos; Java no ejecuta este texto. CAMBIAR COLORES MANUALMENTE
681		Línea en blanco: separa visualmente bloques para que el código sea más legible.
682	public void setAmarilloPrincipal(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
683	Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
684		Línea en blanco: separa visualmente bloques para que el código sea más legible.
685	amarilloPrincipal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
686	color;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
687		Línea en blanco: separa visualmente bloques para que el código sea más legible.
688	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
689	}	Cierra el bloque abierto anteriormente.
690		Línea en blanco: separa visualmente bloques para que el código sea más legible.
691		Línea en blanco: separa visualmente bloques para que el código sea más legible.
692	public void setRojoPrincipal(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
693	Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
694		Línea en blanco: separa visualmente bloques para que el código sea más legible.
695	rojoPrincipal =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
696	color;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
697		Línea en blanco: separa visualmente bloques para que el código sea más legible.
698	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
699	}	Cierra el bloque abierto anteriormente.
700		Línea en blanco: separa visualmente bloques para que el código sea más legible.
701		Línea en blanco: separa visualmente bloques para que el código sea más legible.
702	public void setFondoCampo(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
703	Color color) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
704		Línea en blanco: separa visualmente bloques para que el código sea más legible.
705	fondoCampo =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
706	color;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
707		Línea en blanco: separa visualmente bloques para que el código sea más legible.
708	repaint();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
709	}	Cierra el bloque abierto anteriormente.
710	}	Cierra el bloque abierto anteriormente.

16.111 src/Login/Login.form

Metadatos visuales de NetBeans para diseñar Login. Define componentes, propiedades, eventos y posiciones; se sincroniza con el archivo Java del mismo nombre.

Tipo: .form | Líneas: 203 | Archivo 111 de 157

1	<?xml version="1.0" encoding="UTF-8" ?>	Cabecera XML: informa a NetBeans la versión y codificación del archivo.
2		Línea en blanco para organizar el XML del formulario.
3	<Form version="1.3" maxVersion="1.9" type="org.netbeans.modules.form.forminfo.JFrameFormInfo">	Comienza la definición del formulario que administra el GUI Builder.
4	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
5	<Property name="defaultCloseOperation" type="int" value="3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
6	</Properties>	Cierra la sección XML abierta anteriormente.
7	<SyntheticProperties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
8	<SyntheticProperty name="formSizePolicy" type="int" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
9	<SyntheticProperty name="generateCenter" type="boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
10	</SyntheticProperties>	Cierra la sección XML abierta anteriormente.
11	<AuxValues>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
12	<AuxValue name="FormSettings_autoResourcing" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
13	<AuxValue name="FormSettings_autoSetComponentName" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
14	<AuxValue name="FormSettings_generateFQN" type="java.lang.Boolean" value="true"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
15	<AuxValue name="FormSettings_generateMnemonicsCode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
16	<AuxValue name="FormSettings_i18nAutoMode" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
17	<AuxValue name="FormSettings_layoutCodeTarget" type="java.lang.Integer" value="1"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
18	<AuxValue name="FormSettings_listenerGenerationStyle" type="java.lang.Integer" value="0"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
19	<AuxValue name="FormSettings_variablesLocal" type="java.lang.Boolean" value="false"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
20	<AuxValue name="FormSettings_variablesModifier" type="java.lang.Integer" value="2"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
21	<AuxValue name="designerSize" type="java.awt.Dimension" value="-84,-19,0,5,115,114,0,18,106,97,118,97,46,97,119,116,46,68,105,109,101,110,115,105,111,110,65,-114,-39,-41,-84,95,68,20,2,0,2,73,0,6,104,101,105,103,104,116,73,0,5,119,105,100,116,104,120,112,0,0,4,56,0,0,7,-128"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
22	</AuxValues>	Cierra la sección XML abierta anteriormente.
23		Línea en blanco para organizar el XML del formulario.
24	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
25	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
26	</Layout>	Cierra la sección XML abierta anteriormente.
27	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
28	<Container class="javax.swing.JPanel" name="jPanel1">	Declara un componente o contenedor visual y su clase Java.
29	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
30	<Property name="background" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
31	<Color blue="fff" green="ff" red="ff" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
32	</Property>	Cierra la sección XML abierta anteriormente.
33	</Properties>	Cierra la sección XML abierta anteriormente.
34	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
35	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
36	<AbsoluteConstraints x="-20" y="0" width="1940" height="1090"/>	Define la posición y el tamaño del componente dentro del diseño.
37	</Constraint>	Cierra la sección XML abierta anteriormente.

38	</Constraints>	Cierra la sección XML abierta anteriormente.
39		Línea en blanco para organizar el XML del formulario.
40	<Layout class="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout">	Indica el administrador de distribución que organiza los componentes hijos.
41	<Property name="useNullLayout" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
42	</Layout>	Cierra la sección XML abierta anteriormente.
43	<SubComponents>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
44	<Component class="javax.swing.JLabel" name="jLabel1">	Declara un componente o contenedor visual y su clase Java.
45	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
46	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
47		Metadato de NetBeans que conserva el estado editable del diseñador visual.
48	</Property>	Cierra la sección XML abierta anteriormente.
49	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
50	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
51	</Property>	Cierra la sección XML abierta anteriormente.
52	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
53	<Property name="text" type="java.lang.String" value="Contraseña"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
54	</Properties>	Cierra la sección XML abierta anteriormente.
55	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
56	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
57	<AbsoluteConstraints x="1080" y="520" width="240" height="40"/>	Define la posición y el tamaño del componente dentro del diseño.
58	</Constraint>	Cierra la sección XML abierta anteriormente.
59	</Constraints>	Cierra la sección XML abierta anteriormente.
60	</Component>	Cierra la sección XML abierta anteriormente.
61	<Component class="Componentes.BotonDerretido" name="botonDerretido1">	Declara un componente o contenedor visual y su clase Java.
62	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
63	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
	>	
64	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
65	</Property>	Cierra la sección XML abierta anteriormente.
66	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
67		Metadato de NetBeans que conserva el estado editable del diseñador visual.
68	</Property>	Cierra la sección XML abierta anteriormente.
69	<Property name="label" type="java.lang.String" value="Iniciar Sesion"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
70	</Properties>	Cierra la sección XML abierta anteriormente.
71	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
72	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="botonDerretido1ActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
73	</Events>	Cierra la sección XML abierta anteriormente.
74	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
75	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
76	<AbsoluteConstraints x="1070" y="660" width="440" height="80"/>	Define la posición y el tamaño del componente dentro del diseño.
77	</Constraint>	Cierra la sección XML abierta anteriormente.
78	</Constraints>	Cierra la sección XML abierta anteriormente.

79	</Component>	Cierra la sección XML abierta anteriormente.
80	<Component class="Labels.LabelEscalable" name="labelEscalable3">	Declara un componente o contenedor visual y su clase Java.
81	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
82	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
83	<Image iconType="3" name="/Imagenes/Panel2.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
84	</Property>	Cierra la sección XML abierta anteriormente.
85	<Property name="text" type="java.lang.String" value="labelEscalable3"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
86	</Properties>	Cierra la sección XML abierta anteriormente.
87	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
88	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
89	<AbsoluteConstraints x="20" y="0" width="670" height="1080"/>	Define la posición y el tamaño del componente dentro del diseño.
90	</Constraint>	Cierra la sección XML abierta anteriormente.
91	</Constraints>	Cierra la sección XML abierta anteriormente.
92	</Component>	Cierra la sección XML abierta anteriormente.
93	<Component class="javax.swing.JLabel" name="jLabel2">	Declara un componente o contenedor visual y su clase Java.
94	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
95	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
96		Metadato de NetBeans que conserva el estado editable del diseñador visual.
97	</Property>	Cierra la sección XML abierta anteriormente.
98	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
99	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
100	</Property>	Cierra la sección XML abierta anteriormente.
101	<Property name="horizontalAlignment" type="int" value="2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
102	<Property name="text" type="java.lang.String" value="Usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
103	</Properties>	Cierra la sección XML abierta anteriormente.
104	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
105	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
106	<AbsoluteConstraints x="1080" y="372" width="190" height="40"/>	Define la posición y el tamaño del componente dentro del diseño.
107	</Constraint>	Cierra la sección XML abierta anteriormente.
108	</Constraints>	Cierra la sección XML abierta anteriormente.
109	</Component>	Cierra la sección XML abierta anteriormente.
110	<Component class="javax.swing.JLabel" name="jLabel3">	Declara un componente o contenedor visual y su clase Java.
111	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
112	<Property name="font" type="java.awt.Font" editor="org.netbeans.beaninfo.editors.FontEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
113		Metadato de NetBeans que conserva el estado editable del diseñador visual.
114	</Property>	Cierra la sección XML abierta anteriormente.
115	<Property name="foreground" type="java.awt.Color" editor="org.netbeans.beaninfo.editors.ColorEditor"	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
>		
116	<Color blue="0" green="0" red="0" type="rgb"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
117	</Property>	Cierra la sección XML abierta anteriormente.
118	<Property name="horizontalAlignment" type="int" value="0"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
119	<Property name="text" type="java.lang.String" value="¡Bienvenido a Waldonald's!"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.

120	</Properties>	Cierra la sección XML abierta anteriormente.
121	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
122	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
123	<AbsoluteConstraints x="910" y="260" width="730" height="60"/>	Define la posición y el tamaño del componente dentro del diseño.
124	</Constraint>	Cierra la sección XML abierta anteriormente.
125	</Constraints>	Cierra la sección XML abierta anteriormente.
126	</Component>	Cierra la sección XML abierta anteriormente.
127	<Component class="javax.swing.JButton" name="jButton2">	Declara un componente o contenedor visual y su clase Java.
128	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
129	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
130	<Image iconType="3" name="/Imagenes/visualizar.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
131	</Property>	Cierra la sección XML abierta anteriormente.
132	<Property name="borderPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
133	<Property name="contentAreaFilled" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
134	<Property name="cursor" type="java.awt.Cursor" editor="org.netbeans.modules.form.editors2.CursorEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
135	<Color id="Hand Cursor"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
136	</Property>	Cierra la sección XML abierta anteriormente.
137	<Property name="focusPainted" type="boolean" value="false"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
138	<Property name="maximumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
139	<Dimension value="[40, 40]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
140	</Property>	Cierra la sección XML abierta anteriormente.
141	<Property name="minimumSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
142	<Dimension value="[40, 40]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
143	</Property>	Cierra la sección XML abierta anteriormente.
144	<Property name="preferredSize" type="java.awt.Dimension" editor="org.netbeans.beaninfo.editors.DimensionEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
145	<Dimension value="[40, 40]"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
146	</Property>	Cierra la sección XML abierta anteriormente.
147	</Properties>	Cierra la sección XML abierta anteriormente.
148	<Events>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
149	<EventHandler event="actionPerformed" listener="java.awt.event.ActionListener" parameters="java.awt.event.ActionEvent" handler="jButton2ActionPerformed"/>	Asocia un evento visual con el método Java que lo atenderá.
150	</Events>	Cierra la sección XML abierta anteriormente.
151	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
152	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
153	<AbsoluteConstraints x="1520" y="570" width="50" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
154	</Constraint>	Cierra la sección XML abierta anteriormente.
155	</Constraints>	Cierra la sección XML abierta anteriormente.
156	</Component>	Cierra la sección XML abierta anteriormente.
157	<Component class="Labels.LabelEscalable" name="labelEscalable2">	Declara un componente o contenedor visual y su clase Java.
158	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.

159	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
160	<Image iconType="3" name="/Imágenes/MiniLogo 2.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
161	</Property>	Cierra la sección XML abierta anteriormente.
162	<Property name="text" type="java.lang.String" value="labelEscalable2"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
163	</Properties>	Cierra la sección XML abierta anteriormente.
164	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
165	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
166	<AbsoluteConstraints x="1210" y="80" width="140" height="180"/>	Define la posición y el tamaño del componente dentro del diseño.
167	</Constraint>	Cierra la sección XML abierta anteriormente.
168	</Constraints>	Cierra la sección XML abierta anteriormente.
169	</Component>	Cierra la sección XML abierta anteriormente.
170	<Component class="Labels.LabelEscalable" name="labelEscalable1">	Declara un componente o contenedor visual y su clase Java.
171	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
172	<Property name="icon" type="javax.swing.Icon" editor="org.netbeans.modules.form.editors2.IconEditor">	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
173	<Image iconType="3" name="/Imágenes/Usuarios.png"/>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
174	</Property>	Cierra la sección XML abierta anteriormente.
175	<Property name="text" type="java.lang.String" value="labelEscalable1"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
176	</Properties>	Cierra la sección XML abierta anteriormente.
177	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
178	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
179	<AbsoluteConstraints x="1010" y="420" width="50" height="50"/>	Define la posición y el tamaño del componente dentro del diseño.
180	</Constraint>	Cierra la sección XML abierta anteriormente.
181	</Constraints>	Cierra la sección XML abierta anteriormente.
182	</Component>	Cierra la sección XML abierta anteriormente.
183	<Component class="Labels.TextBox_Login" name="textBox_Login2">	Declara un componente o contenedor visual y su clase Java.
184	<Properties>	Metadato de NetBeans que conserva el estado editable del diseñador visual.
185	<Property name="placeholder" type="java.lang.String" value="Ingresa tu usuario"/>	Configura una propiedad visual, por ejemplo texto, color, tamaño o comportamiento.
186	</Properties>	Cierra la sección XML abierta anteriormente.
187	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
188	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
189	<AbsoluteConstraints x="990" y="400" width="-1" height="-1"/>	Define la posición y el tamaño del componente dentro del diseño.
190	</Constraint>	Cierra la sección XML abierta anteriormente.
191	</Constraints>	Cierra la sección XML abierta anteriormente.
192	</Component>	Cierra la sección XML abierta anteriormente.
193	<Component class="Labels.Textbox_Contrasena" name="textbox_Contrasena1">	Declara un componente o contenedor visual y su clase Java.
194	<Constraints>	Define la posición y el tamaño del componente dentro del diseño.
195	<Constraint layoutClass="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout" value="org.netbeans.modules.form.compat2.layouts.DesignAbsoluteLayout\$AbsoluteConstraintsDescription">	Define la posición y el tamaño del componente dentro del diseño.
196	<AbsoluteConstraints x="990" y="550" width="-1" height="-1"/>	Define la posición y el tamaño del componente dentro del diseño.
197	</Constraint>	Cierra la sección XML abierta anteriormente.
198	</Constraints>	Cierra la sección XML abierta anteriormente.

199	</Component>	Cierra la sección XML abierta anteriormente.
200	</SubComponents>	Cierra la sección XML abierta anteriormente.
201	</Container>	Cierra la sección XML abierta anteriormente.
202	</SubComponents>	Cierra la sección XML abierta anteriormente.
203	</Form>	Cierra la sección XML abierta anteriormente.

16.112 src/Login/Login.java

Pantalla de autenticación. Busca al usuario, verifica la contraseña, valida estado y turno, guarda la sesión y abre el panel correcto según el rol.

Tipo: java | Líneas: 396 | Archivo 112 de 157

Clases/tipos detectados: Login

Línea	Constructor o método detectado
38	public Login() {
68	public void keyPressed(java.awt.event.KeyEvent e) {
80	private Font cargarFuente(String archivo, float tamaño) {
100	private void iniciarSesion() {
237	private void agregarKeyListenerRecursoivo(java.awt.Component componente, java.awt.event.KeyListener listener) {
253	private void initComponents() {
283	public void actionPerformed(java.awt.event.ActionEvent evt) {
314	public void actionPerformed(java.awt.event.ActionEvent evt) {
338	private void botonDerretidoActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_botonDerretidoActionPerformed
342	private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {GEN-FIRST:event_jButton2ActionPerformed
351	public static void main(String args[]) {
377	public void run() {

1	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
2	* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
3	* Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	package Login;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	import Componentes.BarraTituloForm;	Importa Componentes.BarraTituloForm para poder usar esa clase sin escribir su nombre completo.
8	import Conexion.Conexion;	Importa Conexion.Conexion para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	import GUI_ADMINISTRADOR.InicioAdminForm;	Importa GUI_ADMINISTRADOR.InicioAdminForm para poder usar esa clase sin escribir su nombre completo.
11	import GUI_CAJERO.Cajero;	Importa GUI_CAJERO.Cajero para poder usar esa clase sin escribir su nombre completo.
12	import Utilidades.SeguridadContrasena;	Importa Utilidades.SeguridadContrasena para poder usar esa clase sin escribir su nombre completo.
13	import Utilidades.SesionUsuario;	Importa Utilidades.SesionUsuario para poder usar esa clase sin escribir su nombre completo.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
16	import java.io.InputStream;	Importa java.io.InputStream para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	import java.sql.Connection;	Importa java.sql.Connection: representa una sesión abierta con la base de datos.
19	import java.sql.PreparedStatement;	Importa java.sql.PreparedStatement: ejecuta SQL con parámetros y evita concatenar valores del usuario.
20	import java.sql.ResultSet;	Importa java.sql.ResultSet: permite recorrer las filas devueltas por una consulta.

21	import java.sql.SQLException;	Importa java.sql.SQLException: representa errores ocurridos al comunicarse con MySQL.
22	import java.awt.event.ActionEvent;	Importa java.awt.event.ActionEvent para poder usar esa clase sin escribir su nombre completo.
23	import java.awt.event.ActionListener;	Importa java.awt.event.ActionListener para poder usar esa clase sin escribir su nombre completo.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	import javax.swing.JOptionPane;	Importa javax.swing.JOptionPane para poder usar esa clase sin escribir su nombre completo.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
28	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
29	* @author Computacion	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
30	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
31	public class Login extends javax.swing.JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	private boolean contraseñaVisible = false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
36	* Creates new form Login	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
37	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
38	public Login() {	Constructor de Login: prepara un objeto nuevo y deja sus componentes listos para usarse.
39	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	initComponents();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	BarraTituloForm.instalar(this, "Inicio de sesión");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	jLabel1.setFont(	Configura una propiedad mediante setFont en el objeto indicado.
46	cargarFuente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
47	"DMSans-Bold.ttf",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	28f	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	// Placeholders	Comentario para humanos; Java no ejecuta este texto. Placeholders
53	textBox_Login2.setPlaceholder("Ingresa tu usuario");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
54	textBox_Contrasena1.setPlaceholder("Ingresa contraseña");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	// Evitar que un textbox obtenga el foco automáticamente	Comentario para humanos; Java no ejecuta este texto. Evitar que un textbox obtenga el foco automáticamente
57	jPanel1.setFocusable(true);	Configura una propiedad mediante setFocusable en el objeto indicado.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	javax.swing.SwingUtilities.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	jPanel1.requestFocusInWindow();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	// -----	Comentario para humanos; Java no ejecuta este texto.
64	// A PARTIR DE AQUÍ (LÍNEA 49 APROXIMADAMENTE):	Comentario para humanos; Java no ejecuta este texto. A PARTIR DE AQUÍ (LÍNEA 49 APROXIMADAMENTE):

65	// -----	Comentario para humanos; Java no ejecuta este texto. -----
66	java.awt.event.KeyAdapter enterKeyAdapter = new java.awt.event.KeyAdapter() {	Crea una nueva instancia de java.awt.event.KeyAdapter y la usa o guarda para continuar el proceso.
67	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
68	public void keyPressed(java.awt.event.KeyEvent e) {	Declara el método keyPressed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
69	if (e.getKeyCode() == java.awt.event.KeyEvent.VK_ENTER) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
70	iniciarSesion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
71	}	Cierra el bloque abierto anteriormente.
72	}	Cierra el bloque abierto anteriormente.
73	};	Cierra el bloque abierto anteriormente y termina la declaración.
74		Línea en blanco: separa visualmente bloques para que el código sea más legible.
75	// Agregar el listener a los componentes y a sus campos de texto internos	Comentario para humanos; Java no ejecuta este texto. Agregar el listener a los componentes y a sus campos de texto internos
76	agregarKeyListenerRecursivo(textBox_Login2, enterKeyAdapter);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	agregarKeyListenerRecursivo(textbox_Contrasena1, enterKeyAdapter);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	}	Cierra el bloque abierto anteriormente.
79		Línea en blanco: separa visualmente bloques para que el código sea más legible.
80	private Font cargarFuente(String archivo, float tamaño) {	Declara el método cargarFuente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
81	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
82	InputStream fuente = getClass().getResourceAsStream(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
83	"/Font/DMSans/" + archivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
84	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	if (fuente == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
87	System.out.println("No se encontró la fuente: " + archivo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	return new Font("Arial", Font.PLAIN, (int) tamaño);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
89	}	Cierra el bloque abierto anteriormente.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	return Font.createFont(Font.TRUETYPE_FONT, fuente)	Finaliza el método y devuelve el valor indicado a quien lo llamó.
92	.deriveFont(tamaño);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93		Línea en blanco: separa visualmente bloques para que el código sea más legible.
94	} catch (Exception e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
95	e.printStackTrace();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	return new Font("Arial", Font.PLAIN, (int) tamaño);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
97	}	Cierra el bloque abierto anteriormente.
98	}	Cierra el bloque abierto anteriormente.
99		Línea en blanco: separa visualmente bloques para que el código sea más legible.
100	private void iniciarSesion() {	Declara el método iniciarSesion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	// Obtener datos de los campos personalizados	Comentario para humanos; Java no ejecuta este texto. Obtener datos de los campos personalizados
103	String usuario = textBox_Login2.getText().trim();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
104	String contraseña = new String(textbox_Contrasena1.getPassword());	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
105		Línea en blanco: separa visualmente bloques para que el código sea más legible.
106	// Verificar campos vacíos	Comentario para humanos; Java no ejecuta este texto. Verificar campos vacíos
107	if (usuario.isEmpty() contraseña.isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
110	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
111	"Por favor ingrese su usuario y contraseña.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
112	"Campos incompletos",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	JOptionPane.WARNING_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
114	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
116	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
117	}	Cierra el bloque abierto anteriormente.
118		Línea en blanco: separa visualmente bloques para que el código sea más legible.
119	// Conectar a MySQL	Comentario para humanos; Java no ejecuta este texto. Conectar a MySQL
120	Connection conexion = Conexion.conectar();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
121		Línea en blanco: separa visualmente bloques para que el código sea más legible.
122	if (conexion == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
123		Línea en blanco: separa visualmente bloques para que el código sea más legible.
124	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	"No se pudo conectar con la base de datos.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
127	"Error de conexión",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	JOptionPane.ERROR_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130		Línea en blanco: separa visualmente bloques para que el código sea más legible.
131	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
132	}	Cierra el bloque abierto anteriormente.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	String sql = ""	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
135	SELECT id_usuario,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
136	nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
137	apellido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
138	rol,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
139	password_hash	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140	FROM usuario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
141	WHERE usuario = ?	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
142	AND estado = TRUE	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
143	"";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	try (Connection con = conexion; PreparedStatement ps = con.prepareStatement(sql)) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	ps.setString(1, usuario);	Configura una propiedad mediante setString en el objeto indicado.
148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
149	try (ResultSet rs = ps.executeQuery()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
150		Línea en blanco: separa visualmente bloques para que el código sea más legible.
151	if (rs.next()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

152	String hashAlmacenado = rs.getString("password_hash");	Define o usa información sensible de contraseña. En un sistema real debe venir de configuración segura y nunca publicarse.
153	if (!SeguridadContrasena.verificar(contraseña, hashAlmacenado)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
154	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
156	"Usuario o contraseña incorrectos.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	"Inicio de sesión",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
158	JOptionPane.ERROR_MESSAGE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	textbox_Contrasena1.setText("");	Cambia el texto visible de un componente de la interfaz.
160	textbox_Contrasena1.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
162	}	Cierra el bloque abierto anteriormente.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	// Migra automáticamente cuentas antiguas con SHA-256 simple.	Comentario para humanos; Java no ejecuta este texto. Migra automáticamente cuentas antiguas con SHA-256 simple.
165	if (hashAlmacenado == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
166	!hashAlmacenado.startsWith("PBKDF2-SHA256\$")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	try (PreparedStatement migracion = con.prepareStatement(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
168	"UPDATE usuario SET password_hash = ? WHERE id_usuario = ?")) {	Define o usa información sensible de contraseña. En un sistema real debe venir de configuración segura y nunca publicarse.
169	migracion.setString(1, SeguridadContrasena.hash(contraseña));	Configura una propiedad mediante setString en el objeto indicado.
170	migracion.setInt(2, rs.getInt("id_usuario"));	Configura una propiedad mediante setInt en el objeto indicado.
171	migracion.executeUpdate();	Ejecuta un INSERT, UPDATE o DELETE y obtiene cuántas filas fueron afectadas.
172	} catch (SQLException migracionFallida) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
173	// La autenticación ya fue válida; la migración se reintentará después.	Comentario para humanos; Java no ejecuta este texto. La autenticación ya fue válida; la migración se reintentará después.
174	}	Cierra el bloque abierto anteriormente.
175	}	Cierra el bloque abierto anteriormente.
176		Línea en blanco: separa visualmente bloques para que el código sea más legible.
177	int idUsuario = rs.getInt("id_usuario");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
178	String nombre = rs.getString("nombre");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
179	String apellido = rs.getString("apellido");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
180	String rol = rs.getString("rol");	Lee una columna de la fila actual del ResultSet y la convierte al tipo Java solicitado.
181		Línea en blanco: separa visualmente bloques para que el código sea más legible.
182	// Guarda la identidad mínima que necesitan las pantallas.	Comentario para humanos; Java no ejecuta este texto. Guarda la identidad mínima que necesitan las pantallas.
183	SesionUsuario.iniciar(idUsuario, nombre, apellido, rol);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
184		Línea en blanco: separa visualmente bloques para que el código sea más legible.
185	System.out.println("Sesión iniciada");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
186	System.out.println("ID: " + idUsuario);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	System.out.println("Nombre: " + nombre + " " + apellido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188	System.out.println("Rol: " + rol);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
189		Línea en blanco: separa visualmente bloques para que el código sea más legible.
190	if (rol.equalsIgnoreCase("ADMINISTRADOR")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
191		Línea en blanco: separa visualmente bloques para que el código sea más legible.
192	InicioAdminForm ventanaAdministrador	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	= new InicioAdminForm();	Crea una nueva instancia de InicioAdminForm y la usa o guarda para continuar el proceso.
194		Línea en blanco: separa visualmente bloques para que el código sea más legible.
195	ventanaAdministrador.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.

196	ventanaAdministrador.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
198	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
199		Línea en blanco: separa visualmente bloques para que el código sea más legible.
200	} else if (rol.equalsIgnoreCase("CAJERO")) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
201		Línea en blanco: separa visualmente bloques para que el código sea más legible.
202	Cajero ventanaCajero = new Cajero();	Crea una nueva instancia de Cajero y la usa o guarda para continuar el proceso.
203		Línea en blanco: separa visualmente bloques para que el código sea más legible.
204	ventanaCajero.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
205	ventanaCajero.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
206		Línea en blanco: separa visualmente bloques para que el código sea más legible.
207	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
208	}	Cierra el bloque abierto anteriormente.
209		Línea en blanco: separa visualmente bloques para que el código sea más legible.
210	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	"Usuario o contraseña incorrectos.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	"Inicio de sesión",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216	JOptionPane.ERROR_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
217	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
218		Línea en blanco: separa visualmente bloques para que el código sea más legible.
219	textbox_Contrasena1.setText("");	Cambia el texto visible de un componente de la interfaz.
220	textbox_Contrasena1.requestFocus();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
221	}	Cierra el bloque abierto anteriormente.
222	}	Cierra el bloque abierto anteriormente.
223		Línea en blanco: separa visualmente bloques para que el código sea más legible.
224	} catch (SQLException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
225		Línea en blanco: separa visualmente bloques para que el código sea más legible.
226	JOptionPane.showMessageDialog(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	this,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	"Error al iniciar sesión:\n" + e.getMessage(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
229	"Error",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
230	JOptionPane.ERROR_MESSAGE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
231	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
232		Línea en blanco: separa visualmente bloques para que el código sea más legible.
233	e.printStackTrace();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
234	}	Cierra el bloque abierto anteriormente.
235	}	Cierra el bloque abierto anteriormente.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.
237	private void agregarKeyListenerRecursivo(java.awt.Component componente, java.awt.event.KeyListener listener)	Declara el método agregarKeyListenerRecursivo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
238	{	
238	componente.addKeyListener(listener);	Registra un listener: el código asociado se ejecutará cuando el usuario produzca ese evento.
239	if (componente instanceof java.awt.Container container) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

240	for (java.awt.Component c : container.getComponents()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
241	agregarKeyListenerRecursivo(c, listener);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
242	}	Cierra el bloque abierto anteriormente.
243	}	Cierra el bloque abierto anteriormente.
244	}	Cierra el bloque abierto anteriormente.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
247	* This method is called from within the constructor to initialize the form.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
248	* WARNING: Do NOT modify this code. The content of this method is always	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
249	* regenerated by the Form Editor.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
250	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
251	@SuppressWarnings("unchecked")	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
252	// <editor-fold defaultstate="collapsed" desc="Generated Code"> // GEN-BEGIN: initComponents	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
253	private void initComponents() {	Declara el método initComponents. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255	jPanel1 = new javax.swing.JPanel();	Crea una nueva instancia de javax.swing.JPanel y la usa o guarda para continuar el proceso.
256	jLabel1 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
257	botonDerretido1 = new Componentes.BotonDerretido();	Crea una nueva instancia de Componentes.BotonDerretido y la usa o guarda para continuar el proceso.
258	labelEscalable3 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
259	jLabel2 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
260	jLabel3 = new javax.swing.JLabel();	Crea una nueva instancia de javax.swing.JLabel y la usa o guarda para continuar el proceso.
261	jButton2 = new javax.swing.JButton();	Crea una nueva instancia de javax.swing.JButton y la usa o guarda para continuar el proceso.
262	labelEscalable2 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
263	labelEscalable1 = new Labels.LabelEscalable();	Crea una nueva instancia de Labels.LabelEscalable y la usa o guarda para continuar el proceso.
264	textBox_Login2 = new Labels.TextBox_Login();	Crea una nueva instancia de Labels.TextBox_Login y la usa o guarda para continuar el proceso.
265	textBox_Contrasena1 = new Labels.Textbox_Contrasena();	Crea una nueva instancia de Labels.Textbox_Contrasena y la usa o guarda para continuar el proceso.
266		Línea en blanco: separa visualmente bloques para que el código sea más legible.
267	setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
268	getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
269		Línea en blanco: separa visualmente bloques para que el código sea más legible.
270	jPanel1.setBackground(new java.awt.Color(255, 255, 255));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
271	jPanel1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteLayout y la usa o guarda para continuar el proceso.
272		Línea en blanco: separa visualmente bloques para que el código sea más legible.
273	jLabel1.setFont(new java.awt.Font("Dialog", 1, 24)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
274	jLabel1.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
275	jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
276	jLabel1.setText("Contraseña");	Cambia el texto visible de un componente de la interfaz.
277	jPanel1.add(jLabel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1080, 520, 240, 40));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
278		Línea en blanco: separa visualmente bloques para que el código sea más legible.
279	botonDerretido1.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
280	botonDerretido1.setFont(new java.awt.Font("Arial", 1, 18)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
281	botonDerretido1.setLabel("Iniciar Sesión");	Configura una propiedad mediante setLabel en el objeto indicado.
282	botonDerretido1.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
283	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

284	botonDerretido1ActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
285	}	Cierra el bloque abierto anteriormente.
286	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
287	jPanel1.add(botonDerretido1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1070, 660, 440, 80));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
288		Línea en blanco: separa visualmente bloques para que el código sea más legible.
289	labelEscalable3.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/Panel2.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
290	labelEscalable3.setText("labelEscalable3");	Cambia el texto visible de un componente de la interfaz.
291	jPanel1.add(labelEscalable3, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 0, 670, 1080));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	jLabel2.setFont(new java.awt.Font("Dialog", 1, 24)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
294	jLabel2.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
295	jLabel2.setHorizontalAlignment(javax.swing.SwingConstants.LEFT);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
296	jLabel2.setText("Usuario");	Cambia el texto visible de un componente de la interfaz.
297	jPanel1.add(jLabel2, new org.netbeans.lib.awtextra.AbsoluteConstraints(1080, 372, 190, 40));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
298		Línea en blanco: separa visualmente bloques para que el código sea más legible.
299	jLabel3.setFont(new java.awt.Font("Dialog", 1, 48)); // NOI18N	Crea una nueva instancia de java.awt.Font y la usa o guarda para continuar el proceso.
300	jLabel3.setForeground(new java.awt.Color(0, 0, 0));	Crea una nueva instancia de java.awt.Color y la usa o guarda para continuar el proceso.
301	jLabel3.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);	Configura una propiedad mediante setHorizontalAlignment en el objeto indicado.
302	jLabel3.setText(";Bienvenido a Waldonald's!");	Cambia el texto visible de un componente de la interfaz.
303	jPanel1.add(jLabel3, new org.netbeans.lib.awtextra.AbsoluteConstraints(910, 260, 730, 60));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
304		Línea en blanco: separa visualmente bloques para que el código sea más legible.
305	jButton2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/visualizar.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
306	jButton2.setBorderPainted(false);	Configura una propiedad mediante setBorderPainted en el objeto indicado.
307	jButton2.setContentAreaFilled(false);	Configura una propiedad mediante setContentAreaFilled en el objeto indicado.
308	jButton2.setCursor(new java.awt.Cursor(java.awt.Cursor.HAND_CURSOR));	Crea una nueva instancia de java.awt.Cursor y la usa o guarda para continuar el proceso.
309	jButton2.setFocusPainted(false);	Configura una propiedad mediante setFocusPainted en el objeto indicado.
310	jButton2.setMaximumSize(new java.awt.Dimension(40, 40));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
311	jButton2.setMinimumSize(new java.awt.Dimension(40, 40));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
312	jButton2.setPreferredSize(new java.awt.Dimension(40, 40));	Crea una nueva instancia de java.awt.Dimension y la usa o guarda para continuar el proceso.
313	jButton2.addActionListener(new java.awt.event.ActionListener() {	Crea una nueva instancia de java.awt.event.ActionListener y la usa o guarda para continuar el proceso.
314	public void actionPerformed(java.awt.event.ActionEvent evt) {	Declara el método actionPerformed. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
315	jButton2ActionPerformed(evt);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
316	}	Cierra el bloque abierto anteriormente.
317	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
318	jPanel1.add(jButton2, new org.netbeans.lib.awtextra.AbsoluteConstraints(1520, 570, 50, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
319		Línea en blanco: separa visualmente bloques para que el código sea más legible.
320	labelEscalable2.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/MiniLogo 2.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
321	labelEscalable2.setText("labelEscalable2");	Cambia el texto visible de un componente de la interfaz.
322	jPanel1.add(labelEscalable2, new org.netbeans.lib.awtextra.AbsoluteConstraints(1210, 80, 140, 180));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
323		Línea en blanco: separa visualmente bloques para que el código sea más legible.
324	labelEscalable1.setIcon(new javax.swing.ImageIcon(getClass().getResource("/Imágenes/Usuarios.png"))); // NOI18N	Crea una nueva instancia de javax.swing.ImageIcon y la usa o guarda para continuar el proceso.
325	labelEscalable1.setText("labelEscalable1");	Cambia el texto visible de un componente de la interfaz.

326	jPanel1.add(labelEscalable1, new org.netbeans.lib.awtextra.AbsoluteConstraints(1010, 420, 50, 50));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
327		Línea en blanco: separa visualmente bloques para que el código sea más legible.
328	textBox_Login2.setPlaceholder("Ingresa tu usuario");	Configura una propiedad mediante setPlaceholder en el objeto indicado.
329	jPanel1.add(textBox_Login2, new org.netbeans.lib.awtextra.AbsoluteConstraints(990, 400, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
330	jPanel1.add(textbox_Contrasena1, new org.netbeans.lib.awtextra.AbsoluteConstraints(990, 550, -1, -1));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
331		Línea en blanco: separa visualmente bloques para que el código sea más legible.
332	getContentPane().add(jPanel1, new org.netbeans.lib.awtextra.AbsoluteConstraints(-20, 0, 1940, 1090));	Crea una nueva instancia de org.netbeans.lib.awtextra.AbsoluteConstraints y la usa o guarda para continuar el proceso.
333		Línea en blanco: separa visualmente bloques para que el código sea más legible.
334	pack();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
335	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
336	}// </editor-fold>//GEN-END: initComponents	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	private void botonDerretido1ActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST: event_botonDerretido1ActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
339	iniciarSesion();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
340	} //GEN-LAST: event_botonDerretido1ActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
341		Línea en blanco: separa visualmente bloques para que el código sea más legible.
342	private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST: event_jButton2ActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
343	contraseñaVisible = !contraseñaVisible;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
344		Línea en blanco: separa visualmente bloques para que el código sea más legible.
345	textBox_Contrasena1.mostrarPassword(contraseñaVisible);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
346	} //GEN-LAST: event_jButton2ActionPerformed	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
349	* @param args the command line arguments	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
350	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
351	public static void main(String args[]) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
352	/* Set the Nimbus look and feel */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
353	//<editor-fold defaultstate="collapsed" desc="Look and feel setting code (optional) ">	Comentario para humanos; Java no ejecuta este texto. <editor-fold defaultstate="collapsed" desc="Look and feel setting code (optional) ">
354	/* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
355	* For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
356	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
357	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
358	for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels())	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
359	{	
359	if ("Nimbus".equals(info.getName())) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
360	javax.swing.UIManager.setLookAndFeel(info.getClassName());	Configura una propiedad mediante setLookAndFeel en el objeto indicado.
361	break;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
362	}	Cierra el bloque abierto anteriormente.
363	}	Cierra el bloque abierto anteriormente.
364	} catch (ClassNotFoundException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
365	java.util.logging.Logger.getLogger(Login.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
366	} catch (InstantiationException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.

367	java.util.logging.Logger.getLogger(Login.class.getName()).log(java.util.logging.Level.SEVERE, null,	Ejecuta una instrucción Java completa; el punto y coma marca su final.
368	ex);	
368	} catch (IllegalAccessException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
369	java.util.logging.Logger.getLogger(Login.class.getName()).log(java.util.logging.Level.SEVERE, null,	Ejecuta una instrucción Java completa; el punto y coma marca su final.
369	ex);	
370	} catch (javax.swing.UnsupportedLookAndFeelException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
371	java.util.logging.Logger.getLogger(Login.class.getName()).log(java.util.logging.Level.SEVERE, null,	Ejecuta una instrucción Java completa; el punto y coma marca su final.
371	ex);	
372	}	Cierra el bloque abierto anteriormente.
373	//</editor-fold>	Comentario para humanos; Java no ejecuta este texto. </editor-fold>
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.
375	/* Create and display the form */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
376	java.awt.EventQueue.invokeLater(new Runnable() {	Crea una nueva instancia de Runnable y la usa o guarda para continuar el proceso.
377	public void run() {	Declara el método run. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
378	new Login().setVisible(true);	Crea una nueva instancia de Login y la usa o guarda para continuar el proceso.
379	}	Cierra el bloque abierto anteriormente.
380	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
381	}	Cierra el bloque abierto anteriormente.
382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
383	// Variables declaration - do not modify//GEN-BEGIN:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
384	private Componentes.BotonDerretido botonDerretido1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
385	private javax.swing.JButton jButton2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
386	private javax.swing.JLabel jLabel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
387	private javax.swing.JLabel jLabel2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
388	private javax.swing.JLabel jLabel3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
389	private javax.swing.JPanel jPanel1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
390	private Labels.LabelEscalable labelEscalable1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
391	private Labels.LabelEscalable labelEscalable2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
392	private Labels.LabelEscalable labelEscalable3;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
393	private Labels.TextBox_Login textBox_Login2;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
394	private Labels.TextBox_Contrasena textbox_Contrasena1;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
395	// End of variables declaration//GEN-END:variables	Marca protegida del GUI Builder de NetBeans. No debe borrarse porque delimita código administrado por el diseñador visual.
396	}	Cierra el bloque abierto anteriormente.

16.113 src/Main/main.java

Punto de entrada. Inicia la interfaz en el hilo de Swing, muestra la pantalla de carga y luego abre el inicio de sesión.

Tipo: java | Líneas: 33 | Archivo 113 de 157

Clases/tipos detectados: main

Línea	Constructor o método detectado
7	public static void main(String[] args) {
10	public void run() {

1	package Main;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	import Vistas.PantallaCarga;	Importa Vistas.PantallaCarga para poder usar esa clase sin escribir su nombre completo.
3	import Login.Login;	Importa Login.Login para poder usar esa clase sin escribir su nombre completo.
4	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
5	public class main {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	public static void main(String[] args) {	Constructor de main: prepara un objeto nuevo y deja sus componentes listos para usarse.
8	java.awt.EventQueue.invokeLater(new Runnable() {	Crea una nueva instancia de Runnable y la usa o guarda para continuar el proceso.
9	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
10	public void run() {	Declara el método run. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	PantallaCarga carga = new PantallaCarga();	Crea una nueva instancia de PantallaCarga y la usa o guarda para continuar el proceso.
13	carga.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
14	carga.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	// Esperar 3 segundos	Comentario para humanos; Java no ejecuta este texto. Esperar 3 segundos
17	Timer timer = new Timer(8000, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	// Cerrar pantalla de carga	Comentario para humanos; Java no ejecuta este texto. Cerrar pantalla de carga
20	carga.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	// Abrir siguiente interfaz	Comentario para humanos; Java no ejecuta este texto. Abrir siguiente interfaz
23	Login login = new Login();	Crea una nueva instancia de Login y la usa o guarda para continuar el proceso.
24	login.setLocationRelativeTo(null);	Configura una propiedad mediante setLocationRelativeTo en el objeto indicado.
25	login.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
26	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	timer.setRepeats(false);	Configura una propiedad mediante setRepeats en el objeto indicado.
29	timer.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	}	Cierra el bloque abierto anteriormente.
31	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	}	Cierra el bloque abierto anteriormente.
33	}	Cierra el bloque abierto anteriormente.

16.114 src/Modelos/ArticuloInventario.java

Modelo de datos ArticuloInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 70 | Archivo 114 de 157

Clases/tipos detectados: ArticuloInventario

Línea	Constructor o método detectado
19	public ArticuloInventario(String tipo, int id, String nombre,
34	public String getTipo() { return tipo; }
35	public int getId() { return id; }
36	public String getNombre() { return nombre; }
37	public String getCategoria() { return categoria; }
38	public String getUnidad() { return unidad; }
39	public BigDecimal getStockActual() { return stockActual; }
40	public BigDecimal getStockMinimo() { return stockMinimo; }
41	public boolean isActive() { return activo; }
42	public LocalDateTime getUltimoMovimiento() { return ultimoMovimiento; }
44	public boolean esProducto() {
48	public String getTipoVisible() {
52	public String getCodigoVisible() {
56	public String getEstadoVisible() {
67	public String toString() {

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4 import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa fecha y hora juntas.
5	Línea en blanco: separa visualmente bloques para que el código sea más legible.
6 /** Producto directo o ingrediente que sí posee existencias controlables. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
7 public class ArticuloInventario {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
8	Línea en blanco: separa visualmente bloques para que el código sea más legible.
9 private final String tipo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
10 private final int id;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
11 private final String nombre;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
12 private final String categoria;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13 private final String unidad;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
14 private final BigDecimal stockActual;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15 private final BigDecimal stockMinimo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
16 private final boolean activo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.

17	private final LocalDateTime ultimoMovimiento;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	public ArticuloInventario(String tipo, int id, String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	String categoria, String unidad, BigDecimal stockActual,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21	BigDecimal stockMinimo, boolean activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	LocalDateTime ultimoMovimiento) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
23	this.tipo = tipo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
24	this.id = id;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
25	this.nombre = nombre;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
26	this.categoria = categoria;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
27	this.unidad = unidad;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
28	this.stockActual = stockActual;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
29	this.stockMinimo = stockMinimo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
30	this.activo = activo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
31	this.ultimoMovimiento = ultimoMovimiento;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
32	}	Cierra el bloque abierto anteriormente.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	public String getTipo() { return tipo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	public int getId() { return id; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	public String getNombre() { return nombre; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	public String getCategoria() { return categoria; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	public String getUnidad() { return unidad; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	public BigDecimal getStockActual() { return stockActual; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	public BigDecimal getStockMinimo() { return stockMinimo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	public boolean isActive() { return activo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	public LocalDateTime getUltimoMovimiento() { return ultimoMovimiento; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	public boolean esProducto() {	Declara el método esProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
45	return "PRODUCTO".equals(tipo);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
46	}	Cierra el bloque abierto anteriormente.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	public String getTipoVisible() {	Declara el método getTipoVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
49	return esProducto() ? "Producto directo" : "Ingrediente";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
50	}	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	public String getCodigoVisible() {	Declara el método getCodigoVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	return (esProducto() ? "P-" : "I-") + String.format("%04d", id);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	public String getEstadoVisible() {	Declara el método getEstadoVisible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
57	if (!activo) return "Inactivo";	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
58	if (stockActual.signum() <= 0) return "Agotado";	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
59	if (stockMinimo.signum() > 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
60	&& stockActual.compareTo(stockMinimo) <= 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

61	return "Stock bajo";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
62	}	Cierra el bloque abierto anteriormente.
63	return "Disponible";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
64	}	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
67	public String toString() {	Declara el método toString. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
68	return getCodigoVisible() + " . " + nombre;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
69	}	Cierra el bloque abierto anteriormente.
70	}	Cierra el bloque abierto anteriormente.

16.115 src/Modelos/ComponenteMenu.java

Modelo de datos ComponenteMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 11 | Archivo 115 de 157

Clases/tipos detectados: ComponenteMenu

Línea	Constructor o método detectado
5	public record ComponenteMenu(int idProducto, String nombre, int cantidad,

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record ComponenteMenu(int idProducto, String nombre, int cantidad,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	String tipoStock, boolean personalizable,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	String disponibilidadMenu, List<IngredienteProducto> ingredientes) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	public ComponenteMenu {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	ingredientes = List.copyOf(ingredientes == null ? List.of() : ingredientes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
10	}	Cierra el bloque abierto anteriormente.
11	}	Cierra el bloque abierto anteriormente.

16.116 src/Modelos/ConfiguracionProducto.java

Modelo de datos ConfiguracionProducto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 11 | Archivo 116 de 157

Clases/tipos detectados: ConfiguracionProducto

Línea	Constructor o método detectado
5	public record ConfiguracionProducto(int idProducto, String nombre,

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record ConfiguracionProducto(int idProducto, String nombre,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	String descripcion, String imagen, String disponibilidadMenu,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	List<PresentacionMenu> presentaciones) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	public ConfiguracionProducto {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	presentaciones = List.copyOf(presentaciones == null ? List.of() : presentaciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
10	}	Cierra el bloque abierto anteriormente.
11	}	Cierra el bloque abierto anteriormente.

16.117 src/Modelos/GrupoMenu.java

Modelo de datos GrupoMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 12 | Archivo 117 de 157

Clases/tipos detectados: GrupoMenu

Línea	Constructor o método detectado	
5	public record GrupoMenu(int idGrupo, String nombre, int minimo, int maximo,	
1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record GrupoMenu(int idGrupo, String nombre, int minimo, int maximo,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	boolean permiteRepetir, boolean visible, boolean permitePersonalizar,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	List<OpcionMenu> opciones) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	public GrupoMenu {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	opciones = List.copyOf(opciones == null ? List.of() : opciones);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
10	}	Cierra el bloque abierto anteriormente.
11	@Override public String toString() { return nombre; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
12	}	Cierra el bloque abierto anteriormente.

16.118 src/Modelos/Ingrediente.java

Modelo de datos Ingrediente. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 33 | Archivo 118 de 157

Clases/tipos detectados: Ingrediente

Línea	Constructor o método detectado
13	public int getIdIngrediente() { return idIngrediente; }
14	public void setIdIngrediente(int idIngrediente) { this.idIngrediente = idIngrediente; }
15	public String getNombre() { return nombre; }
16	public void setNombre(String nombre) { this.nombre = nombre; }
17	public String getUnidadMedida() { return unidadMedida; }
18	public void setUnidadMedida(String unidadMedida) { this.unidadMedida = unidadMedida; }
19	public BigDecimal getStockActual() { return stockActual; }
20	public void setStockActual(BigDecimal stockActual) { this.stockActual = stockActual; }
21	public BigDecimal getStockMinimo() { return stockMinimo; }
22	public void setStockMinimo(BigDecimal stockMinimo) { this.stockMinimo = stockMinimo; }
23	public boolean isActive() { return activo; }
24	public void setActive(boolean activo) { this.activo = activo; }
26	public boolean tieneStockBajo() {

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4	Línea en blanco: separa visualmente bloques para que el código sea más legible.
5 public class Ingrediente {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6 private int idIngrediente;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
7 private String nombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
8 private String unidadMedida;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
9 private BigDecimal stockActual = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
10 private BigDecimal stockMinimo = BigDecimal.ZERO;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
11 private boolean activo = true;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
12	Línea en blanco: separa visualmente bloques para que el código sea más legible.
13 public int getIdIngrediente() { return idIngrediente; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14 public void setIdIngrediente(int idIngrediente) { this.idIngrediente = idIngrediente; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
15 public String getNombre() { return nombre; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16 public void setNombre(String nombre) { this.nombre = nombre; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
17 public String getUnidadMedida() { return unidadMedida; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18 public void setUnidadMedida(String unidadMedida) { this.unidadMedida = unidadMedida; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
19 public BigDecimal getStockActual() { return stockActual; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

20	public void setStockActual(BigDecimal stockActual) { this.stockActual = stockActual; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
21	public BigDecimal getStockMinimo() { return stockMinimo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22	public void setStockMinimo(BigDecimal stockMinimo) { this.stockMinimo = stockMinimo; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
23	public boolean isActive() { return activo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	public void setActive(boolean activo) { this.activo = activo; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	public boolean tieneStockBajo() {	Declara el método tieneStockBajo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
27	return stockMinimo != null && stockActual != null	Finaliza el método y devuelve el valor indicado a quien lo llamó.
28	&& stockMinimo.signum() > 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	&& stockActual.compareTo(stockMinimo) <= 0;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	}	Cierra el bloque abierto anteriormente.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	@Override public String toString() { return nombre; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
33	}	Cierra el bloque abierto anteriormente.

16.119 src/Modelos/IngredienteProducto.java

Modelo de datos IngredienteProducto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 14 | Archivo 119 de 157

Clases/tipos detectados: IngredienteProducto

Línea	Constructor o método detectado
5	public record IngredienteProducto(int idProductoIngrediente, int idIngrediente,

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record IngredienteProducto(int idProductoIngrediente, int idIngrediente,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	String nombre, BigDecimal cantidadDefault, boolean permiteQuitar,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	boolean permiteExtra, BigDecimal cantidadExtra, BigDecimal precioExtra,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	int maxExtras) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	public IngredienteProducto {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10	cantidadDefault = cantidadDefault == null ? BigDecimal.ZERO : cantidadDefault;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
11	cantidadExtra = cantidadExtra == null ? BigDecimal.ONE : cantidadExtra;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
12	precioExtra = precioExtra == null ? BigDecimal.ZERO : precioExtra;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
13	}	Cierra el bloque abierto anteriormente.
14	}	Cierra el bloque abierto anteriormente.

16.120 src/Modelos/LineaPedido.java

Modelo de datos LineaPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 51 | Archivo 120 de 157

Clases/tipos detectados: LineaPedido

Línea	Constructor o método detectado
8	public record LineaPedido(String idLinea, int idProducto, int idPresentacion,
22	public BigDecimal precio() {
34	public BigDecimal subtotal() { return precio().multiply(BigDecimal.valueOf(cantidad)); }
36	public LineaPedido conCantidad(int nuevaCantidad) {
41	public String resumen() {

1 package Modelos;		Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;		Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4 import java.util.List;		Importa java.util.List: colección ordenada que admite varios elementos.
5 import java.util.UUID;		Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7 /** Una línea del carrito. Dos configuraciones distintas nunca se mezclan. */		Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8 public record LineaPedido(String idLinea, int idProducto, int idPresentacion,		Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
9 String nombre, String presentacion, BigDecimal precioBase,		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10 List<OpcionPedido> opciones, int cantidad) {		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
11 public LineaPedido {		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
12 if (idLinea == null idLinea.isBlank()) idLinea = UUID.randomUUID().toString();		Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
13 if (idProducto <= 0 idPresentacion <= 0 nombre == null		Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
14 presentacion == null precioBase == null		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15 precioBase.signum() < 0 precioBase.scale() > 2		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16 cantidad < 1 cantidad > 999) {		Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17 throw new IllegalArgumentException("Producto, presentación, precio o cantidad inválidos.");		Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
18 }		Cierra el bloque abierto anteriormente.
19 opciones = List.copyOf(opciones == null ? List.of() : opciones);		Ejecuta una instrucción Java completa; el punto y coma marca su final.
20 }		Cierra el bloque abierto anteriormente.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22 public BigDecimal precio() {		Declara el método precio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
23 BigDecimal total = precioBase;		Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24 for (OpcionPedido opcion : opciones) {		Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
25 total = total.add(opcion.precioExtra());		Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
26 for (ProductoPedido producto : opcion.productos()) {		Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
27 for (ModificacionPedido modificacion : producto.modificaciones())		Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
28 total = total.add(modificacion.precioExtra());		Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
29 }		Cierra el bloque abierto anteriormente.

30	}	Cierra el bloque abierto anteriormente.
31	return total.setScale(2);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
32	}	Cierra el bloque abierto anteriormente.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	public BigDecimal subtotal() { return precio().multiply(BigDecimal.valueOf(cantidad)); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	public LineaPedido conCantidad(int nuevaCantidad) {	Declara el método conCantidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
37	return new LineaPedido(idLinea, idProducto, idPresentacion, nombre,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
38	presentacion, precioBase, opciones, nuevaCantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	}	Cierra el bloque abierto anteriormente.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	public String resumen() {	Declara el método resumen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	StringBuilder texto = new StringBuilder(nombre).append(" · ").append(presentacion);	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
43	opciones.stream().filter(o -> !o.grupo().startsWith("__"))	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
44	.forEach(o -> texto.append(" ").append(o.opcion()));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	long cambios = opciones.stream().flatMap(o -> o.productos().stream())	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
46	.mapToLong(p -> p.modificaciones().size()).sum();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	if (cambios > 0) texto.append(" ").append(cambios)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
48	.append(cambios == 1 ? " cambio" : " cambios");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	return texto.toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
50	}	Cierra el bloque abierto anteriormente.
51	}	Cierra el bloque abierto anteriormente.

16.121 src/Modelos/ModificacionPedido.java

Modelo de datos ModificacionPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 15 | Archivo 121 de 157

Clases/tipos detectados: ModificacionPedido

Línea	Constructor o método detectado
5	public record ModificacionPedido(int idProductoIngrediente, int idIngrediente,

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4	Línea en blanco: separa visualmente bloques para que el código sea más legible.
5 public record ModificacionPedido(int idProductoIngrediente, int idIngrediente,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6 String tipo, String ingrediente, BigDecimal cantidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7 int veces, BigDecimal precioExtra) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8 public ModificacionPedido {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 if (!"SIN".equals(tipo) && !"EXTRA".equals(tipo))	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
10 throw new IllegalArgumentException("Modificación inválida.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
11 cantidad = cantidad == null ? BigDecimal.ZERO : cantidad;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
12 precioExtra = precioExtra == null ? BigDecimal.ZERO : precioExtra;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
13 if (veces < 1) veces = 1;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
14 }	Cierra el bloque abierto anteriormente.
15 }	Cierra el bloque abierto anteriormente.

16.122 src/Modelos/MovimientoInventario.java

Modelo de datos MovimientoInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 15 | Archivo 122 de 157

Clases/tipos detectados: MovimientoInventario

Línea	Constructor o método detectado
6	public record MovimientoInventario(

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4	import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa fecha y hora juntas.
5		Línea en blanco: separa visualmente bloques para que el código sea más legible.
6	public record MovimientoInventario(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7	int id,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	LocalDateTime fecha,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	String tipoArticulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10	String articulo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
11	String tipoMovimiento,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
12	BigDecimal cantidad,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
13	String motivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14	Integer idPedido) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15	}	Cierra el bloque abierto anteriormente.

16.123 src/Modelos/OpcionMenu.java

Modelo de datos OpcionMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 16 | Archivo 123 de 157

Clases/tipos detectados: OpcionMenu

Línea	Constructor o método detectado
6	public record OpcionMenu(int idOpcion, String nombre, BigDecimal incrementoPrecio,

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4 import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
5	Línea en blanco: separa visualmente bloques para que el código sea más legible.
6 public record OpcionMenu(int idOpcion, String nombre, BigDecimal incrementoPrecio,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7 boolean determinada, List<ComponenteMenu> componentes) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8 public OpcionMenu {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 incrementoPrecio = incrementoPrecio == null ? BigDecimal.ZERO : incrementoPrecio;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
10 componentes = List.copyOf(componentes == null ? List.of() : componentes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
11 }	Cierra el bloque abierto anteriormente.
12 @Override public String toString() {	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
13 return nombre + (incrementoPrecio.signum() > 0	Finaliza el método y devuelve el valor indicado a quien lo llamó.
14 ? " + Q" + incrementoPrecio.setScale(2).toString() : "";	Configura una propiedad mediante setScale en el objeto indicado.
15 }	Cierra el bloque abierto anteriormente.
16 }	Cierra el bloque abierto anteriormente.

16.124 src/Modelos/OpcionPedido.java

Modelo de datos OpcionPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 16 | Archivo 124 de 157

Clases/tipos detectados: OpcionPedido

Línea	Constructor o método detectado
6	public record OpcionPedido(int idGrupo, int idOpcion, int numero,
13	public OpcionPedido conProductos(List<ProductoPedido> nuevos) {

1 package Modelos;			Declara el paquete o carpeta lógica a la que pertenece esta clase.
2			Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;			Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4 import java.util.List;			Importa java.util.List: colección ordenada que admite varios elementos.
5			Línea en blanco: separa visualmente bloques para que el código sea más legible.
6 public record OpcionPedido(int idGrupo, int idOpcion, int numero,			Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7 String grupo, String opcion, BigDecimal precioExtra,			Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8 List<ProductoPedido> productos) {			Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 public OpcionPedido {			Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10 precioExtra = precioExtra == null ? BigDecimal.ZERO : precioExtra;			Ejecuta una instrucción Java completa; el punto y coma marca su final.
11 productos = List.copyOf(productos == null ? List.of() : productos);			Ejecuta una instrucción Java completa; el punto y coma marca su final.
12 }			Cierra el bloque abierto anteriormente.
13 public OpcionPedido conProductos(List<ProductoPedido> nuevos) {			Declara el método conProductos. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
14 return new OpcionPedido(idGrupo, idOpcion, numero, grupo, opcion, precioExtra, nuevos);			Finaliza el método y devuelve el valor indicado a quien lo llamó.
15 }			Cierra el bloque abierto anteriormente.
16 }			Cierra el bloque abierto anteriormente.

16.125 src/Modelos/PaginaIngredientes.java

Modelo de datos PaginaIngredientes. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 6 | Archivo 125 de 157

Clases/tipos detectados: PaginaIngredientes

Línea	Constructor o método detectado	
5	public record PaginaIngredientes(List<Ingrediente> ingredientes, int totalRegistros) {	
1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record PaginaIngredientes(List<Ingrediente> ingredientes, int totalRegistros) {	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	}	Cierra el bloque abierto anteriormente.

16.126 src/Modelos/PaginalInventario.java

Modelo de datos PaginalInventario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 8 | Archivo 126 de 157

Clases/tipos detectados: PaginalInventario

Línea	Constructor o método detectado
5	public record PaginalInventario(

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record PaginaInventario(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	List<ArticuloInventario> articulos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	int totalRegistros) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	}	Cierra el bloque abierto anteriormente.

16.127 src/Modelos/PaginaProductos.java

Modelo de datos PaginaProductos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 9 | Archivo 127 de 157

Clases/tipos detectados: PaginaProductos

Línea	Constructor o método detectado
5	public record PaginaProductos(

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	public record PaginaProductos(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
6	List<Producto> productos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7	int totalRegistros	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	}	Cierra el bloque abierto anteriormente.

16.128 src/Modelos/PaginaUsuarios.java

Modelo de datos PaginaUsuarios. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 11 | Archivo 128 de 157

Clases/tipos detectados: PaginaUsuarios

Línea	Constructor o método detectado	
6	public record PaginaUsuarios(List<Usuario> usuarios, int totalRegistros) {	
1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	/** Resultado de una página y cantidad total que coincide con los filtros. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
6	public record PaginaUsuarios(List<Usuario> usuarios, int totalRegistros) {	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	public PaginaUsuarios {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	usuarios = List.copyOf(usuarios);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
10	}	Cierra el bloque abierto anteriormente.
11	}	Cierra el bloque abierto anteriormente.

16.129 src/Modelos/PresentacionMenu.java

Modelo de datos PresentacionMenu. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 14 | Archivo 129 de 157

Clases/tipos detectados: PresentacionMenu

Línea	Constructor o método detectado
6	public record PresentacionMenu(int idPresentacion, int idProductoPrincipal,

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4 import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
5	Línea en blanco: separa visualmente bloques para que el código sea más legible.
6 public record PresentacionMenu(int idPresentacion, int idProductoPrincipal,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7 String nombre, String tipo, BigDecimal precio, boolean predeterminada,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8 List<GrupoMenu> grupos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 public PresentacionMenu {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10 precio = precio == null ? BigDecimal.ZERO : precio;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
11 grupos = List.copyOf(grupos == null ? List.of() : grupos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
12 }	Cierra el bloque abierto anteriormente.
13 @Override public String toString() { return nombre; }	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
14 }	Cierra el bloque abierto anteriormente.

16.130 src/Modelos/Producto.java

Modelo de datos Producto. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 88 | Archivo 130 de 157

Clases/tipos detectados: Producto

Línea	Constructor o método detectado
22	public Producto() {
25	public int getIdProducto() { return idProducto; }
26	public void setIdProducto(int idProducto) { this.idProducto = idProducto; }
28	public int getIdCategoria() { return idCategoria; }
29	public void setIdCategoria(int idCategoria) { this.idCategoria = idCategoria; }
31	public String getCategoria() { return categoria; }
32	public void setCategoria(String categoria) { this.categoria = categoria; }
34	public String getSubCategoria() { return subCategoria; }
35	public void setSubCategoria(String subCategoria) {
39	public String getNombre() { return nombre; }
40	public void setNombre(String nombre) { this.nombre = nombre; }
42	public String getDescripcion() { return descripcion; }
43	public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
45	public BigDecimal getPrecioBase() { return precioBase; }
46	public void setPrecioBase(BigDecimal precioBase) { this.precioBase = precioBase; }
48	public String getImagen() { return imagen; }
49	public void setImagen(String imagen) { this.imagen = imagen; }
51	public String getDisponibilidadMenu() { return disponibilidadMenu; }
52	public void setDisponibilidadMenu(String disponibilidadMenu) {
56	public String getTipoStock() { return tipoStock; }
57	public void setTipoStock(String tipoStock) { this.tipoStock = tipoStock; }
59	public boolean isPersonalizable() { return personalizable; }
60	public void setPersonalizable(boolean personalizable) {
64	public int getStockActual() { return stockActual; }
65	public void setStockActual(int stockActual) { this.stockActual = stockActual; }
67	public int getStockMinimo() { return stockMinimo; }
68	public void setStockMinimo(int stockMinimo) { this.stockMinimo = stockMinimo; }
70	public boolean isActive() { return activo; }
71	public void setActive(boolean activo) { this.activo = activo; }

Línea	Constructor o método detectado
73	public String getTipo() {
81	public boolean tieneStockBajo() {
87	public String toString() { return nombre; }

```
1 package Modelos;
2
3 import java.math.BigDecimal;
4
5 public class Producto {
6
7     private int idProducto;
8     private int idCategoria;
9     private String categoria;
10    private String subCategoria;
11    private String nombre;
12    private String descripcion;
13    private BigDecimal precioBase = BigDecimal.ZERO;
14    private String imagen;
15    private String disponibilidadMenu = "TODO_DIA";
16    private String tipoStock = "DIRECTO";
17    private boolean personalizable = true;
18    private int stockActual;
19    private int stockMinimo;
20    private boolean activo = true;
21
22    public Producto() {
23    }
24
25    public int getIdProducto() { return idProducto; }
26    public void setIdProducto(int idProducto) { this.idProducto = idProducto; }
27
28    public int getIdCategoria() { return idCategoria; }
29    public void setIdCategoria(int idCategoria) { this.idCategoria = idCategoria; }
30
31    public String getCategoria() { return categoria; }
32    public void setCategoria(String categoria) { this.categoria = categoria; }
33
34    public String getSubCategoria() { return subCategoria; }
35    public void setSubCategoria(String subCategoria) {
36        this.subCategoria = subCategoria;
37    }
```

Declara el paquete o carpeta lógica a la que pertenece esta clase.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Constructor de Producto: prepara un objeto nuevo y deja sus componentes listos para usarse.
Cierra el bloque abierto anteriormente.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
Línea en blanco: separa visualmente bloques para que el código sea más legible.
Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
Declara el método setSubCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
Accede al atributo del objeto actual para asignarlo o utilizarlo.
Cierra el bloque abierto anteriormente.

38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	public String getNombre() { return nombre; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	public void setNombre(String nombre) { this.nombre = nombre; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
41		Línea en blanco: separa visualmente bloques para que el código sea más legible.
42	public String getDescripcion() { return descripcion; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
43	public void setDescripcion(String descripcion) { this.descripcion = descripcion; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	public BigDecimal getPrecioBase() { return precioBase; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	public void setPrecioBase(BigDecimal precioBase) { this.precioBase = precioBase; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	public String getImagen() { return imagen; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	public void setImagen(String imagen) { this.imagen = imagen; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	public String getDisponibilidadMenu() { return disponibilidadMenu; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	public void setDisponibilidadMenu(String disponibilidadMenu) {	Declara el método setDisponibilidadMenu. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	this.disponibilidadMenu = disponibilidadMenu;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
54	}	Cierra el bloque abierto anteriormente.
55		Línea en blanco: separa visualmente bloques para que el código sea más legible.
56	public String getTipoStock() { return tipoStock; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	public void setTipoStock(String tipoStock) { this.tipoStock = tipoStock; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	public boolean isPersonalizable() { return personalizable; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	public void setPersonalizable(boolean personalizable) {	Declara el método setPersonalizable. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
61	this.personalizable = personalizable;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
62	}	Cierra el bloque abierto anteriormente.
63		Línea en blanco: separa visualmente bloques para que el código sea más legible.
64	public int getStockActual() { return stockActual; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	public void setStockActual(int stockActual) { this.stockActual = stockActual; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	public int getStockMinimo() { return stockMinimo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	public void setStockMinimo(int stockMinimo) { this.stockMinimo = stockMinimo; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	public boolean isActive() { return activo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	public void setActive(boolean activo) { this.activo = activo; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.
73	public String getTipo() {	Declara el método getTipo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
74	return switch (tipoStock == null ? "" : tipoStock) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
75	case "RECETA" -> "Preparado por receta";	Define una de las rutas posibles dentro del switch.
76	case "NINGUNO" -> "Sin control de stock";	Define una de las rutas posibles dentro del switch.
77	default -> "Stock directo";	Define una de las rutas posibles dentro del switch.
78	};	Cierra el bloque abierto anteriormente y termina la declaración.
79	}	Cierra el bloque abierto anteriormente.
80		Línea en blanco: separa visualmente bloques para que el código sea más legible.

81	public boolean tieneStockBajo() {	Declara el método tieneStockBajo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
82	return activo && !"NINGUNO".equals(tipoStock)	Finaliza el método y devuelve el valor indicado a quien lo llamó.
83	&& stockMinimo > 0 && stockActual <= stockMinimo;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85		Línea en blanco: separa visualmente bloques para que el código sea más legible.
86	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
87	public String toString() { return nombre; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	}	Cierra el bloque abierto anteriormente.

16.131 src/Modelos/ProductoPedido.java

Modelo de datos ProductoPedido. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 15 | Archivo 131 de 157

Clases/tipos detectados: ProductoPedido

Línea	Constructor o método detectado
6	public record ProductoPedido(String idInterno, int idProducto, String nombre,
12	public ProductoPedido conModificaciones(List<ModificacionPedido> nuevas) {

```
1 package Modelos;
2
3 import java.util.List;
4 import java.util.UUID;
5
6 public record ProductoPedido(String idInterno, int idProducto, String nombre,
7     int cantidad, String tipoStock, List<ModificacionPedido> modificaciones) {
8     public ProductoPedido {
9         if (idInterno == null || idInterno.isBlank()) idInterno = UUID.randomUUID().toString();
10        modificaciones = List.copyOf(modificaciones == null ? List.of() : modificaciones);
11    }
12    public ProductoPedido conModificaciones(List<ModificacionPedido> nuevas) {
13        return new ProductoPedido(idInterno, idProducto, nombre, cantidad, tipoStock, nuevas);
14    }
15 }
```

Declara el paquete o carpeta lógica a la que pertenece esta clase.

Línea en blanco: separa visualmente bloques para que el código sea más legible.

Importa java.util.List: colección ordenada que admite varios elementos.

Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.

Línea en blanco: separa visualmente bloques para que el código sea más legible.

Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.

Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

Ejecuta una instrucción Java completa; el punto y coma marca su final.

Cierra el bloque abierto anteriormente.

Declara el método conModificaciones. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

Finaliza el método y devuelve el valor indicado a quien lo llamó.

Cierra el bloque abierto anteriormente.

Cierra el bloque abierto anteriormente.

16.132 src/Modelos/Productos.java

Modelo de datos Productos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 70 | Archivo 132 de 157

Clases/tipos detectados: Productos

Línea	Constructor o método detectado
13	public Productos(
24	public Productos(
43	public int getIdProducto() {
47	public int getIdCategoria() {
51	public String getNombre() {
55	public String getDescripcion() {
59	public double getPrecio() {
63	public String getImagen() {
67	public boolean isDisponibleHorario() {

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	public class Productos {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	private int idProducto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
6	private int idCategoria;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
7	private String nombre;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
8	private String descripcion;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
9	private double precio;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
10	private String imagen;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
11	private final boolean disponibleHorario;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	public Productos(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14	int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15	int idCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	String descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18	double precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19	String imagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21	this(idProducto, idCategoria, nombre, descripcion, precio, imagen, true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
22	}	Cierra el bloque abierto anteriormente.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	public Productos(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

25	int idProducto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26	int idCategoria,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
28	String descripcion,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
29	double precio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30	String imagen,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	boolean disponibleHorario	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	this.idProducto = idProducto;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
35	this.idCategoria = idCategoria;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
36	this.nombre = nombre;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
37	this.descripcion = descripcion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	this.precio = precio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
39	this.imagen = imagen;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
40	this.disponibleHorario = disponibleHorario;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
41	}	Cierra el bloque abierto anteriormente.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	public int getIdProducto() {	Declara el método getIdProducto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
44	return idProducto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
45	}	Cierra el bloque abierto anteriormente.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	public int getIdCategoria() {	Declara el método getIdCategoria. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
48	return idCategoria;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
49	}	Cierra el bloque abierto anteriormente.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	public String getNombre() {	Declara el método getNombre. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
52	return nombre;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
53	}	Cierra el bloque abierto anteriormente.
54		Línea en blanco: separa visualmente bloques para que el código sea más legible.
55	public String getDescripcion() {	Declara el método getDescripcion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
56	return descripcion;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
57	}	Cierra el bloque abierto anteriormente.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	public double getPrecio() {	Declara el método getPrecio. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
60	return precio;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
61	}	Cierra el bloque abierto anteriormente.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	public String getImagen() {	Declara el método getImagen. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
64	return imagen;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
65	}	Cierra el bloque abierto anteriormente.
66		Línea en blanco: separa visualmente bloques para que el código sea más legible.
67	public boolean isDisponibleHorario() {	Declara el método isDisponibleHorario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
68	return disponibleHorario;	Finaliza el método y devuelve el valor indicado a quien lo llamó.

69	}	Cierra el bloque abierto anteriormente.
70	}	Cierra el bloque abierto anteriormente.

16.133 src/Modelos/ResumenIngredientes.java

Modelo de datos ResumenIngredientes. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 5 | Archivo 133 de 157

Clases/tipos detectados: ResumenIngredientes

Línea	Constructor o método detectado	
3	public record ResumenIngredientes(int total, int stockBajo, int activos, int inactivos) {	
1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	public record ResumenIngredientes(int total, int stockBajo, int activos, int inactivos) {	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4	}	Cierra el bloque abierto anteriormente.
5		Línea en blanco: separa visualmente bloques para que el código sea más legible.

16.134 src/Modelos/ResumenProductos.java

Modelo de datos ResumenProductos. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 9 | Archivo 134 de 157

Clases/tipos detectados: ResumenProductos

Línea	Constructor o método detectado
3	public record ResumenProductos(

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 public record ResumenProductos(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
4 int total,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
5 int stockBajo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
6 int activos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7 int inactivos	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 }	Cierra el bloque abierto anteriormente.

16.135 src/Modelos/ResumenUsuarios.java

Modelo de datos ResumenUsuarios. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 9 | Archivo 135 de 157

Clases/tipos detectados: ResumenUsuarios

Línea	Constructor o método detectado
4	public record ResumenUsuarios(

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 /** Valores mostrados en las cuatro tarjetas superiores. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4 public record ResumenUsuarios(	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
5 int total,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
6 int administradoresActivos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
7 int cajerosActivos,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
8 int inactivos) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9 }	Cierra el bloque abierto anteriormente.

16.136 src/Modelos/SolicitudPago.java

Modelo de datos SolicitudPago. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 37 | Archivo 136 de 157

Clases/tipos detectados: SolicitudPago

Línea	Constructor o método detectado
7	public record SolicitudPago(String clave, int usuario, List<LineaPedido> lineas,
33	public BigDecimal total() {
36	public BigDecimal cambio() { return recibido.subtract(total()); }

1	package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
4	import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
5	import java.util.UUID;	Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	public record SolicitudPago(String clave, int usuario, List<LineaPedido> lineas,	Declara un registro inmutable. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
8	String servicio, String metodo, BigDecimal recibido, String referencia) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
9	public SolicitudPago {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10	UUID.fromString(clave);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
11	lineas = List.copyOf(lineas);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
12	if (usuario <= 0 lineas.isEmpty()) throw new IllegalArgumentException("Inicie sesión y agregue produc	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
13	tos.");	
14	if (!List.of("COMER_AQUI", "PARA_LLEVAR").contains(servicio)	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
15	!List.of("EFECTIVO", "TARJETA").contains(metodo)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	throw new IllegalArgumentException("Servicio o método inválido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
17	}	Cierra el bloque abierto anteriormente.
18	if (lineas.stream().map(LineaPedido::idLinea).distinct().count() != lineas.size()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
19	throw new IllegalArgumentException("Hay líneas repetidas en el pedido.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
20	}	Cierra el bloque abierto anteriormente.
21	BigDecimal total = lineas.stream().map(LineaPedido::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	if (total.signum() <= 0 total.compareTo(new BigDecimal("99999999.99")) > 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
23	recibido == null recibido.scale() > 2 recibido.compareTo(total) < 0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	recibido.compareTo(new BigDecimal("99999999.99")) > 0) {	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
25	throw new IllegalArgumentException("Ingrese un monto válido, con hasta dos decimales, que cubra el t	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
26	otal.");	
27	}	Cierra el bloque abierto anteriormente.
28	referencia = referencia == null ? "" : referencia.trim();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	if (referencia.length() > 100 (metodo.equals("TARJETA")	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
30	&& (referencia.isEmpty() recibido.compareTo(total) != 0))) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	throw new IllegalArgumentException("Confirme el pago externo e ingrese su referencia (máximo 100 car	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
32	acteres).");	
33	}	Cierra el bloque abierto anteriormente.

31	}	Cierra el bloque abierto anteriormente.
32		Línea en blanco: separa visualmente bloques para que el código sea más legible.
33	public BigDecimal total() {	Declara el método total. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
34	return lineas.stream().map(LineaPedido::subtotal).reduce(new BigDecimal("0.00"), BigDecimal::add);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
35	}	Cierra el bloque abierto anteriormente.
36	public BigDecimal cambio() { return recibido.subtract(total()); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	}	Cierra el bloque abierto anteriormente.

16.137 src/Modelos/Usuario.java

Modelo de datos Usuario. Transporta información entre la interfaz, la lógica y la base de datos sin dibujar pantallas.

Tipo: java | Líneas: 66 | Archivo 137 de 157

Clases/tipos detectados: Usuario

Línea	Constructor o método detectado
23	public Usuario(int idUsuario, String nombre, String apellido,
30	public Usuario(int idUsuario, String nombre, String apellido,
47	public int getIdUsuario() { return idUsuario; }
48	public String getNombre() { return nombre; }
49	public String getApellido() { return apellido; }
50	public String getNombreUsuario() { return nombreUsuario; }
51	public String getCorreo() { return correo; }
52	public String getRol() { return rol; }
53	public boolean isActive() { return activo; }
54	public LocalDateTime getFechaCreacion() { return fechaCreacion; }
55	public String getTurno() { return turno; }
56	public LocalTime getHoralnicio() { return horalnicio; }
57	public LocalTime getHoraFin() { return horaFin; }
58	public String getTurnoEtiqueta() {
65	public String getNombreCompleto() { return nombre + " " + apellido; }

1 package Modelos;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa fecha y hora juntas.
4 import java.time.LocalTime;	Importa java.time.LocalTime: representa una hora sin fecha.
5	Línea en blanco: separa visualmente bloques para que el código sea más legible.
6 /**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
7 * Representa una fila de la tabla usuario.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8 * La contraseña se maneja aparte y nunca vuelve desde la base de datos.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9 */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10 public class Usuario {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11 private final int idUsuario;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
12 private final String nombre;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13 private final String apellido;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
14 private final String nombreUsuario;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
15 private final String correo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
16 private final String rol;	Declara una constante o referencia que no puede reasignarse después de inicializarse.

17	private final boolean activo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18	private final LocalDateTime fechaCreacion;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	private final String turno;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
20	private final LocalTime horaInicio;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
21	private final LocalTime horaFin;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	public Usuario(int idUsuario, String nombre, String apellido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	String nombreUsuario, String correo, String rol, boolean activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25	LocalDateTime fechaCreacion) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26	this(idUsuario, nombre, apellido, nombreUsuario, correo, rol, activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
27	fechaCreacion, null, null, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	}	Cierra el bloque abierto anteriormente.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	public Usuario(int idUsuario, String nombre, String apellido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	String nombreUsuario, String correo, String rol, boolean activo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	LocalDateTime fechaCreacion, String turno, LocalTime horaInicio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	LocalTime horaFin) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	this.idUsuario = idUsuario;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
35	this.nombre = nombre;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
36	this.apellido = apellido;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
37	this.nombreUsuario = nombreUsuario;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	this.correo = correo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
39	this.rol = rol;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
40	this.activo = activo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
41	this.fechaCreacion = fechaCreacion;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
42	this.turno = turno;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
43	this.horaInicio = horaInicio;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
44	this.horaFin = horaFin;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
45	}	Cierra el bloque abierto anteriormente.
46		Línea en blanco: separa visualmente bloques para que el código sea más legible.
47	public int getIdUsuario() { return idUsuario; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	public String getNombre() { return nombre; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	public String getApellido() { return apellido; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50	public String getNombreUsuario() { return nombreUsuario; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	public String getCorreo() { return correo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
52	public String getRol() { return rol; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	public boolean isActive() { return activo; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	public LocalDateTime getFechaCreacion() { return fechaCreacion; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	public String getTurno() { return turno; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	public LocalTime getHoraInicio() { return horaInicio; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	public LocalTime getHoraFin() { return horaFin; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
58	public String getTurnoEtiqueta() {	Declara el método getTurnoEtiqueta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
59	if (turno == null horaInicio == null horaFin == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
60	return "Sin turno";	Finaliza el método y devuelve el valor indicado a quien lo llamó.

61	}	Cierra el bloque abierto anteriormente.
62	return (turno.equals("MANANA") ? "Mañana" : "Tarde")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
63	+ " (" + horaInicio + "-" + horaFin + ");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	}	Cierra el bloque abierto anteriormente.
65	public String getNombreCompleto() { return nombre + " " + apellido; }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
66	}	Cierra el bloque abierto anteriormente.

16.138 src/Utilidades/GestorImagenProducto.java

Utilidad compartida GestorImagenProducto. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 462 | Archivo 138 de 157

Clases/tipos detectados: GestorImagenProducto

Línea	Constructor o método detectado
41	private GestorImagenProducto() {
48	public static Path obtenerCarpeta()
70	public static String guardarImagen(
127	public static Imagen cargarIcono(
179	public static Imagen cargarIcono(
212	private static Imagen escalar(
272	public static void eliminarImagenGestionada(
329	public static boolean existe(
356	public static void validarImagen(
417	public static Path resolverRuta(
441	private static String obtenerExtension(

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Image;	Importa java.awt.Image para poder usar esa clase sin escribir su nombre completo.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
8	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
11	import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
12	import java.nio.file.StandardCopyOption;	Importa java.nio.file.StandardCopyOption para poder usar esa clase sin escribir su nombre completo.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	import java.util.UUID;	Importa java.util.UUID para poder usar esa clase sin escribir su nombre completo.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	import javax.swing.ImageIcon;	Importa javax.swing.ImageIcon para poder usar esa clase sin escribir su nombre completo.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	public final class GestorImagenProducto {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.

22	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
23	* Carpeta que se encuentra junto a /src.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
25	private static final String CARPETA =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
26	"imagenes_productos";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
29	* Las imágenes agregadas desde el programa	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
30	* comenzarán con este prefijo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
31	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
32	* Así evitamos eliminar accidentalmente imágenes	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
33	* originales que tú pusiste manualmente.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
34	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
35	private static final String PREFIJO_GESTIONADA =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
36	"prod_";	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	private static final long TAMANO_MAXIMO =	Declara una constante o referencia que no puede reasignarse después de inicializarse.
39	10L * 1024L * 1024L;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	private GestorImagenProducto() {	Constructor de GestorImagenProducto: prepara un objeto nuevo y deja sus componentes listos para usarse.
42	}	Cierra el bloque abierto anteriormente.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	// =====	Comentario para humanos; Java no ejecuta este texto. =====
45	// CARPETA	Comentario para humanos; Java no ejecuta este texto. CARPETA
46	// =====	Comentario para humanos; Java no ejecuta este texto. =====
47		Línea en blanco: separa visualmente bloques para que el código sea más legible.
48	public static Path obtenerCarpeta()	Declara el método obtenerCarpeta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
49	throws IOException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
50		Línea en blanco: separa visualmente bloques para que el código sea más legible.
51	Path carpeta =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
52	Path.of(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
53	System.getProperty(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	"user.dir"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	CARPETA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58		Línea en blanco: separa visualmente bloques para que el código sea más legible.
59	Files.createDirectories(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	carpeta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62		Línea en blanco: separa visualmente bloques para que el código sea más legible.
63	return carpeta;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
64	}	Cierra el bloque abierto anteriormente.

65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	// =====	Comentario para humanos; Java no ejecuta este texto. =====
67	// GUARDAR	Comentario para humanos; Java no ejecuta este texto. GUARDAR
68	// =====	Comentario para humanos; Java no ejecuta este texto. =====
69		Línea en blanco: separa visualmente bloques para que el código sea más legible.
70	public static String guardarImagen(71 File archivoOrigen)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	throws IOException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73		Línea en blanco: separa visualmente bloques para que el código sea más legible.
74	if (archivoOrigen == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	throw new IOException(77 "No se seleccionó ninguna imagen." 78);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
79	}	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80		Cierra el bloque abierto anteriormente.
81	validarImagen(82 archivoOrigen	Línea en blanco: separa visualmente bloques para que el código sea más legible. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
83	);	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.
84		Línea en blanco: separa visualmente bloques para que el código sea más legible.
85	String extension =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
86	obtenerExtension(87 archivoOrigen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
88	.getName()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
89	);	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	String nuevoNombre =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	PREFIJO_GESTIONADA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
93	+ UUID.randomUUID()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
94	+ ". "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	+ extension;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	Path destino =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
98	obtenerCarpeta()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
99	.resolve(100 nuevoNombre	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	);	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	Files.copy(104 archivoOrigen.toPath(), 105 destino, 106 StandardCopyOption.REPLACE_EXISTING 107);	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas. Ejecuta una instrucción Java completa; el punto y coma marca su final.

108		Línea en blanco: separa visualmente bloques para que el código sea más legible.
109	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
110	* IMPORTANTE:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
111	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
112	* En MySQL NO se guarda C:\Users\...	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
113	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
114	* Se guarda:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
115	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
116	* imagenes_productos/prod_xxx.png	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
117	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
118	return CARPETA	Finaliza el método y devuelve el valor indicado a quien lo llamó.
119	+ "/"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
120	+ nuevoNombre;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
121	}	Cierra el bloque abierto anteriormente.
122		Línea en blanco: separa visualmente bloques para que el código sea más legible.
123	// =====	Comentario para humanos; Java no ejecuta este texto. =====
124	// CARGAR DESDE RUTA GUARDADA	Comentario para humanos; Java no ejecuta este texto. CARGAR DESDE RUTA GUARDADA
125	// =====	Comentario para humanos; Java no ejecuta este texto. =====
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	public static ImageIcon cargarIcono(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	String ruta,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
129	int ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	int alto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131		Línea en blanco: separa visualmente bloques para que el código sea más legible.
132	if (ruta == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
133	ruta.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
134		Línea en blanco: separa visualmente bloques para que el código sea más legible.
135	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
136	}	Cierra el bloque abierto anteriormente.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
139		Línea en blanco: separa visualmente bloques para que el código sea más legible.
140	Path rutaArchivo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
141	resolverRuta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
142	ruta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
143	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
144		Línea en blanco: separa visualmente bloques para que el código sea más legible.
145	if (!Files.exists(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
146	rutaArchivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
147	)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
148		Línea en blanco: separa visualmente bloques para que el código sea más legible.
149	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
150	}	Cierra el bloque abierto anteriormente.

151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	ImageIcon original =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
153	new ImageIcon(	Crea una nueva instancia de ImageIcon y la usa o guarda para continuar el proceso.
154	rutaArchivo.toString()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	if (original.getIconWidth()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
158	<= 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
159		Línea en blanco: separa visualmente bloques para que el código sea más legible.
160	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	return escalar(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
164	original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
165	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
166	alto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
167	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168		Línea en blanco: separa visualmente bloques para que el código sea más legible.
169	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
170		Línea en blanco: separa visualmente bloques para que el código sea más legible.
171	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
172	}	Cierra el bloque abierto anteriormente.
173	}	Cierra el bloque abierto anteriormente.
174		Línea en blanco: separa visualmente bloques para que el código sea más legible.
175	// =====	Comentario para humanos; Java no ejecuta este texto. =====
176	// CARGAR ARCHIVO SELECCIONADO	Comentario para humanos; Java no ejecuta este texto. CARGAR ARCHIVO SELECCIONADO
177	// =====	Comentario para humanos; Java no ejecuta este texto. =====
178		Línea en blanco: separa visualmente bloques para que el código sea más legible.
179	public static ImageIcon cargarIcono(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	File archivo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
181	int ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
182	int alto) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
183		Línea en blanco: separa visualmente bloques para que el código sea más legible.
184	if (archivo == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
185	!archivo.exists()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
186		Línea en blanco: separa visualmente bloques para que el código sea más legible.
187	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
188	}	Cierra el bloque abierto anteriormente.
189		Línea en blanco: separa visualmente bloques para que el código sea más legible.
190	ImageIcon original =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
191	new ImageIcon(	Crea una nueva instancia de ImageIcon y la usa o guarda para continuar el proceso.
192	archivo.getAbsolutePath()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
193	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

194		Línea en blanco: separa visualmente bloques para que el código sea más legible.
195	if (original.getIconWidth()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
196	<= 0) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
197		Línea en blanco: separa visualmente bloques para que el código sea más legible.
198	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
199	}	Cierra el bloque abierto anteriormente.
200		Línea en blanco: separa visualmente bloques para que el código sea más legible.
201	return escalar(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
202	original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
203	ancho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
204	alto	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
205	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
206	}	Cierra el bloque abierto anteriormente.
207		Línea en blanco: separa visualmente bloques para que el código sea más legible.
208	// =====	Comentario para humanos; Java no ejecuta este texto. =====
209	// ESCALAR CONSERVANDO PROPORCIÓN	Comentario para humanos; Java no ejecuta este texto. ESCALAR CONSERVANDO PROPORCIÓN
210	// =====	Comentario para humanos; Java no ejecuta este texto. =====
211		Línea en blanco: separa visualmente bloques para que el código sea más legible.
212	private static ImageIcon escalar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
213	ImageIcon original,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
214	int anchoMaximo,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
215	int altoMaximo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
216		Línea en blanco: separa visualmente bloques para que el código sea más legible.
217	int anchoOriginal =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
218	original.getIconWidth();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
219		Línea en blanco: separa visualmente bloques para que el código sea más legible.
220	int altoOriginal =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
221	original.getIconHeight();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
222		Línea en blanco: separa visualmente bloques para que el código sea más legible.
223	double escala =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
224	Math.min(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
225	anchoMaximo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
226	/ (double) anchoOriginal,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
227	altoMaximo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
228	/ (double) altoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
229	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
230		Línea en blanco: separa visualmente bloques para que el código sea más legible.
231	escala =	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
232	Math.min(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
233	escala,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
234	1.0	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
235	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
236		Línea en blanco: separa visualmente bloques para que el código sea más legible.

237	int nuevoAncho =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
238	Math.max(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
239	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
240	(int) Math.round(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
241	anchoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
242	* escala	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
243	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
244	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
245		Línea en blanco: separa visualmente bloques para que el código sea más legible.
246	int nuevoAlto =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
247	Math.max(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
248	1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
249	(int) Math.round(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
250	altoOriginal	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
251	* escala	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
252	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
253	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
254		Línea en blanco: separa visualmente bloques para que el código sea más legible.
255	Image imagen =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
256	original.getImage()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
257	.getScaledInstance(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
258	nuevoAncho,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
259	nuevoAlto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
260	Image.SCALE_SMOOTH	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
261	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
262		Línea en blanco: separa visualmente bloques para que el código sea más legible.
263	return new ImageIcon(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
264	imagen	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
265	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
266	}	Cierra el bloque abierto anteriormente.
267		Línea en blanco: separa visualmente bloques para que el código sea más legible.
268	// =====	Comentario para humanos; Java no ejecuta este texto. =====
269	// ELIMINAR IMAGEN CREADA POR EL PROGRAMA	Comentario para humanos; Java no ejecuta este texto. ELIMINAR IMAGEN CREADA POR EL PROGRAMA
270	// =====	Comentario para humanos; Java no ejecuta este texto. =====
271		Línea en blanco: separa visualmente bloques para que el código sea más legible.
272	public static void eliminarImagenGestionada(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
273	String ruta) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
274		Línea en blanco: separa visualmente bloques para que el código sea más legible.
275	if (ruta == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
276	ruta.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
277		Línea en blanco: separa visualmente bloques para que el código sea más legible.
278	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
279	}	Cierra el bloque abierto anteriormente.

280		Línea en blanco: separa visualmente bloques para que el código sea más legible.
281	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
282		Línea en blanco: separa visualmente bloques para que el código sea más legible.
283	Path archivo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
284	resolverRuta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
285	ruta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
286	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
287		Línea en blanco: separa visualmente bloques para que el código sea más legible.
288	String nombre =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
289	archivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
290	.getFileName()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
291	.toString();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
292		Línea en blanco: separa visualmente bloques para que el código sea más legible.
293	/*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
294	* Solo borramos imágenes creadas	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
295	* mediante guardarImagen().	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
296	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
297	* NO eliminamos:	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
298	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
299	* imagenes_productos/bigwald.png	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
300	*	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
301	* porque puede ser una imagen inicial	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
302	* que forma parte de tu proyecto.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
303	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
304	if (!nombre.startsWith(	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
305	PREFIJO_GESTIONADA	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
306	)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
307		Línea en blanco: separa visualmente bloques para que el código sea más legible.
308	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
309	}	Cierra el bloque abierto anteriormente.
310		Línea en blanco: separa visualmente bloques para que el código sea más legible.
311	Files.deleteIfExists(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
312	archivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
313	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
314		Línea en blanco: separa visualmente bloques para que el código sea más legible.
315	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
316		Línea en blanco: separa visualmente bloques para que el código sea más legible.
317	System.out.println(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
318	"No fue posible eliminar "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
319	+ "la imagen anterior: "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
320	+ ex.getMessage()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
321	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
322	}	Cierra el bloque abierto anteriormente.
323	}	Cierra el bloque abierto anteriormente.

324		Línea en blanco: separa visualmente bloques para que el código sea más legible.
325	// =====	Comentario para humanos; Java no ejecuta este texto. =====
326	// SABER SI EXISTE	Comentario para humanos; Java no ejecuta este texto. SABER SI EXISTE
327	// =====	Comentario para humanos; Java no ejecuta este texto. =====
328		Línea en blanco: separa visualmente bloques para que el código sea más legible.
329	public static boolean existe(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
330	String ruta) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
331		Línea en blanco: separa visualmente bloques para que el código sea más legible.
332	if (ruta == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
333	ruta.isBlank()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
334		Línea en blanco: separa visualmente bloques para que el código sea más legible.
335	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
336	}	Cierra el bloque abierto anteriormente.
337		Línea en blanco: separa visualmente bloques para que el código sea más legible.
338	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
339		Línea en blanco: separa visualmente bloques para que el código sea más legible.
340	return Files.exists(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
341	resolverRuta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
342	ruta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
343	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
344	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
345		Línea en blanco: separa visualmente bloques para que el código sea más legible.
346	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
347		Línea en blanco: separa visualmente bloques para que el código sea más legible.
348	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
349	}	Cierra el bloque abierto anteriormente.
350	}	Cierra el bloque abierto anteriormente.
351		Línea en blanco: separa visualmente bloques para que el código sea más legible.
352	// =====	Comentario para humanos; Java no ejecuta este texto. =====
353	// VALIDAR	Comentario para humanos; Java no ejecuta este texto. VALIDAR
354	// =====	Comentario para humanos; Java no ejecuta este texto. =====
355		Línea en blanco: separa visualmente bloques para que el código sea más legible.
356	public static void validarImagen(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
357	File archivo)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
358	throws IOException {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
359		Línea en blanco: separa visualmente bloques para que el código sea más legible.
360	if (archivo == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
361	!archivo.exists()) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
362		Línea en blanco: separa visualmente bloques para que el código sea más legible.
363	throw new IOException(	Interrompe el flujo lanzando un error con una explicación para quien llamó al método.
364	"El archivo seleccionado no existe."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
365	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

366	}	Cierra el bloque abierto anteriormente.
367		Línea en blanco: separa visualmente bloques para que el código sea más legible.
368	if (!archivo.isFile()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
369		Línea en blanco: separa visualmente bloques para que el código sea más legible.
370	throw new IOException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
371	"El archivo seleccionado no es válido."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
372	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
373	}	Cierra el bloque abierto anteriormente.
374		Línea en blanco: separa visualmente bloques para que el código sea más legible.
375	if (archivo.length()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
376	> TAMANO_MAXIMO) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
377		Línea en blanco: separa visualmente bloques para que el código sea más legible.
378	throw new IOException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
379	"La imagen no puede superar los 10 MB."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
380	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
381	}	Cierra el bloque abierto anteriormente.
382		Línea en blanco: separa visualmente bloques para que el código sea más legible.
383	String extension =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
384	obtenerExtension(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
385	archivo.getName()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
386	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
387		Línea en blanco: separa visualmente bloques para que el código sea más legible.
388	boolean permitida =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
389	extension.equals("png")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
390	extension.equals("jpg")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
391	extension.equals("jpeg");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
392		Línea en blanco: separa visualmente bloques para que el código sea más legible.
393	if (!permitida) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
394		Línea en blanco: separa visualmente bloques para que el código sea más legible.
395	throw new IOException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
396	"Solo puedes utilizar imágenes PNG, JPG o JPEG."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
397	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
398	}	Cierra el bloque abierto anteriormente.
399		Línea en blanco: separa visualmente bloques para que el código sea más legible.
400	BufferedImage imagen =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
401	ImageIO.read(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
402	archivo	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
403	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
404		Línea en blanco: separa visualmente bloques para que el código sea más legible.
405	if (imagen == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
406		Línea en blanco: separa visualmente bloques para que el código sea más legible.
407	throw new IOException(	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
408	"El archivo seleccionado no contiene una imagen válida."	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
409	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

410	}	Cierra el bloque abierto anteriormente.
411	}	Cierra el bloque abierto anteriormente.
412		Línea en blanco: separa visualmente bloques para que el código sea más legible.
413	// =====	Comentario para humanos; Java no ejecuta este texto. =====
414	// RUTA	Comentario para humanos; Java no ejecuta este texto. RUTA
415	// =====	Comentario para humanos; Java no ejecuta este texto. =====
416		Línea en blanco: separa visualmente bloques para que el código sea más legible.
417	public static Path resolverRuta(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
418	String ruta) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
419		Línea en blanco: separa visualmente bloques para que el código sea más legible.
420	Path path =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
421	Path.of(ruta);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
422		Línea en blanco: separa visualmente bloques para que el código sea más legible.
423	if (path.isAbsolute()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
424		Línea en blanco: separa visualmente bloques para que el código sea más legible.
425	return path;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
426	}	Cierra el bloque abierto anteriormente.
427		Línea en blanco: separa visualmente bloques para que el código sea más legible.
428	return Path.of(	Finaliza el método y devuelve el valor indicado a quien lo llamó.
429	System.getProperty(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
430	"user.dir"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
431	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
432	).resolve(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
433	ruta	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
434	).normalize());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
435	}	Cierra el bloque abierto anteriormente.
436		Línea en blanco: separa visualmente bloques para que el código sea más legible.
437	// =====	Comentario para humanos; Java no ejecuta este texto. =====
438	// EXTENSIÓN	Comentario para humanos; Java no ejecuta este texto. EXTENSIÓN
439	// =====	Comentario para humanos; Java no ejecuta este texto. =====
440		Línea en blanco: separa visualmente bloques para que el código sea más legible.
441	private static String obtenerExtension(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
442	String nombre) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
443		Línea en blanco: separa visualmente bloques para que el código sea más legible.
444	int posicion =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
445	nombre.lastIndexOf(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
446	','	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
447	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
448		Línea en blanco: separa visualmente bloques para que el código sea más legible.
449	if (posicion < 0	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
450	posicion	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
451	== nombre.length() - 1) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

452		Línea en blanco: separa visualmente bloques para que el código sea más legible.
453	return "";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
454	}	Cierra el bloque abierto anteriormente.
455		Línea en blanco: separa visualmente bloques para que el código sea más legible.
456	return nombre	Finaliza el método y devuelve el valor indicado a quien lo llamó.
457	.substring(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
458	posicion + 1	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
459	)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
460	.toLowerCase();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
461	}	Cierra el bloque abierto anteriormente.
462	}	Cierra el bloque abierto anteriormente.

16.139 src/Utilidades/HorarioMenu.java

Utilidad compartida HorarioMenu. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 33 | Archivo 139 de 157

Clases/tipos detectados: HorarioMenu

Línea	Constructor o método detectado
11	private HorarioMenu() {
19	public static String disponibilidadActual(LocalTime hora) {
26	public static boolean estaDisponible(String disponibilidad, LocalTime hora) {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.time.LocalTime;	Importa java.time.LocalTime: representa una hora sin fecha.
4		Línea en blanco: separa visualmente bloques para que el código sea más legible.
5	/** Reglas de disponibilidad del menú según la hora local del restaurante. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
6	public final class HorarioMenu {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7		Línea en blanco: separa visualmente bloques para que el código sea más legible.
8	/** A las 11:00 termina el horario de desayuno. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
9	public static final LocalTime INICIO_ALMUERZO = LocalTime.of(11, 0);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
10		Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	private HorarioMenu() {	Constructor de HorarioMenu: prepara un objeto nuevo y deja sus componentes listos para usarse.
12	}	Cierra el bloque abierto anteriormente.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
15	* Devuelve el valor que utiliza producto.disponibilidad_menu para la hora	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
16	* indicada. El almuerzo/cena empieza a las 11:00; antes de esa hora es	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
17	* desayuno.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
18	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	public static String disponibilidadActual(LocalTime hora) {	Declara el método disponibilidadActual. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
20	if (hora == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
21	throw new IllegalArgumentException("La hora no puede ser nula.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
22	}	Cierra el bloque abierto anteriormente.
23	return hora.isBefore(INICIO_ALMUERZO) ? "DESAYUNO" : "ALMUERZO";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
24	}	Cierra el bloque abierto anteriormente.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	public static boolean estaDisponible(String disponibilidad, LocalTime hora) {	Declara el método estaDisponible. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
27	if (disponibilidad == null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
28	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
29	}	Cierra el bloque abierto anteriormente.
30	return "TODO_DIA".equalsIgnoreCase(disponibilidad)	Finaliza el método y devuelve el valor indicado a quien lo llamó.
31	disponibilidadActual(hora).equalsIgnoreCase(disponibilidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

32	}	Cierra el bloque abierto anteriormente.
33	}	Cierra el bloque abierto anteriormente.

16.140 src/Utilidades/HorarioTurno.java

Utilidad compartida HorarioTurno. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 41 | Archivo 140 de 157

Clases/tipos detectados: HorarioTurno

Línea	Constructor o método detectado
15	private HorarioTurno() {
18	public static boolean estaEnTurno(
34	public static String etiqueta(String turno) {

1 package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.time.LocalDateTime;	Importa java.time.LocalDateTime: representa una hora sin fecha.
4	Línea en blanco: separa visualmente bloques para que el código sea más legible.
5 /** Horarios asignados a los turnos disponibles para cajeros. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
6 public final class HorarioTurno {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
7 public static final String MANANA = "MANANA";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
8 public static final String TARDE = "TARDE";	Declara una constante o referencia que no puede reasignarse después de inicializarse.
9	Línea en blanco: separa visualmente bloques para que el código sea más legible.
10 public static final LocalDateTime INICIO_MANANA = LocalDateTime.of(6, 0);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
11 public static final LocalDateTime FIN_MANANA = LocalDateTime.of(14, 0);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
12 public static final LocalDateTime INICIO_TARDE = LocalDateTime.of(14, 0);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
13 public static final LocalDateTime FIN_TARDE = LocalDateTime.of(22, 0);	Declara una constante o referencia que no puede reasignarse después de inicializarse.
14	Línea en blanco: separa visualmente bloques para que el código sea más legible.
15 private HorarioTurno() {	Constructor de HorarioTurno: prepara un objeto nuevo y deja sus componentes listos para usarse.
16 }	Cierra el bloque abierto anteriormente.
17	Línea en blanco: separa visualmente bloques para que el código sea más legible.
18 public static boolean estaEnTurno(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19 String turno,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20 LocalDateTime inicio,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21 LocalDateTime fin,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
22 LocalDateTime ahora) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
23 if (turno == null inicio == null fin == null	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
24 ahora == null inicio.equals(fin)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25 return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
26 }	Cierra el bloque abierto anteriormente.
27 if (inicio.isBefore(fin)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
28 return !ahora.isBefore(inicio) && ahora.isBefore(fin);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
29 }	Cierra el bloque abierto anteriormente.
30 // También admite turnos que cruzan la medianoche.	Comentario para humanos; Java no ejecuta este texto. También admite turnos que cruzan la medianoche.
31 return !ahora.isBefore(inicio) ahora.isBefore(fin);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
32 }	Cierra el bloque abierto anteriormente.

33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	public static String etiqueta(String turno) {	Declara el método etiqueta. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35	return switch (turno == null ? "" : turno) {	Finaliza el método y devuelve el valor indicado a quien lo llamó.
36	case MANANA -> "Mañana (06:00-14:00)";	Define una de las rutas posibles dentro del switch.
37	case TARDE -> "Tarde (14:00-22:00)";	Define una de las rutas posibles dentro del switch.
38	default -> "Sin turno";	Define una de las rutas posibles dentro del switch.
39	};	Cierra el bloque abierto anteriormente y termina la declaración.
40	}	Cierra el bloque abierto anteriormente.
41	}	Cierra el bloque abierto anteriormente.

16.141 src/Utilidades/IconosAdmin.java

Utilidad compartida IconosAdmin. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 75 | Archivo 141 de 157

Clases/tipos detectados: IconosAdmin

Línea	Constructor o método detectado
10	private IconosAdmin() {
13	public static void pintarTicket(

IconosAdmin.java		
1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.geom.Path2D;	Importa java.awt.geom.Path2D para poder usar esa clase sin escribir su nombre completo.
6		Línea en blanco: separa visualmente bloques para que el código sea más legible.
7	/** Dibujo vectorial del icono de menú utilizado por InicioAdminForm. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8	public final class IconosAdmin {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	private IconosAdmin() {	Constructor de IconosAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
11	}	Cierra el bloque abierto anteriormente.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	public static void pintarTicket(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
14	Graphics2D g2,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
15	int centroX,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
16	int centroY,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	TemaAdmin tema) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	int x = centroX - tema.px(40);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
20	int y = centroY - tema.px(53);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
21	int ancho = tema.px(80);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
22	int alto = tema.px(112);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	int doblez = tema.px(18);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	Path2D documento = new Path2D.Double();	Crea una nueva instancia de Path2D.Double y la usa o guarda para continuar el proceso.
26	documento.moveTo(x + tema.px(10), y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27	documento.lineTo(x + ancho - doblez, y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28	documento.lineTo(x + ancho, y + doblez);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	documento.lineTo(x + ancho, y + alto - tema.px(8));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	documento.quadTo(x + ancho, y + alto, x + ancho - tema.px(8), y + alto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	documento.lineTo(x + tema.px(8), y + alto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	documento.quadTo(x, y + alto, x, y + alto - tema.px(8));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
33	documento.lineTo(x, y + tema.px(10));	Ejecuta una instrucción Java completa; el punto y coma marca su final.

34	documento.quadTo(x, y, x + tema.px(10), y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	g2.setStroke(new BasicStroke(	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
37	Math.max(3f, tema.px(5)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	BasicStroke.CAP_ROUND,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	BasicStroke.JOIN_ROUND	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
41	g2.setColor(TemaAdmin.AMARILLO);	Configura una propiedad mediante setColor en el objeto indicado.
42	g2.draw(documento);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	g2.drawLine(x + ancho - doblez, y, x + ancho - doblez, y + doblez);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	g2.drawLine(x + ancho - doblez, y + doblez, x + ancho, y + doblez);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45		Línea en blanco: separa visualmente bloques para que el código sea más legible.
46	int burgerY = y + tema.px(43);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	g2.drawArc(x + tema.px(18), burgerY, tema.px(44), tema.px(27), 5, 170);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	g2.drawLine(x + tema.px(17), burgerY + tema.px(17),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	x + tema.px(63), burgerY + tema.px(17));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	g2.drawArc(x + tema.px(18), burgerY + tema.px(15),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	tema.px(44), tema.px(18), 185, 170);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52		Línea en blanco: separa visualmente bloques para que el código sea más legible.
53	g2.setStroke(new BasicStroke(	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
54	Math.max(2f, tema.px(3)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	BasicStroke.CAP_ROUND,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
56	BasicStroke.JOIN_ROUND	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	g2.drawLine(x + tema.px(29), burgerY + tema.px(10),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
59	x + tema.px(31), burgerY + tema.px(10));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	g2.drawLine(x + tema.px(42), burgerY + tema.px(7),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	x + tema.px(44), burgerY + tema.px(7));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	g2.drawLine(x + tema.px(51), burgerY + tema.px(11),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
63	x + tema.px(53), burgerY + tema.px(11));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	g2.setStroke(new BasicStroke(	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
66	Math.max(3f, tema.px(5)),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
67	BasicStroke.CAP_ROUND,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	BasicStroke.JOIN_ROUND	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	g2.drawLine(x + tema.px(18), y + tema.px(91),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	x + tema.px(62), y + tema.px(91));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	g2.drawLine(x + tema.px(18), y + tema.px(103),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	x + tema.px(62), y + tema.px(103));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	}	Cierra el bloque abierto anteriormente.
75	}	Cierra el bloque abierto anteriormente.

16.142 src/Utilidades/IconosUsuarios.java

Utilidad compartida IconosUsuarios. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 172 | Archivo 142 de 157

Clases/tipos detectados: IconosUsuarios, Tipo, IconoLinea

Línea	Constructor o método detectado
21	private IconosUsuarios() {
24	public static Icon crear(Tipo tipo, Color color, int tamano) {
34	private IconoLinea(Tipo tipo, Color color, int tamano) {
41	public int getIconWidth() {
46	public int getIconHeight() {
51	public void paintIcon(Component c, Graphics graphics, int x, int y) {
79	private void pintarBuscar(Graphics2D g2) {
84	private void pintarEditar(Graphics2D g2) {
97	private void pintarEliminar(Graphics2D g2) {
105	private void pintarFiltro(Graphics2D g2) {
114	private void pintarExportar(Graphics2D g2) {
121	private void pintarAgregar(Graphics2D g2) {
127	private void pintarFlecha(Graphics2D g2, boolean derecha) {
134	private void pintarUsuarios(Graphics2D g2) {
141	private void pintarAdministrador(Graphics2D g2) {
152	private void pintarCajero(Graphics2D g2) {
157	private void pintarInactivo(Graphics2D g2) {
163	private void pintarOjo(Graphics2D g2) {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.geom.Path2D;	Importa java.awt.geom.Path2D para poder usar esa clase sin escribir su nombre completo.
10	import javax.swing.Icon;	Importa javax.swing.Icon para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	/** Iconos vectoriales del módulo; no dependen de imágenes externas. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.

13	public final class IconosUsuarios {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	public enum Tipo {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
16	BUSCAR, EDITAR, ELIMINAR, FILTRO, EXPORTAR, AGREGAR,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	ANTERIOR, SIGUIENTE, USUARIOS, ADMINISTRADOR, CAJERO,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18	INACTIVO, OJO	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19	}	Cierra el bloque abierto anteriormente.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private IconosUsuarios() {	Constructor de IconosUsuarios: prepara un objeto nuevo y deja sus componentes listos para usarse.
22	}	Cierra el bloque abierto anteriormente.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	public static Icon crear(Tipo tipo, Color color, int tamano) {	Declara el método crear. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
25	return new IconoLinea(tipo, color, Math.max(12, tamano));	Finaliza el método y devuelve el valor indicado a quien lo llamó.
26	}	Cierra el bloque abierto anteriormente.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	private static final class IconoLinea implements Icon {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
29		Línea en blanco: separa visualmente bloques para que el código sea más legible.
30	private final Tipo tipo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
31	private final Color color;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
32	private final int tamano;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	private IconoLinea(Tipo tipo, Color color, int tamano) {	Declara el método IconoLinea. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35	this.tipo = tipo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
36	this.color = color;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
37	this.tamano = tamano;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
38	}	Cierra el bloque abierto anteriormente.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
41	public int getIconWidth() {	Declara el método getIconWidth. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	return tamano;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
46	public int getIconHeight() {	Declara el método getIconHeight. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
47	return tamano;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
48	}	Cierra el bloque abierto anteriormente.
49		Línea en blanco: separa visualmente bloques para que el código sea más legible.
50	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
51	public void paintIcon(Component c, Graphics graphics, int x, int y) {	Declara el método paintIcon. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
52	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	g2.translate(x, y);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	g2.scale(tamano / 24.0, tamano / 24.0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
56	RenderingHints.VALUE_ANTIALIAS_ON);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

57	g2.setColor(color);	Configura una propiedad mediante setColor en el objeto indicado.
58	g2.setStroke(new BasicStroke(1.9f, BasicStroke.CAP_ROUND,	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
59	BasicStroke.JOIN_ROUND));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60		Línea en blanco: separa visualmente bloques para que el código sea más legible.
61	switch (tipo) {	Selecciona una ruta entre varias según el valor evaluado.
62	case BUSCAR -> pintarBuscar(g2);	Define una de las rutas posibles dentro del switch.
63	case EDITAR -> pintarEditar(g2);	Define una de las rutas posibles dentro del switch.
64	case ELIMINAR -> pintarEliminar(g2);	Define una de las rutas posibles dentro del switch.
65	case FILTRO -> pintarFiltro(g2);	Define una de las rutas posibles dentro del switch.
66	case EXPORTAR -> pintarExportar(g2);	Define una de las rutas posibles dentro del switch.
67	case AGREGAR -> pintarAgregar(g2);	Define una de las rutas posibles dentro del switch.
68	case ANTERIOR -> pintarFlecha(g2, false);	Define una de las rutas posibles dentro del switch.
69	case SIGUIENTE -> pintarFlecha(g2, true);	Define una de las rutas posibles dentro del switch.
70	case USUARIOS -> pintarUsuarios(g2);	Define una de las rutas posibles dentro del switch.
71	case ADMINISTRADOR -> pintarAdministrador(g2);	Define una de las rutas posibles dentro del switch.
72	case CAJERO -> pintarCajero(g2);	Define una de las rutas posibles dentro del switch.
73	case INACTIVO -> pintarInactivo(g2);	Define una de las rutas posibles dentro del switch.
74	case OJO -> pintarOjo(g2);	Define una de las rutas posibles dentro del switch.
75	}	Cierra el bloque abierto anteriormente.
76	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	}	Cierra el bloque abierto anteriormente.
78		Línea en blanco: separa visualmente bloques para que el código sea más legible.
79	private void pintarBuscar(Graphics2D g2) {	Declara el método pintarBuscar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
80	g2.drawOval(4, 4, 11, 11);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	g2.drawLine(14, 14, 20, 20);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	}	Cierra el bloque abierto anteriormente.
83		Línea en blanco: separa visualmente bloques para que el código sea más legible.
84	private void pintarEditar(Graphics2D g2) {	Declara el método pintarEditar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
85	Path2D lapiz = new Path2D.Double();	Crea una nueva instancia de Path2D.Double y la usa o guarda para continuar el proceso.
86	lapiz.moveTo(5, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	lapiz.lineTo(6, 13);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	lapiz.lineTo(16, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	lapiz.lineTo(21, 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	lapiz.lineTo(11, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	lapiz.closePath();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	g2.draw(lapiz);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	g2.drawLine(14, 5, 19, 10);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	g2.drawLine(5, 18, 4, 20);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	}	Cierra el bloque abierto anteriormente.
96		Línea en blanco: separa visualmente bloques para que el código sea más legible.
97	private void pintarEliminar(Graphics2D g2) {	Declara el método pintarEliminar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
98	g2.drawRoundRect(6, 7, 12, 14, 2, 2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
99	g2.drawLine(4, 6, 20, 6);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	g2.drawLine(9, 3, 15, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

101	g2.drawLine(10, 10, 10, 17);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	g2.drawLine(14, 10, 14, 17);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	}	Cierra el bloque abierto anteriormente.
104		Línea en blanco: separa visualmente bloques para que el código sea más legible.
105	private void pintarFiltro(Graphics2D g2) {	Declara el método pintarFiltro. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
106	g2.drawLine(4, 6, 20, 6);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107	g2.drawLine(7, 12, 20, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
108	g2.drawLine(10, 18, 20, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	g2.drawOval(5, 10, 4, 4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	g2.drawOval(11, 4, 4, 4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
111	g2.drawOval(14, 16, 4, 4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	}	Cierra el bloque abierto anteriormente.
113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	private void pintarExportar(Graphics2D g2) {	Declara el método pintarExportar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
115	g2.drawRoundRect(4, 10, 16, 10, 2, 2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	g2.drawLine(12, 16, 12, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	g2.drawLine(8, 7, 12, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	g2.drawLine(16, 7, 12, 3);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119	}	Cierra el bloque abierto anteriormente.
120		Línea en blanco: separa visualmente bloques para que el código sea más legible.
121	private void pintarAgregar(Graphics2D g2) {	Declara el método pintarAgregar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
122	g2.drawOval(3, 3, 18, 18);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	g2.drawLine(12, 7, 12, 17);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
124	g2.drawLine(7, 12, 17, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
125	}	Cierra el bloque abierto anteriormente.
126		Línea en blanco: separa visualmente bloques para que el código sea más legible.
127	private void pintarFlecha(Graphics2D g2, boolean derecha) {	Declara el método pintarFlecha. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
128	int inicio = derecha ? 8 : 16;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
129	int finalX = derecha ? 16 : 8;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
130	g2.drawLine(inicio, 5, finalX, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
131	g2.drawLine(finalX, 12, inicio, 19);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
132	}	Cierra el bloque abierto anteriormente.
133		Línea en blanco: separa visualmente bloques para que el código sea más legible.
134	private void pintarUsuarios(Graphics2D g2) {	Declara el método pintarUsuarios. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
135	g2.drawOval(8, 3, 8, 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	g2.drawArc(4, 11, 16, 10, 0, 180);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	g2.drawArc(1, 8, 8, 9, 70, 120);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	g2.drawArc(15, 8, 8, 9, -10, 120);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
139	}	Cierra el bloque abierto anteriormente.
140		Línea en blanco: separa visualmente bloques para que el código sea más legible.
141	private void pintarAdministrador(Graphics2D g2) {	Declara el método pintarAdministrador. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
142	pintarUsuarios(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	Path2D corona = new Path2D.Double();	Crea una nueva instancia de Path2D.Double y la usa o guarda para continuar el proceso.
144	corona.moveTo(6, 5);	Ejecuta una instrucción Java completa; el punto y coma marca su final.

145	corona.lineTo(8, 1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146	corona.lineTo(12, 4);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	corona.lineTo(16, 1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	corona.lineTo(18, 5);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
149	g2.draw(corona);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
150	}	Cierra el bloque abierto anteriormente.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	private void pintarCajero(Graphics2D g2) {	Declara el método pintarCajero. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
153	g2.drawOval(8, 3, 8, 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
154	g2.drawArc(4, 11, 16, 10, 0, 180);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156		Línea en blanco: separa visualmente bloques para que el código sea más legible.
157	private void pintarInactivo(Graphics2D g2) {	Declara el método pintarInactivo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
158	pintarCajero(g2);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	g2.drawLine(16, 15, 22, 21);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
160	g2.drawLine(22, 15, 16, 21);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161	}	Cierra el bloque abierto anteriormente.
162		Línea en blanco: separa visualmente bloques para que el código sea más legible.
163	private void pintarOjo(Graphics2D g2) {	Declara el método pintarOjo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
164	Path2D ojo = new Path2D.Double();	Crea una nueva instancia de Path2D.Double y la usa o guarda para continuar el proceso.
165	ojo.moveTo(2, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
166	ojo.curveTo(7, 5, 17, 5, 22, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
167	ojo.curveTo(17, 19, 7, 19, 2, 12);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
168	g2.draw(ojo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
169	g2.drawOval(9, 9, 6, 6);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
170	}	Cierra el bloque abierto anteriormente.
171	}	Cierra el bloque abierto anteriormente.
172	}	Cierra el bloque abierto anteriormente.

16.143 src/Utilidades/ImageCache.java

Utilidad compartida ImageCache. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 80 | Archivo 143 de 157

Clases/tipos detectados: ImageCache

Línea	Constructor o método detectado
22	private static final ExecutorService EXECUTOR = Executors.newFixedThreadPool(
29	private ImageCache() { }
31	public static void cargarAsync(String ruta, int ancho, int alto,
52	private static Imagemcon cargarYEScalar(String ruta, int ancho, int alto) {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Image;	Importa java.awt.Image para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
7	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
8	import java.net.URL;	Importa java.net.URL para poder usar esa clase sin escribir su nombre completo.
9	import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
10	import java.util.concurrent.ConcurrentHashMap;	Importa java.util.concurrent.ConcurrentHashMap para poder usar esa clase sin escribir su nombre completo.
11	import java.util.concurrent.ExecutorService;	Importa java.util.concurrent.ExecutorService para poder usar esa clase sin escribir su nombre completo.
12	import java.util.concurrent.Executors;	Importa java.util.concurrent.Executors para poder usar esa clase sin escribir su nombre completo.
13	import java.util.function.Consumer;	Importa java.util.function.Consumer para poder usar esa clase sin escribir su nombre completo.
14	import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.ImageIcon;	Importa javax.swing.ImageIcon para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
17		Línea en blanco: separa visualmente bloques para que el código sea más legible.
18	/** Carga y escala imágenes fuera del EDT, reutilizando resultados en memoria. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
19	public final class ImageCache {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private static final Map<String, ImageIcon> CACHE = new ConcurrentHashMap<>();	Crea una nueva instancia de ConcurrentHashMap y la usa o guarda para continuar el proceso.
22	private static final ExecutorService EXECUTOR = Executors.newFixedThreadPool(	Declara una constante o referencia que no puede reasignarse después de inicializarse.
23	4, runnable -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
24	Thread hilo = new Thread(runnable, "waldonald-imagenes");	Crea una nueva instancia de Thread y la usa o guarda para continuar el proceso.
25	hilo.setDaemon(true);	Configura una propiedad mediante setDaemon en el objeto indicado.
26	return hilo;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
27	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	private ImageCache() { }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.

31	public static void cargarAsync(String ruta, int ancho, int alto,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	Consumer<ImageIcon> callback) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	if (ruta == null ruta.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
34	SwingUtilities.invokeLater(() -> callback.accept(null));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	String clave = ruta + " " + ancho + "x" + alto;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
39	ImageIcon cacheada = CACHE.get(clave);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
40	if (cacheada != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
41	SwingUtilities.invokeLater(() -> callback.accept(cacheada));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	return;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
43	}	Cierra el bloque abierto anteriormente.
44		Línea en blanco: separa visualmente bloques para que el código sea más legible.
45	EXECUTOR.execute(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	ImageIcon resultado = cargarYEscalar(ruta, ancho, alto);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	if (resultado != null) CACHE.putIfAbsent(clave, resultado);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
48	SwingUtilities.invokeLater(() -> callback.accept(resultado));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	}	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	private static ImageIcon cargarYEscalar(String ruta, int ancho, int alto) {	Declara el método cargarYEscalar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
54	BufferedImage original;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
55	URL recurso = ImageCache.class.getResource(ruta);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
56	if (recurso != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
57	original = ImageIO.read(recurso);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
58	} else {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
59	File archivo = new File(ruta);	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
60	if (!archivo.exists()) return null;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
61	original = ImageIO.read(archivo);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
62	}	Cierra el bloque abierto anteriormente.
63	if (original == null) return null;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
64		Línea en blanco: separa visualmente bloques para que el código sea más legible.
65	BufferedImage escalada = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
66	ancho, alto, BufferedImage.TYPE_INT_ARGB);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	Graphics2D graphics = escalada.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	graphics.setRenderingHint(RenderingHints.KEY_INTERPOLATION,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
69	RenderingHints.VALUE_INTERPOLATION_BILINEAR);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	graphics.setRenderingHint(RenderingHints.KEY_RENDERING,	Configura una propiedad mediante setRenderingHint en el objeto indicado.
71	RenderingHints.VALUE_RENDER_SPEED);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	graphics.drawImage(original.getScaledInstance(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	ancho, alto, Image.SCALE_SMOOTH, 0, 0, null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	graphics.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.

75	return new ImageIcon(escalada);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
76	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
77	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
78	}	Cierra el bloque abierto anteriormente.
79	}	Cierra el bloque abierto anteriormente.
80	}	Cierra el bloque abierto anteriormente.

16.144 src/Utilidades/PagoPendienteStore.java

Utilidad compartida PagoPendienteStore. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 107 | Archivo 144 de 157

Clases/tipos detectados: PagoPendienteStore

Línea	Constructor o método detectado
16	public PagoPendienteStore(int usuario) { this(Path.of("pagos_pendientes", "caja-" + usuario + ".properties")); }
17	public PagoPendienteStore(Path archivo) { this.archivo = archivo; }
18	public void guardar(SolicitudPago pago) throws IOException {
64	public SolicitudPago cargar() throws IOException {
106	public void eliminar() throws IOException { Files.deleteIfExists(archivo); }

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Modelos.LineaPedido;	Importa Modelos.LineaPedido para poder usar esa clase sin escribir su nombre completo.
4	import Modelos.ModificacionPedido;	Importa Modelos.ModificacionPedido para poder usar esa clase sin escribir su nombre completo.
5	import Modelos.OpcionPedido;	Importa Modelos.OpcionPedido para poder usar esa clase sin escribir su nombre completo.
6	import Modelos.ProductoPedido;	Importa Modelos.ProductoPedido para poder usar esa clase sin escribir su nombre completo.
7	import Modelos.SolicitudPago;	Importa Modelos.SolicitudPago para poder usar esa clase sin escribir su nombre completo.
8	import java.io.*;	Importa java.io.* para poder usar esa clase sin escribir su nombre completo.
9	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
10	import java.nio.file.*;	Importa java.nio.file.* para poder usar esa clase sin escribir su nombre completo.
11	import java.util.*;	Importa java.util.* para poder usar esa clase sin escribir su nombre completo.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	/** Guarda la solicitud antes del cobro para poder recuperarla tras reiniciar la caja. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
14	public final class PagoPendienteStore {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
15	private final Path archivo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
16	public PagoPendienteStore(int usuario) { this(Path.of("pagos_pendientes", "caja-" + usuario + ".properties"));	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	};	
18	public PagoPendienteStore(Path archivo) { this.archivo = archivo; }	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
19	public void guardar(SolicitudPago pago) throws IOException {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
20	Properties p = new Properties();	Crea una nueva instancia de Properties y la usa o guarda para continuar el proceso.
21	p.setProperty("clave", pago.clave()); p.setProperty("usuario", "" + pago.usuario());	Configura una propiedad mediante setProperty en el objeto indicado.
22	p.setProperty("servicio", pago.servicio()); p.setProperty("metodo", pago.metodo());	Configura una propiedad mediante setProperty en el objeto indicado.
23	p.setProperty("recibido", pago.recibido().toString()); p.setProperty("referencia", pago.referencia());	Configura una propiedad mediante setProperty en el objeto indicado.
24	});	
25	p.setProperty("cantidad", "" + pago.lineas().size());	Configura una propiedad mediante setProperty en el objeto indicado.
26	for (int i=0; i<pago.lineas().size(); i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
27	LineaPedido l=pago.lineas().get(i); String prefijo="línea."+i+";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	p.setProperty(prefijo+"idLinea", l.idLinea());	Configura una propiedad mediante setProperty en el objeto indicado.
29	p.setProperty(prefijo+"idProducto", ""+l.idProducto());	Configura una propiedad mediante setProperty en el objeto indicado.
30	p.setProperty(prefijo+"idPresentacion", ""+l.idPresentacion());	Configura una propiedad mediante setProperty en el objeto indicado.

29	p.setProperty(prefijo+"nombre", l.nombre());	Configura una propiedad mediante setProperty en el objeto indicado.
30	p.setProperty(prefijo+"presentacion", l.presentacion());	Configura una propiedad mediante setProperty en el objeto indicado.
31	p.setProperty(prefijo+"precioBase", l.precioBase().toString());	Configura una propiedad mediante setProperty en el objeto indicado.
32	p.setProperty(prefijo+"cantidad", ""+l.cantidad());	Configura una propiedad mediante setProperty en el objeto indicado.
33	p.setProperty(prefijo+"opciones", ""+l.opciones().size());	Configura una propiedad mediante setProperty en el objeto indicado.
34	for (int j=0; j<l.opciones().size(); j++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
35	opcionPedido o=l.opciones().get(j); String po=prefijo+"opcion."+j+ ".";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
36	p.setProperty(po+"idGrupo", ""+o.idGrupo()); p.setProperty(po+"idOpcion", ""+o.idOpcion());	Configura una propiedad mediante setProperty en el objeto indicado.
37	p.setProperty(po+"numero", ""+o.numero()); p.setProperty(po+"grupo", o.grupo());	Configura una propiedad mediante setProperty en el objeto indicado.
38	p.setProperty(po+"opcion", o.opcion()); p.setProperty(po+"precio", o.precioExtra().toString());	Configura una propiedad mediante setProperty en el objeto indicado.
39	p.setProperty(po+"productos", ""+o.productos().size());	Configura una propiedad mediante setProperty en el objeto indicado.
40	for (int k=0; k<o.productos().size(); k++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
41	ProductoPedido pr=o.productos().get(k); String pp=po+"producto."+k+ ".";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42	p.setProperty(pp+"idInterno", pr.idInterno()); p.setProperty(pp+"id", ""+pr.idProducto());	Configura una propiedad mediante setProperty en el objeto indicado.
43	p.setProperty(pp+"nombre", pr.nombre()); p.setProperty(pp+"cantidad", ""+pr.cantidad());	Configura una propiedad mediante setProperty en el objeto indicado.
44	p.setProperty(pp+"tipoStock", pr.tipoStock()); p.setProperty(pp+"cambios", ""+pr.modificaciones().size());	Configura una propiedad mediante setProperty en el objeto indicado.
45	for (int m=0; m<pr.modificaciones().size(); m++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
46	ModificacionPedido mo=pr.modificaciones().get(m); String pm=pp+"cambio."+m+ ".";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	p.setProperty(pm+"idProductoIngrediente", ""+mo.idProductoIngrediente());	Configura una propiedad mediante setProperty en el objeto indicado.
48	p.setProperty(pm+"idIngrediente", ""+mo.idIngrediente()); p.setProperty(pm+"tipo", mo.tipo());	Configura una propiedad mediante setProperty en el objeto indicado.
49	p.setProperty(pm+"ingrediente", mo.ingrediente()); p.setProperty(pm+"cantidad", mo.cantidad().toString());	Configura una propiedad mediante setProperty en el objeto indicado.
50	p.setProperty(pm+"veces", ""+mo.veces()); p.setProperty(pm+"precio", mo.precioExtra().toString());	Configura una propiedad mediante setProperty en el objeto indicado.
51	}	Cierra el bloque abierto anteriormente.
52	}	Cierra el bloque abierto anteriormente.
53	}	Cierra el bloque abierto anteriormente.
54	}	Cierra el bloque abierto anteriormente.
55	Files.createDirectories(archivo.toAbsolutePath().getParent());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	Path temporal = Files.createTempFile(archivo.toAbsolutePath().getParent(), "pago-", ".tmp");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
58	try (FileOutputStream salida = new FileOutputStream(temporal.toFile())) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
59	p.store(salida, "Cobro pendiente; no borrar hasta verificar el resultado"); salida.getFD().sync();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	}	Cierra el bloque abierto anteriormente.
61	Files.move(temporal, archivo.toAbsolutePath(), StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.ATOMIC_MOVE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	} finally { Files.deleteIfExists(temporal); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
63	}	Cierra el bloque abierto anteriormente.
64	public SolicitudPago cargar() throws IOException {	Declara el método cargar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
65	if (!Files.exists(archivo)) return null;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
66	Properties p = new Properties();	Crea una nueva instancia de Properties y la usa o guarda para continuar el proceso.
67	try(InputStream entrada=Files.newInputStream(archivo)) { p.load(entrada); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
68	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

69	List<LineaPedido> lineas=new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
70	int cantidad=Integer.parseInt(p.getProperty("cantidad"));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
71	if(cantidad < 1 cantidad > 10000) throw new IllegalArgumentException();	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
72	for(int i=0;i<cantidad;i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
73	String prefijo="linea."+i+ ".";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
74	List<OpcionPedido> opciones=new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
75	int cantidadOpciones=Integer.parseInt(p.getProperty(prefijo+"opciones"));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
76	for(int j=0;j<cantidadOpciones;j++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
77	String po=prefijo+"opcion."+j+ "."; List<ProductoPedido> productos=new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
78	int cantidadProductos=Integer.parseInt(p.getProperty(po+"productos"));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
79	for(int k=0;k<cantidadProductos;k++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
80	String pp=po+"producto."+k+ "."; List<ModificacionPedido> cambios=new ArrayList<>();	Crea una nueva instancia de ArrayList y la usa o guarda para continuar el proceso.
81	int cantidadCambios=Integer.parseInt(p.getProperty(pp+"cambios"));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
82	for(int m=0;m<cantidadCambios;m++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
83	String pm=pp+"cambio."+m+ ".";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
84	cambios.add(new ModificacionPedido(Integer.parseInt(p.getProperty(pm+"idProductoIngr ediente")),	Crea una nueva instancia de ModificacionPedido y la usa o guarda para continuar el proceso.
85	Integer.parseInt(p.getProperty(pm+"idIngrediente")),p.getProperty(pm+"tipo")	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	, p.getProperty(pm+"ingrediente"),new BigDecimal(p.getProperty(pm+"cantidad"))	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
87	, Integer.parseInt(p.getProperty(pm+"veces")),new BigDecimal(p.getProperty(pm+ "precio"))));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
88	}	Cierra el bloque abierto anteriormente.
89	productos.add(new ProductoPedido(p.getProperty(pp+"idInterno"), Integer.parseInt(p.getPro perty(pp+"id")),	Crea una nueva instancia de ProductoPedido y la usa o guarda para continuar el proceso.
90	p.getProperty(pp+"nombre"), Integer.parseInt(p.getProperty(pp+"cantidad")),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
91	p.getProperty(pp+"tipoStock"), cambios));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
92	}	Cierra el bloque abierto anteriormente.
93	opciones.add(new OpcionPedido(Integer.parseInt(p.getProperty(po+"idGrupo")),	Crea una nueva instancia de OpcionPedido y la usa o guarda para continuar el proceso.
94	Integer.parseInt(p.getProperty(po+"idOpcion")), Integer.parseInt(p.getProperty(po+"nu mero")),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	p.getProperty(po+"grupo"),p.getProperty(po+"opcion"),new BigDecimal(p.getProperty(po +"precio")),productos));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
96	}	Cierra el bloque abierto anteriormente.
97	lineas.add(new LineaPedido(p.getProperty(prefijo+"idLinea"), Integer.parseInt(p.getProperty(prefi jo+"idProducto")),	Crea una nueva instancia de LineaPedido y la usa o guarda para continuar el proceso.
98	Integer.parseInt(p.getProperty(prefijo+"idPresentacion")),p.getProperty(prefijo+"nombre"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
99	), p.getProperty(prefijo+"presentacion"),new BigDecimal(p.getProperty(prefijo+"precioBase")	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
100	), opciones, Integer.parseInt(p.getProperty(prefijo+"cantidad"))));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	}	Cierra el bloque abierto anteriormente.
102	return new SolicitudPago(p.getProperty("clave"), Integer.parseInt(p.getProperty("usuario")), lineas,	Finaliza el método y devuelve el valor indicado a quien lo llamó.
103	p.getProperty("servicio"),p.getProperty("metodo"),new BigDecimal(p.getProperty("recibido")), p.getProperty("referencia"));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
104	} catch(RuntimeException ex) { throw new IOException("El archivo de cobro pendiente no se puede leer. Ve rifique la venta antes de continuar.",ex); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
105	}	Cierra el bloque abierto anteriormente.

106	public void eliminar() throws IOException { Files.deleteIfExists(archivo); }	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
107	}	Cierra el bloque abierto anteriormente.

16.145 src/Utilidades/ReporteCsv.java

Utilidad compartida ReporteCsv. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 53 | Archivo 145 de 157

Clases/tipos detectados: ReporteCsv

Línea	Constructor o método detectado
9	private ReporteCsv() {}
11	public static String generar(LocalDate fecha, ReporteDAO.Resumen r) {
34	private static void seccion(StringBuilder b, String titulo, String[] columnas, List<Object[]> filas) {
41	private static void fila(StringBuilder b, Object... valores) {

1 package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import DAO.ReporteDAO;	Importa DAO.ReporteDAO para poder usar esa clase sin escribir su nombre completo.
4 import java.time.LocalDate;	Importa java.time.LocalDate: representa una fecha sin hora.
5 import java.util.List;	Importa java.util.List: colección ordenada que admite varios elementos.
6	Línea en blanco: separa visualmente bloques para que el código sea más legible.
7 /** Exporta el resultado consultado sin modificar datos. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
8 public final class ReporteCsv {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
9 private ReporteCsv() {}	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
10	Línea en blanco: separa visualmente bloques para que el código sea más legible.
11 public static String generar(LocalDate fecha, ReporteDAO.Resumen r) {	Declara el método generar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
12 StringBuilder b = new StringBuilder("\uFEFF");	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
13 fila(b, "Reporte de ventas", fecha);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
14 fila(b, "Ventas pagadas (Q)", r.ventas());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
15 fila(b, "Pedidos cobrados", r.pedidos());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
16 fila(b, "Ticket promedio pagado (Q)", r.ticket());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
17 fila(b, "Pedidos cancelados", r.cancelados());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
18 fila(b, "Efectivo (Q)", r.efectivo());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
19 fila(b, "Tarjeta (Q)", r.tarjeta());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
20 fila(b, "Otros m■todos (Q)", r.otros());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
21 fila(b, "Productos activos con stock bajo (actual)", r.stockBajo());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
22 if (!r.ventasDetalle().isEmpty()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
23 seccion(b, "Detalle de ventas", new String[]{"Orden", "Fecha/Hora",	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
24 "Cajero", "Servicio", "Estado", "M■todo de pago",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
25 "Monto recibido", "Total", "Cantidad de ■tems",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
26 r.ventasDetalle());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27 }	Cierra el bloque abierto anteriormente.
28 seccion(b, "Pedidos del día", new String[]{"Orden", "Cajero", "Hora", "Servicio", "Estado", "Total (Q)"}	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
29 , r.recientes());	
seccion(b, "10 productos más vendidos (líneas principales pagadas)", new String[]{"Producto", "Categoría", "Unidades", "Estado actual"}, r.productos());	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.

30	seccion(b, "Alertas de inventario actual", new String[]{"Tipo", "Nombre", "Existencias", "Mínimo", "Unidad", "Estado"}, r.alertas());	Crea una nueva instancia de String y la usa o guarda para continuar el proceso.
31	return b.toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
32	}	Cierra el bloque abierto anteriormente.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	private static void seccion(StringBuilder b, String titulo, String[] columnas, List<Object[]> filas) {	Declara el método seccion. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35	b.append("\r\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	fila(b, titulo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	fila(b, (Object[]) columnas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38	for (Object[] valores : filas) fila(b, valores);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
39	}	Cierra el bloque abierto anteriormente.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	private static void fila(StringBuilder b, Object... valores) {	Declara el método fila. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	for (int i = 0; i < valores.length; i++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
43	if (i > 0) b.append(',');	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
44	String texto = valores[i] == null ? "" : valores[i].toString();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
45	String inicio = texto.stripLeading();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
46	// Nombres que empiezan con operadores no deben ejecutarse como fórmulas.	Comentario para humanos; Java no ejecuta este texto. Nombres que empiezan con operadores no deben ejecutarse como fórmulas.
47	if (!(valores[i] instanceof Number) && !inicio.isEmpty()	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
48	&& "+-@".indexOf(inicio.charAt(0)) >= 0) texto = "" + texto;	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	b.append('').append(texto.replace("\n", "\\n")).append('');	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	}	Cierra el bloque abierto anteriormente.
51	b.append("\r\n");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52	}	Cierra el bloque abierto anteriormente.
53	}	Cierra el bloque abierto anteriormente.

16.146 src/Utilidades/SeguridadContrasena.java

Utilidad compartida SeguridadContrasena. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 105 | Archivo 146 de 157

Clases/tipos detectados: SeguridadContrasena

Línea	Constructor o método detectado
21	private SeguridadContrasena() {
24	public static String validarPolitica(String contrasena, String usuario) {
43	public static String hash(String contrasena) {
52	public static boolean verificar(String contrasena, String almacenada) {
73	private static byte[] derivar(char[] contrasena, byte[] sal, int iteraciones) {
89	public static String sha256(String contrasena) {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
4	import java.security.MessageDigest;	Importa java.security.MessageDigest para poder usar esa clase sin escribir su nombre completo.
5	import java.security.NoSuchAlgorithmException;	Importa java.security.NoSuchAlgorithmException para poder usar esa clase sin escribir su nombre completo.
6	import java.security.GeneralSecurityException;	Importa java.security.GeneralSecurityException para poder usar esa clase sin escribir su nombre completo.
7	import java.security.SecureRandom;	Importa java.security.SecureRandom para poder usar esa clase sin escribir su nombre completo.
8	import java.util.Base64;	Importa java.util.Base64 para poder usar esa clase sin escribir su nombre completo.
9	import javax.crypto.SecretKeyFactory;	Importa javax.crypto.SecretKeyFactory para poder usar esa clase sin escribir su nombre completo.
10	import javax.crypto.spec.PBEKeySpec;	Importa javax.crypto.spec.PBEKeySpec para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	/** Funciones compartidas por el login y el mantenimiento de usuarios. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	public final class SeguridadContrasena {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	public static final int LONGITUD_MINIMA = 8;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
16	private static final int ITERACIONES = 600_000;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
17	private static final int TAMANO_SAL = 16;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18	private static final int TAMANO_HASH = 32;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	private static final SecureRandom ALEATORIO = new SecureRandom();	Crea una nueva instancia de SecureRandom y la usa o guarda para continuar el proceso.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	private SeguridadContrasena() {	Constructor de SeguridadContrasena: prepara un objeto nuevo y deja sus componentes listos para usarse.
22	}	Cierra el bloque abierto anteriormente.
23		Línea en blanco: separa visualmente bloques para que el código sea más legible.
24	public static String validarPolitica(String contrasena, String usuario) {	Declara el método validarPolitica. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
25	if (contrasena == null contrasena.length() < LONGITUD_MINIMA) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
26	return "La contraseña debe tener al menos " + LONGITUD_MINIMA + " caracteres.";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
27	}	Cierra el bloque abierto anteriormente.
28	if (contrasena.chars().noneMatch(Character::isUpperCase)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.

29	return "La contraseña debe incluir al menos una letra mayúscula.";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
30	}	Cierra el bloque abierto anteriormente.
31	if (contrasena.chars().noneMatch(Character::isLowerCase)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
32	return "La contraseña debe incluir al menos una letra minúscula.";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
33	}	Cierra el bloque abierto anteriormente.
34	if (contrasena.chars().noneMatch(Character::isDigit)) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
35	return "La contraseña debe incluir al menos un número.";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
36	}	Cierra el bloque abierto anteriormente.
37	if (contrasena.chars().noneMatch(c -> !Character.isLetterOrDigit(c))) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
38	return "La contraseña debe incluir al menos un carácter especial.";	Finaliza el método y devuelve el valor indicado a quien lo llamó.
39	}	Cierra el bloque abierto anteriormente.
40	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
41	}	Cierra el bloque abierto anteriormente.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	public static String hash(String contrasena) {	Declara el método hash. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
44	byte[] sal = new byte[TAMANO_SAL];	Crea una nueva instancia de byte y la usa o guarda para continuar el proceso.
45	ALEATORIO.nextBytes(sal);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	byte[] derivado = derivar(contrasena.toCharArray(), sal, ITERACIONES);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
47	return "PBKDF2-SHA256\$" + ITERACIONES + "\$"	Finaliza el método y devuelve el valor indicado a quien lo llamó.
48	+ Base64.getEncoder().withoutPadding().encodeToString(sal) + "\$"	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	+ Base64.getEncoder().withoutPadding().encodeToString(derivado);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	}	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	public static boolean verificar(String contrasena, String almacenada) {	Declara el método verificar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	if (contrasena == null almacenada == null almacenada.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
54	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
55	}	Cierra el bloque abierto anteriormente.
56	if (!almacenada.startsWith("PBKDF2-SHA256\$")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
57	// Compatibilidad con cuentas antiguas; las nuevas nunca usan SHA-256 simple.	Comentario para humanos; Java no ejecuta este texto. Compatibilidad con cuentas antiguas; las nuevas nunca usan SHA-256 simple.
58	return MessageDigest.isEqual(sha256(contrasena).getBytes(), almacenada.getBytes());	Finaliza el método y devuelve el valor indicado a quien lo llamó.
59	}	Cierra el bloque abierto anteriormente.
60	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
61	String[] partes = almacenada.split("\\\$", -1);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
62	if (partes.length != 4) return false;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
63	int iteraciones = Integer.parseInt(partes[1]);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
64	byte[] sal = Base64.getDecoder().decode(partes[2]);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
65	byte[] esperado = Base64.getDecoder().decode(partes[3]);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	byte[] actual = derivar(contrasena.toCharArray(), sal, iteraciones);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	return MessageDigest.isEqual(actual, esperado);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
68	} catch (IllegalArgumentException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
69	return false;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
70	}	Cierra el bloque abierto anteriormente.
71	}	Cierra el bloque abierto anteriormente.
72		Línea en blanco: separa visualmente bloques para que el código sea más legible.

73	private static byte[] derivar(char[] contrasena, byte[] sal, int iteraciones) {	Declara el método derivar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
74	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
75	PBEKeySpec especificacion = new PBEKeySpec(	Crea una nueva instancia de PBEKeySpec y la usa o guarda para continuar el proceso.
76	contrasena, sal, iteraciones, TAMANO_HASH * 8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
77	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
78	return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")	Finaliza el método y devuelve el valor indicado a quien lo llamó.
79	.generateSecret(especificacion).getEncoded();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
81	especificacion.clearPassword();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	}	Cierra el bloque abierto anteriormente.
83	} catch (GeneralSecurityException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
84	throw new IllegalStateException("PBKDF2 no está disponible", ex);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
85	}	Cierra el bloque abierto anteriormente.
86	}	Cierra el bloque abierto anteriormente.
87		Línea en blanco: separa visualmente bloques para que el código sea más legible.
88	/** Compatibilidad de lectura para hashes antiguos. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
89	public static String sha256(String contrasena) {	Declara el método sha256. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
90	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
91	MessageDigest digest = MessageDigest.getInstance("SHA-256");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	byte[] hash = digest.digest(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
93	contrasena.getBytes(StandardCharsets.UTF_8));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	StringBuilder resultado = new StringBuilder(hash.length * 2);	Crea una nueva instancia de StringBuilder y la usa o guarda para continuar el proceso.
95	for (byte valor : hash) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
96	resultado.append(String.format("%02x", valor));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	}	Cierra el bloque abierto anteriormente.
98	return resultado.toString();	Finaliza el método y devuelve el valor indicado a quien lo llamó.
99	} catch (NoSuchAlgorithmException ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
100	// SHA-256 forma parte obligatoria de Java; esta excepción indica	Comentario para humanos; Java no ejecuta este texto. SHA-256 forma parte obligatoria de Java; esta excepción indica
101	// una instalación de Java dañada o no compatible.	Comentario para humanos; Java no ejecuta este texto. una instalación de Java dañada o no compatible.
102	throw new IllegalStateException("SHA-256 no está disponible", ex);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
103	}	Cierra el bloque abierto anteriormente.
104	}	Cierra el bloque abierto anteriormente.
105	}	Cierra el bloque abierto anteriormente.

16.147 src/Utilidades/SesionUsuario.java

Utilidad compartida SesionUsuario. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 43 | Archivo 147 de 157

Clases/tipos detectados: SesionUsuario

Línea	Constructor o método detectado
13	private SesionUsuario() {
16	public static void iniciar(
26	public static void cerrar() {
32	public static int getIdUsuario() {
36	public static String getNombreCompleto() {
40	public static String getRol() {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	/**	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
4	* Mantiene los datos mínimos del usuario autenticado durante la ejecución.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
5	* Permite impedir que un administrador se desactive a sí mismo.	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
6	*/	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
7	public final class SesionUsuario {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
8		Línea en blanco: separa visualmente bloques para que el código sea más legible.
9	private static int idUsuario;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
10	private static String nombreCompleto = "Administrador";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
11	private static String rol = "ADMINISTRADOR";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	private SesionUsuario() {	Constructor de SesionUsuario: prepara un objeto nuevo y deja sus componentes listos para usarse.
14	}	Cierra el bloque abierto anteriormente.
15		Línea en blanco: separa visualmente bloques para que el código sea más legible.
16	public static void iniciar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
17	int nuevoId,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18	String nombre,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
19	String apellido,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
20	String nuevoRol) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
21	idUsuario = nuevoId;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
22	nombreCompleto = (nombre + " " + apellido).trim();	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
23	rol = nuevoRol;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
24	}	Cierra el bloque abierto anteriormente.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	public static void cerrar() {	Declara el método cerrar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
27	idUsuario = 0;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
28	nombreCompleto = "Administrador";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.

29	rol = "ADMINISTRADOR";	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
30	}	Cierra el bloque abierto anteriormente.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	public static int getIdUsuario() {	Declara el método getIdUsuario. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
33	return idUsuario;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
34	}	Cierra el bloque abierto anteriormente.
35		Línea en blanco: separa visualmente bloques para que el código sea más legible.
36	public static String getNombreCompleto() {	Declara el método getNombreCompleto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
37	return nombreCompleto;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
38	}	Cierra el bloque abierto anteriormente.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	public static String getRol() {	Declara el método getRol. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
41	return rol;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
42	}	Cierra el bloque abierto anteriormente.
43	}	Cierra el bloque abierto anteriormente.

16.148 src/Utilidades/TemaAdmin.java

Utilidad compartida TemaAdmin. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 76 | Archivo 148 de 157

Clases/tipos detectados: TemaAdmin

Línea	Constructor o método detectado
21	public TemaAdmin() {
27	public int px(double valor) {
31	public Font regular(float tamano) {
35	public Font media(float tamano) {
39	public Font negrita(float tamano) {
43	private Font cargarFuente(String archivo, int estiloAlternativo) {
57	public static void aplicarCalidad(Graphics2D g2) {

1	package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.FontFormatException;	Importa java.awt.FontFormatException para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.GraphicsEnvironment;	Importa java.awt.GraphicsEnvironment para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
9	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
10	import java.io.InputStream;	Importa java.io.InputStream para poder usar esa clase sin escribir su nombre completo.
11		Línea en blanco: separa visualmente bloques para que el código sea más legible.
12	/** Tipografía y calidad gráfica utilizadas por las pantallas administrativas. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
13	public final class TemaAdmin {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
14		Línea en blanco: separa visualmente bloques para que el código sea más legible.
15	public static final Color AMARILLO = new Color(255, 188, 0);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
16		Línea en blanco: separa visualmente bloques para que el código sea más legible.
17	private final Font fuenteRegular;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
18	private final Font fuenteMedia;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
19	private final Font fuenteNegrita;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
20		Línea en blanco: separa visualmente bloques para que el código sea más legible.
21	public TemaAdmin() {	Constructor de TemaAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
22	fuenteRegular = cargarFuente("DMSans-Regular.ttf", Font.PLAIN);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
23	fuenteMedia = cargarFuente("DMSans-Medium.ttf", Font.PLAIN);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
24	fuenteNegrita = cargarFuente("DMSans-Bold.ttf", Font.BOLD);	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
25	}	Cierra el bloque abierto anteriormente.
26		Línea en blanco: separa visualmente bloques para que el código sea más legible.

27	public int px(double valor) {	Declara el método px. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
28	return Math.max(1, (int) Math.round(valor));	Finaliza el método y devuelve el valor indicado a quien lo llamó.
29	}	Cierra el bloque abierto anteriormente.
30		Línea en blanco: separa visualmente bloques para que el código sea más legible.
31	public Font regular(float tamano) {	Declara el método regular. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
32	return fuenteRegular.deriveFont(tamano);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
33	}	Cierra el bloque abierto anteriormente.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	public Font media(float tamano) {	Declara el método media. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
36	return fuenteMedia.deriveFont(tamano);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
37	}	Cierra el bloque abierto anteriormente.
38		Línea en blanco: separa visualmente bloques para que el código sea más legible.
39	public Font negrita(float tamano) {	Declara el método negrita. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
40	return fuenteNegrita.deriveFont(tamano);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
41	}	Cierra el bloque abierto anteriormente.
42		Línea en blanco: separa visualmente bloques para que el código sea más legible.
43	private Font cargarFuente(String archivo, int estiloAlternativo) {	Declara el método cargarFuente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
44	try (InputStream entrada = TemaAdmin.class.getResourceAsStream(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
45	"/Font/DMSans/" + archivo)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
46	if (entrada != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
47	Font fuente = Font.createFont(Font.TRUETYPE_FONT, entrada);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
48	GraphicsEnvironment.getLocalGraphicsEnvironment().registerFont(fuente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
49	return fuente;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
50	}	Cierra el bloque abierto anteriormente.
51	} catch (FontFormatException IOException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
52	System.err.println("No se pudo cargar " + archivo + ": " + e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53	}	Cierra el bloque abierto anteriormente.
54	return new Font("SansSerif", estiloAlternativo, 14);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
55	}	Cierra el bloque abierto anteriormente.
56		Línea en blanco: separa visualmente bloques para que el código sea más legible.
57	public static void aplicarCalidad(Graphics2D g2) {	Declara el método aplicarCalidad. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
58	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
59	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
63	RenderingHints.KEY_TEXT_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	RenderingHints.VALUE_TEXT_ANTIALIAS_LCD_HRGB	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
65	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
67	RenderingHints.KEY_RENDERING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
68	RenderingHints.VALUE_RENDER_QUALITY	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
69	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
70	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.

71	RenderingHints.KEY_STROKE_CONTROL,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	RenderingHints.VALUE_STROKE_PURE	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
74	}	Cierra el bloque abierto anteriormente.
75		Línea en blanco: separa visualmente bloques para que el código sea más legible.
76	}	Cierra el bloque abierto anteriormente.

16.149 src/Utilidades/UltimoComprobanteStore.java

Utilidad compartida UltimoComprobanteStore. Resuelve una tarea transversal usada por varias pantallas.

Tipo: java | Líneas: 46 | Archivo 149 de 157

Clases/tipos detectados: UltimoComprobanteStore

Línea	Constructor o método detectado
13	public UltimoComprobanteStore(int usuario) {
17	public UltimoComprobanteStore(Path archivo) {
21	public void guardar(String comprobante) throws IOException {
40	public String cargar() throws IOException {

1 package Utilidades;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2	Línea en blanco: separa visualmente bloques para que el código sea más legible.
3 import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
4 import java.nio.charset.StandardCharsets;	Importa java.nio.charset.StandardCharsets para poder usar esa clase sin escribir su nombre completo.
5 import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
6 import java.nio.file.Path;	Importa java.nio.file.Path para poder usar esa clase sin escribir su nombre completo.
7 import java.nio.file.StandardCopyOption;	Importa java.nio.file.StandardCopyOption para poder usar esa clase sin escribir su nombre completo.
8	Línea en blanco: separa visualmente bloques para que el código sea más legible.
9 /** Persiste el último comprobante confirmado para restaurarlo tras reiniciar la caja. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
10 public final class UltimoComprobanteStore {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11 private final Path archivo;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
12	Línea en blanco: separa visualmente bloques para que el código sea más legible.
13 public UltimoComprobanteStore(int usuario) {	Constructor de UltimoComprobanteStore: prepara un objeto nuevo y deja sus componentes listos para usarse.
14 this(Path.of("comprobantes_ultimo", "caja-" + usuario + ".txt"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
15 }	Cierra el bloque abierto anteriormente.
16	Línea en blanco: separa visualmente bloques para que el código sea más legible.
17 public UltimoComprobanteStore(Path archivo) {	Constructor de UltimoComprobanteStore: prepara un objeto nuevo y deja sus componentes listos para usarse.
18 this.archivo = archivo;	Accede al atributo del objeto actual para asignarlo o utilizarlo.
19 }	Cierra el bloque abierto anteriormente.
20	Línea en blanco: separa visualmente bloques para que el código sea más legible.
21 public void guardar(String comprobante) throws IOException {	Declara el método guardar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
22 if (comprobante == null comprobante.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
23 throw new IOException("El comprobante está vacío.");	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
24 }	Cierra el bloque abierto anteriormente.
25	Línea en blanco: separa visualmente bloques para que el código sea más legible.
26 Path destino = archivo.toAbsolutePath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27 Path directorio = destino.getParent();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28 Files.createDirectories(directorio);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29 Path temporal = Files.createTempFile(directorio, "comprobante-", ".tmp");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30 try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.

31	Files.writeString(temporal, comprobante, StandardCharsets.UTF_8);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	Files.move(temporal, destino,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
33	StandardCopyOption.REPLACE_EXISTING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	StandardCopyOption.ATOMIC_MOVE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
36	Files.deleteIfExists(temporal);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	}	Cierra el bloque abierto anteriormente.
38	}	Cierra el bloque abierto anteriormente.
39		Línea en blanco: separa visualmente bloques para que el código sea más legible.
40	public String cargar() throws IOException {	Declara el método cargar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
41	Path destino = archivo.toAbsolutePath();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
42	if (!Files.exists(destino)) return null;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
43	String comprobante = Files.readString(destino, StandardCharsets.UTF_8);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
44	return comprobante.isBlank() ? null : comprobante;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
45	}	Cierra el bloque abierto anteriormente.
46	}	Cierra el bloque abierto anteriormente.

16.150 src/Vistas/PantallaCarga.java

Archivo de apoyo del proyecto: src/Vistas/PantallaCarga.java.

Tipo: java | Líneas: 156 | Archivo 150 de 157

Clases/tipos detectados: PantallaCarga, FondoCarga

Línea	Constructor o método detectado
35	public PantallaCarga() {
41	private void configurarVentana() {
49	private void crearComponentes() {
78	private Font cargarFuente(String archivo, float tamano, int estiloAlternativo) {
91	private void iniciarAnimacionTexto() {
103	public void cerrar() {
121	protected void paintComponent(Graphics graphics) {
150	public static void main(String[] args) {

1	package Vistas;	Declara el paquete o carpeta lógica a la que pertenece esta clase.
2		Línea en blanco: separa visualmente bloques para que el código sea más legible.
3	import Componentes.CargaW;	Importa Componentes.CargaW para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.BasicStroke;	Importa java.awt.BasicStroke para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.Color;	Importa java.awt.Color para poder usar esa clase sin escribir su nombre completo.
6	import java.awt.Font;	Importa java.awt.Font para poder usar esa clase sin escribir su nombre completo.
7	import java.awt.FontFormatException;	Importa java.awt.FontFormatException para poder usar esa clase sin escribir su nombre completo.
8	import java.awt.GradientPaint;	Importa java.awt.GradientPaint para poder usar esa clase sin escribir su nombre completo.
9	import java.awt.Graphics;	Importa java.awt.Graphics para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.RenderingHints;	Importa java.awt.RenderingHints para poder usar esa clase sin escribir su nombre completo.
12	import java.io.IOException;	Importa java.io.IOException para poder usar esa clase sin escribir su nombre completo.
13	import java.io.InputStream;	Importa java.io.InputStream para poder usar esa clase sin escribir su nombre completo.
14	import javax.swing.JFrame;	Importa javax.swing.JFrame para poder usar esa clase sin escribir su nombre completo.
15	import javax.swing.JLabel;	Importa javax.swing.JLabel para poder usar esa clase sin escribir su nombre completo.
16	import javax.swing.JPanel;	Importa javax.swing.JPanel para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.SwingConstants;	Importa javax.swing.SwingConstants para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.Timer;	Importa javax.swing.Timer: ejecuta una acción periódica sin bloquear la interfaz Swing.
19		Línea en blanco: separa visualmente bloques para que el código sea más legible.
20	public class PantallaCarga extends JFrame {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
21		Línea en blanco: separa visualmente bloques para que el código sea más legible.
22	private static final int ANCHO_VENTANA = 560;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
23	private static final int ALTO_VENTANA = 360;	Declara una constante o referencia que no puede reasignarse después de inicializarse.
24	private static final Color ROJO_SUPERIOR = new Color(207, 31, 25);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
25	private static final Color ROJO_INFERIOR = new Color(166, 19, 18);	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.

26		Línea en blanco: separa visualmente bloques para que el código sea más legible.
27	private CargaW cargaw;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
28	private FondoCarga pnlFondo;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
29	private JLabel lblCargando;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	private JLabel lblDerechos;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
31		Línea en blanco: separa visualmente bloques para que el código sea más legible.
32	private Timer timerCargando;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33	private int frameText;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
34		Línea en blanco: separa visualmente bloques para que el código sea más legible.
35	public PantallaCarga() {	Constructor de PantallaCarga: prepara un objeto nuevo y deja sus componentes listos para usarse.
36	configurarVentana();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	crearComponentes();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38	iniciarAnimacionTexto();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	}	Cierra el bloque abierto anteriormente.
40		Línea en blanco: separa visualmente bloques para que el código sea más legible.
41	private void configurarVentana() {	Declara el método configurarVentana. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
42	setSize(ANCHO_VENTANA, ALTO_VENTANA);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	setResizable(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	setUndecorated(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	setLocationRelativeTo(null);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	}	Cierra el bloque abierto anteriormente.
48		Línea en blanco: separa visualmente bloques para que el código sea más legible.
49	private void crearComponentes() {	Declara el método crearComponentes. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
50	pnlFondo = new FondoCarga();	Crea una nueva instancia de FondoCarga y la usa o guarda para continuar el proceso.
51	pnlFondo.setLayout(null);	Configura una propiedad mediante setLayout en el objeto indicado.
52	setContentPane(pnlFondo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	cargaw = new CargaW();	Crea una nueva instancia de CargaW y la usa o guarda para continuar el proceso.
55	cargaw.setTamanoLogo(198);	Configura una propiedad mediante setTamanoLogo en el objeto indicado.
56	cargaw.setVelocidad(1.0f);	Configura una propiedad mediante setVelocidad en el objeto indicado.
57	cargaw.setAnchoLuz(70);	Configura una propiedad mediante setAnchoLuz en el objeto indicado.
58	cargaw.setOpacidadBase(0.20f);	Configura una propiedad mediante setOpacidadBase en el objeto indicado.
59	cargaw.setBounds(130, 25, 300, 205);	Configura una propiedad mediante setBounds en el objeto indicado.
60	pnlFondo.add(cargaw);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61		Línea en blanco: separa visualmente bloques para que el código sea más legible.
62	lblCargando = new JLabel("Preparando tu experiencia", SwingConstants.CENTER);	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.
63	lblCargando.setFont(cargarFuente("DMSans-Bold.ttf", 15f, Font.BOLD));	Configura una propiedad mediante setFont en el objeto indicado.
64	lblCargando.setForeground(Color.WHITE);	Configura una propiedad mediante setForeground en el objeto indicado.
65	lblCargando.setBounds(100, 238, 360, 28);	Configura una propiedad mediante setBounds en el objeto indicado.
66	pnlFondo.add(lblCargando);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67		Línea en blanco: separa visualmente bloques para que el código sea más legible.
68	lblDerechos = new JLabel(	Crea una nueva instancia de JLabel y la usa o guarda para continuar el proceso.

69	"\u00A9 2026. Todos los derechos reservados.",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
70	SwingConstants.CENTER	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
71	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
72	lblDerechos.setFont(cargarFuente("DMSans-Regular.ttf", 11f, Font.PLAIN));	Configura una propiedad mediante setFont en el objeto indicado.
73	lblDerechos.setForeground(new Color(255, 218, 214));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
74	lblDerechos.setBounds(60, 306, 440, 22);	Configura una propiedad mediante setBounds en el objeto indicado.
75	pnlFondo.add(lblDerechos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
76	}	Cierra el bloque abierto anteriormente.
77		Línea en blanco: separa visualmente bloques para que el código sea más legible.
78	private Font cargarFuente(String archivo, float tamano, int estiloAlternativo) {	Declara el método cargarFuente. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
79	try (InputStream fuente = getClass().getResourceAsStream(	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
80	"/Font/DMSans/" + archivo)) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
81	if (fuente != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
82	return Font.createFont(Font.TRUETYPE_FONT, fuente).deriveFont(tamano);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
83	}	Cierra el bloque abierto anteriormente.
84	} catch (FontFormatException IOException e) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
85	System.err.println("No se pudo cargar la fuente " + archivo + ": "	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	+ e.getMessage());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	}	Cierra el bloque abierto anteriormente.
88	return new Font("SansSerif", estiloAlternativo, Math.round(tamano));	Finaliza el método y devuelve el valor indicado a quien lo llamó.
89	}	Cierra el bloque abierto anteriormente.
90		Línea en blanco: separa visualmente bloques para que el código sea más legible.
91	private void iniciarAnimacionTexto() {	Declara el método iniciarAnimacionTexto. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
92	timerCargando = new Timer(360, e -> {	Crea una nueva instancia de Timer y la usa o guarda para continuar el proceso.
93	frameTexto = (frameTexto + 1) % 4;	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
94	lblCargando.setText(	Cambia el texto visible de un componente de la interfaz.
95	"Preparando tu experiencia" + ".".repeat(frameTexto)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
96	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
97	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	timerCargando.setCoalesce(true);	Configura una propiedad mediante setCoalesce en el objeto indicado.
99	timerCargando.start();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	}	Cierra el bloque abierto anteriormente.
101		Línea en blanco: separa visualmente bloques para que el código sea más legible.
102	/** Detiene los timers y libera la ventana al pasar al login. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
103	public void cerrar() {	Declara el método cerrar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
104	if (timerCargando != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
105	timerCargando.stop();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
106	}	Cierra el bloque abierto anteriormente.
107	if (cargaw != null) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
108	cargaw.detener();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
109	}	Cierra el bloque abierto anteriormente.
110	setVisible(false);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
111	dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	}	Cierra el bloque abierto anteriormente.

113		Línea en blanco: separa visualmente bloques para que el código sea más legible.
114	private static class FondoCarga extends JPanel {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
115		Línea en blanco: separa visualmente bloques para que el código sea más legible.
116	FondoCarga() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
117	setOpaque(true);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	}	Cierra el bloque abierto anteriormente.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	@Override	Anotación de Java: agrega metadatos o indica al compilador cómo tratar el elemento siguiente.
121	protected void paintComponent(Graphics graphics) {	Declara el método paintComponent. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
122	Graphics2D g2 = (Graphics2D) graphics.create();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
123	g2.setRenderingHint(	Configura una propiedad mediante setRenderingHint en el objeto indicado.
124	RenderingHints.KEY_ANTIALIASING,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125	RenderingHints.VALUE_ANTIALIAS_ON	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127		Línea en blanco: separa visualmente bloques para que el código sea más legible.
128	g2.setPaint(new GradientPaint(	Crea una nueva instancia de GradientPaint y la usa o guarda para continuar el proceso.
129	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
130	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
131	ROJO_SUPERIOR,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	getHeight(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
134	ROJO_INFERIOR	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
136	g2.fillRect(0, 0, getWidth(), getHeight());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137		Línea en blanco: separa visualmente bloques para que el código sea más legible.
138	g2.setColor(new Color(255, 255, 255, 12));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
139	g2.fillOval(-92, -126, 270, 270);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
140	g2.setStroke(new BasicStroke(2f));	Crea una nueva instancia de BasicStroke y la usa o guarda para continuar el proceso.
141	g2.setColor(new Color(255, 199, 44, 26));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
142	g2.drawOval(getWidth() - 146, getHeight() - 138, 210, 210);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143		Línea en blanco: separa visualmente bloques para que el código sea más legible.
144	g2.setColor(new Color(255, 255, 255, 16));	Crea una nueva instancia de Color y la usa o guarda para continuar el proceso.
145	g2.fillRoundRect(85, 288, getWidth() - 170, 1, 1, 1);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146	g2.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
147	}	Cierra el bloque abierto anteriormente.
148	}	Cierra el bloque abierto anteriormente.
149		Línea en blanco: separa visualmente bloques para que el código sea más legible.
150	public static void main(String[] args) {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
151	java.awt.EventQueue.invokeLater(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
152	PantallaCarga pantalla = new PantallaCarga();	Crea una nueva instancia de PantallaCarga y la usa o guarda para continuar el proceso.
153	pantalla.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
154	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
155	}	Cierra el bloque abierto anteriormente.
156	}	Cierra el bloque abierto anteriormente.

16.151 test/FormReportesTest.java

Prueba ejecutable FormReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 73 | Archivo 151 de 157

Clases/tipos detectados: FormReportesTest

Línea	Constructor o método detectado
13	private static void comprobar(boolean condicion, String mensaje) {
19	public static void main(String[] args) throws Exception {
60	private static void registrar(

1	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
2	import java.nio.file.Files;	Importa java.nio.file.Files para poder usar esa clase sin escribir su nombre completo.
3	import java.util.HashMap;	Importa java.util.HashMap para poder usar esa clase sin escribir su nombre completo.
4	import java.util.Map;	Importa java.util.Map: colección de pares clave-valor.
5	import javax.xml.parsers.DocumentBuilderFactory;	Importa javax.xml.parsers.DocumentBuilderFactory para poder usar esa clase sin escribir su nombre completo.
6	import org.w3c.dom.Document;	Importa org.w3c.dom.Document para poder usar esa clase sin escribir su nombre completo.
7	import org.w3c.dom.Element;	Importa org.w3c.dom.Element para poder usar esa clase sin escribir su nombre completo.
8	import org.w3c.dom.NodeList;	Importa org.w3c.dom.NodeList para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	/** Comprueba que el formulario visual y el código de Reportes sigan sincronizados. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
11	public class FormReportesTest {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
12		Línea en blanco: separa visualmente bloques para que el código sea más legible.
13	private static void comprobar(boolean condicion, String mensaje) {	Declara el método comprobar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
14	if (!condicion) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
15	throw new AssertionError(mensaje);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
16	}	Cierra el bloque abierto anteriormente.
17	}	Cierra el bloque abierto anteriormente.
18		Línea en blanco: separa visualmente bloques para que el código sea más legible.
19	public static void main(String[] args) throws Exception {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
20	File archivoForm = new File(	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
21	"src/GUI_ADMINISTRADOR/ReportesPanel.form");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
22	Document documento = DocumentBuilderFactory.newInstance()	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	.newDocumentBuilder().parse(archivoForm);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
24		Línea en blanco: separa visualmente bloques para que el código sea más legible.
25	Map<String, String> componentes = new HashMap<>();	Crea una nueva instancia de HashMap y la usa o guarda para continuar el proceso.
26	registrar(documento.getElementsByTagName("Component"), componentes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
27	registrar(documento.getElementsByTagName("Container"), componentes);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
28		Línea en blanco: separa visualmente bloques para que el código sea más legible.
29	Map<String, String> requeridos = Map.ofEntries(	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
30	Map.entry("botonActualizar", "Componentes.BotonDerretido"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
31	Map.entry("campoBusqueda", "Componentes.CampoBusquedaAdmin"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
32	Map.entry("filtroCajero", "Componentes.BotonDesplegable"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

33	Map.entry("filtroMetodo", "Componentes.BotonDesplegable"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	Map.entry("filtroServicio", "Componentes.BotonDesplegable"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
35	Map.entry("botonLimpiarFiltros", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
36	Map.entry("tablaPedidos", "Componentes.TablaAdministrativa"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
37	Map.entry("botonAnterior", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
38	Map.entry("botonPagina1", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	Map.entry("botonPagina2", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	Map.entry("botonPagina3", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
41	Map.entry("botonSiguiente", "Componentes.BotonRedondeado"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	Map.entry("panelResumen", "Componentes.PanelFlotante"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43		Línea en blanco: separa visualmente bloques para que el código sea más legible.
44	String fuente = Files.readString(new File(	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
45	"src/GUI_ADMINISTRADOR/ReportesPanel.java").toPath());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	for (Map.Entry<String, String> requerido : requeridos.entrySet()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
47	comprobar(requerido.getValue().equals(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
48	componentes.get(requerido.getKey()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
49	"Componente incorrecto o ausente: " + requerido.getKey());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	comprobar(fuente.contains(requerido.getKey()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
51	"El código no declara: " + requerido.getKey());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52	}	Cierra el bloque abierto anteriormente.
53		Línea en blanco: separa visualmente bloques para que el código sea más legible.
54	comprobar(documento.getElementsByTagName("Table").getLength() == 1,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
55	"Reportes debe tener una sola tabla principal.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	System.out.println(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
57	"OK: ReportesPanel.form es editable y contiene filtros, tabla y paginación.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	}	Cierra el bloque abierto anteriormente.
59		Línea en blanco: separa visualmente bloques para que el código sea más legible.
60	private static void registrar(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
61	NodeList nodos, Map<String, String> componentes) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
62	for (int indice = 0; indice < nodos.getLength(); indice++) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
63	Elemento:	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
64	{	Abre un bloque de instrucciones perteneciente a la declaración anterior.
65	Element elemento = (Element) nodos.item(indice);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	String nombre = elemento.getAttribute("name");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
67	if (!nombre.isBlank()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
68	componentes.put(nombre, elemento.getAttribute("class"));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	}	Cierra el bloque abierto anteriormente.
70	}	Cierra el bloque abierto anteriormente.
71	}	Cierra el bloque abierto anteriormente.
72	}	Cierra el bloque abierto anteriormente.
73	}	Cierra el bloque abierto anteriormente.

16.152 test/InventarioPanelTest.java

Prueba ejecutable InventarioPanelTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 189 | Archivo 152 de 157

Clases/tipos detectados: InventarioPanelTest

Línea	Constructor o método detectado	
28	private static Object campo(Object objeto, String nombre) throws Exception {	
34	private static void verificar(boolean condicion, String mensaje) {	
38	private static <T extends Component> T buscarComponente(	
52	private static void capturar(Component componente, String nombre)	
66	private static void acomodar(Component componente) {	
72	public static void main(String[] args) throws Exception {	
1	import GUI_ADMINISTRADOR.CatalogoIngredientesPanel;	Importa GUI_ADMINISTRADOR.CatalogoIngredientesPanel para poder usar esa clase sin escribir su nombre completo.
2	import GUI_ADMINISTRADOR.GestionInventarioPanel;	Importa GUI_ADMINISTRADOR.GestionInventarioPanel para poder usar esa clase sin escribir su nombre completo.
3	import GUI_ADMINISTRADOR.InventarioPanel;	Importa GUI_ADMINISTRADOR.InventarioPanel para poder usar esa clase sin escribir su nombre completo.
4	import GUI_ADMINISTRADOR.IngredienteFormPanel;	Importa GUI_ADMINISTRADOR.IngredienteFormPanel para poder usar esa clase sin escribir su nombre completo.
5	import GUI_ADMINISTRADOR.MenuAdmin;	Importa GUI_ADMINISTRADOR.MenuAdmin para poder usar esa clase sin escribir su nombre completo.
6	import GUI_ADMINISTRADOR.MovimientoInventarioFormPanel;	Importa GUI_ADMINISTRADOR.MovimientoInventarioFormPanel para poder usar esa clase sin escribir su nombre completo.
7	import GUI_ADMINISTRADOR.MovimientosInventarioPanel;	Importa GUI_ADMINISTRADOR.MovimientosInventarioPanel para poder usar esa clase sin escribir su nombre completo.
8	import Modelos.ArticuloInventario;	Importa Modelos.ArticuloInventario para poder usar esa clase sin escribir su nombre completo.
9	import Modelos.Ingrediente;	Importa Modelos.Ingrediente para poder usar esa clase sin escribir su nombre completo.
10	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
11	import java.awt.Container;	Importa java.awt.Container para poder usar esa clase sin escribir su nombre completo.
12	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
13	import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
14	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
15	import java.math.BigDecimal;	Importa java.math.BigDecimal: maneja dinero y cantidades decimales sin los errores de aproximación de double.
16	import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
17	import javax.swing.JButton;	Importa javax.swing.JButton para poder usar esa clase sin escribir su nombre completo.
18	import javax.swing.JFrame;	Importa javax.swing.JFrame para poder usar esa clase sin escribir su nombre completo.
19	import javax.swing.JLayeredPane;	Importa javax.swing.JLayeredPane para poder usar esa clase sin escribir su nombre completo.
20	import javax.swing.JTable;	Importa javax.swing.JTable para poder usar esa clase sin escribir su nombre completo.
21	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
22		Línea en blanco: separa visualmente bloques para que el código sea más legible.
23	/** Comprobación visual y de datos del módulo de inventario. */	Parte de un comentario de bloque o documentación Javadoc; describe intención y no se ejecuta.
24	public class InventarioPanelTest {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
25		Línea en blanco: separa visualmente bloques para que el código sea más legible.
26	private static MenuAdmin menu;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
27		Línea en blanco: separa visualmente bloques para que el código sea más legible.
28	private static Object campo(Object objeto, String nombre) throws Exception {	Declara el método campo. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

29	var campo = objeto.getClass().getDeclaredField(nombre);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
30	campo.setAccessible(true);	Configura una propiedad mediante setAccessible en el objeto indicado.
31	return campo.get(objeto);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
32	}	Cierra el bloque abierto anteriormente.
33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	private static void verificar(boolean condicion, String mensaje) {	Declara el método verificar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35	if (!condicion) throw new AssertionError(mensaje);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
36	}	Cierra el bloque abierto anteriormente.
37		Línea en blanco: separa visualmente bloques para que el código sea más legible.
38	private static <T extends Component> T buscarComponente(	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	Container contenedor, Class<T> tipo) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
40	for (Component componente : contenedor.getComponents()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
41	if (tipo.isInstance(componente) && componente.isVisible()) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
42	return tipo.cast(componente);	Finaliza el método y devuelve el valor indicado a quien lo llamó.
43	}	Cierra el bloque abierto anteriormente.
44	if (componente instanceof Container hijo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
45	T encontrado = buscarComponente(hijo, tipo);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
46	if (encontrado != null) return encontrado;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
47	}	Cierra el bloque abierto anteriormente.
48	}	Cierra el bloque abierto anteriormente.
49	return null;	Finaliza el método y devuelve el valor indicado a quien lo llamó.
50	}	Cierra el bloque abierto anteriormente.
51		Línea en blanco: separa visualmente bloques para que el código sea más legible.
52	private static void capturar(Component componente, String nombre)	Declara el método capturar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
53	throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
54	componente.setSize(componente.getPreferredSize());	Configura una propiedad mediante setSize en el objeto indicado.
55	acomodar(componente);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	BufferedImage imagen = new BufferedImage(componente.getWidth(),	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
57	componente.getHeight(), BufferedImage.TYPE_INT_ARGB);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
58	Graphics2D graficos = imagen.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
59	componente.printAll(graficos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
60	graficos.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
61	File salida = new File("build/inventory-check/" + nombre);	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
62	salida.getParentFile().mkdirs();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	ImageIO.write(imagen, "png", salida);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
64	}	Cierra el bloque abierto anteriormente.
65		Línea en blanco: separa visualmente bloques para que el código sea más legible.
66	private static void acomodar(Component componente) {	Declara el método acomodar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
67	if (!(componente instanceof Container contenedor)) return;	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
68	contenedor.doLayout();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
69	for (Component hijo : contenedor.getComponents()) acomodar(hijo);	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
70	}	Cierra el bloque abierto anteriormente.
71		Línea en blanco: separa visualmente bloques para que el código sea más legible.
72	public static void main(String[] args) throws Exception {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.

73	SwingUtilities.invokeLaterAndWait(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
74	menu = new MenuAdmin();	Crea una nueva instancia de MenuAdmin y la usa o guarda para continuar el proceso.
75	menu.setExtendedState(JFrame.NORMAL);	Configura una propiedad mediante setExtendedState en el objeto indicado.
76	menu.setSize(1920, 1080);	Configura una propiedad mediante setSize en el objeto indicado.
77	menu.setVisible(true);	Muestra u oculta una ventana o componente según el valor booleano enviado.
78	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
79	((JButton) campo(menu, "botonInventario")).doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
81	throw new RuntimeException(ex);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
82	}	Cierra el bloque abierto anteriormente.
83	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	Thread.sleep(1500);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	SwingUtilities.invokeLaterAndWait(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
86	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
87	Container contenido = (Container) campo(menu, "panelContenido");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
88	InventarioPanel modulo = buscarComponente(	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
89	contenido, InventarioPanel.class);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	verificar(modulo != null, "No se abrió la tarjeta Inventario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	GestionInventarioPanel inventario =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
92	(GestionInventarioPanel) campo(modulo, "existencias");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
93	JTable tabla = (JTable) campo(inventario, "tablaProductos");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
94	verificar(tabla.getRowCount() > 0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
95	"La tabla de inventario no cargó filas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
96	verificar(tabla.getValueAt(0, 1) instanceof String,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
97	"El nombre debe mostrarse como texto, sin imagen");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
98	verificar(campo(modulo, "botonIngredientes") instanceof JButton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
99	"Falta la vista interna de Ingredientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	verificar(campo(modulo, "botonMovimientos") instanceof JButton,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
101	"Falta la vista interna de Movimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102		Línea en blanco: separa visualmente bloques para que el código sea más legible.
103	JButton botonIngredientes =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
104	(JButton) campo(modulo, "botonIngredientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
105	JButton botonExistencias =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
106	(JButton) campo(modulo, "botonExistencias");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
107	botonIngredientes.doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
108	verificar(botonIngredientes.isShowing(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
109	"La pestaña Ingredientes desapareció al seleccionarla");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
110	CatalogoIngredientesPanel catalogo =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	(CatalogoIngredientesPanel) campo(modulo, "ingredientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
112	verificar(catalogo.isVisible(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
113	"No se abrió la vista interna de Ingredientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	JTable tablaIngredientes =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
115	(JTable) campo(catalogo, "tablaIngredientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
116	verificar(tablaIngredientes.getRowCount() > 0,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.

117	"El catálogo de ingredientes no cargó filas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
118	capturar(modulo, "inventario-ingredientes.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
119		Línea en blanco: separa visualmente bloques para que el código sea más legible.
120	JButton botonMovimientos =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
121	(JButton) campo(modulo, "botonMovimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
122	botonMovimientos.doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	verificar(botonIngredientes.isShowing()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
124	&& botonMovimientos.isShowing()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
125	&& botonExistencias.isShowing(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
126	"Las pestañas desaparecieron al abrir Movimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	verificar("Existencias".equals(botonExistencias.getText()),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
128	"La pestaña Existencias perdió su texto");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
129	JLayeredPane capas = (JLayeredPane) campo(modulo, "panelCapas");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
130	Component vistas = (Component) campo(modulo, "panelVistas");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	verificar(capas.getLayer(botonExistencias)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
132	> capas.getLayer(vistas),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
133	"Las pestañas no están en una capa superior");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
134	verificar(capas.getComponentZOrder(botonExistencias)	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
135	< capas.getComponentZOrder(vistas),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
136	"Existencias quedó detrás del panel de vistas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	MovimientosInventarioPanel historial =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
138	(MovimientosInventarioPanel) campo(modulo, "movimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
139	verificar(historial.isVisible(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
140	"No se abrió la vista interna de Movimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	JTable tablaMovimientos =	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
142	(JTable) campo(historial, "tablaMovimientos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
143	verificar(tablaMovimientos.getColumnCount() == 8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
144	"La tabla de movimientos no tiene la estructura esperada");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
145	capturar(modulo, "inventario-movimientos.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
146		Línea en blanco: separa visualmente bloques para que el código sea más legible.
147	((JButton) campo(modulo, "botonExistencias")).doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
148	verificar(botonIngredientes.isShowing()	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
149	&& botonMovimientos.isShowing(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
150	"Las pestañas desaparecieron al volver a Existencias");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
151		Línea en blanco: separa visualmente bloques para que el código sea más legible.
152	Container raiz = menu.getContentPane();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
153	BufferedImage imagen = new BufferedImage(	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
154	raiz.getWidth(), raiz.getHeight(),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
155	BufferedImage.TYPE_INT_RGB);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
156	Graphics2D graficos = imagen.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
157	raiz.printAll(graficos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
158	graficos.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
159	File salida = new File(	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
160	"build/inventory-check/inventario-panel.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.

161	salida.getParentFile().mkdirs();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	ImageIO.write(imagen, "png", salida);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
163		Línea en blanco: separa visualmente bloques para que el código sea más legible.
164	Ingrediente ejemplo = new Ingrediente();	Crea una nueva instancia de Ingrediente y la usa o guarda para continuar el proceso.
165	ejemplo.setIdIngrediente(7);	Configura una propiedad mediante setIdIngrediente en el objeto indicado.
166	ejemplo.setNombre("Carne de res");	Configura una propiedad mediante setNombre en el objeto indicado.
167	ejemplo.setUnidadMedida("unidad");	Configura una propiedad mediante setUnidadMedida en el objeto indicado.
168	ejemplo.setStockActual(new BigDecimal("1500"));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
169	ejemplo.setStockMinimo(new BigDecimal("150"));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
170	ejemplo.setActivo(true);	Configura una propiedad mediante setActivo en el objeto indicado.
171	capturar(new IngredienteFormPanel(null, ejemplo),	Crea una nueva instancia de IngredienteFormPanel y la usa o guarda para continuar el proceso.
172	"ingrediente-form.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
173		Línea en blanco: separa visualmente bloques para que el código sea más legible.
174	ArticuloInventario articulo = new ArticuloInventario(	Crea una nueva instancia de ArticuloInventario y la usa o guarda para continuar el proceso.
175	"INGREDIENTE", 7, "Carne de res", "Ingredientes",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
176	"unidad", new BigDecimal("1500"),	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
177	new BigDecimal("150"), true, null);	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
178	capturar(new MovimientoInventarioFormPanel(	Crea una nueva instancia de MovimientoInventarioFormPanel y la usa o guarda para continuar el proceso.
179	null, articulo, "ENTRADA"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
180	"movimiento-form.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
181	menu.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
182	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
183	throw new RuntimeException(ex);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
184	}	Cierra el bloque abierto anteriormente.
185	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
186	System.out.println("OK: navegación, datos y nombres sin imágenes.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
187	System.exit(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
188	}	Cierra el bloque abierto anteriormente.
189	}	Cierra el bloque abierto anteriormente.

16.153 test/MenuAnimacionTest.java

Prueba ejecutable MenuAnimacionTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 49 | Archivo 153 de 157

Clases/tipos detectados: MenuAnimacionTest

Línea	Constructor o método detectado
31	public static void main(String[] args) throws Exception {
1	import GUI_ADMINISTRADOR.MenuAdmin;
2	import Componentes.BotonMenuLateral;
3	import java.awt.*;
4	import java.awt.image.BufferedImage;
5	import javax.swing.*;
6	import javax.imageio.ImageIO;
7	import java.io.File;
8	
9	public class MenuAnimacionTest {
10	static MenuAdmin menu;
11	static Object campo(String n) throws Exception { var f=MenuAdmin.class.getDeclaredField(n); f.setAccessible(true); return f.get(menu); }
12	static void llamar(String n) throws Exception { var m=MenuAdmin.class.getDeclaredMethod(n);m.setAccessible(true);m.invoke(menu); }
13	static void edt(Runnable r) throws Exception { SwingUtilities.invokeLaterAndWait(r); }
14	static void espera() throws Exception { Thread.sleep(450); edt(()->{}); }
15	static void verificar(boolean ok,String m) { if(!ok) throw new AssertionError(m); }
16	static void capturar(String nombre) throws Exception {
17	Container c=menu.getContentPane();
18	BufferedImage b=new BufferedImage(c.getWidth(),c.getHeight(),BufferedImage.TYPE_INT_RGB);
19	Graphics2D g=b.createGraphics();c.printAll(g);g.dispose();
20	File salida=new File("build/reportes-check/"+nombre+".png");
21	salida.getParentFile().mkdirs();
22	ImageIO.write(b,"png",salida);
23	}
24	static void ancho(boolean abierto) throws Exception {
25	JPanel lateral=(JPanel)campo("panelLateral");
26	int esperado=abierto?340:90;
27	verificar(Math.abs(lateral.getWidth()-esperado)<=2,"Ancho lateral incorrecto: "+lateral.getWidth()+" esperado "+esperado);
28	verificar(Math.abs((double)campo("aperturaMenu")-(abierto?1:0))<0.0001,"Estado final");
29	verificar(!((BotonMenuLateral)campo("botonReportes")).getText().isBlank(),"Texto accesible conservado");
30	}
31	public static void main(String[] args) throws Exception {
32	edt(()->{menu=new MenuAdmin();menu.setExtendedState(JFrame.NORMAL);menu.setSize(1920,1080);menu.setVisible(true);}); espera();

33	edt()->{try {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
34	((JButton)campo("botonUsuarios")).doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	capturar("usuarios-completo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	((JButton)campo("botonGestionMenu")).doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	capturar("gestion-menu-completo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38	((JButton)campo("botonReportes")).doClick();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	ancho(true); capturar("menu-abierto"); llamar("alternarMenuLateral");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	}catch(Exception ex){throw new RuntimeException(ex);}}; espera();	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
41	edt()->{try { ancho(false);capturar("menu-cerrado");llamar("alternarMenuLateral");}catch(Exception ex){	Crea una nueva instancia de RuntimeException y la usa o guarda para continuar el proceso.
	throw new RuntimeException(ex);}}; espera();	
42	edt()->{try {ancho(true);capturar("menu-abierto-final");llamar("alternarMenuLateral");}catch(Exception	Crea una nueva instancia de RuntimeException y la usa o guarda para continuar el proceso.
	ex){throw new RuntimeException(ex);}};	
43	Thread.sleep(90);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	edt()->{try {llamar("alternarMenuLateral");}catch(Exception ex){throw new RuntimeException(ex);}}; esp	Crea una nueva instancia de RuntimeException y la usa o guarda para continuar el proceso.
	era();	
45	edt()->{try {ancho(true);menu.dispose();}catch(Exception ex){throw new RuntimeException(ex);}};	Crea una nueva instancia de RuntimeException y la usa o guarda para continuar el proceso.
46	System.out.println("OK: abrir/cerrar, invertir durante animación y conservar textos accesibles sin escal	Ejecuta una instrucción Java completa; el punto y coma marca su final.
	ado.");	
47	System.exit(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	}	Cierra el bloque abierto anteriormente.
49	}	Cierra el bloque abierto anteriormente.

16.154 test/PagosTest.java

Prueba ejecutable PagosTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 167 | Archivo 154 de 157

Clases/tipos detectados: PagosTest

Línea	Constructor o método detectado
63	public static void main(String[] args) throws Exception {
1	import DAO.PagoDAO;
2	import Modelos.*;
3	import GUI_CAJERO.PedidoPanel;
4	import java.math.BigDecimal;
5	import java.sql.*;
6	import java.nio.file.*;
7	import java.util.*;
8	import java.util.concurrent.*;
9	import java.lang.reflect.*;
10	Línea en blanco: separa visualmente bloques para que el código sea más legible.
11	/** Integra contra MySQL real, exclusivamente en una base temporal nueva. */
12	public class PagosTest {
13	static String base;
14	static int pruebas;
15	static Connection abrir() throws SQLException {
16	Connection c = Conexion.Conexion.conectar();
17	if (c == null) throw new SQLException("MySQL no disponible");
18	c.setCatalog(base); return c;
19	}
20	static void sql(String texto) throws SQLException {
21	try(Connection c=abrir(); Statement s=c.createStatement()) { s.execute(texto); }
22	}
23	static int numero(String texto) throws SQLException {
24	try(Connection c=abrir(); Statement s=c.createStatement(); ResultSet r=s.executeQuery(texto)) { r.next(); return r.getInt(1); }
25	}
26	static void comprobar(boolean ok, String nombre) {
27	if(!ok) throw new AssertionError(nombre);
28	System.out.println("OK: " + nombre); pruebas++;
29	}
30	static SolicitudPago pago(int cantidad) {
31	ProductoPedido producto = new ProductoPedido(UUID.randomUUID().toString(),1,
32	"Producto de prueba",1,"DIRECTO",List.of());
33	OpcionPedido opcion = new OpcionPedido(1,1,1,"_Producto principal",

34	"Producto de prueba",BigDecimal.ZERO,List.of(producto));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	LineaPedido linea = new LineaPedido(UUID.randomUUID().toString(),1,1,	Crea una nueva instancia de LineaPedido y la usa o guarda para continuar el proceso.
36	"Producto de prueba","Individual",new BigDecimal("10.25"),	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
37	List.of(opcion),cantidad);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
38	return new SolicitudPago(UUID.randomUUID().toString(),1,List.of(linea),	Finaliza el método y devuelve el valor indicado a quien lo llamó.
39	"COMER_AQUI", "EFECTIVO",new BigDecimal("100.00"),"");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
40	}	Cierra el bloque abierto anteriormente.
41	static SolicitudPago comboPersonalizado() {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
42	ModificacionPedido sin = new ModificacionPedido(1,1,"SIN","Pepinillo",	Crea una nueva instancia de ModificacionPedido y la usa o guarda para continuar el proceso.
43	BigDecimal.ONE,1,BigDecimal.ZERO);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	ModificacionPedido extra = new ModificacionPedido(1,1,"EXTRA","Pepinillo",	Crea una nueva instancia de ModificacionPedido y la usa o guarda para continuar el proceso.
45	BigDecimal.ONE,1,new BigDecimal("2.00"));	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
46	ProductoPedido primero = new ProductoPedido(UUID.randomUUID().toString(),2,	Crea una nueva instancia de ProductoPedido y la usa o guarda para continuar el proceso.
47	"Hamburguesa",1,"RECETA",List.of(sin));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
48	ProductoPedido segundo = new ProductoPedido(UUID.randomUUID().toString(),2,	Crea una nueva instancia de ProductoPedido y la usa o guarda para continuar el proceso.
49	"Hamburguesa",1,"RECETA",List.of(extra));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	OpcionPedido uno = new OpcionPedido(2,2,1,"Elige tus hamburguesas",	Crea una nueva instancia de OpcionPedido y la usa o guarda para continuar el proceso.
51	"Hamburguesa",BigDecimal.ZERO,List.of(primero));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
52	OpcionPedido dos = new OpcionPedido(2,2,2,"Elige tus hamburguesas",	Crea una nueva instancia de OpcionPedido y la usa o guarda para continuar el proceso.
53	"Hamburguesa",BigDecimal.ZERO,List.of(segundo));	Ejecuta una instrucción Java completa; el punto y coma marca su final.
54	LineaPedido linea = new LineaPedido(UUID.randomUUID().toString(),3,2,	Crea una nueva instancia de LineaPedido y la usa o guarda para continuar el proceso.
55	"Combo de dos","Combo",new BigDecimal("20.00"),List.of(uno,dos),1);	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
56	return new SolicitudPago(UUID.randomUUID().toString(),1,List.of(linea),	Finaliza el método y devuelve el valor indicado a quien lo llamó.
57	"PARA_LLEVAR", "EFECTIVO",new BigDecimal("100.00"),"");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
58	}	Cierra el bloque abierto anteriormente.
59	static void rechazar(Callable<?> accion, String nombre) throws Exception {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
60	try { accion.call(); throw new AssertionError(nombre); }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
61	catch (IllegalArgumentException esperado) { comprobar(true,nombre); }	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
62	}	Cierra el bloque abierto anteriormente.
63	public static void main(String[] args) throws Exception {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
64	base = "waldonalds_test_pagos_" + UUID.randomUUID().toString().replace("-", "");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
65	boolean creada=false;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
66	try(Connection original=Conexion.Conexion.conectar()) {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
67	if(original==null) throw new SQLException("MySQL no disponible");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
68	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
69	try(Statement s=original.createStatement()) { s.execute("CREATE DATABASE `" +base+"`"); creada=true; }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
70	for(String tabla: List.of("categoria","usuario","ingrediente","producto",	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
71	"producto_ingrediente","presentacion_menu","grupo_presentacion",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
72	"opcion_grupo","opcion_componente","pedido","pedido_detalle",	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	"pedido_opcion","pedido_producto","pedido_modificacion","movimiento_inventario")) {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
74	try(Statement s=original.createStatement();ResultSet r=s.executeQuery("SHOW CREATE TABLE `" +	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
75	tabla+"`")) { r.next(); sql(r.getString(2)); }	
76	}	Cierra el bloque abierto anteriormente.
76	sql(Files.readString(Path.of("sql/pagos.sql")));	Ejecuta una instrucción Java completa; el punto y coma marca su final.

77	sql("INSERT INTO categoria(id_categoria,nombre) VALUES(1, 'Prueba')");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	sql("INSERT INTO usuario(id_usuario,nombre,apellido,usuario,correo,password_hash,rol) VALUES(1, 'Prueba', 'Caja', 'prueba', 'prueba@local.test', 'sin-acceso', 'CAJERO')");	Define o usa información sensible de contraseña. En un sistema real debe venir de configuración segura y nunca publicarse.
79	sql("INSERT INTO producto(id_producto,id_categoria,nombre,precio_base,tipo_stock,stock_actual) VALUES(1,1, 'Producto de prueba',10.25, 'DIRECTO',10)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
80	sql("INSERT INTO presentacion_menu(id_presentacion,id_producto_principal,nombre,tipo,precio,predeterminada) VALUES(1,1, 'Individual', 'INDIVIDUAL',10.25,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	sql("INSERT INTO grupo_presentacion(id_grupo,id_presentacion,nombre,minimo,maximo,visible) VALUES(1,1, 'Producto principal',1,1,0)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	sql("INSERT INTO opcion_grupo(id_opcion,id_grupo,nombre,predeterminada) VALUES(1,1, 'Producto de prueba',1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
83	sql("INSERT INTO opcion_componente(id_opcion,id_producto,cantidad) VALUES(1,1,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	sql("INSERT INTO producto(id_producto,id_categoria,nombre,precio_base,tipo_stock,stock_actual) VALUES(2,1, 'Hamburguesa',10, 'RECETA',0), (3,1, 'Combo de dos',20, 'NINGUNO',0)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
85	sql("INSERT INTO ingrediente(id_ingrediente,nombre,unidad_medida,stock_actual,stock_minimo) VALUES(1, 'Pepinillo', 'unidad',10,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
86	sql("INSERT INTO producto_ingrediente(id_producto_ingrediente,id_producto,id_ingrediente,cantidad_default,permite_quitar,permite_extra,cantidad_extra,precio_extra,max_extras) VALUES(1,2,1,1,1,1,2,2)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
87	sql("INSERT INTO presentacion_menu(id_presentacion,id_producto_principal,nombre,tipo,precio,predeterminada) VALUES(2,3, 'Combo', 'COMBO',20,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
88	sql("INSERT INTO grupo_presentacion(id_grupo,id_presentacion,nombre,minimo,maximo,permite_repetir,visible) VALUES(2,2, 'Elige tus hamburguesas',2,2,1,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
89	sql("INSERT INTO opcion_grupo(id_opcion,id_grupo,nombre,predeterminada) VALUES(2,2, 'Hamburguesa',1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
90	sql("INSERT INTO opcion_componente(id_opcion,id_producto,cantidad) VALUES(2,2,1)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
91	PagoDAO dao=new PagoDAO(PagosTest::abrir);	Crea una nueva instancia de PagoDAO y la usa o guarda para continuar el proceso.
92	comprobar(PedidoPanel.monto("10.25")).equals(new BigDecimal("10.25")), "monto con coma decimal");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
93	rechazar()->PedidoPanel.monto("1.001"), "rechaza más de dos decimales");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
94	rechazar()->PedidoPanel.monto("-5"), "rechaza montos negativos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
95	SolicitudPago p=pago(2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
96	Path respaldoArchivo=Files.createTempDirectory("waldonalds-pago-test").resolve("pendiente.properties");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
97	Utilidades.PagoPendienteStore respaldo=new Utilidades.PagoPendienteStore(respaldoArchivo);	Crea una nueva instancia de Utilidades.PagoPendienteStore y la usa o guarda para continuar el proceso.
98	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
99	respaldo.guardar(p);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
100	comprobar(p.equals(respaldo.cargar()),"recupera la misma clave y montos tras reiniciar");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
101	respaldo.eliminar();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
102	comprobar(respaldo.cargar()==null,"retira el respaldo al completar");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
103	} finally { Files.deleteIfExists(respaldoArchivo); Files.deleteIfExists(respaldoArchivo.getParent()); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
104	comprobar(p.total().equals(new BigDecimal("20.50")) && p.cambio().equals(new BigDecimal("79.50")), "total y cambio exactos");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
105	rechazar()->new SolicitudPago(UUID.randomUUID().toString(),1,List.of(),"COMER_AQUI", "EFECTIVO", BigDecimal.TEN,""), "rechaza pedido vacío");	Crea una nueva instancia de SolicitudPago y la usa o guarda para continuar el proceso.
106	rechazar()->new SolicitudPago(UUID.randomUUID().toString(),1,p.lineas(),"COMER_AQUI", "EFECTIVO", BigDecimal.ONE,""), "rechaza efectivo insuficiente");	Crea una nueva instancia de SolicitudPago y la usa o guarda para continuar el proceso.
107	rechazar()->new SolicitudPago(UUID.randomUUID().toString(),1,p.lineas(),"COMER_AQUI", "TARJETA", p.total(),""), "tarjeta requiere referencia");	Crea una nueva instancia de SolicitudPago y la usa o guarda para continuar el proceso.
108	String ticket=dao.cobrar(p);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
109	int ventas = numero("SELECT COUNT(*) FROM pedido WHERE estado='PAGADO'");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

110	int stock = numero("SELECT stock_actual FROM producto WHERE id_producto=1");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
111	comprobar(ticket.contains("79.50") && ventas==1 && stock==8,	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
112	"efectivo guarda venta y descuento stock (ventas="+ventas+", stock="+stock+)");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
113	comprobar(dao.cobrar(p).equals(ticket) && numero("SELECT COUNT(*) FROM pedido")==1 && numero("SELECT COUNT(*) FROM movimiento_inventario")==1, "reintento no duplica venta ni inventario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
114	rechazar()->dao.cobrar(pago(9)), "rechaza stock insuficiente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
115	sql("UPDATE presentacion_menu SET precio=11 WHERE id_presentacion=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
116	rechazar()->dao.cobrar(pago(1)), "rechaza precio modificado");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
117	sql("UPDATE presentacion_menu SET precio=10.25 WHERE id_presentacion=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
118	sql("UPDATE producto SET estado=0 WHERE id_producto=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
119	rechazar()->dao.cobrar(pago(1)), "rechaza producto inactivo");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
120	sql("UPDATE producto SET estado=1 WHERE id_producto=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
121	sql("UPDATE usuario SET estado=0 WHERE id_usuario=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
122	rechazar()->dao.cobrar(pago(1)), "rechaza usuario desactivado");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
123	sql("UPDATE usuario SET estado=1 WHERE id_usuario=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
124	sql("CREATE TRIGGER fallo_prueba BEFORE INSERT ON movimiento_inventario FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Fallo controlado'");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
125	try { dao.cobrar(pago(1)); throw new AssertionError("Debe fallar el movimiento"); } catch(SQLException esperado) { }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
126	comprobar(numero("SELECT COUNT(*) FROM pedido")==1 && numero("SELECT stock_actual FROM producto WHERE id_producto=1")==8 && numero("SELECT COUNT(*) FROM pago_operacion")==1, "rollback completo tras fallo al registrar inventario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
127	sql("DROP TRIGGER fallo_prueba");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
128	SolicitudPago tarjeta=new SolicitudPago(UUID.randomUUID().toString(),1,pago(1).lineas(),"PARALLEL", "TARJETA", new BigDecimal("10.25"), "TERMINAL-PRUEBA");	Crea una nueva instancia de SolicitudPago y la usa o guarda para continuar el proceso.
129	comprobar(dao.cobrar(tarjeta).contains("TERMINAL-PRUEBA") && numero("SELECT COUNT(*) FROM pedido WHERE metodo_pago='TARJETA' AND monto_recibido=total")==1,"registra tarjeta externa y referencia");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
130	SolicitudPago incierto=pago(1);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
131	PagoDAO ambiguo=new PagoDAO()->{	Crea una nueva instancia de PagoDAO y la usa o guarda para continuar el proceso.
132	Connection real=abrir();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
133	return (Connection)Proxy.newProxyInstance(PagosTest.class.getClassLoader(),new Class[]{Connection.class}, (obj,method,params)->{	Finaliza el método y devuelve el valor indicado a quien lo llamó.
134	try { Object resultado=method.invoke(real,params); if(method.getName().equals("commit")) throw new SQLException("Respuesta de commit perdida"); return resultado; }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
135	catch(InvocationTargetException e) { throw e.getCause(); }	Captura un error producido en el try para mostrarlo, convertirlo o recuperarse de forma controlada.
136	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
137	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
138	try { ambiguo.cobrar(incierto); throw new AssertionError("Simular respuesta perdida"); } catch(SQLException esperado) { }	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
139	int antes=numero("SELECT COUNT(*) FROM pedido");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
140	dao.cobrar(incierto);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
141	comprobar(numero("SELECT COUNT(*) FROM pedido")==antes,"respuesta de commit perdida se recupera sin duplicar");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
142	String noDisponible=Utilidades.HorarioMenu.disponibilidadActual(java.time.LocalDateTime.now()).equals("DESAYUNO") ? "ALMUERZO" : "DESAYUNO";	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
143	sql("UPDATE producto SET disponibilidad_menu='"+noDisponible+"' WHERE id_producto=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
144	rechazar()->dao.cobrar(pago(1)), "rechaza productos fuera de horario");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
145	sql("UPDATE producto SET disponibilidad_menu='TODO_DIA', stock_actual=1 WHERE id_producto=1");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
146	ExecutorService ejecutor=Executors.newFixedThreadPool(2);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.

147	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
148	CountDownLatch inicio=new CountDownLatch(1);	Crea una nueva instancia de CountDownLatch y la usa o guarda para continuar el proceso.
149	Callable<Boolean> intento=()->{ inicio.await(); try { dao.cobrar(pago(1)); return true; } catch(IllegalArgumentException ex) {return false;} };	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
150	Future<Boolean> a=ejecutor.submit(intento), b=ejecutor.submit(intento); inicio.countDown();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
151	comprobar(a.get()!=b.get() && numero("SELECT stock_actual FROM producto WHERE id_producto=1"))==0,"dos cajas compiten por última existencia sin sobreventa");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
152	} finally { ejecutor.shutdownNow(); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
153	SolicitudPago personalizado = comboPersonalizado();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
154	String ticketCombo = dao.cobrar(personalizado);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
155	comprobar(personalizado.total().compareTo(new BigDecimal("22.00"))==0	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
156	&& numero("SELECT stock_actual FROM ingrediente WHERE id_ingrediente=1")==8	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
157	&& numero("SELECT COUNT(*) FROM pedido_producto WHERE id_producto=2")==2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
158	&& numero("SELECT COUNT(*) FROM pedido_modificacion")==2	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
159	&& ticketCombo.contains("SIN Pepinillo") && ticketCombo.contains("EXTRA Pepinillo"),	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
160	"combo repetido conserva y descuento cada personalización por separado");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
161	System.out.println("PRUEBAS COMPLETADAS: "+pruebas);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
162	} finally {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
163	if(creada) try(Statement s=original.createStatement()) { s.execute("DROP DATABASE ``"+base+"``");	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
164	}	Cierra el bloque abierto anteriormente.
165	}	Cierra el bloque abierto anteriormente.
166	}	Cierra el bloque abierto anteriormente.
167	}	Cierra el bloque abierto anteriormente.

16.155 test/ReporteDAOTest.java

Prueba ejecutable ReporteDAOTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 48 | Archivo 155 de 157

Clases/tipos detectados: ReporteDAOTest

Línea	Constructor o método detectado
11	public static void main(String[] args) throws Exception {
1	package DAO;
2	
3	import Conexion.Conexion;
4	import java.math.BigDecimal;
5	import java.sql.Connection;
6	import java.time.LocalDate;
7	
8	/** Tablas TEMPORARY de esta conexión: no modifica las tablas reales. */
9	public class ReporteDAOTest {
10	static void verificar(boolean ok, String mensaje) { if (!ok) throw new AssertionError(mensaje); }
11	public static void main(String[] args) throws Exception {
12	try (Connection c = Conexion.conectar()) {
13	if (c == null) throw new AssertionError("MySQL no disponible");
14	try (var s = c.createStatement()) {
15	// MySQL oculta los nombres reales solo dentro de esta conexión.
16	s.execute("CREATE TEMPORARY TABLE usuario (id_usuario INT,nombre VARCHAR(50),apellido VARCHAR(50))");
17	s.execute("CREATE TEMPORARY TABLE categoria (id_categoria INT,nombre VARCHAR(50))");
18	s.execute("CREATE TEMPORARY TABLE producto (id_producto INT,id_categoria INT,nombre VARCHAR(100),tipo_stock VARCHAR(20),estado BOOLEAN,stock_actual INT,stock_minimo INT)");
19	s.execute("CREATE TEMPORARY TABLE ingrediente (nombre VARCHAR(100),estado BOOLEAN,stock_actual DECIMAL(10,2),stock_minimo DECIMAL(10,2),unidad_medida VARCHAR(20))");
20	s.execute("CREATE TEMPORARY TABLE presentacion_menu (id_presentacion INT,id_producto_principal INT)");
21	s.execute("CREATE TEMPORARY TABLE pedido (id_pedido INT,numero_orden INT,id_usuario INT,fecha_hora DATETIME,tipo_servicio VARCHAR(20),estado VARCHAR(20),metodo_pago VARCHAR(20),monto_recibido DECIMAL(10,2),total DECIMAL(10,2))");
22	s.execute("CREATE TEMPORARY TABLE pedido_detalle (id_pedido INT,id_presentacion INT,cantidad INT)");
23	s.execute("INSERT INTO usuario VALUES (1,'Prueba','Temporal')");
24	s.execute("INSERT INTO categoria VALUES (1,'Comida')");
25	s.execute("INSERT INTO producto VALUES (1,1,'Combo','DIRECTO',1,5,5),(2,1,'Componente','DIRECTO',1,0,0),(3,1,'Inactivo','DIRECTO',0,0,5),(4,1,'Disponible','DIRECTO',1,10,5)");
26	s.execute("INSERT INTO presentacion_menu VALUES (1,1),(2,2),(3,3),(4,4)");
27	s.execute("INSERT INTO ingrediente VALUES ('Harina',1,1.25,2.00,'kg'),('Inactivo',0,0,5,'kg')");

28	s.execute("INSERT INTO pedido VALUES (1,1,1,'2026-09-25 00:00:00','COMER_AQUI','PAGADO','EFFECTIVO',100,100),(2,2,1,'2026-09-25 23:59:59','A_DOMICILIO','PAGADO','TARJETA',50,50),(3,3,1,'2026-09-25 12:00:00','PARA_LLEVAR','PENDIENTE',NULL,0,30),(4,4,1,'2026-09-25 14:00:00','PARA_LLEVAR','CANCELADO',NULL,0,200),(5,5,1,'2026-09-26 00:00:00','COMER_AQUI','PAGADO','EFFECTIVO',900,900)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
29	s.execute("INSERT INTO pedido_detalle VALUES (1,1,2),(2,4,1),(3,4,9),(4,4,20),(5,4,30)");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
30	}	Cierra el bloque abierto anteriormente.
31	var dao = new ReporteDAO();	Crea una nueva instancia de ReporteDAO y la usa o guarda para continuar el proceso.
32	var r = dao.cargar(c, LocalDate.of(2026,9,25));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
33	verificar(r.ventas().compareTo(new BigDecimal("150"))==0,"Ventas pagadas y límites del día");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
34	verificar(r.pedidos()==2,"Conteo de pedidos cobrados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	verificar(r.recientes().size()==4,"Listado de todos los estados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	verificar(r.cancelados()==1,"Conteo de cancelados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	verificar(r.ticket().compareTo(new BigDecimal("75"))==0,"Promedio solo de pagados");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
38	verificar(r.stockBajo()==2 && r.alertas().size()==3,"Umbral inclusivo y solo activos");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	verificar(r.productos().size()==2 && "Combo".equals(r.productos().get(0)[0]),"Top sin componentes ni cancelados/pendientes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
40	verificar(new BigDecimal(r.productos().get(0)[2].toString()).intValue()==2,"Cantidad del combo");	Crea una nueva instancia de BigDecimal y la usa o guarda para continuar el proceso.
41	verificar("A domicilio".equals(r.recientes().get(0)[3]),"Servicio y orden reciente");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	var vacio=dao.cargar(c,LocalDate.of(2026,9,24));	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
43	verificar(vacio.pedidos()==0 && vacio.cancelados()==0 && vacio.ventas().signum()==0 && vacio.ticket().signum()==0,"Día sin ventas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
44	verificar(vacio.stockBajo()==2,"Inventario siempre actual");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
45	System.out.println("OK: límites de día, pagados/pendientes/cancelados, combos, stock inclusivo, inactivos, día vacio.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	}	Cierra el bloque abierto anteriormente.
47	}	Cierra el bloque abierto anteriormente.
48	}	Cierra el bloque abierto anteriormente.

16.156 test/ReportesTest.java

Prueba ejecutable ReportesTest. Verifica automáticamente un comportamiento sin modificar la aplicación productiva.

Tipo: java | Líneas: 85 | Archivo 156 de 157

Clases/tipos detectados: ReportesTest

Línea	Constructor o método detectado
30	public static void main(String[] args) throws Exception {
1	import DAO.ReporteDAO;
2	import GUI_ADMINISTRADOR.ReportesPanel;
3	import GUI_ADMINISTRADOR.MenuAdmin;
4	import java.awt.*;
5	import java.awt.image.BufferedImage;
6	import java.io.File;
7	import java.math.BigDecimal;
8	import java.time.LocalDate;
9	import java.util.List;
10	import javax.imageio.ImageIO;
11	import javax.swing.*;
12	
13	/** Pruebas de solo lectura y render de reportes. No inserta ni cambia registros. */
14	public class ReportesTest {
15	static Object campo(Object o, String nombre) throws Exception {
16	var f = o.getClass().getDeclaredField(nombre); f.setAccessible(true); return f.get(o);
17	}
18	static void comprobar(boolean valor, String mensaje) {
19	if (!valor) throw new AssertionError(mensaje);
20	}
21	static void organizar(Container c) {
22	c.doLayout();
23	for (Component hijo : c.getComponents()) if (hijo instanceof Container h) organizar(h);
24	}
25	static void imagen(Container c, String destino) throws Exception {
26	organizar(c);
27	BufferedImage img = new BufferedImage(c.getWidth(), c.getHeight(), BufferedImage.TYPE_INT_RGB);
28	Graphics2D g = img.createGraphics(); c.printAll(g); g.dispose(); ImageIO.write(img, "png", new File(destino));
29	}
30	public static void main(String[] args) throws Exception {
31	var ejemplo = new ReporteDAO.Resumen(new BigDecimal("125.50"), 2, new BigDecimal("62.75"), 0, 0,
32	List.<Object[]>of(new Object[]{1,"=SUM(1,2) \"cajero\"", "12:00", "En salón", "PAGADO",new BigDecimal("125.50")}), List.of(), List.of());
33	String csv = Utilidades.ReporteCsv.generar(LocalDate.of(2026,9,27), ejemplo);

34	comprobar(csv.startsWith("\uFEFF"), "UTF-8 BOM");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
35	comprobar(csv.contains("\n"=SUM(1,2) \n\"cajero\n\"\"), "Escape CSV y formula");	Calcula o toma el valor de la derecha y lo asigna al elemento de la izquierda.
36	comprobar(csv.contains("125.50"), "Importe decimal sin pérdida");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
37	final ReportesPanel[] panel = new ReportesPanel[1];	Crea una nueva instancia de ReportesPanel y la usa o guarda para continuar el proceso.
38	SwingUtilities.invokeLaterAndWait(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
39	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
40	panel[0] = new ReportesPanel(); panel[0].setSize(1580,980);	Crea una nueva instancia de ReportesPanel y la usa o guarda para continuar el proceso.
41	comprobar(campo(panel[0],"carga") == null, "Constructor sin consultas");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
42	comprobar(!((JTable) campo(panel[0],"tablaPedidos")).getModel().isCellEditable(0,0), "Tabla solo lectura");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
43	var fecha = (JSpinner) campo(panel[0],"fechaReporte");	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
44	((JSpinner.DateEditor) fecha.getEditor()).getTextField().setText("31/02/2026");	Cambia el texto visible de un componente de la interfaz.
45	panel[0].actualizarReporte();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
46	comprobar(campo(panel[0],"carga") == null, "Fecha inválida rechazada");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
47	fecha.setValue(new java.util.Date());	Crea una nueva instancia de java.util.Date y la usa o guarda para continuar el proceso.
48	} catch (Exception ex) { throw new RuntimeException(ex); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
49	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
50	System.out.println("OK: CSV, constructor sin DB, tabla de solo lectura y fecha inválida.");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
51	if (args.length > 0 && args[0].equals("--db")) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
52	LocalDate fecha = LocalDate.now();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
53	ReporteDAO.Resumen r = new ReporteDAO().cargar(fecha);	Crea una nueva instancia de ReporteDAO y la usa o guarda para continuar el proceso.
54	comprobar(r.pedidos() == r.recientes().stream().filter(f -> "PAGADO".equals(f[4])).count(), "Cuento de pedidos cobrados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
55	comprobar(r.cancelados() == r.recientes().stream().filter(f -> "CANCELADO".equals(f[4])).count(), "Cuento de cancelados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
56	BigDecimal suma = BigDecimal.ZERO; int pagados = 0;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
57	for (Object[] fila : r.recientes()) if ("PAGADO".equals(fila[4])) { suma = suma.add(BigDecimal) fila[5]; pagados++; }	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
58	comprobar(suma.compareTo(r.ventas()) == 0, "Ventas excluyen no pagados");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
59	BigDecimal promedio = pagados == 0 ? BigDecimal.ZERO : suma.divide(BigDecimal.valueOf(pagados),2,java.math.RoundingMode.HALF_UP);	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
60	comprobar(promedio.compareTo(r.ticket().setScale(2,java.math.RoundingMode.HALF_UP)) == 0, "Ticket promedio");	Configura una propiedad mediante setScale en el objeto indicado.
61	comprobar(r.alertas().stream().filter(f -> "Producto".equals(f[0])).count() == r.stockBajo(), "Stock y alertas coherentes");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
62	comprobar(r.productos().size() <= 10, "Top 10");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
63	for (Object[] fila : r.alertas()) comprobar(new BigDecimal(fila[2].toString()).compareTo(new BigDecimal(fila[3].toString())) <= 0, "Umbral stock");	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
64	var vacio = new ReporteDAO().cargar(LocalDate.of(1000,1,1));	Crea una nueva instancia de ReporteDAO y la usa o guarda para continuar el proceso.
65	comprobar(vacio.pedidos()==0 && vacio.ventas().signum()==0 && vacio.ticket().signum()==0, "Día vacio sin división por cero");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
66	SwingUtilities.invokeLaterAndWait(() -> panel[0].actualizarReporte());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
67	long fin = System.currentTimeMillis()+30000;	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
68	while (System.currentTimeMillis()<fin) {	Bucle while: repite el bloque mientras la condición continúe siendo verdadera.
69	final boolean[] pendiente={false}; SwingUtilities.invokeLaterAndWait(() -> { try { pendiente[0]=campo(panel[0],"carga")!=null; } catch(Exception ex) { throw new RuntimeException(ex); } });	Crea una nueva instancia de RuntimeException y la usa o guarda para continuar el proceso.
70	if (!pendiente[0]) break; Thread.sleep(50);	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
71	}	Cierra el bloque abierto anteriormente.

72	SwingUtilities.invokeLaterAndWait(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
73	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
74	comprobar(campo(panel[0], "ultimoReporte")!=null, "Carga SwingWorker completada");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
75	JFrame marco = new JFrame(); marco.setContentPane(panel[0]); marco.pack();	Crea una nueva instancia de JFrame y la usa o guarda para continuar el proceso.
76	panel[0].setSize(1580, 980);	Configura una propiedad mediante setSize en el objeto indicado.
77	imagen(panel[0], "build/reportes-check/reportes.png");	Ejecuta una instrucción Java completa; el punto y coma marca su final.
78	marco.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
79	} catch(Exception ex) { throw new RuntimeException(ex); }	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
80	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
81	System.out.println("OK: MySQL real (solo lectura), indicadores, alertas, día vacío y carga asíncrona . Pedidos del día: "+r.pedidos());	Ejecuta una instrucción Java completa; el punto y coma marca su final.
82	}	Cierra el bloque abierto anteriormente.
83	System.exit(0);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
84	}	Cierra el bloque abierto anteriormente.
85	}	Cierra el bloque abierto anteriormente.

16.157 VisualCheckMenuAdmin.java

Archivo de apoyo del proyecto: VisualCheckMenuAdmin.java.

Tipo: java | Líneas: 42 | Archivo 157 de 157

Clases/tipos detectados: VisualCheckMenuAdmin

Línea	Constructor o método detectado
11	private VisualCheckMenuAdmin() {
14	public static void main(String[] args) throws Exception {
34	private static void organizar(Container contenedor) {

1	import GUI_ADMINISTRADOR.MenuAdmin;	Importa GUI_ADMINISTRADOR.MenuAdmin para poder usar esa clase sin escribir su nombre completo.
2	import java.awt.Component;	Importa java.awt.Component para poder usar esa clase sin escribir su nombre completo.
3	import java.awt.Container;	Importa java.awt.Container para poder usar esa clase sin escribir su nombre completo.
4	import java.awt.Graphics2D;	Importa java.awt.Graphics2D para poder usar esa clase sin escribir su nombre completo.
5	import java.awt.image.BufferedImage;	Importa java.awt.image.BufferedImage para poder usar esa clase sin escribir su nombre completo.
6	import java.io.File;	Importa java.io.File para poder usar esa clase sin escribir su nombre completo.
7	import javax.imageio.ImageIO;	Importa javax.imageio.ImageIO para poder usar esa clase sin escribir su nombre completo.
8	import javax.swing.SwingUtilities;	Importa javax.swing.SwingUtilities para poder usar esa clase sin escribir su nombre completo.
9		Línea en blanco: separa visualmente bloques para que el código sea más legible.
10	public final class VisualCheckMenuAdmin {	Declara un tipo Java. Aquí comienza la definición de responsabilidades y datos de esta pieza del sistema.
11	private VisualCheckMenuAdmin() {	Constructor de VisualCheckMenuAdmin: prepara un objeto nuevo y deja sus componentes listos para usarse.
12	}	Cierra el bloque abierto anteriormente.
13		Línea en blanco: separa visualmente bloques para que el código sea más legible.
14	public static void main(String[] args) throws Exception {	Declara el método main. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
15	final MenuAdmin[] ventana = new MenuAdmin[1];	Crea una nueva instancia de MenuAdmin y la usa o guarda para continuar el proceso.
16	SwingUtilities.invokeLaterAndWait(() -> ventana[0] = new MenuAdmin());	Crea una nueva instancia de MenuAdmin y la usa o guarda para continuar el proceso.
17	SwingUtilities.invokeLaterAndWait(() -> {	Forma parte del bloque actual y completa la lógica indicada por las líneas cercanas.
18	Container contenido = ventana[0].getContentPane();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
19	contenido.setSize(1920, 1080);	Configura una propiedad mediante setSize en el objeto indicado.
20	organizar(contenido);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
21	BufferedImage imagen = new BufferedImage(1920, 1080, BufferedImage.TYPE_INT_ARGB);	Crea una nueva instancia de BufferedImage y la usa o guarda para continuar el proceso.
22	Graphics2D graficos = imagen.createGraphics();	Declara una variable o atributo: reserva un nombre y tipo para guardar un dato usado por esta clase.
23	contenido.printAll(graficos);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
24	graficos.dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
25	try {	Inicia una operación que puede fallar. Si hay recursos entre paréntesis, Java los cerrará automáticamente.
26	ImageIO.write(imagen, "png", new File(args[0]));	Crea una nueva instancia de File y la usa o guarda para continuar el proceso.
27	} catch (Exception ex) {	Cierra el bloque anterior y abre la ruta alternativa o de manejo de error.
28	throw new RuntimeException(ex);	Interrumpe el flujo lanzando un error con una explicación para quien llamó al método.
29	}	Cierra el bloque abierto anteriormente.
30	ventana[0].dispose();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
31	});	Ejecuta una instrucción Java completa; el punto y coma marca su final.
32	}	Cierra el bloque abierto anteriormente.

33		Línea en blanco: separa visualmente bloques para que el código sea más legible.
34	private static void organizar(Container contenedor) {	Declara el método organizar. Sus parámetros son datos de entrada y el tipo anterior al nombre indica qué devuelve.
35	contenedor.doLayout();	Ejecuta una instrucción Java completa; el punto y coma marca su final.
36	for (Component componente : contenedor.getComponents()) {	Bucle for: repite el bloque para una secuencia, rango o colección de elementos.
37	if (componente instanceof Container hijo) {	Evalúa una condición; el bloque siguiente solo se ejecuta cuando el resultado es verdadero.
38	organizar(hijo);	Ejecuta una instrucción Java completa; el punto y coma marca su final.
39	}	Cierra el bloque abierto anteriormente.
40	}	Cierra el bloque abierto anteriormente.
41	}	Cierra el bloque abierto anteriormente.
42	}	Cierra el bloque abierto anteriormente.